ARCH LINUX 分册教材

第四册

网络与服务

网络配置、DNS、VPN、SSH、服务器与网络服务

依据 Arch Linux 中文维基全量条目编写

共 9 册 · 997 篇 · 安装 / 系统 / 网络 / 硬件 / 桌面 / 软件 / 开发 / 安全

安装系统时，你大概只是接上网线或连上 Wi-Fi，拿到一个临时地址，之后就没再关心过网络。本册要补的正是这背后的完整图景：网卡如何被识别和命名，地址从哪儿来，域名怎么解析，流量怎样被防火墙拦住或放行，代理与 VPN 如何改变出口，以及一台普通机器怎样转身成为别人能访问的服务器。

读完之后，你应该能做到：亲手配好有线与无线连接，说清网络管理器和底层工具各自管什么；在"连不上"时按链路、地址、路由、解析、端口的顺序把原因缩小到一处；用 SSH 安全登录远端主机；把机器配成网页、文件或邮件之类服务的宿主；必要时用监控工具看清线路上发生了什么。

它在九册里的位置

第四册站在前三册的肩膀上。第一册交代了 Arch 的哲学与查资料的方法，第二册给了你一个装好、能用包管理器维护的系统，第三册讲清了服务管理、内核与存储——本册几乎每个网络服务最终都要落成 systemd 单元，每块网卡都离不开内核驱动，缺了这层服务概念会读得很吃力。

往后看，第五册接手网卡之外的硬件，第六册讲桌面怎样接管网络状态，第七册收录联网应用，第八册把网络当成开发工具的背景，而第九册《安全与加密》与本册贴得最近：本册讲怎样让服务可达，第九册讲怎样让可达的部分不被滥用。

阅读顺序

主线建议这样走：先慢读第 1 章《网络》，十八节是全册地基；再读网络管理器与网络配置几章，把工具层补齐；然后读 DNS 与防火墙，二者分别决定"能不能访问别人"和"能不能被别人访问"。之后转 SSH，再按需进入服务器与 VPN，最后挑服务细项。

可以跳读的是一批应用型章节：串流、BitTorrent、志愿计算、网络电话、搜索、目录服务，以及即时通信、文件分享这类兴趣向分支。它们回答的是"这台机器还能拿来做什么有趣的事"，与系统服务主线并列，可挑读。

前置知识与误区

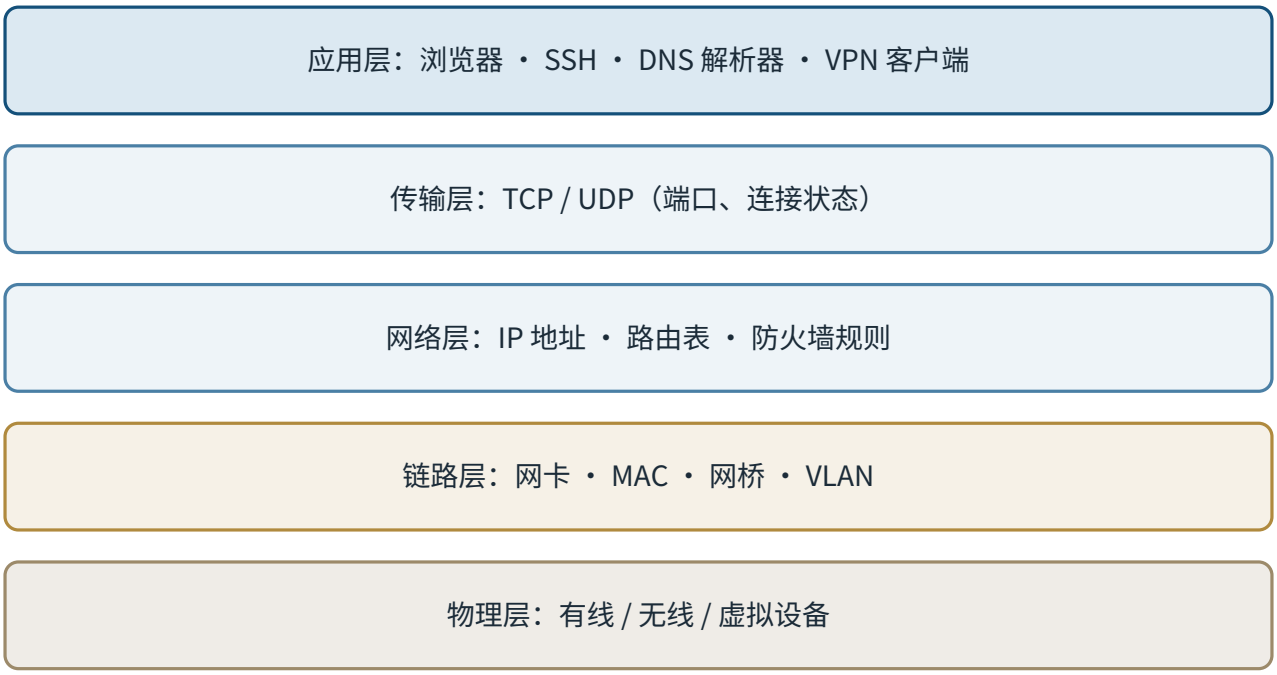
前置要求不多但很具体：会用 systemctl 管理服务状态，会编辑 `/etc` 下的配置，懂一点权限，并对地址、掩码、网关、端口、协议有基本印象；这组若是空白，建议先补半天 TCP/IP 入门。

三个误区值得先记住。其一，以为装好网络管理器就等于网络正常——它只是调度层，地址来自 DHCP 客户端，解析来自 DNS 服务，任何一环掉链子都得单独查。其二，把 `/etc/resolv.conf` 当成可以随手改的永久配置，它常被别的组件重写，改完似乎生效，重启又变回去。其三，把"ping 得通"当成"服务可用"：端口没监听或被防火墙挡住时，ping 照样成功，连接却依旧失败。

另外，本册的服务器不必另买机器，同一台 Arch 也能兼任，只是安全边界的功课要提前做。

网络栈

分层排查是基本功



排查网络问题先确认卡在哪一层：能 ping 通不代表 DNS 正常，DNS 正常不代表代理生效

TCP/IP 五层模型与常见的排查切入点

目　录

第 1 章　网络（18 节）

1.1 Isatapd　1.2 VnStat　1.3 Telnet　1.4 Sslh　1.5 Drcom　1.6 DNS-over-HTTPS　1.7 I2P　1.8 Netatalk　1.9 OpenStack　1.10 VLAN　1.11 网桥　1.12 MentoHUST　1.13 业余无线电　1.14 进阶网络流量控制　1.15 Systemd-resolved　1.16 IPv6　1.17 网络配置　1.18 InfiniBand

第 2 章　防火墙（9 节）

2.1 Ipset　2.2 Shorewall　2.3 Firewalld　2.4 Fail2ban　2.5 Uncomplicated Firewall　2.6 Sshguard　2.7 Iptables　2.8 Simple stateful firewall　2.9 Nftables

第 3 章　代理服务器（8 节）

3.1 NaïveProxy　3.2 Squid　3.3 V2Ray　3.4 Privoxy　3.5 Qv2ray　3.6 Polipo　3.7 Shadowsocks　3.8 代理服务器

第 4 章　互联网应用（7 节）

4.1 Mikutter　4.2 Twister　4.3 ZeroNet　4.4 Google Earth　4.5 AMule　4.6 Unison　4.7 Tor

第 5 章　即时通信（7 节）

5.1 Koreader　5.2 WhatsApp　5.3 Discord　5.4 微信　5.5 TouchFish　5.6 Telegram　5.7 腾讯 QQ

第 6 章　网络共享（7 节）

6.1 Shfs　6.2 IPhone 网络共享　6.3 网络分享　6.4 路由器　6.5 Samba/Active Directory 域控制器　6.6 NFS　6.7 Samba

第 7 章　网络服务器（7 节）

7.1 Caddy　7.2 Artanis　7.3 Wordpress　7.4 Lighttpd　7.5 Tomcat　7.6 Apache HTTP 服务器　7.7 Nginx

第 8 章　SSH（6 节）

8.1 Mosh　8.2 安全外壳协议　8.3 通过 SSH 安装 Arch Linux　8.4 SSHFS　8.5 SSH 密钥　8.6 OpenSSH

第 9 章　VPN（6 节）

9.1 PPTP server　9.2 PPTP Client　9.3 Linux 容器/使用 VPN　9.4 StrongSwan　9.5 WireGuard　9.6 OpenVPN

第 10 章　串流（6 节）

10.1 Streamlink　10.2 Yt-dlp　10.3 流式传输至 twitch.tv　10.4 Open Broadcaster Software　10.5 VLC 媒体播放器　10.6 Mpv

第 11 章　BitTorrent（5 节）

11.1 Ktorrent　11.2 QBittorrent　11.3 Aria2　11.4 RTorrent　11.5 Transmission

第 12 章　无线网络（5 节）

12.1 Ad-hoc networking　12.2 博通无线网卡　12.3 软件接入点　12.4 Wireless bonding　12.5 网络配置/无线网络配置

第 13 章　网络应用（5 节）

13.1 Ghost　13.2 KiwiIRC　13.3 Home Assistant Supervised　13.4 Matomo　13.5 Home Assistant

第 14 章　远程服务器（5 节）

14.1 Remmina　14.2 X11vnc　14.3 Xrdp　14.4 X2Go　14.5 TigerVNC

第 15 章　邮件客户端（5 节）

15.1 NBSMTP　15.2 Aerc　15.3 Notmuch　15.4 Sup　15.5 Mutt

第 16 章　DNS（4 节）

16.1 Cloudflared　16.2 BIND　16.3 Unbound　16.4 域名解析

第 17 章　服务器（4 节）

17.1 YOURLS　17.2 FHEM　17.3 Git 服务器　17.4 Minecraft/Java 版服务端

第 18 章　网络管理器（4 节）

18.1 Wifi Radar　18.2 ConnMan　18.3 Netctl　18.4 Systemd-networkd

第 19 章　网络管理界面（4 节）

19.1 Webmin　19.2 Adminer　19.3 PhpPgAdmin　19.4 PhpMyAdmin

第 20 章　网络配置（4 节）

20.1 网络配置/以太网　20.2 Dhcpcd　20.3 Wpa supplicant　20.4 Iwd

第 21 章　邮件服务器（4 节）

21.1 UW IMAP　21.2 Sendmail　21.3 邮件服务器　21.4 Postfix

第 22 章　FTP（3 节）

22.1 CurlFtpFS　22.2 SCP 与 SFTP　22.3 SFTP chroot

第 23 章　NTP（3 节）

23.1 Systemd-timesyncd　23.2 Chrony　23.3 Network Time Protocol daemon

第 24 章　WebDAV（3 节）

24.1 AgenDAV　24.2 Davfs2　24.3 WebDAV

第 25 章　下载工具（3 节）

25.1 Bookget　25.2 CURL　25.3 Wget

第 26 章　搜索（3 节）

26.1 Surfraw　26.2 Baloo　26.3 Locate

第 27 章　DHCP（2 节）

27.1 Dhcpd　27.2 Dnsmasq

第 28 章　XMPP（2 节）

28.1 Jabberd2　28.2 Pidgin

第 29 章　互联网中继聊天（2 节）

29.1 Arch 的 IRC 频道　29.2 Irssi

第 30 章　志愿计算（2 节）

30.1 Folding@home　30.2 BOINC

第 31 章　文件分享（2 节）

31.1 Dropbox　31.2 Nextcloud

第 32 章　网络框架（2 节）

32.1 Django　32.2 Ruby on Rails

第 33 章　邮件（2 节）

33.1 Mailman　33.2 GnuPG

第 34 章　FTP 服务器（1 节）

34.1 Very Secure FTP Daemon

第 35 章　Usenet 客户端（1 节）

35.1 Hellanzb

第 36 章　分布式计算（1 节）

36.1 Distcc

第 37 章　多播DNS（1 节）

37.1 Avahi

第 38 章　目录服务（1 节）

38.1 OpenLDAP

第 39 章　网络应用程序（1 节）

39.1 应用程序列表/互联网

第 40 章　网络电话（1 节）

40.1 Matrix

第 41 章　网络监控（1 节）

41.1 Smokeping

第 42 章　网页（1 节）

42.1 网络订阅源

网络

本章把网络从「连不上」讲到「按需分流」：确认连接、配置地址与解析，再到网卡上的隔离与转发。《网络配置》与《Systemd-resolved》必读：前者管连接检查与网络管理，后者决定解析走向；再读《IPv6》，留意邻居发现与地址自动配置；需要隔离时看《网桥》《VLAN》。《VnStat》《Telnet》随手翻；Isatapd、I2P、业余无线电等节无对应环境可跳过。

最常见的坑是远程改网桥或 VLAN，一落盘就切断自己所在的接口；另一个误解是地址配通即网络就绪，故障多在解析与 PPPoE。

1.1　Isatapd

难度 入门 | **篇幅** 1.5 KB | **所属** 网络

本节将讲到

- 安装
- 用法

isatapd 是一个 ISATAP 客户端。它是一个守护程序，用于在 Linux 中创建和维护 ISATAP 隧道 RFC 5214。

安装

安装 isatapd包 包。

用法

该软件包提供了 `isatapd@.service` 系统服务。启动/启动时，将所选路由器（例如 isatap.tsinghua.edu.cn）作为参数传递给服务。

如果要修改其他选项而不是使用默认值，请查看项目主页以供进一步参考。

1.2　VnStat

难度 入门 | **篇幅** 1.8 KB | **所属** 网络

本节将讲到

- 安装
- 配置
- 用法

vnStat 是一个轻量级（命令行）网络流量监视器。它监视可选接口，并将网络流量日志存储在数据库中，以供以后分析。

安装

安装 vnstat包 软件包。

配置

启动/启用 `vnstat.service` 守护程序。

选择一个首选的网络接口，并相应地在 `/etc/vnstat.conf` 中编辑 `Interface` 变量。 要列出 vnstat 可用的所有接口，请使用 `vnstat --iflist`。

要开始监视启动守护程序时配置文件中未引用的特定接口，必须首先初始化数据库。每个接口都需要自己的数据库。初始化 `eth0` 接口的命令是：

```
# vnstat --add -i eth0
```

添加新接口后，请记得重新启动 `vnstat.service` 守护程序。

用法

查询网络流量：

```
# vnstat --query
```

查看实时网络流量使用情况：

```
# vnstat --live
```

要查找更多选项，请使用：

```
# vnstat --help
```

或查看所有选项，使用：

```
# vnstat --longhelp
```

数据的直观呈现也可以通过 vnstati(1)（它是 vnstat包 软件包的一部分）来实现。

1.3 Telnet

难度 入门 | **篇幅** 2.3 KB | **所属** 网络

本节将讲到

- 简单介绍
- 安装
- 配置

**这篇文章或章节的翻译质量不佳。**

原因：No updates since 2012, still references `rc.d`.（在 Talk:Telnet# 中讨论）

简单介绍

telnet是一种基于TCP的传统命令行远程控制协议。telnet使用非加密的通道，因此不太安全。现在主要用于链接一些旧设备。

下面这些介绍主要适用于在Arch Linux系统中配置一个telnet服务器。

安装

如果只使用telnet联接到别的机器，只需要安装netkit-telnet即可：

```
# pacman -S inetutils
```

如果需要安装和配置telnet服务器，还需要安装xinetd：

```
# pacman -S xinetd
```

- 重要提示, "Telnetd是传统脚本，代码不佳并且不可信赖 - 除非不得已，否则不要运行他。" 引用自netkit-telnet的README文件。

如果你非得要telnet服务器，有一个更好的选择。安装AUR里面的telnet-bsd软件包取代netkit-telnet（同样也支持IPv6）。

配置

1\. 要允许通过xinetd联接telnet，需要编辑/etc/xinetd.d/telnet文件：

```
# vi /etc/xinetd.d/telnet
```

将'disable'的值从'yes'修改为'no'。

2\. 要允许telnet从其他机子联接到本机，需要添加允许规则。打开文件/etc/hosts.allow，添加如下行：

```
in.telnetd: ALL
```

3\. 如果需要开机自动开启该服务，将xinetd加入到/etc/rc.conf的"DAEMONS"中：

```
DAEMONS=(syslog-ng network netfs crond ..... xinetd)
```

4\. 重新启动电脑。或者重新启动xinetd（如下）：

```
# /etc/rc.d/xinetd restart
```

测试

先试试看在本地用telnet联接自己：

```
$ telnet localhost
```

提示：你不能通过telnet登录为根用户｛包括使用bsd telnet）。

1.4 Sslh

难度 入门 | **篇幅** 2.6 KB | **所属** 网络

本节将讲到

- 安装
- 配置
- 运行
- 参考

sslh是一个ssl/ssh多路复用软件。

安装

安装sslh包软件包。

配置

软件的默认配置在/etc/sslh.cfg文件中，支持ssh、openvpn、xmpp、http、ssl和anyprot协议。

软件包中还包括2个额外的配置文件：

- /usr/share/doc/sslh/basic.cfg, 基础配置文件，提供“标准安装”所需要的合理配置。
- /usr/share/doc/sslh/example.cfg, 此配置文件被用作说明文档，其不应当被当做默认配置使用，也不应该在任何生产环境中使用。

运行

将sslh-fork.service或sslh-select.service设置为开机自启动。

sslh-fork为每一个新连接创建一个新的进程。sslh-fork经过广泛测试，具有良好的可靠性，但会榨干许多处理器的性能。 如果在一个“小环境”下使用sslh（只有几个ssh连接的低流量https服务器），使用sslh-fork非常合适。

sslh-select只使用一个线程，在同一时间监听所有连接。sslh-select功能较新且测试不足，但每个连接最多只使用16字节。当然，如果sslh-select停止工作，所有连接都会中止，因此你不能对其远程升级。 如果在一个“中型环境”下使用sslh（上千个ssh连接和上千个ssl连接），sslh-select更合适。

如果要在非常大的环境下使用sslh（上万个连接），可能需要某些应用了libevent或类似技术的“概念版”软件（译者注：vapourware，又称“雾件”，指发布后从未真正发行的软件，类似于“概念车”）。

参考

Difference between sslh-fork and sslh-select

basic.cfg

example.cfg

1.5 Drcom

难度 入门 | **篇幅** 2.6 KB | **所属** 网络

本节将讲到

- 安装
- 配置
- 启动
- 注意事项以及疑难解答

drcom作为中国大陆众多高校采用拨号上网的认证方式，某些学校不提供Linux客户端，或者不对各个发行版都支持. dogcom通过分析drcom认证数据包，支持发送心跳包，保持握手.从而解决了校园网认证问题

安装

安装软件包 dogcom-gitAUR.

配置

drcom分为DHCP和PPPOE二个版本，二个版本配置文件不同.

DHCP配置

使用 Wireshark 在官方客户端登录前开始截包，做一次完整的截包动作然后登出，保存为 `wireshark` 截包文件, 比如 `dr.pcapng` (扩展名为pcapng).

配置文件生成

下载配置文件生成器将其与第一步的截包文件放到同一个目录下，并且将 `filename = 'dr.pcapng'` 中的 dr.pcapng 改为第一步保存的文件名。

修改配置文件

进入命令提示符(或shell)进入 drcom_d_config.py 所在目录，输入 `python drcom_d_config.py > config.txt`，打开 `config.txt` 根据提示，将如下段落拷贝到 `/etc/drcom.d/dhcp.conf`。

```
server = '172.17.254.6'
username='hahahahaha'
password=
CONTROLCHECKSTATUS = '\x20'
ADAPTERNUM = '\x01'
host_ip = '172.17.1.1'
IPDOG = '\x01'
host_name = 'DRCOMFUCKER'
PRIMARY_DNS = '114.114.114.114'
dhcp_server = '0.0.0.0'
AUTH_VERSION = '\x0f\x00'
mac = 0x28d244090e15
```

```
host_os = 'WINDIAOS'
KEEP_ALIVE_VERSION = '\xd4\x02'
```

PPPOE配置

配置文件生成

抓包同上

下载 p版配置生成器 , 将其与第一步的截包文件放到同一个目录下，并且将 `filename = 'dr.pcapng'` 中的 drp.pcapng 改为第一步保存的文件名

修改配置文件

进入命令提示符(或shell)进入 drcom_p_config.py 所在目录，输入 `python drcom_p_config.py > config.txt` ，打开 `config.txt` 将类似

```
server = '1.1.1.1'
pppoe_flag = '\x1d'
keep_alive2_flag = '\xd8'
```

的输出覆盖 `/etc/dogcom.d/pppoe.conf`

启动

请启动 `dogcom-d.service` DHCP认证方式或 `dogcom-p.service` 服务。

关于用法请参阅 systemd 页面。

注意事项以及疑难解答

更多配置以及事项请见github wiki页面

- 大部分学校配置可能无法生成，可参阅一下链接

可用学校列表

1.6 DNS-over-HTTPS

难度 入门 | **篇幅** 3.3 KB | **所属** 网络

本节将讲到

- 安装
- 客户端启动
- 客户端配置
- 故障排除

DNS-over-HTTPS是 DNS over HTTPS 的一个实现。它可以作为一个本地解析器。

安装

安装dns-over-https包包。

客户端启动

禁用任何绑定到53端口的服务

为了查看是否有程序在使用53端口，运行：

```
$ ss -lp 'sport = :domain'
```

如果输出超过了一行，你需要禁用任何使用53端口的服务。一旦以上命令只输出以下内容，您就可以继续操作了：

```
Netid State   Recv-Q  Send-Q   Local Address:Port     Peer Address:Port Process
```

更改系统DNS服务器

将系统的DNS服务器更改为配置文件`listen =`中的地址。如果你不知道自己在做什么，建议使用`127.0.0.1`。

这可以通过Network Manager或编辑/etc/resolv.conf来完成。

启动

启动或启用 `doh-client.service`。

测试配置

要测试你系统的DNS是否有效，请在命令行输入`nslookup www.baidu.com`。假定您在安装之前有配置好DNS，这会在配置好DNS over HTTPS之前成功执行。

客户端配置

客户端配置文件默认是`/etc/dns-over-https/doh-client.conf`。

选择首选的上游DNS服务器

要选择首选DNS服务器，请取消注释其中一个配置文件。

如果你使用的DNS服务器未列出，你可以在`[upstream]` 部分使用此模板：

```
/etc/dns-over-https/doh-client.conf
[[upstream.upstream_ietf]]
    url = "https://[IP or web address]/dns-query"
    weight = 20
```

故障排除

服务在有线连接中无法正常启动

正如开发者所说:

: ArchLinux没有默认的网络管理器，因此systemd没有预先配置的在线检测。 : 如果你用Wi-Fi,我建议确保systemd的在线检测能正常运行。我相信你的系统已经安装网络管理器，例如NetworkManager。以帮助你管理Wi-Fi密码。 : 或者，如果你用有线网

络，简单地修改.service文件以禁用在线检测将是最简单的解决方案。为非移动机器安装NetworkManager可能违反K.I.S.S.原则，我们不希望这样做。

上游建议对服务文件使用附加配置片段：

```
/etc/systemd/system/doh-client.service.d/override.conf
[Unit]
After=multi-user.target

[Service]
Type=idle
```

1.7 I2P

难度 入门 | **篇幅** 3.4 KB | **所属** 网络

本节将讲到

- 安装
- 使用
- 匿名站点
- 参见

I2P 是一个动态的分布式匿名网络，它提供了一个简单的代理层帮助身份敏感的应用程序进行匿名通信。I2P 网络中不存在可信方，经过网络的数据将会被多层加密，很多在公网上可能威胁用户隐私的活动可以在 I2P 中匿名进行。许多支持 I2P 代理接口的应用程序都可用，包括邮件，P2P，IRC聊天等等。

安装

安装使用 C++ 编写的客户端 i2pd包 （或开发版本 i2pd-gitAUR ），它可以适应资源有限的硬件。

标准 I2P 套件位于 i2pAUR 和 i2p-binAUR 包。两者都需要Java运行时环境（如 OpenJDK）。建议在 ARM 平台使用时选择 Oracle Java。

I2P 主页还提供了预编译二进制文件，其中包括命令行（无界面）选项，可以安装到用户的主目录当中。在这种情况下，I2P 将通过 i2p 网络自动更新自身。

使用

如果您安装了 i2pd（C++版本），您需要启动/启用服务项 `i2pd.service` 。守护进程的配置文件位于 `/etc/i2pd/i2pd.conf` 。如果安装了 i2p（Java版本），则此时您需要启动或启用服务项 `i2prouter.service`。

打开（您选择的）浏览器访问 I2P 的欢迎页面， `127.0.0.1:7070` 为守护程序页面， `127.0.0.1:7657` 为路由控制台页面（请参阅常见问题解答）。在这里您将被

导航到 I2P 配置和统计页面，以及指向 Eepsites 的链接。 请注意，在守护进程引导进入网络之前，匿名站点将不可使用，这可能需要几分钟时间。

为了访问匿名站点，您需要配置浏览器使用以下本地代理：

```
HTTP  127.0.0.1 4444
SOCKS 127.0.0.1 4447
```

匿名站点

要搭建匿名站点请查阅 I2P 说明。但请记住，i2p（Java版本）用户的主目录位于 `/opt/i2p` ，参阅 AUR 文件 i2p.install 。

参见

- I2P Homepage\[失效链接 2021-11-11 ⓘ\]
- I2P Homepage mirror
- I2P FAQ\[失效链接 2021-11-11 ⓘ\]
- I2P on Wikipedia
- i2pd daemon homepage
- i2pd documentation
- GetI2P

1.8 Netatalk

难度 入门 | **篇幅** 3.7 KB | **所属** 网络

本节将讲到

- 安装
- 配置
- IP Tables
- 开启 Bonjour/Zeroconf

Netatalk 是一个 Apple Filing Protocol (AFP) 的开源实现。它为 Unix 风格系统提供了与 Macintosh 文件共享的功能。

安装

安装 netatalkAUR 包。

配置

使用 systemd 来启用或启动 `netatalk.service`

除了一并安装的配置文件（升级的时候会检查这些文件），另外 Netatalk 可能会生成 `/etc/netatalk/afp_signature.conf`
或 `/var/state/netatalk/afp_signature.conf`，来保存系统的

UUID；`/etc/netatalk/afp_voluuid.conf` 或 `/var/state/netatalk/afp_voluuid.conf` 来保存 Time Machine 的 UUID。为了消除本地网络的服务广播的歧义，移除包的时候可能并不会删除这些文件。

Netatalk 3.x 只使用一个配置文件 `/etc/afp.conf`。 查看 `man afp.conf` 来配置该文件（需要进程拥有 `afpd.log` 的写权限）：

```
/etc/afp.conf
[Global]
 mimic model = TimeCapsule6,106
 log level = default:warn
 log file = /var/log/afpd.log
 hosts allow = 192.168.1.0/16

[Homes]
 basedir regex = /home

[TimeMachine]
 path = /mnt/timemachine
 valid users = tmuser
 time machine = yes

[Shared Media]
 path = /srv/share/media
 valid users = joe sam
```

警告：不要在 `afp.conf` 里面使用符号链接（symbolic link）

来客访问

如果要让来客**只读**地访问你的共享文件夹，在 `[Global]` 这一行的下面加入以下片段：

```
/etc/afp.conf
[Global]
uam list = uams_guest.so
```

要让来客拥有**读/写**权限，首先要像上面一样建立只读权限，再添加下面这一行到指定的分享片段：

```
/etc/afp.conf
[Your Share]
path = /mnt/public/share
rwlist = nobody
```

IP Tables

如果你使用 iptables 包来做防火墙，可能需要加入下面几行：(如果需要的话，可以用用 `-A` 来 替换掉 `-I`)

```
Bonjour/Zeroconf
iptables -I INPUT -p udp --dport mdns -d 224.0.0.251 -j ACCEPT
iptables -I OUTPUT -p udp --dport mdns -d 224.0.0.251 -j ACCEPT
AFP
iptables -I INPUT -p tcp --dport afpovertcp -j ACCEPT
SLP
iptables -I INPUT -p tcp --dport slp -j ACCEPT
iptables -I OUTPUT -p tcp --dport slp -j ACCEPT
iptables -I INPUT -p udp --dport slp -j ACCEPT
```

```
iptables -I OUTPUT -p udp --dport slp -j ACCEPT
AppleTalk
iptables -I INPUT -p tcp -m multiport --dport at-rtmp,at-nbp,at-echo,at-zis -j ACCEPT
iptables -I OUTPUT -p tcp -m multiport --dport at-rtmp,at-nbp,at-echo,at-zis -j ACCEPT
```

开启 Bonjour/Zeroconf

Bonjour/Zeroconf 是 Netatalk 的必要组件，在默认状况下是编译好了的。不需要额外的配置，Netatalk 就可以使用 dbus 链接来注册服务。 你要确认设置 `-mimicmodel` 为你想要的字符串（查看 Mac 上的 `/System/Library/CoreServices/CoreTypes.bundle/Contents/Info.plist` 来获得一个完整的列表）。

如果 `avahi-daemon.service` 没有运行的话，可能需要使用 systemd 启用或启动该服务。

1.9 OpenStack

难度 入门 | **篇幅** 5.7 KB | **所属** 网络

本节将讲到

- 组件
- 部署 OpenStack
- 镜像
- 参阅

OpenStack 是由来自全球的开发者及云计算专家合作开放的一套被广泛使用的公有/私有云计算平台。该项目着重于为所有类型的云创造一套易于实现、高可扩展性且功能丰富的解决方案。该平台由一系列互相关联的项目组成，为云基础设施方案提供各种组件。

组件

- **计算 (Nova)** — Nova 是用于提供计算实例（即虚拟机）的 Openstack 项目。

: https://docs.openstack.org/nova/latest/ || openstack-novaAUR

- **网络 (Neutron)** — Neutron 是用于在其它 OpenStack 服务（例如 Nova）管理的接口设备（例如 vNIC）之间提供“网络即服务”功能的 OpenStack 项目。它实现了 OpenStack 网络 API。

: https://docs.openstack.org/neutron/latest/ || openstack-neutronAUR

- **镜像服务 (Glance)** — 镜像服务 (glance) 项目提供了让用户上传并发现用于让其它服务使用的数据资产的功能。

: https://docs.openstack.org/glance/latest/ || openstack-glanceAUR

- **块存储 (Cinder)** — Cinder 是用于为 Nova 虚拟机、Ironic 裸金属、容器等服务提供存储盘的 OpenStack 服务。

: https://docs.openstack.org/cinder/latest/ || openstack-cinderAUR

- **鉴权 (Keystone)** — Keystone 是通过 OpenStack 身份 API 提供 API 用户验证，服务发现及分布式多租户授权的 OpenStack 服务。

: https://docs.openstack.org/keystone/latest/ || openstack-keystoneAUR

- **控制面板 (Horizon)** — Horizon 是 canonical 对于 OpenStack 面板的实现，它为 Nova、Swift、Keystone 等 OpenStack 服务提供了网页用户界面。

: https://docs.openstack.org/horizon/latest/ || openstack-horizonAUR

部署 OpenStack

镜像

可用镜像

Openstack 官方镜像包含了大部分流行的 GNU/Linux 发行版。

Arch Linux 官方云镜像可以在以下链接下载：
https://gitlab.archlinux.org/archlinux/arch-boxes

自己创建镜像

OpenStack 镜像需要满足一些要求。可以手动或是使用工具来协助准备镜像。

若要使用工具，可以考虑将 image-bootstrap 搭配 `--openstack` 选项使用。

若要手动创建，下面为一系列*必须*步骤：

- 将一个硬盘分区为单一的 ext3/4 分区。
- 安装基础系统到分区中（例如使用 arch-install-scripts包 中的 `pacstrap`）
- 安装引导程序（例如 GRUB 或是 extlinux）
- 安装并配置 cloud-init
- 添加一个可以无密码运行 sudo 的用户
- 为 `eth0` 配置 DHCP
- 配置 udev 将网络接口命名为 `eth*`
- 配置 systemd-networkd 在 `eth0` 上使用 DHCP
- 安装 SSH 服务器

- 调整 initramfs 配置并重新生成 initramfs 镜像
- Disabling the `autodetect` hook (since autodetection works differently from a chroot)
- Either activating hook `growfs` from mkinitcpio-growrootfsAUR or installing `growpart` from cloud-utils包 and have cloud-init do resizing by itself
- 通过 enabling 来使服务自动启动
- 删除生成的密钥（例如 SSH 服务器及 pacman)；可以配置为在系统初次启动时自动生成密钥
- 删除设备ID（`/etc/machine-id` 及 `/var/lib/dbus/machine-id`）以使系统间不会冲突

参阅

- Wikipedia:OpenStack

1.10 VLAN

难度 入门 | **篇幅** 5.8 KB | **所属** 网络

本节将讲到

- 配置
- 排错

翻译状态：

- 本文（或部分内容）译自 VLAN，最近一次同步于 2015-04-28，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

Virtual LANs give you the ability to sub-divide a LAN. Linux can accept **VLAN** tagged traffic and presents each **VLAN ID** as a different network interface (eg: `eth0.100` for **VLAN ID** `100`)

本文介绍如何通过 iproute2包 和 systemd-networkd 或 netctl 配置 VLAN 。

配置

此前 Arch Linux 用 `vconfig` 命令设置 VLANs ，该命令已被 `ip` 命令取代。请确认 iproute2包 已安装。

下面的范例假定**网口**是 `eth0`，**名字**是 `eth0.100` ， **vlan id** 是 `100`。

创建 VLAN 设备

用下列命令添加 VLAN 网口：

```
# ip link add link eth0 name eth0.100 type vlan id 100
```

执行 `ip link` 命令确认 VLAN 已创建。

这个 VLAN 网口就像一个普通的物理网口，所有流经这个网口的数据包将被加上 VLAN tag 并流经它关联的物理网口（本例中的 `eth0`）。仅配置为相同 VLAN 的设备可接收这些数据包，否则将被丢弃。 Using a **name** like `eth0.100` is just convention and not enforced; you can alternatively use `eth0_100` or something descriptive like `IPTV`. To see the VLAN ID on an interface, in case you used an unconventional name:

```
# ip -d link show eth0.100
```

The `-d` flag shows full details on an interface:

```
# ip -d addr show
4: eno1.100@eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
group default
   link/ether 96:4a:9c:84:36:51 brd ff:ff:ff:ff:ff:ff promiscuity 0
   vlan protocol 802.1Q id 100 <REORDER_HDR>
   inet6 fe80::944a:9cff:fe84:3651/64 scope link
      valid_lft forever preferred_lft forever
```

添加 IP

Now add an IPv4 address to the just created vlan link, and activate the link:

```
# ip addr add 192.168.100.1/24 brd 192.168.100.255 dev eth0.100
# ip link set dev eth0.100 up
```

关闭设备

To cleanly shutdown the setting before you remove the link, you can do:

```
# ip link set dev eth0.100 down
```

移除设备

ex Removing a VLAN interface is significantly less convoluted

```
# ip link delete eth0.100
```

开机启动

systemd-networkd

Use the following configuration files:

```
/etc/systemd/network/eno1.network
[Match]
Name=eno1

[Network]
DHCP=v4
VLAN=eno1.100
VLAN=eno1.200
/etc/systemd/network/'eno1.100.netdev
[Netdev]
Name=eno1.100
Kind=vlan
```

```
[VLAN]
Id=100
/etc/systemd/network/'eno1.200.netdev
[Netdev]
Name=eno1.200
Kind=vlan

[VLAN]
Id=200
```

Then enable `systemd-networkd.service`. See systemd-networkd for details.

netctl

You can use netctl for this purpose, see the self-explanatory example profiles in {{ic|/etc/netctl/examples/vlan-{dhcp,static} }}.

排错

udev 重命名虚拟设备

An annoyance is that udev may try to rename virtual devices as they are added, thus ignoring the **name** configured for them (in this case `eth0.100`).

For instance, if the following commands are issued:

```
# ip link add link eth0 name eth0.100 type vlan id 100
# ip link show
```

This could generate the following output:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP qlen 1000
    link/ether aa:bb:cc:dd:ee:ff brd ff:ff:ff:ff:ff:ff
3: rename1@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state DOWN
    link/ether aa:bb:cc:dd:ee:ff brd ff:ff:ff:ff:ff:ff
```

udev has ignored the configured virtual interface name `eth0.100` and autonamed it **rename1**.

The solution is to edit `/etc/udev/rules.d/network_persistent.rules` and append **DRIVERS=="?*"** to the end of the physical interface's configuration line.

For example, for the interface **aa:bb:cc:dd:ee:ff** (eth0):

```
/etc/udev/rules.d/network_persistent.rules
SUBSYSTEM=="net", ATTR{address}=="aa:bb:cc:dd:ee:ff", NAME="eth0", DRIVERS=="?*"
```

A reboot should mean that VLANs configure correctly with the names assigned to them.

1.11 网桥

难度 进阶 | **篇幅** 6.3 KB | **所属** 网络

本节将讲到

- 创建网桥
- 分配 IP 地址
- 提示与技巧
- 参阅

网桥是一种软件配置，用于连结两个或更多个不同网段。网桥的行为就像是一台虚拟的网络交换机，工作于透明模式（即其他机器不必关注网桥的存在与否）。任意的真实物理设备（例如 `eth0`）和虚拟设备（例如 `tap0`）都可以连接到网桥。

本文讲述如何创建至少包含一个以太网设备的网桥，这种应用常见于QEMU、设置基于软件的访问点等场景。

创建网桥

创建网桥有多种途径。

通过 bridge-utils

本节讲述用 bridge-utils包 软件包里面的 *brctl* 工具管理网桥。该软件包已进入官方仓库（official repositories）。*brctl* 的完整选项清单请参阅 brctl(8)。

新建一个网桥：

```
# brctl addbr bridge_name
```

添加一个设备（例如`eth0`）到网桥：

```
# brctl addif bridge_name eth0
```

显示当前存在的网桥及其所连接的网络端口：

```
$ brctl show
```

启动网桥：

```
# ip link set up dev bridge_name
```

删除网桥，需要先关闭它：

```
# ip link set dev bridge_name down
# brctl delbr bridge_name
```

通过 iproute2

本节讲述用 iproute2包 软件包里面的 *ip* 工具管理网桥。该软件包包含在 base包 包组中。

创建一个网桥并设置其状态为已启动：

```
# ip link add name bridge_name type bridge
# ip link set dev bridge_name up
```

添加一个网络端口（比如 eth0）到网桥中，要求先将该端口设置为混杂模式并启动该端口：

```
# ip link set dev eth0 promisc on
# ip link set dev eth0 up
```

把该端口添加到网桥中，再将其所有者设置为 `*bridge_name*` 就完成了配置：

```
# ip link set dev eth0 master bridge_name
```

要显示现存的网桥及其关联的端口，可以用 *bridge* 工具（它也是 iproute2包 的组成部分）。详阅 bridge(8)。

```
# bridge link show
```

若要删除网桥，应首先移除它所关联的所有端口，同时关闭端口的混杂模式并关闭端口以将其恢复至原始状态。

```
# ip link set eth0 promisc off
# ip link set eth0 down
# ip link set dev eth0 nomaster
```

当网桥的配置清空后就可以将其删除：

```
# ip link delete bridge_name type bridge
```

通过 netctl

参阅 Bridge with netctl.

通过 systemd-networkd

参阅 systemd-networkd#Bridge interface.

通过 NetworkManager

Gnome's NetworkManager can create bridges, but currently will not auto-connect to them. Open Network Settings, add a new interface of type Bridge, add a new bridged connection, and select the MAC address of the device to attach to the bridge.

Now, find the UUID of the attached device (by default named "bridge0 slave 1"):

```
$ nmcli connection
```

Finally, enable that connection:

```
$ nmcli con up <UUID>
```

If NetworkManager's default interface for the device you added to the bridge connects automatically, you may want to disable that by clicking the gear next to it in Network Settings, and unchecking "Connect automatically" under "Identity."

分配 IP 地址

When the bridge is fully set up, it can be assigned an IP address:

```
# ip addr add dev bridge_name 192.168.66.66/24
```

提示与技巧

网桥使用无线网络端口

To add a wireless interface to a bridge, you first have to assign the wireless interface to an access point or start an access point with hostapd. Otherwise the wireless interface will not be added to the bridge.

See also Bridging with a wireless NIC on Debian wiki.

参阅

- Official documentation for bridge-utils
- Official documentation for iproute2

1.12 MentoHUST

难度 进阶 | **篇幅** 9.4 KB | **所属** 网络

本节将讲到

- 安装
- 配置与运行
- 故障排除
- 二次拨号
- 参见

本页面正在考虑归档。

原因： The package in the AUR, as well as the upstream package, has stopped maintenance.（在 Talk:MentoHUST 中讨论）

锐捷是国内众多大学采用的拨号客户端，但其 Linux 版久无更新，很多使用它的用户无法通过认证，少数能通过但容易掉线。虽然网上第三方 Linux 版锐捷客户端不少，但都大同小异，难以通过锐捷的客户端校验。

本条目旨在提供一个在 Linux 下能与锐捷兼容性很好的认证客户端 MentoHUST 使用教程，方便 Arch Linux 用户实现锐捷拨号并上网。

安装

安装 AUR 中的 mentohust-gitAUR 即可。

注意：mentohust-gitAUR 目前还在开发中，主要针对锐捷v4算法，暂不兼容旧版本的mentohust，若使用出现问题，可尝试旧版本的mentohust,亦可安装AUR中的旧版本mentohustAUR 。

配置与运行

网络参数配置

MentoHUST 直接依赖 Arch Linux 内置的相关网络参数，例如 IP、网关、DNS 等等，需要事先进行此方面的配置。

注意：以下包含的 `eth0` 为一种网卡，且每种 PC 所拥有的网卡名称却不一定是这个，请自行执行 `ip link` 以查询本机上正确的网卡名称即可。

不论获取 IP 方式如何，用户需事先从校方相关部门索取用户名和密码。

静态 IP 用户

首先一般需要事先从校方相关部门索取 IP、子网掩码、网关、DNS 信息等，不妨假设分别为 `10.10.45.49, 24, 10.10.173.1, 210.32.24.21`，以及主机所用网络接口为`eth0`。执行以下命令设置即可：

启动网络接口

```
$ ip link set eth0 up
```

在其网络接口上设置 IP（后缀紧跟了子网掩码）

```
$ ip addr add 10.10.45.49/24 dev eth0
```

在路由表上添加网关记录

```
$ ip route add default via 10.10.45.1
```

在 `/etc/resolv.conf` 上添加 DNS 地址

```
$ echo "nameserver 210.32.24.21 > /etc/resolv.conf
```

动态 IP 用户

首先同样地要启动网络接口

```
$ ip link set eth0 up
```

但无须设置其它网络参数，直接启动 dhpcd 即可：

```
$ systemctl start dhcpcd@eth0.service
```

MentoHUST 配置

MentoHUST 参数丰富，以最大程度适应不同学校的不同锐捷认证环境。本程序使用配置文件 `/etc/mentohust.conf` 保存参数，虽然该配置文件是还算标准的 ini 格式文件，并不复杂。但还是有人因多加空格或；导致配置出现问题，所以不提倡手工修改配置文件来设置参数，而是直接靠命令参数配置。

参数说明

`-h` 显示帮助信息。

`-k` MentoHUST 支持 daemon 运行，也就是认证成功后可以关闭终端而认证不会中断。当进入 daemon 运行方式后，是不能像没有进入这一模式时一样通过Ctrl+C退出的，这时如果需要退出，可以使用以下命令：

```
$ mentohust -k
```

或者

```
$ pkill mentohust
```

`-w` 在命令行参数中指定的参数默认不会保存到配置文件，如果需要保存，请加上该参数，例如下面这个命令将会把用户名更新为hust，密码更新为123456。

```
$ mentohust -u hust -p 123456 -w
```

`-u`，`-p`，`-n` 分别指定用户名、密码、网卡。如果均不指定，就会自动判断是否需要输入。

`-i`，`-m`，`-g`，`-s` 静态 IP 用户可通过这些参数分别指定学校分配的 IP、子网掩码、网关、DNS。但由于已经在#网络参数配置事先设定好，一般不用再考虑。

`-o` 指定智能重连时用来 ping 的目标 IP，例如未认证时，1.2.3.4 无法 ping 通，认证成功后可以 ping 通，就可以加上参数 `-o 1.2.3.4`，当掉线且未收到服务器下线通知时会在掉线1分钟之内重连。除非网络不好，一般不会掉线，掉线且收到服务器下线通知时会在掉线后立即重连。

`-t` 指定认证时多少秒后仍未收到服务器回应则重启认证，一般保持默认即可。

`-e` 指定认证成功后每隔多少秒向服务器发送一次数据以表明自己仍然在线，一般保持默认即可。

`-r` 由于有些学校会规定认证失败后一定时间内不允许再次认证，所以在这期间不论发多少数据服务器都不会响应，为了减少这种垃圾数据，MentoHUST 会在认证失败后等待一段时间或者服务器向客户端请求数据时再认证，这个时间就由此参数指定，一般保持默认即可。 `-r15` 并不是说在认证失败后15秒才会再次认证，如果在15秒内服务器发来一个数据包要求开始认证，MentoHUST会放弃等待，立即开始再次认证。

`-a` 指定组播地址或客户端类型，`-a0` 为标准 `-a1` 为锐捷私有，这两个分别对应于锐捷中的标准和私有，有些学校只能用标准，有些学校只能用私有，所以如果提示“找不到服务器”而网卡并没有选错，就检查是不是这里设置错了。`-a2` 表示将MentoHUST用于赛尔认证（赛尔用 `-a0` 标准也行）。

`-d` 指定 DHCP 方式，使用动态 IP 的同学应该在这里正确设置，一般不是 `1` 就是 `2`，如果用 `3` 认证成功却无法上网，请改成 `1` 试试。使用静态 IP 的同学应该将这里设为 `0`。

`-b` 指定后台（daemon）运行方式，`-b0` 不后台运行，认证成功后，不能关闭终端；`-b1`、`-b2` 均为后台运行，但前者无输出，后者则保留输出；`-b3` 后台运行并将输出保存到 `/tmp/mentohust.log`，可以随时打开该文件查看输出。

`-y` 指定是否显示通知（notify），`-y0` 不显示，`1~20` 显示，其中数字指定通知持续时间。

`-c` 指定动态 IP 用户 DHCP 时运行的脚本，一般保持默认即可。如果觉得这个输出太多影响用户，可以改为 `-cdhclient>/dev/null`。

`-q` 很多锐捷用户不清楚他们的 DHCP 方式是什么，本参数用于显示解密后的锐捷 `SuConfig.dat` 文件内容。假如锐捷所在目录为/path/to，则执行以下命令即可：

```
# mentohust -q '/path/to/SuConfig.dat'
```

示范

某用户获取 IP 的方式是静态 IP，用户名为`hust`，密码为`123456`，ping 地址为 `192.168.1.254`，同时以 deamon 方式保存输出到文件。

首先按#网络参数配置配置好 IP、网关、子网掩码、DNS 等，然后执行：

```
# mentohust -u hust -p 123456 -o 192.168.1.254 -b 3 -w
```

运行

```
$ systemctl start mentohust.service
```

若要开机自启动，执行以下一次即可：

```
$ systemctl enable mentohust.service
```

故障排除

提示“在网卡 eth0 上获取 IP 失败”

如果获取 IP 方式是是动态 IP 的话，无须理会；否则按#网络参数配置进行排错。

提示“IP 地址类型错误”

DHCP 方式设置错误。使用 `-q` 参数查看正确的 DHCP 方式并按需修改。

提示“IP 端口绑定错误"

若是静态 IP，原因是在 MentoHUST 中将绑定 IP 设置错误，可以通过 `-i` 参数修改。

提示“找不到服务器”

一般是选错了组播模式，在“标准”与“锐捷”中切换试试。

如果 ping 任何 IP 均出现 `Destination Host Unreacheable` 错误，请检查下网线。

提示“不允许使用的客户端类型”

学校禁用了 xrgsu， 使用 `-v` 参数指定版本号，或者复制相关文件（`8021x.exe` 和 `W32N55.dll`，可能还需要 `SuConfig.dat`）到 `/etc/mentohust/`。

提示“客户端版本过低”

和#提示“不允许使用的客户端类型”的解决方法相同。

提示“客户端完整性被破坏”

说明校方开启了客户端校验，复制相关文件（`8021x.exe` 和 `W32N55.dll`，可能还需要 `SuConfig.dat`）到 `/etc/mentohust/`。

认证成功但仍无法上网

静态 IP 用户未正确设置 IP 及 DNS，或动态 IP 用户未能正确地获取到 IP 及 DNS。

二次拨号

可参见 MentoHUST 及『二次撥號』的臨時解決方案

参见

MentoHUST 在 Google Code 的主页

Ubuntu Wiki 条目『锐捷、赛尔认证MentoHUST』

acgtyrant 在浙江理工大学进行锐捷拨号的自动化脚本

1.13 业余无线电

难度 进阶 | **篇幅** 12.5 KB | **所属** 网络

本节将讲到

- 通用信息
- 硬件
- 软件列表

翻译状态：

- 本文（或部分内容）译自 Amateur radio，最近一次同步于 2025-07-21，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

业余无线电爱好者（又称“火腿”、“HAM”）自从无线电存在的早期就活跃在相关实验与开发的前沿。在各个无线电频段上，有多种常用的通信模式。 本页面列举了 AUR 中与业余无线电有关的软件，其中有些可以独立运行，但多数处理数字通信的程序则需要配合无线电硬件或者声卡使用。相关硬件可以购买，也可以自制。

警告：无线电频谱资源属于国家所有。根据《业余无线电台管理办法》等法律法规规定，在中国大陆，如果你想要发射业余无线电信号，需要获得**中国无线电协会**颁发的**业余无线电台操作证书**以及无线电管理机构颁发的**业余电台执照**。**违规发射将面临严重法律后果。**

通用信息

下面的很多程序都需要使用串口（如: /dev/ttyS0）与发信机通信。首先，你的用户需要在uucp用户组中。要将一个用户添加到uucp组中，以root身份运行下面的命令：

```
$ gpasswd -a 用户名 uucp
```

之后注销再重新登录。

硬件

Digirig是适用于许多无线电硬件的接口之一。尽管有着许多型号，但它们大致上都通过ALSA以“C-Media Electronics USB Audio Device”呈现给操作系统，包含着音频输入/输出的两个通道。通过/dev/ttyUSB0串口设备，Digirig也可用于控制电台。

其它设备，例如ICOM IC-705，具有内置声卡和CAT控制器。你可以直接通过USB连接电台，它应该作为USB音频设备(Burr-Brown USB Audio)和串口设备(/dev/ttyUSB0 和 /dev/ttyUSB1)。

软件列表

- **Hamlib** — 为无线电硬件和控制程序之间提供了一个接口。它只是用于帮助控制电台等硬件(例如记录日志，数字模式等)，并不是一个能独立使用的程序。

: https://sourceforge.net/projects/hamlib/ || hamlib包

- **Soundmodem** — 由Tom Sailer（HB9JNX/AE4WA）编写，它能将声卡作为一个分组无线电的调制解调器，从而能使用多种AX.25通信模式。

: https://web.archive.org/web/20130824024508/http://www.baycom.org/~tom/ham/soundmodem/ || soundmodemAUR

- **Grig** — 基于Hamlib的简单的控制程序。

: http://groundstation.sourceforge.net/grig/ || grigAUR

- **gMFSK** — 支持多种数字模式，使用hamlib和xlog记录日志。

: http://gmfsk.connect.fi || gmfskAUR

- **lysdr** — 一个SDR程序。

: https://github.com/gordonjcp/lysdr || 未被打包？在 AUR 里搜索

- **linrad** — SM5BSZ编写的SDR控制程序。

: https://www.sm5bsz.com/linuxdsp/linrad.htm || linradAUR

- **quisk** — N2ADR编写的SDR控制程序

: https://james.ahlstrom.name/quisk/ || quiskAUR

- **owx** — 使用CSV表格给欧讯电台写频的命令行工具。

: http://owx.chmurka.net || owxAUR

- **fldigi** — W1HKJ开发的GUI程序，支持多种数字通信模式。

: http://w1hkj.com/Fldigi.html || fldigiAUR

- **libfap** — APRS数据包解析程序。

: http://pakettiradio.net/libfap/ || libfapAUR

- **aprx** — 轻量的APRS数字接口与i-Gate接口。

: https://thelifeofkenneth.com/aprx/ || aprxAUR

- **xdx** — 一个DX Cluster客户端。

: https://www.qsl.net/pg4i/linux/xdx.html || xdxAUR

- **qsstv** — SSTV(慢扫描电视)编解码程序。

: || qsstvAUR

- **linpsk** — 一个PSK31程序。

: || linpskAUR

- **xpsk31** — 一个PSK31程序，使用GTK+编写GUI。

: https://www.qsl.net/5b4az/pkg/psk31/xpsk31/xpsk31.html || xpsk31AUR

AX.25

AX.25 — 一个广泛应用于分组无线电的数据链路层协议。它既支持有线连接，也支持无线连接（如APRS）。Linux内核中有对AX.25的原生支持。这里有更多信息：guideAUR中有以下软件可用：

- ax25-appsAUR
- ax25-toolsAUR
- libax25AUR
- nodeAUR

: http://www.ax25.net/ || 原生内核已包含

WSJT-X

WSJT-X) (Weak Signal Communication by K1JT) — 提供了丰富的包括针对流星散射、电离层散射和VHF/UHF的EME(月面反射)以及HF天波传播优化的特定数字模式(如FT8和WSPR)。WSJT由诺贝尔物理学奖得主Joe Taylor领导开发，他的业余无线电呼号为KIJT。该程序可以解码流星余迹反射通信中几分之一秒的信号和低于可听阈值10 dB的稳定信号。WSJT需要串口访问权限，确保你的用户在uucp用户组中(见上文)。

: https://wsjt.sourceforge.io/wsjtx.html || wsjtxAUR

WSJT-X_Improved

wsjt-x_improved by DG2YCB — 是WSJT-X的加强版。WSJT-X_Improved提供了为DX(Distant eXchange，远距离通信)特殊的优化和其它创新性的提升和特性。有三个不同的可用软件包，它们分别采用标准GUI, Alternate Layout (AL) GUI, 和 Widescreen GUI。

: https://sourceforge.net/projects/wsjt-x-improved || wsjtx-improvedAUR | wsjtx-improved-alAUR | wsjtx-improved-widescreenAUR

JTDX_Improved

jtdx_improved by DG2YCB — 是一个JTDX(由Igor Chernikov et al开发，https://sourceforge.net/projects/jtdx/)的加强版。DG2YCB和Igor的两个版本都是基于WSJT-X的。有两个不同的GUI布局可供选择，分别为改进版GUI和标准JTDX GUI。

: https://sourceforge.net/projects/jtdx-improved || jtdx-improvedAUR | jtdx-improved-jtdxguiAUR

Xastir

Xastir — 意思是X Amateur Station and Information Reporting，是一个APRS客户端。Xastir是一个开源程序，提供对APRS的完全支持。该软件正处于一个积极开发的阶段当中。 Xastir具有很高的灵活性，有许多配置它的方法。例如，在有网络连接的情况下，可以通过网络进行操作而无需无线电设备。https://xastir.org 上的wiki非常详细，包含了关于软件功能和软件设置的所有信息。 该软件具有一个可选的语音功能，可以在安装软件包festival后启用。此外还需要安装如festival-en或festival-english这样的语音库软件包。如果想要启用该功能，festival必须在编译xastir之前安装到系统上。在启动xastir之前，运行festival：

```
$ festival --server
```

或者也可以写一个简单的脚本来自动化这个按顺序的启动程序。如果有其它程序(像是一个播放器)正在同时访问音频，可能会出现问题。PKGBUILD会自动下载850kB大小的一组.wav文件放置在`/usr/share/xastir/sounds/`中。它们是一个不需要

festival渲染的北美英语的警报录音。configure菜单的音频播放命令`play`可能不起作用，请用`aplay`代替。

: http://www.xastir.org || xastirAUR

数字语音

FreeDV — 是一个用于 HF 频段的数字语音模式。它使用开源免费的 Codec2 语音编解码器，可以在窄带宽、低比特率的情况下高效率进行通信，特别适用于短波无线电通联。使用FreeDV所需要的只有一台运行Free DV GUI程序的计算机，以及 SSB 模式的电台。对于 Arch Linux，FreeDV 和 Codec2 都可以从 AUR 获取到。两者都需要安装好才能使用 FreeDV。

- https://www.rowetel.com/codec2.html codec2包

: https://freedv.org || freedvAUR

分析工具

- gpredictAUR – 用于实时卫星追踪、卫星轨道预测的软件
- hamsolarAUR – 在桌面上显示当前太阳活动指数
- splatAUR – 无线电信号传播、损耗和地形分析
- sunclockAUR – 用于预测灰线传播路径
- xnec2cAUR – 天线模拟软件

通联日志

- cqrlog-binAUR – 常见的Linux日志程序
- fdlogAUR – 一个有网络节点的Field Day日志记录器
- klogAUR – 运行在KDE上的业余无线电日志程序
- qleAUR – QSO 日志记录器和编辑器，用 Perl 编写
- tlfAUR – 一个日志与竞赛控制台程序
- trustedqslAUR – 用于上传LoTW(Logbook of the World)日志
- xlogAUR – 一个业余无线电日志系统
- yfklogAUR – 通用的*nix业余无线电日志程序
- yfktestAUR – 无线电爱好者的通联竞赛日志程序

工具

- ctyAUR – 包含实体（国家）、前缀、呼号等信息，供业余无线电日志程序使用
- dxccAUR – 一个确认指定呼号DXCC实体的小程序

摩尔斯电码练习工具

- aldoAUR
- cutecwAUR
- ebook2cwAUR
- gtkmmorseAUR
- kochmorseAUR
- qrqAUR
- unixcwAUR

其他

- cwircAUR – 通过IRC收发摩尔斯电码

1.14 进阶网络流量控制

难度 进阶 | **篇幅** 13.0 KB | **所属** 网络

本节将讲到

- 排队
- Filters
- Example of ingress traffic shaping with SNAT
- See also

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： Only the intro is translated.（在 Talk:进阶网络流量控制# 中讨论）

Linux内核的网络模块具有网络流量控制功能。包iproute2包安装了`tc` 命令通过命令行来操控。 这篇文章展示了如何使用排队规则来控制流量。比如，若因为用户滥用网络带宽，要禁止所管理的网络范围内下载或种子下载行为，可以用排队规则(queueing disciplines)，禁调此类流量，保证整个网络的速度。 这篇文章需要读者对于网络设备、iptables有一定的了解。

排队

排队控制着数据的**发送**方式；接收数据的反应性更强，面向网络的控制更少。然而，由于TCP/IP数据包是使用慢速启动发送的，因此系统开始缓慢发送数据包，并保持越来越快的发送速度，直到数据包开始被拒绝——因此，可以通过在转发之前丢弃到达路由器的数据包来控制局域网上接收的流量。还有更多相关细节，但它们没有直接涉及排队逻辑。为了完全控制流量的大小，我们需要成为链中最慢的环节。也就是说，如果连接的最大下载速度为500k，如果你不将输出限制在450k或以下，那么将由调制

解调器而不是我们来限制流量。每个网络设备都有一个*根*目录，可以在其中设置qdisc。默认情况下，此根具有fq_codel qdisc。（更多信息见下文）

有两种类型：有分类和无分类。

类qdisc允许您创建类，其工作方式类似于树上的分支。然后，您可以设置规则，将数据包过滤到每个类中。每个类本身都可以分配其他有类或无类qdisc。

无类qdisc不允许向其中添加更多qdisc。在开始配置qdisc之前，首先我们需要从根目录中删除任何现有的qdisc。这将从eth0设备中删除任何qdisc：

```
# tc qdisc del root dev eth0
```

无类Qdiscs

这些队列通过重新排序、减慢或丢弃数据包来对流量进行基本管理。此qdiscs不允许创建类。fifo_fast这是systemd 217之前的默认qdisc。在没有应用自定义qdisc配置的每个网络设备中，fifo_fast是根上设置的qdisc。fifo的意思是先进先出，也就是说，第一个进入的数据包将是第一个被发送的。这样，没有包裹会得到特殊待遇。令牌桶过滤器（TBF）只要不超过特定的速率限制，此qdisc就允许传递字节。它的工作原理是创建一个虚拟桶，然后以一定的速度丢弃代币，填充该桶。每个包都从bucket中获取一个虚拟令牌，并使用它来获得通过的权限。如果到达的数据包太多，bucket将没有剩余的令牌，剩余的数据包将等待一定时间以获得新的令牌。如果令牌到达的速度不够快，数据包将被丢弃。在相反的情况下（发送的数据包太少），令牌可用于允许一些突发（上传尖峰）发生。

这意味着这个 qdisc 在减慢界面速度方面很有用。

举例:

上传可能会填满调制解调器的队列，因此，当您上传一个巨大的文件时，交互性会被破坏。

```
# tc qdisc add dev ppp0 root tbf rate 220kbit latency 50ms burst 1540
```

请注意，上述上传速度应更改为您的上传速度减去几个百分点（即链中最慢的环节）。此配置为`ppp0` 设备设置TBF，将上传速度限制为220k，为丢弃前的包设置50ms的延迟，并设置1540的突发。 它的工作原理是将排队保持在Linux机器上（可以在那里进行整形），而不是调制解调器上。

Stochastic Fairness Queueing (SFQ)

This is a round-robin qdisc. Each conversation is set on a fifo queue, and on each round, each conversation has the possibility to send data. That is why it is called "Fairness". It is also called "Stochastic" because it does not really create a queue for

each conversation, instead it uses a hashing algorithm. For the hash, there is a chance for multiple sessions on the same bucket. To solve this, SFQ changes its hashing algorithm often to prevent that this becomes noticeable.

Example:

This configuration sets SFQ on the root on the eth0 device, configuring it to perturb (alter) its hashing algorithm every 10 seconds.

```
# tc qdisc add dev eth0 root sfq perturb 10
```

CoDel and Fair Queueing CoDel

Since systemd 217, fq_codel is the default. CoDel (Controlled Delay) is an attempt to limit buffer bloating and minimize latency in saturated network links by distinguishing good queues (that empty quickly) from bad queues that stay saturated and slow. The fair queueing Codel utilizes fair queues to more readily distribute available bandwidth between Codel flows. The configuration options are limited intentionally, since the algorithm is designed to work with dynamic networks, and there are some corner cases to consider that are discussed on the bufferbloat wiki concerning Codel, including issues on very large switches and sub megabit connections.

Additional information is available via the tc-codel(8) and tc-fq_codel(8).

警告：Make sure your ethernet driver supports Byte Queue Limits before using CoDel. Here is a list of drivers supported as of kernel 3.6

Classful Qdiscs

Classful qdiscs are very useful if you have different kinds of traffic which should have differing treatment. A classful qdisc allows you to have branches. The branches are called classes.

Setting a classful qdisc requires that you name each class. To name a class,the `classid` parameter is used . The `parent` parameter, as the name indicates, points to the parent of the class.

All the names should be set as `x:y` where `x` is the name of the root, and `y` is the name of the class. Normally, the root is called `1:` and its children are things like `1:10`

Hierarchical Token Bucket (HTB)

HTB is well suited for setups where you have a fixed amount of bandwidth which you want to divide for different purposes, giving each purpose a guaranteed bandwidth, with the possibility of specifying how much bandwidth can be borrowed. Here is an example with comments explaining what each line does:

```
# This line sets a HTB qdisc on the root of eth0, and it specifies that the class 1:30
is used by default. It sets the name of the root as 1:, for future references.
tc qdisc add dev eth0 root handle 1: htb default 30

# This creates a class called 1:1, which is direct descendant of root (the parent is
1:), this class gets assigned also an HTB qdisc, and then it sets a max rate of 6mbits,
with a burst of 15k
```

```
tc class add dev eth0 parent 1: classid 1:1 htb rate 6mbit burst 15k

# The previous class has this branches:

# Class 1:10, which has a rate of 5mbit
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 5mbit burst 15k

# Class 1:20, which has a rate of 3mbit
tc class add dev eth0 parent 1:1 classid 1:20 htb rate 3mbit ceil 6mbit burst 15k

# Class 1:30, which has a rate of 1kbit. This one is the default class.
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 1kbit ceil 6mbit burst 15k

# Martin Devera, author of HTB, then recommends SFQ for beneath these classes:
tc qdisc add dev eth0 parent 1:10 handle 10: sfq perturb 10
tc qdisc add dev eth0 parent 1:20 handle 20: sfq perturb 10
tc qdisc add dev eth0 parent 1:30 handle 30: sfq perturb 10
```

Filters

Once a classful qdisc is set on root (which may contain classes with more classful qdiscs), it is necessary to use filters to indicate which package should be processed by which class.

On a classless-only environment, filters are not necessary.

You can filter packets by using tc, or a combination of tc + iptables.

Using tc only

Here is an example explaining a filter:

```
# This command adds a filter to the qdisc 1: of dev eth0, set the
# priority of the filter to 1, matches packets with a
# destination port 22, and make the class 1:10 process the
# packets that match.
tc filter add dev eth0 protocol ip parent 1: prio 1 u32 match ip dport 22 0xffff flowid
1:10

# This filter is attached to the qdisc 1: of dev eth0, has a
# priority of 2, and matches the ip address 4.3.2.1 exactly, and
# matches packets with a source port of 80, then makes class
# 1:11 process the packets that match
tc filter add dev eth0 parent 1: protocol ip prio 2 u32 match ip src 4.3.2.1/32 match
ip sport 80 0xffff flowid 1:11
```

Using tc + iptables

iptables has a method called fwmark that can be used to mark packets across interfaces.

First, this makes packets marked with 6, to be processed by the 1:30 class

```
# tc filter add dev eth0 protocol ip parent 1: prio 1 handle 6 fw flowid 1:30
```

This sets that mark 6, using iptables

```
# iptables -A PREROUTING -t mangle -i eth0 -j MARK --set-mark 6
```

You can then use iptables normally to match packets and then mark them with fwmark.

Example of ingress traffic shaping with SNAT

Qdiscs on ingress traffic provide only policing with no shaping. In order to shape ingress, the IFB (Intermediate Functional Block) device has to be used. However, another problem arises if SNAT or MASQUERADE is in use, as all incoming traffic has the same destination address. The Qdisc intercepts the incoming traffic on the external interface before reverse NAT translation so it can only see the router's IP as destination of the packets.

The following solution is implemented on OpenWRT and can be applied to Arch Linux: First the outgoing packets are marked with MARK and the corresponding connections (and related connections) with CONNMARK. On the incoming packets an ingress u32 filter redirects the traffic to IFB (action mirred), and also retrieves the mark of the packet from CONNTRACK (action connmark) thus providing information as to which IP behind the NAT initiated the traffic).

This function is integrated in kernel since linux包\-3.19 and in iproute2包 since 4.1.

The following is a small script with only 2 HTB classes on ingress to demonstrate it. Traffic defaults to class 3:30. Outgoing traffic from 192.168.1.50 (behind NAT) to the Internet is marked with "3" and thus incoming packets from the Internet going to 192.168.1.50 are marked also with "3" and are classified on 3:33.

```
#!/bin/sh -x

# Maximum allowed downlink. Set to 90% of the achievable downlink in kbits/s
DOWNLINK=1800

# Interface facing the Internet
EXTDEV=enp0s3

# Load IFB, all other modules all loaded automatically
modprobe ifb
ip link set dev ifb0 down

# Clear old queuing disciplines (qdisc) on the interfaces and the MANGLE table
tc qdisc del dev $EXTDEV root    2> /dev/null > /dev/null
tc qdisc del dev $EXTDEV ingress 2> /dev/null > /dev/null
tc qdisc del dev ifb0 root       2> /dev/null > /dev/null
tc qdisc del dev ifb0 ingress    2> /dev/null > /dev/null
iptables -t mangle -F
iptables -t mangle -X QOS

# appending "stop" (without quotes) after the name of the script stops here.
if [ "$1" = "stop" ]
then
        echo "Shaping removed on $EXTDEV."
        exit
fi

ip link set dev ifb0 up

# HTB classes on IFB with rate limiting
tc qdisc add dev ifb0 root handle 3: htb default 30
tc class add dev ifb0 parent 3: classid 3:3 htb rate ${DOWNLINK}kbit
tc class add dev ifb0 parent 3:3 classid 3:30 htb rate 400kbit ceil ${DOWNLINK}kbit
tc class add dev ifb0 parent 3:3 classid 3:33 htb rate 1400kbit ceil ${DOWNLINK}kbit

# Packets marked with "3" on IFB flow through class 3:33
tc filter add dev ifb0 parent 3:0 protocol ip handle 3 fw flowid 3:33
```

```
# Outgoing traffic from 192.168.1.50 is marked with "3"
iptables -t mangle -N QOS
iptables -t mangle -A FORWARD -o $EXTDEV -j QOS
iptables -t mangle -A OUTPUT -o $EXTDEV -j QOS
iptables -t mangle -A QOS -j CONNMARK --restore-mark
iptables -t mangle -A QOS -s 192.168.1.50 -m mark --mark 0 -j MARK --set-mark 3
iptables -t mangle -A QOS -j CONNMARK --save-mark

# Forward all ingress traffic on internet interface to the IFB device
tc qdisc add dev $EXTDEV ingress handle ffff:
tc filter add dev $EXTDEV parent ffff: protocol ip \
        u32 match u32 0 0 \
        action connmark \
        action mirred egress redirect dev ifb0 \
        flowid ffff:1

exit 0
```

See also

- Linux Advanced Routing & Traffic Control
- Wikipedia page for the tc command)

1.15 Systemd-resolved

难度 进阶 | **篇幅** 15.5 KB | **所属** 网络

本节将讲到

- 安装
- 配置
- 查找
- 故障排除
- 另请参阅

翻译状态：

- 本文（或部分内容）译自 Systemd-resolved，最近一次同步于 2025-01-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

*systemd-resolved*是一个通过D-Bus接口、NSS(nss-resolve(8))解析服务和一个监听`127.0.0.53`的本地DNS解析器为本地应用提供网络名称解析的systemd服务。

安装

*systemd-resolved*是默认安装的systemd包包的一部分。

配置

*systemd-resolved*为DNS（包括DNSSEC和Dns over TLS）、多播DNS（mDNS）和链路本地多播名称解析（LLMNR）提供解析服务。

可以通过编辑/etc/systemd/resolved.conf
或 /etc/systemd/resolved.conf.d/中的附加配置片段文件来配置解析器。请参阅resolved.conf(5)。

要使用*systemd-resolved*，启用并启动systemd-resolved.service。

提示：要了解有关选项和开关的上下文，可以如Systemd#诊断一个服务中所述打开*systemd-resolved*的详细调试信息。

DNS

依赖 glibc getaddrinfo(3)（或类似API）的软件直接就能使用。如果存在nss-resolve(8)，/etc/nsswitch.conf的默认配置将使用它进行主机名解析。

有些软件会直接读取/etc/resolv.conf进行域名解析，例如网络浏览器、Go 和 GnuPG 等，为了支持这些软件，*systemd-resolved*提供了四种不同的文件处理模式：本地、静态、上联和外部（file-stub, static, uplink和foreign），systemd-resolved(8) § /ETC/RESOLV.CONF中对这些方式进行了详细说明。

/run/systemd/resolve/stub-resolv.conf包含作为唯一DNS服务器和搜索域名的本地存根127.0.0.53。这是将*systemd-resolved*托管配置传播到所有客户端的推荐操作模式。要使用它，将/etc/resolv.conf替换为指向它的符号链接：

```
# ln -sf ../run/systemd/resolve/stub-resolv.conf /etc/resolv.conf
```

注意：

- 以../开头的目标路径是*相对于链接位置的*，而不是当前目录。
- 未正确配置/etc/resolv.conf将会导致DNS解析中断。
- 在*arch-chroot*内部时将无法创建/etc/resolv.conf符号链接，因为该文件从外部系统绑定安装。应该在chroot外创建符号链接。例如：

```
# ln -sf ../run/systemd/resolve/stub-resolv.conf /mnt/etc/resolv.conf
```

设置DNS服务器

提示：要检查*systemd-resolved*当前正在使用的DNS, 请运行resolvectl status.

自动配置

对于使用/etc/resolv.conf的网络管理器，*systemd-resolved*将开箱即用，不需要特定的配置，因为*systemd-resolved*将通过/etc/resolv.conf符号链接探测，systemd-networkd、NetworkManager和iwd就是这样。

但是，如果DHCP和VPN客户端使用resolvconf程序来设置名称服务器和搜索域名（请参阅来获得使用*resolvconf*程序的软件的列表）则需要额外的软件包systemd-resolvconf包来提供/usr/bin/resolvconf符号链接。

注意：

- *systemd-resolved*有一个受限的*resolvconf*接口，可能与所有客户端一起工作，请参阅resolvectl(1) § COMPATIBILITY WITH RESOLVCONF(8))来获得更多信息。
- systemd-resolvconf包只在`systemd-resolved.service`运行时才工作。 如果你没有使用*systemd-resolved*确保systemd-resolvconf包包已经删除，否则会导致需要使用`/usr/bin/resolvconf`二进制文件的网络软件出问题。

手动配置

在本地（stub）和静态（static）模式中，自定义DNS服务器可以在resolved.conf(5)文件中设置：

```
/etc/systemd/resolved.conf.d/dns_servers.conf
[Resolve]
DNS=192.168.35.1 fd7b:d0bd:7a6e::1
Domains=~.
```

注意：

- 在resolved.conf(5)中没有`Domains=~.`选项时，如果在per-link的配置中，其中任何一个设置了`Domains=~.`， *systemd-resolved*可能使用每per-link的DNS服务器。
- 此选项不会影响任何在per-link配置中指定了更具体的搜索域名的域名解析.

要获得更多关于per-link配置的信息，请参阅Systemd-networkd#network_文件。

备用

如果*systemd-resolved*没有从网络管理器收到任何DNS服务器地址并且没有DNS服务器在Systemd-resolved#手动配置中配置了， *systemd-resolved*会回退到备用DNS地址来确保DNS解析始终工作。

注意：备用DNS依次为：Cloudflare、Quad9和Google。请参阅定义DNS服务器的systemd PKGBUILD。

地址可以通过编辑resolved.conf(5)中的`FallbackDNS`来更改， 例如：

```
/etc/systemd/resolved.conf.d/fallback_dns.conf
[Resolve]
FallbackDNS=127.0.0.1 ::1
```

要禁用备用DNS功能，请在不指定任何地址的情况下设置`FallbackDNS`：

```
/etc/systemd/resolved.conf.d/fallback_dns.conf
[Resolve]
FallbackDNS=
```

DNSSEC

警告：截至2023年6月， *systemd-resolved*中的DNSSEC支持被认为是实验性和不完整的。 因此，systemd包默认禁用DNSSEC验证。

可以通过更改resolved.conf(5)中的`DNSSEC`设置来启用DNSSEC验证。

- 设置`DNSSEC=allow-downgrade`以仅在上游DNS服务器支持DNSSEC的情况下验证DNSSEC。
- 设置`DNSSEC=true`以始终验证DNSSEC，因此停止不支持它的名称服务器的DNS解析。例如：

```
/etc/systemd/resolved.conf.d/dnssec.conf
[Resolve]
DNSSEC=true
```

通过查询一个具有无效签名的域名来测试DNSSEC验证：

```
$ resolvectl query badsig.go.dnscheck.tools
badsig.go.dnscheck.tools: resolve call failed: DNSSEC validation failed: invalid
```

现在来测试一个具有有效签名的域名：

```
$ resolvectl query go.dnscheck.tools
go.dnscheck.tools: 2604:a880:400:d0::256e:b001 -- link: enp2s0
                   142.93.10.179               -- link: enp2s0

-- Information acquired via protocol DNS in 122.2ms.
-- Data is authenticated: yes; Data was acquired via local or encrypted transport: no
-- Data from: network
```

DNS over TLS

DNS over TLS默认禁用。要启用它，请更改resolved.conf(5)中`[Resolve]`部分的`DNSOverTLS`设置。要启用对你的DNS提供商服务器证书的验证，请在DNS设置中以`*ip_address*#*hostname*`的格式包含它们的主机名。例如：

```
/etc/systemd/resolved.conf.d/dns_over_tls.conf
[Resolve]
DNS=9.9.9.9#dns.quad9.net
DNSOverTLS=yes
```

注意：

- 如果`DNSOverTLS=yes`，使用的DNS服务器必须支持DNS over TLS，否则所有DNS请求都将失败。
- 或者，只有当服务器支持`DNSOverTLS=opportunistic`时，才可以使用DNS over TLS。如果使用的DNS服务器不支持DNS over TLS，*systemd-resolved*将回退到常规的未加密DNS。

ngrep包可用于测试DNS over TLS是否工作，因为DNS over TLS始终使用853端口而且从不使用53端口。当通过DNS over TLS解析主机名时，ngrep命令53端口应该不产生输出，并且853端口应该产生加密输出。

使用Wireshark可以检查 TLS DNS 的详细数据包内容。

额外的监听接口

默认情况下，*systemd-resolved*通过环回接口响应本地应用程序的DNS请求。要使*systemd-resolved*在默认接口之外的额外接口或地址上响应DNS请求，请在resolved.conf(5)中为每个额外接口设置`DNSStubListenerExtra`选项。例如：

```
/etc/systemd/resolved.conf.d/additional-listening-interfaces.conf
[Resolve]
```

```
DNSStubListenerExtra=192.168.10.10
DNSStubListenerExtra=2001:db8:0:f102::10
DNSStubListenerExtra=192.168.10.11:9953
```

提示：这在将*systemd-resolved*用作路由器的DNS服务器时非常有用。

mDNS

*systemd-resolved*可以作为mDNS的解析程序和响应程序工作。

解析器使用"*主机名*.local"命名方案提供主机名（\[网络配置#设置计算机名\]）解析。

只有同时启用systemd-resolved的mDNS支持并且当前活动的网络管理器为网络连接启用了mDNS支持，mDNS才会被激活。

*systemd-resolved*的mDNS支持默认启用，可以通过它的`MulticastDNS`设置禁用（请参阅resolved.conf(5) § OPTIONS）。

启用每个连接的mDNS支持依赖网络管理器：

- 对于systemd-networkd，设置每个连接的设置文件的`[Network]`部分中的`MulticastDNS`选项。你可能也需要在`[Link]`部分中设置`Multicast=yes`。请参阅systemd.network(5)。
- 对于NetworkManager，在网络连接的配置文件的`[connection]`部分中设置`mdns`，可以运行`nmcli connection modify *interface_name* connection.mdns *{yes|no|resolve}*`来设置。请参阅nm-settings(5)。

注意：

- 如果安装了Avahi，请考虑禁用或屏蔽`avahi-daemon.service`和`avahi-daemon.socket`以防止和*systemd-resolved*发生冲突。
- 如果你计划使用mDNS和防火墙确保打开UDP端口 `5353`。

提示：可以通过在`/etc/NetworkManager/conf.d/`中创建配置文件并在`[connection]`部分设置`connection.mdns=2`来设置所有NetworkManager连接的默认值。请参阅NetworkManager.conf(5) § CONNECTION SECTION和。

LLMNR

Link-Local Multicast Name Resolution是由微软创造的主机名解析服务。

LLMNR将只会在systemd-resolved的全局设置（resolved.conf(5) § OPTIONS中的`LLMNR`）和网络管理器的每个连接的设置都启用时才会活动。默认情况下，*systemd-resolved*启用LLMNR响应程序，systemd-networkd和NetworkManager为连接启用它。

- 对于systemd-networkd，选项是`[Network]`部分中的`LLMNR`。请参阅[systemd.network(5) § \[NETWORK\] SECTION OPTIONS](https://man.archlinux.org/man/systemd.network.5#%5BNETWORK%5D_SECTION_OPTIONS)。
- 对于NetworkManager，选项是`[connection]`部分中的`llmnr`。请参阅nm-settings(5) § connection setting。

提示：所有NetworkManager连接的默认值可以通过在`/etc/NetworkManager/conf.d/`中创建配置文件并在`[connection]`部分中设置`connection.llmnr`来进行设置。请参阅NetworkManager.conf(5) § CONNECTION SECTION。

如果你计划使用LLMNR和防火墙，确保打开UDP和TCP端口`5355`。

查找

要解析DNS记录、mDNS或LLMNR主机，你可以使用*resolvectl*组件。

例如，要解析一个DNS记录：

```
$ resolvectl query archlinux.org
archlinux.org: 2a01:4f8:172:1d86::1
               138.201.81.199

-- Information acquired via protocol DNS in 48.4ms.
-- Data is authenticated: no
```

故障排除

systemd-resolved不搜索本地域名

当只给定主机名时，即使`UseDomains=yes`和`Domains=[domain-list]`存在于相应的systemd-networkd的*.network*文件中，并且该文件在`resolv.conf`中生成预期的`search [domain-list]`，*systemd-resolved*也可能不会搜索本地域名。你可以运行`networkctl status`或`resolvectl status`来检查域名搜索是否真的被选中了。

可能的解决方法：

- 禁用LLMNR以使systemd-resolved立刻继续附加DNS后缀。
- 修改`/etc/nsswitch.conf`的`hosts`数据库（例如在`resolve`服务后删除`[!UNAVAIL=return]`选项。）
- 切换到使用完全限定域名（fully-qualified domain names）。
- 使用`/etc/hosts`来解析主机名。
- 回退到使用glibc的`dns`以代替systemd的`resolve`。

systemd-resolved不解析没有后缀的主机名

为了使systemd-resolved解析不完全限定域名的主机名，请向`/etc/systemd/resolved.conf`中添加`ResolveUnicastSingleLabel=yes`。

警告：这将把single-label names转发到可能不在你控制之下的全局DNS服务器。这种行为不符合标准，且可能导致隐私和安全风险。请参阅resolved.conf(5)来获得详细信息。

这似乎只在禁用LLMNR的情况下工作（`LLMNR=no`）。

如果你在使用systemd-networkd，你可能希望DHCP服务器或IPv6路由器提供的域名用作搜索域名，这在默认情况下是禁用的，将它添加到接口的*.network*文件中来启用：

```
[Network]
UseDomains=true
```

你可以使用以下工具检查每个接口的systemd-resolved结果：

```
$ resolvectl domain
```

另请参阅

- 我们通过Francisco Ros在外面发现的systmed-resolved的名称解析问题
- 查看resolvectl(1) § EXAMPLES以获取更多示例。

1.16 IPv6

难度 高级 | **篇幅** 21.0 KB | **所属** 网络

本节将讲到

- 邻居发现（Neighbor discovery，ND）
- 无状态地址自动配置 (SLAAC)
- 隐私拓展
- 稳定的隐私地址
- 静态地址

翻译状态：

- 本文（或部分内容）译自 IPv6，最近一次同步于 2022-11-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

在 Arch Linux 中，IPv6 是默认开启的。

tldp Linux+IPv6-HOWTO 这篇文章比较旧， 且缺乏维护。不过其涵盖了本文所提及的许多主题，从基础的东西讲起并且循序渐进到高级配置。而且还有很多命令的例子。初学者可以先读一读这篇文章再接着阅读本文。

邻居发现（Neighbor discovery，ND）

使用 Ping 命令来 ping `ff02::1` 这个多播地址，会得到本地链路层的所有主机的响应。需要手动指定接口（`%eth0`）

```
$ ping ff02::1%eth0
```

Ping 完之后，您可以用如下命令来获取本地子网中的所有主机。

```
$ ip -6 neigh
```

如果您 Ping 的是 `ff02::2` 这个多播地址，则只有网络中的路由器会响应。

如果您添加一个 `-I *your-global-ipv6*` 选项，本地链路上的主机会以他们的本地链路地址来响应。这种情况下可以省略接口。

```
$ ping -I 2001:4f8:fff6::21 ff02::1
```

用下面这个脚本可以 Ping 所有接口上的所有其他主机，并且把你的地址告诉所有人。

```
#!/usr/bin/bash
declare -a l_ifs
readarray l_ifs < <(/sbin/ip -6 -j address | jq -r '.[] | .ifname ')
for l_if in ${l_ifs[@]} ; do
 echo $l_if
 declare -a l_addrs
 readarray l_addrs < <(/sbin/ip -6 -j address show dev "$l_if" | \
                       jq -r  '..addr_info[].local')
 for l_addr in ${l_addrs[@]} ; do
   echo $l_addr
   ping -c 4 -6 -I "$l_addr" ff02::1%"$l_if"
 done
done
```

无状态地址自动配置 (SLAAC)

想要获得一个IPv6地址，最简单的办法就是用SLAAC。只要您的网络配置好了「无状态地址自动配置」（Stateless address autoconfiguration，简称SLAAC），您的设备就会自动根据路由器广播（Router Advertisement，RA）的前缀为自己配置一个地址。既不需要进一步配置，也不需要专门的软件（如DHCP客户端）

客户端配置

如果您正在使用 netctl，您要做的只是往接口配置文件里加上这么一行：

```
IP6=stateless
```

如果您使用的是 NetworkManager，那么只要网络内有路由器广播，它就会自动配置上 IPv6 地址。

不过需要注意的是，SLAAC只有在网络允许 IPv6 ICMP 包通过的时候才能工作。所以你需要在客户端防火墙上允许 `ipv6-icmp` 类型的数据包传入。如果您用的是 Simple stateful firewall/iptables，加上下面这行就可以了：

```
-A INPUT -p ipv6-icmp -j ACCEPT
```

如果您用的是其他防火墙，请参考对应的文档来放行 `ipv6-icmp` 数据包。

如果你选择的网络管理方案不支持通过SLAAC配置DNS（比如netctl），那么你可以考虑用ndisc6包包里的rdnssd(8)命令来配置DNS。

路由器（网关）配置

要正确给网络内的客户端分配 IPv6，我们要用到一个广播守护进程，一般是radvd 包。其配置过程其实相当简单。参考下面的配置来编辑 `/etc/radvd.conf`文件。

```
# 把LAN替换为你实际的局域网接口
interface LAN {
  AdvSendAdvert on;
  MinRtrAdvInterval 3;
  MaxRtrAdvInterval 10;
  prefix ::/64 {
    AdvOnLink on;
    AdvAutonomous on;
    AdvRouterAddr on;
  };
};
```

以上配置会让客户端自己从 /64 地址段中选择地址来使用。请注意如果配置了 `prefix ::/64`，RADVD会向局域网广播那个接口上的*所有*可用的前缀。如果您想要广播指定的前缀，请手动指定，比如 `prefix 2001:DB8::/64`。`prefix` 节可以根据需要多次重复来广播多个不同前缀。

您可以通过 RDNSS 功能向客户端广播 DNS，以 Google DNS 为例，配置如下：

```
RDNSS 2001:4860:4860::8888 2001:4860:4860::8844 {
};
```

网关必须在所有链上允许 `ipv6-icmp` 类型数据包通过。以 Simple stateful firewall/iptables 为例：

```
-A INPUT -p ipv6-icmp -j ACCEPT
-A OUTPUT -p ipv6-icmp -j ACCEPT
-A FORWARD -p ipv6-icmp -j ACCEPT
```

根据其他防火墙前端进行相应的调整，最后别忘了激活 `radvd.service` 服务。

隐私拓展

当一个客户端通过 SLAAC 获得一个地址时，其 IPv6 地址是经过客户端网络接口广播的前缀和 MAC 地址计算而得出的。这可能会引起隐私问题，因为别人可以轻易通过 IPv6 SLAAC 地址反推客户端 MAC。为了解决这个问题，RFC 4941 中提出了 *IPv6 隐私拓展*标准。启用隐私扩展后，内核会生成一个*临时*地址，这个地址是原来的

SLAAC 地址经过二次计算得出的随机地址。当主动连接到远程服务器时，会优先选择使用私有地址，从而帮助隐藏原始的 SLAAC地址。启用隐私扩展的步骤如下:

向 `/etc/sysctl.d/40-ipv6.conf` 中添加如下内容：

```
# Enable IPv6 Privacy Extensions
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
net.ipv6.conf.nic0.use_tempaddr = 2
...
net.ipv6.conf.nicN.use_tempaddr = 2
```

上面的 `nic0` 到 `nicN` 是你的网络接口。你可以参考网络配置#Listing network interfaces 来列出你的设备上的接口。 `all.use_tempaddr` 或者 `default.use_tempaddr` 参数对于执行 sysctl 设置时已经存在的接口无效，也就是说不会即时生效。

不过当您重启设备之后，隐私拓展应该就会生效了。

dhcpcd

dhcpcd 的默认配置已经包括了 `slaac private`，这会启用 RFC 7217 中定义的「稳定的私有IPv6地址而不是基于硬件的地址」功能。因此，您无需修改任何设置，除非您希望更频繁地获得新的随机地址，而不仅在连接到新网络的时候改变IPv6地址。您还可以设置 `slaac hwaddr` 来使用普通的基于 MAC 生成的固定 IPv6 后缀。

NetworkManager

您可以修改 NetworkManager 的全局配置文件 NetworkManager.conf(5) 或连接配置中的 `ipv6.ip6-privacy` 选项，来决定是否启用 IPv6 隐私拓展。如果全局配置和连接配置都没有设置该选项，NetworkManager 会根据 `/proc/sys/net/ipv6/conf/default/use_tempaddr` 内核参数来决定。

如果您想明确默认启用 IPv6 隐私拓展，请在 NetworkManager.conf(5) 中添加以下内容：

```
/etc/NetworkManager/conf.d/ip6-privacy.conf
[connection]
ipv6.ip6-privacy=2
```

然后应用配置并重新连接所有活动接口。

想要为每一个使用 NetworkManager 管理的接口单独设置 IPv6 隐私拓展功能，请编辑其对应的 `/etc/NetworkManager/system-connections/` 目录下的 keyfile，并把 `ip6-privacy=2` 添加到 `[ipv6]` 一节下方，示例如下：

```
/etc/NetworkManager/system-connections/example_connection.nmconnection
...
[ipv6]
method=auto
ip6-privacy=2
...
```

最后，重新加载连接配置并再次连接。

注意：尽管隐私拓展生成的 `scope global temporary` IPv6 地址看上去永远不会被更新（在其 `valid_lft` 所显示的生命周期内，该地址永远不会变为 `deprecated` 状态），但经过较长时间的验证，这个地址 **确实** 会发生变化。

systemd-networkd

systemd-networkd 同样不遵守 `/etc/sysctl.d/40-ipv6.conf` 中配置的 `net.ipv6.conf.xxx.use_tempaddr` 内核参数，除非您在 *.network* 文件中指定 `IPv6PrivacyExtensions` 选项的值为 `kernel`。

不过 systemd-networkd 遵守其他 IPv6 隐私拓展的内核参数，比如：

```
net.ipv6.conf.xxx.temp_prefered_lft
net.ipv6.conf.xxx.temp_valid_lft
```

注意：`temp_prefered_lft` 才是正确的变量名，「preferred」应该是拼错了

请参考 systemd-networkd 和 systemd.network(5) 以了解更多信息。

ConnMan

在服务文件（比如 `/var/lib/connman/*service*/settings`）中设置：

```
IPv6.privacy=preferred
```

请参考 ConnMan 以了解更多信息。

稳定的隐私地址

另外一个选择，是使用 RFC 7217 中定义的稳定隐私地址。它可以在不暴露 MAC 地址的情况下固定住网络中的 IPv6 地址。

Another option is a stable private IP address (RFC 7217). This allows for IPs that are stable within a network without exposing the MAC address of the interface.

以 `wlan0` 接口为例，使用以下命令来让内核为其生成一个密钥：

```
# sysctl net.ipv6.conf.wlan0.addr_gen_mode=3
```

把接口关闭再启用，然后运行 `ip addr show dev wlan0`，您应该会看到每个 IPv6 地址旁边会有 `stable-privacy` 标志。内核已经为该接口生成了一个 128 位的密钥用于产生稳定 IPv6 地址，您可以运行 `sysctl net.ipv6.conf.wlan0.stable_secret` 来查看这一密钥。我们需要固定这个密钥以让他每次重启后都获得同样的地址，所以请在 `/etc/sysctl.d/40-ipv6.conf` 中进行如下配置：

```
# Enable IPv6 stable privacy mode
net.ipv6.conf.wlan0.stable_secret = 刚才查看的密钥
net.ipv6.conf.wlan0.addr_gen_mode = 2
```

注意：`stable-privacy` 标志 **不会** 被应用到由 dhcpcd 获得的 IPv6 地址上

NetworkManager

NetworkManager 不遵守以上设置，但是它默认已经启用了稳定隐私地址。

静态地址

有些情况下使用静态地址可以增强安全性。使用SLAAC分配动态地址时，你的计算机的地址会由其网卡MAC推算出，这不利于安全，因为即使地址的网络号改变，你的电脑依然可以被追踪到。

要想用netctl分配一个静态地址，可以参照`/etc/netctl/examples/ethernet-static`。如下的部分尤其重要：

```
...
# For IPv6 static address configuration
IP6=static
Address6=('1234:5678:9abc:def::1/64' '1234:3456::123/96')
Routes6=('abcd::1234')
Gateway6='1234:0:123::abcd'
```

注意：如果你只有IPv6连接，那么你要给出IPv6的DNS服务器，例如：

```
DNS=('6666:6666::1' '6666:6666::2')
```

如果你的ISP没有告诉你IPv6的DNS服务器地址，并且你也没有自己的服务器，你可以从resolv.conf这篇文章中选一个。

IPv6与PPPoE

PPPoE的软件包`pppd`提供了对PPPoE之上的IPv6的支持（前提是你的ISP和调制解调器支持）。只需要将如下内容加入`/etc/ppp/pppoe.conf`：

```
+ipv6
```

如果你使用netctl，那么就向你的netctl配置文件加入如下内容：

```
PPPoEIP6=yes
```

地址委派 (DHCPv6-PD)

注意：这个部分是针对自行使用配置网关的内容，不是客户端。如果你使用从市场购得的路由器，请查阅其附带的文档以开启地址委派。

地址委派是一种常见的 IPv6 部署方式，被许多 ISP 所采用。具体的做法是将一个地址前缀分配给用户（局域网），即路由器配置为将不同的前缀分配给不同的子网；ISP 通过 DHCPv6 将地址前缀（通常是`/56`或`/64`）分发出去，DHCP 客户端再将前缀分配给局域网。对于一个拥有两个网卡的简单网关来说，它的工作就是将从 WAN 口（或虚拟接口，比如ppp）获取的前缀分配给局域网。

DHCPv6 客户端需要在 UDP 546 接口上接受传入的连接。对于基于 nftables 的防火墙，可以编辑 `/etc/nftables.conf`：

```
table inet filter {
  chain input {
```

```
    udp dport dhcpv6-client accept
    ...
  }
...
}
```

使用dhcpcd

Dhcpcd在 IPv4 之外也提供了一个完整的支持 DHCPv6-PD 的客户端。如果你使用 `dhcpcd`，需要修改 `/etc/dhcpcd.conf`。你可能已经在用dhcpcd来配置 IPv4,，所以只需要对现有的配置进行小幅修改：

```
duid
noipv6rs
waitip 6
# Uncomment this line if you are running dhcpcd for IPv6 only.
#ipv6only

# use the interface connected to WAN
interface WAN
ipv6rs
iaid 1
# use the interface connected to your LAN
ia_pd 1 LAN
#ia_pd 1/::/64 LAN/0/64
```

这种配置下，客户端会从`WAN`接口获取一个前缀，分配给`LAN`接口。 如果ISP分配的是`/64`的地址，你需要用第二个`ia_pd`选项。 这也会禁用除`WAN`接口之外的所有路由器请求。

提示：查阅手册页**`dhcpcd(8)`**与**`dhcpcd.conf(5)`**获得更多信息。

使用WIDE-DHCPv6

WIDE-DHCPv6是原本由KAME计划开发的DHCPv6开源实现。它在AUR中：wide-dhcpv6AUR。

如果你使用`wide-dhcpv6`，修改`/etc/wide-dhcpv6/dhcp6c.conf`：

```
# use the interface connected to your WAN
interface WAN {
  send ia-pd 0;
};

id-assoc pd 0 {
  # use the interface connected to your LAN
  prefix-interface LAN {
    sla-id 1;
    sla-len 8;
  };
};
```

注意：`sla-len`应设置为满足`(WAN-prefix) + (sla-len) = 64`的值。这里示范的情况是针对一个长度`/56`的前缀，56+8=64。对于前缀长度`/64`的网络，`sla-len`应为`0`。

要启用或运行wide-dhcpv6，使用如下命令。把`WAN`改为连接到ISP的网卡:

```
# systemctl enable/start dhcp6c@WAN.service
```

提示：查阅手册页**`dhcp6c(8)`**与**`dhcp6c.conf(5)`**获取更多信息。

systemd-networkd

配置您的上游(wan)和下游(lan)接口。这将在运行 DHCPv6 客户端的接口上启用 DHCPv6-PD，并通过 IPv6 路由广播在下游网络中分发委派的地址前缀。

```
/etc/systemd/network/wan.network
[Network]
# Use 'yes' instead of 'ipv6' for both ipv4 and ipv6.
DHCP=ipv6
/etc/systemd/network/lan.network
[Network]
IPv6SendRA=yes
DHCPv6PrefixDelegation=yes
```

其他客户端

dhclient 也可以请求前缀；但想要把整个或者部分前缀分配到接口上则需要使用 *dhclient* 退出脚本来完成。请参考 https://github.com/jaymzh/v6-gw-scripts/blob/master/dhclient-ipv6 中的例子。

NAT64

Wikipedia:NAT64 是让只有 IPv6 地址的主机通过使用 NAT 来和 IPv4 主机通信的过渡机制。

Linux 内核并不原生支持 NAT64，但您可以使用以下这些软件包来添加 NAT64 支持：

- **Jool** — SIIT and NAT64 for Linux

: https://nicmx.github.io/Jool/ || jool-dkmsAUR, jool-toolsAUR

- **TAYGA** — NAT64 daemon (unmaintained)

: http://www.litech.org/tayga/ || taygaAUR

禁用 IPv6

注意：Arch直接将IPv6支持编译进内核，因此不能禁用对应的内核模块。

关闭 IPv6 功能

如果您遇到一些问题，可以试着向内核参数加入 `ipv6.disable=1` ，来完全关闭 IPv6 功能， 参考内核参数来了解更多信息.

此外，只加入`ipv6.disable_ipv6=1`内核参数可以保留IPv6功能，但不会向网卡分配IPv6地址。

也可以通过向`/etc/sysctl.d/40-ipv6.conf`加入如下配置来避免系统给网卡分配IPv6地址：

```
# Disable IPv6
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.nic0.disable_ipv6 = 1
...
```

```
net.ipv6.conf.nicN.disable_ipv6 = 1
```
重启 `systemd-sysctl.service` 单元来应用上述设置。

注意你必须在这里清楚地列出所有不需要分配IPv6地址的网卡，仅仅设置`all.disable_ipv6`并不会立刻对已经连接的网卡起作用。

注意：如果通过 *sysctl* 禁用IPv6， 你应该在`/etc/hosts`中删除所有的IPv6主机，否则当你使用主机名连接它们的时候会解析到不通的IPv6地址

其他应用程序

在内核中关闭 IPv6 功能不会阻止应用程序尝试使用 IPv6。多数情况下，这样不会有问题，但如果你发现程序无法正常运行，你应该查阅该应用程序的手册页，以找到关闭 IPv6 的合适方法。

dhcpcd

*dhcpcd*会依然尝试发送 IPv6 路由器请求。要禁用这种行为，可以依照其手册页`dhcpcd.conf (5)`，向`/etc/dhcpcd.conf`加入如下内容：

```
noipv6rs
noipv6
```

NetworkManager

如果要禁用 NetworkManager 的 IPv6 功能，右键点击网络状态图标，然后选择*配置网络连接* > *有线以太网* > 网络连接名称 > *编辑* > *IPv6* > *方法* > *忽略/已禁用*然后点击应用。

你也可以通过命令行来设置，先输入：

```
# nmcli connection modify ConnectionName ipv6.method "disabled"
```
然后再重启连接：

```
# nmcli connection up ConnectionName
```
要确认设置是否被应用，使用 `ip address show` 看看还有没有显示 IPv6 地址（inet6）。或者检查 `/proc/sys/net/ipv6/conf/*interface*/disable_ipv6` 这个内核参数，其值应该为 1。

ntpd

依照Systemd#Drop-in files， 修改`ntpd.service`的启动方式。

```
# systemctl edit ntpd.service
```
这样会产生一个drop-in snippet，替代原有的`ntpd.service`来加载ntpd。参数`-4`关闭了ntpd的IPv6支持。向drop-in snippet中加入如下内容：

```
[Service]
ExecStart=
ExecStart=/usr/bin/ntpd -4 -g -u ntp:ntp
```

第一行清除了之前的`ExecStart`配置，接下来的一行将该配置设置为带有`-4`参数的ntpd。

GnuPG

在 *dirmngr* 配置文件中设置如下参数：

```
~/.gnupg/dirmngr.conf
disable-ipv6
```

然后重启 `dirmngr.service` 服务.

sshd

修改 `sshd_config` 来让 sshd 只使用 IPv4：

```
/etc/ssh/sshd_config
AddressFamily inet
```

然后重启 `sshd.service` 服务

systemd-timesyncd

有时 Systemd-timesyncd 会尝试查询IPv6时间服务器，即使IPv6已被禁用。这可能导致系统时钟未更新，日志显示类似以下错误：

```
systemd-timesyncd: Failed to set up connection socket: Address family not supported by protocol
```

`systemd timesyncd` 在其「状态」中会显示尝试连接 IPv6 地址的记录，类似于：

```
Status: "Connecting to time server [2001:19f0:8001:afd:5400:1ff:fe9d:cba]:123 (2.pool.ntp.org)"
```

根据 FS#59806, 只有 2.ntp.org 池支持 IPv6。所以将 `2.arch.pool.ntp.org` 和 `2.pool.ntp.org` 从配置文件 `/etc/systemd/timesyncd.conf` 的 NTP 和 FallbackNTP 部分中删去即可避免这一现象。

systemd-networkd

networkd 支持按接口禁用IPv6。当网络单元的 `[Network]` 部分的 `LinkLocalAddressing=ipv4` 或 `LinkLocalAddressing=no` 时，networkd不会尝试在匹配的接口上配置IPv6。

然而，请注意，即使在使用上述选项时，如果 IPv6 未被全局禁用，*networkd* 仍会收到路由器通告。如果接口未接收到IPv6流量（例如，由于 sysctl 或 ip6tables 设置），它将保持在配置状态，并可能导致等待网络完全配置的服务超时。为了避免这种情况，还应在 `[Network]` 部分设置 `IPv6AcceptRA=no` 选项。

参见

- IPv6 — kernel.org documentation
- IPv6 temporary addresses — a summary about temporary addresses and privacy extensions

- IPv6 prefixes — a summary of prefix types
- net.ipv6 options — documentation of kernel parameters

1.17 网络配置

难度 高级 | **篇幅** 31.7 KB | **所属** 网络

本节将讲到

- 检查连接
- 网络管理
- 设置计算机名
- 更多设置
- 疑难解答

翻译状态：

- 本文（或部分内容）译自 Network configuration，最近一次同步于 2022-12-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：本文久未同步，内容与英语原文相比，已有较大差异。（在 Talk:网络配置# 中讨论）

本页解释了如何在 OSI 第三层及之上配置网络连接。有部分内容在网络配置/以太网和网络配置/无线网络配置两个页面中。

注意：配置网络连接时请不要同时安装启用多个网络管理器，例如：不要同时启用 iwctl 与 NetworkManager，这可能会导致网络连接故障

提示：建议参阅systemd-networkd

检查连接

要排查网络连接问题，请依次检查以下条件，确保满足每一项:

1. 你的网络接口已列出并处于启用状态。否则请检查设备驱动－请查阅 Network configuration/Ethernet#Device driver 或 Wireless network configuration#设备驱动\[损坏的链接：无效的章节\] 页面。
2. 你已连接到网络。网线已接好或者已经连接到无线局域网。
3. 你的网络接口拥有一个IP 地址。
4. 你的路由表已正确配置。
5. 你可以 ping 通一个本地 IP 地址（例如你的默认网关）。

6. 你可以 ping 通一个公网 IP 地址（例如 `9.9.9.9`，即 Quad9 的 DNS 服务器，常用于测试连通性）。
7. 检查是否能解析域名（例如 `archlinux.org`）。

Ping

ping) 用于测试你是否可连接到某个主机, 用 `Ctrl+c` 停止测试。

```
$ ping www.example.com
PING www.example.com (93.184.216.34): 56(84) data bytes
64 bytes from 93.184.216.34: icmp_seq=0 ttl=56 time=11.632 ms
64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time=11.726 ms
64 bytes from 93.184.216.34: icmp_seq=2 ttl=56 time=10.683 ms
...
```

如上所示，Ping 命令对每个收到的回应都会显示一行信息，直到你手动中断（按 ctrl+c）。详细的解释请阅读 ping(8) 手册。注意，计算机可以配置为不响应 ICMP 回应请求。

如果你收到错误信息（参见ping错误提示#Error_indications)）或未收到任何回复，原因可能与默认网关配置或者网络接入服务商（ISP）有关。可以运行 traceroute 命令以进一步诊断到对端主机的路由路径。

网络管理

通过以下步骤来设置网络链接：

1. 确保网络接口已连接并已开启。
2. 连接到网络。网线已接好或者已经连接到无线局域网。
3. 配置网络连接：
4. 静态 IP 地址
5. 动态 IP 地址：使用 DHCP

注意：安装镜像使用 systemd-resolved 和 systemd-networkd 为, WLAN 和 WWAN网络配置 DHCP 客户端。

网络工具

Arch Linux 已经弃用了 net-tools包 转而使用 iproute2包。

已弃用的命令	替换命令
arp	ip neighbor
ifconfig	ip address, ip link
netstat	ss
route	ip route

想要更多更完整的总结，请参阅这篇文章。

iproute2

iproute2 是 base包 元包的依赖，提供 ip(8) 命令行接口，用于管理网络接口，IP 地址和路由表。注意使用 `ip` 进行的配置会在重启后丢失。要进行永久配置，可以使用网络管理器或通过脚本和 systemd 单元使 *ip* 命令自动化。同时需要注意的是 `ip` 命令通常能够缩写，为了清楚起见，本文对其进行了详细说明。

网络接口

默认情况下， udev 使用可预测的网络接口名称分配给你的网络接口，该名称以 `en` (有线/以太网)， `wl` (无线/WLAN) 或 `ww` (WWAN)开头 。参见 systemd.net-naming-scheme(7)。

提示：要改变接口名称，请参阅 #更改接口名称 和 #改回传统接口名称。

列出网络接口

有线和无线接口名称都可以通过 `ls /sys/class/net` 或 `ip link` 找到。要注意的是 `lo` 是 Loop 设备，不被用于建立网络连接。

同样，也可以用 `iw dev` 来检索无线设备的名称。可以参阅无线网#获取有用信息\[损坏的链接：无效的章节\] 。

如果未列出您的网络接口，请确保已成功加载设备驱动。参阅以太网#设备驱动或无线网#设备驱动\[损坏的链接：无效的章节\] 。

启用和禁用网络接口

可以使用 `ip link set *interface* up|down` 来启用/禁用网络接口，参阅 ip-link(8) 。

要想检查接口 `enp2s0` 的状态：

```
$ ip link show dev enp2s0
2: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master br0 state
DOWN mode DEFAULT qlen 1000
...
```

`<BROADCAST,MULTICAST,UP,LOWER_UP>` 中的 `UP` 表示接口已经启动，而非表示稍后的 `state DOWN` 。

注意：如果您的默认路由是通过接口 `enp2s0` 进行的，则将其删除也会删除该路由，而恢复备份将不会自动重新建立默认路由。参阅 #路由表 以重新建立它。

静态还是动态 IP 地址?

如果你在使用 WiFi 或路由器，你很可能会使用动态 IP 地址。你的计算机应该被配置为使用由 WiFi 或路由器分配的 IP 地址。如果你处于家庭环境且电脑连接到网络接入服务商提供的调制解调器，比如一个线缆调制解调器，那也会使用一个动态 IP 地址。

动态 IP 地址会在每次开机时改变。在工作环境中，你可能具有静态 IP 地址或动态 IP 地址。在家里，你可以将路由器配置为始终为计算机分配相同的 IP 地址，在这种情况下，你将使用静态 IP 地址。使用动态 IP 地址时，需要使用 DHCP 以便用正确的 IP 地址设置网络接口。除了配置 IP 地址外，DHCP 还可以配置路由（如何从你的位置到达任意网络中你要访问的的位置）以及域名服务器，域名服务器会转换主机名（例如 google.com）到 IP 地址（带点号的数字）。

静态 IP 地址

可以通过绝大多数的网络管理器或 dhcpcd 来配置静态 IP 地址。

要手动配置静态 IP 地址，请按照 #IP 地址中的说明添加 IP 地址，设置路由表并配置 DNS 服务器。

IP 地址

通过 ip-address(8) 来管理 IP 地址。

列出 IP 地址：

```
$ ip address show
```

将 IP 地址添加到接口：

```
# ip address add address/prefix_len broadcast + dev interface
```

: 注意：

: - 地址格式是 CIDR 表示法，这种格式也可以用来提供子网掩码。

- `+` 是使 `ip` 从 IP 地址和子网掩码派生多播地址的特殊符号。

: > **注意**：要确保手动分配的 IP 地址与 DHCP 分配的 IP 地址不冲突。

将 IP 地址从接口中删除：

```
# ip address del address/prefix_len dev interface
```

删除所有符合条件的地址，例如某个特定接口的地址：

```
# ip address flush dev interface
```

提示：可通过 ipcalc (ipcalc包) 计算 IP 地址。

路由表

路由表被用于确定是否可以直接访问某个 IP 地址或该使用哪个网关（路由器）。如果别的路由与这个 IP 地址匹配，将使用默认网关。

通过 ip-route(8) 来管理路由表。

应在 *PREFIX* 处使用 CIDR 表示法，或填入 `default` 表示默认网关。

列出 IPv4 路由：

```
$ ip route show
```

列出 IPv6 路由：

```
$ ip -6 route
```

添加路由：

```
# ip route add PREFIX via address dev interface
```

删除路由：

```
# ip route del PREFIX via address dev interface
```

DHCP

动态主机配置协议(DHCP) 服务器为客户端提供动态 IP 地址，子网掩码，默认网关 IP 地址，有时还可以提供域名服务器。

要使用 DHCP ，需要网络中有 DHCP 服务器和 DHCP 客户端：

客户端	软件包	Archiso	备注	Systemd 单元
dhcpcd	dhcpcd包	是	DHCP, DHCPv6, ZeroConf, static IP	`dhcpcd.service`, `dhcpcd@*interface*.service`
ISC dhclient	dhclient包	是	DHCP, DHCPv6, BOOTP, static IP	`dhclient@*interface*.service`

注意：

- ISC 从 2022 年初开始已经停止 ISC DHCP 客户端开发，程序已经不再维护，请不要在生产系统中使用。
- 不能同时运行两个 DHCP 客户端。
- 除了直接使用独立的 DHCP 客户端，还可以使用具有内置 DHCP 客户端的网络管理器 。
- 另外，iwd 提供了内置的 DHCP 客户端，可以通过某些配置来使用它：iwd#启用内置网络配置 。

提示：

- 可以通过 dhcping包 检查一个 DHCP 服务器是否在运行。
- 在等待分配 IP 时，可以运行 `watch -n 1 ping -c 1 archlinux.org` 之类的命令来自动确认网络是否配置好。

服务器

服务器	软件包	IPv4	IPv6	GUI	接口	后端存储	备注
dhcpd	dhcp包	是	是	Glass-ISC-DHCP	?	文件	
dnsmasq	dnsmasq包	是	是	否	?	文件	还可运行 DNS，PXE 和 TFTP 服务
Kea	kea包	是	是	Stork		文件, MySQL,	还可运行 DNS 服务

服务器	软件包	IPv4	IPv6	GUI	接口	后端存储	备注
					REST, RADIUS 和 NETCONF	PostgreSQL 和 Cassandra	

网络管理器

网络管理器可以在网络配置文件中管理网络连接设置，以便切换网络。

注意：有许多可选的方案，但他们都相互排斥，因此不要同时运行两个守护程序

网络管理器	GUI	Archiso	CLI 工具	点对点协议支持 （例如 3G 调制解调器）	DHCP 客户端	Systemd 单元
ConnMan	8 种，非官方	否	connmanctl(1)	是（通过 ofonoAUR）	内置	`connman.service`
netctl	2 种，非官方	否	netctl(1), wifi-menu	是	dhcpcd 或 dhclient包	`netctl-ifplugd@*interface*.service`, `netctl-auto@*interface*.service`
NetworkManager	是	否	nmcli(1), nmtui(1)	是	内置或 dhclient包	`NetworkManager.service`
systemd-networkd	否	是 (base 包)	networkctl(1)	否	内置	`systemd-networkd.service`, `systemd-resolved.service`

设置计算机名

主机名是一个网络中唯一标识一台机器的名称。主机名通过文件 `/etc/hostname` 进行配置，参阅 hostname(5) 和 hostname(7) 获取详细介绍。`/etc/hostname` 可以包含系统使用的域名（如果有）。要设置主机名，请编辑 `/etc/hostname` 使之包含单独一行 `*myhostname*`：

```
/etc/hostname
myhostname
```

提示：有关选取一个主机名的建议，参阅 RFC 1178。

也可以使用 hostnamectl(1)：

```
# hostnamectl set-hostname myhostname
```

要临时设置主机名（直到下次重启为止），使用 inetutils包 中的 hostname(1) 命令：

```
# hostname myhostname
```

要设置“美观”的主机名和其它机器元数据，请参考 machine-info(5)。

局域网主机名解析

要想使用主机名在你的局域网上访问你的机器，你可以使用下列方法：

- 编辑你局域网上所有设备的 `/etc/hosts`，参见 hosts(5)
- 设立一个 DNS 服务器来解析你的主机名，并让局域网中的设备都使用这个 DNS 服务器（比如，通过 #DHCP）
- 或者最简单的方式：使用 Zero-configuration networking 服务：
- 通过微软的 NetBIOS 进行主机名解析。在 Linux 上由 Samba 提供。它只需要启用 `nmb.service` 模块。运行 Windows，macOS 或运行 `nmb` 的 Linux 的计算机都能够发现你的设备。
- 通过 mDNS 进行主机名解析。 由带有 Avahi 的 `nss_mdns`（阅读 Avahi#Hostname resolution 获取详细配置）或 systemd-resolved 提供。 运行 macOS，或者带有 Avahi 的 Linux 或者运行 systemd-resolved 的 Linux 的计算机能够发现你的设备。老版本的 Win32 API 不支持 mDNS，一些老的 Windows 应用可能无法访问你的设备。

更多设置

更改接口名称

注意：当你改变接口命名规则时，不要忘记更新所有与网络相关的配置文件和自定义的 systemd 单元文件以反映更改。

你可以通过编辑 udev 规则来手动更改设备名称。例如：

```
/etc/udev/rules.d/10-network.rules
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="aa:bb:cc:dd:ee:ff", NAME="net1"
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="ff:ee:dd:cc:bb:aa", NAME="net0"
```

这些规则将在启动时自动应用。

值得注意的两点：

- 使用这条命令来获得每张网卡的 MAC 地址：

`cat /sys/class/net/**设备名**/address`

- 确保你的 udev 规则中使用小写的十六进制值，而不是大写。

如果网卡的 MAC 地址是会动态变化的，你可以使用 `DEVPATH`，例如：

```
/etc/udev/rules.d/10-network.rules
SUBSYSTEM=="net", DEVPATH=="/devices/platform/wemac.*", NAME="int"
SUBSYSTEM=="net", DEVPATH=="/devices/pci*/*1c.0/*/net/*", NAME="en"
```

要得到所有当前已连接设备的 `DEVPATH`，请查看 `/sys/class/net/` 中符号链接指向的位置。例如：

```
file /sys/class/net/*
/sys/class/net/enp0s20f0u4u1: symbolic link
to ../../devices/pci0000:00/0000:00:14.0/usb2/2-4/2-4.1/2-4.1:1.0/net/enp0s20f0u4u1
/sys/class/net/enp0s31f6:     symbolic link
to ../../devices/pci0000:00/0000:00:1f.6/net/enp0s31f6
/sys/class/net/lo:            symbolic link to ../../devices/virtual/net/lo
/sys/class/net/wlp4s0:        symbolic link
to ../../devices/pci0000:00/0000:00:1c.6/0000:04:00.0/net/wlp4s0
```

设备路径应与新设备名称和旧设备名称均匹配，因为该规则可能在启动时执行多次。例如，在第二条规则中若写 `"/devices/pci*/*1c.0/*/net/enp*"` 是错误的，因为一旦设备名称更改为 `en`，它将停止匹配。 只有系统默认规则会第二次触发，从而使设备名称改回例如 `enp1s0`。

如果你正在使用具有动态 MAC 地址的 USB 网络设备（如 Android 手机网络共享），并且你希望能够使用不同的 USB 端口，你可以使用匹配供应商和产品 ID 的规则：

```
/etc/udev/rules.d/10-network.rules
SUBSYSTEM=="net", ACTION=="add", ATTRS{idVendor}=="12ab", ATTRS{idProduct}=="3cd4",
NAME="net2"
```

要测试\[损坏的链接：无效的章节\]自定义规则，可以直接从用户空间触发这些规则，例如使用 `udevadm --debug test /sys/class/net/*`。切记先关闭您要重命名的接口（例如，`ip link set enp1s0 down`）。

注意：选择静态名称时，**应该避免使用形如“eth*X*”或“wlan*X*”的名称**，因为这可能在引导时导致内核与 udev 之间的竞争状态。相反，最好用内核默认不会使用的接口名称，例如：`net0`、`net1`、`wifi0`、`wifi1`。更多细节请查看 systemd 文档。

改回传统接口名称

如果希望使用像 eth0 这样的传统接口命名,可以通过下面方法禁用可预测网络接口名:

```
# ln -s /dev/null /etc/udev/rules.d/80-net-setup-link.rules
```

设定设备的 MTU 和队列长度

你可以手动定义一条 udev 规则来改变设备的 MTU 和队列长度。举例来说：

```
/etc/udev/rules.d/10-network.rules
ACTION=="add", SUBSYSTEM=="net", KERNEL=="wl*", ATTR{mtu}="1500", ATTR{tx_queue_len}
="2000"
```

`mtu`：使用大于 1500 的值（所谓的巨型帧）可以大大加快网络传输速度。请注意，所有网络接口（包括本地网络中的交换机）都必须支持相同的 MTU 才能使用巨型帧。对于 PPPoE，MTU 不应大于 1492。您还可以通过 systemd.netdev(5) 设置MTU。

`tx_queue_len`：对于具有高延迟的速度较慢的设备（如调制解调器链路和 ISDN），使用较小的值。对于通过高速 Internet 连接进行大型数据传输的服务器，建议使用较大的值。

绑定和链路聚合

参阅 netctl\[损坏的链接：无效的章节\] 或 systemd-networkd 或无线网络绑定。

IP 别名

IP 别名是指给同一个网络接口分配多个 IP 地址。这样一个网络节点可以有多个网络连接，每个实现不同的作用。典型的用处是 Web 服务器和 FTP 服务器的虚拟主机，或者是重组服务器时不需要更新其他任何机器上的 IP （对于名称服务器来说很有用）。

例子

要手动设置别名，可以使用 iproute2包 执行：

```
# ip addr add 192.168.2.101/24 dev enp2s0 label eth0:1
```

删除别名：

```
# ip addr del 192.168.2.101/24 dev enp2s0:1
```

默认情况下，发往子网的数据包将使用主 IP。如果目标 IP 子别名，原始 IP 也会被相应设置。如果有多个网卡，可以通过 `ip route` 查看默认路由。

混杂模式

切换网卡到混杂模式可以让（无线）网卡接收所有数据然后转交给操作系统处理。而正常模式下，网卡会丢掉不是发给自己的数据。这通常用来检查网络问题或进行数据包嗅探。

```
/etc/systemd/system/promiscuous@.service
[Unit]
Description=Set %i interface in promiscuous mode
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/bin/ip link set dev %i promisc on
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

如果要在 `enp2s0` 上启用全接收模式，只需要启用 `promiscuous@enp2s0.service`.

检测套接字

ss 是用来检测网络端口的工具，是 iproute2包 软件包的一部分。它与已弃用的 netstat 工具具有相似的功能。

一般的用途包括:

显示所有 TCP 连接及相应的服务名：

```
$ ss -at
```

显示所有 TCP 连接及其端口号：

```
$ ss -atn
```

显示所有 UDP 连接：

```
$ ss -au
```

更多信息请参阅 ss(8)。

疑难解答

TCP窗口扩缩（window scaling）故障

TCP 包头有个“窗口”（window）值表明其它主机可以发送多少数据回来。这个值只有16个bit，也就是说窗口大小最多只有 64KiB。TCP包会被缓存一段时间（ 它们要重新排序），如果内存有限（过去经常是）的话，主机会很容易用完内存。

回到 1992 年，内存逐渐增加，RFC 1323 被发布以改善情况：窗口扩缩(Window Scaling)。所有包里的窗口值，可以被初始连接时定义的一个缩放因子（Scale Factor）所改变。8bit 的缩放因子使得窗口可以是初始 64KiB 的 32 倍。

但是 Internet 上有些有故障的路由器和防火墙会将缩放因子重写为 0，这导致主机之间产生误解。Linux内核 2.6.17 引入了新的计算方式生成更高的缩放因子，间接的使得这些有故障的路由器和防火墙引发的后果更明显。

这导致连接缓慢甚至中断。

如何诊断故障

首先，我们要明白：这个问题很怪异。在某些案例中，你根本无法使用(HTTP, FTP, ...)，而有时候，你可以连接某些主机（很罕见）。

当你碰到这个故障时，`dmesg` 的输出正确，日志也没问题，`ip addr` 报告状态正常……实际上一切看起来都很正常。

如果你无法浏览任何网站，不过你能 ping 通某些主机，很可能你是遇到了这个问题：ping 使用 ICMP 协议所以不受 TCP 问题的影响。

你可以尝试使用 Wireshark。你也许会看到 UDP 和 ICMP 通讯成功，但是 TCP 通讯不成功（仅对外部主机）。

如何修复

糟糕的方法

用比较糟糕的方法修复的话，你可以修改缩放因子计算所基于的 `tcp_rmem` 值。虽然它对大部分主机有效，但并不担保一定都有效，特别是某些很远的主机。

```
# echo "4096 87380 174760" > /proc/sys/net/ipv4/tcp_rmem
```

好点的方法

只需要禁止窗口缩放。虽然窗口缩放是个不错的TCP特性，但它也可能令人不安，特别是当你没法修改出了问题的路由器的时候。有几种方法可以禁止窗口缩放，而看来最可靠的（适用于大部分内核）是将下面一行加入到你的 `/etc/sysctl.d/99-disable_window_scaling.conf` 中 （参见 sysctl）：

```
net.ipv4.tcp_window_scaling = 0
```

最佳的方法

这个故障是由有毛病的路由器/防火墙引起的，所以最好换了它。有些用户报告说那些有故障的路由是他们自己的 DSL 路由。

更多

本段内容是基于 LWN文章 TCP window scaling and broken routers 和一篇 Kernel Trap 文章：Window Scaling on the Internet。

在 LKML 上也有几篇相关的帖子。

连接第二台电脑后无法进行桥接

第一台电脑有两个 LAN 口。第二台电脑有一个 LAN 口并连接到第一台电脑。让第二台电脑桥接网络接口并可以访问 LAN 口：

```
# sysctl net.bridge.bridge-nf-filter-pppoe-tagged=0
# sysctl net.bridge.bridge-nf-filter-vlan-tagged=0
# sysctl net.bridge.bridge-nf-call-ip6tables=0
# sysctl net.bridge.bridge-nf-call-iptables=0
# sysctl net.bridge.bridge-nf-call-arptables=0
```

本地主机名解析

systemd包 的 `myhostname` Name Service Switch (NSS) 模块提供`localhost` 和本地域名到 IP 地址的解析，默认是启用的。

然而一些客户端应用仍然会依赖于 `/etc/hosts`，相关例子参见 。

要阻止程序通过网络非安全的解析 localhost，请将 `localhost` 加入 hosts(5) 文件:

```
/etc/hosts
127.0.0.1        localhost
::1              localhost
```

注意：Report any software affected by this issue in FS#56684. This may help in getting `localhost` entries added to the default `/etc/hosts`.

要配置 `/etc/hosts`，将以下内容加入其中：

```
127.0.0.1        localhost
::1              localhost
127.0.0.1        myhostname.localdomain        myhostname
```

注意：`/etc/hosts` 中在 IP 地址后的主机名或别名的顺序是重要的。第一个字符串会被认为是标准主机名，其后可以加上以点逐级分隔的上级域名（也就是上面的 `.localdomain`）。所有同一行中其后的字符串都被认为是别名。更多信息请参阅 hosts(5)。

这样系统就会解析每条配置：

```
$ getent hosts
127.0.0.1       localhost
127.0.0.1       localhost
127.0.0.1       myhostname.localdomain myhostname
```

对于有固定 IP 地址的系统，应该用固定 IP 地址替换 `127.0.0.1`。For a system with a fully qualified domain name, insert the fully qualified domain name before the hostname (see the following link for the reasoning). For example:

```
/etc/hosts
127.0.0.1         localhost
::1               localhost
203.0.113.45      host1.fqdomain.example host1
```

注意：The order of hostnames/aliases that follow the IP address in `/etc/hosts` is significant. The first string is considered the canonical hostname and may be appended with parent domains, where domain components are separated by a dot. All following strings on the same line are considered aliases. See hosts(5) for more info.

As a result the system resolves to both entries:

```
$ getent hosts
127.0.0.1       localhost
127.0.0.1       localhost
127.0.0.1       myhostname
```

扩展阅读

- Linux 网络管理员指南
- Debian 参考手册：网络安装
- RHEL7：网络指南
- 监视和调优 Linux 网络栈：接收数据
- 监视和调优 Linux 网络栈：发送数据
- 使用 Linux tracepoints、perf 和 eBPF 跟踪数据包

1.18 InfiniBand

难度 高级 | **篇幅** 33.9 KB | **所属** 网络

本节将讲到

- 简介
- 术语解释
- 安装
- TCP/IP (IPoIB)
- Soft RoCE (RXE)

本页介绍了如何设置、诊断和测试 InfiniBand 网络。

简介

概述

InfiniBand（简称 IB）是以太网和光纤通道的替代方案。IB 提供高带宽和低延迟。IB 可以将数据直接从一台机器上的存储设备传输到另一台机器的用户空间，从而绕过并避免系统调用的开销。IB 适配器可以处理的网络协议与在 CPU 上运行的以太网协议不同，它允许操作系统和 CPU 在进行高带宽传输时保持低占用。这在10Gb+以太网中是个重要问题。

IB 硬件由迈络思（Mellanox，与 Voltaire 合并，并得到 Oracle 的大力支持）和英特尔（2012 年收购了 QLogic 的 IB 部门）制造。迈络思于 2019 年被英伟达收购。IB 最常用于超算、集群和数据中心。IBM、惠普和 Cray 也是 InfiniBand 委员会的成员。Facebook、Twitter、eBay、YouTube 和 PayPal 都是 IB 用户。

IB 软件的开发基于 OpenFabrics Open Source Alliance。

实惠的二手设备

由于大型企业从使用更新的设备中获益，无源 IB 线缆的最大长度限制，有源 IB 线缆的高成本，以及比以太网更复杂的技术设置，二手 IB 市场已经严重供大于求。因此，在家庭或小型企业的内部网络中使用二手 IB 设备性价比很高。

带宽

信号传输速率

IB 传输速率一开始与 PCI Express（简称 PCIe）支持的最大速率一致。后来随着 PCIe 的发展，传输速率与其他 I/O 技术一样，每个端口的 PCIe 通道数也随之增加。IB 推出的 SDR（单数据速率）每通道信号传输速率为 2.5Gb/s（与 PCIe v1.0 相对应），后来又增加了：5Gb/s 的 DDR（双数据速率，PCIe 2.0）；10Gb/s 的 QDR（四数据速率，与 PCIe 3.0 的吞吐量相匹配，改进了 PCIe 3.0 的编码而非信号传输速率）；以及 14.0625 Gbps 的 FDR（十四数据速率，与 16G FC 光纤通道相匹配）。IB 目前正在提供 25Gb/s 的 EDR（增强数据速率，与 25Gb 以太网相匹配）。计划在 2017 年左右推出 50Gb/s 的 HDR（高数据速率）。

有效吞吐量

由于 SDR、DDR 和 QDR 版本使用 8/10 编码（8 位数据需要 10 位信令），这些版本的有效吞吐量降低到 80% 。 SDR 为 2Gb/s/link；DDR 为 4Gb/s/link；QDR 为 8Gb/s/link。从 FDR 开始，IB 使用 64/66 编码，使有效吞吐量与信令速率的比率提高

到 96.97% 。 FDR 为 13.64Gb/s/link；EDR 为 24.24Gb/s/lane；HDR 为 48.48Gb/s/link。

IB 设备能够通过多个链路发送数据，不过商用标准是每条线缆大约 4 个链路。

使用普通 4X 链路设备时，总有效吞吐量可达 SDR 为 8Gb/s；DDR 为 16Gb/s；QDR 为 32Gb/s；FDR 为 54.54Gb/s；EDR 为 96.97Gb/s；HDR 为 193.94Gb/s。

延迟

IB 的延迟小得令人难以置信：SDR（5us）；DDR（2.5us）；QDR（1.3us）；FDR（0.7us）；EDR（0.5us）；HDR（< 0.5us）。相比之下，10Gb 以太网的延迟为 7.22us，是 FDR 延迟的 10 倍。

向后兼容性

IB 设备几乎总是向后兼容的。连接应建立在最小带宽上。用于 PCIe 8x 插槽的 DDR 适配器应能在 PCIe 4x 插槽中使用（带宽减半）。

线缆

IB 无源铜缆使用 QDR 时最长可达 7 米，使用 FDR 时最长可达 3 米。

IB 有源光纤（光）电缆使用 FDR 时最长可达 300 米（FDR10 仅为 100 米）。

Mellanox MetroX 设备可提供长达 80 公里的连接。延迟每公里增加约 5us。

IB 线缆可用于直接连接两台计算机，无需交换机；不存在 IB 交叉线缆。

术语解释

硬件

适配器、交换机、路由器和网桥/网关必须专为 IB 而设计。

HCA (主机通道适配器) : 类似以太网 NIC（网络接口卡）。它将 IB 电缆连接到 PCIe 总线上，如果使用的是适当型号的 HCA，则可达到总线的全速。IB 网络的终端节点，执行传输级功能，并支持 IB verbs接口。 **交换机** : 与以太网网卡类似。在同一 IB 子网中将数据包从一个链路传送到另一个链路。 **路由器** : 类似以太网路由器。在不同 IB 子网之间传输数据包。 **网桥/网关** : 一个独立的硬件或执行此功能的计算机。桥接 IB 和以太网网络。

GUID

与以太网 MAC 地址类似，但一个设备有多个 GUID。由硬件制造商分配，重启后保持不变。64 位地址（24 位制造商前缀和 40 位设备标识符）。分配给适配器、交换机、路由器和网桥/网关。

Node GUID : 标识 HCA、交换机或路由器 **Port GUID** : 标识 HCA、交换机或路由器上的端口（即使一个 HCA 通常也有多个端口） **System GUID** : 允许将多个 GUID 视为一个实体 **LID（本地标识符）** : 子网管理器接收时分配的 16 位地址。用于路由数据包。重启后不能持久使用。

网络管理

SM（子网管理器） : 主动管理 IB 子网。可以作为连接到 IB 网络的计算机上的一个软件程序、内置到 IB 交换机中或作为专门的 IB 设备来实现。初始化和配置子网中的其他一切，包括分配 LID（本地标识符）。建立通过子网的流量路径。隔离故障。防止其他未经授权的子网管理器。一个子网管理器下可以有多个交换机。一个子网中可以有多个子网管理器，但同时只能有一个处于活动状态。 **MAD（管理数据报）** : 子网管理器与 IB 设备之间通信的标准信息格式，由 UD（不可靠数据报）承载。 **UD（不可靠数据报）**

安装

首先安装 rdma-core包 其中包含所有核心库和守护进程。

更新固件

使用最新固件可显著提高性能，并修复连接问题。

警告：请小心操作，设备可能会损坏！

适用于 Mellanox

- 安装 mstflintAUR
- 确定适配器的 PCI device ID (在这个例子中, "05:00.0" 就是适配器的 PCI device ID)

```
$ lspci | grep Mellanox
05:00.0 InfiniBand: Mellanox Technologies MT25418 [ConnectX VPI PCIe 2.0 2.5GT/s - IB DDR / 10GigE] (rev a0)
```

- 确定适配器的固件版本，以及适配器的 PSID（比型号更加具体 - 用于区分不同修订版本的兼容性设置）

```
# mstflint -d <adapter PCI device ID> query
...
FW Version:      2.7.1000
...
PSID:            MT_04A0110002
```

- 检查最新固件版本

- 访问 Mellanox's firmware download page（本指南采用了该链接的 "固件刻录说明"，使用的是 mstflint 选项）
- 选择您的设备类别
- 在 mstflint 给您的列表中找到设备的 PSID
- 检查固件映像文件名是否比适配器的 FW 版本更新，比如 `fw-25408-2_9_1000-MHGH28-XTC_A1.bin.zip` 的版本就是 `2.9.1000`
- 如果有最新版本，请下载新固件并将其刻录到适配器中。

```
$ unzip <firmware .bin.zip file name>
# mstflint -d <adapter PCI device ID> -i <firmware .bin file name> burn
```

适用于 Intel/QLogic

在 Intel Download Center 上搜索型号（或子串），然后按说明操作。下载的软件可能需要在 RHEL/CentOS 或 SUSE/OpenSUSE 中运行。

内核模块

按照自己的喜好编辑 `/etc/rdma/modules/rdma.conf` 和 `/etc/rdma/modules/infiniband.conf`。 然后加载这些文件中记录的内核模块，如 `ib_ipoib`, 或直接重启系统。

提示：尽管通常不需要这样操作，但如果内核模块未正确加载，请启动并启用 `rdma-load-modules@rdma.service` 和 `rdma-load-modules@infiniband.service`，然后重启系统。

注意：由于内核堆栈的处理方式，对 `/etc/rdma/modules/rdma.conf` 的修改只能在每次启动时生效一次， 即首次启动 `rdma-load-modules@*.service` 时， 重启 `rdma-load-modules@*.service` 没有任何作用。

子网管理器

每个 IB 网络至少需要一个子网管理器。如果没有子网管理器，设备可能会显示有链接，但状态永远不会从 `Initializing` 变为 `Active`。子网管理器会经常（通常为每 5 或 30 秒）检查网络中是否有新适配器，并将其添加到路由表中。如果 IB 交换机内嵌子网管理器，则可以使用该管理器，或者将其禁用，改用软件子网管理器。也有专用的 IB 子网管理器设备。

启用端口

如果端口处于物理状态 `Sleep`（可通过 `ibstat` 验证）则首先需要运行 `ibportstate --Direct 0 1 enable` 来启用端口。如果链路两端的端口都处于睡眠状态，可能需要在启动时自动执行此操作。

软件子网管理器

在一个系统上:

- 安装 opensmAUR
- 修改 systemd 文件 `/usr/lib/systemd/system/opensm.service` 为如下所示。
- 启用并启动 `opensm.service`

当前的 opensm 配置与 RDMA 的 systemd 配置不兼容。即编辑 `/usr/lib/systemd/system/opensm.service` 中的两行（注释为原始内容）。

```
# /usr/lib/systemd/system/opensm.service
Requires=rdma-load-modules@rdma.service # Requires=rdma.service
After=rdma-load-modules@rdma.service # After=rdma.service
```

现在，所有连接的 IB 端口都应处于 `Active` 状态，物理状态应为 `LinkUp`。您可以通过运行 ibstat 来检查。

```
$ ibstat
... (look at the ports shown you expect to be connected)
State: Active
Physical state: LinkUp
...
```

或通过检查 `/sys` 文件系统:

```
$ cat /sys/class/infiniband/kernel_module/ports/port_number/phys_state
5: LinkUp
$ cat /sys/class/infiniband/kernel_module/ports/port_number/state
4: ACTIVE
```

TCP/IP (IPoIB)

您可以创建一个在 HCA 上运行的虚拟以太网适配器。这样，设计用于 TCP/IP 但不用于 IB 的程序就可以（间接）使用 IB 网络。由于通过正常的 TCP 协议栈发送所有流量，需要在 CPU 而不是 HCA 上运行系统调用、内存拷贝和网络协议，因此性能会受到负面影响。

IB 接口将在加载模块 `ib_ipoib` 时出现。 使其出现的简单配置是在 `/etc/rdma/modules/infiniband.conf` 添加 `ib_ipoib` 行，然后重启系统。模块 `ib_ipoib` 随系统启动后，应使用命令 `ip link` 确认名称为 `ibp16s0` 的链接。

还可对 IB 接口进行详细配置 (例如，将其重命名为 `ib0` 并分配 IP 地址就像传统的以太网适配器一样).

连接模式

IPoIB 可在 datagram（默认）或 connected 模式下运行。Connected 模式允许您设置更高的 MTU，但短报文的 TCP 延迟会比 datagram 模式增加5%.

查看当前使用的模式：

```
$ cat /sys/class/net/interface/mode
```

注意： ConnectX-6（或更新版本）的 Mellanox 卡不再支持 Connected 模式。

MTU

在 datagram 模式下，使用 UD（不可靠数据报）传输，这通常会强制 MTU 为 2044 字节。从技术上讲，IB L2 MTU 将 4 字节用于 IPoIB 封装头，2044 字节传输数据。

在 connected 模式下，使用 RC（可靠连接）传输，允许 MTU 达到最大 IP 数据包大小（65520 字节）。

查看您的 MTU：

```
$ ip link show interface
```

调优 connection 模式和 MTU

只有更改默认 connection 模式 和/或 MTU 时，才需要 `ipoibmodemtu`。

- 安装并设置 TCP/IP over IB (IPoIB))
- 安装 ipoibmodemtuAUR
- 通过 `/etc/ipoibmodemtu.conf` 配置 `ipoibmodemtu`，其中包含相关说明。
- 默认情况下，它将单个 IB 端口 `ib0` 设置为 `connected` 模式和 MTU `65520` 。
- 启动并启用 `ipoibmodemtu.service`

不同的设置会产生不同的结果。有些人通过使用 `connected` 模式和 MTU `65520` 看到了速度的大幅提升（双倍以上），而有些人看到的速度则与之差不多，甚至更差。请使用 qperf 和 iperf 对系统进行微调。

使用本文给出的 qperf 示例，下面是一个 SDR 网络（理论速度为 8 Gb/s）的示例结果，其中包含各种微调:

Mode	MTU	MB/s	us latency
datagram	2044	707	19.4
connected	2044	353	18.9
connected	65520	726	19.6

提示：整个子网使用相同的 connection 模式和 MTU 设置。混用和匹配无法达到最佳效果。

Soft RoCE (RXE)

Soft ROCE 是 RoCE 的软件实现，可通过任何以太网适配器使用 Infiniband。

- 安装 iproute2包
- 运行 `rdma link add rxe_eth0 type rxe netdev ethN` 以在以太网设备 ethN 上配置 RXE 实例。

现在应该有一个 rxe0 设备了:

```
# rdma link
link rxe_eth0/1 state ACTIVE physical_state LINK_UP netdev enp1s0
```

远程数据存储

您可以使用 iSCSI、带有 iSER 的 iSCSI 或 SRP，通过 IB 网络将物理或虚拟设备从target（主机/服务器）共享到initiator（访客/客户端）系统。 这些方法与传统的文件共享（Samba 或 NFS）不同，因为启动程序系统会将共享设备视为自己的块级设备，而不是传统的挂载网络共享文件夹。 例如 `fdisk /dev/*block_device_id*`, `mkfs.btrfs /dev/*block_device_id_with_partition_number*`

缺点是每个共享设备一次只能有一个系统使用；试图在目标系统（target system）或其他发起系统（initiator system）上挂载共享设备会失败（最初的发起系统当然可以在上面运行传统的文件共享）。

这样做的好处是带宽更快、控制能力更强，甚至可以远程物理定位发起系统的根文件系统（远程启动）。

targetcli

`targetcli`是一个命令，将其复杂的（不值得手动创建的）`/etc/target/saveconfig.json` 作为虚拟文件系统的配置文件。

安装和使用

在目标系统上:

- 安装 targetcli-fbAUR
- 启动并启用 `target.service`

在 `targetcli` 中：

- 在任意虚拟目录中，运行 `help` 可查看“‘该伪目录中可用的命令’”或 `help *command*`（如 `help create`）以获得更详细的帮助。
- 许多命令还提供Tab补全
- 运行 `ls` 查看当前虚拟目录及其下的整个虚拟文件系统

创建后端存储

输入配置命令:

```
# targetcli
```

在 `targetcli` 中，为每个要共享的设备或虚拟设备设置一个后端存储:

- 要共享真实的块设备，请运行 `cd /backstores/block` 和 `create *name* *dev*`
- 要将文件作为虚拟块设备共享，请运行 `cd /backstores/fileio` 和 `create *name* *file*`
- 要以直通方式共享物理 SCSI 设备，请运行 `cd /backstores/pscsi` 和 `create *name* *dev*`
- 要共享内存盘，请运行 `cd /backstores/ramdisk` 和 `create *name* *size*`
- 其中 *name* 表示后端存储的名称
- 其中 *dev* 是要共享的块设备（如 `/dev/sda`、`/dev/sda4`、`/dev/disk/by-id/*XXX*` 或一个 LVM 逻辑卷 `/dev/vg0/lv1`）
- 其中 *file* 是要共享的文件（如 `/path/to/file`）
- 其中 *size* 是要创建的内存盘的大小（如 512MB 或 20GB）

iSCSI

iSCSI 允许通过网络使用存储设备和虚拟存储设备。对于 IB 网络，存储设备可以通过 IPoIB 或 iSER 运行。

与 iSCSI Target、iSCSI Initiator 和 iSCSI Boot 文章有很多重叠之处，但由于很多内容需要根据 IB 的使用情况进行定制，因此将讨论必要的内容。

Over IPoIB

首先执行模板系统指令，这将指导您何时临时切换到发起系统指令。

- 在目标系统和发起系统上，安装 TCP/IP over IB)
- 在目标系统上，对于要共享的每个设备或虚拟设备，使用 `targetcli`：
- 创建后端存储
- 为每个后备存储创建一个 IQN（iSCSI 限定名称）（其他系统配置将看到的存储名称）
- 运行 `cd /iscsi` 和 `create`. 它将为您提供一个 *randomly_generated_target_name*, 例如 `iqn.2003-01.org.linux-iscsi.hostname.x8664:sn.3d74b8d4020a`
- 设置 TPG（目标门户组），在上一步中自动创建为 tpg1
- 创建 lun（逻辑单元编号）
- 运行 `cd *randomly_generated_target_name*/tpg1/luns` 和 `create *storage_object*`。其中 `*storage_object*` 是指向现有存储对象的完整路径，例如 `/backstores/block/*name*`
- 创建 acl （访问控制列表）

- 运行: `cd ../acls`; 和 `create *wwn*`, 此处 `*wwn*` 是initiator系统的 IQN（iSCSI 合格名称），又称其（全局名称）。
- 在发起系统（'不是目标系统）上运行 `*wwn*` 获取（在其上安装 open-iscsi包之后）`cat /etc/iscsi/initiatorname.iscsi`
- 通过运行 `cd /`；`saveconfig`；`exit` 来保存并退出。
- 在发起系统上：
- 安装 open-iscsi包
- 此时，您可以获取发起系统的 IQN（iSCSI 合格名称），又称 wwn（全局名称），用于设置目标系统的 `lun`：
- `pacman` 应显示 `>>> Setting Initiatorname *wwn*`
- 否则，请运行 `cat /etc/iscsi/initiatorname.iscsi` 查看 `InitiatorName=*wwn*`
- 启动并启用 `iscsid.service`
- 要在启动时自动登录已发现的目标系统，请在发现目标系统前编辑 `/etc/iscsi/iscsid.conf` 设置 `node.startup = automatic`。
- 要寻找在线的目标系统。请以 root 身份运行 `iscsiadm -m discovery -t sendtargets -p *portal*` 其中 *portal* 是 IPv4/IPv6 地址或主机名
- 如果使用主机名，请确保它路由到 IB IP 地址而不是以太网 - 只使用 IB IP 地址可能更佳。
- 要在启动时自动登录已发现的目标系统，启动并启用 `iscsi.service`
- 要手动登录已发现的目标，请以 root 身份运行 `iscsiadm -m node -L all`。
- 查看每个登录目标的块设备 ID。以 root 身份运行 `iscsiadm -m session -P 3 | grep Attached` 块设备 ID 将出现在每个目标树中的最后一行（`-P` 是打印命令，其选项是冗余级别，只有第 3 级才会列出块设备 ID）

Over iSER

iSER（iSCSI Extensions for RDMA）利用 IB 的 RDMA 协议，而不是使用 TCP/IP。它消除了 TCP/IP 的开销，提供了更高的带宽、零拷贝时间、更低的延迟和更低的 CPU 利用率。

根据 iSCSI Over IPoIB 介绍， 并做出以下更改:

- 如果您愿意，可以直接安装 RDMA 以加载内核模块，而不是安装 IPoIB)。
- 在目标系统上，完成其他设置后，仍在 `targetcli` 中启用目标系统上的 iSER:

- 对于每一个您希望使用 iSER 而不是 IPoIB的 *iqn*， 运行 `cd /iscsi/*iqn*/tpg1/portals/0.0.0.0:3260`
- 其中 *iqn* 是随机生成的目标名称，例如 `iqn.2003-01.org.linux-iscsi.hostname.x8664:sn.3d74b8d4020a`
- 运行 `enable_iser true`
- 保存并退出: `cd /`; `saveconfig`; 和 `exit`
- 在 initiator 系统上，运行 `iscsiadm` 寻早在线 target 时，使用附加参数 `-Iiser`，登录 target 时，应该会看到: `Logging in to [iface: iser...`

添加到 /etc/fstab

您上次寻找目标系统时，自动登录一定是打开的。

就像添加本地块设备一样在 `/etc/fstab` 中添加挂载项，只是需要添加 `_netdev` 选项，以避免在网络初始化前尝试挂载。

网络划分

一个 IB 子网可以为不同的客户或应用分区，从而提供安全性和服务质量保证。每个分区由 PKEY（分区密钥）标识。

SDP (Sockets Direct Protocol)

使用 `librdmacm` 和 `LD_PRELOAD` (rsockets和libspd的后续版本）和`LD_PRELOAD`来拦截非IB程序的套接字调用，并通过RDMA以透明方式（对程序而言）将其发送到IB上。这大大加快了为 TCP/IP 构建的程序的运行速度，远远超过使用 IPoIB 所能达到的速度。它避免了为使用 IB 而修改程序源代码，甚至可用于闭源程序。它不适用于静态链接套接字库的程序。

诊断和基准测试

所有 IB 专用工具都包含在 rdma-core包 和 ibutilsAUR 中。

ibstat - 查看计算机的 IB GUIDs

ibstat 会显示计算机上运行的每个 IB 适配器的详细信息，包括：型号；端口数；固件和硬件版本；节点，系统映像和端口 GUID；端口状态，物理状态，速率，base lid，lmc，SM lid，capability mask，和链路层。

```
$ ibstat
CA 'mlx4_0'
        CA type: MT25418
        Number of ports: 2
        Firmware version: 2.9.1000
        Hardware version: a0
        Node GUID: 0x0002c90300002f78
        System image GUID: 0x0002c90300002f7b
        Port 1:
```

```
                State: Active
                Physical state: LinkUp
                Rate: 20
                Base lid: 3
                LMC: 0
                SM lid: 3
                Capability mask: 0x0251086a
                Port GUID: 0x0002c90300002f79
                Link layer: InfiniBand
        Port 2:
                State: Down
                Physical state: Polling
                Rate: 10
                Base lid: 0
                LMC: 0
                SM lid: 0
                Capability mask: 0x02510868
                Port GUID: 0x0002c90300002f7a
                Link layer: InfiniBand
```

本例显示的是 Mellanox Technologies (MT) 适配器。报告的是它的 PCI Device ID（25418），而不是产品型号。它显示的状态是 “Active”，这意味着它已正确连接到子网管理器。它显示的物理状态为 “LinkUp”，这意味着它通过线缆进行了电气连接，但不一定与子网管理器正确连接。它显示的总速率为 20 Gb/s（对该卡来说，它来自 5.0 Gb/s 信号速率和 4 个虚拟通道）。它显示子网管理器为该端口分配了 3 个 Base lid 。

ibhosts - 查看 IB 网络上的所有主机

ibhosts 会显示 IB 网络上每台主机的节点 GUID、端口数量和设备名称。

```
# ibhosts
Ca      : 0x0002c90300002778 ports 2 "MT25408 ConnectX Mellanox Technologies"
Ca      : 0x0002c90300002f78 ports 2 "hostname mlx4_0"
```

ibswitches - 查看 IB 网络上的所有交换机

ibswitches 会显示 IB 网络上每个交换机的节点 GUID、端口数量和设备名称。如果只使用直接连接，则不会显示任何内容。

```
# ibswitches
```

iblinkinfo - 查看 IB 网络链接信息

iblinkinfo 将显示设备名称、端口 GUID、虚拟通道数、信号传输速率、状态、物理状态以及所连接的设备。

```
# iblinkinfo
CA: MT25408 ConnectX Mellanox Technologies:
      0x0002c90300002779      4    1[  ] ==( 4X           5.0 Gbps Active/  LinkUp)==>
3    1[  ] "kvm mlx4_0" ( )
CA: hostname mlx4_0:
      0x0002c90300002f79      3    1[  ] ==( 4X           5.0 Gbps Active/  LinkUp)==>
4    1[  ] "MT25408 ConnectX Mellanox Technologies" ( )
```

该示例展示了不使用交换机直接连接的两个适配器，使用 5.0 Gb/s 信号传输速率和 4 个虚拟通道 (4X)。

ibping - Ping 另一个 IB 设备

ibping 将尝试 ping 另一个 IB GUID。 ibping 必须在一台计算机上以服务器模式运行，在另一台计算机上以客户端模式运行。

ibping 必须在一台计算机上以服务器模式运行。

```
# ibping -S
```

而在另一个端口上则是客户端模式。它正在 ping 一个特定端口，因此不能使用 CA 名称、节点或系统 GUID。它需要`-G`和端口 GUID，或`-L`和Lid。

```
# ibping -G 0x0002c90300002779
-or-
# ibping -L 1
Pong from hostname.(none) (Lid 1): time 0.053 ms
Pong from hostname.(none) (Lid 1): time 0.074 ms
^C
--- hostname.(none) (Lid 4) ibping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1630 ms
rtt min/avg/max = 0.053/0.063/0.074 ms
```

如果运行的是 IPoIB，则可以使用通过 TCP/IP 协议栈 ping 的常规 `ping`。 ibping 使用 IB 接口，不使用 TCP/IP 协议栈。

ibdiagnet - 显示整个子网的诊断信息

ibdiagnet 会显示子网中的潜在问题。您可以在不带选项的情况下运行它。 `-lw <1x|4x|12x>` 指定适配器的预期链路宽度(虚拟通道数)，这样它就能检查是否按预期运行。 `-ls <2.5|5|10>` 指定计算机适配器的预期链接速度（信号传输速率），以便检查其是否按预期运行，但对于 FDR+ 设备，它还不支持速度超过 10 的选项。 `-c <count>` 覆盖默认发送 10 个数据包。

```
# ibdiagnet -lw 4x -ls 5 -c 1000
Loading IBDIAGNET from: /usr/lib/ibdiagnet1.5.7
-W- Topology file is not specified.
    Reports regarding cluster links will use direct routes.
Loading IBDM from: /usr/lib/ibdm1.5.7
-I- Using port 1 as the local port.
-I- Discovering ... 2 nodes (0 Switches & 2 CA-s) discovered.

-I---------------------------------------------------
-I- Bad Guids/LIDs Info
-I---------------------------------------------------
-I- No bad Guids were found

-I---------------------------------------------------
-I- Links With Logical State = INIT
-I---------------------------------------------------
-I- No bad Links (with logical state = INIT) were found

-I---------------------------------------------------
-I- General Device Info
-I---------------------------------------------------

-I---------------------------------------------------
-I- PM Counters Info
-I---------------------------------------------------
-I- No illegal PM counters values were found
```

```
-I---------------------------------------------------
-I- Links With links width != 4x (as set by -lw option)
-I---------------------------------------------------
-I- No unmatched Links (with width != 4x) were found

-I---------------------------------------------------
-I- Links With links speed != 5 (as set by -ls option)
-I---------------------------------------------------
-I- No unmatched Links (with speed != 5) were found

-I---------------------------------------------------
-I- Fabric Partitions Report (see ibdiagnet.pkey for a full hosts list)
-I---------------------------------------------------
-I-    PKey:0x7fff Hosts:2 full:2 limited:0

-I---------------------------------------------------
-I- IPoIB Subnets Check
-I---------------------------------------------------
-I- Subnet: IPv4 PKey:0x7fff QKey:0x00000b1b MTU:2048Byte rate:10Gbps SL:0x00
-W- Suboptimal rate for group. Lowest member rate:20Gbps > group-rate:10Gbps

-I---------------------------------------------------
-I- Bad Links Info
-I- No bad link were found
-I---------------------------------------------------
----------------------------------------------------------------
-I- Stages Status Report:
    STAGE                                    Errors Warnings
    Bad GUIDs/LIDs Check                     0      0
    Link State Active Check                  0      0
    General Devices Info Report              0      0
    Performance Counters Report              0      0
    Specific Link Width Check                0      0
    Specific Link Speed Check                0      0
    Partitions Check                         0      0
    IPoIB Subnets Check                      0      1

Please see /tmp/ibdiagnet.log for complete log
----------------------------------------------------------------

-I- Done. Run time was 0 seconds.
```

qperf - 通过 RDMA 或 TCP/IP 测试性能

qperf 可通过 RDMA（SDP、UDP、UD 和 UC）或 TCP/IP（包括 IPoIB）测量带宽和延迟

qperf 必须在一台计算机上以服务器模式运行。

```
$ qperf
```

而在另一台上以客户端模式运行。SERVERNODE 可以是主机名，或者 IPoIB 的 TCP/IP 地址。其中有许多测试。下面是一些最有用的测试。

```
$ qperf SERVERNODE [OPTIONS] TESTS
```

TCP/IP over IPoIB

```
$ qperf 192.168.2.2 tcp_bw tcp_lat
tcp_bw:
    bw  =  701 MB/sec
tcp_lat:
    latency  =  19.8 us
```

iperf - 通过 TCP/IP 测试性能

iperf 不是一个 IB 感知的程序，它的目的是通过 TCP/IP 或 UDP 进行测试。尽管qperf 可以使用 IPoIB 测试 IB TCP/IP 性能，但 iperf 仍是另一个可以使用的程序。

iperf 必须在一台计算机上以服务器模式运行。

```
$ iperf3 -s
```

在另一台上使用客户端模式

```
$ iperf3 -c 192.168.2.2
[  4] local 192.168.2.1 port 20139 connected to 192.168.2.2 port 5201
[ ID] Interval           Transfer     Bandwidth
[  4]   0.00-1.00   sec   639 MBytes  5.36 Gbits/sec
...
[  4]   9.00-10.00  sec   638 MBytes  5.35 Gbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  4]   0.00-10.00  sec  6.23 GBytes  5.35 Gbits/sec                  sender
[  4]   0.00-10.00  sec  6.23 GBytes  5.35 Gbits/sec                  receiver

iperf Done.
```

iperf 以 10 GB 为基准显示传输量，以 2 GB 为基准显示带宽。因此，本例显示 10 秒内传输 6.23GB（base 10）。也就是 10 秒内传输 6.69GB（base 2）。(6.23 \ *2^30 / 10^9) 即 5.35 Gb/s（base 2），如 iperf 所示。 (6.23* \ 2^30 / 10^9 \ *8 / 10) 即 685 MB/s（base 2），与 qperf 报告的速度大致相同。(6.23* \ 2^30 / 10^9 \ *8 / 10* \ 1024 / 8)

常见问题 / FAQ

链接问题

链路、物理状态和端口状态

- 查看系统是否能识别 IB 硬件模块。如果您有英特尔适配器，则必须在此处使用 Intel，如果您有其他英特尔硬件，则需要查看几行内容:

```
# dmesg | grep -Ei "Mellanox|InfiniBand|QLogic|Voltaire"
[    6.287556] mlx4_core: Mellanox ConnectX core driver v2.2-1 (Feb, 2014)
[    8.686257] <mlx4_ib> mlx4_ib_add: mlx4_ib: Mellanox ConnectX InfiniBand driver
v2.2-1 (Feb 2014)
$ ls -l /sys/class/infiniband
mlx4_0 -> ../../devices/pci0000:00/0000:00:03.0/0000:05:00.0/infiniband/mlx4_0
```

如果没有任何显示，则说明内核无法识别适配器。本示例显示了使用 mlx4_0 内核模块的 Mellanox ConnectX 适配器的大致情况。

- 检查端口和物理状态。运行 ibstat 或检查 `/sys`.

```
$ ibstat
(look at the port shown that you expect to be connected)
```

或

```
$ cat /sys/class/infiniband/<kernel module>/ports/<port number>/phys_state
 5: LinkUp
$ cat /sys/class/infiniband/<kernel module>/ports/<port number>/state
```

`4: ACTIVE`

物理状态应为 “LinkUp”。如果不是，则可能是线缆没有插好、没有连接到另一端的任何设备或线缆有缺陷。端口状态应为 "Active"。如果是 "Initializing" 或 "INIT" ，则说明子网管理器不存在、未运行或未将端口添加到网络路由表中。

- 您能否成功 ibping 其他直接使用 IB 的设备而不是 IPoIB？ 如果运行 IPoIB，能否成功 `ping`？
- 考虑更新固件.

getaddrinfo failed: Name or service not known

- 运行 ibhosts ，查看每行末尾带引号的 CA 名称

速度问题

- 首先要仔细检查您的期望值。

您是如何确定存在速度问题的？您使用的是qperf还是iperf，它们都是从内存而不是硬盘传输数据。或者，您是在对实际文件传输进行基准测试吗？除非你运行 RAID 来提高速度，否则即使使用 2015 年中期可用的最快固态硬盘，单个硬盘（有时甚至是多个硬盘）也会对 IB 传输速度造成瓶颈。您是通过 IPoIB 使用 RDMA 还是 TCP/IP？如果是，使用 IPoIB 而不使用 RDMA 会影响性能)。

- 检查您的链接速度，运行 ibstat, iblinkinfo, 或者检查 `/sys`.

```
$ ibstat
(look at the Rate shown on the port you are using)
```

或

```
# iblinkinfo
(look at the middle part formatted like "4X     5.0 Gbps")
```

或

```
$ cat /sys/class/infiniband/<kernel module>/ports/<port number>/rate
20 Gb/sec (4X DDR)
```

这是否符合您预期的带宽和虚拟通道数？

- 检查整个子网的诊断信息。 运行 #ibdiagnet - 显示整个子网的诊断信息. 确保使用 `-ls` 和适当的信号传输速率，这可能是您 IB 卡的宣传速度除以 4.

```
# ibdiagnet -lw <expected number of virtual lanes -ls <expected signaling rate> -c 1000
```

- 考虑更新固件.

防火墙

网络边界上的取舍有三处：哪些包能进来、地址集合怎么收拢、反复试探的来路封不封。iptables 与 nftables 是两套并行写法，其上架着 firewalld、UFW 这类前端，Fail2ban 按日志批量封禁。

只护一台机器，先读《Simple stateful firewall》再读《Nftables》，《Iptables》作旧写法对照，《Shorewall》等整网方案可跳。最常踩的坑是前端与底层工具混配，各写一套规则互相覆盖；改规则时忘掉默认拒绝，第一条命令就把自己挡在门外。

2.1　Ipset

难度 入门 | **篇幅** 3.4 KB | **所属** 防火墙

本节将讲到

- 安装
- 配置
- 其他命令
- 优化

ipset是 Linux 防火墙 iptables 的一个协助工具。 通过这个工具可以轻松愉快地屏蔽一组IP地址。

安装

安装软件包 ipset包。

配置

屏蔽一组地址

先创建一个新的网络地址的"集合"。下面的命令创建了一个新的叫做"myset"的"net"网络地址的"hash"集合。

```
# ipset create myset hash:net
```

或

```
# ipset -N myset nethash
```

把希望屏蔽的IP地址添加到集合中。

```
# ipset add myset 14.144.0.0/12
# ipset add myset 27.8.0.0/13
# ipset add myset 58.16.0.0/15
```

最后，配置 iptables，屏蔽这个集合中的所有地址。这个命令将会向"INPUT"链顶端添加一个规则来从 ipset 中 "-m" 匹配名为 "myset" 的集合，当匹配到的包是一个"src"包时，"DROP"屏蔽掉它。

```
# iptables -I INPUT -m set --match-set myset src -j DROP
```

屏蔽多个 IP 地址

先创建一个 IP 地址"集合"，下面命令创建一个 "myset-ip" 散列集合。

```
# ipset create myset-ip hash:ip
```

或

```
# ipset -N myset-ip iphash
```

将要屏蔽的地址加入此集合：

```
# ipset add myset-ip 1.1.1.1
# ipset -A myset-ip 2.2.2.2
```

最后，用 iptables 屏蔽集合中的所有地址.

```
# iptables -I INPUT -m set --match-set myset-ip src -j DROP
```

使ipset持久化

上面创建的 ipset 存在于内存中，重启后将会消失。要使ipset持久化，你要这样做：

首先把 ipset 保存到 /etc/ipset.conf:

```
# ipset save > /etc/ipset.conf
```

然后启用 `ipset.service`, 与 `iptables.service`相似，这个服务用于恢复 iptables 规则。

使用PeerGuardian和其它列表屏蔽

maeyanie.com所作的pg2ipset-gitAUR与ipset-update.sh 脚本配合可以用cron来自动更新多个屏蔽列表。 当前实现了按默认国家屏蔽，tor退出节点屏蔽，和来自Bluetack的pg2列表屏蔽。

其他命令

查看集合。

```
# ipset list
```

删除名为"myset"的集合。

```
# ipset destroy myset
```

删除所有集合。

```
# ipset destroy
```

更多信息请参考ipset的man手册页。

优化

iprangeAUR工具可以通过合并相邻范围或消除重复范围来帮助减少ipset.conf中的项目。 在表的大小很大时，这有助于改善路由/防火墙的性能。这个工具也可以把一个主机名的列表转换成IP列表。

虽然 ipset 设计上可以支持非常大的屏蔽，但并不是无限的。有些国家的 IP 地址空间非常庞大，所以按区域屏蔽的效果并不理想。

2.2 Shorewall

难度 进阶 | **篇幅** 7.6 KB | **所属** 防火墙

本节将讲到

- 安装
- 配置
- 启动
- Traffic shaping

The Shoreline Firewall, more commonly known as "Shorewall", is high-level tool for configuring Netfilter.

You describe your firewall/gateway requirements using entries in a set of configuration files. Shorewall reads those configuration files and with the help of the iptables utility, Shorewall configures Netfilter to match your requirements.

Shorewall can be used on a dedicated firewall system, a multi-function gateway/router/server or on a standalone GNU/Linux system. Shorewall does not use Netfilter's ipchains compatibility mode and can thus take advantage of Netfilter's connection state tracking capabilities.

安装

Install the shorewallAUR or shorewall6AUR package.

配置

These settings are based on the two-interface documentation on the Shorewall web site.

Use some example configuration files that come with the shorewall package

```
# cp /usr/share/doc/shorewall/Samples/one-interface/* /etc/shorewall/     # If you have
a desktop-type system with a single network interface
# cp /usr/share/doc/shorewall6/Samples6/one-interface/* /etc/shorewall6/  # If you have
a desktop-type system with a single network interface, pkg shorewall6
# cp /usr/share/doc/shorewall/Samples/two-interfaces/* /etc/shorewall/    # If you have
a router with two network interfaces
# cp /usr/share/doc/shorewall/Samples/three-interfaces/* /etc/shorewall/  # If you have
a router with three network interfaces
```

/etc/shorewall/interfaces

Change the interface settings to match the names used for our Ethernet devices and to allow DHCP traffic on the local network. Edit `/etc/shorewall/interfaces`

original

```
net     eth0            dhcp,tcpflags,nosmurfs,routefilter,logmartians
loc     eth1            tcpflags,nosmurfs,routefilter,logmartians
```

new

```
net     wan           dhcp,tcpflags,nosmurfs,routefilter,logmartians
loc     lan           dhcp,tcpflags,nosmurfs,routefilter,logmartians
```

/etc/shorewall/policy

Change the policy file to allow the router (this machine) to access the Internet. Edit `/etc/shorewall/policy`

original

```
###############################################################################
#SOURCE          DEST            POLICY          LOG LEVEL       LIMIT:BURST

loc             net             ACCEPT
net             all             DROP            info
# THE FOLLOWING POLICY MUST BE LAST
all             all             REJECT          info
```

new

```
###############################################################################
#SOURCE          DEST            POLICY          LOG LEVEL       LIMIT:BURST
$FW             net             ACCEPT
loc             net             ACCEPT
net             all             DROP            info
# THE FOLLOWING POLICY MUST BE LAST
all             all             REJECT          info
```

/etc/shorewall/rules

DNS look-ups are handled (actually forwarded) by dnsmasq, so Shorewall needs to allow those connections. **Add** these lines to `/etc/shorewall/rules`

```
#       Accept DNS connections from the local network to the firewall
#
DNS(ACCEPT)     loc               $FW
```

/etc/shorewall/masq

Change the network interface to the one connected to your external (WAN) network and **change** the IP to the one used in your local network.

```
eth0        192.168.1.0/24
```

SSH

OPTIONAL: You can **add** these lines to /etc/shorewall/rules if you want to be able to SSH into the router from computers on the Internet

```
#       Accept SSH connections from the internet for administration
#
SSH(ACCEPT)     net             $FW         TCP      <SSH port used>
```

Port forwarding (DNAT)

- /etc/shorewall/rules : here is an example for a webserver on our LAN with IP 10.0.0.85. You can reach it on port 5000 of our "external" IP.

```
DNAT        net        loc:10.0.0.85:80        tcp        5000
```

/etc/shorewall/stoppedrules

If you have a network name other than eth1 for the network interface in /etc/shorewall/interfaces, you need to update stoppedrules with the correct name.

/etc/shorewall/shorewall.conf

When you are finished making above changes, enable shorewall by a **change** in its config file `/etc/shorewall/shorewall.conf`:

original

```
STARTUP_ENABLED=No
```

new

```
STARTUP_ENABLED=Yes
```

See shorewall(8)\[失效链接 2022-06-25\] for more info.

启动

Start/enable `shorewall.service`.

Traffic shaping

Read Shorewall's Traffic Shaping/Control guide.

Here is my config as an example:

- /etc/shorewall/tcdevices : here is where you define the interface you want to have shaped and its rates. I have got a ADSL connection with a 4MBit down/256KBit up profile.

```
ppp0        4mbit        256kbit
```

- /etc/shorewall/tcclasses : here you define the minimum (rate) and maximum (ceil) throughput per class. You will assign each one to a type of traffic to shape.

```
# interactive traffic (ssh)
ppp0            1       full    full    0
# online gaming
ppp0            2       full/2  full    5
# http
ppp0            3       full/4  full    10
# rest
ppp0            4       full/6  full    15              default
```

- /etc/shorewall/tcrules : this file contains the types of traffic and the class it belongs to.

```
1       0.0.0.0/0       0.0.0.0/0       tcp     ssh
2       0.0.0.0/0       0.0.0.0/0       udp     27000:28000
3       0.0.0.0/0       0.0.0.0/0       tcp     http
3       0.0.0.0/0       0.0.0.0/0       tcp     https
```

I have split it up my traffic in 4 groups:

1. interactive traffic or ssh: although it takes up almost no bandwidth, it is very annoying if it lags due to leechers on the LAN. This gets the highest priority.
2. online gaming: needless to say you cannot play when your ping sucks. ;)
3. webtraffic: can be a bit slower
4. everything else: every sort of download, they are the cause of the lag anyway.

2.3 Firewalld

难度 进阶 | **篇幅** 7.9 KB | **所属** 防火墙

本节将讲到

- 安装
- 使用
- 配置
- 提示和技巧
- 参阅

firewalld 是由红帽开发的防火墙守护进程。 默认使用 nftables 。据项目主页：

: Firewalld 提供了一个动态管理的防火墙，支持使用区域来标识网络连接/接口的可信等级。支持 IPv4、IPv6 防火墙设置、以太网桥接和 IP sets。使用分离的运行时配置和永久设置。也提供了一个接口用来直接为服务或应用添加防火墙规则。

安装

安装 firewalld包。

使用

启用并启动 `firewalld.service`.

你可以使用控制台工具 `firewall-cmd` 来控制防火墙规则。

`firewall-offline-cmd` 可在 firewalld 未运行时进行配置，与 `firewall-cmd` 类似。

随 firewalld包 带的 `firewall-config` 命令提供了一个图形界面。

配置

使用 `firewall-cmd` 可用于运行时更改配置

注意：大部分命令只会改变运行时配置而其将会在重启后丢失，要想进行永久配置有两种方式：

- 使用 `--permanent` 参数。这 **不会** 更改运行时配置直至防火墙服务重启，或使用 `--reload` 重载规则。
- 持久化规则可见：#将运行时配置设为永久

区域

区域是一系列可用于指定接口的规则。

要查看当前区域及其应用的接口，使用：

```
# firewall-cmd --get-active-zones
```

一些命令（例如 add/remove port/service） 需要指定区域/

区域可以通过 `--zone=*zone_name*` 参数传递。

若未指定一个确切的区域，则使用默认区域。

区域信息

你可以列出所有区域的配置：

```
# firewall-cmd --list-all-zones
```

或指定一个区域

```
# firewall-cmd --info-zone=zone_name
```

更改接口区域

```
# firewall-cmd --zone=zone --change-interface=interface_name
```

此处 `*zone*` 是你想分配给接口的区域。

使用 NetworkManager 管理区域

NetworkManager 能够为不同的连接分配不同的区域。例如：将一个家庭 WiFi 分配到 "home" 区域， 将工作室 WiFi 分配到 "work" 区域，并将剩余 WiFi 分配到 "public" 区域。

列出连接配置：

```
$ nmcli connection show
```

将 "myssid" 设置为 "home" 区域：

```
$ nmcli connection modify myssid connection.zone home
```

默认区域

默认区域将会被自动应用到新的接口上，查询默认区域使用：

```
# firewall-cmd --get-default-zone
```

更改默认区域使用：

```
# firewall-cmd --set-default-zone=zone
```

注意：此改变是临时的

服务

服务是为特定守护程序预配置的规则。例如匹配 SSH 的 服务 {{ic|ssh} 在被分配到一个区域后会开放 22 端口。

使用下面的命令列出所有可用的服务：

```
# firewall-cmd --get-services
```

查询特定服务的信息：

```
# firewall-cmd --info-service service_name
```

从区域中添加或移除服务

添加一个服务到区域：

```
# firewall-cmd --zone=zone_name --add-service service_name
```

移除服务：

```
# firewall-cmd --zone=zone_name --remove-service service_name
```

端口

可以直接在指定区域上开放端口：

```
# firewall-cmd --zone=zone_name --add-port port_num/protocol
```

此处 `*protocol*` 应为 `tcp` 或 `udp` 之一。

关闭端口使用带有相同的端口号和协议的 `--remove-port` 选项。

NAT 地址伪装

此命令与 `iptables -t nat -A POSTROUTING -j MASQUERADE` 具有相同的效果：

```
# firewall-cmd --zone=public --add-masquerade
```

从版本 1.0.0 开始，如果想要 NAT 地址伪装在不同的防火墙区域之间工作，您必须创建一个新的策略对象，用于过滤它们之间的流量：

```
# firewall-cmd --new-policy NAT_int_to_ext --permanent
# firewall-cmd --permanent --policy NAT_int_to_ext --add-ingress-zone internal
# firewall-cmd --permanent --policy NAT_int_to_ext --add-egress-zone public
# firewall-cmd --permanent --policy NAT_int_to_ext --set-target ACCEPT
```

端口转发

如果你在路由器上配置了 firewalld，而且还如上开启了 NAT 地址伪装，则通过 firewalld 设置端口转发很简单：

```
# firewall-cmd --zone=public --add-forward-
port=port=12345:proto=tcp:toport=22:toaddr=10.20.30.40
```

这会将外部端口上的 12345/tcp 端口转发到内部网络 10.20.30.40 的 22 （标准 SSH） 端口。移除此端口转发：

```
# firewall-cmd --zone=public --remove-forward-
port=port=12345:proto=tcp:toport=22:toaddr=10.20.30.40
```

不幸的是你必须键入完整的转发声明以移除它，只指定端口和协议是不行的。

Rich 规则

通过使用 Rich 规则，可以以一种易于理解的方式创建更复杂的防火墙规则。

要添加 Rich 规则：

```
# firewall-cmd [--zone=zone_name] [--permanent] --add-rich-rule='rich_rule'
```

其中 `*rich_rule*` 是一条丰富语言规则。

如果要允许来所有的连接自网络 192.168.1.0/24 到 NFS service：

```
# firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" service name="nfs" accept'
```

要允许来自 192.168.2.3 到端口 1234/tcp 的连接：

```
# firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.2.3" port port="1234" protocol="tcp" accept'
```

有关更多 Rich 规则语法，请参阅 firewalld.richlanguage(5)。

要删除 Rich 规则：

```
# firewall-cmd  [--zone=zone_name] [--permanent] --remove-rich-rule='rich_rule'
```

提示和技巧

端口或服务有效时间

使用 --timeout=*value* 可以设置服务或端口的有效时间。时间默认为秒，m 后缀代表分钟， h 后缀代表小时。 例如：为 SSH 开放三小时的使用时间：

```
# firewall-cmd --add-service ssh --timeout=3h
```

注意：超时选项与 --permanent 互斥。也就是说，超时只能是运行时配置，不可能持久化。

将运行时配置设为永久

你可以将运行时配置持久化（这意味着重启后也会保留配置）

```
# firewall-cmd --runtime-to-permanent
```

检查服务配置

/usr/lib/firewalld/services/ 存储了默认支持的服务的配置文件，/etc/firewalld/services/ 存储了用户创建的服务文件。

移除程序/托盘图标

托盘图标与 firewalld包 一起打包。其自启动脚本位于 /etc/xdg/autostart/firewall-applet.desktop 可以被隐藏，参见 XDG Autostart#目录。另一种方法是，通过将其添加到 /etc/pacman.conf 中的 NoExtract 来阻止该文件的安装。

参阅

- firewall-cmd(1)
- Official documentation
- Fedora:Firewalld
- Fedora:Features/FirewalldRichLanguage

2.4 Fail2ban

难度 进阶 | **篇幅** 8.6 KB | **所属** 防火墙

本节将讲到

- 安装
- 用法
- 配置
- 小技巧
- 参考

Fail2ban 会检查日志文件（例如 `/var/log/httpd/error_log`）并封禁存在恶意行为的 IP,例如大量身份验证请求，漏洞扫描等。Fail2ban 通常会在防火墙规则上将这些 IP 封禁一段时间，但也可以配置像发送邮件等其它操作。

警告：

- IP 封禁软件可以防止一些简单的攻击，但依赖于额外的守护进程和正确的日志记录配置。
- 在类似为 sshd 启用仅接受公钥认证等情况下，没有必要使用 fail2ban。
- 这不是 VPN 的替代品，请尽量不要将服务暴露到互联网上。
- 如果攻击者知道你的 IP，他们可以发送修改了源地址头字段的包使你的 IP 被服务器封禁，因此务必将你的 IP 添加到 `ignoreip` 中。

安装

安装以下软件包之一：

- fail2ban包 - 最新的稳定版
- fail2ban-gitAUR - 包含添加到代码库的最新更改

用法

配置 Fail2ban 并启用/启动 `fail2ban.service`。

fail2ban-client

fail2ban-client 可以用于管理 jail 的状态（包括重新加载、重启、查看状态等）。要查看所有可用命令：

```
$ fail2ban-client
```

要查看已启用的 jail：

```
# fail2ban-client status
```

要查看 jail 的状态（以 *sshd* 为例）：

```
# fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 1
|  |- Total failed:     9
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd<br>`- Actions<br>|-
Currently banned: 1<br>|- Total banned:     1<br>`- Banned IP list:   0.0.0.0<br>```
```

要查看所有 `jail` 的简略信息（包括已封禁的 `IP`）：

[{'sshd': ['192.168.100.50']}, {'apache-auth': []}]

配置

[jail.conf(5) § CONFIGURATION FILES FORMAT](https://man.archlinux.org/man/jail.conf.5#CONFIGURATION_FILES_FORMAT) 建议用户创建 `/etc/fail2ban/jail.local` 文件，否则在更新时可能会为 `/etc/fail2ban/jail.conf` 创建新的 [Pacnew 和 Pacsave 文件](../../01_安装与包管理/包管理器/Pacman_pacnew%20与%20pacsave.md)。

以将默认封禁时间设为一天为例：
/etc/fail2ban/jail.local

[DEFAULT] bantime = 1d

也可以在 `/etc/fail2ban/jail.d` 目录下创建单独的 `*name*.local` 文件（例如 `/etc/fail2ban/jail.d/sshd.local`）。

重载 `fail2ban.service` 以应用配置更改。

启用 jail

所有 jail 默认都是禁用的。要启用 jail，需要在对应项添加 `enabled = true`。以启用 [OpenSSH](../SSH/OpenSSH.md) jail 为例：
/etc/fail2ban/jail.local

[sshd] enabled = true

具体配置请参考 [#配置 SSH jail](#配置_SSH_jail)。

接收告警邮件

如果你想在封禁时收到邮件提醒，需要参考下方配置 SMTP 客户端（例如 msmtp）并修改默认操作：
/etc/fail2ban/jail.local

[DEFAULT] destemail = yourname@example.com sender = yourname@example.com

action = %(action_mw)s

防火墙和服务

Fail2ban 默认使用 [iptables](Iptables.md)，但要配置大部分防火墙和服务也很简单，以 [nftables](Nftables.md) 为例：
/etc/fail2ban/jail.local

[DEFAULT] banaction = nftables banaction_allports = nftables[type=allports]

其它示例请参考 `/etc/fail2ban/action.d/`，例如 [ufw.conf](https://github.com/fail2ban/fail2ban/blob/master/config/action.d/ufw.conf)。

小技巧

配置 SSH jail

> **警告：**如果攻击者知道你的 IP，他们可以发送修改了源地址头字段的包使你的 IP 被服务器封禁。[SSH 密钥](../SSH/SSH%20密钥.md)为解决暴力攻击提供了简洁的方案，而不会产生这些问题。

编辑 `/etc/fail2ban/jail.d/sshd.local`，添加以下内容，并在 `ignoreip` 中修改可信 IP 清单：
/etc/fail2ban/jail.d/sshd.local

[sshd] enabled = true filter = sshd banaction = iptables backend = systemd maxretry = 5 findtime = 1d bantime = 2w ignoreip = 127.0.0.1/8

> **注意：**
>
> - 为使 fail2ban 具有完整监控能力，可能需要在 `/etc/ssh/sshd_config` 中设置 `LogLevel VERBOSE`，否则像密码错误等可能不会被正确记录。
> - Fail2ban 从 0.10 版本开始支持 IPv6，请按需调整防火墙（例如启动/启用 `ip6tables.service`）。
> - 在使用 journal 命名空间（即在单元文件中使用 `LogNamespace=*something*`）时，可以像这样配置 `backend` 来让 fail2ban 读取这些日志：`backend = systemd[journalfiles="/var/log/journal/*.*something*/system.journal"]`

> **提示：**
>
> - 如果使用像 [ufw](Uncomplicated%20Firewall.md) 这样的 [iptables](Iptables.md) 前端，可以使用 `banaction = ufw` 替代 iptables。
> - 在使用 [Shorewall](Shorewall.md) 时，可以在 `/etc/shorewall/shorewall.conf` 中使用 `banaction = shorewall`，并将 `BLACKLIST` 设为 `ALL`，否则新的 IP 封禁规则只会影响新连接。

Systemd 后端：journald 过滤

When using the *systemd* backend to improve performance, configure a filter with `journalmatch`. For example, to parse only kernel-level log messages:
/etc/fail2ban/filter.d/fwdrop.local

[Definition] failregex = ^.*DROP_.SRC=<ADDR> DST=.*$ journalmatch = _TRANSPORT=kernel

See also [systemd.journal-fields(7)](https://man.archlinux.org/man/systemd.journal-fields.7).

服务加固

现在 Fail2ban 只能以*根用户*权限运行。因此，你可能会想使用 [systemd](../../01_安装与包管理/初始化/Systemd.md) 加固进程。

为 `fail2ban.service` 创建[附加配置片段](../../01_安装与包管理/初始化/Systemd.md#附加配置片段)文件：
/etc/systemd/system/fail2ban.service.d/override.conf

[Service] PrivateDevices=yes PrivateTmp=yes ProtectHome=read-only ProtectSystem=strict ReadWritePaths=-/var/run/fail2ban ReadWritePaths=-/var/lib/fail2ban ReadWritePaths=-/var/log/fail2ban.log ReadWritePaths=-/var/spool/postfix/maildrop ReadWritePaths=-/run/xtables.lock CapabilityBoundingSet=CAP_AUDIT_READ CAP_DAC_READ_SEARCH CAP_NET_ADMIN CAP_NET_RAW

`CapabilityBoundingSet` 的 `CAP_DAC_READ_SEARCH` 参数会允许 Fail2ban 读取所有目录和文件，`CAP_NET_ADMIN` 和 `CAP_NET_RAW` 会允许 Fail2ban 操作任何具有[命令行解释器](../../07_开发与工具/命令行解释器/命令行解释器.md)界面的防火墙。具体信息请参考 [capabilities(7)](https://man.archlinux.org/man/capabilities.7)。

使用 `ProtectSystem=strict` 会使[文件系统](../../02_系统管理与内核/文件系统/文件系统.md)层次结构设为只读，`ReadWritePaths` 会允许 Fail2ban 对特定路径具有写入权限。

最后[重载 systemd 守护进程](../../01_安装与包管理/初始化/Systemd.md#使用单元)以应用对单元文件的修改，并重启 `fail2ban.service`。

参考

- [使用 Fail2Ban Jail 将用户加入白名单](https://www.the-art-of-web.com/system/fail2ban-action-whitelist/)
- [Fail2Ban 过滤器优化](https://www.the-art-of-web.com/system/fail2ban-filters/)
- [Fail2Ban 与 sendmail](https://www.the-art-of-web.com/system/fail2ban-sendmail/)
- [Fail2Ban 与 iptables](https://www.the-art-of-web.com/system/fail2ban/)
- [Fail2Ban 0.8.3 用法](https://www.the-art-of-web.com/system/fail2ban-howto/)
- [监控 fail2ban 日志](https://www.the-art-of-web.com/system/fail2ban-log/)

2.5 Uncomplicated Firewall

难度 进阶 | **篇幅** 9.6 KB | **所属** 防火墙

本节将讲到

- 安装
- 基本配置
- 转发策略
- 添加其他应用
- 删除应用

来自项目主页：

: Ufw（即 Uncomplicated Firewall 的缩写）是一个管理网络防火墙的程序。它提供了一个命令行界面，并旨在简单（uncomplicated）易用。

注意：注意 UFW 可以使用 iptables包 或 nftables包 作为防火墙后端。此外，由于 `nft` 接受 iptables 语法（例如在 `/etc/ufw/before.rules` 中），因此使用 UFW 管理规则的用户无需学习怎么对 iptables 或者 nftables 进行底层调用。

安装

安装软件包 ufw包。

启动并启用 `ufw.service` 使其开机自启。注意，如果 `iptables.service`（或者其 ipv6 部分）也被启用了，`ufw.service` 将不会生效。

基本配置

以下是一个非常简单的配置示例，它将会默认拒绝所有连接，允许来自 192.168.0.1-192.168.0.255 局域网内的所有连接，允许所有目标为 Deluge（英语：Deluge）（一个 BitTorrent 客户端）的连接，并启用了来自*任何地方*的 SSH 连接速率限制：

```
# ufw default deny
# ufw allow from 192.168.0.0/24
# ufw allow Deluge
# ufw limit ssh
```

要想允许一个端口而不是*任何地方*进行连接，请参考下面的例子。这三条命令分别允许端口 51312 的 UDP 和 TCP 连接，允许端口 51313 的 UDP 连接，和允许范围从 51312 到 51314 端口的连接：

```
# ufw allow 51312
# ufw allow 51312/udp
# ufw allow 51312:51314
```

仅当首次安装后*第一次使用* UFW 时需要执行:

```
# ufw enable
```

注意：请确保 `ufw.service` 已经被启用。

最后,使用以下命令确认更改已被应用:

```
# ufw status
Status: active
To                         Action      From
--                         ------      ----
Anywhere                   ALLOW       192.168.0.0/24
Deluge                     ALLOW       Anywhere
SSH                        LIMIT       Anywhere
```

注意：状态报告仅限于用户添加的规则。在大多数情况下不会有什么问题，但最好还是了解一下内置规则是否存在。这些规则包括允许 UPNP、AVAHI 和 DHCP 回复的过滤器。具体信息请参考 ufw README 的“Default ruleset”一节。

使用以下命令来查看额外信息（包括默认策略）：

```
# ufw status verbose
```

但这依然限制于用户指定的规则。为了查看所有设置的规则，可以使用：

```
# ufw show raw
```

或者手册中列出的其他报告。由于这些报告也总结了流量情况，其输出有些难以阅读，为了方便阅读，你可以使用另一种检查传入流量的方法：

```
# iptables -S | grep ACCEPT
# ip6tables -S | grep ACCEPT
```

请注意，只要你还在使用 `ufw` 来管理 `iptables`，就不要将后者启用。

注意：如果你在 `/etc/sysctl.d/*` 中设置了特殊的网络环境变量，请相应更新 `/etc/ufw/sysctl.conf` 中的内容（因为此配置会覆盖默认配置）。

转发策略

如果你需要使用 VPN（例如 OpenVPN 或 WireGuard），可以将 `/etc/default/ufw` 中的 **DEFAULT_FORWARD_POLICY** 变量从 **"DROP"** 调整到 **"ACCEPT"**，以便无论用户如何设置如何都能转发所有数据包。要针对特定接口（例如 **wg0**）进行转发，用户可以在 ***filter** 块中添加以下行：

```
/etc/ufw/before.rules
```

```
# End required lines

-A ufw-before-forward -i wg0 -j ACCEPT
-A ufw-before-forward -o wg0 -j ACCEPT
```

你可能还需要取消以下行的注释：

```
/etc/ufw/sysctl.conf
net/ipv4/ip_forward=1
net/ipv6/conf/default/forwarding=1
net/ipv6/conf/all/forwarding=1
```

添加其他应用

软件包还提供了一些常见守护进程和程序的默认端口配置。可以通过查看 `/etc/ufw/applications.d` 目录下的文件或使用如下指令查看：

```
# ufw app list
```

如果用户正在使用非标准端口运行任何应用程序，建议创建 `/etc/ufw/applications.d/custom` 文件，并按照默认配置做模板添加所需的数据。

警告：如果用户修改了任何由软件包提供的规则集，这些修改将在第一次更新 ufw 软件包时被覆盖。这就是为什么建议将自定义应用程序规则放在外面的原因！

例如，使用自定义的 TCP 端口范围 20202-20205 进行 deluge 下载：

```
[Deluge-my]
title=Deluge
description=Deluge BitTorrent client
ports=20202:20205/tcp
```

如果需要为同一应用程序定义 TCP 和 UDP 端口，只需使用竖线进行分隔，如下所示：该应用程序打开 TCP 端口 10000-10002 和 UDP 端口 10003：

```
ports=10000:10002/tcp|10003/udp
```

还可以使用逗号来定义端口。此示例打开 TCP 端口 10000-10002（包括端口 10000 和 10002）以及 UDP 端口 10003 和 10009：

```
ports=10000:10002/tcp|10003,10009/udp
```

删除应用

根据上面的 Deluge/Deluge-my 示例，以下操作将删除标准的 Deluge 规则，并用上面示例中的 Deluge-my 规则替换它们：

```
# ufw delete allow Deluge
# ufw allow Deluge-my
```

通过 status 指令查询结果：

```
# ufw status
Status: active
To                         Action      From
--                         ------      ----
Anywhere                   ALLOW       192.168.0.0/24
SSH                        ALLOW       Anywhere
Deluge-my                  ALLOW       Anywhere
```

IP地址黑名单

将 IP 地址添加到黑名单可能是一个可取的做法，可以简单地编辑 `/etc/ufw/before.rules` 文件，在文件底部在 "COMMIT" 的上方插入一个 iptables DROP 行来轻松实现。

```
/etc/ufw/before.rules
...
## blacklist section
# block just 199.115.117.99
-A ufw-before-input -s 199.115.117.99 -j DROP
# block 184.105.*.*
-A ufw-before-input -s 184.105.0.0/16 -j DROP

# don't delete the 'COMMIT' line or these rules won't be processed
COMMIT
```

连接速率限制

ufw 有能力拒绝来自在过去 30 秒内尝试建立 6 次或更多连接的 IP 地址的连接。用户应该考虑在 SSH 之类的服务中使用这个选项。

利用上述基本配置，要启用速率限制，我们只需用 limit 参数替换 allow 参数。然后新规则将取代先前的规则。

```
# ufw limit SSH
Rule updated
# ufw status
Status: active
To                         Action      From
--                         ------      ----
Anywhere                   ALLOW       192.168.0.0/24
SSH                        LIMIT       Anywhere
Deluge-my                  ALLOW       Anywhere
```

用户规则

所有的用户规则都储存在 `etc/ufw/user.rules` 和 `etc/ufw/user6.rules` 中，分别用于 IPv4 与 IPv6。

技巧和窍门

禁用远程 ping

在以下文件中将 `ACCEPT` 改为 `DROP`：

```
/etc/ufw/before.rules
# ok icmp codes
...
-A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT
```

如果你正在使用 IPv6, 请一同更改 `/etc/ufw/before6.rules` 中的规则。

禁用 IPv6 处理

将 `/etc/default/ufw` 中的 `IPV6=yes` 改为 `IPV6=no`，然后重新加载规则（`ufw reload`）。这样会移除原有 IPv6 规则，并且不再生成新的 IPv6 规则。

禁用 UFW 日志

禁用 UFW 日志可以让 UFW 停止填充内核（dmesg）和消息日志：

```
# ufw logging off
```

UFW 与 Docker

Docker 在默认状态下会编写 iptables 规则并忽视 ufw 规则，这可能会造成一些安全隐患。一种解决方案是使用 https://github.com/chaifeng/ufw-docker 。

提示：你可以安装 ufw-dockerAUR，并通过执行 `ufw-docker install` 来自动修复 iptables 规则。这个脚本也能帮你管理 Docker 相关的 ufw 规则，详细参见 `ufw-docker help`。

GUI 前端

如果你在使用 KDE Plasma，可以安装 plasma-firewall包，然后直接在 *Wi-Fi & Networking* > *Firewall* 中调整防火墙配置。

Gufw

gufw包 是一个用于 Ufw 的 GTK 前端，旨在使 Linux 防火墙的管理尽可能简单易用。它具有常见端口和 p2p 应用程序的预设设置。其依赖于软件包 python包，ufw包 以及 GTK 支持。

参考

- Ubuntu UFW 文档
- ufw(8)
- 使用 ipsets 为 ufw 添加屏蔽列表：https://github.com/poddmo/ufw-blocklist

2.6 Sshguard

难度 进阶 | **篇幅** 10.4 KB | **所属** 防火墙

本节将讲到

- 安装
- 设置
- 用法
- 配置文件
- 提示与技巧

警告：使用 IP 黑名单将会阻挡普通的攻击，但它依赖于一个额外的守护进程以及成功的日志记录（包含 /var 的分区可能会被占满，尤其是当攻击者正在攻击服务器时）。另外，由于知道您的 IP 地址，攻击

者可以发送伪装源头的包，并将您阻挡在服务器之外。SSH 密钥 为防止暴力破解提供了一个优雅的解决方案，而没有上述问题。

sshguard 是一个用于保护 SSH 以及其他服务免受暴力破解攻击的守护进程，类似于 fail2ban。

sshguard 与后者的不同之处在于，它是使用 C 编写的，使用起来更轻量、简单，功能更少，但同时也能出色地执行核心功能。

sshguard 不会受到大多数（或任何）日志分析漏洞的影响，而这些漏洞曾导致同类型工具出现问题。

安装

安装 sshguard包 软件包.

设置

sshguard 通过监控 `/var/log/auth.log`，syslog-ng 或 systemd 日志来获得失败的登录尝试。对于每次失败的尝试，违规的主机都会在有限时间内被禁止，无法进行通信。对于违规主机，被禁止的默认时间长度从 120 秒开始，之后每次登录失败都会增加 1.5 倍。sshguard 可以被配置为永久禁止一个失败尝试次数过多的主机。

临时和永久禁止都是通过在 iptables 的 "sshguard" 链中添加条目完成的，该条目会删除来自违规主机的所有数据包。随后该封禁会被记录到 syslog 并保存到 `/var/log/auth.log`，如果使用 systemd 日志，则会保存到 systemd 日志中。

您必须配置下列防火墙之一来让 sshguard 的屏蔽发挥作用。

firewalld

sshguard 可以与 firewalld 一同工作。确保您已安装、配置并启用 firewalld。为了让 sshguard 写入设置区域，请执行以下命令：

```
# firewall-cmd --permanent --zone=public --add-rich-rule="rule source ipset=sshguard4 drop"
```

如果您使用 ipv6，您可以执行相同的命令，但需要把 sshguard4 替换为 sshguard6。完成后，执行以下命令：

```
# firewall-cmd --reload
```

您可以通过以下命令确认上述内容：

```
# firewall-cmd --info-ipset=sshguard4
```

最后，在 /etc/sshguard.conf 中，找到 BACKEND 行并变更如下：

```
BACKEND="/usr/lib/sshguard/sshg-fw-firewalld"
```

UFW

如果安装并启用了 UFW，则必须赋予它将 DROP 控制传递给 sshguard 的能力。可以通过在 `/etc/ufw/before.rules` 中包含以下行来实现。这些行应该被插入到回环设备部分之后。

注意：在非标准端口上运行 sshd 的用户需要修改最后一行中的端口号（标准端口为 22）。

```
/etc/ufw/before.rules
# 允许全部回环
-A ufw-before-input -i lo -j ACCEPT
-A ufw-before-output -o lo -j ACCEPT

# 将 sshd 的控制权交给 sshguard
:sshguard - [0:0]
-A ufw-before-input -p tcp --dport 22 -j sshguard
```

在进行上述变更后需要重新启动 ufw。

iptables

注意：请先参阅 iptables 和 简单的有状态防火墙 来设置防火墙。

必要的设置是在 iptables 上创建一个名为 `sshguard` 的链。sshguard 将会在这个链上自动插入规则以丢弃来自坏主机的数据包：

```
# iptables -N sshguard
```

接下来添加一条从 `INPUT` 链跳转到 `sshguard` 链的规则。这条规则必须被添加在任何其他处理 sshguard 保护端口的规则之前。使用下列行来保护 FTP 和 SSH，或查看了解更多实例。

```
# iptables -A INPUT -m multiport -p tcp --destination-ports 21,22 -j sshguard
```

若要保存这些规则：

```
# iptables-save > /etc/iptables/iptables.rules
```

注意：For IPv6, repeat the same steps with *ip6tables* and save the rules with *ip6tables-save* to `/etc/iptables/ip6tables.rules`.

nftables

编辑 `/etc/sshguard.conf` 并将 `BACKEND` 的值更改如下：

```
/etc/sshguard.conf
BACKEND="/usr/lib/sshguard/sshg-fw-nft-sets"
```

When you start/enable the `sshguard.service`, two new tables named `sshguard` in the `ip` and `ip6` address families are added which filter incoming traffic through sshguard's list of IP addresses. The chains in the `sshguard` table have a priority of -10 and will be processed before other rules of lower priority. See sshguard-setup(7) and nftables for more information.

用法

systemd

启用并启动 `sshguard.service`。

syslog-ng

如果您安装了 syslog-ng包，您可以直接从命令行启动 sshguard。

```
/usr/sbin/sshguard -l /var/log/auth.log -b /var/db/sshguard/blacklist.db
```

配置文件

Configuration is done in `/etc/sshguard.conf` which is required for *sshguard* to start. A commented example is located at `/usr/share/doc/sshguard/sshguard.conf.sample` or can also be found on Bitbucket sshguard.conf.sample.

注意：Piped commands and runtime flags in *sshguard's* systemd units are not supported. Such flags can be modified in the configuration file.

Blacklisting threshold

By default in the Arch-provided configuration file, offenders become permanently banned once they reach a "danger" level of 120 (or 12 failed logins; see attack dangerousness for more details). This behavior can be modified by prepending a danger level to the blacklist file.

```
BLACKLIST_FILE=200:/var/db/sshguard/blacklist.db
```

The `200:` in this example tells sshguard to permanently ban a host after achieving a danger level of 200.

Finally restart `sshguard.service`

Moderate banning example

A slightly more aggressive banning rule than the default one is proposed here to illustrate various options:

- It monitors sshd and vsftpd via logs from the systemd/日志
- It blocks attackers after 2 attempts (each having a cost of 10, explaining the `20` value of the `THRESHOLD` parameter) for 180 seconds with subsequent block time longer by a factor of 1.5. Note that this 1.5 multiplicative delay is internal and not controlled in the settings
- Attackers are permanently blacklisted after 10 attempts (10 attempts having each a cost of 10, explaining the `100` value in the `BLACKLIST_FILE` parameter)
- It blocks not only the attacker's IP but all the IPv4 subnet 24 (CIDR notation)

```
/etc/sshguard.conf
# Full path to backend executable (required, no default)
BACKEND="/usr/lib/sshguard/sshg-fw-iptables"
```

```
# Log reader command (optional, no default)
LOGREADER="LANG=C.UTF-8 /usr/bin/journalctl -afb -p info -n1 -t sshd -t vsftpd -o cat"

# How many problematic attempts trigger a block
THRESHOLD=20
# Blocks last at least 180 seconds
BLOCK_TIME=180
# The attackers are remembered for up to 3600 seconds
DETECTION_TIME=3600

# Blacklist threshold and file name
BLACKLIST_FILE=100:/var/db/sshguard/blacklist.db

# IPv6 subnet size to block. Defaults to a single address, CIDR notation. (optional,
default to 128)
IPV6_SUBNET=64
# IPv4 subnet size to block. Defaults to a single address, CIDR notation. (optional,
default to 32)
IPV4_SUBNET=24
```

Aggressive banning

For some users under constant attack, a more aggressive banning policy can be adopted. If you are confident that accidental failed logins are unlikely, you can instruct SSHGuard to permanently ban hosts after a single failed login. Modify the parameters in the configuration file in the following way:

```
THRESHOLD=10
BLACKLIST_FILE=10:/var/db/sshguard/blacklist.db
```

最后重新启动 `sshguard.service`。

另外，为了防止在单个连接期间进行多次身份验证尝试，您需要在 `/etc/ssh/sshd_config` 中定义如下：

```
MaxAuthTries 1
```

重新启动 `sshd.service` 来使此更改生效。

提示与技巧

解除封禁

如果您封禁了自己，你可以等待自动解封，或者使用 iptables 或 nftables 来解除封禁。

要永久解除您的封禁，您还需要从 `/var/db/sshguard/blacklist.db` 移除您的 IP 地址。

iptables

首先检查您的 IP 是否被 sshguard 阻止：

```
# iptables --list sshguard --line-numbers --numeric
```

然后使用下列命令来解禁，使用前一个命令中标识的行号：

```
# iptables --delete sshguard line-number
```

nftables

从 `attackers` 中移除您的 IP 地址：

```
# nft delete element family sshguard attackers { ip_address }
```

其中 *family* 可以为 ip 或 ip6。

日志记录

要查看传递给 sshguard 的内容，请检查 /usr/lib/systemd/scripts/sshguard-journalctl 中的脚本和 systemd 服务 sshguard.service。一个在终端中查看日志的等效命令：

```
# journalctl -afb -p info SYSLOG_FACILITY=4 SYSLOG_FACILITY=10
```

2.7 Iptables

难度 高级 | **篇幅** 28.2 KB | **所属** 防火墙

本节将讲到

- 安装
- 基本概念
- 配置并运行 iptables
- 日志
- 参见

iptables 是一个配置 Linux 内核防火墙的命令行工具，是 Netfilter 项目的一部分。术语 *iptables* 也经常代指该内核级防火墙。iptables 可以直接配置，也可以通过许多控制台和图形化前端配置。iptables 用于 ipv4，*ip6tables* 用于 IPv6。*iptables*和 *ip6tables* 拥有相同的语法，但是有些特别的选项，对 IPv4 和 IPv6 有些不同的。

注意：*iptables*是一个遗留的框架，nftables 致力于提供一个现代化的替代产品，包括一个兼容层。

安装

Arch Linux 内核已经编译了 iptables 支持。只需要安装用户层工具 iptables包 包即可。iptables包是base包meta package的间接依赖，因此系统中已经默认安装了 iptables 包。

前端

控制台

- **Arno's firewall** — 可用于单机器和多连接机器的安全防火墙。配置简单，管理方便，高度定制化。支持：NAT和SNAT, 端口转发，拥有静态和动态IP分配的ADSL网络调制解调器，MAC地址过滤，隐身端口扫描检测，DMZ和DMZ-2-LAN转发，保护免SYN/ICMP洪范攻击，使用限速的可自定义的用户日志防御日志洪范攻击，所有IP协议和VPN(如IPsec)，插件支持额外添加功能。

: https://rocky.eld.leidenuniv.nl/ || arno-iptables-firewallAUR

- **ferm** — 维护复杂防火墙的工具，无需一遍遍的向防火墙写入复杂的规则。它允许将整个防火墙规则储存在一个单独的文件中，之后使用一个单独的命令进行加载。 防火墙配置类似于结构变成语言，可以包含levels和lists。

: http://ferm.foo-projects.org/ || ferm包

- **FireHOL** — 用来表达防火墙规则的语言，不仅仅是生成某种防火墙的脚本。它可以以一种你想要的方式，使构建一个很复杂的防火墙变得简单。

: http://firehol.sourceforge.net/ || fireholAUR

- **Firetable** — 维护IPtables防火墙的工具。每个接口可以通过其自己的配置文件单独配置，配置文件是以一种简单且人类可读的语法书写的。

: https://gitlab.com/hsleisink/firetable || firetableAUR

- **firewalld (firewall-cmd)** — 守护进程和控制台接口，以用来配置网络和防火墙保护区域，和设置与配置防火墙规则一样。

: https://firewalld.org/ || firewalld包

- **Shorewall** — 配置Netfilter的高级工具。您可以使用一组配置文件中的条目描述防火墙/网关的需求。

: http://www.shorewall.net/ || shorewallAUR

- **Uncomplicated Firewall** — iptables的简单前端。

: https://launchpad.net/ufw || ufw包

- **PeerGuardian (pglcmd)** — 面向隐私的防火墙应用。它使用一个指定的巨大的阻塞链表（包含数千或数百万的IP的范围）来阻止主机的出站和入站连接。

: https://sourceforge.net/projects/peerguardian/ || pglAUR

- **Vuurmuur** — 强大的防火墙管理器。它拥有一个简单易学配置方式，并且支持简单或复杂的配置。配置完全可以使用ncurses包GUI进行，并且该GUI支持通过SSH或控制台进行的安全远程管理。Vuurmuur支持流量整形，拥有一个强大的监控功能，它允许管理员实时查看日志，链接和带宽使用。

: https://www.vuurmuur.org/ || vuurmuurAUR

- **Servicewall** — 简单的自定义的iptables前端，使你可以定义在连接到特定领域时允许的服务，并且在需要时自动的切换配置文件。它的服务定义通过

jhansonxi提供，通过ufw使用。它依赖于ulogd来向日志文件提供丢弃的包的日志，并且提供了一个注重日志访问限制的日志检查框架。

: https://github.com/lafleurdeboum/servicewall || servicewall-gitAUR\[损坏的链接：package not found\]

图形化

- **Firewall Builder** — GUI防火墙配置管理工具，支持iptables (netfilter), ipfilter, pf, ipfw, 思科 PIX (FWSM, ASA) 和思科路由拓展访问链表。程序可以运行在Linux，FreeBSD，OpenBSD，Windows和macOS上，并且可以管理本地和远程的防火墙。

: https://fwbuilder.sourceforge.net/ || fwbuilder包

- **firewalld (firewall-config)** — 用于配置网络和防火墙区域，以及设置和配置防火墙规则的守护进程和图形界面。

: https://firewalld.org/ || firewalld包

- **Gufw** — 用于ufw包的基于GTK的前端，这正好是一个十分简单好用的对于iptables(gufw->ufw->iptables)的用户前端。

: https://gufw.org/ || gufw包

- **PeerGuardian GUI (pglgui)** — 面向隐私的防火墙应用。它使用一个指定的巨大的阻塞链表（包含数千或数百万的IP的范围）来阻止主机的出站和入站连接。

: https://sourceforge.net/projects/peerguardian/ || pglAUR

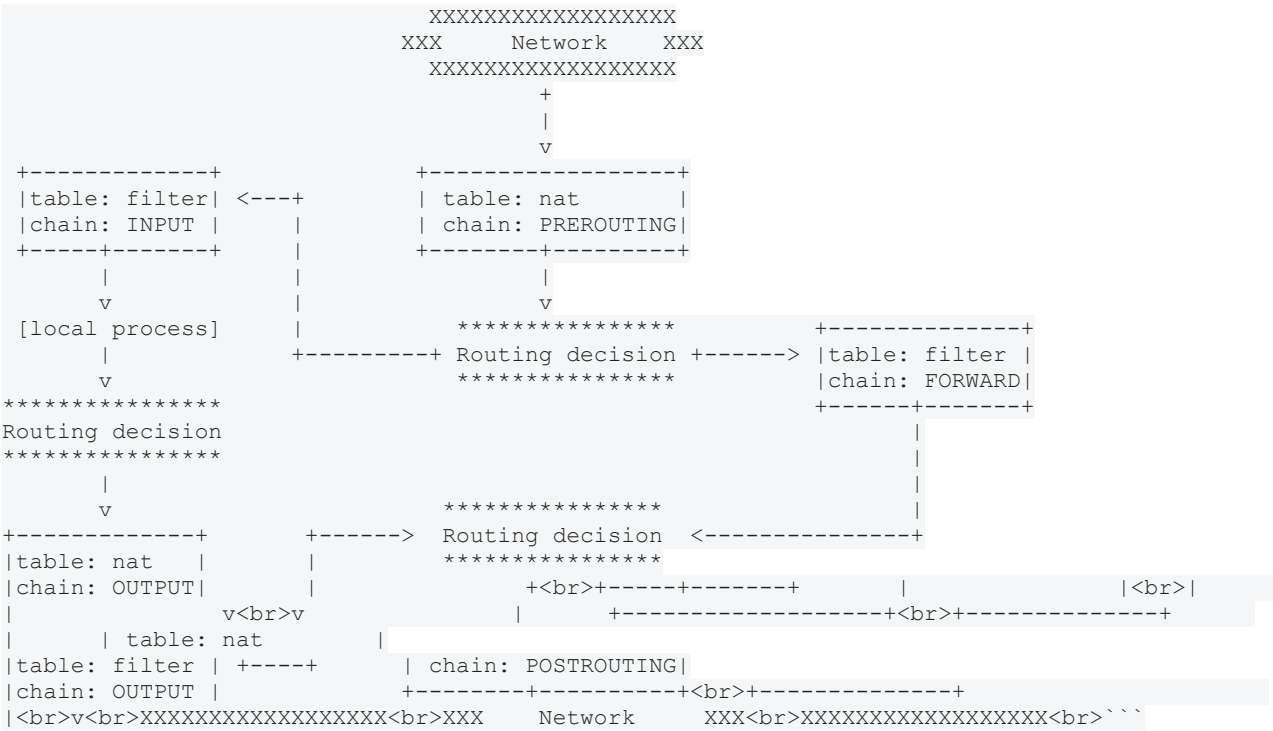
基本概念

iptables 可以检测、修改、转发、重定向和丢弃 IPv4 数据包。过滤 IPv4 数据包的代码已经内置于内核中，并且按照不同的目的被组织成**表**的集合。**表**由一组预先定义的**链**组成，**链**包含遍历顺序规则。每一条规则包含一个谓词的潜在匹配和相应的动作（称为**目标**），如果谓词为真，该动作会被执行。也就是说条件匹配。iptables 是用户工具，允许用户使用**链**和**规则**。很多新手面对复杂的 linux IP 路由时总是感到气馁，但是，实际上最常用的一些应用案例（NAT 或者基本的网络防火墙）并不是很复杂。

理解 iptables 如何工作的关键是这张图。图中在上面的小写字母代表**表**，在下面的大写字母代表**链**。**从任何网络端口**进来的每一个 IP 数据包都要从上到下的穿过这张图。

一种常见的错误认知是认为 iptables 对从内部端口进入的数据包和从面向互联网端口进入的数据包采取不同的处理方式，相反，iptables 对从任何端口进入的数据包都会采取相同的处理方式。可以定义规则使 iptables 采取不同的方式对待从不同端口进入的数据包。当然一些数据包是用于本地进程的，因此在图中表现为从顶端进入，到 <Local Process> 停止，而另一些数据包是由本地进程生成的，因此在图中表现为从 <Local Process> 发出，一直向下穿过该流程图。一份关于该流程图如何工作的详细解释请参考这里。

在大多数使用情况下都不会用到 **raw**，**mangle** 和 **security** 表。下图简要描述了网络数据包通过 **iptables** 的过程：



表（Tables）

iptables 包含 5 张表（tables）：

1. `raw` 用于配置数据包，`raw` 中的数据包不会被系统跟踪。
2. `filter` 是用于存放所有与防火墙相关操作的默认表。
3. `nat` 用于[网络地址转换](https://en.wikipedia.org/wiki/Network_address_translation)（例如：端口转发）。
4. `mangle` 用于对特定数据包的修改（参考[损坏数据包](https://en.wikipedia.org/wiki/Mangled_packet)）。
5. `security` 用于[强制访问控制](../../08_安全与加密/安全.md#Mandatory_access_control)网络规则（例如：SELinux -- 详细信息参考[该文章](https://lwn.net/Articles/267140/)）。

大部分情况仅需要使用 **filter** 和 **nat**。其他表用于更复杂的情况—包括多路由和路由判定—已经超出了本文介绍的范围。

链（Chains）

表由链组成，链是一些按顺序排列的规则的列表。默认的 `filter` 表包含 `INPUT`， `OUTPUT` 和 `FORWARD` 3条内建的链，这3条链作用于数据包过滤过程中的不同时间点，如该[流程图](https://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg)所示。`nat` 表包含`PREROUTING`， `POSTROUTING` 和 `OUTPUT` 链。

使用 [iptables(8)](https://man.archlinux.org/man/iptables.8) 查看其他表中内建链的描述。

默认情况下，任何链中都没有规则。可以向链中添加自己想用的规则。链的默认规则通常设置为 `ACCEPT`，如果想确保任何包都不能通过规则集，那么可以重置为 `DROP`。默认的规则总是在一条链的最后生效，所以在默认规则生效前数据包需要通过所有存在的规则。

用户可以加入自己定义的链，从而使规则集更有效并且易于修改。如何使用自定义链请参考 [Simple stateful firewall](Simple%20stateful%20firewall.md)。

规则 (Rules)

数据包的过滤基于**规则**。**规则**由一个*目标*（数据包匹配所有条件后的动作）和很多*匹配*（导致该规则可以应用的数据包所满足的条件）指定。一个规则的典型匹配事项是数据包进入的端口（例如：eth0 或者 eth1）、数据包的类型（ICMP，TCP，或者 UDP）和数据包的目的端口。

目标使用 `-j` 或者 `--jump` 选项指定。目标可以是用户定义的链（例如，如果条件匹配，跳转到之后的用户定义的链，继续处理）、一个内置的特定目标或者是一个目标扩展。内置目标是 `ACCEPT`， `DROP`， `QUEUE` 和 `RETURN`，目标扩展是 `REJECT` and `LOG`。如果目标是内置目标，数据包的命运会立刻被决定并且在当前表的数据包的处理过程会停止。如果目标是用户定义的链，并且数据包成功穿过第二条链，目标将移动到原始链中的下一个规则。目标扩展可以被*终止*（像内置目标一样）或者*不终止*（像用户定义链一样）。详细信息参阅 [iptables-extensions(8)](https://man.archlinux.org/man/iptables-extensions.8)。

遍历链 (Traversing Chains)

该[流程图](https://www.frozentux.net/iptables-tutorial/chunkyhtml/images/tables_traverse.jpg)描述了链在任何接口上收到的网络数据包是按照怎样的顺序穿过表的交通管制链。第一个路由策略包括决定数据包的目的地是本地主机（这种情况下，数据包穿过 `INPUT` 链），还是其他主机（数据包穿过 `FORWARD` 链）；中间的路由策略包括决定给传出的数据包使用哪个源地址、分配哪个接口；最后一个路由策略存在是因为先前的 mangle 与 nat 链可能会改变数据包的路由信息。数据包通过路径上的每一条链时，链中的每一条规则按顺序匹配；无论何时匹配了一条规则，相应的 target/jump 动作将会执行。最常用的3个 target 是 `ACCEPT`, `DROP` ,或者 jump 到用户自定义的链。内置的链有默认的策略，但是用户自定义的链没有默认的策略。在 jump 到的链中，若每一条规则都不能提供完全匹配，那么数据包像[这张图片](https://www.frozentux.net/iptables-tutorial/images/table_subtraverse.jpg)描述的一样返回到调用链。在任何时候，若 `DROP` target 的规则实现完全匹配，那么被匹配的数据包会被丢弃，不会进行进一步处理。如果一个数据包在链中被 `ACCEPT`，那么它也会被所有的父链 `ACCEPT`，并且不再遍历其他父链。然而，要注意的是，数据包还会以正常的方式继续遍历其他表中的其他链。

模块 (Modules)

有许多模块可以用来扩展 iptables，例如 connlimit, conntrack, limit 和 recent。这些模块增添了功能，可以进行更复杂的过滤。

配置并运行 iptables

iptables 是一个 [Systemd](../../01_安装与包管理/初始化/Systemd.md) 服务，因此可以这样启动：
但是，除非有 `/etc/iptables/iptables.rules` 文件，否则服务不会启动，Arch [iptables](https://archlinux.org/packages/?name=iptables)包 包不包含默认的 `iptables.rules` 文件。因此，第一次启动服务时使用以下命令：
或者
和其他服务一样，如果希望启动时自动加载 iptables，必须启用该服务：

从命令行

显示当前规则

使用以下命令查看当前规则和匹配数：
Chain INPUT (policy ACCEPT 0 packets, 0 bytes) pkts bytes target prot opt in out source destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes) pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0K packets, 0 bytes) pkts bytes target prot opt in out source destination

上面的结果表明还没有配置规则。没有数据包被阻止。

在输入中添加 `--line-numbers` 选项，可以在列出规则的时候显示行号。这在添加单独的规则的时候很有用。

重置规则

使用这些命令刷新和重置 iptables 到默认状态：
没有任何参数的 `-F` 命令在当前表中刷新所有链。同样的， `-X` 命令删除表中所有非默认链。

用下面的带 `[chain]` 参数的 `-F` 和 `-X` 命令刷新或删除单独的链。

编辑规则

有两种方式添加规则，一种是在链上附加规则，另一种是将规则插入到链上某个特定位置。这里将讲解这两种方式。

首先，由于电脑不是路由器(unless, of course, it [is a router](../网络共享/路由器.md))，因此将 `FORWARD` 链默认的规则由 `ACCEPT` 改成 `DROP`。
> **警告：**本章节其余部分主要讲解 iptables 规则的语法和其背后的思想，而不是作为一种保护服务器的手段，如果想要提高系统的安全性，参考 [Simple stateful firewall](Simple%20stateful%20firewall.md) 来获得一个最低限度的 iptables 安全配置，参考 [Security](../../08_安全与加密/安全/安全.md) 来提高 Arch Linux 的安全性。

[Dropbox](https://en.wikipedia.org/wiki/Dropbox_(service)) 的局域网同步特性—[每30秒广播数据包](https://isc.sans.edu/port.html?port=17500)\[[失效链接](https://zh.wikipedia.org/wiki/Wikipedia:%E5%A4%B1%E6%95%88%E9%93%BE%E6%8E%A5) 2022-09-18 ⓘ\]到所有可视的计算机，如果我们碰巧在一个拥有 Dropbox 客户端的局域网中，但是我们不想使用这个特性，那么我们希望拒绝这些数据包。
Chain INPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination 1 0 0 REJECT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:17500 reject-with icmp-port-unreachable

Chain FORWARD (policy DROP 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

> **注意：**这里使用 `REJECT` 而不是 `DROP`，因为 [RFC 1122 3.3.8](https://tools.ietf.org/html/rfc1122#page-69) 要求主机尽可能返回 ICMP 错误而不是丢弃数据包。[这里](https://www.chiark.greenend.org.uk/~peterb/network/drop-vs-reject) 介绍了为什么倾向于 REJECT 而不是 DROP 数据包。

现在，我们改变对 Dropbox 的看法，并且决定安装其到我们的计算机，我们也希望局域网同步，但是我们的网络只有一个特定的 IP 地址，因此需要使用 `-R` 参数来替换旧规则，`10.0.0.85` 是我们的另一个 IP 地址。

Chain INPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination 1 0 0 REJECT tcp -- !10.0.0.85 0.0.0.0/0 tcp dpt:17500 reject-with icmp-port-unreachable

Chain FORWARD (policy DROP 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

现在我们替换了原来的规则，使 `10.0.0.85` 可以访问计算机 `17500` 端口。但是我们意识到这条规则是不可升级的。如果友好的 Dropbox 用户想要访问设备上的 `17500` 端口。我们应该马上允许，而不是在测试过所有防火墙规则后再允许。

因此我们写一条新规则来立即允许受信任的用户。使用 `-I` 参数来在旧规则前插入一条新规则：

Chain INPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination 1 0 0 ACCEPT tcp -- 10.0.0.85 0.0.0.0/0 tcp dpt:17500 / *Friendly Dropbox* / 2 0 0 REJECT tcp -- !10.0.0.85 0.0.0.0/0 tcp dpt:17500 reject-with icmp-port-unreachable

Chain FORWARD (policy DROP 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

更改第二条规则，使其拒绝 `17500` 端口的任何数据包：
我们的最终规则如下：

Chain INPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination 1 0 0 ACCEPT tcp -- 10.0.0.85 0.0.0.0/0 tcp dpt:17500 / *Friendly Dropbox* / 2 0 0 REJECT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:17500 reject-with icmp-port-unreachable

Chain FORWARD (policy DROP 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes) num pkts bytes target prot opt in out source destination

配置文件

在 Arch Linux 中， Iptables 规则默认存放在 `/etc/iptables/iptables.rules` 文件中，它们不会被自动加载，因此需要启用 `iptables.service`，服务会在启动时读取文件并且加载规则，或者直接启动服务：
ipv6 规则默认保存在 `/etc/iptables/ip6tables.rules`,`ip6tables.service` 服务会使用这个规则，可以用类似的方式启动服务。

> **注意：**iptables 1.4.21-1 包中的 `iptables.service` 和 `ip6tables.service` 文件已经过期了。因为安全原因，自 systemd 214 起推荐防火墙在 `network-pre.target` 目标之前启动，这样防火墙就可以在任何网络配置之前运行。等待 iptables 数据包更新，创建深层目录 `/etc/systemd/system/iptables.service.d`，创建 `00-pre-network.conf` 文件，添加如下片段：

>
> ```
> [Unit]
> Before=network-pre.target
> [Install]
> RequiredBy=network-pre.target
> ```
>
> 如果系统使用 `ip6tables.service`，那么在 `/etc/systemd/system/ip6tables.service.d` 目录中做同样的配置。更多细节参考 [systemd bug report](https://bugs.freedesktop.org/show_bug.cgi?id=79600)，[systemd release announcement](https://lists.freedesktop.org/archives/systemd-devel/2014-June/019925.html) 和 [FS#33478](https://bugs.archlinux.org/task/33478)

通过命令行添加规则，配置文件不会自动改变，所以必须手动保存：
修改配置文件后，需要重新加载服务：
或者通过 iptables 直接加载：
指南

- [Simple stateful firewall](Simple%20stateful%20firewall.md)
- [Router](../网络共享/路由器.md)

日志

LOG 目标可以用来记录匹配某个规则的数据包。和 ACCEPT 或 DROP 规则不同，进入 LOG 目标之后数据包会继续沿着链向下走。所以要记录所有丢弃的数据包，只需要在 DROP 规则前加上相应的 LOG 规则。但是这样会比较复杂，影响效率，所以应该创建一个`logdrop`链。

创建 logdrop 链：
定义规则：
[下文](#限制日志级别)会给出 `limit` 和 `limit-burst` 选项的解释。

现在任何时候想要丢弃数据包并且记录该事件，只要跳转到 `logdrop` 链，例如：
限制日志级别

上述 `logdrop` 链使用限制（limit）模式来防止 *iptables* 日志来过大或者造成不必要的硬盘读写。没有限制的话，一个试图链接的错误配置服务、或者一个攻击者，都会使 iptables 日志写满整个硬盘（或者至少是 `/var` 分区）。

限制模式使用 `-m limit`，可以使用 `--limit` 来设置平均速率或者使用 `--limit-burst` 来设置起始触发速率。在上述 `logdrop` 例子中：
iptables -A logdrop -m limit --limit 5/m --limit-burst 10 -j LOG

```
appends a rule which will log all packets that pass through it. The first 10
consecutive packets will be logged, and from then on only 5 packets per minute will be
logged. The "limit burst" count is reset every time the "limit rate" is not broken,
i.e. logging activity returns to normal automatically.
```

添加一条记录所有通过其的数据包的规则。开始的连续10个数据包将会被记录，之后每分钟只会记录5个数据包。The "limit burst" count is reset every time the "limit rate" is not broken，例如，日志记录活动自动恢复到正常。

查看记录的数据包

记录的数据包作为内核信息，可以在 [systemd journal](../../02_系统管理与内核/日志/Systemd_Journal.md) 看到。

使用以下命令查看所有最近一次启动后所记录的数据包：
使用 syslog-ng

使用 Arch 默认的 syslog-ng 可以控制 iptables 日志的输出文件：
filter f_everything { level(debug..emerg) and not facility(auth, authpriv); };

修改为
filter f_everything { level(debug..emerg) and not facility(auth, authpriv) and not filter(f_iptables); };

iptables 的日志就不会输出到 `/var/log/everything.log`。

iptables 也可以不输出到 `/var/log/iptables.log`，只需设置`syslog-ng.conf` 中的 d_iptables 为需要的日志文件。
destination d_iptables { file("/var/log/iptables.log"); };

使用 ulogd

[ulogd](https://www.netfilter.org/projects/ulogd/index.html) 是专门用于 netfilter 的日志工具，可以代替默认的 LOG 目标。软件包 [ulogd](https://archlinux.org/packages/?name=ulogd)包 位于 `[community]` 源。

参见

[Wikipedia:iptables](https://en.wikipedia.org/wiki/iptables)

- [Simple stateful firewall](Simple%20stateful%20firewall.md)
- [官方网站](https://www.netfilter.org/projects/iptables/index.html)
- [Iptables 教程 1.2.2](https://www.frozentux.net/iptables-tutorial/iptables-tutorial.html)

2.8 Simple stateful firewall

难度 高级 | **篇幅** 30.2 KB | **所属** 防火墙

本节将讲到

- 前提条件
- Firewall for a single machine
- Setting up a NAT gateway
- See Also

这篇文件解释了如何使用 iptables 建立状态防火墙。也解释了规则的含义以及为何需要这些规则，为简单起见，文章分成两个主要章节，第一章节讲解在单独一台计算机上使用 iptables，第二章节在第一章节基础上建立一个 NAT 网关。

警告：规则是按照执行的顺序给与的，如果是远程登录计算机，那么在设置规则时可能会被计算机拒之门外，所以只能在本地登录计算机的时候做下面的设置。

example config file 可以用来解决这个问题。

前提条件

注意：Linux内核在编译时需要加入 iptables 支持，所有的常用 Arch Linux 内核都有 iptables 支持。

首先，安装用户层工具 iptables包 或者确保其已被安装。

本文章假设现在系统中没有设置 iptable 规则。运行如下命令检测当前规则集并且确保当前没有规则：

```
# iptables-save
# Generated by iptables-save v1.4.19.1 on Thu Aug  1 19:28:53 2013
*filter
:INPUT ACCEPT [50:3763]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [30:3472]
COMMIT
# Completed on Thu Aug  1 19:28:53 2013
```

或者

```
# iptables -nvL --line-numbers
Chain INPUT (policy ACCEPT 156 packets, 12541 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 82 packets, 8672 bytes)
num   pkts bytes target     prot opt in     out     source               destination
```

如果设置了规则，需要将规则重置，并且加载默认规则，运行：

```
# iptables-restore < /etc/iptables/empty.rules
```

否则，参阅 Iptables (简体中文)#重置规则。

Firewall for a single machine

注意：因为 iptables 按照线性顺序从上到下处理链的规则，所以建议按照规则执行的逻辑将频繁命中的规则放到靠近链开始的地方。规则有相关的运行成本，所以规则不能仅根据字节/数据包数量的经验观测来重新排列。

创建必要的链

对于基本的设置，我们将创建两条用户定义的链 — 用来打开防火墙的端口。

```
# iptables -N TCP
# iptables -N UDP
```

链可以有任意的名字，我们选择这两个名字只是为了匹配在以后规则中想要处理的协议，其经常使用协议参数指定，例如，使用 `-p tcp`。

```
# iptables -N TCP
# iptables -N UDP
```

FORWARD 链

如果想要设置计算机为NAT网关，请参阅 #Setting up a NAT gateway。然而对于单独计算机，只需要简单设置 **FORWARD** 链规则为 **DROP**，然后继续。

```
# iptables -P FORWARD DROP
```

The OUTPUT chain

我们不需要过滤任何输出流，这会使设置十分复杂，并且需要其他额外的思路，因为这个简单的愿意，我们设置 **OUTPUT** 链规则为 **ACCEPT**。

```
# iptables -P OUTPUT ACCEPT
```

The INPUT chain

和上述的链一样，设置 **INPUT** 链默认策略为 **DROP** 已避免某些东西莫名其妙的穿过规则。停止所有的数据交换，然后指定可以被允许的数据是建立一个安全的防火墙的最好方式。

警告：如果使用 SSH 登录，之后的操作将导致丢失 SSH 链接。通过下面的步骤避免出现这个问题：（1）将下面的语句添加到第一 INPUT 链规则中（将保持界面开启）。（2）

```
# iptables -P INPUT DROP
```

Every packet that is received by any network interface will pass the **INPUT** chain first, if it is destined for this machine. In this chain, we make sure that only the packets that we want are accepted.

The first rule added to the INPUT chain will allow traffic that belongs to established connections, or new valid traffic that is related to these connections such as ICMP errors, or echo replies (the packets a host returns when pinged). **ICMP** stands for **Internet Control Message Protocol**. Some ICMP messages are very important and help to manage congestion and MTU, and are accepted by this rule.

The connection state `ESTABLISHED` implies that either another rule previously allowed the initial (`--ctstate NEW`) connection attempt or the connection was already active (for example an active remote SSH connection) when setting the rule:

```
# iptables -A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

The second rule will accept all traffic from the "loopback" (lo) interface, which is necessary for many applications and services.

注意：You can add more trusted interfaces here such as "eth1" if you do not want/need the traffic filtered by the firewall, but be warned that if you have a NAT setup that redirects any kind of traffic to this interface from anywhere else in the network (let us say a router), it will get through, regardless of any other settings you may have.

```
# iptables -A INPUT -i lo -j ACCEPT
```

The third rule will drop all traffic with an "INVALID" state match. Traffic can fall into four "state" categories: NEW, ESTABLISHED, RELATED or INVALID and this is what makes this a "stateful" firewall rather than a less secure "stateless" one. States are tracked using the "nf_conntrack_*" kernel modules which are loaded automatically by the kernel as you add rules.

注意：

- This rule will drop all packets with invalid headers or checksums, invalid TCP flags, invalid ICMP messages (such as a port unreachable when we did not send anything to the host), and out of sequence packets which can be caused by sequence prediction or other similar attacks. The "DROP" target will drop a packet without any response, contrary to REJECT which politely refuses the packet. We use DROP because there is no proper "REJECT" response to packets that are INVALID, and we do not want to acknowledge that we received these packets.
- ICMPv6 Neighbor Discovery packets remain untracked, and will always be classified "INVALID" though they are not corrupted or the like. Keep this in mind, and accept them before this rule! iptables -A INPUT -p 41 -j ACCEPT

```
# iptables -A INPUT -m conntrack --ctstate INVALID -j DROP
```

The next rule will accept all new incoming **ICMP echo requests**, also known as pings. Only the first packet will count as NEW, the rest will be handled by the RELATED,ESTABLISHED rule. Since the computer is not a router, no other ICMP traffic with state NEW needs to be allowed.

```
# iptables -A INPUT -p icmp --icmp-type 8 -m conntrack --ctstate NEW -j ACCEPT
```

Now we attach the TCP and UDP chains to the INPUT chain to handle all new incoming connections. Once a connection is accepted by either TCP or UDP chain, it is handled by the RELATED/ESTABLISHED traffic rule. The TCP and UDP chains will either accept new incoming connections, or politely reject them. New TCP connections must be started with SYN packets.

注意： NEW but not SYN is the only invalid TCP flag not covered by the INVALID state. The reason is because they are rarely malicious packets, and they should not just be dropped. Instead, we simply do not accept them, so they are rejected with a TCP RST by the next rule.

```
# iptables -A INPUT -p udp -m conntrack --ctstate NEW -j UDP
# iptables -A INPUT -p tcp --syn -m conntrack --ctstate NEW -j TCP
```

We reject TCP connections with TCP RST packets and UDP streams with ICMP port unreachable messages if the ports are not opened. This imitates default Linux behavior (RFC compliant), and it allows the sender to quickly close the connection and clean up.

```
# iptables -A INPUT -p udp -j REJECT --reject-with icmp-port-unreachable
# iptables -A INPUT -p tcp -j REJECT --reject-with tcp-rst
```

For other protocols, we add a final rule to the INPUT chain to reject all remaining incoming traffic with icmp protocol unreachable messages. This imitates Linux's default behavior.

```
# iptables -A INPUT -j REJECT --reject-with icmp-proto-unreachable
```

Example iptables.rules file

Example of `iptables.rules` file after running all the commands from above:

```
/etc/iptables/iptables.rules
# Generated by iptables-save v1.4.18 on Sun Mar 17 14:21:12 2013
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:TCP - [0:0]
:UDP - [0:0]
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate INVALID -j DROP
-A INPUT -p icmp -m icmp --icmp-type 8 -m conntrack --ctstate NEW -j ACCEPT
-A INPUT -p udp -m conntrack --ctstate NEW -j UDP
-A INPUT -p tcp --tcp-flags FIN,SYN,RST,ACK SYN -m conntrack --ctstate NEW -j TCP
-A INPUT -p udp -j REJECT --reject-with icmp-port-unreachable
-A INPUT -p tcp -j REJECT --reject-with tcp-reset
-A INPUT -j REJECT --reject-with icmp-proto-unreachable
COMMIT
# Completed on Sun Mar 17 14:21:12 2013
```

This file can be generated with:

```
# iptables-save > /etc/iptables/iptables.rules
```

and can be used to continue with the following sections. If you are setting up the firewall remotely via SSH, append the following rule to allow new SSH connections before continuing (adjust port as required):

```
-A TCP -p tcp --dport 22 -j ACCEPT
```

The TCP and UDP chains

The TCP and UDP chains contain rules for accepting new incoming TCP connections and UDP streams to specific ports.

注意：This is where you need to add rules to accept incoming connections, such as SSH, HTTP or other services that you want to access remotely.

Opening ports to incoming connections

To accept incoming TCP connections on port 80 for a web server:

```
# iptables -A TCP -p tcp --dport 80 -j ACCEPT
```

To accept incoming TCP connections on port 443 for a web server (HTTPS):

```
# iptables -A TCP -p tcp --dport 443 -j ACCEPT
```

To allow remote SSH connections (on port 22):

```
# iptables -A TCP -p tcp --dport 22 -j ACCEPT
```

To accept incoming UDP streams on port 53 for a DNS server:

```
# iptables -A UDP -p udp --dport 53 -j ACCEPT
```

See iptables(8) for more advanced rules, like matching multiple ports.

Port knocking

Port knocking is a method to externally open ports that, by default, the firewall keeps closed. It works by requiring connection attempts to a series of predefined closed ports. When the correct sequence of port "knocks" (connection attempts) is received, the firewall opens certain port(s) to allow a connection. See Port knocking for more information.

Protection against spoofing attacks

注意：`rp_filter` is currently set to `1` by default in `/usr/lib/sysctl.d/50-default.conf`, so the following step is not necessary.

Blocking reserved local addresses incoming from the internet or local network is normally done through setting `rp_filter` (Reverse Path Filter) in sysctl to 1. To do so, add the following line to your `/etc/sysctl.d/90-firewall.conf` file (see sysctl for details) to enable source address verification which is built into Linux kernel itself. The verification by the kernel will handle spoofing better than individual iptables rules for each case.

```
net.ipv4.conf.all.rp_filter=1
```

This can be done with netfilter instead if statistics (and better logging) are desired:

```
# iptables -t raw -I PREROUTING -m rpfilter --invert -j DROP
```

注意：There is no reason to enable this in both places. The netfilter method is the modern choice and works with IPv6 too.

For niche setups where asynchronous routing is used, the `rp_filter=2` sysctl option needs to be used instead. Passing the `--loose` switch to the `rpfilter` module will accomplish the same thing with netfilter.

"Hide" your computer

If you are running a desktop machine, it might be a good idea to block some incoming requests.

Block ping request

A 'Ping' request is an ICMP packet sent to the destination address to ensure connectivity between the devices. If your network works well, you can safely block all ping requests. It is important to note that this *does not* actually hide your computer — any packet sent to you is rejected, so you will still show up in a simple nmap "ping scan" of an IP range.

This is rudimentary "protection" and makes life difficult when debugging issues in the future. You should only do this for education purposes.

To block echo requests, add the following line to your `/etc/sysctl.d/90-firewall.conf` file (see sysctl for details):

```
net.ipv4.icmp_echo_ignore_all = 1
```

More information is in the iptables man page, or reading the docs and examples on the webpage http://www.snowman.net/projects/ipt_recent/

Tricking port scanners

注意：This opens you up to a form of DoS. An attack can send packets with spoofed IPs and get them blocked from connecting to your services.

Port scans are used by attackers to identify open ports on your computer. This allows them to identify and fingerprint your running services and possibly launch exploits against them.

The INVALID state rule will take care of every type of port scan except UDP, ACK and SYN scans (-sU, -sA and -sS in nmap respectively).

ACK scans are not used to identify open ports, but to identify ports filtered by a firewall. Due to the SYN check for all TCP connections with the state NEW, every single packet sent by an ACK scan will be correctly rejected by a TCP RST packet. Some firewalls drop these packets instead, and this allows an attacker to map out the firewall rules.

The recent module can be used to trick the remaining two types of port scans. The recent module is used to add hosts to a "recent" list which can be used to fingerprint and stop certain types of attacks. Current recent lists can be viewed in `/proc/net/xt_recent/`.

SYN scans

In a SYN scan, the port scanner sends SYN packet to every port. Closed ports return a TCP RST packet, or get dropped by a strict firewall. Open ports return a SYN ACK packet regardless of the presence of a firewall.

The recent module can be used to keep track of hosts with rejected connection attempts and return a TCP RST for any SYN packet they send to open ports as if the port was closed. If an open port is the first to be scanned, a SYN ACK will still be returned, so running applications such as ssh on non-standard ports is required for this to work consistently.

First, insert a rule at the top of the TCP chain. This rule responds with a TCP RST to any host that got onto the TCP-PORTSCAN list in the past sixty seconds. The `--update` switch causes the recent list to be updated, meaning the 60 second counter is reset.

```
# iptables -I TCP -p tcp -m recent --update --seconds 60 --name TCP-PORTSCAN -j REJECT --reject-with tcp-rst
```

Next, the rule for rejecting TCP packets need to be modified to add hosts with rejected packets to the TCP-PORTSCAN list.

```
# iptables -D INPUT -p tcp -j REJECT --reject-with tcp-rst
# iptables -A INPUT -p tcp -m recent --set --name TCP-PORTSCAN -j REJECT --reject-with tcp-rst
```

UDP scans

UDP port scans are similar to TCP SYN scans except that UDP is a "connectionless" protocol. There are no handshakes or acknowledgements. Instead, the scanner sends UDP packets to each UDP port. Closed ports should return ICMP port unreachable messages, and open ports do not return a response. Since UDP is not a "reliable" protocol, the scanner has no way of knowing if packets were lost, and has to do multiple checks for each port that does not return a response.

The Linux kernel sends out ICMP port unreachable messages very slowly, so a full UDP scan against a Linux machine would take over 10 hours. However, common ports could still be identified, so applying the same countermeasures against UDP scans as SYN scans is a good idea.

First, add a rule to reject packets from hosts on the UDP-PORTSCAN list to the top of the UDP chain.

```
# iptables -I UDP -p udp -m recent --update --seconds 60 --name UDP-PORTSCAN -j REJECT --reject-with icmp-port-unreachable
```

Next, modify the reject packets rule for UDP:

```
# iptables -D INPUT -p udp -j REJECT --reject-with icmp-port-unreachable
# iptables -A INPUT -p udp -m recent --set --name UDP-PORTSCAN -j REJECT --reject-with icmp-port-unreachable
```

Restore the Final Rule

If either or both of the portscanning tricks above were used the final default rule is no longer the last rule in the INPUT chain. It needs to be the last rule otherwise it will intercept the trick port scanner rules you just added and they will never be used.

Simply delete the rule (-D), then add it once again using append (-A) which will place it at the end of the chain.

```
# iptables -D INPUT -j REJECT --reject-with icmp-proto-unreachable
# iptables -A INPUT -j REJECT --reject-with icmp-proto-unreachable
```

Protection against other attacks

See the sysctl#TCP/IP stack hardening for relevant kernel parameters.

Bruteforce attacks

Unfortunately, bruteforce attacks on services accessible via an external IP address are common. One reason for this is that the attacks are easy to do with the many tools available. Fortunately, there are a number of ways to protect the services against them. One is the use of appropriate `iptables` rules which activate and blacklist an IP after a set number of packets attempt to initiate a connection. Another is the use of specialised daemons that monitor the logfiles for failed attempts and blacklist accordingly.

警告： Using an IP blacklist will stop trivial attacks but it relies on an additional daemon and successful logging (the partition containing /var can become full, especially if an attacker is pounding on the server). Additionally, if the attacker knows your IP address, they can send packets with a spoofed source header and get you locked out of the server. SSH keys provide an elegant solution to the problem of brute forcing without these problems.

Two packages that ban IPs after too many password failures are Fail2ban or, for `sshd` in particular, Sshguard. These two applications update iptables rules to reject future connections from blacklisted IP addresses.

The following rules give an example configuration to mitigate SSH bruteforce attacks using `iptables`.

```
# iptables -N IN_SSH
# iptables -A INPUT -p tcp --dport ssh -m conntrack --ctstate NEW -j IN_SSH
# iptables -A IN_SSH -m recent --name sshbf --rttl --rcheck --hitcount 3 --seconds 10 -
j DROP
# iptables -A IN_SSH -m recent --name sshbf --rttl --rcheck --hitcount 4 --seconds 1800
-j DROP
# iptables -A IN_SSH -m recent --name sshbf --set -j ACCEPT
```

Most of the options should be self-explanatory, they allow for three connection packets in ten seconds. Further tries in that time will blacklist the IP. The next rule adds a quirk by allowing a total of four attempts in 30 minutes. This is done because some bruteforce attacks are actually performed slow and not in a burst of attempts. The rules employ a number of additional options. To read more about them, check the original reference for this example: compilefailure.blogspot.com

Using the above rules, now ensure that:

```
# iptables -A INPUT -p tcp --dport ssh -m conntrack --ctstate NEW -j IN_SSH
```

is in an appropriate position in the iptables.rules file.

This arrangement works for the IN_SSH rule if you followed this entire wiki so far:

```
*
-A INPUT -p icmp -m icmp --icmp-type 8 -m conntrack --ctstate NEW -j ACCEPT
```

```
-A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW -j IN_SSH
-A INPUT -p udp -m conntrack --ctstate NEW -j UDP
*
```

The above rules can, of course, be used to protect any service, though protecting the SSH daemon is probably the most often required one.

提示：For self-testing the rules after setup, the actual blacklist happening can slow the test making it difficult to fine-tune parameters. One can watch the incoming attempts via `cat /proc/net/xt_recent/sshbf`. To unblock the own IP during testing, root is needed `# echo / > /proc/net/xt_recent/sshbf`

Saving the rules

The ruleset is now finished and should be saved to your hard drive so that it can be loaded on every boot.

The systemd unit file points to the location where the rule configuration will be saved:

```
iptables=/etc/iptables/iptables.rules
ip6tables=/etc/iptables/ip6tables.rules
```

Save the rules with this command:

```
# iptables-save > /etc/iptables/iptables.rules
```

and make sure your rules are loaded on boot enabling the **iptables** daemon.

Check that the rules load correctly by starting `iptables.service` and then checking the status of the service.

IPv6

If you do not use IPv6 (most ISPs do not support it), you should disable it.

Otherwise, you should enable the firewall rules for IPv6. After copying the IPv4 rules as a base:

```
# cp /etc/iptables/iptables.rules /etc/iptables/ip6tables.rules
```

the first step is to change IPs referenced in the rules from IPv4 format to IPv6 format.

Next, a few of the rules (built as example in this article for IPv4) have to be adapted. IPv6 obtained a new ICMPv6 protocol, replacing ICMP. Hence, the reject error return codes `--reject-with icmp-port-unreachable` and `--reject-with icmp-proto-unreachable` have to be converted to ICMPv6 codes.

The available ICMPv6 error codes are listed in RFC 4443, which specifies connection attempts blocked by a firewall rule should use `--reject-with icmp6-adm-prohibited`. Doing so will basically inform the remote system that the connection was rejected by a firewall, rather than a listening service.

If it is preferred not to explicitly inform about the existence of a firewall filter, the packet may also be rejected without the message:

```
 -A INPUT -j REJECT
```

The above will reject with the default return error of `--reject-with icmp6-port-unreachable`. You should note though, that identifying a firewall is a basic feature of port scanning applications and most will identify it regardless.

In the next step make sure the protocol and extension are changed to be IPv6 appropriate for the rule regarding all new incoming ICMP echo requests (pings):

```
# ip6tables -A INPUT -p icmpv6 --icmpv6-type 128 -m conntrack --ctstate NEW -j ACCEPT
```

Netfilter conntrack does not appear to track ICMPv6 Neighbor Discovery Protocol (the IPv6 equivalent of ARP), so we need to allow ICMPv6 traffic regardless of state for all directly attached subnets. The following should be inserted after dropping `--ctstate INVALID`, but before any other DROP or REJECT targets, along with a corresponding line for each directly attached subnet:

```
# ip6tables -A INPUT -s fe80::/10 -p icmpv6 -j ACCEPT
```

Since there is no kernel reverse path filter for IPv6, you may want to enable one in *ip6tables* with the following:

```
# ip6tables -t raw -A PREROUTING -m rpfilter -j ACCEPT
# ip6tables -t raw -A PREROUTING -j DROP
```

After the configuration is done, enable the **ip6tables** service, it is meant to run in parallel to *iptables*.

Setting up a NAT gateway

This section of the guide deals with NAT gateways. It is assumed that you already read the first part of the guide and set up the **INPUT**, **OUTPUT**, **TCP** and **UDP** chains like described above. All rules so far have been created in the **filter** table. In this section, we will also have to use the **nat** table.

Setting up the filter table

Creating necessary chains

In our setup, we will use another two chains in the filter table, the **fw-interfaces** and **fw-open** chains. Create them with the commands

```
# iptables -N fw-interfaces
# iptables -N fw-open
```

Setting up the FORWARD chain

Setting up the **FORWARD** chain is similar to the **INPUT** chain in the first section.

Now we set up a rule with the **conntrack** match, identical to the one in the **INPUT** chain:

```
# iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

The next step is to enable forwarding for trusted interfaces and to make all packets pass the **fw-open** chain.

```
# iptables -A FORWARD -j fw-interfaces
# iptables -A FORWARD -j fw-open
```

The remaining packets are denied with an **ICMP** message:

```
# iptables -A FORWARD -j REJECT --reject-with icmp-host-unreachable
# iptables -P FORWARD DROP
```

Setting up the fw-interfaces and fw-open chains

The meaning of the **fw-interfaces** and **fw-open** chains is explained later, when we deal with the **POSTROUTING** and **PREROUTING** chains in the **nat** table, respectively.

Setting up the nat table

All over this section, we assume that the outgoing interface (the one with the public internet IP) is **ppp0**. Keep in mind that you have to change the name in all following rules if your outgoing interface has another name.

Setting up the POSTROUTING chain

Now, we have to define who is allowed to connect to the internet. Let us assume we have the subnet **192.168.0.0/24** (which means all addresses that are of the form 192.168.0.*) on **eth0**. We first need to accept the machines on this interface in the FORWARD table, that is why we created the **fw-interfaces** chain above:

```
# iptables -A fw-interfaces -i eth0 -j ACCEPT
```

Now, we have to alter all outgoing packets so that they have our public IP address as the source address, instead of the local LAN address. To do this, we use the **MASQUERADE** target:

```
# iptables -t nat -A POSTROUTING -s 192.168.0.0/24 -o ppp0 -j MASQUERADE
```

Do not forget the **\-o ppp0** parameter above. If you omit it, your network will be screwed up.

Let us assume we have another subnet, **10.3.0.0/16** (which means all addresses 10.3.\.\), on the interface **eth1**. We add the same rules as above again:

```
# iptables -A fw-interfaces -i eth1 -j ACCEPT
# iptables -t nat -A POSTROUTING -s 10.3.0.0/16 -o ppp0 -j MASQUERADE
```

The last step is to enable IP Forwarding (if it is not already enabled):

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

Then edit the relevant line in `/etc/sysctl.d/90-firewall.conf` so it persists through reboot (see sysctl for details):

```
net.ipv4.ip_forward = 1
```

Machines from these subnets can now use your new NAT machine as their gateway. Note that you may want to set up a DNS and DHCP server like **dnsmasq** or a combination of **bind** and **dhcpd** to simplify network settings DNS resolution on the client machines. This is not the topic of this guide.

Setting up the PREROUTING chain

Sometimes, we want to change the address of an incoming packet from the gateway to a LAN machine. To do this, we use the **fw-open** chain defined above, as well as the **PREROUTING** chain in the **nat** table in the following two simple examples.

First, we want to change all incoming SSH packets (port 22) to the ssh server of the machine **192.168.0.5**:

```
# iptables -t nat -A PREROUTING -i ppp0 -p tcp --dport 22 -j DNAT --to 192.168.0.5
# iptables -A fw-open -d 192.168.0.5 -p tcp --dport 22 -j ACCEPT
```

The second example will show you how to change packets to a different port than the incoming port. We want to change any incoming connection on port **8000** to our web server on **192.168.0.6**, port **80**:

```
# iptables -t nat -A PREROUTING -i ppp0 -p tcp --dport 8000 -j DNAT --to 192.168.0.6:80
# iptables -A fw-open -d 192.168.0.6 -p tcp --dport 80 -j ACCEPT
```

The same setup also works with udp packets.

Saving the rules

Save the rules:

```
# iptables-save > /etc/iptables/iptables.rules
```

and make sure your rules are loaded when you boot enabling the **iptables** daemon.

See Also

- Internet sharing
- Router
- Firewalls
- Uncomplicated Firewall
- Methods to block SSH attacks
- Using iptables to block brute force attacks
- 20 Iptables Examples For New SysAdmins\[失效链接 2021-11-19 ⓘ\]
- 25 Most Frequently Used Linux IPTables Rules Examples

2.9 Nftables

难度 高级 | **篇幅** 31.3 KB | **所属** 防火墙

本节将讲到

- 安装
- 用法
- 配置
- 示例
- 提示和技巧

翻译状态：

- 本文（或部分内容）译自 nftables，最近一次同步于 2023-2-24，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

nftables 是一个网络过滤器项目，旨在替换现有的 {ip,ip6,arp,eb}tables 框架。它提供了一个新的数据包过滤框架、一个新的用户空间实用程序（nft）和一个针对 {ip,ip6} tables 的兼容层。它使用了现有的钩子、连接跟踪系统、用户空间队列组件和网络过滤器的日志子系统。

它由三个主要组件组成：内核实现、libnl 网络链接通信和 nftables 用户空间前端。 内核提供了一个网络链接配置接口，以及运行时规则集计算，libnl 包含用于与内核通信的底层函数，nftables 前端供用户通过 nft 进行交互。

你也可以访问官方 nftables wiki 页获取更多信息。

安装

安装用户空间实用程序包 nftables包 或 git 版本 nftables-gitAUR。

或者，安装 iptables-nft包，它将 nftables包 作为一个依赖项，将自动卸载 iptables包（base包元包的间接依赖项），并在一起使用时防止 iptables包 和 nftables包 之间的冲突。

提示：大多数 iptables 前端都不直接或间接支持 nftables，但可能会引入它。 一个同时支持 nftables 和 iptables 的图形前端是 firewalld。

前端

- **firewalld (firewall-cmd)** — 用于配置网络和防火墙域以及建立和配置防火墙规则的守护进程和控制台界面。

: https://firewalld.org/ || firewalld包

- **nft-blackhole** — 脚本/守护进程可以按国家和黑名单阻止 nftables 中的 IP。

: https://github.com/tomasz-c/nft-blackhole || nft-blackholeAUR

- **ufw** — Ufw（即 Uncomplicated Firewall 的缩写）是一个管理网络防火墙的程序。它提供了一个命令行界面，旨在简单（uncomplicated）易用。

: https://launchpad.net/ufw || ufw包

用法

提示：如果你已经拥有 iptables 规则，那么可以将 iptables 规则转换为 nftables 规则，参见。

nftables 不区分命令行中制作的临时规则和从文件加载或保存到文件中的永久规则。

所有规则都必须使用 `nft` 命令行工具创建或加载。

请参考 #配置章节了解如何使用。

可以打印当前规则集：

```
# nft list ruleset
```

删除所有规则集，使系统没有防火墙：

```
# nft flush ruleset
```

通过重启 `nftables.service` 从 `/etc/nftables.conf` 读取规则集。

简单的防火墙

nftables包 附带了一个简单而安全的防火墙配置，存储在 `/etc/nftables.conf` 文件中。

`nftables.service` 将在启动或启用时从该文件加载规则。

配置

nftables 的用户空间实用程序 `nft` 在将规则集传递给内核之前执行大多数规则集评估。规则存储在链中，而链又存储在表中。以下部分说明如何创建和修改这些构件。

要从文件读取输入，请使用 `-f`/`--file` 选项：

```
# nft --file 文件名
```

请注意，任何已加载的规则都不会自动刷新。

有关所有命令的完整列表，请参见 nft(8)。

表

表包含#链。与 iptables 中的表不同，nftables 中没有内置表。表的数量及其名称由用户决定。不过，每个表只有一个地址族，并且仅适用于该族的数据包。表可以指定五个族之一：

nftables 族	iptables 实用程序
ip	iptables
ip6	ip6tables
inet	iptables 和 ip6tables
arp	arptables
bridge	ebtables

`ip`（即 IPv4）是默认族，如果未指定族，则将使用该族。

要创建同时适用于 IPv4 和 IPv6 的规则，请使用 `inet`。`inet` 允许统一 `ip` 和 `ip6` 族，从而使同时定义两者的规则更加容易。

参见 nft(8) § ADDRESS FAMILIES 获得关于地址族的完整描述。

在以下所有示例中，`*族类型*` 是可选的，如果未指定，则设置为 `ip`。

创建表

以下示例添加一个新表：

```
# nft add table 族类型 表名
```

列出表

要列出所有表：

```
# nft list tables
```

列出表中的链和规则

要列出指定表的所有链和规则，请执行：

```
# nft list table 族类型 表名
```

例如，要列出 `inet` 族的 `my_table` 表的所有规则：

```
# nft list table inet my_table
```

删除表

要删除一个表，请执行：

```
# nft delete table 族类型 表名
```

这会销毁表中的所有链。

刷新表

要刷新来自表的所有规则，请执行：

```
# nft flush table 族类型 表名
```

链

链的用途是包含#规则。与 iptables 中的链不同，nftables 中没有内置链。这意味着，如果没有链在网络过滤器框架中使用任何类型或钩子，则与 iptables 不同，nftables 将不会触及流经这些链的数据包。

链有两种类型。*基本*链是来自网络堆栈的数据包的入口点，其中指定了一个钩子值。为了更好地组织，可以使用*常规*链作为跳转目标。

在以下所有示例中 *族类型* 都是可选的，如果未指定则设为 `ip`。

创建链

基本链

要添加基本链，请指定钩子和优先级值：

```
# nft add chain 族类型 表名 链名 '{ type 链类型 hook 钩子类型 priority 优先级值 ; }'
```

链类型 可以是 `filter`, `route`, 或 `nat`。

对于 IPv4/IPv6/Inet 地址族，*钩子类型* 可以是 `prerouting`, `input`, `forward`, `output`, 或 `postrouting`。见 nft(8) § ADDRESS FAMILIES 获取其他族的钩子列表。

优先级值 采用优先级名称或整数值。见 nft(8) § CHAINS 获取标准优先级名称和值的列表。数字较低的链首先被处理，并且可以是负数。

例如，要添加过滤输入数据包的基本链：

```
# nft add chain inet my_table my_chain '{ type filter hook input priority 0; }'
```

在上述任意情况中用 `create` 替换 `add` 以添加新链，但如果该链已经存在，则返回错误。

常规链

下面将常规链 *链名* 添加到表 *表名* 中：

```
# nft add chain 族类型 表名 链名
```

例如，要将名为 `my_tcp_chain` 的常规链添加到 `inet` 地址族的 `my_table` 表，请执行：

```
# nft add chain inet my_table my_tcp_chain
```

列出规则

下面列出链的所有规则：

```
# nft list chain 族类型 表名 链名
```

例如，下面列出名为 `my_table` 的 `inet` 表中名为 `my_output` 的链的规则：

```
# nft list chain inet my_table my_output
```

编辑链

要编辑链，只需按其名称调用它并定义要更改的规则。

```
# nft chain 族类型 表名 链名 '{ [ type 链类型 hook 钩子类型 device 设备名 priority 优先级值 ; policy 策略类型 ; ] }'
```

例如，要将 `my_table` 表的 `my_input` 链策略从 `accept` 更改为 `drop`

```
# nft chain inet my_table my_input '{ policy drop ; }'
```

删除链

要删除一个链，请执行：

```
# nft delete chain 族类型 表名 链名
```

链不能包含任何规则或是跳转目标。

刷新来自链的规则

要刷新来自链的规则，请执行：

```
# nft flush chain 族类型 表名 链名
```

规则

规则由表达式或语句构成，并包含在链中。

添加规则

要向链添加规则，请执行：

```
# nft add rule 族类型 表名 链名 handle 句柄值 语句
```

规则附加在 *句柄值* 处，这是可选的。如果未指定，规则将附加到链的末端。

应当用可以添加到任何有效列表命令的 `--handle` 开关确定规则句柄。此开关告诉 `nft` 在其输出中列出句柄。 `--numeric` 参数用于查看某些数值输出，如未解析的 IP 地址。

```
# nft --handle --numeric list chain inet my_table my_input
table inet my_table {
     chain input {
          type filter hook input priority 0;
          ip saddr 127.0.0.1 accept # handle 10
     }
}
```

要将规则添加到指定位置，请执行：

```
# nft insert rule 族类型 表名 链名 handle 句柄值 语句
```

如果未指定 *句柄值*，则规则将附加到链的开头。

表达式

通常，一个 *语句* 包括一些要匹配的表达式，然后是一个判断语句。判断语句包括 `accept`, `drop`, `queue`, `continue`, `return`, `jump` *链名*, 和 `goto` *链名*。存在判断语句以外的其他语句。有关详细信息，见 nft(8)。

nftables 中有各种可用的表达式，并且在很大程度上与 iptables 对应的表达式一致。最明显的区别是没有通用或隐式匹配。通用匹配总是可用的，例如 `--protocol` 或 `--source`。隐式匹配是特定于协议的，例如，当数据包被确定为 TCP 时，即为 `--sport`。

以下是可用匹配项的不完整列表：

- meta (元属性，例如 interfaces)
- icmp (ICMP 协议)
- icmpv6 (ICMPv6 协议)
- ip (IP 协议)
- ip6 (IPv6 协议)
- tcp (TCP 协议)
- udp (UDP 协议)
- sctp (SCTP 协议)
- ct (连接跟踪)

以下是匹配参数的不完整列表（有关更完整的列表，见 nft(8)）：

```
meta:
  oif <output interface INDEX>
  iif <input interface INDEX>
  oifname <output interface NAME>
  iifname <input interface NAME>

  （oif 和 iif 接受字符串参数并转换为接口索引）
  （oifname 和 iifname 更动态，但由于字符串匹配，速度较慢）

icmp:
  type <icmp type>

icmpv6:
  type <icmpv6 type>

ip:
  protocol <protocol>
  daddr <destination address>
  saddr <source address>

ip6:
  daddr <destination address>
  saddr <source address>

tcp:
  dport <destination port>
  sport <source port>

udp:
  dport <destination port>
  sport <source port>

sctp:
  dport <destination port>
  sport <source port>

ct:
  state <new | established | related | invalid>
```

删除

单个规则只能通过其句柄删除。获取句柄在#添加规则中展示。假设

```
# nft --handle --numeric list chain inet my_table my_input
table inet my_table {
     chain input {
          type filter hook input priority 0;
          ip saddr 127.0.0.1 accept # handle 10
     }
}
# nft delete rule inet my_table my_input handle 10
```

删去它。

可以使用 `nft flush table` 命令刷新表中的所有链。使用 `nft flush chain` 或 `nft delete rule` 命令可以刷新单个链。

```
# nft flush table 表名
# nft flush chain 族类型 表名 链名
# nft delete rule 族类型 表名 链名
```

第一个命令刷新 ip *表名* 表中的所有链。第二个刷新 *族类型* *表名* 表中的 *链名* 链。第三个删除 *族类型* *表名* 表中 *链名* 链中的所有规则。

集

集是命名的或匿名的，由一个或多个元素组成，用逗号分隔，用大括号括起来。匿名集嵌入到规则中，无法更新，必须删除并重新添加规则。例如，下面您不能从 dports 的匿名集中只删除 “http”：

```
# nft add rule ip6 filter input tcp dport {telnet, http, https} accept
```

命名集可以被更新，并且可以被类型化和标记。*sshguard* 为被阻止主机的 IP 地址使用命名集。

```
table ip sshguard {
       set attackers {
              type ipv4_addr
              flags interval
              elements = { 1.2.3.4 }
       }
```

要从该集*添加（add）*或*删除（delete）*元素（element），使用：

```
# nft add element ip sshguard attackers { 5.6.7.8/32 }
# nft delete element ip sshguard attackers { 1.2.3.4/32 }
```

注意，*ipv4_addr* 类型可以包含 CIDR 网络掩码（此处的 “/32” 不是必需的，但为了完整起见，包含了它）。此外请注意，这里由 “TABLE ip sshguard { SET attackers }” 定义的集被称为 `ip sshguard attackers`。

原子重载

刷新当前规则集：

```
# echo "flush ruleset" > /tmp/nftables
```

转储当前规则集：

```
# nft -s list ruleset >> /tmp/nftables
```

现在，您可以编辑 `/tmp/nftables` 并通过以下方式应用更改：

```
# nft -f /tmp/nftables
```

示例

工作站

```
/etc/nftables.conf
flush ruleset

table inet my_table {
       set LANv4 {
              type ipv4_addr
              flags interval

              elements = { 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16 }
       }
       set LANv6 {
              type ipv6_addr
              flags interval
```

```
                elements = { fd00::/8, fe80::/10 }
        }

        chain my_input_lan {
                udp sport 1900 udp dport >= 1024 meta pkttype unicast limit rate 4/second
burst 20 packets accept comment "接受 UPnP IGD 端口映射回复"

                udp sport netbios-ns udp dport >= 1024 meta pkttype unicast accept
comment "接受 Samba Workgroup 浏览回复"

        }

        chain my_input {
                type filter hook input priority filter; policy drop;

                iif lo accept comment "接受所有本机流量"
                ct state invalid drop comment "丢弃无效连接"
                ct state established,related accept comment "接受来自我们（us）的流量"

                meta l4proto ipv6-icmp accept comment "接受 ICMPv6"
                meta l4proto icmp accept comment "接受 ICMP"
                ip protocol igmp accept comment "接受 IGMP"

                udp dport mdns ip6 daddr ff02::fb accept comment "接受 mDNS"
                udp dport mdns ip daddr 224.0.0.251 accept comment "接受 mDNS"

                ip6 saddr @LANv6 jump my_input_lan comment "来自专用 IP 地址范围的连接"
                ip saddr @LANv4 jump my_input_lan comment "来自专用 IP 地址范围的连接"

                counter comment "统计任何其他流量"
        }

        chain my_forward {
                type filter hook forward priority filter; policy drop;
                # 丢弃转发给我们的所有内容。我们不转发。这是路由器的工作。
        }

        chain my_output {
                type filter hook output priority filter; policy accept;
                # 接受每个出站连接
        }

}
```

服务器

```
/etc/nftables.conf
flush ruleset

table inet my_table {
        set LANv4 {
                type ipv4_addr
                flags interval

                elements = { 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16 }
        }
        set LANv6 {
                type ipv6_addr
                flags interval

                elements = { fd00::/8, fe80::/10 }
        }
```

```
	chain my_input_lan {
		meta l4proto { tcp, udp } th dport 2049 accept comment "接受 NFS"

		udp dport netbios-ns accept comment "接受 NetBIOS 名称服务 (nmbd)"
		udp dport netbios-dgm accept comment "接受 NetBIOS 数据报服务 (nmbd)"
		tcp dport netbios-ssn accept comment "接受 NetBIOS 会话服务 (smbd)"
		tcp dport microsoft-ds accept comment "接受 Microsoft 目录服务 (smbd)"

		udp sport { bootpc, 4011 } udp dport { bootps, 4011 } accept comment "接受
PXE"
		udp dport tftp accept comment "接受 TFTP"
	}

	chain my_input {
		type filter hook input priority filter; policy drop;

		iif lo accept comment "接受所有本机流量"
		ct state invalid drop comment "丢弃无效链接"
		ct state established,related accept comment "接受来自我们（us）的流量"

		meta l4proto ipv6-icmp accept comment "接受 ICMPv6"
		meta l4proto icmp accept comment "接受 ICMP"
		ip protocol igmp accept comment "接受 IGMP"

		udp dport mdns ip6 daddr ff02::fb accept comment "接受 mDNS"
		udp dport mdns ip daddr 224.0.0.251 accept comment "接受 mDNS"

		ip6 saddr @LANv6 jump my_input_lan comment "来自专用 IP 地址范围的连接"
		ip saddr @LANv4 jump my_input_lan comment "来自专用 IP 地址范围的连接"

		tcp dport ssh accept comment "在 端口 22 上接受 SSH"

		tcp dport ipp accept comment "在端口 631 上接受 IPP/IPPS"

		tcp dport { http, https, 8008, 8080 } accept comment "接受 HTTP (端口 80,
443, 8008, 8080)"

		udp sport bootpc udp dport bootps ip saddr 0.0.0.0 ip daddr
255.255.255.255 accept comment "接受 DHCPDISCOVER (对于 DHCP-Proxy)"
	}

	chain my_forward {
		type filter hook forward priority filter; policy drop;
		# 丢弃转发给我们的所有内容。我们不转发。这是路由器的工作。
	}

	chain my_output {
		type filter hook output priority filter; policy accept;
		# 接受每个出站连接
	}

}
```

限速

```
table inet my_table {
	chain my_input {
		type filter hook input priority filter; policy drop;

		iif lo accept comment "接受所有本机流量"
		ct state invalid drop comment "丢弃无效链接"
```

```
            meta l4proto icmp icmp type echo-request limit rate over 10/second burst 4 packets drop comment "无 ping 洪流"
            meta l4proto ipv6-icmp icmpv6 type echo-request limit rate over 10/second burst 4 packets drop comment "无 ping 洪流"

            ct state established,related accept comment "接受来自我们（us）的流量"

            meta l4proto ipv6-icmp icmpv6 type { destination-unreachable, packet-too-big, time-exceeded, echo-reply, parameter-problem, mld-listener-query, mld-listener-report, mld-listener-reduction, nd-router-solicit, nd-router-advert, nd-neighbor-solicit, nd-neighbor-advert, ind-neighbor-solicit, ind-neighbor-advert, mld2-listener-report } accept comment "接受 ICMPv6"
            meta l4proto icmp icmp type { destination-unreachable, router-solicitation, router-advertisement, time-exceeded, parameter-problem } accept comment "接受 ICMP"
            ip protocol igmp accept comment "接受 IGMP"

            tcp dport ssh ct state new limit rate 15/minute accept comment "避免在 SSH 上使用暴力破解"

        }

}
```

跳转（Jump）

在配置文件中使用跳转时，必须首先定义目标链。否则可能会出现 `Error: Could not process rule: No such file or directory`。

```
table inet my_table {
    chain web {
        tcp dport http accept
        tcp dport 8080 accept
    }
    chain my_input {
        type filter hook input priority filter;
        ip saddr 10.0.2.0/24 jump web
        drop
    }
}
```

不同接口的不同规则

如果您的机箱有多个网络接口，并且您希望对不同的接口使用不同的规则，那么您可能需要使用 “分派（dispatching）” 过滤器链，然后使用特定于接口的过滤器链。例如，假设您的机箱充当家庭路由器，您希望运行可通过 LAN 访问的网络服务器（接口 `enp3s0`），但不能从公共网络访问（接口 `enp2s0`）。您可能需要考虑这样的结构：

```
table inet my_table {
  chain my_input { # 该链充当分派器
    type filter hook input priority filter;

    iif lo accept comment "始终接受环回"
    iifname enp2s0 jump my_input_public
    iifname enp3s0 jump my_input_private

    reject with icmpx port-unreachable comment "拒绝来自所有其他接口的流量"
  }
  chain my_input_public { # 适用于公共接口界面的规则
    ct state {established,related} accept
    ct state invalid drop
```

```
    udp dport bootpc accept
    tcp dport bootpc accept
    reject with icmpx port-unreachable comment "所有其它流量"
  }
  chain my_input_private {
    ct state {established,related} accept
    ct state invalid drop
    udp dport bootpc accept
    tcp dport bootpc accept
    tcp port http accept
    tcp port https accept
    reject with icmpx port-unreachable comment "所有其它流量"
  }
  chain my_output { # 我们把一切都放出来
    type filter hook output priority filter;
    accept
  }
}
```

或者，您可以只选择一个 `iifname` 语句，例如针对单个上游接口，并将所有其他接口的默认规则放在一个位置，而不是为每个接口分派。

伪装（Masquerading）

nftables 有一个特别的关键字 `masquerade`“其中源地址被自动设为输出接口的地址”（来源）。这对于接口的 IP 地址不可预测或不稳定的情况尤其有用，例如连接到许多 ISP 的路由器的上游接口。如果没有它，每次接口的 IP 地址更改时都必须更新网络地址转换规则。

要使用它：

- 确保在内核中启用伪装（如果使用默认内核，则已启用），否则请在内核配置期间，设置 `CONFIG_NFT_MASQ=m`。
- `masquerade` 关键字只能在 `nat` 类型的链中使用。
- 伪装是一种源网络地址转换（NAT），因此只在输出路径中工作。

具有两个接口的机器示例：LAN 连接到 `enp3s0`，公共网络连接到 `enp2s0`：

```
table inet my_nat {
  chain my_masquerade {
    type nat hook postrouting priority srcnat;
    oifname "enp2s0" masquerade
  }
}
```

由于表类型为 `inet`，IPv4 和 IPv6 数据包都将被伪装。如果您只希望 ipv4 数据包被伪装（因为 IPv6 的额外地址空间不需要 NAT），可以在 `oifname "enp2s0"` 前面使用 `meta nfproto ipv4` 表达式，或者可以将表类型更改为 `ip`。

带端口转发的 NAT

**本文或本章节的事实准确性存在争议。**

原因： I think `my_postrouting` chain will cause the destination computer see that connections are made by router rather than from some global IP. Also this does not masquerade outbound traffic.（在 Talk:Nftables 中讨论）

此示例将端口22和80转发到 `*destination_ip*`。您需要通过 sysctl 将 `net.ipv4.ip_forward` 和 `net.ipv4.conf.*wan_interface*.forwarding` 设置为 `1`。

```
table ip my_nat {
  chain my_prerouting {
    type nat hook prerouting priority dstnat;

    tcp dport { ssh, http } dnat to destination_ip
  }

  chain my_postrouting {
    type nat hook postrouting priority srcnat;

    ip daddr destination_ip masquerade
  }
}
```

计算每个 IP 的新连接数

使用此代码段计算 HTTPS 连接数：

```
/etc/nftables.conf
table inet filter {
    set https {
        type ipv4_addr;
        flags dynamic;
        size 65536;
        timeout 60m;
    }

    chain input {
        type filter hook input priority filter;
        ct state new tcp dport 443 update @https { ip saddr counter }
    }
}
```

要打印计数器，请运行 `nft list set inet filter https`。

动态黑洞

使用此代码段可以在1分钟内从超过 10/秒 限制的源 IP 断开所有 HTTPS 连接。

```
/etc/nftables.conf
table inet dev {
    set blackhole {
        type ipv4_addr;
        flags dynamic, timeout;
        size 65536;
    }

    chain input {
        ct state new tcp dport 443 \
                meter flood size 128000 { ip saddr timeout 10s limit rate over
10/second } \
                add @blackhole { ip saddr timeout 1m }

        ip saddr @blackhole counter drop
    }
}
```

要打印黑洞的 IP，请运行 `nft list set inet dev blackhole`。

提示和技巧

保存当前规则集

`nft list ruleset` 命令的输出也是它的有效输入文件。当前规则集可以保存到文件中，稍后再加载回。

```
$ nft -s list ruleset | tee filename
```

注意：`nft list` 不输出变量定义，如果原始文件中有任何变量定义，它们将丢失。规则中使用的任何变量都将被其值替换。

简单的状态防火墙

本文或本章节的事实准确性存在争议。

原因： 这不是一个非常简单的防火墙。我认为 Arch Linux 在 /etc/nftables.conf 中提供的内容很简单。建议用该脚本替换此部分，并给出如何根据特定需要扩展该部分的一些指导。（在 Talk:Nftables 中讨论）

见 Simple stateful firewall 获取更多信息。

单机

刷新当前规则集：

```
# nft flush ruleset
```

添加一个表：

```
# nft add table inet my_table
```

添加输入（input）、转发（forward）和输出（output）基本链。输入和转发的策略将是丢弃。输出的策略是接受。

```
# nft add chain inet my_table my_input '{ type filter hook input priority 0 ; policy drop ; }'
# nft add chain inet my_table my_forward '{ type filter hook forward priority 0 ; policy drop ; }'
# nft add chain inet my_table my_output '{ type filter hook output priority 0 ; policy accept ; }'
```

添加两个将与 tcp 和 udp 关联的常规链：

```
# nft add chain inet my_table my_tcp_chain
# nft add chain inet my_table my_udp_chain
```

将接受相关（related）和已建立（established）的流量：

```
# nft add rule inet my_table my_input ct state related,established accept
```

将接受所有环回接口流量：

```
# nft add rule inet my_table my_input iif lo accept
```

丢弃所有无效流量：

```
# nft add rule inet my_table my_input ct state invalid drop
```

接受 ICMP 和 IGMP：

```
# nft add rule inet my_table my_input meta l4proto ipv6-icmp accept
```

```
# nft add rule inet my_table my_input meta l4proto icmp accept
# nft add rule inet my_table my_input ip protocol igmp accept
```

新的 udp 流量将跳转到 UDP 链：

```
# nft add rule inet my_table my_input meta l4proto udp ct state new jump my_udp_chain
```

新的 tcp 流量将跳转到 TCP 链：

```
# nft add rule inet my_table my_input 'meta l4proto tcp tcp flags & (fin|syn|rst|ack)
== syn ct state new jump my_tcp_chain'
```

拒绝其他规则未处理的所有流量：

```
# nft add rule inet my_table my_input meta l4proto udp reject
# nft add rule inet my_table my_input meta l4proto tcp reject with tcp reset
# nft add rule inet my_table my_input counter reject with icmpx port-unreachable
```

此时，您应该决定要向传入连接打开哪些端口，这些端口由 TCP 和 UDP 链处理。例如，要打开网络服务器的连接，请添加：

```
# nft add rule inet my_table my_tcp_chain tcp dport 80 accept
```

要接受网络服务器的 HTTPS 连接在端口 443：

```
# nft add rule inet my_table my_tcp_chain tcp dport 443 accept
```

要接受 SSH 流量在端口 22：

```
# nft add rule inet my_table my_tcp_chain tcp dport 22 accept
```

要接受传入的DNS请求：

```
# nft add rule inet my_table my_tcp_chain tcp dport 53 accept
# nft add rule inet my_table my_udp_chain udp dport 53 accept
```

确保在满意后使您的更改永久化。

防止暴力攻击

Sshguard 是个程序，它可以检测暴力攻击，并根据临时列入黑名单的 IP 地址修改防火墙。见 Sshguard#nftables 了解如何设置与之一起使用的 nftables。

记录流量

你可以使用 `log` 操作记录数据包。记录所有传入流量的最简单规则是：

```
# nft add rule inet filter input log
```

见 nftables wiki 获取详情。

故障排除

与 Docker 一起工作

注意：

- 使用以下设置，即使具有 `--net host --privileged`，您也无法在容器内使用 `AF_BLUETOOTH` 等协议。
- 无根 Docker 容器已经在单独的网络命名空间中运行。你可能不需要做任何事情。

使用 nftables 可能会干扰 Docker 网络（以及其他容器运行时）。您可以在互联网上找到各种解决方案，其中包括修补 iptables 规则并确保定义的服务启动顺序，或者完全禁用 dockers iptables 管理，这使得使用 docker 非常受限（想想端口转发或 docker-compose）。

一种可靠的方法是让 docker 在一个单独的网络命名空间中运行，在那里它可以为所欲为。最好不要使用 iptables-nft包 来防止 docker 混合 nftables 和 iptables 规则。

使用以下 docker 服务放置文件\[损坏的链接：无效的章节\]：

```
/etc/systemd/system/docker.service.d/netns.conf
[Service]
PrivateNetwork=yes

# 清理
ExecStartPre=-nsenter -t 1 -n -- ip link delete docker0

# 添加虚拟接口
ExecStartPre=nsenter -t 1 -n -- ip link add docker0 type veth peer name docker0_ns
ExecStartPre=sh -c 'nsenter -t 1 -n -- ip link set docker0_ns netns "$$BASHPID" &&
true'
ExecStartPre=ip link set docker0_ns name eth0

# 使主机联机
ExecStartPre=nsenter -t 1 -n -- ip addr add 10.0.0.1/24 dev docker0
ExecStartPre=nsenter -t 1 -n -- ip link set docker0 up

# 使命名服务器联机
ExecStartPre=ip addr add 10.0.0.100/24 dev eth0
ExecStartPre=ip link set eth0 up
ExecStartPre=ip route add default via 10.0.0.1 dev eth0
```

如果 `10.0.0.*` IP 地址不适合您的设置，请调整它们。

使用以下后路由规则为 docker0 启用 IP 转发并设置 NAT：

```
iifname docker0 oifname eth0 masquerade
```

然后，确保已启用内核 IP 转发。

现在，您可以使用 nftables 为 `docker0` 接口设置防火墙和端口转发，而不受任何干扰。

另见

- 网络过滤器 nftables wiki
- debian:nftables
- gentoo:nftables
- First release of nftables
- nftables quick howto
- The return of nftables
- What comes after ‘iptables’? Its successor, of course: \C0

代理服务器

代理的麻烦大多不在安装，而在装好之后如何让程序真正把流量交出去：谁走代理、走哪一层。八节因此分成几类：Squid、Privoxy、Polipo 管本机缓存与转发，V2Ray、Shadowsocks、NaïveProxy 管对外连接，Qv2ray 只是 V2Ray 的前端。

先读《代理服务器》：环境变量、libproxy、GNOME3 设置代理，决定了装好的软件能否真正生效，是其余配置的前提。之后按场景只挑一节落地——缓存转发看 Squid 或 Privoxy，对外连接看 V2Ray 或 Shadowsocks，NaïveProxy、Polipo 与 Qv2ray 按需查阅。最容易踩的坑是把「装上」当成「配好」：桌面代理设置管不到命令行工具与 systemd 服务，libproxy 也只对链接它的程序生效，不显式设置 http_proxy，流量照样直连。

3.1　NaïveProxy

难度 入门 | **篇幅** 2.8 KB | **所属** 代理服务器

本节将讲到

- 安装
- 配置

翻译状态：

- 本文（或部分内容）译自 NaïveProxy，最近一次同步于 2024-07-22，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

NaïveProxy 是一个跨平台的代理工具，使用了 Chromium 的网络栈来伪装流量，并在绕过中国国家防火墙时提供了强大的反审查能力和低可检测性。它可以减轻由 TLS 指纹特征造成的被识别问题，并在中国大规模封锁基于 TLS 的审查规避工具时成功存活。它需要一个 `naiveproxy` 客户端和一个带有 `forwardproxy` 模块的 Caddy 服务器来工作。

安装

安装 naiveproxyAUR 或 naiveproxy-gitAUR 来获取最新的开发版本，并执行 `naiveproxy config.json`。下面是一个配置文件示例：

config.json
```
{
  "listen": "socks://127.0.0.1:1080",
  "proxy": "https://myUsername:myStrongPassword@my.domain"
}
```

配置

如果没有一个带有 forwardproxy 模块的 caddy 服务器，Naiveproxy 将无法工作。你可以用 `xcaddy` 来构建它：

```
$ go install github.com/caddyserver/xcaddy/cmd/xcaddy@latest
$ ~/go/bin/xcaddy build --with
github.com/caddyserver/forwardproxy@caddy2=github.com/klzgrad/forwardproxy@naive
```

接着，配置 caddy：

```
/etc/caddy/Caddyfile
{
  order forward_proxy before file_server
}
:443, my.domain:443 {
  tls /etc/caddy/ssl.cer /etc/caddy/ssl.key
  forward_proxy {
    basic_auth myUsername myStrongPassword
    hide_ip
    hide_via
    probe_resistance
  }
  file_server {
    root /var/www/html
  }
}
```

注意，为了让这个 Caddyfile 能正常工作，`:443` 必须放在最前面。有关如何配置 TLS 证书的说明，请参阅 Caddyfile 文档。

然后，启动 caddy 服务器：

```
# setcap cap_net_bind_service=+ep ./caddy && ./caddy start
```

您可能也想要把 caddy 作为一个守护进程运行。

3.2 Squid

难度 入门 | **篇幅** 4.2 KB | **所属** 代理服务器

本节将讲到

- 安装
- 配置
- 启动

**这篇文章或章节的翻译质量不佳。**

原因：Still references `rc.d`.（在 Talk:Squid# 中讨论）

来自 Squid 官方网站：

: ***Squid** 是一个 Web 缓存代理，支持 HTTP, HTTPS, FTP, 以及更多。它通过缓存与重用经常请求的web页面，减少带宽使用同时提升了响应时间。Squid 具有可扩展的*

访问控制功能，同时可以使服务器加速。它运行在 Unix 和 Windows 中，采用 GNU GPL 协议发布。

尽管 squid 在大型公司和学校中工作的很好，它也可以为个人家庭用户使用。

安装

安装位于官方软件仓库的 squid包。

配置

默认设置下，缓存目录会创建于 `/var/cache/squid`,同时正确的权限会被同时设置好。然而，为了更好的控制，我们需要深入 `/etc/squid/squid.conf`.

所有的配置项都有注释，但是如果你想去掉注释的话，你可以运行：

```
sed -i "/^#/d;/^ *$/d" /etc/squid/squid.conf
```

下面的选项会对你有用。如果你没有这些选项，你可以添加进去。

- `http_port` - 设置 Squid 绑定的代理端口。你可以将 Squid 绑定到多个端口，使用多个 http_port 行。默认情况下，Squid 绑定到 3128 端口。

```
http_port 3128
http_port 3129
```

- `http_access` - 这是访问控制列表，即谁被允许使用这个代理。默认只有 localhost 被允许访问这个代理。作为测试目的，你可以改变这个选项 `http_access deny all` 为 `http_access allow all`，这会允许所有人链接到你的代理。如果你想仅仅允许子网内访问，你可以使用：

```
acl ip_acl src 192.168.1.0/24
http_access allow ip_acl
http_access deny all
```

- `cache_mgr` - 这是缓存管理员的 email 地址。

```
cache_mgr squid.admin@example.com
```

- `shutdown_lifetime` - 指定当 squid 的 rc.d 脚本请求停止的时候等待的时间。如果你在你的桌面电脑上运行 squid，你也许需要设成短一点的时间值。

```
shutdown_lifetime 10 seconds
```

- `cache_mem` - 这是你想让 squid 使用内存而不写入硬盘的内存量。Squid 的总内存使用量会超过这个值！默认这个值是 8MB，所以如果你有很多内存的话，你也许想增加这个值。

```
cache_mem 64 MB
```

- `visible_hostname` - 可以在状态/错误信息中看到的 hostname。

```
visible_hostname cerberus
```

- `cache_peer` - 如果你想让 Squid 穿过另一个代理服务器，而不是直接连到 Internet，你需要在这里指定。
- `login` - 如果上层代理需要认证的话使用这个选项。
- `never_direct` - 如果你设定了上面的选项的话，可以告诉缓存不要直接连接到 Internet 请求页面。

```
cache_peer 10.1.1.100 parent 8080 0 no-query default login=user:password
never_direct allow all
```

- `maximum_object_size` - 这是缓存部件的最大大小。默认这个值很小 (我认为是 256KB)，所以如果你有很多磁盘空间，你需要将这个值增大到一个合理的值。

```
maximum_object_size 10 MB
```

- `cache_dir` - 这是你的缓存文件夹，存储缓存文件的位置。有很多个选项，但是格式大概是这样子：

```
cache_dir diskd <directory> <size in MB> 16 256
```

所以，在一个学校的代理服务器中：

```
cache_dir diskd /cache0 200000 16 256
```

如果你改变了默认的缓存文件夹，你需要在启动 squid 前为文件夹设置合适的权限。否则会无法创建缓存文件夹并且停止运行。

启动

配置完成之后，你需要检查一下配置文件是否正确：

```
# squid -k check
```

然后创建缓存目录：

```
# squid -z
```

最后你可以启动 Squid:

```
# systemctl start squid
```

开机启动：

```
# systemctl enable squid
```

3.3 V2Ray

难度 入门 | **篇幅** 4.4 KB | **所属** 代理服务器

本节将讲到

- 安装
- 配置
- 使用
- 提示与技巧
- 故障排除

V2Ray 是 Project V 的核心工具，其主要负责网络协议和功能的实现，与其它 Project V 通信。

安装

安装 v2ray包，如果需要最新开发版本请安装 v2ray-gitAUR。

如果需要图形化管理 v2ray，可安装使用 WebUI 管理的 v2rayaAUR (或 v2raya-binAUR)。

Qv2ray 与 Nekoray 已经停止维护，不再推荐安装。

配置

V2Ray 使用 Json 配置， 请参考配置文件格式和配置。

使用

命令行

在命令行中使用 `v2ray` 命令启动，参考 Commandline.

服务

启动/启用 `v2ray.service`。

多文件配置

要使用多文件配置，请编辑 `/etc/systemd/system/v2ray.service`，加入以下内容：

```
/etc/systemd/system/v2ray.service
[Unit]
Description=V2Ray Service
After=network.target nss-lookup.target

[Service]
User=nobody
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
ExecStart=/usr/bin/v2ray -confdir /etc/v2ray/

[Install]
WantedBy=multi-user.target
```

配置方式

V2Ray 有多种配置方式，目前较为有效的是 WebSocket+TLS。

提示：将 TLS 交由 Nginx 配置，是一种更合理，也更易于管理的方式。

提示与技巧

路由规则

参见路由和 V2Ray 路由。

预设的路由规则文件是 `geosite.dat` 和 `geoip.dat`，位于 `/usr/share/v2ray`。这两个文件分别由软件包 v2ray-domain-list-community包和 v2ray-geoip包 提供。这两个软件包将被作为 v2ray包 的依赖项安装。

替换预设的路由规则文件

有些路由规则文件有现成的软件包。比如对 @Loyalsoldier/v2ray-rules-dat 来说，可以安装 v2ray-rules-dat-gitAUR 软件包。

对于没有现成软件包的路由规则文件，可以创建软件包并安装，或者在不移除依赖包的前提下移除 v2ray-domain-list-community包 和 v2ray-geoip包 然后将路由规则文件放在 `/usr/share/v2ray` 目录下。

警告：直接替换 `geosite.dat` 和 `geoip.dat` 可能导致文件冲突"_错误)。

故障排除

Failed with result 'exit-code'

如果您在日志里看到了此错误：`Failed with result 'exit-code'`，这是因为 V2Ray 对目录 /var/log/v2ray/access.log 没有写入权限。使用以下命令可解决此问题。

```
chown -R nobody /var/log/v2ray
```

另请参阅

- V2Fly 官方网站和文档
- V2Ray 新手指南
- v2rayA 官方网站和文档
- Nekoray 官方网站和文档
- 不再推荐使用 Qv2ray (项目已经停止维护) 。

3.4 Privoxy

难度 入门 | **篇幅** 4.5 KB | **所属** 代理服务器

本节将讲到

- 安装
- 配置
- 使用

- 参阅

Privoxy 是一个 HTTP 协议过滤代理，常结合 Tor 使用。Privoxy 是有着先进的过滤能力和保护隐私的代理工具，它可以过滤网页内容，管理cookies，控制访问，除广告、横幅、弹出窗口等等，它同时支持单系统和多用户网络。

当用户直接使用 SOCKS 代理访问网络时，浏览器会泄漏 DNS 请求，降低匿名性，这时应该使用 Privoxy。

安装

安装 privoxy包。

配置

使用Privoxy必须指定转发规则，Privoxy的主配置文件在`/etc/privoxy/config`。

监听地址

Privoxy默认只提供给本机使用（即localhost或者127.0.0.1），如果需要将Privoxy提供给网络中的其他计算机使用，需要`/etc/privoxy/config`在添加监听信息，格式：

```
listen-address [SERVER-IP]:[PORT]
```

例如提供给192.168.1.1的8118端口使用:

```
listen-address 192.168.1.1:8118
```

转发协议

编辑 `/etc/privoxy/config`文件添加相关转发规则：

- 转发socks5

示例（注意9050后面有一个空格和点号）：

```
 forward-socks5 / localhost:9050 .
```

- 转发.i2p

通过I2P路由转发.i2p，示例：

```
 forward .i2p localhost:4444
```

- 转发onion

示例（注意9050后面有一个空格和点号）：

```
forward-socks4a .onion localhost:9050 .
```

广告屏蔽

警告：Blocking advertisements can reduce anonymity, since it creates a unique browser signature. This should not be done when using tor or another proxy for anonymity.

Using an ad blocking extension in a web browser can increase page load time. Additionally, extensions like AdBlock Plus are not supported by all browsers. A useful alternative is to install system-wide ad blocking by setting a proxy address in your preferred browser.

You can use adblock plus filters. The privoxy blocklist script automatically downloads adblock plus filters, converts them to a privoxy friendly format, and edits privoxy's config file to include those filters:

1. Run the script once to create `/etc/conf.d/privoxy-blacklist`
2. Edit `/etc/conf.d/privoxy-blacklist` to uncomment the line `PRIVOXY_USER=` and the two lines below it.
3. Run the script again to download and install the blocklists.
4. Restart privoxy.

To block tracking via embedded Facebook "Like" button, Twitter "follow", and Google Plus "+1", edit `/etc/privoxy/user.action` and add these lines to the end:

```
{+block-as-image{Facebook "like" and similar tracking URLs.}}
www.facebook.com/(extern|plugins)/(login_status|like(box)?|activity|fan)\.php
platform.twitter.com/widgets/follow_button?
plusone.google.com
```

使用

使用Systemd (简体中文) 启用privoxy服务。

对程序进行代理设置，默认的地址是:

```
localhost:8118
```

Firefox浏览器:进入 首选项 > 高级 > 网络 > 设置，添加http代理。

Chromium

```
$ chromium --proxy-server="localhost:8118"
```

你也可以添加`http_proxy`环境变量，例如添加到`~/.bashrc`中:

```
http_proxy="http://localhost:8118"
```

参看Proxy settings

参阅

- Privoxy 官方网站
- Tor 官方网站

3.5 Qv2ray

难度 入门 | **篇幅** 4.6 KB | **所属** 代理服务器

本节将讲到

- 安装

- 配置
- 使用
- 疑难解答
- 参阅

Qv2ray 是一个使用 Qt 编写的跨平台的 V2Ray 图形前端，可以在图形化界面中方便地配置代理。

安装

注意：Shadowsocks-NET/Qv2ray 不与 Qv2ray/Qv2ray 的插件、配置文件兼容 。

Qv2ray/Qv2ray

Qv2ray/Qv2ray 是 Qv2ray 的原始版本。

安装 qv2rayAUR 软件包。对于开发版本，安装 qv2ray-dev-gitAUR 软件包，也可以在 archlinuxcn 储存库中找到。

注意：此版本已停止维护。

Shadowsocks-NET/Qv2ray

Shadowsocks-NET/Qv2ray 是新的原创 Qv2ray 项目。

安装 qv2ray-v3AUR 软件包。对于开发版本，安装 qv2ray-gitAUR 软件包。对于预构建二进制每日版本，安装 qv2ray-static-nightly-binAUR 软件包。

注意：此版本尚处于开发阶段，存在缺陷如显示连接后无法上网 和功能缺失如无多语言支持。

插件

参见使用插件。

注意：Qv2ray 本身并不包含 V2Ray 核心。如果不使用插件，就需要手动安装 V2Ray，否则 Qv2ray 将无法使用。若使用插件，Qv2ray 可以在不依赖 V2Ray 核心的情况下运行 .

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Use Template:App.（在Talk:Qv2ray讨论）

对于 Qv2ray/Qv2ray：

- 安装 qv2ray-plugin-ssr-gitAUR 以支持 ShadowsocksR 协议。
- 安装 qv2ray-plugin-trojanAUR 以支持 Trojan-GFW 协议。
- 安装 qv2ray-plugin-trojan-go-gitAUR 以支持 Trojan-Go 协议。
- 安装 qv2ray-plugin-naiveproxy-gitAUR 以支持 NaiveProxy 协议。
- 安装 qv2ray-plugin-commandAUR 以进行自动化。

上述插件也可以在 *archlinuxcn* 储存库找到，可以安装 *qv2ray-plugin* 组以获得上述所有插件。

注意：AUR 中尚没有适用于 Shadowsocks-NET/Qv2ray 的插件的包。

配置

见 Qv2ray 手册。

打开 Qv2ray 中的*首选项*进行配置。Qv2ray 配置文件位于 `~/.config/qv2ray/` 目录。

注意：若要从 Qv2ray/Qv2ray 迁移到 Shadowsocks-NET/Qv2ray，请删除旧配置并重新进行配置 。

使用

见导入节点到 Qv2ray 和配置浏览器/软件以使用 Qv2ray。也可以看看 Shadowsocks#为其它程序配置代理。

疑难解答

见常见问题。

参阅

- Qv2ray 官方网站 & 文档

3.6 Polipo

难度 进阶 | **篇幅** 5.9 KB | **所属** 代理服务器

本节将讲到

- 安装
- 优化 Polipo
- 启动守护进程
- 配置

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： squid、privoxy 等 (在Talk:Polipo讨论)

来自Polipo's site:

: "*Polipo 是一个小而快速的缓存 web 代理程序(web 缓存, HTTP 代理, 代理服务器)。尽管 Polipo 是为一个人或一小群人使用而设计的，但并不妨碍它为一大群人所使用。*"

下面包括了 Polipo 的安装和设置：

安装

安装软件包 polipoAUR\[损坏的链接：package not found\]。

优化 Polipo

作为指定的用户运行

Polipo 需要使用普通用户运行。这个用户可以建立新的也可以使用旧的：

```
# mkdir /var/cache/polipo
# groupadd -r polipo
# useradd -d /var/cache/polipo -g polipo -r -s /bin/false polipo
```

为了在用特定用户运行之前确保所有的文件和目录都已经被建立，启动然后停止 Polipo。

```
# /etc/rc.d/polipo start
# /etc/rc.d/polipo stop
```

与其他的守护程序以root运行然后尽快的降低权限不同，polipo 是以启动它的用户权限运行的。如果 polipo 是在`/etc/rc.d/polipo`调用的，改变调用行

```
[[ ! -d /var/run/$DAEMON ]] && install -d $DAEMON /var/run/$DAEMON
/usr/bin/$DAEMON $ARGS >/dev/null 2>&1
```

为

```
[[ ! -d /var/run/$DAEMON ]] && install -d $DAEMON --group=polipo --
owner=polipo /var/run/$DAEMON
su -c "/usr/bin/$DAEMON $ARGS" -s /bin/sh polipo >/dev/null 2>&1
```

然后也有必要改变几个 polipo 需要写入的目录的所有者/权限：

```
# chown polipo:polipo /var/log/polipo
# chown -R polipo:polipo /var/run/polipo
# chown -R polipo:polipo /var/cache/polipo
```

尽管更好的选择是建立一个被指定用户所有的目录 `/var/log/polipo` 然后通过配置文件的 logFile 变量设置 polipo 的 log 文件到 `/var/log/polipo/polipo.log` 。

启动守护进程

以下命令启动 Polipo 守护进程：

```
# /etc/rc.d/polipo start
```

将它添加到 `/etc/rc.conf` 使它在启动时自动运行：

```
DAEMONS=(syslog-ng network netfs polipo crond)
```

多线程

Polipo 也可以不需要超级用户权限即可运行。想要这样的话，首先复制 `/etc/polipo/config.sample` 到一个合适的目录：

```
$ cp /etc/polipo/config.sample ~/.poliporc
```

编辑并将 `/var/cache/polipo` 替换成一个可写入的目录:

```
# Uncomment this if you want to put the on-disk cache in a
# non-standard location:
diskCacheRoot = "~/.polipo-cache/"
```

创建 cache 目录：

```
$ mkdir ~/.polipo-cache
```

最后，使用新配置文件启动：

```
$ polipo -c ~/.poliporc
```

配置

管理通常是在 `/etc/polipo/config` 进行。大多数用户可以选择使用示例配置文件，它适合大多数情况，注释很详细。

```
# cd /etc/polipo; cp config.sample config
```

必须要提到的 Polipo 配置中的一个要素是 polipo 的默认行为是通过端口屏蔽外部连接。Polipo 的配置文件有两个变量控制允许的外部端口。`allowedPorts` 制定外部 HTTP连接的端口。默认是 80-100 和 1024-65535. `tunnelAllowedPorts` 制定 Polipo 隧道流量允许的端口，以及 HTTPS 流量的端口。默认它是更严格的限制的："*它默认允许 ssh, HTTP, https, rsync, IMAP, imaps, POP, pops, Jabber, CVS 和 Git 流量。*"

如果你看到来自 Polipo 的 "403 Forbidden Port" 错误信息，你需要配置polipo接收更多 HTTP 或 HTTPS的端口。要设定全都允许，添加下面的几行到 `/etc/polipo/config`:

```
allowedPorts = 1-65535
tunnelAllowedPorts = 1-65535
```

与其他代理软件不同，Polipo 需要重启后才能应用改变。

浏览器

设置浏览器，让它使用 `localhost:8123` 作为代理。确保禁用浏览器的磁盘缓存来避免冗余的 IO 操作以及低下的性能。

隧道

注意：按照 Polipo FAQ 关于 "intercepting proxy" 这是不被支持的!

注意：这需要用 Polipo 自己的用户运行它。

除了手动配置每个浏览器和其他程序使用 Polipo 的缓存之外，也可以使用 iptables 来通过 polipo 路由。

在安装了 iptables 之后，添加合适的规则到 `/etc/iptables/iptables.rules`：

```
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
```

```
-A OUTPUT -p tcp --dport 80 -m owner ! --uid-owner polipo -j ACCEPT
-A OUTPUT -p tcp --dport 80 -j REDIRECT --to-ports 8123
COMMIT
```

这会通过 Polipo 路由所有的 HTTP 流量。移除浏览器的所有代理设置，然后重新启动 iptables。

Privoxy

Privoxy 是一个可以有效拦截广告和其他恶意程序的代理软件。

按照 Polipo 开发人员的建议，为了得到 Privoxy 的隐私性和 Polipo 的大部分性能，你需要把 Polipo 放在 Privoxy 的上游。

也就是说：

- 浏览器指向 Privoxy: `localhost:8118`
- 然后将 Privoxy 流量通过 Polipo: `forward / localhost:8123` 在 Privoxy 配置文件设置。

Tor

Tor是一个匿名代理网络。

与 Tor 一起使用 Polipo，需要在 `/etc/polipo/config` 取消注释下面的几行：

```
socksParentProxy = localhost:9050
socksProxyType = socks5
```

DansGuardian

DansGuardian 是一个 web 内容过滤器。 DansGuardian 和 Polipo (而不是 squid 或者 tinyproxy) 共同使用，唯一的不同是在 `dansguardian.conf` 中 proxyport 需要设定为 polipo 的 8123:

```
# the port DansGuardian connects to proxy on
proxyport = 8123
```

3.7 Shadowsocks

难度 进阶 | **篇幅** 8.1 KB | **所属** 代理服务器

本节将讲到

- 安装
- 配置
- 使用
- 参阅

Shadowsocks 是一个轻量级 SOCKS5#SOCKS5) 代理。

安装

安装 shadowsocks-rust包。

其他可选项

- shadowsocks-goAUR—使用Go语言编写。
- shadowsocks-gitAUR—使用Python语言编写。已不再开发。
- shadowsocks-libev-staticAUR—使用C语言编写。已不再开发。
- go-shadowsocks2AUR—使用Go语言编写。已不再开发。

配置

Shadowsocks 以 JSON 为配置文件格式，以下是软件包中的样例：

```
/etc/shadowsocks/config.json
{
    "server":"my_server_ip",
    "server_port":8388,
    "local_address": "127.0.0.1",
    "local_port":1080,
    "password":"mypassword",
    "timeout":300,
    "method":"chacha20-ietf-poly1305",
    "fast_open": false,
    "workers": 1,
    "prefer_ipv6": false
}
```

提示：

- 若需同时指定多个服务端地址，使用如下语法 `"server":["1.1.1.1", "2.2.2.2"]`.
- 要找出在你的机器上运行最快的方式，可以运行这个脚本。

名称	解释
server	服务端监听地址
server_port	服务端端口
local_address	本地监听地址
local_port	本地端口
password	用于加密的密码
timeout	超时时间（秒）
method	加密方式，默认为 `chacha20-ietf-poly1305`
mode	是否启用 TCP / UDP 转发，参阅 shadowsocks-libev(8)
fast_open	是否启用 TCP Fast Open
workers	worker 数量

使用

客户端

警告：udns包 用作 DNS 的存根解析程序。为了防止客户端应用程序（如浏览器）的 DNS 请求泄漏，必须使用其他应用程序。例如客户端上的 Privoxy 或完整的 DNS 解析程序。

通过命令行

使用 `ss-local`（shadowsocks-libev）或`sslocal` （shadowsocks-rust）命令启动客户端。 要使用 `/etc/shadowsocks/*config*.json` 配置文件启动客户端:

```
$ sslocal -c /etc/shadowsocks/config.json
```

或者，也可以直接在命令中指定配置：

```
$ sslocal -s 服务器地址 -p 服务器端口 -l 本地端口 -k 密码 -m 加密方式
```

要输出详细日志，添加 `-v` 命令:

```
$ sslocal -s 服务器地址 -p 服务器端口 -l 本地端口 -k 密码 -m 加密方式 -v
```

守护进程管理

Shadowsocks 客户端可以通过 systemctl 使用 `shadowsocks@.service` 或 `shadowsocks-libev@.service` 实例控制。

要使用配置文件 `/etc/shadowsocks/*config*.json`，启动并启用 `shadowsocks-libev-server@*config*.service` 或 `shadowsocks-server@*config*.service`.

您可能也会对在网络启动后\[损坏的链接：无效的章节\]运行 `shadowsocks@` 或 `shadowsocks-libev@` 实例感兴趣。

提示：以 root 运行 `journalctl -u shadowsocks@*foo*` 来查看日志。

为其它程序配置代理

Shadowsocks 客户端启动后，其它程序并不会直接使用，可使用以下方法对其进行配置。

全局代理

使用 iptables 等工具。

注意：使用全局代理会使所有的连接通过 Shadowsocks 服务器中转，一般不建议使用全局代理。

程序设置

不少程序都能在其设置中添加代理。只需要在其设置中找到网络相关配置，添加 SOCKS5 代理，参照本地配置文件中的地址和端口填写即可（例如浏览器可参考下文 #浏览器配置）。

使用工具进行临时代理

例如 proxychains-ng包 和 tsocks包. 参见 Proxy server#Using a SOCKS proxy.

转换为 HTTP 代理

直接使用 SOCKS5 代理有时未必是用户的期望，可使用 Privoxy 或 Squid 等软件转换为 HTTP 代理。

以 Privoxy 为例，编辑配置文件，添加 SOCKS5 转发：

```
forward-socks5 / 127.0.0.1:1080 .
```

默认监听的是本机的 8118 端口，即 `localhost:8118`，可更改为监听其他端口：

```
listen-address  127.0.0.1:8010
```

提示：如果希望网络上其他主机也能使用该 Privoxy 配置，可以更改 `127.0.0.1` 为 `0.0.0.0` 或直接删除 `127.0.0.1`.

重启 `privoxy.service`.

假设转换后的 HTTP 代理为 `127.0.0.1:8118`，则在终端中启动（以 Chromium 为例）：

```
$ chromium --proxy-server="http://127.0.0.1:8118"
```

浏览器配置

提示：浏览器直接使用 SOCKS 代理时，你可能需要使用 Privoxy 等辅助程序，因为一般浏览器会泄漏你的DNS请求，从而降低你的匿名性，参见 #转换为 HTTP 代理

- Firefox
  - 使用如 SwitchyOmega 或 FoxyProxy 等扩展。
  - 直接在*首选项* > *常规* > *网络代理*中设置*手动代理配置*或者*自动代理配置的 URL（PAC）*。
  - 使用*手动代理配置*：在 *SOCKS 主机*栏填上 Shadowsocks 设置的本地地址和端口，点选 *SOCKS v5*，保存即可。
  - 使用*自动代理配置的 URL（PAC）*：可使用 genpac 工具生成。
- Chrome/Chromium
  - 使用 SwitchyOmega 等插件。

服务端

守护进程管理

shadowsocks-libev-staticAUR: start/enable `shadowsocks-libev-server@*`配置文件`*.service`

shadowsocks-rust包: start/enable `shadowsocks-rust-server@*`配置文件`*.service`

多端口运行

将配置文件中的 "server_port" 和 "password" 字段删去，添加 "port_password" 字段配置端口及其密码。

参见 Configure Multiple Users 中的示例。

性能优化

- 使用常用端口如 443 等。Great Firewall 为减轻压力，对常用端口检查相对较少。
- 启用 TCP Fast Open.
- 启用 BBR.
- 安装 python-gevent包 提升 shadowsocks包 的速度。
- 优化内核参数，参见 Optimizing Shadowsocks.

加密

参见 AEAD Ciphers\[失效链接 2022-09-23 ⓘ\].

安装 python-m2crypto包 包将会使加密更快一点。

要使用 Salsa20 或 *ChaCha20* 加密，安装 libsodium包 包。

参阅

- shadowsocks-libev(8)
- Linux Kernel 4.9 中的 BBR 算法与之前的 TCP 拥塞控制相比有什么优势?

3.8 代理服务器

难度 进阶 | **篇幅** 12.9 KB | **所属** 代理服务器

本节将讲到

- HTTPS MITM代理
- 环境变量
- 关于libproxy
- 网络代理选项
- 通过socks代理

根据维基百科 Wikipedia:

: 在计算机网络中，代理服务器是一个作为客户从其他服务器请求资源的中介的服务器。

HTTPS MITM代理

当调试HTTPS连接时，在浏览器之外进行拦截是很有用的。为了让TLS MITM工作，你需要信任浏览器或系统范围内的代理certificate authority。

- **Charles** — 用Java编写的图形化试用软件。

: https://www.charlesproxy.com/ || charlesAUR

- **Fiddler** — 在Mono上运行的专有图形化软件。

: https://www.telerik.com/fiddler || fiddlerAUR

- **mitmproxy** — 命令行和网页界面的软件，用Python编写，也提供API。

: https://mitmproxy.org/ || mitmproxy包

- **sslsplit** — 适用于任何TLS连接，但不能作为浏览器中的HTTP代理，用C语言编写。

: https://www.roe.ch/SSLsplit || sslsplit包

警告：*mitmproxy* 创建可被所有用户读取的私钥。

环境变量

一些程序，例如 wget 还有 (pacman基于此) *curl*, 使用 `*protocol*_proxy` 形式的环境变量来确定一个给定网络协议 (例如 HTTP, FTP, 等等)。

下面是一个在shell中设置这些变量的例子:

```
export http_proxy=http://10.203.0.1:5187/
export https_proxy=$http_proxy
export ftp_proxy=$http_proxy
export rsync_proxy=$http_proxy
export no_proxy="localhost,127.0.0.1,localaddress,.localdomain.com"
```

有些程序会寻找环境变量为大写字母版本。

如果代理环境变量要提供给所有的用户和所有的应用程序，上述的导出命令可以添加到一个脚本中，比如说在 `/etc/profile.d/`中的`proxy.sh` . 脚本必须被设置成可执行的executable. 当使用像Xfce这样不提供代理配置选项的桌面环境时，这种方法很有帮助。例如，Chromium浏览器将在运行XFCE时使用此方法设置的变量。

另外，还有一个名为proxyman-gitAUR的工具，这个工具声称可以轻松配置全系统的代理设置。它还可以处理其他软件的代理配置，比如git、npm、Dropbox等。这个项目的灵感来源于Alan Pope制作脚本。

另外，您也可以通过在`.bashrc`中添加一个函数来自动切换变量。

```
function proxy_on() {
    export no_proxy="localhost,127.0.0.1,localaddress,.localdomain.com"

    if (( $# > 0 )); then
        valid=$(echo $@ | sed -n 's/\([0-9]\{1,3\}.\?\)\{4\}:\([0-9]\+\)/&/p')
        if [[ $valid != $@ ]]; then
            >&2 echo "Invalid address"
            return 1
        fi
        local proxy=$1
        export http_proxy="$proxy" \
               https_proxy=$proxy \
               ftp_proxy=$proxy \
               rsync_proxy=$proxy
        echo "Proxy environment variable set."
        return 0
    fi

    echo -n "username: "; read username
    if [[ $username != "" ]]; then
        echo -n "password: "
        read -es password
        local pre="$username:$password@"
    fi

    echo -n "server: "; read server
    echo -n "port: "; read port
    local proxy=$pre$server:$port
    export http_proxy="$proxy" \
           https_proxy=$proxy \
           ftp_proxy=$proxy \
           rsync_proxy=$proxy \
           HTTP_PROXY=$proxy \
           HTTPS_PROXY=$proxy \
           FTP_PROXY=$proxy \
           RSYNC_PROXY=$proxy
}

function proxy_off(){
    unset http_proxy https_proxy ftp_proxy rsync_proxy \
          HTTP_PROXY HTTPS_PROXY FTP_PROXY RSYNC_PROXY
    echo -e "Proxy environment variable removed."
}
```

如果不需要用户名和密码可以设置为缺省值。

作为一个替代方案，你可能想使用以下脚本。 修改字符串`YourUserName`、`ProxyServerAddress:Port`、`LocalAddress`和`LocalDomain`来匹配你自己的数据，然后编辑你的`~/.bashrc`来包含编辑过的函数。任何新的bash窗口都会有新的函数。在现有的bash窗口中，输入`source ~/.bashrc`。 你也可以把函数定义放在一个单独的文件中，比如`functions`，然后把`source functions`添加到`.bashrc`中，而不是把所有内容都放在`.bashrc`中。 "myProxy "这个名字改成一个简短易写的名字也是可以的。

```
#!/bin/bash

assignProxy(){
   PROXY_ENV="http_proxy ftp_proxy https_proxy all_proxy HTTP_PROXY HTTPS_PROXY
FTP_PROXY ALL_PROXY"
   for envar in $PROXY_ENV
   do
      export $envar=$1
   done
```

```
   for envar in "no_proxy NO_PROXY"
   do
      export $envar=$2
   done
}

clrProxy(){
    PROXY_ENV="http_proxy ftp_proxy https_proxy all_proxy HTTP_PROXY HTTPS_PROXY
FTP_PROXY ALL_PROXY"
    for envar in $PROXY_ENV
    do
       unset $envar
    done
}

myProxy(){
   user=YourUserName
   read -p "Password: " -s pass &&  echo -e " "
   proxy_value="http://$user:$pass@ProxyServerAddress:Port"
   no_proxy_value="localhost,127.0.0.1,LocalAddress,LocalDomain.com"
   assignProxy $proxy_value $no_proxy_value
}
```

通过sudo保持代理

如果代理环境变量environment variables只为一般用户设置，那么当使用sudo运行命令时（或者当程序在内部使用sudo时）无法获取代理。

防止这种情况发生的方法是在sudo配置文件中添加以下一行。

```
/etc/sudoers.d/05_proxy
Defaults env_keep += "*_proxy *_PROXY"
```

network managers的自动化

- NetworkManager 不会改变环境变量.
- netctl 可以设置环境变量，但它们不会被其他应用程序应用，因为它们不是netctl的子程序。

关于libproxy

libproxy (在额外的仓库中可以找到）是一个抽象库，所有想要访问网络资源的应用程序都应该使用它。这个项目仍在开发中，但如果被广泛采用的话，可能会在GNU/Linux中实现对代理的统一和自动化处理。

libproxy的作用是从不同的源头读取代理设置，并将它们提供给使用该库的应用程序。libproxy的有趣之处在于，它提供了一个实现了 Web Proxy Autodiscovery Protocol 和 Proxy Auto-Config 的实现。

`/usr/bin/proxy` 以URL作为参数，并返回可用于获取这些网络资源的代理。

注意：0.4.11 版本 不支持 `http_proxy='wpad:'` 因为 `{ pkg-config 'mozjs185 >= 1.8.5'; }` 会产生错误。

从2009年4月6日开始，libproxy被libsoup依赖，也被midori包浏览器间接使用。

网络代理选项

- Squid 是一个非常流行的缓存/优化代理。
- Privoxy 是一个匿名和广告屏蔽的代理。
- tinyproxy包 是一个小型、高效的HTTP/SSL代理守护进程。
- 对于一个简单的代理，可以使用带端口转发的ssh。

简单的ssh代理

ssh连接到服务器，如下所示。

```
$ ssh -D PORT USER@HOST
```

对于 PORT，选择一些非 IANA 注册端口的数字。ssh将作为SOCKS的服务器，支持SOCKS代理服务器的软件可以简单地配置连接到localhost上的*PORT*。支持SOCKS代理服务器的软件可以简单地配置为连接到localhost的*PORT*。

通过socks代理

有如下两种情形:

- 想使用的应用程序处理SOCKS5代理（例如Firefox），那么你只需要配置它来使用代理。
- 要使用的应用程序不处理 SOCKS 代理，那么可以尝试使用 tsocks包 或 proxychains-ng包。

在Firefox中，你可以在菜单 *偏好* > *网络* > *设置*中使用SOCKS代理。选择*Manual Proxy Configuration*，然后设置SOCKS Host（只有这个，确保其他字段，如HTTP Proxy或SSL Proxy为空）。例如，如果SOCKS5代理运行在localhost 8080端口上，则在SOCKS Host字段输入`127.0.0.1`，在Port字段输入`8080`，然后验证。

如果使用*proxychains-ng*，配置将在`/etc/proxychains.conf`中进行。只需要取消最后一行的注释（默认设置为使用Tor），并将其替换为SOCKS代理的参数。例如，如果你使用的是和上面一样的SOCKS5代理，你就必须用以下方式替换最后一行：

```
socks5 127.0.0.1 8080
```

然后可以通过*proxychains-ng* 启动程序

```
$ proxychains program
```

其中`*program*`可以是您系统中已经安装的任何程序（例如xterm、gnome-terminal等）。

如果使用*tsocks*，配置将在`/etc/tsocks.conf`中进行。参见tsocks.conf(5)中的选项。一个最少参数配置的例子是这样的。

```
/etc/tsocks.conf
```

```
server = 127.0.0.1
server_port = 8080
server_type = 5
default_user = ""
default_pass = ""
```

curl 和 pacman

你可以设置 `all_proxy` 环境变量，让 curl 和 pacman (使用 curl) 使用你的 socks5 代理。

```
$ export all_proxy="socks5://your.proxy:1080"
```

GNOME3设置代理

一些程序，如Chromium和Firefox可以使用GNOME存储的设置。这些设置可以通过 gnome-control-center 前端和*gsettings*来修改。

```
gsettings set org.gnome.system.proxy mode 'manual'
gsettings set org.gnome.system.proxy.http host 'proxy.localdomain.com'
gsettings set org.gnome.system.proxy.http port 8080
gsettings set org.gnome.system.proxy.ftp host 'proxy.localdomain.com'
gsettings set org.gnome.system.proxy.ftp port 8080
gsettings set org.gnome.system.proxy.https host 'proxy.localdomain.com'
gsettings set org.gnome.system.proxy.https port 8080
gsettings set org.gnome.system.proxy.socks host 'proxy.localdomain.com'
gsettings set org.gnome.system.proxy.socks port 8080
gsettings set org.gnome.system.proxy ignore-hosts "['localhost', '127.0.0.0/8',
'10.0.0.0/8', '192.168.0.0/16', '172.16.0.0/12' , '*.localdomain.com' ]"
```

这个配置也可以通过proxydriverAUR包设置成在NetworkManager连接到特定网络时自动执行。

Microsoft NTLM 代理

在Windows网络中，NT LAN Manager (NTLM)是一套微软安全协议，它为用户提供认证、完整性和保密性。

来自AUR的cntlmAUR站在您的应用程序和NTLM代理之间，即时添加NTLM认证。你可以指定几个 "父"代理，Cntlm会一个接一个地尝试，直到有一个成功为止。所有经过认证的连接都会被缓存和重用，以实现高效率。

```
(NTLM PROXY IP:PORT + CREDENTIALS + OTHER INFO) -----> (127.0.0.1:PORT)
```

配置

根据需要更改`/etc/cntlm.conf`中的设置，除了密码。然后运行。

```
$ cntlm -H
```

这将根据您的代理主机名、用户名和密码生成加密的密码哈希。

警告：ettercap包 当使用纯文本密码而不是加密哈希时，可以很容易地通过局域网嗅探你的密码。

再次编辑`/etc/cntlm.conf`并包含所有三个生成的哈希值，然后启用enabled。`cntlm.service`。

测试配置可使用一下指令:

```
$ cntlm -v
```

使用方法

使用 127.0.0.1:*port* 或 localhost:*port* 作为代理地址。*port*匹配/etc/cntlm.conf中的Listen参数，默认为3128。

互联网应用

本章处理的是浏览器之外的一批互联网应用：社交客户端 Mikutter 与 Twister、去中心化站点 ZeroNet、Google Earth、电骡 AMule、双向同步工具 Unison，以及 Tor 的网页浏览与即时通信。它们的共同难点不在安装，而在装完之后配置文件、服务单元与图形前端如何配合起来。

Tor 与 Unison 建议读完：前者覆盖配置、用法与网页浏览、即时通信两条完整链路；后者把版本不兼容性和小技巧讲得最细，出问题时最管用。AMule 与 Twister 按需查阅，Mikutter、ZeroNet、Google Earth 只取安装与故障处理部分即可。最容易踩的坑是把安装成功当成配好可用——AMule 还要单独配置服务与 amulegui，Unison 两端版本不一致会直接失败。

4.1　Mikutter

难度 入门 | **篇幅** 1.8 KB | **所属** 互联网应用

本节将讲到

- 安装
- 配置
- 参阅

mikutter 是由 toshi_a 桑开发的一个开源的 Twitter 客户端. mikutter 的开发使用了 Arch Linux (简体中文) 和 ruby (其中GUI使用了 GTK).(我爱miku)

安装

mikutter 可以通过AUR来安装。同时你应该安装ruby包.

```
$ yaourt -S mikutter
```

配置

插件

mikutter可以通过安装插件来扩展机能(比如煮咖啡←雾)。 要安装插件的话, 请放置 `~/.mikutter/plugin`下的`.rb`文件, 创建插件名的目录, 并且在那个目录中对名为插件名`.rb`的文件进行设置.

miku商店 : 插件管理。

参阅

- mikutter 公式サイト
- mikutter news
- Writing mikutter plugin
- mikutter開発日記

4.2 Twister

难度 入门 | **篇幅** 2.6 KB | **所属** 互联网应用

本节将讲到

- 安装
- 准备
- Gui
- JSON/CLI

Twister twister是一个还在测试阶段的P2P微博软件。

安装

AUR中有 twister-core-gitAUR 和 twister-html-gitAUR这两个包。

准备

启动伺服机：

```
# systemctl start twister
```

让twister随系统启动：

```
# systemctl enable twister
```

这会同时加载twister-core daemon和twister-html gui.

然后尝试进入: http://127.0.0.1:28332/home.html.

进入“网络”（network）选项卡，然后等待块链（blockchain）下载完成。 进入“注册”（setup account）选项卡，注册。等待你的注册完毕，然后进行你的个人设置，你的设置需要被另一个块接收，所以，请耐心等待“保存”（save）按钮的出现。

Gui

Twister-html包含一个在web上的GUI交互界面，你可以访问从而完全操纵你计算机上的twister；你也可以发推（post）和收推。

JSON/CLI

twister有基于JSON-RPC的CLI界面，但是这基本上使用来debug或者develop的，详细介绍请看，以下给出此页面的一些摘要

创建用户

此命令创建用户：

```
# twisterd createwalletuser myname
```

此命令将用户传播到Twister网络上：

```
# twisterd sendnewusertransaction myname
```

发推和看推

```
# twisterd newpostmsg myname 1 "hello world"
# twisterd getposts 5 '[{"username":"myname"},{"username":"myfriend"}]'
```

私信

```
# twisterd newdirectmsg myname 2 myfriend "secret message"
# twisterd getdirectmsgs myname 10 '[{"username":"myfriend"}]'
```

设置管理

```
# twisterd dhtput myname profile s '{"fullname":"My Name","bio":"just another
user","location":"nowhere","url":"twister.net.co"}' myname 1
# twisterd dhtget myfriend profile s
```

帮助

```
# twisterd help
```

4.3 ZeroNet

难度 入门 | **篇幅** 2.8 KB | **所属** 互联网应用

本节将讲到

- 安装
- 配置
- 创建 ZeroNet 站点
- 参阅

ZeroNet 可以“利用比特币加密和 BitTorrent 网络创建公开、自由和不受审查的网站”。

安装

安装 zeronetAUR 软件包。

最新的开发版是 zeronet-gitAUR 包。

配置

启动

启动 ZeroNet： 启动/启用 `zeronet.service`。

Tor

默认情况下，ZeroNet 使用 clearnet 和 Tor（如果可用）。要启用对 Tor 的支持需要安装 Tor（tor包）。然后执行下列命令让 ZeroNet 使用 Tor：

```
# usermod -a -G tor zeronet
# mkdir -m 750 /var/lib/tor-auth
# chown tor:tor /var/lib/tor-auth
```

在 `/etc/tor/torrc` 中添加下列行：

```
ControlPort 9051
CookieAuthentication 1
CookieAuthFileGroupReadable 1
CookieAuthFile /var/lib/tor-auth/control_auth_cookie
```

创建 ZeroNet 站点

所有操作，包括 ZeroNet 站点文件的编辑，都应该以 `zeronet` 用户执行。使用 `--config_file` 选项可以指定要使用的配置文件。`/var/lib/zeronet` 中默认的数据文件目录是`/var/lib/zeronet`。示例:

```
$ sudo -u zeronet python2 zeronet.py --config_file /etc/zeronet.conf
```

或

```
$ sudo su - zeronet
$ cd /opt/zeronet
$ python2 zeronet.py --config_file /etc/zeronet.conf
```

创建的站点的初始数据位于 `/var/lib/zeronet/[address]` 中，关于创建 Zeronet 站点的更多信息，请阅读 Zeronet FAQ.

参阅

- ZeroNet 文档

4.4 Google Earth

难度 入门 | **篇幅** 5.0 KB | **所属** 互联网应用

本节将讲到

- 安装
- 故障处理

来自项目网站：:

"Google Earth可以让您在一个虚拟的地球上随意的旅行并观看卫星影像,地形,地图,3D建筑等元素. 随着Google earth中地理内容的丰富, 您将能探索更真实的世界，并能够飞到您最喜欢的地方, 搜索商户或者只是用于导航。"

安装

AUR中已提供了Google earth 的安装包:

- 最新版本 - google-earthAUR\[损坏的链接：package not found\]
- 之前的版本 - google-earth6AUR\[损坏的链接：package not found\]

故障处理

提示：参见 官方支持社区 来查阅本段中未包括的内容。

启动时出现错误

Google Earth可能会因多种错误而不能启动，下面列出了几个常见错误和错误的处理方法。

Startup tips 被启用

即使startup tips非常有用，但是它被人所更为了解的原因是startup tip容易造成软件的崩溃，所以请使它无效化，把以下字段加入 `~/.config/Google/GoogleEarthPlus.conf`中:

```
[General]
enableTips=false
```

`~/.drirc`文件的结尾没有换行

```
$ echo >> ~/.drirc
```

缓存文件损坏

如果缓存文件已损坏并需要重建,请移除它:

```
$ rm -r ~/.googleearth/Cache/
```

Another crash happened while handling crash!

您可以尝试移除以下文件:

```
$ rm -f ~/.googleearth/Cache/cookies ~/.googleearth/instance-running-lock
```

占用的内存过多

您可以在 *Tools* > *Options* > *Cache*中调整最大缓存限制来控制内存的使用大小，或者您可以在*3D View*页面中通过降低画面质量来降低内存负荷。

Panoramio上的照片无法使用

在Google Earth forums中有几个针对不同原因所给出的不同解决方案 。 如果在google-earthAUR\[损坏的链接：package not found\]中的PKGBUILD设置 `_attempt_fix` 变量仍不起作用，回滚到旧版的google-earth6AUR\[损坏的链接: package not found\] 可能会解决这个问题.

地球除了黄色的边框以外什么都不显示

根据更新日志7.0.3这个问题已得到大幅改善，但是这个错误仍然会存在:

这个问题可能会出现在使用`Intel`系列`GPU`的`Linux`用户中

解决方法同样适用于 google-earth6AUR\[损坏的链接：package not found\]。

4.5 AMule

难度 入门 | **篇幅** 5.1 KB | **所属** 互联网应用

本节将讲到

- 安装
- 服务
- 配置
- amulegui
- 使用 Docker 安装

aMule 是一个类似于 eMule（电骡）的客户端，用于 eDonkey 网络和 Kademlia 网络，支持多平台。

安装

安装软件包 amule包。

amuled 是 aMule 的后台守护进程，无界面但功能齐全。其前端有 GTK 的 aMuleGUI、网页版的aMuleWeb、命令行的 aMuleCmd。

服务

软件包提供了两个 *systemd* 服务： amuled 和 amuleweb。先进行配置，设置外部访问的密码和 `amuleweb` 管理员密码，然后按照需要启动/启用 `amuled` 和 `amuleweb` 服务。

amulweb启动后可以在本机通过 `[http://127.0.0.1:4711](http://127.0.0.1:4711)` 访问（如果需要外部访问，则要更改监听地址），默认的管理员密码是 **amule**。

配置

软件安装时会创建用户**amule**，运行 systemd 服务时会使用此用户。

配置文件和临时文件位于 amule 的主目录`/var/lib/amule`：

- amuled 的配置位于 `/var/lib/amule/.aMule/amule.conf`
- amuleweb 的配置位于`/var/lib/amule/.aMule/remote.conf`

安装时 pacman 会生成一个带外部访问密码的 amule.conf 文件，amuleweb 配置文件也使用相同的密码。外部配置工具可以使用此密码远程访问。要重新生成密码，可以使用：

```
$ echo -n <your password here> | md5sum | cut -d ' ' -f 1
```

生成密码后，通过 **\[ExternalConnect\]** 参数设置。

```
/var/lib/amule/.aMule/amule.conf
[ExternalConnect]
AcceptExternalConnections=1
ECPassword=<encrypted password>
```

注意 `/var/lib/amule` 下的全部文件所有者应当是 **amule** 用户.

```
# chown amule:amule -R /var/lib/amule
```

amuleweb

注意：较之amulegui，amuleweb功能单薄，输出的下载信息也少，而且经常要求输入密码（让浏览器记住密码会好一些）。基于以上原因，建议使用amulegui，并忽略本节。

创建配置文件

还是使用之前配置amuled时的那个新用户，启动amuleweb以初始化配置文件：

```
$ amuleweb --write-config --password=<这里是密码> --admin-pass=<这个是网页登录密码>
```

<这里是密码>处填写之前配置 amuled 使用的密码（未加密的），<这个是网页登录密码>处填写登录网页界面时输入的密码。

提示：如果 Kad nodes.dat 用的默认 URL 无法连接，可以使用在 获取 URL.

amulegui

Amulegui 是 aMule 的 GTK+ 前端。

配置通知

Settings → Events 包含自动触发的命令. 核心命令是 *notify-send* (需要安装 libnotify 包)，可以用 amule 参数设置通知。例如在 *Download completed* 中设置如下值会在下载完成后显示下载大小：:

```
$ notify-send -i amule "%NAME completed (%SIZE bytes)"
```

"-i amule" 选项是用来设置 amule 图标的（如果想自定义，把图标名 amule 换成自定义的文件，路径用斜线隔开）。

使用 Docker 安装

如果你在使用 docker包，安装和配置会很简单，用下面模板启动容器

```
$ sudo docker run -d \
--name=amule \
-e PUID=1000 \
-e PGID=1000 \
-e TZ=Asia/Shanghai \
-p 4711:4711 \
-p 4712:4712 \
-p 4662:4662 \
-p 4665:4665/udp \
-p 4672:4672/udp \
-v <fill_amule_configuration_path>:/home/amule/.aMule \
-v <fill_amule_completed_downloads_path>:/incoming \
-v <fill_amule_incomplete_downloads_path>:/temp \
--restart unless-stopped \
ngosang/amule
```

把 **fill_amule_configuration_path** 替换为配置路径，通常是 */home/you/.config/amule*，把 **ill_amule_completed_downloads_path**, **fill_amule_incomplete_downloads_path** 换成宿主机上的目录（要存放下载文件的目录绝对路径）。

如果容器成功启动了，会运行 **amuled** 和 **amuleweb** 在里面，使用这个命令查看生成的密码

```
$ sudo docker logs amule
Creating group amule with GID 1000 ...
```

```
Creating user amule with UID 1000 ...
Remote GUI password: Passwd1xxxxxxxxxxxxxxxxxxxxxxxx
Web UI password: Passwd2xxxxxxxxxxxxxxxxxxxxxxxx
/home/amule/.aMule/amule.conf file NOT found. Generating new default configuration ...
/home/amule/.aMule/amule.conf successfullly generated.
```

接着打开 `[http://127.0.0.1:4711](http://127.0.0.1:4711)` 并输入 *Web UI password*。

参阅

- Getting_Started at aMule wiki。

4.6 Unison

难度 进阶 | **篇幅** 6.9 KB | **所属** 互联网应用

本节将讲到

- 安装
- 配置
- 使用
- 版本不兼容性
- 提醒与小技巧

Unison是一款可以在类UNIX操作系统(包括 Linux, Mac OS X, 和Solaris) 和Windows环境下运行的双向文件同步工具。他可以把一个文件或目录的两个备份分别储存在两个不同的主机(或同一个主机的不同的磁盘上)，分别修改，并且通过把双方的改变传递到对方来完成同步。同时，他也不限制哪一方做主机。

安装

请从official repositories 安装 unison包, 那里有提供 CLI, GTK+ 和 GTK+ 2.0 接口. 如果有线下文档需求的话，请从 AUR 安装 unison-docAUR\[损坏的链接：package not found\].

配置

为了使用Unison，你需要创建一份配置文件.

GUI

如果想在GUI环境下配置的话 请运行:

```
$ unison-gtk2
```

手动操作

或者，在`~/.unison`里手动创建配置文件，并且将接下来的几行加入到默认配置文件`~/.unison/*profilename*.prf`里。

为被同步文件定义根目录

```
root=/home/user/
```
定义一个远程目录，文件将被同步到那里

```
root=ssh://example.com//path/to/server/storags
```
为SSH提供参数(可选)

```
sshargs=-p 4000
```
定义同步哪些文件和目录:

```
# dirs
path=Documents
path=Photos
path=Study
# files
path=.bashrc
path=.vimrc
```
你还可以定义无视哪些文件:

```
ignore=Name temp.*
ignore=Name .*~
ignore=Name *.tmp
```
注意：如若需要更多咨询请看 User Manual and Reference Guide中的 Sample profiles。

使用

设定好配置文件以后就可以开始同步了:

```
$ unison profilename
```
如果你选择使用GUI工具的话就这么干:

```
$ unison-gtk2
```
然后选择配置文件。Unison的界面很赞，你可以看到变动和进度。

版本不兼容性

如果你希望Unison能正常工作的话，请确保每一个客户端上安装的版本都是一样的。举个例子，如果一套电脑上安装了2.40版本而另外一台上安装的是2.32，那他们就没法互相同步了。这对于所有全部一切你希望进行同步作业的计算机都适用。

由于Linux发行版数目众多，Unison的release错综复杂，所以你很有可能会陷入老版本的泥潭。Arch Linux在上游的Extra repository里提供有最新版本的Unison。同时在 AUR 有非官方的 2.32版本 (unison-232AUR\[损坏的链接：package not found\])和2.27版本(unison-227AUR\[损坏的链接：package not found\]) 的 PKGBUILD，这样各种发行版的用户们都可以在他们的系统上愉快的使用Unison啦

提醒与小技巧

人生苦短，少敲键盘

如果哪位在一个有能力维持一个合适的scrollback buffer的VDT环境下运行Unison的话，他就没有确认每一个无冲突改变的必要了；设定 `auto` 为 true 来避开这些提示。

常规配置文件同步

在同步那些在不同的系统(比如服务器，工作站，笔记本，智能手机)但却含有通用构造(比如键盘映射，基本shell同义名)的配置文件(比如针对本地程序，对安全性敏感的配置)时，最好把这些内容放到分散的配置文件中(比如`.bashrc_common`)并且只对他们进行同步。

警告：通过让被同步方(甚至也许还包括其他与被同步方同步的机器)的配置文件接受恶意的篡改，配置文件的双向同步可能会打开一条入侵的光明大道。这对于对手来说很有吸引力，尤其是在比如公共shell服务器vs个人工作站这样双方"实力悬殊"的情况下，因为要瓦解一个安全等级更低的系统看起来真不是什么难事。你并不需要在两台特定的机器间进行双向同步时一直使用`noupdate`；如果真的有这个必要，请在同步时确认每一处变动。在对待自动的双向同步行为的时候,当心点。

参阅

- Unison (file synchronizer)
- Official website
- Yahoo! user group\[失效链接 2021-05-17 ⓘ\]
- *Liberation through data replication\[失效链接 2021-05-17 ⓘ\]* by Philip Guo
- *Setting up Unison for your mom\[失效链接 2021-11-19 ⓘ\]* by Philip Guo
- *Replication using Unison* on TWiki

4.7 Tor

难度 进阶 | **篇幅** 17.3 KB | **所属** 互联网应用

本节将讲到

- 介绍
- 安装
- 配置
- 用法
- 网页浏览

**这篇文章或章节的翻译质量不佳。**

原因：No updates since 2020, still references `rc d`.（在 Talk:Tor# 中讨论）

翻译状态：

- 本文（或部分内容）译自 Tor，最近一次同步于 2020-04-02，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

注意：本文正在一个临时页面重译，请前往该临时页面查看进度。

Tor 是一个开源实现，第二代洋葱路由匿名代理网络，提供免费访问。其首要目标是通过对流量分析攻击保护，使网上的匿名性。

介绍

使用Tor网络在他们的机器上运行洋葱代理。这个软件向外连接到Tor，定期的通过Tor网络连接一个虚拟的电子回路。Tor在一个分层方式采用加密(因此有了'洋葱'的比喻), 确保路由器之间的完全保密。 同时， 洋葱代理服务器软件，向客户端提出了SOCKS接口。SOCKS监听程序是Tor的特点，通过Tor的虚拟电路的流量，然后复。

警告：仅仅使用Tor不能保证匿名。这里可以看出几个主要的误导 (see:)。

通过这个过程洋葱代理将会管理终端用户的匿名网络流量。它通过加密流量来保持用户的匿名， 通过Tor的其他节点来发送信息，在发送信息到你指定的服务器之前解密。虽然Tor相对的来说比一般的使用DNS目录连接安全(例如：没有使用代理), 由于大量的数据加密所以它相对来说比较慢。 此外，尽Tor阻止了流量分析但是Tor不能阻Tor网络边界流量的确认。(例如：例如流量进入或者退出网络)。

Wikipedia:Tor (anonymity network))

安装

安装位于官方软件仓库的软件包 tor包。

此外，还可以安装 Qt 前端 vidaliaAUR。除了控制 Tor 进程外，Vidalia 可以查看 Tor 的配置和状态、监控带宽使用并查看、过滤和搜索日志信息。如果喜欢使用命令行，并能获得和 Vidalia 一样的效果，可以使用 nyx包

配置

为了更好的理解Tor`/etc/tor/torrc`配置文件. 配置选项在 man tortor(1)和Tor website有介绍.

默认配置应该能够很好的为大多数Tor用户服务除了哪些是用Vidalia的用户, Vidalia是一Tor的图形界面。 在AUR有可用包vidaliaAUR。除了控制过程中的Vidalia允许您查看Tor状态监视带宽使用情况，查看，过滤，搜索日志消息;和配置Tor的某些方面。

使用`TOR_MAX_FD`变量，您可以设置自定义文件为Tor的描述 ulimits`/etc/conf.d/tor`。

默认情况下的Tor记录在日志级别的通知"STDOUT"。如果你能够记录在自己的文件 `torrc`, 他们默认为 `/usr/local/var/log/tor/`.

中继配置

中继 (relay) 是 Tor 网络中重要一环，负责数据传输和隐藏真实 ip 地址。如果没有配置 Tor 配置文件中的 `AccountingMax` 且没有布置 Web 服务器，应该考虑修改你的洋葱路由端口 (ORPort) 为 `443`，因为防火墙会阻止 `tor`，只能用于访问网页。修改后可以绕过防火墙，也可以考虑 `22` , `110` , `143` 和 `9001` 端口。由于低于1024端口属于特级端口，需要在 `tor.service` 中设置 `User=root` 和 `/etc/tor/torrc` 中设置 `User tor` 以 root 权限运行。realy setup

启用 Tor ControlPort

大多数用户是不需要这个功能的，但一些应用程序需要这个功能来实现对 Tor 节点的低级访问。 通过 ControlPort，其他应用程序可以更改和查看 Tor Node，在 Tor 运行时修改配置文件和查看 Tor 网络。

添加到文件

```
ControlPort 9051
```

即可启用

警告：由于 ControlPort 会修改 Tor 配置文件，所以一般情况下不要启用，需要启用时应搭配 `cookie` 文件或控制密钥来使用，且只对受信任的程序或用户提供!

关于配置 Tor ControlPort，请参考设置 cookie 文件

用法

可以通过命令行或 **vidalia** 等图形程序启动 Tor.

要永久连接，可以启动 **tor** daemon 并将其加入 `DAEMONS` 数组。

程序配置成实验 127.0.0.1 或 localhost 作为 SOCKS5 代理就可以使用 Tor，端口要设置成 9050 (Tor 标准设置) 或 9051 (用 **vidalia**标准配置).

访问 Tor、Harvard\[失效链接 2021-05-17 ⓘ\] 或 Xenobite.eu 可以检查 Tor 是否正确运行。

网页浏览

Tor主要支持Firefox，但是也支持Chromium。

Firefox

如果你使用Firefox, 你可以安装这个插件: TorButton。这将允许你很容易地在Tor的网络和正常网络切换。

如果你正在使用多代理(例如：如果你想使用 TOR 和 "ssh -D") 也有一个插件叫作 "FoxyProxy" 他允许你对于不同网址或者全部网址指定多个代理。只需将它指向localhost上的端口8118（polipo运行中）。

为了测试他, 在你的浏览器打开或者关闭Tor浏览这个网址Tor check。 更多信息请查看这个网址the official doc。

Chromium

当你使用Tor和Chromium时不需要polipo。只需要简单的运行Tor daemon,

然后运行：

```
$ chromium --proxy-server="socks://localhost:9050"
```

即时通信

要让即时通信程序使用 Tor，并不需要 polipo/privoxy 等 http 代理。直接使用 tor 守护进程监听的端口 9050 即可。

Pidgin

通过 preferences -> proxy 进入编辑，设置为：

```
Proxy type    SOCKS5
Host           127.0.0.1
Port           9150
```

之后 pidgin 会使用 Tor 进行通信。有时根据不同账号的 IM 服务配置，需要修改代理设置。在 Accounts -> Manage Accounts 中选择要修改的账号，在 Proxy 标签页中选择：

```
Proxy type    Use Global Proxy Settings
```

Irssi

Freenode 不推荐使用 Irssi 和 Privoxy。他们推荐 `mapaddress` 方式，通过运行 `torify irssi` 启动。将下行加入 `/etc/tor/torrc`:

```
mapaddress  10.40.40.40 p4fsi4ockecnea7l.onion
```

Freenode 需要 charybdis 和 ircd-seven 的 SASL 机制在连接时进行 nickserv 确认。从 Freenode 网站(即 https://web.archive.org/web/20150423190608/http://www.freenode.net/sasl/cap_sasl.pl) 下载启用 SASL 的 `cap_sasl.pl`，保存到 `~/.irssi/scripts/cap_sasl.pl`

用 pacman 安装需要的 perl 模块和 AUR 中的 perl-crypt-dhAUR\[损坏的链接：package not found\].

```
$ pacman -S perl-crypt-openssl-bignum perl-crypt-blowfish
```

也可以通过 perl 直接安装:

```
$ perl -MCPAN -e 'install Crypt::OpenSSL::Bignum Crypt::DH Crypt::Blowfish'
```
启动 irssi

```
$ torify irssi
```
加载使用 SASL 机制的脚本：

```
/script load cap_sasl.pl
```
将身份设置到 nickserv，连接时会读取这个值，支持的机制是 PLAIN 和 DH-BLOWFISH。

```
/sasl set <network> <username> <password> <mechanism>
```
连接到 Freenode:

```
/connect -network <network> 10.40.40.40
```
如果遇到问题，请访问 Arch 论坛上的 Cannot Connect to Freenode IRC using Irssi & Tor。

外部链接

- Accessing freenode Via Tor
- SASL README
- IRC/SILC torproject.org 上的 Wiki

使用 HTTP 代理

如果你需要一个 HTTP 代理。

注意：现在 Tor 开发组推荐在 Privoxy 上使用 Polipo.

Privoxy

Privoxy 是一个 HTTP 代理，它使用 SOCKS4a 代理进行 html/cookie 过滤。可以安装 Privoxy 文章安装。

在Firefox使用Tor 和 Privoxy

最简单的方法就是使用Torbutton 扩展。

或者, 你可以使用Foxyproxy。然后重启Firefox你就会发现一个新的工具条。 点*Add*, 选*Standard proxy type*. 选 你要的*Proxy Label* , 例如*Tor*。进入“HTTP Proxy”和“SSL Proxy”区域：

```
Hostname: 127.0.0.1 Port: 8118
```
然后Firefox将会用代理连接。你也可以在*No Proxy for* 添加例外。

现在,返回 https://whatsmyip.net/ 检查你的ip地址是否和以前不同了。

在其他程序使用Tor 和Privoxy

你可以在像即时通信, Jabber, IRC,等软件使用这个方法。

你可以指定代理(127.0.0.1 port 8118)在那些支持HTTP代理的软件里面。

如果使用SOCKS 代理，你可以指定你的程序到Tor (127.0.0.1 port 9050)。用这种方法的一个问题是，虽然自己做DNS解析的应用程序可能会泄漏信息。

你可以考虑使用Socks4A (e.g. via privoxy)来取代他。

运行Tor 服务

配置

你的带宽必须至少20kb/s:

```
Nickname <tornickname>
ORPort 9001
BandwidthRate 20 KB             # Throttle traffic to 20KB/s
BandwidthBurst 50 KB            # But allow bursts up to 50KB/s
ExitRelay 0                   # Disallow exits from your relay
```

允许 Irc 在 6660-6667 端口输出：

```
ExitPolicy accept *:6660-6667,reject *:* # Allow irc ports but no more
```

Tor 作为输出节点:

```
ExitPolicy accept *:119         # Accept nntp as well as default exit policy
```

Tor 作为中间人：

```
ExitPolicy reject *:*
```

运行 Tor 网桥

Tor 网桥是旨在 ISP 运营商或政府阻止访问中继 (relay) 时提供的连接方案，通过访问bridges来了解网桥的工作原理。

需要使用网桥时需在 `/etc/tor/torrc` 添加一下4行：

```
/etc/tor/torrc
SOCKSPort 0
ORPort 443
BridgeRelay 1
ExitRelay 0
```

TorDNS

Tor 0.2.x系列提供了一个内置的DNS转发器。在Tor配置文件添加如下文件来启动它。

```
/etc/tor/torrc
 DNSPort 9053
 AutomapHostsOnResolve 1
 AutomapHostsSuffixes .exit,.onion
```

然后重新启动 Tor 来加载更新过的配置文件：

```
/etc/rc.d/tor restart
```

这将让Tor接受DNS请求(在这个例子里面监听着9053端口)。并通过Tor网络域名。一个缺点是，它是仅能够解决一个记录的DNS查询； MX 和NS 请求没有回应。

更多信息请查看 Debian-based introduction。

DNS查询，也可以通过命令行`tor-resolve`来查询。例如：

```
$ tor-resolve archlinux.org
66.211.214.131
```

Torsocks

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

torsocks包 will allow you use an application via the Tor network without the need to make configuration changes to the application involved. From the man page:

torsocks is a wrapper between the torsocks library and the application in order to make every Internet communication go through the Tor network.

Usage example:

```
$ torsocks elinks checkip.dyndns.org
$ torsocks wget -qO- https://check.torproject.org/ | grep -i congratulations
```

Torify

`torify` 允许你的程序不需要更改配置来通过访问Tor网络。

man page:

```
 torify  is  a simple wrapper that calls tsocks with a tor specific configuration file.

 tsocks itself is a wrapper between the tsocks library and the  application  that
 you would like to run socksified
```

使用例子：

```
$ torify elinks checkip.dyndns.org
$ torify wget -qO- https://check.torproject.org/ | grep -i congratulations
```

Torify 不会通过Tor网络来查询DNS。其中一种解决方法就是和`tor-resolve` 来解决(前文所述)。在这种情况下, 上面的例子程序将像这样

```
$ tor-resolve checkip.dyndns.org
208.78.69.70
$ torify elinks 208.78.69.70
```

问题解决

User value的问题

如果*tor* daemon启动失败，你可以在root环境下运行一下命令(或者使用sudo)

```
# tor
```

如果你收到以下 error

```
May 23 00:27:24.624 [warn] Error setting groups to gid 43: "Operation not permitted".
May 23 00:27:24.624 [warn] If you set the "User" option, you must start Tor as root.
May 23 00:27:24.624 [warn] Failed to parse/validate config: Problem with User value.
See logs for details.
May 23 00:27:24.624 [err] Reading config failed--see warnings above.
```

它意味着你的User value有问题。通过以下的步骤解决：

运行以下命令检查`/var/lib/tor`目录的权限

```
# ls -l /var/lib/
```

如果`/var/lib/tor`权限显示如下

```
drwx------ 2 tor    tor    4096 May 12 21:03 tor
```

这意味着它被*tor*用户和 *tor*组所拥有

通过以下命令把拥有者和组改为*root*, *root*

```
# chown -R root:root /var/lib/tor
```

如果你重新检查权限,他现在应该显示为

```
drwx------ 2 root   root   4096 May 12 21:03 tor
```

现在打开 `/etc/tor/torrc`找到以下文字

```
## Uncomment this to start the process in the background... or use
## --runasdaemon 1 on the command line.
RunAsDaemon 1
User tor
Group tor
```

注释掉*User tor* 和*Group tor*, 所以他应该显示为

```
## Uncomment this to start the process in the background... or use
## --runasdaemon 1 on the command line.
RunAsDaemon 1
#User tor
#Group tor
```

保存更改然后重启*tor* daemon, 他应该能够工作了

```
# /etc/rc.d/tor restart
```

外部链接

- Official Website
- “如何翻墙”系列：关于 Tor 的常见问题解答 @ 编程随想的博客
- Unix-based Tor Articles
- Software commonly integrated with Tor
- How to set up a Tor *Hidden Service*
- 新中继的生命周期
- 贡献服务器作为 Tor 节点
- Running a bridge

即时通信

聊天软件在 Arch 上往往没有官方包，或官方只覆盖少数平台，得在原生版本、虚拟机、Wine 与第三方客户端之间挑一条路。本章按这几条路给出装法，并把装不上、装上却不好用的排查一并写清。

先读《腾讯 QQ》与《微信》：两节篇幅最长，Linux 版、虚拟机、Wine、第三方客户端到疑难解答几条路都讲到了，其余各节照此类推即可。要自建服务的再读《TouchFish》，《Telegram》的提示和技巧、《Discord》的故障排除遇事再查，《Koreader》与《WhatsApp》只有特点与安装，可略。最容易踩的坑是把缺官方 Linux 版当成用不了，一条路不通便止步，或装完不收尾、跳过疑难解答。

5.1　Koreader

难度 入门 | **篇幅** 1.9 KB | **所属** 即时通信

本节将讲到

- 主要特点
- 安装

KOReader 是一个主要针对电子墨水阅读器的文档查看器。

主要特点

便携：在嵌入式设备（Cervantes、Kindle、Kobo、PocketBook、reMarkable）、Android 和 Linux 计算机上运行。开发人员可以在 Linux 和 MacOS 中运行 KOReader 模拟器。

多格式文档：支持固定页面格式（PDF、DjVu、CBT、CBZ）和可重排的电子书格式（EPUB、FB2、Mobi、DOC、CHM、TXT）。扫描的 PDF/DjVu 文档也可以使用内置的 K2pdfopt 库进行重排。

全功能阅读：多语言用户界面，具有高度可定制的阅读器视图和许多排版选项。您可以设置任意页边距、覆盖行间距并选择外部字体和样式。它具有捆绑到应用程序中的多语言连字词典。

与calibre包（搜索元数据、无线接收电子书、通过 OPDS 浏览图书馆）、Wallabag 、维基百科、谷歌翻译和其他内容提供商集成。

针对电子墨水设备进行了优化：没有动画的自定义 UI、分页菜单、可调节的文本对比度以及轻松缩放以适应分页媒体中的内容或页面。

可扩展：通过插件

快速：在一些较旧的设备上，经测量，它的翻页延迟不到内置阅读软件的一半。

还有更多：使用 StarDict 词典/维基百科查找单词，添加您自己的在线 OPDS 目录和 RSS 提要、无线软件更新、FTP 客户端、SSH 服务器……

请查看用户指南和wiki以发现更多功能并帮助我们记录它们。

安装

目前来说想要在arch安装koreader有两种选择

koreader-bin

koreader-binAUR 是直接用koreader官方提供的deb包重新打包所得的包。

koreader-appimage

koreader-appimageAUR 则是直接用koreader官方的appimage打包所得。

5.2 WhatsApp

难度 入门 | **篇幅** 1.9 KB | **所属** 即时通信

本节将讲到

- 安装

WhatsApp 是 Facebook 拥有的移动优先即时消息服务，具有端到端加密功能。它需要有效的手机号码和数据连接才能运行。

没有适用于 Linux 的官方 WhatsApp 桌面客户端，Facebook 严格尝试禁止使用其协议的第三方客户端和插件。由于这种公司行为，我们建议人们避免完全使用 WhatsApp 来支持具有更开放性的 IM 服务，例如 XMPP，Telegram 或 ICQ。

安装

您可以使用以下方法之一来在 Arch 中使用 WhatsApp：

基于 Web 的客户端

- WhatsApp Web\[失效链接 2022-09-23 ⓘ\]，这是在 Linux 上使用 WhatsApp 的唯一官方方式。
- whatsapp-nativefierAUR，是使用 nativefier 构建的 WhatsApp Web 的桌面包装。
- franzAUR，这是一个基于 Chrome 的开源应用程序，可用于各种即时消息传递软件（包括 WhatsApp）的基于 Web 的界面。

- rambox-binAUR，类似于 Franz，它是基于 Web 的包装程序，用于各种即时消息传递软件（包括 WhatsApp）。

5.3 Discord

难度 入门 | **篇幅** 3.8 KB | **所属** 即时通信

本节将讲到

- 安装
- 故障排除

Discord 是一款专有的跨平台多合一语音和文本聊天应用程序。它是专门为游戏玩家量身定制的。但是，许多开源社区也有官方的 Discord 服务器。可以通过 Web 浏览器或通过 Electron 制作的桌面应用程序使用 Discord。

安装

您可以使用以下软件包之一来安装 Discord 的桌面应用程序：

图形客户端

官方应用：

- 稳定版：discord包
- 测试版：discord-ptbAUR
- 每夜构建版：discord-canaryAUR

第三方客户端：

- ripcordAUR，有关更多信息，请参阅 Ripcord 文章

命令行客户端

尽管目前只有少数几个打包在 AUR 上，但在 GitHub 之类的地方（例如 Discline 或 terminal-discord）上托管着更多基于 CLI 的第三方客户端。

聊天客户端插件

- 通过使用 purple-discord-gitAUR，您可以在基于 libpurple包 的图形或终端聊天软件（例如 Pidgin）上使用 Discord。
- 通过使用 bitlbee-discord-gitAUR，您可以通过 Bitlbee 使用 Discord。

自定义 CSS 和插件

通过 betterdiscord-gitAUR 软件包，可以将 Discord 修改为使用自定义 CSS 和插件。

请注意，使用这些软件包或修改 Discord 客户端的任何软件包或应用程序均违反 Discord 的服务条款，并可能会封禁您的帐户。使用这些或类似软件包时，您将承担全部责任。

故障排除

- 如果您在进行语音聊天时遇到啪啪作响的声音，则应尝试执行此处概述的步骤：PulseAudio/Troubleshooting
- 如果不能在多显示器设置上共享单个显示器，则应尝试使用 mon2cam-gitAUR 作为此错误的解决方法：Discord Trello
- 当使用 Flatpak 版本的 Discord 时，Rich Presence 将无法开箱即用。要使其正常工作，必须创建从 `$XDG_RUNTIME_DIR/discord-ipc-0` 到 `$XDG_RUNTIME_DIR/app/com.discordapp.Discord/discord-ipc-0` 的符号链接。要为当前用户会话创建符号链接，请运行：

```
$ ln -sf {app/com.discordapp.Discord,$XDG_RUNTIME_DIR}/discord-ipc-0
```

: 要自动创建符号链接，可以通过在 `~/.config/user-tmpfiles.d/` 中的扩展名为 `.conf` 的文件中添加以下行来使用 systemd-tmpfiles：

```
L %t/discord-ipc-0 - - - - app/com.discordapp.Discord/discord-ipc-0
```

提示：可以通过 `--start-minimized` 参数最小化 Discord。

5.4 微信

难度 进阶 | **篇幅** 8.0 KB | **所属** 即时通信

本节将讲到

- Linux 版
- 虚拟机
- Wine
- 统信 UOS 魔改版

微信（WeChat）是腾讯公司于 2011 年 1 月 21 日推出的一个为智能终端提供即时通讯服务的免费专有应用程序。

目前微信官方适配了部分 Linux 发行版（见下），如有兴趣可测试补充其他发行版上的兼容情况。

除安装原生版本外，本文还将介绍如何通过其他方法安装微信，以满足不同的需求。

Linux 版

提示：✓这是当前比较好的解决方案。

2024年3月，腾讯推出了基于原生跨平台方案的微信（Universal），旨在逐步实现微信 Windows、Mac 和 Linux 版本在功能和更新节奏上的一致性。腾讯官方提供了 rpm、deb 和 appimage 格式的安装包，但目前尚未实现更新源，因此 deb 和 rpm 用户无法通过包管理器实现自动更新。

微信原版打包于 *wechat-binAUR*, 并有包 *wechatAUR* 提供功能增强和修复。后者有进程管理，沙盒，输入法及 HiDPI 修复等功能。也可以使用 Flatpak 从 Flathub 安装 `com.tencent.WeChat`。

*wechat-universal-bwrapAUR*提供了基本沙盒封装的微信，它最初是基于现已不存在的 *wechat-uos-qtAUR*，因此它不依赖于*wechat-binAUR*。其使用Bubblewrap沙盒而非 portable 沙盒，不具备 D-Bus 过滤等功能，**有沙盒逃逸风险**。

沙盒

wechatAUR 可选启用 portable 沙盒（在2026.02.06之后的版本中沙盒为必选项，对于希望禁用沙盒的用户，您可能需要portable-legacyAUR）. 它有一些独特功能:

- 支持 GNOME 后台监视
- 简易的文件分享
- D-Bus 安全过滤
- 请注意: 截图功能仅在 GNOME 桌面环境可用, 因为一些桌面实现无须用户授权即可截图
- 为双显卡用户隐藏独立显卡, 避免唤醒耗电
- 自动输入法配置
- 隐藏一些系统和硬件信息

右键任务栏图标以访问实用功能:

- 打开或关闭沙盒
- 彻底退出微信

同时, 还可以使用环境变量 `bwBindPar`向沙盒暴露某个路径, 请注意这将会授予微信完全的读写权限.

wechatAUR 的沙盒需要 XDG 桌面门户正常工作. 正常来说, 使用桌面环境 (DE) 时应无须特别处理, 窗口管理器用户可能需要根据文档自行选择实现。

*wechat-universal-bwrapAUR*使用Bubblewrap沙盒，以 `~/Documents/WeChat_Data/home`为家目录，可以编写`~/.config/wechat-universal/binds.list`来设置更多的被暴露到容器内的文件/文件夹，每行一个路

径，要将微信文件的主要路径修改至`~/Documents/WeChat_Data`外的其他路径，请设置环境变量 `WECHAT_DATA_DIR`

输入法与 HiDPI

wechatAUR 将会自动设置环境变量以绕过此类问题. 未设置`XMODIFIERS` 变量可能导致探测错误.

在 *wechat-binAUR* 4.1.13.9 版本, 可能需要携带以下环境变量运行:

```
QT_IM_MODULE=text-input-unstable-v3
```

这样可以强制微信使用 `text-input-v3` 协议. 否则 Fcitx5 可能无法正常工作. 使用 wechatAUR 的用户可以直接将上述内容写入 `~/.local/share/WeChat_Data/portable.env` 中.

Emoji 字体显示问题

安装 ttf-twemojiAUR，并升级 *wechatAUR* 到 1.0.0.241-27 及以上版本。

虚拟机

可以在 VMware 或 VirtualBox 等虚拟机里安装 Windows 系统来使用微信。

Wine

注意： 目前 Wine-based 微信小程序功能全部不可用，大于一定尺寸的图片无法发送。详情请参阅 这里。

安装

兼容层 Wine 可用于在 Linux 中运行微信。

此外，deepin-wine-wechatAUR（无补丁）也提供了它，这是一个为 Arch 配置的 Wine 容器。版本为最新官方版本。

使用 archlinuxcn 仓库的 wine-for-wechat

如果微信在使用过程中在其他窗口之上有窗口阴影，可以尝试使用 archlinuxcn 仓库中的 *wine-for-wechat*。 此版本的 Wine 使用这个补丁修复微信窗口阴影。

另外，建议安装 wine-wechat-setupCNRepo 包来安装，运行和配置微信（微信的 Windows 安装包需要自行下载）。

deepin-wine 微信

可以安装 com.qq.weixin.deepinAUR软件包，这是 Deepin 官方配置的 Wine 化应用，但版本较旧 (3.2.1)。该版本的小程序功能不可用。

若需要更新的版本可选择安装 com.qq.weixin.sparkAUR或deepin-wine-wechatAUR。基本保持与官方 Windows 版本更新同步，且可以使用小程序。

Deepin-wine 还提供了其他的相关软件包，比如企业微信 com.qq.weixin.work.deepinAUR。

在容器中运行

docker-wechat 在一个 Docker 容器中运行 Wine wechat。

配置

高分辨率屏幕

如果屏幕 DPI 比较高，界面和字体会比较小。需要通过如下方式配置 wine 的屏幕分辨率 DPI:

1. 打开对应的 winecfg, 下面的是 deepin-wine-wechatAUR 的示例，请按实际情况修改 `WINEPREFIX` 和 `deepin-wine6-stable`:

```
$ WINEPREFIX=~/.deepinwine/Deepin-WeChat/ deepin-wine6-stable winecfg
```

1. 点击显示，调整*屏幕分辨率*为合适的数值。
2. 微信中，启用适配系统缩放比例选项(设置-->通用设置-->适配系统缩放比例)

统信 UOS 魔改版

安装 wechat-uosAUR 软件包即可。

注意：原统信 UOS 软件包会验证运行环境是否是统信 UOS 专业版，在非 UOS 专业版的环境下会工作异常。

本解决方案通过使用 bubblewrap 替换 `os-release` 和 `lsb-release`，在没有修改其分发的二进制文件的前提下，实现模拟 UOS 专业版运行环境，使其能正常运行。

此包功能较少，基本是微信网页版套壳，建议使用原生版本。

5.5 TouchFish

难度 进阶 | **篇幅** 10.4 KB | **所属** 即时通信

本节将讲到

- 安装
- 系统要求
- 启动和使用（v5）
- 启动服务端
- 启动（v4）

TouchFish 是一款轻量级的局域网（LAN）即时通讯工具，无需登录或添加好友即可在同一子网内创建聊天室。它支持服务端/客户端模式，适用于机房、办公室等内部网络环境。

安装

AUR

TouchFish 的 AUR 虚拟包名为 touchfishAUR，此包名拥有 3 个包 touchfish-clientAUR，touchfish-serverAUR 和 touchfish-v4AUR。

其中 client 和 server 指 v5，touchfish-v4 包同时具有客户端和服务器功能。

手动安装

访问 链接页 下载对应的版本。

系统要求

- **Linux**：无特殊限制，服务端建议使用源码版本而非二进制文件。
- **Windows**：Windows 10 或更新版本原生支持；Windows 8.1 及更早版本可能需要额外 DLL。若提示写入失败，请以管理员身份运行。
- **macOS**：需要 macOS 10.8 Mountain Lion 或更高版本，并允许“任何来源”的应用运行权限。

启动和使用（v5）

客户端（TouchFish Client）

登录和配置

首次启动 TouchFish Client 时，你需要配置服务器连接：

1. 在欢迎界面点击"配置服务器"
2. 输入你的 TouchFish 服务器地址
3. 选择服务器，出现对勾代表确认服务器可访问
4. 点击"保存"完成配置

登录时会显示 RSA 公钥指纹，为防止中间人攻击，请核对指纹是否一致，就像在 SSH 中那样。 配置完服务器后，你可以：

- **登录**：如果你已有账号，输入用户名和密码登录
- **注册**：点击"注册新账号"创建新用户

主界面

TouchFish Client 的主界面分为以下几个部分：

- **侧边栏**：显示联系人列表和聊天列表
- **聊天窗口**：显示当前对话内容
- **输入框**：发送消息和文件

应用设置

登录前点击右上角的设置图标可以访问。 登录后进入“账户”页面，选择“应用设置”访问。

发送消息

1. 在侧边栏选择一个联系人或群组
2. 在输入框中输入消息
3. 发送

发送文件

1. 点击输入框左侧的附件图标
2. 选择要发送的文件
3. 等待上传完成后发送

支持的文件类型：

- 图片（可预览）
- 视频（可预览）
- 音频（可预览）
- 文档
- 其他文件类型

查看图片和视频

- 点击聊天中的图片可以全屏查看，支持缩放与旋转
- 点击视频可以直接播放

发起通话

1. 打开与好友的聊天窗口
2. 点击右上角的**视频通话**按钮
3. 等待对方接听

媒体流点对点传输，不经过服务器。详细说明见语音视频通话。

服务端（Server）

创建新实例

```
python main.py --create-new-config
```

按向导输入 API 端口与 TCP 端口后，会创建完整的运行目录，包括：

- RSA 密钥对与 JWT 签名密钥（`res/<api_port>/secret/`）
- SQLite 数据库文件
- 默认头像和资源文件
- 服务器配置文件

配置会追加到 `server_config.json`，其编号用于后续启动。

服务端配置项

生成后可在 `res/<api_port>/config.json` 中修改配置。常用配置项及其默认值：

配置项	说明	默认值
`server_name`	服务器名称	`TouchFish`
`captcha`	注册是否需要验证码	`false`
`email_activate`	发件邮箱，空为不启用邮箱验证	`""`
`file_last_time`	文件保留时间（小时）	`72`
`max_file_size`	单文件最大大小（字节）	`-1`（不限制）
`user_storage_quota`	用户存储配额（字节）	`-1`（不限制）
`max_message_length`	单条消息最大长度	`10000`
`jwt_max_per_user`	单用户可同时持有的 token 数	`5`
`legacy_auth_enabled`	是否接受旧版 uid+password 认证	`true`
`storage_backend`	文件存储后端（`local` / `oss2`）	`local`

所有配置项的详细说明（反向代理、RTC/TURN、频率限制、对象存储等）请参阅 开服指导。

启动服务端

```
# 使用指定编号的配置启动（编号从 0 开始，见 server_config.json）
python main.py --start-api --use-config <编号>

# 仅调试时使用 Flask 开发服务器，并输出请求日志
python main.py --start-api --use-config <编号> --dev-server --debug --log
```

不带 `--start-api` 时只会启动 TCP（WebSocket）服务，REST API 需要单独暴露；`--start-api` 会在同一进程内启动内置 waitress 服务器，可通过 `--waitress-threads`（默认 16）与 `--waitress-connection-limit`（默认 1000）调整。

启动后：

- REST API 监听 `http://0.0.0.0:<api_port>`

- WebSocket 监听 `ws://0.0.0.0:<tcp_port>`

启动（v4）

作为服务端（创建聊天室）

1. 获取本机内网 IP 地址：

: Linux：执行 `ip a` 或 `ifconfig`，查找 `inet` 后的 IPv4 地址。

1. 检查端口可用性：

: 选择一个 0~65535 的端口，执行 `netstat -an | grep <端口号>` 或 `lsof -i :<端口号>`，确保端口未被占用。

1. 启动服务器：

: 运行 `LTS.py` 或二进制文件，按 `Enter` 开始配置，按以下顺序填写：

1. 启动模式：`Server`\[1\]
2. 内网 IP 地址
3. 空闲端口号
4. 服务端昵称
5. 最大用户数（建议 ≤128）
6. : 配置完成后将你的 IP 地址和端口分享给其他用户。
7. 后续启动：再次运行时按 `Ctrl+C`（Windows/Linux）或 `Ctrl+D`（macOS）即可自动使用上次配置。

作为客户端（加入聊天室）

1. 运行 `LTS.py` 或二进制文件，按 `Enter` 开始配置。
2. 选择启动模式为 `Client`\[1\]，填写服务器提供的 IP、端口和你的个人昵称。
3. 后续连接：再次运行时按 `Ctrl+C`（Windows/Linux）或 `Ctrl+D`（macOS）即可自动连接。

注：你需要有写入当前文件夹的权限来写入和读取配置文件

使用（v4）

概述

聊天室界面分为两种模式：

- **输出模式（默认）**：行首没有符号，此时输入的指令将被忽略且不会显示在屏幕上。

- **输入模式**：行首显示 > 符号。按下 Enter（或输入任意指令）即可从输入模式返回输出模式。在输入模式下，新的消息会等待退出输入模式后才显示。

用户状态共有以下 7 种：

状态	说明
Root	聊天室房主
Admin	聊天室管理员
Online	聊天室普通用户
Pending	等待加入审核的用户
Offline	主动离开聊天室的用户
Kicked	被踢出聊天室的用户
Rejected	连接被拒绝的用户

只有状态为 Root、Admin、Online 和 Pending 的用户会被计入在线用户数。状态为 Admin 或 Root 的用户有权**查看别人的私聊消息和私有文件**。

聊天室内可用的指令共 17 条（25 项），支持使用方括号内的缩略形式（不区分大小写）。所有 <user> 字段可以输入 UID 或用户名（优先解析为 UID，遇到冲突时采纳 UID 最小的合法解析结果）。简易命令行模式（flood）允许直接输入并发送单行消息而省略 send，但会禁用其他指令。

基本命令

以下指令所有用户均可使用：

缩略	完整格式	说明
D	dashboard	展示聊天室各项数据（Root 用户可以看到所有用户的 IP 地址）
F	distribute <filename>	发送文件（filename 可以包含空格而无需使用引号，transfer 同理）
Q	evaluate <input>	像 Python IDLE 那样计算输入数据（使用 eval()，二进制发行版 Python 版本为 3.6）。 **警告**：请勿注入恶意代码（如 exit()、locals() 等），后果自负。
E	exit	退出或关闭聊天室
L	flood	开启简易命令行模式（禁用其他指令，直接发送单行消息）
H	help	显示帮助文本
S	send	发送多行消息
S	send <message>	发送单行消息
J	shell <command>	执行 Shell 指令 **警告**： 请勿执行危害本程序或设备的指令，后果自负。
T	transfer <user> <filename>	向某个用户发送私有文件
P	whisper <user>	向某个用户发送多行私聊消息
P	whisper <user> <message>	向某个用户发送单行私聊消息

管理员

以下指令需要管理员（Admin）或房主（Root）权限。标注 * 的指令只有 Admin 或 Root 可使用，标注 ** 的指令只有 Root 可使用。

缩略	完整格式	权限	说明
I+	ban ip add <ip>	*	封禁 IP 或 IP 段（支持 CIDR 格式如 a.b.c.d/e，前缀长度 ≥24）
I-	ban ip remove <ip>	*	解除封禁 IP 或 IP 段
W+	ban words add <word>	*	屏蔽某个词语
W-	ban words remove <word>	*	解除屏蔽某个词语
B	broadcast	*	向全体用户广播多行消息
B	broadcast <message>	*	向全体用户广播单行消息
C	config <key> <value>	*	修改聊天室配置项（<key> 格式以 dashboard 输出为准，<value> 格式以修改示例为准）
G+	doorman accept <user>	*	通过某个用户的加入申请
G-	doorman reject <user>	*	拒绝某个用户的加入申请
K	kick <user>	*	踢出某个用户 • Root 可踢出 Admin 或 Online 用户 • Admin 只能踢出 Online 用户
A+	admin add <uid>	\\	添加管理员（仅 Root）
A-	admin remove <uid>	\\	移除管理员（仅 Root）
V	save	\\	保存聊天室配置项信息（仅 Root）

更多联机帮助请参见 TouchFish GitHub 仓库。

故障排除

无法连接聊天室

- 确保所有设备处于同一局域网（LAN）或可互通的子网内。
- 服务端需开放指定端口的入站连接（例如：检查防火墙规则，使用 iptables 或 ufw 放行端口）。

客户端或服务端闪退

- 检查端口是否已被其他进程占用，改用其他空闲端口。
- 运行 python 版本时，确认 Python 环境为 3.x 版本。

1. ↑ 1.0 1.1 此处必须开头大写

5.6 Telegram

难度 进阶 | **篇幅** 11.0 KB | **所属** 即时通信

本节将讲到

- 安装
- 提示和技巧
- 参见

Telegram) 是一种基于云的跨平台即时消息服务，具有可选的端到端加密。创建帐户需要手机号。

官方客户端是开源的，但最新版本的代码并不总是立即发布。服务器端代码是专有的。

安装

在 Arch Linux 上，你可以这样使用 Telegram:

聊天软件的插件

- telegram-tdlib-purple-gitAUR 为基于 libpurple包 的聊天软件 （例如 Pidgin ）提供了支持。
- 基于 Telepathy) （例如 GNOME 的 empathyAUR）的软件可以使用 telepathy-haze包，它也支持libpurple包，所以也可以用 telegram-tdlib-purple-gitAUR 提供支持。
- KDE 用户可以使用 telepathy-morse包 将默认聊天程序设置为 Telegram。

图形界面

telegram-desktop包 提供了官方桌面版客户端。

第三方客户端:

- **Kotatogram Desktop** — Kotatogram Desktop 基于 Telegram Desktop，具有 Telegram Desktop 的所有功能，但也有一些更实用、更美观的功能。

: https://kotatogram.github.io/ || kotatogram-desktopAUR 或者 kotatogram-desktop-binAUR

终端客户端

- **Telegram messenger CLI** — Telegram 的命令行界面，使用 readline 界面。

: https://github.com/vysheng/tg || telegram-cli-gitAUR

- **ncTelegram** — 用 Python 和 urwid 库开发的 Ncurses Telegram 客户端，依赖于 telegram-cli-gitAUR。

: https://github.com/Nanoseb/ncTelegram || nctelegram-gitAUR

- **tg** — Telegram 终端客户端。

: https://github.com/paul-nameless/tg || telegram-tgAUR

- **telegram-send** — Telegram-send 不是完整的客户端,是一款命令行工具，用于通过 Telegram 向账户、群组或频道发送消息和文件。 它提供了一个简单的界面，可以方便地从其他程序中调用。

: https://github.com/rahiel/telegram-send || python-telegram-sendAUR

- **nchat** — nchat 是一款基于终端的聊天客户端，适用于 Linux 和 macOS，支持 Telegram 和 WhatsApp。

: https://github.com/d99kris/nchat || nchat-gitAUR

- **telega.el** — GNU Emacs Telegram 平台的全功能非官方客户端。

: https://github.com/zevlg/telega.el || 此软件并不在 AUR 中

- **TelegramTUI** — 基于 Python 的 TUI telegram

: https://github.com/vtr0n/TelegramTUI || 此软件并不在 AUR 中，你可以通过 `pipx install telegramtui` 安装

基于网页的客户端

- 官方的 Telegram WebK。
- 官方网页应用 Telegram Web。
- franzAUR 是开源的 Web 客户端，可以作为 Telegram), WhatsApp, Facebook 等应用的客户端。
- rambox-binAUR 是 Franz 的备选之一，也是开源的，提供了各个功能的对标组件。
- Telegram Web 的 Chrome app。

提示和技巧

在 Telegram 中使用 GTK 对话框

如果你想要使用 GTK 对话框来代替 Qt 对话框，请将环境变量 `QT_QPA_PLATFORMTHEME` 设置为 `gtk3`。

Telegram 桌面中的 KDE 对话框

如果想使用 KDE 文件对话框来代替 Qt 对话框，请将环境变量 `QT_QPA_PLATFORMTHEME` 设置为 `xdgdesktopportal`。

使用 SVG 图标主题

如果您想使用基于 SVG 图像的图标主题 (e.g. papirus-icon-theme包)，请安装 qt5-svg包。参见 Qt#Icon theme is not applied。

Wayland 支持

参见 Wayland#Qt。

Telegram Desktop 中的 xdg-open

如果你想在 t.me 链接上使用 xdg-open 并收到一个错误，找不到 tg 的处理程序：

```
xdg-mime default telegramdesktop.desktop application/x-xdg-protocol-tg
xdg-mime default telegramdesktop.desktop x-scheme-handler/tg
```

Failed to set real-time priority for thread: Operation not permitted

如果获得以下错误信息：

```
$ telegram-desktop
[ALSOFT] (EE) Failed to set real-time priority for thread: Operation not permitted (1)
```

安装 realtime-privileges包，并添加您自己到 `realtime` 用户组并重启。参见 Realtime process management#Configuring PAM。

HiDPI 缩放

如果系统启用了 Qt 缩放，且缩放比例不是整数，你可能遇到图片和图标出现马赛克的问题。此时可能需要单独针对 Telegram 关闭高 DPI 缩放。

将 `/usr/share/applications/telegramdesktop.desktop` 复制到用户特定应用程序目录下，并像这样编辑：

```
$HOME/.local/share/applications/telegramdesktop-no-scaling.desktop
...
Exec=env -u QT_SCREEN_SCALE_FACTORS telegram-desktop -- %u
...
```

您可能需要更新桌面项目数据库。

音频后端

Telegram 使用 OpenAL，可以通过编辑此配置文件（`~/.config/alsoft.conf`）或者通过此处列出的环境变量进行修改。

如果使用了无效的音频后端导致声音故障，可以通过修改环境变量 `ALSOFT_DRIVERS` 或者修改 OpenAL 配置文件 `[general]` 部分中的属性 `drivers`

来覆盖设置。比如 drivers 值为 "pulse" 时，系统将优先使用 pulseaudio, 失败则回滚到默认驱动程序。

选择正确的摄像头

Telegram 版本 3.7.1 不允许切换用于视频会议的摄像头。如果要强制 telegram 使用其他摄像头, 可以禁用不需要的摄像头，参见 https://askubuntu.com/a/166819 。

Fcitx support for Qt 6 Telegram

Telegram-desktop 自 3.4.2-2 版本起已迁移至基于 Qt 6 构建。从旧版本升级的用户可能会发现 Fcitx 输入法在此应用中停止工作。要恢复功能，请安装 fcitx-qt6包 软件包或 fcitx-im包组 软件包组。若使用Fcitx5输入框架，则应安装 fcitx5-im包组 软件包组。

Changing the default file browser

在未设置 XDG_CURRENT_DESKTOP 环境变量的环境中（如不运行桌面环境，即仅运行窗口管理器时），telegram-desktop 会退回到非常基本的文件浏览器。 使用上述变量，用户可以将默认浏览器更改为自己喜欢的、属于特定桌面环境的浏览器。 该变量的有效值可以在这里找到。 例如，使用 XFCE 项目中的Thunar：

```
XDG_CURRENT_DESKTOP=XFCE telegram-desktop
```

Arch Linux 中文社区的 Telegram 群组和频道

- Arch Linux CN News - 翻译 Arch Linux 官方网站的最新消息，和社区内大家遇到的踩坑预警。
- #archlinux-cn - 同步到 IRC #archlinux-cn 的中文讨论。
- #archlinux-cn-offtopic - 同步到 IRC #archlinux-cn-offtopic， 中文水群。

参见

- Arch Linux - 讨论有关 Arch Linux 的一切内容的非官方群组。
- ArchWikiBot - Arch Linux Wiki 搜索机器人。
- Planet Arch Linux & News - Channel with recent Planet Arch updates and Latest News in one place.
- Arch Linux: Recent package updates - Arch Linux 软件包更新通知频道。
- Arch Linux News - Arch 网站的新闻频道 *(2018 年起不再更新)*。
- Planet Arch - Planet Arch 网站的文章频道 *(2018 年起不再更新)*。

5.7 腾讯 QQ

难度 进阶 | **篇幅** 13.7 KB | **所属** 即时通信

本节将讲到

- 原生 Linux 版本
- 虚拟机
- Wine
- 第三方客户端
- 疑难解答

QQ 是腾讯公司开发的即时通讯软件，为 ICQ 的仿制品，是中国最流行的 IM 软件之一。本页面列出了在 Arch Linux 上使用 QQ 的各种解决方案。

原生 Linux 版本

使用 Electron 技术的 Linux QQ 对应的软件包为 linuxqqAUR。

注意：官方版本提供的是普通 QQ，TIM 没有官方版本。如果要使用 TIM，请使用 wine 方案。

带有 bubblewrap 沙盒的版本

此外还有 linuxqq-nt-bwrapAUR 可选，带有 bubblewrap 沙盒和一些调整。

- 要传递给 **bwrap** 的自定义参数应该写在 `~/.config/qq-bwrap-flags.conf`，可在此文件中设置自定义挂载目录。
- 要传递给 **electron** 的自定义参数应该写在 `~/.config/qq-electron-flags.conf`。

虚拟机

您可以在虚拟机中运行一个完整的 Windows 系统，并在此中运行 QQ。相比于其他的方案，这种方案出错的几率是最小的，缺点是占用的资源较多。

提示：

- 根据许可条款，在每个虚拟设备上运行 Windows 都需要单独的授权。
- 如果您使用 VirtualBox，建议您开启**无缝模式**，这个功能能让您在宿主机的桌面下无缝操作虚拟机中的窗口。

Wine

Wine 是类 UNIX 系统下运行微软 Windows 程序的"兼容层"，可以用它模拟 Windows 环境来运行 QQ/TIM。

警告：Wine QQ/TIM 在平铺式窗口管理器下的样式可能会大规模失控，需要进行额外的配置。

Ukylin Wine（优麒麟Wine）

ukylin-wineAUR 是基于 Wine 开发的一款软件，用于在 Linux 系统上兼容运行 Windows 软件。其对微信和QQ/TIM进行了大量的修复以确保软件稳定。如果您在使

用 TIM 时遇到了不停的卡顿（包括但不限于群详情页、右键菜单和表情面板），不妨尝试使用 ukylin-wine 替换其他版本的 wine 来启动 TIM。

例：替换 Spark-TIM 启动脚本中的 APPRUN_CMD

```
/opt/apps/com.qq.tim.spark/files/run.sh:14
export APPRUN_CMD="ukylin-wine"
```

Crossover

可以使用 CrossOver 运行 QQ 和 TIM。更多详情可以参阅 CrossOver 的兼容性列表。

手动 Wine 方案

TIM

安装 wine包、wine-gecko包 和 wine-mono包。

执行 `winetricks riched20`，也可使用 `winecfg` 设置函数库顶替。

可能需要配置中文字体显示，见#字体配置。

安装 TIM。

生成图标

安装的 TIM 可能没有在程序列表中生成图标。若要自行添加图标，新建 `tim.desktop` 文件，写入以下内容：

```
tim.desktop
[Desktop Entry]
Encoding=UTF-8
Version=1
Name=TIM
Comment=Tencent TIM
Exec=wine '~/.wine/drive_c/Program Files/Tencent/TIM/Bin/TIM.exe'
Icon=~/.wine/drive_c/Program Files/Tencent/TIM/TIMUninst.ico
Terminal=false
Type=Application
Categories=Network;
```

将 `tim.desktop` 移动到 `~/.local/share/applications` 或 `/usr/share/applications`文件夹下即可。

第三方客户端

注意：请尽量使用含 wine 在内的方案或来自腾讯官方的客户端。第三方客户端尽管不存在官方客户端的一系列问题，但随时可能因为种种原因而失效。

Icalingua++

Icalingua++ 是 Icalingua 的分支，为已经删除的 Icalingua 提供有限的更新。此项目希望为 Linux 打造一个会话前端框架，通过实现 Adapter 后端接口来适配各种聊天平台。目前已经拥有基于 oicq 以及 Icalingua 自有协议的后端。

要使用 Icalingua++ ， 可以从 AUR 源安装icalingua++AUR。

疑难解答

原生版本(Linux QQ)

无法使用通知功能

如果在 QQ 中无法使用桌面环境的通知功能，可以尝试安装XDG桌面门户中对应的桌面后端。也可能是在设置-消息通知中关闭了通知，但Linux版本没有此设置，需要在Windows或macOS上的QQ中修改。

打开窗口/设置/接受消息闪退

你可以尝试以下命令:

```
sudo rm -rf ~/.config/QQ/crash_files/ && sudo chattr +i
~/.config/QQ/crash_files/
```

每次登录都需要重新扫码

如果即使在登录时勾选了无需在手机端确认，下次登录时还需要扫码，这可能是因为其是根据网卡mac地址来判断是否为同一设备，你可以尝试禁用虚拟网卡(例如:Docker网卡,VMWare网卡)来解决这个问题。

wine版

无法调用输入法

如果在 QQ 中使用输入法无反应，可考虑在 QQ 的启动脚本中配置有关环境变量，示例如下（将 fcitx 改成你用的输入法）。

```
export GTK_IM_MODULE=fcitx
export QT_IM_MODULE=fcitx
export XMODIFIERS=@im=fcitx
```

com.qq.im.deepinAUR\[损坏的链接：package not found\]等默认脚本中可能存在将输入法配置为 ibus 的行，如果你要使用其它输入法，需将这些行删去。

字体配置

如果中文的显示遇到问题，可以尝试先执行 `winetricks fakechinese`。

另请参阅字体和字体配置#不支持 Fontconfig 的程序。

字体替换不完全/显示宋体发虚

先使用 fakechinese 替换完字体后，再参考 https://bbs.deepin.org/zh/post/213530 和 https://bbs.deepin.org/post/213153 使用伪装的宋体。

文件被占用

杀死 QQ 或 TIM 的进程即可。 在退出 QQ/TIM 之后，某些相关进程仍然在后台运行。也可以使用如下脚本来启动 QQ/TIM，它会首先查找已有的进程，杀死该进程后启动新的 QQ/TIM。

```
start-tim.sh
#!/bin/sh
# script to start TIM
# kill TIM before start TIM
for pid in `pgrep TIM.exe`; do
       if [ -n ${pid} ]; then
              kill ${pid}
       fi
done
# start TIM
wine '~/.wine/drive_c/Program Files/Tencent/TIM/Bin/TIM.exe'
```

上面的例子适用于 TIM，稍作修改之后即可应用于 QQ。

xfce4（xfwm4）下无法输入表情

打开设置管理器-窗口管理器微调-焦点，取消勾选激活焦点防窃取和遵照标准的 ICCCM 焦点提示即可。

原因是表情窗口获取焦点时会发生不兼容现象。

在非中文 locale 下无法输入中文

修改 `.desktop` 文件的 `Exec`，这个文件一般位于 `/usr/share/applications/` 或者 `~/.local/share/applications/`。

在 `Exec` 行中加入 `env LC_ALL=zh_CN.UTF-8`。 例如，原来的 `Exec` 为：

```
Exec=".wine/drive_c/Program Files/QQ/Bin/QQ.exe"
```

则应改为：

```
Exec=env LC_ALL=zh_CN.UTF-8 wine ".wine/drive_c/Program Files/QQ/Bin/QQ.exe"
```

deepinwine方案在非中文 locale 下字体显示为黑框且能够调用输入法但是无法输入字符

首先查看是否安装有中文locale。在 `/etc/locale.gen` 中启用zh_CN.UTF-8 UTF-8(删除#符号)。而后运行命令 `locale-gen`。

安装了中文locale后，修改deepinwine的启动脚本，其位于 `/opt/deepinwine/tools/` 。其中有三个脚本，分别为 `run.sh`, `run_v2.sh`, `run_v3.sh`。将三个脚本中的

```
WINE_CMD="deepin-wine"
```

都修改为

```
WINE_CMD="LC_ALL=zh_CN.UTF-8 deepinwine"
```

也可以根据对应deepinwine容器的启动脚本（一般路径为 `/usr/share/applications/*.desktop` ）中的Exec行确定要修改的脚本。

HiDPI 支持

在 HiDPI 显示器上，QQ/TIM 的界面可能会过小。在较新版本的 QQ/TIM 中已经加入了对 HiDPI 的支持。只需手动调整 Wine 的 DPI 即可。

执行 `winecfg`，在打开的窗口中切换到**显示**选项卡并调整 DPI。

注意：如果您使用的不是默认的 Wine 容器（例如使用了deepin QQ/TIM），那么需要在执行 `winecfg` 时指定`WINEPREFIX` 变量。例如`env WINEPREFIX=$HOME/.deepinwine/Deepin-QQ deepin-wine winecfg` 或是 `env WINEPREFIX=$HOME/.deepinwine/Deepin-TIM deepin-wine winecfg`。

平铺式窗口管理器下的配置

Awesome

Wine QQ/TM 在平铺式窗口管理器下可能会失控，需要进行一些配置。

下面的配置有这些作用：

- 将所有 TM 的窗口设置为浮动。
- 清除不需要的窗口边框、避免菜单弹出时焦点移动到菜单上。
- 在使用标签式会话窗口时，增加使用 Alt+数字来切换标签页的快捷键，需要安装 xdotool。
- 自动关闭弹出的新闻窗口。

将以下内容添加到 Awesome 配置：

```
function myfocus_filter(c)
  if awful.client.focus.filter(c) then
    -- This works with tooltips and some popup-menus
    if c.class == 'Wine' and c.above == true then
      return nil
    elseif c.class == 'Wine'
      and c.type == 'dialog'
      and c.skip_taskbar == true
      and c.size_hints.max_width and c.size_hints.max_width < 160
      then
      -- for popup item menus of Photoshop CS5
      return nil
    else
      return c
    end
  end
end

awful.rules.rules = {
  -- All clients will match this rule.
  {
    rule = { },
    properties = {
      -- 这里使用我们自己的函数
      focus = myfocus_filter,
```

```
      -- 以下是默认的部分
      border_width = beautiful.border_width,
      border_color = beautiful.border_normal,
      keys = clientkeys,
      buttons = clientbuttons,
    }
  }, {
    rule_any = {
      instance = {'TM.exe', 'QQ.exe'},
    },
    properties = {
      -- This, together with myfocus_filter, make the popup menus flicker taskbars less
      -- Non-focusable menus may cause TM2013preview1 to not highlight menu
      -- items on hover and crash.
      focusable = true,
      floating = true,
      -- 去掉边框
      border_width = 0,
    }
  }, {
    -- 其它规则
  }
}

alt_switch_keys = awful.util.table.join(
    -- it's easier for a vimer to manage this than figuring out a nice way to loop and
concat
    awful.key({'Mod1'}, 1, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+1') end),
    awful.key({'Mod1'}, 2, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+2') end),
    awful.key({'Mod1'}, 3, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+3') end),
    awful.key({'Mod1'}, 4, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+4') end),
    awful.key({'Mod1'}, 5, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+5') end),
    awful.key({'Mod1'}, 6, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+6') end),
    awful.key({'Mod1'}, 7, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+7') end),
    awful.key({'Mod1'}, 8, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+8') end),
    awful.key({'Mod1'}, 9, function(c) awful.util.spawn('xdotool key --window ' ..
c.window .. ' ctrl+9') end)
)
function bind_alt_switch_tab_keys(client)
    client:keys(awful.util.table.join(client:keys(), alt_switch_keys))
    end -- }}}

    client.connect_signal("manage", function (c, startup)
  -- 其它配置

  if c.instance == 'TM.exe' then
    -- 添加 Alt+n 支持
    bind_alt_switch_tab_keys(c)
    -- 关闭各类新闻通知小窗口
    if c.name and c.name:match('^腾讯') and c.above then
      c:kill()
    end
  end

  -- 其它配置
end)
```

也可以看看完整 Awesome 配置。

i3

原生配置下，启动 `qq` 时会自动最大化，且边框不美观，可在 i3 的 `config` 设置如下两条规则以改善：

```
for_window [instance="QQ.exe"] floating enable
for_window [instance="QQ.exe"] border none
```

在i3status中，tim托盘可能无法正常显示。推荐您使用i3status-rust作为替代。可以安装包：

```
i3status-rust-gitAUR
i3status-rust包
```

参阅

- openSUSE wiki
- Web 端的 QQ 群空间当所使用 QQ 客户端不支持群空间时，可以此用该服务代替。
- IM QQ-QQ 手机版移动端也未尝不也是一种代替方案。
- hillwoodroc/winetricks-zh hillwoodroc/winetricks-zh

网络共享

一台机器把文件目录借给别的机器读写，或让别的机器借道它上网，是本章要解决的事。先读《网络分享》，建立依赖与配置的总图；再读《路由器》的接口配置、ADSL/PPPoE、DNS 和 DHCP 与连接共享，IPv6 说明可后置。主干是《NFS》与《Samba》：前者按服务端配置、客户端配置、故障排查推进，后者看服务器与客户端配置两节，两者的疑难解答都值得留着。《Samba/Active Directory 域控制器》只在需要集中认证时才看，《iPhone 网络共享》与《Shfs》属特定场景，可跳过。

最容易踩的坑是把 NFS 与 Samba 当成可互相替代的两个选项，或只配完服务端就等客户端自动生效；不通时先看故障排查，核对两侧的导出与共享定义，而非只改本机网络。

6.1　Shfs

难度 入门 | **篇幅** 2.6 KB | **所属** 网络共享

本节将讲到

- 安装
- 配置
- 参考

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： SSHFS、rclone包 等 (在Talk:Shfs讨论)

Shfs是一个简单易用的Linux内核模块。该模块可以让你通过ssh连接挂载远程文件系统。使用shfs访问远程文件就和访问本地文件一样，但其访问权限由ssh安全传输功能进行管理。

注意：由于shfs自2004年后就再也没有更新过，目前基于FUSE的SSHFS应用更加广泛。

安装

安装#安装软件包)shfs-utils包软件包。

警告：要使用shfs，需要在客户端而不是服务器端进行安装及配置。服务器端只需要运行ssh服务即可。

配置

**本文或本章节的事实准确性存在争议。**

原因： Do not simply recommend setting the SUID bit on a binary since this can have severe security implications（在 Talk:Shfs 中讨论）

如果想作为普通用户使用shfs挂载功能，你需要使用`chmod +s /usr/bin/shfsmount`和`chmod + /usr/bin/shfsumount`两条命令。为了更方便地使用shfs，可以在`/etc/fstab`中增加挂载项，例如：

```
remoteuser@Server:/data    /mnt/data    shfs     rw,noauto,uid=localuser,persistent    0
0
remoteuser@Server:/crap    /mnt/crap    shfs     rw,noauto,uid=localuser,persistent    0
0
remoteuser@Server:/backup /mnt/backup shfs     rw,noauto,uid=localuser,persistent    0
0
remoteuser@Server:/home    /mnt/home    shfs     rw,noauto,uid=localuser,persistent    0
0
```

使用SSH keys可以免去输入密码的步骤。

你可能需要在挂载项中添加端口号（`port=<端口号>`）

添加fstab挂载项

如要在fstab添加shfs卷，在`/etc/fstab`中按以下格式增加内容：

```
userid@remoteMachine:/remoteDirectory /home/userid/remoteDirectory shfs rw,user,noauto
0 0
```

参考

- SSHFS (简体中文)
- Ubuntu Forums
- Reference
- Another reference

6.2 IPhone 网络共享

难度 进阶 | **篇幅** 6.0 KB | **所属** 网络共享

本节将讲到

- 通过 USB 进行共享
- 通过蓝牙进行共享

您可以通过 **Wi-Fi**、**USB** 或**蓝牙**来在设备上共享您的 iPhone 的移动数据连接，除非您的服务提供商（运营商）禁用了此功能：

- 只要您的电脑能够连接无线网络，Wi-Fi 就不需要其他额外配置，
- 下文提供使用 USB 与蓝牙的操作指南。

通过 USB 进行共享

通过原生的 USB 进行共享是最理想的方式，因为其可以提供更加稳定的连接，且较蓝牙和 Wi-Fi 共享消耗的电量较少。

要使用 USB 进行 iPhone 网络共享，您需要安装 libimobiledevice包 与 usbmuxd包。**usbmuxd** 依赖于 **libimobiledevice** ，其中 usbmuxd 负责系统与 iOS 设备间的底层连接。**usbmuxd** 包也包含了一条 udev 规则，使其能够在设备连接或断开连接时自动启动与停止该守护进程。相关内容详见 iOS。

连接 iOS 设备，并确保 `usbmuxd.service` 已经自动启动。

下一步，在您的 iPhone 上启用*个人热点*（*Personal Hotspot*），并将其用数据线连接至您的电脑。这时您应该会多出一个可用的以太网连接，并能够使用任意网络管理器通过新加入的 iPhone 以太网设备连接到 Internet，与使用其他普通的以太网连接没有什么区别。

如果您在使用 MAC 地址伪装，您可能需要将不支持 MAC 复制的 `ipheth` 驱动程序列为例外。

```
/etc/NetworkManager/conf.d/your-file-here.conf
[your-section]
match-device=*,except:driver:ipheth
```

使用 systemd-networkd

如果您使用 systemd-networkd 进行网络管理，您能轻易将其配置为通过 iPhone 连接 Internet，与使用其他普通网络适配器时别无二致。

假设 *enp0s26u1u2c4i2* 是运行 `networkctl list` 命令时展示的 iPhone 创建的网络设备的名称，创建下面的 *.network* 文件：

```
/etc/systemd/network/30-tethering.network
[Match]
Name=enp0s26u1u2c4i2

[Network]
DHCP=yes
```

故障排除

若设备列表中有 iPhone 但无法连接，您有可能需要在使用网络共享之前先将 iPhone 与您的电脑连接并进行配对。过程中，使用 PIN 的 iPhone 需要解锁。

```
# idevicepair pair
```

缺少驱动程序

如果您按照上面的步骤操作，但 `networkctl list` 命令仍无法检测到 iPhone，您可能缺少了ipheth 驱动。您可以运行 `modprobe ipheth` 命令来检查您是否安装了该驱动程序。如果出现了错误消息，可能是您自行编译内核时发生的问题。请在编译内核时设置 `ipheth` flag。

```
.config
CONFIG_USB_IPHETH=y
```

通过蓝牙进行共享

使用蓝牙进行网络共享会相对较快地将电池耗尽。如果您同时连接了 USB 电源供应器，情况可能会稍好一些。

硬件要求

- 运行 iPhone OS 3.0 及以上的开启了网络共享的 iPhone。在*设置 > 通用 > 网络*中开启网络共享选项。
- 蓝牙控制器或类似的硬件设备，最好支持 EDR (*Enhanced Data Rate*) 以便获得相对较好的速度。在 *Belkin F8T016NE* 上经过了测试。

设置

参考主文章蓝牙并设置好蓝牙守护程序。

Gnome/XFCE

安装 Blueman GTK 蓝牙管理器。

您的通知区域中应该会出现一个蓝牙图标（注意：如果在开机时未启用蓝牙，图标可能不会出现）。点击该图标，并搜索附近的设备，添加您的 iPhone。（注意，要使 iPhone 可以被发现，您可能需要在 iPhone 上打开设置中的蓝牙页面。）

一旦 iPhone 被添加到了设备列表中，打开设备菜单并选择 配对 。通常这需要先在电脑后在 iPhone 上输入 PIN 码。连接好后，再次打开设备菜单，选择*网络访问 > 网络访问点*。如果一切正常的话，blueman 会报告成功，您的 iPhone 上的状态栏也会变为蓝色，表示成功建立了网络共享。

Blueman 将会创建一个新的网络设备，一般为 bnep0。要连接到该设备，以 root 身份运行以下命令：

```
# dhcpcd bnep0
```

netcfg

您也可以创建一个 netcfg 网络档案，以便在命令行界面中就可以轻松地进行网络共享而不需要使用 Blueman 或 Gnome。假设您有一部地址为 '00:00:DE:AD:BE:EF' 的已经配对好的 iPhone，只需要在 `/etc/network.d called` 中创建一个档案，例如 'tether'：

```
CONNECTION="ethernet"
DESCRIPTION="Ethernet via pand tethering to iPhone"
INTERFACE="bnep0"
IPHONE="00:00:DE:AD:BE:EF"
PRE_UP="pand -E -S -c ${IPHONE} -e ${INTERFACE} -n 2>/dev/null"
POST_DOWN="pand -k ${IPHONE}"
IP="dhcp"
```

接下来执行：

```
# netcfg tether
```
要关闭网络连接并停止网络共享：

```
# netcfg down tether
```

6.3 网络分享

难度 进阶 | **篇幅** 10.6 KB | **所属** 网络共享

本节将讲到

- 依赖
- 配置
- 问题解决
- 另请参见

这篇文章解释了如何从一台机器向其他机器分享网络连接。

依赖

作为服务器的机器应该有一个额外的网络设备。这个网络设备需要一个数据链路层来连接到将要获得网络访问的机器：

- 如果想给若干台机器分享网络连接，交换机可以提供数据连接。
- 一个无线设备同样可以给若干台机器分享网络访问，参见Software access point。
- 如果只需分享网络给一台机器，一根交叉网线就可以了。 如果两台电脑的网卡支持MDI-X，交叉网线也不是必须的，一根普通直连网线也可以。 运行 `ethtool *interface* | grep MDI` 确认网卡是否支持MDI-X。

配置

这个章节假定，连接到客户机的网络设备被命名为 ***net0***而连接到互联网的网络设备被命名为***internet0***。

提示：为了此事，你可以使用Udev#Setting static device names所述的方法重命名你的设备。

所有的配置都是在服务器计算机上完成的，除了最后一步#给客户机分配IP地址.

静态IP地址

在服务器计算机上，给要连接到其他机器的网卡分配一个静态的IP地址。IP地址的前3个字节不能和其他网卡的一模一样，除非两个网卡都有一个严格大于/24的子网掩码。

```
# ip link set up dev net0
# ip addr add 192.168.123.100/24 dev net0 # arbitrary address
```
为了使你的静态IP在启动时被分配，你可以使用network manager。

启用包转发

警告：如果没有正确配置防火墙则会有安全隐患。

检查当前的包转发设置：

```
# sysctl -a | grep forward
```

这将显示全局和端口默认转发设置以及每个接口分别的 IPv4 和 IPv6 转发设置。内核文档（英文）中有各个选项的详细描述。

要启用 IPv4 和 IPv6 包转发，配置 sysctl 并设置这些选项：

```
net.ipv4.ip_forward = 1
net.ipv4.conf.all.forwarding = 1
net.ipv6.conf.all.forwarding = 1
```

提示：要只为某个接口启用包转发，设置 `net.*ipv*.conf.*interface_name*.forwarding=1`。

警告：如果系统使用了systemd-networkd来控制网卡，就不能为单个网卡配置 IPv4 的设置，换言之，systemd 的操作规则会将所有配置过的转发发送到一个全局（针对所有网卡）的 IPv4 的设置。建议的变通方法是使用防火墙在选中的网卡上禁止转发。参考 systemd.network(5) 手册页以获取更多信息。在之前的 systemd 220/221 版本中引入的执行内核设置的 `IPForward=kernel` 已不再适用。

要是更改在重启后仍生效，见 Sysctl#配置。可以考虑将设置写到 `/etc/sysctl.d/30-ipforward.conf` 这种有描述性文件名的文件中。

建议在重启之后再次检查，确认转发已经按要求启用。

启用 NAT

使用 iptables

安装iptables包包。使用iptables来启用NAT：

```
# iptables -t nat -A POSTROUTING -o internet0 -j MASQUERADE
# iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
# iptables -A FORWARD -i net0 -o internet0 -j ACCEPT
```

注意：当然了，这对移动宽带连接（一般在电脑上叫做ppp0）也有用。

阅读iptables的文章以获取更多信息（特别是关于保存规则并在重启之后应用的东西）。 还有一篇关于iptables的很好的文章：Simple stateful firewall。

使用 nftables

安装 nftables包包。要通过nftables启用NAT，你必须在一个新的或者现有的表中创建prerouting和postrouting链（即使它们是空的你也还是需要这两个链）：

```
# nft add table ip nat
# nft add chain ip nat prerouting { type nat hook prerouting priority 0 \; }
# nft add chain ip nat postrouting { type nat hook postrouting priority 100 \; }
```

注意：如果想要使用IPv6，你需要将所有的`ip`替换为`ip6`。

然后，你还要将net0的地址转换为internet0的地址（这里就是网络地址转换NAT了）：

```
# nft add rule nat postrouting oifname internet0 masquerade
```

你可能还需在forwarding链上添加更多的防火墙显示（假如filter表已经存在，就像Nftables#Server中配置的那样）：

```
# nft add chain inet filter forward { type filter hook forward priority 0 \; policy drop}
# nft add rule filter forward ct state related,established accept
# nft add rule filter forward iifname net0 oifname internet0 accept
```

你可以在nftables Wiki)找到更多的关于nftables中的NAT的信息。如果你想要持久化这些更改，照着nftables说的做。

给客户机分配IP地址

如果你计划经常要有几台机器使用这台电脑共享的网络，那么建议你安装一个DHCP服务器，比如dhcpd 或者 dnsmasq。然后在每一台连这个网的电脑上配置DHCP客户端（比如说dhcpcd）。

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：这不是 iptables 指南。使用 `iptables -I` 扩展链可能会跳过其他重要规则；如果需要为此编写一个开关脚本，请使用自定义链，并在 INPUT 链中小心地放置一个跳转。（在Talk:网络分享讨论）

必须把67端口的UDP传入连接开给DHCP服务器。为了允许DNS请求，还要打开53端口的UDP/TCP传入连接。

```
# iptables -I INPUT -p udp --dport 67 -i net0 -j ACCEPT
# iptables -I INPUT -p udp --dport 53 -s 192.168.123.0/24 -j ACCEPT
# iptables -I INPUT -p tcp --dport 53 -s 192.168.123.0/24 -j ACCEPT
```

如果你不打算经常使用这套配置，你也可以手动给每台电脑配置IP。

手动添加IP地址

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Duplicates 网络配置#Static IP address.（在Talk:网络分享讨论）

除了使用DHCP，你还可以在每台客户端电脑上添加IP地址和默认网关：

```
# ip addr add 192.168.123.201/24 dev eth0  # 地址随便填，但是网段和子网掩码必须和服务端在这个子网的配置一样
# ip link set up dev eth0
# ip route add default via 192.168.123.100 dev eth0   # 服务端的地址
```

为每一个客户端电脑配置DNS服务器, 细节位于resolv.conf。

搞定。客户端电脑现在应该有网了。

问题解决

如果你可以连接两台电脑但没法传输数据（比如说，客户端发送了DHCP请求给服务器，服务器受到了并且给了一个IP，但是客户端没收到，最后超时了），检查以确保你没有其他Iptables规则在干扰。

客户端无法连接Wi-Fi或者无法上网

问题还包括：很久都拿不到ip，DHCP offer穿不过虚拟网桥，设备ping电脑的时候得到"host is down", ping外部网络的时候得到"no route to host"……

目前已知docker可能导致这个问题。禁用docker.service和docker.socket就可以解决了。

在github上的issue

另请参见

- Xyne关于设置一个有DHCP和DNS服务器的子网的指引和脚本
- NetworkManager 可以用来配置网络共享。

6.4 路由器

难度 进阶 | **篇幅** 12.0 KB | **所属** 网络共享

本节将讲到

- 硬件需求
- 网络接口配置
- ADSL连接/PPPoE
- DNS 和 DHCP
- 连接共享

此文章是将一台电脑配置为一台网关/路由器的指南。为加强安全性，不应在此机器上运行对外服务。对于局域网（LAN），只运行特定的网关服务而不运行httpd、ftpd、samba、nfsd等属于局域网服务器的服务，因为这样会带来风险。

此文章不涉及如何设置两台PC用交叉网线进行网络连接共享。简单网络连接共享，参见Internet sharing。

注意：在本文中，**intern0**和**extern0**是网络接口的名称。进一步说明参见#网络接口名称约定.

硬件需求

- 至少 1GB 硬盘空间。基本安装约需 500MB 硬盘空间，如果使用web代理缓存还需额外预留缓存空间。

- 至少两个实体网络接口：网关要将两个网络彼此连通（实际上路由器可以用一个物理网络接口实现两个VLAN接口并连接到VLAN交换机，这称为单臂路由/router-on-a-stick，但本文不讨论这种情况）。需要将两个网络连接到同一台实体机器，其中一个网口连接到外部网络，另一个连接内部网络。其中一个接口必须连接到外部网络，其他接口连接到内部网络。
- 一个集线器、交换机或UTP线缆：将其他机器连接到网关。

网络接口配置

网络接口名称约定

Systemd 会自动为网口选择唯一接口名称。命名是持续不变的，直到重启。你可能希望更改接口名称，以便标记不同的网络接口。在以下的说明中，我们约定使用以下名称：

- **intern0**：连接到局域网（LAN）的网络接口。真实名称可能是 enp2s0、enp1s1 等等。
- **extern0**：连接到外部网络或广域网的网络接口。真实名字可能是 enp2s0、enp1s1 等等。

你可以通过Systemd-networkd#Renaming an interface中说明的方法使用Systemd-networkd的配置文件，或通过网络配置#Change interface name中说明的方法使用udev规则重新分配设备名称。由于本文是示例性质，使用以上约定名称。

IP 配置

首先需要配置网络接口。配置网络接口最好的方式是编辑 netctl 配置文件。下面需要创建两个配置文件。

注意：如果仅通过PPPoE连接到互联网（只有一个广域网接口），则**不要**设置或启用外网口配置文件（extern0-profile）。参见#PPPoE配置

- `/etc/netctl/extern0-profile`

```
Description='Public Interface.'
Interface=extern0
Connection=ethernet
IP='dhcp'
```

- `/etc/netctl/intern0-profile`

```
Description='Private Interface'
Interface=intern0
Connection=ethernet
IP='static'
Address=('10.0.0.1/24')
```

注意：上面示例中假定路由器面向整个子网。如果只要面向少数几台机器，可以修改 CIDR 后缀缩减范围。例如 `/27` 是限定范围在 `10.0.0.1` 至 `10.0.0.30`。有很多在线CIDR计算器可用，例如sipcalc包。

下一步是用 netctl 启用网络接口配置。

```
# netctl enable extern0-profile
# netctl enable intern0-profile
```

ADSL连接/PPPoE

可以用 rp-pppoe 将防火墙的 `extern0` 接口连接到ADSL modem并管理该连接。确认已将modem设置为*桥接*（或半桥接或RFC1483）模式，否则modem工作于路由器模式。安装 rp-pppoe包 包。

需要注意的是：如果仅能通过PPPoE连接到互联网（除了能连接modem之外没有其它广域网接口），则不必启用 `extern0-profile` ，因为此时外网口使用名为 ppp0 的伪接口。

PPPoE配置

可以使用 netctl 设置 pppoe 连接。首先，

```
# cp /etc/netctl/examples/pppoe /etc/netctl/
```

然后编辑它。选择连接到modem的接口并配置。如果你仅仅通过PPPoE连接到互联网，这就是 `extern0` 接口。其他字段填入ISP的信息。详情参阅 netctl.profile(5) 手册。

DNS 和 DHCP

可以使用专为小型站点设计的 dnsmasq 为局域网提供DNS和DHCP服务。安装 dnsmasq包 包即可。

Dnsmasq 要配置为DHCP服务器。编辑 `/etc/dnsmasq.conf`：

```
/etc/dnsmasq.conf
interface=intern0 # 使dnsmasq只侦听来自内网口（intern0，即局域网）的请求
expand-hosts      # 为 /etc/hosts 中的主机名添加一个域名
domain=foo.bar    # 允许DHCP主机的完全限定域名（需要启用"expand-hosts"）
dhcp-range=10.0.0.2,10.0.0.255,255.255.255.0,1h # 定义局域网中DHCP地址范围：从 10.0.0.2 至 10.0.0.255，子网掩码为 255.255.255.0，DHCP 租期为 1 小时 （可按需修改）
```

下面将看到，只能添加***静态***DHCP租约。例如，分配一个IP地址给局域网中某一台指定MAC地址的机器。这样，无论这台机器何时请求DHCP，都获得同一个IP地址。对于需要DNS记录的网络服务来说这非常有用。这也可以拒绝特定MAC的机器获取IP地址。

现在启动 `dnsmasq.service` 服务。

连接共享

现在可以连通两个网络接口。

Manual

首先，需要允许数据包从一个网络接口跳转到另一个网络接口。这样需要通过内核中的 sysctl(8) 功能启用包转发功能。详情参阅 Internet sharing#Enable packet forwarding。

假设 `net.**forwarding` 设置正确（即设置为 `1`），数据包仍需要正确地发送和接收。因此，必须在外部网路和内部子网之间转换IP地址。这种技术称为“钳制”（*masquerading*）。还需要两个转发规则保持连接，并使LAN能够转发到WAN。为此，我们可以使用iptables：

```
/etc/iptables/iptables.rules
*nat
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A POSTROUTING -o extern0 -j MASQUERADE
COMMIT

*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
-A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -i intern0 -o extern0 -j ACCEPT
COMMIT
```

如果通过PPPoE连接，还需要将 mss 钳制到 pmtu，防止产生碎片：

```
/etc/iptables/iptables.rules
*mangle
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A FORWARD -o ppp0 -p tcp -m tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu
COMMIT
```

Start 和 enable `iptables.service`。路由器现在应该完全正常运行，并路由数据。但是，要面向公共Internet使用，还需要通过Simple stateful firewall进行额外的保护。

用shorewall共享连接

详细配置说明参见Shorewall。

IPv6说明

很有用的参考： IPv6 和 wikipedia:IPv6。

唯一本地地址

即使ISP没有提供IPv6地址，仍然可以将路由器设置为IPv6模式。除非禁用IPv6，否则所有接口都会分配唯一的 `fe80::/10` 地址。

对于内部网络，已保留地址块 `fc00::/7`。这些地址保证是唯一的，且不可从开放的互联网路由。属于 `fc00::/7` 的地址称为 Unique Local Addresses。要在内部网络生成和使用 generate a ULA /64 block，如希望使用 `fd00:aaaa:bbbb:cccc::/64`，我们首先要在内部接口上分配静态IPv6地址。修改上面的 `intern0-profile`，添加以下一行：

```
Address6=('fd00:aaaa:bbbb:cccc::1/64')
```

这样将ULA添加到内部网络接口。就路由器而言，这是唯一需要配置的地方。

全局单播地址

如果ISP或WAN网络可以访问IPv6 Internet，则可以将全局链路地址分配给路由器，并通过 SLAAC)传播到内部网络。全局单播前缀通常是*静态*或通过*前缀委派*提供的。

静态IPv6前缀

如果ISP提供静态前缀，则编辑 `/etc/netctl/extern0-profile`配置文件，简单添加IPv6地址和前缀即可：

```
Address6=('2002:1:2:3:4:5:6:7/64')
```

可以在使用上述ULA地址之外，同时使用此功能。

通过DHCPv6-PD获取IPv6前缀

如果ISP通过前缀委派处理IPv6，则可以按照main IPv6 article)中的说明进行操作。按照本说明的约定，WAN接口为 `extern0`（或如果通过PPPoE连接时是 `ppp0`），LAN接口为 `intern0`。

路由通告与无状态自动配置(SLAAC)

要向网络客户端正确分发IPv6，需要使用通告守护进程。按照 main IPv6 article 中关于如何设置 `radvd`的说明进行操作。按照本说明的约定，LAN接口是 `intern0`。可以通告所有前缀，或选择哪些前缀分配给本地网络。

可选附加功能

UPnP

上述 shorewall 的配置不包含 UPnP 支持。尽管可能使路由器遭到来自局域网内部的攻击，然而某些应用要求必须使用UPnP。

要在路由器上启用UPnP，需要安装一个UPnPInternet Gateway Device (IGD) protocol。可从官方仓库安装 miniupnpd包 。

更详细信息请查阅 Shorewall guide on UPnP 。

远程管理

OpenSSH 可用于远程管理路由器，十分适合运行在“**无头**”模式（无显示器或输入设备）的机器。

缓存网页代理

参阅 Squid 如何配置代理为网页浏览加速和/或添加额外的安全层。

时间服务器

将路由器用作一台时间服务器，参阅系统时间#时间同步，配置网络时间协议（NTP）服务器。

然后配置 shorewall 或者 iptables 允许 NTP 流量流出。

内容过滤

如果需要内容过滤，可以安装配置 DansGuardian 或者 Privoxy 。

流量共享

流量共享很有用，尤其当局域网中不止一台机器时。基本思路是为不同类型的流量分配优先级。交互型流量（ssh、在线游戏等）可能需要高优先级，而 P2P 流量可分配低优先级。其他流量介于二者之间。

参阅

- Simple stateful firewall
- Internet sharing

6.5 Samba/Active Directory 域控制器

难度 高级 | **篇幅** 27.1 KB | **所属** 网络共享

本节将讲到

- 安装
- 创建新目录
- Adding a second domain controller to an existing domain
- 额外服务
- Tips and tricks

翻译状态：

- 本文（或部分内容）译自 Samba/Active Directory domain controller，最近一次同步于 2024-05-04，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面/Active_Directory_domain_controller_(%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87)?action=history)找到本文翻译的**原始**修订历史。

- 本文可能与英文原文存在出入。

本文描述了如何使用 Samba 搭建 Active Directory 域控制器。它假设所有配置文件都处于安装后的状态，未作任何更改。文章内容已在全新安装系统上验证过，没有（也不需要）除静态 IPv4 地址外的其它网络配置。最后，下面的大多数命令都需要提权。在不考虑操作规范的前提下，使用根用户会话运行命令会比按需提权更方便些。

安装

注意：确保在你的网络上可以通过主机名访问到服务器。详细信息请查看网络配置#局域网主机名解析。

除 Samba 的自带程序外，一个完整可用的 samba 域控制器还需要一些其它软件。先从官方仓库安装 krb5包， ntp包， python-dnspython包， python-markdown包,openresolv包 和 samba包。

Samba 内置了一个全功能的 DNS 服务器，但如果你需要为外部域名维护 DNS 域，强烈建议换用 BIND。如果需要共享打印机，同时也要用到 CUPS。如有这些需要，请安装 bind包 和/或 cups包。

创建新目录

置备

创建 Active Directory 域的第一步是进行置备。这包括配置内部 LDAP、Kerberos 和 DNS 服务器，并按目录需求进行基础设置。如果你有配置目录服务器的经验，那你肯定知道多组件协同工作很可能会产生些问题。这也是为什么 Samba 的开发者选择提供内置版本的这些组件，上面安装的服务器包仅作为客户端工具进行安装。为 Samba 进行置备较为简单，只需执行以下命令：

```
# samba-tool domain provision --use-rfc2307 --interactive
```

参数说明

\--use-rfc2307 : 该参数将 POSIX 属性（UID/GID）添加到了 AD Schema 中。如果你需要认证 Linux，BSD 或是 macOS 客户端（包括本地主机），就需用到该参数。

\--interactive : 该参数强制置备脚本以交互模式运行。

你也可以使用 `samba-tool domain provision --help` 查看置备步骤的文档。

Interactive provision explanations

Realm : **INTERNAL.DOMAIN.COM** - This should be the same as the DNS domain in all caps. It is common to use an internal-only sub-domain to separate your internal domain from your external DNS domains, but it is not required.

Domain : **INTERNAL** - This will be the NetBIOS domain name, usually the leftmost DNS sub-domain but can be anything you like. For example, the name INTERNAL would not be very descriptive. Perhaps company name or initials would be appropriate. This should be entered in all caps, and should have a 15 character maximum length for compatibility with older clients.

Server Role : **dc** - This article assumes that your are installing the first DC in a new domain. If you select anything different, the rest of this article will likely be useless to you.

DNS Backend : **BIND9_DLZ** or **SAMBA_INTERNAL** - This is down to personal preference of the server admin. Again, if you are hosting DNS for external domains, you are strongly encouraged to use the **BIND9_DLZ** backend so that flat zone files can continue to be used and existing transfer rules can co-exist with the internal DNS server. If unsure, use the **SAMBA_INTERNAL** backend.

DNS forwarder IP address : **xxx.xxx.xxx.xxx** or **none** - This option is only presented when using the SAMBA_INTERNAL DNS backend. Supply the IP address of a DNS server for forwarding non local DNS queries, or use the string **none** to always do root lookups.

Administrator password : **xxxxxxxx** - You must select a *strong* password for the administrator account. The minimum requirements are one uppercase letter, one number, and at least eight characters. If you attempt to use a password that does not meet the complexity requirements, provisioning will fail.

配置守护进程

NTPD

为你网络下的时间服务器创建 NTP 配置文件。相关介绍及额外配置可参考 Network Time Protocol daemon 一文。

按照如下内容修改 `/etc/ntp.conf`：

```
/etc/ntp.conf
# Please consider joining the pool:
#
#     http://www.pool.ntp.org/join.html
#
# For additional information see:
# - https://wiki.archlinux.org/index.php/Network_Time_Protocol_daemon
# - http://support.ntp.org/bin/view/Support/GettingStarted
# - the ntp.conf man page

# 关联到 Arch 的 NTP 池
server 0.arch.pool.ntp.org
server 1.arch.pool.ntp.org
server 2.arch.pool.ntp.org
server 3.arch.pool.ntp.org

# 限制
restrict default kod limited nomodify notrap nopeer mssntp
restrict 127.0.0.1
restrict ::1
```

```
restrict 0.arch.pool.ntp.org mask 255.255.255.255 nomodify notrap nopeer noquery
restrict 1.arch.pool.ntp.org mask 255.255.255.255 nomodify notrap nopeer noquery
restrict 2.arch.pool.ntp.org mask 255.255.255.255 nomodify notrap nopeer noquery
restrict 3.arch.pool.ntp.org mask 255.255.255.255 nomodify notrap nopeer noquery

# 偏移文件位置
driftfile /var/lib/ntp/ntpd.drift

# Location of the update directory
ntpsigndsocket /var/lib/samba/ntp_signd/
```

创建状态目录并配置权限：

```
# install -d /var/lib/samba/ntp_signd
# chown root:ntp /var/lib/samba/ntp_signd
# chmod 0750 /var/lib/samba/ntp_signd
```

启用并启动 `ntpd.service`。

BIND

如果你选择了 **BIND9_DLZ** 作为 DNS 后端，需要安装 bind包 并创建如下 BIND 配置，相关介绍及额外配置可参考 BIND 一文。别忘了将 **x** 替换成实际值：

创建 `/etc/named.conf` 文件：

```
/etc/named.conf
// vim:set ts=4 sw=4 et:
acl local-networks {
    127.0.0.0/8;
    xxx.xxx.xxx.xxx/xx;
// 如要使用 IPv6，需移除下面的注释
    //::1/128;
    //xxxx:xxxx:xxxx:xxxx::/64;
};

options {
    directory "/var/named";
    pid-file "/run/named/named.pid";
    session-keyfile "/run/named/session.key";

    // 将该行取消注释以启用 IPv6 连接支持
    //  listen-on-v6 { any; };
    // 为无 IPv4 环境添加该行:
    //  listen-on { none; };

    // 将允许的子网或主机加入到 local-networks acl 中
    allow-query       { local-networks; };
    allow-recursion   { local-networks; };
    allow-query-cache { local-networks; };
    allow-transfer    { none; };
    allow-update      { none; };

    version none;
    hostname none;
    server-id none;

    auth-nxdomain yes;
    datasize default;
    empty-zones-enable no;
    tkey-gssapi-keytab "/var/lib/samba/bind-dns/dns.keytab";

    // 将该行取消注释以使用 ISP 的转发器
    //  forwarders { xxx.xxx.xxx.xxx; xxx.xxx.xxx.xxx; };
};
```

```
zone "localhost" IN {
    type master;
    file "localhost.zone";
};

zone "0.0.127.in-addr.arpa" IN {
    type master;
    file "127.0.0.zone";
};

zone "1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.ip6.arpa" {
    type master;
    file "localhost.ip6.zone";
};

// 加载 AD 内置区域
include "/var/lib/samba/bind-dns/named.conf";

//zone "example.org" IN {
//    type slave;
//    file "example.zone";
//    masters {
//        192.168.1.100;
//    };
//    allow-query { any; };
//    allow-transfer { any; };
//};

logging {
    channel xfer-log {
        file "/var/log/named.log";
            print-category yes;
            print-severity yes;
            severity info;
        };
        category xfer-in { xfer-log; };
        category xfer-out { xfer-log; };
        category notify { xfer-log; };
};
```

配置权限：

```
# chgrp named /var/lib/samba/private/dns.keytab
# chmod g+r /var/lib/samba/private/dns.keytab
# touch /var/log/named.log
# chown root:named /var/log/named.log
# chmod 664 /var/log/named.log
```

启用并启动 `named.service`。

转发器值可以选用你的 ISP 的 DNS 服务器，谷歌（8.8.8.8，8.8.4.4，2001:4860:4860::8888 和 2001:4860:4860::8844）及 OpenDNS（208.67.222.222，208.67.220.220，2620:0:ccc::2 和 2620:0:ccd::2）也免费提供了公共 DNS 服务器，具体最佳值取决于你的现有网络。

Kerberos 客户端工具

上面的置备步骤会创建一个可与 Samba 域控制器搭配使用的 krb5.conf 文件。使用如下命令进行安装：

```
# mv /etc/krb5.conf{,.default}
# cp /var/lib/samba/private/krb5.conf /etc
```

DNS

你现在需要使用本地 DNS 服务器了，重新配置 resolvconf 以仅使用 localhost 进行 DNS 查询。创建 `/etc/resolv.conf.tail` 文件（不要忘了将 **internal.domain.tld** 替换为你的内部域名）：

```
# Samba 配置
search internal.domain.tld
# 如要使用 IPv6，需移除下行注释
#nameserver ::1
nameserver 127.0.0.1
```

配置权限并生成新的 `/etc/resolv.conf` 文件：

```
# chmod 644 /etc/resolv.conf.tail
# resolvconf -u
```

Samba

启用并启动 `samba.service`。如果你想使用 LDB 工具，需额外通过创建 `/etc/profile.d/sambaldb.sh` 来设置 **LDB_MODULES_PATH**：

```
export LDB_MODULES_PATH="${LDB_MODULES_PATH}:/usr/lib/samba/ldb"
```

设置权限并对其使用 source：

```
# chmod 0755 /etc/profile.d/sambaldb.sh
# . /etc/profile.d/sambaldb.sh
```

测试

DNS

首先验证 DNS 是否正常。执行如下命令，将 **internal.domain.com** 和 **server** 替换为对应的值：

```
# host -t SRV _ldap._tcp.internal.domain.com.
# host -t SRV _kerberos._udp.internal.domain.com.
# host -t A server.internal.domain.com.
```

输出应该如下：

```
_ldap._tcp.internal.domain.com has SRV record 0 100 389 server.internal.domain.com.
_kerberos._udp.internal.domain.com has SRV record 0 100 88 server.internal.domain.com.
server.internal.domain.com has address xxx.xxx.xxx.xxx
```

NT 认证

接着，确认密码认证是否正常工作：

```
# smbclient //localhost/netlogon -U Administrator -c 'ls'
```

将会出现提示要求输入密码（即你之前设定的密码），并将出现如下目录列表：

```
  .                                   D        0  Wed Nov 27 23:59:07 2013
  ..                                  D        0  Wed Nov 27 23:59:12 2013

                50332 blocks of size 2097152. 47185 blocks available
```

Kerberos

现在验证 KDC 是否正常工作。确保替换了 **INTERNAL.DOMAIN.COM**，并且使用了全大写字符：

```
# kinit administrator@INTERNAL.DOMAIN.COM
```
将会出现提示要求输入密码，并产生如下输出：

```
Warning: Your password will expire in 41 days on Wed 08 Jan 2014 11:59:11 PM CST
```
检查是否确实拿到了票据：

```
# klist
```
输出应该如下：

```
Ticket cache: FILE:/tmp/krb5cc_0
Default principal: administrator@INTERNAL.DOMAIN.COM

Valid starting       Expires              Service principal
11/28/2013 00:22:17  11/28/2013 10:22:17
krbtgt/INTERNAL.DOMAIN.COM@INTERNAL.DOMAIN.COM
       renew until 11/29/2013 00:22:14
```
最后，通过 smbclient 使用刚拿到的票据进行验证。将 **server** 替换为对应的服务器名：

```
# smbclient //server/netlogon -k -c 'ls'
```
输出应与上述密码认证测试的输出一致。

Additional configuration

DNS

You will also need to create a reverse lookup zone for each subnet in your environment in DNS. It is important that this is kept in Samba's DNS as opposed to BIND to allow for dynamic updates by cleints. For each subnet, create a reverse lookup zone with the following commands. Replace **server**.**internal**.**domain**.**tld** and **xxx**.**xxx**.**xxx** with appropriate values. For **xxx**.**xxx**.**xxx**, use the first three octets of the subnet in reverse order (for example: 192.168.0.0/24 becomes 0.168.192):

```
# samba-tool dns zonecreate server.internal.domain.tld xxx.xxx.xxx.in-addr.arpa -U
Administrator
```
Now, add a record for you server (if your server is multi-homed, add for each subnet) again substituting appropriate values as above. **zzz** will be replaced by the fourth octet of the IP for the server:

```
# samba-tool dns add server.internal.domain.tld xxx.xxx.xxx.in-addr.arpa zzz PTR
server.internal.domain.tld -U Administrator
```
Finally, test the lookup. Replace **xxx**.**xxx**.**xxx**.**xxx** with the IP of your server:

```
# host -t PTR xxx.xxx.xxx.xxx
```
You should get output similar to the following:

```
xxx.xxx.xxx.xxx.in-addr.arpa domain name pointer server.internal.domain.tld.
```

TLS

默认不启用 TLS，但在初始化 DC 时会创建一个默认证书。从 Samba 4.3.8 和 4.2.2 开始，默认禁用非加密 LDAP 绑定。在不影响安全性的前提下，你必须要配置 TLS 才能将 Samba 作为认证源使用。如要使用默认证书，请将下列内容添加到 `/etc/samba/smb.conf` 的“\[global\]”部分下：

```
/etc/samba/smb.conf
```

```
[global]
tls enabled  = yes
tls keyfile  = tls/key.pem
tls certfile = tls/cert.pem
tls cafile   = tls/ca.pem
```

如果需要可信证书，请创建一个签名密钥和一个证书请求（详细步骤参见 OpenSSL）。让你选择的证书机构对请求签名，然后将其放置在配置的文件夹下。如果证书机构要求使用中间证书，可以将证书串在一起（先是服务器证书，然后是中间证书），然后将 **tls cafile** 留空。

重启 `samba` 以应用更改。

Adding a second domain controller to an existing domain

Prerequisites

As with the provisioning setup when setting up a new domain, you must have ntp包 configured per the above instructions. Additionally, some of the arguments and parameters on the original domain setup must be replicated here.

Argument explanations

\--option='idmap_ldb:use rfc2307 = yes' : this is required if you elected to include Unix UID/GID support on your existing domain (using the **\--use-rfc2307** option for Samba's provision step or applied the RFC 2307 schema extensions).

\--dns-backend=DNSTYPE** **: replace** DNSTYPE **with** BIND9_DLZ **or** SAMBA_INTERNAL **- This is again down to personal preference of the server admin. If using** BIND9_DLZ** backend, you will need to configure bind包 as per the above instructions after joining the domain.

\--option="dns forwarder="xxx.xxx.xxx.xxx**"** : this is only valid for the SAMBA_INTERNAL DNS backend which allows you to specify a DNS forwarder. Replace **xxx.xxx.xxx.xxx** with appropriate value.

\--site=SITE**** : if you have multiple sites defined, use this to join directly in that site.

See the output of `samba-tool domain join --help` for additional options.

Joining an existing domain as a new DC

Execute the following command (adding any necessary parameters above to the end of the command):

```
# samba-tool domain join internal.domain.tld DC -U"INTERNAL\administrator"
```

Now copy the `krb5.conf`:

```
# cp /var/lib/samba/private/krb5.conf /etc/krb5.conf
```

If you used the RFC 2307 schema extensions, you need to copy the idmap from an existing DC. If using Samba, execute the following command from another DC:

```
# tdbbackup -s .bak /var/lib/samba/private/idmap.ldb
```

This will generate a file `/var/lib/samba/private/idmap.ldb.bak`, transfer this file to the new server in the `/var/lib/samba/private` directory, removing the .bak extension. If you intend to keep multiple DCs, you will need to automate this process going forward using one of the methods listed on the Samba website here. This also applies to transferring the idmap from Windows DCs.

Enable and start the `samba.service` unit.

If using BIND9_DLZ DNS backend, you'll need to follow the BIND section above. Check if the `/var/lib/samba/private/dns` directory is created, and if not, run the following command (ignore warning about updating named.conf):

```
# samba_upgradedns --dns-backend=BIND9_DLZ
```

Restart the `named.service` and then update the DNS records with the following command:

```
# samba_dnsupdate --all-names --use-samba-tool --verbose
```

Now proceed with LDB configuration and testing as with a new domain here.

Transferring the FSMO roles

If this is intended to replace an existing domain controller, you will need to transfer the FSMO roles before demoting the existing DC. This is currently outside the scope of this document. See the Samba wiki here.

额外服务

打印

作为域控制器的 Samba 服务器默认不会启用打印服务。你需要在 `/etc/samba/smb.conf` 的 global 部分添加下列内容：

```
/etc/samba/smb.conf
[global]
...
        rpc_server:spoolss = external
        rpc_daemon:spoolssd = fork
        printing = CUPS
...
[printers]
       path = /var/spool/samba/
       printable = yes
```

以上示例会为所有 CUPS 打印队列启用自动共享。如果你只想共享特定打印队列，可以添加如下内容（先将上面的 **\[printers\]** 共享移除）：

```
/etc/samba/smb.conf
[global]
...
        load printers = no
...
# 打印共享示例
[HPDJ3050]
       path = /var/spool/samba/
       printable = yes
       printer name = hpdj3050
```

Tips and tricks

DHCP with dynamic DNS updates

It should be noted that using this method will affect functionality of windows clients, as they will still attempt to update DNS on their own. When this occurs, the machine will be denied permission to do so as the record will be owned by the dhcp user rather than the machine account. While this is essentially harmless, it will generate warnings in the system log of the offending machine. You should create a GPO to overcome this, but unfortunately, Samba does not yet have a command line utility to modify GPOs. You will need a Windows PC with the RSAT tools installed. Simply create a dedicated GPO with the Group Policy Editor, and apply only to OUs that contain workstations using DHCP (so that Samba/Windows servers and statically configured Samba/Windows clients can still update using 'ipconfig /registerdns') and configure the following settings:

```
Computer Configuration
  Policies
    Administrative Templates
      Network
        DNS Client
          Dynamic Update = Disabled
          Register PTR Records = Disabled
```

Install the dhcp包, sudo包, and the samba-dhcpd-updateAUR packages.

Create an unprivileged user in AD for performing the updates. When prompted for password, use a secure password. 63 random, mixed case, alpha-numeric characters is sufficient. Optionally samba-tool also takes a random argument:

```
# samba-tool user create dhcp --description="Unprivileged user for DNS updates via DHCP server"
```

Since this is a service account, disabling password expiration on the user account is recommended, but not required:

```
# samba-tool user setexpiry dhcp --noexpiry
```

Give the user privileges to administer DNS:

```
# samba-tool group addmembers DnsAdmins dhcp
```

Create an SPN and export the users credentials to a private keytab:

```
# samba-tool spn add server/server.internal.domain.tld@INTERNAL.DOMAIN.TLD dhcp
# samba-tool domain exportkeytab --principal=dhcp@INTERNAL.DOMAIN.TLD dhcpd.keytab
# install -vdm 755 /etc/dhcpd
# mv dhcpd.keytab /etc/dhcpd
# chown root:root /etc/dhcpd/dhcpd.keytab
# chmod 400 /etc/dhcpd/dhcpd.keytab
```

Modify the `dhcpd-update-samba-dns.conf` file with the following commands (substituting correct values for **server**, **internal**.**domain**.**tld**, and **INTERNAL**.**DOMAIN**.**TLD**):

```
/etc/dhcpd/dhcpd-update-samba-dns.conf
# Variables
KRB5CC="/tmp/dhcpd4.krb5cc"
KEYTAB="/etc/dhcpd/dhcpd.keytab"
DOMAIN="internal.domain.tld"
REALM="INTERNAL.DOMAIN.TLD"
PRINCIPAL="dhcp@${REALM}"
NAMESERVER="server.${DOMAIN}"
```

```
ZONE="${DOMAIN}"
```

Grant the dhcp user account permissions to run the update script without a password prompt (replace **server** with the hostname of the server):

```
/etc/sudoers.d/dhcp-update
dhcp server = (root) NOPASSWD: /usr/bin/dhcpd-update-samba-dns.sh
```

Configure the dhcpd server following the dhcpd article and add the following to all subnet declarations in the `/etc/dhcpd.conf` file that provide DHCP service:

```
  on commit {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "add", ClientIP,
ClientName);
  }

  on release {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "delete", ClientIP,
ClientName);
  }

    on expiry {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "delete", ClientIP,
ClientName);
```

Here is a complete example `/etc/dhcpd.conf` file for reference:

```
/etc/dhcpd.conf
subnet 192.168.1.0 netmask 255.255.255.0 {
  range 192.168.1.100 192.168.1.199;
  option subnet-mask 255.255.255.0;
  option routers 192.168.1.254;
  option domain-name "internal.domain.tld";
  option domain-name-servers 192.168.1.1;
  option broadcast-address 192.168.1.255;
  default-lease-time 28800;
  max-lease-time 43200;
  authoritative;

  on commit {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "add", ClientIP,
ClientName);
  }

  on release {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "delete", ClientIP,
ClientName);
  }

    on expiry {
    set ClientIP = binary-to-ascii(10, 8, ".", leased-address);
    set ClientName = pick-first-value(option host-name, host-decl-name);
    execute("/usr/bin/sudo", "/usr/bin/dhcpd-update-samba-dns.sh", "delete", ClientIP,
ClientName);
  }
}
```

Finally, enable and start (or restart) the `dhcpd4` service.

Transferring users from one directory to another

Unfortunately, there is no built-in utility to export users from one directory to another. This is one way, albeit exceptionally ugly, to get the user specific fields out of your existing SAM and into a suitable LDIF format for ldbmodify:

```
# ldbsearch -H /var/lib/samba/private/sam.ldb \
    -s sub -b cn=Users,dc=internal,dc=domain,dc=tld '(objectClass=user)' | \
    grep -e "^\# record" -e "^accountExpires:" -e "^c:" -e "^cn:" -e "^co:" -e
"^codePage:" \
         -e "^comment:" -e "^company:" -e "^countryCode:" -e "^department:" \
         -e "^description:" -e "^displayName" -e "^displayNamePrintable:" \
         -e "^distinguishedName" -e "^division:" -e "^dn:" -e "^employeeID:" \
         -e "^facsimileTelephoneNumber:" -e "^generationQualifier:" \
         -e "^givenName" -e "^homeDirectory:" -e "^homeDrive:" -e "^homePhone:" \
         -e "^homePostalAddress:" -e "^info:" -e "^initials:" \
         -e "^internationalISDNNumber:" -e "^ipPhone:" -e "^l:" -e "^mail:" \
         -e "^manager:" -e "^middleName:" -e "^mobile:" -e "^name:" -e "^o:" \
         -e "^objectClass" -e "^otherFacsimileTelephoneNumber:" \
         -e "^otherHomePhone:" -e "^otherIpPhone:" -e "^otherMailbox:" \
         -e "^otherMobile:" -e "^otherPager:" -e "^otherTelephone:" -e "^pager:" \
         -e "^personalTitle:" -e "^physicalDeliveryOfficeName:" -e "^postalAddress:" \
         -e "^postalCode:" -e "^postOfficeBox:" -e "^proxyAddresses\: SMTP" \
         -e "^proxyAddresses: smtp" -e "^referredDeliveryMethod:" \
         -e "^primaryInternationalISDNNumber:" -e "^primaryTelexNumber:" \
         -e "^profilePath:" -e "^registeredAddress:" -e "^sAMAccountName:" \
         -e "^scriptPath:" -e "^sn:" -e "^st:" -e "^street:" -e "^streetAddress:" \
         -e "^telephoneNumber:" -e "^teletexTerminalIdentifier:" \
         -e "^telexNumber:" -e "^title:" -e "^userAccountControl:" -e
"^userPrincipalName:"\
         -e "^url:" -e "^userSharedFolder:" -e "^userSharedFolderOther:" -e
"^wWWHomePage:" | \
    sed '/^dn:.*/ a\changetype: add' | sed '/^# record/ i\\n' > user-export.ldif
```

Explanation: Run an ldbsearch in the users container only, using sub-tree search for objectclass=user. If you need the whole directory, you can modify the search base to use the root or some other OU. The output from ldbsearch is then piped into a really long grep command that returns only appropriate attributes to keep in the new directory. This is obviously subjective, and probably should be tailored to your specific use case. Finally, we use sed to insert the changetype line (needed to tell ldbmodify that we are adding a user), and prefix with a blank line (to make it easier to read) for each exported object.

注意：You will need to modify the output file and remove any objects that you do not want transferred. The output file will contain objects (service users, built-ins, etc.) that can break your new directory if you fail to remove them! It will also contain the old domain in both the "dn" and "distinguishedName" attributies that must be changed before import.

To import, after editing the file and transferring to the new server, simply run the following command on your new samba domain controller:

```
# ldbmodify -H /var/lib/samba/private/sam.ldb user-export.ldif
```

密码复杂度

Samba 默认要求使用强密码。使用以下命令可以禁用复杂度检查：

```
# samba-tool domain passwordsettings set --complexity=off
```

详细信息请参考 Samba 维基的密码设置对象一节。

6.6 NFS

难度 高级 | **篇幅** 32.8 KB | **所属** 网络共享

本节将讲到

- 安装
- 服务端配置
- 客户端配置
- 提示和技巧
- 故障排查

来自维基百科：NFS 网络文件系统（Network File System）是由 Sun 公司 1984 年发布的分布式文件系统协议。它允许客户端上的用户像访问本地文件一样地访问网络上的文件。

注意：

- 默认情况下，NFS不提供加密功能. 在处理敏感数据时,请使用 #TLS 加密 或 Kerberos（使用 `sec=krb5p` 提供基于 Kerberos 的加密），也可以使用类似 WireGuard 的加密 VPN) 协议传输NFS流量.
- 与 Samba 不同, NFS默认不提供任何验证用户的方法,客户端访问限制是通过 IP 地址和/或计算机名实现的。如果需要更强的加密方式，也可以使用 Kerberos。
- NFS期望用户和/或用户组的 ID 在客户端与服务端上是相同的（使用 Kerberos 的情况下除外）。使用启用 NFSv4 ID 映射,或使用`anonuid`/`anongid`并在`/etc/exports`中启用`all_squash`,手动改变 UID/GID 来解决这一问题.
- NFS 不支持 POSIX ACL。NFS 服务器还是会强制应用 ACL 规则，但客户端无法观察或是对其进行修改。

安装

客户端和服务端都只需要安装 nfs-utils包 包。

强烈建议使用时间同步守护进程以保持客户端/服务器之间的时间同步，如果各个节点上没有精确同步的时钟，NFS 可能产生非预期的延迟。

服务端配置

全局设置选项在 `/etc/nfs.conf` 中被列出.只进行简单配置的用户无需编辑此文件.

NFS 服务器需要按照 `/etc/exports` 或 `/etc/exports.d/*.exports` 文件中定义的“导出”文件夹列表进行共享(详细介绍参见 exports(5))。默认情况下，这些目录都会按路径原样导出，例如：

```
/etc/exports
/data/music    192.168.1.0/24(rw)
```

以上配置将把 /data/music 暴露到 MyServer:/data/music 路径下，可通过 NFSv3 或 NFSv4 进行挂载。

指定导出根目录

共享对象是相对于所谓的“NFS 根目录”的.出于安全考虑，建议定义一个单独的目录为 NFS 根，这可以将用户限制在该挂载点中。绑定的挂载点（bind mounts）将文件系统上他处的目录与被分享的挂载点连接起来。之前 NFSv4 强制要求使用 NFS 根目录，现在成了可选项（从内核 2.6.33 和 nfs-utils 1.2.2 版本开始实现了虚拟根目录）。

查看下面的例子。在本例中：

1. NFS 根目录是 /srv/nfs
2. 将要共享的目录是 /srv/nfs/music，该目录以绑定挂载方式指向了它实际的位置/mnt/music。

```
# mkdir -p /srv/nfs/music /mnt/music
# mount --bind /mnt/music /srv/nfs/music
```

注意：ZFS 文件系统对于 bindmounts 方式挂载要特殊处理，参阅 ZFS#Bind mount。

为了让服务器重启后共享仍旧有效，增加绑定到 fstab 文件：

```
/etc/fstab
/mnt/music /srv/nfs/music  none   bind   0   0
```

增加允许被挂载的目录和使其只能被属于特定 CIDR 所指定的 IP 范围或拥有特定主机名的客户端所访问的限制至 /etc/exports 文件,例如：

```
/etc/exports
/srv/nfs         192.168.1.0/24(rw,fsid=root)
/srv/nfs/music   192.168.1.0/24(rw,sync)
/srv/nfs/home    192.168.1.0/24(rw,sync)
/srv/nfs/public 192.168.1.0/24(ro,all_squash,insecure)
desktop(rw,sync,all_squash,anonuid=99,anongid=99) # 将访客映射到特定用户组 - 在本例中是 nobody
```

在使用 NFSv4 时，fsid=root 或 fsid=0 选项指定了导出目录“根”的位置；如果使用这些选项指定了导出目录，那其它文件夹都必须位于该文件夹下。/etc/nfs.conf 文件中的 rootdir 选项在这种情况下不起效。如果没有指定 fsid=0，那默认行为与 NFSv3 一致。

在上面的实例中，/srv/nfs 被指定为根，且可以通过 NFSv4 按 MyServer:/music 目录挂载 /srv/nfs/music 文件夹 - 注意，根目录前缀被省略了。

提示：

- 对 NFSv3 而言（不适用于 NFSv4），`crossmnt` 选项使客户端可以访问所有挂载在文件系统上并标记有 `crossmnt` 的文件系统并且客户端不需要单独逐个挂载子共享. 请注意,你可能不希望在子共享同时被共享到另一端地址时使用该选项.
- 除了 `crossmnt` 之外，你也可以在子共享上使用 `nohide` 选项，这样的话,子共享就会在根共享被挂载时自动挂载。与 `crossmnt` 不同的是，`nohide` 仍然会遵守子共享的地址范围。注意，该选项仅适用于 NFSv3，NFSv4 *永远*表现为该选项已启用。
- `insecure` 选项使客户端可以用高于 1023 的端口进行连接。（大概是只有 root 用户可以使用较低编号的端口，因此阻断其它端口可以作为简单的访问控制方法。在实际使用中，使用 `insecure` 选项与否并不能带来任何安全方面的提升或是下降。）
- 使用一个通配符（`*`）以允许来自所有接口的访问.

如果服务运行时修改了 `/etc/exports` 文件， 你需要重新导出使其生效:

```
# exportfs -arv
```

想要查看已经加载的共享的详细信息,请使用:

```
# exportfs -v
```

有关所有可用选项的详细介绍,请参阅exports(5).

提示：ip2cidr 是一个可以将 IP 范围转换为 CIDR 的工具.

注意：如果共享是tmpfs文件系统,你需要指定`fsid=1`选项.

开始运行服务

- 要提供 NFSv3 和 NFSv4 服务，启动和启用 `nfs-server.service`。
- 要仅提供 NFSv4 服务，启动和启用 `nfsv4-server.service`。

v4 版本协议的用户可能需要至少屏蔽掉 `rpcbind.service` 和 `rpcbind.socket`，以防止不需要的服务自启，详细信息请参考 。另外，也可以禁用 `nfs-server.service` 以防止其由于某些原因而被拉起。

注意：在启用ZFS共享的同时，请一并启动/启用 `zfs-share.service`。如果不这么做的话，在重启之后 ZFS 目录不会被共享.参阅 ZFS#NFS.

限制 NFS 使其只允许来自特定接口/IP 地址的访问

默认情况下，启动 `nfs-server.service` 会忽略 `/etc/exports` 文件的内容，而是在所有网络接口上监听连接。可以通过定义监听的IP和/或主机名来改变这一行为。

```
/etc/nfs.conf
[nfsd]
host=192.168.1.123
# 或者也可以使用主机名
# host=myhostname
```

在修改完后，重启 `nfs-server.service` 以应用设置。

防火墙配置

如果要访问防火墙后的 NFSv4 服务器，就需要开放 `2049` 端口的 TCP 上行连接。（NFSv4 使用静态网络端口；它不使用任何像 mountd 或是 portmapper 这样的辅助服务。）

如果要访问 NFSv3 服务器，就还需要额外为 portmapper（rpcbind）开放 `111` 端口的 TCP/UDP 连接，还需要为 MOUNT（rpc.mountd）开放端口。默认情况下，rpc.mountd 会动态选择端口，所以如果你要访问防火墙后的服务，就需要通过编辑 `/etc/nfs.conf` 配置来选择一个固定端口。可以使用 `rpcinfo -p` 查看 NFSv3 服务器使用的具体端口信息：

```
$ rpcinfo -p
100003    3   tcp   2049  nfs
100003    4   tcp   2049  nfs
100227    3   tcp   2049  nfs_acl
...
```

客户端配置

打算将 NFS4 与 Kerberos 一起使用的用户需要启动并启用 `nfs-client.target`.

手动挂载

对于NFSv3，请使用以下命令显示服务器分享的文件系统：

```
$ showmount -e servername
```

对于NFSv4，请挂载NFS根目录，并查看可用的挂载：

```
# mount servername:/ /mountpoint/on/client
```

然后挂载分享.挂载时省略服务器的NFS分享根目录：

```
# mount -t nfs -o vers=4 servername:/music /mountpoint/on/client
```

如果挂载失败，请尝试包括服务器的分享根目录（对于Debian/RHEL/SLES是必需的，某些发行版需要使用`-t nfs4`而不是`-t nfs`）：

```
# mount -t nfs -o vers=4 servername:/srv/nfs/music /mountpoint/on/client
```

注意：`*servername*` 必须被替换为有效的主机名（而不仅仅是IP地址）。否则，挂载远程共享将挂起。

使用/etc/fstab挂载

对于服务器保持开启，且无论客户端何时启动皆可用的 NFS 共享而言，fstab 非常有用。编辑 `/etc/fstab`，然后根据需要添加一行新配置。同样，服务端的 NFS 导出根路径已被缺省。

```
/etc/fstab
servername:/music   /mountpoint/on/client   nfs   defaults,timeo=900,retrans=5,_netdev
0 0
```

注意：更多挂载选项可参考 nfs(5) 和 mount(8)。

以下为一些可参考的挂载选项：

rsize 和 wsize：`rsize` 指单次请求从服务器读取的字节数，`wsize` 指单次请求向服务器写入的字节数。默认情况下，如果未指定值，客户端和服务端会协商使用它们都支持的最大值（参考 nfs(5)）。修改值后，建议进行性能测试（参考#性能调优）。

soft 或 hard：决定在 NFS 请求超时后的恢复行为。如果没有指定（或者指定了 `hard` 选项），NFS 会无限重试请求。如果指定了 `soft` 选项，那么 NFS 客户端会在重试 *retrans* 指定的次数后失败，导致 NFS 客户端向发起应用返回错误。

警告：`soft`（软）超时在某些情况下会导致数据静默损坏。因此，建议仅在客户端响应度优先于于数据完整性的情况下使用 `soft` 选项。使用基于 TCP 传输的 NFS 或增加 `retrans` 选项数值有助于降低使用 `soft` 导致的故障风险。

timeo：`timeo` 表示在遇到 RPC 超时后重发前的等待时间，单位为十分之一秒。对基于 TCP 的 NFS 而言默认值为 600（即 60 秒）。在首次超时后，超时值都将加倍，最大不超过 60 秒或是出现重大超时。在连接到慢速服务器或是使用高负载网络时，增加超时值有助于提高连接稳定性。

retrans：NFS 客户端在执行下一步恢复操作前的请求重试次数。如果未指定 `retrans` 值，那 NFS 客户端会为请求重试 3 次。在 *retrans* 次重试后，NFS 客户端会生成一条“server not responding”消息，然后进行下一步恢复尝试（取决于是否使用了硬挂载选项）。

_netdev：`_netdev` 选项告知系统在网络就绪后再尝试挂载该共享 - systemd 会自动为 NFS 使用该项。

注意：将第六项（`fs_passno`）设为非零值可能会导致预期外行为，例如当 systemd 自动挂载等待永不会发生的检查时挂起

搭配 /etc/fstab 和 systemd 进行挂载

另一种方法是使用 x-systemd.automount 选项，它会在访问时挂载文件系统：

```
/etc/fstab
servername:/home    /mountpoint/on/client   nfs   _netdev,noauto,x-systemd.automount,x-systemd.mount-timeout=10,timeo=14,x-systemd.idle-timeout=1min 0 0
```

为了让 systemd 意识到 fstab 的更改，需要重新加载 systemd 并重启 `remote-fs.target`。

**本文或本章节的事实准确性存在争议。**

原因： 不是所有人都用 NetworkManager。是否可以用Systemd#在网络已连接后再启动某服务替代？（在 Talk:NFS 中讨论）

提示：

- `noauto` 选项使得在访问 NFS 共享前不会进行挂载；使用 `auto` 会立刻进行挂载。

如果遇到了网络未拉起/不可用导致的挂载失败，可以启用 `NetworkManager-wait-online.service`。它会确保在激活前 `network.target` 已准备好所有链接可用。

- `users` 选项允许普通用户进行挂载，但请注意，它同时启用了如 `noexec` 等隐藏挂载选项。
- `x-systemd.idle-timeout=1min` 会在 1 分钟内未使用 NFS 共享的情况下自动将其卸载。该选项对于随时会断网的笔记本环境来说比较有用。
- 如果由于 NFS 导致关机/重启时间过长，可以启用 `NetworkManager-wait-online.service` 来确保 NetworkManager 在 NFS 卷卸载前不会关闭。
- 不要添加 `x-systemd.requires=network-online.target` 挂载选项，该选项可能会导致 systemd 出现启动顺序循环 。systemd 会根据 `_netdev` 选项自动将 `network-online.target` 添加为单元依赖。
- `nocto` 选项可能会提高只读挂载的性能，但应只用在服务器上数据不经常出现变更的情况下。

作为 systemd 单元

在`/etc/systemd/system`目录下创建一个新`.mount` 文件,例如`mnt-home.mount`. 有关详细信息，请参见systemd.mount(5).

注意：确保文件名与您要使用的挂载点相对应.例如,仅当要将共享挂载到 `/mnt/home` 时才能使用单元名称 `mnt-home.mount`，否则可能会发生以下错误：`systemd: mnt-myshare.mount: Where= setting does not match unit name. Refusing.`。如果挂载点包含非 ASCII 字符，请使用 systemd-escape。

`What=` 分享的路径

`Where=` 分享应当被挂载的路径

`Options=` 挂载分享的选项

{{注意|

- 网络安装单元会自动获取对 `remote-fs-pre.target`, `network.target` 和 `network-online.target` 的 `After` 依赖，并获得对 `remote-fs.target` 的`Before` 依赖，除非设置了 `nofail` 挂载选项。在后一种情况下,还会添加一个 `Wants` 单元。
- 添加 `noauto` 到 `Options` 可以防止启动时自动挂载（除非被其它单元触发）。
- 如果你要为共享服务器使用域名而不是 IP 地址，则需要添加 `nss-lookup.target` 到 `After` 下。这可能会避免测试中没有问题，但启动时挂载出错的情况出现。

```
/etc/systemd/system/mnt-home.mount
[Unit]
Description=Mount home at boot
```

```
[Mount]
What=172.16.24.192:/home
Where=/mnt/home
Options=vers=4
Type=nfs
TimeoutSec=30

[Install]
WantedBy=multi-user.target
```

提示：为处理网络出现中断的情况，可添加 `ForceUnmount=true` 到 `[Mount]`，该选项使挂载可以被强制卸载。

要使用 `mnt-home.mount`，启动该单元，然后将其启用以在系统启动时进行挂载。

自动挂载

要想自动挂载一个分享,你可以使用下面的自动挂载单元:

```
/etc/systemd/system/mnt-home.automount
[Unit]
Description=Automount home

[Automount]
Where=/mnt/home

[Install]
WantedBy=multi-user.target
```

禁用/停止 `mnt-home.mount` 单元，然后启用/启动 `mnt-home.automount` 以在路径可用时自动挂载共享。

提示：添加 `TimeoutIdleSec` 以启用自动卸载，详细信息可参考 systemd.automount(5)。

使用 autofs 挂载

Using autofs is useful when multiple machines want to connect via NFS; they could both be clients as well as servers. The reason this method is preferable over the earlier one is that if the server is switched off, the client will not throw errors about being unable to find NFS shares. See autofs#NFS network mounts for details.

提示和技巧

启用 NFSv4 ID 映射

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：The instructions are too convoluted. Separate sections for client and server configuration would be better than notes like "Enabling/starting nfs-idmapd.service should not be needed on the client" or "Do not confuse the nfsidmap (only for nfs clients) with nfs-idmapd.service".（在Talk:NFS讨论）

注意：

- NFSv4 ID 映射无法解决默认 `sec=sys` 挂载选项导致的所有问题。更多信息请参考 NFS#静态映射 和
- 在客户端和服务端上均需要启用 NFSv4 ID 映射

- 另一个选择是确保用户 ID 和组 ID（UID 和 GID）在客户端和服务器上一致。
- 不需要启用/启动 `nfs-idmapd.service`，因为它已被新的 ID 映射器替换：

```
# dmesg | grep id_resolver
[ 3238.356001] NFS: Registering the id_resolver key type
[ 3238.356009] Key type id_resolver registered
```

提示：

- Do not confuse the `nfsidmap` (only for nfs **clients**) with `nfs-idmapd.service` which is used by the NFS **server** and forks the process `rpc.idmapd`.
- Both rpc.idmapd and nfsidmap share also some configurations from idmapd.conf(5).
- 更多信息请参考 idmapd(8) 和 nfsidmap(8)。

NFSv4 协议将客户端的 UID 和 GID 值表示为 `*user*@*domain*` 形式的字符串.将UID和字符串互相转换的过程称为 *ID 映射*。

域名

注意：

- By default, the domain part of the string is the system's DNS domain name. It can also be specified in /etc/idmapd.conf if the system is multi-homed, or if the system's DNS domain name does not match the name of the system's Kerberos realm.
- When the domain is not specified in /etc/idmapd.conf the local DNS server will be queried for the _nfsv4idmapdomain text record. If the record exists that will be used as the domain. When the record does not exist, the domain part of the DNS domain will used.

Display the system's effective NFSv4 domain name on stdout.

```
# nfsidmap -d
domain.tld
```

Edit to match up the Domain on the server and/or client:

```
/etc/idmapd.conf
[General]
Domain = guestdomain.tld
```

静态映射

提示：

- 这个映射仅用于客户端本地映射 uid。如果您创建一个属于某个在服务器上不被识别的 uid（例如 1005）的文件。该文件会以 uid 1005 的形式存储在服务器上，但在传输过程中不会再显示正确的 uid。
- 在与服务器交互（例如列出文件）之后，您可以查看密钥环中的所有条目。

```
# nfsidmap -l
7 .id_resolver keys found:
uid:nobody
user:1
uid:bin@domain.tld
uid:foo@domain.tld
gid:foo@domain.tld
uid:remote_user@domain.tld
uid:root@domain.tld
```

- 您可以使用命令 `nfsidmap -c` 清除密钥环，但在默认设置中，条目会在 10 分钟后过期，因此并不需要手动清除。

这些步骤只在服务器和客户端具有不同的用户/组名称时才需要进行。

更改只会在客户端的配置文件中进行。

```
/etc/idmapd.conf
[Translation]
# The default is nsswitch and other methods exist.
method = static,nsswitch

[Static]
foo@domain.tld = local_foo
remote_user@domain.tld = user
```

后备映射

Only in the client configuration. Local user/group name to be used when a mapping cannot be completed:

```
/etc/idmapd.conf
[Mapping]
Nobody-User = nobody
Nobody-Group = nobody
```

性能调优

When using NFS on a network with a significant number of clients one may increase the default NFS threads from *8* to *16* or even a higher, depending on the server/network requirements:

```
/etc/nfs.conf
[nfsd]
threads=16
```

It may be necessary to tune the `rsize` and `wsize` mount options to meet the requirements of the network configuration.

In recent linux kernels (>2.6.18) the size of I/O operations allowed by the NFS server (default max block size) varies depending on RAM size, with a maximum of 1M (1048576 bytes), the max block size of the server will be used even if nfs clients requires bigger `rsize` and `wsize`. See https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/5/html/5.8_technical_notes/known_issues-kernel It is possible to change the default max block size allowed by the server by writing to the `/proc/fs/nfsd/max_block_size` before starting *nfsd*. For example, the following command restores the previous default iosize of 32k:

```
# echo 32768 > /proc/fs/nfsd/max_block_size
```

注意：This is mainly useful for 32-bit servers when dealing with the large numbers of nfsd threads. Lowering the `max_block_size` may decrease NFS performance on modern hardware.

To make the change permanent, create a systemd-tmpfile:

```
/etc/tmpfiles.d/nfsd-block-size.conf
w /proc/fs/nfsd/max_block_size - - - - 32768
```

To mount with the increased `rsize` and `wsize` mount options:

```
# mount -t nfs -o rsize=32768,wsize=32768,vers=4
servername:/srv/nfs/music /mountpoint/on/client
```

Furthermore, despite the violation of NFS protocol, setting `async` instead of `sync` or `sync,no_wdelay` may potentially achieve a significant performance gain especially on spinning disks. Configure exports with this option and then execute `exportfs -arv` to apply.

```
/etc/exports
/srv/nfs         192.168.1.0/24(rw,async,crossmnt,fsid=0)
/srv/nfs/music   192.168.1.0/24(rw,async)
```

警告：Using `async` comes with a risk of possible data loss or corruption if the server crashes or restarts uncleanly.

处理自动挂载

This trick is useful for NFS-shares on a wireless network and/or on a network that may be unreliable. If the NFS host becomes unreachable, the NFS share will be unmounted to hopefully prevent system hangs when using the `hard` mount option .

Make sure that the NFS mount points are correctly indicated in fstab:

```
/etc/fstab
lithium:/mnt/data            /mnt/data              nfs noauto 0 0
lithium:/var/cache/pacman    /var/cache/pacman    nfs noauto 0 0
```

注意：

- Use hostnames in fstab for this to work, not IP addresses.
- In order to mount NFS shares with non-root users the `users` option has to be added.
- The `noauto` mount option tells systemd to not automatically mount the shares at boot, otherwise this may cause the boot process to stall.

Create the `auto_share` script that will be used by cron or systemd/Timers to use ICMP ping to check if the NFS host is reachable:

警告：

- 如果你在使用 btrfs，请将 `sed -e '/^.*#/d' -e '/^.*:/!d' -e 's/\t/ /g' /etc/fstab` 替换为如 `sed -n -e '/^[a-zA-Z0-9.-]*\(\.[a-zA-Z]\+\)\{0,1\}:/p' /etc/fstab`，并测试其输出是否仅包含网络共享，以防如压缩选项等参数导致本地分区被包含进 *NET_MOUNTS* 变量中。

```
/usr/local/bin/auto_share
#!/bin/bash

function net_umount {
  umount -l -f $1 &>/dev/null
}

function net_mount {
  mountpoint -q $1 || mount $1
}

NET_MOUNTS=$(sed -e '/^.*#/d' -e '/^.*:/!d' -e 's/\t/ /g' /etc/fstab | tr -s " ")$'\n'b

printf %s "$NET_MOUNTS" | while IFS= read -r line
do
  SERVER=$(echo $line | cut -f1 -d":")
  MOUNT_POINT=$(echo $line | cut -f2 -d" ")
```

```
  # Check if server already tested
  if [[ "${server_ok[@]}" =~ "${SERVER}" ]]; then
    # The server is up, make sure the share are mounted
    net_mount $MOUNT_POINT
  elif [[ "${server_notok[@]}" =~ "${SERVER}" ]]; then
    # The server could not be reached, unmount the share
    net_umount $MOUNT_POINT
  else
    # Check if the server is reachable
    ping -c 1 "${SERVER}" &>/dev/null

    if [ $? -ne 0 ]; then
      server_notok[${#server_notok[@]}]=$SERVER
      # The server could not be reached, unmount the share
      net_umount $MOUNT_POINT
    else
      server_ok[${#server_ok[@]}]=$SERVER
      # The server is up, make sure the share are mounted
      net_mount $MOUNT_POINT
    fi
  fi
done
```

注意：Test using a TCP probe instead of ICMP ping (default is tcp port 2049 in NFS4) then replace the line:

```
# Check if the server is reachable
ping -c 1 "${SERVER}" &>/dev/null
```

with:

```
# Check if the server is reachable
timeout 1 bash -c ": < /dev/tcp/${SERVER}/2049"
```

in the `auto_share` script above.

Make sure the script is executable.

Next check configure the script to run every X, in the examples below this is every minute.

Cron

```
# crontab -e
* * * * * /usr/local/bin/auto_share
```

systemd/Timers

```
/etc/systemd/system/auto_share.timer
[Unit]
Description=Automount NFS shares every minute

[Timer]
OnCalendar=*-*-* *:*:00

[Install]
WantedBy=timers.target
/etc/systemd/system/auto_share.service
[Unit]
Description=Automount NFS shares
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/auto_share

[Install]
WantedBy=multi-user.target
```

Finally, enable and start `auto_share.timer`.

Using a NetworkManager dispatcher

NetworkManager can also be configured to run a script on network status change.

The easiest method for mount shares on network status change is to symlink the `auto_share` script:

```
# ln -s /usr/local/bin/auto_share /etc/NetworkManager/dispatcher.d/30-nfs.sh
```

However, in that particular case unmounting will happen only after the network connection has already been disabled, which is unclean and may result in effects like freezing of KDE Plasma applets.

The following script safely unmounts the NFS shares before the relevant network connection is disabled by listening for the `down`, `pre-down` and `vpn-pre-down` events, make sure the script is executable:

```
/etc/NetworkManager/dispatcher.d/30-nfs.sh
#!/bin/sh

# Find the connection UUID with "nmcli con show" in terminal.
# All NetworkManager connection types are supported: wireless, VPN, wired...
WANTED_CON_UUID="CHANGE-ME-NOW-9c7eff15-010a-4b1c-a786-9b4efa218ba9"

if [ "$CONNECTION_UUID" = "$WANTED_CON_UUID" ]; then

    # Script parameter $1: network interface name, not used
    # Script parameter $2: dispatched event

    case "$2" in
        "up")
            mount -a -t nfs4,nfs
            ;;
        "down"|"pre-down"|"vpn-pre-down")
            umount -l -a -t nfs4,nfs -f >/dev/null
            ;;
    esac
fi
```

注意：This script ignores mounts with the `noauto` option, remove this mount option or use `auto` to allow the dispatcher to manage these mounts.

在`/etc/NetworkManager/dispatcher.d/pre-down`中创建一个符号链接以捕获`pre-down`事件:

```
# ln -s /etc/NetworkManager/dispatcher.d/30-
nfs.sh /etc/NetworkManager/dispatcher.d/pre-down.d/30-nfs.sh
```

TLS 加密

NFS traffic can be encrypted using TLS as of Linux 6.5 using the `xprtsec=tls` mount option. To begin, install the ktls-utilsAUR package on the client and server, and follow the below configuration steps for each.

服务端

Create a private key and obtain a certificate containing your server's DNS name (see Transport Layer Security for more detail). These files do not need to be added to the system's trust store.

注意：Using a self-signed certificate that has also been encrypted is currently not supported and will result in a mount failure.

Edit `/etc/tlshd.conf` to use these files, using your own values for `x509.certificate` and `x509.private_key`

```
/etc/tlshd.conf
[authenticate.server]
x509.certificate= /etc/nfsd-certificate.pem
x509.private_key= /etc/nfsd-private-key.pem
```

Now start and enable `tlshd.service`.

客户端

Add the server's TLS certificate generated in the previous step to the system's trust store (see Transport Layer Security for more detail).

Start and enable `tlshd.service`.

Now you should be able to mount the server using the server's DNS name:

```
# mount -o xprtsec=tls servername.domain:/ /mountpoint/on/client
```

Checking journalctl on the client should show that the TLS handshake was successful:

```
$ journalctl -b -u tlshd.service
Sep 28 11:14:46 client tlshd: Built from ktls-utils 0.10 on Sep 26 2023 14:24:03
Sep 28 11:15:37 client tlshd: Handshake with servername.domain (192.168.122.100) was
successful
```

故障排查

参考单独的 NFS/Troubleshooting 页面。

更多参考

- See also Avahi, a Zeroconf implementation which allows automatic discovery of NFS shares.
- HOWTO: Diskless network boot NFS root
- Microsoft Services for Unix NFS Client info
- NFS on Snow Leopard
- http://chschneider.eu/linux/server/nfs.shtml
- How to do Linux NFS Performance Tuning and Optimization
- Linux: Tune NFS Performance
- Configuring an NFSv4-only Server

6.7 Samba

难度 高级 | **篇幅** 56.0 KB | **所属** 网络共享

本节将讲到

- 服务器
- 客户端配置

- 提示与技巧
- 疑难解答
- addition for iOS/iPadOS 14.5+ Files transfer-to server

翻译状态：

- 本文（或部分内容）译自 Samba，最近一次同步于 2024-04-29，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Samba 是用于 Linux 和 Unix的标准 Windows 互操作性程序套件。自 1992 年以来，Samba 为所有使用 SMB/CIFS 协议的客户端提供了安全、稳定和快速的文件和打印服务，例如所有版本的 DOS 和 Windows、OS/2、Linux 和许多其他系统。

要通过 Samba 共享文件，请参阅#服务器部分；要访问其他机器上通过 Samba 共享的文件，请参见#客户端部分。

服务器

安装

安装 samba包 软件包。

Samba 服务的配置文件是 `/etc/samba/smb.conf`，smb.conf(5)提供了详细的文档。

samba包 软件包没有提供此文件，启动 `smb.service` 前需要先创建这个文件。

从 Samba 的 Git 仓库可以获取到示例文件 `smb.conf.default`，可参考其配置 `/etc/samba/smb.conf`。

注意：

提示：修改 `smb.conf` 文件后，运行 testparm(1) 命令看看有没有语法错误。

启用并启动服务

要通过 SMB 提供基础文件共享服务，请启用/启动 `smb.service`。详情请参考 smbd(8)。

如果你要让服务器可通过 NetBIOS 主机名访问，需在 `smb.conf` 中的 `netbios name` 选项配置目标名称，并启用/启动 `nmb.service`。详情请参考 smbd(8)。

注意：无需用到 `nmb.service`。但对于部分主机，需要该服务才能通过主机名（例如 `smb://hostname/`）访问 Samba 服务器。如果你的网络中只包含运行 Windows 10 及更高版本的设备，可以考虑同时安装 WSD 守护进程来使你的服务器出现在“网络”视图中。

使服务器可被发现

安装 avahi包 软件包， 然后启用/启动 `avahi-daemon.service` 以通过 Zeroconf 使 Samba 服务器可被发现。这应当在多数非 Windows 文件管理器上可用（macOS Finder，Linux 和 BSD 上各种基于 GUI 的文件管理器，等等）。

如果 `avahi-daemon.service` 尚未运行，服务器仍可被访问，仅不可被发现。也就是说，它不会出现在文件管理器中，但你仍可直接通过 IP 或者域名连接它。

Windows 资源管理器单独依赖 WS-Directory 协议；参见 #Windows 1709 及更高版本无法在“网络”视图中发现 Samba 服务器。

配置防火墙

如果使用了防火墙，请记得打开需要的端口（通常是 137-139 + 445）。完整列表请查看 Samba 所需端口。

UFW 规则

SMB/CIFS 的 UFW 应用程序配置文件在安装 Ufw 时已默认包含在 `\/etc/ufw/applications.d/ufw-fileserver\` 中。

使用 root 执行 `ufw allow CIFS` 。

如果您删除了配置文件，请创建/编辑 /etc/ufw/applications.d/samba 并添加以下内容：

```
[Samba]
title=LanManager-like file and printer server for Unix
description=The Samba software suite is a collection of programs that implements the SMB/CIFS protocol for unix systems, allowing you to serve files and printers to Windows, NT, OS/2 and DOS clients. This protocol is sometimes also referred to as the LanManager or NetBIOS protocol.
ports=137,138/udp|139,445/tcp
```

Then load the profile into UFW run `ufw app update Samba` as root.

Then finally, allow Samba by running `ufw allow Samba` as root.

firewalld 服务

To configure firewalld to allow Samba in the **home** zone, run:

```
# firewall-cmd --permanent --add-service={samba,samba-client,samba-dc} --zone=home
```

The three services listed are:

- `samba`: for sharing files with others.
- `samba-client`: to browse shares on other machines on the network.
- `samba-dc`: for Samba/Active Directory domain controller.

`--permanent` ensures the changes remain after `firewalld.service` is restarted.

基础配置

用户管理

下列章节描述了如何创建本地（tdbsam）Samba 用户数据库。出于用户验证及其他用途，Samba 也可以被绑定到一个 Active Directory 域中，或是自己作为 Active Directory 域控制器，或者被用作 LDAP 服务器。

添加用户

Samba 需要 Linux 账户才能使用 - 可以使用已有账户或创建新用户。

注意：用户/用户组 *nobody* 应已在系统中创建。它被作为默认来宾账户，可用于包含 `guest ok = yes` 的共享，使得用户无需登录即可使用该共享。

虽然 Samba 和 Linux 系统共享用户名，但 Samba 使用单独的密码管理。将下面的 `samba_user` 替换为选择的 Samba 用户:

```
# smbpasswd -a samba_user
```

根据服务器角色的差异，可能需要为 Samba 用户账户修改已有的文件权限与属性。

要让新创建的用户仅能访问 Samba 远程文件服务器，可以禁用其它登录选项：

- 禁用 shell - `usermod --shell /usr/bin/nologin --lock *samba_user*`
- 禁用 SSH 登录 - 编辑 `/etc/ssh/sshd_config` 文件，修改 `AllowUsers` 选项

另请参阅安全一文来加固系统。

查询用户

使用 pdbedit(8) 命令查询现有用户：

```
# pdbedit -L -v
```

更改 samba 用户的密码

使用 `smbpasswd` 修改 samba 用户的密码：

```
# smbpasswd samba_user
```

创建匿名共享

1\. 创建新 Linux 用户，匿名 Samba 用户将映射到该用户。

```
# useradd guest -s /bin/nologin
```

注意：用户名可以是任意合规 Linux 用户名，不限于“guest”。该用户不需要作为 Samba 用户。

2\. 在 `/etc/samba/smb.conf` 添加下列内容：

```
/etc/samba/smb.conf
...
[global]
```

```
security = user
map to guest = bad user
guest account = guest

[guest_share]
    comment = guest share
    path = /tmp/
    public = yes
    only guest = yes
    writable = yes
    printable = no
```

现在匿名用户将被映射到 Linux 用户 `guest`，并可以访问 `guest_share.path` 中定义的任何目录，在该例子中为 `/tmp/`。

注意：共享名称不一定要包含“guest”，它可以是任何合规 Samba 共享名称。

请确保 Linux 用户 `guest` 拥有访问 `guest_share.path` 中文件的相应权限。

另外，请确保共享已正确按照 smb.conf.default 中的 *Share Definitions* 一节进行配置。

高级配置

Enable symlink following

警告：Enabling the `follow symlinks` option can be a security risk.

```
/etc/samba/smb.conf
...
[global]
   follow symlinks = yes
   wide links = yes
   unix extensions = no
```

Then, restart `smb.service`.

注意：When using AppArmor, if the symlink points to a directory outside the user's home or the usershare directory, then you need to modify the AppArmor profile permissions.

为 macOS 客户端启用服务端复制

服务端复制使得在服务端复制文件时不需要在服务器及客户端间进行数据传输。该选项默认启用，但在 macOS 客户端下无效。如果你有 macOS 客户端，需要将下面的配置添加到 `smb.conf`，然后重启 `smb.service`。

```
/etc/samba/smb.conf
...
[global]
   fruit:copyfile = yes
```

启用 Usershare

注意：此为可选功能，如无需要可以跳过。

Usershares 可以让不具有 root 权限的用户拥有进行添加、修改和删除自己的文件夹的操作权限。参见 smb.conf(5) § USERSHARES。

1. 为 usershares 创建目录：

```
# mkdir /var/lib/samba/usershares
```

1. 创建一个用户组：

```
# groupadd -r sambashare
```

1. 将目录的所有者修改为 `root`，组修改为 `sambashare`：

```
# chown root:sambashare /var/lib/samba/usershares
```

1. 修改 `usershares` 目录的权限，使 `sambashare` 组中的用户可以创建文件。该命令同时配置了黏着位，防止用户删除其他用户的 usershares：

```
# chmod 1770 /var/lib/samba/usershares
```

在 `smb.conf` 配置文件中设置如下配置：

```
/etc/samba/smb.conf
[global]
  usershare path = /var/lib/samba/usershares
  usershare max shares = 100
  usershare allow guests = yes
  usershare owner only = yes
```

将用户添加到 *sambashare* 组，将 `*your_username*` 替换为你用户的名称：

```
# gpasswd sambashare -a your_username
```

重启 `smb.service` 和 `nmb.service` 服务。

注销并重新登录。

如果你希望共享主目录下的路径，其必须允许 *others* 组的用户访问。

**本文或本章节的事实准确性存在争议。**

原因： 不确定该需求是否正确，也不确定指的是哪个路径：是 smb.conf 的 'usershare path' 选项路径还是共享目录的路径？（在 Talk:Samba#permissions 中讨论）

在 GUI 下，你可以使用 Thunar 或 Dolphin - 右键点击任何目录，然后通过网络共享。

在命令行下，使用下列任一命令，替换掉斜体的 *sharename*，*user*，...：

```
# net usershare add sharename abspath [comment] [user:{R|D|F}] [guest_ok={y|n}]
# net usershare delete sharename
# net usershare list wildcard-sharename
# net usershare info wildcard-sharename
```

设置并强制应用权限

权限可同时应用于服务器和共享：

```
/etc/samba/smb.conf
[global]
  ;inherit owner = unix only ; Inherit ownership of the parent directory for new files
and directories
  ;inherit permissions = yes ; Inherit permissions of the parent directory for new
files and directories
  create mask = 0664
  directory mask = 2755
  force create mode = 0644
  force directory mode = 2755
  ...
```

```
[media]
  comment = Media share accessible by greg and pcusers
  path = /path/to/media
  valid users = greg @pcusers
  force group = +pcusers
  public = no
  writable = yes
  create mask = 0664
  directory mask = 2775
  force create mode = 0664
  force directory mode = 2775

[public]
  comment = Public share where archie has write access
  path = /path/to/public
  public = yes
  read only = yes
  write list = archie
  printable = no

[guests]
  comment = Allow all users to read/write
  path = /path/to/guests
  public = yes
  only guest = yes
  writable = yes
  printable = no
```

参见 smb.conf(5) 查看完整的可用权限标志和设置。

限制协议以增强安全性

警告：默认情况下，4.11 前的 Samba 版本允许使用过时和不安全的 SMB1 协议连接。使用这些版本时，强烈建议设置 `server min protocol = SMB2_02` 以保护自己免遭勒索软件攻击。在 Samba 4.11 及更新版本，SMB2 已是默认最低协议，因此不再需要此更改。

在 `/etc/samba/smb.conf` 中添加 `server min protocol` 和 `server max protocol` 以强制最低和最高可用协议版本；

```
/etc/samba/smb.conf
[global]
  server min protocol = SMB2_10
  ; server max protocol = SMB3
```

参见 smb.conf(5) 中的 `server max protocol` 部分以获取所支持协议的概述。

为了和旧版本的客户端和/或服务器兼容，你或许需要将 `client min protocol` 或 `server min protocol` 设置到旧一点的版本，但请注意你将更易于遭受攻击。

提示：当客户端只应使用最新的 SMB3 协议连接时，使用 `server min protocol = SMB3`，例如运行 Windows 10 及之后版本的客户端。

使用 `mount.cifs` 的客户端也许需要指定正确的 `vers=*`，例如：

```
# mount -t cifs //SERVER/sharename /mnt/mountpoint -o
username=username,password=password,iocharset=utf8,vers=3.1.1
```

参见 mount.cifs(8) 获取更多信息.

使用原生 SMB 传输加密

原生 SMB 传输加密在 SMB 3.0 及更新版本可用。支持该类型加密的客户端包括 Windows 8 以及更新版本，Windows Server 2012 及更新版本，以及 Samba 4.1 及更新版本的 smbclient。

为了默认使用原生 SMB 传输加密，需全局和/或按共享设置 `server smb encrypt` 参数。可用的值有`off`，`enabled`（默认值），`desired` 或 `required`：

```
/etc/samba/smb.conf
[global]
  server smb encrypt = desired
```

To configure encryption for on the client side, use the option `client smb encrypt`.

参见 smb.conf(5) 获取更多信息，特别是**对 SMB1 的影响**和**对 SMB2 的影响**的段落。

提示：当 #手动挂载 一个共享时，指定 `seal` 挂载选项以强制使用加密。

禁用打印机共享

默认情况下 Samba 会共享由 CUPS 设置的打印机。

如果你不想打印机被共享，使用以下设置：

```
/etc/samba/smb.conf
[global]
  load printers = no
  printing = bsd
  printcap name = /dev/null
  disable spoolss = yes
  show add printer wizard = no
```

防止 Samba 共享特定后缀的文件

注意：设置这项参数会影响 Samba 的性能，因为它会被强制要求检查所有扫描到的文件和目录是否匹配要求。

Samba 提供了一个选项以屏蔽满足特定命名模式的文件，比如文件扩展名。该选项可用于防止病毒传播或阻止用户用特定的文件浪费空间。更多关于此选项的信息可在 smb.conf(5) 找到。

```
/etc/samba/smb.conf
...
[myshare]
  comment = Private
  path = /mnt/data
  read only = no
  veto files = /*.exe/*.com/*.dll/*.bat/*.vbs/*.tmp/*.mp3/*.avi/*.mp4/*.wmv/*.wma/
```

提高吞吐量

警告：注意，下列操作有可能会造成损坏/连接问题，可能会损坏你的 TCP/IP 栈。

默认设置应满足多数用户的用例。虽然正确配置 'socket options' 可以提升性能，但错误配置也会同比例降低性能。在进行大幅更改前请确保进行了测试。

在应用下列任何配置前请阅读 smb.conf(5) 文档。

下列设置需要附加到 `/etc/samba/smb.conf` 的 `[global]` 一节中。

设置 deadtime 可防止大量非活跃连接消耗系统资源：

```
deadtime = 30
```

sendfile 可能会更高效利用 CPU 资源，让 Samba 变得更快：

```
use sendfile = yes
```

配置最低 receivefile 大小可以利用零复制直接从网络套接字缓冲区写入到操作系统缓存（如果可用）。这一操作可能会提高性能，但建议用户进行测试：

```
min receivefile size = 16384
```

提高接收/发送缓存大小及配置套接字优化标志可能有助于提升吞吐量。由于在某些网络下可能会出现问题，建议对各个标志进行单独测试：

```
socket options = IPTOS_LOWDELAY TCP_NODELAY IPTOS_THROUGHPUT SO_RCVBUF=131072 SO_SNDBUF=131072
```

注意：部分选项可能需要调整网络接口，详见 Sysctl#网络。

Enable access for old clients/devices

Latest versions of Samba no longer offer older authentication methods and protocols which are still used by some older clients (IP cameras, etc). These devices usually require Samba server to allow NTMLv1 authentication and NT1 version of the protocol, known as CIFS. For these devices to work with latest Samba, you need to add these two configuration parameters into `[global]` section:

```
server min protocol = NT1
ntlm auth = yes
```

Anonymous/guest access to a share requires just the first parameter. If the old device will access with username and password, you also need the add the second line too.

Enable Spotlight searching

Spotlight allows supporting clients (e.g. MacOS Finder) to quickly search shared files.

Install and start/enable OpenSearch. Install fs2es-indexerAUR, configure the directories you want to index in `/etc/fs2es-indexer/config.yml`, and start/enable `fs2es-indexer.service` for periodic indexing.

Edit `smb.conf` as described in the Samba wiki to enable Spotlight per share, and restart `smb.service` to apply the changes.

客户端配置

要使用类似 `ftp` 的命令行界面，请安装软件包 smbclient包。常用命令请参考 smbclient(1)。

如需更轻量级的替代品（没有可用共享查询等功能），可安装 cifs-utils包，其提供了 `/usr/bin/mount.cifs`。

部分桌面环境可能提供了图形界面，搭配文件管理器的用法请参考#文件管理器配置。

注意：

- smbclient包 requires a `/etc/samba/smb.conf` file (see #Installation), which you can create as an empty file using the `touch` utility.
- 安装 cifs-utils包 或 smbclient包 后，请加载 `cifs` 内核模块 或重启以避免挂载失败。

显示可用共享

下面命令会显示服务器上的可用共享:

```
$ smbclient -L hostname -U%
```

smbtree 可用显示共享目录树，不建议再有大量计算机的网络上使用此功能。可用它检查共享名是否可用。

另外，使用 `$ smbtree -N` 会以树状图显示所有可用共享，它使用了广播查询，因此不建议在有大量设备的网络中使用，但有助于检查共享名是否可用。使用 `-N`（`-no-pass`）选项可以忽略密码输入请求。

注意：`smbtree` 使用了 SMB1 和 NetBIOS，意味着它们必须已在服务器上启用，且需要在客户端的 `smb.conf` 文件中设置 `client min protocol = NT1`，否则 `smbtree` 只会显示空输出。

NetBIOS/WINS 主机名

Samba clients handle NetBIOS host names automatically by default (the behavior is controlled by the `name resolve order` option in `smb.conf`). Other programs (including `mount.cifs`) typically use Name Service Switch, which does not handle NetBIOS by default.

The smbclient包 package provides a libnss driver to resolve NetBIOS host names. To use it, install it along with the samba包 package (which provides the *winbindd* daemon), start/enable `winbind.service` and add `wins` to the `hosts` line in nsswitch.conf(5):

```
/etc/nsswitch.conf
...
hosts: mymachines resolve [!UNAVAIL=return] files myhostname dns wins
...
```

注意：Due to a current mistake in `winbind.service`, you may have to modify the unit file as described in this bug-report

Now, during host resolving (e.g. when using `mount.cifs` or just `ping *netbios-name*`), *winbindd* will resolve the host name by sending queries using NetBIOS Name Service (NBNS, also known as WINS) protocol.

By default it sends a broadcast query to your local network. If you have a WINS server, you can add `wins server = *wins-server-ip*` to `smb.conf` and restart `winbind.service`, then *winbindd* and other Samba clients will send unicast queries to the specified IP.

If you want to resolve your local host name (specified in the `netbios name` option in `smb.conf`), start/enable `nmb.service`, which will handle incoming queries.

You can test WINS resolution with `nmblookup`. By default it sends broadcast queries to your local network regardless of the `wins server` option.

Note that WINS resolution requires incoming traffic originating from port 137.

Disable NetBIOS/WINS support

When not using NetBIOS/WINS host name resolution, it may be preferred to disable this protocol:

```
/etc/samba/smb.conf
[global]
  disable netbios = yes
  dns proxy = no
```

Finally disable/stop `winbind.service`.

手动挂载

使用 `mount.cifs` 作为挂载类型 `type`，下面列出的选项并不是全部都需要：

```
# mount --mkdir -t cifs //SERVER/sharename /mnt/mountpoint -o
username=username,password=password,workgroup=workgroup,iocharset=utf8,uid=username,gid
=group
```

其中 `uid` 和 `gid` 对应了被授予特定路径读写权限的本地（例如客户端）用户/用户组。

注意：

- If the `uid` and `gid` being used does not match the user of the server, the `forceuid` and `forcegid` options may be helpful. However note permissions assigned to a file when `forceuid` or `forcegid` are in effect may not reflect the real (server) permissions. See the *File And Directory Ownership And Permissions* section in mount.cifs(8) § FILE AND DIRECTORY OWNERSHIP AND PERMISSIONS for more information.
- To mount a Windows share without authentication, use `"username=*"`.

**本文或本章节的事实准确性存在争议。**

原因： Regardless of recommendation, there's no substantial evidence for the claimed I/O error risk. The warning was added without comment or reference in 2013 and challenged without defense in 2018. It's unclear whether it's true at all.（在 Talk:Samba#Unfounded warning regarding I/O errors and manual mounting? 中讨论）

注意：请注意这里有 **s**,其它文件系统一般用的是 *user*。

使用 `uid` 和 `gid` 挂载选项时，请注意文件权限，否则会出现 I/O 错误。}}

注意：使用 `uid` 和/或 `gid` 作为挂载选项可能会导致 I/O 错误，建议转而设置并检查 文件权限与属性。

- `*SERVER*`——服务器名称
- `*sharename*`——共享目录
- `*mountpoint*`——本地挂载点
- `[-o *options*]`——详情请参考 {{man|8|mount.cifs}}。

注意：

- 结尾不要加 `/`. `//*SERVER*/*sharename***/**` 无法工作.
- 如果挂载工作不稳定，出现卡顿和掉线问题，请尝试用 `vers=` 设置不同的 SMB 协议版本。例如，挂载 Vista 用 `vers=2.0`。
- 如果挂载了 cifs 的机器上出现关机超时，请参考 wpa_supplicant#Problem with mounted network shares (cifs) and shutdown_and_shutdown)。

保存共享密码

不建议将密码保存在所有人都可读的文件中，一个更安全的方式是创建凭证文件，例如写入到 `/etc/samba/credentials`：

```
/etc/samba/credentials/share
username=myuser
password=mypass
```

将挂载命令中的 `username=myuser,password=mypass` 替换为 `credentials=/etc/samba/credentials/share`。

应仅根用户允许有凭证文件的读写权限：

```
# chown root:root /etc/samba/credentials
# chmod 700 /etc/samba/credentials
# chmod 600 /etc/samba/credentials/share
```

自动挂载

注意：You may need to enable `systemd-networkd-wait-online.service` or `NetworkManager-wait-online.service` (depending on your setup) to proper enable booting on start-up.

Using NetworkManager and GIO/gvfs

NetworkManager can be configured to run a script on network status change. This script uses the *gio* command so that it mounts the Samba shares automatically, the same way your file manager does, as explained below. The script also safely unmounts the Samba shares before the relevant network connection is disabled by listening for the `pre-down` and `vpn-pre-down` events. Make the script is executable after creating it.

```
/etc/NetworkManager/dispatcher.d/30-samba.sh
```

```
#!/bin/sh

# Find the connection UUID with "nmcli con show" in terminal.
# All NetworkManager connection types are supported: wireless, VPN, wired...
WANTED_CON_UUID="CHANGE-ME-NOW-9c7eff15-010a-4b1c-a786-9b4efa218ba9"

# The user the share will be mounted under
USER="yourusername"
# The path that appears in your file manager when you manually mount the share you want
SMB_URL="smb://servername/share"

# Get runtime user directory. If it does not exist, do nothing and just exit
XDG_RUNTIME_DIR=$(loginctl show-user --property=RuntimePath --value "$USER") || exit 0

if [ "$CONNECTION_UUID" = "$WANTED_CON_UUID" ]; then

    # Script parameter $1: network interface name, not used
    # Script parameter $2: dispatched event

    case "$2" in
        "up"|"vpn-up")
            su $USER -c "DBUS_SESSION_BUS_ADDRESS=unix:path=$XDG_RUNTIME_DIR/bus gio
mount $SMB_URL"
            ;;
        "pre-down"|"vpn-pre-down")
            su $USER -c "DBUS_SESSION_BUS_ADDRESS=unix:path=$XDG_RUNTIME_DIR/bus gio
mount -uf $SMB_URL"
            ;;
    esac
fi
```

Create a symlink inside `/etc/NetworkManager/dispatcher.d/pre-down` to catch the `pre-down` events:

```
# ln -s /etc/NetworkManager/dispatcher.d/30-
samba.sh /etc/NetworkManager/dispatcher.d/pre-down.d/30-samba.sh
```

注意：Since this script uses the user bus, it will only work if the user has active sessions. This means that the share will not mount automatically after boot if the connection is established before you are logged in.

作为挂载路径

This is a simple example of a `cifs` mount entry that requires authentication:

```
/etc/fstab
//SERVER/sharename /mnt/mountpoint cifs _netdev,nofail,username=myuser,password=mypass
0 0
```

注意：

- See examples below on better security for authentication credentials
- Spaces in sharename should be replaced by `\040` (ASCII code for space in octal). For example, `//*SERVER*/share name` on the command line should be `//*SERVER*/share\040name` in `/etc/fstab`.
- To allow users to mount it as long as the mount point resides in a directory controllable by the user; i.e. the user's home, append the `users` mount option. The option is user**s** (plural). For other filesystem types handled by mount, this option is usually *user*; sans the "**s**".

提示：Use `x-systemd.automount` if you want them to be mounted only upon access. See Fstab#Remote file system for details.

作为 systemd 单元

在 `/etc/systemd/system` 下创建一个新的 `.mount` 文件，例如：`mnt-myshare.mount`。详细信息请查看 systemd.mount(5)。

注意：请确保文件名与你要使用的挂载点对应。 例如 `mnt-myshare.mount` 只能用于挂载位于 `/mnt/myshare` 下的共享，否则可能会出现像这样的报错：`systemd: mnt-myshare.mount: Where= setting does not match unit name. Refusing`。

`What=` 要使用的共享路径

`Where=` 将共享挂载到的位置

`Options=` 共享挂载选项

注意：

- 网络挂载单元会自动获取 `remote-fs-pre.target`、`network.target` 和 `network-online.target` 中 `After` 部分的依赖，并在没有设置 `nofail` 挂载选项的情况下会获得对 `remote-fs.target` 的 `Before` 依赖。对于后者，还会添加一个 `Wants` 单元。
- 将 `noauto` 添加到 `Options` 可以防止在启动阶段自动挂载（除非被其它单元拉起）。
- 如果你想用主机名而不是 IP 来连接共享服务器，需要在 `After` 中加入 `nss-lookup.target`。这可以避免在测试时正常，但启动时挂载报错的情况出现。

```
/etc/systemd/system/mnt-myshare.mount
[Unit]
Description=Mount Share at boot

[Mount]
What=//server/share
Where=/mnt/myshare
Options=_netdev,credentials=/etc/samba/credentials/myshare,iocharset=utf8,rw
Type=cifs
TimeoutSec=30

[Install]
WantedBy=multi-user.target
```

提示：

- 遇到无法连通的主机时，将 `ForceUnmount=true` 添加到 `[Mount]` 以允许强制卸载共享。
- 如果你的共享使用了只读权限的组，可以添加 `uid=*username*` 或者 `gid=*group*` 到 `Options=` 来指定拥有写入权限的用户/用户组。

要使用 `mnt-myshare.mount`，需启动该单元，然后通过启用使其在系统启动时运行。

自动化挂载

可以使用如下单元来在访问时自动挂载共享（类似 autofs）：

```
/etc/systemd/system/mnt-myshare.automount
[Unit]
Description=Automount myshare
```

```
[Automount]
Where=/mnt/myshare

[Install]
WantedBy=multi-user.target
```

禁用/停止 `mnt-myshare.mount`，然后启用/启动 `mnt-myshare.automount` 来在访问挂载路径时自动挂载共享。

提示：添加 `TimeoutIdleSec` 以启用自动卸载。详细信息可参考 systemd.automount(5)。

smbnetfs

注意：smbnetfs needs an intact Samba server setup. See above on how to do that.

First, check if you can see all the shares you are interested in mounting:

```
$ smbtree -U remote_user
```

If that does not work, find and modify the following line in `/etc/samba/smb.conf` accordingly:

```
domain master = auto
```

Now restart `smb.service` and `nmb.service`.

If everything works as expected, install smbnetfs包.

Then, add the following line to `/etc/fuse.conf`:

```
user_allow_other
```

Now copy the directory `/etc/smbnetfs/.smb` to your home directory:

```
$ cp -a /etc/smbnetfs/.smb ~
```

Then create a link to `smb.conf`:

```
$ ln -sf /etc/samba/smb.conf ~/.smb/smb.conf
```

If a username and a password are required to access some of the shared folders, edit `~/.smb/smbnetfs.auth` to include one or more entries like this:

```
~/.smb/smbnetfs.auth
auth                "hostname" "username" "password"
```

It is also possible to add entries for specific hosts to be mounted by smbnetfs, if necessary. More details can be found in `~/.smb/smbnetfs.conf`.

If you are using the Dolphin or GNOME Files, you may want to add the following to `~/.smb/smbnetfs.conf` to avoid "Disk full" errors as smbnetfs by default will report 0 bytes of free space:

```
~/.smb/smbnetfs.conf
free_space_size 1073741824
```

When you are done with the configuration, you need to run

```
$ chmod 600 ~/.smb/smbnetfs.*
```

Otherwise, smbnetfs complains about 'insecure config file permissions'.

Finally, to mount your Samba network neighbourhood to a directory of your choice, call

```
$ smbnetfs mount_point
```

Daemon

The Arch Linux package also maintains an additional system-wide operation mode for smbnetfs. To enable it, you need to make the said modifications in the directory `/etc/smbnetfs/.smb`.

Then, you can start and/or enable the `smbnetfs` daemon as usual. The system-wide mount point is at `/mnt/smbnet/`.

autofs

See Autofs for information on the kernel-based automounter for Linux.

文件管理器配置

GNOME Files、Nemo、Caja、Thunar 和 PCManFM

为了通过 GNOME Files，Nemo，Caja，Thunar 或 PCManFM 访问 samba 共享，需安装 gvfs-smb包 软件包。

按 `Ctrl+l` 然后在地址栏输入 `smb://*servername*/*share*` 以访问您的共享。

共享很可能挂载到了文件系统中的 `/run/user/*your_UID*/gvfs` 或 `~/.gvfs` 位置下。

KDE

KDE 有内建的浏览 Samba 共享的能力。为了使用 KDE 系统设置的 GUI，你需要安装 kdenetwork-filesharing包 软件包。

KDE 应用（例如 Dolphin）自带 Samba 共享浏览功能。在地址栏输入 `smb://*servername*/*share*`就可以浏览文件。如果你想在非 KDE 应用访问文件，可以安装 kio-fuse包。

To use a GUI in the KDE System Settings, you will need to install the kdenetwork-filesharing包 package.

其它图形环境

There are a number of useful programs, but they may need to have packages created for them. This can be done with the Arch package build system. The good thing about these others is that they do not require a particular environment to be installed to support them, and so they bring along less baggage.

- pyneighborhoodAUR
- LinNeighborhood, RUmba, xffm-samba plugin for Xffm are not available in the official repositories or the AUR. As they are not officially (or even unofficially supported), they may be obsolete and may not work at all.

提示与技巧

Discovering network shares

If nothing is known about other systems on the local network, and automated tools such as smbnetfs are not available, you can manually probe for Samba shares.

First, install the nmap包 and smbclient包 packages.

Use nmap to scan your local network to find systems with TCP port 445 open, which is the port used by the SMB protocol. Note that you may need to use `-Pn` or set a custom ping scan type (e.g. `-PS445`) because Windows systems are usually firewalled.

```
$ nmap -p 445 "192.168.1.*"
Starting Nmap 7.92 ( https://nmap.org ) at 2022-03-13 12:00 UTC
Nmap scan report for 192.168.1.1
Host is up (0.0011s latency).

PORT    STATE  SERVICE
445/tcp open  microsoft-ds

Nmap scan report for 192.168.1.2
Host is up (0.00011s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Nmap done: 256 IP addresses (2 hosts up) scanned in 2.45 seconds
```

The first result is another system; the second happens to be the client from where this scan was performed.

Now you can connect to their IP addresses directly, but if you want to use NetBIOS host names, you can use nmblookup(1) to check for NetBIOS names. Note that this will not work if NetBIOS is disabled on the server.

```
$ nmblookup -A 192.168.1.1
Looking up status of 192.168.1.1
        PUTER           <00> -         B <ACTIVE>
        HOMENET         <00> - <GROUP> B <ACTIVE>
        PUTER           <03> -         B <ACTIVE>
        PUTER           <20> -         B <ACTIVE>
        HOMENET         <1e> - <GROUP> B <ACTIVE>
        USERNAME        <03> -         B <ACTIVE>
        HOMENET         <1d> -         B <ACTIVE>
        MSBROWSE        <01> - <GROUP> B <ACTIVE>
```

Regardless of the output, look for **<20>**, which shows the host with open services.

Use smbclient(1) to list which services are shared on these systems. You can use NetBIOS host name (`PUTER` in this example) instead of IP when available. If prompted for a password, pressing enter should still display the list:

```
$ smbclient -L \\192.168.1.1
Sharename       Type      Comment
---------       ----      -------
MY_MUSIC        Disk
SHAREDDOCS      Disk
PRINTER$        Disk
PRINTER         Printer
IPC$            IPC       Remote Inter Process Communication
```

```
Server               Comment
---------            -------
PUTER

Workgroup            Master
---------            -------
HOMENET               PUTER
```

Remote control of Windows computer

Samba offers a set of tools for communication with Windows. These can be handy if access to a Windows computer through remote desktop is not an option, as shown by some examples.

Send shutdown command with a comment:

```
$ net rpc shutdown -C "comment" -I IPADDRESS -U USERNAME%PASSWORD
```

A forced shutdown instead can be invoked by changing -C with comment to a single -f. For a restart, only add -r, followed by a -C or -f.

Stop and start services:

```
$ net rpc service stop SERVICENAME -I IPADDRESS -U USERNAME%PASSWORD
```

To see all possible net rpc command:

```
$ net rpc
```

疑难解答

启动 Samba SMB/CIFS 服务器失败

可能的解决方法：

- 通过 testparm(1) 检查 `smb.conf` 中的配置问题。
- 在 `/var/cache/samba/` 中配置正确的权限，然后重启 `smb.service`：

```
# chmod 0755 /var/cache/samba/msg
```

SELinux 权限问题

SELinux 默认不允许 samba 访问用户的主目录。要解决此问题，执行：

```
# setsebool -P samba_enable_home_dirs 1
```

类似地，`samba_export_all_ro` 和 `samba_export_all_rw` 使 Samba 拥有读取或“读和写”所有文件的能力。

AppArmor 权限问题

如果使用了一个在家目录或用户共享目录之外的共享路径，请在 `/etc/apparmor.d/local/usr.sbin.smbd` 中将其加入白名单。例如：

```
/etc/apparmor.d/local/usr.sbin.smbd
"/data/" rk,
"/data/**" lrwk,
```

No dialect specified on mount

The client is using an unsupported SMB/CIFS version that is required by the server.

See #Restrict protocols for better security for more information.

Unable to overwrite files, permissions errors

**本文或本章节的事实准确性存在争议。**

原因： An user should set/check for server/client permissions, instead of using incorrect/possible insecure flags.（在 Talk:Samba 中讨论）

Possible solutions:

- Append the mount option `nodfs` to the `/etc/fstab` entry.
- Add `msdfs root = no` to the `[global]` section of the server's `/etc/samba/smb.conf`.

Windows clients keep asking for password even if Samba shares are created with guest permissions

Set `map to guest` inside the `global` section of `/etc/samba/smb.conf`:

```
map to guest = Bad Password
```

If you are still using Samba < 4.10.10, use `Bad User` instead of `Bad Password`.

Windows 10 1709 和更高版本的连接性问题 - "Windows cannot access" 0x80004005

This error affects some machines running Windows 10 version 1709 and later. It is not related to SMB1 being disabled in this version but to the fact that Microsoft disabled insecure logons for guests on this version for some, but not others.

To fix, open Group Policy Editor (`gpedit.msc`). Navigate to *Computer configuration**administrative templates**network**Lanman Workstation* > *Enable insecure guest logons* and enable it. Alternatively,change the following value in the registry:

```
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\LanmanWorkstation\Parameters]
"AllowInsecureGuestAuth"=dword:1
```

Error: Failed to retrieve printer list: NT_STATUS_UNSUCCESSFUL

If you are a home user and using samba purely for file sharing from a server or NAS, you are probably not interested in sharing printers through it. If so, you can prevent this error from occurring by adding the following lines to your `/etc/samba/smb.conf`:

```
/etc/samba/smb.conf
[global]
  load printers = No
  printing = bsd
  printcap name = /dev/null
  disable spoolss = Yes
```

Restart the samba service, `smb.service`, and then check your logs:

```
# cat /var/log/samba/smbd.log
```

and the error should now no longer be appearing.

Sharing a folder fails

It means that while you are sharing a folder from *Dolphin* (file manager) and everything seems ok at first, after restarting *Dolphin* the share icon is gone from the shared folder, and also some output like this in terminal (*Konsole*) output:

```
'net usershare' returned error 255: net usershare: usershares are currently disabled
```

To fix it, enable usershare as described in #启用 Usershares.

"Browsing" network fails with "Failed to retrieve share list from server"

And you are using a firewall (iptables) because you do not trust your local (school, university, hotel) network. This may be due to the following: When the smbclient is browsing the local network it sends out a broadcast request on udp port 137. The servers on the network then reply to your client but as the source address of this reply is different from the destination address iptables saw when sending the request for the listing out, iptables will not recognize the reply as being "ESTABLISHED" or "RELATED", and hence the packet is dropped. A possible solution is to add:

```
iptables -t raw -A OUTPUT -p udp -m udp --dport 137 -j CT --helper netbios-ns
```

to your iptables setup.

For Uncomplicated Firewall, you need to add `nf_conntrack_netbios_ns` to the end of the following line in `/etc/default/ufw`

```
IPT_MODULES="nf_conntrack_ftp nf_nat_ftp nf_conntrack_irc nf_nat_irc"
```

and then run the following commands as root:

```
echo 1 > /proc/sys/net/netfilter/nf_conntrack_helper
ufw allow CIFS
ufw reload
```

To make this change persistent across reboots, add the following line at the end of `/etc/ufw/sysctl.conf`:

```
net.netfilter.nf_conntrack_helper=1
```

Protocol negotiation failed: NT_STATUS_INVALID_NETWORK_RESPONSE

The client probably does not have access to shares. Make sure clients' IP address is in `hosts allow =` line in `/etc/samba/smb.conf`.

Another problem could be, that the client uses an invalid protocol version. To check this try to connect with the `smbclient` where you specify the maximum protocol version manually:

```
$ smbclient -U <user name> -L //<server name> -m <protocol version: e. g. SMB2> -W
<domain name>
```

If the command was successful then create a configuration file:

```
~/.smb/smb.conf
[global]
  workgroup = <domain name>
  client max protocol = SMB2
```

Connection to SERVER failed: (Error NT_STATUS_UNSUCCESSFUL)

You are probably passing a wrong server name to `smbclient`. To find out the server name, run `hostnamectl` on the server and look at "Transient hostname" line

Connection to SERVER failed: (Error NT_STATUS_CONNECTION_REFUSED)

Make sure that the server has started. The shared directories should exist and be accessible.

Protocol negotiation failed: NT_STATUS_CONNECTION_RESET

Probably the server is configured not to accept protocol SMB1. Add option `client max protocol = SMB2` in `/etc/samba/smb.conf`. Or just pass argument `-m SMB2` to `smbclient`.

Password Error when correct credentials are given (error 1326)

Samba 4.5 has NTLMv1 authentication disabled by default. It is recommend to install the latest available upgrades on clients and deny access for unsupported clients.

If you still need support for very old clients without NTLMv2 support (e.g. Windows XP), it is possible force enable NTLMv1, although this is **not recommend** for security reasons:

```
/etc/samba/smb.conf
[global]
  lanman auth = yes
  ntlm auth = yes
```

If NTLMv2 clients are unable to authenticate when NTLMv1 has been enabled, create the following file on the client:

```
/home/user/.smb/smb.conf
[global]
  sec = ntlmv2
  client ntlmv2 auth = yes
```

This change also affects samba shares mounted with **mount.cifs**. If after upgrade to Samba 4.5 your mount fails, add the **sec=ntlmssp** option to your mount command, e.g.

```
mount.cifs //server/share /mnt/point -o sec=ntlmssp,...
```

See the mount.cifs(8) man page: **ntlmssp** - Use NTLMv2 password hashing encapsulated in Raw NTLMSSP message. The default in mainline kernel versions prior to v3.8 was **sec=ntlm**. In v3.8, the default was changed to **sec=ntlmssp**.

Mapping reserved Windows characters

Starting with kernel 3.18, the cifs module uses the "mapposix" option by default. When mounting a share using unix extensions and a default Samba configuration, files and directories containing one of the seven reserved Windows characters `: \ * < > ?` are listed but cannot be accessed.

Possible solutions are:

- Use the undocumented `nomapposix` mount option for cifs

```
# mount.cifs //server/share /mnt/point -o nomapposix
```

- Configure Samba to remap `mapposix` ("SFM", Services for Mac) style characters to the correct native ones using fruit

```
/etc/samba/smb.conf
[global]
```

```
  vfs objects = catia fruit
  fruit:encoding = native
```

- Manually remap forbidden characters using catia

```
/etc/samba/smb.conf
[global]
  vfs objects = catia
  catia:mappings = 0x22:0xf022, 0x2a:0xf02a, 0x2f:0xf02f, 0x3a:0xf03a, 0x3c:0xf03c,
0x3e:0xf03e, 0x3f:0xf03f, 0x5c:0xf05c, 0x7c:0xf07c, 0x20:0xf020
```

The latter approach (using catia or fruit) has the drawback of filtering files with unprintable characters.

Folder shared inside graphical environment is not available to guests

This section presupposes:

1. Usershares are configured following previous section
2. A shared folder has been created as a non-root user from GUI
3. Guests access has been set to shared folder during creation
4. Samba service has been restarted at least once since last `/etc/samba/smb.conf` file modification

For clarification purpose only, in the following sub-sections is assumed:

- Shared folder is located inside user home directory path (`/home/yourUser/Shared`)
- Shared folder name is *MySharedFiles*
- Guest access is read-only.
- Windows users will access shared folder content without login prompt

Verify correct samba configuration

Run the following command from a terminal to test configuration file correctness:

```
$ testparm
```

Verify correct shared folder creation

Run the following commands from a terminal:

```
$ cd /var/lib/samba/usershares
$ ls
```

If everything is fine, you will notice a file named `mysharedfiles`

Read the file contents using the following command:

```
$ cat mysharedfiles
```

The terminal output should display something like this:

```
/var/lib/samba/usershares/mysharedfiles
path=/home/yourUser/Shared
comment=
usershare_acl=S-1-1-0:r
guest_ok=y
sharename=MySharedFiles
```

Verify folder access by guest

Run the following command from a terminal. If prompted for a password, just press Enter:

```
$ smbclient -L localhost
```
If everything is fine, MySharedFiles should be displayed under `Sharename` column

Run the following command in order to access the shared folder as guest (anonymous login)

```
$ smbclient -N //localhost/MySharedFiles
```
If everything is fine samba client prompt will be displayed:

```
smb: \>
```
From samba prompt verify guest can list directory contents:

```
smb: \> ls
```
If the `NTFS_STATUS_ACCESS_DENIED` error is displayed, the issue is likely to be with Unix directory permissions. Ensure that your samba user has access to the folder and all parent folders. You can test this by sudoing to the user and attempting to list the mount directory, and all of its parents.

Mount error: Host is down

This error might be seen when mounting shares of Synology NAS servers. Use the mount option `vers=1.0` to solve it.

注意：SMB version 1 is known to have security vulnerabilities and was used in successful ransomware attacks.

Software caused connection abort

File managers that utilizes gvfs-smb包 can show the error `Software caused connection abort` when writing a file to a share/server. This may be due to the server running SMB/CIFS version 1, which many routers use for USB drive sharing (e.g. Belkin routers). To write to these shares specify the CIFS version with the option `vers=1.0`. E.g.:

```
/etc/fstab
//SERVER/sharename /mnt/mountpoint cifs
_netdev,guest,file_mode=0777,dir_mode=0777,vers=1.0 0 0
```
This can also happen after updating Samba to version 4.11, which deactivates SMB1 as default, and accessing any Samba share. You can reenable it by adding

```
/etc/samba/smb.conf
[global]
client min protocol = CORE
```

Connection problem (due to authentification error)

Be sure that you do not leave any space characters before your username in Samba client configuration file as follows:

```
~/.samba
username= user
password=pass
```
The correct format is:

```
~/.samba
username=user
password=pass
```

Windows 1709 及更高版本无法在“网络”视图中发现 Samba 服务器

随着 Windows 10 1511 版本的推出，对 SMBv1 的支持以及由此的 NetBIOS 设备发现被默认禁用。根据实际版本不同，从1709版本（"秋季创意者更新"）开始的 Windows 版本不允许再安装 SMBv1 客户端。这导致运行 Samba 的主机无法在资源管理器的“网络（网上邻居）”视图中被列出。虽然并无连接问题，而且 Samba 仍然可以正常运行，但用户可能想让他们的 Samba 主机被 Windows 自动列出。wsddAUR 实现了一个 Web Service Discovery 宿主守护进程。这使得（Samba）主机，比方说你的本地 NAS 设备，能够被像 Windows 这样的 Web Service Discovery 客户端找到。默认设置应该适用于大多数用例，你要做的就是启用 `wsdd.service`。

默认配置（在组 "WORKGROUP "中使用机器主机名公示自己）应适用于绝大多数情况。如果有需要，你可以通过在 `/etc/conf.d/wsdd` 中添加额外的参数来改变配置选项（详见wsdd的手册页）。

wsdd2AUR 的功能相同，但它是用 C 语言而不是 Python 编写的。默认情况下，它将在 `smb.conf` 中寻找 `netbios name` 和 `workgroup` 值。

GNOME Files not showing Windows machines (version 1709 or up) with shared folders in Network view

See GNOME/Files#Windows machines (version 1709 or up) with shared folders don't show up in Network view_with_shared_folders_don't_show_up_in_Network_view).

iOS/iPadOS Files can no longer copy-to Samba share on Arch Linux beginning with iOS/iPadOS 14.5

Beginning with iOS/iPadOS 14.5 attempting to transfer from a device running iOS/iPadOS using the "Files" app to a samba share on Arch Linux will result in the error:

```
The operation couldn't be completed
Operation canceled
```

To correct this problem, add add the following to the global section of your `smb.conf` and restart `smb.service`. Comment optional:

```
## addition for iOS/iPadOS 14.5+ Files transfer-to server
vfs object = fruit streams_xattr
```

See https://apple.stackexchange.com/q/424681 Apple.Stackexchange.com - "The operation couldn't be completed"/"Operation canceled" error message when saving to a Samba share via Files app.

Slow initial connections from certain clients without other performance problems

Some SMB clients, such as Solid Explorer for Android, take significantly longer to connect to Samba if they fail to resolve the NetBIOS name. Enabling `nmb.service`

will greatly speed up initial connections if this is the case. Since this is a bug in the client software, please report such cases to the authors of conflicting software.

Windows 复制文件时会随机卡住不动

使用 windows 文件管理器复制 samba 服务器中的文件时，会随机出现卡住不动的情况。原因是复制 linux 的 pipe 特殊文件会占用连接数，当连接数消耗完毕以后，windows 的复制窗口就会一直卡住不动。解决办法是在 `smb.conf` 中添加隐藏特殊文件配置项。

参考来源：https://forum.level1techs.com/t/samba-server-and-windows-clients-getting-stuck/161676

```
/etc/samba/smb.conf
[global]
   hide special files = yes
```

更多参考

- 官方网站
- Samba: An Introduction
- Samba 3.2.x HOWTO and Reference Guide（已过时，但仍是最详细的文档）
- 维基百科)
- Gentoo:Samba/Guide
- Debian:Samba/ServerSimple
- KSMBD - A linux kernel server which implements SMB3 protocol in kernel space for sharing files over network.

网络服务器

本章解决在Arch上把服务器跑起来、往上放内容的问题：七节覆盖静态站点、反向代理、应用部署与WordPress的安装、配置和排错。

Nginx与Apache篇幅最长，建议先读这两节并动手跑一遍；Caddy配置更省事，可以先跳过；Tomcat仅在要部署Java时读；Lighttpd、Artanis与WordPress按需查阅。最容易踩的坑是把装完包当成服务已在跑：几节都单列了启动、停止和改配置后的重启，又带故障排除，可见首次配置很少一次成功；WordPress本身不是服务器，Caddy插件需另装。

7.1　Caddy

难度 入门 | **篇幅** 3.1 KB | **所属** 网络服务器

本节将讲到

- 安装
- 插件
- 配置
- 用法
- 故障排除

Caddy 是具有 HTTP/2 功能的网页服务器，具有自动 HTTPS 配置功能。

安装

安装 caddy包 软件包。

插件

如果你需要比默认 caddy 更多的功能，可以使用 xcaddy-binAUR 来自定义 caddy server 构建。这对于需要如 DNS challenge 插件等场景非常有用。 另外，在可用的情况下，你也可以直接从 AUR 挑选要要安装的带插件预构建版本，例如 caddy-cloudflareAUR。

你可以使用 xcaddy 构建带特定插件的定制 caddy server：

```
$ xcaddy build [<caddy_version>]
   [--output <file>]
   [--with <module[@version][=replacement]>...]
```

更多信息可参考 xcaddy 存储库。

配置

Caddy 2 支持多种配置格式，详情可参考配置适配器（包括 caddyfile，nginx，json，yaml，toml 等）。

最常用的做法是使用被称为 Caddyfile 的纯文本文件。`Caddyfile` 由（可选的全局选项块及）要发布的网页地址开头，后接数个次级指令。

一个简单的 `Caddyfile` 如下所示，在 `localhost:2020` 发布了一个网页：

```
{
  http_port 2020
}

localhost:2020
file_server
```

用法

```
$ caddy help
$ caddy help run
```

Caddy 可以由页面目录中的任何用户运行，并且 `Caddyfile` 应该位于同一目录中：

```
$ caddy run
```

或者，您可以指定一个自定义的 `Caddyfile`：

```
$ caddy run --config ../path/to/Caddyfile
```

故障排除

证书错误

如果你遇到了任何 SSL 证书相关问题（特别是在非公开域名上），基本上都由 caddy 实例无将证书添加到系统信任存储位置的权限导致。在使用 `caddy.service` 自动启动 caddy 时似乎都由于该问题导致。

要修复该问题，请使用 root 权限执行以下命令。你只需隔很长时间运行一次该命令（根证书的寿命周期）。

```
# XDG_DATA_HOME=/var/lib caddy trust
```

参见

- Caddy 官方网站
- Caddy 用户指南
- Caddyfile 文档

7.2 Artanis

难度 入门 | **篇幅** 5.4 KB | **所属** 网络服务器

本节将讲到

- 安装

- 配置
- 使用
- 参见

GNU Artanis 是 Scheme 编程语言的第一个产品级现代网络框架。它的设计和维护都以稳健、快速和易于使用为目标，适用于专业的网络开发。

GNU Artanis 以 GPLv3+ & LGPLv3+ 发布。它非常轻量级——新手也能轻松破解和学习。它具有一个完整的网络服务器实现，包括一个错误页面处理程序。它支持的数据库（通过 guile-dbi）有 MySQL、SQLite、PostgreSQL。

安装

Guix

提示：推荐的方案。

如果您已经安装 Guix 包管理器，可以直接使用以下命令安装：

```
guix install artanis
```

手动安装

安装 guile包、base-devel包、nss包 等基础软件包。

安装 guile-curlAUR、guile-jsonAUR、guile-redisAUR。

切换至一个新目录，下载最新版本源代码压缩文件并解压，切换至解压得到的目录，运行：

```
$ ./autogen.sh -no-configure
$ ./configure
$ make
# make install
```

AUR

安装 artanisAUR 或 artanis-gitAUR。

配置

首次运行 Artanis 时需要配置文件。

- 如果使用最小模式，例如，所有代码都在脚本文件中，而不在应用程序目录下。配置文件必须命名为 `/etc/artanis/artanis.conf`。
- 如果使用应用程序目录，配置文件 `conf/artanis.conf` 会自动生成。

使用

在控制台中键入 `guile`，进入 **Guile REPL**。屏幕上将显示以下文本：

```
GNU Guile 3.0.9
Copyright (C) 1995-2023 Free Software Foundation, Inc.
```

```
Guile comes with ABSOLUTELY NO WARRANTY; for details type `,show w'.
This program is free software, and you are welcome to redistribute it
under certain conditions; type `,show c' for details.

Enter `,help' for help.
scheme@(guile-user)>
```

本文许多对 Artanis 的操作都会在这里完成。

运行

简单 HTTP 服务器

在控制台中运行这段代码：

```
guile -c "(use-modules (artanis artanis))(init-server)(run)"
```

您会看到这个屏幕：

```
Anytime you want to quit just try Ctrl+C, thanks!
http://127.0.0.1:3000
```

运行一个使用 GNU Artanis 的站点

这是最简单的网站运行方式：

```
#!/bin/env guile
!#
(use-modules (artanis artanis))
(init-server)
(get "/hello" (lambda () "hello world"))
(run)
```

运行服务器

```
scheme@(guile-user)> (run #:host #f #:port #f #:debug #f #:use-db? #f #:dbd #f #:db-username #f #:db-passwd #f #:db-name #f)
```

关键字的值为 #f，默认情况下将从配置文件中获取值。

但你也可以定义它们：

- **#:host**：主机名。
- **#:port**：服务器的套接字端口。
- **#:debug**：设置 #t 如果您想要启用调试模式，日志将更加详细。
- **#:use-db?**：设置 #t 如果您想使用数据库，GNU Artanis 将初始化数据库连接。
- **#:dbd**：选择一个 dbd。这些是支持的三个：postgresql、mysql 和 sqlite3。
- **#:db-username**：你的数据库服务器的用户名。
- **#:db-passwd**：上述用户的数据库密码。
- **#:db-name**：要使用的数据库名称。

与 Nginx 一起工作

您可以使用反向代理尝试 GNU Artanis+Nginx。

虽然 GNU Artanis 有很好的服务器核心，但官方还是建议使用 Nginx 作为前端服务器。除了性能增强外，它还不易受到攻击。

以下是 `/etc/nginx/nginx.conf` 的一些示例行：

```
location / {
proxy_pass http://127.0.0.1:''1234'';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

要使其正常工作，请在编辑文件后重启 Nginx，并运行 GNU Artanis：

```
scheme@(guile-user)> (run #:port 1234)
```

参见

- GNU Artanis web-framework Manual
- `$ info artanis`
- 官方网站
- Savannah 页面
- Gitlab 仓库

7.3 Wordpress

难度 进阶 | **篇幅** 13.0 KB | **所属** 网络服务器

本节将讲到

- 安装
- 配置
- WordPress安装
- 使用
- 故障排除

翻译状态：

- 本文（或部分内容）译自 Wordpress，最近一次同步于 2017-04-05，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2017 (473190), out of sync with English page（在 Talk:Wordpress# 中讨论）

WordPress是由Matt Mullenweg创建的免费开源内容管理系统(CMS)，并于2003年首次发行。WordPress拥有庞大而充满活力的社区，提供数以万计的免费插件和主题，让用户轻松自定义WordPress的外观和功能。 WordPress是根据GPLv2授权的。

Wordpress的最大特性是易于配置与管理。搭建WordPress将使用5分钟（实际上远远不止）. WordPress管理面板允许用户轻松地配置其网站的几乎每个方面，包括获取和安装插件和主题。WordPress提供方便的自动更新。

安装

WordPress需要安装和配置PHP及MySQL 。 详细请查阅LAMP文档。在配置中,某些WordPress功能需要PHP extensions 可能不是默认打开。

使用pacman安装

建议手动安装！！！ Install the wordpress包 package.

警告：虽然使用pacman能更容易的安装和升级WordPress, 但这不是必须的。WordPress具有内置功能，用于管理更新，主题和插件。 如果你决定使用pacman安装官方仓库的,如果不配置必须的权限的话，您将无法使用WordPress管理面板安装插件和主题,或以root身份登录FTP. 无论您是否手动或以其他方式将数据添加到目录，pacman都不会从系统中卸载WordPress安装目录。

手动安装

访问wordpress.org下载WordPress的最新版本，将其解压到你的webserver目录中. 为目录提供足够的权限，允许您的FTP用户写入目录（WordPress需要使用）。

```
cd /srv/http/whatever
wget https://wordpress.org/latest.tar.gz
tar xvzf latest.tar.gz
```

配置

这里使用的配置方法假设您是在本地网络上使用WordPress。

主机配置

确保你的`/etc/hosts`已经正确配置. 当从本地网络访问WordPress CMS时，这将非常重要。你的`/etc/hosts`文件应该如下所示,

```
#<ip-address>   <hostname.domain.org>   <hostname>
127.0.0.1       lithium.kaboodle.net    localhost lithium
::1             lithium.kaboodle.net    localhost lithium
```

注意：如果计划使用主机名安装WordPress，则需要使用代理服务器从移动设备访问您的WordPress安装，否则你的网站将会崩溃 #Appearance is broken (no styling)).

配置apache

注意：你将需要 Apache 已经配置运行PHP 和 MySQL，检查 LAMP#PHP\[损坏的链接：无效的章节\] 和 LAMP#MySQL/MariaDB\[损坏的链接：无效的章节\] 说明的部分。

你将需要为apache创建一个配置文件来找到你安装的WordPress。使用你最爱的编辑器创建以下的文件：

```
# /etc/httpd/conf/extra/httpd-wordpress.conf
```

```
Alias /wordpress "/usr/share/webapps/wordpress"
<Directory "/usr/share/webapps/wordpress">
       AllowOverride All
       Options FollowSymlinks
       Require all granted
</Directory>
```

修改第一行的`/wordpress` 为你想要的。例如，`/myblog` 将要求你导航到 `http://hostname/myblog` 来访问你的`WordPress`.

如果你是手动安装的话，还需要修改你的WordPress安装文件夹的路径。 不要忘记将父目录添加到`php_admin_value`如下所示。

```
# /etc/httpd/conf/extra/httpd-wordpress.conf
Alias /myblog "/mnt/data/srv/wordpress"
<Directory "/mnt/data/srv/wordpress">
       AllowOverride All
       Options FollowSymlinks
       Require all granted
</Directory>
```

然后编辑Apache配置文件，添加以下行：

```
# /etc/httpd/conf/httpd.conf
Include conf/extra/httpd-wordpress.conf
```

现在使用systemd重启`httpd.service` (Apache)。

配置MySQL

MySQL 可以使用多种工具进行配置,但是最常用的是使用命令行和phpMyAdmin.

提示：确保MariaDB已经安装和正确配置。至少要遵循 installation instructions\[损坏的链接：无效的章节\] for Arch Linux.

使用MariaDB命令行工具

首先,以root身份登录。您将被要求输入您的MariaDB root密码：

```
$ mysql -u root -p
```

然后创建一个用户和数据库

数据库名称是

注意：`wordpress`，用户名是`wp-user`。你可以修改它们为你想要的.。还可以使用您的新数据库密码替换`choose_db_password`。你需要在下一部分`localhost`提供这些值

```
MariaDB> CREATE DATABASE wordpress;
MariaDB> GRANT ALL PRIVILEGES ON wordpress.* TO "wp-user"@"localhost" IDENTIFIED BY
"choose_db_password";
MariaDB> FLUSH PRIVILEGES;
MariaDB> EXIT
```

访问WordPress.org official instructions获取更多细节.

使用phpMyAdmin

访问phpMyAdmin来安装和配置phpMyAdmin.

在你的浏览器中,登录到你的phpMyAdmin并执行以下操作:

1. 登录phpMyAdmin。

2. 点击"user"然后点击"Add user"。
3. 在弹出窗口输入名字和密码。
4. 选择"Create database with same name and grant all privileges"给予全部权限。
5. 点击"Add user"按钮来创建一个用户。

WordPress安装

一旦你花了几个小时的时间设置你的http服务器、php和mysql，这将是是最后的安装WordPress的五分钟，让我们开始吧。

1. 访问`[http://hostname/wordpress](http://hostname/wordpress)`。
2. 点击"Create a Configuration File" 按钮。
3. 点击"Let's go!"按钮。
4. 填写上一节中创建的数据库信息。
5. 点击"Submit"。

假如你使用的是pacman安装的WordPress, 那么这个安装过程将无法创建WordPress使用的wp-config.php文件的正确权限。您将不得不使用WordPress将提供的信息以root身份执行此步骤。

将出现一个页面，说WordPress无法写入wp-config.php文件。复制编辑框中的文本，并在文本编辑器中以root身份打开`/usr/share/webapps/wordpress/wp-config.php`。 将复制的文本粘贴中并保存文件。

之后,您必须使用chown将/usr/share/webapps/wordpress/及其中的所有文件的权限修改为 用户为`http` 组为`http`，以便网络服务器可以访问它。

最后，点击"Run the install" ，WordPress将使用您的信息填充数据库。一旦完成,页面将显示"Success!"。点击login按钮结束你的安装。

现在是从所有设备访问您的网站的好时机，确保您的WordPress安装设置正确。

使用

安装一个主题

寻找新的主题

WordPress有数以万计的可用主题。在谷歌搜索一个好的主题就好像通过一个充满垃圾的河流。寻找主题的好地方包括：

- Official WordPress theme website
- Smashing Magazine
- The Theme Factory
- Woo Themes

提示：可以使用WordPress的管理界面来安装插件和主题。为此， make the user that serves WordPress the owner of your WordPress directory. For Apache this user is normally http.

使用管理面板安装主题

Before installing a theme using the admin panel, you will need to setup an FTP server on your WordPress host. To maintain a high level of protection, you might set up a user on your system specifically for WordPress, give it the home directory of `<path to your WordPress install>/wp-content`, disallow anonymous login, and allow no more users to log in than for WordPress (and obviously others as required by your setup).

Once the FTP server is setup, login to your WordPress installation and click "Appearance->Install Themes->Upload". From there select your zip file that contains your theme and click "Install Now". You will be presented with a box asking for FTP information, enter it and click "Proceed". You might need to update file ownership and rights if WordPress reports that it is unable to write to the directory. If you have been following along closely, you should now have an installed theme. Activate it if you wish.

手动安装主题

Download the archive and extract into the **wp-content/themes** folder

```
# Example for a theme named "MyTheme"
cd /path/to/wordpress/root/directory
cd wp-content/themes
# get the theme archive and extract
wget http://www.example.com/MyTheme.zip
unzip MyTheme.zip
# remove the archive (optional)
rm MyTheme.zip
```

Be sure to follow any additional instructions as provided by the theme author.

Select your new theme from the theme chooser ("Appearance->Themes")

Installing a plugin

The steps for installing a plugin are the same as they are for installing a theme. Just click the "Plugins" link in the left navigation bar and follow the steps.

升级

Every now and then when you log into wordpress there will be a notification informing you of updates. If you have correctly installed and configured an FTP client, and have the correct filesystem permissions to write in the WordPress install path then you should be able to perform updates at the click of a button. Just follow the steps.

Alternatively, you can use SSH to update your installation with the SSH SFTP Updater Support plugin.

故障排除

Appearance is broken (no styling)

Your WordPress website will appear to have no styling to it when viewing it in a web browser (desktop or mobile) that does not have its hostnames mapped to ip addresses correctly.

This occurs because you used a url with the hostname of your server, instead of an ip address, when doing the initial setup and WordPress has used this as the default website URL.

To fix this, you will either need to edit your /etc/hosts file or setup a proxy server. For an easy to setup proxy server, see Polipo, or if you want something with a little more configuration, see Squid.

Another option is changing a value in the database table of your WordPress, specifically the wp_options table. The fix is to change the siteurl option to point directly to the domain name and not "localhost".

Tips and tricks

See also

- WordPress
- Content management system

7.4 Lighttpd

难度 进阶 | **篇幅** 13.4 KB | **所属** 网络服务器

本节将讲到

- 安装
- 设置
- 参见

lighttpd是一个安全，快速，标准，且非常灵活的网页服务器，并对高性能环境做了最佳化。相较于其他网页服务器,它占用的内存很少，且注重CPU负载量。它的进阶功能集 （FastCGI，CGI，验证，输出压缩，网址重定向等等）让Lighttpd成为每个试图摆脱瓶颈的服务器的完美网页服务器软件。

安装

Lighttpd可以从\[extra\]仓库获得:

```
# pacman -S lighttpd
```

设置

基本设置

lighttpd配置文件： `/etc/lighttpd/lighttpd.conf`。在安装完成后它会自动生成一个用于测试的测试页面。

想要检查 `lighttpd.conf` 中的语法错误，可以使用以下命令来快速查找错误：

```
$ lighttpd -t -f /etc/lighttpd/lighttpd.conf
```

预设配置文件会把 `/srv/http/` 指定为提供HTTP服务的文件目录。

测试安装结果：

```
# chmod 755 /srv/http/index.html
# echo 'TestMe!' >> /srv/http/index.html
```

修改日志目录权限:

```
# chown -R http:http /var/log/lighttpd
```

启动服务:

```
# systemctl start lighttpd
```

然后就可以使用浏览器打开网址 `localhost` ，你应该能见到测试页面。

开机自动启动:

```
# systemctl enable lighttpd
```

示例配置文件在 `/usr/share/doc/lighttpd/`。

基本日志

lighttpd 能够输出错误与访问记录到日志文件中。错误日志默认启用 (`server.errorlog` 选项). 如下编辑`/etc/lighttpd/lighttpd.conf`启用访问日志：

```
server.modules += (
   "mod_accesslog",
)

accesslog.filename = "/var/log/lighttpd/access.log"
```

通过SSL启动HTTPS

警告：计划独立实现SSL/TLS的用户应当注意，SSL/TLS的一些变种和实现难以抵抗攻击。参阅 OpenSSL

提示：

- Mozilla 有一个能用于lighttpd的SSL 配置生成器。
- 设置SSL后, 你可以用 Qualys SSL Labs' SSL服务器检验来检查你的配置

自签名证书

安装openssl包后，自签名SSL证书用以下方式生成：

```
# mkdir /etc/lighttpd/certs
# openssl req -x509 -nodes -days 7300 -newkey rsa:2048 -sha256 -
keyout /etc/lighttpd/certs/server.pem -out /etc/lighttpd/certs/server.pem
# chmod 600 /etc/lighttpd/certs/server.pem
```

修改`/etc/lighttpd/lighttpd.conf`，增加以下内容来启用HTTPS。

```
server.modules += ( "mod_openssl" )
$SERVER["socket"] == ":443" {
    ssl.engine                  = "enable"
    ssl.pemfile                 = "/etc/lighttpd/certs/server.pem"
 }
```

详情请参阅 lighttpd TLS 配置。

Let's Encrypt

你也可以生成被Let's Encrypt签名的证书。

在`/etc/lighttpd/lighttpd.conf`增加：

```
$SERVER["socket"] == ":443" {
    ssl.engine                  = "enable"
    ssl.privkey                 = "/etc/letsencrypt/live/domain/privkey.pem"
    ssl.pemfile                 = "/etc/letsencrypt/live/domain/fullchain.pem"
}
```

详情请参阅 lighttpd 文档中的启用 Let's Encrypt。

重定向HTTP请求到HTTPS

你需要在`/etc/lighttpd/lighttpd.conf`中的server.modules数组中增加`"mod_redirect"`:

```
server.modules += ( "mod_redirect" )

$SERVER["socket"] == ":80" {
  $HTTP["host"] == "example.org" {
    url.redirect = ( "^/(.*)" => "https://example.org/$1" )
    server.name                 = "example.org"
  }
}

$SERVER["socket"] == ":443" {
  ssl.engine = "enable"
  ssl.pemfile = "/etc/lighttpd/certs/server.pem"
  server.document-root = "..."
}
```

为了重定向所有网络主机到它们的HTTPS安全访问方式，用以下内容取代之前的socket 80端口配置：

```
$SERVER["socket"] == ":80" {
  $HTTP["host"] =~ ".*" {
    url.redirect = (".*" => "https://%0$0")
  }
}
```

如仅需重定向所有网络主机的一部分 (例如，“secure” or ”phpmyadmin“):

```
$SERVER["socket"] == ":80" {
  $HTTP["url"] =~ "^/secure" {
    url.redirect = ( "^/(.*)" => "https://example.com/$1" )
  }
}
```

用密码保护目录

注意：此模块需要mariadb-libs包。一个近似于系统的/etc/passwd的密码文件需要被用于用户的认证。安装需要特定的格式和经md5sum散列的密码，但用户可以用下列命令轻松创建：

```
$ user=foo
$ password=b@R102
$ realm='Password Required'
$ hash=`echo -n "$user:$realm:$password" | md5sum | cut -b -32`

# echo "$user:$realm:$hash" >> /etc/lighttpd/lighttpd.user
```

修改/etc/lighttpd/lighttpd.conf，增加下列内容以启用目录保护:

```
server.modules += ( "mod_auth", "mod_authn_file" )

auth.backend               = "htdigest"
auth.backend.htdigest.userfile = "/etc/lighttpd/lighttpd.user"

# 注意：这里的路径是相对于server.document-root变量的
auth.require = ( "/secret" =>
   (
    "method" => "basic",
    "realm" => "Password Required",
    "require" => "valid-user"
   )
)
```

注意： 写入/etc/lighttpd/lighttpd.conf的"user"和"realm"项必须与/etc/lighttpd/lighttpd.user中相符，以使认证正常运作。

CGI

Common Gateway Interface (CGI) 脚本对于lighttpd可以开箱即用, 你只需要开启CGI模块, 指定配置文件，并确保指定语言的解释器已经安装。 (如：使用python需要安装python包)

创建 /etc/lighttpd/conf.d/cgi.conf， 添加以下内容:

```
server.modules += ( "mod_cgi" )

cgi.assign                 = ( ".pl"  => "/usr/bin/perl",
                               ".cgi" => "/usr/bin/perl",
                               ".rb"  => "/usr/bin/ruby",
                               ".erb" => "/usr/bin/eruby",
                               ".py"  => "/usr/bin/python",
                               ".php" => "/usr/bin/php-cgi" )

index-file.names           += ( "index.pl",   "default.pl",
                               "index.rb",   "default.rb",
                               "index.erb",  "default.erb",
                               "index.py",   "default.py",
                               "index.php",  "default.php" )
```

对于PHP脚本，请确认在 /etc/php/php.ini添加了下列配置：

```
cgi.fix_pathinfo = 1
```

在你的lighttpd配置文件中/etc/lighttpd/lighttpd.conf添加：

```
include "conf.d/cgi.conf"
```

FastCGI

安装 fcgi包，之后你的lighttpd就有了fcgi支持， 以下内容是给需要使用Ruby，PHP或者Python的人的指导。

注意： lighttpd现在默认以用户 `http` 运行。

首先复制一份默认配置，

从 `/usr/share/doc/lighttpd/config/conf.d/fastcgi.conf` 复制到 `/etc/lighttpd/conf.d`

将以下内容添加到 `/etc/lighttpd/conf.d/fastcgi.conf`

```
server.modules += ( "mod_fastcgi" )

#server.indexfiles += ( "dispatch.fcgi" ) #这是强烈不建议的
index-file.names += ( "dispatch.fcgi" ) # 如果选用rails，增加dispatch.fcgi

server.error-handler-404   = "/dispatch.fcgi" #同上
fastcgi.server = (
    ".fcgi" => (
      "localhost" => (
        "socket" => "/run/lighttpd/rails-fastcgi.sock",
        "bin-path" => "/path/to/rails/application/public/dispatch.fcgi"
      )
    )
)
```

然后在 `/etc/lighttpd/lighttpd.conf`中添加:

```
include "conf.d/fastcgi.conf"
```

下面是关于PHP的指导。

PHP

安装 php包 和 php-cgi包 (可以参阅 PHP 、 LAMP).

确认php-cgi可以工作： `php-cgi --version`

```
PHP 5.4.3 (cgi-fcgi) (built: May  8 2012 17:10:17)
Copyright (c) 1997-2012 The PHP Group
Zend Engine v2.4.0, Copyright (c) 1998-2012 Zend Technologies
```

如果输出内容与上面相仿则php已经正确安装。

创建一份新的配置文件:

```
/etc/lighttpd/conf.d/fastcgi.conf
# 请确保php和php-cgi已正确安装。参阅:
# https://wiki.archlinux.org/index.php/Fastcgi_and_lighttpd#PHP

server.modules += ("mod_fastcgi")

# FCGI server
# ===========
#
# 配置一个处理PHP请求的FastCGI服务器
#
index-file.names += ("index.php")
fastcgi.server = (
```

```
    ".php" => (
        # 下列FastCGI服务器中的负载均衡配置。
        # 服务器的命名仅用作日志中服务器的标识名。
        "localhost" => (
            "bin-path" => "/usr/bin/php-cgi",
            "socket" => "/tmp/php-fastcgi.sock",
            # 分解SCRIPT_FILENAME 以让PHP能够从中解析出PATH_INFO
            "broken-scriptfilename" => "enable",
            # 启动 (max-procs + (max-procs * PHP_FCGI_CHILDREN)) 个进程，其中
            # max-procs 个是"watcher"进程，其余是"worker"进程。参阅：
            #
https://redmine.lighttpd.net/projects/1/wiki/frequentlyaskedquestions#How-many-php-CGI-
processes-will-lighttpd-spawn
            "max-procs" => 4, # 默认值
            "bin-environment" => (
                "PHP_FCGI_CHILDREN" => "1" # 默认值
            )
        )
    )
)
```

在/etc/lighttpd/lighttpd.conf中添加以下内容使新配置能够应用:

```
/etc/lighttpd/lighttpd.conf
include "conf.d/fastcgi.conf"
```

注意：模块顺序十分重要, 正确地模块加载顺序位于 `/usr/share/doc/lighttpd/config/modules.conf`. 任何错误配置都可能导致 *lighttpd* 崩溃。

重新加载 lighttpd：

1. systemctl restart lighttpd

注意：

- 如果你在访问php文件时遇到诸如 *No input file found* 的错误, 有很多原因可以导致这个错误，请参阅 this FAQ 。
- 确认没有其他模块 (如 `mod_cgi`) 负责处理php文件。

使用php-fpm

为了动态管理PHP进程，你可以安装php-fpm包，启动`php-fpm.service`。

注意：你可以编辑`/etc/php/php-fpm.conf`以调整池中的服务器数量，以及其他配置项。更多关于php-fpm的细节可在php-fpm 网站上找到。注意：你对`/etc/php/php.ini`做出改动后需要重启`php-fpm.service`。

在 `/etc/lighttpd/conf.d/fastcgi.conf` 添加:

```
server.modules += ( "mod_fastcgi" )

index-file.names += ( "index.php" )

fastcgi.server = (
    ".php" => (
      "localhost" => (
        "socket" => "/run/php-fpm/php-fpm.sock",
        "broken-scriptfilename" => "enable"
      ))
)
```

uWSGI

在 /etc/lighttpd/lighttpd.conf中加入

```
server.modules += ("mod_scgi")

$HTTP["url"] =~ "^/uwsgi/" {
    scgi.protocol = "uwsgi"
    scgi.server   = (
        "/uwsgi/foo" => ((
            "socket"             => "/path/to/socket",
            "check-local"        => "disable"
        )),
        "/uwsgi/bar" =>
            "host"               => "127.0.0.1",
            "port"               => "8080",
            "check-local"        => "disable"
        ))
    )
}
```

可以采用systemd unit\]或\[direct的方式启动ywsgi应用。 此处是一个来自 digitalocean 的小巧的教程，指出如何从零开始创建一个Flask框架应用程序。

输出压缩

复制示例配置文件：

```
# mkdir /etc/lighttpd/conf.d
# cp /usr/share/doc/lighttpd/config/conf.d/deflate.conf /etc/lighttpd/conf.d/
```

在/etc/lighttpd/lighttpd.conf中增加如下内容：

```
include "conf.d/deflate.conf"
```

Finally, reload lighttpd.service, 后续将会自动压缩纯文本和 html 内容。

注意：不能通过在/etc/lighttpd/lighttpd.conf增加内容代替复制deflate.conf的作用

/etc/lighttpd/conf.d/deflate.conf中的deflate.mimetypes参数配置何种类型的内容应该被压缩：

```
deflate.mimetypes           = ("text/plain", "text/html", "text/javascript",
"text/css", "text/xml")
```

可以创建缓存目录保存压缩后的文件:

```
# mkdir /var/cache/lighttpd/compress
# chown http:http /var/cache/lighttpd/compress
```

取消 /etc/lighttpd/conf.d/deflate.conf 中 deflate.cache-dir 选项的注释并修改为：

```
deflate.cache-dir = "/var/cache/lighttpd/compress"
```

参见

- Lighttpd wiki

7.5 Tomcat

难度 进阶 | **篇幅** 15.9 KB | **所属** 网络服务器

本节将讲到

- 安装
- 初始化配置
- 启动/停止 Tomcat
- 部署和处理 Web 程序
- 日志记录

Tomcat 是一个由 Apache Software Foundation 开发的开源的 Java Servlet 容器。

注意：目前存在四个版本的稳定分支： 7, 8, 9 和 10, 。没一个版本能完全替代另一个。相反，每个分支是一部分 "Servlet" 和 "JSP" Java 标准的实现。两个版本都被 Arch Linux 官方支持：tomcat8包, tomcat9包 and tomcat10包。选用哪个版本主要由你的 web 程序的需求来决定的。

安装

安装 tomcat8包, tomcat9包, 或 tomcat10包 中的一个。

如果打算部署 Tomcat 到一个生产环境，请考虑安装 tomcat-native包 。Tomcat-native 将配置服务器来使用 Apache Portable Runtime (APR) 库的网络连接（socket）和 RNG 实现。它将使用原生的32位或64位代码来提升性能，对速度非常敏感的生产环境经常会用到。Tomcat 的安装不需要额外的配置。更多的信息请参阅 Tomcat 官方文档。

使用 tomcat-native 将会移除位于 `catalina.err` 的下列警告：

```
INFO: The APR based Apache Tomcat Native library which allows optimal performance in production environments was not found on the java.library.path [...]
```

文件系统层次结构

将 `n` 用实际安装的版本 (8, 9, 10) 替代。

路径	用途
`/etc/tomcat*n*`	配置文件。包括： `tomcat-users.xml` (defines users allowed to use administration tools and their roles), `server.xml` (Main Tomcat configuration file), `catalina.policy` (security policies configuration file)
`/usr/share/tomcat*n*`	Tomcat 主文件夹。包括脚本文件及到其它文件夹的链接
`/usr/share/java/tomcat*n*`	Tomcat 的 Java 库（jar文件）
`/var/log/tomcat*n*`	不被 `systemd` 记录的日志文件 (参阅 #日志记录)
`/var/lib/tomcat*n*/webapps`	Tomcat 部署你的 Web 程序的地方
`/var/tmp/tomcat*n*`	Tomcat 存储你的 Web 程序数据的地方

初始化配置

要使用 manager 和 admin 管理界面，请编辑下面这个文件 `/etc/tomcat*n*/tomcat-users.xml`

取消 XML 声明中"role and user"这一块的注释，然后根据你的需要，修改并启用 `tomcat`, {{Ic|admin-{gui,script} }} 和 {{Ic|manager-{gui,script,jmx,status} }} 等角色（详情请参阅 Configuring Manager Application Access）。

简单来说，`tomcat` 用于运行 Tomcat 服务器，`manager-*` 用于管理 Tomcat 服务器上的 web 程序，`admin-*` 是 Tomcat 服务器的全权管理员。

下面是一个配置文件的样例，里面定义了一些包含用户名和密码的角色（请务必修改把 \[CHANGE_ME\] 改成更为更安全的密码）：

```
/etc/tomcatn/tomcat-users.xml
<?xml version='1.0' encoding='utf-8'?>
<tomcat-users>
  <role rolename="tomcat"/>
  <role rolename="manager-gui"/>
  <role rolename="manager-script"/>
  <role rolename="manager-jmx"/>
  <role rolename="manager-status"/>
  <role rolename="admin-gui"/>
  <role rolename="admin-script"/>
  <user username="tomcat" password="[CHANGE_ME]" roles="tomcat"/>
  <user username="manager" password="[CHANGE_ME]" roles="manager-gui,manager-
script,manager-jmx,manager-status"/>
  <user username="admin" password="[CHANGE_ME]" roles="admin-gui"/>
</tomcat-users>
```

请记住，每一次修改了这个文件都必须要重启 Tomcat 服务器。

这篇博文更好地介绍了这些角色。

为了读取配置文件，以及为了和一些 IDE 更好地配合，需要加入 `tomcat*n*` 用户组：

```
gpasswd -a <user> tomcat<number>
```

启动/停止 Tomcat

启动 `tomcat*n*.service` 服务。

一旦 Tomcat 启动了，你可以通过访问这个页面来查看结果：http://localhost:8080 。如果显示了一个漂亮的 Tomcat 本地页面，表明你的 Servlet 容器正在运行，并且可以装载你的 web 程序了。如果启动脚本失败了，又或者你在浏览器里只看到了一个 Java 错误页面，你可以通过 systemd 的 journalctl 命令来查看启动日志。Google 上有 Tomcat 日志里常见问题的解决办法。

注意：To improve security, Arch Linux's Tomcat packages use the jsvc binary from Apache's commons-daemons. Tomcat's `systemd` service runs this Apache binary with root privileges which itself starts Tomcat with an underprivileged user (`tomcat*n*:tomcat*n*` in Arch Linux). This prevents malicious code that could be executed in a bad web application from causing too much damage. This also enables the use of ports under 1024 if needed. See `man jsvc` for options available and pass them through the `CATALINA_OPTS` environment variable declared in `/etc/conf.d/tomcat*n*`.

备用“手动”方法

Tomcat 也能通过上游脚本来直接控制：

```
/usr/share/tomcat/bin/{startup.sh,shutdown.sh,..}
```

这种方法对于调试程序甚至调试 Tomcat 非常有帮助。但不要在首次启动 Tomcat 的时候用这种方法，因为这会导致一些权限错误，从而使一些程序停止运行。为了使用这些脚本，可能需要做一些额外的配置。并且使用这些脚本将会阻止上面所提到的 jsvc 安全优势。

部署和处理 Web 程序

Tomcat 7 捆绑了5个已经部署了的 Web 程序（有必要的话，把 localhost 换成你的服务器的全称域名）：

- 默认主页: http://localhost:8080/
- Tomcat 7 的本地文档: http://localhost:8080/docs/
- Servlets 和 JSP 的示例: http://localhost:8080/examples/
- 用于处理虚拟主机的 host-manager: http://localhost:8080/host-manager/
- 用于管理 web 程序的 manager: http://localhost:8080/manager/html/

GUI 方法

最简单的方法是通过 manager 程序来部署 http://localhost:8080/manager/html。 使用你在 `tomcat-users.xml` 定义的 `manager` 角色的用户名和密码来登录。登录进去后，你将看到5个已经部署了的 Web 程序。通过 “Deploy” 区域来添加你自己的 Web 程序，通过 “Applications” 区域来停止/启动/取消部署。

CLI 方法

你也可以通过复制程序的 WAR 文件到 `/usr/share/tomcat*n*/webapps` 目录来部署。确保 `autoDeploy` 选项如下面所示，设置在了正确的主机：

```
/etc/tomcatn/server.xml
...
<Host name="localhost"  appBase="webapps"
      unpackWARs="true" autoDeploy="true">
...
```

把项目部署在别的目录

你也可以通过设置 `Context` 来把你的项目部署在别的目录。 你需要在 `/etc/tomcat*n*/Catalina/localhost/` 目录下创建你的 context。Context 是一个用于指定 tomcat 查找目录的 xml 文件，其基本格式如下：

```
/etc/tomcatn/Catalina/localhost/whatShouldFollowLocalhost.xml
<Context path="/whatSholdFollwLocalhost" docBase="/where/your/project/is/"
reloadable="true"/>
```

假设项目放在用户 /home 目录下的某个文件夹里，则 context 像这样写：

```
/etc/tomcatn/Catalina/localhost/myProject.xml
<Context path="/myProject" docBase="/home/archie/code/jsp/myProject" reloadable="true"/
>
```

现在项目文件可以放在 `/home/archie/code/jsp/myProject/` 目录下了。要想在浏览器里查看这个项目，访问 http://localhost:8080/myProject 。 如果 tomcat 无法加载这些文件，可能是遇到了权限问题，运行 `chmod o+x /home/archie/code/jsp/myProject` 应该可以解决这个问题。

日志记录

如果使用的是 Arch Linux 官方的 Tomcat 包，则使用 systemd 的 journalctl 来记录**启动日志**。这意味着 `/var/log/tomcat*n*/catalina.err` 和 `/var/log/tomcat*n*/catalina.out` 将**不会**被使用。而其它的日志，例如作为 `Valve` 定义在 `/etc/tomcat*n*/server.xml` 里的 access logs 和 business logs 则默认会记录在 `/var/log/tomcat*n*/` 目录下。

为了保存上游风格的日志，把 systemd 文件 `/lib/systemd/system/tomcat*n*.service` 复制到 `/etc/systemd/system/tomcat*n*.service` ，然后把两处 `SYSLOG` 改为日志文件的绝对路径。

进一步设置

基本的配置，可以通过虚拟主机管理程序来设置：http://localhost:8080/host-manager/html 。使用你在 `tomcat-users.xml` 中设置的用户名和密码来登录。其它选项可通过修改 `/etc/tomcat*n*` 下的配置文件来设置，最重要的莫过于 `server.xml`。对这些文件的修改已经超过了本入门页面的范围，请访问官方 Tomcat 7 文档来获得更多支持。

从以前版本的 Tomcat 中迁移

正如介绍里说的，“Tomcat 8 并不替代 Tomcat 7”。他们都是 Servlet/JSP 标准的实现。因此，程序使用了哪个版本的 Servlet/JSP 决定了需要使用哪个版本的 Tomcat 。如果需要迁移，官方网站会给你建议。

使用不同版本的 JRE/JDK 来运行 Tomcat

除了安装需要的 JRE/JDK 外，唯一要做的就是设置 Tomcat `systemd` service 文件的 TOMCAT_JAVA_HOME 变量。

安装后，使用 drop-in file 配置 `TOMCAT_JAVA_HOME`:

```
/etc/systemd/system/tomcatn.service.d/start.conf
[Service]
Environment=TOMCAT_JAVA_HOME=/usr/lib/jvm/java-8-openjdk
```

安全配置

本页面最低限度地使你的第一个 web 应用运行在 Tomcat 上，并不意味着这是一个管理 Tomcat （这是一项单独的工作）的明确的指导。Tomcat 官方网站提供了所有必须的官方事项。你也可以参考这个 O'Reilly 页面和 这个页面。这里还是提供了一些安全方面的建议：

- 保持你的 Tomcat 更新到了最新版，以便获得安全问题的最新修复
- 移除不需要的默认程序，比如 `examples`，`docs`，默认的主页 `ROOT` ("_" 在 `manager` 程序里) 。这可以防止潜在的安全漏洞被利用。使用 `manager` 来管理。

为了更安全，你甚至可以删除 host-manager 和 manager 应用。但是别忘了，后者对部署 web 应用非常有用。

- 关闭 WAR 自动部署选项。这可以防止某些获得限制权限的人复制 WAR 文件到 `/usr/share/java/webapps` 来直接运行它。编辑 `server.xml` 并把 `autoDeploy` 设为 `false`：

```
/etc/tomcatn/server.xml
...
<Host name="localhost"  appBase="webapps"
      unpackWARs="true" autoDeploy="false">
...
```

- 匿名化 Tomcat 的默认错误页面来防止潜在的攻击者取得 Tomcat 的版本。想知道 Tomcat 默认显示什么，只需要访问一个不存在的页面，比如 http://localhost:8080/I_dont_exist ，你将会看到一个404错误页面，最底下标明了 Tomcat 的版本。

要想匿名化这个，编辑或打开这个 JAR （像 `vim` 等编辑器能直接编辑 zip 文件）

```
/usr/share/tomcatn/lib/catalina.jar
```

然后编辑这个文件

```
org/apache/catalina/util/ServerInfo.properties
...
server.info=
server.number=
server.built=
...
```

- 禁用 `server.xml` 里不使用的 `connectors`
- 保持对 `/etc/tomcat*n*/server.xml` 的访问限制。只有 `tomcat` 用户，和/或者 `root` 能读写这个文件

- 保持 `jsvc` 的使用。不要使用上游启动脚本，除非有上文安全部分提到的特别原因。
- 在 `tomcat-users.xml` 里为每个用户启用强壮并且不同的密码，给真正需要的用户角色，并且禁用你用不到的用户或者角色。

你还可以用使用下列上游脚本来加密 `tomcat-users.xml` 里的密码：

```
/usr/share/tomcatn/bin/digest.sh -a SHA NEW_PASSWORD
```

这条命令将会输出类似于下面的信息：

```
NEW_PASSWORD:b7bbb48a5b7749f1f908eb3c0c021200c72738ce
```

然后用哈希的部分替换掉 `tomcat-users.xml` 里的密码部分，然后把下文加入到 `server.xml` 中：

```
/etc/tomcatn/server.xml
<Host
  ...
  <Realm
    ...
    className="org.apache.catalina.realm.MemoryRealm" digest="SHA"
    ...
  />
  ...
/>
```

注意，这个方法可能意义并不大。因为只有 root 和/或者 tomcat 能读写这个文件，而如果一个入侵者成功获得了 root 权限，那么他将不需要这些密码来混乱你的程序和数据。请确保对这个文件的读写限制！永远弄清楚你在部署什么。

故障排除

Tomcat 服务已经启动，但是无法加载页面

首先，检查 `/etc/tomcat*n*/tomcat-users.xml` 有没有语法错误。如果一切正常并且 `tomcat*n*` 正在运行，输入 `journalctl -r` 查看log，看看有没有抛出什么异常（参照 Logging ）。 如果你看到有类似 `java.lang.Exception: Socket bind failed: Address already in use` 的异常，说明其它服务正在监听这个端口。例如，有可能 Apache 和 Tomcat 都在监听同一个端口（例如你在8080端口运行的 Apache 使用在80端口运行的Nginx 作为代理）。如果是这种情况，编辑 `/etc/tomcat*n*/server.xml` 文件，然后修改 `<Service name="Catalina">` 下的 Connector port 为其它的值：

```
/etc/tomcatn/server.xml
<?xml version='1.0' encoding='utf-8'?>
...
...
<Service name="Catalina">
    <Connector executor="tomcatThreadPool"
               port="8090" protocol="HTTP/1.1"
               connectionTimeout="20000"
               redirectPort="8443" />
...
```

```
...
</Service>
```

最后重启 `tomcat*n*` 和 `httpd` 服务。

如果还没有解决，而且是在虚拟机中，请尝试删除并重建 `/dev/random` (cf. Solution: FUTEX_WAIT hangs Java on Linux / Ubuntu in vmware or virtual box):

```
# rm /dev/random
# mknod -m 644 /dev/random c 1 9
```

或者修改 `/usr/lib/jvm/java-8-openjdk/jre/lib/security/java.security` 为指向 `/dev/urandom/`

7.6 Apache HTTP 服务器

难度 进阶 | **篇幅** 18.3 KB | **所属** 网络服务器

本节将讲到

- 安装
- 配置
- 扩展
- 问题处理
- 参阅

LAMP是指在许多web 服务器上使用的一个软件组合：Linux,Apache,MySQL/MariaDB以及PHP。

Apache HTTP 服务器，简称 Apache，是非常流行的Web服务器软件。通常和脚本语言比如 PHP，数据库 MySQL 一起工作，合称为 LAMP) 栈(**L**inux, **A**pache, **M**ySQL, **P**HP). 本文介绍。本文档描述了怎样安装设置 Apache 网页服务器。以及选择安装 PHP和 MySQL 并集成到Apache服务器中。

安装

安装软件包 apache包.

配置

Apache 配置文件位于 `/etc/httpd/conf`，主要的配置文件是 `/etc/httpd/conf/httpd.conf`, 此文件会引用其它文件。

用默认配置可以启动一个简单的服务，有用户访问时会提供目录 `/srv/http` 下的内容。

启动 `httpd.service` systemd 服务，Apache 就会启动，从浏览器中访问 http://localhost/ 会显示一个简单的索引页面。

高级选项

请参考 Apache 完整 directives 配置选项和 directive 快速参考.

请关注一下 `/etc/httpd/conf/httpd.conf` 中的下面选项:

`User http`
: 出于安全原因，Apache以root用户身份启动(直接的或者通过启动脚本)后将立即切换为 `/etc/httpd/conf/httpd.conf`中指定的 UID，默认配置是 *http*, 安装时会自动创建此用户。

`Listen 80`
: Apache 监听的端口，要被外网访问，请在路由器开放此端口。 : 如果是本地调试用，可以用下面命令设置为仅供本地访问 `Listen 127.0.0.1:80`.

`ServerAdmin you@example.com`
: 管理员的电子邮件，在错误页面会展示给用户。

`DocumentRoot "/srv/http"`
: 网页的目录. : 如果需要可以修改这个目录，请记得同步修改 `<Directory "/srv/http">` 和`DocumentRoot`,否则访问新位置时可能出现 **403 Error** (缺少权限)问题。不要忘记修改 `Require all denied` 行到 `Require all granted`，否则会出现 **403 Error**. DocumentRoot 目录及其父目录必须有可执行权限，这样再能被服务器进程使用的用户访问到(用 `chmod o+x /path/to/DocumentRoot` 设置)，否则会出现 **403 Error**.

`AllowOverride None`
: 在 `<Directory>` 段落中的这个设置会让 Apache 完全忽略 `.htaccess` 文件。从 Apache 2.4，这个设置以及是默认的，所以如果要使用 `.htaccess`，请允许Overide. 如果要在 `.htaccess` 中使用 `mod_rewrite` 或其它设置, 可以指定哪些目录允许覆盖服务器配置。更多信息请访问 Apache 文档.

提示：可以用 `apachectl configtest` 检查配置文件是否存在问题。

更多设置可以访问 `/etc/httpd/conf/extra/httpd-default.conf`, 例如

关闭服务器签名:

`ServerSignature Off`
隐藏 Apache 和 PHP 版本等属性:

`ServerTokens Prod`

用户目录

在默认设置下，可以通过 http://localhost/~yourusername/ 访问用户的主目录并显示 `~/public_html` 中的内容 (可以通过 `/etc/httpd/conf/extra/httpd-`

userdir.conf 设置). 要禁用这个访问，请注释掉 /etc/httpd/conf/httpd.conf 文件中的如下行：

```
Include conf/extra/httpd-userdir.conf
```

**本文或本章节的事实准确性存在争议。**

原因： It is not necessary to set +x for every users, setting it only for the webserver via ACLs suffices (see Access Control Lists#Granting execution permissions for private files to a Web Server).（在 Talk:Apache HTTP 服务器 中讨论）

请正确设置目录的权限，使得 Apache 可以访问到文件。主目录和 ~/public_html 必须是可被其它用户执行:

```
$ chmod o+x ~
$ chmod o+x ~/public_html
$ chmod -R o+r ~/public_html
```

重启 httpd.service 服务以应用更改。参考 Umask#Set the mask value.

TLS/SSL

警告：如果计划使用 SSL/TLS，请注意某些版本和实现 依然 有安全漏洞. 访问 https://disablessl3.com/ 和 https://weakdh.org/sysadmin.html 可以查看当前的安全漏洞和服务器处理方式。

OpenSSL 提供了 TLS/SSL 支持，默认已经安装在 Arch 中。

在 /etc/httpd/conf/httpd.conf 中，取消下面行的注释:

```
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
Include conf/extra/httpd-ssl.conf
```

TLS/SSL 需要密钥和认证，如果你有公开域名，可以使用 Let's Encrypt 免费获取认证，如果没有，请参考 #创建密钥并自签名.

获取密钥和认证之后，请将 /etc/httpd/conf/extra/httpd-ssl.conf 中的 SSLCertificateFile 和 SSLCertificateKeyFile 指向对应的文件。如果还生成了 CA 认证链，请将文件名设置到 SSLCertificateChainFile.

重启 httpd.service.

提示：Mozilla 的 SSL/TLS 文章 包含了 Apache 相关 配置的指南和一个 自动生成工具，可以有助于创建更安全的配置。

创建密钥并自签名

创建一个私钥并自己签名认证，对于不需要 CSR 的大部分使用来说已经足够:

```
# cd /etc/httpd/conf
# openssl req -new -x509 -nodes -newkey rsa:4096 -keyout server.key -out server.crt -
days 1095
# chmod 400 server.key
```

注意：\-days 参数是可选的，RSA 密钥大小最低是 2048 (default).

如果需要创建 CSR，用下面的密钥创建方:

```
# openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out server.key
# chmod 400 server.key
# openssl req -new -sha256 -key server.key -out server.csr
# openssl x509 -req -days 1095 -in server.csr -signkey server.key -out server.crt
```

注意：openssl 手册 和 opnssl 文档 包含了更多信息。

Virtual Hosts

注意：You will need to add a separate <VirtualHost *:443> section for virtual host SSL support. See #管理多个主机 for an example file.

如果需要不止一个主机，取消 `/etc/httpd/conf/httpd.conf`行的注释:

```
Include conf/extra/httpd-vhosts.conf
```

在 `/etc/httpd/conf/extra/httpd-vhosts.conf` 中设置虚拟主机，默认文件包含了一个示例。

要在本地机器测试虚拟主机，将虚拟名称加入 `/etc/hosts` 文件:

```
127.0.0.1 domainname1.dom
127.0.0.1 domainname2.dom
```

重启 `httpd.service` 服务。

管理多个主机

如果要管理的主机非常多，希望更方便的维护，建议为每一个虚拟主机创建一个配置文件并文件存储到一个文件夹中 `/etc/httpd/conf/vhosts`。

创建目录:

```
# mkdir /etc/httpd/conf/vhosts
```

编写单独的配置文件:

```
# nano /etc/httpd/conf/vhosts/domainname1.dom
# nano /etc/httpd/conf/vhosts/domainname2.dom
...
```

在 `/etc/httpd/conf/httpd.conf` 中 `Include` 单独的配置文件:

```
#Enabled Vhosts:
Include conf/vhosts/domainname1.dom
Include conf/vhosts/domainname2.dom
```

通过注释或取消注释可以单独启用或禁用一个虚拟主机。

基本的 vhost 文件：

```
/etc/httpd/conf/vhosts/domainname1.dom
<VirtualHost *:80>
    ServerAdmin webmaster@domainname1.dom
    DocumentRoot "/home/user/http/domainname1.dom"
    ServerName domainname1.dom
    ServerAlias domainname1.dom
    ErrorLog "/var/log/httpd/domainname1.dom-error_log"
    CustomLog "/var/log/httpd/domainname1.dom-access_log" common

    <Directory "/home/user/http/domainname1.dom">
```

```
        Require all granted
    </Directory>
</VirtualHost>

<VirtualHost *:443>
    ServerAdmin webmaster@domainname1.dom
    DocumentRoot "/home/user/http/domainname1.dom"
    ServerName domainname1.dom:443
    ServerAlias domainname1.dom:443
    SSLEngine on
    SSLCertificateFile "/etc/httpd/conf/apache.crt"
    SSLCertificateKeyFile "/etc/httpd/conf/apache.key"
    ErrorLog "/var/log/httpd/domainname1.dom-error_log"
    CustomLog "/var/log/httpd/domainname1.dom-access_log" common

    <Directory "/home/user/http/domainname1.dom">
        Require all granted
    </Directory>
</VirtualHost>
```

扩展

PHP

首先，参考 PHP 页面，完成 PHP 的安装。

有多种方式可以在 Apache 下使用 PHP，#使用 libphp 最简单，但是扩展性最差，libphp 还需要修改 mpm 模块，可能影响其它扩展，比如和 #HTTP2 不兼容。

使用 libphp

安装软件包 php-apache包。

php-apache包 中包含的 `libphp7.so` 不支持 `mod_mpm_event`，仅支持 `mod_mpm_prefork`(FS#39218)。需要在 `/etc/httpd/conf/httpd.conf` 中注释掉:

```
#LoadModule mpm_event_module modules/mod_mpm_event.so
```

取消下面行的注释:

```
LoadModule mpm_prefork_module modules/mod_mpm_prefork.so
```

不然将发生下面的错误:

```
Apache is running a threaded MPM, but your PHP Module is not compiled to be threadsafe.
You need to recompile PHP.
AH00013: Pre-configuration failed
httpd.service: control process exited, code=exited status=1
```

另外在本小节的下方还有两种处理高并发的方案供选择。 （使用php-fpm管理进程 和 使用mod_fcgid管理进程 ）

要启用 PHP，在 `/etc/httpd/conf/httpd.conf` 中添加如下行：

- 将这一行放在`LoadModule` 的末尾：

```
LoadModule php_module modules/libphp.so
AddHandler php-script php
```

- 将这一行放到`Include`列表的末尾：

```
Include conf/extra/php_module.conf
```

重启 `httpd.service`。

使用 php-fpm 和 mod_proxy_fcgi

这种方式是使用php-fpm来管理进程的，进程不是由apache启动和管理的.

注意：与使用ProxyPass的广泛设置不同，使用SetHandler的代理配置遵守Apache指令，例如DirectoryIndex。 这是为了确保与为libphp7、mod_fastcgi和mod_fcgid而设计的软件有更好的兼容性。 如果您仍然想尝试使用ProxyPass，请尝试使用如下所示的行：

```
ProxyPassMatch ^/(.*\.php(/.*)?)$ unix:/run/php-fpm/php-fpm.sock|
fcgi://localhost/srv/http/$1
```

安装官方软件包 php-fpm包 .

启用代理模块:

```
/etc/httpd/conf/httpd.conf
LoadModule proxy_module modules/mod_proxy.so
LoadModule proxy_fcgi_module modules/mod_proxy_fcgi.so
```

创建文件： `/etc/httpd/conf/extra/php-fpm.conf` 写入以下内容:

```
/etc/httpd/conf/extra/php-fpm.conf
DirectoryIndex index.php index.html
<FilesMatch \.php$>
    SetHandler "proxy:unix:/run/php-fpm/php-fpm.sock|fcgi://localhost/"
</FilesMatch>
```

把以下这句添加到配置文件 `/etc/httpd/conf/httpd.conf` 中 include 部份的最后

```
Include conf/extra/php-fpm.conf
```

注意：在 `sock` 和 `fcgi` 中间的管道符两边不要有空格! `localhost` 可以替换成任何的字符串. 详细请见 here

你可以自行配置 PHP-FPM 通过这个编辑这个配置文件 `/etc/php/php-fpm.d/www.conf`, 但是默认的配置已经工作的很好了.

重启 `httpd.service` 和 `php-fpm.service` 这两个服务.

注意：如果之前在 `httpd.conf` 加入了下面内容，请删除它们，已经不再需要：

```
LoadModule php7_module modules/libphp7.so
Include conf/extra/php7_module.conf
```

使用 apache2-mpm-worker 和 mod_fcgid

这种方式和上一种方式(php-fpm)的区别：

```
   php-fgi进程是由apache模块启动并管理，而不需要配置和使用php-fpm来管理进程。
   在php-cgi进程以apache用户身份运行，php程序写的文件，其权限为apache用户（而不像php-fpm下写文件为php-fpm用户所有，默认是nobody），这样在目录权限管理方面一致性高些。
```

安装软件包 mod_fcgidAUR(详情)和 php-cgi包。

创建需要的目录并建立软链接:

```
# mkdir /srv/http/fcgid-bin
# ln -s /usr/bin/php-cgi /srv/http/fcgid-bin/php-fcgid-wrapper
```

创建 `/etc/httpd/conf/extra/php-fcgid.conf`，内容是:

```
/etc/httpd/conf/extra/php-fcgid.conf
# Required modules: fcgid_module

<IfModule fcgid_module>
    AddHandler php-fcgid .php
    AddType application/x-httpd-php .php
    Action php-fcgid /fcgid-bin/php-fcgid-wrapper
    ScriptAlias /fcgid-bin/ /srv/http/fcgid-bin/
    SocketPath /var/run/httpd/fcgidsock
    SharememPath /var/run/httpd/fcgid_shm
        # If you don't allow bigger requests many applications may fail (such as
WordPress login)
        FcgidMaxRequestLen 536870912
        # Path to php.ini - defaults to /etc/phpX/cgi
        DefaultInitEnv PHPRC=/etc/php/
        # Number of PHP childs that will be launched. Leave undefined to let PHP
decide.
        #DefaultInitEnv PHP_FCGI_CHILDREN 3
        # Maximum requests before a process is stopped and a new one is launched
        #DefaultInitEnv PHP_FCGI_MAX_REQUESTS 5000
    <Location /fcgid-bin/>
        SetHandler fcgid-script
        Options +ExecCGI
    </Location>
</IfModule>
```

编辑 `/etc/httpd/conf/httpd.conf`，启用 actions 模块：

```
LoadModule actions_module modules/mod_actions.so
```

并添加如下配置:

```
LoadModule fcgid_module modules/mod_fcgid.so
Include conf/extra/httpd-mpm.conf
Include conf/extra/php-fcgid.conf
```

Restart `httpd.service`.

测试 PHP

在 apache 文档根目录（即`/srv/http/`或`~public_html`）中创建test.php文件，在其中写入：

```
<?php phpinfo(); ?>
```

然后访问： http://localhost/test.php 或者 http://localhost/~myname/test.php

高级的配置和扩展，请设置 PHP.

HTTP2

要启用 http2，安装 libnghttp2包 软件包(属于core仓库,一般默认已经安装)。然后取消 `httpd.conf` 中下面行前的注释:

```
LoadModule http2_module modules/mod_http2.so
```

并加入:

```
Protocols h2 http/1.1
```

更多信息请参考 mod_http2 文档。

问题处理

Apache 的状态和日志

状态信息可以用 systemctl 查询。

Apache 默认的系统日志位于 `/var/log/httpd/`。

启动后出现 Error: PID file /run/httpd/httpd.pid not readable

在 `httpd.conf` 中注释掉 `unique_id_module` 行：

```
#LoadModule unique_id_module modules/mod_unique_id.so
```

AH00534: httpd: Configuration error: No MPM loaded.

最近的升级需要修改 `httpd.conf` 配置文件，取消下面行前的注释：

```
/etc/httpd/conf/httpd.conf
LoadModule mpm_prefork_module modules/mod_mpm_prefork.so
```

AH00072: make_sock: could not bind to address

多种都可能导致此问题，最常见的问题是已经有程序监听了设置的端口，通过下面命令确认：

```
# netstat -lnp | grep -e :80 -e :443
```

如该能查到结果，关闭占用端口的程序，然后重试。

还有一个原因是 Apache 没有以 root 执行，运行下面命令看看问题是否依然发生：

```
# httpd -k start
```

最后，可能配置有问题，导致程序同时监听了端口两次，例如下面的配置就有这个问题：

```
Listen 0.0.0.0:80
Listen [::]:80
```

php.ini 中的 max_execution_time 设置无效

`php.ini` 中的 `max_execution_time` 设置为大于 30 (秒), 还会受到 `503 Service Unavailable` 的话，还需要添加 `ProxyTimeout` 到 `<FilesMatch \.php$>` 段落之前:

```
/etc/httpd/conf/httpd.conf
ProxyTimeout 300
```

重启 `httpd.service`.

参阅

- Apache 官方网站
- Apache wiki
- 生成ssh_test_certificate的教程
- Apache故障排除Wiki

7.7 Nginx

难度 高级 | **篇幅** 39.4 KB | **所属** 网络服务器

本节将讲到

- 安装
- 启动
- 配置
- 在chroot环境下安装
- 建议和技巧

Nginx (读作"engine X") 由Igor Sysoev(俄罗斯)于2005年编写，是一个免费、开源、高性能的HTTP web服务器和反向代理，也可以作为一个IMAP/POP3代理服务器。Nginx因为稳定，丰富的功能集，配置简单，资源占用低而闻名世界。

这篇文章描述了如何设置nginx并且如何通过#FastCGI集成PHP.

安装

安装以下其中一个软件包：

- nginx-mainline包 - 主线分支：新功能、更新、错误修复。
- nginx包 - 稳定分支：仅重大错误修复。
- angieAUR - nginx 的分支和直接替代品，具有更多功能。
- freenginx-mainlineAUR - 保留 nginx 自由和开放开发的直接替代品（主线分支）。
- freenginx-libresslAUR - 保留 nginx 自由和开放开发的直接替代品（主线分支，支持 LibreSSL）。
- freenginxAUR - 保留 nginx 自由和开放开发的直接替代品（稳定分支）。

推荐使用主线分支。使用稳定分支的主要原因是担心新功能可能带来的不良影响，例如与第三方模块不兼容或开发者在引入新功能时疏忽导致的错误。

注意：所有在official repositories可用的模块都需要*nginx*（与*nginx-mainline*相反）作为依赖。你可能需要在下决定选*nginx*还是*nginx-mainline*前，先查看下模块名单。*nginx-mainline*的模块能在Arch User Repository找到。

对于在基于chroot环境下的额外安全性，可查阅#在chroot环境下安装。

启动

启动/启用 `nginx.service` 或 `angie.service`（如果你使用 Angie）。

默认在 http://127.0.0.1 页面服务的页面
是 `/usr/share/nginx/html/index.html`。

配置

安装nginx后的第一步该干什么写在用户手册里了。你可以通过编辑
在`/etc/nginx/`下的文件来修改配置。主配置文件在`/etc/nginx/nginx.conf`。

更多细节和例子，你可以在官方文档找到。

下面的例子包含了最常见的使用案例。我们假定你使用的是默认文件路径
(`/usr/share/nginx/html`)。如果你改了路径，用你自己的路径替代。

提示：DigitalOcean 提供了一个 Nginx 配置工具。

配置例子

```
/etc/nginx/nginx.conf
user http;
worker_processes auto;
worker_cpu_affinity auto;

events {
    worker_connections 1024;
}

http {
    charset utf-8;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    server_tokens off;
    log_not_found off;
    types_hash_max_size 4096;
    client_max_body_size 16M;

    # MIME
    include mime.types;
    default_type application/octet-stream;

    # logging
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log warn;

    # load configs
    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

通用配置

进程和连接

你应该为`worker_processes`选一个合适的值。这项设置最终决定了nginx接受多少连接和它会使用多少处理器。通常来说，把它设置成你系统里的硬件线程数就好了。可选的是，自从 1.3.8 和 1.2.5版本以后， `worker_processes` 接受 `auto` 作为值了，它会自动检测最优值 (source)。

nginx的最大连接数有下面的公式给出`max_clients = worker_processes * worker_connections`。

不同用户间的使用

默认的，nginx包 用 `root` 身份运行主进程而用`http`用户运行worker进程。如果要用其它用户运行worker进程，改变`nginx.conf`里的`user`参数就好了：

```
/etc/nginx/nginx.conf
user user [group];
```

如果组（group）忽略不写的话，就会用和*user*相同的名字来代替。

提示：其实也可以使用systemd以`root`身份在没有任何东西运行的情况下运行nginx。可以看 #使用systemd无权限运行 和 #使用 systemd 用户服务运行。

server代码块

通过使用 `server` 模块，可以实现服务多个域名。这些模块可以类比为Apache HTTP Server中的"VirtualHosts"。也可查阅上游文献。

下面的例子，服务器监听IPv4和IPv6的80端口的进入流量的两个域名，分别是`domainname1.tld` 和`domainname2.tld`：

```
/etc/nginx/nginx.conf
...
server {
    listen 80;
    listen [::]:80;
    server_name domainname1.tld;
    root /usr/share/nginx/domainname1.tld/html;
    location / {
        index index.php index.html index.htm;
    }
}

server {
    listen 80;
    listen [::]:80;
    server_name domainname2.tld;
    root /usr/share/nginx/domainname2.tld/html;
    ...
}
```

Restart(重启) `nginx.service` 来让改变的配置生效。

确保主机名是可以被设置好的DNS服务器比如BIND 或者 dnsmasq解析的，或者可以查看下网络配置#Local network hostname resolution。

管理服务器入口

把不同的`server`模块放到不同的文件里是可能的。这样的话可以让你很轻易的开启和禁用特定的站点。

**本文或本章节的事实准确性存在争议。**

原因： 有人认为使用 `sites-enabled` 和 `sites-available` 的方法已经不再有用，反而会带来更多问题，可以参考 比较这两种方法 和 使用这种方法可能引发的问题。

相反，可以直接在 `/etc/nginx/conf.d/` 目录下创建文件，这种方式符合标准的配置文件放置方式。然后在主配置文件中加入 `include /etc/nginx/conf.d/*.conf`，类似于包含其他目录中的文件模式。这样，可以通过将文件重命名为 `original_name.conf.disabled` 来禁用站点，因为只有以 *.conf* 结尾的文件才会被包含。

（在 Talk:Nginx 中讨论）

如果仍然想使用 `sites-enabled` 和 `sites-available` 的方法，可以创建以下文件夹：

```
# mkdir /etc/nginx/sites-available
# mkdir /etc/nginx/sites-enabled
```

在`sites-available`文件夹下创建一个文件包含一个或更多服务器模块：

```
/etc/nginx/sites-available/example.conf
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;

    ...
}
```

把 `include sites-enabled/*;`放到`http` 模块的末尾：

```
/etc/nginx/nginx.conf
http {
    ...
    include sites-enabled/*;
}
```

如果要启用一个网站，只需要简单的创建一个符号链接：

```
# ln -s /etc/nginx/sites-available/example.conf /etc/nginx/sites-enabled/example.conf
```

如果要移除一个网站：

```
# unlink /etc/nginx/sites-enabled/example.conf
```

Reload/restart `nginx.service` 来让配置生效。

TLS

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：不要仅仅把 OpenSSL#Usage 的配置搬过来。（在Talk:Nginx讨论）

OpenSSL 提供了TLS支持并且默认在Arch系统安装时安装了。

提示：

- 你可能需要在配置SSL之前阅读 ngx_http_ssl_module 文档。
- Let's Encrypt 是一个免费的、自动的、开放的证书颁发机构。有一个插件可以从命令行请求有效的证书并自动配置。
- Mozilla有一篇有用的 TLS文章，也有一个自动化工具来创建一个更安全的配置。

警告：如果你打算实现TLS，你必须知道一些变化和实现 仍然被攻击时很脆弱。如果想知道TLS内现存的漏洞和怎样用合适的方法到nginx上，可访问 https://weakdh.org/sysadmin.html

创建一个私钥和自签名的证书，这对大多数不需要 CSR的安装来说足够了：

```
# mkdir /etc/nginx/ssl
# cd /etc/nginx/ssl
# openssl req -new -x509 -nodes -newkey rsa:4096 -keyout server.key -out server.crt -
days 1095
# chmod 400 server.key
# chmod 444 server.crt
```

注意：`-days` 是可选的并且RSA键大小最低可到2048 (默认值)。

如果你需要创建一个CSR，用下面的教程而不是上面的：

```
# mkdir /etc/nginx/ssl
# cd /etc/nginx/ssl
# openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out server.key
# chmod 400 server.key
# openssl req -new -sha256 -key server.key -out server.csr
# openssl x509 -req -days 1095 -in server.csr -signkey server.key -out server.crt
```

注意：如果你需要更多的 *openssl*选项，可以阅读 openssl(1ssl) 或扩展文档。

使用TLS的 `/etc/nginx/nginx.conf` 的基本例子可以参考 Mozilla的SSL配置生成器。

Restart `nginx.service` 来让配置生效。

分用户目录

如果要复制Apache式的 `~user` URLs 到用户的 `~/public_html` 目录，可以尝试下面的。(注意：如果下面的两个规则都用了的话一定要把更清楚地PHP规则放在前面。)

```
/etc/nginx/nginx.conf
...
server {
    ...
    # PHP in user directories, e.g. http://example.com/~user/test.php
    location ~ ^/~(.+?)(/.+\.php)$ {
        alias          /home/$1/public_html$2;
        fastcgi_pass   unix:/run/php-fpm/php-fpm.sock;
        fastcgi_index  index.php;
        include        fastcgi.conf;
    }

    # User directories, e.g. http://example.com/~user/
    location ~ ^/~(.+?)(/.*)?$ {
        alias     /home/$1/public_html$2;
        index     index.html index.htm;
        autoindex on;
    }
    ...
}
...
```

查阅 #PHP实现来阅读更多 `nginx`的PHP配置。

Restart `nginx.service` 来启用新配置。

FastCGI

FastCGI, 也叫 FCGI, 是适用于web服务器和交互式程序的接口的协议. FastCGI 是早期的 Common Gateway Interface (CGI)的变种; FastCGI的主要目的是减少CGI程序和web服务器交互的开销,来允许服务器同时处理更多网页请求.

FastCGI技术被引进nginx是为了与许多外部工具，比如. Perl, PHP 和 Python

PHP实现

PHP-FPM 是建议使用的用作PHP的FastCGI服务器的解决方案.

安装 php-fpm包 然后确保 PHP 被安装配置正确. PHP-FPM的主要配置文件是 `/etc/php/php-fpm.conf`. 基础使用的话默认配置就足够了.

最后, 启动/启用 `php-fpm.service`.

你也可以使用 php-legacy-fpm包 代替， 参见 #使用 php-legacy.

注意：

- 如果你 用不同的用户运行nginx, 确保PHP-FPM套接字文件能被这个用户访问，或者通过一个TCP套接字.
- 如果你在chroot的环境下运行nginx (chroot 是 `/srv/nginx-jail`, web页面在 `/srv/nginx-jail/www`), 你必须修改文件 `/etc/php/php-fpm.conf`， 把 `chroot = /srv/nginx-jail` 和 `listen = /srv/nginx-jail/run/php-fpm/php-fpm.sock` 两行加入到指令部分 (默认是`[www]`). 如果缺少的话请为套接字文件创建目录。此外, 动态链接到依赖的模块，你需要将这些依赖复制到(比如. 对于php-imagick, 你需要复制ImageMagic的库复制到chroot, 而不是imagick.so他自己).

nginx 配置

当服务一个PHP web程序时，一个PHP-FPM的`location`应该包括在 #server代码块 里 ,比如.:

```
/etc/nginx/sites-available/example.conf
server {
    root /usr/share/nginx/html;

    location / {
        index index.html index.htm index.php;
    }

    location ~ \.php$ {
        # 404
        try_files $fastcgi_script_name =404;

        # default fastcgi_params
        include fastcgi_params;

        # fastcgi settings
        fastcgi_pass                unix:/run/php-fpm/php-fpm.sock;
        fastcgi_index                       index.php;
        fastcgi_buffers                     8 16k;
```

```
        fastcgi_buffer_size         32k;

        # fastcgi params
        fastcgi_param DOCUMENT_ROOT       $realpath_root;
        fastcgi_param SCRIPT_FILENAME     $realpath_root$fastcgi_script_name;
        #fastcgi_param PHP_ADMIN_VALUE    "open_basedir=$base/:/usr/lib/php/:/tmp/";
    }
}
```

如果需要用PHP处理其他文件扩展名 (比如. *.html* and *.htm*):

```
location ~ [^/]\.(php|html|htm)(/|$) {
    ...
}
```

非 *.php* 的PHP-FPM扩展处理都需要被直接加入到 `/etc/php/php-fpm.d/www.conf`:

```
security.limit_extensions = .php .html .htm
```

注意：需要注意下 `fastcgi_pass` 参数, 因为它被选定的FastCGI服务器在它的配置文件里定义TCP或Unix套接字. 对应`php-fpm`的**default** (Unix) 套接字是 是:

```
fastcgi_pass unix:/run/php-fpm/php-fpm.sock;
```

你可能想要使用常见的 TCP 套接字, **not default**,

```
fastcgi_pass 127.0.0.1:9000;
```

Unix 的域名套接字应该会快一点.

提示：如果你使用多个 `server` 块来启用PHP支持, 创建一个 `php_fastcgi.conf` 配置文件可能更简单:

```
/etc/nginx/php_fastcgi.conf
location ~ \.php$ {
# 404
try_files $fastcgi_script_name =404;
# default fastcgi_params
include fastcgi_params;
# fastcgi settings
...
}
```

为特定的服务器启用PHP支持, 只需要简单的加入 `include php.conf`就好了:

```
/etc/nginx/sites-available/example.conf
server {
server_name example.com;
...

include /etc/nginx/php_fastcgi.conf;
}
```

测试配置

你需要restart `php-fpm.service` 和 `nginx.service` 服务单元来让配置生效，如果配置之前被改变过的话.

测试FastCGI实现, 在`root`目录下创建一个新的PHP文件，内容是:

```
<?php phpinfo(); ?>
```

把这个文件用浏览器打开，你应该就可以看到现在的PHP配置的信息页了.

CGI 实现

这个实现是CGI程序所需要的.

fcgiwrap

安装 fcgiwrap包. 配置文件是 `/usr/lib/systemd/system/fcgiwrap.socket`. 启用并且启动 `fcgiwrap.socket`.

多个工人线程

如果你想生成多个工人线程建议你使用multiwatchAUR, 它会处理崩溃的子进程. 如果你需要使用`spawn-fcgi` 来创建Unix套接字,因为multiwatch无法处理systemd创建的套接字，尽管fcgiwarp它自己在单元文件中直接调用页没有任何问题.

覆盖 `fcgiwrap.service` (和`fcgiwrap.socket` 单元, 如果存在的话), 修改`ExecStart`行到满足你需要的地方. 下面是使用multiwatchAUR的单元文件. 确保`fcgiwrap.socket` 没有被start（开始）或enable（启用）, 因为他会和这些单元冲突:

```
/etc/systemd/system/fcgiwrap.service
[Unit]
Description=Simple CGI Server
After=nss-user-lookup.target

[Service]
ExecStartPre=/bin/rm -f /run/fcgiwrap.socket
ExecStart=/usr/bin/spawn-fcgi -u http -g http -s /run/fcgiwrap.sock -n
-- /usr/bin/multiwatch -f 10 -- /usr/sbin/fcgiwrap
ExecStartPost=/usr/bin/chmod 660 /run/fcgiwrap.sock
PrivateTmp=true
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

可以自定义 `-f 10` 来改变生成的子进程的数量.

警告：`ExecStartPost`行是需要的，因为当为`spawn-fcgi`使用`-M 660`参数选项时会出现奇怪的情况. 这有可能是一个Bug

nginx配置

在每个服务了一个CGI程序的`server` 块内都应该有一个像下面的 `location` 块:

```
location ~ \.cgi$ {
     root           /path/to/server/cgi-bin;
     fastcgi_pass   unix:/run/fcgiwrap.sock;
     include        fastcgi.conf;
}
```

`fcgiwrap` 的默认套接字文件是 `/run/fcgiwrap.sock`.

警告：如果使用了 SCRIPT_NAME 和 DOCUMENT_ROOT，fcgiwrap 会*丢弃* nginx 中设置的其他 fastcgi_params。你必须使用 SCRIPT_FILENAME 才能通过 Nginx 配置设置其他参数（如 PATH_INFO）。参见 这个 GitHub 问题。

如果你一直出现 `502 - bad Gateway` 错误,你应该检查你的CGI程序是否宣告了下面内容的mime类型。如果是html，他应该是`Content-type: text/html`.

如果你遇到 403 错误，请确保 CGI 可执行文件对 `http` 用户可读且可执行，并且每个父文件夹对 `http` 用户可读。

在chroot环境下安装

**本文或本章节的事实准确性存在争议。**

原因： 本节内容来自2013年。自那时起，systemd已被引入，可以更高效且无需太多麻烦地替代此方法。（在 Talk:Nginx 中讨论）

在chroot环境里运行nginx是有一个额外的安全层的。为了最大化安全性，chroot环境应该只包括nginx服务器需要的文件，并且所有的文件都应该尽可能有严格的权限，比如像`/usr/bin`这样的文件夹应该不可读和不可写。

Arch默认有一个`http`用户和组来运行服务器，chroot将会在`/srv/http`。

创建这个监狱（指chroot环境）的PERL脚本能在jail.pl gist找到。你可以直接用那个脚本或者跟着这篇文章的指示继续阅读下去。它应该用root运行。你需要在它起作用之前取消一行注释。

创建必要的设备

nginx需要 `/dev/null`, `/dev/random`, 和 `/dev/urandom`. 为了在chroot环境中安装这些设备，需要创建 `/dev/` 目录并使用*mknod*添加设备。尽量避免挂载整个`/dev/`目录，以确保即使chroot被攻破，攻击者也必须突破chroot才能访问像`/dev/sda1`这样的重要设备。

提示：

- 确保 `/srv/http` 挂载时没有使用nodev选项
- 查看 mknod(1) 和 `ls -l /dev/{null,random,urandom}` 来更好地理解*mknod*选项。

```
# export JAIL=/srv/http
# mkdir $JAIL/dev
# mknod -m 0666 $JAIL/dev/null c 1 3
# mknod -m 0666 $JAIL/dev/random c 1 8
# mknod -m 0444 $JAIL/dev/urandom c 1 9
```

创建必要目录

nginx需要大量文件来运转正常。在把它们拷贝过去之前创建一个文件夹来存储它们。假设你的nginx文档目录在 `/srv/http/www`。

```
# mkdir -p $JAIL/etc/nginx/logs
# mkdir -p $JAIL/usr/{lib,bin}
# mkdir -p $JAIL/usr/share/nginx
# mkdir -p $JAIL/var/{log,lib}/nginx
```

```
# mkdir -p $JAIL/www/cgi-bin
# mkdir -p $JAIL/{run,tmp}
# cd $JAIL; ln -s usr/lib lib
# cd $JAIL; ln -s usr/lib lib64
# cd $JAIL/usr; ln -s lib lib64
```

然后挂载 `$JAIL/tmp` 和 `$JAIL/run` 作为tmpfs。大小应该限制确保攻击者无法吃掉所有的内存。

```
# mount -t tmpfs none $JAIL/run -o 'noexec,size=1M'
# mount -t tmpfs none $JAIL/tmp -o 'noexec,size=100M'
```

为了在重启后保留挂载，下面的入口应该被添加到`/etc/fstab`:

```
/etc/fstab
tmpfs   /srv/http/run   tmpfs   rw,noexec,relatime,size=1024k   0       0
tmpfs   /srv/http/tmp   tmpfs   rw,noexec,relatime,size=102400k 0       0
```

填充chroot

首先复制简单的文件。

```
# cp -r /usr/share/nginx/* $JAIL/usr/share/nginx
# cp -r /usr/share/nginx/html/* $JAIL/www
# cp /usr/bin/nginx $JAIL/usr/bin/
# cp -r /var/lib/nginx $JAIL/var/lib/nginx
```

然后复制必要的库。使用*ldd*来列出它们并将它们复制到正确的位置。复制优于硬链接，以确保即使攻击者获得文件的写权限，也无法破坏或修改真实的系统文件。

```
$ ldd /usr/bin/nginx
linux-vdso.so.1 (0x00007fffc41fe000)
libpthread.so.0 => /usr/lib/libpthread.so.0 (0x00007f57ec3e8000)
libcrypt.so.1 => /usr/lib/libcrypt.so.1 (0x00007f57ec1b1000)
libstdc++.so.6 => /usr/lib/libstdc++.so.6 (0x00007f57ebead000)
libm.so.6 => /usr/lib/libm.so.6 (0x00007f57ebbaf000)
libpcre.so.1 => /usr/lib/libpcre.so.1 (0x00007f57eb94c000)
libssl.so.1.0.0 => /usr/lib/libssl.so.1.0.0 (0x00007f57eb6e0000)
libcrypto.so.1.0.0 => /usr/lib/libcrypto.so.1.0.0 (0x00007f57eb2d6000)
libdl.so.2 => /usr/lib/libdl.so.2 (0x00007f57eb0d2000)
libz.so.1 => /usr/lib/libz.so.1 (0x00007f57eaebc000)
libGeoIP.so.1 => /usr/lib/libGeoIP.so.1 (0x00007f57eac8d000)
libgcc_s.so.1 => /usr/lib/libgcc_s.so.1 (0x00007f57eaa77000)
libc.so.6 => /usr/lib/libc.so.6 (0x00007f57ea6ca000)
/lib64/ld-linux-x86-64.so.2 (0x00007f57ec604000)
```

对于 `/usr/lib` 中的文件，你可以使用以下命令：

```
# cp $(ldd /usr/bin/nginx | grep /usr/lib/ | sed -sre 's/(.+)(\/usr\/lib\/\S+).+/\2/g')
$JAIL/usr/lib
```

使用以下命令复制 `ld-linux-x86-64.so`：

```
# cp /lib64/ld-linux-x86-64.so.2 $JAIL/lib
```

注意：不要尝试复制 `linux-vdso.so`：它不是真实存在的库，不在 `/usr/lib` 文件夹中。

复制一些复杂但必要的库和系统文件。

```
# cp /usr/lib/libnss_* $JAIL/usr/lib
# cp -rfvL /etc/
{services,localtime,nsswitch.conf,nscd.conf,protocols,hosts,ld.so.cache,ld.so.conf,reso
lv.conf,host.conf,nginx} $JAIL/etc
```

为chroot创建严格的用户/组文件。这样只有chroot正常工作所需的用户才会被chroot知道，其他用户/组不会泄露给攻击者，即使他们已经获取了chroot权限。

```
$JAIL/etc/group
```

```
http:x:33:
nobody:x:99:
$JAIL/etc/passwd
http:x:33:33:http:/:/bin/false
nobody:x:99:99:nobody:/:/bin/false
$JAIL/etc/shadow
http:x:14871::::::
nobody:x:14871::::::
$JAIL/etc/gshadow
http:::
nobody:::
# touch $JAIL/etc/shells
# touch $JAIL/run/nginx.pid
```

最后，设置非常严格的权限。尽可能多的文件应归root所有并设置为不可写。

```
# chown -R root:root $JAIL/

# chown -R http:http $JAIL/www
# chown -R http:http $JAIL/etc/nginx
# chown -R http:http $JAIL/var/{log,lib}/nginx
# chown http:http $JAIL/run/nginx.pid

# find $JAIL/ -gid 0 -uid 0 -type d -print | xargs chmod -rw
# find $JAIL/ -gid 0 -uid 0 -type d -print | xargs chmod +x
# find $JAIL/etc -gid 0 -uid 0 -type f -print | xargs chmod -x
# find $JAIL/usr/bin -type f -print | xargs chmod ug+rx
# find $JAIL/ -group http -user http -print | xargs chmod o-rwx
# chmod +rw $JAIL/tmp
# chmod +rw $JAIL/run
```

如果你的端口绑定为80（或其他在\[1-1023\]范围内的端口），给chroot无需root即可绑定的权限。

```
# setcap 'cap_net_bind_service=+ep' $JAIL/usr/bin/nginx
```

修改nginx.service来开启chroot

覆盖单元文件 `nginx.service`。升级nginx不会修改你自定义的 *.service* 文件。

systemd单元必须在开启chroot里的nginx之前修改，以http用户的身份，将pid文件放到chroot里。

注意：我不确定pid文件是否需要被放到chroot里.

```
/etc/systemd/system/nginx.service
[Unit]
Description=A high performance web server and a reverse proxy server
After=network.target

[Service]
Type=forking
PIDFile=/srv/http/run/nginx.pid
ExecStartPre=/usr/bin/chroot --userspec=http:http /srv/http /usr/bin/nginx -t -q -g
'pid /run/nginx.pid; daemon on; master_process on;'
ExecStart=/usr/bin/chroot --userspec=http:http /srv/http /usr/bin/nginx -g
'pid /run/nginx.pid; daemon on; master_process on;'
ExecReload=/usr/bin/chroot --userspec=http:http /srv/http /usr/bin/nginx -g
'pid /run/nginx.pid; daemon on; master_process on;' -s reload
ExecStop=/usr/bin/chroot --userspec=http:http /srv/http /usr/bin/nginx -g
'pid /run/nginx.pid;' -s quit

[Install]
WantedBy=multi-user.target
```

注意：用pacman升级nginx并不会升级chroot里的nginx。如果你要升级的话可能需要手动重复上面的步骤。不要忘记更新它链接的库.

你现在可以安全的写在非chroot环境下的nginx安装.

```
# pacman -Rsc nginx
```

如果你不移除非chroot环境的nginx安装，你可能想要确认运行的nginx是不是chroot环境里的, 你可以查询 `/proc/*PID*/root` 链接到哪里. 如果链接到 `/srv/http` 而不是/那运行的就是chroot环境下的nginx.

```
# ps -C nginx | awk '{print $1}' | sed 1d | while read -r PID; do ls -l /proc/
$PID/root; done
```

建议和技巧

使用 systemd 以非特权用户身份运行

使用 drop-in unit file 为 `nginx.service` 设置 `User` 和可选的 `Group` 选项，放在 `[Service]` 下：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
User=user
Group=group
```

我们可以通过以下方式加固服务，防止其提升权限：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
...
NoNewPrivileges=yes
```

提示：更多限制选项请参阅 systemd.exec(5)。

然后我们需要确保 `*user*` 拥有访问所需资源的权限。按照以下子章节进行配置，然后启动 nginx。

提示：相同的设置可能也适用于你的 FastCGI 服务器。

端口

默认情况下，Linux 不允许非 `root` 进程绑定到 1024 以下的端口。可以使用 1024 以上的端口：

```
/etc/nginx/nginx.conf
server {
        listen 8080;
}
```

提示：如果你希望 nginx 在端口 80 或 443 上可访问，可以配置你的 防火墙 将请求从 80 或 443 重定向到 nginx 监听的端口。

或者你可以授予 nginx 进程 CAP_NET_BIND_SERVICE 能力，使其能够绑定到 1024 以下的端口：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
```

```
...
CapabilityBoundingSet=
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=
AmbientCapabilities=CAP_NET_BIND_SERVICE
```

另外，你可以使用 systemd 套接字激活。在这种情况下，systemd 将监听端口，并在建立连接时启动 nginx，并将套接字作为文件描述符传递。这意味着 nginx 不需要特殊权限，因为套接字在启动时已经存在。这依赖于 nginx 用于传递套接字的内部环境变量 ，因此不受官方支持。与其设置 `CapabilityBoundingSet` 和 `AmbientCapabilities`，不如编辑服务覆盖文件，设置 `NGINX` 环境变量，告诉 nginx 套接字将作为哪些文件描述符传递：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
...
Environment=NGINX=3:4;
```

每个监听端口将有一个套接字，从文件描述符 3 开始，因此在此示例中，我们告诉 nginx 期望两个套接字。现在创建一个 `nginx.socket` 单元，指定要监听的端口：

```
/etc/systemd/system/nginx.socket
[Socket]
ListenStream=0.0.0.0:80
ListenStream=0.0.0.0:443
After=network.target
Requires=network.target

[Install]
WantedBy=sockets.target
```

套接字将按照此单元中定义的顺序传递，因此端口 80 将是文件描述符 3，端口 443 将是文件描述符 4。如果你之前启用或启动了服务，现在应该停止它，并启用 `nginx.socket`。当系统启动时，nginx 将不会运行，但在你通过浏览器访问网站时启动。通过这种方式，你可以进一步加固服务；例如，在许多情况下，你现在可以在服务文件中设置 `PrivateNetwork=True`，阻止 nginx 访问外部网络，因为 systemd 创建的套接字足以通过它提供网站服务。请注意，这将在 nginx 服务的日志中打印一条警告：`2020/08/29 19:33:20 [notice] 254#254: using inherited sockets from "3:4;"`

PID 文件

nginx包 默认编译为使用 `/run/nginx.pid`，而 *user* 无法写入该文件。我们可以创建一个 *user* 可以写入的目录，并将 PID 文件放在那里。例如，可以使用 `RuntimeDirectory` (systemd.exec(5))。

编辑 `nginx.service` 以配置 PID 文件：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
...
RuntimeDirectory=nginx
PIDFile=/run/nginx/nginx.pid
ExecStart=
```

```
ExecStart=/usr/bin/nginx -g 'pid /run/nginx/nginx.pid; error_log stderr;'
ExecReload=
ExecReload=/usr/bin/nginx -s reload -g 'pid /run/nginx/nginx.pid; error_log stderr;'
```

/var/lib/nginx

nginx包 默认编译为将临时文件存储在 `/var/lib/nginx` 中。

提示：通过运行 `$ nginx -V` 查看所有编译时选项。

你可以通过例如使用 `StateDirectory` (systemd.exec(5)) 来授予 *user* 对该目录的写权限：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
...
StateDirectory=nginx
```

/var/log/nginx

nginx包 默认编译为将访问日志存储在 `/var/log/nginx` 中。

你可以通过例如使用 `LogsDirectory` (systemd.exec(5)) 来授予 *user* 对该目录的写权限：

```
/etc/systemd/system/nginx.service.d/user.conf
[Service]
...
LogsDirectory=nginx
```

使用 systemd 运行用户服务

如果你想运行一个完全由非特权用户控制和配置的服务器实例，请考虑使用 nginx-user-serviceAUR。

针对systemd的可选脚本

在纯systemd的情况下，你可以享受chroot + systemd的好处。 基于设置用户组和pid：

```
/etc/nginx/nginx.conf
user http;
pid /run/nginx.pid;
```

文件的绝对路径是`/srv/http/etc/nginx/nginx.conf`。

```
/etc/systemd/system/nginx.service
[Unit]
Description=nginx (Chroot)
After=network.target

[Service]
Type=forking
PIDFile=/srv/http/run/nginx.pid
RootDirectory=/srv/http
ExecStartPre=/usr/bin/nginx -t -c /etc/nginx/nginx.conf
ExecStart=/usr/bin/nginx -c /etc/nginx/nginx.conf
ExecReload=/usr/bin/nginx -c /etc/nginx/nginx.conf -s reload
ExecStop=/usr/bin/nginx -c /etc/nginx/nginx.conf -s stop

[Install]
WantedBy=multi-user.target
```

没必要设置默认路径，nginx默认加载 `-c /etc/nginx/nginx.conf`，但这仍然是一个好主意。

作为可选项，你可以在chroot环境下**仅仅**运行 `ExecStart`，`RootDirectoryStartOnly`参数要设置为`yes`（参见 systemd.service(5)），或者在挂载点生效前启用它，或者使用一个 systemd路径。

```
/etc/systemd/system/nginx.path
[Unit]
Description=nginx (Chroot) path
[Path]
PathExists=/srv/http/site/Public_html
[Install]
WantedBy=default.target
```

Enable（启用）创建的`nginx.path` 然后把 `/etc/systemd/system/nginx.service`里的`WantedBy=default.target` 变成 `WantedBy=nginx.path`。

单元文件里的`PID`文件允许systemd监控进程（需要绝对路径）。如果这不是你想要的，你可以改变默认的“一击”类型，然后删除单元文件里的参照。

Nginx Beautifier （nginx美化器）

nginxbeautifierAUR 是一个命令行工具，用来美化和格式化nginx配置文件。

更好的头文件管理

Nginx 有一个非常不直观的头文件管理系统，头文件只能在一个上下文中定义，任何其它的头文件都会被忽略。为了补救这个问题，我们可以安装 headers-more-nginx 模块。

安装 nginx-mod-headers-more包 包。这样会把模块安装到 `/usr/lib/nginx/modules` 目录下。

如果要加载模块，把下面这行加入到你的 nginx 配置文件的第一行。

```
/etc/nginx/nginx.conf
load_module "/usr/lib/nginx/modules/ngx_http_headers_more_filter_module.so";
...
```

基本认证

基本认证需要创建一个密码文件。密码文件可以使用 apache包 包提供的 `htpasswd` 程序进行管理，或者使用 nginx_passwdAUR 提供的 `nginx-passwd` 进行管理 - 详细信息可在 GitHub 源码上找到。

使用 php-legacy

安装 php-legacy-fpm包 而不是 php-fpm包，并确保 PHP 已正确安装和配置。

PHP-LEGACY-FPM 的主配置文件是 `/etc/php-legacy/php-fpm.conf`。对于基本使用，默认配置应该足够。

`fastcgi_pass` 参数的 Unix 套接字也需要调整，通常为：

```
fastcgi_pass unix:/run/php-fpm-legacy/php-fpm.sock;
```

然后启动/启用 `php-legacy-fpm.service`。

故障排除

配置验证

```
# nginx -t
 nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
 nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Error: The page you are looking for is temporarily unavailable. Please try again later. (502 Bad Gateway)

这是因为FastCGI服务器还没有开启，或者套接字有错误的权限。

试一试这个回答来解决502问题。

在Arch Linux，上面提到的配置文件在 `/etc/php/php-fpm.conf`。

Error: No input file specified

1\. 确定`/etc/php/php.ini`里的 `open_basedir`变量包含正确的路径，要和 `nginx.conf` (通常在`/usr/share/nginx/`)里的`root`变量一致. 当使用PHP-FPM作为PHP的FastCGI服务器时, 你可以添加 `fastcgi_param PHP_ADMIN_VALUE "open_basedir=$document_root/:/tmp/:/proc/";` 到`nginx.conf`里的 `location` 代码块里，它能处理PHP文件.

2\. 另一种情况是, `nginx.conf`文件里，`location ~ \.php$`代码块里的错误的 `root` 变量. 确保`root` 指向相同服务器里的`location /`的同样的目录. 或者你直接把root设为全局，不要把它定义到任何地方

3\. 检查权限: 比如. `http` 是用户/组, `755` 是目录权限，`644` 是文件权限. 记住，到 `html`目录的路径目录也必须有相同的权限. 可以查看 File permissions and attributes#Bulk chmod 来批量修改大量文件夹的权限.

4\. 你没有让 `SCRIPT_FILENAME` 包含你脚本的全部路径. 如果nginx的配置 (`fastcgi_param SCRIPT_FILENAME`) 是对的的话, 这种错误意味着PHP无法加载请求的脚本. 通常它知识简单的权限问题, 你可以用root运行php-cgi:

```
# spawn-fcgi -a 127.0.0.1 -p 9000 -f /usr/bin/php-cgi
```

或者你应该创建一个组和用户来开启php-cgi:

```
# groupadd www
```

```
# useradd -g www www
# chmod +w /srv/www/nginx/html
# chown -R www:www /srv/www/nginx/html
# spawn-fcgi -a 127.0.0.1 -p 9000 -u www -g www -f /usr/bin/php-cgi
```

5\. 如果你是在chroot的环境下的nginx运行php-fpm，确保`/etc/php-fpm/php-fpm.d/www.conf`里的`chroot` 设置正确(或者 `/etc/php-fpm/php-fpm.conf` 如果是旧版本的话)

Warning: Could not build optimal types_hash

当开启 `nginx.service`, 进程可能会有下面的日志信息:

```
[warn] 18872#18872: could not build optimal types_hash, you should increase either types_hash_max_size: 1024 or types_hash_bucket_size: 64; ignoring types_hash_bucket_size
```

解决这个警告, 提升`http`代码块的下面的键的值 :

```
/etc/nginx/nginx.conf
http {
    types_hash_max_size 4096;
    server_names_hash_bucket_size 128;
    ...
}
```

不能指定请求地址

来自`nginx.service` 单元状态的全部错误是

```
[emerg] 460#460: bind() to A.B.C.D:443 failed (99: Cannot assign requested address)
```

尽管，你的 nginx 单元文件配置为在 `network.target` 之后运行，nginx 可能仍然会尝试监听一个已配置但尚未添加到任何接口的地址。通过手动 start （启动）nginx 来验证这种情况（从而确保 IP 地址配置正确）。配置 nginx 监听任何地址都会解决这个问题。如果你的使用场景需要监听特定地址，一个可能的解决方案是重新配置 systemd。

在所有配置的网络设备启动并分配 IP 地址后启动 nginx，在 `nginx.service` 中将 `network-online.target` 附加到 `After=`，然后 start/enable `systemd-networkd-wait-online.service`。

参见

- WebDAV with nginx
- nginx configuration pitfalls
- Very good in-depth 2014 look at nginx security and Reverse Proxying
- Installing LEMP (nginx, PHP, MySQL with MariaDB engine and PhpMyAdmin) in Arch Linux
- Using SSL certificates generated with Let's Encrypt

第 8 章　共 6 节　138 KB

SSH

本章把登录、传输与挂载做成一条不必反复输入口令的通道：OpenSSH 讲客户端、服务端用法和排障，SSH 密钥讲密钥对、公钥分发与 SSH 代理，SSHFS 挂载远端目录，Mosh 管弱网漫游；另有协议背景，以及通过 SSH 远程安装 Arch Linux 的做法。

先读 OpenSSH 与 SSH 密钥，这是其余各节的前提；SSHFS、Mosh 与远程装机按需查阅，协议背景可略过。最容易踩的坑是把密钥当口令：生成后忘了把公钥送到远端，或私钥权限、代理没配好，便以为密钥失效；SSHFS 也常只试手动挂载、不试自动挂载与断线重连，重启后才发现目录空了。

8.1　Mosh

难度 入门 | **篇幅** 1.8 KB | **所属** SSH

本节将讲到

- 安装
- 用法

Mosh 是另一种交互式 SSH 终端。它支持漫游和本地回显。它还旨在提高对间歇性和高延迟连接的响应能力。请注意，真彩色支持仅在后者中提供。

安装

注意：Mosh 必须同时安装在服务器和客户端上。

安装 mosh包 包，或安装 mosh-gitAUR 以获得最新版本。

用法

注意：Mosh 设计不允许您访问会话历史记录，请考虑安装终端多路复用器，例如 tmux 或 GNU Screen。

要进行连接，请运行：

```
$ mosh user@server-address
```

发送 ssh 连接选项：

```
$ mosh --ssh="ssh -p 2222" user@server-address
```

您可以使用通常的 OpenSSH Client Configuration 来使选项永久化。

注意：Mosh 有一个未公开的命令行选项 `--predict=experimental`，它可以更积极地响应本地击键。 对键盘输入的低延迟视觉确认感兴趣的用户可能更喜欢此预测模式。

8.2　安全外壳协议

难度 入门 | **篇幅** 3.0 KB | **所属** SSH

本节将讲到

- 软件
- 保护
- 另请参见

根据维基百科：

: Secure Shell（安全外壳，简称 SSH）是一种加密的网络传输协议，可在不安全的网络中为网络服务提供安全的传输环境。SSH 最常见的用途是远程登录系统命令行界面和远程执行命令。

可以使用 SSH 的服务示例包括 Git，rsync 和 X11 转发。始终使用 SSH 的服务是 SCP 和 SFTP。

默认情况下，SSH 服务器侦听标准 TCP 端口 22。SSH 客户端程序通常用于建立与接受远程连接的 *sshd* 守护程序的连接。 两者都普遍存在于大多数现代操作系统中，包括 macOS，GNU/Linux，Solaris 和 OpenVMS。存在各种复杂程度和完整性的专有，免费软件和开源版本。

软件

- **Dropbear)** — 轻型 SSH 服务器。命令行 ssh 客户端名为 dbclient(1)。

: https://matt.ucc.asn.au/dropbear/dropbear.html || dropbear包

- **OpenSSH** — 用于通过 SSH 协议远程登录的高级连接工具

: https://www.openssh.com/portable.html || openssh包

仅服务端

- **TinySSH** — 一种简约的 SSH 服务器，仅实现 SSHv2 功能的一部分；只依赖 glibc。

: https://tinyssh.org/ || tinyssh包

仅客户端

- **PuTTY** — 集成终端的 SSH/Telnet 客户端

: https://www.chiark.greenend.org.uk/~sgtatham/putty/ || putty包

保护

请参阅安全#SSH。

另请参见

- 终端复用器（通常通过 SSH 使用）
- Wikipedia:Comparison of SSH clients
- Wikipedia:Comparison of SSH servers
- SSH 密钥指纹格式

8.3 通过 SSH 安装 Arch Linux

难度 进阶 | **篇幅** 8.4 KB | **所属** SSH

本节将讲到

- 在远程（目标）计算机上
- 在本地计算机上
- 在无头服务器上安装

本文介绍如何通过 SSH 连接远程安装 Arch。如果计算机位于远程位置或者希望在安装 Arch 时使用 SSH 客户端的复制/粘贴功能，则可以考虑这种方法。

在远程（目标）计算机上

注意：这些步骤需要物理访问计算机。如果计算机位于其他位置，则可能需要与其他人协调。

通过 Live CD/USB 镜像启动计算机进入 Live Arch 环境：用户将以 root 用户身份登录。

这时，按照安装指南#连接到互联网在目标计算机上设置网络。

为 SSH 连接设置 root 密码，因为 Arch 中 root 的默认密码为空：

```
# passwd
```

确认在 `/etc/ssh/sshd_config` 中设置了 `PermitRootLogin yes`。如果没有，请设置并重新加载OpenSSH 守护进程 `sshd.service` 使更改生效。

注意：如果目标计算机在 NAT 路由器后面，并且需要从外部访问， 则需要将 SSH 端口（默认为 22）转发到目标计算机的 LAN IP 地址。也可以设置一个到本地（客户端）机器的反向 SSH 隧道，并通过它连接，请参见 ssh(1) 的 `-R` 标志。

在本地计算机上

在本地计算机上，使用以下命令通过 SSH 连接到目标计算机：

```
$ ssh -o StrictHostKeyChecking=no -o "UserKnownHostsFile /dev/null" root@目标计算机的_IP_地址
```

注意：`-o StrictHostKeyChecking=no -o "UserKnownHostsFile /dev/null"` 选项将阻止验证 Live 环境的 SSH 主机密钥并将其写入到 `~/.ssh/known_hosts`。如果您以前连接过这一 IP 地址，这将避免 `REMOTE HOST IDENTIFICATION HAS CHANGED` 警告。

从这里可以看到 Live 环境的欢迎信息，并且能够像坐在物理键盘旁一样管理目标计算机。此时，如果只想安装 Arch，请遵循安装指南。如果要编辑现有的已损坏的 Linux 安装，参见从现有 Linux 发行版安装 Arch Linux。

提示：考虑在目标计算机上使用 GNU Screen 或 tmux（在 Live 环境中两者都可用）。这样，如果断开连接，还可以重新连接到多路复用器的会话。

在无头服务器上安装

注意：这些步骤需要物理访问远程计算机。需要有人插入安装介质并启动服务器。

本节介绍在没有键盘、鼠标和显示器的无头服务器上安装 Arch Linux。这种方法使用一个带有 cloud-init NoCloud 配置的额外驱动器来自动配置 OpenSSH 授权密钥和可选的 iwd 连接。

准备 cloud-init 配置文件

需要两个 cloud-init 配置文件：`meta-data` 和 `user-data`。

`meta-data` 文件可以为空：

```
$ printf "" > meta-data
```

`user-data` 将包含相关配置：

```
user-data
#cloud-config
users:
  - name: root
    ssh_authorized_keys:
      - ssh-ed25519
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
      - ssh-ed25519
YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
```

将 `ssh-ed25519 *XXXXXXXXXXXXXXXXXXXX

```
runcmd:
- systemctl restart iwd.service
```

创建了这两个文件之后，需要将其放置在一个标签为 `CIDATA` 的 ISO 9660 或 FAT 卷的驱动器上。

使用额外的 FAT 格式驱动器

使用 FAT 格式的驱动器。将 `meta-data` 和 `user-data` 复制到驱动器并将文件系统的 LABEL 更改为 `CIDATA`。

除了包含官方 ISO 的驱动器外，还需要将这个驱动器连接到无头计算机上。

使用额外的 ISO

使用 libisoburn包 中的 ***xorriso*** 创建一个 `cloud-init.iso` 文件：

```
$ xorrisofs -output cloud-init.iso -volid CIDATA -joliet -rational-rock meta-data user-data
```

将 cloud-init 数据介质刻录到光盘，如果部署选项允许的话，按原样使用 ISO。

使用单个 U 盘

如果将安装映像写入到了 U 盘之类的驱动器，并且驱动器上有足够的空间，就可以创建一个额外的分区来存储 cloud-init 数据。

安装 dosfstools包、mtools包 和 libisoburn包。

首先创建一个 LABEL 设置为 `CIDATA` 的 FAT 镜像：

```
$ mkfs.fat -C -n CIDATA cloud-init.img 2048
```

将 `meta-data` 和 `user-data` 文件复制到其根目录：

```
$ mcopy -i cloud-init.img meta-data user-data ::
```

重新打包官方 ISO，将 FAT 镜像作为第三个分区包括进来：

```
$ xorriso -indev archlinux-version-x86_64.iso -outdev archlinux-version-x86_64-with-cidata.iso -append_partition 3 0x0c cloud-init.img -boot_image any replay
```

最后按照 USB flash installation medium#Using the ISO as is (BIOS and UEFI)) 使用重新打包的 ISO（`archlinux-*version*-x86_64-with-cidata.iso`）准备 U 盘安装介质。

使用单个定制的 ISO

除了上述方法，也使用 Archiso 创建一个自定义 ISO。这样，无论类型如何，都可以只使用一个驱动器。

以 releng 配置文件作为基础。将 cloud-init 配置文件放在 `airootfs/var/lib/cloud/seed/nocloud/` 中，然后构建 ISO。

从安装介质启动

完成后，使用适当的方法将安装介质和 cloud-init 数据介质（如果是单独的）部署到无头计算机。

启动无头计算机，从安装介质启动到 Live Arch 环境。等待一分钟左右来让无头计算机启动并连接到网络。

从您现有的计算机（带键盘和显示器）通过 SSH 连接到无头服务器上的 Live Arch 环境，并按照安装指南完成安装。

注意：显然，启动镜像中的所有 Wi-Fi 和 SSH 配置都需要在实际的 Arch Linux 安装中再次配置，以便在安装后允许 WiFi SSH 访问无头计算机。

8.4 SSHFS

难度 进阶 | **篇幅** 12.6 KB | **所属** SSH

本节将讲到

- 安装
- 选项
- Chrooting
- Automounting
- Troubleshooting

翻译状态：

- 本文（或部分内容）译自 SSHFS，最近一次同步于 2023-03-09，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 部分内容未翻译（在 Talk:SSHFS# 中讨论）

SSHFS 是一个通过 SSH 挂载基于 FUSE 的文件系统的客户端程序。

注意：开发者已经停止开发此项目，rclone包 提供了相同的挂载功能。

安装

安装软件包 sshfs包。

提示：

- If you often need to mount sshfs filesystems you may be interested in using an sshfs helper, such as qsshfsAUR, sftpman, sshmntAUR or fmount.py.

- You may want to use Google Authenticator providing additional security as in two-step authentication.
- SSH keys may be used over traditional password authentication.

挂载

只能挂载 SSH 用户可以访问的目录，用 *sshfs* 挂载远程目录：

```
$ sshfs [user@]host:[dir] mountpoint [options]
```

示例:

```
$ sshfs myuser@mycomputer:/remote/path /local/path -C -p 9876
```

`-p 9876` 指定端口号，`-C` 启用压缩。

未指定是默认挂载远程用户的主目录，默认的用户名和密码可以通过 `~/.ssh/config` 按主机进行配置，这样可以简化 *sshfs* 命令。详情请参考 OpenSSH#Client usage。

需要密码时，SSH 会提示输入，如果不想一直手动输入，可以使用 SSH keys。

卸载

要卸载远程系统:

```
$ fusermount3 -u mountpoint
```

例如:

```
$ fusermount3 -u /local/path
```

选项

sshfs can automatically convert between local and remote user IDs. Use the `idmap=user` option to translate the UID of the connecting user to the remote user `myuser` (GID remains unchanged):

```
$ sshfs myuser@mycomputer:/remote/path /local/path -o idmap=user
```

If you need more control over UID and GID translation, look at the options `idmap=file`, `uidfile` and `gidfile`.

A complete list of options can be found in sshfs(1).

Chrooting

You may want to restrict a specific user to a specific directory on the remote system. This can be done by editing `/etc/ssh/sshd_config`:

```
/etc/ssh/sshd_config
.....
Match User someuser
       ChrootDirectory /chroot/%u
       ForceCommand internal-sftp
       AllowTcpForwarding no
       X11Forwarding no
.....
```

注意：The chroot directory **must** be owned by root, otherwise you will not be able to connect.

See also SFTP chroot. For more information check the sshd_config(5) man page for `Match`, `ChrootDirectory` and `ForceCommand`.

Automounting

Automounting can happen on boot, or on demand (when accessing the directory). For both, the setup happens in the fstab.

注意：Keep in mind that automounting is done through the root user, therefore you cannot use hosts configured in `.ssh/config` of your normal user.

To let the root user use an SSH key of a normal user, specify its full path in the `IdentityFile` option.

And most importantly, use each sshfs mount at least once manually **while root** on the client machine so the host's signature is added to the client's `/root/.ssh/known_hosts` file. This can also be done manually by appending one or more of the the SSH server's public host keys (the `/etc/ssh/ssh_host_*key.pub` files) to `/root/.ssh/known_hosts`.

On demand

With systemd on-demand mounting is possible using `/etc/fstab` entries.

Example:

```
user@host:/remote/path /local/path  fuse.sshfs noauto,x-systemd.automount,_netdev,users,idmap=user,IdentityFile=/home/user/.ssh/id_rsa,allow_other,reconnect 0 0
```

The important mount options here are *noauto,x-systemd.automount,_netdev*.

- *noauto* tells it not to mount at boot
- *x-systemd.automount* does the on-demand magic
- *_netdev* tells it that it is a network device, not a block device (without it "No such device" errors might happen)

注意：After editing `/etc/fstab`, reload the systemd configuration and the required services, which can be found by running `systemctl list-unit-files --type automount`

On boot

An example on how to use sshfs to mount a remote filesystem through `/etc/fstab`

```
user@host:/remote/path  /local/path  fuse.sshfs  defaults,_netdev  0  0
```

Take for example the *fstab* line

```
llib@192.168.1.200:/home/llib/FAH  /media/FAH2  fuse.sshfs  defaults,_netdev  0  0
```

The above will work automatically if you are using an SSH key for the user. See Using SSH Keys.

If you want to use sshfs with multiple users, add the following option:

```
user@domain.org:/home/user  /media/user  fuse.sshfs  defaults,allow_other,_netdev  0  0
```

In order to ensure the network is available before trying to mount, it is not only important to set the `_netdev` mount option, but also to add either `--any` or a specific `--interface` to the appropriate wait-online service for your network manager.

Secure user access

When automounting via fstab, the filesystem will generally be mounted by root. By default, this produces undesireable results if you wish access as an ordinary user and limit access to other users.

An example mountpoint configuration:

```
user@host:/remote/path /local/path  fuse.sshfs noauto,x-systemd.automount,_netdev,user,idmap=user,follow_symlinks,identityfile=/home/user/.ssh/id_rsa,allow_other,default_permissions,uid=USER_ID_N,gid=USER_GID_N 0 0
```

Summary of the relevant options:

- *allow_other* - Allow other users than the mounter (i.e. root) to access the share.
- *default_permissions* - Allow kernel to check permissions, i.e. use the actual permissions on the remote filesystem. This allows prohibiting access to everybody otherwise granted by *allow_other*.
- *uid*, *gid* - set reported ownership of files to given values; *uid* is the numeric user ID of your user, *gid* is the numeric group ID of your user.

Troubleshooting

Checklist

Read OpenSSH#Checklist first. Further issues to check are:

1. Is your SSH login sending additional information from server's `/etc/issue` file e.g.? This might confuse SSHFS. You should temporarily deactivate server's `/etc/issue` file:

```
$ mv /etc/issue /etc/issue.orig
```

1. Keep in mind that most SSH related troubleshooting articles you will find on the web are **not** systemd related. Often `/etc/fstab` definitions wrongly begin with

```
sshfs#user@host:/mnt/server/directory ... fuse ...
```

instead of using the syntax

```
user@host:/mnt/server/directory ... fuse.sshfs ... x-systemd, ...
```

1. Check that the owner of server's source directory and content is owned by the server's user.

```
$ chown -R user_s: /mnt/servers/directory
```

1. The server's user ID can be different from the client's one. Obviously both user names have to be the same. You just have to care for the client's user IDs. SSHFS will translate the UID for you with the following mount options:

```
uid=USER_C_ID,gid=GROUP_C_ID
```

1. Check that the client's target mount point (directory) is owned by the client user. This directory should have the same user ID as defined in SSHFS's mount options.

```
$ chown -R user_c: /mnt/client/directory
```

1. Check that the client's mount point (directory) is empty. By default you cannot mount SSHFS directories to non-empty directories.

Connection reset by peer

- If you are trying to access the remote system with a hostname, try using its IP address, as it can be a domain name resolving issue. Make sure you edit `/etc/hosts` with the server details.
- Make sure your user can log into the server (especially when using `AllowUsers`).
- Make sure `Subsystem sftp /usr/lib/ssh/sftp-server` is enabled in `/etc/ssh/sshd_config` on the remote system.
- If you are using a non-default key name and passing it as `-i .ssh/my_key`, this will not work. You have to use `-o IdentityFile=/home/*user*/.ssh/*my_key*`, with the full path to the key.
- If your `/root/.ssh/config/` is a symlink, you will be getting this error as well. See this serverfault topic
- Adding the option `sshfs_debug` (as in `sshfs -o sshfs_debug *user*@*server* ...`) can help in resolving the issue.
- If that does not reveal anything useful, you might also try adding the option `debug`.
- If you are trying to sshfs into a router running DD-WRT or the like, there is a solution here. (Note that the `-osftp_server=/opt/libexec/sftp-server` option can be used to the sshfs command instead of patching dropbear).
- If you see this only on boot, it may be that systemd is attempting to mount prior to a network connection being available. Enabling the appropriate wait-online service for your network manager fixes this.
- Old Forum thread: sshfs: Connection reset by peer.

注意：When providing more than one option for sshfs, they must be comma separated. Like so: `sshfs -o sshfs_debug,IdentityFile=*/path/to/key* *user*@*server* ...`).

Remote host has disconnected

If you receive this message directly after attempting to use *sshfs*:

- First make sure that the **remote** machine has *sftp* installed! It will not work, if not.
- Then, check that the path of the `Subsystem sftp` in `/etc/ssh/sshd_config` on the remote machine is valid.

fstab mounting issues

To get verbose debugging output, add the following to the mount options:

```
ssh_command=ssh\040-vv,sshfs_debug,debug
```

注意：Here, `\040` represents a space which fstab uses to separate fields.

To be able to run `mount -av` and see the debug output, remove the following:

```
noauto,x-systemd.automount
```

Some directories are empty

sshfs by default does not support symlinks. If those directories happened to be symlinks, use:

```
$ sshfs user@host:/remote/path /local/path -o follow_symlinks
```

Files not refreshed

If you see old content on remote, consider using option `dir_cache=no`:

```
$ sshfs user@host:/remote/path /local/path -o dir_cache=no
```

Limited transfer on fast network

If you observe transfer than is lower than your network capabilities and high CPU usage on the party where files are copied from, disable compression (remove `-C` option or set `-o compression=no`).

See also

- How to mount chrooted SSH filesystem, with special care with owners and permissions questions.

8.5 SSH 密钥

难度 高级 | **篇幅** 46.0 KB | **所属** SSH

本节将讲到

- 背景
- 生成密钥对
- 将公钥复制到远程服务器上
- SSH 代理
- 疑难排解

对于使用公钥加密与质询-应答式认证的 SSH 服务器，SSH 密钥可以作为证明身份的方法。基于密钥的认证主要优点在于，与密码认证相比，它不易遭受暴力破解攻击，且在服务器被攻破的情况下也不会泄露您的有效凭证（详细信息请参考 RFC 4251 9.4.4）。

不仅如此，SSH 密钥认证也会比传统的密码认证更加方便。当与被称作 SSH agent 的程序共用时，SSH 密钥可以让您无需记住或输入每个系统的密码，就能够连接到一个或多个服务器。

基于密钥的认证并非没有缺点，可能也并非适用于一切环境，但在很多情况下可以提供一些有力的优势。对 SSH 密钥如何工作的概略理解会帮助您决定如何以及何时使用它们，以满足您的需求。

本文假定您已经对于安全外壳协议有了基本理解，并安装了 openssh包 软件包。

背景

SSH 密钥都是成对生成的，其一称为公钥，另一则称为私钥。私钥只由您所知，必须安全保管。相对地，公钥可以向您想连接的任何 SSH 服务器自由地共享。

如果一个 SSH 服务器在文件中存有您的公钥，并收到了您的连接请求，就会使用您的公钥构建一个质询问题并发送给您。这一质询问题是一条加密信息，必须得到正确应答，服务器才能允许您访问。这条编码信息特别安全是在于，只有私钥持有者才能理解它。公钥可以用来加密信息，但不能用来解密同一条信息。只有您，私钥的持有者，能够正确理解这一质询问题并产生合适的应答。

这一质询-应答过程发生在后台，对用户不可见。只要您持有私钥（一般存放在 `~/.ssh/` 目录下），您的 SSH 客户端就应当能够向服务器回复正确的应答。

私钥是受保护的秘密，因此建议以加密形式将其存储在磁盘上。当需要被加密的私钥时，为了解密它，需要先输入密码。虽然这在表面上可能像是您在向 SSH 服务器提供登录密码，但该密码只用于解密本地系统上的私钥。该密码并未通过网络传输。

生成密钥对

通过运行 `ssh-keygen` 命令可以生成密钥对，具体“*一般被认为充足*”且应当兼容于几乎所有客户端和服务器的选项请参考 ssh-keygen(1)：

```
$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/username/.ssh/id_ed25519):
Created directory '/home/username/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/username/.ssh/id_ed25519
Your public key has been saved in /home/username/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:RLy4JBv7jMK5qYhRKwHB3af0rpMKYwE2PBhALCBV3G8 username@hostname
The key's randomart image is:
+--[ED25519 256]--+
|%oooo. ..        |
|== ..o.o.        |
|==  . +o..       |
|+ o o.ooE        |
|...  *.oS        |
| o..o ..         |
|o=.. +o          |
|+o*..+o          |
|+.o+. .          |<br>+----[SHA256]-----+<br>```
```

`[randomart 图像](https://www.cs.berkeley.edu/~dawnsong/papers/randomart.pdf)`作为一种简便的密钥指纹视觉辨识方式在 `[OpenSSH 5.1](https://www.openssh.com/txt/release-5.1)` 中引入。

`> **注意：**可以使用 `-a` 开关来指定密码加密的 KDF rounds 数量。`

也可以用 `-C` 开关对公钥添加可选的注释栏，从而更容易在 `~/.ssh/known_hosts`、`~/.ssh/authorized_keys` 以及 `ssh-add -L` 输出等处进行辨识。例如：

```
$ ssh-keygen -C "$(whoami)@$(uname -n)-$(date -I)"
```

会添加一条注释，说明是哪个用户何时在哪台机器上创建的密钥。

选择认证密钥类别

OpenSSH（对于认证密钥）支持数种签名算法，按照所采用的数学性质可分为两类：

1. [Ed25519](https://zh.wikipedia.org/wiki/Curve25519) 以及 [ECDSA](https://zh.wikipedia.org/wiki/%E6%A4%AD%E5%9C%86%E6%9B%B2%E7%BA%BF%E6%95%B0%E5%AD%97%E7%AD%BE%E5%90%8D%E7%AE%97%E6%B3%95)，依赖于椭圆曲线[离散对数](https://zh.wikipedia.org/wiki/%E7%A6%BB%E6%95%A3%E5%AF%B9%E6%95%B0)问题（ECDLP）；（[例子](https://www.certicom.com/content/certicom/en/52-the-elliptic-curve-discrete-logarithm-problem.html)）
2. [RSA](https://zh.wikipedia.org/wiki/RSA%E5%8A%A0%E5%AF%86%E7%AE%97%E6%B3%95)，依赖于对两个大质数之积进行分解的[实际困难度](https://zh.wikipedia.org/wiki/%E6%95%B4%E6%95%B0%E5%88%86%E8%A7%A3#.E9.9A.BE.E5.BA.A6.E4.B8.8E.E5.A4.8D.E6.9D.82.E5.BA.A6)。

[椭圆曲线密码学](https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography)（ECC）算法是对于公钥密码体制的[较新发展](https://zh.wikipedia.org/wiki/%E6%A4%AD%E5%9C%86%E6%9B%B2%E7%BA%BF%E5%AF%86%E7%A0%81%E5%AD%A6#.E6.AD.B7.E5.8F.B2)。其主要优势之一是[以较小密钥提供同等安全性](https://zh.wikipedia.org/wiki/%E6%A4%AD%E5%9C%86%E6%9B%B2%E7%BA%BF%E5%AF%86%E7%A0%81%E5%AD%A6#Rationale)的能力，使得计算密集操作更少（*也就是*密钥创建、加密及解密更快）且存储及传输的要求更低。

由于安全问题，DSA 密钥已被[废弃](https://archlinux.org/news/openssh-70p1-deprecates-ssh-dss-keys/)，且 SSH 实现正逐渐将其移除。OpenSSH 9.8 构建默认不带 DSA 密钥支持，Dropbear 2022.83 禁用了 DSA 密钥支持，libssh 0.11.0 完全移除了 DSA 密钥支持。因此[密码体制](https://zh.wikipedia.org/wiki/%E5%AF%86%E7%A0%81%E4%BD%93%E5%88%B6)的选择范围就限定在了 RSA 或两种 ECC 之一当中。

[#RSA](#RSA) 密钥会提供最大的可移植性，而 [#Ed25519](#Ed25519) 会提供最佳的安全性，但需要新版本的客户端与服务器。. #ECDSA 兼容性可能比 Ed25519 更好（虽然还是不如 RSA），但存在对于其安全性的怀疑（见下文）。

默认的 Ed25519 会提供最佳安全性和不错的性能。ECDSA 比 Ed25519 更慢，但比 RSA 快，在安全性上存在些疑问（具体参见下文）。RSA 密钥提供了对旧服务器的最佳兼容性，但需要更大的密钥才能提供足够的安全性。

> **注意：**这些密钥只用于认证您的身份；选择更强的密钥并不会在通过 SSH 传输数据时加重 CPU 负担。

> **提示：**为防止 SSH 密钥从设备上外泄，可以将密钥储存在 [FIDO/U2F 实体认证器](../../08_安全与加密/通用第二因素/通用第二因素.md)或[可信平台模块](../../04_硬件与外设/硬件/可信平台模块.md)中。
>
> - 在生成密钥时，可以使用特殊的"安全密钥"类型将 Ed25519 或 ECDSA 密钥储存在 FIDO/U2F 实体认证器上。具体信息请参考 #FIDO/U2F。
> - 可信平台模块支持 ECDSA 和 RSA，因此可以将 SSH 密钥封装在 TPM 内。具体信息请参考 [可信平台模块 #SSH](../../04_硬件与外设/硬件/可信平台模块.md#SSH)。

Ed25519

[Ed25519](https://ed25519.cr.yp.to/) 在2014年1月被引入 [OpenSSH 6.5](https://www.openssh.com/txt/release-6.5)："*Ed25519 是一种能够提供比 ECDSA 及 DSA 更佳安全性及良好性能的椭圆曲线签名体系*"。其主要长处在于速度、时间恒定运行时（从而能够对抗侧信道攻击）以及无需晦

暗不明的硬编码常数。 关于其工作方式，参见由一位 Mozilla 开发者撰写的[博客文章](https://blog.mozilla.org/warner/2011/11/29/ed25519-keys/)。

它已经在[众多应用及库中](https://zh.wikipedia.org/wiki/EdDSA#.E8.BD.AF.E4.BB.B6)得到实现，并且在 OpenSSH 中是[默认的密钥交换算法](https://www.libssh.org/2013/11/03/openssh-introduces-curve25519-sha256libssh-org-key-exchange/)（与密钥*签名*不同）。

Ed25519 密钥对生成方法如下：

[ssh-keygen(1)](https://man.archlinux.org/man/ssh-keygen.1) 默认使用 Ed25519，因此不需要使用 `-t ed25519` 选项进行指定。使用以下命令生成密钥：

```
$ ssh-keygen
```

所有 Ed25519 密钥都是 256 位，故无需设置密钥尺寸。

注意较老的 SSH 客户端与服务器可能不支持这些密钥。

ECDSA

椭圆曲线数字签名算法（ECDSA）是 [OpenSSH 5.7](https://www.openssh.com/txt/release-5.7)（2011-01-24）到 OpenSSH 6.5（2014-01-30）间的首选认证（密钥交换）算法。

对于它的担忧有两种：

1. *政治上的担忧*：在 NSA（美国国家安全局）被曝在软件、硬件组件及发布的标准中蓄意植入后门之后，由 NIST（美国国家标准技术研究所）产生的曲线的可信性[受到了质疑](https://crypto.stackexchange.com/questions/10263/should-we-trust-the-nist-recommended-ecc-parameters)；密码领域的知名人士对于 NIST 曲线的设计方式[表示](https://www.schneier.com/blog/archives/2013/09/the_nsa_is_brea.html#c1675929)[了](https://safecurves.cr.yp.to/rigid.html)[怀疑](https://www.hyperelliptic.org/tanja/vortraege/20130531.pdf)，并且之前已经有主动的污染[被](https://www.schneier.com/blog/archives/2007/11/the_strange_sto.html)[证实](https://www.scientificamerican.com/article/nsa-nist-encryption-scandal/)。
2. *技术上的担忧*：对于[正确实现该标准的困难](https://blog.cr.yp.to/20140323-ecdsa.html)以及在实现不够谨慎情况下降低安全性的[迟缓与设计缺陷](https://www.gossamer-threads.com/lists/openssh/dev/57162#57162)。

[libssh curve25519 介绍](https://git.libssh.org/projects/libssh.git/tree/doc/curve25519-sha256@libssh.org.txt#n4)当中很好地总结了上述担忧。尽管政治上的担忧仍受到争议，但有[明确的共识](https://news.ycombinator.com/item?id=7597653)认为 [#Ed25519](#Ed25519) 技术上更优越，因此应当优先采用。

可以使用以下方法生成 ECDSA 密钥对：

```
$ ssh-keygen -t ecdsa
```

ECDSA 密钥支持三种椭圆曲线大小：256，384 和 521 位，默认使用 256 位。如果想要生成更安全的 ECDSA 密钥对，可以使用 `-b` 选项进行指定：

```
$ ssh-keygen -t ecdsa -b 384
```

RSA

RSA 提供所有算法当中最佳的兼容性，但提供充足安全性所需的密钥尺寸较大。最低密钥大小为 1024 位，默认为 3072（具体请参考 [ssh-keygen(1)](https://man.archlinux.org/man/ssh-keygen.1)），最大为 16384。

使用以下命令生成 RSA 密钥对：

```
$ ssh-keygen -t rsa
```

如果您想生成较强的 RSA 密钥对（*例如*为了防范先进或未知攻击以及较高级的攻击者），只需为 `-b` 选项指定比默认更高的位值：

$ ssh-keygen -t rsa -b 4096

需要明白的是，使用更长的密钥存在收益递减。 GnuPG 常见问答记载："*如果你需要的安全性比 RSA-2048 所提供的更强，应该做的是改用椭圆曲线密码学—而不是接着使用 RSA*。"

另一方面，最新版本的 [NSA Fact Sheet Suite B Cryptography](https://web.archive.org/web/20160305203915/https://www.nsa.gov/ia/programs/suiteb_cryptography/index.shtml) 建议对于 RSA 使用最少 3072 位的模，同时"*为即将到来的抗量子算法迁移（做准备）*"。

FIDO/U2F

[OpenSSH 8.2 版本](https://www.openssh.com/txt/release-8.2)添加了 FIDO/[U2F](../../08_安全与加密/通用第二因素/通用第二因素.md) [实体认证器](https://zh.wikipedia.org/wiki/%E5%AE%89%E5%85%A8%E4%BB%A4%E7%89%8C)支持，可以使用上文描述的两种椭圆曲线签名方案。这使得可以将连接的实体令牌作为多重验证方案与私钥搭配使用。

实体令牌需要用到 [libfido2](https://archlinux.org/packages/?name=libfido2)包。

> **注意：**
>
> - 客户端和服务器都需要支持 `ed25519-sk` 和 `ecdsa-sk` 密钥类型。
> - OpenSSH 使用中间件库来与实体令牌通信，其带有支持 USB 令牌的内部中间件。如果要使用其它中间件，可以通过[sshd_config(5) § SecurityKeyProvider](https://man.archlinux.org/man/sshd_config.5#SecurityKeyProvider) 配置项或为 `ssh-keygen` 和 `ssh-add` 配置 `SSH_SK_PROVIDER` 环境变量来实现。

连接兼容的 FIDO 密钥后，使用以下命令生成密钥对：

$ ssh-keygen -t ed25519-sk

这一步通常需要输入 PIN 码并/或触摸令牌进行确认。除非在生成时使用了 `-O no-touch-required` 命令行选项或服务器指定了 [sshd(8) § no-touch-required](https://man.archlinux.org/man/sshd.8#no-touch-required) `authorized_keys` 选项，否则连接服务器时通常需要触摸令牌进行确认。

要生成不需要触摸确认的密钥，需在生成密钥对时使用 `no-touch-required` 选项，例如：

$ ssh-keygen -O no-touch-required -t ed25519-sk

> **注意：**并不是所有实体令牌都支持该选项。如果你使用的是 YubiKey，需要 5.2.3 固件版本才能使用 ed25519-sk 类型密钥。

另外，`sshd` 默认会拒绝 `no-touch-required` 密钥。为使用通过该选项生成的密钥，需在 `authorized_keys` 文件内对每个密钥单独进行启用：

no-touch-required sk-ssh-ed25519@openssh.com AAAAInN... user@example.com

或是在 `/etc/ssh/sshd_config` 中对整个系统进行启用：

PubkeyAuthOptions none

> **提示：**[GitHub](https://github.com/orgs/community/discussions/10593) 和 [GitLab](https://gitlab.com/gitlab-org/gitlab/-/issues/367682) 不支持 `no-touch-required`。

基于 ECDSA 的密钥对可通过 `ecdsa-sk` 密钥类型生成，但同样会有 #ECDSA 一节提到的问题：

$ ssh-keygen -t ecdsa-sk

选择密钥存储位置以及密码短语

运行 `ssh-keygen` 时，它会询问您希望的私钥文件名称及位置。默认情况下，密钥保存到 `~/.ssh` 目录下，并根据所使用的加密类型命名。为使下文中的示例代码正确工作，建议您接受默认的名称和位置。

当系统向您询问密码短语时，如果您在乎私钥的安全性，请选择难以猜到的密码。更长、更随机的密码一般会更强，当私钥落入贼人之手时更不容易被破解掉。

在没有密码短语的情况下生成私钥也是可能的。虽然也许很方便，但您需要明白随之而来的风险。在没有密码短语的情况下，您的私钥会以未加密形式存储在硬盘上。任何能接触到您私钥文件的人之后都能够在您使用基于密钥认证连接的任何 SSH 服务器面前冒用您的身份。更进一步，没有密码短语，您也必须信任 root 用户，因为他可以绕过文件权限并能够随时访问您未加密的私钥文件。

> **注意：**从前，私钥密码都是以一种不安全的方式编码的：仅一遍 MD5 散列。OpenSSH 6.5 及之后版本支持一种新的、更安全的格式来编码您的私钥。从 [OpenSSH 7.8 版本](https://www.openssh.com/txt/release-7.8)开始默认使用该格式。Ed25519 密钥一直采用新的编码格式。只需按下节所述更改密钥的密码短语即可升级到新格式。

不修改密钥对的情况下修改密码短语

如果不希望使用原本选择的 SSH 密钥密码短语或者必须更换，可以使用 `ssh-keygen` 命令来修改密码短语，而无需改动实际密钥。此法也可用于将密码编码格式改为新标准。

$ ssh-keygen -f ~/.ssh/id_rsa -p

管理多组密钥对

如果你有多个 SSH 身份，可以在配置文件中通过 `Host` 和 `IdentityFile` 为不同的主机和远程用户使用不同的密钥：

~/.ssh/config

Host SERVER1 IdentitiesOnly yes IdentityFile ~/.ssh/id_rsa_IDENTITY1

Host SERVER2 SERVER3 IdentitiesOnly yes IdentityFile ~/.ssh/id_ed25519_IDENTITY2

对于这些选项的完整描述见 [ssh_config(5)](https://man.archlinux.org/man/ssh_config.5)。

在硬件令牌上存储 SSH 密钥

SSH 密钥也可以存储在智能卡或 USB 令牌等安全令牌上。这样做的好处是私钥安全地存储在令牌上，而不是存储在磁盘中。当使用安全令牌时，敏感的私钥也从来不会存在于 PC 的内存当中；密码学操作在令牌自身上进行。密码学令牌还有一点好处是，它不与单台电脑绑定；可以将其从一台电脑上轻松取下，四处携带之后在其他电脑上使用。

硬件令牌的例子可参考：

- #FIDO/U2F
- [YubiKey#SSH notes](../../08_安全与加密/Smartcards/YubiKey.md#SSH_notes) 用于 FIDO/U2F keys 的原生 OpenSSH 支持
- [YubiKey#SSH keys](../../08_安全与加密/Smartcards/YubiKey.md#SSH_keys)
- [可信平台模块#SSH](../../04_硬件与外设/硬件/可信平台模块.md#SSH)

将公钥复制到远程服务器上

创建好密钥对之后，您需要将公钥上传到远程服务器上，以便用于 SSH 密钥认证登录。公钥和私钥的文件名相同，只不过公钥文件带有扩展名 `.pub` 而私钥文件名则没有。千万不要将私钥上传，私钥应该保存在本地。

简易方式

如果您的公钥文件为 `~/.ssh/id_rsa.pub`，只需要输入命令：
$ ssh-copy-id remote-server.org

如果您的远程服务器用户名与本地的不同，您需要在远程主机名前以 `@` 隔开指定用户名：
$ ssh-copy-id username@remote-server.org

如果您的公钥文件名不是默认的 `~/.ssh/id_rsa.pub`，会出现报错 `/usr/bin/ssh-copy-id: ERROR: No identities found`。这种情况下，您需要显式提供公钥的位置：
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub username@remote-server.org

如果远程服务器监听端口不是默认的 22，请在主机参数中指明：
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 221 username@remote-server.org

手动方式

对于 OpenSSH，默认需要将公钥添加到 `~/.ssh/authorized_keys`。先从复制公钥到远程服务器开始。
$ scp ~/.ssh/id_ecdsa.pub username@remote-server.org:

以上示例通过 `scp` 将公钥（`id_ecdsa.pub`）复制到您在远程服务器上的主目录。别忘了加上服务器地址末尾的 `:`。并请注意，您的公钥名称可能与此例所示不同。

在远程服务器上，如果还没有 `~/.ssh` 目录，您需要创建一个；然后将您的公钥追加到 `authorized_keys` 文件内。
$ ssh username@remote-server.org username@remote-server.org's password: $ install -dm700 ~/.ssh $ cat ~/id_ecdsa.pub >> ~/.ssh/authorized_keys $ rm ~/id_ecdsa.pub $ chmod 600 ~/.ssh/authorized_keys

上面最后两个命令将公钥文件从服务器上删除，并设置 `authorized_keys` 文件的权限，这样只有作为文件拥有者的您能够读取及写入。

SSH 代理

如果您的私钥使用密码短语来加密了的话，每一次试图连接到使用公钥认证的 SSH 服务器的时候，您都必须输入密码短语。每次唤起 `ssh` 或 `scp` 时都需要密码短语来解密您的私钥以便认证能够继续。

而 SSH 代理程序能够将您的已解密的私钥缓存起来，并代表您将其提供给 SSH 客户端。这样子，您就只需要将私钥加入 SSH 代理缓存的时候输入一次密码短语就可以了。在经常使用 SSH 连接时，将会提供不少便利。

SSH 代理一般会设置成在登录会话的时候自动启动，并在整个会话中保持运行。有多种代理、前端及配置都能够达成这一效果。本节概述了多种不同的解决方案，能够适应以满足您的特定需求。

ssh-agent

`ssh-agent` 是 OpenSSH 自带的默认代理。它可以直接使用，或者作为下文所述的几种前端解决方案的后端。`ssh-agent` 运行时会自动 fork 到后台，然后打印出其所需的环境变量，例如：
$ ssh-agent

SSH_AUTH_SOCK=/tmp/ssh-vEGjCM2147/agent.2147; export SSH_AUTH_SOCK; SSH_AGENT_PID=2148; export SSH_AGENT_PID; echo Agent pid 2148;

要使用这些环境变量，请使用 `eval` 命令来运行它。如果使用的是 `fish` shell，需要转而使用 `ssh-agent -c`。
$ eval $(ssh-agent)

Agent pid 2157

`ssh-agent` 运行起来之后，您还需要将您的私钥加入它的缓存。

$ ssh-add ~/.ssh/id_ed25519

Enter passphrase for /home/user/.ssh/id_ed25519: Identity added: /home/user/.ssh/id_ed25519 (/home/user/.ssh/id_ed25519)

如果您的私钥已加密，`ssh-add` 会提示您输入密码短语。一旦您的私钥被成功添加到代理，您不需要再输入密码短语就能够建立 SSH 连接了。

> **提示：**要让包括 `git` 在内的所有 `ssh` 客户端在第一次使用时就把密钥存储在代理中，请将配置 `AddKeysToAgent yes` 添加到 `~/.ssh/config`。其他可能的值有 `confirm`、`ask` 和 `no`（默认）。

为了自动启动代理并确保同一时间只有一个 `ssh-agent` 进程在运行，添加以下内容到您的 `~/.bashrc`：

if ! pgrep -u "$USER" ssh-agent > /dev/null; then ssh-agent -t 1h > "$XDG_RUNTIME_DIR/ssh-agent.env" fi if [[! -f "$SSH_AUTH_SOCK"]]; then source "$XDG_RUNTIME_DIR/ssh-agent.env" >/dev/null fi

如果还没有 `ssh-agent` 进程在运行的话，该脚本会自动启动一个，并保存其输出。如果已经有一个在运行了，就会读取缓存的 `ssh-agent` 输出并执行之，从而设置必要的环境变量。解密密钥的存活时间设置为1小时。

本节后面会介绍几种 `ssh-agent` 的前端及其它代理，也能够避免这一问题。

用 systemd user 启动 ssh-agent

如果想要在登录时运行 SSH agent 而不论 X 是否在运行的话，从 9.4p1-3 版本开始 [openssh](https://archlinux.org/packages/?name=openssh)包 提供了 `ssh-agent.service`，可以作为[用户单元](../../02_系统管理与内核/系统管理/Systemd_用户.md)启用。

然后将[环境变量](../../02_系统管理与内核/系统管理/环境变量.md#按用户) `SSH_AUTH_SOCK` 设为 `$XDG_RUNTIME_DIR/ssh-agent.socket`。

> **注意：**如果您使用 GNOME，该环境变量默认会被覆盖。见 [GNOME/Keyring#禁用钥匙环的守护进程组件](../../05_桌面与图形/GNOME/GNOME_Keyring.md#禁用钥匙环的守护进程组件)。

转发 ssh-agent

> [](https://wiki.archlinuxcn.org/wzh/File:Tango-edit-clear.png)**本文或本章节的语言、语法或风格需要改进。参考：[帮助:风格](https://wiki.archlinuxcn.org/wzh/Help:%E9%A3%8E%E6%A0%BC)**
>
> **原因：**This is not specific to ssh-agent, e.g. gpg-agent uses the same environment variable: [GnuPG#Forwarding gpg-agent and ssh-agent to remote](../邮件/GnuPG.md#Forwarding_gpg-agent_and_ssh-agent_to_remote)（在[Talk:SSH 密钥](https://wiki.archlinuxcn.org/wiki/Talk:SSH_%E5%AF%86%E9%92%A5)讨论）

When [forwarding](OpenSSH.md#Agent_forwarding) a local `ssh-agent` to remote (e.g., through command-line argument `ssh -A remote` or through `ForwardAgent yes` in the configuration file), it is important for the remote machine not to overwrite the environment variable `SSH_AUTH_SOCK`. So if the remote machine uses a [systemd unit](#Start_ssh-agent_with_systemd_user) shown previously to start the agent, `SSH_AUTH_SOCK` must not be set in the environment when a user is logged in through SSH. Otherwise, the forwarding may fail, and you may see errors (for example: `The agent has no identities`) when checking the existing keys with `ssh-add -l` on the remote machine.

For example, if using bash, the `.bashrc` could be something like:

~/.bashrc

... if [[-z "${SSH_CONNECTION}"]]; then export SSH_AUTH_SOCK="$XDG_RUNTIME_DIR/ssh-agent.socket" fi ...

In this way, `SSH_AUTH_SOCK` is only set when the current session is **not** a SSH login. And when this is a SSH session, `SSH_AUTH_SOCK` on the remote machine is then set by the local machine to make the forwarding work.

作为包装程序运行 ssh-agent

[加州大学伯克利分校实验室的这份 ssh-agent 教程](https://upc.lbl.gov/docs/user/sshagent.shtml)还介绍了一种随每个 X 会话启动 `ssh-agent` 的方法。基本用法是，如果您使用命令 `startx` 正常启动 X，您可以改成像这样在前面加上 `ssh-agent`：

$ ssh-agent startx

您也可以在 `.bash_aliases` 或等同文件中设置别名，就不用再费心了：

alias startx='ssh-agent startx'

这样做可以避免多个会话中存在多余的 `ssh-agent` 进程。整个 X 会话中只会有一个 `ssh-agent` 在运行。

> **注意：**`ssh-askpass` 需要 `DISPLAY` 或 `WAYLAND_DISPLAY` 环境变量才能正常工作，所以你可能会需要在 `~/.xinitrc` 中运行 `ssh-agent` 以配置 `DISPLAY` 选项，而不是使用 `ssh-agent startx`（如将 `exec ssh-agent dbus-launch i3` 添加到 `~/.xinitrc`）。或者也可以将 `eval $(ssh-agent)` 添加到 `~/.xinitrc` 以使用 `ssh-agent` 作为包装程序运行。

见[下文有关协同使用 x11-ssh-askpass 与 ssh-add 的说明](#Calling_x11-ssh-askpass_with_ssh-add)了解如何立即将密钥添加到 agent。

OpenPGP card ssh-agent

This ssh-agent specializes on OpenPGP card integration. It uses private keys that are stored in OpenPGP card authentication slots.

> **注意：** When also using [GnuPG](../邮件/GnuPG.md) while running this agent, it is required to [use it with pcscd](../邮件/GnuPG.md#GnuPG_with_pcscd_(PCSC_Lite)) and configure [shared access with pcscd](../邮件/GnuPG.md#Shared_access_with_pcscd).

Install [openpgp-card-ssh-agent](https://archlinux.org/packages/?name=openpgp-card-ssh-agent)包 and enable and start the `openpgp-card-ssh-agent.socket` user unit.

Afterwards add the relevant environment variable for this agent:

export SSH_AUTH_SOCK="$XDG_RUNTIME_DIR/openpgp-card/ssh-agent.sock"

User PIN handling

The user PIN for the OpenPGP card is persisted via an [org.freedesktop.secrets](https://archlinux.org/packages/?q=org.freedesktop.secrets) provider (such as [GNOME Keyring](../../05_桌面与图形/GNOME/GNOME_Keyring.md), KeePassXC or [KDE Wallet](../../05_桌面与图形/KDE/KDE%20Wallet.md)) by default. The PIN storage backend is [configurable and extendable](https://codeberg.org/openpgp-card/state).

The user PIN needs to be persisted only once for each OpenPGP card. Prior to the first SSH connection with this agent, list the available SSH public keys and add their respective card identifiers:

$ ssh-add -L ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJUz6VnFprMe33G88Pq8NLw3wnIKOsBg0CDrwFeUVrU6 FFFE:01234567 $ ssh-add -s FFFE:01234567 Enter passphrase for PKCS#11:

> **注意：** The prompt mentions *PKCS#11* as it is a hardcoded message in [ssh-add(1)](https://man.archlinux.org/man/ssh-add.1). However, in this context the OpenPGP card user PIN must be entered.

GnuPG Agent

[gpg-agent](../邮件/GnuPG.md) 模拟了 OpenSSH 的代理协议，必要配置请参考 [GnuPG#SSH 代理](../邮件/GnuPG.md#SSH_代理)。

Keychain

[Keychain](https://www.funtoo.org/Keychain) 是一个用来方便管理 SSH 密钥对的程序，它能尽最大努力去减少对用户的打扰。实际上，它就是一个 shell 脚本，驱动 *ssh-agent* 或者 *gpg-add* 来工作。一个值得注意的特性是，Keychain 在多个会话中重复使用同一个 *ssh-agent* 进程。这意味着您只需要在机器启动时输入一次密码短语即可。

安装

安装 [keychain](https://archlinux.org/packages/?name=keychain)包 软件包。

配置

> **警告：**直至 2015-09-26 为止，`-Q, --quick` 选项存在预期之外的副作用，导致 *keychain* 在重新登录时切换到新产生的 *ssh-agent* （至少在使用 [GNOME](../../05_桌面与图形/GNOME/GNOME.md) 的系统上如此），使您不得不重新添加所有先前注册的密钥。

将类似以下的行加入到您 [shell](../../07_开发与工具/命令行解释器/命令行解释器.md) 的配置文件中，*例如*若您使用[Bash](../../07_开发与工具/命令行解释器/Bash.md)：

~/.bashrc

eval $(keychain --eval --quiet id_ed25519 id_rsa ~/.keys/my_custom_key)

> **注意：**之所以使用 `~/.bashrc` 而非上游建议的 `~/.bash_profile`，是因为在 Arch 上登录与非登录 shell 都会使用它，因此同样适用于文字与图形环境。关于二者之间差异的更多信息见[Bash#调用](../../07_开发与工具/命令行解释器/Bash.md#调用)。

上例当中，

- `--eval` 开关可以输出供 `eval` 命令执行的行；这样可以设置必要的环境变量，让 SSH 客户端能够找到您的 agent。
- `--quiet` 会把输出限制为只包含警告、错误及用户提示。

如例子所示，可在命令行中指定多个密钥。Keychain 默认会在 `~/.ssh/` 目录中寻找密钥对，但对于位于非标准位置的密钥可以使用绝对路径。也可使用 `--confhost` 选项告知 keychain 在 `~/.ssh/config` 中寻找为特定主机定义的 `IdentityFile` 设置，并使用这些路径来定位密钥。

关于如何为其他 shell 设置 *keychain* 的详情请参考 `keychain --help` 或 [keychain(1)](https://man.archlinux.org/man/keychain.1)。

要测试 Keychain，只需打开一个终端模拟器，或注销当前会话再重新登录。它应该会使用 `$SSH_ASKPASS` 中设置的程序或在终端上提示您输入指定私钥的密码短语（如果有）。

因为 Keychain 重新使用之前的登录中的 *ssh-agent* 进程，所以您再次登录或打开新终端时应该就无需输入密码短语了。每当您重启机器时，您才会被要求再次输入密码短语。

提示

- *keychain* 需要公钥文件存放在相应私钥的同一目录下，并带有 `.pub` 扩展名。如私钥为符号链接，公钥需要与符号链接一起存放或与符号链接目标在同一目录下（该功能需要系统上有 `readlink` 命令）。

- 要禁用图形提示，以始终在终端中输入密码短语，请使用 `--nogui` 选项。举例来说，这样可以从密码管理器中复制粘贴长密码。

- 如果您想要等到需要时再提示解锁密钥，而不是一开始就提示，请使用 `--noask` 选项。

> **注意：**Keychain 能够以同样的方式管理 [GPG](../邮件/GnuPG.md) 密钥。它默认只会尝试启动 *ssh-agent*，但使用 `--agents` 选项可以改变这一行为，*例如* `--agents ssh,gpg`。详细信息请参考 [keychain(1)](https://man.archlinux.org/man/keychain.1)。

> **注意：**
>
> - Keychain 能够以同样的方式管理 [GPG](../邮件/GnuPG.md) 密钥。它默认只会尝试启动 *ssh-agent*，但使用 `--agents` 选项可以改变这一行为，*例如* `--agents ssh,gpg`。详细信息请参考 [keychain(1)](https://man.archlinux.org/man/keychain.1)。
> - 根据 [keychain issue 148](https://github.com/funtoo/keychain/issues/148)，在使用 Wayland 时可能需要添加 `--inherit any-once` 参数。

x11-ssh-askpass

The [x11-ssh-askpass](https://archlinux.org/packages/?name=x11-ssh-askpass)包 package provides a graphical dialog for entering your passhrase when running an X session. *x11-ssh-askpass* depends only on the [libx11](https://archlinux.org/packages/?name=libx11)包 and [libxt](https://archlinux.org/packages/?name=libxt)包 libraries, and the appearance of *x11-ssh-askpass* is customizable. While it can be invoked by the *ssh-add* program, which will then load your decrypted keys into [ssh-agent](#ssh-agent), the following instructions will, instead, configure *x11-ssh-askpass* to be invoked by the aforementioned [Keychain](#Keychain) script.

Install the [keychain](https://archlinux.org/packages/?name=keychain)包 and [x11-ssh-askpass](https://archlinux.org/packages/?name=x11-ssh-askpass)包 packages.

Edit your `~/.xinitrc` file to include the following lines, replacing the name and location of your private key if necessary. Be sure to place these commands **before** the line which invokes your window manager.

~/.xinitrc

keychain ~/.ssh/id_ecdsa [-f ~/.keychain/$HOSTNAME-sh] && . ~/.keychain/$HOSTNAME-sh 2>/dev/null [-f ~/.keychain/$HOSTNAME-sh-gpg] && . ~/.keychain/$HOSTNAME-sh-gpg 2>/dev/null ... exec openbox-session

In the above example, the first line invokes *keychain* and passes the name and location of your private key. If this is not the first time *keychain* was invoked, the following two lines load the contents of `$HOSTNAME-sh` and `$HOSTNAME-sh-gpg`, if they exist. These files store the environment variables of the previous instance of *keychain*.

Calling x11-ssh-askpass with ssh-add

The *ssh-add* manual page specifies that, in addition to needing the `DISPLAY` or `WAYLAND_DISPLAY` variable defined, you also need `SSH_ASKPASS` set to the name of your askpass program (in this case *x11-ssh-askpass*). It bears keeping in mind that the default Arch Linux installation places the *x11-ssh-askpass* binary in `/usr/lib/ssh/`, which will not be in most people's `PATH`. This is a little annoying, not only when declaring the `SSH_ASKPASS` variable, but also when theming. You have to specify the full path everywhere. Both inconveniences can be solved simultaneously by symlinking:

$ ln -sv /usr/lib/ssh/x11-ssh-askpass ~/bin/ssh-askpass

This is assuming that `~/bin` is in your `PATH`. So now in your `.xinitrc`, before calling your window manager, one just needs to export the `SSH_ASKPASS` environment variable:

$ export SSH_ASKPASS=ssh-askpass

and your [X resources](../../01_安装与包管理/配置文件/X%20资源.md) will contain something like:

ssh-askpass*background: #000000

Doing it this way works well with [the above method on using *ssh-agent* as a wrapper program](#ssh-agent_as_a_wrapper_program). You start X with `ssh-agent startx` and then add *ssh-add* to your window manager's list of start-up programs.

Theming

The appearance of the *x11-ssh-askpass* dialog can be customized by setting its associated [X resources](../../01_安装与包管理/配置文件/X%20资源.md). Some examples are the .ad files at https://github.com/sigmavirus24/x11-ssh-askpass. See [x11-ssh-askpass(1)](https://man.archlinux.org/man/x11-ssh-askpass.1) for full details.

Alternative passphrase dialogs

There are other passphrase dialog programs which can be used instead of *x11-ssh-askpass*. The following list provides some alternative solutions.

- [seahorse](https://archlinux.org/packages/?name=seahorse)包 (provides `/usr/lib/seahorse/ssh-askpass`) uses the [GTK](../../07_开发与工具/控件工具集/GTK.md) library. .
- [gnome-ssh-askpass3](https://aur.archlinux.org/packages/gnome-ssh-askpass3/)AUR uses the [GTK](../../07_开发与工具/控件工具集/GTK.md) library.
- [ksshaskpass](https://archlinux.org/packages/?name=ksshaskpass)包 uses the [KDE Wallet](../../05_桌面与图形/KDE/KDE%20Wallet.md).
- [openssh-askpass](https://aur.archlinux.org/packages/openssh-askpass/)AUR uses the [Qt5](../../07_开发与工具/控件工具集/Qt.md) library.
- [lxqt-openssh-askpass](https://archlinux.org/packages/?name=lxqt-openssh-askpass)包

pam_ssh

[pam_ssh](http://pam-ssh.sourceforge.net/) 项目为 SSH 私钥提供了一个[可插拔认证模块](https://zh.wikipedia.org/wiki/%E5%8F%AF%E6%8F%92%E6%8B%94%E8%AE%A4%E8%AF%81%E6%A8%A1%E5%9D%97)（PAM），该模块可以为 SSH 连接提供类似单点登录验证的特性。在认证完成后，pam_ssh 模块会让 ssh-agent 在整个会话期间缓存您已解密的私钥。

要在 tty 模式下使用单点登录特性，需要安装非官方的 [pam_ssh](https://aur.archlinux.org/packages/pam_ssh/)AUR 包。

> **注意：**pam_ssh 2.0** 要求所有用于认证的私钥文件必须保存在 `~/.ssh/login-keys.d/` 下。

可以为您的私钥文件创建一个软链接，并放到 `~/.ssh/login-keys.d/`。将以下案例中的 `id_rsa` 替换为你的私钥的名称：

$ mkdir ~/.ssh/login-keys.d/ $ cd ~/.ssh/login-keys.d/ $ ln -s ../id_rsa

编辑 `/etc/pam.d/login`，将下面例子中高亮加粗的那几行加进去。请注意配置内容的顺序会影响到登录行为，应当按照例子中的来。

> **警告：**PAM 配置错误会导致系统中的所有用户被锁定。因此，在进行任何更改前，你需要大致了解 PAM 配置的工作方式，并确保具有备用的 PAM 配置文件访问手段（如 Arch Live CD），以确保被锁定后仍能撤回更改。可以参阅 IBM developerWorks 的 [这篇文章](https://developer.ibm.com/tutorials/l-pam/) 详细了解一下 PAM 的配置细节。

/etc/pam.d/login

auth required pam_securetty.so auth requisite pam_nologin.so auth include system-local-login auth optional pam_ssh.so try_first_pass account include system-local-login session include system-local-login session optional pam_ssh.so

在上面的例子中，登录认证在起始阶段正常进行，并要求用户输入登录密码。附加到认证栈末尾的额外 `auth` 认证规则会让 pam_ssh 尝试解密 `~/.ssh/login-keys.d` 目录下的所有私钥。`try_first_pass` 选项会被传递给 pam_ssh 模块，要求它尝试使用用户之前输入的密码解密 SSH 私钥。如果用户密码与私钥密码一致，用户就无需再次输入相同的密码。如果两者不同，pam_ssh 模块会在用户输入登录密码后要求输入私钥密码。`optional` 选项可以保证在没有私钥时用户也能正常登录系统。这样，没有 SSH 私钥的用户对 pam_ssh 是无感的。

如果你使用 X11 显示管理器（如 [SLiM](../../05_桌面与图形/显示管理器/SLiM.md) 或 [XDM](../../05_桌面与图形/显示管理器/XDM.md)）等其它方式进行登录，必须仿照如上配置编辑 PAM 配置文件。提供 PAM 支持的软件包通常会将默认配置文件放置在 `/etc/pam.d/` 目录下。

Further details on how to use pam_ssh and a list of its options can be found in the [pam_ssh(8)](https://linux.die.net/man/8/pam_ssh) man page.

使用不同密码解锁 SSH 密钥

If you want to unlock the SSH keys or not depending on whether you use your key's passphrase or the (different!) login password, you can modify `/etc/pam.d/system-auth` to
/etc/pam.d/system-auth

auth [success=1 new_authtok_reqd=1 ignore=ignore default=ignore] pam_unix.so try_first_pass nullok auth required pam_ssh.so use_first_pass auth optional pam_permit.so auth required pam_env.so

account required pam_unix.so account optional pam_permit.so account required pam_time.so

password required pam_unix.so try_first_pass nullok sha512 shadow password optional pam_permit.so

session required pam_limits.so session required pam_unix.so session optional pam_permit.so session optional pam_ssh.so

For an explanation, see .

pam_ssh 已知问题

pam_ssh 项目并不活跃，其提供的文档也比较少。你需要注意该软件未提及的一些使用限制，比如：

- 2.0 版本前的 pam_ssh 不支持 ECDSA（椭圆曲线）SSH 密钥，需要使用旧版本的必须使用 RSA 密钥。

- pam_ssh 调用的 `ssh-agent` 进程仅限于当前用户登录，无法在多次登录间持久化。如果你在登录期间保留了一个 [GNU Screen](../../06_软件与应用/GNU/GNU%20Screen.md) 会话，会发现重新连接 screen 会话后无法与 ssh-agent 进行通信。这是因为 GNU Screen 环境和其子进程仍在引用调用 screen 时存在的 ssh-agent，但其已随上次登出被关闭。上文提到的 [Keychain](#Keychain) 前端可以在多次登录间保留 ssh-agent 进程，从而避免该问题。

pam_exec-ssh

As an alternative to [pam_ssh](#pam_ssh) you can use [pam_exec-ssh-git](https://aur.archlinux.org/packages/pam_exec-ssh-git/)AUR. It is a shell script that

uses pam_exec. Help for configuration can be found [upstream] (https://github.com/x70b1/pam_exec-ssh).

GNOME Keyring

[GNOME Keyring](../../05_桌面与图形/GNOME/GNOME_Keyring.md) 可用作 ssh-agent 的封装，并提供图形界面和/或密钥自动解锁功能。详情请参考 [GNOME Keyring#SSH密钥](../../05_桌面与图形/GNOME/GNOME_Keyring.md#SSH密钥)。

使用 Kwallet 存储 SSH 密钥

For instructions on how to use kwallet to store your SSH keys, see [KDE Wallet#Using the KDE Wallet to store ssh key passphrases](../../05_桌面与图形/KDE/KDE%20Wallet.md#Using_the_KDE_Wallet_to_store_ssh_key_passphrases).

KeePass2 搭配 KeeAgent 插件

[KeeAgent](https://lechnology.com/software/keeagent/) is a plugin for KeePass that allows SSH keys stored in a KeePass database to be used for SSH authentication by other programs.

- Supports both PuTTY and OpenSSH private key formats.
- Works with native SSH agent on Linux/Mac and with PuTTY on Windows.

See KeePass#Plugin installation in KeePass or install the [keepass-plugin-keeagent] (https://archlinux.org/packages/?name=keepass-plugin-keeagent)包 package.

This agent can be used directly, by matching KeeAgent socket: `KeePass -> Tools -> Options -> KeeAgent -> Agent mode socket file -> %XDG_RUNTIME_DIR%/keeagent.socket`\- and environment variable: `export SSH_AUTH_SOCK="$XDG_RUNTIME_DIR"'/keeagent.socket'`.

KeePassXC

The KeePassXC fork of KeePass [can act as a client for an existing SSH agent] (https://keepassxc.org/docs/#faq-ssh-agent-how). SSH keys stored in its database can be automatically (or manually) added to the agent. It is also compatible with KeeAgent's database format.

疑难排解

密钥被服务器忽略

如果您的 SSH 服务器忽略了您的 SSH 密钥对，您需要检查一下相关文件的权限是否正确。

: 本地机器上：
$ chmod 700 ~/.ssh $ chmod 600 ~/.ssh/key

: 服务器上:
$ chmod 700 ~/.ssh $ chmod 600 ~/.ssh/authorized_keys

如果这样还不能解决您的问题，您可以试试将 `/etc/ssh/sshd_config` 中的 `StrictModes` 设为 `no`。如果使用 `StrictModes off` 就能够顺利认证的话，说明相关文件的权限还没有改对。

- 确保 `~/.ssh/authorized_keys` 中的密钥正确并只有一行。
- 确保远程设备支持使用的密钥类型：有些服务器不支持 ECDSA 密钥，可以试试 RSA 密钥，具体请参考 #生成密钥对。
- 你可能会想在连接时启用调试模式并监控输出：
- 如果你的密钥使用其它名字（如 `id_rsa_server`），需要使用 `-i` 选项进行连接：
$ ssh -i id_rsa_server user@server

代理拒绝操作

```
If your private key requires a password (or, for instance, you have a hardware key with a PIN) but ssh-agent is not provided with one, `ssh` will fail:
```
sign_and_send_pubkey: signing failed for ECDSA-SK user@host from agent: agent refused operation

```
One potential cause for this is ssh-agent being unable to prompt for a password. Ensure that ssh-agent has access to either a display server (via the `DISPLAY` environment variable) or a TTY.

Another cause, if using a hardware authenticator, could be the key malfunctioning or being unplugged.

There is currently an open [bug](https://bugzilla.mindrot.org/show_bug.cgi?id=3572) that triggers with the "agent refused operation" error when using authenticator keys like ED25519-sk and ECDSA-SK that were created with the option `-O verify-required`. To avoid this issue, use the `-o IdentityAgent=none -o IdentitiesOnly=yes` option for the `ssh` command or add it to your `ssh_config` file for the relevant hosts:
```
Host myserver.tld IdentityAgent none IdentitiesOnly yes
```

## 参考

- OpenSSH 密钥管理：[第一部分](https://www.funtoo.org/OpenSSH_Key_Management,_Part_1)，[第二部分](https://www.funtoo.org/OpenSSH_Key_Management,_Part_2)，[第三部分](https://www.funtoo.org/OpenSSH_Key_Management,_Part_3)
- [Secure Secure Shell](https://stribika.github.io/2015/01/04/secure-secure-shell.html)
```

8.6 OpenSSH

难度 高级 | **篇幅** 66.4 KB | **所属** SSH

本节将讲到

- 安装
- 客户端用法
- 服务端用法
- 小技巧
- 排障

```
![](https://wiki.archlinuxcn.org/wzh/File:Tango-preferences-desktop-locale.png)**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**
```

OpenSSH（OpenBSD Secure Shell）是使用安全外壳协议（SSH），在计算机网络上提供加密通信会话的软件集。它是 SSH Communications Security 专有 SSH 软件的开源替代方案。OpenSSH 是 OpenBSD 项目的一部分，该项目由 Theo de Raadt 领导。

OpenSSH 有时候会与名字相似的 OpenSSL 相混淆；然而，它们的目的不同，开发团队也不同，名字相似只因为两者目标相似。

安装

安装 openssh包。

客户端用法

连接服务器：

```
$ ssh -p 端口 用户名@服务器地址
```

如果服务器只允许公钥认证，参考 SSH 密钥。

配置

客户端可以配置一些通用的参数和主机信息。所有的参数都可以全局配置或者限制为特定主机，例如：

```
~/.ssh/config
# 全局选项
User 用户名

# 特定于主机的选项
Host myserver
    Hostname 服务器地址
    Port     端口
```

配置之后，下面的两条命令是等价的：

```
$ ssh -p 端口 用户名@服务器地址
$ ssh myserver
```

参考 ssh_config(5) 可以获得更多的信息。

一些参数没有对应的命令行选项，但你可以在命令行中使用 `-o` 指定配置选项。例如 `-oKexAlgorithms=+diffie-hellman-group1-sha1`。

服务端用法

`sshd` 是 OpenSSH 服务器守护程序，通过 `/etc/ssh/sshd_config` 配置并由 `sshd.service` 管理。每次更改配置后，请在重新启动服务前在测试模式下运行 `sshd` 以确保它能够干净地启动。有效配置将不会产生输出：

```
# sshd -t
```

配置

要仅允许某些用户访问，请添加以下一行：

```
AllowUsers    用户1 用户2
```

要仅允许某些用户组访问：

```
AllowGroups    用户组1 用户组2
```

要添加漂亮的欢迎消息（例如，输出 `/etc/issue` 文件），请配置 `Banner` 选项：

```
Banner /etc/issue
```

sshdgenkeys 服务会自动在 `/etc/ssh` 中生成缺失的任何算法的公钥和私钥，即使 `sshd_config` 中的 `HostKeyAlgorithms` 选项只允许部分加密算法。OpenSSH 提

供了三种基于 ed25519、ecdsa 和 rsa 算法的密钥对。要让 sshd 使用特定的密钥，请指定以下选项：

```
HostKey /etc/ssh/ssh_host_ed25519_key
```

如果服务器要暴露在公网下，建议将默认端口从 22 更改为更高的随机端口，例如：

```
Port 39901
```

提示：

- 要选择尚未分配给常见服务的备用端口，请查看 TCP 和 UDP 端口号列表，还可以在本地的 `/etc/services` 中查找端口信息。更改 OpenSSH 所使用的端口有助于减少由攻击者使用自动程序尝试通过 SSH 登录你的服务器的情况（但无法彻底解决）。查看 Port knocking 了解相关内容。
- 建议完全禁用密码登录。这将大大提高安全性，更多信息请参见#强制公钥认证。更多推荐的安全方法，请参阅#保护。
- OpenSSH 可以监听多个端口，只需在配置文件中添加多行 `Port *端口号*`。
- 可以通过从 `/etc/ssh` 中删除要替换的主机密钥对并以 root 身份运行 `ssh-keygen -A` 来生成新的（或缺失的）主机密钥对。

管理守护程序

启动/启用 `sshd.service`。它将保持 SSH 守护程序始终处于活动状态，并为每个传入连接 fork。

注意：因易受到拒绝服务攻击的影响，openssh包 8.0p1-3删除了 `sshd.socket`，更多信息请参考 FS#62248。如果在更新到 openssh包 8.0p1-3 时启用了 `sshd.socket`，则 `sshd.socket` 和 `sshd@.service` 单元将被复制到 `/etc/systemd/system/` 并重新启用。这样做只是为了不破坏现有设置；仍然建议用户迁移到 `sshd.service`。

警告：如果你继续使用 `sshd.socket`, 请注意下列问题:

- `sshd.socket` 单元可能会失败 (例如：在内存耗尽的情况下) 并且 `Restart=always` 不能被用于 socket 单元. 点击 systemd issue 11553 了解更多内容。
- 使用套接字激活会导致拒绝服务，因为太多的连接会导致拒绝进一步激活服务。见 FS#62248。

注意： `sshd.socket` 使得 `ListenAddress` 设置失效，因此它会允许连接任意地址。为了使 `ListenAddress` 设置起作用，你必须编辑 `sshd.socket` 来为 `ListenStream` 指定端口*和*IP（例如 `ListenStream=192.168.1.100:22`）。你还必须在 `[Socket]` 添加 `FreeBind=true`，否则设置 IP 地址会有 `ListenAddress` 设置一样的缺点：如果网络没有及时启动，套接字会启动失败。

提示：当使用套接字激活时，每个连接都会启动一个 `sshd@.service` 的瞬态实例（实例名称会不同）。因此，不论是 `sshd.socket` 还是守护进程常规的 `sshd.service` 都会不被允许在日志中监控连接尝试。可以通过 root 运行 `journalctl -u "sshd@*"` 或者 `journalctl /usr/bin/sshd` 查看 SSH 套接字激活实例的日志。

保护

允许通过 SSH 远程登录有利于管理，但可能会威胁服务器的安全。由于 SSH 访问通常是暴力攻击的目标，因此需要适当限制 SSH 访问以防止第三方访问服务器。

ssh-audit包 提供对服务端和客户端配置的自动分析。关于这个主题，还有其他几个很好的指南和工具，例如：

- Mozilla Infosec 团队的文章
- SSH Hardening Guides

强制公钥认证

如果客户端无法通过公钥进行身份验证，SSH 服务端默认会退回到使用密码进行身份验证，从而允许恶意用户尝试通过暴力穷举密码来获取访问权限。防止这种攻击的最有效方法之一是完全禁用密码登录，并强制使用 SSH 密钥。这可以通过在守护程序配置文件中设置以下选项来完成：

```
/etc/ssh/sshd_config.d/20-force_publickey_auth.conf
PasswordAuthentication no
AuthenticationMethods publickey
```

警告：在将此添加到配置之前，请确保所有需要 SSH 访问的帐户都在相应的 `authorized_keys` 文件中设置了公钥身份验证。详细信息请参阅 SSH 密钥#将公钥复制到远程服务器上。

双重身份认证和公钥

SSH 可以被设置成需要多重验证。你可以通过 `AuthenticationMethods` 选项指定使用哪些验证方式，这将同时启用公钥和双重验证。

身份验证提供程序

设置 Google 身份验证器的方法请查看 Google 身份验证器。

对于 Duo，安装提供 `pam_duo.so` 模块的包 duo_unixAUR 。阅读 Duo Unix 文档以了解如何设置 Duo 凭据（Integration Key，密钥和 API 主机名）。

PAM 设置

**本文或本章节的事实准确性存在争议。**

原因： The distribution defaults to `KbdInteractiveAuthentication no` since . Later, the lexical order of the defaults was adjusted due to FS#79285 to allow for higher priority user snippets, which would match below `20-pam.conf`. However, a BBS thread appears to be resolved by ordering custom after `99-archlinux.conf` defaults only.（在 Talk:OpenSSH 中讨论）

要在 OpenSSH 中使用 PAM，请编辑下列文件：

```
/etc/ssh/sshd_config.d/20-pam.conf
KbdInteractiveAuthentication yes
AuthenticationMethods publickey keyboard-interactive:pam
```

然后你可以通过公钥或在 PAM 中设置的用户身份验证方法登录。

另一方面，如果您想要根据 PAM 设置的要求使用公钥和用户身份验证对用户进行验证，请使用逗号而不是空格来分隔 AuthenticationMethods：

```
/etc/ssh/sshd_config.d/20-pam.conf
KbdInteractiveAuthentication yes
AuthenticationMethods publickey,keyboard-interactive:pam
```

为了要求同时使用公钥和 pam 身份验证，您可能希望禁用密码登录：

```
/etc/pam.d/sshd
auth       required  pam_securetty.so     #disable remote root
#Require google authenticator
auth       required  pam_google_authenticator.so
#But not password
#auth       include   system-remote-login
account    include   system-remote-login
password   include   system-remote-login
session    include   system-remote-login
```

防止暴力穷举攻击

暴力攻击的概念非常简单：使用大量随机用户名和密码组合尝试登录网页或类似 SSH 等服务。

请参考 ufw#连接速率限制，如果使用的是 iptables，请参考 Simple stateful firewall#Bruteforce attacks。

从 9.8 版本开始，OpenSSH 引入了类似 fail2ban 的基本保护实现：`PerSourcePenalties` 选项默认配置了一个合理值，会对同一地址上的客户端针对各种条件进行惩罚，导致在一段时间内被拒绝连接。

另外，你也可以使用自动化脚本封禁任何尝试暴力穷举登录的对象。

- 只允许可信源地址进行 SSH 连接
- 使用 fail2ban 或 sshguard 自动封禁多次密码登录失败的 IP
- 使用 pam_shield 封禁一段时间内多次尝试登录的 IP。与 fail2ban 或 sshguard 不同，该程序不会对成功和失败登录请求进行区分。

限制root用户登录

It is generally considered bad practice to allow the root user to log in without restraint over SSH. There are two methods by which SSH root access can be restricted for increased security.

拒绝

Sudo selectively provides root rights for actions requiring these without requiring authenticating against the root account. This allows locking the root account against access via SSH and potentially functions as a security measure against brute force attacks, since now an attacker must guess the account name in addition to the password.

SSH can be configured to deny remote logins with the root user by editing the "Authentication" section in the daemon configuration file. Simply set `PermitRootLogin` to `no`:

```
/etc/ssh/sshd_config.d/20-deny_root.conf
PermitRootLogin no
```

Next, restart the SSH daemon.

You will now be unable to log in through SSH under root, but will still be able to log in with your normal user and use su or sudo to do system administration.

限制

Some automated tasks such as remote, full-system backup require full root access. To allow these in a secure way, instead of disabling root login via SSH, it is possible to only allow root logins for selected commands. This can be achieved by editing `~root/.ssh/authorized_keys`, by prefixing the desired key, e.g. as follows:

```
command="rrsync -ro /" ssh-ed25519 ...
```

This will allow any login with this specific key only to execute the command specified between the quotes.

The increased attack surface created by exposing the root user name at login can be compensated by adding the following to `sshd_config`:

```
PermitRootLogin forced-commands-only
```

This setting will not only restrict the commands which root may execute via SSH, but it will also disable the use of passwords, forcing use of public key authentication for the root account.

A slightly less restrictive alternative will allow any command for root, but makes brute force attacks infeasible by enforcing public key authentication. For this option, set:

```
PermitRootLogin prohibit-password
```

对 authorized_keys 文件加锁

警告：Locking this file only protects against user mistakes and a particular naive in-person attack. It **does not** provide any protection against malicious programs or breaches. Use multi-factor authentication, firewalling and practice defence in depth to prevent breaches in the first place.

If, for whatever reason, you think that the user in question should not be able to add or change existing keys, you can prevent them from manipulating the file.

On the server, make the `authorized_keys` file read-only for the user and deny all other permissions:

```
$ chmod 400 ~/.ssh/authorized_keys
```

To prevent the user from simply changing the permissions back, set the immutable bit on the `authorized_keys` file. To prevent the user from renaming the `~/.ssh` directory and creating a new `~/.ssh` directory and `authorized_keys` file, set the immutable bit on the `~/.ssh` directory too. To add or remove keys, you will have to remove the immutable bit from `authorized_keys` and make it writable temporarily.

提示： It is recommended to log changes to any `authorized_keys` file via e.g auditd.

SSH certificates

While common SSH keys and manual fingerprint verification may be easy to use with a handful of hosts that are managed by a single administrator, this method of authentication does not scale at all. When a number of servers need to be accessed through SSH by several users, manually verifying ssh public key fingerprints of every host becomes nearly impossible to do securely and reliably.

The solution for this is to use SSH certificates that provide automatic verification of public key identities through a chain of trust that scales significantly better than the default trust-on-first-use approach of SSH. SSH certificates are basically nothing else than normal public SSH keys, but with an additional signature from a trusted certificate authority that verifies the key identity.

Create a host certificate authority key for your infrastructure

```
$ ssh-keygen -t ed25519 -f ~/.ssh/ca_host_key -C 'Host certificate authority for *.example.com'
```

The private certificate authority key should be stored securely, ideally on a smartcard or hardware token that prevents key extraction like the Nitrokey or YubiKey.

Sign a server's public SSH host key

Copy the public server key to your local system containing the private certificate authority key to sign it:

```
$ ssh-keygen -h -s ~/.ssh/ca_key -I certLabel -n server01.example.com ./ssh_host_ed25519_key.pub
```

Move the new certificate and configure sshd to use it

The generated certificate `ssh_host_ed25519_key-cert.pub` should be copied to the server at `/etc/ssh/`.

```
/etc/ssh/sshd_config.d/20-ed25519_key.conf
HostCertificate /etc/ssh/ssh_host_ed25519_key-cert.pub
```

Configure all clients to trust the certificate authority

```
~/.ssh/known_hosts
@cert-authority  *.example.com ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKL8gB/pjuff005YNazwMCqJpgsXAbQ3r4VStd/CRKwU Host certificate authority for *.example.com
```

警告： When a server does not provide a certificate for identification, public key authentication is used as fallback by default.

SSH user certificates

Depending on the number of users and method of deployment, SSH User keys can also be used with Certificates. For organizations with many ssh users, this is strongly advised to manage User key deployment securely.

The deployment of user certificates works basically the same as for server identities. More details and instructions can be found at Wikibooks:OpenSSH/Cookbook/Certificate-based Authentication.

Certificate deployment automation

Automated deployment of SSH certificates can be provided by a number of open source tools. Popular examples are:

- Ansible - openssh_cert module
- privacyIDEA - authentication server
- Theo App - authorized keys manager

SSHFP record

The Secure Shell fingerprint record (SSHFP) is an optional resource record in the domain name system that associates SSH keys to a host name. It can be used to verify the SSH fingerprint on public servers by using DNSSEC instead of deploying trusted CA certificates, which allows even unmanaged clients to verify the validity of key fingerprints.

Generate record entry

To generate the required hexadecimal key fingerprint to be stored in the DNS record, create the hash on the target server.

```
$ ssh-keygen -r host.example.com
```

This will read all available SSH keys for the specified domain and output valid SSHFP records that can then be stored in the DNS entries of the affected domain.

Client configuration

In order to automatically retrive and trust SSH key fingerprints stored as SSHFP records, add the following to your ssh client configuration file:

```
~/.ssh/config
# global options
Match all
    VerifyHostKeyDNS yes
```

If the target host has a valid SSHFP record and this record is verified with a valid DNSSEC signature, the fingerprint is automatically accepted without prompting the user to verify the hosts identity. In case the DNS record is not verified by DNSSEC, the user will be prompted to verify the fingerprint instead.

Generate SSHFP records

To determine the SSH fingerprint of a specific domain, use *ssh-keyscan* to retrieve the ssh fingerprints in a valid DNS record format. (Note that by default fingerprints for every available key type is provided as both SHA1 and SHA256.)

```
$ ssh-keyscan -D github.com
; github.com:22 SSH-2.0-babeld-57ca1323
; github.com:22 SSH-2.0-babeld-57ca1323
github.com IN SSHFP 1 1 6f4c60375018bae0918e37d9162bc15ba40e6365
github.com IN SSHFP 1 2
b8d895ced92c0ac0e171cd2ef5ef01ba3417554a4a6480d331ccc2be3ded0f6b
; github.com:22 SSH-2.0-babeld-57ca1323
github.com IN SSHFP 3 1 3358ab5dd3e306c461c840f7487e93b697e30600
github.com IN SSHFP 3 2
a764003173480b54c96167883adb6b55cf7cfd1d415055aedff2e2c8a8147d03
; github.com:22 SSH-2.0-babeld-57ca1323
github.com IN SSHFP 4 1 e9619e2ed56c2f2a71729db80bacc2ce9ccce8d4
github.com IN SSHFP 4 2
f83898df0bef57a4ee24985ba598ac17fccb0c0d333cc4af1dd92be14bc23aa5
```

```
; github.com:22 SSH-2.0-babeld-57ca1323
```
Since the SSHFP record stores the key fingerprints as hexadecimal values while the common output for SSH fingerprints is the base64 encoded SHA256 hash of the public key, it is necessary to convert the record back to the base64 format in order to compare it with values in the known_hosts file or other documentation that commonly stores fingerprints as SHA256.

```
$ echo "SSHFP-fingerprint" | xxd -r -p | base64
```
Example for github.com using the hex value for the sha256 fingerprint of the key type ed25519

```
$ echo "f83898df0bef57a4ee24985ba598ac17fccb0c0d333cc4af1dd92be14bc23aa5" | xxd -r -p | base64
+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU=
```
Compare with known_hosts entries:

```
$ ssh-keygen -l -f ~/.ssh/known_hosts
```
Manually retrieve SSHFP records from DNS
```
$ dig SSHFP targetdomain.tld +short
```

小技巧

使用加密的 SOCKS 隧道

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：Written like a blog post.（在Talk:OpenSSH讨论）

这对连接到各种不安全无线连接的笔记本电脑用户非常有用。你唯一需要的是在一个比较安全的位置运行的 SSH 服务器，比如你的家或工作地点。使用动态 DNS 服务可能很有用，如DynDNS，这样你就不必记住你的 IP 地址。

步骤 1：开始连接

你只需要执行这一条命令就可以开始连接：

```
$ ssh -TND 4711 user@host
```
其中 `*user*` 是你在 `*host*` 上运行的 SSH 服务器的用户名。它将询问你的密码，然后建立连接。`N` 标志禁用了交互式提示，`D` 标志指定了监听的本地端口（如果你愿意，可以选择任何端口号）。{Ic|T}} 标志禁用伪终端分配。

可以添加 verbose（`-v`）标志，这样就可以通过输出验证是否成功连接。

步骤 2 (方法 A)：配置你的浏览器（或其它程序）

上述步骤仅与使用新创建的 SOCKS 隧道的网页浏览器或其他程序结合使用时有用。SSH 现在同时支持 SOCKS v4 和 SOCKS v5，你可以选择它们中的任意一种。

- 对于火狐浏览器：进入*首选项* > *通用*，跳到页面底部并点击网络设置项右边的*设置...*按钮。接下来，在打开的二级窗口中，选中*手动代理配置*并在 SOCKS

主机名框内输入 `localhost`，并在旁边的*端口*项输入端口号（上述例子使用 `4711`）。

: 火狐不会自动通过 SOCKS 隧道发出 DNS 请求，该潜在隐私问题可通过选中页面下方的*使用 SOCKS v5 时代理 DNS 查询*解决。显然，这只在使用 SOCKS v5 而不是 v4 时有效。 : 重启火狐来应用设置。

- 对于 Chromium，可以将 SOCKS 设置为环境变量或通过命令行选项进行配置。例如，可以在 `.bashrc` 中添加以下函数：

```
function secure_chromium {
    port=4711
    export SOCKS_SERVER=localhost:$port
    export SOCKS_VERSION=5
    chromium &
    exit
}
```

或者是

```
function secure_chromium {
    port=4711
    chromium --proxy-server="socks://localhost:$port" &
    exit
}
```

现在打开终端，然后执行：

```
$ secure_chromium
```

就可以使用安全隧道了！

步骤 2（方法 B）：配置本地 TUN 接口

这种方法在前期稍微复杂，但以后无需手动配置每个应用程序来使用 SOCKS 代理，具体原理是建立一个本地 TUN 接口并通过该接口路由流量。

具体方法请参考 VPN over SSH#Set up badvpn and tunnel interface。

X11 转发

使用 X11 转发可以在本地客户端上显示远程系统运行的 X11 程序图形界面。对于 X11 转发，远程主机不需要安装完整的 X11 环境，但至少要安装 *xauth*。*xauth*是维护 `Xauthority` 配置的工具，该配置被服务端和客户端用于认证 X11 会话（源）。

警告：X11 转发在安全方面有很强的影响，需至少要阅读 ssh(1)、sshd_config(5) 和 ssh_config(5) 手册来了解相关信息。另外也请参考该 StackExchange 提问。

设置

服务端

- 安装 xorg-xauth包
- 在 `/etc/ssh/ssh**d**_config` 中：

- 设置 `X11Forwarding` 为 *yes*
- 检查 `AllowTcpForwarding` 和 `X11UseLocalhost` 选项是否已设为 *yes*，并且 `X11DisplayOffset` 设为 *10*（未修改情况下是默认值，参考 sshd_config(5)）
- 然后重启 *sshd* daemon.

客户端

- 安装 xorg-xauth包
- 启用 `ForwardX11` 选项，可以在命令行中使用 `-X` 选项，也可以在客户端配置文件中设置 `ForwardX11` 为 *yes*。

提示：如果 GUI 显示质量不佳或者出现错误，可以启用 `ForwardX11Trusted` 选项（命令行下使用 `-Y`），这会防止 X11 转发受到 X11 SECURITY extension 的影响。在操作前，确保你已经看过警告信息。

用法

正常登录到远程机器，如果客户端的配置文件中没有启用 *ForwardX11*，则指定 `-X` 开关：

```
$ ssh -X user@host
```

如果你在试图运行图形应用程序时收到错误，请尝试用 *ForwardX11Trusted* 代替：

```
$ ssh -Y user@host
```

如果出现 `X11 forwarding request failed` 报错，则需要为你的远程机器重新进行设置。一旦 X11 转发请求成功，你可以在远程服务器上启动任何 X 程序，它将被转发到你的本地会话：

```
$ xclock
```

如果出现了 `Can't open display` 报错，表示 `DISPLAY` 设置有误。

对一些应用程序要小心，因为它们会检查本地机器上是否有正在运行的实例。Firefox 就是一个例子：要么关闭正在运行的 Firefox 实例，要么使用下面的启动参数在本地机器上启动一个远程实例：

```
$ firefox --no-remote
```

如果你在连接时出现“X11 forwarding request failed on channel 0”报错（并且服务器 `/var/log/errors.log` 显示“Failed to allocate internet-domain X11 display socket”），请确保已安装 xorg-xauth包。如果安装了也还是不行，可以尝试以下方法之一：

- 在*服务端*的 `ssh**d**_config` 中启用 `AddressFamily any` 选项，或者
- 在*服务端*的 `ssh**d**_config` 中将 `AddressFamily` 选项设置为 inet。

将其设置为 inet 可能会解决 Ubuntu 客户端在 IPv4 上的问题。

对于在 SSH 服务器上以另一个用户身份运行 X 应用程序，you need to `xauth add` the authentication line taken from `xauth list` of the SSH logged in user.

提示：关于 `X11` 转发 排障可参考以下链接：

- https://unix.stackexchange.com/a/12772
- https://unix.stackexchange.com/a/46748
- https://superuser.com/a/805060

转发其他端口

除了SSH对X11的内置支持外，它还可以通过本地转发或远程转发使用安全隧道传输任何 TCP 连接。

本地转发打开本地计算机上的一个端口，连接将被转发到远程主机，并从那里转发到给定的目的地。通常，转发目的地将与远程主机相同，从而为同一机器提供安全外壳，例如安全 VNC 连接。本地转发会使用到 `-L` 选项，后接 `<tunnel port>:<destination address>:<destination port>` 格式的转发规则。

因此：

```
$ ssh -L 1000:mail.google.com:25 192.168.0.100
```

将使用 SSH 进行登录到 `192.168.0.100` 并打开 shell，同时创建本地 TCP 端口 1000 到 mail.google.com 端口 25 的隧道。连接成功后，发送到 `localhost:1000` 的连接会连到 Gmail 的 SMTP 端口。除非使用了其它手段，否则对于谷歌来说，他们会观察到所有这些连接都是从 `192.168.0.100` 发送出来的（但对于连接传送的数据又是另一回事），而不是 `192.168.0.100`。

类似地:

```
$ ssh -L 2000:192.168.0.100:6001 192.168.0.100
```

将允许连接到 `localhost:2000`，然后数据会被透明发送到远程主机的 6001 端口。The preceding example is useful for VNC connections using the vncserver utility--part of the tightvnc package--which, though very useful, is explicit about its lack of security.

Remote forwarding allows the remote host to connect to an arbitrary host via the SSH tunnel and the local machine, providing a functional reversal of local forwarding, and is useful for situations where, e.g., the remote host has limited connectivity due to firewalling. It is enabled with the `-R` switch and a forwarding specification in the form of `<tunnel port>:<destination address>:<destination port>`.

Thus:

```
$ ssh -R 3000:irc.libera.chat:6667 192.168.0.200
```

will bring up a shell on `192.168.0.200`, and connections from `192.168.0.200` to itself on port 3000 (the remote host's `localhost:3000`) will be sent over the tunnel to the local machine and then on to irc.libera.chat on port 6667, thus, in this example, allowing the use of IRC programs on the remote host to be used, even if port 6667 would normally be blocked to it.

Both local and remote forwarding can be used to provide a secure "gateway", allowing other computers to take advantage of an SSH tunnel, without actually running SSH or the SSH daemon by providing a bind-address for the start of the tunnel as part of the forwarding specification, e.g. `<tunnel address>:<tunnel port>:<destination address>:<destination port>`. The `<tunnel address>` can be any address on the machine at the start of the tunnel. The address `localhost` allows connections via the `localhost` or loopback interface, and an empty address or `*` allow connections via any interface. By default, forwarding is limited to connections from the machine at the "beginning" of the tunnel, i.e. the `<tunnel address>` is set to `localhost`. Local forwarding requires no additional configuration; however, remote forwarding is limited by the remote server's SSH daemon configuration. See the `GatewayPorts` option in sshd_config(5) and `-L address` option in ssh(1) for more information about remote forwarding and local forwarding, respectively.

跳板机

有时，可能无法直接连接到目标的 SSH 守护程序，因而需要使用一个跳转服务器（或堡垒服务器）。为此，我们试图将两个或更多的 SSH 隧道连接在一起，并假设你的本地密钥对链上的每个服务器都有授权。这可以通过 SSH 代理转发（`-A`）和使用伪终端分配（`-t`）转发你的本地密钥实现：

```
$ ssh -A -t -l user1 bastion1 \
  ssh -A -t -l user2 intermediate2 \
  ssh -A -t -l user3 target
```

这一流程可以使用 ProxyCommand 选项进行自动化：

```
$ ssh -o ProxyCommand="ssh -W %h:%p bastion.example.org" targetserver.example.org
```

一个更简单和更安全的方法是将 ProxyJump 选项结合 `-J` 使用：

```
$ ssh -J user1@bastion1,user2@intermediate2 user3@target
```

`-J` 指令中的多个主机可以用逗号隔开，它们将按照列出的顺序被连接。`user...@` 部分不是必须的，但可以使用。The host specifications for `-J` use the ssh configuration file, so specific per-host options can be set there, if needed.

The main difference between the ProxyCommand and ProxyJump options is that the later does not require a shell on the jumphost. Consequently, this also means that the jumpserver does not require access to the users login credentials or SSH agent forwarding. With the ProxyJump option, the ssh client connects through the

jumpserver directly to the target server, establishing an end-to-end encrypted channel between client and target server.

`-J` 标志相当于配置文件中 `ProxyJump` 选项，相关细节请参考 ssh_config(5)。

Reverse SSH through a relay

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：The idea of SSH tunneling is classic, so some references for detailed explanation would be nice. E.g. which includes other scenarios.（在Talk:OpenSSH讨论）

The idea is that the client connects to the server via another relay while the server is connected to the same relay using a reverse SSH tunnel. This is useful when the server is behind a NAT, and the relay is a publicly accessible SSH server used as a proxy to which the user has access. Therefore, the prerequisite is that the client's keys are authorized against both the relay and the server, and the server needs to be authorized against the relay as well for the reverse SSH connection.

The following configuration example assumes that user1 is the user account used on client, user2 on relay and user3 on server. First, assuming we will use port 2222, the server needs to establish the reverse tunnel with:

```
ssh -R 2222:localhost:22 -N user2@relay
```

Which can also be automated with a startup script, systemd service, autossh or sidedoorAUR.

At the client side, the connection is established with:

```
ssh -t user2@relay ssh user3@localhost -p 2222
```

注意：`ssh *user3*@*relay* -p 2222` would require you to open this port in the firewall of your relay server, as well as to allow connections to this port from other addresses.

The remote command to establish the connection to reverse tunnel can also be defined in relay's `~/.ssh/authorized_keys` by including the `command` field as follows:

```
command="ssh user3@localhost -p 2222" ssh-ed25519 KEY2 user1@client
```

In this case the connection is established with:

```
ssh user2@relay
```

Alternatively, you can add an entry to your ssh configuration that specifies both `RemoteCommand` and `RequestTTY`:

```
~/.ssh/config
Host jump-destination
    Hostname relay
    User user2
    RemoteCommand ssh user3@localhost -p 2222
    RequestTTY yes
```

Which will reduce connecting to:

```
ssh jump-destination
```

注意：SCP's autocomplete function in client's terminal will not work and even the SCP transfers themselves are not working under some configurations.

多路复用

SSH 守护进程通常监听 22 端口。然而，许多公共热点会屏蔽所有不在常规 HTTP/S 端口（80 和 443）的流量，从而阻止了 SSH 连接。最直接解决这个问题的办法是让 `sshd` 额外监听白名单中的一个端口。

```
/etc/ssh/sshd_config
Port 22
Port 443
```

然而，443 端口很可能已经被提供 HTTPS 内容的网页服务器使用，在这种情况下，可以使用类似 sslh包 的多路复用器，它会监听多路复用端口，并智能地将数据包转发给多个服务。

SSH 提速

有多个客户端配置项可以全局或为指定的主机提升连接性能。关于这些选项的完整描述请参考 ssh_config(5)。

- *使用更快的加密算法*：在带有 AESNI 指令集的现代 CPU 上，使用 `aes128-gcm@openssh.com` 和 `aes256-gcm@openssh.com` 替代 openssh 的默认加密方法（通常是 `chacha20-poly1305@openssh.com`）应可以显著提升性能。在命令行中可以通过 `-c` 选项选择加密算法。如果要将配置持久化，可以在 `~/.ssh/config` 中添加 `Ciphers` 选项，然后按照偏好顺序填入加密算法，例如：

```
    Ciphers aes128-gcm@openssh.com,aes256-gcm@openssh.com,chacha20-
poly1305@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr
```

- *启用或禁用压缩*：在低速连接上启用压缩可以提升性能，该功能可通过 `Compression yes` 选项或 `-C` 标志进行启用。但是，该选项使用的压缩算法是 gzip(1)，相对较慢，在高速网络上可能会成为瓶颈。为提高性能，在本地或高速网络上应使用 `Compression no` 选项。
- *共享连接*：可以使用以下选项为连到同一主机的所有会话共用一个连接：

```
ControlMaster auto
ControlPersist yes
ControlPath ~/.ssh/sockets/socket-%r@%h:%p
```

: 其中 `~/.ssh/sockets` 可以是任何其他用户不可写的目录。

- `ControlPersist` 指定了在客户端连接关闭后，需要在后台等待新客户端连接的时间。可选值包括：
- `no`：最后一个客户端断开后立即关闭连接
- 以秒指定的时间

- `yes`：始终等待，连接永远不会被自动关闭
- 可以使用 `AddressFamily inet` 选项或 `-4` 标志绕过 IPv6 查找来缩短登录时间。
- 另外，如果有将 SSH 用于 SFTP 或 SCP 的需求，高性能 SSH/SCP 可以通过动态提升 SSH 缓存大小来显著提升传输速度。可以安装 openssh-hpnAUR 来使用带有该补丁的 OpenSSH。

使用 SSHFS 挂载远程文件系统

关于将 SSH 可访问的远程系统挂载到本地目录的内容请查看 SSHFS 文章。这样您就可以使用任何工具对挂载的文件执行任何操作（复制、重命名、使用 vim 编辑等）。*sshfs* 一般优于 *shfs*，后者自 2004 年以来一直没有更新。

保活

默认情况下，如果 SSH 会话空闲了一段时间，它将自动注销。为了保持会话，如果在一段时间内没有收到数据，客户端可以向服务器发送保持活动的信号，或者如果服务器没有收到客户端的消息，则可以对称地定期发送消息。

- 在**服务端**，`ClientAliveInterval` 以秒为单位设置超时，如果没有从客户端接收到数据，*sshd* 将发送响应请求。默认值为 0（不发送消息）。例如，要每 60 秒从客户端请求一次响应，请在服务器配置中设置 `ClientAliveInterval 60` 选项。另见 `ClientAliveCountMax` 和 `TCPKeepAlive` 选项。
- 在 '*客户端*，`ServerAliveInterval` 控制从客户端发送到服务器的响应请求之间的间隔。例如，要请求服务器每 120 秒响应一次，请将 `ServerAliveInterval 120` 选项添加到客户端配置中。另请参见 `ServerAliveCountMax` 和 `TCPKeepAlive` 选项。

注意：为了确保会话保持活动状态，客户机或服务器中只有一个需要发送 keep alive 请求。如果同时控制服务器和客户机，一个合理的选择是只在需要持久会话的客户机上配置 `ServerAliveInterval`，并使其他客户机和服务器保持默认配置。

Automatically restart SSH tunnels with systemd

systemd can automatically start SSH connections on boot/login *and* restart them when they fail. This makes it a useful tool for maintaining SSH tunnels.

The following service can start an SSH tunnel on login using the connection settings in your ssh configuration. If the connection closes for any reason, it waits 10 seconds before restarting it:

```
~/.config/systemd/user/tunnel.service
[Unit]
Description=SSH tunnel to myserver
```

```
[Service]
Type=simple
Restart=always
RestartSec=10
ExecStart=/usr/bin/ssh -F %h/.ssh/config -N myserver
```

Then enable and start the Systemd/User service. See #Keep alive for how to prevent the tunnel from timing out. If you wish to start the tunnel on boot, you might want to rewrite the unit as a system service.

Autossh - automatically restarts SSH sessions and tunnels

When a session or tunnel cannot be kept alive, for example due to bad network conditions causing client disconnections, you can use autossh包 to automatically restart them.

Usage examples:

```
$ autossh -M 0 -o "ServerAliveInterval 45" -o "ServerAliveCountMax 2" username@example.com
```

Combined with SSHFS:

```
$ sshfs -o reconnect,compression=yes,transform_symlinks,ServerAliveInterval=45,ServerAliveCountMax=2,ssh_command='autossh -M 0' username@example.com: /mnt/example
```

Connecting through a SOCKS-proxy set by Proxy settings:

```
$ autossh -M 0 -o "ServerAliveInterval 45" -o "ServerAliveCountMax 2" -NCD 8080 username@example.com
```

With the `-f` option autossh can be made to run as a background process. Running it this way however means the passphrase cannot be entered interactively.

The session will end once you type `exit` in the session, or the autossh process receives a SIGTERM, SIGINT of SIGKILL signal.

Run autossh automatically at boot via systemd

If you want to automatically start autossh, you can create a systemd unit file:

```
/etc/systemd/system/autossh.service
[Unit]
Description=AutoSSH service for port 2222
After=network.target

[Service]
Environment="AUTOSSH_GATETIME=0"
ExecStart=/usr/bin/autossh -M 0 -NL 2222:localhost:2222 -o TCPKeepAlive=yes foo@bar.com

[Install]
WantedBy=multi-user.target
```

Here `AUTOSSH_GATETIME=0` is an environment variable specifying how long ssh must be up before autossh considers it a successful connection, setting it to 0 autossh also ignores the first run failure of ssh. This may be useful when running autossh at boot. Other environment variables are available at autossh(1). Of course, you can make this unit more complex if necessary (see the systemd documentation for details), and obviously you can use your own options for autossh, but note that the `-f` implying `AUTOSSH_GATETIME=0` does not work with systemd.

Remember to start and/or enable the service afterwards.

You may also need to disable ControlMaster e.g.

```
ExecStart=/usr/bin/autossh -M 0 -o ControlMaster=no -NL 2222:localhost:2222 -o
TCPKeepAlive=yes foo@bar.com
```

提示：It is also easy to maintain several autossh processes, to keep several tunnels alive. Just create multiple service files with different names.

Alternative service should SSH daemon fail

For remote or headless servers which rely exclusively on SSH, a failure to start the SSH daemon (e.g., after a system upgrade) may prevent administration access. systemd offers a simple solution via `OnFailure` option.

Let us suppose the server runs `sshd` and telnet is the fail-safe alternative of choice. Create a file as follows. Do **not** enable `telnet.socket`!

```
/etc/systemd/system/sshd.service.d/override.conf
[Unit]
OnFailure=telnet.socket
```

That's it. Telnet is not available when `sshd` is running. Should `sshd` fail to start, a telnet session can be opened for recovery.

Terminal background color based on host

为了更好地区分在不同主机上的情况，可以设置基于主机类型的不同背景颜色。

这种解决方案仅限于 ZSH。

Network specific configuration

You can use host configuration specific to the network you are connected to using a `Match exec`.

For example, when using nmcli(1), and the connection is configured (manually or through DHCP) to use a search-domain:

```
Match exec "nmcli | grep domains: | grep example.com"
  CanonicalDomains example.com
  # Should you use a different username on this network
  #User username
  # Use a different known_hosts file (for private network or synchronisation)
  #UserKnownHostsFile <network>_known_hosts
```

Another example for `Match host ... exec "..."`: Consider that connecting to `internal.example.com` requires a bastion/proxy (via `ProxyJump`) unless you are already connected via VPN. The fragment `!exec "host internal.example.com"` applies only when `internal.example.com` cannot be looked up via DNS. Various alternatives are discussed at .

```
Match host internal.example.com !exec "host internal.example.com"
  ProxyJump bastion.example.com
Host internal.example.com
  User foobar
```

Private networks hostkeys verification

Because different servers on different networks are likely to share a common private IP address, you might want to handle them differently.

**本文或本章节的事实准确性存在争议。**

原因： The best solution would not need a warning to use something else in practice.（在 Talk:OpenSSH 中讨论）

The best solution is to use the #Network specific configuration to use a different `UserKnownHostsFile` depending on the network you are on. The second solution, best used as default when you are working on new/prototype networks, would be to simply ignore hostkeys for private networks:

```
Host 10.* 192.168.*.* 172.31.* 172.30.* 172.2?.* 172.1?.*
    # Disable HostKey verification
    # Trust HostKey automatically
    StrictHostKeyChecking no
    # Do not save the HostKey
    UserKnownHostsFile=/dev/null
    # Do not display: "Warning: Permanently Added ..."
    LogLevel Error
```

**本文或本章节的事实准确性存在争议。**

原因： The `known_hosts` file records an IP address even when you use hostname to access the server.（在 Talk:OpenSSH 中讨论）

警告：In a production environment, make sure to either use the hostname to access the host and/or to use network specific known_hosts files.

在登录时运行命令

如果你使用的是交互式会话，有多种方法可以在登录时执行一个命令：

- 使用远程主机上的 `authorized_keys` 文件（参见 sshd(8) § AUTHORIZED_KEYS FILE FORMAT）
- 如果服务器启用了 `~/.ssh/rc` 选项，则使用远程主机上的 `PermitUserRC` 选项
- 使用远程主机上的 shell 配置文件，例如 `.bashrc`。

转发代理

SSH 代理转发允许你在连接到一个服务器时使用你的本地密钥。建议只对选定的主机启用代理转发。

```
~/.ssh/config
Host myserver.com
    ForwardAgent yes
```

接下来，配置 SSH 代理并使用 *ssh-add* 添加你的本地密钥。

如果你现在连接到一个远程服务器，你将能够使用你的本地密钥连接到其他服务。

生成新的密钥对

新的服务器私钥可以通过以下方式生成:

1. 删除已有的所有密钥，例如：

```
# rm /etc/ssh/ssh_host_*_key*
```

1. 重启 `sshdgenkeys.service` 或者以根用户身份运行 `ssh-keygen -A`

Run sshd as non-privileged user

You may want to run `sshd` as non-privileged user in containers, or for testing, etc.

Since non-privileged user cannot read host keys in `/etc/ssh`, new host keys must be generated:

```
$ ssh-keygen -q -N "" -t rsa -b 4096 -f /path/to/host/keys/ssh_host_rsa_key
$ ssh-keygen -q -N "" -t ecdsa -f /path/to/host/keys/ssh_host_ecdsa_key
$ ssh-keygen -q -N "" -t ed25519 -f /path/to/host/keys/ssh_host_ed25519_key
```

Create an `sshd_config` file. The example below uses a port higher than 1024, provides a new path to the host keys and disables PAM:

```
/path/to/sshd_config
Port 2022
HostKey /path/to/host/keys/ssh_host_rsa_key
HostKey /path/to/host/keys/ssh_host_ecdsa_key
HostKey /path/to/host/keys/ssh/ssh_host_ed25519_key
UsePAM no
```

Run *sshd* with the created config. The `-D` flag disables daemon mode and `-e` redirects output to stderr to allow easy monitoring.

```
$ sshd -f /path/to/sshd_config -D -e
```

排障

Checklist

Check these simple issues before you look any further.

1. The configuration directory `~/.ssh`, its contents should be accessible only by the user (check this on both the client and the server), and the user's home directory should only be writable by the user:

```
$ chmod go-w ~
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/*
$ chown -R $USER ~/.ssh
```

1. Check that the client's public key (e.g. `id_ed25519.pub`) is in `~/.ssh/authorized_keys` on the server.
2. Check that you did not limit SSH access with `AllowUsers` or `AllowGroups` in the server config.
3. Check if the user has set a password. Sometimes new users who have not yet logged in to the server do not have a password.
4. Append `LogLevel DEBUG` to `/etc/ssh/sshd_config`.
5. Run `journalctl -xe` as root for possible (error) messages.

6. Restart `sshd` and logout/login on both client and server.

Connection refused or timeout problem

Port forwarding

If you are behind a NAT mode/router (which is likely unless you are on a VPS or publicly addressed host), make sure that your router is forwarding incoming ssh connections to your machine. Find the server's internal IP address with `$ ip addr` and set up your router to forward TCP on your SSH port to that IP. portforward.com can help with that.

Is SSH running and listening?

The ss utility shows all the processes listening to a TCP port with the following command line:

```
$ ss --tcp --listening
```

If the above command do not show the system is listening to the port `ssh`, then SSH is not running: check the journal for errors etc.

Are there firewall rules blocking the connection?

Iptables may be blocking connections on port `22`. Check this with:

```
# iptables -nvL
```

and look for rules that might be dropping packets on the `INPUT` chain. Then, if necessary, unblock the port with a command like:

```
# iptables -I INPUT 1 -p tcp --dport 22 -j ACCEPT
```

For more help configuring firewalls, see firewalls.

Is the traffic even getting to your computer?

Start a traffic dump on the computer you are having problems with:

```
# tcpdump -lnn -i any port ssh and tcp-syn
```

This should show some basic information, then wait for any matching traffic to happen before displaying it. Try your connection now. If you do not see any output when you attempt to connect, then something outside of your computer is blocking the traffic (e. g., hardware firewall, NAT router etc.).

Your ISP or a third party blocking default port?

注意：Try this step if you **know** you are not running any firewalls and you know you have configured the router for DMZ or have forwarded the port to your computer and it still does not work. Here you will find diagnostic steps and a possible solution.

In some cases, your ISP might block the default port (SSH port 22) so whatever you try (opening ports, hardening the stack, defending against flood attacks, et al) ends up useless. To confirm this, create a server on all interfaces (0.0.0.0) and connect remotely.

If you get an error message comparable to this:

```
ssh: connect to host www.inet.hr port 22: Connection refused
```

That means the port is **not** being blocked by the ISP, but the server does not run SSH on that port (See security through obscurity).

However, if you get an error message comparable to this:

```
ssh: connect to host 111.222.333.444 port 22: Operation timed out
```

That means that something is rejecting your TCP traffic on port 22. Basically that port is stealth, either by your firewall or 3rd party intervention (like an ISP blocking and/or rejecting incoming traffic on port 22). If you know you are not running any firewall on your computer, and you know that Gremlins are not growing in your routers and switches, then your ISP is blocking the traffic.

To double check, you can run Wireshark on your server and listen to traffic on port 22. Since Wireshark is a Layer 2 Packet Sniffing utility, and TCP/UDP are Layer 3 and above (see IP Network stack), if you do not receive anything while connecting remotely, a third party is most likely to be blocking the traffic on that port to your server.

Diagnosis

Install either tcpdump包 or Wireshark with the wireshark-cli包 package.

For tcpdump:

```
# tcpdump -ni interface "port 22"
```

For Wireshark:

```
$ tshark -f "tcp port 22" -i interface
```

where `*interface*` is the network interface for a WAN connection (see `ip a` to check). If you are not receiving any packets while trying to connect remotely, you can be very sure that your ISP is blocking the incoming traffic on port 22.

Possible solution

The solution is just to use some other port that the ISP is not blocking. Open the `/etc/ssh/sshd_config` and configure the file to use different ports. For example, add:

```
Port 22
Port 1234
```

Also make sure that other "Port" configuration lines in the file are commented out. Just commenting "Port 22" and putting "Port 1234" will not solve the issue because then sshd will only listen on port 1234. Use both lines to run the SSH server on both ports.

Restart the server `sshd.service` and you are almost done. You still have to configure your client(s) to use the other port instead of the default port. There are numerous solutions to that problem, but let us cover two of them here.

Read from socket failed: connection reset by peer

Recent versions of OpenSSH sometimes fail with the above error message when connecting to older ssh servers. This can be worked around by setting various client

options for that host. See ssh_config(5) for more information about the following options.

The problem could be the `ecdsa-sha2-nistp*-cert-v01@openssh` elliptical host key algorithms. These can be disabled by setting `HostKeyAlgorithms` to a list excluding those algorithms. On the client side, the `HostKeyAlgorithms` that the client wants to use can also be set by preceding the `HostKeyAlgorithms` list with a `-` to remove the specified algorithms (including wildcards) from the default set (see `ssh_config(5)` man page). You can check the actually used host key algorithm with `ssh -v server_to_connect_to` in the line that contains `kex: host key algorithm:`.

If that does not work, it could be that the list of ciphers is too long. Set the `Ciphers` option to a shorter list (fewer than 80 characters should be enough). Similarly, you can also try shortening the list of `MACs`.

See also the discussion on the OpenSSH bug forum.

"\[your shell\]: No such file or directory" / ssh_exchange_identification problem

One possible cause for this is the need of certain SSH clients to find an absolute path (one returned by `whereis -b [your shell]`, for instance) in `$SHELL`, even if the shell's binary is located in one of the `$PATH` entries.

"Terminal unknown" or "Error opening terminal" error message

If you receive the above errors upon logging in, this means the server does not recognize your terminal. Ncurses applications like nano may fail with the message "Error opening terminal".

The correct solution is to install the client terminal's terminfo file on the server. This tells console programs on the server how to correctly interact with your terminal. You can get info about current terminfo using `$ infocmp` and then find out which package owns it.

If you cannot install it normally, you can copy your terminfo to your home directory on the server:

```
$ ssh myserver mkdir -p  ~/.terminfo/${TERM:0:1}
$ scp /usr/share/terminfo/${TERM:0:1}/$TERM myserver:~/.terminfo/${TERM:0:1}/
```

After logging in and out from the server the problem should be fixed.

TERM hack

注意：This should only be used as a last resort.

Alternatively, you can simply set `TERM=xterm` in your environment on the server (e.g. in `.bash_profile`). This will silence the error and allow ncurses applications to run again, but you may experience strange behavior and graphical glitches unless your terminal's control sequences exactly match xterm's.

Connection closed by x.x.x.x \[preauth\]

If you are seeing this error in your sshd logs, make sure you have set a valid HostKey

```
HostKey /etc/ssh/ssh_host_ed25519_key
```

subsystem request failed

Since *OpenSSH* 8.8, *scp* uses *SFTP* as the default protocol for data transfers by requesting the subsystem named `sftp`. If you run *scp* in verbose mode, `scp -v`, you can determine which subsystem your client is using (e.g. `Sending subsystem: <subsystem-name>`). Errors such as `subsystem request failed on channel 0` may be fixed by configuring the server's Subsystem settings: sshd_config(5) § Subsystem. The server configuration should resemble the example below.

```
/etc/ssh/sshd_config
...
Subsystem subsystem-name /path/to/subsystem-executable
...
```

id_dsa refused

OpenSSH 7.0 deprecated DSA public keys for security reasons and OpenSSH 9.8 is built without support for DSA keys by default. The first OpenSSH release of 2025 will remove DSA support entirely. For now, if you absolutely must use them, you will need to rebuild openssh包 while passing `--enable-dsa-keys` to `configure`.

No matching key exchange method found by OpenSSH 7.0

OpenSSH 7.0 deprecated the diffie-hellman-group1-sha1 key algorithm because it is weak and within theoretical range of the so-called Logjam attack (see https://www.openssh.com/legacy.html). If the key algorithm is needed for a particular host, ssh will produce an error message like this:

```
Unable to negotiate with 127.0.0.1: no matching key exchange method found.
Their offer: diffie-hellman-group1-sha1
```

The best resolution for these failures is to upgrade/configure the server to not use deprecated algorithms. If that is not possible, you can force the client to reenable the algorithm with the client option `KexAlgorithms +diffie-hellman-group1-sha1`.

tmux/screen session killed when disconnecting from SSH

If your processes get killed at the end of the session, it is possible that you are using socket activation and it gets killed by systemd包 when it notices that the SSH session process exited. In that case there are two solutions. One is to avoid using socket activation by using `ssh.service` instead of `ssh.socket`. The other is to set `KillMode=process` in the Service section of `ssh@.service`.

The `KillMode=process` setting may also be useful with the classic `ssh.service`, as it avoids killing the SSH session process or the screen包 or tmux包 processes when the server gets stopped or restarted.

SSH session stops responding

SSH responds to flow control commands `XON` and `XOFF`. It will freeze/hang/stop responding when you hit `Ctrl+s`. Use `Ctrl+q` to resume your session.

Broken pipe

If you attempt to create a connection which results in a `Broken pipe` response for `packet_write_wait`, you should reattempt the connection in debug mode and see if the output ends in error:

```
debug3: send packet: type 1
packet_write_wait: Connection to A.B.C.D port 22: Broken pipe
```

The `send packet` line above indicates that the reply packet was never received. So, it follows that this is a *QoS* issue. To decrease the likely-hood of a packet being dropped, set `IPQoS`:

```
/etc/ssh/ssh_config
Match all
    IPQoS reliability
```

The `reliability` (`0x04`) type-of-service should resolve the issue, as well as `0x00` and `throughput` (`0x08`).

Terminate unresponsive SSH connection

If a client session is no longer responding and cannot be terminated by instructing the running program (e.g. shell), you can still terminate the session by pressing `Enter`, `~` and `.` one after another in that order.

The `~` is a pseudo-terminal escape character (see ssh(1) § ESCAPE CHARACTERS), which can be added multiple times depending on the client session to terminate. For example, if you connected from A to B and then from B to C and the session from B to C freezes, you can terminate it by pressing `Enter` and typing `~~.`, which will leave you in a working session on B.

WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!

If the client warns that the key of an ssh server has changed, you should verify that the newly offered key really belongs to the server operator via an authenticated (not necessarily encrypted) channel. Then remove the old key from the `known_hosts` file with `ssh-keygen -R $SSH_HOST` and accept the new key as if it was a new server.

Connecting to a remote without the appropriate terminfo entry

When connecting to hosts that do not have a terminfo entry for your terminal, for example, when using a terminal emulator whose terminfo entry is not shipped with ncurses包 (e.g. kitty and rxvt-unicode), or when connecting to hosts with a limited terminfo database (e.g. systems running OpenWrt), various issues will occur with software that relies on terminfo(5).

A proper solution is to place the appropriate terminfo entry on the host. If that is not feasible, an alternative is to set `TERM` to a value that is both supported by the remote host and compatible with the terminal.

Since OpenSSH 8.7, a custom `TERM` environment variable can be passed to remote hosts with a simple configuration snippet:

```
~/.ssh/config
Host example.com
  SetEnv TERM=xterm-256color
```

Connection through jump host fails with "bash: No such file or directory"

If you do not have the `SHELL` environment variable set to a full valid path (on the jump server), connection will fail with an error message simmilar to this one:

```
bash: No such file or directory
kex_exchange_identification: Connection closed by remote host
Connection closed by UNKNOWN port 65535
```

You can simply solve this by setting your `SHELL` to a full path name of a shell that will also be valid on the jump server or by setting a specific `SHELL` variable for each server in your `~/.ssh/config` file.

参见

- Wikibooks:OpenSSH
- 防御暴力 ssh 攻击
- OpenSSH 密钥管理：第一部分在 IBM developerWorks 上, 第二部分、第三部分在 funtoo.org 上
- Secure Secure Shell

第 9 章　共 6 节　80 KB

VPN

本章解决的是让一台机器既能充当 VPN 服务端供别人接入，也能作为客户端连出去：装软件、备好密钥或证书、配好路由、放行防火墙、启动服务，每一步都得对得上。建议先读《OpenVPN》与《WireGuard》：一个从内核配置、自建 PKI 走到三层 IP 路由，一个从密钥对走到服务端配置与疑难解答，读透其一其余可自行迁移；《Linux 容器/使用 VPN》在容器网络里必读；《StrongSwan》重在证书与多种 VPN 模式，按需查阅；《PPTP server》与《PPTP Client》最短，只讲安装、配置、建立与断开连接，快读即可。

最容易踩的坑是把装上软件当成已经连通，或把服务端与客户端的配置混在一起：证书与 Secrets 对不上、路由没配、防火墙没放行、容器里还要单独配防火墙，症状常常只是连不上。

9.1　PPTP server

难度 入门 | **篇幅** 3.6 KB | **所属** VPN

本节将讲到

- 安装
- 配置
- 启动

点对点隧道协议(PPTP)是一种实现虚拟专用网(VPN)的方法。PPTP 在TCP之上使用一个控制通道和 GRE 隧道操作加密 PPP 数据包。

本条目说明如何在 Arch 中创建 PPTP 服务器。

安装

安装位于官方软件仓库的软件包pptpd包。

配置

然后编辑文件`/etc/pptpd.conf`

```
/etc/pptpd.conf
option /etc/ppp/pptpd-options
localip 172.16.36.1
remoteip 172.16.36.2-254
```

接着编辑文件`/etc/ppp/pptpd-options`

```
/etc/ppp/pptpd-options
name pptpd
refuse-pap
refuse-chap
refuse-mschap
require-mschap-v2
require-mppe-128
proxyarp
lock
nobsdcomp
```

```
novj
novjccomp
nologfd
ms-dns 8.8.8.8
ms-dns 8.8.4.4
```

在 /etc/ppp/chap-secrets中添加用户名和密码：

```
/etc/ppp/chap-secrets
<username>     pptpd     <password>   *
```

在 /etc/sysctl.d/99-sysctl.conf中启用IP转发：

```
/etc/sysctl.d/99-sysctl.conf
net.ipv4.ip_forward=1
```

应用 sysctl.conf 修改：

```
# sysctl --system
```

iptables 防火墙配置

配置 iptables 设置，允许 PPTP 客户端访问：

```
iptables -A INPUT -i ppp+ -j ACCEPT
iptables -A OUTPUT -o ppp+ -j ACCEPT

iptables -A INPUT -p tcp --dport 1723 -j ACCEPT
iptables -A INPUT -p 47 -j ACCEPT
iptables -A OUTPUT -p 47 -j ACCEPT

iptables -F FORWARD
iptables -A FORWARD -j ACCEPT

iptables -A POSTROUTING -t nat -o eth0 -j MASQUERADE
iptables -A POSTROUTING -t nat -o ppp+ -j MASQUERADE
```

使用下面命令保存新的 iptables 规则：

```
# rc.d save iptables
```

systemd 用户需要使用：

```
# iptables-save > /etc/iptables/iptables.rules
```

阅读 Iptables 条目获得更多信息。

ufw 防火墙配置

在 /etc/default/ufw中，修改默认的转发规则

```
/etc/default/ufw
DEFAULT_FORWARD_POLICY=”ACCEPT”
```

修改/etc/ufw/before.rules，添加如下配置到 *filter 之前

```
/etc/ufw/before.rules
# nat Table rules
*nat
:POSTROUTING ACCEPT [0:0]

# Allow traffic from clients to eth0
-A POSTROUTING -s 172.16.36.0/24 -o eth0 -j MASQUERADE

# don.t delete the .COMMIT. line or these nat table rules won.t be processed
COMMIT
```

代开端口 1723：

```
ufw allow 1723
```

重启ufw

```
ufw disable
ufw enable
```

启动

pptpd 软件包已经提供了 service 文件.

启动服务：

```
# systemctl start pptpd.service
```

开机自动启动：

```
# systemctl enable pptpd.service
```

9.2 PPTP Client

难度 进阶 | **篇幅** 6.3 KB | **所属** VPN

本节将讲到

- 安装PPTPClient
- 配置
- 创建你的连接
- 断开连接
- 把一个VPN连接配为默认启动

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2011, out of sync with English page（在 Talk:PPTP Client# 中讨论）

pptpclient是一个实现Microsoft PPTP协议的程序。因此它能够被用来接入另一个Microsoft VPN网络，比如学校和单位。

安装PPTPClient

pptpclient由安装包pptpclient提供，运行下列命令可以安装：

```
# pacman -S pptpclient
```

配置

你需要从网络管理员获取以下信息来配置pptpclient:

- VPN服务器的ip或者域名
- VPN隧道名称
- Windows域（不是所有网络都需要）
- VPN用户名
- VPN密码

编辑配置文件

用你称手的编辑器打开/etc/ppp/options.pptp。这个文件为你的VPN连接启用了一系列默认安全设置。如果你连接时候出现问题，你可以自定义配置。你的options.pptp文件最少需要包含以下内容：

```
lock
noauth
nobsdcomp
nodeflate
```

编辑密码文件

下一步，打开或者创建/etc/ppp/chap-secrets。我们将在这个文件里面存储你的密码，记得修改权限让除root之外所有用户不能访问它。这个文件的格式如下：

```
<DOMAIN>\\<USERNAME> PPTP <PASSWORD> *
```

如果你的服务器不要求域，则配置如下：

```
<USERNAME> PPTP <PASSWORD> *
```

替换掉上文中范例中的占位符。注意，如果你的密码包含特殊字符，比如“$”，你需要用双引号把它们包起来。

命名你的VPN隧道

用你称手的编辑器创建类似/etc/ppp/peers/<TUNNEL>的文件，把<TUNNEL>这里替换成你的VPN连接名。这个文件设置之后看起来如下：

```
pty "pptp <SERVER> --nolaunchpppd"
name <DOMAIN>\\<USERNAME>
remotename PPTP
require-mppe-128
file /etc/ppp/options.pptp
ipparam <TUNNEL>
```

注意：跟刚才一样，如果你的连接不要求域，忽略范例中的"<DOMAIN>\\\\"

注意：PPTP远程主机使用Chap-Secrets文件中的<PASSWORD>

<SERVER>是VPN服务器的地址，<DOMAIN>是你所属的域，<USERNAME>是你将要用来连接服务器的用户名，<TUNNEL>是连接的名称。

注意：如果你不需要使用MPPE，你应当从/etc/ppp/options.pptp中移除require-mppe-128这个选项

创建你的连接

用root执行以下命令来确保配置是正确的：

```
# pon $TUNNEL debug dump logfd 2 nodetach
```

如果一切都配置好了，pon命令应当不会自动结束。一旦你感觉差不多OK了，就可以终止这个命令。

注意：另一个用来确保配置正确的命令是ifconfig -a，看看里面时候有一个名叫ppp0的新驱动，并且还是可用的

执行以下命令来连接VPN隧道：

```
# pon <TUNNEL>
```

<TUNNEL>是你之前命名过的VPN隧道名称。注意使用root命令执行。

配置路由

一旦你成功连接上VPN，你就可以和VPN服务器交互了。当然在此之前，咱们需要添加一个新的路由到你的路由表，从而可以接入远程网络。

注意：根据你的环境配置，你可能需要每次都重复添加路由信息

你可以阅读PPTP Routing Howto来获得更多如何添加路由的信息，里面还有很多范例。

选择路由

对我来说，只有传输到VPN网络的数据包才应该走VPN连接，所以我添加如下路由条目：

```
# ip route add 192.168.10.0/24 dev ppp0
```

这将路由所有目的地址为191.168.10.xxx的数据到VPN连接。

配置为默认路由

如果你想要所有数据从VPN连接走，下面这条命令包你爽：

```
# ip route add default dev ppp0
```

注意：所有数据从VPN连接走的话会比正常连接慢一些

使用ip-up.d设置默认路由

注意：/etc/ppp/ip-up.d/下所有脚本将在VPN连接建立的时候自动执行

```
#!/bin/bash

# This script is called with the following arguments:
# Arg Name
# $1 接口名字
# $2 The tty
# $3 连接速度
# $4 本地ip地址
# $5 Peer IP number
# $6 可选``ipparam 的参数

route add default gw $4
```

把这个脚本命名为01-routes.sh,然后放在/etc/ppp/ip-up.d/下面。

断开连接

下面这条命令用来断开VPN连接：

```
# poff <TUNNEL>
```

<TUNNEL>是你VPN连接的名称。

把一个VPN连接配为默认启动

你可以在rc.d创建一个快捷命令来实现自动在后台连接VPN网络。

注意：和平常一样，<TUNNEL>是你隧道的名字，<ROUTING COMMAND>是你加入路由表的命令。

```
#!/bin/bash

. /etc/rc.conf
. /etc/rc.d/functions

DAEMON=<TUNNEL>-vpn
ARGS=

[ -r /etc/conf.d/$DAEMON ] && . /etc/conf.d/$DAEMON

case "$1" in
 start)
   stat_busy "Starting $DAEMON"
   pon <TUNNEL> updetach persist &> /dev/null && <ROUTING COMMAND> &>/dev/null
   if [ $? = 0 ]; then
     add_daemon $DAEMON
     stat_done
   else
     stat_fail
     exit 1
   fi
   ;;
 stop)
   stat_busy "Stopping $DAEMON"
   poff MST &>/dev/null
   if [ $? = 0 ]; then
     rm_daemon $DAEMON
     stat_done
   else
     stat_fail
     exit 1
   fi
   ;;
 restart)
   $0 stop
   sleep 1
   $0 start
   ;;
 *)
   echo "usage: $0 {start|stop|restart}"
esac
```

注意，我们可以使用updetach和persist这两个附加命令在pon上。updetach保证pon阻塞知道连接被建立。另外一个命令persist保证网络自动重练。如果需要开机自动启动，则添加@<TUNNEL>-vpn到rc.conf的DAEMONS中去。

注意

你可以在pptpclient website查到更多关于pptpclient的配置信息。Ubuntu的帮助手册也有一些帮助你配置的信息。这些范例能够很轻松的稍加变换从而添加到daemons中去，从而帮助你自动化运行。

9.3 Linux 容器/使用 VPN

难度 进阶 | **篇幅** 9.5 KB | **所属** VPN

本节将讲到

- 配置容器
- 服务器模式的OpenVPN
- 客户端模式的 OpenVPN
- for OpenVPN
- WireGuard

本文介绍如何设置 Linux_容器来运行多个带有“终止开关”的 VPN 协议，以实现安全/隐私的互联网使用。 与使用 VirtualBox 或者 QEMU 等全套的虚拟化相比， 这样做提供了更小的资源开销和在低功耗设备上运行等明显优势。

配置容器

需要对 Linux_容器进行基本配置和了解。 本文假设读者已经设置了可运行的基本 LXC。

服务器模式的OpenVPN

本小节详细介绍了在容器中提供 OpenVPN 服务所需的一些额外设置。 想要使用给出的 OpenVPN 配置文件的用户无需阅读本小节。

主机设置

1. 主机操作系统需要网桥设置才能允许容器运行。请参考 Linux_容器#宿主机网络配置。
2. 需要启用数据包转发。请参考网络分享#启用包转发。
3. 尽管不是必须的，但强烈建议使用防火墙。

客户端模式的 OpenVPN

需要修改容器的配置才能使用 OpenVPN，如下所示：

```
/var/lib/lxc/playtime/config
...

## for OpenVPN
lxc.mount.entry = /dev/net dev/net none bind,create=dir
lxc.cgroup2.devices.allow = c 10:200 rwm
```

安装 openvpn包. 如果使用容器连接到第三方 VPN 提供商，只需将配置文件 `foo.conf` 放在 `/etc/openvpn/client/foo.conf` 即可。 To v要验证容器内的 OpenVPN 功能，请启动 `openvpn-client@foo.service` 来开启OpenVPN。一旦对当前配置感到满意，启用服务来实现系统启动后自动开启。

有关其他用例和设置，请参阅 OpenVPN。

注意：在“非特权”容器中运行 OpenVPN 的用户需要创建一个自定义 systemd 单元才能在容器中启动它。在替换单元文件中注释掉以下开头的行：`LimitNPROC...`

WireGuard

安装wireguard-tools包。用户将拥有由第三方 VPN 服务提供的 WireGuard 配置，或者自己设置好的WireGuard 对应的配置。 如果使用容器连接到 VPN 提供商， 只需将配置文件 `foo.conf` 放入 `/etc/wireguard/` 。

要验证容器内的 WireGuard 功能，请通过启动 `wg-quick@foo.service` 来开启 WireGuard。一旦对当前配置感到满意，启用服务来实现系统启动后自动开启。

有关其他用例和设置，请参阅 WireGuard。

容器内的防火墙配置

*强烈*建议在容器内运行正确配置的防火墙。容器内防火墙的作用有两个：

1. 提供“终止开关”，以便在 VPN 连接失败时维护隐私。
2. 把不想要东西拒之门外。

本指南使用易于配置的 ufw包，当然也可以使用其他防火墙。

提示：要完全重置 ufw 的配置文件，使用reset: `ufw reset`.

设置“终止开关”的方法是通过设置一个拒绝政策然后只允许VPN设备上的特定服务和流量。这样，如果该设备的连接中断时不会出现本地政策回退。

注意：下面显示的方法的一个限制是 VPN 配置文件不得使用 www.myvpn.com 等域名，它们需要使用相应的 IP 地址。如上所述，当 VPN 未连接时，容器 DNS 解析将被禁用。因此，为了连接，必须提供数字 IP。

编辑 `/etc/default/ufw` 并将 DEFAULT_OUTPUT_POLICY 从 "ACCEPT" 更改为 "DROP":

```
/etc/default/ufw
DEFAULT_OUTPUT_POLICY="DROP"
```

注意：以下命令调用 `ufw` 需要以root用户执行； 按照 wiki 标准命令的前缀“#”符号已被省略，以允许干净地复制/粘贴到终端中。

设置拒绝政策：

```
ufw default deny outgoing
ufw default deny incoming
```

（可选）添加例如`/etc/ufw/applications.d/custom`等文件中定义的任何预定义或自定义规则：

```
ufw allow ssh
ufw allow from my-custom-app1
ufw allow from my-custom-app2
```

（可选）进一步限制来自内部 LAN IP 范围甚至单个 IP 地址的访问：

```
ufw allow from 192.168.1.0/24
```

WireGuard 用户将拥有一个与相应配置文件，比如`/etc/wireguard/foo.conf`，同名的接口，而 OpenVPN 用户很可能使用`tun0`. 在下面的命令中，将“foo”替换为 WireGuard 配置文件的名称（省略 .conf 后缀），或者如果使用 OpenVPN，则将“foo”替换为 tun0 或正在使用的任何设备：

```
ufw allow out on foo from any to any
```

最后，打开 VPN 提供商的 IP 地址和端口，并定义相对应的协议。在下面的行中，需要考虑三个变量：

- “xxx”代表 WireGuard 的 peer 或者 OpenVPN 的服务器的IP地址。它将在 VPN 提供商提供的相应配置文件中。
- “yyy”代表要进行通信的端口。同样，这将位于配置文件中。
- “zzz” 代表要使用的协议，可以从 udp 或 tcp 中选择。请注意，WireGuard 仅支持 udp，而 OpenVPN 两者都支持。

```
ufw allow out to xxx port yyy proto zzz
```

注意：如果预计使用多个服务器，请对 VPN 提供商定义的每个 IP 地址 (xxx) 重复此操作。

启动 ufw 并启用 `ufw.service` 以实现开机启动。

在配置文件中使用 VPN 域名的一个非正式方法

如果需要在 VPN 配置文件中使用域名，主机上的 shell 脚本可以将其预先解析为数字 IP，然后通过将该 IP 地址存储在写在容器中文件的一个变量来将该 IP 地址传递给容器。该文件又可以由修改后的 VPN systemd 服务读取。 它可以用, 但不正规。

编辑两个变量以匹配相对应的容器名称和服务器名称：

在主机上

安装 bind包 (dig 需要)并创建以下脚本：

```
/path/to/container-start.sh
#!/bin/bash
# this script should be called as root
container=foo
server=www.myvpnserver.org

if ! systemctl is-active lxc@"$container" &>/dev/null; then
  ToUse=$(dig +short "$server")
  [[ -d /var/lib/lxc/$container/rootfs/etc/conf.d ]] || mkdir -p /var/lib/lxc/
$container/rootfs/etc/conf.d
  echo "SERVER=$ToUse" > /var/lib/lxc/$container/rootfs/etc/conf.d/server.hack.txt
  systemctl start lxc@"$container"
fi
```

从现在开始，调用该脚本来启动容器。它将使用 dig 从域名中获取 IP 地址，然后启动容器。

从容器内部

修改启动 VPN 的 systemd 服务并创建一个框架配置文件，该配置文件可以使用我们刚刚创建的脚本创建的`/var/lib/lxc/$container/rootfs/etc/conf.d/server.hack.txt` 中定义的 IP 地址进行修改。

要制作框架配置文件，只需将使用的配置文件重命名为另一个名称即可。

例如使用 WireGuard：

```
mv /etc/wireguard/foo.conf /etc/wireguard/foo.skel
```

现在编辑 `/etc/wireguard/foo.skel` 以将 **Endpoint = www.myvpnserver.org** 替换为**@@@**，例如：

```
Endpoint = @@@:51820
```

或者如果使用 OpenVPN：

```
mv /etc/openvpn/client/foo.conf /etc/openvpn/client/foo.skel
```

编辑 `/etc/openvpn/client/foo.skel` 以将远程**www.myvpnserver.org**替换为**@@@**，例如：

```
remote @@@
```

最后，创建一个附加配置片段, 以读取 IP 并将其替换为实际配置文件。

使用 WireGuard 的示例：

```
/etc/systemd/system/wg-quick@foo.service.d/override.conf
[Service]
EnvironmentFile=-/etc/conf.d/server.hack.txt
ExecStartPre=/bin/bash -ac "sed s/@@@/$SERVER/ </etc/wireguard/foo.skel
>/etc/wireguard/foo.conf"
```

使用 OpenVPN 的示例：

```
/etc/systemd/system/openvpn-client@foo.service.d/override.conf
[Service]
EnvironmentFile=-/etc/conf.d/server.hack.txt
ExecStartPre=/bin/bash -ac "sed s/@@@/$SERVER/ </etc/openvpn/client/foo.skel
>/etc/openvpn/client/foo.conf"
```

测试服务

在正在运行的容器内 (通过 ssh 或者 `lxc-attach -n playtime`) 通过将浏览器导出到主机的计算机 X server 来测试设置:

```
$ DISPLAY=:0 firefox
```

提示：通过 ssh 连接需要允许本地显示接受连接。执行此操作 `xhost +SI:localuser:yourusername` 然后通过 ssh 连接到容器。

结果应该是主机的 X server 中出现一个标题为"Mozilla Firefox (playtime)" 的 Firefox 窗口。许多网站可用于验证 IP 地址和 DNS 的状态，例如 ipleak dot net.

此时，仅应显示与配置文件中定义的条目相对应的 DNS 条目。

9.4 StrongSwan

难度 进阶 | **篇幅** 12.0 KB | **所属** VPN

本节将讲到

- 安装
- 证书
- 不同的 VPN 模式
- Secrets
- Networking

IPSec 是一个加密和认证标准，可以组建安全虚拟专用网路(VPN).

Linux kernel 已经原生支持这个标准，但是用户需要配置加密密钥。IPSec VPN 使用 IKE 协议自动协商密钥的交换，交换方式包括认证、提前共享或同时使用。

通常用服务器上的一个后台进程实现，strongSwan 是一个完全支持 IKEv1 和 IKEv2 的 IKE 后台进程。大部分现代的客户端都支持，包括 Linux, Windows 7, Apple iOS, Mac OSX, FreeBSD and BlackBerry OS.

安装

安装软件包 strongswan包。

证书

第一步是生成 X.509 认证，包括一个认证机构(CA)证书，一个服务器证书和至少一个客户端证书。

认证机构

首先创建一个自己签名的根 CA 证书:

生成 4096 位 RSA 私钥 `strongswanKey.pem`:

```
$ cd /etc/ipsec.d/
$ ipsec pki --gen --type rsa --size 4096 --outform pem > private/strongswanKey.pem
$ chmod 600 private/strongswanKey.pem
```

生成自签名的 CA 证书 `strongswanCert.pem`，期限是 10 年(3650天),用 PEM 编码格式保存：

```
$ ipsec pki --self --ca --lifetime 3650 --in private/strongswanKey.pem --type rsa --dn
"C=CH, O=strongSwan, CN=strongSwan Root CA" \
          --outform pem > cacerts/strongswanCert.pem
```

可以根据需要修改 Distinguished Name (DN)， country (C), organization (O), and common name (CN)。

通过下面命令查看新生成的证书：

```
$ ipsec pki --print --in cacerts/strongswanCert.pem
```

输出:

```
cert:      X509
subject:  "C=CH, O=strongSwan, CN=strongSwan Root CA"
issuer:   "C=CH, O=strongSwan, CN=strongSwan Root CA"
validity:  not before Nov 22 11:55:41 2013, ok
           not after  Nov 20 11:55:41 2023, ok (expires in 3649 days)
serial:    65:39:93:df:a0:f8:40:03
flags:     CA CRLSign self-signed
authkeyId: 45:30:11:da:a4:0e:0b:0a:a3:41:a5:81:41:ab:d8:04:7a:40:6c:c0
subjkeyId: 45:30:11:da:a4:0e:0b:0a:a3:41:a5:81:41:ab:d8:04:7a:40:6c:c0
pubkey:    RSA 4096 bits
keyid:     dc:15:91:95:04:07:a5:13:69:5f:77:65:26:d7:02:3f:60:ec:73:c8
subjkey:   45:30:11:da:a4:0e:0b:0a:a3:41:a5:81:41:ab:d8:04:7a:40:6c:c0
```

警告：CA 的私钥 `/etc/ipsec.d/private/strongswanKey.pem` 必须保存在安全位置，最好放在一个没有网络访问的签名专用服务器。如果此密钥泄露，整个公钥系统就土崩瓦解了。

主机证书

此证书被用来认证 VPN 服务器，运行下面命令：

生成 2048 位 RSA 私钥：

```
$ cd /etc/ipsec.d/
$ ipsec pki --gen --type rsa --size 2048 --outform pem > private/vpnHostKey.pem
$ chmod 600 private/vpnHostKey.pem
```

导出公钥并生成被 CA 签名的 `vpnHostCert.pem`，证书有效期是两年(730天),使用完整域名 `vpn.example.com` 作为身份

```
$ ipsec pki --pub --in private/vpnHostKey.pem --type rsa | \
        ipsec pki --issue --lifetime 730 \
         --cacert cacerts/strongswanCert.pem \
         --cakey private/strongswanKey.pem \
         --dn "C=CH, O=strongSwan, CN=vpn.example.com" \
         --san vpn.example.com \
         --flag serverAuth --flag ikeIntermediate \
         --outform pem > certs/vpnHostCert.pem
```

后面的客户端连接配置中，会使用 VPN 服务器的域名或 IP 地址，必须将其加入 Distinguished Name 或 Alternative Name (line11), 最好都包含。请注意把 vpn.example.com 修改为 VPN 的主机名 - 否则客户端和服务器的连接会失败。

注意：如果要使用 Win7 内置的 VPN 客户端，必须按上面例子在主机证书中添加 `serverAuth` 标记。OS X 10.7.3 和更早的版本需要 `ikeIntermediate` 标记。这些标记不影响其它功能，可以放心使用。

查看新生成的证书：

```
$ ipsec pki --print --in certs/vpnHostCert.pem
```

Output:

```
cert:      X509
subject:  "C=CH, O=strongSwan, CN=vpn.example.com"
issuer:   "C=CH, O=strongSwan, CN=strongSwan Root CA"
validity:  not before Nov 22 21:16:51 2013, ok
           not after  Nov 22 21:16:51 2015, ok (expires in 729 days)
serial:    0c:05:d7:d5:57:0e:d9:48
altNames:  vpn.zeitgeist.se
```

```
flags:      serverAuth iKEIntermediate
authkeyId: 9b:57:35:fb:cd:9e:2d:20:37:1d:61:4c:e7:c4:5b:5e:dc:64:ad:fc
subjkeyId: 5f:12:c2:06:ee:2b:1e:cc:5f:78:54:ff:f0:f3:7b:a0:2b:c0:b4:d6
pubkey:     RSA 2048 bits
keyid:      6f:a7:99:60:27:27:09:96:02:c1:b9:d9:7d:c1:b0:10:e3:e1:d5:45
subjkey:    5f:12:c2:06:ee:2b:1e:cc:5f:78:54:ff:f0:f3:7b:a0:2b:c0:b4:d6
```

客户端证书

客户端需要个人证书才能使用 VPN. 和生成主机证书类似，只是这里用客户端的电子邮件而不是主机名:

生成 2048 位 RSA 私钥:

```
$ cd /etc/ipsec.d/
$ ipsec pki --gen --type rsa --size 2048 --outform pem > private/ClientKey.pem
$ chmod 600 private/ClientKey.pem
```

导出公钥并用 CA 签名，证书时间是两年。

```
$ ipsec pki --pub --in private/ClientKey.pem --type rsa | \
        ipsec pki --issue --lifetime 730 \
         --cacert cacerts/strongswanCert.pem \
         --cakey private/strongswanKey.pem \
         --dn "C=CH, O=strongSwan, CN=myself@example.com" \
         --san myself@example.com \
         --outform pem > certs/ClientCert.pem
```

将所有证书和密钥用密码打包成 PKCS#12 文件，以方便客户端使用。

```
$ openssl pkcs12 -export -inkey private/ClientKey.pem \
         -in certs/ClientCert.pem -name "My own VPN client certificate" \
         -certfile cacerts/strongswanCert.pem \
         -caname "strongSwan Root CA" \
         -out Client.p12
```

不同的 VPN 模式

最简单的配置方式是用隧道模式运行 IPSec。

IPSec 隧道模式

可以在 `/etc/ipsec.conf` 找到 VPN 配置，下面配置包含了基本 VPN 服务器需要的选项：

```
/etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file
config setup

  # By default only one client can connect at the same time with an identical
  # certificate and/or password combination. Enable this option to disable
  # this behavior.
  # uniqueids=never

  # Slightly more verbose logging. Very useful for debugging.
  charondebug="cfg 2, dmn 2, ike 2, net 2"

# Default configuration options, used below if an option is not specified.
# See: https://wiki.strongswan.org/projects/strongswan/wiki/ConnSection
conn %default

  # Use IKEv2 by default
  keyexchange=ikev2
```

```
  # Prefer modern cipher suites that allow PFS (Perfect Forward Secrecy)
  ike=aes128-sha256-ecp256,aes256-sha384-ecp384,aes128-sha256-modp2048,aes128-sha1-
modp2048,aes256-sha384-modp4096,aes256-sha256-modp4096,aes256-sha1-modp4096,aes128-
sha256-modp1536,aes128-sha1-modp1536,aes256-sha384-modp2048,aes256-sha256-
modp2048,aes256-sha1-modp2048,aes128-sha256-modp1024,aes128-sha1-modp1024,aes256-
sha384-modp1536,aes256-sha256-modp1536,aes256-sha1-modp1536,aes256-sha384-
modp1024,aes256-sha256-modp1024,aes256-sha1-modp1024!
  esp=aes128gcm16-ecp256,aes256gcm16-ecp384,aes128-sha256-ecp256,aes256-sha384-
ecp384,aes128-sha256-modp2048,aes128-sha1-modp2048,aes256-sha384-modp4096,aes256-
sha256-modp4096,aes256-sha1-modp4096,aes128-sha256-modp1536,aes128-sha1-
modp1536,aes256-sha384-modp2048,aes256-sha256-modp2048,aes256-sha1-modp2048,aes128-
sha256-modp1024,aes128-sha1-modp1024,aes256-sha384-modp1536,aes256-sha256-
modp1536,aes256-sha1-modp1536,aes256-sha384-modp1024,aes256-sha256-modp1024,aes256-
sha1-modp1024,aes128gcm16,aes256gcm16,aes128-sha256,aes128-sha1,aes256-sha384,aes256-
sha256,aes256-sha1!

  # Dead Peer Discovery
  dpdaction=clear
  dpddelay=300s

  # Do not renegotiate a connection if it is about to expire
  rekey=no

  # Server side
  left=%any
  leftsubnet=0.0.0.0/0
  leftcert=vpnHostCert.pem

  # Client side
  right=%any
  rightdns=8.8.8.8,8.8.4.4
  rightsourceip=%dhcp

# IKEv2: Newer version of the IKE protocol
conn IPSec-IKEv2
  keyexchange=ikev2
  auto=add

# IKEv2-EAP
conn IPSec-IKEv2-EAP
  also="IPSec-IKEv2"
  rightauth=eap-mschapv2
  rightsendcert=never
  eap_identity=%any

# IKEv1 (Cisco-compatible version)
conn CiscoIPSec
  keyexchange=ikev1
  # forceencaps=yes
  rightauth=pubkey
  rightauth2=xauth
  auto=add
```

IPSec in transport mode

IPSec/L2TP

L2TP/IPsec VPN client setup 页面包含了如何建立一个连接到 IPSec/L2TP 服务器的客户端。这种 IPSec VPN 和纯 IPSec 相比，可以将隧道用于非 IP 包，缺点是需要运行一个额外的 L2TP 后台进程。

Secrets

strongSwan 需要知道哪些客户端是允许连接到 VPN 的，通过 `/etc/ipsec.secrets` 文件进行配置：

```
/etc/ipsec.secrets
# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.  Suitable public keys, for ipsec.conf, DNS,
# or configuration of other implementations, can be extracted conveniently
# with "ipsec showhostkey".

: RSA ClientKey.pem
user1 : EAP "topsecretpassword"
user2 : XAUTH "evenmoretopsecretpassword"
```

当 strongSwan 运行时，如果编辑了 /etc/ipsec.secrets，需要重新加载：

```
$ ipsec rereadsecrets
```

Networking

需要执行下面设备让 VPN 服务器正常的进行 VPN 隧道传输：

```
/etc/sysctl.d/10-net-forward.conf
# VPN
net.ipv4.ip_forward = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
```

上面的 VPN 配置用 DHCP 自动给客户端分配 IP 地址，所以需要一个 DHCP 服务，如果这个服务是运行在 strongSwan 相同的服务器上，需要编辑 `/etc/strongswan.d/charon/dhcp.conf`:

```
/etc/strongswan.d/charon/dhcp.conf
dhcp {
 force_server_address = yes
 server = 192.168.0.255
}
```

在防火墙中开放如下协议：

- ESP (Encrypted Secure Payload): Standard IPSec traffic
- UDP 4500: IPSec traffic in "NAT Traversal" mode
- UDP 500: Key exchanges (IKE)

最后，start 并 enable `strongswan` 服务。

在容器中运行 Strongswan

要在容器中运行 `strongswan`，比如在 systemd-nspawn 中，需要类似下面的服务文件：

```
/etc/systemd/system/systemd-nspawn@.service.d/override.conf}
[Service]
ExecStart=
ExecStart=/usr/bin/systemd-nspawn --quiet --keep-unit --boot --link-journal=try-guest
--settings=override --machine=%I --capability=CAP_NET_ADMIN --network-veth
```

Troubleshooting

参阅

- strongSwan 5: How to create your own VPN——初始版本的参考，获得作者授权。

9.5 WireGuard

难度 进阶 | **篇幅** 18.7 KB | **所属** VPN

本节将讲到

- 安装
- 使用
- 配置一个 VPN 服务器
- 使用技巧
- 疑难解答

**这篇文章或章节的翻译质量不佳。**

原因：翻译已经过期，请阅读英文页面中的内容。（在 Talk:WireGuard# 中讨论）

引用自 WireGuard 项目主页：

: WireGuard 是一种极其简单但快速且现代的 VPN，它利用了最先进的加密技术。它的目标是比 IPsec 更快、更简单、更精简和更有用，同时避免令人头疼的问题。其旨在提供比 OpenVPN 更高的性能。WireGuard 被设计为在嵌入式接口和超级计算机等上运行的通用 VPN，适用于许多不同的环境。虽然最初仅支持 Linux，但现在可以跨平台（Windows、macOS、BSD、iOS、Android）广泛部署。

本文中的主要概念在 WireGuard 项目主页上有粗略的介绍。2019 年底以后，Linux 内核中包含 WireGuard。

安装

安装 wireguard-tools包，其提供了用户空间实用程序。

另外，如果 Peer 公钥可用，也可以借助其他网络管理器的 WireGuard 支持。详见#持久化配置。

图形客户端

- **wireguird** — 一个运行在Linux上的GTK GUI WireGuard客户端

: https://github.com/UnnoTed/wireguird || wireguirdAUR

- **wireguard-gui** — 一个使用nextauri制作的Wireguard Linux图形化客户端

: https://github.com/0xle0ne/wireguard-gui || wireguard-gui-binAUR

命令行工具

- **wg_tool** — 用于管理服务器和用户 wireguard 配置的工具。

: https://github.com/gene-git/wg_tool || wg_toolAUR

- **wg-client** — 一个同时具有图形化界面和命令行的Linux客户端

: https://github.com/gene-git/wg-client || wg-clientAUR

- **wg2nd** — 将 WireGuard 配置从 wg-quick(8) 格式转换为 systemd-networkd 兼容的配置的工具。

: https://git.flu0r1ne.net/wg2nd/about || wg2ndAUR

使用

以下命令展示了如何在两个对等节点之间建立一条使用以下设置的基本的隧道：

	外部（公共）地址	**内部 IP 地址**	**端口**			
域名	IPv4 地址	IPv6 地址	IPv4 地址	IPv6 地址		
对等节点 A		198.51.100.101	2001:db8:a85b:70a:ffd4:ec1b:4650:a001	10.0.0.1/24	fdc9:281f:04d7:9ee9::1/64	UDP/51871
对等节点 B	peer-b.example	203.0.113.102	2001:db8:40f0:147a:80ad:3e88:f8e9:b002	10.0.0.2/24	fdc9:281f:04d7:9ee9::2/64	UDP/51902
对等节点 C		*动态（不固定）*	*动态（不固定）*	10.0.0.3/24	fdc9:281f:04d7:9ee9::3/64	UDP/51993

提示：对等节点可使用相同的 UDP 端口。

外部地址应该已经存在。例如，如果 ICMP 回显请求未被阻止，则对等节点 A 应该能够通过 ping 通对等点 B 的公共 IP 地址，反之亦然。

内部地址将是使用 ip(8) 实用程序手动创建或通过网络管理软件创建的新地址，这些地址将在新的 WireGuard 网络内部使用。以下示例使用 10.0.0.0/24 和 fdc9:281f:04d7:9ee9::/64 作为内部网络。 IP 地址中的 `/24` 和 `/64` 是 CIDR。

生成密钥

为每个对等节点创建公钥和私钥。如果要连接很多节点，可以考虑 vanity keypair 个性化 Base64 编码的公钥字符串，参见#Vanity keys。

生成私钥：

```
$ (umask 0077; wg genkey > peer_A.key)
```

注意：建议仅允许所有者读写。以上命令使用子 shell 临时改变 umask，从而确保只有所有者可访问（有读写权限）。

生成公钥：

```
$ wg pubkey < peer_A.key > peer_A.pub
```

或者，一次性生成公钥和私钥：

```
$ wg genkey | (umask 0077 && tee peer_A.key) | wg pubkey > peer_A.pub
```

还可以生成预共享密钥，以添加一层额外的对称密钥加密技术，将其混合到现有的公钥加密技术中，以实现后量子抵抗。应为每个对等节点对生成预共享密钥，且不应重复使用。例如，三个互连的对等节点 A、B 和 C 将需要三个单独的预共享密钥，每个对等节点对一个。

使用以下命令为每个对等节点对生成预共享密钥（确保也使用 `umask 0077`）：

```
$ wg genpsk > peer_A-peer_B.psk
$ wg genpsk > peer_A-peer_C.psk
$ wg genpsk > peer_B-peer_C.psk
```

Vanity keys

Currently, WireGuard does not support comments or attaching human-memorable names to keys. This makes identifying the key's owner difficult particularly when multiple keys are in use. One solution is to generate a public key that contains some familiar characters (perhaps the first few letters of the owner's name or of the hostname etc.), wireguard-vanity-addressAUR does this.

For example:

```
$ wireguard-vanity-address --in 8 leslie
searching for 'leslie' in pubkey[0..10], one of every 214748364 keys should match
one core runs at 2.69e6 keys/s, CPU cores available: 16
est yield: 5.0 seconds per key, 200.10e-3 keys/s
hit Ctrl-C to stop
private wEoVMj92P+E3fQXVf9IixWJqpCqcnP/4OfvrB1g3zmY=  public
LEsliEny+aMcWcRbh8Qf414XsQHSBOAFk3TaEk/aSD0=
private EAOwlGGqpHVbZ9ehaCspdBJt+lkMcCfkwiA5T5a4JFs=  public
VlesLiEB5BFd//OD2ILKXviolfz+hodG6uZ+XjoalC8=
private UDWG4VWI+RzAGzNSnlC+0X4d3nk9goWPs/NRC5tX524=  public
9lESlieIFOlJFV6dG7Omao2WS+amWgshDdBYn8ahRjo=
```

**这篇文章或章节的翻译质量不佳。**

原因：以下翻译已经过期，请阅读英文页面中的内容。（在 Talk:WireGuard# 中讨论）

在 peer 上生成公钥和私钥：

```
$ wg genkey | tee privatekey | wg pubkey > publickey
```

下面的指令会演示如何以表中的配置建立一条两个 peer 之间的隧道

	Peer A	Peer B
公网地址	10.10.10.1/24	10.10.10.2/24
内网地址	10.0.0.1/24	10.0.0.2/24
wireguard 监听端口	UDP/4857	UDP/3981

Peer 应当已经拥有公网地址。例如，peer A 应当能够通过 `ping 10.10.10.2` ping 通 peer B，反之亦然。内部地址是由下文 `ip` 命令创建的新地址，并且将在 WireGuard 网络中共享。IP地址中 `/24` 的含义详见 CIDR

Peer A 配置

这个 peer 将监听 UDP 端口 48574，通过将 peer B 的公钥与其内部和外部 IP 地址关联，接受来自 peer B 的连接。

```
# ip link add dev wg0 type wireguard
# ip link set dev wg0 mtu 1420
# ip addr add 10.0.0.1/24 dev wg0
# wg set wg0 listen-port 4857 private-key ./privatekey
# wg set wg0 peer [Peer B public key] persistent-keepalive 25 allowed-ips 10.0.0.2/32
endpoint 10.10.10.2:3981
# ip link set wg0 up
```

`[Peer B public key]` 的格式应当如同 `EsnHH9m6RthHSs+sd9uM6eCHe/mMVFaRh93GYadDDnM=`。`allowed-ips` 是 peer A 能够向之发送流量的地址列表。`allowed-ips 0.0.0.0/0` 将允许向任意地址发送流量。

Peer B 配置

如同 Peer A，只不过 wireguard 守护监听 UDP 端口 39814 并且只接受 peer A 的连接

```
# ip link add dev wg0 type wireguard
# ip link set dev wg0 mtu 1420
# ip addr add 10.0.0.2/24 dev wg0
# wg set wg0 listen-port 3981 private-key ./privatekey
# wg set wg0 peer [Peer A public key] persistent-keepalive 25 allowed-ips 10.0.0.1/32
endpoint 10.10.10.1:4857
# ip link set wg0 up
```

基本检查

不带任何参数使用 wg 命令可以快速查看当前的配置。

例如，当 peer A 配置好之后，我们可以看见它的身份和与之关联的 peers。

```
 peer-a$ wg
```

```
interface: wg0
  public key: UguPyBThx/+xMXeTbRYkKlP0Wh/QZT3vTLPOVaaXTD8=
  private key: (hidden)
  listening port: 4857

peer: 9jalV3EEBnVXahro0pRMQ+cHlmjE33Slo9tddzCVtCw=
  endpoint: 10.10.10.2:3981
  allowed ips: 10.0.0.2/32
```

此时我们可以 ping 通隧道的另一端：

```
peer-a$ ping 10.0.0.2
```

配置持久化

配置可以通过 `showconf` 来保存

```
# wg showconf wg0 > /etc/wireguard/wg0.conf
# wg setconf wg0 /etc/wireguard/wg0.conf
```

示例 peer 配置

```
/etc/wireguard/wg0.conf
[Interface]
PrivateKey = [CLIENT PRIVATE KEY]
MTU = 1420

[Peer]
PublicKey = [SERVER PUBLICKEY]
AllowedIPs = 10.0.0.0/24, 10.123.45.0/24, 1234:4567:89ab::/48
Endpoint = [SERVER ENDPOINT]:5182
PersistentKeepalive = 25
```

配置一个 VPN 服务器

WireGuard 自带一个快速创建和销毁 VPN 服务器的工具，`wg-quick`。注意这里使用的配置文件不是一个能被 `wg setconf` 有效的配置文件，并且你可能至少要把 `eth0` 改成你实际使用的。

服务器

```
/etc/wireguard/wg0server.conf
[Interface]
Address = 10.0.0.1/24  # This is the virtual IP address, with the subnet mask we will
use for the VPN
PostUp   = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j ACCEPT;
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i -j ACCEPT;
iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE
ListenPort = 5182
PrivateKey = [SERVER PRIVATE KEY]
MTU = 1420

[Peer]
PublicKey = [CLIENT PUBLIC KEY]
AllowedIPs = 10.0.0.2/32  # 这表示客户端只有一个 IP。
```

要使 iptables 规则生效，启用 IPv4 转发：

```
# sysctl net.ipv4.ip_forward=1
```

永久保留这项改变，向 `/etc/sysctl.d/99-sysctl.conf` 添加 `net.ipv4.ip_forward = 1`。

使用 `wg-quick up wg0server` 启用 Interface，`wg-quick down wg0server` 用以关闭

客户端 (转发所有流量)

```
/etc/wireguard/wg0.conf
[Interface]
Address = 10.0.0.2/24  # The client IP from wg0server.conf with the same subnet mask
PrivateKey = [CLIENT PRIVATE KEY]
DNS = 10.0.0.1
MTU = 1420

[Peer]
PublicKey = [SERVER PUBLICKEY]
AllowedIPs = 0.0.0.0/0, ::0/0
Endpoint = [SERVER ENDPOINT]:5182
PersistentKeepalive = 25
```

使用 `wg-quick up wg0` 来启用 Interface, 使用 `wg-quick down wg0` 来关闭。

使用 `systemctl enable wg-quick@wg0` 来自动启动。

如果你使用 NetworkManager, 可能有必要启用 NetworkManager-wait-online.service
`systemctl enable NetworkManager-wait-online.service`

或者你使用的是 systemd-networkd, 启用 systemd-networkd-wait-online.service
`systemctl enable systemd-networkd-wait-online.service`

等待所有设备就绪再尝试 WireGuard 连接

使用技巧

以加密的形式存储私钥

以加密的形式存储私钥可能是可以实现的，例如通过使用 pass包。只需将配置文件中 \[Interface\]下的 PrivateKey 一行替换为。

```
 PostUp = wg set %i private-key <(su user -c "export
PASSWORD_STORE_DIR=/path/to/your/store/; pass WireGuard/private-keys/%i")
```

where user is your username. See the \`wg-quick(8)\` man page for more details.

将其中的user替换成你的用户名。更多细节请参见 \`wg-quick(8)\` man 页面。

疑难解答

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

路由定期被重置

NetworkManager 用户应确保其未管理 WireGuard 接口。例如，创建以下配置文件：

```
/etc/NetworkManager/conf.d/unmanaged.conf
[keyfile]
unmanaged-devices=type:wireguard
```

DNS 无法解析

When tunneling all traffic through a WireGuard interface, the connection can become seemingly lost after a while or upon new connection. This could be caused by a network manager or DHCP client overwriting `/etc/resolv.conf`.

By default *wg-quick* uses *resolvconf* to register new DNS entries (from the `DNS` keyword in the configuration file). This will cause issues with network managers and DHCP clients that do not use *resolvconf*, as they will overwrite `/etc/resolv.conf` thus removing the DNS servers added by wg-quick.

The solution is to use networking software that supports resolvconf.

注意：Users of systemd-resolved should make sure that systemd-resolvconf包 is installed.

Users of NetworkManager should know that it does not use resolvconf by default. It is recommended to use systemd-resolved. If this is undesirable, install openresolv包 and configure NetworkManager to use it: NetworkManager#Use openresolv.

MTU 过低

Due to too low MTU (lower than 1280), wg-quick may have failed to create the WireGuard interface. This can be solved by setting the MTU value in WireGuard configuration in Interface section on client.

```
foo.config
[Interface]
Address = 10.200.200.2/24
MTU = 1420
PrivateKey = PEER_FOO_PRIVATE_KEY
DNS = 10.200.200.1
```

密钥长度或格式错误

To avoid the following error, put the key value in the configuration file and not the path to the key file.

```
# wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
Key is not the correct length or format: `/path/example.key'
Configuration parsing error
[#] ip link delete dev wg0
```

无法在 NAT 或防火墙后建立持久连接

By default, WireGuard peers remain silent while they do not need to communicate, so peers located behind a NAT and/or firewall may be unreachable from other peers until they reach out to other peers themselves (or the connection may time out). Adding `PersistentKeepalive = 25` to the `[Peer]` settings of a peer located behind a NAT and/or firewall can ensure that the connection remains open.

To temporarily set the persistent-keepalive setting via command line, run the following command:

```
# wg set wg0 peer public_key persistent-keepalive 25
```

循环路由

Adding the endpoint IP to the allowed IPs list, the kernel will attempt to send handshakes to said device binding, rather than using the original route. This results in failed handshake attempts.

As a workaround, the correct route to the endpoint needs to be manually added using

```
# ip route add endpoint_ip via gateway dev network_interface
```

E.g. for peer B from above in a standard LAN setup:

```
# ip route add 203.0.113.102 via 192.168.0.1 dev eth0
```

To make this route persistent, the command can be added as `PostUp = ip route ...` to the `[Interface]` section of `wg0.conf`. However, on certain setups (e.g. using `wg-quick@.service` in combination with NetworkManager) this might fail on resume. Furthermore, this only works for a static network setup and fails if gateways or devices change (e.g. using Ethernet or Wi-Fi on a laptop).

Using NetworkManager, a more flexible solution is to start WireGuard using a dispatcher script. As root, create

```
/etc/NetworkManager/dispatcher.d/50-wg0.sh
#!/bin/sh
case $2 in
  up)
    wg-quick up wg0
    ip route add <endpoint ip> via $IP4_GATEWAY dev $DEVICE_IP_IFACE
    ;;
  pre-down)
    wg-quick down wg0
    ;;
esac
```

If not already running, start and enable `NetworkManager-dispatcher.service`. Also make sure that NetworkManager is not managing routes for `wg0`, see #Routes are periodically reset.

使用 systemd-networkd 时睡眠导致连接丢失

**本文或本章节可能需要合并到systemd-networkd。**

附注： Might reach more people on the dedicated page.（在 Talk:WireGuard#Merge proposal of Connection lost after sleep using systemd-networkd 中讨论）

systemd version 253 introduced a change in how network interfaces are reconfigured when resuming from a suspended state. In doing so, network connections managed by systemd-networkd will lose connection to the wireguard interface. Unless a kill switch is configured, this risks exposing the public IP address after resuming from suspend. To fix this, uncomment and change the value to `no` for `ManageForeignRoutingPolicyRules` in `/etc/systemd/networkd.conf`.

参见

- Wikipedia:WireGuard
- Jason Donenfeld 的演讲
- 邮件列表
- 非官方 WireGuard 文档
- Debian:Wireguard

9.6 OpenVPN

难度 高级 | **篇幅** 30.3 KB | **所属** VPN

本节将讲到

- 安装
- 内核配置
- 连接到第三方提供的 VPN
- 从头开始创建公钥基础设施 (PKI)
- 一个基本的三层 IP 路由配置

**这篇文章或章节的翻译质量不佳。**

原因：Still out of date, even after the 2021-12-01 update, e.g. this page still references `rc.conf`（在 Talk:OpenVPN# 中讨论）

本文介绍了 OpenVPN 的基本的安装与配置过程，适用于个人使用与小型商业使用。要了解更多信息，请访问官方网站 HOWTO、OpenVPN 2.4 手册页以及 Manual。OpenVPN 是一个健壮的、高度灵活的 VPN 守护进程。它支持 SSL/TLS 安全、以太网桥接)、经由代理的 TCP 或 UDP 隧道和 NAT。另外，它也支持动态 IP 地址以及 DHCP，可伸缩性足以支持数百或数千用户的使用场景，同时可移植至大多数主流操作系统平台上。

OpenVPN 与 OpenSSL 库紧密相关，并由此获得许多加密功能。它除了支持使用预共享密钥（静态密钥模式）或使用非对称加密的公钥（SSL/TLS 模式）这两种传统加密方式之外。还支持不加密的 TCP/UDP 隧道。

OpenVPN 配合大多数平台上都有的 TUN/TAP 虚拟网络接口一起使用。总体而言，它旨在提供 IPSec 的许多关键功能，但占用空间相对较小。OpenVPN 由 James Yonan 编写，并在 GNU 通用公共许可证 (GPL) 下发布。

安装

安装 openvpn包，它同时支持服务器和客户端模式。

可用的前端：

- **NetworkManager OpenVPN** — OpenVPN 的 NetworkManager VPN 插件。

: https://wiki.gnome.org/Projects/NetworkManager/VPN || networkmanager-openvpn包

- **QOpenVPN** — 用 PyQt 为基于 systemd 的发行版编写的简单 OpenVPN GUI。

: https://github.com/xmikos/qopenvpn || qopenvpn包

- **eOVPN** — 用于连接、管理和更新 OpenVPN 配置的应用程序。

: https://github.com/jkotra/eOVPN || eovpnAUR

内核配置

OpenVPN 需要 TUN/TAP 的支持，默认内核已经进行了正确的配置。自定义的内核需要启用 `tun` 模块。

```
Kernel config file
 Device Drivers
  --> Network device support
    [M] Universal TUN/TAP device driver support
```

详情参阅 Kernel modules

连接到第三方提供的 VPN

要连接到第三方提供的 VPN 服务，可以忽略以下大部分内容，尤其是在服务器设置方面。 从 #客户端配置文件开始，然后跳到 #启动 OpenVPN。 应使用提供商证书和说明，请参阅 Category:VPN providers 以获取可适用于其他提供商的示例。 Linux Containers 中的 OpenVPN 客户端也有一般适用的说明，同时它更进一步，将 OpenVPN 客户端进程隔离到容器中。

注意：大多数免费的 VPN 提供商会（通常只）提供 PPTP, 它更容易设置和配置，但 不安全。

从头开始创建公钥基础设施 (PKI)

在设置 OpenVPN 服务器时，用户需要创建一个公开密钥基础建设 (PKI)，这在 Easy-RSA 文章中有详细说明。 按照单独文章中的步骤创建所需的证书、私钥和相关文件后，此时 `/etc/openvpn/server` 中应该有 5 个文件：

```
ca.crt
dh.pem
servername.crt
servername.key
ta.key
```

或者，从 OpenVPN 2.4 开始，可以使用 Easy-RSA 使用椭圆曲线生成证书和密钥。有关详细信息，请参阅 OpenVPN 文档。

一个基本的三层 IP 路由配置

注意：除非另有明确说明，本文的其余部分均假设基本的第 3 层 IP 路由配置。

OpenVPN 是一个非常通用的软件，可以进行多种配置，实际上机器既可以是服务器，也可以是客户端。

随着 v2.4 的发布，服务器配置存储在 `/etc/openvpn/server` 中，客户端配置存储在 `/etc/openvpn/client` 中，每种模式都有自己的systemd 单元，即 `openvpn-client@.service` 和 `openvpn-server@.service`。

示例配置

OpenVPN 软件包附带了一系列用于不同目的的示例配置文件。示例服务器和客户端配置文件是基本 OpenVPN 设置的理想起点，具有以下功能：

- 使用 Public Key Infrastructure (PKI) 进行身份验证。
- 使用虚拟 TUN 网络接口（OSI 第 3 层 IP 路由）创建 VPN。
- 监听 UDP 端口 1194 上的客户端连接（OpenVPN 的官方 IANA 端口号 ）。
- 将虚拟地址分配给来自 10.8.0.0/24 子网的连接客户端。

有关更高级的配置，请参阅 openvpn(8) 手册页和 OpenVPN 文档。

服务器配置文件

注意：请注意，如果服务器位于防火墙或 NAT 转换路由器后面，则必须将 OpenVPN 端口转发到服务器。

将示例服务器配置文件 `/usr/share/openvpn/examples/server.conf` 复制到 `/etc/openvpn/server/server.conf`。

编辑文件，至少进行以下更改：

```
/etc/openvpn/server/server.conf
ca ca.crt
cert servername.crt
key servername.key
dh dh.pem

tls-crypt ta.key # Replaces tls-auth ta.key 0

user nobody
group nobody
```

如果使用带有椭圆曲线的 TLS，请指定 `dh none` 和 `ecdh-curve secp521r1`。使用椭圆曲线时不使用 DH 参数文件。从 OpenVPN 2.4.8 开始，需要在服务器配置中指

定椭圆曲线的类型。否则服务器将无法识别曲线类型并可能使用不兼容的曲线类型，从而导致身份验证错误。

强化服务器

如果优先考虑安全性，则建议进行其他配置，包括：限制服务器使用强密码/身份验证方法和（可选）将启用的 TLS 密码集限制为较新的密码。从 OpenVPN 2.4 开始，服务器和客户端将在 TLS 模式下自动协商 **AES-256-GCM**。

将以下内容添加到 `/etc/openvpn/server/server.conf`：

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：使用说明属于维基文本，不属于代码块。避免诸如“边缘安全性”之类的主观术语。（在Talk:OpenVPN讨论）

最安全

```
/etc/openvpn/server/server.conf
cipher AES-256-GCM
auth SHA512
tls-version-min 1.3
#Uncomment  tls-cipher to limit possible negotiation options to the strongest ciphers, doing so it's no longer possible to generate certs with current easyrsa, more information
#tls-cipher TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
```

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：兼容性选项不适合强化部分。（在Talk:OpenVPN讨论）

与大多数设备的兼容性

```
/etc/openvpn/server/server.conf
cipher AES-256-GCM
auth SHA512
tls-version-min 1.2
tls-cipher TLS-DHE-RSA-WITH-AES-256-GCM-SHA384:TLS-DHE-RSA-WITH-AES-128-GCM-SHA256:TLS-DHE-RSA-WITH-AES-256-CBC-SHA:TLS-DHE-RSA-WITH-CAMELLIA-256-CBC-SHA:TLS-DHE-RSA-WITH-AES-128-CBC-SHA:TLS-DHE-RSA-WITH-CAMELLIA-128-CBC-SHA
```

注意：

- .ovpn 客户端配置文件“必须”包含匹配的密码和身份验证行才能正常工作（至少对于 iOS 和 Android 客户端）。
- 错误地使用 `tls-cipher` 可能会导致调试连接困难，并且可能没有必要。有关更多信息，请参阅 OpenVPN 的社区维基。

启用压缩

上游不建议启用压缩；这样做会向服务器打开所谓的 VORACLE 攻击向量。请参阅这篇文章。

非标准端口和/或协议

一般建议使用 OpenVPN over UDP，因为 TCP over TCP 不是一个好主意。

某些网络可能不允许默认端口和/或协议上的 OpenVPN 连接。规避这一点的一种策略是模拟 HTTPS 流量，这很可能是畅通无阻的。

为此，请将 `/etc/openvpn/server/server.conf` 配置为：

```
/etc/openvpn/server/server.conf
port 443
proto tcp
```

注意：.ovpn 客户端配置文件 **必须**包含匹配的端口和原型线才能正常工作！

在物理机的不同端口上运行多个 OpenVPN 实例

可以在同一台机器上运行多个并发的 OpenVPN 实例。每个服务器都需要在 `/etc/openvpn/server/` 中定义为单独的 .conf 文件。至少，并行服务器需要在不同的端口上运行。一个简单的设置将流量连接到一个单独的 IP 池。更高级的设置超出了本指南的范围。

考虑这个例子，运行 2 个并发服务器，一个端口 443/udp，另一个端口 80/tcp。

首先修改创建的 `/etc/openvpn/server/server.conf` 如下：

```
/etc/openvpn/server/server.conf
port 443
proto udp
server 10.8.0.0 255.255.255.0
```

现在复制它并修改副本以在 80/tcp 上运行：

```
/etc/openvpn/server/server2.conf
port 80
proto tcp
server 10.8.1.0 255.255.255.0
```

请务必在防火墙中设置相应的条目，请参阅#防火墙配置中的相关部分。

客户端配置文件

将示例客户端配置文件 `/usr/share/openvpn/examples/client.conf` 复制到 `/etc/openvpn/client/`。

编辑以下内容：

- `remote` 指令反映服务器的 Fully Qualified Domain Name、主机名（客户端已知）或其 IP 地址。
- 取消注释 `user` 和 `group` 指令以删除权限。
- `ca`、`cert` 和 `key` 参数反映密钥和证书的路径和名称。
- 启用 TLS HMAC 握手保护（`--tls-crypt` 或 `--tls-auth`）。

```
/etc/openvpn/client/client.conf
```

```
client
remote elmer.acmecorp.org 1194

user nobody
group nobody
ca ca.crt
cert client.crt
key client.key

tls-crypt ta.key # Replaces tls-auth ta.key 1
```

以非特权用户身份运行

在配置文件中使用选项 `user nobody` 和 `group nobody` 会使*OpenVPN*在建立连接后放弃其 `root` 权限。缺点是在 VPN 断开连接后，守护进程无法再次删除其设置的网络路由。如果想要在没有 VPN 连接的情况下限制传输流量，那么延迟路由可能被认为是有益的。但是，也可能发生 OpenVPN 服务器在隧道运行时向路由推送更新的情况。具有删除权限的客户端将无法执行更新并退出并出现错误。

由于似乎需要手动操作来管理路由，因此选项 `user nobody` 和 `group nobody` 可能看起来不受欢迎。然而，根据设置，有不同的方法来处理这些情况：

- 对于单元的错误，一个简单的方法是edit它并在`[Service]`部分添加一个`Restart=on-failure`。但是，仅此一项不会删除任何过时的路由，因此可能会发生重新启动的隧道未正确路由的情况。
- 包中包含`/usr/lib/openvpn/plugins/openvpn-plugin-down-root.so`，可以用来让*openvpn*fork一个只有root权限的进程从主进程接收到关闭信号时执行自定义脚本的任务，该进程正在处理具有删除权限的隧道（另请参阅其 README）。

下面链接的 OpenVPN HowTo 更进一步，创建了一个专用的非特权用户/组，而不是现有的 `nobody`。这样做的好处是避免了在守护进程之间共享用户时的潜在风险：

- OpenVPN HowTo 解释了如何创建非特权用户模式和包装脚本以自动恢复路由的另一种方法。
- 可以让 OpenVPN 首先以非特权用户身份启动，而无需以 root 身份运行，请参阅这个 OpenVPN wiki（howto）。 howto 假设存在 System V init，而不是 Systemd 并且不包括 `--up`/`--down` 脚本的处理 - 那些应该被处理与 *ip* 命令相同的方式，额外注意访问权限。
- 也可以从无特权的 podman 容器中运行 OpenVPN，请参阅这部分 OpenVPN HowTo

提示：#openvpn-unroot 描述了一种自动化上述设置的工具。

将证书转换为加密的 .p12 格式

某些软件只会读取存储在密码加密的 .p12 文件中的 VPN 证书。这些可以使用以下命令生成：

```
# openssl pkcs12 -export -inkey keys/bugs.key -in keys/bugs.crt -certfile keys/ca.crt -
out keys/bugs.p12
```

测试 OpenVPN 配置

在服务器上（以 root 用户）运行 `openvpn /etc/openvpn/server/server.conf`，在客户端（以 root 用户）运行 `openvpn /etc/openvpn/client/client.conf`。示例输出应类似于以下内容：

```
# openvpn /etc/openvpn/server/server.conf
Wed Dec 28 14:41:26 2011 OpenVPN 2.2.1 x86_64-unknown-linux-gnu [SSL] [LZO2] [EPOLL]
[eurephia] built on Aug 13 2011
Wed Dec 28 14:41:26 2011 NOTE: OpenVPN 2.1 requires '--script-security 2' or higher to
call user-defined scripts or executables
Wed Dec 28 14:41:26 2011 Diffie-Hellman initialized with 2048 bit key
...
Wed Dec 28 14:41:54 2011 bugs/95.126.136.73:48904 MULTI: primary virtual IP for
bugs/95.126.136.73:48904: 10.8.0.6
Wed Dec 28 14:41:57 2011 bugs/95.126.136.73:48904 PUSH: Received control message:
'PUSH_REQUEST'
Wed Dec 28 14:41:57 2011 bugs/95.126.136.73:48904 SENT CONTROL [bugs]:
'PUSH_REPLY,route 10.8.0.1,topology net30,ping 10,ping-restart 120,ifconfig 10.8.0.6
10.8.0.5' (status=1)
# openvpn /etc/openvpn/client/client.conf
Wed Dec 28 14:41:50 2011 OpenVPN 2.2.1 i686-pc-linux-gnu [SSL] [LZO2] [EPOLL]
[eurephia] built on Aug 13 2011
Wed Dec 28 14:41:50 2011 NOTE: OpenVPN 2.1 requires '--script-security 2' or higher to
call user-defined scripts or executables
...
Wed Dec 28 14:41:57 2011 GID set to nobody
Wed Dec 28 14:41:57 2011 UID set to nobody
Wed Dec 28 14:41:57 2011 Initialization Sequence Completed
```

查找IP地址分配给服务器上的tunX接口，从客户端ping。

查找IP地址分配给客户端的tunX接口，从服务器ping。

注意：如果使用防火墙，请确保 TUN 设备上的 IP 数据包没有被阻止。

使用 Fragment 和 MSS 配置 MTU

如果在通过 OpenVPN 使用（远程）服务时遇到问题（例如网页浏览、DNS、NFS），则可能需要手动设置 MTU 值。

以下消息可能指示应调整 MTU 值：

```
read UDPv4 [EMSGSIZE Path-MTU=1407]: Message too long (code=90)
```

为了获得最大段大小 (MSS)，客户端需要沿着到服务器的路径发现最小的 MTU。为了做到这一点 ping 服务器并禁用碎片，然后指定最大数据包大小：

```
# ping -M do -s 1500 -c 1 example.com
```

每次将 1500 值减 10，直到 ping 成功。

注意：不支持"片段"指令的客户端（例如 OpenELEC，应用程序) 无法连接到使用 `fragment` 指令的服务器。请参阅 `mtu-test` 作为替代解决方案。

更新客户端配置以使用成功的 MTU 值，例如：

```
/etc/openvpn/client/client.conf
remote example.com 1194

tun-mtu 1400
mssfix 1360
```

可能会指示 OpenVPN 每次在客户端连接时测试 MTU。请耐心等待，因为客户端可能不会通知正在运行的测试，并且连接在完成之前可能会显示为无效。以下将增加大约 3 分钟的 OpenVPN 启动时间。建议配置片段大小，除非客户端将通过许多不同的网络进行连接并且瓶颈不在服务器端：

```
/etc/openvpn/client/client.conf
remote example.com 1194

mtu-test
```

IPv6

通过 IPv6 连接到服务器

从 OpenVPN 2.4 开始，当仅使用 `proto udp` 或 `proto tcp` 时，OpenVPN 将使用操作系统定义的 `AF_INET`，在大多数情况下，这将仅是 IPv4。要同时使用 IPv4 和 IPv6，请使用 `proto udp6` 或 `proto tcp6`。要仅强制执行 IPv4，请使用 `proto udp4` 或 `proto tcp4`。在较旧的 OpenVPN 版本上，一个服务器实例只能支持 IPv4 或 IPv6。

在隧道内提供 IPv6

为了在隧道内提供 IPv6，请将 IPv6 前缀路由到 OpenVPN 服务器。要么在网关上设置静态路由（如果分配了静态块），要么使用 DHCPv6 客户端通过 DHCPv6 前缀委派获取前缀（详情请参阅IPv6 Prefix delegation)）。还可以考虑使用来自地址块 fc00::/7 的唯一本地地址。这两种方法各有优缺点：

- 许多 ISP 仅提供动态更改的 IPv6 前缀。 OpenVPN 不支持前缀更改，因此每次更改前缀时都要更改 server.conf（也许可以通过脚本自动实现）。
- ULA 地址不会路由到 Internet，并且设置 NAT 不像使用 IPv4 那样简单。这意味着不能通过隧道路由整个流量。那些希望通过 IPv6 连接两个站点而不需要通过隧道连接到 Internet 的人可能希望使用 ULA 地址来轻松。

或者，NDP 代理应该可以工作。请参阅这个 StackExchange 帖子。

收到前缀（建议使用 /64）后，将以下内容附加到 server.conf：

```
server-ipv6 2001:db8:0:123::/64
```

这是相当于 OpenVPN 的默认 10.8.0.0/24 网络的 IPv6，需要从 DHCPv6 客户端获取。或者使用例如 fd00:1234::/64。

那些想要将路由推送到家庭网络（192.168.1.0/24 等效）的人，还需要附加：

```
push "route-ipv6 2001:db8:0:abc::/64"
```

OpenVPN 尚不包括 DHCPv6，因此没有方法例如通过 IPv6 推送 DNS 服务器。这需要通过 IPv4 来完成。 OpenVPN Wiki 提供了一些其他配置选项。

启动 OpenVPN

手动启动

systemd 服务配置

让 NetworkManager 开始连接

NetworkManager-native VPN 配置

图形界面配置

CLI 配置

与连接同步状态

故障排除

没有证书密码

通过服务器路由客户端流量

防火墙配置

防火墙

ufw
iptables
如果 VPN 出现故障，防止泄漏
ufw
vpnfailsafe
第 3 层 IPv4 路由
路由 LAN 的先决条件
路由表
将服务器 LAN 连接到客户端
将客户端 LAN 连接到服务器
连接客户端和服务器 LANs
连接客户端和客户端局域网
域名解析
pull-resolv-conf 自定义脚本
update-resolv-conf 自定义脚本
update-systemd-resolved 自定义脚本
使用 NetworkManager 覆盖 DNS 服务器
第 2 层以太网桥接

配置生成器

ovpngen

openvpn-unroot

故障排除

客户端守护进程在挂起后没有重新连接

一段时间不活动后连接断开

PID 文件不存在

路由配置失败， systemd-networkd

tls-crypt 解包错误：数据包太短

另见

准备证书和密钥数据

现在要创建所需的证书和密钥，可以在任何机器上完成，即使没有联网也可以进行。设置证书和密钥生成脚本的默认值。编辑 `/etc/openvpn/easy-rsa/vars`，设置 KEY_COUNTRY, KEY_PROVINCE, KEY_CITY, KEY_ORG 和 KEY_EMAIL 参数(不要留空任何参数)，然后导出环境变量。

```
# source ./vars
```

清理之前的密钥：

```
# ./clean-all
```

build-ca 脚本创建了 certificate authority (CA) ca.key，密钥认证机器需要这个密钥。服务器和客户端需要 ca.crt 证书。

```
# ./build-ca
```

`build-key-server` 为服务器创建一个证书和密钥对。使用中不要输入简单的密码或公司名。

```
# ./build-key-server <server-name>
```

`build-dh` 脚本创建服务器需要的 Diffie-Hellman pem 文件。

```
# ./build-dh
```

`build-key` 脚本创建客户端证书和密钥对。可以生成任意多个以给不同的客户端使用。只要保证客户端名 <client> 是唯一的。如果要用密码认证客户端，请使用 `build-key-pass` 脚本。

```
# ./build-key <client1>
# ./build-key <client2>
```

生成的文件都保存在 /etc/openvpn/easy-rsa/keys。如果有错误，可以通过运行 clean-all 脚本，然后从头开始。注意这将删除之前生成的证书和密钥。

```
# ./clean-all
```

最后一步是将所有需要的文件通过安全通道放到正确的机器上。ca.crt 需要放到所有服务器和客户端。server.crt, server.key 和 dh{n}.pem 文件放到服务器，client.crt 和 client.key 文件放到客户端。

配置服务器

复制默认服务器配置文件

```
# cp /usr/share/openvpn/examples/server.conf /etc/openvpn/openvpn.conf
```

使用 PAM 和密码认证

```
port 1194
proto udp
dev tap
ca /etc/openvpn/easy-rsa/keys/ca.crt
cert /etc/openvpn/easy-rsa/keys/<MYSERVER>.crt
key /etc/openvpn/easy-rsa/keys/<MYSERVER>.key
dh /etc/openvpn/easy-rsa/keys/dh1024.pem
server 192.168.56.0 255.255.255.0
ifconfig-pool-persist ipp.txt
;learn-address ./script
client-to-client
;duplicate-cn
keepalive 10 120
;tls-auth ta.key 0
comp-lzo
;max-clients 100
;user nobody
;group nobody
persist-key
persist-tun
status /var/log/openvpn-status.log
verb 3
client-cert-not-required
username-as-common-name
plugin /usr/lib/openvpn/openvpn-auth-pam.so login
```

使用证书认证

```
port 1194
proto tcp
dev tun0

ca /etc/openvpn/easy-rsa/keys/ca.crt
cert /etc/openvpn/easy-rsa/keys/<MYSERVER>.crt
key /etc/openvpn/easy-rsa/keys/<MYSERVER>.key
dh /etc/openvpn/easy-rsa/keys/dh1024.pem

server 10.8.0.0 255.255.255.0
ifconfig-pool-persist ipp.txt
keepalive 10 120
comp-lzo
user nobody
group nobody
persist-key
persist-tun
status /var/log/openvpn-status.log
verb 3

log-append /var/log/openvpn
```

```
status /tmp/vpn.status 10
```

通过服务器路由

将下面内容写入服务器的 `openvpn.conf` 配置文件，"192.168.1.1" 修改为外部 DNS IP 地址。

```
push "dhcp-option DNS 192.168.1.1"
push "redirect-gateway def1"
```

使用 iptable 进行 NAT 转发：

```
echo 1 > /proc/sys/net/ipv4/ip_forward
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
```

如果运行在 OpenVZ VPS 环境，参见 :

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o venet0 -j SNAT --to (venet0 ip)
```

如果一切正常，保存修改，编辑 `/etc/conf.d/iptables` 设置 IPTABLES_FORWARD=1

```
/etc/rc.d/iptables save
```

客户端配置

配置客户端的 .conf 文件

使用密码认证

```
client
dev tap
proto udp
remote <address> 1194
resolv-retry infinite
nobind
persist-tun
comp-lzo
verb 3
auth-user-pass passwd
ca ca.crt
```

被 `auth-user-pass` 引用的 `passwd` 文件必须包含如下两行:

- 第一行 - username
- 第二行 - password

证书验证

```
client
remote <MYSERVER> 1194
dev tun0
proto tcp
resolv-retry infinite
nobind
persist-key
persist-tun
verb 2
ca ca.crt
cert client1.crt
key client1.key
comp-lzo
```

将`ca.crt`, `client1.crt` 和 `client1.key`复制到远程计算机：

安装隧道模块:

```
 # sudo modprobe tun
```

要让 **tun** 模块自动启动，请将其加入 `/etc/rc.conf` 的 Modules 行。

DNS

系统使用的 DNS 服务器定义在`/etc/resolv.conf`。通常此文件由控制系统网络的模块(Wicd, NetworkManager 等)维护。然而，如果希望通过远程服务器解析地址，OpenVPN 需要修改这个文件。

安装 **openresolv**软件包，它可以实现多个程序互不影响的修改 `resolv.conf`。安装后通过重启网络连接，保证 resolv.conf 是由 "resolvconf" 创建而且 DNS 解析工作正常。openresolv 不需要配置，它会自动检测和使用网络系统。

然后将如下脚本保存到`/usr/share/openvpn/update-resolv-conf`:

```
#!/bin/bash
#
# Parses DHCP options from openvpn to update resolv.conf
# To use set as 'up' and 'down' script in your openvpn *.conf:
# up /etc/openvpn/update-resolv-conf
# down /etc/openvpn/update-resolv-conf
#
# Used snippets of resolvconf script by Thomas Hood <jdthood@yahoo.co.uk>
# and Chris Hanson
# Licensed under the GNU GPL.  See /usr/share/common-licenses/GPL.
#
# 05/2006 chlauber@bnc.ch
#
# Example envs set from openvpn:
# foreign_option_1='dhcp-option DNS 193.43.27.132'
# foreign_option_2='dhcp-option DNS 193.43.27.133'
# foreign_option_3='dhcp-option DOMAIN be.bnc.ch'

[ -x /usr/sbin/resolvconf ] || exit 0

case $script_type in

up)
   for optionname in ${!foreign_option_*} ; do
      option="${!optionname}"
      echo $option
      part1=$(echo "$option" | cut -d " " -f 1)
      if [ "$part1" == "dhcp-option" ] ; then
         part2=$(echo "$option" | cut -d " " -f 2)
         part3=$(echo "$option" | cut -d " " -f 3)
         if [ "$part2" == "DNS" ] ; then
            IF_DNS_NAMESERVERS="$IF_DNS_NAMESERVERS $part3"
         fi
         if [ "$part2" == "DOMAIN" ] ; then
            IF_DNS_SEARCH="$part3"
         fi
      fi
   done
   R=""
   if [ "$IF_DNS_SEARCH" ] ; then
           R="${R}search $IF_DNS_SEARCH
"
   fi
   for NS in $IF_DNS_NAMESERVERS ; do
           R="${R}nameserver $NS
```

```
"
   done
   echo -n "$R" | /usr/sbin/resolvconf -a "${dev}.inet"
   ;;
down)
   /usr/sbin/resolvconf -d "${dev}.inet"
   ;;
esac
```

设置脚本可执行属性：

```
$ chmod +x /usr/share/openvpn/update-resolv-conf
```

然后将下面内容加入 OpenVPN 客户端的配置文件：

```
script-security 2
up /usr/share/openvpn/update-resolv-conf
down /usr/share/openvpn/update-resolv-conf
```

现在再启动 OpenVPN 连接，就能发现 `resolv.conf` 文件已经更新，关闭连接后恢复正常。

启动 OpenVPN

手动启动

如需对 VPN 连接进行调试，可以以 root 身份手动运行 `openvpn /etc/openvpn/client.conf` 以启动客户端守护程序。服务器端同样可以如此启动，只需替换为服务器端的配置文件（例，`openvpn /etc/openvpn/server.conf`）。

systemd 服务配置

若需在系统启动时自动启动 OpenVPN，对服务器端与客户端，都可以采用在对应机器上启用 `openvpn@*<configuration>*.service` 的方式配置。

例如，如果客户端配置文件是 `/etc/openvpn/client.conf`，则服务名称应为 `openvpn@client.service`。或者，如果服务器端配置文件是 `/etc/openvpn/server.conf`，则服务名称应为 `openvpn@server.service`。

让 NetworkManager 启动连接

在客户端，您可能并不需要一直运行 VPN 隧道，或者您仅仅想要为特定的 NetworkManager 连接建立隧道。要实现这一点，您可以向 `/etc/NetworkManager/dispatcher.d/` 添加脚本。在下列示例中，"Provider" 是 NetworkManager 连接的名称：

```
/etc/NetworkManager/dispatcher.d/10-openvpn
#!/bin/sh
case "$2" in
  up)
    if [ "$CONNECTION_ID" = "Provider" ]; then
      systemctl start openvpn@client
    fi
```

```
  ;;
  down)
    systemctl stop openvpn@client
  ;;
esac
```

请查看 NetworkManager#Network services with NetworkManager dispatcher 以了解详情。

Gnome 配置

如果您想要经由 Gnome 内置的网络配置来连接到一个 OpenVPN 服务器，请按照如下步骤进行配置： 首先，安装 `networkmanager-openvpn`。 然后，在设置菜单选中网络设置，在VPN点击+新建连接。从这里你可以选择OpenVPN并手动输入设置，或者您也可以选择导入配置文件，如果您已经有一个的话。如果您按照上述说明，配置文件会在`/etc/openvpn/client.conf`。打开开关就可以连接VPN。

参见

- OpenVPN 官方站点

串流

本章处理两条方向相反的数据流：把网络上的直播与点播内容拉回本地播放、留存，以及把本机画面与声音推给直播平台。前者用 Streamlink 与 yt-dlp，后者用 OBS。建议的顺序是：先 Streamlink 的安装与用法，再 yt-dlp 的安装、配置与用法，然后借《流式传输至 twitch.tv》走通一次推送，最后看 OBS 的配置与疑难解答。

VLC 与 Mpv 篇幅最长，更像长期使用的播放器手册，时间紧时先读安装，其余留作查阅。最容易踩的坑是方向：拉流工具不会替你推流，OBS 也不负责下载，装好了却没有画面，多半是把两者当成了同一类工具。

10.1　Streamlink

难度 入门 | **篇幅** 3.4 KB | **所属** 串流

本节将讲到

- 安装
- 使用
- 另请参阅

Streamlink 是一个用 Python 编写的命令行工具，可将各类服务的视频流传输至常见的视频播放器（如 VLC、MPlayer、mpv）。完整的播放器支持列表请参阅播放器兼容性。

该项目是从已不再维护的 Livestreamer 分支而来。

Streamlink 基于插件系统构建，可轻松添加对新流媒体服务的支持。绝大多数主流流媒体平台均已兼容，包括 Dailymotion、Livestream、Twitch、UStream、YouTube Live、哔哩哔哩等。完整的插件列表请参阅插件页面。

安装

安装 streamlink包 或其开发版本 streamlink-gitAUR。

使用

该软件包提供了 `streamlink` 命令行工具，使用起来非常简单：

```
$ streamlink -p 播放器 url stream
```

- *播放器*——所使用的媒体播放器的可执行文件名，例如 `vlc`。也可以指定完整路径，例如 `/usr/bin/vlc`。如果安装，默认使用 VLC。

- `*url*`——流媒体的 URL 地址，HTTP(S) 的协议前缀（`http://`、`https://`）通常可以省略，具体取决于所用插件的实现方式。
- `*stream*`——要播放的具体流，主要用于选择视频画质：`best` 表示最高画质，`worst` 表示最低画质，某些插件可能还支持其他画质选项。如果不提供此参数，则会输出所有当前可用流的列表。

例如：

```
$ streamlink -p mpv dailymotion.com/embed/video/x1b1h6o worst
```

有关可用选项的完整列表，请参阅 streamlink(1)。

若要将视频流保存到本地硬盘：

```
$ streamlink -o ~/$current_time.m2t url best,high
```

Twitch

使用以下命令播放 Twitch 直播：

```
$ streamlink -p 播放器 twitch.tv/频道名 画质
```

例如：

```
$ streamlink -p vlc twitch.tv/archlinux best
```

可用的画质选项可能（视原画质而定）包括：`audio_only`、`160p（worst）`、`360p`、`480p`、`720p60`、`1080p60（best）`。

哔哩哔哩

使用以下命令播放哔哩哔哩直播：

```
$ streamlink -p 播放器 live.bilibili.com/房间号 画质
```

例如：

```
$ streamlink -p vlc live.bilibili.com/22102223 best
```

可用的画质选项包括：`hls_alt（worst）`、`hls_alt2`、`hls`、`httpstream_alt` 和 `httpstream（best）`。

另请参阅

- Streamlink 的 GitHub 仓库
- Streamlink 官方文档（英文）

10.2 Yt-dlp

难度 入门 | **篇幅** 5.4 KB | **所属** 串流

本节将讲到

- 安装
- 配置

- 用法
- 方法与技巧
- 另请参阅

翻译状态：

- 本文（或部分内容）译自 yt-dlp，最近一次同步于 2025-10-03，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

yt-dlp 是一款可让您轻松从一千多个网站下载视频和音频的命令行程序。查看受支持的网站列表。

注意：yt-dlp 是 youtube-dl 的一个分支，创建于原项目停滞之后。上游的 youtube-dl 仍可通过 youtube-dlAUR 来安装；本页的命令通常仍可正常使用，但请参阅默认行为差异列表以了解具体区别。

安装

安装yt-dlp包。同时建议安装 FFmpeg（用来封装部分网站的文件），请参阅可选依赖项。

yt-dlp 还有各种图形前端，如 tartubeAUR 和 yt-dlg-gitAUR。

您还可以安装 yt-dlp-drop-inAUR，它提供了一个占位的 `/usr/bin/youtube-dl` 可执行文件（只是重定向到 *yt-dlp*），以兼容那些仍在寻找 *youtube-dl* 可执行文件的过时程序。

配置

系统全局配置文件是 `/etc/yt-dlp.conf`，用户的配置文件为`~/.config/yt-dlp/config`。配置示例：

```
--ignore-errors
# --no-playlist

# 保存至 ~/Videos
-o ~/Videos/%(title)s.%(ext)s

# 首选1080p或更低的分辨率
-f bestvideo[height<=?1080]+bestaudio/best
```

更多信息请参见

自定义配置文件也可以指定：

```
$ yt-dlp URL --config-locations PATH
```

用法

请参见 yt-dlp(1) 手册。

```
$ yt-dlp [OPTIONS] URL
```

提示：在某些情况下（如 YouTube）`*URL*` 可以用视频ID代替。

格式选择

当多种格式的视频可用时，*yt-dlp*将默认下载最好的格式。

获取可用格式的列表：

```
$ yt-dlp -F URL
```

选择一个特定的要下载：

```
$ yt-dlp -f format URL
```

提取音频

使用`-x`下载音频（需要 FFmpeg）：

```
$ yt-dlp -x -f bestaudio URL
```

根据可用的源流，这通常可以修正纯音频容器。若无可用的纯音频流，请从上述示例中移除 `-f bestaudio` 参数。此操作会先下载视频文件，并在后续处理中提取其音频。默认情况下会删除已下载的视频文件，添加 `-k` 参数可保留视频文件。

同时包括专辑封面：（需要atomicparsley包）：

```
$ yt-dlp -x -f bestaudio[ext=m4a] --add-metadata --embed-thumbnail URL
```

字幕

查看哪些语言可用：

```
$ yt-dlp --list-subs URL
```

下载带有选定字幕（用逗号分隔）的视频：

```
$ yt-dlp --write-sub --sub-lang LANG URL
```

对于自动生成的字幕：

```
$ yt-dlp --write-auto-sub --sub-lang LANG URL
```

添加`--skip-download`仅获取字幕。

Cookie

要导入 cookie，添加 `--cookies-from-browser <browser>`。

例如，从 chromium 导入：

```
$ yt-dlp --cookies-from-browser chromium URL
```

方法与技巧

更快的下载

一些网站限制了传输速度。您通常可以通过选择非DASH流或使用支持多连接下载的外部下载器 aria2 来绕过这种限制。例如：

```
$ yt-dlp --downloader aria2c --downloader-args '-c -j 3 -x 3 -s 3 -k 1M' URL
```

播放列表

将 yt-dlp 用于播放列表通常归结为以下选项：

```
$ yt-dlp --ignore-errors --continue --no-overwrites --download-archive progress.txt
usual options URL
```

这组选项允许下载在中断后仍能有效继续。如果您正在进行归档操作，请添加您可能需要的常规选项，例如 `--write-xxx` 和 `--embed-xxx`。

修剪（部分下载）

可以通过将 `yt-dlp -g -f *format* *URL*` 的输出作为 *ffmpeg* 的输入，并结合 `-ss`（用于输入）、`-t` 和 `-c copy` 选项来下载视频片段。

来自剪贴板的URL

可以通过设置一个 Shell 别名、桌面启动器（英语：desktop launcher）或键盘快捷键，从 X Window 选区中输出选中（或复制）的 URL 以下载视频或音频。具体方法可参阅剪贴板工具。

另请参阅

- GitHub 代码仓库

10.3 流式传输至 twitch.tv

难度 进阶 | **篇幅** 8.3 KB | **所属** 串流

本节将讲到

- Twitch 广播要求
- 图形用户界面解决方案
- 命令行界面解决方案

Twitch.tv 是一个流行的 RTMP 的流媒体服务。因此，本页面为 Twitch.tv 提供流媒体服务的解决方案列表。使用案例可能包括流媒体游戏、Linux 桌面等。

Twitch 广播要求

根据 Twitch.tv 支持页面:

视频要求

- 编解码器：H.264 (x264)
- 模式：严格恒定比特率
- 关键帧间隔：2 秒

音频要求

- 编解码器：AAC-LC，立体声或单声道
- 最大比特率：160 kbps
- 采样频率：任意

注意：已取消支持 MP3 编解码器

其他要求

- 页面未列出对 Y'UV420p 像素格式的要求，因为对 Y'UV444 的支持还不广泛。

图形用户界面解决方案

- Open Broadcaster Software（obs-studio包）是一款流行的流媒体程序。Alpha Linux 构建的安装包（obs-studio-gitAUR）也可用于编译和测试。
- Castawesome（castawesomeAUR）是一款用于 ffmpeg 流媒体的 Gtk3 前端，内置 Twitch.tv 支持。
- SimpleScreenRecorder（lib32-simplescreenrecorder包）可对 Twitch 串流（直播）进行配置：
- 容器需设置为 FLV
- RTMP URL 需要放在‘另存为’栏中
- 确保没有选中“文件分段”
- 视频编解码器设置为 libx264（不是 H.264）
- 设定合理的比特率，如 2000 kbps
- 在自定义选项栏中，输入 `preset=fast,minrate=2000,maxrate=2000,bufsize=2000,keyint=60`

: : > **注意**：“minrate”、“maxrate”和“bufsize”的值应等于比特率。

命令行界面解决方案

- FFmpeg (ffmpeg包)
- PyLivestream (pylivestream-gitAUR)
- GStreamer (gstreamer包)
- gst-live

FFmpeg

下面的解决方案利用了 FFmpeg 来实现向 Twitch.tv 串流（直播）。

shell 脚本方法

使用 FFmpeg 实现的向 Twitch.tv 串流（直播）的 shell 脚本。它支持对桌面和 OpenGL 元素进行串流。可通过运行 `stream-to-twitch path/to/stream_key` 在 shell 中调用，密钥则使用 pass 安全存储。脚本内容如下：

```
/usr/local/sbin/stream-to-twitch
#!/usr/bin/env sh
#
# Stream screen and audio (speakers and microphone) to Twitch.tv using FFmpeg.
#
# Usage: stream-to-twitch path/to/key

set -euo pipefail

######################################
# Stream to Twitch.tv.
# Globals:
#   None.
# Arguments:
#   Stream key. A string.
# Returns:
#   None.
######################################
stream_to_twitch() {
    res_input="1920x1080" # input resolution
    res_output="1280x720" # output resolution
    fps="30" # target FPS
    gop="60" # i-frame interval, should be double of fps
    gop_min="30" # min i-frame interval, should be equal to fps
    probesize="42M" # https://stackoverflow.com/a/57904380
    threads="2" # max 6
    cbr="1000k" # constant bitrate (should be between 1000k-3000k)
    quality="ultrafast" # one of the many FFmpeg presets
    audio_input_speakers="0" # speakers' sink id
    audio_input_mic="default" # microphone's sink id
    audio_rate="44100"
    stream_server="live-prg" # see https://stream.twitch.tv/ingests for list
    stream_key="${1}" # key will be passed as an agument from the command line
    loglevel="warning" # supress unecessary information from printing

    ffmpeg \
        -loglevel "${loglevel}" \
        -f x11grab -s "${res_input}" -r ${fps} -probesize ${probesize} -i :0.0 \
        -f pulse -i "${audio_input_speakers}" \
        -f pulse -i "${audio_input_mic}" \
        -filter_complex "highpass=f=200,lowpass=f=3000[hl]; [hl]amix=inputs=2[a]" \
        -map 0:v -map [a] \
        -f flv -ac 2 -ar ${audio_rate} \
        -vcodec libx264 -g ${gop} -keyint_min ${gop_min} -b:v ${cbr} \
        -minrate ${cbr} -maxrate ${cbr} -pix_fmt yuv420p \
        -s ${res_output} -preset "${quality}" -tune film -acodec aac \
        -threads ${threads} -strict normal \
        -bufsize ${cbr} \
        "rtmp://${stream_server}.twitch.tv/app/${stream_key}"
}

# Get stream key securely stored with the password manager "pass"
# and pass the key to the script to start the stream.
stream_to_twitch "$(pass "${1}")"
```

提示：如果使用 PulseAudio，请运行 `pactl list sinks short` 寻找输入音频流。也可使用混音器（例如 pulsemixer包 或 pavucontrol包）在脚本运行时编辑音源。

注意：根据你的网络上传速度，你可能需要修改 FFmpeg 的参数。请使用参数明细作为参考。

<table class="wikitable"><caption>FFmpeg 参数明细</caption><tbody><tr><th>参数</th><th>描述</th></tr><tr><td><code>ffmpeg</code></td><td>转换器</td></tr><tr><td><code>-loglevel "${LOGLEVEL}"</code></td><td>-loglevel 设置日志输出级别</td></tr><tr><td><code>-f x11grab</code></td><td>-f 强制从 x11grab 输入</td></tr><tr><td><code>-s $RES_INPUT</code></td><td>-s 从输入源设置一个指定的图像尺寸，由变量 $RES_INPUT 提供</td></tr><tr><td><code>-r $FPS</code></td><td>-r 设置帧率为 $FPS</td></tr><tr><td><code>-probesize "${PROBESIZE}"</code></td><td>-probesize 设置要分析的数据大小，以获得流信息</td></tr><tr><td><code>-i :0.0</code></td><td>-i 获取显示（器）输入，在这里，是从 x11 的 :0.0 显示（器）获取。可以调整，例如：-i :0.0+500,100 是指在屏幕位置 500/100 处开始获取</td></tr><tr><td><code>-f pulse</code></td><td>强制从 PulseAudio 输入</td></tr><tr><td><code>-i "${AUDIO_INPUT_SPEAKERS</code>"}</td><td>选择扬声器的 sink ID</td></tr><tr><td><code>-i "${AUDIO_INPUT_MIC</code>"}</td><td>选择麦克风的 sink ID</td></tr><tr><td><code>-filter_complex ...</code></td><td>将过滤器应用到麦克风，以减少噪音并合并音频流</td></tr><tr><td><code>-map 0:v</code></td><td>映射视频流</td></tr><tr><td><code>-map [a]</code></td><td>映射音频流</td></tr><tr><td><code>-f flv</code></td><td>强制格式为 FLV</td></tr><tr><td><code>-ac 2</code></td><td>设置声道为 2</td></tr><tr><td><code>-ar "${AUDIO_RATE}"</code></td><td>-ar 设置音频比特率</td></tr><tr><td><code>-vcodec libx264</code></td><td>设置视频编解码器为 libx264</td></tr><tr><td><code>-b:v "$CBR"</code></td><td>-b:v 指定要改变到的视频比特率。比特率的值由 $CBR 决定</td></tr><tr><td><code>-pix_fmt yuv420p</code></td><td>设置像素格式为 Y'UV420p。否则默认使用 Y'UV444，这与 Twitch 不兼容</td></tr><tr><td><code>-s $RES_OUTPUT</code></td><td>-s 为输出设置一个指定的图像尺寸，由变量 $RES_OUTPUT 提供</td></tr><tr><td><code>-preset "{$QUALITY}"</code></td><td>设置预设的压缩质量和速度</td></tr><tr><td><code>-acodec aac</code></td><td>设置音频编解码器以使用 AAC</td></tr><tr><td><code>-threads 0</code></td><td>设置启动的 CPU 线程，为 0 则根据 CPU 核心数自动启动线程</td></tr></tbody></table>

10.4 Open Broadcaster Software

难度 进阶 | **篇幅** 8.9 KB | **所属** 串流

本节将讲到

- 安装
- 配置
- 提示和技巧
- 疑难解答
- 参见

翻译状态：

- 本文（或部分内容）译自 Open Broadcaster Software，最近一次同步于 2022-11-27，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

Open Broadcaster Software（OBS）是一款开源的跨平台视频录制和直播应用程序。它提供一项容易上手和可扩展的工作流程，具有可定制的场景、音量混合器、转场、过滤器等功能。

安装

可选择 obs-studio包 或 obs-studio-gitAUR 开发版本安装。

提供其他功能的客户端也可以使用。

- obs-studio-tytan652AUR
- obs-studio-browserAUR
- obs-vaapiAUR

配置

为方便配置，*工具* > *自动配置向导*可以快速设置录制和直播功能。该向导根据你的硬件（如果设置流媒体，还包括网络连接）自动选择比特率、分辨率和编码器。

硬件加速

硬件加速的编码和解码在性能、CPU/GPU 使用和质量方面是最好的。编码器可以在 *设置* > *输出* > *流媒体* > *编码器*中改变（可能要先将 *设置* > *输出* > *输出模式*为 *高级*）。如果没有检测到硬件编码器，请参阅硬件视频加速。

录制输出

OBS 默认在用户的家目录中输出录像，视频文件名中带有空格，并选择相同的编码器用于流媒体。输出路径、文件大小、文件格式、文件名样式等可以在 *设置* > *输出* > *录制*中改变。

热键

OBS 默认没有指定热键。所有的热键对在选择时以红色显示，可以使用相同的键来切换热键对的功能。

虚拟摄像机输出

从26.1版开始，OBS 支持 Linux 虚拟摄像机输出。要使用它，安装 v4l2loopback，然后 OBS 中会出现*启动虚拟摄像机*按钮。如果`v4l2loopback`包中的内核模块尚未加载，OBS 会自动尝试加载它，并要求获得管理权限（使用 pkexec(1)）。

Wayland

由于 OBS 是一款 Qt 应用程序，请参见 Wayland#Qt 以使其在 Wayland 下工作。OBS 从版本 28 开始从 Qt5 更新到了 Qt6，因此更新的版本需要安装 qt6-wayland 包。参见 PipeWire#WebRTC_屏幕共享以启用 Wayland 屏幕捕获。

KDE中全局快捷键无法使用

在OBS中设置的全局快捷键只有在OBS处于焦点状态时才有效。作为解决方法，您可以通过OBS的WebSocket接口来控制它，该接口可以在OBS中启用，方法是转到*工具* > *WebSocket服务器设置*，然后选择*启用WebSocket服务器*。

注意：`obs-studio`仍然缺少WebSocket支持 FS#76710，但Flatpak和一些AUR软件包具有此支持（例如obs-studio-gitAUR）。

然后可以使用`[obsws-python](https://github.com/aatikturk/obsws-python)`或`[obs-websocket-py](https://github.com/Elektordi/obs-websocket-py)`（版本>=1.0）来控制WebSocket。可以使用pip安装其中任何一个。然后在*系统设置* > *快捷键*中，您可以为`obsws-python`添加以下命令，以设置自定义快捷键以切换录制：

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：避免复杂的一行命令。（在Talk:Open Broadcaster Software讨论）

```
python -c "import obsws_python;obsws_python.ReqClient(host='localhost',port=4455,password=yourwebsocketpassword).toggle_record()"
```

或者，如果您使用`obs-websocket-py`，则使用以下命令：

```
python -c "from obswebsocket import obsws,requests;c=obsws('localhost',4455,yourwebsocketpassword);c.connect();c.call(requests.ToggleRecord());c.disconnect()"
```

WebSocket密码和端口可以在*WebSocket服务器设置* > *显示连接信息*中找到。

如果您希望避免复杂的一行命令，请使用recording-toggle.py脚本。

提示和技巧

浏览器来源

obs-browser 插件提供在画布中使用网页的功能，通常用于基于网络的覆盖。该网页可以交互，并像其他源类型一样工作。

默认的 obs-studio包 不提供这个插件，但可以通过其他客户端或插件包添加。

- obs-studio-gitAUR 使用浏览器插件进行编译。
- obs-studio-tytan652AUR 是一款定制的客户端，提供浏览器插件以及浏览器停靠栏，还有其他改进。

通过 Vulkan/OpenGL 捕获

obs-vkcapture 插件增加通过直接连接到 Vulkan 或 OpenGL 的功能，而不是使用通用的 Xorg 或 Wayland 窗口捕获 API。要使用它，安装 obs-vkcaptureAUR，以及 lib32-obs-vkcaptureAUR（如果捕获32位应用程序）。按照 GitHub 仓库中的说明，使用该插件设置*游戏捕获*。

使用 GStreamer 进行编码

obs-gstreamer 项目提供：

- 使用 GStreamer 进行编码的编码器插件。
- 使用 GStreamer pipeline 作为源、视频过滤器或音频过滤器的插件。这是个高级功能，提供给熟悉使用 GStreamer 的用户。

AMD GPU 用户报告说，通过 GStreamer 的 VA-API 胜过 OBS 默认的 VA-API 视频编码能力。要使用 obs-gstreamer 进行编码，安装 obs-gstreamerAUR 并将 OBS 的编码器改为*GStreamer 编码器*。如果 OBS 在编码器方面出现错误，你可能需要安装 gstreamer-vaapi包。

手动插件安装

可以将插件手动安装到 `~/.config/obs-studio/plugins/`. 目录结构如下:

```
~/.config/obs-studio/plugins/plugin_name/bin/64-bit/plugin_name.so
~/.config/obs-studio/plugins/plugin_name/data/locale/en-US.ini
```

录制矩形区域

在 KDE 上，您可以用 obs-rectangle-area-selector 脚本选择并录制一块矩形区域。

疑难解答

QuickSync 提示 "Error creating a MFX session"

该错误会在没有安装正确的英特尔 QuickSync 驱动时发生。

```
[AVHWDeviceContext @ 0x6111a37c1040] Error creating a MFX session: -9.
Device creation failed: -1313558101.
```

要使用 QuickSync，请为 Iron Lake (Gen5) 到 Ice Lake (Gen10)) 的GPU 安装 intel-media-sdk包，对于 Tiger Lake (Gen11) 及更新的 GPU，请安装 vpl-gpu-rt包。

关于更多信息，请参阅 FFmpeg#Intel QuickSync (QSV))。

参见

- OBS Wiki
- 修复 Arch Linux 缺失的 OBS 功能

10.5　VLC 媒体播放器

难度 进阶 | **篇幅** 19.1 KB | **所属** 串流

本节将讲到

- 安装
- 语言
- 皮肤
- Web 界面
- 提示与技巧

引自项目主页：

: VLC 是一款自由、开源的跨平台多媒体播放器及框架，可播放大多数多媒体文件，以及 DVD、音频 CD、VCD 及各类流媒体协议。

安装

安装 vlc包 拆分包，然后从以下插件软件包中选择安装：

- vlc-plugins-base包——提供必要的基础插件。
- vlc-plugins-extra包——提供一些额外插件，包括：基于 FFmpeg 的音视频解码器和 H.264/AVC、H.265/HEVC 编码器；VA-API 视频解码器；MPEG 传输流解复用器；MPEG I/II 视频解码器，支持 DVD 和音频 CD 的播放。
- vlc-plugins-visualization包——提供一些音频可视化插件，包括 Goom 和 ProjectM（英语：ProjectM） 效果。

- vlc-plugins-all包——提供所有可用的官方插件，包括上述所有插件和 FluidSynth MIDI 合成器。

提示：可以通过在包数据库中查询 `vlc-plugin` 获取可用插件列表并独选择单个插件安装。

替代前端

- vlc-cli包——CLI 前端（用 `cvlc` 或 `rvlc` 运行）。
- vlc-gui-ncurses包——ncurses TUI 前端（用 `nvlc` 运行），详见官方文档。
- vlc-gui-skins2包——skins2 GUI 前端（用 `svlc` 运行），参见 #皮肤。

语言

VLC 没有在其“偏好设置”中提供更改语言的选项，但可通过通过修改C0 环境变量改变其界面语言。例如，可以把其桌面项 `vlc.desktop` 中这一部分：

```
Exec=/usr/bin/vlc %U
```

修改为：

```
Exec=env LANGUAGE=zh_CN /usr/bin/vlc %U
```

来将界面语言设置为中文（中国）。

皮肤

VLC 的“皮肤”功能由 vlc-gui-skins2包 提供（需使用 `svlc` 运行），可更换外观和风格。可以从 VLC 皮肤网站下载皮肤。

打开 VLC，点击菜单栏中**“工具 > 偏好设置”**。“简明”显示下，皮肤设置位于“界面”选项卡。

选择“使用自定义皮肤”，然后选择已下载的皮肤资源文件。

重启 VLC 以使更改生效。

Web 界面

使用参数 `--extraintf=http` 运行 VLC 可同时开启桌面界面和 Web 界面。参数 `--http-host` 用于指定绑定地址，默认是 `localhost`。必须使用 `--http-password` 设置密码，否则 VLC 不允许登录。

```
$ vlc --extraintf=http --http-host 0.0.0.0 --http-port 8080 --http-password 密码
```

在图形界面中，可以通过点击菜单栏中**“视图 > 添加界面 > Web”**启动 Web 界面。通过**“工具 > 偏好设置（显示设置为‘全部’）> 界面 > 主界面 > Lua > Lua HTTP > 密码”**设置密码。登录时用户名留空。

VLC 默认使用端口 `8080`：http://localhost:8080

提示与技巧

使用 VLC 观看直播

可以使用 Streamlink 通过 VLC 观看直播。

从本地 DLNA 服务器播放流媒体内容

安装 vlc-plugin-upnp包 后，通过进入**“视图 > 播放列表 > 本地网络 > 通用即插即播（UPnP）”**可以播放 UPnP DLNA 内容。如果 VLC 无法检测到本地网络中的 DLNA 服务器，请确保防火墙未阻止 UPnP 使用的 UDP 1900 端口，只有开放这一端口才能播放本地 DLNA（使用 UPnP）内容。

使用快捷键或命令行控制

安装 openbsd-netcat包。

从 CrunchBang 论坛的存档界面获取脚本。

按照脚本中的说明为 VLC 设置一个 socket。

可以从命令行运行该脚本，或者通过桌面环境将其注册为键盘快捷键。

另外，你也可以使用 MPRIS 与 VLC 交互。

也可以使用 CLI 或 ncurses TUI 启动 VLC（参见 #替代前端）。

保持单实例运行

VLC 的默认设置是每次打开文件就启动一个新的程序实例。如果使用 VLC 来播放音乐收藏之类的内容，这可能会很烦人。可以在文件管理器中多选文件后一起打开，也在菜单中可以关闭此行为：勾选**“工具 > 偏好设置（显示设置为‘简明’）> 界面 > 播放列表与实例管理 > 仅允许一个实例”**，并勾选**“在单实例模式中将项目添加到播放列表队列中”**，这样新的文件将被添加到当前播放列表中而不会中断当前播放的文件。

还有一个选项**“从文件管理器启动时，只允许一个实例”**勾选后，通过文件管理器用 VLC 打开的文件都会在一个实例中播放。**“在单实例模式中将项目添加到播放列表队列中”**选项对此仍然生效。

注意：当前 VLC 版本存在以下 bug：勾选**“从文件管理器启动时，只允许一个实例”**而不勾选**“仅允许一个实例”**时无法（取消）勾选**“在单实例模式中将项目添加到播放列表队列中”**选项。

硬件视频加速

详见硬件视频加速。

VLC 会自动尝试使用可用的 API，但也可以手动覆盖设置。在菜单栏中进入：**工具 > 偏好设置（显示设置为“简明”）> 输入/编解码器**，在“硬件加速解码”中选择合适的选项。

systemd 服务

可以通过 systemd 自启动 VLC 的 #Web 界面。首先，创建一个专用用户：

```
# useradd -c "VLC daemon" -d / -G audio -M -p \! -r -s /usr/bin/nologin -U vlcd
```

接着创建 systemd 服务文件：

```
/etc/systemd/system/vlc.service
[Unit]
Description=VideoOnLAN Service
After=network.target

[Service]
Type=forking
User=vlcd
ExecStart=/usr/bin/vlc --daemon --syslog -I http --http-port 8090 --http-password 密码
Restart=on-abort

[Install]
WantedBy=multi-user.target
```

启动并启用 `vlc.service`。登录时用户名留空，密码为以上服务文件中所设。

Chromecast 支持

从 3.0 版本（Vetinari 分支）开始，VLC 可以向同一网络中的 Chromecast 设备进行流媒体投放。

安装以下软件包：

- vlc-plugin-chromecast包
- libmicrodns包——使 VLC 能发现 Chromecast 设备，并在菜单 “播放 > 渲染器” 中显示。

然后，编辑文件 `/etc/nsswitch.conf`，在 `hosts` 这一行的 `resolve` 和 `dns` 之前加入 `mdns_minimal [NOTFOUND=return]`：

```
hosts: ... mdns_minimal [NOTFOUND=return] resolve [!UNAVAIL=return] dns ...
```

注意：

- 如果解析 `*主机名*.local` 速度较慢，可以尝试将 `mdns_minimal` 替换为 `mdns4_minimal`，仅解析 IPv4 地址。
- 更多信息请参阅 avahi。

单击自动暂停/恢复插件

安装 vlc-pause-click-pluginAUR，该插件实现了许多人习惯的在直接点击 VLC 播放窗口中的视频时暂停或恢复播放的功能。

安装后插件不会自动启用，需要按照 https://github.com/nurupo/vlc-pause-click-plugin#usage 的说明在设置里手动启用。

从 V4L2 音视频采集设备串流

例如，从 USB 摄像头或 USB HDMI 采集设备进行实时串流。

查找 V4L2 设备的视频设备节点：

```
$ v4l2-ctl --list-devices
HDMI Capture: HDMI Capture (usb-0000:07:00.1-2):
/dev/video2
/dev/video3
/dev/media1
```

通常为列出的第一个 `/dev/video*` 节点，此处为 `/dev/video2`。

然后，查找该采集设备的 PulseAudio 音频源名称：

```
$ pactl list short sources | grep --invert-match --regexp=.monitor
 (…部分设备已省略…)
8750 alsa_input.usb-XF_HDMI_Capture_20000130041415-02.iec958-stereo PipeWire s16le 2ch 48000Hz SUSPENDED
```

此处的名称即为 `alsa_input.usb-XF_HDMI_Capture_20000130041415-02.iec958-stereo`。

使用图形用户界面

现在打开 VLC 媒体播放器，在顶部菜单栏中选择“**媒体(M) > 打开捕获设备(C)...**”。

在“设备选择”部分，“视频设备名称”选择上面查询到的 V4L2 设备的视频设备节点（此处为 `/dev/video2`）。 将“音频设备名称”留空。1

在左下角勾选“显示更多选项(M)”复选框。此时应会显示“编辑选项”文本框。 在该文本框末尾添加一个空格后添加以下内容（斜体处替换为上面查询到的实际 PulseAudio 音频源名称）：

```
:input-slave=pulse://alsa_input.usb-XF_HDMI_Capture_20000130041415-02.iec958-stereo
```

然后点击“播放(P)”按钮。

1 这是因为该输入仅接受 ALSA 设备，而 VLC 通常无法访问它们，因为这些设备已被 PipeWire 或 PulseAudio 独占使用。

通过命令行

或者，也可以通过以下命令从命令行运行 VLC 媒体播放器（斜体处分别替换为上面查询到的实际 PulseAudio 音频源名称和 V4L2 设备的视频设备节点）：

```
$ vlc --v4l2-standard= --live-caching=300 --input-slave=pulse://alsa_input.usb-XF_HDMI_Capture_20000130041415-02.iec958-stereo -- v4l2:///dev/video2
```

问题解决

更新后视频异常或其他问题

VLC 的配置在更新时常常会出现问题，即使是小版本更新。在提交错误报告之前，请先删除或重命名位于 `~/.config/vlc` 的配置文件，并确认问题是否仍然存在。

如果使用了 AUR 中的 FFmpeg 变种，请确保它也已更新。Pacman 不会自动升级这些包，而版本不匹配可能会导致 VLC 出现故障。

段错误

启动 VLC 时出现

在启动 VLC 时，可能会遇到段错误。如果已经排除了诸如微码等常见因素，可以尝试重新生成 VLC 的插件缓存并重装 VLC，执行：

```
# /usr/lib/vlc/vlc-cache-gen /usr/lib/vlc/plugins
```

然后重新安装 VLC。

播放视频时出现

如果 VLC 能正常播放音频文件，但在播放视频时出现段错误，那么可能是硬件视频加速配置不当，导致 VLC 无法访问图形设备。这种情况尤其容易发生在一台电脑使用多个显卡时。

下拉菜单中图标缺失

在 XFCE 桌面环境下，下拉菜单中图标（如 PCI 卡图标）可能会缺失。

执行以下命令可以重新启用这些图标：

```
$ gconftool-2 --type boolean --set /desktop/gnome/interface/buttons_have_icons true
$ gconftool-2 --type boolean --set /desktop/gnome/interface/menus_have_icons true
```

打开 VDPAU 后端失败

参见硬件视频加速#打开 VDPAU 后端失败。

由于系统可能并不支持 VDPAU，需要让 VLC 使用 VA-API 作为替代，请参见 #硬件视频加速。

无法通过 SFTP 播放媒体文件

如果 VLC 无法通过 SFTP 播放任何视频或音频文件，请确保已安装 sshfs包。

如果 VLC 拒绝播放任何文件名包含空格的媒体文件而要求身份验证，请修改 `vlc.desktop` 文件中的 Exec 行如下：

```
Exec=/usr/bin/vlc --started-from-file %F
```

参见 。

iOS 或 tvOS 上的 VLC 无法通过 SFTP 连接到 Arch

由于 App Store 的许可限制，VLC 的 iOS 与 tvOS 应用使用的是不完整的 SSH 实现。为使其兼容，需修改 openssh包 配置。创建配置文件 `/etc/ssh/sshd_config.d/vlc.conf`，内容如下：

```
HostKeyAlgorithms +ssh-rsa
KexAlgorithms +diffie-hellman-group16-sha512
```

然后重新加载 `sshd.service`。

无法打开 DVD

若希望通过菜单 “媒体 > 打开光盘” 播放 DVD，需安装 VLC 的可选依赖 vlc-plugin-dvd包 及该插件的可选依赖 libdvdcss包（用于解码使用 CSS 加密的 DVD）。

部分 DVD 还可能需要其它插件，例如 vlc-plugin-dca包。也可以选择安装 vlc-plugins-extra包，该软件包包含上述两个插件及其他额外插件。

如果已安装上述所有软件包仍无法播放 DVD，可尝试删除 `~/.dvdcss` 文件夹，让 VLC 会重新下载 CSS 密钥，有时可以解决播放问题。

无法打开蓝光光盘/蓝光菜单无法加载

参见蓝光#VLC。

媒体无法加载

若某些 RTP、RTSP、DVB-T 流媒体或蓝光视频出现无限缓冲状态，或在日志中无报错却静默加载失败（例如法国 Free 互联网服务提供商的 IPTV 服务），请安装 vlc-plugin-aribb24包 插件。

Wayland 支持

vlc-gitAUR 默认编译时启用了 Wayland 支持。设置环境变量 `QT_QPA_PLATFORM=wayland` 可启用 Wayland，详见 Wayland#Qt。

需要注意的是，尽管 vlc包 的 PKGBUILD 中使用了 `--enable-wayland` 编译参数（当前使用的是 VLC 3.0 版本），但实际上仍会通过 Xwayland 运行，因为 VLC 3 的 Wayland 支持尚未完善。如果未安装 xorg-xwayland包，VLC 的视频输出可能出现裁切或其他异常。

可使用以下命令让 VLC 3 使用 Wayland 启动：

```
$ env -u DISPLAY vlc
```

该命令的原理是：VLC 3 启动时会检查 `DISPLAY` 环境变量，若未设置则尝试使用 Wayland。不推荐在系统范围内移除 `DISPLAY`，因为一些旧应用程序仍依赖该变量。

无法连接到 RTSP 流

如果连接 RTSP 流时出现 `failed to setup rtsp session` 错误，请安装 vlc-plugin-live555包。

通过 Streamlink 观看直播时花屏

请安装可选依赖 vlc-plugin-aribb24包。

使用 AMDGPU 播放 HEVC 视频时系统无响应

在播放 HEVC（H265）编码的视频时，系统可能会完全冻结，无法进行任何操作（包括关机）。

可以通过将**“工具 > 偏好设置（显示设置为“全部”）> 视频 > 输出模块 > OpenGL > Open GL/GLES hardware converter”**从“自动”更改为其他合适选项（例如“VDPAU OpenGL surface converter”或“VA-API OpenGL surface converter for Wayland”）来解决该问题。

详见 https://gitlab.freedesktop.org/drm/amd/-/issues/2113#note_1602599

更改播放进度后声音短暂消失

将**“工具 > 偏好设置（显示设置为‘简明’）> 音频 > 输出模块”**设置为目前使用的音频服务器（对于 PipeWire，需要安装 vlc-plugin-pipewireAUR，否则只能使用兼容层）。

VLC 不显示字幕和屏显信息（OSD）

即使解码字幕的插件已经安装，如果没有安装 vlc-plugin-freetype包，字幕和屏显信息仍然不会显示（而且“消息”和控制台中也不会报错）。注意，该插件不包含在 vlc-plugins-base包 中，但包含在 vlc-plugins-extra包 中。

MKV 文件中的字幕导致“VLC could not identify the audio or video codec”

如果从外部文件加载字幕时显示正常，但 MKV 文件中的内嵌字幕无法被识别，那么很可能是缺少 vlc-plugin-matroska包 软件包。该软件包不包含在 vlc-plugins-base包 中，而包含在 vlc-plugins-extra包 中。

KDE Plasma 未对 VLC 应用深色主题

参见 KDE#主题 提示框中的第一项。

参见

- 维基百科页面
- 应用程序列表/多媒体

- VLC 主页
- playerctl：一个用于控制媒体播放器的命令行工具和库
- VideoLAN's Wiki：通过浏览器控制 VLC（英文）

10.6 Mpv

难度 高级 | **篇幅** 31.9 KB | **所属** 串流

本节将讲到

- 安装
- 配置
- 脚本
- Vapoursynth
- 提示与技巧

mpv 是一款基于 MPlayer 和 *mplayer2* 的多媒体播放器（*mplayer2* 现已不再维护）。它支持多种类型的视频文件格式、音视频编解码器和字幕。这里有一份不太完整的关于 *mpv* 与上述播放器之间差异的详细列表。

安装

安装 mpv包 软件包或开发者版本的 mpv-gitAUR。

前端

参见应用程序列表/多媒体#基于_mpv。

配置

mpv 拥有良好且全面的默认设置，即使电脑上的显卡老旧或性能较弱，*mpv* 也能良好运行。然而，在配备有更新更现代的显卡的电脑上，*mpv* 经过详尽配置，即可获得更好的视频品质（视频品质的高低取决于显卡性能）。要实现这一点，只需要创建一些配置文件（这些配置文件默认不存在）。

注意：系统级配置文件会从 `/etc/mpv/` 中读取，用户级配置文件在未设置 `XDG_CONFIG_HOME` 环境变量的情况下将从 `~/.config/mpv/` 中读取，用户级配置会覆盖系统级配置，而在命令行使用的配置将覆盖上述所有配置。配置时可能需要反复试错，因此，建议使用用户级配置。

mpv 提供了包含默认设置的配置文件示例，将这些文件复制一份然后开始配置：

```
$ cp -r /usr/share/doc/mpv/ ~/.config/
```

`mpv.conf` 是 *mpv* 的主要配置文件，`input.conf` 包含了按键绑定的配置。通读这两个文件，了解它们的工作原理以及可用选项。

通用设置

将以下设置写入 `~/.config/mpv/mpv.conf`。

字幕配置

启用模糊搜索：

```
sub-auto=fuzzy
```

更改字体：

```
sub-font="fontName"
```

将字幕加粗以增加可读性：

```
sub-bold=yes
```

高品质配置

默认设置下，mpv 兼顾质量和性能。此外，它还提供了两个预定义的配置：代表最高性能的 `fast` 和代表卓越渲染质量的 `high-quality`。使用 `--profile=*name*` 选项以应用某个特定的配置，使用 `--show-profile=*name*` 查看该配置的内容。

```
profile=high-quality
```

mpv 的实时性能统计数据可通过 `i` 键调出查看。此功能在确认硬件是否以当前配置运行，或用于对比不同配置下的运行情况时非常有用。

最后两个选项稍微有些复杂。`video-sync=display-resample` 选项能够在音频与视频不同步时，通过保留视频帧，并对音频进行重新采样以保证音视频同步（相较于视频丢帧，音频音调的轻微改变通常难以分辨）。相关的深入探讨位于 mpv 维基上的显示同步。`interpolation` 选项通过改变视频帧的显示方式，让源视频帧更好地贴合显示器的刷新率，以此让画面上的移动显得更加平滑（不要与 SVP 的技术相混淆，后者实际上是将视频转换至 60 帧）。相关的深入探讨位于 mpv 维基上的 Interpolation（Interpolation 又常被称为 *smoothmotion*）。

```
profile=high-quality
video-sync=display-resample
interpolation
```

注意： 使用 NVIDIA Optimus 时，`video-sync=display-resample` 选项可能会导致视频加速。同时，在某些系统上，该选项会彻底扰乱帧速率，插值（interpolation）将完全失去作用。

你能做的远不止于此，但同时，随着配置深入，这些配置细节将更加复杂，对显卡的性能要求也更高。简而言之，可以加载那些实现了奇特的缩放和锐化技术的特殊着色器，这其中包括那些实际上使用了经过图像训练（真实世界和动画化的图像均有）的深度神经网络的着色器。要了解更多相关细节，请查阅 mpv wiki，尤其是 user shader's section 这部分。

更多其他配置项参见 mpv(1)。在命令行运行 *mpv* 检查配置的错误信息也能获取帮助。

自定义配置

自定义配置可在 `mpv.conf` 中创建，这些“配置”实际上只是“选项组”，它们可用于：

- 在不同的配置之间快速切换，无需重写文件。
- 为特殊内容创建特殊配置。
- 嵌套多个配置，将简单的配置组合成更复杂的配置。

创建一个配置很简单。`mpv.conf` 文件的开头区域被称为“顶层”，写在此处的所有选项都会在 *mpv* 启动后生效。要创建一个配置，首先写下一对中括号，然后在中括号内写下配置的名称。一旦创建了配置，接下来写的所有选项（直到下一个配置开始之前）都属于该配置。这是一份 `mpv.conf` 文件的示例：

```
profile=myprofile2          #“顶层”区域，加载 myprofile2
ontop=yes                   #窗口置顶

[myprofile1]                #一份简单的配置，“顶层”区域到此处结束
profile-desc="a profile"    #可选的配置描述
fs=yes                      #以全屏模式启动

[myprofile2]                #另一份简单的配置
profile=high-quality        #mpv 自带的一个配置
log-file=~~/log             #设置日志文件的位置，~~/ 会被转换成 ~/.config/mpv
```

上述示例文件中，“顶层”区域仅有两行，区域下方定义了两个单独的配置。*mpv* 启动时，读取第一行的内容，加载 `myprofile2` 的选项，这相当于加载了 `high-quality` 中的选项以及 `log-file=~~/log` 选项，最终加载 `ontop=yes` 选项，至此启动完成。注意，由于没有在“顶层”区域调用 `myprofile1` 配置，该配置将不会被加载。

又或者，直接从命令行调用 *mpv*：

```
$ mpv --profile=myprofile1 video.mkv
```

此时，除 `myprofile1` 这一配置内的选项，其余所有选项都会被忽略。

自动配置

mpv 会根据文件扩展名或是使用的协议，自动加载特定类型的配置。

根据匹配的文件扩展名，分别为扩展名是 *.mkv* 和 *.gif* 的所有文件加载其对应的配置：

```
[extension.mkv]
keep-open
volume-max=150

[extension.gif]
osc=no
loop-file
```

当播放任意的 http 或 https 流时加载下面的配置（例如 `mpv https://example.com/video.mp4`）：

```
[protocol.https]
speed=2
keep-open

[protocol.http]
profile=protocol.https
```

运行 `mpv --list-protocols` 以查看 mpv 支持的不同协议。

按键绑定

根据 `/usr/share/doc/mpv/input.conf` 和 mpv(1) § COMMAND INTERFACE 中的示例，按键绑定简单明了。

将下面的示例添加至 `~/.config/mpv/input.conf`：

```
shift+s          screenshot each-frame
Shift+UP         seek  600
Shift+DOWN       seek -600
=                cycle video-unscaled
- cycle-values window-scale 2 3 1 .5
WHEEL_UP         add volume 5
WHEEL_DOWN       add volume -5
WHEEL_LEFT       ignore
WHEEL_RIGHT      ignore
Alt+RIGHT        add video-rotate 90
Alt+LEFT         add video-rotate -90
Alt+-            add video-zoom -0.25
Alt+=            add video-zoom 0.25
Alt+j            add video-pan-x -0.05
Alt+l            add video-pan-x 0.05
Alt+i            add video-pan-y 0.05
Alt+k            add video-pan-y -0.05
Alt+BS           set video-zoom 0; set video-pan-x 0; set video-pan-y 0
```

有关在 mpv 中重现 MPC-HC 按键绑定的尝试，请参考这个链接。

额外配置文件

此外，还可以创建一些配置文件和目录，其中包括：

- `~/.config/mpv/script-opts/osc.conf` 负责管理屏幕控制器。更多信息请参阅 mpv(1) § ON SCREEN CONTROLLER。
- `~/.config/mpv/scripts/*script-name*.lua` 表示 Lua 脚本。获取示例请参考这个链接。

要了解其他文件和目录的相关信息，请参阅 mpv(1) § FILES。

脚本

mpv 有大量可扩展播放器功能的脚本（参见用户脚本）。为此，它为 Lua 和 JavaScript 提供了内部绑定。

脚本的安装方式通常是将它们放在 `~/.config/mpv/scripts/` 目录内（可能需要先创建该目录）。之后，它们将在 mpv 启动时被自动加载（这个行为可以被其他 *mpv* 选项修改）。有些脚本自带安装和配置说明，请务必查看。此外，有些脚本已经陈旧、损坏且不再维护。

JavaScript

JavaScript（ES5 via MuJS）自 2014 年起作为 mpv 的脚本语言得到支持。目前仅有少量脚本可用，如果想要自己制作脚本，mpv(1) § JAVASCRIPT 有文档可供参考。

在 mpv 的 `scripts` 目录下放入一个扩展名为 `.js` 的脚本以开始使用，例如：

```
~/.config/mpv/scripts/fullscreen-off-on-pause.js
function onPauseChange (prop, enabled) {
    if (enabled) {
        mp.set_property('fullscreen', 'no')
    }
}

mp.observe_property('pause', 'bool', onPauseChange)
```

想要了解更多细节，例如，关于使用 `require` 加载 CommonJS 模块，请参见 mpv(1) § CommonJS modules and require(id))。

mpv包 软件包和 AUR 中的某些软件包（例如 mpv-fullAUR 和 mpv-full-gitAUR）已支持 JavaScript。

Lua

mpv 的 Lua 脚本开发记录在 mpv(1) § LUA SCRIPTING 和 TOOLS/lua 的示例中，这些脚本会被安装到 `/usr/share/mpv/scripts`。

例如，启用内置脚本，自动裁剪带黑条的视频：

```
$ ln -s /usr/share/mpv/scripts/autocrop.lua ~/.config/mpv/scripts
```

mpv-ytdlAutoFormat

mpv-ytdlautoformat 是一个 Lua 脚本，能够将 Youtube 的 ytdl-format、Twitch 或者其他域的内容，自动转换成 480p 或是你需要的视频品质。

mpv-webm

mpv-webm（或简称为 *webm*）是一个易于使用的 Lua 脚本， 允许在观看视频的同时创建 WebM 文件。它包含了若干特性，并且，它没有任何额外的依赖项（完全依赖于 mpv）。

ytdl-preload

ytdl-preload Lua 脚本能够预加载播放列表中的下一个 ytdl-link。

注意：该脚本仍处于积极开发的阶段。

C

mpv-mpris

mpv-mpris 插件由 C 语言开发，它允许其他应用程序通过 MPRIS 协议与 *mpv* 集成。例如，安装 *mpv-mpris* 后，kdeconnect包 能够在电话来电时自动暂停 *mpv* 中播放的视频。另一个例子是集成蓝牙音频设备上的播放或暂停等按钮。

安装 mpv-mpris包 以使用该插件。

Vapoursynth

Vapoursynth 是 AviSynth 的替代品，可用于 Linux，并允许通过 python 脚本进行视频操作。Vapoursynth 的 python 脚本可被用作 *mpv* 的视频过滤器。

mpv包 软件包现已默认启用 Vapoursynth 支持。

SVP 4 Linux (SmoothVideoProject)

SmoothVideoProject SVP 是一个主要用于转换视频帧率至 60 帧的程序。它功能齐全，适用于 64 位 Linux。

注意：目前 SVP 已改为 30天试用后收费。

它有三个主要功能，每个功能都可以任意禁用或启用（不会强制使用运动插补）。

1. 运动插补（youtube 视频）- 一种将视频帧率转换至 60 帧的算法。这会产生具有争议性的“肥皂剧效应”，有人喜欢，有人讨厌。遗憾的是，这种算法并不完美，还会产生许多奇怪的伪影。它可以根据性能或品质进行调整（通过滑块）。它也提供了一些减少伪影的设置，将实际帧与生成帧插补以减少伪影的明显程度。帧率检测可设置成自动或手动（手动似乎能解决某些用户的性能问题）。
2. 黑条光照（Black bar lighting）（youtube 视频）- 如果图像的宽高比在显示屏上产生黑条，SVP 将用屏幕内容生成的“光照”使黑条发光。可对它进行一些自定义设置，但默认设置非常接近最佳状态。
3. LED 环境光照控制(youtube 视频）- 可控制电视机上的 LED 环境光照。

一旦编译了支持 Vapoursynth 的 *mpv*，让 SVP 与 *mpv* 协同工作就非常简单。仅需安装 svp-binAUR，打开 SVP 程序，让它评估系统性能（可能需要先关闭其他程序以获取准确的结果）,最后在 mpv.conf 中添加下面的 *mpv* 配置 ：

```
mpv.conf
[svp]
input-ipc-server=/tmp/mpvsocket      # 接收来自 SVP 的输入
hr-seek-framedrop=no                 # 修复音频不同步
watch-later-options-remove=vf        # 不保留 SVP 的视频过滤器
```

```
# 某些情况下可修复卡顿，其他情况下可能引起卡顿。如遇卡顿可以尝试。
#opengl-early-flush=yes
```

然后，为了使用 SVP，在以该配置运行 *mpv* 并打开文件之前，必须在后台运行 SVP 程序。要么像这样使用：

```
$ mpv --profile=svp video.mkv
```

要么在 *mpv* 配置的“顶层”部分设置 `profile=svp`。

要使用硬件解码，必须使用回传解码器（copy-back decoder），因为普通解码器不兼容 Vapoursynth（请选择一个以 `-copy` 结尾的 `hwdec` 选项）。例如：

```
hwdec=auto-copy
hwdec-codecs=all
```

mpv 的开发人员无论如何都不鼓励使用硬件解码，而且硬件解码也不可能在性能上产生显著的差异。

提示与技巧

图像

硬件视频加速

参见硬件视频加速。

硬件加速视频解码可通过 `--hwdec=*API*` 选项启用。要获取所有受支持的接口和其他选项，请参见 mpv(1) § hwdec。

将该选项添加到配置文件以使其始终生效（例如在桌面环境播放视频）：

```
~/.config/mpv/mpv.conf
hwdec=auto
```

要允许 CPU 处理视频滤镜，请从 `*-copy` 中选择某个接口。

视频播放时，可使用键盘快捷键 `Ctrl+h` 切换硬件解码。

要排除硬件加速故障，可能需要调整日志记录级别（参见 mpv(1) § msg-level）。例如，`--msg-level=vd=v,vo=v,vo/gpu/vaapi-egl=trace` 将启用以下功能：

- 来自视频解码模块 (`vd`) 和视频输出模块 (`vo`) 的详细信息。
- 负责视频解码模块的更详细的跟踪信息。根据经验，在没有调整任何日志记录级别的情况下运行 mpv 之后，相关模块应该是 `vo/gpu/vaapi-egl`。

在宽高比之间快速切换

使用 `Shift+a` 在宽高比之间切换。

忽略宽高比

使用 `--keepaspect=*no*` 忽略宽高比。要使该选项始终生效，请在配置文件中添加 `keepaspect=*no*` 这一行。

绘制至根窗口

使用 `--wid=0` 参数运行 *mpv*。*mpv* 将绘制到 ID 为 0 的窗口中。

始终显示应用程序窗口

要让从命令行启动的 mpv（即使播放的是音频文件）显示应用程序窗口，请使用 `--force-window` 选项。要使该选项始终生效，请在配置文件中添加 `force-window=yes` 这一行。

禁用视频输出

要在命令行启动时禁用视频输出，请使用 `--vid=no` 选项，或是其别名 `--no-video`。

终端输出视频

- `--vo=tct` “彩色 Unicode 艺术的视频输出驱动程序，可在文本控制台中运行。”
- `--vo=caca` “彩色 ASCII 艺术的视频输出驱动程序，可在文本控制台中运行。”由于存在漏洞（参见 FS#70962），Arch Linux 禁用了 libcaca包 支持，并且由于其上游停止活动，此包尚未被重新加入：安装 mpv-fullAUR。

音频

音量过低

为配置文件中的 `volume-max=*value*` 设置一个合适的值，例如 `volume-max=150`，这将使音量提高至 150%，也就是两倍多的音量。将音量调得过高会产生削波假象（clipping artefacts）。此外（或作为替代方法），还可以使用 `af=acompressor` 动态压缩范围。

指定音频输出

运行以下命令获取可用音频输出设备列表：

```
$ mpv --audio-device=help
```

然后将其中的某个设备添加至 `~/.config/mpv/mpv.conf`。例如：

```
audio-device=alsa/hdmi:CARD=NVidia,DEV=1
```

高清音频直通

要使 TrueHD 和 DTS-MA 等高清音频编解码器直通影音接收器，请将以下内容添加至 `~/.config/mpv/mpv.conf`。

```
audio-spdif=ac3,eac3,dts-hd,truehd
```

音量标准化

不同信号源的响度可能不同或不一致，因此 *mpv* 用户可能需要配置自动音量标准化。例如：

```
~/.config/mpv/input.conf
n cycle_values af loudnorm=I=-30 loudnorm=I=-15 anull
```

这会绑定按键 `n`，使音频过滤器设置（`af`）在指定的值之间循环：

- `loudnorm=I=-30`：loudnorm 设置为 `I=-30`，音量小，可能适合背景音乐
- `loudnorm=I=-15`：音量大一些，可能适合当前播放的视频
- `anull`：音量过滤器重置为空（null），即禁用音频过滤器

注意：绑定一个按键并不会改变默认的音频过滤器。要改变默认值，请将例如 `af=loudnorm=I=-30` 的配置项添加到主配置文件中。

mpv 中的音频过滤功能由后端的 FFmpeg 提供。参见 Wikipedia:EBU R 128 和 ffmpeg loudnorm filter 以获取细节信息。

另请参见上游问题 和 ，其中提到了不同的选项。

在 mpv 用作音乐播放器时使用 Lua 脚本增强其功能

这篇博文介绍了 music.lua 脚本，该脚本展示了如何在 mpv 用作音乐播放器时，使用 Lua 脚本增强其功能。

退出时保存播进度

默认情况下，按下快捷键 `Shift+q` 可保存播放进度并退出。可在按键绑定的配置文件中设置 `quit_watch_later` 以修改此快捷键。

要在退出时自动保存当前播放位置，请以 `--save-position-on-quit` 启动 *mpv*，或将 `save-position-on-quit` 添加至配置文件。

保存在播放列表中的位置以及在播放下一个文件时暂停播放

播放列表可以是一个文件列表，参见 mpv(1) § playlist。要播放一个列表并记住播放位置：

```
$ mpv --save-position-on-quit --pause --reset-on-next-file=pause --
playlist=/path/to/playlist
```

以 `--pause` 选项启动时，*mpv* 将处于暂停状态，当切换至下一个文件时，`--reset-on-next-file=pause` 将重置暂停模式。

播放 DVD

mpv 不支持 DVD 菜单。要以视频 DVD 的最长标题启动主流，使用这个命令：

```
$ mpv dvd://
```

可选的标题指定符是一个数字（从 0 开始），可在 DVD 上单独的视频流之间进行选择：

```
$ mpv dvd://[title]
```

已被复制到本地文件系统（例如使用了 dvdbackup（英语：dvdbackup） 工具）的 DVD 可通过指定本地拷贝将其包含进来：`--dvd-device=*PATH*`。

要从本地文件系统播放 DVD，请参考下面的桌面项示例：

```
[Desktop Entry]
Type=Application
Name=mpv Media Player DVD
GenericName=Multimedia player
Comment=Play movies and songs
Icon=mpv
Exec=mpv dvd:// --player-operation-mode=pseudo-gui --force-window --idle --dvd-
device=%f
Terminal=false
Categories=AudioVideo;Audio;Video;Player;TV;
# (MimeType 和 X-KDE-Protocols 已省略，参见原 mpv.desktop 文件)
```

将 Exec 这一行替换为：

```
Exec=mpv dvd://0 dvd://1 dvd://2 dvd://3 dvd://4 dvd://5 dvd://6 dvd://7 dvd://8
dvd://9  --player-operation-mode=pseudo-gui --force-window --idle --dvd-device=%f
```

mpv 播放器会将 DVD 标题以从 0 到 9 的顺序排列在播放列表内，这样用户就可以连续地播放或通过 mpv 用户界面在 DVD 标题之间前后跳转。

安装 libdvdcss包 以解决以下问题：

```
[dvdnav] Error getting next block from DVD 1 (Error reading from DVD.)
```

恢复旧的屏幕控件

自 0.21.0 版本起，*mpv* 已经将屏幕控件（on-screen controls）替换为底部栏（bottombar）。如果想要恢复屏幕控件，可以按照此处的说明编辑 *mpv* 的配置。

可重现的屏幕截图

屏幕截图的模板选项可以包含截屏帧的精确时间码（HH:MM:SS.mmm）。有意义的文件名更易于了解屏幕截图的来源。可以像这样设置它：

```
~/.config/mpv/mpv.conf
screenshot-template="%F - [%P] (%#01n)"
```

这将扩展至 `*filename* - [HH:MM:SS.mmm] (*number*).jpg`。结果示例：

```
Gunsmith Cats/
├── Gunsmith Cats Ep. 01 - [00:00:50.217] (1).jpg
```

```
├── Gunsmith Cats Ep. 01 - [00:22:55.874] (1).jpg
├── Gunsmith Cats Ep. 01 - [00:22:55.874] (2).jpg
└── Gunsmith Cats Ep. 02 - [00:15:05.778] (1).jpg
```

这样的一个好处是，文件排序得当，因为时间码在章节编号内部以字母顺序进行排列。

参见 mpv(1) § screenshot-template 以获取更多信息。

创建单张屏幕截图

这是使用开始时间（`HH:MM:SS`）创建单张屏幕截图的示例：

```
$ mpv --no-audio --start=00:01:30 --frames=1 /path/to/video/file --
o=/path/to/screenshot.png
```

屏幕截图将被保存至 /path/to/screenshot.png。

Streaming

Twitch.tv streaming over mpv

安装好 yt-dlp 或 youtube-dlAUR 后，*mpv* 能直接打开 Twitch 的直播流。

或参见 Streamlink#Twitch。

另一个选择是基于 Livestreamer 的 Lua 脚本。

youtube-dl 和格式选择

`--ytdl-format` 的默认值是 `bestvideo+bestaudio/best`。这可能意味着，对于那些有 4K 分辨率的 youtube 视频，即使连接着的显示器分辨率低得多，软件内解码 VP9 编码的 4K 视频对于设备而言依然十分困难。

然而，设置正确的 youtube-dl 格式选择器能轻易地解决这个问题。下面的配置示例中，只有垂直分辨率小于等于 1080 像素的视频会被选择。

```
ytdl-format="bestvideo[height<=?1080]+bestaudio/best"
```

因无法硬件解码而希望完全禁用特定的编解码器，可以将其添加至格式选择器内。例如，可额外选择忽略 VP9，如下所示：

```
ytdl-format="bestvideo[height<=?1080][vcodec!~='vp0?9']+bestaudio/best"
```

若更倾向于开源编解码器（VP9 和 Opus），请使用：

```
ytdl-
format="((bestvideo[vcodec^=vp9]/bestvideo)+(bestaudio[acodec=opus]/bestaudio[acodec=vo
rbis]/bestaudio[acodec=aac]/bestaudio))/best"
```

youtube-dl 音频搜索

要从终端模拟器中使用 `yta *search terms*` 寻找并串流音频，将下面的函数添加至 `.bashrc`：

```
function yta() {
    mpv --ytdl-format=bestaudio ytdl://ytsearch:"$*"
}
```

系统集成

从 KDE 的剪切板中打开视频链接

如果已安装 youtube-dlAUR 或 yt-dlp包，并且正在使用 KDE Plasma，可以在 KDE 的剪切板中创建一个自定义操作来方便快捷地播放来自视频分享站点的链接。

1. 打开剪切板配置菜单（典型的做法是右键它的系统托盘图标）然后前往 *Actions* 标签页。
2. 点击 *Add Action* 然后输入用于探测想要从中播放视频的网站的正则表达式（例如，`^http.+(youtu|twitch)` 用于探测 YouTube 和 Twitch 的网站）。
3. 点击 *Add Command*，在 *Command* 下输入 `mpv %s` 并在 *Description* 下输入 `mpv`。

此时，在剪切板内按下快捷键 `Ctrl+Alt+r` 并在上下文菜单中选择 *mpv* 即可播放来自剪切板的视频链接。有可能需要前往 *Advanced Settings* 并将火狐浏览器从 *Disable Actions for Windows of Type WM_CLASS* 这一小节移除。

故障排除

常规调试

若使用 *mpv* 播放时遇到问题（或完全无法运行），首先需要做这三件事：

1. 从命令行启动 *mpv*（-v 标识用于增加输出信息详细程度）。幸运的话，你会看到一条错误信息，告诉你什么地方出错了。
2. `$ mpv -v video.mkv`
3. 让 *mpv* 输出日志文件。日志内容可能难以筛选，但如果有什么东西坏了，你应该能在日志里看到。
4. `$ mpv -v --log-file=./log video.mkv`
5. 无配置文件运行 *mpv*。如果运行正常，那么问题应该在配置文件的某处（或许是硬件无法跟上你的设置）。
6. `$ mpv --no-config video.mkv`

如果 *mpv* 能够运行但运行状态不佳，那么第四件值得一试的事就是查看实时统计数据（使用快捷键 `i`），确认其具体运行情况。

修复播放卡顿和撕裂问题

在支持 OpenGL 的硬件上，*mpv* 默认使用 OpenGL 视频输出设备设置。在尝试使用英特尔 HD4XXX 系列显卡或类似显卡在 4K 显示器上播放视频时，使用任何 OpenGL 输出设置都会出现播放卡顿（有时甚至完全停止）和严重的撕裂问题。如果遇到以上问题，使用 XV 视频输出（仅适用于 Xorg）或许有用：

```
~/.config/mpv/mpv.conf
vo=xv
```

注意：这是 X 上兼容性最佳的视频输出，但视频品质可能很低，在屏幕显示（OSD）和字幕显示方面也存在问题。

这有可能进一步提升播放性能（尤其在低性能硬件上），但大多数情况下，这会大幅降低视频品质。

要提升视频播放性能，可以考虑以下配置项：

```
~/.config/mpv/mpv.conf
vd-lavc-fast
vd-lavc-skiploopfilter=skipvalue
vd-lavc-skipframe=skipvalue
vd-lavc-framedrop=skipvalue
vd-lavc-threads=threads
```

窗口合成器相关问题

如 mpv(1) § Window 中所述，mpv 在全屏模式下默认禁用任何活动窗口合成器。这用于防止播放时可能出现的性能问题。

对于 KWin 或 Mutter 等窗口合成器而言，即使在窗口化模式下，禁用窗口合成也是有好处的。这可以通过设置 `x11-bypass-compositor=yes` 选项实现。

禁用合成有两个缺点：

- 重新启用合成可能会出现一段时间的卡顿，尤其在使用 KWin 合成时。
- 由合成提供的任何效果都将暂时不可用（直至 mpv 重新启用合成），这在 Plasma 中会阻止默认的应用程序切换器工作。

为避免这些问题，可使用 `x11-bypass-compositor=no` 令合成器保持启用状态。

没有音量条，无法调整音量

在音量图标上滚动鼠标滚轮。

GNOME Blank screen (Wayland)

在 Wayland 下 *mpv* 可能不会暂停 GNOME 的省电设置，导致屏幕保护程序在视频播放时关闭显示器。解决方法是在 `mpv.desktop` 中 `Exec=` 这一行开头添加 `gnome-session-inhibit`。

要仅在播放时阻止屏保，请使用 mpv_inhibit_gnomeAUR。作为替代方案，可使用这个基于 `gnome-session-inhibit` 的 mpv Lua 脚本。

提示：flatpak 应用程序 `io.mpv.Mpv` 已包含 mpv_inhibit_gnome 插件。

光标主题在 GNOME Wayland 下不受约束

参见 GNOME/Troubleshooting#Cursor size or theme issues on Wayland（英语：GNOME/Troubleshooting#Cursor size or theme issues on Wayland）。

关于 AMD GPU 上缺少 CUDA 库的错误信息

v0.34.1 以及更早版本的 mpv 在 AMD GPU 上使用 VAAPI 硬件加速时，可能会遇到持续的错误信息：`Cannot load libcuda.so.1`。设置 `gpu-hwdec-interop=vaapi` 可阻止此错误信息。

相关错误报告：Github issue #9691，Github issue #8765。

上游已在 pull request #9842 中修复该问题。

无法在 PipeWire 被屏蔽时播放音频

如果 *mpv* 在屏蔽 PipeWire 的系统上崩溃或无法播放音频（报告无输出或管道破损），请设置 `--ao` 选项以匹配环境。在 `mpv.conf` 中设置该选项以使其始终生效。

mpv 无法从文件播放 DVD

如果 mpv 无法从纯 VIDEO_TS/VOB 结构的文件播放 DVD，则可能是恢复播放进度的功能存在问题。请尝试清理 `.config/mpv/watch_later` 文件夹，或以 `no-resume-playback`（也可设置 `save-position-on-quit=no` 选项）启动 *mpv*。

修复暂停后恢复播放时出现的卡顿问题

如果在使用 PulseAudio 时出现视频卡顿，请尝试 mpv(1) § --pulse-latency-hacks 中讨论的 `pulse-latency-hacks` 选项：

```
~/.config/mpv/mpv.conf
pulse-latency-hacks=yes
```

BitTorrent

本章解决在 Arch Linux 上挑一个 BitTorrent 客户端并长期运行的问题：装哪个、怎么配、怎么挂到终端或 Web 界面。Transmission 与 Aria2 篇幅最厚，覆盖守护程序、命令行与 Web 界面，必读；rTorrent 靠 Screen 挂机，要长期保种再细看；qBittorrent 兼顾图形界面、搜索引擎与主题，适合桌面；Ktorrent 最短，当速查表翻即可跳过。

最容易踩的坑是把「装上」当成「能用」：图形界面配置与守护程序配置各算一套，Aria2 要配前端才顺手，rTorrent 得借 Screen 才能退出后继续跑，接成 makepkg 的下载工具也要另行设置。

11.1　Ktorrent

难度 入门 | **篇幅** 1.5 KB | **所属** BitTorrent

本节将讲到

- 安装
- 问题解决
- 在命令行中用脚本管理

KTorrent 是 KDE 的 BitTorrent 客户端。

安装

安装 ktorrent包 软件包。

问题解决

无法看到底部面板的某些工具

要显示 files、seeders、leechers 和 current trackers 的任何信息，安装可选依赖 geoip包。建议在设置中启用 DHT，以避免速度过慢和 seeders 数量过少。

在命令行中用脚本管理

虽然 KTorrent 仅是一个 GUI 应用程序，但是幸好它有一个 DBUS 接口，因此您可以使用脚本在命令行（即从 SSH）中对其进行管理。详情请参见 linuxquestions 论坛回答。

11.2　QBittorrent

难度 入门 | **篇幅** 5.5 KB | **所属** BitTorrent

本节将讲到

- 安装
- 配置

- 搜索引擎
- Web 界面
- 主题

qBittorrent是一个用C++、Qt和Python（可选搜索引擎）编写的开源跨平台BitTorrent客户端，使用libtorrent-rasterbar包库。

它速度快，稳定性高，体积轻，支持unicode并提供集成搜索引擎。它支持UPnP端口转发 / NAT-PMP，加密（与Vuze兼容），FAST扩展（主线）和PeX支持（与uTorrent兼容）。

安装

在官方仓库中有两个软件包可以用于安装qBittorrent：

- **qBittorrent** — 具有Qt图形界面和#Web 界面。

: https://github.com/qbittorrent/qBittorrent || qbittorrent包

- **qBittorrent-nox** — 无图形界面的版本(nox -> no X server)。

: https://github.com/qbittorrent/qBittorrent || qbittorrent-nox包

配置

在第一次运行程序时，配置文件会保存在`~/.config/qBittorrent/qBittorrent.conf`中。

自启动

如果你安装了qbittorrent-nox包，你将会得到一个systemd单元模板：`qbittorrent-nox@.service`。

如果你启用、启动`qbittorrent-nox@username.service`，QBittorrent 将以用户`*username*`运行。参见)。

默认的下载文件夹会是该用户的`Downloads`目录。

提示：

- 如果你把它作为一个可以访问的服务运行，可以添加一个叫做qbittorrent的用户并让它在那个用户下运行，使得该服务可以在程序退出后重新启动。
- 要修改设置（例如端口），你可以为它的systemd单元添加一个环境变量（例如端口的变量就是QBT_WEBUI_PORT）。执行`qbittorrent-nox --help`可以查看更多其他的环境变量（此信息在手册中并未给出）。

搜索引擎

你可以通过菜单*视图*> *搜索引擎*启用可选搜索引擎，这将启用*搜索*标签页。

搜索插件

默认的搜索插件可通过下面的操作添加/启用：转到*搜索*标签页，中打开追踪器窗口后通过*搜索插件...*（右下角按钮），并点击*查找更新*。

更多非官方的搜索插件可以在这里找到。

Web 界面

默认端口

qBittorrent默认会在所有接口上的8080端口监听传入连接。因此，你可以通过`[http://HOST_IP:8080](http://HOST_IP:8080)`访问。

注意：HTTPS默认不启用，所以，`[https://HOST_IP:8080](https://HOST_IP:8080)`将无法访问。

默认用户名 & 密码

默认用户名为`admin`，在5.0.0版本之前的默认密码为`adminadmin`。版本5.0.0及以上会生成一个只在控制台可见的随机密码，直到手动保存以防止未经授权的访问。有关此更改的更多信息，请查看这里。

允许无用户名&密码访问

在家庭环境下，通常希望能够无需输入用户名和密码就能访问Web UI。这可以在使用默认用户名和密码登录后在Web UI本身中配置。

或者，为了避免首次登录，可以将以下部分添加到`~/.config/qBittorrent/qBittorrent.conf`中：

```
[Preferences]
WebUI\AuthSubnetWhitelist=192.168.1.0/24
WebUI\AuthSubnetWhitelistEnabled=true
WebUI\UseUPnP=false
```

上述配置项将：

- 允许从192.168.1.x登录的客户端无需输入用户名和密码即可以访问Web UI。
- 为Web UI禁用UPnP，这样Web UI将无法从网络外部访问。

然后，重载 `qbittorrent-nox@*用户名*.service`。

反向代理配置

nginx

请参考qbittorrent wiki

主题

非官方主题

- 如何使用自定义UI主题
- 已知的qBittorrent主题列表

故障排查

另见

- qBittorrent官方网站
- qBittorrent在GitHub的wiki
- Wikipedia:qBittorrent

11.3 Aria2

难度 进阶 | **篇幅** 13.4 KB | **所属** BitTorrent

本节将讲到

- 安装
- 配置
- 前端
- 提示与技巧
- 另请参见

来自 aria2 主页:

: aria2 是一个轻量级多协议多源命令行下载工具，支持 HTTP/HTTPS， FTP，BitTorrent 和 Metalink。用户可以通过内置的 JSON-RPC 和 XML-RPC 接口操作 aria2。

安装

安装 aria2包 软件包。

要以守护进程运行 aria2，你可以写一个 systemd 用户单元。

配置

aria2.conf

aria2 默认从 `$XDG_CONFIG_HOME/aria2/aria2.conf` 中寻找全局配置，该行为可以通过 `--conf-path` 选项改变：

- 下面的示例用配置文件 `/file/aria2.rapidshare` 中的选项下载 `aria2.example.rar`：

```
$ aria2c --conf-path=/file/aria2.rapidshare
http://rapidshare.com/files/12345678/aria2.example.rar
```

如果存在 `$XDG_CONFIG_HOME/aria2/aria2.conf`，但希望使用 `/file/aria2.rapidshare` 中的选项，需要在命令中加入 `--no-conf` 选项：

- 若不使用默认的配置文件，只使用 `/file/aria2.rapidshare` 中的选项下载 `aria2.example.rar`：

```
$ aria2c --no-conf --conf-path=/file/aria2.rapidshare
http://rapidshare.com/files/12345678/aria2.example.rar
```

如果不存在 `$XDG_CONFIG_HOME/aria2/aria2.conf`，并且你想要简化配置管理：

```
$ touch $XDG_CONFIG_HOME/aria2/aria2.conf
```

示例 aria2.conf

```
continue
dir=${HOME}/Desktop
file-allocation=none
input-file=${HOME}/.aria2/input.conf
log-level=warn
max-connection-per-server=4
min-split-size=5M
on-download-complete=exit
```

这与执行下面的命令等价：

```
$ aria2c --dir=${HOME}/Desktop --file-allocation=none --input-
file=${HOME}/.aria2/input.conf --on-download-complete=exit --log-level=warn FILE
```

注意：上面示例 aria.conf 可能错误地使用了 $HOME 变量，一些用户报告说，使用花括号语法会在 aria2 的工作目录下创建名叫 ${HOME} 的文件夹。因为 bash 会把 $HOME 当成环境变量，这样的文件夹很难访问。目前建议在 aria2.conf 中使用绝对路径。

选项细节

- `continue`: 如果存在下载任务对应的控制文件，自动继续未完成的下载任务。
- `dir=${HOME}/Desktop`: 在 `~/Desktop` 中存储下载文件。
- `file-allocation=none`: 不要在下载开始前预分配磁盘空间（默认值：prealloc1
- `input-file=${HOME}/.aria2/input.conf`: 从 `~/.aria2/input.conf` 中的 URI 下载文件，URI 以 TAB 分割。

- `log-level=warn`: 设置日志级别，只输出警告和错误。（默认值：debug）
- `max-connection-per-server=4`: 设置每个服务器的最大链接数为 4。（默认值：1）
- `min-split-size=5M`: 只在文件大小超过 2*5MB = 10MB 时分割文件。（默认值： 20M）
- `on-download-complete=exit`: 当下载任务完成时，运行 `exit` 命令并退出 shell。

示例输入文件 #1

从两个分别的地址下载 `aria2-1.10.0.tar.bz2` 到 `~/Desktop`，并合并为 `aria2-1.10.0.tar.bz2`

```
http://aria2.net/files/stable/aria2-1.10.0/aria2-1.10.0.tar.bz2
http://sourceforge.net/projects/aria2/files/stable/aria2-1.10.0/aria2-1.10.0.tar.bz2
```

示例输入文件 #2

下载 `aria2-1.9.5.tar.bz2`，命名为 `aria2.old.tar.bz2`， 保存到 `/file/old`

```
http://aria2.net/files/stable/aria2-1.9.5/aria2-1.9.5.tar.bz2
  dir=/file/old
  out=aria2.old.tar.bz2
```

下载 `aria2-1.10.0.tar.bz2`，命名为 `aria2.new.tar.bz2`，保存到 `~/Desktop`

```
http://aria2.net/files/stable/aria2-1.10.0/aria2-1.10.0.tar.bz2
  out=aria2.new.tar.bz2
```

备注

1 `--file-allocation=falloc`: 推荐在新的文件系统，比如 ext4 （具有 extents 支持的）、btrfs 或者 XFS 上，使用这个选项，因为它们几乎可以瞬间分配 GB 级别的大文件。不要在传统的文件系统，比如 ext3 上，使用 falloc，因为预分配会消耗与正常分配相同的时间，还会阻塞 aria2 下载。

提示：要查看完整的配置选项，参见 `aria2c --help=#all` 和 aria2c(1) § OPTIONS。

示例 aria2.rapidshare

```
http-user=USER_NAME
http-passwd=PASSWORD
allow-overwrite=true
dir=/file/Downloads
file-allocation=falloc
enable-http-pipelining=true
input-file=/file/input.rapidshare
log-level=error
max-connection-per-server=2
summary-interval=120
```

选项细节

- `http-user=USER_NAME`: 设置 HTTP 用户名)为 USER_NAME。这会影响所有的 URI。
- `http-passwd=PASSWORD`: 设置 HTTP 密码为 PASSWORD。这会影响所有的 URI。
- `allow-overwrite=true`: 如果控制文件不存在，重新下载。（默认值：false）
- `dir=/file/Downloads`: 存储下载文件到 `/file/Downloads`。
- `file-allocation=falloc`: 在下载开始前，调用 posix_fallocate(3) 分配磁盘空间。（默认值：prealloc）
- `enable-http-pipelining=true`: 启用 HTTP/1.1 流水线来降低网络延迟的影响，并减少数据用量。（默认值：false）
- `input-file=/file/input.rapidshare`:
  从 `/file/input.rapidshare` 中的 URI 下载文件，URI 以 TAB 分割。
- `log-level=error`: 设置日志级别，只输出错误。（默认值：debug）
- `max-connection-per-server=2`: 设置每台服务器每个文件的最大链接数为 2。（默认值：1）
- `summary-interval=120`: 每 120 秒输出下载进度汇总。（默认值：60）2

备注

- 因为 `aria2.rapidshare` 包含用户名和密码，建议将该文件权限设置为 600 或类似的权限。

```
$ cd /file
$ chmod 600 /file/aria2.rapidshare
$ ls -l
total 128M
-rw------- 1 arch users  167 Aug 20 00:00 aria2.rapidshare
```

2 `summary-interval=0`: 不显示下载进度汇总，可以提高总体性能。日志仍然会按照 `log-level` 选项的配置输出。

提示：示例的配置文件可以用于Hotfile，DepositFiles 等网站。

注意：命令行选项永远优先于配置文件中的选项。

示例 aria2.bittorrent

```
bt-seed-unverified
max-overall-upload-limit=1M
max-upload-limit=128K
seed-ratio=5.0
seed-time=240
```

选项细节

- `bt-seed-unverified=false`: 在做种前不检查文件哈希。（默认值：true）
- `max-overall-upload-limit=1M`: 设置全局最大上传速度为 1MB/sec。（默认值：0）
- `max-upload-limit=128K`: 设置每个种子的最大上传速度为 128K/sec。（默认值：0）
- `seed-ratio=5.0`: 下载完成后持续做种直到分享率达到 5.0。（默认值：1.0）
- `seed-time=240`: 下载完成后做种 240 分钟。

注意：如果同时设置了 `seed-ratio` 和 `seed-time`，当至少一个条件满足时就会停止做种。

示例 aria2.daemon

这个配置可以用于以服务启动的 aria2。可以与下面的前端一起使用。注意 rpc-user 与 rpc-pass 选项已经被弃用，但许多前端还不支持新的认证系统。记得修改用户名、密码和下载目录。

```
continue
daemon=true
dir=/home/aria2/Downloads
file-allocation=falloc
log-level=warn
max-connection-per-server=4
max-concurrent-downloads=3
max-overall-download-limit=0
min-split-size=5M
enable-http-pipelining=true

enable-rpc=true
rpc-listen-all=true
rpc-user=rpcuser
rpc-passwd=rpcpass
```

前端

注意：前端的设置不会影响 `aria2` 本身的配置，不确定不同的前端是否会在提供了自定义配置时，复用 `aria2` 的配置。用户应当确认选择的工具能效实现，并能持久保存配置。（比如 uGet 提供了自己的 `aria2` 命令行，并且会在重启间保存配置）。

Web UIs

注意：这些前端需要 `aria2c` 以 `--enable-rpc` 选项启动。它们应该在你本地的设备，而不是远程服务器上运行。

- **WebUI** — Aria2 的 HTML 前端。

: https://github.com/ziahamza/webui-aria2 || webui-aria2-gitAUR

- **aria2rpc** — 用于连接远程 `aria2c` 实例的命令行工具。如果安装了 `aria2c`，该程序可以在 `/usr/share/doc/aria2/xmlrpc/aria2rpc` 下找到。

: https://github.com/tatsuhiro-t/aria2/blob/master/doc/xmlrpc/aria2rpc || aria2包

- **AriaNg** — 纯 HTML & JavaScript 实现的aria2前端，也提供桌面客户端。

: https://github.com/mayswind/AriaNg || ariang-gitAUR, ariang-native-electron-gitAUR

其他 UIs

注意：这些前端不需要 `aria2c` 以 `--enable-rpc` 选项启动。

- **aria2fe** — 一个命令行下载工具 aria2 的 GUI。

: https://sourceforge.net/projects/aria2fe/ || aria2feAUR

- **Persepolis** — Aria2 下载管理器的图形前端，具有许多功能，支持 HTTP 和 FTP。

: https://github.com/persepolisdm/ || persepolis包

- **uGet** — 功能丰富的 GTK/CLI 下载管理器，可以通过内置的插件，使用 aria2 作为后端。

: https://ugetdm.com || uget包

- **Motrix** — 基于 aria2 的全功能下载管理器，支持 HTTP、FTP、BitTorrent 和 Magnet。

: https://github.com/agalwood/Motrix || motrixAUR

提示与技巧

下载但不安装软件包

只需要用下面的命令：

```
# pacman -Sp packages | aria2c -i -
```

`pacman -Sp` 在标准输出里打印软件包的 URL，而不是下载它们，然后用 `|` 管道传给下一个命令。最后，`aria2c -i -` 里的 `-i` 意味着下载文件的 URL 应该从文件里指定。但如果传入了 `-`，就从 stdin 里读取 URL。

pacman XferCommand

参见 Pacman/提示和技巧#aria2.

改变 User Agent

一些站点会根据你的 User Agent 过滤请求，由于 aria2 并不出名，使用主流下载器或浏览器的 User Agent 可能会更好。只需要像这样使用 `-U` 选项：

```
$ aria2c -UWget http://some-url-to-download/file.xyz
```

你可以任意指定 User Agent，比如 **\-UMozilla/5.0** 等。

在 makepkg 中使用 aria2

你可以使用 aria2 代替 curl包 来下载源文件，只需要像下面一样更改 `DLAGENTS` 变量：

```
/etc/makepkg.conf
[...]
DLAGENTS=('ftp::/usr/bin/aria2c -UWget -s4 -x4 %u -o %o --follow-metalink=mem'
          'http::/usr/bin/aria2c -UWget -s4 -x4 %u -o %o --follow-metalink=mem'
          'https::/usr/bin/aria2c -UWget -s4 -x4 %u -o %o --follow-metalink=mem'
          'rsync::/usr/bin/rsync --no-motd -z %u %o'
          'scp::/usr/bin/scp -C %u %o')
[...]
```

注意：用 `-UWget` 选项改变 user agent 为 Wget，可以防止下载文件时，站点根据不同的 user agent 提供不同的响应。由于 aria2 不是著名的下载器，可能会被站点识别为浏览器而不是下载器，所以使用 Wget 的 user agent 可以在大多数情况下解决问题。使用`--follow-metalink=mem`选项来避免将metalink下载为单独的文件导致校验检查失败。

作为守护进程使用 aria2

要使用 aria2 作为守护进程，你可以写一个systemd用户单元。比如：

```
~/.config/systemd/user/aria2cd.service
[Unit]
Description=aria2 Daemon

[Service]
ExecStart=/usr/bin/aria2c --conf-path=/path/to/conf

[Install]
WantedBy=default.target
```

#示例 aria2.daemon 展示了配置文件示例。

使用 aria2 做种

要做种下载好的种子，使用 `--check-integrity` 选项：

```
$ aria2c --check-integrity=true --seed-ratio=0.0 --dir="/path/to/iso"
"/path/to/torrent/archlinux-2020.03.01-x86_64.iso.torrent"
```

指定 `--seed-ratio=0.0` 选项可以永久做种。

另请参见

- aria2 手册 - 官方网站
- aria2 用例 - 官方网站

11.4 RTorrent

难度 进阶 | **篇幅** 14.6 KB | **所属** BitTorrent

本节将讲到

- 安装
- 配置
- 管理
- 通过Screen使用rTorrent
- 其他提示

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： Last updated in 2011（在 Talk:RTorrent# 中讨论）

rTorrent 是一个非常简洁、优秀、非常轻量的BT客户端. 它使用了 ncurses 库以 C++ 编写, 因此它完全基于文本并在终端中运行. 将 rTorrent 用在安装有 GNU Screen 和 Secure Shell 的低端系统上作为远程的 BT 客户端是非常理想的。

安装

从官方源 安装 rtorrent包 包。

你也可以从 AUR 中安装 rtorrent-gitAUR 或 rtorrent-extendedAUR\[损坏的链接：package not found\]。

配置

注意：查看关于此部分的 rTorrent Wiki 文章获取更多信息： Common Tasks in rTorrent for Dummies

在运行 rTorrent 前,首先要创建默认配置文件，该文件可以从 `/usr/share/doc/rtorrent/rtorrent.rc` 上找到，并将其另存为 `.rtorrent.rc` 放到你的主目录中.

```
$ cp /usr/share/doc/rtorrent/rtorrent.rc ~/.rtorrent.rc
```

保存后, 使用你喜爱的文本编辑器打开它并进行有必要的改动. 可以简单地去掉那些你需要使用的选项的注释， 如需更详细的资料,请访问rTorrent Common Tasks

性能优化

注意：查看关于此部分的 rTorrent wiki 文章获得更多信息： 性能优化

下面这些配置的值主要取决于你的系统和网络速度. 要寻找最佳配置，请访问下面网站并按照指示操作: 优化 BitTorrent 下载速度

```
# 每个种子所允许的最大最小连接数
#min_peers = 40
```

```
max_peers = 52

# 同上，针对的是已经完成的种子 （-1 表示和下载中的种子一致）
#min_peers_seed = 10
max_peers_seed = 52

# 每个种子的最大同时上传数
max_uploads = 8

# 全局的上传与下载速度限制（以 KB 为单位），"0"表示无限制
download_rate = 200
upload_rate = 28
```

check_hash 选项将在下载完成或 rTorrent 重新启动时对文件进行 Hash 校验。这将确保你获得/做种的文件没有错误：

```
check_hash = yes
```

创建和管理文件

directory 选项将决定你的 torrent 数据被保存在哪. 如果你想改变默认保存目录，请务必输入正确的绝对路径; rtorrent 有一个奇怪的 Bug，一些时候它不考虑相对路径 (如 ~/torrents):

```
directory = /home/[user]/torrents/
```

session 选项将允许 rTorrent 保存你的 torrents 的进度. 一定要创建一个会话目录 .session (普通用户运行:$ mkdir ~/.session):

```
session = /home/[user]/.session/
```

schedule 选项将使 rTorrent 监视一个特定目录中的 .torrent 文件。使用这一选项时一定要小心，这将移动 .torrent 文件到你的会话目录下，并将之重命名为其 Hash 值。 当你看到了一个你想要下载的种子文件，你可以使用你的浏览器将这一种子文件保存到这一监视目录下， rTorrent 将会自动的开始下载这一种子文件的内容。必须确保要监视的目录已经被创建(以普通用户身份执行: mkdir ~/watch):

```
# 监听目录中的新的种子文件，并停止那些已经被删除部分的种子
#schedule = watch_directory,5,5,load_start=./watch/*.torrent
#schedule = untied_directory,5,5,stop_untied=
schedule = watch_directory,5,5,load_start=/home/[user]/watch/*.torrent
schedule = untied_directory,5,5,stop_untied=
schedule = tied_directory,5,5,start_tied=
```

下面的 schedule 选项将使 rTorrent 在磁盘空间不足时停止。 这对于Seedbox这种磁盘空间非常有限的设备来说是很有用的。按照你的喜好来改变下面的数值：

```
# 当磁盘空间不足时停止下载
schedule = low_diskspace,5,60,close_low_diskspace=100M
```

端口配置

port_range 选项将指定选用哪一个端口去侦听。建议使用高于 49152 的端口。虽然 rTorrent 允许使用多个的端口，还是建议使用单个的端口。

```
port_range = 49164-49164
```

Additionally, make sure port forwarding is enabled for the proper port(s) (see: Port Forward Guides).

附加设置

`encryption` 选项使能加密功能。 使用加密功能十分重要not only for yourself, but also for your peers in the torrent swarm. 人们也许需要对它们的网络提供商模糊其带宽使用。即使你不需要这些保护，开启加密也不会对你有所负作用。详细信息请见：Bittorrent Protocol Encryption

```
# 加密选项，设为0（默认情况）或下面的任何一个：
# allow_incoming, try_outgoing, require, require_RC4, enable_retry, prefer_plaintext
#
# 如下例中的值将允许将接入连接加密，开始时以非加密方式作为连接的输出方式，
# 如行不通则以加密方式进行重试，在加密握手后，优先选择将纯文本以 RC4 加密
#
# encryption = allow_incoming,enable_retry,prefer_plaintext
encryption = allow_incoming,try_outgoing,enable_retry
```

下面的选项用以加入 DHT 支持。如果你使用了 public trackers，你可能希望使能 DHT 以获得更多的连接。如果你仅仅使用了私有的连接，**请不要使能 DHT**，因为这将降低你的速度，并可能造成一些隐密的危险。一些私有连接甚至会就使用 DHT 而向你发出警告。

```
# 使能对 trackerless torrents 或当所有的 tracker 都停止情况下的 DHT 支持。
# 可以被设置为 "disable" （完全禁止 DHT），"off"（不启动 DHT）， "auto"（按需要启
# 动或停止 DHT）或者 "on" （即时开启 DHT）。
# 默认设置为 "off"。要使 DHT 工作，会话目录必须被定义。
#
# dht = auto

# UDP 使用 UHT的端口
#
# dht_port = 6881

# 使能 peer 的交换 （对那些没有被标记为私有的 torrent）
#
# peer_exchange = yes
```

注意：See the rTorrent wiki article on this subject for more information: Using DHT

管理

Cmd	Action
Ctrl-q	Quit application
Left	Returns to the previous screen
Right	Goes to the next screen
Backspace	Adds the specified *.torrent

rTorrent依靠用户输入的专有的快捷键.可在官方网站上查看完整的: rTorrent用户指南

注意：快速敲击 `Ctrl-q` 两次会使得 rTorrent 关闭，而不会等待向连接的 tracker 发送停止声明。

这里是一些最基础的快速参考：

- Control-q : closes rTorrent, done twice makes the program shutdown without waiting to send stopping information to the trackers.

- Left arrow : returns to the previous screen.
- Right arrow : goes to the next screen.
- a|s|d : increase global upload throttle about 1|5|50 KB/s
- A|S|D : increase global download throttle about 1|5|50 KB/s
- z|x|c : decrease global upload throttle about 1|5|50 KB/s
- Z|X|C : decrease global download throttle about 1|5|50 KB/s
- Control-S : starts download
- Control-D : stops an active download, removes a stopped download.
- \+ or - : changes the download priority of selected torrent.
- Backspace : adds the specified .torrent. After pressing this button write full path or URL of .torrent file. You can use Tab and other tricks from bash.

Redundant mapping

`Ctrl-s` is often used for terminal control to stop screen output while `Ctrl-q` is used to start it. These mappings may interfere with rTorrent. Check to see if these terminal options are bound to a mapping:

```
$ stty -a
...
swtch = <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^V;
...
```

To remove the mappings, change the terminal characteristics to undefine the aforementioned special characters (i.e. `stop` and `start`):

```
# stty stop undef
# stty start undef
```

To remove these mappings automatically at startup you may add the two preceding commands to your `~/.bashrc` file.

通过Screen使用rTorrent

Screen 是允许CLI应用程序在后台运行并支持X运行的一个程序.

安装:

```
pacman -S screen
```

然后复制screenrc到你的用户主目录:

```
cp /etc/screenrc ~/.screenrc
```

要总是通过Screen来运行rTorrent,在.screenrc文件中添加以下内容:

```
screen -t rtorrent rtorrent
```

要运行screen和rTorrent, 只要简单的在终端运行*screen*就可以了. Control-a followed by d will detach screen, and running *screen -r* will open screen again.

On a remote machine

Most setups of rTorrent on a remote machine involve using Screen. Supposing you have a detached Screen session with rtorrent on the remote machine (and the option to SSH into it), you can gain access to it using:

```
/usr/bin/ssh -t -p <ssh port on remote machine> <user>@<remote machine> screen -RD
```

If you want immediate access on startup, you will need to upload a key from your machine to remote host (so you will not be prompted for a password) and setup a terminal to run the command above. An inittab example using rungettyAUR\[损坏的链接：package not found\] on virtual console 4:

```
sam:45:respawn:/sbin/rungetty tty4 -u <local user> -- /usr/bin/ssh -t -p <ssh port on
remote machine> <remote user>@<remote machine> screen -RD
```

Screen后台运行rTorrent

I use this on my home server to run rtorrent w/ screen as a daemon. With the username *rtorrent* Just create an *rtorrent* file in your */etc/rc.d/* and add the following code.

```
#!/bin/bash

. /etc/rc.conf
. /etc/rc.d/functions

case "$1" in
  start)
    stat_busy "Starting rtorrent"
    su rtorrent -c 'screen -d -m rtorrent' &> /dev/null
    if [ $? -gt 0 ]; then
      stat_fail
    else
      add_daemon rtorrent
      stat_done
    fi
    ;;
  stop)
    stat_busy "Stopping rtorrent"
    killall -w -s 2 /usr/bin/rtorrent &> /dev/null
    if [ $? -gt 0 ]; then
      stat_fail
    else
      rm_daemon rtorrent
      stat_done
    fi
    ;;
  restart)
    $0 stop
    sleep 1
    $0 start
    ;;
  *)
    echo "usage: $0 {start|stop|restart}"
esac
exit 0
```

On a remote computer I use the following script to connect to the server's daemon process:

```
ssh -t rtorrent@192.168.1.10 'screen -r'
```

其他提示

- To use rTorrent with a tracker that uses https, do the following as root:

```
cd /etc/ssl/certs
wget --no-check-certificate
https://web.archive.org/web/20160505025153/https://www.geotrust.com/resources/root_cert
ificates/certificates/Equifax_Secure_Global_eBusiness_CA-1.cer
mv Equifax_Secure_Global_eBusiness_CA-1.cer Equifax_Secure_Global_eBusiness_CA-1.pem
c_rehash
```

And from now on run rTorrent with:

```
rtorrent -o http_capath=/etc/ssl/certs
```

Be sure to change .screenrc to reflect this change if you use screen:

```
screen -t rtorrent rtorrent -o http_capath=/etc/ssl/certs
```

- To create .torrent files, I recommend mktorrent包 for commandline interface or RubyTorrent for GUI.

Send Text Message Upon Torrent Completion Using GMail

Cell phone providers allow you to "email" your phone:

```
Verizon: 10digitphonenumber@vtext.com
Former AT&T customers: 10digitphonenumber@mmode.com
Sprint: 10digitphonenumber@messaging.sprintpcs.com
T-Mobile: 10digitphonenumber@tmomail.net
Nextel: 10digitphonenumber@messaging.nextel.com
Cingular: 10digitphonenumber@cingularme.com
Virgin Mobile: 10digitphonenumber@vmobl.com
Alltel: 10digitphonenumber@alltelmessage.com OR
10digitphonenumber@message.alltel.com
CellularOne: 10digitphonenumber@mobile.celloneusa.com
Omnipoint: 10digitphonenumber@omnipointpcs.com
Qwest: 10digitphonenumber@qwestmp.com
```

If you have Verizon, your cell phone's "email" is 5551234567@vtext.com

- Install **Heirloom's mailx** program:

```
pacman -S mailx-heirloom
```

- Clear the /etc/nail.rc file and enter:

```
set smtp=smtp.gmail.com:587
set smtp-use-starttls
set ssl-verify=ignore
set ssl-auth=login
set smtp-auth-user=USERNAME@gmail.com
set smtp-auth-password=PASSWORD
```

Now to send the text, we must pipe a message to the mailx program.

- Make a bash script (/path/to/mail.sh):

```
echo "Torrent Done" | mailx 5551234567@vtext.com
```

- And finally, add the important ~/.rtorrent.rc line:

```
on_finished = move_complete,"execute=/path/to/mail.sh"
```

Note: The "move complete" will not affect anything. I could not get it to work without it.

See Also

Screen Tips

A Detailed Intro to Bittorrent including definition of terms terms

11.5 Transmission

难度 进阶 | **篇幅** 16.2 KB | **所属** BitTorrent

本节将讲到

- 安装
- 图形界面的配置
- Transmission 守护程序和命令行
- 通知
- Web 界面

Transmission 是一个轻量级、跨平台的 BitTorrent 客户端。

安装

有多种可选的软件包：

- transmission-cli包 - 包括命令行工具、守护程序和 #Web 界面。
- transmission-gtk包 - GTK\+ 4 图形界面。
- transmission-qt包 - Qt 6 图形界面。
- tremcAUR 或 tremc-gitAUR - 用于守护程序的 Curses 界面。
- stigAUR 或 stig-gitAUR – 用于守护程序的 Curses 界面。
- transmission-remote-gtk包 – 用于守护程序的 GTK+ 3 界面。

图形界面的配置

两种带 GUI 的版本 *transmission-gtk* 和 *transmission-qt* 都可以在没有后端守护程序的情况下独立运行。

GUI 版本都被配置为开箱即用，但用户可能仍需要更改一些设置。GUI 配置文件的默认路径是 `~/.config/transmission`。

在 Transmission 的 Github 上，有一份各种选项如何配置的教程：https://github.com/transmission/transmission/blob/main/docs/Editing-Configuration-Files.md.

Transmission 守护程序和命令行

transmission-cli 支持的命令包括：

: *transmission-daemon*: 启动守护程序。 : *transmission-remote*: 调用本地或远程守护程序的命令行界面，后面跟着希望守护程序运行的命令。 : *transmission-show*: 返回指定种子文件的信息。 : *transmission-create*: 创建一个新的种子文件。 : *transmission-edit*: 新增、删除或替换一个 tracker 服务器的 announce URL。 : *transmission-cli*: (已弃用) 启动一个非守护程序的本地 *transmission* 实例，用于手动下载一个种子。 : *tremc*: (需要 tremc-gitAUR) 为本地或远程守护程序启动一个 curses) 界面。

守护程序的启动与停止

就像 #选择一个用户里面的说明那样，transmission 的守护程序可以按以下方式运行：

- 作为 *transmission* 用户运行，只要用 systemd start/enable `transmission.service` 服务即可。

: 你可以按照 #选择一个用户里的说明更改默认用户。

- 用你自己的账户，只要在账户下运行：

```
    $ transmission-daemon
```

启动守护程序将会创建一个初始的 *transmission* 配置文件。详情参阅 #配置守护程序。

守护程序可以这样来停止：

```
$ pkill -3 transmission-daemon
```

另一种停止 transmission 的方法是使用 *transmission-remote* 命令：

```
$ transmission-remote --exit
```

减少多余的日志

运行 transmission-daemon 可能会产生许多不需要的日志条目。只要用一个小脚本包装一下再启动就可以过滤日志输出。以下示例可以提供一些启发：

```
transwrap.sh
#!/bin/zsh
killall transmission-daemon 2> /dev/null
transmission-daemon --foreground --log-info 2>&1 | while read line; do
       echo $line |
              grep -v "announcer.c:\|platform.c:\|announce done (tr-dht.c:" |
              grep -v "Saved.*variant.c:" |
              while read line; do
                     echo $line | grep -q "Queued for verification (verify.c:" &&
                            notify-send --app-name="Transmission Started" "${line#*
* }"
                     echo $line | grep -q "changed from .Incomplete. to .Complete." &&
                            notify-send --app-name="Transmission Complete" "${line#*
* }"
                     echo $line | systemd-cat --identifier="TransWrap" --priority=5
              done 2>&1 > /dev/null
       done&disown
```

仅当连接到网络时运行

Netctl

你可能只想在连接某些网络时运行 transmission。下面的脚本会先检查连接是否在授权的网络列表中，是的话再继续启动 transmission-daemon。

```
/etc/netctl/hooks/90-transmission.sh
#!/bin/bash

# The SSIDs for which we enable this.
declare -A ssids=(
```

```
    ["network_1"]=y
    ["network_2"]=y
)

if [[ ${ssids[$SSID]} ]]; then
    case $ACTION in
        CONNECT|REESTABLISHED)
            # Need to wait, otherwise doesn't seem to bind to 9091.
            sleep 30
            systemctl start transmission
            ;;
        *)
            systemctl stop transmission
            ;;
    esac
fi
```

选择一个用户

选择你想如何运行 `transmission`：

- 作为独立用户运行，默认用户是 `transmission`（推荐，因为安全性更高）。

默认情况下，*transmission* 会创建名为 `transmission` 的用户和组（它的主目录的文件位于 `/var/lib/transmission/`），并且用这个“账户”运行。这是一种安全措施，因为这样 *transmission* 及其下载的文件将无法访问 `/var/lib/transmission/` 目录之外的文件。配置、操作和访问下载的文件需要使用 “root” 权限完成（比如用 sudo）。

- 在你自己的账户下运行。设置这种方式需要重写已提供的服务文件并在其中指定你的用户名：

```
/etc/systemd/system/transmission.service.d/username.conf
[Service]
User=your_username
```

配置守护程序

启动守护程序来创建缺省的配置文件。

- 如果用 `transmission` 这个用户来运行 Transmission，配置文件会存放在 `/var/lib/transmission/.config/transmission-daemon/settings.json`。
- 如果用你自己的账户运行 Transmission，配置文件会存放在 `~/.config/transmission-daemon/settings.json`。

使用一个 Transmission 客户端或者用自带的 web 界面（在支持的浏览器中用 http://localhost:9091 来访问）可以自定义守护程序。

在 Transmission 的网站上，有一份各种选项如何配置的教程：https://github.com/transmission/transmission/blob/main/docs/Editing-Configuration-Files.md

注意：

- 如果你想用一个文本编辑器手动编辑配置文件，请先 停止守护程序；否则当它停止时将会覆盖配置文件。
- 另一种方式是，可以用 SIGHUP 信号指示守护程序重新加载其配置，运行 `kill -s SIGHUP pidof transmission-daemon `即可。

一个给那些用 `transmission` 账户运行的用户的建议是，用适当的权限创建一个共享的下载目录，这样就能同时允许 `transmission` 账户和其他账户访问下载文件，也能相应地更新配置文件。例如：

```
# mkdir /mnt/data/torrents
# chown -R facade:transmission /mnt/data/torrents
# chmod -R 775 /mnt/data/torrents
```

现在 `/mnt/data/torrents` 目录可供系统账户 `facade` 和 `transmission` 用户所属的 `transmission` 组访问。非常不鼓励使这个目录可读写（即不要将目录 *chmod* 到 *777*），而是应该将个别用户/组的部分权限授予部分目录。

注意：如果 `/mnt/data/torrents` 位于一个可移除的设备上，比如这是一个 `/etc/fstab` 条目且带有 `nofail` 选项，那么 Transmission 会报告说没有找到文件。要解决这个问题，可以在 `/etc/systemd/system/transmission.service.d/transmission.conf` 文件的 `[Unit]` 一节中加入 `RequiresMountsFor=/mnt/data/torrents`。

另一种方法是把你的账户加入 `transmission` 用户组 (`#usermod -a -G transmission yourusername`)，然后修改 `/var/lib/transmission` 和 `/var/lib/transmission/Downloads` 目录的权限设置，以允许 `transmission` 组成员对这两个目录的 `rwx` 访问权。

Host 白名单

如果你想用远程主机的主机名来访问位于该主机上的 Transmission 守护程序，你需要把主机名加入 `settings.json` 文件中的 `rpc-host-whitelist` 字段中。 否则你会在连接主机时收到一条 "421 Misdirected Request" 错误。

如果你用该服务器的 IP 地址来访问守护程序，这一步可以省略。

监视目录

如果你想*自动添加某个文件夹中的 .torrent 文件*，但是配置文件中的 `watch-dir` 和 `watch-dir-enabled` 选项都不起作用，可以带 `-c /path/to/watch/dir` 参数启动 transmission 守护程序。

如果你在用 systemd，按照 systemd#修改现存单元文件里面的描述编辑 `transmission.service` 单元文件。

启用 IPv6

默认情况下，守护程序仅监听 IPv4 连接。要同时监听 IPv6 连接，把 `settings.json` 里的 `rpc-bind-address` 选项改为 `"::"`。

CLI 使用示范

如果你想删除所有已完成的种子，可以用下面的命令（用你自己的用户名和密码）：

```
# transmission-remote -n 'username:password' -l | grep 100% | awk '{print $1}'| paste -d, -s | xargs -i transmission-remote -t {} -r
```

为已下载的文件做种：

```
# transmission-remote [HOST] --torrent=example.torrent -a example.torrent --verify --download-dir=/dir/to/folder --start
```

通知

停止 transmission-daemon 并在 `.config/transmission-daemon/settings.json` 中添加以下内容：

```
   "script-torrent-added-enabled": true,
   "script-torrent-added-filename": "path/to/transmission-handler.sh",
   "script-torrent-done-enabled": true,
   "script-torrent-done-filename": "path/to/transmission-handler.sh",
transmission-handler.sh
#!/bin/bash

percentage=$(transmission-remote -t $TR_TORRENT_ID -l | awk -v ID="$TR_TORRENT_ID" '$0 ~ ID{print $2}')

if [ $percentage != "100%" ]
then
     notify-send --app-name="Transmission Started" "Transmission: started $TR_TORRENT_NAME"
else
     notify-send --app-name="Transmission Complete" "Transmission: downloaded $TR_TORRENT_NAME"
fi
```

Web 界面

GUI 方式

当 Transmission 安装完毕后就可以轻易架起 web 界面。你只需要单击 *edit* 菜单，选择 *preferences*。单击 *Remote* 选项卡并启用 *Allow remote access*。

此处你可以修改默认的监听端口 9091。

单击 *Use authentication* 并填写用户名和密码就可以启用身份认证。

要提高安全性，可以启用 ***Only allow these IP addresses*** 限制来自任意 IP 的访问。

现在你可以单击 *Open web client* （用你的默认浏览器打开）或手动访问 `http://*TARGET_IP_ADDRESS*:*PORT*` 来启动 web 界面。

如果没有修改监听端口，默认是 9091。在这种情况下访问链接是 `[http://localhost:9091](http://localhost:9091)`。

注意：记住，必须安装 transmission-cli包。

CLI 方式

设置 web 界面甚至不需要借助图形界面，因为守护程序提供了类似的选项。你无需指定任何参数即可访问 web 界面，可参阅 #守护程序的启动与停止。

不过，你仍然可以指定在前一节中看到的所有选项：

```
$ transmission-daemon --auth --username arch --password linux --port 9091 --allowed "127.0.0.1"
```

等同于

```
$ transmission-daemon -t -u arch -v linux -p 9091 -a "127.0.0.1"
```

注意：如果你设置了用户名和密码，那么所有后续对`transmission-remote`的命令都需要使用`--auth username:password`进行身份验证。请参阅：transmission-remote commands are erroring with Unauthorized user

可选的网页 UI 主题

如果觉得 web app 的默认用户界面过时、不好看，试试这个简单CSS主题：
https://git.eigenlab.org/sbiego/transmission-web-soft-theme

另一个完整替代方案，可以和默认界面共存：
https://github.com/ronggang/transmission-web-control

作为makepkg DLAGENT的用法

Transmission可以连同transmission-dlagentAUR下载代理一起，作为makepkg的磁链下载代理。

磁链URI需要将其前缀从`magnet:?`更改为`magnet://`。

故障排除

无法使用网络连接至守护程序

在 `network.service` 初始化后守护程序就会开始运行。但是，如果启用了 `dhcpcd` 服务而不是特定设备的服务（比如 `dhcpcd@enp1s0.service`），可能 Transmission 会因为启动过早而无法绑定到相应的网络接口。于是 web 界面无法访问。一种可能的解决方案是在 systemd 单元的配置文件里增加 `Requires` 行。

```
/etc/systemd/system/transmission.service.d/fixdep.conf
[Unit]
Requires=network.target
```

无法显示 Web 界面

```
404: Not Found
```

```
Couldn't find Transmission's web interface files!

Users: to tell Transmission where to look, set the TRANSMISSION_WEB_HOME environment
variable to the folder where the web interface's index.html is located.

Package Builders: to set a custom default at compile time, #define PACKAGE_DATA_DIR in
libtransmission/platform.c or tweak tr_getClutchDir () by hand.
```

即使你用图形界面，也需要安装 transmission-cli包 来支持 web 界面。

接收/发送缓冲区设置失败

在启动时 Transmission 可能会在日志中显示如下的消息：

```
UDP Failed to set receive buffer: requested 4194304, got 425984
UDP Failed to set send buffer: requested 1048576, got 425984
```

通过使用sysctl设置 `net.core.rmem_max=4194304` 和 `net.core.wmem_max=1048576` 可以解决。

transmission-remote频繁超时或网络界面无响应

Transmission并未异步处理磁盘IO，因此在进行大量写入时，可能会变得无响应，详情参见。建议将Transmission的下载目录放在更快的存储设备上，如SSD而非HDD。

transmission-gtk缺少托盘图标

GTK 4版本的托盘图标目前无法工作。有一个选择是使用GTK 3版本的transmission-gtk3AUR。

401: 未授权

在为web界面设置用户名和密码后，使用`transmission-remote`时可能会出现以下错误：

```
< h1>401: Unauthorized\Unauthorized User
```

这种情况是因为设置了用户名和密码。此后，所有来自`transmission-remote`的命令都必须经过身份验证。详情参见：transmission-remote命令因未授权的用户而出错

更多信息

- Transmission wiki (old)
- Transmission documentation on GitHub
- Headless Usage

无线网络

无线网络的麻烦多半不在配置，而在驱动、固件与芯片支持度这几层。本章把机器从“认不出网卡”带到“稳定连上”，再进一步做成接入点，或把多条无线链路绑成一条。

先读《网络配置/无线网络配置》，它把驱动、工具、iw 与认证讲透，是其余各节的地基；再读《软件接入点》与《Wireless bonding》；《Ad-hoc networking》《博通无线网卡》仅在设备或拓扑特殊时跳读。最易踩的坑是以为装好驱动就完事——固件缺失、内核模块未加载，以及默认任意网卡都能开接入点或参与绑定，都会让报错看似无解。

12.1　Ad-hoc networking

难度 入门 | **篇幅** 5.4 KB | **所属** 无线网络

本节将讲到

- 要求
- Wifi 链路层
- 网络配置
- 技巧
- See also

Independent Basic Service Set(独立基本服务集)，缩写是 IBSS，也被称为自组网络，是一种不使用中心控制节点，就能让一组设备相互通讯的方法。是点对点网络的一种，网络中的设备直接通讯，不需要中转。自组网络可以用来进行网络共享。

要求

所有要连接到网络的设备都具有兼容 的无线网卡。

Wifi 链路层

因为 IBSS 网络是点对点网络，所有设备都应该使用相同的 wifi 连接设置。

提示：可以设置更复杂的网络拓扑，Linux 无线文档 包含更高级的示例。

手动设置

警告：此方法创建的是 **非加密** 自组网络，要创建加密网络，请参考 #wpa_supplicant.

详细信息请参考网络配置/Wireless#iw。请先安装软件包 iw包。

设置操作模式为 ibss:

```
# iw interface set type ibss
```

启动接口(可能需要额外的 `rfkill unblock wifi`):

```
# ip link set interface up
```

现在可以创建自组网络了，将 *your_ssid* 替换为实际的网络，*frequency* 替换为以 MHz 为单位的频率。频道和频率的对应关系，请参考, 这里。

```
# iw interface ibss join your_ssid frequency
```

wpa_supplicant

确保已经安装了 wpa_supplicant包，并进行了配置(参考 wpa_supplicant).

```
/etc/wpa_supplicant-adhoc.conf
ctrl_interface=DIR=/run/wpa_supplicant GROUP=wheel

# use 'ap_scan=2' on all devices connected to the network
# this is unnecessary if you only want the network to be created when no other networks
are available
ap_scan=2

network={
    ssid="MySSID"
    mode=1
    frequency=2432
    proto=RSN
    key_mgmt=WPA-PSK
    pairwise=CCMP
    group=CCMP
    psk="secret passphrase"
}
```

所有连接到网络的设备运行下面 *wpa_supplicant* 命令:

```
# wpa_supplicant -B -i interface -c /etc/wpa_supplicant-adhoc.conf -D nl80211,wext
```

网络配置

最后一步是给网络中的所有设备分配 IP 地址，有多种方法：

- 分配静态 IP 地址，参考：网络配置#Static IP address.
- 在一个设备上运行 DHCP，参考 dhcpd 或 dnsmasq.
- 运行 *avahi-autoipd*. 参考 Avahi#Obtaining IPv4LL IP address.

要向自组网络共享外网连接，请参考 Internet sharing.

技巧

使用 NetworkManager

如果使用 NetworkManager，可以使用 *nm-applet* 进行自组网络配置，不用使用手动方法。详情参考 NetworkManager#Sharing internet connection over Wi-Fi。

自定义 systemd 服务(使用 wpa_supplicant 和静态 IP)

可以使用下面模板启用无线自组网络：

```
/etc/conf.d/network-wireless-adhoc@interface
addr=192.168.0.2
mask=24
/etc/systemd/system/network-wireless-adhoc@.service
[Unit]
```

```
Description=Ad-hoc wireless network connectivity (%i)
Wants=network.target
Before=network.target
BindsTo=sys-subsystem-net-devices-%i.device
After=sys-subsystem-net-devices-%i.device

[Service]
Type=oneshot
RemainAfterExit=yes
EnvironmentFile=/etc/conf.d/network-wireless-adhoc@%i

# perhaps rfkill is not needed for you
ExecStart=/usr/bin/rfkill unblock wifi
ExecStart=/usr/bin/ip link set %i up
ExecStart=/usr/bin/wpa_supplicant -B -i %i -D nl80211,wext -c /etc/wpa_supplicant-
adhoc.conf
ExecStart=/usr/bin/ip addr add ${addr}/${mask} dev %i

ExecStop=/usr/bin/ip addr flush dev %i
ExecStop=/usr/bin/ip link set %i down

[Install]
WantedBy=multi-user.target
```

See also

- Ubuntu community wiki WifiDocs/Adhoc
- Manual about creating an Ad-Hoc network on UbuntuGeek
- Share your 3G Internet connection over wifi

12.2 博通无线网卡

难度 进阶 | **篇幅** 8.1 KB | **所属** 无线网络

本节将讲到

- 1 历史
- 查看你可以使用何种驱动
- 获取驱动
- 加载多博通网卡的内核驱动模块
- 故障排除

这篇文章详细地介绍了如何安装和设置博通（Broadcom）无线网卡。

1 历史

博通（Broadcom）对于 GNU/Linux 的 Wi-Fi 设备支持方面有着悠久的历史。在它最开始一段历史的大部分时间里，博通的设备要么完全就不支持，要么就得让用户修改固件才能用。有一些无线设备可以通过使用通过逆向工程的驱动程序来获得支持。

2008年8月，博通官方发布了 802.11 Linux STA 驱动程序以在 GNU/Linux 上提供无线设备的驱动程序。尽管这是一个限制许可的驱动程序，也不适用于隐藏的 ESSID，但博通承诺未来将使用更开放的方法。

2010年9月，博通发布了完全开源的无线网卡硬件驱动。这个名为 brcm80211 的驱动程序已被列入到自2.6.37之后的内核中。随着 Linux 内核 2.6.39 发布，这些驱动程以下是可用的驱动程序列表：序已被重新命名为 `brcmsmac`和 `brcmfmac`。

一

驱动	描述
bcrm80211	n
主线（推荐使用）版本ub43	逆向工程内核驱动
broadcom-wl	限制许可的 Broadcom STA 驱动

查看你可以使用何种驱动

首先，向你的终端输入以下内容来检测网卡的 PCI-ID:

```
$ lspci -vnn | grep 14e4:
```

然后在以下列表中检查 b43的支持设备列表以及 brcm80211的支持设备列表.

获取驱动

brcm80211

Kernel内建了两个开源驱动: **brcmfmac** 提供原生硬MAC支持， **brcmsmac** 提供基于mac80211的软MAC支持。 它们应该会在启动时自行加载。

注意：

- **brcmfmac** 提供较新的芯片支持，并且支持AP模式，P2P模式，高级加密。
- **brcmsmac** 仅提供对于较老芯片的支持，例如BCM4313, BCM43224, BCM43225。

b43/b43legacy

b43 以及 **b43legacy**两个逆向工程驱动已经被内建在Kernel中。b43支持大部分的博通无线芯片组，而b43legacy驱动仅支持早期的BCM4301以及BCM4306 rev.2 芯片组。为了避免与别的驱动造成冲突，请 blacklist 未使用的驱动。

这些驱动的运行都需要安装闭源固件，请从AUR安装b43-firmwareAUR， b43-firmware-classicAUR 或者 b43-firmware-legacyAUR\[损坏的链接：package not found\] 。

注意：

- BCM4306 rev.3, BCM4311, BCM4312 与 **b43-firmware** 固件不能良好的工作。对于这些芯片组请使用 b43-firmware-classicAUR 代替。
- BCM4331 与 **b43-firmware-classic** 固件不能良好的工作。 对于这些芯片组请使用 b43-firmwareAUR 代替。

broadcom-wl

AUR 中有两个版本的Broadcom STA闭源驱动：

- 通常的 broadcom-wl包
- 以及 DKMS 版本 broadcom-wl-dkms包

提示：DKMS版本 broadcom-wl-dkms包

- 可以在不同的Kernel中工作 (例如 linux-ckAUR).
- 每次安装新的Kernel时，dkms都会重新构建驱动，如果你使用 broadcom-wl包 或者其他依赖于单个Kernel版本的驱动 (例如 broadcom-wl-ckAUR), 那么在更新Kernel或者使用新的Kernel时都有可能使驱动崩溃。

加载broadcom-wl的内核模块

`wl` 模块可能会与其他模块冲突而无法加载。加载`wl`模块之前， 请移除`b43`或者其他可能造成冲突的模块：

```
# rmmod b43
```

如果 `ssb` 加载了，也请一并移除:

```
# rmmod ssb
```

注意：错误的加载 `ssb` 可能导致无线界面无法被创建。

加载 `wl` 模块：

```
# modprobe wl
```

加载 `wl` 模块的同时 `lib80211` 或者 `lib80211_crypt_tkip` 应该也会被自动加载。 使用 `lsmod` 来进行检查，如果没有，请手动加载二者之一；

```
# modprobe lib80211
```

或

```
# modprobe lib80211_crypt_tkip
```

如果你直接从博通官方网站下载驱动并安装，你可能同时需要更新依赖模块:

```
# depmod -a
```

如果模块无法在启动时加载，请将 `wl` (以及 `lib80211`/`lib80211_crypt_tkip`如果需要) 到 `/etc/rc.conf`的MODULES列：

```
# MODULES=(... wl...)
```

你也可以Blacklist掉可能冲突的模块，在 `/etc/modprobe.d/modprobe.conf`中加入:

```
# blacklist b43
# blacklist ssb
```

警告：Broadcom Corporation BCM4311 802.11b/g WLAN \[14e4:4311\] 在Blacklist `b43` 以及 `ssb`后可能不工作。

加载多博通网卡的内核驱动模块

b43的博通无线网卡驱动与**b44**博通有线网卡驱动可能产生冲突，因此建议您使用**broadcom-wl**驱动

- Put "lib80211_crypt_tkip" and "wl" at the BEFORE b44 (if you have it) position in MODULES= 在 /etc/rc.conf 模块部分，b44(如果你需要这个驱动的话)之前加入lib80211_crypt_tkip wl
- 不要忘记把 b43 模块加入黑名单
- 您的无线网卡设备为eth0
- 您的有线网卡设备为eth1
- 两者能同时正常工作

故障排除

各种可以纠正的错误。

更新 Kernel 后设备无法访问

如果您使用 **brcm80211** 请确保没有被 blacklisted。如果你使用 **b43** 驱动确保它在工作。如果你使用**Broadcom STA**驱动的broadcom-wl包包，请重新安装或一劳永逸地切换成broadcom-wl-dkms包。

使用 broadcom-wl 驱动的设备不工作/不显示

请查看dmesg是否存在报错信息，**broadcom-wl**经常发生奇怪的问题，重新加载Kernel模块或者重启一般能解决问题

使用 broadcom-wl 驱动时接口交换

使用 broadcom-wl 驱动也许会发现它们的以太网和 Wi-Fi 接口交换了。查看这里（reverting to traditional interface names）的解决方案。

接口显示正常但是不能连接

加入以下内核参数:

```
b43.allhwsupport=1
```

奇怪的报错信息

你可能会在启动时接收到奇怪的报错信息，类似于：

```
phy0: brcms_ops_bss_info_changed: arp filtering: enabled true, count 0 (implement)
phy0: brcms_ops_bss_info_changed: qos enabled: false (implement)
phy0: brcms_ops_bss_info_changed: arp filtering: enabled true, count 1 (implement)
enabled, active
```

一般情况下这并不影响驱动的正常工作，如果你想不想收到这些信息，可以提高printk的日志级别。

在`/etc/sysctl.d/`中创建一个`printk.conf`或者类似的配置文件:

```
printk.conf
kernel.printk = 3 3 3 3
```

最后`sysctl -p`确保配置被成功写入。

不能检测到设备 BCM43241

无论是 `lspci` 还是 `lsusb` 都不能检测到设备。这个问题目前无法解决。 请在解决后删除此部分。

在连接到某些路由器时可能不稳定

如果没有其他的解决方法，安装linux-lts包，或者使用低版本的驱动。

12.3 软件接入点

难度 进阶 | **篇幅** 12.5 KB | **所属** 无线网络

本节将讲到

- 要求
- 配置
- 启用802.11n支持
- 启用802.11ac支持
- 启用802.11ax支持

软接入点（也称为虚拟路由器或虚拟Wi-Fi）使计算机能够将其无线接口转换为Wi-Fi热点，无需使用独立无线路由器。

要求

无线网卡必须支持AP模式

无线设备必须兼容 nl80211标准，并且支持 AP 工作模式。可通过 `iw list` 命令来验证, 该命令输出信息的 `Supported interface modes` 段落中要有 `AP` 模式:

```
$ iw list
Wiphy phy1
...
       Supported interface modes:
                * IBSS
                * managed
                * AP
                * AP/VLAN
                * WDS
                * monitor
                * mesh point
...
```

注意：自 2019 年以来，大多数 Intel 无线网卡上不再能支持 5GHz 频段的 AP 服务，原因是固件错误地将位置感知监管（LAR）功能保留为启用状态，即使在 AP 模式下也是如此，但根据 报告，可以通过一个特殊的 hostapd 补丁让它工作。请同时参考 #无法在 5Ghz 频段启用热点模式 。

单个Wi-Fi设备同时作为无线客户端和AP

软热点的创建和系统的网络连接方式(以太网,无线网等) 是独立的。许多无线设备甚至支持*并存*操作，即同时作为热点和无线“客户端“使用。通过这个功能可以为当前连接

网络创建一个热点，就像“无线中继器”一样。`iw list` 命令的输出中会显示设备是否支持并行操作：

```
$ iw list
Wiphy phy1
...
        valid interface combinations:
                 * #{ managed } <= 2048, #{ AP, mesh point } <= 8, #{ P2P-client, P2P-GO } <= 1,
                   total <= 2048, #channels <= 1, STA/AP BI must match
...
```

约束`#channels <= 1`说明创建的热点必须和无线客户端连接处于同一信道。参见下文 `hostapd.conf` 示例中 `channel` 的设置。

如果有线连接不可用, 使用这个功能时需要分别创建两个“虚拟网络接口（virtual interface）”。可以通过如下方式为`wlan0` 创建负责网络连接的 `wlan0_sta` 和负责热点的 `wlan0_ap`。两个虚拟网卡具有不同的MAC地址。

```
# iw dev wlan0 interface add wlan0_sta type managed addr 12:34:56:78:ab:cd
# iw dev wlan0 interface add wlan0_ap  type managed addr 12:34:56:78:ab:ce
```

可以用 macchanger 创建随机 MAC 地址。

配置

接入点的设置包含两个主要部分:

- 设置**Wi-Fi链路层**，这样无线客户端可以与电脑的“软件接入点”建立连接，从而同电脑进行通信。
- 配置电脑上的网络, 使电脑可以在 Internet 和无线客户端之间有效地转发IP包。

无线链路层

我们使用支持WPA2的hostapd包建立Wi-Fi链路。

*hostapd*所自带的默认配置文件中有很多选项列出，且均有相关描述，读者可以参考该配置文件以对各个选项有个大致的了解。

按实际需要调整*hostapd*配置文件中的内容。尤其是修改`ssid`和`wpa_passphrase`。如果你的网卡支持WiFi5和WiFi6的话，启用`ieee80211ac`与`ieee80211ax`。`ht_capab`用于开启若干AP功能，其中包括对传输速度影响显著的频道宽度。 详情请参考 hostapd Linux 手册.

```
/etc/hostapd/hostapd.conf
interface=wlan0_ap
bridge=br0

# 要在 IEEE 802.11 管理框架中使用的 SSID（即热点名）
ssid=YourWiFiName
# 驱动接口类型（hostap/wired/none/nl80211/bsd）
driver=nl80211
# 国家或地区代码（ISO/IEC 3166-1）
```

```
country_code=US

# 工作模式 (a = IEEE 802.11a (5 GHz), b = IEEE 802.11b (2.4 GHz)
hw_mode=g
# 使用信道
channel=7
# 允许最大连接数
max_num_sta=5

# Bit 字段：bit0 = WPA, bit1 = WPA2
wpa=2
# Bit 字段：1=wpa, 2=wep, 3=both
auth_algs=1

# 加密协议；禁用不安全的 TKIP
wpa_pairwise=CCMP
# 加密算法
wpa_key_mgmt=WPA-PSK
wpa_passphrase=Somepassphrase

# hostapd日志设置
logger_stdout=-1
logger_stdout_level=2

## 启用802.11n支持
#ieee80211n=1
## 启用802.11ac支持
#ieee80211ac=1
## 启用802.11ax支持
#ieee80211ax=1
## QoS 支持
#wmm_enabled=1
## 请使用"iw list"查看设备信息并相应地修改ht_capab和vht_capab
#ht_capab=[HT40+][SHORT-GI-40][TX-STBC][RX-STBC1][DSSS_CCK-40]
#vht_capab=[RXLDPC][SHORT-GI-80][TX-STBC-2BY1][RX-STBC-1]
```

提示：虽然iwd包也可以用来开启无线AP，但相比之下更建议使用hostapd，因为hostapd有更多的选项可用，包括影响性能的选项。使用iwd可能会造成性能限制，比如AP可能会被限制在802.11n或频宽被限制在20MHz。

提示：使用UTF-8字符的SSID可以正常显示国际化的字符。配置文件中设置`utf8_ssid=1`可以启用这个功能。但是一些客户端可能无法正确识别编码（例如wpa_supplicant和windows 7）。

如果想开机自启*hostapd*，启用`hostapd.service`。

如果开机时会自启*hostapd*，请确认无线网络接口已经率先启动（bring up），否则 hostapd 会启动失败。为了确保无线接口已就绪，编辑单元配置文件，绑定（BindTo）你的网络接口并在其后（After）启动：

```
/etc/systemd/system/hostapd.service.d/override.conf
[Unit]
BindsTo=sys-subsystem-net-devices-wlan0.device
After=sys-subsystem-net-devices-wlan0.device
```

确保接口不会被其它网络管理器控制。如果你用的是NetworkManager，查阅NetworkManager#忽略特定设备。

警告：不同地区允许作为接入点的无线信道是不同的。由于无线设备的固件设计，你应当设置正确的地区来使用合法的信道。 **不要**选择非本地区域，这可能会非法扰乱网络通讯, 在信号覆盖范围内影响你和他人设备的无线功能！区域信道参见Wireless network configuration#Respecting the regulatory domain.

注意：如果你有一个基于RTL8192CU芯片组的网卡, 请从AUR中安装hostapd-rtl871xdrvAUR并在 `hostapd.conf` 文件中将`driver=nl80211` **替换为** `driver=rtl871xdrv`。

网络配置

有两种基本的实现方法：

1. **网桥**: 在电脑上搭一个网桥，无线客户端就可以像电脑一样访问同一个网络接口和同一个子网。
2. **NAT**: 通过 IP 转发/伪装和 DHCP 服务，无线客户端会专门使用一个子网, 数据进出这个子网是被网络地址转换的(NAT-ted)——就像是连接在你数字用户回路(DSL)或铜轴线(Cabel)调制解调器上的一个普通的无线路由器一样。

使用网桥方式会更加简单, 但它要求无线客户端所需要的任何服务（例如 DHCP）在你电脑的外部接口上是可用的。因此如果你使用拨号连接（如,通过PPPoE或3G调制解调器）或你在使用铜轴线调制解调器，它会通过DHCP提供一个固定的IP地址, 在这种状态下桥接将不能正常工作。

使用NAT方式会更加灵活, NAT把Wi-Fi客户端和你的电脑清晰地分离开来并且对外界完全透明。 对于任何种类的网络连接它都适用，而且（如果必要的话）你可以使用iptables的方式来控制进出策略。

当然，将两者结合物也是可能的，比方说：某个拥有一个静态IP的网桥既包含以太网设备又有无线设备，同时提供DHCP服务，又能够配置NAT将数据转发到另外一个连接了 WAN 的网络设备。

网桥设置

创建一个网桥并把网络接口 (如 `eth0`) 加入其中 。**不要**将无线网络接口（如`wlan0`）添加到网桥；hostapd 会自行添加无线设备。 详情请参阅 Network bridge。

提示：如果已经有网桥（例如，虚拟机使用的）则可以复用使用该网桥。

NAT设置

详细内容请参考Internet sharing，在其中连接 LAN 的设备是 `net0`。这里我们通常指的是无线设备，比如说 `wlan0`。

工具

linux-wifi-hotspot

linux-wifi-hotspotAUR软件包中的脚本可以创建桥接或NAT方式的接入点。它综合使用hostapd包，dnsmasq和iptables使得接入点运作良好。该软件可以通过命令行或图形界面使用。创建一个简单的NAT方式的接入点可以执行：

```
# create_ap wlan0 eth0 MyAccessPoint MyPassPhrase
```

或者可以根据需求修改`/etc/create_ap.conf`配置文件，并指定使用该配置。

```
# create_ap --config /etc/create_ap.conf
```

或者使用图形界面，运行：

```
# wihotspot
```

Enable并start `create_ap.service`会使用`/etc/create_ap.conf`中指定的配置在启动时运行脚本。 详细信息，参见linux-wifi-hotspot的GitHub页面。

注意：在桥接模式下，“create_ap” 可能在启动时与当前网络配置冲突。在这种情况下，请勿配置以太网接口的IP地址，也不要配置DHCP和statip IP，以便于绑定到网桥。

RADIUS

可以参阅这篇博客的说明来运行WPA2企业级加密的 FreeRADIUS 服务。

常见问题与解答

无线局域网很慢

一般情况下可以尝试以下情况：

- `hw_mode`中配置的工作模式不合理会限制路由性能。检查并设置一个更合适的工作模式。
- 设置的`channel`拥挤或信号噪声较强也会影响无线网络质量，尤其是在人口稠密区域。尝试切换到其他频道或者使用其他频率。

受到NetworkManager干扰

如果网络设备被NetworkManager管理，hostapd可能无法正常工作。 可通过MAC屏蔽设备:

```
/etc/NetworkManager/conf.d/unmanaged.conf
[keyfile]
unmanaged-devices=mac:<hwaddr>
```

或通过接口名称屏蔽：

```
/etc/NetworkManager/conf.d/unmanaged.conf
[keyfile]
unmanaged-devices=interface-name:<ifname>
```

无法在 5Ghz 频段启用热点模式

对某些国家码，例如 `00`（全球），所有 5 Ghz 中可使用的频段都被设置了 C1 (*no-initiating-radiation*) 标记，hostapd 无法使用它们。需要安装 wireless-regdb包，并设置国家码，使 hostapd 可以使用你的国家所允许的频段。

相关文章

- Router
- Hostapd : The Linux Way to create Virtual Wi-Fi Access Point
- tutorial and script for configuring a subnet with DHCP and DNS

12.4 Wireless bonding

难度 高级 | **篇幅** 27.7 KB | **所属** 无线网络

本节将讲到

- 支持可移除设备的网络接口绑定
- DHCP 配置
- DHCPv6 配置
- 静态网络配置
- wpa_supplicant 配置

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

支持可移除设备的网络接口绑定

The Linux bonding driver provides a method for aggregating multiple network interfaces into a single logical "bonded" interface. Linux Ethernet Bonding Driver HOWTO

The Linux kernel bonding driver can be used to provide parallel network connections to maximize throughput, or to allow redundant network connections to maximize network availability. Here is an example of using the kernel bonding driver to maximize availability, by allowing network connections to "failover" between a primary network device and any number of secondary devices, or alternatively, by selecting the highest speed connection available. This approach provides Automatic Wired and Wireless Network Configuration with Removable Device Support, using only the kernel bonding module in "active-backup" mode, the sysfs, the iproute2包 commands, and systemd包 "template" Unit files, **without** using systemd-networkd.

This example will run wpa_supplicant continuously on any interface, as needed, and DHCP and DHCPv6 clients on a virtual "bond0" interface. This is useful, for instance, with a portable computer when you want to use the wired interface for speed and/or security when available, and the wireless interface when the wired interface is not available. The basic idea is to have two "always active" wired and wireless interfaces, then "bond" or "enslave" them to a virtual interface "master", and then let

the kernel bonding module handle switching between the interfaces. Of course, this scheme can be applied to any other type of network interface, and extended to more than two physical or virtual network interfaces.

Note that host networking is managed directly with systemd, and that no other "connection manager" is used here, providing a more basic approach. But then also, wpa_supplicant itself can still be managed directly using `wpa_gui` from wpa_supplicant_guiAUR, to scan for, select, and connect to new wireless access points/base stations.

In this example, there are six systemd service unit files used, along with five associated configuration files, for the kernel bonding module, wpa_supplicant, dhcp6c, dhclient包, and for static network configuration and specifying the primary slave network interface name. The six unit files are essentially generic service unit files which do not contain configuration data, and no modification is needed. The various service units may be stopped, started, and restarted individually without ordering errors or failed states. Any network interface device, such as typically a wired or wireless PC Card, may be removed and replaced, and reconfiguration will be automatic.

注意：All of the required systemd service files and configuration files from a working example are shown here because they are **not** the same as the standard files provided with the Arch Linux packages. Edit as required.

DHCP 配置

```
/etc/dhclient.conf
# These time-outs are aggressively short, supposing a sparsely populated network.
initial-interval 2;
reboot 5;
timeout 10;
retry 20;

# RFC 4361          Node-specific Identifiers for DHCPv4     February 2006
send dhcp-client-identifier 00:02:00:02:2e:2d:01:bd:c3:92:9a:44:2a:c4 ;
send host-name "laptop";
```

Make sure that the DHCP Client Identifier and the DHCPv6 Client Identifier are the same DUID. See `xxd -g1 -s2 /var/lib/dhcpv6/dhcp6c_duid`. The DHCP Server, dnsmasq包 for instance, can be configured to give fixed IP addresses based upon multiple MAC addresses, or provided hostname, or provided Client Identifier.

```
/etc/systemd/system/dhclient@.service
[Unit]
Description= ISC dhclient on interface %I
Documentation= man:dhclient(8) man:dhclient.conf(5)

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network.target
Before= network.target
After= network-pre.target

BindsTo= sys-subsystem-net-devices-%i.device

[Service]
ExecStartPre= /usr/bin/sleep 8
```

```
ExecStart= /usr/bin/dhclient -d -pf /run/dhclient-%i -i %I

# Release the current lease and ensure that dhclient has actually stopped.
ExecStop= /usr/bin/dhclient -r -pf /run/dhclient-%i
ExecStop= /usr/bin/sleep 1

Restart= on-abnormal

[Install]
WantedBy= sys-subsystem-net-devices-%i.device
```

There is a particular issue to address. When *starting* kernel bonding, where the only working interface is the *non*\-primary slave - for instance, starting with only a wireless interface available when the wired interface is the primary - then dhclient will quickly start and adopt the MAC address of the initial primary slave, and use that MAC address when attempting to communicate with the DHCP server. When the wireless interface, some short time later, is authenticated, associated, and authorized with the access point/base station, establishing a connection to the network, the bonding driver will make the wireless interface the new active interface, and change the active MAC address on the bond0 interface, to match the wireless MAC address. Because dhclient will continue to use the MAC address from the wired interface, and that MAC address is no longer accepted by the bond0 interface, all DHCP communication will fail. If there is no saved lease file in /var/lib/dhclient/dhclient.leases, then no IPv4 address will be configured, and no IPv4 traffic will be possible. It can also be seen that when dhclient starts quickly, it can read the primary slave's firmware MAC address, rather than any MAC address assigned to the device interface. If the firmware MAC address is "null", then dhclient assigns a random MAC address. BOOTP/DHCP packets using these firmware or random MAC addresses may "succeed" in gaining a reply on the primary slave device and fail on the non-primary slave device. That can be confusing and annoying.

These are only issues with dhclient and IPv4. Fortunately, on a dhclient DHCP request, after the lease expires, dhclient "does the right thing". dhclient will function properly no matter on which slave interface it was started.

This problem **cannot** be solved by configuring the bonding driver with the default `fail_over_mac=none`. Almost all network interface devices will not pass traffic with a MAC address which is not their own. An example of this kind of warning can be seen here. Strange network behavior will be the result, where broadcast packets will pass, but ping/icmp packets will only pass in some circumstances and not others.

Ideally, dhclient would re-determine the bonding interface MAC address each time it initially retried contacting the DHCP server. Without that, a different approach is to simply delay the start of dhclient until after the kernel bonding driver has configured an active slave. If the active slave is to be the wireless interface, then wpa_supplicant will first have authenticated, associated, and authorized with the access point/base station, and dhclient will adopt the correct MAC address. If the active slave is the primary slave, again dhclient will adopt the correct MAC address.

This delay is imposed with the simple `ExecStartPre= /usr/bin/sleep 8` line in the dhclient service unit file, a conservatively long delay between the time systemd starts dhclient and the supplicant and the bonding driver selects the active interface. This selection time is longest during system boot, when many processes are starting. On faster hardware, a shorter delay, perhaps `sleep 4`, may still be effective.

DHCPv6 配置

```
/etc/wide-dhcpv6/dhcp6c.conf
profile default {                       # generic interface using a profile named
"default"
        send ia-na 1;                   # a corresponding identity association
statement must exist with the same ID.
#       send ia-pd 2;
        send rapid-commit;
#       send authentication auth-protocol;
        request domain-name-servers, domain-name, ntp-servers;
#       script "/etc/wide-dhcpv6/dhcp6c-script";
};

id-assoc na 1 {                         # Identity Association for Non-temporary
Addresses request
                                        # Optionally request a specific address.
#       address <ipv6-address> <pltime> [<vltime>];
};

keyinfo kame-key {                      # Required to use the dhcp6ctl tool.
        realm "kame.net";
        keyid 1;
        secret "ErEJ0GEz7HS3uBU03/hKZA==";
                                        # Generate with "sudo sh -c 'openssl rand -
base64 16 > /etc/wide-dhcpv6/dhcp6cctlkey'".
                                        # Copy the key from /etc/wide-
dhcpv6/dhcp6cctlkey
};
```

A simple example configuration file, with no Prefix Delegation being requested. Here, the "Profile" statement is used instead of the "Interface" statement, since the same configuration is to be applied to whatever interface is provided on the dhcp6c command line. A keyfile is required to allow dhcp6ctl to communicate with dhcp6c.

```
/etc/systemd/system/dhcp6c@.service
[Unit]
Description= WIDE-DHCPv6 dhcp6c on interface %I
Documentation= man:dhcp6c(8) man:dhcp6c.conf(5) man:dhcp6ctl(8)

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network.target
Before= network.target
After= network-pre.target

BindsTo= sys-subsystem-net-devices-%i.device

[Service]
ExecStart= /usr/bin/dhcp6c -f -P default -c /etc/wide-dhcpv6/dhcp6c.conf %I

# Configure keyinfo in /etc/wide-dhcpv6/dhcp6c.conf
ExecReload= /usr/bin/dhcp6ctl reload
ExecReload= /usr/bin/dhcp6ctl start interface %I

ExecStop= /usr/bin/dhcp6ctl stop
ExecStop= /usr/bin/sleep 1

Restart= on-abnormal
```

```
[Install]
WantedBy= sys-subsystem-net-devices-%i.device
```

静态网络配置

```
/etc/conf.d/network.conf
# These Environment names are formed by prefixing the base variable name with the
interface names.
# Remember, here, these are systemd.service Environment variables, not shell variables.
# Variables that are not set or that are set to the empty string will have no effect.
# Add a list of interface names here, in the form of a comment, for reference.
# wlp2s0 enp3s0 bond0

# PRIMARY is the name of the preferred network interface in the bonded group of
interfaces.
bond0PRIMARY='enp3s0'

# ADDRS is a single-quoted space separated list of IPv4 and IPv6 addresses to apply to
the interface.
# The ADDRESS must be followed by a slash and a decimal number which encodes the
network prefix length.
# interfaceADDRS='address1/length address2/length ...'
bond0ADDRS='192.168.0.2/24'
wlp2s0ADDRS=
enp3s0ADDRS=

# ROUTES is a single-quoted space separated list of double-quoted ip-route ROUTE
specifications.
# A PREFIX must be followed by a slash and a decimal number which encodes the network
prefix length.
# Remember, to be able to add a route, the host must first be able to reach the
gateway.
# interfaceROUTES='"to prefix1/length] via gateway1" "to prefix2/length via
gateway2" ...'
bond0ROUTES=
```

Here, for instance, a static private IPv4 address will be assigned to the bonding interface as a "fail-safe", were the DHCP server to fail or be otherwise inaccessible. The primary slave interface is also specified in this file.

```
/etc/systemd/system/static@.service
[Unit]
Description= Static Network Configuration on %I
Documentation= man:ip-address(8) man:ip-route(8) man:systemd.service(5)

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network.target
Before= network.target

BindsTo= sys-subsystem-net-devices-%i.device

[Service]
EnvironmentFile= /etc/conf.d/network.conf

Type= oneshot
RemainAfterExit= yes

# Apparently, "ip" is not synchronous/atomic, so allow some time.
ExecStart=-/usr/bin/ip link set %I up
ExecStart=-/usr/bin/sh -c '[ "$%IADDRS" ] && \
        for a in $%IADDRS;do /usr/bin/ip address add local $$a dev %I;done'
ExecStart= /usr/bin/sleep 1
ExecStart=-/usr/bin/sh -c '[ "$%IROUTES" ] && \
        for r in ${%IROUTES};do /usr/bin/ip route replace $$r dev %I;done'

ExecStop=-/usr/bin/sh -c '[ "$%IROUTES" ] && \
        for r in ${%IROUTES};do /usr/bin/ip route del $$r dev %I;done'
```

```
ExecStop= /usr/bin/sleep 1
ExecStop=-/usr/bin/sh -c '[ "$%IADDRS" ] && \
        for a in $%IADDRS;do /usr/bin/ip address del local $$a dev %I;done'

[Install]
WantedBy= sys-subsystem-net-devices-%i.device
```

Of course, static network configuration may be used as an alternative to, or in addition to, dynamic network configuration, or not at all.

wpa_supplicant 配置

```
/etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=wheel
update_config=1
eapol_version=2
ap_scan=1
# fast_reauth=1
country=US

network={
        ssid="MyHome"
        priority=2
        proto=RSN
        group=CCMP
        pairwise=CCMP
        key_mgmt=WPA-PSK
        #psk="SuperSecret"
        psk=404fe69d94ef522ba8e7a0c456a67a583c8f39ba0b29a3ac22ebe9494cf9992b
}
```

Be careful with the actual protocol configuration in the wpa_supplicant configuration file. Using protocols incompatible with the base station can result in unstable and otherwise difficult to troubleshoot wireless connections. Pre-compute the PSK with `wpa_passphrase ssid passphrase`. `wpa_gui` can overwrite this file. Note that `wpa_supplicant` can be run on any wired or wireless interface, as needed.

```
/etc/systemd/system/supplicant@.service
[Unit]
Description= wpa_supplicant on %P
Documentation= man:wpa_supplicant(8) man:wpa_cli(8) man:wpa_supplicant.conf(5)
man:wpa_passphrase(8)

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network-pre.target
Before= network-pre.target

BindsTo= sys-subsystem-net-devices-%i.device

[Service]
# Disable legacy 802.11b bitrates.
ExecStartPre=-/usr/bin/iw %I set bitrates legacy-2.4  6 9 12 18 24 36 48 54

ExecStart= /usr/bin/wpa_supplicant -c/etc/wpa_supplicant/wpa_supplicant.conf -
Dnl80211,wext,wired -i %I
ExecStartPost=-/usr/bin/sh -c "/usr/bin/iw phy phy`iw dev %I info|grep wiphy|cut -d' '
-f2` set distance 10"

ExecReload= /usr/bin/wpa_cli -i %I reconfigure
ExecReload= /usr/bin/wpa_cli -i %I reassociate

ExecStop= /usr/bin/wpa_cli -i %I terminate

# Reset bitrates.
ExecStopPost=-/usr/bin/iw %I set bitrates
```

```
Restart= on-abnormal

[Install]
WantedBy= sys-subsystem-net-devices-%i.device
```

The supplicants and the DHCP and DHCPv6 clients are ordered relative to the network-pre.target on shutdown. The supplicants must not be stopped before the DHCP and DHCPv6 clients release their address leases.

Remember that the `iw` commands do not work with the wired interface drivers or with older wireless drivers which rely upon the Wireless Extensions user-space driver, and will be ignored in those cases.

从属设备配置

```
/etc/systemd/system/slave@.service
[Unit]
Description= %P@%I Slave
Documentation= https://www.kernel.org/doc/html/latest/networking/bonding.html

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network.target
Before= network.target

Requires= master@%i.service
After= master@%i.service

Before= dhclient@%i.service
Before= dhcp6c@%i.service
Before= static@%i.service
Before= supplicant@%p.service

BindsTo= sys-subsystem-net-devices-%p.device
BindsTo= sys-subsystem-net-devices-%i.device

[Service]
Type= oneshot
RemainAfterExit= yes

ExecStart=\
 /usr/bin/ip link set %P down ;\
 /usr/bin/ip address flush dev %P ;\
 /usr/bin/ip link set %P master %I ;\
 /usr/bin/ip link set %P up

ExecStop=\
-/usr/bin/ip link set %P nomaster ;\
-/usr/bin/ip link set %P up

[Install]
WantedBy= master@%i.service
WantedBy= sys-subsystem-net-devices-%p.device
```

There is a "trick" which will be used here, in the naming of the slave service template unit files. Two environment variables are to be passed to the slave unit files, the name of the network interface, and the name of the bonding interface. Notice that there are two particular environment variables passed into a systemd unit file, %p/%P and %i/%I, these being the strings before and after the "@" character in the name of a template unit file. Here, the bonding interface name is specified in that portion of the unit file name after the "@" character, and the network interface name is passed in that portion before the "@" character. This allows two network interface

names to be specified arbitrarily on the command line, without modifying the unit files themselves.

This "slave@.service" unit file will be *hard* linked to files having the same name as the network interfaces, such as "wlp2s0@.service" and "enp3s0@.service". Note that symbolic links cannot be used here, since systemd would then set %p/%P to the target file name "slave", instead of the desired network interface name.

主设备配置

```
/etc/modprobe.d/bonding.conf
# The primary slave will be configured from the systemd master unit file.

options bonding max_bonds=0 miimon=100 mode=active-backup fail_over_mac=active
/etc/systemd/system/master@.service
[Unit]
Description= %I Interface Master
Documentation= https://www.kernel.org/doc/html/latest/networking/bonding.html

Documentation= https://www.freedesktop.org/wiki/Software/systemd/NetworkTarget/
Wants= network.target
Before= network.target

BindsTo= sys-subsystem-net-devices-%i.device

[Service]
# Environment= PRI=enp3s0
# Environment= PRI=wlp2s0
EnvironmentFile=-/etc/conf.d/network.conf

Type= oneshot
RemainAfterExit= yes

# Apparently, "ip" is not synchronous/atomic, so allow some time.
ExecStart=\
-/usr/bin/sh -c ' case %I in \
 *br*) /usr/bin/ip link add name %I type bridge ;; \
    *) /usr/bin/ip link add name %I type bond ; \
       echo -n $%IPRIMARY > /sys/devices/virtual/net/%I/bonding/primary ;; \
 esac' ;\
 /usr/bin/ip link set %I up ;\
 /usr/bin/sleep 1

ExecStop=\
 /usr/bin/ip link delete %I ;\
 /usr/bin/sleep 1

[Install]
RequiredBy= dhclient@%i.service
RequiredBy= dhcp6c@%i.service
RequiredBy= static@%i.service
```

Of course, "Environment=" could be used here instead of the Environment file, if static network configuration is not used, and then the Environment file could be avoided. Settings from Environment files override settings made with "Environment=".

This master service unit file supports creation of a bonding master or a bridging master network interface. The type of master interface created is determined by the name of the interface. A bridging master is created when the interface name includes the character string "br", and a bonding master is created otherwise.

The RequiredBy dependencies are only here to activate the stop ordering of static or dynamic network configuration units during master stop and restart. The network configurations must be taken-down and that process completed before the slave interfaces are freed and the master interface is deleted.

Enable/Install a bonding master unit or bridging master unit *only* when the master interface is also an IP interface for the host, which is to say, when there is a static or dynamic network configuration unit Enabled/Installed on that master interface. If the bonding master or bridging master is *not* also an IP interface, then the master service unit should not be Enabled/Installed, since it will be started manually, or will be started by the slave service units, on boot, or when a network device is plugged.

启用/安装服务单元

With those preliminaries, the interface names must be specified on the command line.

Whenever a unit file is edited, afterward run:

```
# systemctl daemon-reload
```

Next, observe the available network interface names, after inserting any removable devices:

```
# ip address
```

For each interface which will be enslaved, hard link "slave@.service" to "*interface_name*@.service":

```
# ln /etc/systemd/system/slave@.service /etc/systemd/system/enp3s0@.service
# ln /etc/systemd/system/slave@.service /etc/systemd/system/wlp2s0@.service
```

Now, determine which network interface devices will need a supplicant to access the network. Typically this will just be the wireless interface. Enable/Install and start the supplicant@.service unit for each interface, as needed:

```
# systemctl --now enable supplicant@wlp2s0
```

Then, Enable/Install the slave and master units, using any desired interface name. Here, "bond0" is used:

```
# systemctl enable enp3s0@bond0 wlp2s0@bond0 master@bond0
```

Explicitly Enable/Install only the desired network configuration, specifying the interface name, here again, bond0:

```
# systemctl enable dhclient@bond0 dhcp6c@bond0 static@bond0
```

And finally, activate the bonding interface, the DHCP and DHCPv6 clients, and any static network configuration, by starting "master@bond0.service":

```
# systemctl start master@bond0
```

The master and supplicant units will be started automatically when any configured slave device appears, and in particular, when the system boots. Were any of the DHCP, DHCPv6, or slave units to be started independently, the master unit would also be started, but normally these units will have already been started at boot.

测试成果

Check the results:

```
# journalctl -afn100
$ ip a
$ ps wax
$ systemctl list-units '*bond*' '*master*' '*supplicant*'
UNIT                                    LOAD   ACTIVE   SUB     DESCRIPTION
sys-devices-virtual-net-bond0.device    loaded active
plugged /sys/devices/virtual/net/bond0
sys-subsystem-net-devices-bond0.device loaded active
plugged /sys/subsystem/net/devices/bond0
dhclient@bond0.service                  loaded active   running ISC dhclient on
interface bond0
dhcp6c@bond0.service                    loaded active   running WIDE-DHCPv6 dhcp6c on
interface bond0
enp3s0@bond0.service                    loaded active   exited  enp3s0@bond0 Slave
master@bond0.service                    loaded active   exited  bond0 Interface Master
static@bond0.service                    loaded active   exited  Static Network
Configuration on bond0
supplicant@enp3s0.service               loaded inactive dead    wpa_supplicant on enp3s0
supplicant@wlp2s0.service               loaded active   running wpa_supplicant on wlp2s0
wlp2s0@bond0.service                    loaded active   exited  wlp2s0@bond0 Slave
system-master.slice                     loaded active   active  system-master.slice
system-supplicant.slice                 loaded active   active  system-supplicant.slice

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.

12 loaded units listed.
To show all installed unit files use 'systemctl list-unit-files'.
```

Using the wired ethernet interface,

```
$ cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v3.7.1 (April 27, 2011)

Bonding Mode: fault-tolerance (active-backup) (fail_over_mac active)
Primary Slave: enp3s0 (primary_reselect always)
Currently Active Slave: enp3s0
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0

Slave Interface: wlp2s0
MII Status: up
Speed: Unknown
Duplex: Unknown
Link Failure Count: 0
Permanent HW addr: 68:a3:c4:ac:63:d1
Slave queue ID: 0

Slave Interface: enp3s0
MII Status: up
Speed: 100 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: e8:9a:8f:2a:9e:e1
Slave queue ID: 0
```

Using the wireless interface,

```
$ cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v3.7.1 (April 27, 2011)

Bonding Mode: fault-tolerance (active-backup) (fail_over_mac active)
Primary Slave: enp3s0 (primary_reselect always)
```

```
Currently Active Slave: wlp2s0
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0

Slave Interface: wlp2s0
MII Status: up
Speed: Unknown
Duplex: Unknown
Link Failure Count: 0
Permanent HW addr: 68:a3:c4:ac:63:d1
Slave queue ID: 0

Slave Interface: enp3s0
MII Status: down
Speed: Unknown
Duplex: Unknown
Link Failure Count: 0
Permanent HW addr: e8:9a:8f:2a:9e:e1
Slave queue ID: 0
```

To tear-down the bonding interface and shutdown the master, slave, DHCP, and DHCPv6 units, simply run:

```
# systemctl stop master@bond0
```

The supplicant units can be stopped independently with

```
# systemctl stop supplicant@interface_name
```

This approach to bonded wireless networking leaves wpa_supplicant running continuously on whatever interfaces it is started. By running htop包, it can be seen that wpa_supplicant, and the DHCP and DHCPv6 client daemons, seem to behave well, and do not use any noticeable CPU time.

Still, a hardware switch or rfkill can be used to actually disable the radio when desired.

Notice that the various service units are quite independent except for the ordering dependencies that have been explicitly configured. So, for instance, a dhclient configured IPv4 address may be removed without disturbing any other network configuration or functionality with

```
# systemctl stop dhclient@bond0.service
```

Similarly, an address may be released and a new address acquired with

```
# systemctl restart dhclient@bond0.service
```

And a static address or default gateway may be changed by stopping the static service unit:

```
# systemctl stop static@bond0.service
```

editing the network.conf file, and then starting the static service unit again:

```
# systemctl start static@bond0.service
```

Also, wpa_supplicant could be temporarily disabled when only the wired interface is being used, and then started again later.

This bonding interface will function properly even with only one interface available, for instance, when only a wired interface is being used. And then, simply inserting a configured wireless network card, this new wireless interface will be automatically

added to the bonded interface pool, and wpa_supplicant started. Removing this wireless card again will remove the slave interface and stop wpa_supplicant.

Check that the Ethernet cable is actually plugged-in when wired networking is preferred. And use, for instance, `wpa_cli status` or `iwconfig` to verify a connection to the correct Service Set Identifier/SSID when wireless networking is used.

12.5 网络配置/无线网络配置

难度 高级 | **篇幅** 56.8 KB | **所属** 无线网络

本节将讲到

- 设备驱动
- 工具
- iw
- 认证
- 提示和技巧

网络配置的主页面是网络配置。

配置无线网络一般分两步：第一步是识别硬件、安装正确的驱动程序并进行配置，安装盘中已经包含驱动，但是通常需要额外安装；第二步是选择一种管理无线连接的方式。这篇文章涵盖了这两方面，并提供了无线管理工具的链接地址。

#iw 段落描述了如何使用 iw包 来手动管理你的网络接口/无线局域网连接。网络配置#网络管理器段落介绍了几个可以自动管理您无线网络接口的程序。这些程序中的某些会包含一个图形界面，而所有的程序都支持网络配置文件（在频繁切换网络时有用，比如使用笔记本电脑的场合）。

设备驱动

默认的 Arch Linux 内核是*模块化*的，硬件的设备驱动作为内核模块保存在硬盘上。启动时，udev 会根据硬件加载不同的驱动模块，从而允许创建网络接口。

除了相应的驱动程序之外，某些无线芯片组还需要固件。linux-firmware包 提供了很多固件，然而专有固件并不包含在内，需要另行安装。这部分指南请参考 #安装 driver/firmware。

注意：如果 udev 在启动时未加载正确的模块，只需 手动加载 即可。 如果 udev 为一台设备加载多个驱动程序，则产生的冲突可能会导致配置失败。 确保将不需要的模块加入 黑名单。

检查驱动状态

根据设备的连接方式是 PCI(e) 或 USB，对应地执行 `lspci -k` 或 `lsusb -v` 可以检查设备驱动是否已经加载：

```
$ lspci -k
06:00.0 Network controller: Intel Corporation WiFi Link 5100
       Subsystem: Intel Corporation WiFi Link 5100 AGN
       Kernel driver in use: iwlwifi
       Kernel modules: iwlwifi
```

注意：如果网卡是 USB 设备，以 root 权限执行 `grep usbcore` 的输出应当类似于 `usbcore: registered new interface driver rtl8187`。

If you do not see the card at all, it may not be seated/plugged in properly in its PCI(e) slot/USB port. Try re-plugging it in.

同时也要通过 `ip link` 命令来查看无线接口是否已经创建。通常来说，设备名以字母“w”开头，例如`wlan0` 或 `wlp2s1`。确认之后，使用以下命令使设备上线：

```
# ip link set <设备名> up
```

例如，接口设备名是 `wlan0`，此命令就是 `ip link set wlan0 up`。

注意：

- 如果你从命令中得到如下错误：`RTNETLINK answers: Operation not possible due to RF-kill` 错误，请确保设备没有被 软锁定 或 硬锁定，详情请参考 Rfkill 说明。
- 如果你从命令中得到如下错误：`SIOCSIFFLAGS: No such file or directory`，那么大概率您的无线芯片组需要额外的固件来运行。

检查内核信息中关于已加载固件的信息：

```
# dmesg | grep firmware
[    7.148259] iwlwifi 0000:02:00.0: loaded firmware version 39.30.4.1 build 35138
op_mode iwldvm
```

如果没有相关的输出，那么需要在消息中查找关于您之前指定的模块的完整输出（本示例中的模块是`iwlwifi`），然后查找对应的错误信息：

```
# dmesg | grep iwlwifi
[    12.342694] iwlwifi 0000:02:00.0: irq 44 for MSI/MSI-X
[    12.353466] iwlwifi 0000:02:00.0: loaded firmware version 39.31.5.1 build 35138
op_mode iwldvm
[    12.430317] iwlwifi 0000:02:00.0: CONFIG_IWLWIFI_DEBUG disabled
...
[    12.430341] iwlwifi 0000:02:00.0: Detected Intel(R) Corporation WiFi Link 5100 AGN,
REV=0x6B
```

如果需要的内核模块已经加载且网络接口已经上线，您可以跳过下一节的内容。

安装驱动程序/固件

根据获得的信息，在下面网址查找硬件支持：

- 参阅表格 Linux 支持的无线驱动并转到特定驱动程序页面，其中包含受支持设备的列表。

- Ubuntu Wiki 维护了一个设备被内核和用户空间驱动支持状况的列表。
- Linux 无线支持页面和硬件兼容性列表(HCL)也维护了一个内核友好的设备列表。

注意有些厂商的产品名称相同，实际使用的芯片组却不同。usb-id (USB设备) 或 pci-id (PCI设备)才是唯一识别名称。

如果您在上述列表里找到了您的无线网卡，请按照驱动与固件排错中的指示操作，这一节中包含有关安装某些特定无线网卡所需的驱动程序和固件的信息。完成后，再次检查驱动程序状态。

如果您的上述列表里*没有*找到您的无线网卡，那么大概率您的网卡仅在windows操作系统下受支持。对于这些设备，您可以尝试使用 #ndiswrapper。

工具

同其他网络设备一样，无线网络接口可以通过 iproute2包 包提供的 *ip* 命令来管理。

管理一个无线网络连接需要一套基本的工具。可以使用一个网络管理器或直接使用以下表格中的一个：

软件	软件包	WEXT	nl80211	WEP	WPA/WPA2/WPA3	Archiso
wireless_tools[1]	wireless_tools包	是	否	是	否	是
iw	iw包	否	是	是	否	是
wpa_supplicant	wpa_supplicant包	是	是	否	是	是
iwd	iwd包	否	是	否	是	是

1. 过时，不推荐使用.
2. 一些老驱动只支持WEXT

iw 与 wireless_tools 的对比

下面表格给出了 `iw` 和 `wireless_tools` 命令的对比（更多示例参阅这里）。

***iw* 命令**	***wireless_tools* 命令**	**描述**
iw dev *wlan0* link	iwconfig *wlan0*	获取连接状态。
iw dev *wlan0* scan	iwlist *wlan0* scan	扫描可用接入点。
iw dev *wlan0* set type ibss	iwconfig *wlan0* mode ad-hoc	设置操作模式为 *ad-hoc*。
iw dev *wlan0* connect *your_essid*	iwconfig *wlan0* essid *your_essid*	连接到开放网络。
iw dev *wlan0* connect *your_essid* 2432	iwconfig *wlan0* essid *your_essid* freq 2432M	连接到开放网络的特定频道。
iw dev *wlan0* connect *your_essid* key 0:*your_key*	iwconfig *wlan0* essid *your_essid* key *your_key*	用16进制密匙连接到 WEP 加密网络。
iw dev *wlan0* connect *your_essid* key 0:*your_key*	iwconfig *wlan0* essid *your_essid* key s:*your_key*	用 ASCII 密匙连接到 WEP 加密网络。
iw dev *wlan0* set power_save on	iwconfig *wlan0* power on	启用省电功能。

iw

注意：

- 多数命令需要使用 root权限 执行。 使用普通用户执行某些命令,如 *iw list*，不会输出任何错误但并不会产生正确的输出，造成疑惑。
- 根据您硬件或加密类型的不同，某些步骤并非必要。在连接至某个接入点及获取IP地址之前，某些网卡需要首先被 激活 与/或 扫描热点。这可能需要您自己尝试。例如，WPA/WPA2 用户可能会尝试直接从步骤 连接到接入点 开始来激活无线网络。

本章节的示例会假定您的无线接口设备名称是 `*interface*`, 您想要连接到的WiFi接入点名称是 `*your_essid*`。请将这两个字段作相应替换。

获取接口名

提示：更多关于 *iw* 工具的示例，请查看 官方文档 。

获取您无线网卡的名称，运行：

```
$ iw dev
```

接口的名称会在 "Interface" 单词后显示。比如，常见的无线网卡设备名是 `wlan0`。

检查连接状态

用下面的命令检查连接状态:

```
$ iw dev interface link
```

获取统计数据

获取一些统计数据，比如上传下载的数据量，信号强度等：

```
$ iw dev interface station dump
```

激活接口

提示：通常不需要执行此步骤。

一些无线网卡在使用 `iw` 或 `wireless_tools`前需要先激活内核接口：

```
# ip link set interface up
```

注意：如果出现错误 `RTNETLINK answers: Operation not possible due to RF-kill`, 请确保硬件开关已经打开。参阅 #Rfkill 说明。

如果出现错误 `RTNETLINK answers: Operation not possible due to RF-kill`, 请确保硬件开关已经打开。参阅 #Rfkill 说明。

要验证接口确实打开：

```
$ ip link show interface
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state DOWN mode DORMANT
group default qlen 1000
    link/ether 12:34:56:78:9a:bc brd ff:ff:ff:ff:ff:ff
```

`<BROADCAST,MULTICAST,UP,LOWER_UP>` 中的`UP` 显示接口已经打开，不要和后面的 `state DOWN` 混淆。

扫描接入点

查看可用的接入点：

```
# iw dev interface scan | less
```

注意：如果显示 "Interface doesn't support scanning"，可能是忘了安装固件。有时不以 root 运行 `iwlist` 也会产生这个问题。无线网络也可能被软锁定，请运行 `rfkill list all` 进行检查。

提示：Depending on your location, you might need to set the correct regulatory domain in order to see all available networks.

需要关注的信息:

- **SSID:** 网络的名称.
- **Signal:** 用 dbm (-100 to 0) 报告的无线信号强度。数值越接近零，信号越好。观察高质量连接和低质量连接的数值差异可以了解设备的信号范围。
- **Security:** 没有直接报告, 检查 `capability` 开头的行，如果有 `Privacy` 信息，例如 `capability: ESS Privacy ShortSlotTime (0x0411)`, 表示网络具有某种程度的保护，
- 如果有 `RSN` 信息，网络被 Robust Security Network(WPA2) 协议保护。
- 如果有 `WPA` 信息，网络被 Wi-Fi Protected Access 协议保护。
- 在 `RSN` 和 `WPA` 信息块中，可能看到如下信息：
- **Group cipher:** 数值包括 TKIP, CCMP, both, others.
- **Pairwise ciphers:** 数值包括 TKIP, CCMP, both, others. 可能和 Group cipher 数值不同.
- **Authentication suites:** 数值包括 PSK, 802.1x, others. 家用路由器通常可以看到 PSK (*i.e.* 密码). 在大学中，通常会链接到需要登录名和密码的 802.1x 网络。需要知道其使用的密码管理方式(例如 EAP), 封装方法 (例如 PEAP). 详情请参考这里和这里.
- 如果没有看到 `RSN` 或 `WPA`，但是看到了 `Privacy`, 表示使用的是 WEP.

运行模式

设置无线网卡的操作模式，如果连接到漫游网络，需要设置操作模式为 **ibss**

```
# iw dev interface set type ibss
```

注意：有些网卡需要先关闭无线接口(`ip link set *interface* down`)才能修改模式。

连接到接入点

根据加密方式不同，需要使用密码将无线设备关联到接入点。

假设要使用的接入点 ESSID 为 `MyEssid`:

无加密

```
# iw dev interface connect "your_essid"
```

WEP

使用十六进制或 ASCII 密码(格式是自动识别出来的，因为 WEP 密码长度是固定的):

```
# iw dev interface connect "your_essid" key 0:your_key
```

使用十六进制或 ASCII 密码，第三个是默认 (从0计数，共四个):

```
# iw dev interface connect "your_essid" key d:2:your_key
```

其它

- - *iw* 只能处理 WEP。要使用其他加密方案登录, 请查看下文中的 #认证章节。

无论您使用什么认证方法，都可以使用以下命令来确认连接是否成功：

```
# iw dev interface link
```

认证

WPA2 个人模式

WPA2 个人（模式）, 又名 WPA2-PSK, 是一种 受保护Wi-Fi访问（Wi-Fi存取保护）的模式。

您可以通过 wpa_supplicant 或 iwd 来连接, 或者直接使用网络管理器. 如果您只进行了认证，那么要获取功能齐全的连接，您仍然需要进行手动连接或使用 DHCP 客户端。

WPA2 企业模式

*WPA2 企业模式*也是一种 受保护Wi-Fi访问（Wi-Fi存取保护）的模式，提供了比 WPA2 个人版更好的安全性和密钥管理，并支持其他的企业型功能，例如 VLAN 和 NAP。然而，与个人模式不同，此种加密需要额外的 RADIUS 认证服务器来处理用户的认证。个人模式不需要除无线路由器/接入点以外的设备，对所有用户也只要求一个密码或密码短语。

企业模式允许用户使用用户名和密码或者数字证书来登录 Wi-Fi。因为每个用户都拥有一个动态唯一的加密密码，因此企业模式可以防止通过无限网络的用户探听，并提高加密强度。

本章节描述如何使用网络管理器来连接到使用 WPA2 企业模式的无线接入点。

查看软件接入点#RADIUS 来获取关于*设置接入点自身*的相关信息。

注意：企业级加密模式需要更复杂的客户端配置，个人模式则只需要在出现提示时输入密码。客户端可能需要安装服务器的 签名证书(CA certificate)，如果使用 EAP-TLS, 则另需要为每个用户安装证书，再手动配置无线安全和 802.1X 认证设置。

协议间的对照可以参照此表格。

警告：您可以让客户端在使用WPA2企业级时不检查服务器的授权证书。然而，您在任何情况下都该尽最大努力来配置证书，因为如果不验证接入点，连接可能遭受中间人攻击。虽然连接握手可能是加密的，但最常见的密码传输配置会将密码用明文或易于破解的MS-CHAPv2协议协议进行传输。因此，客户端可能会将密码发送到代理连接的恶意接入点。

MS-CHAPv2

有时，除了原配的 ppp包 (Point-to-Point Protocol) 之外，要求 MSCHAPv2 type-2 with PEAP 的 WPA2 企业级接入点有时会需求 pptpclient包。netctl包 似乎能在缺少 ppp-mppe 的情况下开箱即用。虽然其他认证实现方式通常都并不可用，由于其高度脆弱的特性，用户在任何情况下都不被鼓励使用 MSCHAPv2。

eduroam

eduroam 是一项面向科研，高等教育与继续教育领域人员的国际漫游服务。

注意：* 在应用本章节列出的任何配置之前，**首先**向您的机构确认连接的详细信息。示例配置不保证任何安全性或者匹配任何安全需求。

- 存储未加密的连接配置文件时，建议用 root 用户运行以下命令以限制 root 帐户对此文件的的读取访问权限: `chmod 600 *配置文件*`

提示：针对 NetworkManager 的配置可以用 eduroam 配置助手工具 来生成。 此工具需要安装 python包 与 dbus-python包 。

手动/自动配置

- wpa_supplicant 可以直接通过其配置文件或使用其 CLI/GUI 前端进行配置，并与 DHCP 客户端结合使用。 请参阅`/usr/share/doc/wpa_supplicant/wpa_supplicant.conf` 中的示例来结合您的详细连接信息。
- iwd#WPA Enterprise
- NetworkManager 可以使用 *nmcli* 或 graphical front ends 来创建 WPA2 企业级配置文件。*nmtui* 不支持此功能 (NetworkManager issue 376), 但可以使用现有配置。
- ConnMan 需要在连接至网络之前配置一个单独的配置文件。详情请参阅 connman-service.config(5) 和 ConnMan#连接到 eduroam (802.1X))。
- netctl 可以通过填写配置中的`WPAConfigSection=` 字段来支持 wpa_supplicant 配置文件。详情请参阅 netctl.profile(5)。

: > **注意**：特殊引用规则适用: 参考 netctl.profile(5) § SPECIAL QUOTING RULES. : > **提示**：自定义证书可通过在 `WPAConfigSection` 中添加字段 `'ca_cert="/path/to/special/certificate.cer"'` 来配置。

WPA3 个人

WPA3 Personal, 又名 WPA3-SAE, 是一种 受保护Wi-Fi访问（Wi-Fi存取保护）的模式。

wpa_supplicant 及 iwd 都支持 WPA3 个人认证模式。

WPA3 企业模式

WPA3 企业模式 是一种 受保护Wi-Fi访问（Wi-Fi存取保护）的模式。

wpa_supplicant (自版本起 2:2.10-8) 支持 WPA3 企业模式。 参见 FS#65314。

提示和技巧

遵循监管区域要求

监管区域用于配置无线驱动，确保对无线硬件的使用符合由 FCC、ETSI 及其他组织制定的当地法规。监管区域使用ISO 3166-1二位字母国家代码。例如，美国的监管区域代码是“US”，中国则是“CN”，等等。

监管区域影响无线信道的可用性。在2.4GHz波段，美国允许使用1-11信道，日本是1-14，世界上多数其它地区则是1-13。在5GHz波段，允许使用信道的规则要复杂得多。这两种情况的更详细信息见这一WLAN信道列表。

监管区域也影响对来自无线设备的等效各向同性辐射功率（EIRP）所作的限制。这衍生自发射功率（tx power）并以dBm/mBm（1dBm=100mBm）或mW（对数尺度）衡量。在2.4GHz波段，美国和加拿大最大限制30dBm，欧洲大部分是20dBm，世界上其他地区则是20dB-30dBm。在5GHz波段，最大值通常更低。更详细的信息请参考 wireless-regdb（EIRP dBm 值在每行第二对括号里）。

错误设置监管区域可能会有用——例如，在其他信道拥挤时可以使用未用信道，或者提升发射功率以扩大传输范围。然而**不推荐这种做法**，因为可能会违反当地法规以及造成与其他无线电设备的干扰。

要配置监管区域，安装 crda包\[损坏的链接：replaced by wireless-regdb包\] 并重启（以重新加载 `cfg80211` 模块及所有相关驱动）。检查启动日志以确认 `cfg80211` 正在调用CRDA：

```
# dmesg | grep cfg80211
```

以下命令可将当前监管区域设为美国：

```
# iw reg set US
```

用以下命令查询：

```
$ iw reg get
```

注意：

- 你的设备国家代码可能为“00”，它是包含了通用设置的“世界监管区域”。如果无法解除此设置，则CRDA可能配置有误。
- 根据 CRDA 的 README，从内核版本4.15开始就不再需要它，因为内核会自动从固件加载监管数据库。然而也有说法称这一加载操作需要特定的内核设置。

然而设置监管区域可能不会改变您的设置。有些设备在固件/EEPROM中设置了一个监管区域，为设备规定了限制，这意味着通过软件设置监管区域只能增加限制而不能减少。例如，一个 CN（中国）设备可以用软件设置为 US（美国）监管区域，但由于中国的 EIRP 最大值为20dBm，设备将无法以美国的最大值30dBm进行发射。

例如，查看一个 Atheros 设备的固件中是否设置了监管区域：

```
# dmesg | grep ath:
```

对于其他芯片组，搜索“EEPROM”、“regdomain”或设备驱动的名字可能会有用。

要查看监管区域更改是否成功，以及查询可用信道数和允许发射功率：

```
$ iw list | grep -A 15 Frequencies:
```

监管区域的持久设置可以通过编辑 `/etc/conf.d/wireless-regdom` 并解除相应区域的注释实现。 `wpa_supplicant` 也可以在 `/etc/wpa_supplicant/wpa_supplicant.conf` 中的 `country=` 行应用监管区域。

也可以通过添加模块参数（如 `options cfg80211 ieee80211_regdom=EU`），配置 cfg80211 内核模块，从而使用指定的监管区域。然而这是一种老旧的监管实现。

更多信息请阅读 wireless.kernel.org 监管文档。

Rfkill 说明

许多笔记本都有一个硬件按钮或开关用于关闭无线网卡，然而无线网卡也可以通过内核来屏蔽。这一任务可由 *rfkill* 处理。显示当前状态：

```
$ rfkill
ID TYPE      DEVICE      SOFT      HARD
 0 bluetooth hci0    unblocked unblocked
 1 wlan      phy0    unblocked unblocked
```

如果网卡处于*硬件屏蔽（hard-blocked）*状态，请使用硬件按钮或开关来开启它。 如果无线网卡并没有被*硬件屏蔽*但处于*软件屏蔽（soft-blocked）*，请使用以下命令：

```
# rfkill unblock wifi
```

注意：按下硬件按钮可能会让无线网卡从硬件屏蔽，软件未屏蔽状态变为硬件未屏蔽，软件屏蔽状态。可以通过调整 `rfkill` 内核模块的一些选项改变这一现象。

切换无线网卡的硬件按钮是厂商专用的内核模块处理的，经常是 WMI 模块。特别是非常新的硬件型号，可能还没有得到最新稳定内核的完全支持。这种情况下，在内核 bug 跟踪系统搜索信息，并将型号报告给对应厂商内核模块的维护者（如果还没有报告的话）经常会有帮助。

参见 https://askubuntu.com/questions/62166/siocsifflags-operation-not-possible-due-to-rf-kill 。u

节能

见电源管理#网络接口。

排错

这个单元包含一般性的错误排除提示, 目的并非是排除与设备或固件驱动有关的错误。对于驱动与固件上的问题, 请看 #驱动与固件排错.

临时上网

如果您的网络设备出了问题但需要网络访问，例如， 下载一些软件或向社区寻求帮助， 您可以利用安卓的内置功能来通过USB端口共享网络。 详见 Android Tethering#USB tethering。

观察日志文件

分析系统日志文件是排错的良好切入点。在进行操作（如尝试连接到无线网络）的同时，打开第二个终端/控制台窗口并用以下命令监看内核消息：

```
# dmesg -w
```

这样，您就不需要手动一行一行的浏览这些消息了。

当您使用网络管理器时，同样的，可以用 systemd 提供的日志命令：

```
# journalctl -f
```

通常来说，无线网络错误都伴随着具有特定理由代码（reason code）的取消验证（deauthentication）。例如：

```
wlan0: deauthenticating from XX:XX:XX:XX:XX:XX by local choice (reason=3)
```

对照原因代码表可能会给出解决问题的第一个提示。

获取IP地址失败

**本文或本章节的事实准确性存在争议。**

原因： The dhcpcd包 vs dhclient包 is mostly FUD.（在 Talk:网络配置/无线网络配置 中讨论）

- 如果用 dhcpcd包 客户端获取ip地址反复多次均失败, 尝试安装并使用 dhclient 包 。不要忘记在连接管理器中选择 dhclient 作为首选DHCP客户端
- 如果你使用有线接口可以获得IP地址而无线接口不能， 尝试禁用无线网卡的省电功能(指定 `off` 而非 `on`).
- 如果由于等待网络连接而导致超时错误，可能要将特定网卡设备的信道模式改为自动: `auto`

```
# iwconfig wlan0 channel auto
```

在将wifi信道更改为自动之前，请确保无线接口已关闭。成功地更改后，再次打开无线接口并继续。

IP地址有效但无法接解析主机名

如果你正在使用公共无线网络，可能会有认证门户, 确保在浏览器查询认证页面时使用HTTP 而不是HTTPS, 因为有些认证门户会重定向到HTTP. 如果不是这个导致的问题, 检查你是否可以解析域名, 可能有必要使用通过DHCP方式得到的DNS服务器。

设置 RTS 和静态帧分段阈值

无线网卡默认禁用了 RTS 和 分段机制 。这两种方法都以牺牲带宽来换取吞吐量 (i.e. 以速度为代价提高可靠性)。在由于无线干扰和有多个相邻接入点，从而导致连接超时或失败的环境中，这两个设置非常有用。

数据包分段通过将超过阈值的包分割，从而提高吞吐量。最大值（2346）实际上禁用了分段，因为没有任何包可以超过这个值，最小值（256）最大化了吞吐量，但带宽成本很会大。

```
# iw phy0 set frag 512
```

RTS 通过在与接入点传输超过RTS阈值的分组之前来进行握手，从而提高吞吐量。最大值（2347）实际上禁用了RTS，没有分组可以超过这个值。最小值（0）意味着所有分组都启用RTS，但这很极端。

```
# iw phy0 set rts 500
```

注意：`phy0` 是用`iw phy` 命令 列举出的无线网卡的名称

随时可能断开连接

Cause #1

如果你在日志里看到：`wlan0: deauthenticating from MAC by local choice (reason=3)` 并且 Wi-Fi 断开连接, 很可能是你的无线网卡的电源节能设置太激进。尝试禁用无线网卡的省电功能。

如果你的网卡不支持 启用/禁用 节能模式, 看看 BIOS 的电源管理选项。 比如，在 联想 W520 的BIOS中禁用PCI-Express电源管理就解决了这个问题。

Cause #2

如果你经常断连，并且在日志看到如下信息：

```
ieee80211 phy0: wlan0: No probe response from AP
xx:xx:xx:xx:xx:xx after 500ms, disconnecting
```

在你路由器设置面板里把频道带宽改成 `20MHz` 。

Cause #3

有些笔记本有硬件 rfkill 开关（如Thinkpad X200系列），因为磨损或有问题，开关（或与主板的连接）可能会随时间的推移而变松，不小心触摸开关或移动电脑时就可能出现断连。这问题没有软件解决方案，除非开关是电子式的，BIOS提供了禁用开关的选项。如果开关是机械式的（大部分机型都是），就有很多解决方案，其中大多数都是要禁用开关：在主板/无线网卡上的接触点进行焊接，用胶水封堵或阻挡，使用螺丝来拧紧开关，或者直接拆除开关。

Cause #4

导致随时断连或完全无法连接的其他原因，比如路由器质量差，配置不完整，其他无线设备干扰，或信号质量差。

为了排错, 首先要靠近路由器，并且在不需要身份验证的情况下进行连接

如果有效，那重新启用 WPA/WPA2 ，但是选择固定/有限的路由器配置，比如：

- 如果路由器比你用的无线设备客户端老的多，把路由器设置为单一无线模式进行测试。
- 禁用混合验证模式 ，(e.g. 只启用 WPA2 with AES, o如果是老路由器，则是 TKIP)
- 使用固定频道而不是自动频道 (因为附近的路由器可能老旧而且存在干扰)
- 禁用 WPS
- 把路由器的 5 GHz 频道切换为 non-DFS (Dynamic Frequency Selection) 频道. 在这种频道的连接突然被丢弃或者切换，这是因为附近天气雷达的干扰。
- 只使用2.4 GHz ，而不是自动在 5 GHz 和2.4 GHz 切换。 (后者吞吐量低，但是远距离连接更稳定)
- 禁用 `40MHz` 频道带宽 (吞吐量低，但冲突更少) ，用 `cfg80211.cfg80211_disable_40mhz_24ghz=1`
- 如果路由器有 Qos功能, 检查设置的完整性。(e.g. Wi-Fi 多媒体 (WMM) 是 QoS 流控制中可选的功能， 损坏的路由器固件可能会表明他已经存在，即使没有启用这个功能)

Cause #5

对于一些无线网卡适配器 (e.g. Qualcomm Atheros AR9485), 随时断连可能会伴随一个 DMA 报错：

```
# journalctl -xb
ath: phy0: DMA failed to stop in 10 ms AR_CR=0x00000024 AR_DIAG_SW=0x02000020
DMADBG_7=0x0000a400
wlp1s0: authenticate with 56:e7:ee:7b:55:bc
wlp1s0: send auth to 56:e7:ee:7b:55:bc (try 1/3)
wlp1s0: send auth to 56:e7:ee:7b:55:bc (try 2/3)
wlp1s0: send auth to 56:e7:ee:7b:55:bc (try 3/3)
wlp1s0: authentication with 56:e7:ee:7b:55:bc timed out
```

可行的方案是禁用 Intel IOMMU driver (DMA), 把`intel_iommu=off` 加入 kernel parameters .

注意：Intel IOMMU 驱动对于一些高级虚拟机功能是必需的，如PCI直通。

Cause #6

如果你用的是带 `iwlwifi` 或 `iwlmvm` 的设备进行无线连接，使用电池电源的时候，无线网卡似乎消失了，（可能是重启或从休眠中恢复后），可以通过 iwlmvm 里的节能模式设置修复。

创建 `/etc/modprobe.d/iwlmvm.conf`， 再添加以下行：

```
/etc/modprobe.d/iwlmvm.conf
options iwlmvm power_scheme=1
```

`power_scheme` 为 1 表明把 iwlmvm 设置为*总是活跃*。可选项如下：

值	描述
1	总是活跃
2	平衡
3	节能

这个修复方法是在 被找到的。

Cause #7

若设备长期不使用 (e.g. 文件服务器), 可能会因为节能模式导致无法连接，节能模式会阻止流量传入和建立连接。尝试为接口禁用节能模式:

```
# iw dev interface set power_save off
```

可以创建udev规则，让开机时自动执行它。参阅 Power management#Network interfaces.

Cause #8

如果你连接 Mesh 网络时 (e.g., WiFi6) ，会偶尔中断连接，并且你看到了如下日志信息：

```
# journalctl -b
kernel: wlan0: disconnect from AP aa:bb:cc:dd:ee:ff for new auth to 11:22:33:44:55:66
```

你遇到的是漫游问题，可采取以下措施：

- 如果不需要漫游，在 NetworkManager中锁定BSSID (如上面的 `aa:bb:cc:dd:ee:ff`) (参阅 NetworkManager#Regular network disconnects, latency and lost packets (WiFi)).
- 在 Wpa_supplicant#Roaming 中调整 `bgscan`

C

Cause #9

如果你使用Intel WiFi 7 BE200网卡，并且WiFi在启动时完全正常，但暂停/恢复后，找不到任何网络。

问题可能在于，在新版 Linux 内核中，驱动程序或设备本身会声明其具备将设备设置为低功耗 D3cold 睡眠状态的功能，而这种状态本身存在问题。 解决方案是指示 udev 配置该卡，使其不使用该状态。

```
$ /etc/udev/rules.d/99-disable-d3cold-be200.rules
$ ACTION=="add", SUBSYSTEM=="pci", ATTR{vendor}=="0x8086", ATTR{device}=="0x272b",
ATTR{d3cold_allowed}="0"
```

Wi-Fi 由于错误的信道设置而变得不可见

若电脑 Wi-Fi 信道不符合用户所处国家之规定，部分Wi-Fi即使处于信号覆盖范围内，也可能因信道默认不被允许而不可见。参阅 #Respecting the regulatory domain.

驱动与固件排错

这一部分包含了关于特定芯片组安装内核模块与固件的方法与规范，不同于一般的方法。

参阅内核模块了解通用的模块操作方法。

Ralink

rt2x00

Ralink 芯片组的统一驱动，代替了 `rt2500`、`rt61`、`rt73` 等。Linux 内核从 2.6.24 开始包含此驱动，但是有些设备可能需要额外固件。可以使用标准 `wpa_supplicant`包 和 `iwconfig` 工具配置。有些芯片组需要固件文件，可以安装软件包 linux-firmware包。

参见: Using the new rt2x00 beta driver。

- 从内核 3.0 开始，rt2x00 也包含了这些驱动程序：

`rt2800pci`、`rt2800usb`。 `rt2860sta` 被主分支驱动 `rt2800pci` 替代，`rt2870sta` 被 `rt2800usb` 替代。

- 通过 `iwpriv` 可以配置很多参数，文档在 Ralink 源代码包中。

rt3090

对于使用 rt3090 芯片组的设备, 应该使用的驱动是`rt2800pci` ; 然而， 它并没有非常好的适配这个芯片组 (例如：有时候网速甚至不能到 2Mb/s).

rt3290

rt3290 芯片组是由内核模块 `rt2800pci` 识别的。一些用户遇到这些问题，他们选择恢复到打了补丁的 Ralink 驱动，这样效果更好。

rt3573

2012年新出的芯片组，需要 Ralink 的闭源驱动，有不同的厂商使用他们，参阅 Belkin N750 示例。

mt7612u

2014年新发布的芯片组，以新的商用名称联发科发布。这是一款 AC1200 或 AC1300 级别的芯片组。制造商提供了 Linux 驱动程序在支持页面。从内核 5.5 版本开始，该芯片组应该已被包含的 `mt76` 驱动所支持。

Realtek

参见 \[失效链接 2022-11-10\] 查看 Realtek 芯片组和规范列表

rtl8192cu

这个驱动已经被包含在内核里了, 但很多用户报告指出即使扫描到了 wifi 也无法连接。

8192cu-dkmsAUR 包含了很多补丁；如果内核里的驱动并没有很好的起效果可以尝试这个。

rtl8723ae/rtl8723be

`rtl8723ae` 和 `rtl8723be` 模块已经被包含在 Linux 内核主线。

一些用户使用此网卡时可能会遇到省电方面的问题. 这表现为可能偶尔断开连接，但高级网络管理器无法识别 (netctl、NetworkManager)。以 root 身份执行 `dmesg -w` 可以验证这个错误 ，或以 root 身份执行 `journalctl -f` 查找与 powersave 和 `rtl8723ae`/`rtl8723be`

模块相关的输出. 如果你遇到了这个问题，使用 `fwlps=0` 内核选项，这样可以防止无线网卡自动休眠或停止连接。参阅内核模块#配置模块参数。

如果你信号很差，可能你只有一个物理天线连接了，并且天线的自动选择损坏了。可以用 `ant_sel=1` 或 `ant_sel=2` 内核选项强制选择天线。

rtl88xxau

Realtek 芯片组 rtl8811au、rtl8812au、rtl8814au 和 rtl8821au 被设计为用于各种 USB 适配器，范围从 AC600 到 AC1900。有几个包提供对应的内核驱动，需要 DKMS 和内核头文件：

C	P	N
rtl8811au、rtl8812au、rtl8821au	rtl88xxau-aircrack-dkms-gitAUR	Aircrack-ng 给 8811au、8812au、8821au 提供了监控模式和注入支持。
rtl8812au	rtl8812au-dkms-gitAUR	Realtek 最新官方驱动版本，仅适用于 rtl8812au。
rtl8811au、rtl8821au	rtl8821au-dkms-gitAUR	用于 rtl8821au 的新一代驱动版本
rtl8814au	rtl8814au-dkms-gitAUR	rtl8813au 可能也适用

rtl8811cu/rtl8821cu

rtl8821cu-dkms-gitAUR 提供了一个内核模块给 Realtek 8811cu 和 8821cu 芯片组。

需要 DKMS, 确保安装了内核头文件。

如果安装并加载了 `8821cu` 模块，无线接口还是没有显示，需要手动指定 `rtw_RFE_type` 选项。试试例如 `echo "options 8821cu rtw_RFE_type=0x26" | sudo tee /etc/modprobe.d/8821cu.conf`，其他值可能也可以，比如 1 或 2 等等。详见内核模块#配置模块参数

rtl8821ce

rtl8821ce-dkms-gitAUR 为华硕 X543UA 中的 Realtek 8821ce 芯片组提供了一个内核模块。

它需要 DKMS，所以应确保已安装内核头文件。

注意：有报告提出，在 Linux 内核版本 ≥ 5.9 中由 Realtek 提供的 `rtl8821ce` 无线模块是破损的，会导致连接弱，应该优先使用上面的 AUR 版本。详见在 github 上的讨论，使用 `lspci -k` 检查是否在用默认的 (`rtw88_8821ce`)模块。如果是，把它加入黑名单。

rtl8822bu

rtl88x2bu-dkms-gitAUR 给 Edimax EW7822ULC USB3、Asus AC53 Nano USB 802.11ac 和 TP-Link Archer T3U adapter 的 Realtek 8822bu 芯片组提供了内核模块。

确保安装了 DKMS 和内核头文件。

rtl8xxxu

可以通过安装 linux-firmware包 来获得 `rtl8xxxu` 的固件。

`rtl8xxxu` 驱动程序已在主线内核中。但是 linux-libre 内核不包含。

`rtl8xxxu` 主线内核模块的问题可通过编译特定芯片组的第三方模块来解决。源代码可在 GitHub 存储库中找到。 某些驱动程序可能已存在于在 AUR 中，例如 rtl8723bu-dkms-gitAUR。

提示：如果您的 USB 无线适配器基于 `RTL8188GU` 或 `RTL8192FU` 芯片，您可能需要安装 usb_modeswitch包 以将其切换至 **Wifi 模式**。

RTW88

一份 RTW88 内核模块补丁集最近被发到内核邮件列表中，有望被添加进内核主线。

如果正确配置和编译的话，上游内核以及带有补丁集的内核将会支持大部分 RTW88 芯片设备。linux-zen包 和 linux-zen-gitAUR 均包含了这些补丁，其中已打包的版本已经内置了该模块。

驱动支持型号有：882BE、8822BU、8822CE、8822CU、8723DE、8723DU、8821CE 与 8821CU。

Atheros

MadWifi 开发团队维护了三个模块：

- `madwifi` 是最老的驱动，Arch 内核 从 2.6.39.1 开始已经不再包含。
- C0 将逐步替代 `ath_pci`，有些芯片组使用效果很好，但有些还不能很好工作 (后面有说明)
- C0 是新的官方驱动，适用于新 Atheros 硬件。

还有一些驱动适用于其他的 Atheros 设备. 详见 Linux Wireless 文档。

ath5k

参考:

- https://wireless.wiki.kernel.org/en/users/drivers/ath5k
- http://wiki.debian.org/ath5k

若网页有时加载缓慢或设备无法获取 ip 地址，尝试从硬件加密切换为软件加密，使用 `nohwcrypt=1` 内核选项加载 `ath5k` 模块。详见内核模块#配置模块参数

有些笔记本的 LED 指示灯有问题，可以：

```
echo none > "/sys/class/leds/ath5k-phy0::tx/trigger"
echo none > "/sys/class/leds/ath5k-phy0::rx/trigger"
```

ath9k

外部文档资源:

- https://wireless.wiki.kernel.org/en/users/drivers/ath9k
- https://wiki.debian.org/ath9k

截至 Linux 3.15.1，一些用户可能会遇到网络带宽下降的问题，这种情况可以为 `ath9k` 模块设置 `nohwcrypt=1` 的内核选项。 详见内核模块#配置模块参数.

注意：使用命令 `lsmod` 查看正在使用的模块，如果名称不同（例如 `ath9k_htc`），请更改 `ath9k`。

节能/省电模式

虽然 Linux 无线工具表明对于比 AR9280 更新的 Atheros ath9k 单芯片设备已经启用了动态电源节能功能，虽然 Linux Wireless 指出比 AR9280 更新的 Atheros ath9k 单芯片设备已启用动态节能功能，但一些设备仍 (例如 AR9285) powertop包 被指出省电被禁用，这种情况需手动开启。

一些设备 (例如 AR9285)，启用省电功能可能会有如下报错：

```
# iw dev wlan0 set power_save on
command failed: Operation not supported (-95)
```

解决方法：为 `ath9k` 模块设置 `ps_enable=1` 内核选项，详见内核模块#配置模块参数.

Intel

iwlegacy

iwlegacy 是 Intel 3945 和 4965 网卡的无线驱动。linux-firmware包 已经包含此驱动。

udev 会自动加载驱动，否则要手动加载 `iwl3945` 或 `iwl4965`。详见内核模块。

如果连不上网或者连接很弱，尝试启用 802.11n：

```
/etc/modprobe.d/iwl4965.conf
options iwl4965 11n_disable=1
```

若启动过程中一直失败，且你正在使用 Nouveau 驱动, 尝试启用早期 KMS 避免冲突 。

iwlwifi

iwlwifi 是适用于 Intel 当前网卡的无线驱动，例如 5100AGN、5300AGN、和 5350AGN。查看所有受支持的文件列表。该固件已包含在 linux-firmware包 里。linux-firmware-iwlwifi-gitAUR 可能包含更新的驱动。

若联网存在问题或连接很弱，尝试启用 802.11n，也可能需要启用软件加密：

```
/etc/modprobe.d/iwlwifi.conf
options iwlwifi 11n_disable=1 swcrypto=1
```

在 802.11n 模式下网络上行速度很慢，比如只有 20Mbps，尝试启用 TX AMPDU：

```
/etc/modprobe.d/iwlwifi.conf
options iwlwifi 11n_disable=8
```

若上行速度很慢，可以禁用无线网卡的省电模式。

如果你有 802.11ax (WiFi 6) 接入点并且也有连接不稳定等问题，可参阅 Intel Article 54799。

注意：用 `11n_disable=0` 参数将会阻止 802.11ac，只能使用较慢的协议，（5GHz 频段用 802.11a，2.4 GHz 频段用802.11b/g）。

蓝牙共存

若无法连接蓝牙耳机或无法保持良好的数据接受速度，请禁用蓝牙共存，参考：

```
/etc/modprobe.d/iwlwifi.conf
options iwlwifi bt_coex_active=0
```

注意：从内核版本 5.8 起, 对于由 `iwlmvm` 内核模块处理的硬件， `bt_coex_active` 和 `sw_crypto` 模块选项已被禁用，但对于由 `iwldvm` 模块处理的老硬件，这些选项依旧起作用。

固件问题

你可能会遇到驱动程序输出堆栈跟踪与报错，这会导致有点卡顿。

```
# dmesg
Microcode SW error detected.  Restarting 0x2000000.
```

你也可能遇到其他各种各样的问题（例如连接 5GHz 频段时的问题，随时断连，恢复后无法连接）。

要想确认是否是固件的原因，请降级 linux-firmware包 包。

如果确定是固件的问题, 移除有 bug 的固件文件以确保可以加载旧版本固件 (要更新 linux-firmware包 包，因为它不仅仅提供 Intel 无线网卡的固件更新):

```
# for i in {64..73} ; do mv /usr/lib/firmware/iwlwifi-ty-a0-gf-a0-
$i.ucode.xz /usr/lib/firmware/iwlwifi-ty-a0-gf-a0-$i.ucode.xz.bak ; done
```

为了避免每次更新后都需要手动重复这些步骤, 可以在 `pacman.conf` 里用 `NoExtract` 数组搭配通配符以此禁止它们的安装。详见 pacman#在安装时跳过文件。

从 Windows 重启后无法检测到 WiFi 适配器

如果从 Windows 重启到 Linux 后，检测不到 WiFi 适配器，可能是由于 Windows 默认启用的快速启动功能。尝试禁用快速启动。iwlwifi 内核驱动维基页面有关于此问题的条目。

禁用 LED 闪烁

注意：适用于 `iwlegacy` 和 `iwlwifi` 驱动

默认设置中 LED 闪烁是开启的，有些人不喜欢，可以用 systemd-tmpfiles 禁止闪烁：

```
/etc/tmpfiles.d/phy0-led.conf
w /sys/class/leds/phy0-led/trigger - - - - phy0radio
```

运行 `systemd-tmpfiles --create phy0-led.conf` 使更改生效，或重启

查看此 LED 所有可选的触发值：

```
# cat /sys/class/leds/phy0-led/trigger
```

提示：如果你没有 `/sys/class/leds/phy0-led`，尝试使用 `led_mode="1"` 模块参数。它应当同时适用于 `iwlwifi` 和 `iwlegacy` 驱动。

```
# cat /sys/class/leds/phy0-led/trigger
```

Broadcom

参见博通无线网卡。

其他驱动/设备

Tenda w322u

该型号 Tenda 卡 可被视为 `rt2870sta` 设备。参阅 #rt2x00。

orinoco

这应当是内核的一部分，是已经被安装的。

部分 Orinoco 芯片组属于 Hermes 二代系列。应使用 `wlags49_h2_cs` 驱动而非 `orinoco_cs` 以获得WPA 支持。 使用此驱动前需先把 `orinoco_cs` 加入黑名单。

prism54

`p54` 已经包含在内核中，不过必须从此网站下载适合你网卡的驱动并且把它安装进`/usr/lib/firmware` 目录。

过时的 `prism54` 和新内核模块 `p54pci` 或 `p54usb` 同时装入造成冲突，使用 `lsmod | grep prism54` 查看是否装入了过时模块，如果是，那么就屏蔽 `prism54` 并根据上面方法修改固件名称。

zd1211rw

zd1211rw 是 ZyDAS ZD1211 802.11b/g USB WLAN 芯片的驱动，最近的版本的内核已经包括了。有被支持的设备列表。需要安装固件zd1211-firmwareAUR。

hostap_cs

Host AP 是一个 Linux 驱动程序，对于基于 Intersil 的 Prism2/2.5/3 芯片组的无线局域网卡。该驱动程序包含在 Linux 内核中。

注意：确保把 `orinico_cs` 驱动程序加入黑名单。

ACX100/111

警告：适用于这些设备的驱动 已损坏且并不适用于更新的内核版本。

软件包: `tiacx`、`tiacx-firmware` (已被官方仓库 和 AUR 移除)。

详见官方维基。

ndiswrapper

Ndiswrapper并不是一个真正的驱动,但是如果你无法找到适合你的无线网卡驱动, 它就派上用场了.有的时候, 它是非常有用的.为了使用Ndiswrapper, 你需要 Windows 驱动中的 *.*inf 文件*(*.sys 文件应该和 *.info 在同一个目录中)。如果你需要从 `*.exe` 文件解压缩，你可以使用 cabextract包。

警告：确保使用合适架构(也就是 32/64 位)的驱动。

下面是安装 ndiswrapper 的几个步骤:

1\. 安装 ndiswrapper-dkms包 2. 安装驱动到 `/etc/ndiswrapper/*`

```
ndiswrapper -i filename.inf
```

3\. 列出所有的安装的驱动

```
ndiswrapper -l
```

4\. 配置文件写到 `/etc/modprobe.d/ndiswrapper.conf`

```
ndiswrapper -m
depmod -a
```

然后依照内核模块一文设置启动时加载这个模块。

```
modprobe ndiswrapper
iwconfig
```

如果正常的话, 你应该可以看到wlan0接口了。如果有问题的话, 你可以阅读: Ndiswrapper installation wiki\[失效链接 2020-08-04 ⓘ\]. ndiswrapper howto 和 ndiswrapper FAQ.

参见

- Linux Wireless 项目
- Aircrack-ng 驱动程序安装指南
- Wireless Device Database Wiki (由于原维基站点已关闭，该分叉站点由 wi-cat.ru 托管。目前有两个不太完整的版本： TechInfoDepot、deviwiki)

第 13 章　共 5 节　17 KB

网络应用

本章处理的是同一台 Arch 主机上的部署问题：把五个网络应用装起来，让装好的东西按预期跑下去。各节覆盖的深度差别很大，两节 Home Assistant 从安装、配置一路写到用法和排错，而 Ghost 只写到安装。

阅读顺序上，先读两节 Home Assistant 中的一节，把完整流程摸清，再看 Matomo 的安装与配置；Ghost 与 KiwiIRC 内容最薄，只覆盖安装（后者多一节运行），时间紧可以跳过。最容易踩的坑是把两节 Home Assistant 混为一谈——两者是并行的两套安装与排错路径，混着读容易对不上步骤。

13.1　Ghost

难度 入门 | **篇幅** 1.2 KB | **所属** 网络应用

本节将讲到

- 安装

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： Wordpress 等，参见应用程序列表/互联网#博客引擎 (在Talk:Ghost讨论)

Ghost 是一个免费的开源博客平台，使用 JavaScript 编写，并根据 MIT 许可证进行分发，旨在简化单个博客作者和在线出版物的在线发布过程。

安装

安装 ghost-webAUR\[损坏的链接：package not found\]，并修改 `/etc/webapps/ghost/config.js` 。然后启动 `ghost.service` 。 如果对它感到满意，请在系统引导时启用它以自动启动。

请访问 http://127.0.0.1:2368/ghost 进行最终配置。

13.2　KiwiIRC

难度 入门 | **篇幅** 1.4 KB | **所属** 网络应用

本节将讲到

- 安装
- 运行

KiwiIRC 是您可以享受的手工制作的IRC客户端。专为轻松自由使用而设计。

安装

安装 kiwiircAUR 包。

运行

Apache

创建 Apache 配置文件：

```
/etc/httpd/conf/extra/kiwiirc.conf
Alias /kiwiirc "/usr/share/webapps/kiwiirc"
<Directory "/usr/share/webapps/kiwiirc">
    AllowOverride All
    Options FollowSymlinks
    Require all granted
</Directory>
```

并将其包含在 `/etc/httpd/conf/httpd.conf`：

```
# kiwiirc configuration
Include conf/extra/kiwiirc.conf
```

更改 Apache 配置文件后，重新启动 `httpd.service`。

Lighttpd

配置 Lighttpd，确保 `mod_alias` 已启用

将 kiwiirc 的以下别名添加到配置中：

```
alias.url = ( "/kiwiirc" => "/usr/share/webapps/kiwiirc/")
```

13.3 Home Assistant Supervised

难度 入门 | **篇幅** 3.6 KB | **所属** 网络应用

本节将讲到

- 安装
- 配置
- 用法
- 疑难解答

Home Assistant 是一个开源的家庭自动化软件，它把本地控制和隐私放在首位。由全球的 DIY 爱好者和技术爱好者组成的社区提供支持。

Supervised 是一种安装方式，可提供更多功能，例如通过 UI 安装插件。

安装

安装 homeassistant-supervisedAUR 软件包。

并确保 AppArmor 已正确安装并配置。

注意：

- 从 3.0.0 版起，不再强制要求 CGroup v1，因此可删除早期版本所需的 内核参数 `systemd.unified_cgroup_hierarchy=false`。
- Supervised 安装方式已被上游弃用，将不再获得支持。详见 。

配置

Home Assistant Supervised 的配置文件保存在 `/var/lib/homeassistant/` 中。若不存在配置文件，启动时会写入默认配置。

若只想访问 Home Assistant 自身的配置文件，请前往 `/var/lib/homeassistant/homeassistant/`。

若早于 v3.0.0 安装

Supervised 的默认配置目录已由 `/usr/lib/hassio/` 更改为 `/var/lib/homeassistant/`。你可以继续使用旧配置。如需更改路径，请在配置文件 `/etc/hassio.json` 中设置 *data* 值。

用法

启动 Home Assistant，请 启动/启用 `hassio-apparmor.service` 与 `hassio-supervisor.service`。

第一次启动可能需要 20 分钟，因为将下载并安装所需的软件包。 。

默认情况下，Web 界面位于 `[http://localhost:8123](http://localhost:8123)`。

疑难解答

在进行任何操作前，请先确认 Supervisor 已连接且安装状态为良好。可通过访问 监控插件（地址为 `[http://localhost:4357](http://localhost:4357)`）进行检查。

Supervisor 未连接

1. 确认 `hassio-supervisor.service` 已启动/启用。
2. 确认 supervisor 容器正在运行：

:```

```
### Supervisor 已连接但安装状态不良好

1.  确认 [AppArmor](../../08_安全与加密/访问控制/AppArmor.md#显示当前状态) 已正确安装并配置。
2.  确认已启动/启用 `hassio-apparmor.service` 服务。
3.  查看 Supervisor 日志获取更多信息：
```

: ```

```
# docker logs -f hassio_supervisor
```

文件 /etc/hassio.json 不存在

请确保 `/etc/hassio.json` 配置文件存在。

若不存在，请创建该文件并填写以下示例内容：

```
{
    "supervisor": "ghcr.io/home-assistant/amd64-hassio-supervisor",
    "machine": "generic-x86-64",
    "data": "/var/lib/homeassistant"
}
```

13.4 Matomo

难度 入门 | **篇幅** 4.7 KB | **所属** 网络应用

本节将讲到

- 安装
- 配置

Matomo，前身为 Piwik，是一个开源网络分析工具，在 GNU 通用公共许可证 3 下获得许可。该软件是用 php 编写的，可以通过网络浏览器访问。该项目的核心理念是隐私，因为当使用第三方网站分析提供商时，网站所有者会将用户的数据泄露给他们，以便他们将其出售给广告商。

通过在目标网站上加载一些JavaScript，可以用一个运行实例分析多个网站。

安装

安装 matomoAUR 或 matomo-gitAUR 包。git 包已经为你配置了 php-fpm 守护程序。此外，它还为你下载并安装了最新的 GeoIP 数据库。默认情况下，Matomo 通过访客设定的浏览器语言来猜测他们的位置，这不是可靠的地理位置信息。

配置

php 配置

需要正确配置 php 才能使 Matomo 工作。

首先，启用 MySQL 支持，如 PHP#MySQL/MariaDB 中所述。通过编辑 `/etc/php/php.ini` 来做到这一点。通过删除前面的分号取消注释 `;extension=pdo_mysql` 和 `;extension=mysqli`。

通常，注释由前面的分号表示。

`;extension=iconv` 需要启用，并且 `;extension=gd` 对于 Matomo 是可选的。至少取消注释 iconv。

允许 Matomo 访问需要的文件

注意：此处的更改仅适用于 matomoAUR 包而不是 matomo-gitAUR 包，因为后者已经包含此文件。

由于7.4版以来对 `php-fpm.service` 的新限制，其中 `ProtectSystem` 被设置为阻止 Matomo 正常运行（无法安装插件、更改配置等），因此需要手动设置访问某些文件的权限。

下面的文件 `/etc/systemd/system/php-fpm.service.d/override_matomo.conf` 修复了这个问题，同时没有展示出更多非必要的内容，如果不希望这样做，仍然允许用户按照安装清单中的描述修改 ACL。

```
[Service]
ReadWritePaths = /usr/share/webapps/matomo/config
ReadWritePaths = /usr/share/webapps/matomo/matomo.js
ReadWritePaths = /usr/share/webapps/matomo/misc/user/
ReadWritePaths = /usr/share/webapps/matomo/plugins/
```

服务器设置 (nginx)

要启用 php 网站，请安装 php-fpm包 包和 start/enable `php-fpm.service`（参见 Nginx#PHP implementation）。

通过修改`/etc/nginx/nginx.conf`创建服务器。在 "http "上下文中添加以下模板。或者，看看 matomo的GitHub 说明。

```
include /etc/nginx/mime.types;

server
{
    index index.php;
    listen 443 ssl;
    listen [::]:443 ssl;
    root /usr/share/webapps/matomo/;
    server_name matomo.example.com;

    location ~ ^/(\.git/|config/|core/|lang/|tmp/)
    {
        return 403;
    }

    location ~ \.php$
    {
        try_files $uri =404;

        # FastCGI
        include fastcgi.conf;
        fastcgi_pass unix:/run/php-fpm/php-fpm.sock;
        fastcgi_index index.php;
    }

    location ~ \.(avi|css|eot|gif|htm|html|ico|jpg|js|json|mp3|mp4|ogg|png|svg|ttf|wav|
woff|woff2)$
    {
        try_files $uri =404;
    }

    location ~ ^/(libs/|misc/|node_modules/|plugins/|vendor/)
```

```
    {
        return 403;
    }
}
```

要使用加密，您可以从 letsencrypt 获取免费证书。请求并安装证书后，通过将以下代码添加到 "http" 或 "server" 上下文以使用它们：

```
include /etc/letsencrypt/options-ssl-nginx.conf;
ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
ssl_certificate_key /etc/letsencrypt/live/subdomain.domain.me/privkey.pem;
ssl_certificate /etc/letsencrypt/live/subdomain.domain.me/fullchain.pem;
```

通过启动/启用 `nginx.service` 运行 nginx 服务器。

注意：`mariadb.service` 以及 `php-fpm.service` 是必须的。

最后步骤

所有主要设置都完成了。在你的浏览器中调用你的 Matomo 网站，并完成次要安装指南——不外乎是检查所需的一切是否可用，并设置和编写你的配置文件。

13.5 Home Assistant

难度 进阶 | **篇幅** 5.9 KB | **所属** 网络应用

本节将讲到

- 安装
- 配置
- 用法
- 故障排查
- 参见

Home Assistant 是一个开源的家庭自动化软件，它把本地控制和隐私放在首位。由全球的 DIY 爱好者和技术爱好者组成的社区提供支持。

安装

安装 home-assistant包 包。

注意：Home Assistant Core 版本已被弃用，将不再由上游提供技术支持。参见

Home Assistant Supervised

参见 Home Assistant Supervised.

配置

配置文件存储在 `/var/lib/hass/` 中。如果不存在配置，则在启动时将写入默认配置。

用法

要启动 Home Assistant，请启动/启用 `home-assistant.service`。

第一次启动可能需要 20 分钟，因为将下载并安装所需的软件包。 您可以在日志中看到进度。

提示：使用 journalctl 跟踪首次更新的过程：

```
# journalctl -fu home-assistant
```

默认情况下，Web 界面位于 `[http://localhost:8123](http://localhost:8123)`。

备份还原功能在 Web 界面中不可用。如需还原备份，请以 hass 用户身份解压备份的 tar.gz 文件：

```
# tar --strip-components=1 -xzf homeassistant.tar.gz -C /var/lib/hass
```

使用 MariaDB

默认情况下，Home Assistant 的 recorder/history 集成使用 SQLite。使用 MariaDB 可显著提升性能（尤其在配置规模较大时）。Home Assistant 通过 SQLAlchemy 支持多种后端，例如 MySQL、MariaDB、PostgreSQL 或 MS SQL Server。此处仅介绍 MariaDB，其他后端请参考 recorder 集成文档。

如果尚未安装 MariaDB，请参考 MariaDB 了解安装详情。

安装 python-mysqlclient包 依赖包。

创建新数据库，下面示例为每个服务创建一个用户名，并授予该用户名前缀的所有数据库通配符访问权限：

```
$ mysql -u root -p
CREATE USER username@'localhost' IDENTIFIED BY some_pass;
CREATE DATABASE username;
GRANT ALL PRIVILEGES ON username.* TO username@'localhost';
FLUSH PRIVILEGES;
quit;
```

在配置文件中添加：

```
/var/lib/hass/configuration.yaml
recorder:
  db_url: !secret recorder_mariadb_url
```

在 secrets 文件中添加（若未使用 Unix socket，请参考 recorder 集成文档 了解其他 URL 格式）：

```
/var/lib/hass/secrets.yaml
recorder_mariadb_url: "mysql://USER:PASSWORD@localhost/DATABASE?
unix_socket=/var/run/mysqld/mysqld.sock&charset=utf8mb4"
```

最后，重启 `home-assistant.service` 服务。数据库将开始填充数据。更多信息请参考 recorder 集成文档。

使用 PostgreSQL

你需要在 hass 的虚拟环境中安装名为 *psycopg2* 的 Python 模块。

你需要为 systemd 添加一个 附加配置片段：

```
/etc/systemd/system/home-assistant.service.d/postgresql.conf
[Service]
ExecStartPre=/var/lib/hass/.venv/bin/python -m pip install psycopg2
```

这确保了 psycopg2 模块始终被安装。

创建角色并建立数据库：

```
[postgres]$ createuser homeassistant
[postgres]$ createdb homeassistant -O homeassistant
```

配置 Home Assistant，在配置文件中添加 PostgreSQL 数据库 URL：

```
/var/lib/hass/configuration.yaml
recorder:
  db_url: !secret recorder_postgresql_url
```

在 secrets 文件中添加数据库 URL：

```
/var/lib/hass/secrets.yaml
recorder_postgresql_url: "postgresql://homeassistant@127.0.0.1/homeassistant"
```

最后，重启 `home-assistant.service` 服务。数据库将开始填充数据。

故障排查

无法访问 USB 设备

在默认安装中，Home Assistant 可能没有足够权限访问某些设备（如 Zigbee USB 加密狗），因为这些设备受 `uucp` 系统组控制。

编辑 `home-assistant.service` 文件，在 `[Service]` 部分添加 `SupplementaryGroups=uucp`，使默认的 `hass` 用户拥有正确的权限组。

然后，执行 daemon-reload 并 重启 `home-assistant.service` 服务，即可正常运行。

日志中出现 ModuleNotFoundErrors

如果在系统软件包升级后 Home Assistant 出现异常，可能是系统软件包与 Home Assistant 管理的 Python 库之间不兼容导致的。要重置 Home Assistant 的 Python 库并重新安装，请删除 modules 目录：

```
# rm -r /var/lib/hass/deps/lib
```

然后 重启 `home-assistant.service` 服务。

如果首次重启后仍出现 ModuleNotFoundErrors，可能需要再次重启。Home Assistant 会检测到缺失的模块并自动重新安装。

参见

- Home Assistant 主页
- Home Assistant 文档

远程服务器

远程操作 Arch 机的图形界面，可选方案分几层：有的只共享已登录的桌面，有的另开虚拟会话，有的须经显示管理器。本章把几种做法分开交代。

建议先读《Remmina》，它是客户端侧的统一入口，装好就能对接多数服务端；再按需从《Xrdp》《X2Go》《TigerVNC》中挑一个，后者最全，虚拟会话、转发本地显示、XDMCP 三种方式都有，可作主参考；《X11vnc》可跳读，仅在临时共享桌面时翻看。最易踩的坑是混淆两种会话：转发本地显示与新建虚拟会话是两个桌面，来回切换时易误判成连不上。

14.1　Remmina

难度 入门 | **篇幅** 3.9 KB | **所属** 远程服务器

本节将讲到

- 安装
- 配置
- 使用
- 提示与技巧
- 参考链接

Remmina 是用 GTK 编写的远程桌面客户端，托管在 GitLab 上 。 它支持以下协议：SSH、VNC、RDP、NX、XDMCP、X2Go、HTTP（s）。

安装

安装软件包 remmina包。

- 要支持 VNC 请安装 libvncserver包 或 gtk-vnc包 软件包。
- 要支持 SPICE 请安装 spice-gtk包 软件包。
- 要支持 RDP 也请安装可选的 freerdp包 软件包。
- 要支持 X2Go 也请安装可选的 pyhoca-cliAUR 软件包。
- 密码存储依赖于 GNOME Keyring 或者 KDE Wallet。

注意：如果在安装完上述所需软件包后未在 Remmina 的下拉菜单里面找到对应协议（如 RDP），请确保先完全退出 Remmina：执行 `killall remmina`。当您重启 Remmina 后，对应协议应该就可以使用了。

配置

Remmina 在 `$XDG_CONFIG_HOME/remmina` 目录下存储设置和连接文件。

文件	用处
`remmina.perf`	应用设置

文件	用处
`*.remmina`	连接文件

全系统的默认文件来自 `/etc/xdg/remmina` ，在首次运行后会被复制到用户的配置目录下。

使用

Remmina 连接文件会被存起来用于连接您的服务器，文件名是自动生成的，但是您可以使用您喜欢的文件名手动创建您自己的连接文件。

连接文件可以通过 GUI 创建或者手动地在 `$XDG_CONFIG_HOME/remmina` 目录下放一个后缀为 `.remmina` 的文件。

要打开之前保存的连接配置您可以这样做：

```
$ remmina -c /path/to/example.remmina
```

使用 URI 快速连接

Remmina 支持基于 URI 的快速连接。

- RDP：`remmina -c rdp://*user@example.com*`
- 带有域名的 RDP：`remmina -c rdp://*DOMAIN\\user@example.com*`
- VNC：`remmina -c vnc://*user@example.com*`
- 带有搜索字符串的 VNC：`remmina -c vnc://*example.com?VncUsername=user*`
- SSH：`remmina -c ssh://*user@example.com*`
- SPICE：`remmina -c spice://*example.com*`

使用加密的密码

URI 可以被嵌入加密的密码：

- `rdp://*user:ENCRYPTED@server*`
- `vnc://*user:ENCRYPTED@server*`

使用以下命令生成加密的密码：

```
$ remmina --encrypt-password
```

使用命令行来更新连接文件

连接文件可以不需要交互界面就可以更新。 例如：

```
$ echo "mypassword" | remmina --update-profile $XDG_CONFIG_HOME/remmina/example.remmina
\
  --set-option username=user \
  --set-option resolution_mode=2 \
  --set-option password
```

提示与技巧

- 命令行用法请参阅 `remmina --help`。
- 对于高级设置，请参阅上游 Remmina 维基。

参考链接

- Remmina GitLab Repository
- https://gitlab.com/Remmina/Remmina/-/wikis/Home

14.2 X11vnc

难度 入门 | **篇幅** 4.4 KB | **所属** 远程服务器

本节将讲到

- 设置 x11vnc
- SSH端口转发
- 参见

**这篇文章或章节的翻译质量不佳。**

原因：Still out of sync after the update of 2021-05-26（在 Talk:X11vnc# 中讨论）

x11vnc是一个VNC服务器，它允许人们使用任何VNC viewer远程查看并控制真实的X显示器（即与物理显示器、键盘和鼠标相对应的显示器）。虽然它已不再由原作者Karl Runge开发，但LibVNC和GitHub社区已经接管了开发工作。

*x11vnc*不会为远程控制创建一个额外的显示（或X桌面）。相反，它实时显示现有的X11显示器，与Xvnc不同，它是TigerVNC的一部分，是官方软件仓库中的替代VNC服务器。

还要注意的是，x11vnc并没有附带一个客户端。所有VNC viewer都可以完成这项工作，而不必使用其所有功能。TigerVNC的*vncviewer*是推荐的客户端。

设置 x11vnc

安装

安装位于官方软件仓库的 x11vnc包 软件包。

运行

首先，通过*startx*或显示管理器来启动X 服务器。你可能需要将X设置为无头运行。

然后，运行以下命令，所有可用的选项在x11vnc(1)中都有解释。

```
$ x11vnc -display :0
```

另一个选项是将x11vnc命令放在一个脚本中，在登录时调用，例如：

```
x11vnc -wait 50 -noxdamage -passwd PASSWORD -display :0 -forever -o /var/log/x11vnc.log
-bg
```

设置 X authority

您可以为VNC服务器设置X authority。 这可以通过`-auth`参数和相应的文件来实现，这取决于你的X 服务器是如何启动的。一般来说，分配X authority需要以root身份运行"'x11vnc'"。

启动 X

```
$ x11vnc -display :0 -auth ~/.Xauthority
```

如果失败，则你可能需要作为root来运行：

```
# x11vnc -display :0 -autho /home/USER/.Xauthority
```

其中 *USER* 是运行X 服务器的用户的用户名。

GDM

作为root，运行

```
# x11vnc -display :0 -auth /var/lib/gdm/:0.Xauth
```

设置密码

运行:

```
$ x11vnc -usepw
```

将会使用在`~/.vnc/passwd`找到的密码，或是在`~/.vnc/passwdfile`第一行找到的密码，如果这些文件都找不到，它会提示用户输入密码，密码将会保存在`~/.vnc/passwd`中。

VNC viewer应该在连接时提示输入密码。

访问

在其他机器运行VNC客户端，然后输入运行了x11vnc服务器的IP地址。点击连接，然后你需要设置。

SSH端口转发

为了安全地使用`x11vnc`，您首先需要安装并且配置好SSH。

在启动`x11vnc`的时候，指定命令行选项“`-localhost`”，这将导致VNC服务只能绑定到本地网络界面。此时从外界直接连入的连接将被拒绝。

当您需要从另一台电脑上访问这个VNC服务的时候，首先用SSH登录到运行VNC的主机，将VNC服务监听的端口转发到您的本地主机。以下的例子中假设运行VNC的主机名为"foo"，VNC监听5900端口上：

```
ssh foo -L 5900:localhost:5900
```

SSH连接建立以后，打开VNC客户端程序，但是不要让它连接到foo的5900端口，而是连接到本机（localhost）的5900端口。

这样，您就可以通过加密渠道安全地访问远程X服务了。

参见

- original author site
- https://github.com/LibVNC/x11vnc
- Wikipedia:x11vnc

14.3 Xrdp

难度 进阶 | **篇幅** 8.7 KB | **所属** 远程服务器

本节将讲到

- 安装
- 使用说明
- 技巧和窍门
- 疑难解答
- 参阅

xrdp 是一个守护程序，支持 Microsoft 的 Remote Desktop Protocol (RDP)。 它使用 Xvnc 或 xorgxrdp 作为其后端。

安装

安装 xrdpAUR 软件包 (或是选择开发版本 xrdp-gitAUR)。这仅支持XVNC作为后端。

Xorg 后端

要使用 xorgxrdp 代替后端，安装 xorgxrdpAUR 包。

将 `allowed_users=anybody` 添加到 `/etc/X11/Xwrapper.config` 以允许任何人启动 X 服务器。

使用说明

首先，启动 `xrdp` 服务。您应该能够将 RDP 客户端连接到默认 RDP 端口 (3389) 上的主机。如果成功，您将看到“xrdp”会话管理器窗口，该窗口允许您在“Xorg”或“Xvnc”会话之间进行选择，并为用户身份验证提供输入。通过修改 `/etc/xrdp/xrdp.ini` 可以高度自定义会话管理器 UI。

用于启动 *Xorg* 和 *Xvnc* 显示服务器的参数可以在 `/etc/xrdp/sesman.ini` 中配置。

成功启动显示服务器后，*xrdp* 将默认执行 `/etc/xrdp/startwm.sh`。该脚本用于启动一个窗口管理器（类似于 .xinitrc），如果它们从 `~/.xinitrc` 或 `/etc/X11/xinit/xinitrc` 读取存在。建议编辑 `~/.xinitrc` 以启动您的桌面环境或窗口管理器，但您也可以编辑 `/etc/xrdp/startwm.sh`。

如果您只是关闭会话窗口和 RDP 连接，您可以在下次连接 RDP 时再次访问同一会话。当您从会话窗口退出窗口管理器或桌面环境时，会话将关闭，下次将打开一个新会话。

技巧和窍门

开机自动启动

xrdpAUR 软件包包含 systemd 的服务文件。启用 `xrdp.service`。

图形加速

对于 Xorg 会话，您可以通过为 Intel 和 AMD GPU 安装 xorgxrdp-glamorAUR 和为 Nvidia GPU 安装 xorgxrdp-nvidiaAUR 来启用 OpenGL 和 Vulkan 图形加速。

声卡

安装必要的 PulseAudio 与 pulseaudio-module-xrdpAUR 模块或者 PipeWire 与 pipewire-module-xrdpAUR。

非 root 用户

在 `Xwrapper.config` 中添加如下内容:

```
/etc/X11/Xwrapper.config
allowed_users=anybody
needs_root_rights=no
```

如果没有上面的内容，从 Remmina 等程序发起的远程连接将会出现空白屏幕。

疑难解答

X server could not be started并且日志中有waitforx字样

由于 waitforx 的测试版本不稳定，可以将 `/usr/lib/xrdp/waitforx`备份后，编辑新文件`waitforx`：

```
#!/bin/sh
exit 0
```

保存后设置775权限。或者安装 xrdp-gitAUR 包。

光标周围的黑框

如果您在鼠标指针周围遇到黑框，请使用 `Xcursor.core:1` 行创建 `~/.Xresources-xrdp` 并将其加载到 `~/.xinitrc` 中，例如

```
xrdb ~/.Xresources-xrdp
exec startlxde
```

您可能需要安装 xorg-xrdb包。

黑屏

如果您的 `~/.xinitrc` 在 `dbus_args` 中设置了 `--exit-with-session`，则登录会话管理器后可能会出现黑屏。

尝试将 `~/.xinitrc` 复制到 `~/.xrdpinitrc`，删除 `--exit-with-session`，然后更新 `/etc/xrdp/startwm.sh` 调用 `~/.xrdpinitrc` 而不是 `~/.xinitrc`。您可能需要将您的桌面环境添加到对 .xrdpinitrc 的调用中，如 ~/.xinitrc 中所述；例如 `. ~/.xrdpinitrc xfce`。

桌面环境的黑屏

如果使用桌面环境出现黑屏，则可能是 D-Bus 未正确初始化造成的。一些桌面环境 (例如 KDE Plasma) 可能会从之前的会话中恢复应用程序和窗口，所以结果是只缺失了 "plasmashell"。

尝试使用 `~/.xinitrc` 文件中的 `dbus-launch --exit-with-session` 运行桌面环境。对于 KDE Plasma，您可以使用命令 `/usr/lib/plasma-dbus-run-session-if-needed startplasma-x11` 或者 `dbus-launch --exit-with-session startplasma-x11`。

loginctl 或 systemctl --user 不工作

尝试注释掉 `/etc/pam.d/system-auth` 中对 `systemd-home` 的所有引用。请参阅这个问题。

gnome-keyring 或 KDE Wallet 提示

如果在会话开始时提示您登录 gnome-keyring 或 \[KDE Wallet\]，请修改文件 `/etc/pam.d/xrdp-sesman` 如下：

```
/etc/pam.d/xrdp-sesman
#%PAM-1.0

auth            include         system-remote-login
-auth           optional        pam_gnome_keyring.so
-auth   optional  pam_kwallet5.so

account         include         system-remote-login

password        include         system-remote-login
-password       optional        pam_gnome_keyring.so use_authtok

session         optional        pam_keyinit.so force revoke
session         include         system-remote-login
-session                optional        pam_gnome_keyring.so auto_start
-session  optional  pam_kwallet5.so auto_start
```

如果您只使用 gnome-keyring，则不需要包含 kwallet5 行，反之亦然。

阻止自动启动项目启动

要阻止用户定义的 `~/.config/autostart` 项目启动，您可以在 `~/.xinitrc` 中的会话上设置自动启动目录参数以仅使用全局 `/etc/xdg/autostart` 目录。

```
get_session(){
    local dbus_args=(--sh-syntax)
    case "$SESSION" in
        awesome) dbus_args+=(awesome) ;;
        bspwm) dbus_args+=(bspwm-session) ;;
        budgie) dbus_args+=(budgie-desktop) ;;
        cinnamon) dbus_args+=(cinnamon-session -a /etc/xdg/autostart) ;;
```

没有声音

这可能是 *loginctl* 问题的征兆，因此请尝试以以上内容修复。系统journal可能会遇到以下错误：

```
Failed to load module "module-x11-publish" (argument: "display=:10.0 xauthority="): initialization failed.
```

这是 *systemd* 不正确地启动 PulseAudio 的结果。一种解决方法是为您自己的用户或所有用户禁用 user unit 文件 `pulseaudio.service` 和 `pulseaudio.socket`，然后启动 PulseAudio需要时通过在 `/etc/pulse/client.conf` 中将 `autospawn` 设置为 `yes`。

如果仍然没有声音，请尝试使用 `~/.xinitrc` 中的 `pulseaudio &` 手动启动 PulseAudio。

两指滚动过快

使用 xorg 后端，确保 xorgxrdp 版本 0.9.19 或更新。然后添加 `XRDP_XORG_TOUCHPAD_SCROLL_HACK=yes` 到 `/etc/xrdp/sesman.ini` 中的 `[SessionVariables]` 段 。

参见此问题以获得详细信息。

远程系统的功能限制

远程用户的权限可能会比直接登录用户的权限要低。需要额外的 polkit 权限配置策略：

挂载的设备

要远程访问挂载的设备，请参考 udisks#Permissions.

NetworkManager

要远程访问网络管理器，请阅读 NetworkManager#Set up PolicyKit permissions。

参阅

- TigerVNC - VNC，一个 RDP 的替代品，同时在此用作后端
- freerdp包 一个支持 RDP 7.1 功能的 rdesktop 分支，包括网络级身份验证 (NLA)。它主要由 `xfreerdp` 客户端组成。此外，`freerdp-shadow-cli` 命令提供了一种快速简便的方法来启动 RDP 服务器。

1. ↑ https://github.com/neutrinolabs/xrdp/issues/2788

14.4 X2Go

难度 进阶 | **篇幅** 10.2 KB | **所属** 远程服务器

本节将讲到

- 安装
- 服务器端配置
- 客户端配置
- 排错
- 参阅

X2Go 使你可以通过网络访问一台计算机的图形化桌面。访问时的网络传输使用了 Secure Shell 协议，因而传输是加密的。

注意：X2Go 并不能兼容所有桌面环境。你可以先查阅 X2Go 桌面环境兼容性，尤其是当你要映射当前的桌面环境时。

安装

官方仓库中提供了下列两个软件包供安装：

- x2goserverAUR - X2Go 服务器
- x2goclientAUR - 基于 Qt5 的 X2Go 客户端

服务器端配置

配置 Secure Shell 守护进程

X2Go 使用 Secure Shell 才能工作，所以你首先需要配置 sshd 守护进程使其允许 X11 转发并启动它。参阅 OpenSSH - X11 转发和 OpenSSH - 管理 sshd 守护进程。

加载 fuse 内核模块

为使服务器端能够访问客户端计算机上的文件，你需要加载 `fuse` 内核模块。

你可以查看 `lsmod | grep fuse` 是否返回匹配项，若无，则需要加载 `fuse` 内核模块。

设置 SQLite 数据库

在服务器端执行下列命令以初始化 x2go 服务器运行所需的 SQLite 数据库：

```
# x2godbadmin --createdb
```

Control published applications

X2Go can publish the installed applications in a menu to the client. This is controlled by the files in `/etc/x2go/applications/`. This location however is not created by default and can be created by creating a symlink to `/usr/share/applications/`.

Alternatively instead of creating a symlink one could also create a folder and link only the desired applications instead. See for more information.

启动 X2Go 服务器端守护进程

至此，只需启动 `x2goserver.service` 即可。

客户端配置

运行 X2Go 客户端

```
$ x2goclient
```

注意：确保你可以打开一个从客户端到服务器的 ssh 会话 `ssh *username@host*`

现在，你可以创建多个会话，它们将出现在 X2Go 客户端程序主窗口的右面并且可以用鼠标点击选中。每一项都是由你的用户名、主机名、IP 地址和 SSH 连接端口组成。你还可以进一步基于不同连接速度（从 modem 到 LAN）以及想要远程启动的不同桌面环境定义若干个配置文件。

访问本地桌面

To access the local desktop, the one currently running on the server rather than a new one, one can choose the option "Connection to local desktop" in "session type" in the *X2Go Client* as long as the users match, if it is user *foo* accessing the session of user *foo*.

无论如何，要访问本地不同用户的桌面，你需要安装 x2godesktopsharingAUR 并加载 `x2godesktopsharing`。

客户端与服务器（桌面）交换数据

在 x2go 客户端（例如：笔记本电脑）上，本地目录可被共享。服务器端将通过 fuse 和 sshfs 访问这些目录并将其挂载到服务器端你的家目录下的若干子目录上。这样你就可以在服务器上访问你的笔记本电脑上的数据或者互传文件。也可以每次启动会话时自动挂载共享目录。

临时离开一个会话

X2Go 的另一个特色功能就是可以挂起一个会话。这意味着你可以从一个客户端里离开某一个会话，然后用另一个客户端重新打开这个会话（即便是同一地点的另一个客户端）。这样你可以在局域网启动一个会话，随后在笔记本电脑上重新打开这个会话。在此期间，会话数据由一个 SQLite 数据库存储和管理。会话的状态由一个名为 *x2gocleansessions* 的进程来保持。

排错

桌面环境不启动

本地会话阻止了 X2Go 的新会话

当本地已经有一个桌面会话在运行而 X2Go 尝试新建一个会话时失败。这是典型的 D-Bus 相关的问题，详情参阅 。 如果 D-Bus 启动失败，尝试用*定制桌面*类型代替默认的会话类型。For the command, use the desktop starter as an option of `dbus-launch`, for example `dbus-launch startxfce4`. This is a way to launch a session bus instance, set the appropriate environment variables so that the new session can find the bus.

路径问题

用 ssh 登录时，桌面环境的可执行程序（如 *startkde*、*startgnome* 或 *startxfce4* 等）可能未包含在 `$PATH` 环境变量中。在此情况下，不要简单地选择默认的 KDE、Gnome 或 XFCE，而应使用桌面环境可执行程序的全路径，例如 `/usr/bin/startxfce4`。 也可以启动 openbox 或其他窗口管理器，这会提示你输入服务器的用户名和口令，然后会看到 X2Go 图标一闪而过随后进入桌面。

x2go 客户端中无选择屏幕

由于 iproute2包 的回退导致 *ss* 指定了 `-u` 标记时不显示结果，as done in `/usr/bin/x2golistdesktops`。

详情参阅 ，。

会话没有正常登出

由于这个 bug 导致 X2Go 会话可能不能正常登出。X2Go 可能被会话初始化脚本输出的过量日志行搞懵了。一个简单的解决方法是定制一个会话脚本并且将日志输出重定向到某个文件或空文件`/dev/null`，然后指定把 X2Go 客户端应用程序指向这个定制脚本。

以下是 XFCE 的一个定制脚本范例：

```
#!/bin/sh
```

```
#
#xfce4-session spits out quite a bit of text during logout, which I guess
#confuses x2go so we would get a black screen and session hang.
#adding redirect to a logfile like "~/logfile" or "/dev/null" nicely solved it
# see https://bugs.x2go.org/cgi-bin/bugreport.cgi?bug=914
/usr/bin/xfce4-session > /dev/null
```

通知区无显示

参照#本地会话阻止了 X2Go 的新会话的解决方法。

共享文件夹未挂载（Windows 客户端）

The ssh-daemon used by the X2go windows client uses depreceated ssh-dss keys by default and because Arch does not accept them your shared folders will not mount. Check out this bug report for more information.

This can be solved on the windows side by generating different type of key:

```
C:\Program Files (x86)\x2goclient\ssh-keygen -b 2048 -t rsa
```

And simply replace `c:\Users\User\.x2go\etc\ssh_host_dsa_key` and `c:\Users\User\.x2go\etc\ssh_host_dsa_key.pub` with the newly generated key files.

Other workarrounds from might help, too.

Workaround for failing compositing window manager for remote session

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

This is useful for situations, when the computer running x2goserver is used also for local sessions with e.g. compiz as the window manager. For remote connections with x2goclient, compiz fails to load and metacity should be used instead. The following is for GNOME, but could be modified for other desktop environments. (Getting compiz ready is not part of this how-to.)

Create /usr/local/share/applications/gnome-wm-test.desktop:

```
[Desktop Entry]
Type=Application
Encoding=UTF-8
Name=gnome-wm-test
Exec=/usr/local/bin/gnome-wm-test.sh
NoDisplay=true
```

Create script /usr/local/bin/gnome-wm-test.sh:

```
#!/bin/sh
# Script for choosing compiz when possible, otherwise metacity
# Proper way to use this script is to set the key to mk-gnome-wm
# /desktop/gnome/session/required_components/windowmanager
xdpyinfo 2> /dev/null | grep -q "^ *Composite$" 2> /dev/null
IS_X_COMPOSITED=$?
if [ $IS_X_COMPOSITED -eq 0 ] ; then
    gtk-window-decorator &
    WM="compiz ccp --indirect-rendering --sm-client-id $DESKTOP_AUTOSTART_ID"
else
    WM="metacity --sm-client-id=$DESKTOP_AUTOSTART_ID"
fi
exec bash -c "$WM"
```

Modify the following gconf key to start the session with gnome-wm-test window manager:

```
$ gconftool-2 --type string --
set /desktop/gnome/session/required_components/windowmanager "gnome-wm-test"
```

/bin/bash: No such file or directory when connect (or what ever shell you use)

In you ssh configuration, if you chroot a user, this user need to have his own /bin directory inside his chrooted directory. If not, you will not be able to connect.

参阅

- Screenshot KDE 会话截屏
- Screenshot 配置对话框截屏

14.5 TigerVNC

难度 高级 | **篇幅** 20.6 KB | **所属** 远程服务器

本节将讲到

- 安装
- 为虚拟（无界面）会话运行 vncserver
- 直接转发本地显示内容
- 运行 x0vncserver 来直接控制本地显示内容
- 使用 XDMCP 为需要的会话运行 Xvnc

TigerVNC 是 Virtual Network Computing (VNC) 协议的一种实现。本文着重介绍服务端的功能。

安装

安装 tigervnc包 软件包。

为虚拟（无界面）会话运行 vncserver

初始设置

注意：在内存允许的条件下，Linux 系统可以启动任意数量的 VNC 服务端，它们同时并行运行，互不干扰。

简易教程如下。推荐用户阅读 vncserver 的 man 手册来了解所有的配置项。

1. 用 `vncpasswd` 创建密码，它会将哈希处理之后的密码存储在 `$XDG_CONFIG_HOME/tigervnc/config`。
2. 编辑 `/etc/tigervnc/vncserver.users` 来定义用户映射。该文件中定义的每个用户都会拥有对应的端口来运行会话。该文件中的数字对应的是 TCP 端口。默认情况下，:1 是 TCP 端口 5901 (5900+1)。如果需要运行一个并行的服

务端，第二个实例可以运行在下一个最大的、未被占用的端口，即 5902 (5900+2)。

3. 创建 `$XDG_CONFIG_HOME/tigervnc/config`，其中至少要有一行定义会话的类型，比如 `session=foo` （将foo替换为你想要运行的桌面环境）。你可以通过查看 `/usr/share/xsessions/` 里的 `.desktop` 文件来知道有哪些桌面环境在当前系统上可以使用。

文件内容举例：

```
$XDG_CONFIG_HOME/tigervnc/config
session=lxqt
geometry=1920x1080
localhost
alwaysshared
```

启动与停止 tigervnc

start `vncserver@.service`，如果需要让它随系统启动，enable 它。注意 `/etc/tigervnc/vncserver.users` 中定义的编号需要在@符号后面指定，比如启动:1的命令是：

```
# systemctl start vncserver@:1
```

注意：已经不再支持直接调用 `/usr/bin/vncserver` 了，因为这样做不会建立完整可用的会话环境。systemd 服务是唯一受支持的使用 TigerVNC 的方式。参见 Issue #1096。

直接转发本地显示内容

Tigervnc 带有 libvnc.so，它可以在 X 初始化过程中直接加载，以提供更好的性能。创建如下文件并重启 X：

```
/etc/X11/xorg.conf.d/10-vnc.conf
Section "Module"
Load "vnc"
EndSection

Section "Screen"
Identifier "Screen0"
Option "UserPasswdVerifier" "VncAuth"
Option "PasswordFile" "/root/.vnc/passwd"
EndSection
```

运行 x0vncserver 来直接控制本地显示内容

tigervnc包 同时也提供 `x0vncserver`，允许直接控制物理 X 会话。在使用 *vncpasswd* 定义了会话密码后，请这样启动服务端：

```
$ x0vncserver -rfbauth ~/.vnc/passwd
```

更多信息请参阅 x0vncserver(1)。

注意：

- x11vnc 是另一个可以直接控制当前 X 会话的 VNC 服务端。

- `x0vncserver` 目前不支持客户端和服务端之间的剪贴板共享（即使有 `autocutsel` 支持也不行）。参阅 Issue #529。

从 xprofile 启动 x0vncserver

一种启动 *x0vncserver* 的简单方法是在某个 xprofile 文件中添加如下行：

```
~/.xprofile
...
x0vncserver -rfbauth ~/.vnc/passwd &
```

通过 systemd 启动与停止 x0vncserver

通过系统服务启动

该选项允许用户访问当前显示界面，包括由显示管理器提供的登录界面。

该服务将在用户退出会话后自动重新启动。

下面的例子使用了LightDM，但可以通过修改 `XAUTHORITY` 变量来将其适用于其他显示管理器。

```
/etc/systemd/system/x0vncserver.service
[Unit]
Description=Remote desktop service (VNC) for :0 display
Requires=display-manager.service
After=network-online.target
After=display-manager.service

[Service]
Type=simple
Environment=HOME=/root
Environment=XAUTHORITY=/var/run/lightdm/root/:0
ExecStart=x0vncserver -display :0 -rfbauth $XDG_CONFIG_HOME/tigervnc/passwd
Restart=on-failure
RestartSec=500ms

[Install]
WantedBy=multi-user.target
```

这是一个 system unit, `-rfbauth $XDG_CONFIG_HOME/tigervnc/passwd` 指向 `/root/.vnc/passwd`

Start/enable `x0vncserver.service`.

通过用户服务启动

对大多数用户来说，快速远程访问当前桌面的最简单方法是运行 *x0vncserver* 来得到一个 VNC 服务器。为此可以创建一个 systemd unit，将其中的用户和选项替换为实际值：

```
~/.config/systemd/user/x0vncserver.service
[Unit]
Description=Remote desktop service (VNC)

[Service]
Type=simple
# wait for Xorg started by ${USER}
ExecStartPre=/bin/sh -c 'while ! pgrep -U "$USER" Xorg; do sleep 2; done'
ExecStart=/usr/bin/x0vncserver -rfbauth %h/.vnc/passwd
```

```
# or login with your username & password
#ExecStart=/usr/bin/x0vncserver -PAMService=login -PlainUsers=${USER} -
SecurityTypes=TLSPlain

[Install]
WantedBy=default.target
```

在 systemd用户服务模式下 start 并 enable `x0vncserver.service` 服务，也就是带上 `--user` 参数。

使用 XDMCP 为需要的会话运行 Xvnc

可以让 *systemd* socket activation 与 XDMCP 结合使用，为每个想登录的用户自动生成 VNC 服务器，这样无需为每个用户都准备服务器或端口。这种方法使用显示管理器来验证用户和登录，所以也不需要设置 VNC 密码。这种方法的缺点是会话无法在服务器上保持运行，退出后无法重新连接到它。

该方法首先需要设置 XDMCP，并确保显示管理器正在运行。 然后创建：

```
/etc/systemd/system/xvnc.socket
[Unit]
Description=XVNC Server

[Socket]
ListenStream=5900
Accept=yes

[Install]
WantedBy=sockets.target
/etc/systemd/system/xvnc@.service
[Unit]
Description=XVNC Per-Connection Daemon

[Service]
ExecStart=-/usr/bin/Xvnc -inetd -query localhost -geometry 1920x1080 -once -
SecurityTypes=None
User=nobody
StandardInput=socket
StandardError=syslog
```

使用 systemctl 来 start 并 enable `xvnc.socket`。现在多个用户同时连接到 5900 端口，均可获得单独的桌面。

如果 VNC 服务器直接暴露在公网，请在 `xvnc@.service` 里给 `Xvnc` 添加 `-localhost` 选项（注意 `-query localhost` 和 `-localhost` 是不同的开关），然后遵循 #通过 SSH 隧道连接到 vncserver。由于我们只在连接后才选择用户，所以 VNC 服务端以 *nobody* 用户运行，且直接使用了 `Xvnc` 而不是 `vncserver` 脚本，所以忽略了 `~/.vnc` 中的选项。或者也可以自动启动 *vncconfig* 来打开剪贴板共享（在非 VNC 会话中 *vncconfig* 将立即退出）。一种自动启动的方法是创建：

```
/etc/X11/xinit/xinitrc.d/99-vncconfig.sh
#!/bin/sh
vncconfig -nowin &
```

连接到 vncserver

警告：TigerVNC 默认的连接方式不安全，它缺乏身份验证，且在连接建立过程中无法防止中间人攻击。请确保知晓当前服务器的安全设置，并且不要以不安全的方式连接到可信网络之外的 vncserver。

注意：TigerVNC 默认使用 *TLSVnc* 作为验证和加密方式，也可以用 `SecurityTypes` 参数来改成其他方式。*TLSVnc* 意味着标准的 VNC 身份验证，使用 GNUTLS 加密流量，但不会验证服务器的身份。TigerVNC 支持更换其他安全方案，如 *X509Vnc*，它结合了标准 VNC 身份验证、GNUTLS 加密、服务器身份识别，推荐将它用于安全连接。 如果服务器上的 `SecurityTypes` 设置了非加密方式（如 *None*, *VncAuth*, *Plain*, *TLSNone*, *TLSPlain*, *X509None*, *X509Plain*）为高优先级，这样无法使用加密，是不明智的。运行 *vncviewer* 时，更安全的方式是明确指定 `SecurityTypes` 且不接受未加密的流量。其他的模式仅在 #通过 SSH 隧道连接到 vncserver 时使用。

连接到 vncserver 的客户端不限数量。下面是一个简单的例子，其中 vncserver 运行在 10.1.10.2 的 5901 端口上，端口可以用缩写 :1 来表示：

```
$ vncviewer 10.1.10.2:1
```

无密码验证

`-passwd` 开关允许我们定义服务器上 `~/.vnc/passwd` 文件的位置。用户在服务器上必须有权访问该文件，可以是通过 SSH 访问，也可以是实体访问。两种情况下都应将该文件放在客户端文件系统的一个安全位置，比如一个仅给期望用户以读取权限的位置。

```
$ vncviewer -passwd /path/to/server-passwd-file
```

直接提供密码也是可以的。

注意：下方的密码输入方式是不安全的，本机上任何可以运行 `ps` 的人都可以看到密码。

```
$ vncviewer -passwd <(echo MYPASSWORD | vncpasswd -f)
```

图形界面客户端示例

- gtk-vnc包
- krdc包
- vinagre包
- remmina
- virt-viewer包
- vncviewer-jarAUR

TigerVNC 的 vncviewer 也带有一个简单的图形界面，不带参数运行即可：

```
$ vncviewer
```

通过 SSH 隧道连接到 vncserver

对于提供 SSH 的服务器来说，这种方法有一个优点，除了已经打开的 SSH 端口外不需要再打开其他端口，因为 VNC 流量是在 SSH 隧道中传输的。

服务端配置

服务端必须运行 *vncserver* 或 *x0vncserver*。

无论运行哪个服务端，都建议在 `~/.vnc/config` 添加 `localhost` 选项或使用 `-localhost` 开关（适用于 *x0vncserver*），只允许来自 localhost 的连接，这样可以确保连接都来自于 ssh 或实体机上认证过的用户。例如：

```
~/.vnc/config
session=lxqt
geometry=1920x1080
localhost
alwaysshared
```

确保已经 start 或 restart `vncserver@.service`。例如（参见 #初始设置）：

```
# systemctl start vncserver@:1
```

对于 *x0vncserver* 则是：

```
$ x0vncserver -localhost -SecurityTypes none
```

客户端配置

远程计算机上的 VNC 服务端已被设置为仅接受本地连接。 现在，客户端必须与远程计算机（本例中为 10.1.10.2）建立 SSH 连接，并在客户端的某个端口（如 9901）和服务器的 5901 端口之间建立隧道。关于该功能的详细信息，参见 OpenSSH#Forwarding other ports 和 ssh(1)。

```
$ ssh 10.1.10.2 -L 9901:localhost:5901
```

当通过 SSH 建立连接后，请保持该 shell 窗口打开，它现在是与服务器之间的安全隧道。或者也可以使用 `-f` 选项直接在后台运行 SSH。客户端要想通过这一加密隧道连接到服务端，请指定 *vncviewer* 连接到客户端 localhost 上的转发端口。

```
$ vncviewer localhost:9901
```

实际上就是 vncviewer 连接到了本地的 9901 端口，而该端口映射到了服务器本地的 5901 端口。通过 SSH，向正确的端口建立了连接。

提示：只用一行命令即可实现在连接期间保持端口转发，在连接关闭后立即关闭转发：

```
$ ssh -fL 9901:localhost:5901 10.1.10.2 sleep 10; vncviewer localhost:9901
```

上述命令中 `-f` 选项使 ssh 在后台运行，它将因为 `sleep 10` 而保持运行。接着运行 vncviewer，在 vncviewer 使用隧道期间，ssh 在后台继续运行。当隧道不再使用，ssh 将关闭，这正是我们所需要的行为。

另外，vncviewer 的 `-via` 选项提供了上述命令的快捷方式：

```
$ vncviewer -via 10.1.10.2 localhost::5901
```

（注意双冒号：vncviewer 的语法是 `[host]:[display#]` 或 `[host]::[port]`。）

从 Android 设备通过 SSH 连接到 vncserver

为了用 Android 设备作为客户端，通过 SSH 连接到 VNC 服务端，可以进行以下设置：

1. 服务器运行 SSH
2. 服务器运行 vncserver（带有 `-localhost` 标志以确保安全）
3. Android 设备上的 SSH 客户端：*ConnectBot* 是一个流行的选择，本例中将使用它
4. Android 设备上的 VNC 客户端：此处采用 *androidVNC*

在 *ConnectBot* 中连接到目标主机。轻点 options 按钮，选择 *Port Forwards* 并添加一个端口：

```
Type: Local
Source port: 5901
Destination: 127.0.0.1:5901
```

在 *androidVNC* 中连接到 VNC 端口，即 SSH 连接中设置的本地端口：

```
Password: the vncserver password
Address: 127.0.0.1
Port: 5901
```

提示和技巧

连接到 OS X 系统

参见 https://help.ubuntu.com/community/AppleRemoteDesktop 。 已在 Remmina 上测试。

推荐的安全设置

#通过 SSH 隧道连接到 vncserver 中，SSH 处理身份验证和加密，如果没有采用该方法，则建议用 *X509Vnc*，因为 *TLSVnc* 缺乏身份验证。

```
$ vncserver -x509key /path/to/key.pem -x509cert /path/to/cert.pem -SecurityTypes
X509Vnc :1
```

颁发 x509 证书超出了本指南的范围，而 Let's Encrypt 提供了一种简便方法。也可以使用 OpenSSL 来颁发证书，然后与客户端共享公钥并使用 `-X509CA` 参数指定证书。下面是一个示例（服务器运行在 10.1.10.2 上）：

```
$ vncviewer 10.1.10.2 -X509CA /path/to/cert.pem
```

切换全屏

从 VNC 客户端的菜单可以切换全屏。默认快捷键是 `F8`。

鼠标后退和前进按钮失效的解决方法

VNC 协议目前只支持 7 个鼠标按钮（左、中、右、向上滚动、向下滚动、向左滚动、向右滚动），如果鼠标有后退和前进按钮，这些按钮无法使用，按钮的输入将被忽略。

在单击鼠标后退/前进按钮时，evrouter 将其改为键盘按键发送，可以解决此限制。如果需要，可以在服务器上使用 xautomation包 和 xbindkeys包 中的 xte 将键盘按键映射回鼠标单击。

用键盘 XF86Back/XF86Forward 按键代替鼠标后退/前进按钮

如果只需要在用 Web 浏览器或文件浏览器时后退/前进，则此方法既简单又合适。

客户端安装 evrouterAUR 和 xautomation包。配置 evrouter，想知道如何查找设备名称、窗口名称、按钮名称等，请参阅 Mouse buttons#evrouter 和 evrouter 手册页。以下是示例配置：

```
~/.evrouterrc
Window "OtherComputer:0 - TigerVNC": # 筛选窗口标题

# 使用 Shell 防止重复按键（参见 evrouter 手册）
"USB mouse" "/dev/input/by-id/usb-Mouse-name-event-mouse" none key/275 "Shell/xte 'key XF86Back'"
"USB mosue" "/dev/input/by-id/usb-Mouse-name-event-mouse" none key/276 "Shell/xte 'key XF86Forward'"

# 如果需要重复按键，请使用下面的 XKey（参见 evrouter 手册）
#"Logitech Gaming Mouse G400" "/dev/input/by-id/usb-Logitech_Gaming_Mouse_G400-event-mouse" none key/275 "XKey/XF86Back"
#"Logitech Gaming Mouse G400" "/dev/input/by-id/usb-Logitech_Gaming_Mouse_G400-event-mouse" none key/276 "XKey/XF86Forward"
```

客户端启动 evrouter。按照上面的配置，点击鼠标后退按钮时，向 VNC 服务器发送键盘按键 XF86Back，点击前进按钮时发送 XF86Forward。

在服务端将键盘按键映射回的鼠标点击

必要时可以在服务端将键盘按键映射回鼠标点击。此时最好使用客户端或服务端都不使用的按键值。下面的例子将按键值 XF86Launch8/XF86Launch9 用作鼠标按钮 8/9。

客户端上的 evrouter 配置：

```
~/.evrouterrc
Window "OtherComputer:0 - TigerVNC": # 窗口标题

# 使用 Shell 防止重复按键（参见 evrouter 手册）
"USB mouse" "/dev/input/by-id/usb-Mouse-name-event-mouse" none key/275 "Shell/xte 'key XF86Launch8'"
"USB mosue" "/dev/input/by-id/usb-Mouse-name-event-mouse" none key/276 "Shell/xte 'key XF86Launch9'"
```

服务端安装 xautomation包 和 xbindkeys包。配置 xbindkeys，配置中使用 xte 将键盘按键 XF86Launch8/XF86Launch9 映射到鼠标按钮 8/9。

```
~/.xbindkeysrc
"xte 'mouseclick 8'"
     XF86Launch8

"xte 'mouseclick 9'"
     XF86Launch9
```

启用 xbindkeys `$ xbindkeys -f ~/.xbindkeysrc`。服务端将 XF86Launch8/XF86Launch9 映射为鼠标按钮 8/9。

疑难解答

vncserver 内的终端起始于 / （根目录）

这是上游引入的已知问题。参阅：https://github.com/TigerVNC/tigervnc/issues/1108

无法输入 '<' 符号

如果在客户端上按下 < 却发送了 > 字符，请尝试重新映射键盘 \[失效链接 2020-04-03 ⓘ\]：

```
$ x0vncserver -RemapKeys="0x3c->0x2c"
```

窗口显示为黑色方框

这种情况很可能是应用程序依赖于 Xorg 的 Composite 扩展。如基于 webkit 的应用程序：midori、psi-plus 等。

这种情况下使用下列命令重启 vncserver：

```
$ vncserver -geometry ... -depth 24 :1 +extension Composite
```

VNC 下的 Composite 扩展仅在 24 位色深时才能正常工作。

没有鼠标指针

当使用 `x0vncserver` 时没有可见的鼠标指针，请使用下列命令启动 vncviewer：

```
$ vncviewer DotWhenNoCursor=1 <server>
```

或者将 `DotWhenNoCursor=1` 写入 tigervnc 配置文件，配置文件默认存放在 `~/.vnc/default.tigervnc`。

从远程主机复制剪贴板内容

如果无法从远程主机复制到本地，请在服务端运行 `autocutsel`，如 中所述：

```
$ autocutsel -fork
```

然后按下 F8 打开 VNC 菜单，选择 `Clipboard: local -> remote` 选项。

启动 GNOME 3 时显示 "Authentication is required to create a color managed device" 对话框

一种解决方法是创建一个 "vnc" 组，将 gdm 用户和其他使用 vnc 的用户添加到该组。按如下方式 修改 `/etc/polkit-1/rules.d/gnome-vnc.rules`：

```
   polkit.addRule(function(action, subject) {
      if ((action.id == "org.freedesktop.color-manager.create-device" ||
           action.id == "org.freedesktop.color-manager.create-profile" ||
           action.id == "org.freedesktop.color-manager.delete-device" ||
           action.id == "org.freedesktop.color-manager.delete-profile" ||
           action.id == "org.freedesktop.color-manager.modify-device" ||
           action.id == "org.freedesktop.color-manager.modify-profile") &&
          subject.isInGroup("vnc")) {
         return polkit.Result.YES;
      }
   });
```

没有窗口装饰/边框/标题栏/无法移动窗口

启动一个窗口管理器来修复一个空的 xterm frame。例如在 xfce 上，可在终端中运行 `\xfwm4 &\`。

按用户运行 systemd service unit

```
$ sudoedit /usr/lib/systemd/system/tigervnc@.service
[Unit]
Description=Remote desktop service (VNC)
After=syslog.target network.target

[Service]
Type=simple
ExecStart=/sbin/runuser -l USERNAME -c "/usr/bin/vncserver %i"

[Install]
WantedBy=multi-user.target
```

假设在 display 9 上启动服务：

```
$ sudo systemctl start tigervnc@:9
```

在启动时运行：

```
$ sudo systemctl enable tigervnc@:9
```

另外参见

- https://github.com/TigerVNC/tigervnc

第 15 章　共 5 节　75 KB

邮件客户端

邮件客户端这一层的卡点通常不在界面，而在它下面：投递与发信那一环没配好，再顺手的终端前端也发不出一封测试信。本章要从“能读本地邮箱”走到“能收发、能检索、能管理庞杂邮件”，包括把发信转交给 Gmail 这类外部服务器。

阅读顺序上，先读《NBSMTP》，用最小配置把发信接到 Gmail 的邮件服务器；再读《Mutt》，它篇幅最长，通用选项与多账户示例是后面各节的共同语法。《Notmuch》的初次使用与搜索必读，与 mutt、NeoMutt 集成两节按自己用的客户端挑着看；《Sup》与《Aerc》可跳，除非邮件量大到需要索引、备份与按键绑定。最容易踩的坑是把 Notmuch、Sup 当成独立客户端：它们不投递也不撰写，永久删除邮件也要走它们自己的命令，而不是直接删文件。

15.1　NBSMTP

难度 入门 | **篇幅** 1.6 KB | **所属** 邮件客户端

本节将讲到

- 安装
- 转发到 Gmail 邮件服务器

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： Thunderbird 等，参见应用程序列表/互联网#Email 客户端。 (在Talk:NBSMTP讨论)

来自 nbSMTP manpage：

: nbSMTP is a lightweight SMTP client. It simply takes a message from STDIN and sends it to a relayhost. A relayhost is meant to be a full SMTP server and it will really send the message.

安装

安装 nbsmtpAUR\[损坏的链接：package not found\] 包。

转发到 Gmail 邮件服务器

要配置 nbSMTP，您将必须编辑其配置文件（`~/.nbsmtprc`) 并输入您的帐户设置：

```
~/.nbsmtprc
relayhost=smtp.gmail.com
port=587
use_starttls=True
fromaddr=myusername@gmail.com
auth_user=myusername@gmail.com
auth_pass=myultrasecretpassword
```

请谨慎此文件的权限，建议运行以下命令：

```
chmod 600 ~/.nbsmtprc
```

要测试配置，请创建一个文件（testemail）：

```
testemail
To: myusername@gmail.com
From: myusername@gmail.com
Subject: nbsmtp test
hello email world
```

然后运行：

```
/usr/bin/nbsmtp < testemail
```

15.2 Aerc

难度 入门 | **篇幅** 1.9 KB | **所属** 邮件客户端

本节将讲到

- 安装
- 提示与技巧
- 参见

aerc 是一款基于终端的电子邮件客户端，具有 Vim 风格的按键绑定和 ex-command 系统。它使用嵌入式模板，允许使用任何终端编辑器作为嵌入式组件来撰写电子邮件。*aerc* 内置了 IMAP 和 SMTP 支持，还支持 Maildir 和 Notmuch。虽然 *aerc* 提供了一个简单的向导来设置电子邮件账户，但完整的配置是通过 `$XDG_CONFIG_HOME/aerc` 中的文本文件来处理的。

aerc 是用 Go 编写的，终端用户界面使用 Tcell 软件包。

安装

安装 aerc包 软件包。

提示与技巧

使 Khard 作为通讯录提供者

khard包 可被配置为通讯录提供程序\[失效链接 2024-07-30 ⓘ\]：

```
$XDG_CONFIG_HOME/aerc/aerc.conf
[compose]
address-book-cmd=khard email --parsable --remove-first-line %s
```

参见

- aerc 官方网站
- aerc 仓库
- aerc Wiki

15.3 Notmuch

难度 进阶 | **篇幅** 9.2 KB | **所属** 邮件客户端

本节将讲到

- 概述
- 初次使用
- 前端
- 与 mutt 集成
- 与 NeoMutt 集成

翻译状态：

- 本文（或部分内容）译自 Notmuch，最近一次同步于 2024-8-30，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Notmuch 是一个邮件索引器。从本质上讲，它是 *xapian* 之上的一个非常小的前端。与 Sup 一样，它只专注于一件事：为电子邮件建立索引。Notmuch 可用作电子邮件阅读器，也可简单地用作其他 MUA（如 mutt）的索引器和搜索工具。

概述

Notmuch 由 C 语言编写，速度比 sup-mail 快一个数量级。 Notmuch 可以在索引过程中终止，下次运行时会继续上次的工作。 与 sup-mail 一样，它也不提供永久删除不需要的电子邮件的方法（不过，请参阅#永久删除电子邮件）。 它不能获取或发送邮件，也不能存储你的电子邮件地址，你需要使用 OfflineIMAP、msmtp 和 *abook* 等程序来完成这些任务。

安装 notmuch包 软件包。它提供了 python、vim 和 emacs 绑定。

初次使用

安装完成后，您可以运行以下程序进入交互式设置：

```
notmuch setup
```

程序会提示你输入邮箱地址、主邮箱地址和副邮箱地址。你也可以直接编辑配置文件，该文件默认创建在 `$HOME/.notmuch-config`。

邮件目录的后续重新索引通过

```
notmuch new
```

前端

有一系列使用 notmuch 的方法，包括 CLI 或 Unix `$EDITORS`：

Emacs

notmuch 的默认前端是 Emacs。它是由开发 notmuch 的同一批人开发的。

Vim

notmuch-vim包 软件包中包含一个 Vim 界面。要启动它，请键入：

```
vim -c NotMuch
```

alot

alot 是用 Python 编写的 notmuch 独立 CLI 界面。它以 alot包 和 alot-gitAUR 的形式提供。

Alot 使用 mailcap 来处理不同类型的文件。目前包括 HTML 邮件，这意味着您需要配置一个 `~/.mailcap` 文件才能查看 HTML 邮件。至少要在 `~/.mailcap` 中加入这一行：

```
 text/html; w3m -dump -o -document_charset=%{charset} %s; nametemplate=%s.html; copiousoutput
```

此处使用 w3m包，也可使用其他基于文本的客户端，如 links包 或 lynx包，但它们的参数可能有所不同。

当然，还可以配置更多的文件处理程序。

bower

bower 是另一个 CLI 界面，使用 Mercury 编写。可通过 bower-mailAUR 获取。

Neomutt

Neomutt - 另一个包含许多功能补丁的 mutt fork，其中包括 Notmuch 集成补丁。安装 neomutt包 或 neomutt-gitAUR 软件包。

astroid

Astroid 是一款使用 C++ 和 GTK 编写的图形化 MUA 和 notmuch 界面。可通过 astroid包 或 astroid-gitAUR 安装。GUI 设计非常快速，可预览 HTML 和附件，并可通过键盘导航。它可以进行大量配置，你可以使用自己喜欢的嵌入式编辑器，也可以从外部启动它。查看 Tour 了解如何使用 astroid 以及完整设置的说明，或查看 README 了解更多信息。

与 mutt 集成

如果你使用 mutt 作为你的 MUA，那么 notmuch 就是你索引和搜索邮件的绝佳辅助工具。notmuch-mutt包 软件包提供了将 notmuch 与 mutt 集成的脚本。

安装 notmuch-mutt包 软件包并配置 notmuch 后，在使用 notmuch 从 mutt 进行搜索前，只需添加按键绑定以从 mutt 调用 `notmuch-mutt` perl 脚本。notmuch contrib source 建议在 `.muttrc` 中添加以下内容：

```
macro index <F8> \
```

```
"<enter-command>set my_old_pipe_decode=\$pipe_decode my_old_wait_key=\$wait_key
nopipe_decode nowait_key<enter>\
<shell-escape>notmuch-mutt -r --prompt search<enter>\
<change-folder-readonly>`echo ${XDG_CACHE_HOME:-
$HOME/.cache}/notmuch/mutt/results`<enter>\
<enter-command>set pipe_decode=\$my_old_pipe_decode wait_key=\$my_old_wait_key<enter>"
\
"notmuch: search mail"

macro index <F9> \
"<enter-command>set my_old_pipe_decode=\$pipe_decode my_old_wait_key=\$wait_key
nopipe_decode nowait_key<enter>\
<pipe-message>notmuch-mutt -r thread<enter>\
<change-folder-readonly>`echo ${XDG_CACHE_HOME:-
$HOME/.cache}/notmuch/mutt/results`<enter>\
<enter-command>set pipe_decode=\$my_old_pipe_decode wait_key=\$my_old_wait_key<enter>"
\
"notmuch: reconstruct thread"

macro index <F6> \
"<enter-command>set my_old_pipe_decode=\$pipe_decode my_old_wait_key=\$wait_key
nopipe_decode nowait_key<enter>\
<pipe-message>notmuch-mutt tag -- -inbox<enter>\
<enter-command>set pipe_decode=\$my_old_pipe_decode wait_key=\$my_old_wait_key<enter>"
\
"notmuch: remove message from inbox"
```

上文使用 `F8` 以使用 notmuch 搜索收件箱，使用 `F9` 从搜索结果创建线程，使用 `F6` 标记搜索结果。

与 NeoMutt 集成

如果使用 neomutt包，则不需要 notmuch-mutt包 软件包。取而代之的是，创建一个包含一些基本（虚拟）邮箱的 `~/.mailboxes`。虚拟邮箱并非实际文件夹，而是 notmuch 查询特定标记的结果：

```
~/.mailboxes
virtual-mailboxes "inbox" "notmuch://?query=tag:inbox"
virtual-mailboxes "archive" "notmuch://?query=tag:archive"
virtual-mailboxes "sent" "notmuch://?query=tag:sent"
virtual-mailboxes "newsletters" "notmuch://?query=tag:newsletters"
```

接下来，启用虚拟 spoolfile 并将其作为来源，让 mutt 知晓虚拟邮箱：

```
~/.muttrc
set virtual_spoolfile=yes
set folder=notmuch-root-folder
source ~/.mailboxes
```

此时，mutt 仍然会把你的邮箱列为空邮箱，因为你的邮件还没有被标记，所以查询的邮件也不是很多。要填满虚拟邮箱，首先需要标记邮件目录中的邮件。一个非常简单的方法是创建一个 shell 脚本，如下所示：

```
~/.scripts/notmuch-hook.sh
#!/bin/sh
notmuch new
# retag all "new" messages "inbox" and "unread"
notmuch tag +inbox +unread -new -- tag:new
# tag all messages from "me" as sent and remove tags inbox and unread
notmuch tag -new -inbox +sent -- from:me@example.org or from:me@myself.com
# tag newsletters, but dont show them in inbox
notmuch tag +newsletters +unread -new -- from:newsletter@example.org or
subject:'newsletter*'
```

将 shell 脚本设置为可执行并运行它：

```
 ~/.scripts/notmuch-hook.sh
```

为使上述示例有效，请确保 notmuch 将新信息标记为 'new'：

```
~/.notmuch-config
[new]
tags=new
```

最后，您的钩子脚本需要在新邮件到达时重新运行。如果使用 offlineimap包 将 IMAP 同步到本地 maildir，则需要创建一个同步后钩子：

```
~/.offlineimaprc
[Account myaccount]
postsynchook = ~/.scripts/notmuch-hook.sh
```

一些有用的按键绑定：

```
~/.muttrc
macro index A "<modify-labels>+archive -unread -inbox\\n" "Archive message"
macro index c "<change-vfolder>?" "Change to vfolder overview"
macro index \\\\ "<vfolder-from-query>" "Search mailbox"
```

永久删除电子邮件

一种方法是为希望从磁盘中删除的邮件保留一个标签，例如 "已删除"。然后，你可以将搜索标签与 `xargs` 结合起来，永久删除它们：

```
 notmuch search --output=files --format=text0 tag:killed | xargs -r0 rm
 notmuch new
```

将此钩子放入 notmuch 的 `pre-new` 钩子中，就能确保在更新数据库前删除文件。

参见

- notmuch 官方网站
- notmuch 仓库
- muchsync - 跨设备同步 Notmuch 邮件
- lieer - 快速获取和发送电子邮件，并在 notmuch 和 GMail 之间实现双向标签同步

15.4 Sup

难度 进阶 | **篇幅** 11.9 KB | **所属** 邮件客户端

本节将讲到

- 安裝
- 設定
- 使用
- 備份和復原
- 按鍵綁定列表

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2015, out of sync with English page（在 Talk:Sup# 中讨论）

Sup 是一套爲了擁有大量郵件使用者所開發，嶄新且強大的郵件客戶端程式。您可以將它想成 Mutt 和 Gmail 的混合體，擁有非常快速地操作和搜尋，標籤，自動通訊錄管理，支援大量帳號，以及更多功能。

安裝

從 AUR 安裝 sup-gitAUR。 不過開發團隊建議您用以下方式安裝：

```
$ gem install sup
```

好讓您直接從開發團隊那兒取得最新版本。

設定

Sup 附帶一款名爲 `sup-config` 的簡便設定工具。要使用它，請在終端機內執行它然後完成下面列出的幾項步驟：

1. 輸入您的全名。
2. 輸入您的主要電子郵件信箱，以及任何額外的電子郵件信箱。
3. 輸入您簽名檔的位置，如果您擁有的話。
4. 輸入您要用來撰寫新郵件的編輯器，以及任何您該傳入的參數。
5. 新增您郵件的來源，包括：
6. mbox 檔案
7. maildir 資料夾

支援遠端來源 (POP3, IMAP, IMAPS, 和 mbox + ssh) 的功能已經在 0.12 版釋出時移除。

Sup 目前專注於 MUA (mail user agent) 之功能且不自行處理下載郵件的工作。您可以使用像是 offlineimap, fetchmail, 和 rsync 等工具來將電子郵件傳送到本機系統的 mbox 或 maildir 資料夾。

sup wiki 內的示範簡介了如何使用 offlineimap 設定 gmail+imap 來源。Mutt#設置POP方式接收電郵子章節介紹一些額外的郵件傳送方法。

在新增好電子郵件來源後，`sup-config` 會執行 `sup-sync` 指令來匯入郵件到您的信箱內。

使用

執行 `sup` 指令來開啓 Sup 郵件客戶端程式。該程式應該會顯示由 `sup-config` 匯入的訊息。

對新使用者來說最該記得的按鍵就是 `?` 按鍵。在任何時間點它都會顯示所有的鍵盤指令，提示新使用者該如何導覽程式。

使用箭頭鍵或是 `j` 和 `k` 鍵巡覽討論串 (`Shift+j` 和 `Shift+k` 的功能如同 Page Up 和 Page Down 鍵)。按下 Tab 鍵，則能在有新訊息的討論串中快速穿梭。Sup 預設並不會讀入所有討論串；按下 `Shift+m` 將讀取更多討論串 (更多訊息將被自動讀取直到視窗被填滿)。

選擇欲檢視的討論串然後按下 `Enter` 鍵即可檢視它。在檢視討論串時，如果想要展開或摺疊某一訊息，請選擇該訊息然後按下 `Enter` 鍵。按下 `Shift+n` 將只展開新訊息 (同預設畫面) 或 `Shift+e` 切換所有訊息的狀態。按下 `o` 將顯示或隱藏部分訊息內容 (像是簽名檔)。

按下 `n` 和 `p` 鍵巡覽討論串內的訊息。按下 `h` 鍵顯示訊息的標頭。

按下 `b` 鍵循環切換緩衝，或者按下 `;` 鍵檢視已開啓緩衝的清單。按下 `x` 鍵抹殺某個緩衝。

按下 `a` 鍵封存討論串。這會將該討論串從 inbox 內隱藏直到其他人回覆它，到時候它就會重新出現。按下 `&` 鍵抹殺討論串。這等同於 Gmail 中的 "靜音" 功能，即使有人回覆該討論串它依然會被隱藏。該討論串將永遠不會出現在 inbox 內，不過依然能在搜尋結果找到它。

按下 `*` 鍵替討論串標記星號。按下 `Shift+s` 鍵將討論串標記爲垃圾訊息。Sup 並沒有任何內建的垃圾訊息分類器；要做到這個功能，可以考慮像是 spamassassin包 之類的程式。

按下 `t` 鍵標記討論串。按下 `l` 鍵替討論串內的訊息加上標籤。按下 `Shift+l` 鍵搜尋標籤。輸入欲搜尋的標籤後按下 `Enter` 鍵或是直接按下 `Enter` 鍵來列出所有標籤。按下 `\` 鍵將執行全文字搜尋。

按下 `Shift+c` 鍵來檢視聯絡人清單。要對清單中的人寄送電子郵件，選擇他或她的姓名然後按下 `Enter` 鍵。

備份和復原

備份電子郵件非常重要。爲了保證您不會弄丟任何東西，第一要先備份來源，像是 mbox 檔案和 maildir 目錄，然後執行：

```
$ sup-dump > 檔案名稱
```

這將會備份所有訊息狀態到文字檔案內。要從文字檔案內復原您的訊息狀態，只要執行：

```
$ sup-sync [<來源>+] --restored --restore 檔案名稱
```

謹記，上面的指令只會備份和復原訊息狀態。訊息本身需要另行備份。

按鍵綁定列表

inbox-mode 下的按鍵綁定

```
a：封存討論串 (從 inbox 中移除)
A：封存討論串 (從 inbox 中移除) 且標記爲已讀
```

thread-index-mode 下的按鍵綁定

```
   M：再讀取 20 份討論串
  !!：讀取所有討論串 (可能會列出很多討論串)
  ^G：取消目前的搜尋
   @：刷新畫面
   *：標記星號/取消星號整個討論串內的訊息
   N：切換討論串內所有訊息的未讀/已讀狀態
   l：編輯或新增討論串的標籤
   e：編輯訊息 (僅可對草稿使用)
   S：標記/取消標記討論串爲垃圾郵件
   d：刪除/取消刪除討論串
   &：抹殺討論串 (再也不會出現在 inbox)
   $：馬上儲存變更
 tab：跳到下一個新討論串
   r：回覆討論串內最新的訊息
   f：轉寄討論串內最新的訊息
   t：標記/取消標記所選的討論串
   T：標記/取消標記所有的討論串
   g：標記符合的討論串
+, =：套用下一個操作到所有已標記的討論串
   #：強制所有已標記的討論串合併成一個討論串
   u：重做上一個操作
```

thread-view-mode 下的按鍵綁定

```
      h：切換詳細標頭
      H：顯示完整訊息標頭
      V：顯示完整訊息 (原生格式)
<enter>：開啓/摺疊或啓動項目
      E：開啓/摺疊所有訊息
      e：編輯草稿
      y：寄出草稿
      l：替討論串編輯或新增標籤
      o：開啓/摺疊訊息中所有的引用
      n：跳到下一個開啓的訊息
```

p：跳到前一個開啓的訊息
z：對齊目前訊息在緩衝中的位置
*：替訊息標記星號/取消標記星號
N：切換訊息的未讀/已讀狀態
r：恢復一個訊息
f：轉寄訊息或附件
i：編輯某人的別名/暱稱
D：將訊息視爲新郵件來編輯
s：儲存訊息/附件到磁碟
S：從特定人士搜尋訊息
m：編寫訊息給某人
(：訂閱/取消訂閱郵件清單
)：訂閱/取消訂閱郵件清單
|：將訊息或附件重導給 shell 指令
.a：封存該討論串然後抹殺緩衝
.d：刪除該討論串然後抹殺緩衝
.s：標記該討論串爲垃圾訊息然後抹殺緩衝
.N：標記該討論串爲未讀然後抹殺緩衝
,a：封存該討論串，抹殺緩衝，然後檢視下一個
,d：刪除該討論串，抹殺緩衝，然後檢視下一個
,s：標記該討論串爲垃圾訊息，抹殺緩衝，然後檢視下一個
,N：標記該討論串爲未讀，抹殺緩衝，然後檢視下一個
,n：抹殺緩衝，然後檢視下一個
]a：封存該討論串，抹殺緩衝，然後檢視前一個
]d：刪除該討論串，抹殺緩衝，然後檢視前一個
]s：標記該討論串爲垃圾訊息，抹殺緩衝，然後檢視前一個
]N：標記該討論串爲未讀，抹殺緩衝，然後檢視前一個
]n：抹殺緩衝，然後檢視前一個

contact-list-mode 下的按鍵綁定

M：再讀取 10 個聯絡人
D：放棄通訊錄然後重新讀取
a, i：編輯聯絡人的代稱或是姓名
t：標記/取消標機該行
+：套用下一個操作到所有已標記的項目
S：從特定人士開始搜尋訊息

line-cursor-mode 下的按鍵綁定

<down arrow>, j：將遊標向下移動一行
<up arrow>, k：將遊標向上移動一行
<enter>：選擇該項目

scroll-mode 下的按鍵綁定

J, ^E：向下一行
K, ^Y：向上一行
<left arrow>, h：向左一排
<right arrow>：向右一排
<page down>, <space>, ^F：向下一頁
<page up>, p, <backspace>, ^B：向上一頁
^D：向下半頁
^U：向上半頁
<home>, ^, 1：跳到頁首
<end>, 0：跳到頁尾
[：跳到左邊
/：在目前的緩衝內搜尋

```
                                    n：跳到下個緩衝內搜尋到的項目
```

全域按鍵綁定

```
   q：離開 Sup，但是需要確認
   Q：直接離開 Sup
   ?：顯示幫助
   b：切換到下一個緩衝
   B：切換到前一個緩衝
   x：抹殺目前的緩衝
   ;：列出所有緩衝
   C：列出通訊錄
  ^L：重繪畫面
\, F：搜尋所有訊息
   U：顯示所有未讀訊息
   L：列出標籤
   P：輪查有無新訊息
m, c：撰寫新訊息
  ^G：什麼也不做
   R：編輯最近的訊息草稿
```

疑難排解

因爲 Illegal instruction (core dumped) 而當掉

`sup` 使用一套名爲 Xapian 的搜尋引擎，且它被編譯爲使用 SSE2 指令集。如果您的 CPU 不支援 SSE2 指令集，您將碰上錯誤訊息：

```
   Illegal instruction (core dumped)
```

要解決這問題，您必須在編譯 Xapian 時加入 `--disable-sse` 選項。

1\. 查看 ruby-xapian-rubyAUR 的 `PKGBUILD` 您可以得知它從 https://rubygems.org/gems/xapian-ruby 下載了一份 gem。下載那份 gem。

2\. 執行這些指令

```
   gem unpack xapian-ruby.gem
   gem unpack --spec xapian-ruby.gem
   mv xapian-ruby.gemspec xapian-ruby/
   cd xapian-ruby
```

3\. 您需要編輯 `Rakefile`。您的目標是修改兩行有執行設定參數的指令。您所要做的就是將 `--disable-sse` 加到這些設定指令後面：

```
   system! "./configure --prefix=#{prefix} --exec-prefix=#{prefix} --disable-sse"
   system! "./configure --prefix=#{prefix} --exec-prefix=#{prefix} --with-ruby --
disable-sse"
```

然後儲存所做的修改。

4\. 執行

```
   gem build xapian-ruby.gemspec
   gem install --local xapian-ruby.gem
```

這樣應該能解決舊款 CPU 不能執行 Xapian 的問題。附帶一提，如果您第一次執行 `sup-config` 且正確地完成所有設定，卻依然得到下列錯誤訊息

```
   This Sup version expects a v4 index, but you have an existing v0 index. Please run
sup-dump to save your labels, move /home/user/.sup/xapian out of the way, and run sup-
sync --restore. (RuntimeError)
   Rats, that failed. You may have to do it manually.
```

那麼您可能也碰上了該問題，嘗試執行其他如 `sup` 或 `sup-dump` 等執行檔，看看有無顯示 Illegal instruction (core dumped) 的錯誤訊息。

另請參閱

- 網站
- Wiki
- README
- FAQ
- 哲學闡述

15.5 Mutt

难度 高级 | **篇幅** 50.3 KB | **所属** 邮件客户端

本节将讲到

- 安装
- 配置
- General options
- ACCOUNT1
- ACCOUNT2

Mutt 是一个基于文本的邮件客户端，因其强大的功能而闻名。Mutt 虽然已诞生二十多年了，但仍然是大量高级用户首选的邮件客户端。

Mutt 主要侧重于作为邮件用户代理（Mail User Agent，MUA），最初是为了查看邮件而编写的。稍后实现的功能（检索、发送和过滤邮件）与其他邮件应用程序相比过于简单，因此用户可能希望使用外部应用程序扩展 Mutt 的功能。

尽管如此，Arch Linux 的 mutt包 软件包在编译时包含了对 IMAP、POP3 和 SMTP 的支持，因此不再需要外部应用程序。

本文包含如何使用原生 IMAP 发送和检索邮件，以及一套配置，它使用 OfflineIMAP 或 getmail（POP3）检索邮件，使用 procmail 过滤邮件（使用 POP3 时），并使用 msmtp 发送邮件。

安装

安装 mutt包 软件包，或者考虑使用 #NeoMutt。

可以考虑为 IMAP 配置安装外部帮助程序，例如 isync、OfflineIMAP 或 msmtp。

或者，如果使用 POP3：getmailAUR，fetchmail包 或 fdm包，以及 procmailAUR。

注意：关于认证配置：

- 如果只需要 LOGIN 和 PLAIN，依赖项 libsasl包 已足够
- 如果想（或必须）使用 CRAM-MD5、GSSAPI 或 DIGEST-MD5，安装 cyrus-sasl-gssapi包 软件包
- 如果使用 Gmail 作为 SMTP 服务器，则可能需要安装 cyrus-sasl包 软件包

NeoMutt

NeoMutt 项目旨在汇集 Mutt 的所有补丁。它增加了很多功能。许多旧的 Mutt 补丁已经被更新、整理和记录。

虽然 AUR 中有许多不同的 mutt 包，每个都提供了不同的补丁，NeoMutt 计划通过实现合适的编译选项取代它们。

NeoMutt 可通过 neomutt包 或 neomutt-gitAUR（开发版本）软件包安装。

配置

本节包含 #IMAP、#POP3、#Maildir 和 #SMTP 配置。

Mutt 默认在 6 个位置搜索配置文件：`~/.muttrc`、`~/.mutt/muttrc` 和 `$XDG_CONFIG_HOME/mutt/muttrc`，首先加上 `-MUTT_VERSION` 后缀，然后不加。这之中的任何一个位置都可以。如果决定将初始化文件放在其他地方，使用 `$ mutt -F /path/to/.muttrc`。应当了解 Mutt 配置的一些前提。它的语法非常接近 Bourne Shell。例如，可以获取另一个配置文件的内容：

```
source /path/to/other/config/file
```

可以使用变量并将 shell 命令的结果赋值给它。

```
set editor=`echo \$EDITOR`
```

此处的 `$` 已被转义，因此在传递给 shell 之前不会被 Mutt 替换。还要注意反引号的使用，因为 bash 语法 `$(...)` 不起作用。Mutt 有很多预定义的变量，但是也可以自己定义变量。用户变量**必须以“my”开头！**

```
set my_name = "John Doe"
```

IMAP

内置 IMAP 支持

mutt包 软件包在编译时已包含 IMAP 支持。最少需要在 muttrc 文件中包含 4 行才能访问邮件。

imap_user

```
set imap_user=USERNAME
```

提示：接着前一个例子，记住 Gmail 需要完整的邮件地址（这并非标准）：

```
set imap_user=your.username@gmail.com
```

imap_pass

如果未设置，将提示输入密码。

```
set imap_pass=SECRET
```

提示：如果在 Gmail 中启用了双重验证，并且为 Mutt 添加了特定于应用程序的密码，则应使用这个密码，而不是 Gmail 的密码。

folder

使用服务器（以及在必要时层次结构中最高的文件夹）而不是包含所有邮件（和文件夹）的本地文件夹。

```
set folder=imap[s]://imap.server.domain[:port]/[folder/]
```

你不需要使用 folder，不过它可能会带来方便（例如，如果所有其他的文件夹都在 INBOX 中）。无论在这里把什么设置成文件夹，之后在 Mutt 中都可以通过等号（=）或加号（+）访问。例如：

```
set folder=imaps://imap.gmail.com/
```

应当注意的是，对于几个账户，it is best practice to use different folders -- e.g. for *account-hook*。如果有几个 Gmail 账户，使用

```
set folder=imaps://username@imap.gmail.com/
```

其中账户是 *username@gmail.com*。这样将可以区分不同的文件夹，不然会导致认证错误。

spoolfile

spoolfile 就是（未过滤的）邮件到达的文件夹。大多数邮件服务按照常规将其命名为 *INBOX*。现在可以使用“=”或“+”代替上面配置的 `folder` 路径。例如：

```
set spoolfile=+INBOX
```

mailboxes

在此处列出所有应当定期检查是否有新邮件的 IMAP 文件夹：

```
mailboxes =INBOX =family
mailboxes imaps://imap.gmail.com/INBOX imaps://imap.gmail.com/family
```

或者，检查所有订阅的 IMAP 文件夹（等价于通过 `mailboxes` 添加所有的文件夹）：

```
set imap_check_subscribed
```

如果想订阅所有的文件夹，这两个版本是等价的。因此第二个方法更方便，但第一个更灵活。此外，更新版本的 Mutt 的默认配置包含一个绑定到“y”键的宏，可以改变到 mailboxes 下列出的任何一个文件夹。

如果不设置这个变量，默认将使用 *spoolfile*。这个变量对 #Sidebar 也很重要。

总结

使用这些选项，就可以启动 Mutt、输入 IMAP 密码并开始阅读邮件了。以下是一个 muttrc 片段（适用于 Gmail），同时还有值得考虑的几行，它们可用于获得更好的 IMAP 支持。

```
set folder      = imaps://imap.gmail.com/
set imap_user   = your.username@gmail.com
set imap_pass   = your-imap-password
set spoolfile   = +INBOX
mailboxes       = +INBOX

# Store message headers locally to speed things up.
# If hcache is a folder, Mutt will create sub cache folders for each account which may
speeds things up even more.
set header_cache = ~/.cache/mutt

# Store messages locally to speed things up, like searching message bodies.
# Can be the same folder as header_cache.
# This will cost important disk usage according to your e-mail amount.
set message_cachedir = "~/.cache/mutt"

# Specify where to save and/or look for postponed messages.
set postponed = +[Gmail]/Drafts

# Allow Mutt to open a new IMAP connection automatically.
unset imap_passive

# Keep the IMAP connection alive by polling intermittently (time in seconds).
set imap_keepalive = 300

# How often to check for new mail (time in seconds).
set mail_check = 120
```

外部 IMAP 支持

虽然 Mutt 内置 IMAP 支持，但它并不会下载邮件以供离线使用。可以使用 OfflineIMAP 或者 isync 这样的外部应用程序将邮件下载到本地目录，以便随后由 Mutt 处理。

考虑使用 spamassassin包 或 imapfilterAUR 这样的应用程序整理邮件。

POP3

mutt包 软件包在编译时已包含 POP3 支持，可通过 muttrc(5) 中描述的 `pop_*` 变量配置。

另外，也可以使用外部程序通过 POP3 获取邮件。其中一个流行的选项是使用 getmail 检索邮件，并使用 procmail 过滤邮件。

Maildir

Maildir is a generic and standardized format. Almost every MUA is able to handle Maildirs and Mutt's support is excellent. There are just a few simple things that you need to do to get Mutt to use them. Open your muttrc and add the following lines:

```
set mbox_type=Maildir
set folder=~/mail
```

```
set spoolfile=+/
set header_cache=~/.cache/mutt
```
This is a minimal Configuration that enables you to access your Maildir and checks for new local Mails in INBOX. This configuration also caches the headers of the eMails to speed up directory-listings. It might not be enabled in your build (but it sure is in the Arch-Package). Note that this does not affect OfflineIMAP in any way. It always syncs all of the directories on a Server. `spoolfile` tells Mutt which local directories to poll for new Mail. You might want to add more Spoolfiles (for example the Directories of Mailing-Lists) and maybe other things. But this is subject to the Mutt manual and beyond the scope of this document.

SMTP

Whether you use POP or IMAP to receive mail you will probably still send mail using SMTP.

Folders

There is basically only one important folder here: the one where all your sent e-mails will be saved.

```
set record = +Sent
```
Gmail automatically saves sent e-mail to `+[Gmail]/Sent`, so we do not want duplicates.

```
unset record
```
Native SMTP support

The mutt包 package is compiled with SMTP support.

For example:

```
set my_pass='mysecretpass'
set my_user=myname

set realname = 'Your Real Name'
set from = your-email-address
set use_from = yes

set smtp_pass = $my_pass
set smtp_url=smtps://$my_user@smtp.domain.tld
set ssl_force_tls = yes
```
Note that if your SMTP credentials are the same as your IMAP credentials, then you can use those variables:

```
set smtp_pass = $imap_pass
set smtp_url=smtps://$imap_user@smtp.domain.tld
```
You may need to tweak the security parameters. If you get an error like `SSL routines:SSL23_GET_SERVER_HELLO:unknown protocol`, then your server probably uses the SMTP instead of SMTPS.

```
set smtp_url=smtp://$imap_user@smtp.domain.tld
```
There are other variables that you may need to set. For example for use of STARTTLS:

```
set ssl_starttls = yes
```

注意：Setting `ssl_force_tls = yes` is still needed to prevent STARTTLS stripping attacks.

If your server uses the LOGIN authentication method you might need to specify this explicitly, despite the manual's claim that all methods are tried by default:

```
set smtp_authenticators = "login"
```

See muttrc(5) for more information.

External SMTP support

An external SMTP agent such as msmtp, sSMTP or opensmtpd包 can also be used.

The `sendmail` variable in your `muttrc` determines the program and arguments used to deliver mail in mutt. Any additional arguments must be interpreted by the program as recipient addresses.

For example, if using msmtp:

```
muttrc
set realname='Disgruntled Kangaroo'

set sendmail="/usr/bin/msmtp"

set edit_headers=yes
set folder=~/mail
set mbox=+mbox
set spoolfile=+inbox
set record=+sent
set postponed=+drafts
set mbox_type=Maildir

mailboxes +inbox +lovey-dovey +happy-kangaroos
```

从 Mutt 发送邮件

现在，启动 `mutt`：

You should see all the mail in `~/mail/inbox`. Press `m` to compose mail; it will use the editor defined by your `EDITOR` environment variable. If this variable is not set, you can fix it before starting Mutt:

```
$ export EDITOR=your-favorite-editor
$ mutt
```

You should store the EDITOR value into your shell resource configuration file (such as bashrc). You can also set the editor from Mutt's configuration file:

```
.muttrc
set editor=your-favorite-editor
```

For testing purposes, address the letter to yourself. After you have written the letter, save and exit the editor. You will return to Mutt, which will now show information about your e-mail. Press `y` to send it.

警告：If at this point you press `q` by accident, Mutt will ask you `Postpone this message? ([yes]/no)`. This is really asking whether you want to save the message you just wrote. If you press "n" (perhaps because you want to edit the message again) the message will be permanently deleted. When using Mutt, always remember that "Postpone this message?" really means "Save this message?".

Multiple accounts

Now you should have a working configuration for one account at least. You might wonder how to use several accounts, since we put everything into a single file.

Well all you need is to write account-specific parameters to their respective files and source them. All the IMAP/POP3/SMTP configuration for each account should go to its respective folder.

警告：When one account is setting a variable that is not specified for other accounts, you **must unset** it for them, otherwise configuration will overlap and you will most certainly experience unexpected behaviour.

Mutt can handle this thanks to one of its most powerful features: hooks. Basically a hook is a command that gets executed before a specific action. There are several hooks available. For multiple accounts, you must use account-hooks *and* folder-hooks.

- Folder-hooks will run a command before switching folders. This is mostly useful to set the appropriate SMTP parameters when you are in a specific folder. For instance when you are in your work mailbox and you send a e-mail, it will automatically use your work account as sender.
- Account-hooks will run a command every time Mutt calls a function related to an account, like IMAP syncing. It does not require you to switch to any folder.

Hooks take two parameters:

```
account-hook [!]regex command
folder-hook [!]regex command
```

The regex is the folder to be matched (or not if preceded by the !). The command tells what to do.

Let us give a full example:

```
.muttrc
## General options
set header_cache = "~/.cache/mutt"
set imap_check_subscribed
set imap_keepalive = 300
unset imap_passive
set mail_check = 60
set mbox_type=Maildir

## ACCOUNT1
source "~/.mutt/work"
# Here we use the $folder variable that has just been set in the sourced file.
# We must set it right now otherwise the 'folder' variable will change in the next
sourced file.
folder-hook $folder 'source ~/.mutt/work'

## ACCOUNT2
source "~/.mutt/personal"
folder-hook *user@gmail.com/ 'source ~/.mutt/personal'
folder-hook *user@gmail.com/Family 'set realname="Bob"'
.mutt/work
## Receive options.
set imap_user=user@gmail.com
set imap_pass=****
set folder = imaps://user@imap.gmail.com/
```

```
set spoolfile = +INBOX
set postponed = +Drafts
set record = +Sent

## Send options.
set smtp_url=smtps://user:****@smtp.gmail.com
set realname='User X'
set from=user@gmail.com
set hostname="gmail.com"
set signature="John Doe"
# Connection options
set ssl_force_tls = yes
unset ssl_starttls

## Hook -- IMPORTANT!
account-hook $folder "set imap_user=user@gmail.com imap_pass=****"
```

Finally `.mutt/personal` should be similar to `.mutt/work`.

Now all your accounts are set, start Mutt. To switch from one account to another, just change the folder (`c` key). Alternatively you can use the sidebar.

To change folder for different mailboxes you have to type the complete address -- for IMAP/POP3 folders, this may be quite inconvenient -- let us bind some key to it.

```
## Shortcuts
macro index,pager <f2> '<sync-mailbox><enter-command>source
~/.mutt/personal<enter><change-folder>!<enter>'
macro index,pager <f3> '<sync-mailbox><enter-command>source ~/.mutt/work<enter><change-
folder>!<enter>'
```

With the above shortcuts (or with the sidebar) you will find that changing folders (with `c` by default) is not contextual, *i.e.* it will not list the folders of the current mailbox, but of the one used the last time you changed folders. To make the behaviour more contextual, the trick is to press \= or + for current mailbox. You can automate this with the following macro:

```
macro index 'c' '<change-folder>?<change-dir><home>^K=<enter>'
```

Passwords management

Keep in mind that writing your password in `.muttrc` is a security risk. One solution is to always enter the password manually, but this becomes cumbersome.

GPG

An alternative solution is to encrypt your password with GnuPG in an encrypted file. Setup your own keypair if you have not done so already. Create a file in a tmpfs with the following contents:

```
set my_pass = "password"
```

注意：Remember that user defined variables **must** start with `my_`.

Then encrypt this file, setting yourself as the recipient and move it into an accessible location. In this example the encrypted file resides at $HOME/.my-pwds.gpg.

In your mutt configuration file add the following before any account:

```
source "gpg -dq $HOME/.my-pwds.gpg |"
```

注意：At the end of the line above, there is no space between the pipe and the double quote.

This decrypts the file quietly and sets the variable `my_pass` in this example. This can be used in any variable after it has been sourced. For example:

```
set imap_pass=$my_pass
```

If you use external tools like OfflineIMAP and msmtp, you need to set up an agent (e.g. gpg-agent, see GnuPG#gpg-agent) to keep the passphrase into cache and thus avoiding those tools always prompting for it.

Pass

You can also use pass to encrypt your passwords easily. Just add the passwords to the given passwords as follows:

```
pass add user@domain.tld
```

After that, just add the command `pass show user@domain.tld` in your `muttrc`, for instance:

```
set imap_pass=`pass show user@domain.tld`
```

Note the use of the backquotes. Using regular quotes will not work.

Security concern

If `enter-command` is available from the UI, it is possible to see the password unencrypted, which may be undesired if anybody else than you has access to your session while Mutt is running. You may want to disable it for this reason. As a consequence, every command that the user intends to use must be bound to a key in advance, otherwise it will never be accessible.

```
.muttrc
 bind generic,alias,attach,browser,editor,index,compose,pager,pgp,postpone ':' noop
```

Tips and tricks

Guides to get you started with using and customizing Mutt:

- My first Mutt (maintained by Bruno Postle)
- The Woodnotes Guide to the Mutt Email Client (maintained by Randall Wood)
- The Homely Mutt (by Steve Losh)
- Everything You Need To Know To Start Using GnuPG with Mutt (by Justin R. Miller)
- A List of useful programs that often are used in combination with Neomutt

If you have any Mutt specific questions, feel free to ask in the IRC channel.

Key bindings

The default key bindings are quite far from the more common Emacs-like or Vi-like bindings. You can customize them to your preference. Mutt has a different set of bindings for the pager, the index, the attachment view, etc. Thus you need to specify which *map* you want to modify when you bind a key. You can review the list of commands and key bindings from Mutt's help page (default key: `?`). Example of Vi-like bindings:

```
muttrc
bind pager j next-line
```

```
bind pager k previous-line
bind attach,index,pager \CD next-page
bind attach,index,pager \CU previous-page
bind pager g top
bind pager G bottom
bind attach,index g first-entry
bind attach,index G last-entry
```

Composition

Encrypt and sign mail (GnuPG)

To start encrypting mail in mutt using GnuPG
copy `/usr/share/doc/mutt/samples/gpg.rc` to your mutt configuration folder (e.g. to `~/.mutt/gpg.rc`). Then append the following to your mutt configuration file (e.g. `~/.mutt/muttrc`):

```
source ~/.mutt/gpg.rc
```

Most encryption options are then available by pressing `p` in the compose view.

See the `pgp_*` and `crypt_*` options in muttrc(5).

E-mail character encoding

When using Mutt there are two levels where the character sets that must be specified:

- The text editor used to write the e-mail must save it in the desired encoding.
- Mutt will then check the e-mail and determine which encoding is the more appropriate according to the priority you specified in the `send_charset` variable. Default: "us-ascii:iso-8859-1:utf-8".

So if you write an e-mail with characters allowed in ISO-8859-1 (like 'résumé'), but without characters specific to Unicode, then Mutt will set the encoding to ISO-8859-1.

To avoid this behaviour, set the variable in your `muttrc`:

```
set send_charset="us-ascii:utf-8"
```

or even

```
set send_charset="utf-8"
```

The first compatible charset starting from the left will be used. Since UTF-8 is a superset of US-ASCII it does not harm to leave it in front of UTF-8, it may ensure old MUA will not get confused when seeing the charset in the e-mail header.

Custom mail headers

One of the greatest thing in Mutt is that you can have full control over your mail header.

First, make your headers editable when you write e-mails:

```
set edit_headers=yes
```

Mutt also features a special function `my_hdr` for customizing your header. Yes, it is named just like a variable, but in fact it is a function.

You can clear it completely, which you *should* do when switching accounts with different headers, otherwise they will overlap:

```
unmy_hdr *
```

Other variables have also an impact on the headers, so it is wise to clear them before using `my_hdr`:

```
unset use_from
unset use_domain
unset user_agent
```

Now, you can add any field you want -- even non-standard one -- to your header using the following syntax:

```
my_hdr <FIELD>: <VALUE>
```

Note that <VALUE> can be the result of a command.

Example:

```
## Extra info.
my_hdr X-Info: Keep It Simple, Stupid.

## OS Info.
my_hdr X-Operating-System: `uname -s`, kernel `uname -r`

## This header only appears to MS Outlook users
my_hdr X-Message-Flag: WARNING!! Outlook sucks

## Custom Mail-User-Agent ID.
my_hdr User-Agent: Every email client sucks, this one just sucks less.
```

Signature block

Create a .signature in your home directory. Your signature will be appended at the end of your email. Alternatively you can specify a file in your Mutt configuration:

```
set signature="path/to/sig/file"
```

Random signature

You can use *fortune* (package fortune-mod包) to add a random signature to Mutt.

Create a fortune file and then add the following line to your .muttrc:

```
set signature="fortune pathtofortunefile|"
```

Note the pipe at the end. It tells Mutt that the specified string is not a file, but a command.

Compose and send from command line

Man pages will show all available commands and how to use them, but here are a couple of examples. You could use Mutt to send alerts, logs or some other system information, triggered by login through `.bash_profile`, or as a regular cron job.

Send a message:

```
mutt -s "Subject" somejoeorjane@someserver.com < /var/log/somelog
```

Send a message with attachment:

```
mutt -s "Subject" somejoeorjane@someserver.com -a somefile < /tmp/sometext.txt
```

Compose HTML e-mails

Since Mutt has nothing of a WYSIWIG client, HTML is quite straightforward, and you can do much more than with all WYSIWIG mail clients around since you edit the source code directly. Simply write your mail using HTML syntax. For example:

```
This is normal text<br>
<b>This is bold text</b>
```

Now before sending the mail, use the `edit-type` command (default shortcut `Ctrl+t`), and replace `text/plain` by `text/html`.

注意：HTML e-mails are regarded by many people as useless, cumbersome, and subject to reading issues. Mutt can read HTML mails with a text browser like w3m or lynx, but it has clearly no advantage over a plain-text e-mail. You should avoid writing HTML e-mails when possible.

Display another email while composing

A common complaint with Mutt is that when composing a new mail (or reply), you cannot open another mail (i.e. for checking with another correspondent) without closing the current mail (postponing). The following describes a solution:

First, fire up Mutt as usual. Then, launch another terminal window. Now start a new Mutt with

```
mutt -R
```

This starts Mutt in read-only mode, and you can browse other emails at your convenience. It is strongly recommended to always launch a second Mutt in read-only mode, as conflicts will easily arise otherwise.

注意：When changing folders (with `c` or `y`) the read-only mode is not preserved. Instead `Esc c` has to be used.

提示：This solution calls for a bit of typing, so it is suitable to bind the following command to a keyboard shortcut:

```
$TERMINAL -e mutt -R
```

where `$TERMINAL` is your terminal.

打印

可以安装 muttprintAUR 以获得更好的打印质量。在 muttrc 文件中插入：

```
set print_command="/usr/bin/muttprint %s -p {PrinterName}"
```

Viewing content

Viewing URLs in a web browser

Your should start by creating a .mutt directory in $HOME if not done yet. There, create a file named macros. Insert the following:

```
 macro pager \cb <pipe-entry>'urlview'<enter> 'Follow links with urlview'
```

Then install the urlviewAUR package.

Create a .urlview in $HOME and insert the following:

```
REGEXP (((http|https|ftp|gopher)|mailto)[.:][^ >"\t]*|www\.[-a-z0-9.]+)[^ .,;\t>">\):]
COMMAND <your-browser> %s
```

When you read an email on the pager, hitting ctrl+b will list all the urls from the email. Navigate up or down with arrow keys and hit enter on the desired url. Your browser will start and go to the selected site.

Some browser will require additional arguments to work properly. For example, Luakit will close on Mutt exit. You need to fork it to background, using the `-n` parameter:

```
COMMAND luakit -n %s 2>/dev/null
```

The `2>/dev/null` is to make it quiet, i.e. to prevent useless message printing where you do not want them to.

注意：urlview has a few deficiencies (e.g. the inability to handle certain email encodings) and is fairly feature-bare (e.g. it does not provide context for links it finds). There are a couple alternatives that do better. One, which can handle all email encodings and provides link context, is extract_urlAUR. Another, which can also provide link context but cannot handle all email encodings, is urlscan包. Both are drop-in replacements for urlview, though extract_url has features which benefit from additional configuration changes.

Viewing HTML

It is possible to pass the html body to an external HTML program and then dump it, keeping email viewing uniform and unobtrusive. Three programs are described here: lynx包, w3m包 and elinks包 (make sure the selected package is installed).

If `~/.mutt/mailcap` does not exist you will need to create it and save the following to it.

```
text/html; lynx -assume_charset=%{charset} -display_charset=utf-8 -collapse_br_tags -dump %s; nametemplate=%s.html; copiousoutput
```

or, in case of w3m,

```
text/html; w3m -I %{charset} -T text/html; copiousoutput;
```

or, in case of elinks,

```
text/html; elinks -dump; copiousoutput;
```

or, in case of elinks (with color support),

```
text/html; elinks -dump -dump-color-mode 1; copiousoutput;
```

Edit `~/.muttrc` and add the following,

```
set mailcap_path     = ~/.mutt/mailcap
```

To automatically open HTML messages in lynx, w3m or elinks add this additional line to the muttrc:

```
auto_view text/html
```

The beauty of this is, instead of seeing an html body as source or being opened by a separate program, in this case lynx, you see the formatted content directly, and any url links within the email can be displayed with `Ctrl+b`, assuming you have urlviewAUR installed.

If you receive many emails with multiple or alternate encodings Mutt may default to treating every email as html. To avoid this, add the following variable to your

~/.muttrc to have Mutt default to text when available and use w3m/lynx only when no text version is availble in the email:

```
alternative_order text/plain text/html
```

Some HTML mails may not display correctly in a text-based web browser. As a fallback solution, you can bind a key to open a graphical browser in such cases. The following macro will open the HTML mail selected from the attachment view in the web browser defined in the environment. (Feel free to adapt the `~/.cache/mutt/` folder).

```
 macro attach 'V' "<pipe-entry>iconv -c --to-code=UTF8 >
~/.cache/mutt/mail.html<enter><shell-escape>$BROWSER ~/.cache/mutt/mail.html<enter>"
```

If `$BROWSER` is `firefox`, then set `intl.charset.fallback.utf8_for_file` to `true` in `about:config` as described in to ensure non-ASCII characters in the HTML page are displayed correctly.

Filtering the message view

You can restrict the view to e-mails matching a pattern and specific properties with the `limit` command (default shortcut: `l`).

To view all e-mails containing "foo" in the header, simply write "foo" and you are done. To remove the filter, use the "all" keyword.

To view all flagged messages, use

```
~F
```

To view all unread messages that are either of size ≥1MB or from johndoe, use

```
~U (~z 1M- | ~f johndoe)
```

All possible patterns are listed in the official manual.

Conversation grouping

The default sort order is by date. Use the `sort-mailbox` command (default key: `o`) to change the sorting option. You can group e-mails by conversation/thread, in which case you can define how to sort threads and how to sort within a thread.

In the following example, threads are sorted according to the date of their last e-mail.

```
muttrc
set sort=threads
set sort_aux=last-date-received
```

Configuring editors to work with mutt

vim

- To limit the width of text to 72 characters, edit your .vimrc file and add:

```
au BufRead /tmp/mutt-* set tw=72
```

- Another choice is to use Vim's mail filetype plugin to enable other mail-centric options besides 72 character width. Edit `~/.vim/filetype.vim`, creating it if unpresent, and add:

```
augroup filetypedetect
  " Mail
  autocmd BufRead,BufNewFile *mutt-*              setfiletype mail
```

```
augroup END
```

- To set a different tmp directory, e.g. ~/.tmp, add a line to your muttrc as follows:

```
set tmpdir="~/.tmp"
```

- To reformat a modified text see the Vim context help

```
:h 10.7
```

GNU nano

nano is another nice console editor to use with Mutt.

To limit the width of text to 72 characters, edit your .nanorc file and add:

```
 set fill 72
 set breaklonglines
```

If you do not want to limit the width of text globally, you can pass the column number as an argument to the hard-wrap option in your muttrc file, e.g.:

```
 set editor="nano -r 72 -b"
```

Also, in muttrc file, you can specify the line to start editing so that you will skip the mail header:

```
 set editor="nano +7"
```

Emacs

Emacs has a *mail* and a *message* major mode. To switch to mail-mode automatically when Emacs is called from Mutt, you can add the following to your `.emacs`:

```
.emacs
;; Mutt support.
(add-to-list 'auto-mode-alist '("/tmp/mutt.*" . mail-mode))
;; Neomutt support.
(add-to-list 'auto-mode-alist '("/tmp/neomutt-" . mail-mode))
```

If you usually run Emacs daemon, you may want Mutt to connect to it. Add this to your `.muttrc`:

```
.muttrc
set editor="emacsclient -a \"\" -t"
```

Display settings

Colors

Append the contents of `/usr/share/doc/mutt/samples/colors.linux` to your .muttrc file, or copy and source it. Then adjust to your liking.

The actual color each of these settings will produce depends on the colors set in your ~/.Xresources file.

Alternatively, you can source any file you want containing colors (and thus act as a theme file). See for a theme example.

Index Format

Here follows a quick example to put in your `.muttrc` to customize the Index Format, i.e. the columns displayed in the folder view.

```
set date_format="%y-%m-%d %T"
set index_format="%2C | %Z [%d] %-30.30F (%-4.4c) %s"
```

See the Mutt Reference, strftime(3) and printf(3) for more details.

Display recipient instead of sender in "Sent" folder view

By default Mutt uses the `%L` format string in the `index_format` variable, which will display:

- "To <list-name>", if an address in the "To:" or "Cc:" header field matches an address defined by the user's `subscribe` command.
- Otherwise it displays the author name, or recipient name if the message is from you.

If you use multiple email addresses in the same mailbox, make sure to configure the alternates variable, so that Mutt knows which messages were from you.

```
muttrc
alternates    ^me@example.COM$ ^me@example.NET$ ^example@archlinux.ORG$
```

Variable column width

If you resize the window, the subject might get truncated while there is still unused space left for some fields, like for the sender. You can get the maximum number of columns supported by your terminal (i.e. the width) using a shell call to `tput cols`. With this value, you can set a percentage of the width to fields like Sender and Subject.

Example using the above folder-hook and a sidebar width of 24:

```
muttrc
## From field gets 30% of remaining space, Subject gets 70%.
## Remaining space is the total width minus the other fields (35), minus the sidebar
(24)
set my_index_format_pre='set my_col_from = `echo $((30 * ($(tput cols)-35-24) / 100))`;
set my_col_subject = `echo $((70 * ($(tput cols)-35-24) / 100))`; set index_format="%2C
| %Z [%d] %-$my_col_from.${my_col_from}'
set my_index_format_post=' (%-4.4c) %?M?<%M> ?%-$my_col_subject.${my_col_subject}s"'

folder-hook .*[sS]ent.* "$my_index_format_pre"t"$my_index_format_post"
folder-hook ! .*[sS]ent.* "$my_index_format_pre"F"$my_index_format_post"
```

We *must* set the variables `my_col_from` and `my_col_from` from within the hooks. Otherwise, the column values will not get re-computed.

We can add a binding to force re-computing the index format without changing folder:

```
muttrc
macro index,pager \CL "<enter-
command>$my_index_format_pre"F"$my_index_format_post<enter><redraw-screen>"
```

Sidebar

Example settings for a sidebar are in `/usr/share/doc/mutt/samples/sample.muttrc-sidebar`, including keybindings. Copy, edit, and source that file in your mutt configuration file. Be sure to change `set sidebar_visible = yes`.

Append the following in order to toggle the sidebar visibility:

```
bind index,pager B sidebar-toggle-visible
```

注意：You *must* set the `mailboxes` variables or the `imap_check_subscribed` to tell the sidebar which folder should be displayed. See the mailboxes section.

Note that with the `mailboxes` option, folders appear in the order they were set to `mailboxes` if you do not use the `sidebar_sort_method` option.

提示：To add a separator between different mailboxes, add a fake folder to the list of folders For example add:

```
mailboxes "+-- My mailbox -----------"
```

Display the index above the pager view

Set the following variable in your `muttrc`:

```
set pager_index_lines=10
```

Contact management

Address aliases

Aliases is the way Mutt manages contacts. An alias is **nickname \[longname\] <address>**.

- The **nickname** is what you will type in Mutt to get your contact address. One word only, and should be easy to remember.
- The **longname** is optional. It may be several words.
- An **<address>** must be in a valid form (i.e. with an `@`).

It is quite simple indeed. Add this to `.muttrc`:

```
set alias_file = "~/.mutt/aliases"
set sort_alias = alias
set reverse_alias = yes
source $alias_file
```

Explanation:

- `alias_file` is the file where the information is getting stored when you add an alias from within Mutt.
- `sort_alias` specifies which field to use to sort the alias list when displayed in Mutt. Possible values: alias, address.
- `reverse_alias` if set to yes mutt will display the "personal" name from your aliases in the index menu if it finds an alias that matches the message's sender.
- `source $alias_file` tells Mutt to read aliases on startup. Needed for auto-completion.

Now all you have to do when prompted `To:` is writing the alias instead of the full address. The beauty of it is that you can auto-complete the alias using `Tab`. Autocompleting a wrong or an empty string will display the full list. You can select the alias as usual, or by typing its index number.

There are two ways to create aliases:

- From Mutt, press `a` when an e-mail of the targetted person if selected.

- Edit the alias_file manually. The syntax is really simple:

```
alias nickname Long Name <my-friend@domain.tld>
```

Abook

abookAUR is a stand-alone program dedicated to contact management. It uses a very simple text-based interface and contacts are stored in a plain text, human-readable database. Besides the desired contact properties are extensible (birthday, address, fax, and so on).

Abook is specifically designed to be interfaced with Mutt, so that it can serve as a full, more featured replacement of Mutt internal aliases. If you want to use Abook instead of aliases, remove the aliases configuration in `.muttrc` and add this:

```
muttrc
## Abook
set query_command= "abook --mutt-query '%s'"
macro index,pager  a "<pipe-message>abook --add-email-quiet<return>" "Add this sender
to Abook"
bind editor        <Tab> complete-query
```

See the man pages `abook` and `abookrc` for more details and a full configuration sample.

Goobook

Goobook allows you to search your Google contacts from the command line or from within Mutt and can be installed with the goobookAUR or goobook-gitAUR packages.

Before using goobook you must configure `~/.goobookrc`. To generate the default template:

```
$ goobook config-template > ~/.goobookrc
```

See `~/.goobookrc` for configuration options. At a minimum, you will need to enter your **email** and **password**.

注意：If you have two-step verification enabled with the Google account, you may need to generate an application password.

In order to authenticate with Google, you must generate a Client ID and Client secret. See for details.

If you want to use Goobook instead of aliases, remove any alias configuration in `.muttrc` and add:

```
muttrc
## GooBook
set query_command="goobook query '%s'"
macro index,pager a "<pipe-message>goobook add<return>" "add sender to google contacts"
bind editor <Tab> complete-query
```

When composing an email message within mutt, `Tab` will now search your Google contacts. While viewing messages `a` will add the sender to Google contacts.

Khard

khard包 is a command-line addressbook that uses locally-stored carddav address book entries. You can use vdirsyncer包 to sync those with CardDAV servers.

The integration in mutt is similar to abook, see khard documentation.

Manage multiple sender accounts

If you use multiple sender accounts, you can automatically associate a specific sender account with a recipient. mutt-vidAUR scans sent emails for the most recent "From" details associated with specific recipients, saving these in a file for mutt to source. The next time you email this recipient, mutt will automatically invoke a send-hook with the same email address and real name that you used previously. See mutt-vid's homepage for more details.

Request IMAP mail retrieval manually

If you do not want to wait for the next automatic IMAP fetching (or if you did not enable it), you might want to fetch mails manually. There is a mutt command `imap-fetch-mail` for that. Alternatively, you could bind it to a key:

```
bind index "^" imap-fetch-mail
```

Avoiding slow index on large (IMAP) folders due to coloring

Index highlighting by regex is nice, but can lead to slow folder viewing if your regex checks the body of the message.

Use folder-hook for only highlighting in, for example, the inbox (if you manage to empty your mailbox efficiently):

```
folder-hook . 'uncolor index "~b \"Hi Joe\" ~R !~T !~F !~p !~P"'
folder-hook ""!"" 'color index brightyellow black "~b \"Hi Joe\" ~N !~T !~F !~p !~P"'
```

Speed up folders switch

Add this to your `.muttrc`:

```
set sleep_time = 0
```

Archive treated e-mails

When you read an e-mail, you have four choices: Answer it, Flag it, Archive it or Delete it. If you have this in mind, you can keep your inbox slim and fit with this macro (set up for Gmail):

```
macro index \' "<tag-pattern>~R !~D !~F<enter>\
<tag-prefix><save-message>+[Gmail]/All <enter>" \
"Archive"
```

Migrating mails from one computer to another

In case you are transfering your mails to a new machine (copy&paste), you probably need to delete the header cache (a file or folder like `~/.cache/mutt` if you followed the above configuration) to make Mutt able to read your migrated E-Mails. Otherwise Mutt may freeze.

Note that if you had a folder created for you header cache, all mailboxes will have their own cache file, so you can delete caches individually without having to remove everything.

Default folder for saving attachments

By default Mutt will save attachments to the folder it was started from. If you want to always set the default destination to `~/attachments`, you can create the following alias, which launches Mutt in this folder:

```
alias mutt='cd ~/attachments && mutt'
```

Pager behavior

Show context lines when going to next page:

```
set pager_context=3
```

Stop at the end instead of displaying next mail:

```
set pager_stop=yes
```

Fast reply

By default Mutt will ask to confirm the recipient and the subject when you reply to an e-mail. It will also ask if you want to include the original mail in your answer. If you assume you will always stick to the default values, you can set up Mutt to skip these questions:

```
muttrc
set fast_reply=yes
set include=yes
```

You can still edit the recipient and the subject before sending.

Ignore own e-mail addresses from group-reply

Mutt will include your e-mail address(es) in the recipient list when you group-reply to a mail you were CC'ed. You can ask Mutt to ignore some addresses with:

```
alternates mail1@server1|mail2@server2|...
```

IMAP message cache

When using the built-in IMAP support, e-mails are fetched in memory by default. Retrieving a big e-mail several times will download it from your IMAP server that many times.

Alternatively, you can ask Mutt to store all fetched messages on disk:

```
muttrc
set message_cachedir=~/.cache/mutt/messages
```

(The folder must exist.) This will make any future retrieval instantaneous, even with big attachments.

If you want to purge the cache from its oldest e-mails exceeding a limit of, say, 50MB, you can use a script like the following:

```
~/.mutt/purgecache.sh
#!/bin/sh

## In KB.
CACHE_LIMIT=51200
```

```
cd "$1" 2>/dev/null
[ $? -ne 0 ] && exit

[ $(du -s . | cut -f1 -d'	') -lt $CACHE_LIMIT ] && exit
while IFS= read -r i; do
	rm "$i"
	[ $(du -s . | cut -f1 -d'	') -lt $CACHE_LIMIT ] && exit
done <<EOF
$(find . -type f -exec ls -rt1 {} +)
EOF
```

and call it on startup:

```
muttrc
set message_cachedir=~/.cache/mutt/messages
source "~/.mutt/purgecache.sh '$message_cachedir'|"
```

Open attachments or view HTML email in the background

By default, opening the attachments in the mutt will block mutt. You can edit `~/.mutt/mailcap` to append an `&` so mutt will launch one of the Default applications in the backgroud. Take xdg-open as an example,

```
text/html; xdg-open %s &> /dev/null &; nametemplate=%s.html
application/*; xdg-open %s &> /dev/null &;
image/*; xdg-open %s &> /dev/null &;
```

`&> /dev/null` is used to prevent any error or message from cluttering the mutt terminal window. And `nametemplate=%s.html` is used because chromium refuse to render local files without .html extension as HTML.

Troubleshooting

Backspace does not work in Mutt

This is a common problem with some xterm-like terminals. Two solutions:

- Either rebind the key in `.muttrc`

```
bind index,pager ^? previous-line
```

Note that `^?` is one single character representing backspace in caret notation. To type in Emacs, use `Ctrl+q Backspace`, in Vim `Ctrl+v Backspace`.

- Or fix your terminal:

```
$ infocmp > termbs.src
```

Edit `termbs.src` and change `kbs=^H` to `kbs=\177`, then:

```
$ tic -x termbs.src
```

The change-folder function always prompt for the same mailbox

This is not a bug, this is actually an intended behaviour. See the multiple accounts section for a workaround.

I cannot change folder when using Mutt read-only (Mutt -R)

This is certainly because you are using macros like this one:

```
macro index,pager <f2> '<sync-mailbox><enter-command>source
~/.mutt/personal<enter><change-folder>!<enter>'
```

This macro tells Mutt to sync (which is a write operation) before switching.

Either use the sidebar or set another macro:

```
macro index,pager <f3> '<enter-command>source ~/.mutt/personal<enter><change-folder>!
<enter>'
```

Error sending message, child exited 127 (Exec error.).

This is an SMTP error. It means that mutt does not know how to send the message. You can either try installing sendmail and see if that solves your issue, or you can set the smtp_url variable. If you use gmail, you can add the following to your muttrc to tell mutt to use gmails smtp server.

```
set smtp_url=smtps://$imap_user:$imap_pass@smtp.gmail.com
```

Take note of the smtps protocol, it is important. This should solve the problem.

Character encoding problems

If you are having problems with character encoding, first read this section in the Mutt wiki.

If Chinese text is still garbled, it may help to decode with GBK even when GB2312 is specified in the header. You can do this with `iconv` by adding the following to your `mailcap` file:

```
text/plain; iconv -f gbk -t utf-8 %s; test=echo "%{charset}" | grep -ic "gb2312";
copiousoutput;
```

and enabling it by adding a line to your `.muttrc`:

```
auto_view text/plain
```

For HTML emails, you can edit the relevant line of your mailcap by replacing `%{charset}` with `$(echo %{charset} | sed s/gb2312/gbk/I)`, for example:

```
text/html; w3m -dump -I $(echo %{charset} | sed s/gb2312/gbk/I) %s;
nametemplate=%s.html; copiousoutput
```

Unable to login with GMail

Gmail disables access from applications it considers less secure, including `mutt`. You can enable access by following the instructions here

Not possible to open too long URLs with urlview

Too long URLs are not parsed correctly, because urlview does not decode text (see). You can let mutt decode the e-mails instead. Replace the line for opening urlview with the following code:

```
macro index \cb "\
:set my_tmp_pipe_decode=\$pipe_decode\n\
:set pipe_decode\n\
|urlview\n\<br>:set pipe_decode=\$my_tmp_pipe_decode\n\<br>:unset my_tmp_pipe_decode\n"
\<br>'call urlview to extract URLs out of a message'<br>```

Another option is to use [urlscan](https://github.com/firecat53/urlscan) instead, which
gives a text user interface to select the URL and can deal with much stranger text
formats.

## 文档

新手可能会发现很难找到 Mutt 的帮助。实际上大部分主题都涵盖在官方文档中。强烈推荐阅读这些文档。
```

- [官方手册](http://www.mutt.org/doc/manual/)。Arch Linux 的 [mutt](https://archlinux.org/packages/?name=mutt)包 软件包也在 `/usr/share/doc/mutt/` 安装了 HTML 和纯文本手册。
- `mutt` 和 `muttrc` 手册页。

参见

- [Mutt 官方网站](http://www.mutt.org/)
- [官方手册](http://www.mutt.org/doc/manual/)
- [The Mutt wiki](https://gitlab.com/muttmua/mutt/wikis/home/)
- [Brisbin's great guide on how to setup different IMAP accounts with Mutt, offlineimap, msmtp](https://pbrisbin.com/posts/two_accounts_in_mutt/)
- [Mutt 快速指南](https://srobb.net/mutt.html)
- [Steve Losh on Mutt, offlineimap, msmtp, notmuch (focused on Gmail)](https://stevelosh.com/blog/2012/10/the-homely-mutt/)

DNS

域名解析配错一处，依赖网络的命令就会一起失效。本章讲清解析链：《域名解析》讲 NSS 与 glibc 解析器如何判定查询，是其余各节的地基，必须精读；《Unbound》最长，覆盖递归、缓存与排错，也必读；《BIND》只在自建权威或配 DNSSEC 时再看；《Cloudflared》最短，按需查阅即可。

最常见的坑是跳过解析链直改上游：查询先经 NSS 和 glibc 解析器判定，服务在跑却解析不了，错在这一环而非 DNS 服务器。其次是混淆权威与递归：BIND 只管权威，Unbound 只管递归，角色配错比地址配错更隐蔽。

16.1　Cloudflared

难度 入门 | **篇幅** 3.1 KB | **所属** DNS

本节将讲到

- 安装
- 用法
- 检查
- 上游节点
- 参阅

Cloudflared 可被用于运行本地 DNS over HTTPS（DoH）服务器，也就是存根解析器。

安装

安装 cloudflared包。

用法

运行 `cloudflared proxy-dns` 来启动 DNS over HTTPS 代理服务器。

使用 `--address` 和 `--port` 选项来指定 cloudflared 监听的地址和端口，它们的默认值分别为 `localhost` 和 `53`。详细的可用命令行选项清单可在这里查看。

你可以创建一个 systemd 单元文件，例如：

```
/etc/systemd/system/cloudflared.service
[Unit]
Description=DNS over HTTPS proxy client
Wants=network-online.target nss-lookup.target
Before=nss-lookup.target

[Service]
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
DynamicUser=yes
ExecStart=/usr/bin/cloudflared proxy-dns --port 54
```

```
[Install]
WantedBy=multi-user.target
```

注意：Extra 包不提供服务单元文件。参见 archlinux/packaging/packages/cloudflared#1。

启动服务后，你可以使用 drill(1)（由 ldns包 提供）来确认它是否在正常工作。

```
$ drill archlinux.org @127.0.0.1 -p 54
```

检查

通过 1.1.1.1/help 来检查浏览器是否在使用 *Cloudflare DoH*。

上游节点

默认情况下 cloudflared 会使用 `https://1.1.1.1/dns-query` 和 `https://1.0.0.1/dns-query`，即 Cloudflare DoH 服务器的地址作为上游 URL。

你可以通过 `--upstream` 选项来指定不同的上游节点 URL。

注意：请使用你信任的 DNS 解析服务器，可参考域名解析#第三方 DNS 服务.

参阅

- 官方指引
- 命令行选项
- Running a DNS over HTTPS Client - Cloudflare Resolver
- 隆重推出家庭版 1.1.1.1

16.2 BIND

难度 进阶 | **篇幅** 8.0 KB | **所属** DNS

本节将讲到

- 安装
- 配置
- 权威 DNS 服务器
- 配置 DNSSEC
- 自动监听新的网络接口

伯克利互联网名称服务 (Berkeley Internet Name Daemon，简称 BIND) 是 DNS 协议的一个参考实现。

注意：开发 BIND 的组织在发现安全漏洞之后，会先通知付费客户，四天以后才会通知 Linux 发行版和大众。

安装

安装软件包 bind包。

要使用 BIND 提供系统 DNS 服务，修改 resolv.conf，将`nameserver 127.0.0.1`放到最前面。

开始/启用 `named.service` 服务。

配置

BIND 的配置文件是 `/etc/named.conf`. `named.conf` man 手册页介绍了所有选项。

Reload the `named.service` unit 以应用配置变更.

只允许本地访问

如果希望只允许本地网络访问，编辑 `/etc/named.conf` 并将这行配置加入到 **options** 区域。

```
listen-on { 127.0.0.1; };
```

DNS 转发

要将 DNS 请求请求转发到上游 DNS 服务器（例如说您的 ISP 的服务器，或者 Google、OpenNIC 等知名的服务）。将下面字段加入配置文件的 options 中。.

```
 forwarders { 8.8.8.8; 8.8.4.4; };
```

不要忘记重启 `named.service` 服务。

权威 DNS 服务器

以下为一个设置权威域的简单教程。在这个示例中，我们的权威域名为 "domain.tld"。

更详尽的教程参见 Two-in-one DNS server with BIND9.

1\. 创建一个 zonefile

```
# nano /var/named/domain.tld.zone
$TTL 7200
; domain.tld
@       IN      SOA     ns01.domain.tld. postmaster.domain.tld. (
                                        2007011601 ; Serial
                                        28800      ; Refresh
                                        1800       ; Retry
                                        604800     ; Expire - 1 week
                                        86400 )    ; Minimum
                IN      NS      ns01
                IN      NS      ns02
ns01            IN      A       0.0.0.0
ns02            IN      A       0.0.0.0
localhost       IN      A       127.0.0.1
@               IN      MX 10   mail
imap            IN      CNAME   mail
smtp            IN      CNAME   mail
@               IN      A       0.0.0.0
www             IN      A       0.0.0.0
```

```
mail            IN      A       0.0.0.0
@               IN      TXT     "v=spf1 mx"
```

$TTL 定义了默认的 TTL (time-to-live), 单位为秒。在这个例子中，默认 TTL 为 2 小时。

每次修改 zonefile 的时候，都需要将 Serial (序列号) 加一再重启 **named**。只有当新的 Serial 比最后传输的域的序列号大的时候，从服务器才会请求传输新的域。

2\. *配置主服务器*

将您的 zone 文件加到 `/etc/named.conf`:

```
zone "domain.tld" IN {
        type master;
        file "domain.tld.zone";
        allow-update { none; };
        notify no;
};
```

重新加载 `named.service` 服务。

3\. *将其设置为默认 DNS 服务器*

如果您自己已经在运行 DNS 服务器的话，可以考虑同时将其用来处理 DNS 查询请求。服务器必须支持 **recursive** (递归) 查询。为了防止 DNS 放大攻击，许多 DNS 解析程序都默认禁用了递归功能。Arch 的默认 `/etc/named.conf` 配置只允许本机地址使用递归：

```
allow-recursion { 127.0.0.1; };
```

resolv.conf 配置文件必须包含 127.0.0.1 地址以使用您的 DNS 服务器。 参见 Domain name resolution#Overwriting of /etc/resolv.conf 以了解确保这个文件不会被覆盖的方法。

如果您想为您的局域网提供 DNS 服务的话（例如 192.168.0 IP 段），您必须将对应的 IP 段加入到 `/etc/named.conf` 中：

```
allow-recursion { 192.168.0.0/24; 127.0.0.1; };
```

配置 DNSSEC

- https://www.dnssec.net/practical-documents
- https://www.cymru.com/Documents/secure-bind-template.html **(configuration template!)**
- https://www.bind9.net/manuals\[失效链接 2021-11-07 ⓘ\]
- https://www.bind9.net/BIND-FAQ\[失效链接 2021-11-07 ⓘ\]
- https://web.archive.org/web/20100528023716/http://blog.techscrawl.com:80/2009/01/13/enabling-dnssec-on-bind/
- Or use an external mechanisms such as OpenDNSSEC (fully-automatic key rollover)

自动监听新的网络接口

BIND 会每个每隔几个小时扫描新的网络接口并停止在已经不再不存在的上监听。如果您想修改这个时间的话，可以在 `/etc/named.conf` 中增加这个项：

```
interface-interval <扫描间隔>;
```

最大间隔为 28 天 (40230 分钟)。

如果需要禁用这个功能的话，可以将时间值设置为 **0**。

最后，请重启服务。

在 chroot 环境运行 BIND

在 \[chroot\] 环境运行可以提高安全性。

创建 Jail House

首先，我们需要创建一个 jail。我们可以使用 `/srv/named`, 并将相关文件都放到里面去。

```
mkdir -p /srv/named/{dev,etc,usr/lib/engines,var/{run,log,named}}
# Copy over required system files
cp -av /etc/{localtime,named.conf} /srv/named/etc/
cp -av /usr/lib/engines-1.1/* /srv/named/usr/lib/engines/
cp -dfprv /var/named /srv/named/var/
# Set up required dev nodes
mknod /srv/named/dev/null c 1 3
mknod /srv/named/dev/random c 1 8
# Set Ownership of the files
chown -R named:named /srv/named
```

这些步骤可以配置 jail 的文件系统。

服务文件

接下来我们需要创建服务文件 (service file)，以强制 BIND 在 chroot 环境中启动。

```
cp -av /usr/lib/systemd/system/named.service /etc/systemd/system/named-chroot.service
```

我们需要修改 service 启动 BIND 的方法。

```
/etc/systemd/system/named-chroot.service
  ExecStart=/usr/bin/named -4 -f -u named -t "/srv/named"
}
```

最后，重新加载 systemd `systemctl daemon-reload`。然后，启动 `named-chroot.service`。

参见

- BIND 9 Administrator Reference Manual
- BIND 9 DNS Administration Reference Book
- DNS and BIND by Cricket Liu and Paul Albitz
- Pro DNS and BIND

- Internet Systems Consortium, Inc. (ISC)
- DNS Glossary\[失效链接 2021-11-07 ⓘ\]
- Archived mailing list discussion on BIND's future

16.3 Unbound

难度 进阶 | **篇幅** 18.6 KB | **所属** DNS

本节将讲到

- 安装
- 配置
- 使用
- 提示与技巧
- 疑难解答

Unbound 是一个具有验证，递归和缓存等功能的 DNS 解析器。根据Wikipedia)：

: Unbound has supplanted the Berkeley Internet Name Domain (BIND) as the default, base-system name server in several open source projects, where it is perceived as smaller, more modern, and more secure for most applications.

安装

安装 unbound包 软件包。 此外, expat包 是使用DNSSEC验证请求所必须的。

配置

默认配置已经位于`/etc/unbound/unbound.conf`文件中。此外，`/etc/unbound/unbound.conf.example`文件包含了其他的可配置设置项，并以注释的形式给出了示范设置。以下章节重点解释和默认配置文件不同的设置项。如需了解更多细节，参见unbound.conf(5)。

除非特别声明，这一节列出的选项都是放置在配置文件的`server`节中，类似这样：

```
/etc/unbound/unbound.conf
server:
  ...
  setting: value
  ...
```

注意：请确保你的配置文件中已经设置了`do-daemonize: no`,否则`unbound.service`会无法启动.

本地DNS服务器

如果你想要使用*unbound*作为本地DNS服务器，请把resolv.conf中的域名服务器（nameserver）设置到回环地址：`::1`和`127.0.0.1`。你可能想要让你的配置Domain name resolution#覆盖 /etc/resolv.conf。

提示：实现这个目的的一个简便方法是安装openresolv，然后取消文件`/etc/resolvconf.conf`中包含`name_servers="::1 127.0.0.1"`的那一行的注释。然后运行`resolvconf -u`来重新生成`/etc/resolv.conf`。

要了解如何测试设置项，参见Domain name resolution#Lookup utilities。

在把resolv.conf中的设置更改为持久化设置后，请特别注意检查正在使用的服务器是`::1`或`127.0.0.1`。

你还需要对“unbound”进行设置，以使它#转发查询到你所选择的DNS服务器。

访问控制

你可以通过IP地址来指定响应请求的端口。默认监听的是*localhost*。

为了在所有端口上监听，使用以下配置：

```
interface: 0.0.0.0
```

为了通过IP地址来控制可以访问服务器的系统，使用`access-control`选项：

```
access-control: subnet action
```

例如:

```
access-control: 192.168.1.0/24 allow
```

*action*可以是`deny` (drop message), `refuse` (polite error reply), `allow` (recursive ok), or `allow_snoop` (recursive and nonrecursive ok)中的任意一个。默认除了localhost之外的所有东西都会被拒绝。

使用DNS over TLS进行转发

为了使用这个功能，你需要设置`tls-cert-bundle`选项来指定本地系统的根证书认证包，以使得unbound可以转发TLS请求并指定允许DNS over TLS的服务器数量。

对每个服务器你都需要用 @ 来指定连接的端口，同时你也要用 # 来指明它的域名是什么。虽然它看起来像注释，the hashtag name allows for the TLS authentication name to be set for stub-zones and with `unbound-control forward control` command。在 @ 和 # 之间不应该有空格。

```
/etc/unbound/unbound.conf
...
server:
...
       tls-cert-bundle: /etc/ssl/certs/ca-certificates.crt
...
forward-zone:
        name: "."
        forward-tls-upstream: yes
        forward-addr: 1.1.1.1@853#cloudflare-dns.com
```

根域名服务器

为了查询一个没有被缓存成地址的主机，解释器需要从服务器树的根开始、对根服务器进行请求来知道去哪里找到目标地址的顶级域名。Unbound内置了一些根节点，但是推荐你提供一个根节点文件给它以免内置的过于老旧。

首先，告诉*unbound*使用`root.hints`文件：

```
root-hints: root.hints
```

然后把你的*root hints*文件放进*unbound*的配置文件夹。实现这个目标最简单的方法是运行下面的命令：

```
# curl -o /etc/unbound/root.hints https://www.internic.net/domain/named.cache
```

建议每六个月更新一次`root.hints`来保持根服务器列表是最新的。你可以手动完成这个任务，也可以使用Systemd/Timers。详情参见#根域名服务器与systemd timer。

DNSSEC验证

为了使用DNSSEC验证，你需要在`server:`节中添加以下设置来告诉*unbound*服务器根证书文件的位置：

```
/etc/unbound/unbound.conf
trust-anchor-file: trusted-key.key
```

`/etc/unbound/trusted-key.key`是从依赖项dnssec-anchors包所提供的的`/etc/trusted-key.key`复制而来的，它的PKGBUILD按照unbound-anchor(8)生成了`/etc/trusted-key.key`。

如果总的#转发查询设置到了不支持DNSSEC的DNS服务器，那么请确保已经把这些DNS服务器注释掉，否则DNS请求会失败。DNSSEC验证只会在被请求的DNS服务器支持它的时候成功完成。

注意：如果使用DNSSEC，在地址被缓存之前DNS查询时间将会显著增加。

测试DNSSEC

为了测试DNSSEC是否工作，在starting `unbound.service`之后：

```
$ unbound-host -C /etc/unbound/unbound.conf -v sigok.verteiltesysteme.net
```

得到的回应应该是附带`(secure)`字样的ip地址。

```
$ unbound-host -C /etc/unbound/unbound.conf -v sigfail.verteiltesysteme.net
```

这次的回应应该包含`(BOGUS (security failure))`字样。

另外你也可以使用“drill”来测试：

```
$ drill sigok.verteiltesysteme.net
$ drill sigfail.verteiltesysteme.net
```

第一个命令应该返回`NOERROR`的`rcode`；而第二个命令应该返回`SERVFAIL`的`rcode`。

转发查询

如果你只想转发请求到外部的DNS服务器，请跳到#转发所有其余的请求。

允许本地网络使用DNS

使用openresolv

如果你的网络管理器支持openresolv，你可以通过设置来使它提供本地DNS服务器、使用unbound来查询域名。

```
/etc/resolvconf.conf
...
private_interfaces="*"

# Write out unbound configuration file
unbound_conf=/etc/unbound/resolvconf.conf
```

运行`resolvconf -u`来生成文件。

配置unbound读取openresolv生成的文件并允许回应private IP address ranges：

```
/etc/unbound/unbound.conf
include: "/etc/unbound/resolvconf.conf"
...
server:
...
   private-domain: "intranet"
   private-domain: "internal"
   private-domain: "private"
   private-domain: "corp"
   private-domain: "home"
   private-domain: "lan"

   unblock-lan-zones: yes
   insecure-lan-zones: yes
   ...
```

另外你可能想要对私有DNS域名空间禁用DNSSEC：

```
/etc/unbound/unbound.conf
...
server:
...
   domain-insecure: "intranet"
   domain-insecure: "internal"
   domain-insecure: "private"
   domain-insecure: "corp"
   domain-insecure: "home"
   domain-insecure: "lan"
...
```

手动指定DNS服务器

如果你有一个需要DNS请求的本地网络，同时你想要把请求都转发给一个本地的DNS服务器，那么你需要添加这一行：

```
private-address: 本地子网/子网掩码
```

例如:

```
private-address: 10.0.0.0/24
```

注意：你可以使用私有地址来防止DNS劫持攻击。为了达到这个目的，你可能需要允许RFC1918网络(10.0.0.0/8 172.16.0.0/12 192.168.0.0/16 169.254.0.0/16 fd00::/8 fe80::/10)。 Unbound的未来版本可能会默认启用这个功能。

包含本地DNS服务器

为了包含一个本地DNS服务器，以用于转发和反代本地地址，类似下面的一组配置是必要的（请把下面的10.0.0.1替换为本地网络中提供DNS服务的服务器的地址）：

```
local-zone: "10.in-addr.arpa." transparent
```

上面这一行对于让反向查询正常工作是非常重要的。

```
forward-zone:
name: "mynetwork.com."
forward-addr: 10.0.0.1
forward-zone:
local-data: "1.0.0.127.in-addr.arpa. 10800 IN PTR localhost."
```

注意：转发空间和根空间之间的区别是，根空间只有在直接连接到一个authoritative DNS服务器的时候才能正常工作。如果一个BINDDNS服务器提供authoritative DNS，那么这个特性对于来自于它的请求有用——但是如果你在把请求指向一个*unbound*服务器（内部查询都被转发到另一个DNS服务器），那么把这个指向定义为该机器上的根空间是不会起作用的。在这种情况下，你必须把它定义为上面所说的转发空间，因为转发空间可以通过链式查询去到别的DNS服务器上，亦即转发空间可以把请求指向递归DNS服务器。这个区别是很重要的，因为如果你不恰当地使用根空间，你不会得到能够说明问题的错误信息。

你可以通过下面的配置来设定localhost的前向和反向查询：

```
local-zone: "localhost." static
local-data: "localhost. 10800 IN NS localhost."
local-data: "localhost. 10800 IN SOA localhost. nobody.invalid. 1 3600 1200 604800
10800"
local-data: "localhost. 10800 IN A 127.0.0.1"
local-zone: "127.in-addr.arpa." static
local-data: "127.in-addr.arpa. 10800 IN NS localhost."
local-data: "127.in-addr.arpa. 10800 IN SOA localhost. nobody.invalid. 2 3600 1200
604800 10800"
local-data: "1.0.0.127.in-addr.arpa. 10800 IN PTR localhost."
```

转发所有其余的请求

使用openresolv

如果你的网络管理器支持openresolv，你可以通过配置使它提供上游DNS服务器给unbound。

```
/etc/resolvconf.conf
...
# Write out unbound configuration file
unbound_conf=/etc/unbound/resolvconf.conf
```

运行`resolvconf -u`来生成文件。

最后配置unound读取openresolv生成的文件：

```
include: "/etc/unbound/resolvconf.conf"
```

手动指定DNS服务器

为了使本地机器之外的、本地网络外部的默认转发区域使用指定的服务器，请在配置文件中添加一个名字是.的转发区域。在这个例子里，所有的请求都被转发到谷歌的DNS服务器：

```
forward-zone:
  name: "."
  forward-addr: 8.8.8.8
  forward-addr: 8.8.4.4
```

使用

启动unbound

Start/enable `unbound.service` systemd服务。

远程控制unbound

*unbound*安装的时候自带了`unbound-control`工具，利用这个工具我们可以远程控制unbound服务器。它和pdnsd包的pdnsd-ctl命令很类似。

配置unbound-control

在能够使用它之前你需要做下面的事情：

1) 首先，运行：

```
# unbound-control-setup
```

来为你的服务器和客户端生成一个self-signed的证书和private key。生成的文件位于`/etc/unbound`文件夹。

2) 然后，把下面的内容放进`/etc/unbound/unbound.conf`文件。`control-enable: yes`是一定要有的，其余的内容可以按照所需进行调整。

```
remote-control:
   # Enable remote control with unbound-control(8) here.
   # 用unbound-control-setup生成的keys and certificates进行配置。
   control-enable: yes
   # 设定监听哪个地址.
   # give 0.0.0.0 and ::0 to listen to all interfaces.
   control-interface: 127.0.0.1
   # 远程控制用的端口.
   control-port: 8953
   # unbound server key file.
   server-key-file: "/etc/unbound/unbound_server.key"
   # unbound server certificate file.
   server-cert-file: "/etc/unbound/unbound_server.pem"
   # unbound-control key file.
   control-key-file: "/etc/unbound/unbound_control.key"
   # unbound-control certificate file.
   control-cert-file: "/etc/unbound/unbound_control.pem"
```

使用unbound-control

下面是*unbound-control*可以使用的一部分命令：

- 不重置数据的情况下查看统计数据

```
# unbound-control stats_noreset
```

- 把cache dump到stdout

```
# unbound-control dump_cache
```

- 清空cache并且重新加载配置

```
# unbound-control reload
```

请参考unbound-control(8)来了解*unbound-control*支持的操作。

提示与技巧

域名黑名单

你可以打开这个网页adservers，把它的内容保存到`/etc/unbound/adservers`，然后把下面的配置直接添加到unbound配置文件里就可以了：

```
/etc/unbound/unbound.conf
server:
...
  include: /etc/unbound/adservers
```

提示：

- 为了在查询这些hosts的时候返回OK状态指示，你可以更改默认的127.0.0.1重定向，改成重定向到你所控制的服务器并让那台服务器返回空的204回应，参考
- 如果需要把其他格式的hosts文件转换成unbound的格式的，请运行这个命令:

```
$ grep '^0\.0\.0\.0' hostsfile | awk '{print "local-zone: \""$2"\" always_nxdomain"}'
> /etc/unbound/adservers
```

添加一个authoritative DNS服务器

**本文或本章节的事实准确性存在争议。**

原因： 同时运行两个DNS服务器并不一定比只运行一个提供所有功能的DNS服务器更安全。（在 Talk:Unbound 中讨论）

对于想要在一台机器上同时两个DNS服务器（一个是提供验证、递归、缓存功能的DNS服务器，另一个是authoritative DNS服务器）的用户来说，参考NSD的维基页面可能会有所帮助。那个页面提供了一个示范配置。一个服务器专门响应authoritative DNS请求，另一个服务器提供验证、递归、缓存等DNS功能，这样会比一个服务器提供所有功能会更安全。很多用户已经在使用Bind作为DNS服务器，而针对从Bind变成Bind和NSD协同工作的过程的帮助在NSD页面有提供。

WAN facing DNS

通过更改配置文件和服务器所监听的接口（地址）来允许来自本地网络之外的机器的DNS请求进入本地网络（LAN）内的某台特定机器，这个想法是可行的。这个功能对于公开的网站服务器和邮件服务器是非常有用的。这个在bind上已经实现了多年的技

术，通过正确配置防火墙机器上的端口转发——转发这些请求到正确的机器上——也可以在unbound上实现。

根域名服务器与systemd timer

下面是一个systemd服务和timer的示例文件，它用来每隔一个月更新一次`root.hints`，所用的方法与#根域名服务器中的相同：

```
/etc/systemd/system/roothints.service
[Unit]
Description=Update root hints for unbound
After=network.target

[Service]
ExecStart=/usr/bin/curl -o /etc/unbound/root.hints
https://www.internic.net/domain/named.cache
/etc/systemd/system/roothints.timer
[Unit]
Description=Run root.hints monthly

[Timer]
OnCalendar=monthly
Persistent=true

[Install]
WantedBy=timers.target
```

最后Start/enable `roothints.timer` systemd timer就可以了。

疑难解答

有关num-threads的问题

`unbound.conf`的man page提到：

```
     outgoing-range: <number>
           Number of ports to open. This number of file  descriptors  can  be  opened
per thread.
```

网上的一些人建议`num-threads`这个参数应该设置成你的CPU的核心数量。示范配置文件`unbound.conf.example`里关于这个选项只有下面这两行：

```
       # number of threads to create. 1 disables threading.
       # num-threads: 1
```

但是人为地把`num-threads`提高到比`1`就一定会造成*unbound*在启动的时候在log里写一个warning提示说exceeding the number of file descriptors。实际上对于大多数在小型网络或是单机上运行unbound的用户来说，通过让`num-threads`超过`1`来得到性能提升是徒劳的。如果你一定要这么做，那么请参考official documentation。下面这条经验法则应该对你有所帮助：

: *Set `num-threads` equal to the number of CPU cores on the system. E.g. for 4 CPUs with 2 cores each, use 8.*

把`outgoing-range`设置得尽可能大，参考上面的链接来突破总数是`1024`这个限制。这样就会使得unbound可以同时为更多客户端提供服务。1个核心设置`950`，2个

核心设置450，四个核心设置200。num-queries-per-thread最好设置成outgoing-range的一半。

因为outgoing-range是有限制的，同时num-queries-per-thread也因此受到了限制，所以最好在编译的时候带上libevent包，这样就不会有1024限制了。如果你有一个高负荷DNS服务器使得你不得不这样编译，你需要从源码编译unbound而不是直接安装unbound包。

参阅

- Fedora change to Unbound
- Block hosts that contain advertisements

16.4 域名解析

难度 高级 | **篇幅** 22.5 KB | **所属** DNS

本节将讲到

- 名称服务转换（NSS）
- Glibc 解析器
- 查找实用工具
- 解析器性能
- 隐私与安全

通常，一个域名代表了一个 IP 地址，并在域名系统（Domain Name System，DNS）中与其关联。本文将介绍如何配置域名解析和解析域名。

名称服务转换（NSS）

名称服务转换（Name Service Switch，NSS）工具是 GNU C 库（glibc包）的一部分，支持 getaddrinfo(3) API，用于解析域名。NSS 允许系统数据库由单独的服务提供，其搜索顺序可由管理员在 nsswitch.conf(5) 中配置。负责响应域名解析的数据库是 *hosts* 数据库，glibc 为此提供了下列服务：

- *files*（文件）：读取 `/etc/hosts` 文件，参见 hosts(5)
- *dns*：读取 `/etc/resolv.conf` 的 glibc 解析器，参见 resolv.conf(5)

systemd 为主机名解析提供了三项 NSS 服务：

- nss-resolve(8)——缓存 DNS 存根解析器，systemd-resolved 一文中有描述
- nss-myhostname(8)——无需编辑 `/etc/hosts` 即可提供本地主机名解析，网络配置#Local hostname resolution 一文中有描述

- nss-mymachines(8)——为本地 systemd-machined(8) 容器的名称提供本地主机名解析

使用 NSS 解析域名

NSS 数据库可使用 getent(1) 查询。可以使用以下命令通过 NSS 解析域名：

```
$ getent ahosts domain_name（域名）
```

注意：虽然大多数程序使用 NSS 解析域名，但是有些程序可能会直接读取 `/etc/resolv.conf` 和/或 `/etc/hosts`。参见 网络配置#局域网主机名解析。

Glibc 解析器

覆盖 /etc/resolv.conf

网络管理器们会有意覆盖 `/etc/resolv.conf`，具体细节请参见相应文章章节：

- dhcpcd#/etc/resolv.conf
- Netctl#/etc/resolv.conf
- NetworkManager#/etc/resolv.conf
- ConnMan#/etc/resolv.conf

要防止程序覆盖 `/etc/resolv.conf`，还可以通过设置不可变文件属性来为其建立写保护：

```
# chattr +i /etc/resolv.conf
```

提示：如果想要多个进程写入 `/etc/resolv.conf`，可以使用 resolvconf。

使用 nmcli 的替代方案

**本文或本章节可能需要合并到 NetworkManager#/etc/resolv.conf。**

附注： NetworkManager 有专门的章节来介绍这一主题。（在 Talk:域名解析 中讨论）

如果使用 NetworkManager，可以使用 nmcli(1) 为 `/etc/resolv.conf` 设置持久选项。将下面的*Wired*改为连接名称。例如

```
# nmcli con mod Wired +ipv4.dns-options 'rotate,single-request,timeout:1'
```

有关更多选项，请参阅手册 nmcli(1)、nm-settings-nmcli(5) 和 resolv.conf(5)。

限制查找时间

如果您遇到了非常长时间查找主机名的情况（可能是在 pacman 里或者在浏览网页时），定义一个小的超时时间通常能对问题有所帮助，在此之后系统会使用备用域名服务器（nameserver）。为此，请在 `/etc/resolv.conf` 中添加以下内容。

```
/etc/resolv.conf
options timeout:1
```

IPv6 主机名查找延迟

如果在解析主机名时遇到 5 秒延迟的情况，原因可能是 DNS 服务器/防火墙行为不当，只对并行的 A 和 AAAA 请求给出了一个回应。 可以在 `/etc/resolv.conf` 中设置以下选项来修复问题：

```
/etc/resolv.conf
options single-request
```

本地域名

为了能够使用本地机器名称的主机名，而无需使用格式完全合格的域名，请在 `/etc/resolv.conf` 中添加一行本地域名，如：

```
/etc/resolv.conf
search example.org
```

这样当使用 *ssh* 命令时，可以把 `mainmachine1.example.org` 这样的本地主机简单地称为 `mainmachine1`，不过使用 drill 命令时仍需要格式完全合格的域名以便查找。

查找实用工具

要查询特定的 DNS 服务器和 DNS/DNSSEC 记录，可以使用专用 DNS 查询实用工具。这些工具自身实现了 DNS 功能，不使用 NSS。

ldns包 软件包提供了 drill(1) 命令，是一个旨在检索 DNS 信息的工具。

例如，要用 *drill* 查询特定名称服务器（nameserver）获取一个域名的 TXT 记录，可执行：

```
$ drill @nameserver TXT domain（域名）
```

除非指定了一个 DNS 服务器，否则 *drill* 将使用 `/etc/resolv.conf` 中定义的名称服务器。

提示：一些 DNS 服务器附带它们自己的 DNS 查找实用工具。例如：

- knot包 软件包提供了 khost(1) 和 kdig(1) 命令。
- Unbound 有 unbound-host(1)。
- BIND 有 dig(1)、host(1)、nslookup(1) 和一堆 `dnssec-` 工具。
- systemd-resolved 有 resolvectl(1)，它提供了用于DNS 查找的`query`子命令。

解析器性能

Glibc 不缓存查询结果。要实现本地缓存，请使用 systemd-resolved 或者配置一个本地缓存 DNS 服务器将其用作服务器，方法是在 `/etc/resolv.conf`（如果使用 openresolv，则在 `/etc/resolvconf.conf`）中设置 `127.0.0.1` 和 `::1` 为名称服务器。

提示：

- *drill* 或 *dig* 查找实用工具能报告查询时间。
- 路由器通常将自己的缓存解析器设置为网络的 DNS 服务器，从而为整个网络提供 DNS 缓存。
- 如果切换到下一个 DNS 服务器的时间过长，可以尝试减少超时时间。

隐私与安全

传统的 DNS 协议是没有加密的，并且不会考虑传输的机密性、完整性和身份验证问题，所以如果您在使用不受信任的网络或者有恶意行为的网络服务供应商（ISP），您的 DNS 查询可能会被窃听并响应他人的操纵。此外，DNS 服务器可进行 DNS 劫持。

您需要相信您的 DNS 服务器处理您的查询时能够保密。网络服务供应商和第三方会提供 DNS 服务器。 另外您也可以运营自己的递归名称服务器，但这需要花费更多精力。 如果您在不受信任的网络中使用 DHCP 客户端，请一定要设置静态名称服务器，以避免使用和受制于任意 DNS 服务器。 为了保证您与远程 DNS 服务器的通信安全，您可以使用加密协议，例如 DNS over TLS（RFC 7858）、DNS over HTTPS（RFC 8484）或 DNSCrypt，前提是上游服务器和您的解析器都支持该协议。 另一种方法是使用专用软件来加密和解密通信，例如 stunnel。 要验证响应是否来自权威的名称服务器，您可以验证 DNSSEC 记录，前提是上游服务器和您的解析器都支持它。

虽然可以使用加密 DNS 解析器，但浏览器在请求网站证书时仍会泄漏服务器名称指示（Server Name Indication, SNI）中的域名。可以使用 Wireshark 过滤器 `tls.handshake.extensions_server_name_len > 0` 或者下面的命令检查这种泄漏。建议的解决方案是使用ECH (Encrypted Client Hello)，这是 TLS 1.3 协议的扩展。

```
# tshark -p -Tfields -e tls.handshake.extensions_server_name -Y
'tls.handshake.extensions_server_name_len>0'
```

应用级 DNS

值得注意的是，一些客户端软件，例如主流的网页浏览器，都在开始实现 DNS over HTTPS 功能。虽然一般来说，提供加密查询可以视为一种善举，但这也同时意味着软件可以绕过系统解析器配置，另辟蹊径进行查询。

Firefox 提供了配置选项来启用或禁用 DNS over HTTPS 和选择 DNS 服务器。Mozilla 设立了可信递归解析器 (TRR) 计划，并提供了默认提供商的透明度信息。这里要注意，Firefox 支持并自动为 TRR 提供商启用 ECH，请参阅 Firefox/隐私#加密客户端问候。

Chromium 会检查用户的系统解析器，如果得知系统解析器地址也提供 DNS over HTTPS 支持，则启用 DNS over HTTPS 功能。请参见这篇博文获取更多信息，了解如何禁用 DNS over HTTPS。

Mozilla 提议，如果系统解析器无法解析域名 `use-application-dns.net`，就普遍禁用应用级 DNS。目前，这一功能仅在 Firefox 中实现了。

遗忘式 DNS

遗忘式 DNS（Oblivious DNS，ODNS；RFC:9230）是一套解决了若干 DNS 关切隐私问题的系统。参见 Cloudflare 的文章（英文）获取更多信息。

第三方 DNS 服务

注意：在使用第三方 DNS 服务前，请检查其隐私政策，了解其如何处理用户数据。用户数据是有价值的，可以出售给其他各方。

现在有各种第三方 DNS 服务可用，其中有一些还有专用软件：

- **cloudflared** — Clondflare DNS over HTTPS 的 DNS 客户端

: https://developers.cloudflare.com/1.1.1.1/dns-over-https/cloudflared-proxy || cloudflared包

- **dingo** — Google DNS over HTTPS 的 DNS 客户端

: https://github.com/pforemski/dingo || dingo-gitAUR

- **opennic-up** — 可以自动更新为反应最迅速的 OpenNIC DNS 服务器

: https://github.com/kewlfft/opennic-up || opennic-upAUR

- **nextdns** — NextDNS 的 DNS-over-HTTPS 命令行客户端

: https://github.com/nextdns/nextdns || nextdnsAUR

可以使用 dnsperftest 从您所在位置来测试主流 DNS 解析器们的性能。dnsperf.com 提供了全球 DNS 服务供应商的基准。

DNS 服务器

DNS 服务器有权威式和递归式。如果两个都不是，则称为**根解析器**，只将所有查询转发到其他的递归名称服务器。根解析器典型用途是在本地主机或网络上引入 DNS 缓存机制。请注意，同样的功能可以通过技术完全成熟的名称服务器来实现。本小节我们比较了可用的 DNS 服务器，如需更详细的比较，请参考维基百科：DNS 服务器软件比较。

名称	软件包	功能兼容性	resolvconf	受支持的协议							
权威式	递归式	缓存式	验证 DNSSEC	DNS	DNSCrypt	DNS over TLS	DNS over HTTPS	DNS over QUIC			
BIND	bind包	是	是	是	是	是	是	否	服务器[1]	服务器	否
CoreDNS	corednsAUR	是	否	是	否	否	是	否	是	是	否
DNS-over-HTTPS	dns-over-https包	否	否	否	否	?	?	否	否	是	否
Deadwood (MaraDNS recursor)	maradnsAUR	否	是	是	否	否	是	否	否	否	否
dnscrypt-proxy	dnscrypt-proxy包	否	否	是	否	否	服务器	解析器	否	是	?
dnsmasq	dnsmasq包	部分[2]	否	是	是	是	否	否	否	否	否
dnsproxy	dnsproxyAUR	否	否	是	否	否	是	是	是	是	是
Knot Resolver	knot-resolver包	否	是	是	是	否	是	否	是	服务器	否
pdnsd	pdnsd包	部分[2]	是	永久	否	是	是	否	否	否	否
PowerDNS Recursor	powerdns-recursor包	否	是	是	是	是	是	否	部分	否	?
Rescached	rescached-gitAUR	否	否	是	否	是	是	否	是	是	否
RouteDNS	routedns-gitAUR	否	否	是[3]	否	否	是	否	是	是	是
SmartDNS	smartdns包	否	否	是	否	?	是	否	解析器	解析器	否
Stubby	stubby包	否	否	否	是	否	服务器	否	解析器	否	否
systemd-resolved	systemd包	否	否	是	是	是	解析器和有限的服务器	否	解析器	否	否
Unbound	unbound包	部分	是	是[3]	是	是	是	服务器	是	服务器	否

1. BIND 可以为 DNS over TLS 和 DNS over HTTPS 提供服务（请参阅 tls{} 和 listen-on)），但尚无法将查询转发到 DNS over TLS/DNS over HTTPS 上游。

dig 工具可以通过 DNS over TLS 和 DNS over HTTPS（使用 `+tls` 和 `+https` 选项）进行查询，但没有任何证书检查。

2. 根据维基百科：对权威式名称服务器功能支持有限，是旨在为内部网络使用而不是公共互联网使用。
3. 支持使用 Redis 后端进行持久缓存。

仅权威式服务器

名称	软件包	DNSSEC	地理负载均衡
gdnsd	gdnsd包	否	是
Knot DNS	knot包	是	是
MaraDNS	maradnsAUR	否	?
NSD	nsd包	是	否
PowerDNS	powerdns包	是	是

条件转发

在查询特定域名时，可以使用特定的 DNS 解析器。这在连接到虚拟专用网络（VPN）时特别有用，这样，对 VPN 网络的查询由 VPN 的 DNS 解析，而对普通互联网的查询仍由标准 DNS 解析器解析（分流）。这也可以用于本地网络中。

要实现这个目的，您需要使用本地解析器，因为 glibc 并不支持这样做。

在动态环境中（笔记本和某种程度的台式机），需要根据所连接的网络来配置解析器。最好的方式是使用 openresolv，因为它支持多个订阅器。一些网络管理器会通过 openresolv 或者直接通过配置解析器来支持此功能。NetworkManager 支持条件转发而无需 openresolv。

注意：虽然您可以使用其他条件进行转发（比如源 IP 地址），但是“条件转发”一名似乎更用于描述“域名查询”这一条件。

另请参阅

- Linux 网络管理员指南（英文）
- Debian 手册（英文）
- RFC:7706 - 通过在回环上运行一个根服务器来减少对根服务器的访问时间（Decreasing Access Time to Root Servers by Running One on Loopback，英文）
- 域名系统概览（英文） - DNS 有关图表
- 备用 DNS 服务
- What does DNS stand for

服务器

把服务放到自己的机器上，难的不是装，而是装好之后怎么管住访问、出问题怎么查。本章四节正好分处这两端：YOURLS 和 FHEM 只涉及安装、设置或配置、启动，篇幅短、流程常规，读完第零册即可照做，时间紧可先跳过；Git 服务器与 Minecraft/Java 版服务端值得精读，前者的重点在访问控制与 Web 界面，后者篇幅主要落在配置细节、可替代服务端与疑难解答上。

最容易踩的坑是以为选对服务端就算完工。协议通了只是起点，真正难的是限制访问，以及出故障时逐条排查。

17.1　YOURLS

难度 入门 | **篇幅** 1.6 KB | **所属** 服务器

本节将讲到

- 安装
- 设置

YOURLS 是用 PHP 编写的自托管链接缩短程序服务。

安装

安装 yourlsAUR 包。

设置

需要 MySQL 数据库和用户才能完成 `/etc/webapps/yourls/config.php` 处的配置文件。

在同一个配置文件中，将 `YOURLS_SITE` 设置为您的短域名 `https://*example.org*` 或子域名 `http://*example.org*/*s*`，但始终不带 `/` 尾随。

设置一个随机的 cookie 密钥，例如使用 `pwgen 50 1` 和 admin 用户的用户/密码组合。您可以编写哈希或纯文本，但是在后一种情况下，web 服务器需要 `rw` 权限才能对其进行 hash。

17.2　FHEM

难度 入门 | **篇幅** 1.8 KB | **所属** 服务器

本节将讲到

- 安装
- 配置
- 启动

FHEM (TM) 是在 GPL 下分发的 Perl 服务器，用于使房屋自动化。它可以用于自动执行家庭中的一些常见任务，例如开关灯/百叶窗/暖气等，并记录温度/湿度/功耗等。

安装

安装 fhemAUR 软件包。

配置

从版本 5.7 开始，FHEM 软件包对其文件使用不同的目录布局：

- `/usr/share/fhem` Perl 模块和静态内容
- `/var/lib/fhem` 日志和状态文件，动态内容
- `/etc/fhem.cfg` 主配置文件

如果您有旧版配置，请通过将这些行放入 `/etc/fhem.cfg` 来调整路径

如果您想从 Web 前端编辑配置，请确保 `fhem` 用户对 `/etc/fhem.cfg` 具有写权限。

```
/etc/fhem.cfg
attr global logfile /var/lib/fhem/fhem-%Y-%m.log
attr global modpath /usr/share/fhem
attr global statefile /var/lib/fhem/fhem.save

[...]

define Logfile FileLog /var/lib/fhem/fhem-%Y-%m.log fakelog

define autocreate autocreate
attr autocreate filelog /var/lib/fhem/%NAME-%Y.log

define eventTypes eventTypes /var/lib/fhem/eventTypes.txt
```

请访问 FHEM 参考文档以获取更多信息。

启动

只需启动/启用 `fhem.service`。

17.3 Git 服务器

难度 进阶 | **篇幅** 9.5 KB | **所属** 服务器

本节将讲到

- 协议
- 访问控制
- Web 界面

本文概述了如何托管 Git 服务器。有关更多信息，请参阅 Pro Git 书籍的 Git on the Server 一章。

协议

请参阅Git on the Server - The Protocols，了解详细说明以及优缺点。

普通（General）

设置 git 服务器 的分步指南设置 git Server 的分步指南描述了如何在 Arch 上设置不安全的服务器。

默认情况下，git 用户已过期（"您的账户已过期，请联系您的系统管理员"）。使用 chage 可移除过期条件，举例如下：

```
# chage -E -1 git
```

SSH

您只需要设置一个 SSH 服务器.

您可以进一步确保 SSH 用户账户的安全，只允许在该用户账户上执行推拉命令。方法是用 git-shell 代替默认登录 shell。请参阅设置服务器。

在 Arch 上，使用 #General 中的说明和本节(#SSH)中的说明创建 git 服务器时，需要采取以下额外步骤：

1. 更改主目录： 为了让 ssh 能够读取 `/srv/git/.ssh/authorized_keys`，需要将 `/etc/passwd` 中 git 的主目录从`/`更改为 `/srv/git`。
2. home目录更改之时更改基本路径： 如果仓库是从 git 的主目录提供服务的，为了让 git 能够为仓库提供服务，需要将`git-daemon\@.service`中的`--base-path`改为 `/srv/git`。

Dumb HTTP

这里的 "Dump"是指只有 WebDAV 参与了拉取和推送。

nginx

Follow the basic WebDAV instructions for nginx. Pushing via WebDAV also requires Locking. Here is an example location block:

```
/etc/nginx/nginx.conf
location /repos/ {
        auth_basic "Authorized Personnel Only!";
        auth_basic_user_file /etc/nginx/htpasswd;
        dav_methods PUT DELETE MKCOL COPY MOVE;
        dav_ext_methods PROPFIND OPTIONS LOCK UNLOCK;
        dav_access user:rw group:rw all:r;
        dav_ext_lock zone=general;
        create_full_put_path on;
        client_body_temp_path /tmp;
    }
```

Note the `dav_ext_lock zone`. Add the specified locking zone to the http section of your config:

```
/etc/nginx/nginx.conf
dav_ext_lock_zone zone=general:10m;
```

Now do the ususal steps when preparing a git repo for the server:

- `git clone --bare /path/to/myrepo myrepo.git`
- copy the bare repo to the server
- run `git update-server-info` in the bare repo
- chown the repo to be owned by http:http

You might have noticed that I added HTTP Basic Authentication to have at lease some means of access control. Everyone who has an password entry in the htaccess file can push.

Now you can clone as usual:

```
$ git clone https://www.example.com/repos/myrepo.git
Cloning into 'myrepo'...
$
```

Make some changes, add, commit, and push:

```
$ git push origin main
error: Cannot access URL https://www.example.com/repos/myrepo.git/, return code 22
fatal: git-http-push failed
error: failed to push some refs to 'https://www.example.com/repos/myrepo.git'
```

Oh noes! For some reason PROPFIND reports 401 Unauthorized and that's all. Nothing in the nginx error logs. Appearently the git client has a problem passing the username and password for all subsequent requests. Running a git credential cache does not help. The only solution that works so far is editing the ~/.netrc (obviously git uses curl for http):

```
~/.netrc
machine www.example.com
login git
password topsecret
$  > git push origin main
Fetching remote heads...
 refs/
 refs/heads/
 refs/tags/
updating 'refs/heads/main'
 from 03f8860418facfbecedd5e0a81b480131b31bcba
 to   ec5536091e31ebf172a34c6d1ebddfc36e3bd3a6
   sending 3 objects
   done
Updating remote server info
To https://www.example.com/repos/myrepo.git
  0318860..ec55560  main -> main
```

Don't even think to specify the clone URL as `https://username:password@www.example.com/repos/myrepo.git`. This works for the initial clone but for a subsequent push you get an error message in your error log stating that the destination URL is handled by a different repository.

Smart HTTP

The git-http-backend(1) is a CGI program, allowing efficient cloning, pulling and pushing over HTTP(S).

Apache

设置非常简单，只需安装Apache HTTP 服务器，并启用 `mod_cgi`、`mod_alias` 和 `mod_env`，当然还有 git包。

运行基本设置后，在 Apache 配置文件中添加以下内容，该文件通常位于：

```
/etc/httpd/conf/httpd.conf
<Directory "/usr/lib/git-core">
    Require all granted
</Directory>

SetEnv GIT_PROJECT_ROOT /srv/git
SetEnv GIT_HTTP_EXPORT_ALL
ScriptAlias /git/ /usr/lib/git-core/git-http-backend/
```

这假设的是：您的 Git 仓库位于 `/srv/git`，并且您希望通过以下方式访问它们：`http(s)://your_address.tld/git/your_repo.git`。

注意：确保 Apache 可以读写您的版本库。

有关更详细的文档，请访问以下链接：

- https://git-scm.com/book/en/v2/Git-on-the-Server-Smart-HTTP
- https://git-scm.com/docs/git-http-backend

Git

The Git protocol is not encrypted or authenticated, and only allows read access.

The Git daemon (git-daemon(1)) can be started with `git-daemon.socket`.

The service uses the `--export-all` and `--base-path` parameters to serve all repositories placed in `/srv/git/`.

访问控制

对于保险（fine-grained）的访问控制，可采用以下解决方案：

- **Gitolite** — 一个 Git 访问控制层，用 Perl 编写。

: https://github.com/sitaramc/gitolite || gitolite包

- **Gitosis** — 一个用于托管 Git 仓库的软件，用 Python 编写。

: https://github.com/tv42/gitosis || gitosis-gitAUR

请注意，如果您愿意为所有应该访问版本库的人创建用户账户，并且不需要 git 对象（如分支）级别的访问控制，也可以使用标准的文件权限来进行访问控制。

Web 界面

简易 web 应用程序

- Gitweb——Git 自带的默认 Web 界面
- **cgit** — 一个用纯 C 语言编写的 git 网页界面。

: https://git.zx2c4.com/cgit/ || cgit包

高级 web 应用程序

- **Forgejo** — 自托管轻量级软件。Gitea 的社区管理分支。

: https://forgejo.org || forgejo包

- **Gitea** — 无偿自托管 Git 服务。它最初是 Gogs 的一个社区管理分支，但在 2022 年由 Gitea Limited 以商业模式拥有。

: https://gitea.io || gitea包

- **GitLab** — 用 Ruby 编写的项目管理和代码托管应用程序。

: https://gitlab.com/gitlab-org/gitlab-ce || gitlab包

- **Gogs** — 用 Go 编写的自助托管 Git 服务。

: https://gogs.io || gogsAUR

17.4 Minecraft/Java 版服务端

难度 进阶 | **篇幅** 12.5 KB | **所属** 服务器

本节将讲到

- 安装
- 配置
- 可用于替代的服务端
- 提示和技巧
- 疑难解答

Minecraft 是一个多人游戏。它使用客户端-服务端模型，游戏本体是一个可以独立游玩的客户端，连接到服务器时也可以和其他玩家一起游玩。

注意：第三方可以运行 Minecraft 服务端。您应当阅读它们的隐私政策来知晓它们如何处理您的数据。一些服务器需要登录第三方账户，还有一些支持通过小额交易购买物品。即使这可能违反了 Minecraft 最终用户许可协议 (“EULA”)，具体取决于服务器销售的内容。

安装

Java 版 Minecraft 服务端可通过 minecraft-serverAUR 包来安装。它附带一个 systemd 单元文件和一个小巧的控制脚本。

请参阅#可用于替代的服务端来了解一些可替代 Minecraft 服务端的应用的简介。

配置

安装过程会创建 `minecraft` 用户和用户组。出于安全原因，我们推荐创建一个特殊的 minecraft 用户。在一个无特权用户下运行 Minecraft 可以使任何攻破您的服务器的人仅获得该用户的权限，而您自己的则完好无损。 不过，您需要安全地将您的用户添加到 `minecraft` 组，并授予该组 `/srv/minecraft`（默认）目录的写权限以修改 Minecraft 服务端配置。确保`/srv/minecraft` 目录下的所有文件均属于 `minecraft` 用户，或者让其他用户拥有这些文件的读写权。如果服务端无法访问某些文件且没有足够的权限将错误消息写入日志，那么它会出错。

这个软件包提供了一个 systemd 服务和一个计时器来自动备份。默认情况下，备份位于服务器根目录的 `backup` 文件夹下。为节省磁盘空间，服务端只会保留 10 个最新的备份（可通过 `KEEP_BACKUPS` 进行配置）。相关的 systemd 文件是 `minecraftd-backup.timer` 和 `minecraftd-backup.service`。您可凭喜好编辑它们，例如遵循自定义的备份周期。

启动服务端

您可以通过 systemd 或命令行来启动服务端。无论使用哪种方式，服务端都被封装在 `minecraft` 用户的 tmux 会话中。使用 systemd，你可以启动/启用其包含的 `minecraftd.service`。此外，也可以执行：

```
# minecraftd start
```

接受 EULA

要运行 Minecraft 服务端，您必须接受 EULA（End User License Agreement，最终用户许可协议），您只需在安装后接受即可。EULA 文件由服务端软件包创建，位于 `/srv/minecraft/eula.txt`。请编辑该文件来声明您接受 EULA。您只需编辑：

```
eula=false
```

将其值改为 `true`。以下是一份被接受的 EULA 的样例：

```
/srv/minecraft/eula.txt
#By changing the setting below to TRUE you are indicating your agreement to our EULA
(https://account.mojang.com/documents/minecraft_eula).
#Sat Sep 11 11:11:11 PDT 2011
eula=true
```

防火墙配置

`server.properties` 包含三个设置用于确定您使用的端口：

`server-port` 用于确定监听传入连接所使用的 `TCP` 端口，默认为 `25565`。

`query.port` 用于确定共享游戏信息所使用的 `UDP` 端口，默认为 `25565`。

注意：要注意 `server.port` 是 TCP 而 `query.port` 是 UDP，它们可以共用同一端口。要启用 query，您还需要单独设置 `enable-query=true`。

`rcon.port` 用于确定远程连接管理终端所使用的 TCP 端口， 前提是您需要允许远程连接，默认端口为 `25575`。要启用 rcon，您需要单独设置 `enable-rcon=true` 和 `rcon.password=...`。

您至少应当开启 `server-port` 以允许传入连接，建议允许 query 并开启 `query.port`。此外，允许远程控制会带来风险，开启前请三思。

以上是官方服务端的配置方法。若您使用其他替代的服务端，请阅读它们的文档以获取更多细节及配置方法。

参阅 和 以获取更多信息。

服务端管理脚本

您可以使用软件包提供的 `minecraftd` 脚本来容易地管理服务端。它可以执行一些简单的诸如 `start`、`stop`、`restart` 的基本命令或使用 `console` 连接到会话。更进一步地，您可以使用 `status` 查看状态信息，用 `backup` 备份服务端的世界目录，用 `restore` 从备份恢复您的世界或使用 `command *要执行的命令*` 在服务器控制台上运行单条命令。

注意：关于服务器控制台（可通过 `minecraftd console` 访问），请知晓您可以使用 `ctrl+b d` 快捷键退出任意 tmux 会话。

调整

要调整默认设置（如最大 RAM、线程数等），请修改 `/etc/conf.d/minecraft`。

例如，更多的高级用户可能会想将 `IDLE_SERVER` 设为 `true`，这会使服务端在 `IDLE_IF_TIME` 定义的时长（默认为 20 分钟）内没有玩家后暂停。当服务端被暂停时，一个 `idle_server` 会使用来自 nmap包 的 ncat(1)（或其他 netcat 实现）监听 Minecraft 端口，当有连接传入时，会立即启动服务端。这可能会推迟暂停后的第一个连入请求，但它可以显著降低 CPU 和内存占用，实现更合理的资源及功耗水平。

注意：如果在第一次运行时启用该选项，`/srv/minecraft/eula.txt` 将不会被创建，初次启动时您需要禁用它。

可用于替代的服务端

Spigot（区别于Craftbukkit）

Spigot（水龙头服）是世界上使用最广泛的修改版服务端。您可使用 spigotAUR 安装它。spigot 的 PKGBUILD 在 minecraft-serverAUR 上构建，这意味着它具有自己的 systemd 单元文件、spigot 脚本和相应的脚本配置文件。其二进制文件名为 `spigot`，与 `minecraftd` 的命令相同。配置文件位于 `/etc/conf.d/spigot`。

确保您已阅读#配置章节，并无论何处都将所有的 `minecraftd` 替换为 `spigot` 。

它与 Bukkit（水桶服）有点故事，且在 bukkit 陨落后越发受到欢迎。

Cuberite

Cuberite 是一个高性能且可扩展的服务端，由 C++ 和 Lua 编写 ，其性能远超原版 Minecraft 服务端，但其与最新的 Minecraft 客户端不完全兼容（一些游戏特性可能会缺失或不工作）。

Cuberite Minecraft 服务端可作为 cuberiteAUR 安装，其在 `8080` 端口默认提供一个简单的 web 界面，可以通过浏览器设置大部分的服务器选项，cuberite 的 PKGBUILD 在 minecraft-serverAUR 的文件上构建，这意味着 cuberite 服务端提供了它自己的 systemd 单元文件、cuberite 脚本和相应的脚本配置文件。其二进制文件名为 `cuberite`，与 `minecraftd` 的命令相同，配置文件位于 `/etc/conf.d/cuberite`。

确保您已阅读#配置并将您遇到的所有的 `minecraftd` 替换为 `cuberite`。

PaperMC

PaperMC（纸服）是一个 Minecraft 服务端，兼容 Spigot 的插件，旨在提供更佳性能。通过 papermcAUR 安装该服务端。

确保您已阅读#配置并将您遇到的所有 `minecraftd` 替换为 `papermc`。

Forge

Forge 是一个使用广泛的 Minecraft 模组 API。有以下服务端软件包可用：

- forge-serverAUR 最新版本的 Minecraft
- forge-server-1.15.2AUR for Minecraft 1.15.2
- forge-server-1.14.4AUR for Minecraft 1.14.4
- forge-server-1.12.2AUR for Minecraft 1.12.2

- forge-server-1.11.2AUR for Minecraft 1.11.2
- forge-server-1.10.2AUR for Minecraft 1.10.2
- forge-server-1.9.4AUR for Minecraft 1.9.4
- forge-server-1.8.9AUR for Minecraft 1.8.9
- forge-server-1.7.10AUR for Minecraft 1.7.10

确保您已阅读#配置并将遇到的所有 `minecraftd` 替换为 `forged` （若是旧版则还需加上版本号，形同 `forge-x.x.xd`）。

Fabric

Fabric 是为 Minecraft 开发的一个轻量化、实验性的模组工具链。通过 fabric-serverAUR 安装其服务端软件包。

确保您已阅读#配置并将遇到的所有 `minecraftd` 替换为 `fabricd`。

Quilt

Quilt 是一个主要为 Minecraft 开发、开源的、社区驱动的模组工具链。通过 quilt-serverAUR 安装其服务端软件包。

确保您已阅读#配置并将遇到的所有 `minecraftd` 替换为 `quiltd`。

Quilt 是 Fabric 的一个分支，意味着其可以向下兼容一些 Fabric 模组。

提示和技巧

Minecraft 服务器端口

Minecraft 服务端默认在 `25565` 端口上运行，当输入的地址不带端口时默认使用该端口。

大部分 Minecraft 服务器提供商需要购买高级服务以使用默认的 Minecraft 端口。因此若您使用的端口不是 `25565`，您需要在域名或地址后加上一个冒号（**:**）和您的服务器为游戏分配的端口。例如，若您的地址是 `43.12.122.96`，端口是 `28543`，那么您需要连接到`43.12.122.96:28543`。

疑难解答

日志

Screen 日志存储在 `/tmp/spigot_spigot_command_dump.txt` 文件中。如果 *systemctl* 无法启动服务,请检查 *screen* 日志。

Journal 日志存储在 `spigot.service`

另请参阅

- 一些打包好的服务端提供从自动备份到多线程服务器等所有功能，请参阅 Server Wrappers 以了解更多信息。然而 AUR 提供的管理脚本应当可以满足大部分需求。
- 您可能会想要设置一个 systemd 定时器，例如与 mapper 一起使用以为您的世界定时生成地图。
- 确保定时备份地图，可以使用提供的管理脚本（见阅#配置或 rsync）。
- 另见架设 Java 版服务器。

第 18 章　共 4 节　89 KB

网络管理器

本章解决同一台 Arch 机器上由谁托管网络：无线网卡的扫描与连接、有线接口的地址与路由、配置的持久化，以及断网时从哪里查起。

建议先读《Systemd-networkd》和《Netctl》：两节篇幅最长，从基本用法、配置文件讲到故障排查，是主线；《ConnMan》可略读，作为集中托管连接的替代方案；《Wifi Radar》最短，只适合临时连一次无线，可跳过。

最常见的误解是同时启用多套网络管理器，它们会争抢地址与路由，表现为网络时通时断；选定一种后要关掉其余服务，排查时先确认生效的是哪一个。

18.1　Wifi Radar

难度 入门 | **篇幅** 1.4 KB | **所属** 网络管理器

本节将讲到

- 介绍
- 安装
- 配置
- 小技巧
- 附加资料

介绍

Wifi雷达（Wifi Radar）\[失效链接 2020-08-06 ⓘ\]是一个小巧的GUI程序，你可以用它很容易的切换无线网络连接。

安装

要安装Wifi雷达，只需执行：

```
# pacman -S wifi-radar
```

配置

```
# visudo
```

增加下面一行：

```
你的用户名      ALL=(ALL) NOPASSWD: /usr/sbin/wifi-radar
```

按 ESC, 接着冒号 (:), 然后按 x, 然后按 ENTER 保存并退出。

增加 *wifi-radar* 到 /etc/rc.conf 中的DAEMONS组。

从你的程序菜单运行wifi-radar。如果你想查看可用的网络连接，执行*sudo wifi-radar*。

小技巧

1. 你可能需要编辑 /etc/conf.d/wifi-radar，设置适合你的特殊网络接口。
2. 有的用户可能要设置‘Ifup required’为开启，以便wifi-radar能正常工作。

附加资料

Wireless network configuration - 无线网络安装wiki。

18.2 ConnMan

难度 高级 | **篇幅** 19.6 KB | **所属** 网络管理器

本节将讲到

- 安装
- 用法
- 提示与技巧
- 疑难杂症
- 另请参见

ConnMan 是一个命令行网络管理器，为嵌入式设备和快速响应设计。ConnMan 通过插件扩展，但内置了 DHCP 与 NTP 支持。

安装

安装 connman包 软件包。 wpa_supplicant包、bluez包 与 openvpn包 是对应 Wi-Fi、蓝牙与 VPN 功能的可选依赖。

在启用 `connman.service` 之前，确保禁用所有已安装的网络配置。

ConnMan 自带 connmanctl(1) 命令行界面。各种 #前端也可以使用。

前端

- **cmst** — ConnMan 的 QT GUI。

: https://github.com/andrew-bibb/cmst || cmstAUR

- **connman-ncurses** — ConnMan 的简单 ncurses UI；没有实现 ConnMan 的全部功能，但是可以用（有没有 X 都可以运行），参考 wiki。

: https://github.com/eurogiciel-oss/connman-json-client || connman-ncurses-gitAUR

- **ConnMan-UI** — GTK3 客户端托盘程序。

: https://github.com/tbursztyka/connman-ui || connman-ui-gitAUR

- **connman_dmenu** — dmenu 客户端/前端。

: https://github.com/taylorchu/connman_dmenu || connman_dmenu-gitAUR

- **Econnman** — Enlightenment 桌面托盘程序。

: https://www.enlightenment.org || econnmanAUR

- **LXQt-Connman-Applet** — LXQt 桌面托盘程序。

: https://github.com/lxqt/lxqt-connman-applet || lxqt-connman-appletAUR

- **connman-gtk** — GTK 客户端。

: https://github.com/jgke/connman-gtk || connman-gtkAUR

- **gnome-extension-connman** — ConnMan 的 Gnome3 扩展；如果不安装 connman-gtk，提供的功能很少。

: https://github.com/jgke/gnome-extension-connman || https://extensions.gnome.org/extension/981/connman-extension/

用法

ConnMan 自带 `connmanctl` 命令行界面，参考 connmanctl(1)。 如果你不提供任何命令，直接运行 `connmanctl`，将会启动一个交互式终端。

ConnMan 会自动处理有线连接。

Wi-Fi

启用与禁用 wifi

执行 `connmanctl technologies` 来检查 wifi 是否开启，查看类似 `Powered: True/False` 的输出。 执行 `connmanctl enable wifi` 来开启 wifi，或者执行 `connmanctl disable wifi` 来禁用 wifi。 其他启用 wifi 的方法包括使用笔记本电脑上的 `Fn` 键，或者执行 `ip link set <interface> up`。

连接到公开的接入点

`connmanctl` 接受简单的 *technologies* 名称来扫描网络，要扫描附近的 Wi-Fi 网络：

```
$ connmanctl scan wifi
```

在扫描之后，列出可用的网络（示例输出）：

```
$ connmanctl services
*AO MyNetwork               wifi_dc85de828967_68756773616d_managed_psk
    OtherNET                wifi_dc85de828967_38303944616e69656c73_managed_psk
    AnotherOne              wifi_dc85de828967_3257495245363836_managed_wep
    FourthNetwork           wifi_dc85de828967_4d7572706879_managed_wep
    AnOpenNetwork           wifi_dc85de828967_4d6568657272696e_managed_none
```

要连接到公开的网络，使用以 `wifi_` 开头的第二个参数:

```
$ connmanctl connect wifi_dc85de828967_4d6568657272696e_managed_none
```

提示：网络名称可以用 tab 补全。

你现在应该已经连上网络了，使用 `connmanctl state` 或 `ip addr` 来检查。

连接到受保护的接入点

你需要提供更多的信息，起码是密码或者密码短语，给 ConnMan 守护程序来连接受保护的接入点。

这一节的命令展示了如何在交互模式执行 `connmanctl`，只有交互模式下才能使用 `agent` 命令。要开启交互模式，只需要输入：

```
$ connmanctl
```

接下来的操作和之前的类似，首先扫描所有的 Wi-Fi *technologies*：

```
connmanctl> scan wifi
```

列出服务：

```
connmanctl> services
```

现在你需要注册一个 agent 来处理用户请求，命令是：

```
connmanctl> agent on
```

现在连接到受收保护的服务，使用 tab 补全来输入 wifi_ 服务，比如要连接到上面例子中的 OtherNET， 输入：

```
connmanctl> connect wifi_dc85de828967_38303944616e69656c73_managed_psk
```

Agent 会向你询问守护进程连接到网路需要的一切信息，这些信息会依据与你要连接的网络类型的不同而变化。 Agent 也会像下面一样显示有关信息的额外数据：

```
Agent RequestInput wifi_dc85de828967_38303944616e69656c73_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory ]
  Passphrase?
```

提供要求的信息，在这个例子里是密码短语，然后输入：

```
connmanctl> quit
```

如果你提供的信息正确，那么你应该已经连接到受保护的接入点了。

使用 iwd 代替 wpa_supplicant

ConnMan 可以使用 iwd包 连接无线网络。Community 中提供的的软件包已经支持使用 iwd包 来连接无线网络。因为只要 connman包 找到 wpa_supplicant包 就会启动它，这里建议卸载 wpa_supplicant包。

目前 iwd包 的 `-i` 选项似乎会导致 connman包 无法找到 Wi-Fi 接口。

创建下面两个服务文件，可以让 connman包 无论是否安装了wpa_supplicant包，都一律使用 iwd包 来连接无线网络。

```
/etc/systemd/system/iwd.service
[Unit]
Description=Internet Wireless Daemon (IWD)
```

```
Before=network.target
Wants=network.target

[Service]
ExecStart=/usr/lib/iwd/iwd

[Install]
Alias=multi-user.target.wants/iwd.service
/etc/systemd/system/connman_iwd.service
[Unit]
Description=Connection service
DefaultDependencies=false
Conflicts=shutdown.target
RequiresMountsFor=/var/lib/connman
After=dbus.service network-pre.target systemd-sysusers.service iwd.service
Before=network.target multi-user.target shutdown.target
Wants=network.target
Requires=iwd.service

[Service]
Type=dbus
BusName=net.connman
Restart=on-failure
ExecStart=/usr/bin/connmand --wifi=iwd_agent -n
StandardOutput=null
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE CAP_NET_RAW CAP_SYS_TIME
CAP_SYS_MODULE
ProtectHome=true
ProtectSystem=true

[Install]
WantedBy=multi-user.target
```

然后启用/开始 `connman_iwd` 服务。

使用 iwd包 代替 wpa_supplicant包 的好处是：ping 时间似乎会更加一致，并且连接会更加可靠。

设置

对于用户经常连接的网络，ConnMan 会自动创建设置与配置文件。这些文件包括了密码短语、essid 和其他信息。配置文件存放在 `/var/lib/connman/` 内，以服务为名的文件夹下。要查看所有的网络配置文件，在 root shell 内执行：

```
# cat /var/lib/connman/*/settings
```

注意：VPN 设置存放在 `/var/lib/connman-vpn/`。

Technologies

ConnMan 将各种硬件接口称为 *Technologies*。

要列出所有可用的 *Technologies*，执行：

```
$ connmanctl technologies
```

只列出 *Technologies* 类型的名字，可以使用这一行命令：

```
$ connmanctl technologies | awk '/Type/ { print $NF }'
```

注意：字段 `Type = tech_name` 里的“tech_name”，就是 `connmanctl` 命令所使用的 technology 类型

要与这些接口交互，必须使用 technology 的类型。要开启或关闭 *Technologies* 可以用：

```
$ connmanctl enable technology_type
```

与：

```
$ connmanctl disable technology_type
```

比如要关闭 wifi：

```
$ connmanctl disable wifi
```

警告：connman 会监听 rfkill 事件。使用 connman 时，尽管硬件锁可能可以正常工作，但基本无法使用 `rfkill` 或 `bluetoothctl` 来开关设备。 请使用 `connmanctl enable|disable`

提示与技巧

避免改变 hostname

ConnMan 默认会更改临时 hostname。与 X authority 一起使用时会有问题：如果用旧的 hostname 生成了 xauth magic coookie 之后，ConnMan 改变了你的 hostname，那么将无法创建新的窗口，会显示诸如 `No protocol specified` 和 `Can't open display: :0.0` 的错误。手动重置 hostname 可以解决这个问题，但根本的解决办法是从一开始就防止 ConnMan 改变你的 hostname。把下面的配置加入 `/etc/connman/main.conf` 即可：

```
[General]
AllowHostnameUpdates=false
```

改变这个文件后记得重启 `connman.service`。

要测试的话，建议一边观察 systemd 日志，一边插拔几次网线接口。

有线网络与无线网络共存时，优先使用有线网络

ConnMan 默认不会优先使用有线网络，这会导致 ConnMan 即使在可以使用有线网络时，也只使用慢速的无线网络。你可以将下面的配置加入 `/etc/connman/main.conf`，让 ConnMan 优先使用有线网络。

```
[General]
PreferredTechnologies=ethernet,wifi
```

互斥连接

ConnMan 允许你同时连接有线与无线网络。这样允许程序在通过 wifi 建立了连接后，再连接到有线网络，通过 wifi 建立的连接也不会中断。但有些人更喜欢同一时间只使用一种网络，将下面的配置加入 `/etc/connman/main.conf` 可以实现这一点：

```
[General]
SingleConnectedTechnology=true
```

连接到 eduroam (802.1X)

在连接到诸如 eduroam 的 WPA2 企业模式网络之前，需要建立单独的配置文件。 比如，创建 `/var/lib/connman/eduroam.config`：

```
eduroam.config
[service_eduroam]
Type=wifi
Name=eduroam
EAP=peap
CACertFile=/etc/ssl/certs/certificate.cer
Phase2=MSCHAPV2
Identity=user@foo.edu
AnonymousIdentity=anonymous@foo.edu
Passphrase=password
```

重启 `wpa_supplicant.service` 和 `connman.service` 来连接到新网络上。

注意：

- 选项是大小写敏感的，比如应该用 `EAP = ttls` 而不是 `EAP = TTLS`。
- 请向提供 eduroam 网络的机构咨询各种设置，比如用户名、密码、`EAP`、`Phase2output`和需要的凭证。

更多信息请参考 connman-service.config(5) 和 Wireless network configuration#eduroam.

避免与本地 DNS 服务冲突

如果你在运行本地的 DNS 服务，在安装了 ConnMan 之后，你很可能会遇到 53 端口（TCP 和/或 UDP）被占用的问题。这是因为 ConnMan 内置的 DNS 代理也会尝试绑定 53 端口。如果你看到了来自 BIND 或 dnsmasq 的日志诸如：

```
named: could not listen on UDP socket: address in use
```

就是这个问题引起的。要确认是哪个应用占用了 53 端口，可以以 root 身份执行 `ss -tulpn`。

要修复这个问题，可以修改 systemd 服务文件，以 `-r` 或 `--nodnsproxy` 选项启动 connmand。 创建文件夹 `/etc/systemd/system/connman.service.d/` 并新增文件 `disable_dns_proxy.conf`：

```
/etc/systemd/system/connman.service.d/disable_dns_proxy.conf
[Service]
ExecStart=
ExecStart=/usr/bin/connmand -n --nodnsproxy
```

确保在新增此文件后 reload systemd 守护程序，并 restart `connman.service` 和你的 DNS 代理。

/etc/resolv.conf

如果你既想知道从 DHCP 获得的 DNS 服务器地址，同时又想保留自定义的 `/etc/resolv.conf`，那么就在上面的文件里追加

`RuntimeDirectory=connman`（如果不需要 `ExecStart`，就去掉 ExecStart 行）。这样 connman 就会把从 DHCP 获得的 DNS 服务器地址写入 `/var/run/connman/resolv.conf`。

记得在增加这个文件之后重载 systemd 守护程序，并重启 `connman.service`和你的 DNS 代理。

屏蔽接口

如果有 Docker 之类的的程序创建了虚拟接口，当网络连接中断时，ConnMan 可能会尝试连接这些虚拟接口，而不是真实的硬件接口。简单的解决办法是将你不想要 ConnMan 连接的接口加入黑名单。 ConnMan 默认会屏蔽以 `vmnet`、`vboxnet`、`virbr` 与 `ifb` 开头的接口，所以配置新的黑名单时也要加入这些接口。

屏蔽接口还可以避免竞态条件：在 systemd/udev 改变接口名称到形如 `enp4s0` 的可预测的网络接口名称之前，connman 可能就会去连接接口。屏蔽惯用（且不可预测）的接口前缀可以让 connman 在接口重命名之后再连接。

把下面的配置加入 `/etc/connman/main.conf`（如果不存在就创建这个文件）：

```
[General]
NetworkInterfaceBlacklist=vmnet,vboxnet,virbr,ifb,docker,veth,eth,wlan
```

重启 `connman.service` 之后，在 Econnman 这样的 GUI 工具中，所有形如 `veth#######` 的接口也都会被隐藏。

疑难杂症

Error /net/connman/technology/wifi: Not supported

目前，connman 不支持用 iwd包 扫描 Wi-Fi 网络，这个功能暂时只能由 `wpa_supplicant` 提供（参考 ）。要用 iwd连接 wifi，enable 并 start `iwd.service`，然后参考 Iwd 中的指示，或者使用#前端的其中之一。为了让 connman 能扫描 Wi-Fi，安装 wpa_supplicant包 软件包，并在停用 `iwd.service` 之后重启`connman.service`。

Error /net/connman/technology/wifi: No carrier

在你使用这条命令启用 wifi 后：

```
$ connmanctl enable wifi
```

如果扫描无线网络时产生了这条错误，这可能是因为一个未解决的 Bug。\[失效链接 2022-09-17 ⓘ\] 如果在满足了这些条件后还未解决，请在禁用其他网络管理器，并重启系统之后再尝试。

也可能仅仅是因为无线接口被 rfkill 关闭了，重启 wpa_supplicant 时有可能会这样。使用 `rfkill list` 来检查。

"Not registered", or "Method "Connect" with signature ... doesn't exist"

在执行命令时你可能会看到这样的错误：

来自 `connmanctl` 提示符：

```
connmanctl> connect service_id
Error /net/connman/service/SSID: Method "Connect" with signature "" on interface
"net.connman.Service" doesn't exist
```

来自终端：

```
# connmanctl connect service_id
Error /net/connman/service/service_id: Not registered
```

这些错误是由于没有运行 agent。在 `connmanctl` 提示符里执行 `agent on` 来启动 agent，然后再试一试。

Error Failed to set hostname/domainname

Connman 可能会因没有 `CAP_SYS_ADMIN` 而无法设置 hostanme 或 domainname。

你需要编辑 `connman.service`（以及其他类似的文件，如 `connman-vpn.service`）来修改 `CapabilityBoundingSet` 行，增加 `CAP_SYS_ADMIN`。

更多细节请参考 `EPERM` 下的 sethostname(2) § ERRORS 或 setdomainname(2) § ERRORS。

连接时出现未知路由

每次连接完成后，日志中可能出现未知的路由，比如：

```
...
connmand: wlp2s0 {add} route 82.165.8.211 gw 10.20.30.4 scope 0 <UNIVERSE>
connmand: wlp2s0 {del} route 82.165.8.211 gw 10.20.30.4 scope 0 <UNIVERSE>
...
```

这可能是由于 ConnMan 在通过检查与 ipv4.connman.net 的连接，来测试网络（在本例里，`82.165.8.211` 就是 ipv4.connman.net 被解析后的 IP地址）。\[失效链接 2022-09-17 ⓘ\] 详细信息，包括 ConnMan 为什么要这么做，以及 ConnMan 除了连接 IP 还传递了什么信息，请参考 ConnMan README。

该行为可以通过在 `/etc/connman/main.conf` 中增加下面的配置来阻止：

```
[General]
EnableOnlineCheck=false
```

这项配置也会导致默认设备状态不会从 READY 切换到 ONLINE。connman.conf(5) 但网络连接是正常的。

也可以用防火墙规则阻止连接检查，这不会影响正常的网络连接（除非有 captive portal）：

```
# ip6tables -A OUTPUT -d ipv6.connman.net -j REJECT
# iptables -A OUTPUT -d ipv4.connman.net,ipv6.connman.net -j REJECT
```

File /proc/net/pnp doesn't exist

如果你在错误日志中看到了这个，这是由 connman 的 bug 引起的 ，可以无视。Bug Report\[失效链接 2022-09-17 ⓘ\]

另请参见

- Git 仓库文档——更多细节文档

18.3 Netctl

难度 高级 | **篇幅** 28.7 KB | **所属** 网络管理器

本节将讲到

- 安装
- 配置
- 用法
- 技巧和窍门
- 故障排除

netctl 是一个基于命令行与配置文件的网络管理器和一个Arch项目。

安装

安装 netctl包 软件包.

netctl 用于自动连接的的 #特殊 systemd 单元需要一些额外的依赖项目。有关更多信息，请参阅该部分。

其他可选依赖如下表所示。

功能	依赖
WPA	wpa_supplicant包
DHCP	dhcpcd包 or dhclient包
wifi-menu	dialog包
PPPoE	ppp包

警告：不要启用并发、导致冲突的网络服务。在启用 *netctl* 服务或配置文件前，先使用 `systemctl --type=service` 命令来确保没有其他网络服务正在运行。

配置

netctl 通过配置文件来管理网络连接和设置自动/手动/按需启动等不同操作模式。

netctl 配置文件存放于 `/etc/netctl/` 文件夹中，而示例配置存放在 `/etc/netctl/examples/` 里。

要使用示例配置文件，只需把它从 `/etc/netctl/examples/` 下复制到 `/etc/netctl/` 下，然后按照自己的需要进行配置。请参考下文中的 #示例配置。创建配置文件所需的第一个参数是网络接口（`Interface`），详情请参考网络配置#网络接口。

提示：

- 对于无线设置，您可以以 root 权限运行 `wifi-menu` as 命令来在 `/etc/netctl/` 下创建配置文件。您需要 dialog包 软件包才能使用 *wifi-menu* 命令
- 在您的配置文件中使用 `SkipNoCarrier=yes` 指令，可以在无论网线是否连接的情况下都启用静态 IP 配置文件。

参考 netctl.profile(5) 来获得配置文件选项的完整列表。for a complete list of profile options.

用法

参考 netctl(1) 来获得 *netctl* 指令的完整列表。

启动配置文件

创建配置文件后，您就可以尝试建立连接了。以下命令中的 *profile* 仅为配置文件名称，而非完整路径：

```
# netctl start profile
```

如果以上命令运行出错，请以 root 权限运行 `journalctl -xn` 或运行 `netctl status *profile*` 来获得更详细的错误说明。

启用配置文件

您可以用以下命令来让配置文件随系统启动：

```
# netctl enable profile
```

这会创建一个随计算机一起启动的 systemd 服务。但对配置文件的修改不会自动反映到服务文件中，所以在您修改配置文件之后，请重新启用它：

```
# netctl reenable profile
```

启用配置文件后，它将在下次启动时启动。显然，只有插好网线或者设备位于要使用的无线接入点范围内才能成功。

如果您需要频繁切换多个配置文件（例如，携带笔记本电脑旅行），请使用 #特殊 systemd 单元，而不是启用配置文件。

特殊 systemd 单元

netctl 为自动切换有线和无线连接配置文件提供了一个特殊的 systemd 服务。有关特殊 systemd 单元的完整列表，请参见 netctl.special(7)。

有线网络

安装 ifplugd包 软件包然后启动或启用 `netctl-ifplugd@*interface*.service` systemd 单元。DHCP 配置文件会在网线插入或拔出时自动启动/停止。

- `netctl-ifplugd@*interface*.service` 服务将首选使用 DHCP 的配置文件。
- 要自动启动静态IP配置文件，需要在其中设置 `ExcludeAuto=no` 选项。
- 要想调高静态 IP 配置文件的优先级，您可以设置 `Priority=2`，这高于 DHCP 配置文件的默认优先级 `Priority=1`。

无线网络

启动或启用 `netctl-auto@*interface*.service` systemd 单元。 *netctl* 会在您从一个无线网络移动到另一个无线网络的时候自动切换配置文件（也就是无线漫游）。

- 配置文件必须设置 `Security=wpa-configsection` 或者 `Security=wpa` 才能使用 *netctl-auto*。不能用 *`Security=wpa-config`*
- 如果你希望某些无线网络配置文件不被 `netctl-auto@*interface*.service` 自动启动，您必须在配置里明确指定 `ExcludeAuto=yes` 选项。
- 您可以在 *WPAConfigSection* 里设置 `priority=` 属性(参考 `/etc/netctl/examples/wireless-wpa-configsection`) 当多个无线接入点可用时，该选项的值越大表示对应的配置文件优先级越高。

记得用您设备的实际接口名称来替换上面的 *interface* 。比如 `netctl-auto@wlp4s0.service`。有关详细信息，请参考 netctl.profile(5) 。

注意：* 如果任何配置文件包含错误，例如一个空的或错误引用的 `Key=` 变量，该单元将无法加载并报错 `"Failed to read or parse configuration '/run/network/wpa_supplicant_wlan0.conf'`，即使该配置文件未被使用。

- 如果您曾经用 *netctl* 来 #启用配置文件，请运行 `netctl disable *profile*` 来防止该配置文件在系统启动时被启动两次。

可以通过 *netctl-auto* 命令手动控制 *netctl-auto* 管理的接口，而不必停止 `netctl-auto.service`服务。有关可用操作的完整列表，请参见 netctl-auto(1) 。

技巧和窍门

示例配置

有线网络

对于使用 DHCP 的网络连接，您只需把 `/etc/netctl/examples/ethernet-dhcp` 复制到 `/etc/netctl` 下，然后配置其中的 `Interface` 项即可。

例如：

```
/etc/netctl/my_dhcp_profile
Interface=enp1s0
Connection=ethernet
IP=dhcp
```

要设置静态 IP，请把 `/etc/netctl/examples/ethernet-static` 复制到 `/etc/netctl` 下然后自行修改 `Interface`、 `Address` 、`Gateway` 和 `DNS` 选项。

例如：

```
/etc/netctl/my_static_profile
Interface=enp1s0
Connection=ethernet
IP=static
Address=('10.1.10.2/24')
Gateway='10.1.10.1'
DNS=('10.1.10.1')
```

注意其中的子网大小 `/24`。这是用无类别域间路由（英语：Classless Inter-Domain Routing，简称CIDR）表示法表示的，相当于子网掩码 `255.255.255.0`。

要在一个接口上设置多个 IP，可以这样设置

```
Address=('10.1.10.2/24' '192.168.1.2/24')
```

要添加多个 DNS 服务器地址：

```
DNS=('1.1.1.1' '1.0.0.1')
```

无线网络 (WPA-PSK)

以下内容适用于使用预共享密钥（WPA-PSK）的标准无线连接。

```
/etc/netctl/wireless-wpa
Description='A simple WPA encrypted wireless connection using 256-bit PSK'
Interface=wlp2s2
Connection=wireless
Security=wpa
IP=dhcp
ESSID=your_essid
Key=\"64cf3ced850ecef39197bb7b7b301fc39437a6aa6c6a599d0534b16af578e04a
```

注意：

- 确保对 `Key` 变量使用 **特殊引用规则**，如 netctl.profile(5) § SPECIAL QUOTING RULES 中所述。
- 如果密码短语失败，请尝试删除 `Key` 变量中的 `\"`。

- 虽然是 Key 看起来是加密的，但是仅凭您在配置文件配置中输入的密钥就已经足以连接到 WPA-PSK 网络。因此，这只能隐藏明文的密码短语，而不会阻止对该文件具有读访问权限的任何人连接到网络。

混淆无线密码

您还可以按照以下步骤来混淆无线密码短语 (*wifi-menu* 在使用 `-o` 标志时自动执行此操作) :

不希望将其无线网络的密码以明文形式存储的用户可以选择存储相应的 256 位预共享密钥，该密钥使用标准算法从密码和 SSID 计算得出。

使用 wpa_passphrase 命令来计算 256 位 PSK：

```
$ wpa_passphrase your_essid
network={
  ssid="your_essid"
  #psk="passphrase"
  psk=64cf3ced850ecef39197bb7b7b301fc39437a6aa6c6a599d0534b16af578e04a
}
```

然后用*预共享密钥* (PSK) 来替换 `Key` 变量中的明文密码短语。 The *pre-shared key* (psk) now needs to replace the plain text passphrase of the `Key` variable in the profile.

使用实验性 GUI

如果您想要一个图形用户界面来管理 *netctl* 和您的连接，并且您不担心使用高度实验性的非官方软件包，那么有一些选项可用。netctl-guiAUR 提供了一个基于 Qt 的图形界面、DBus 守护程序和 KDE 小部件。netmenuAUR 使用 dmenu包 作为其图形界面，gnome-shell-extension-netctl-auto-gnome-gitAUR 是 gnomeshell 扩展。

还有一个应用程序能显示托盘图标，和在配置文件更改时显示桌面通知 : netctl-trayAUR。

链路聚合

From kernel documentation:

: The Linux bonding driver provides a method for aggregating multiple network interfaces into a single logical "bonded" interface. The behavior of the bonded interfaces depends on the mode. Generally speaking, modes provide either hot standby or load balancing services. Additionally, link integrity monitoring may be performed.

负载均衡

To use bonding with netctl, additional package from official repositories is required: ifenslave包\[损坏的链接：package not found\].

Copy `/etc/netctl/examples/bonding` to `/etc/netctl/bond0` and edit it, for example:

```
/etc/netctl/bond0
Description='Bond Interface'
Interface='bond0'
Connection=bond
BindsToInterfaces=('eth0' 'eth1')
IP=dhcp
IP6=stateless
```

Now you can disable your old configuration and set *bond0* to be started automatically. Switch to the new profile, for example:

```
# netctl switch-to bond0
```

注意：This uses the round-robin policy, which is the default for the `bonding` driver. See official documentation for details.

Setting the MODE in the netctl configuration is not always successful and it may be necessary to pass options directly to the bonding module on load as noted here. This may be needed to use LACP / mode 4.

提示：To check the status and bonding mode:

```
$ cat /proc/net/bonding/bond0
```

有线到无线网络故障切换

This example describes how to use *bonding* to fallback to wireless when the wired Ethernet goes down. This is most useful when both the wired and wireless interface will be connected to the same network. Your wireless router/access point must be configured in bridge mode.

You will need additional packages from the official repositories: ifenslave包\[损坏的链接：package not found\] and wpa_supplicant包.

First, load the module at boot:

```
/etc/modules-load.d/bonding.conf
bonding
```

Then, configure the options of the `bonding` driver to use `active-backup` and configure the `primary` parameter to the device you want to be the active one (normally the wired interface). Also, be sure to use the same device name as returned when running `ip link`:

```
/etc/modprobe.d/bonding.conf
options bonding mode=active-backup miimon=100 primary=eth0 max_bonds=0
```

The `miimon` option is needed, for the link failure detection. The `max_bonds` option avoids the `Interface bond0 already exists` error. More information can be obtained on the kernel documentation.

Next, configure a netctl profile to enslave the two hardware interfaces. Use the name of all the devices you want to enslave. If you have more than two wired or wireless interfaces, you can enslave all of them on a bond interface. But, for most cases you will have only two devices, a wired and a wireless one:

```
/etc/netctl/failover
Description='A wired connection with failover to wireless'
Interface='bond0'
Connection=bond
BindsToInterfaces=('eth0' 'wlan0')
IP='dhcp'
```

Disable any other profiles (specially a wired or wireless) you had enabled before and then enable the failover profile on startup:

```
# netctl enable failover
```

Now you need to configure *wpa_supplicant* to connect to any known network you wish. You should create a file for each interface and enable it on systemd. Create the following file with this content:

```
/etc/wpa_supplicant/wpa_supplicant-wlan0.conf
ctrl_interface=/run/wpa_supplicant
update_config=1
```

And append to the end of this file any network you want to connect to:

```
network={
    ssid="SSID"
    psk=PSK
}
```

To generate the obfuscated PSK you can run *wpa_passphrase* as on the wpa_supplicant#Connecting with wpa_passphrase page.

Now, enable the `wpa_supplicant@` template service on the network interface, for example `wpa_supplicant@wlan0`.

You can try now to reboot your machine and see if your configuration worked.

注意：If you get this error on boot bonding:

```
wlan0 is up - this may be due to an out of date ifenslave
```

Then this is happening because the *wpa_supplicant* is being run before the `failover` netctl profile. This happens because systemd runs everything in parallel, unless told otherwise. *ifenslave* need all the interfaces to be down before bonding them to the `bond0` interface. And, since the *wpa_supplicant* need to put the interface up to be able to scan for networks, this might cause the interface to not be enslaved and your bonding to only have the wired interface.

If this is your case, then you will need to setup a custom dependency on the `wpa_supplicant@wlan0` service in relation with the `netctl@failover` profile. More specifically, the *wpa_supplicant* must be started **after** the netctl profile. To accomplish this, create a custom dependency file based on the instructions provided here: systemd#Handling dependencies

```
/etc/systemd/system/wpa_supplicant@wlan0.service.d/customdependency.conf
[Unit]
After=netctl@failover.service
```

After that you can try to reboot your system again and see if it works. You can check the status of your bonding by checking journalctl for the `netctl@failover.service` unit.

And by checking:

```
# ip link
```

You should see something like this:

```
1: eth0: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master bond0
state UP mode DEFAULT group default qlen 1000
link/ether xx:xx:xx:xx:xx:xx brd ff:ff:ff:ff:ff:ff
2: wlan0: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master bond0 state
UP mode DORMANT group default qlen 1000
```

```
link/ether xx:xx:xx:xx:xx:xx brd ff:ff:ff:ff:ff:ff
3: bond0: <BROADCAST,MULTICAST,MASTER,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode
DEFAULT group default
link/ether xx:xx:xx:xx:xx:xx brd ff:ff:ff:ff:ff:ff
```

Now, you can test your failover setup, by initiating a big download. Unplug your wired interface. Your download should keep going over the wireless interface. Then, plug your wired interface again and it should keep working. You can debug by checking journalctl for the `netctl@failover.service` and `wpa_supplicant@wlan0.service` units.

使用任意接口

在某些情况下，可能需要允许配置文件使用系统上的任何接口。一个常见的用例是在具有不同硬件的许多机器上使用通用的磁盘映像（如果它们没有显示输出，这特别有用）。如果您使用内核的命名方案，并且您的机器只有一个以太网接口，那么 eth0 可能是是正确的接口。如果您使用 udev 的可预测的网络接口名称，则名称将根据特定硬件本身 (例如 enp1s0) 分配，而不是简单地根据检测到硬件的顺序 (例如 eth0、eth1) 分配。这意味着 netctl 配置文件可能仅在一台机器上工作，而不能在另一台机器上工作，因为它们各有不同的接口名称。

一个简单粗暴的解决方案是使用 `/etc/netctl/interfaces/` 目录。首先设置一个接口别名 (本例中为 `en-any`) ，然后在目录下创建一个同名文件 (确保它具有可执行权限)。

```
/etc/netctl/interfaces/en-any
#!/bin/sh
for interface in /sys/class/net/en*; do
        break;
done
Interface=$(basename "$interface")
echo "en-any: using interface $Interface";
```

然后创建一个使用该接口的配置文件。特别注意 `Interface` 指令，请把它设置为刚刚创建的接口别名。其余部分仅作为示例提供。

```
/etc/netctl/wired
Description='Wired'
Interface=en-any
Connection=ethernet
IP=static
Address=('192.168.1.15/24')
Gateway='192.168.1.1'
DNS=('192.168.1.1')
```

当 `wired` 配置文件启动时，使用上述两个文件的任何机器都会自动启动并配置系统上找到的第一个以太网接口，而不管 udev 分配给它什么名称。请注意，这不是配置接口的最可靠的方法。如果您使用多个接口，netctl 可能会尝试为它们分配相同的接口，这可能会导致连接中断。如果您不介意更复杂的解决方案，`netctl-auto` 可能更可靠。

使用钩子

netctl 支持 `/etc/netctl/hooks/` 中的钩子和 `/etc/netctl/interfaces/` 中的每个接口钩子，您可以在钩子中设置任何配置文件支持的选项。其中包括最重要的 `ExecUpPost` 和 `ExecDownPre`。

在读取配置文件前，netctl 会先运行 `hooks` 目录下的**所有可执行的脚本**，然后才读取配置文件来建立连接。最后它会运行 `interfaces` 目录下与接口同名的可执行脚本（接口钩子）。因此，配置文件中的选项可以被接口钩子覆盖，
而 `/etc/netctl/hooks/` 下的钩子选项又可以被配置文件所覆盖。

只有在使用 `netctl-auto` 时，变量 `$INTERFACE` 和 `$ACTION` 才能在钩子或接口钩子中使用。

示例

连接建立时执行命令

```
/etc/netctl/hooks/myservices
#!/bin/sh
ExecUpPost="systemctl start crashplan.service; systemctl start
dropbox@<username>.service"
ExecDownPre="systemctl stop crashplan.service; systemctl stop
dropbox@<username>.service"
```

设置默认 DHCP 客户端

要设置或更改用于所有配置文件的 DHCP 客户端，请执行以下操作：

```
/etc/netctl/hooks/dhcp
#!/bin/sh
DHCPClient='dhclient'
```

不要忘记给文件添加可执行权限

或者也可以对某个特定的网络接口设置单独的 DHCP 客户端，只需要创建一个接口钩子（`/etc/netctl/interfaces/<interface>` 脚本）并设置如下参数就可以了：

```
DHCPClient='dhclient'
```

最简易的 WPAConfigSection 配置

如 netctl.profile(5) § OPTIONS FOR 'WIRELESS' CONNECTIONS 中所述，`WPAConfigSection` 变量是传递给 wpa_supplicant 的配置项数组。因此，一个最简单的 WPAConfigSection 将包含：

```
Description='A wireless connection using a custom network block configuration'
Interface=wlan0
Connection=wireless
Security=wpa-configsection
IP=dhcp
WPAConfigSection=(
    'ssid="University"'
    'psk="very secret passphrase"'
)
```

注意：如果要连接到具有非 ASCII 字符（unicode、表情符号等）的 SSID，可以将 SSID 指定为十六进制而不是字符串，例如 `ssid=F09F90BA` 表示“🐺”。 当不确定十六进制编码时，可以运行 *wifi-menu* 命令来查看（确保删除 \\ 和 x 字符）。

/etc/resolv.conf

如果您在配置文件中使用 `DNS*` 选项，*netctl* 将调用 resolvconf 来覆盖 resolv.conf。

故障排除

Job for netctl@wlan(...).service failed

警告：本节假设在启动 *netctl* 配置文件或服务之前没有其他网络服务正在运行。有关详细信息，请参阅 #安装。

有些人在使用 *netctl* 连接网络时会遇到问题，例如：

```
# netctl start wlan0-ssid
Job for netctl@wlan0\x2ssid.service failed. See 'systemctl status
netctl@wlan0\x2ssid.service' and 'journalctl -xn' for details.
```

当以 root 身份运行 `journalctl -xn` 查看日志时，可能会看到以下错误：

1\. 如果您的网络接口（在本例中为 `wlan0`）已启动：

```
network: The interface of network profile 'wlan0-ssid' is already up
```

禁用网络接口可能能够解决这个问题：

```
# ip link set wlan0 down
```

然后再重试：

```
# netctl start wlan0-ssid
```

2\. 如果错误仍然存在，请在添加 `ForceConnect` 选项后重试：

```
/etc/netctl/wlan0-ssid
...
ForceConnect=yes
```

保存并尝试再次启动配置文件：

```
# netctl start wlan0-ssid
```

dhcpcd: ipv4_addroute: File exists

在某些系统上，把 dhcpcd 与 netctl 结合使用会导致唤醒计算机时出现超时问题，特别是在你切换了网络的时候。netctl 将报告称已成功连接，但你仍然会遇到连接超时等问题。这种情况是因为旧的默认路由仍然存在，并且没有被更新。避免这种问题的一个方法是改用 dhclient 作默认 dhcp 客户端。 有关此问题的更多信息，请参阅此处.

DHCP 超时问题

如果您在通过 DHCP 请求租约时遇到超时问题，则可以将超时值设置得比 netctl 默认的 30 秒长。比如可以在 `/etc/netctl/hooks/` 或 `/etc/netctl/interfaces/` 中创建一个可执行文件，并向其中添加 `TimeoutDHCP=40` 选项。

调试 dhcpcd

如果 dhcpcd 无法获得一个地址，请将 `-d` 选项添加到 `/usr/lib/netctl/dhcp`，然后以 root 用户身份运行 `journalctl -xe` 来查看调试日志。通过查看调试日志，您可能能发现客户端因为 IP 地址已被使用而拒绝了服务器提供的 IP 地址等错误原因。

连接超时问题

如果您遇到与 DHCP 无关的超时问题 (例如在使用静态 IP 的连接上) ，并且在启动配置文件时遇到类似下列错误：

```
# journalctl _SYSTEMD_UNIT=netctl@profile.service
Starting network profile 'profile'...
No connection found on interface 'eth0' (timeout)
Failed to bring the network up for profile 'profile'
```

那么，您应该修改 `TimeoutCarrier=` 和 `TimeoutUp=` 选项来调高检测网线插好和网卡就绪的超时时间。

```
/etc/netctl/profile
...
TimeoutUp=300
TimeoutCarrier=300
```

别忘了用以下命令重新启用配置文件：

```
# netctl reenable profile
```

唤醒时 netctl-auto 故障

有时，当系统从挂起、休眠或混合休眠恢复时，*netctl-auto* 无法重新连接。一个简单的解决方案是重新启动 *netctl-auto* 服务。这可以通过类似 netctl-auto-resumeAUR 这样的附加服务实现自动化：

```
/etc/systemd/system/netctl-auto-resume@.service
[Unit]
Description=restart netctl-auto on resume.
Requisite=netctl-auto@%i.service
After=sleep.target

[Service]
Type=oneshot
ExecStart=/usr/bin/systemctl restart netctl-auto@%i.service

[Install]
WantedBy=sleep.target
```

要为您的无线网卡启用此服务，请以 root 权限启用 `netctl-auto-resume@wlan0.service` 服务，并将其中的 `wlan0` 更改为所需的网络接口。

如果服务单元启动时，对应的网卡尚未恢复运行，则此操作将失败。您可以通过在 *After* 选项中添加要依赖的 Systemd 目标来修复：

```
/etc/systemd/system/netctl-auto-resume@.service
...
After=sleep.target sys-subsystem-net-devices-%i.device
...
```

netctl-auto 无法自动解锁无线网卡

许多笔记本电脑有一个硬件按钮（或开关）来屏蔽无线网卡，但是该卡也可以被内核屏蔽。这可以由 rfkill 处理。

如果您希望 *netctl-auto* 自动解禁您的无线卡并连接到特定网络，请参考 netctl.profile(5) 为您的无线网络连接设置 `RFKill=++auto++` 选项。

在使用多个网卡时提示 RTNETLINK answers: File exists

这是一个非常具有误导性的报错，这实际上意味着您已经在之前的某个 netctl 控制文件配置了一个默认路由。当 netctl 在启动第 n 个网卡并且尝试设置它的本地路由时，因为已经在启动第 n-1 个网卡时设置过默认路由了，所以该操作会失败。

把它删掉后就一切正常了，除了因为没有默认路由而无法正常上网以外。而即使用 `ExecUpPost` 也无法解决这个问题，因为他也会为每个网卡执行一遍。

一种可能的解决方案是创建一个新服务。并将下面的“FIRST_INTERFACE”和“SECOND_INTERFACE”替换为接口名称，将“192.168.xxx.yyy”替换为你的默认网关。

```
/etc/systemd/system/defaultrouter.service
[Unit]
Description="Configure default gateway"
Requires=netctl@FIRST_INTERFACE.service netctl@SECOND_INTERFACE.service
After=netctl@FIRST_INTERFACE.service netctl@SECOND_INTERFACE.service

[Service]
Type=oneshot
ExecStart=/usr/bin/ip route add default via 192.168.xxx.yyy

[Install]
WantedBy=network-online.target
```

使用 eduroam 和其他 MSCHAPv2 连接时遇到问题

参见 wpa_supplicant#Problems with eduroam and other MSCHAPv2 connections。

在配置文件中使用 .include 指令导致日志警告

仍然使用 systemd 旧的 `.include` 指令的配置文件将生成日志警告，例如：

```
systemd: /etc/systemd/system/netctl@<profile>.service:1: .include directives are deprecated, and support for them will be removed in a future version of systemd. Please use drop-in files instead.
```

详见 FS#59494。

执行

```
netctl reenable profile
```

能配置文件更新为新的 drop-in unit file 格式。

钩子无效

如果在 `/etc/netctl/hooks/` 中有多个钩子，那么像 `ExecUpPost` 和 `ExecDownPre` 这样的变量会被重复覆盖，导致只能执行其中一个文件的。要解决此问题，请如下定义变量：

```
/etc/netctl/hooks/test
ExecUpPost="some command ; "$ExecUpPost
ExecDownPre="some command ; "$ExecDownPre
```

这样可以在其他命令前面附加要执行的命令，防止被重复覆盖。

参阅

- 初始邮件列表公告
- 官方公告帖子
- 官方新闻公告

18.4 Systemd-networkd

难度 高级 | **篇幅** 39.6 KB | **所属** 网络管理器

本节将讲到

- 基本用法
- 配置文件
- 容器下的应用
- 提示与技巧
- 故障排查

systemd-networkd 是一个管理网络配置的系统守护进程。它会在网络设备出现时检测和配置它们；它还可以创建虚拟网络设备。这个服务非常适合于为 systemd-nspawn 管理的容器或者虚拟机创建复杂的网络配置。如果只是简单网络的配置，它也同样能胜任。

基本用法

systemd包 是默认 Arch 安装的一部分，包含操作有线网络所需的所有文件。无线适配器可以通过其他服务（比如 wpa_supplicant 或者 iwd）来配置，本文后面的部分也会介绍相关内容。

必需的服务和设置

启动/启用 `systemd-networkd.service` 以使用 ***systemd-networkd***。

注意：必须确认没有其他服务正在管理网络。不同的网络管理服务会互相冲突。通过 `systemctl --type=service` 可以得到正在运行的服务的列表，请停止其他网络管理服务。

配置 systemd-resolved 是可选的，它为本地应用程序提供网络名称解析服务。是否使用它可以考虑下面几条：

- 如果 *.network* 文件中指定了 DNS 条目，则 *systemd-resolved* 服务是必需的
- 想自动从DHCP服务器或IPv6路由器推荐获取 DNS 服务器地址（通过在 `[Network]`中设置(`DHCP=`和/或`IPv6AcceptRA=`，并在对应的`[DHCPv4]`、`[DHCPv6]`、`[IPv6AcceptRA]`中设置`UseDNS=yes`（默认值）来实现，参见 systemd.network(5)）
- 请搞明白 resolv.conf 和 *systemd-resolved* 如何互相影响，以便正确配置要使用的 DNS 服务器。更多相关信息可以参见 systemd-resolved
- 注意：即使没有启用 *systemd-networkd*， *systemd-resolved* 也能够提供服务。

systemd-networkd-wait-online

启用 `systemd-networkd.service` 的同时，也会启用 `systemd-networkd-wait-online.service`。这是一个一次性类型的 systemd 服务，用于等待网络完成配置。该服务具有 `WantedBy=network-online.target`，因此只有当 `network-online.target` 被启用或被其他单元拉入时才会启动。另请参阅当网络启动后执行服务。

默认情况下，`systemd-networkd-wait-online.service` 会等待所有由 *systemd-networkd* 管理的链路完成配置（或失败），并且至少有一个链路在线。

详情请参阅 systemd-networkd-wait-online(8)。

多个网络接口不一定会持续连接

若系统具有多个网络接口，但这些接口并不一定会时时保持连接（例如拥有双口以太网卡，但只插了一根线），启动 `systemd-networkd-wait-online.service` 时会在默认 2 分钟超时后失败，从而导致开机流程出现不必要的延迟。

若希望更改为等待“任一”接口而不是“全部”接口上线，可以编辑该服务，并在 `ExecStart` 行加入 `--any` 参数：

```
/etc/systemd/system/systemd-networkd-wait-online.service.d/wait-for-only-one-interface.conf
[Service]
ExecStart=
ExecStart=/usr/lib/systemd/systemd-networkd-wait-online --any
```

或者，也可以使用 `systemd-networkd-wait-online@.service` 来指定特定接口。例如，若只需等待 `enp1s0` 上线，请禁用 `systemd-networkd-wait-`

online.service，并启用 systemd-networkd-wait-online@enp1s0.service。

提示：若已知某些接口可能不会随时上线，可在 .network 配置文件的 [Link] 区块中加入 RequiredForOnline=no。详情请参见 [systemd.network(5) § \[LINK\]_SECTION_OPTIONS] (https://man.archlinux.org/man/systemd.network.5#%5BLINK%5D_SECTION_OPTIONS)。

等待直到网络接口取得可路由地址

根据 systemd-networkd-wait-online.service(8) 的说明，"online 表示链路的操作状态至少为 degraded"（关于 degraded 与其他操作状态的定义，请参阅 networkctl(1)）。

若希望 systemd-networkd-wait-online.service 在网络接口取得可路由的 IP 地址后才结束（以避免依赖网络的其他服务启动过早），可在 *.network* 配置文件中的 [Link] 区段加入：

```
[Link]
RequiredForOnline=routable
```

配置样例

在本节中，所有配置都存储为在 /etc/systemd/network/ 目录下 形如 foo.network 的文件。有关选项的完整列表和处理顺序可以参考 #配置文件和 systemd.network(5)。

Systemd/udev 会自动为所有本地以太网、WLAN 和 WWAN 接口分配可预测且稳定的网络接口名。使用 networkctl list 以列出系统上所有设备。

在修改了配置文件之后，restart systemd-networkd.service 以使得它们生效。

注意：

- 配置文件中指定的选项区分大小写。
- 在下面的示例中，enp1s0 是有线适配器，wlp2s0 是无线适配器。他们的名字在不同系统上可能会有不同的名字。也可以使用通配符，例如，Name=en*。
- 如果想要禁用 IPv6 的话，参考 IPv6#systemd-networkd。
- 在 [Network] 段设置 DHCP=yes 来同时接收 IPv4 和 IPv6 DHCP 请求。

使用 DHCP 的有线适配器

```
/etc/systemd/network/20-wired.network
[Match]
Name=enp1s0

[Link]
RequiredForOnline=routable

[Network]
DHCP=yes
```

使用静态 IP 的有线适配器

```
/etc/systemd/network/20-wired.network
[Match]
Name=enp1s0

[Network]
Address=10.1.10.9/24
Gateway=10.1.10.1
DNS=10.1.10.1
#DNS=8.8.8.8
```

`Address=` 能够被使用多次来指定多个 IPv4 或者 IPv6 地址。 参见 #network 文件或者 systemd.network(5) 了解更多配置项。

无线适配器

为了能够使用 *systemd-networkd* 连接一个无线网络，需要一个被其他应用，比如 wpa_supplicant 或 iwd，配置好的无线适配器。

```
/etc/systemd/network/25-wireless.network
[Match]
Name=wlp2s0

[Link]
RequiredForOnline=routable

[Network]
DHCP=yes
IgnoreCarrierLoss=3s
```

如果无线适配器有一个静态地址，它的配置（除了接口的名字）跟有线适配器是一样的。

同一台机器上的有线和无线适配器

这些设置将为有线和无线连接自动获取 DHCP IP, 并使用 metric 让内核动态地决定使用的链接。这样，网络在有线连接断开时就不会有明显的中断。内核的 route metric (与 ip 配置一致) 会在具有多个设备时决定使用哪个传出数据包。如果某个连接断开，其他的接口就会自动接管，而不会出现网络中断（正在进行的传输可能会出现问题，不过那是另一层的问题了。）

注意：`Metric` 选项用于静态路由，而 `RouteMetric` 选项则用于不使用静态路由的配置。详情请参见 systemd.network(5)。

```
/etc/systemd/network/20-wired.network
[Match]
Name=enp1s0

[Network]
DHCP=yes

[DHCPv4]
RouteMetric=100

[IPv6AcceptRA]
RouteMetric=100
/etc/systemd/network/25-wireless.network
[Match]
```

```
Name=wlp2s0

[Network]
DHCP=yes

[DHCPv4]
RouteMetric=600

[IPv6AcceptRA]
RouteMetric=600
```

网络接口重命名

作为更改设备名称的替代方案，systemd 使用*.link*文件用于接口重命名。常见的例子是基于 MAC 地址给一个 USB 接口以太网适配器设置一个可预见的接口名称。这类设备依其连接到不同 USB 接口而具有不同的接口名称。

```
/etc/systemd/network/10-ethusb0.link
[Match]
MACAddress=12:34:56:78:90:ab

[Link]
Description=USB to Ethernet Adapter
Name=ethusb0
```

注意：任何由用户提供的*.link*文件名**必须**是依字典顺序先于默认配置文件名`99-default.link`才能生效。例如，必须是`10-ethusb0.link`而不能是 `ethusb0.link`。

配置文件

配置文件位于 `/usr/lib/systemd/network`，非持久化的运行时网络配置目录位于 `/run/systemd/network` ，本地管理网络配置位于 `/etc/systemd/network`。`/etc/systemd/network` 中的配置文件具有最高优先级。

配置文件有三类。它们均使用类似于 systemd 单元文件的格式。

- ***.network*** 文件，为*匹配*的设备提供一个网络配置
- ***.netdev*** 文件，为*匹配*的环境创建一个*虚拟网络设备*
- ***.link*** 文件，当网络设备出现时，udev 将查找第一个*匹配*的*.link*文件

它们均遵循下列规则：

- 如果位于`[Match]`小节的**全部**条件相匹配，配置项将被激活
- 一个空的`[Match]`小节意味着配置项适用任何情况（相当于`*`通配符）
- 所有配置文件将按字典顺序集中保存和处理，不管它们在目录中的实际顺序如何。
- 同名文件将彼此替换

提示：

- 要永久覆盖 `/usr/lib/systemd/network` 中系统提供的文件（即升级之后仍覆盖），请在 `/etc/systemd/network` 中放置一个具有相同名称的文件并将其符号链接到 `/dev/null`
- 星号（`*`）通配符可以在 `VALUE` 中使用（例如 `en*` 将匹配任何以太网设备）， 布尔值可以简单地写为 `yes` 或 `no`。
- 根据这个线索的讨论，最佳实践是 to setup specific container network settings *inside the container* with **networkd** configuration files.
- Systemd 使用`1`, `true`, `yes`, `on`作为逻辑“真”值，`0`, `false`, `no`, `off`作为逻辑“假”值

network 文件

这类文件用于设置网络配置变量，尤其适用于服务器和容器。

*.network*文件含有下列小节：`[Match]`、`[Link]`、`[Network]`、`[Address]`、`[Route]`以及`[DHCP]`。下列为每小节的通用配置。详情及范例请参阅 systemd.network(5)。

\[Match\] 小节

参数	说明	可接受的值	默认值
`Name=`	匹配设备名称，例如 `en*`。可使用 `!` 反向匹配。	以空格分隔的设备名称，支持通配符与逻辑否定（`!`）	
`MACAddress=`	匹配 MAC 地址，例如 `MACAddress=01:23:45:67:89:ab` `00-11-22-33-44-55` `AABB.CCDD.EEFF`	使用冒号、连字符或点号分隔的十六进制 MAC 地址	
`Host=`	匹配主机名称或 machine ID。	含通配符的主机名称字符串，machine-id(5)	
`Virtualization=`	检查是否运行于虚拟环境中。`Virtualization=false` 仅匹配实体机器；`Virtualization=true` 则匹配任何容器或虚拟机。亦可检查特定虚拟化类型或实现，或用户命名空间（`private-users`）。	布尔值、逻辑否定（`!`）、类型（`vm`, `container`）、实现（参见 systemd-detect-virt(1)）、`private-users`	

\[Link\] 小节

参数	说明	可接受的值	默认值
`MACAddress=`	指定设备的硬件地址。可用于 MAC 地址伪造。	使用冒号、连字符或点号分隔的十六进制 MAC 地址	
`MTUBytes=`	指定设备的最大传输单元（MTU）大小（单位：字节）。若启用 IPv6 且设置值小于 1280（IPv6 最小 MTU），系统会自动调整为此值。设置较大的 MTU（例如使用 jumbo frames）可大幅提升传输性能。	整数，可加上单位 K、M、G（以 1024 为底）	
`Multicast=`	是否允许使用 多播。	布尔值	

\[Network\] 小节

参数	说明	可接受的值	默认值
DHCP=	控制 DHCPv4 或 DHCPv6 客户端支持。	布尔值、ipv4, ipv6	no
DHCPServer=	启用时将启动 DHCPv4 服务器。	布尔值	no
MulticastDNS=	启用 多播 DNS 支持。若设为 resolve，仅启用名称解析功能，不注册主机或广播服务。	布尔值、resolve	false
DNSSEC=	控制是否启用 DNSSEC 验证支持。设为 allow-downgrade 时，在不支持 DNSSEC 的网络中将自动停用以提高兼容性。	布尔值、allow-downgrade	false
DNS=	设置静态 DNS 地址。可指定多个值。	inet_pton(3)	
Domains=	指定此连接使用哪些域名解析，[systemd.network(5) § \[NETWORK\] SECTION OPTIONS] (https://man.archlinux.org/man/systemd.network.5#%5BNETWORK%5D_SECTION_OPTIONS)。	域名，可加上波浪号（~）表示仅供解析使用	
IPv4Forwarding= 和 IPv6Forwarding=	启用时，此接口接收的（IPv4 / IPv6）包可根据路由表转发至其他接口。对应内核的 net.ipv4/6.conf.INTERFACE.forwarding sysctl 选项。详见 Internet sharing#Enable packet forwarding。	布尔值	no
IPMasquerade=	启用时，此接口转发的包会被伪装为来自本机。此选项的设置可能会同时启用 IPv4Forwarding 或 IPv6Forwarding。	ipv4, ipv6, both, no	no
IPv6PrivacyExtensions=	设置是否使用会定期变更的无状态临时地址（参见 RFC:4941）。prefer-public 表示启用临时地址，但偏好使用公开地址；kernel 则保留内核默认值。	布尔值、prefer-public, kernel	no

\[Address\] 小节

参数	说明	可接受的值	默认值
Address=	可多次指定此键以设置多个 IP 地址。若未使用 DHCP，则此参数为必要项。若指定为 0.0.0.0（IPv4）或 ::（IPv6），系统会自动从全局未使用段中分配一个适当大小的地址范围。	静态 IPv4 或 IPv6 地址及其前缀长度（见 inet_pton(3)）	

\[Route\] 小节

- `Gateway=` 此选项为“必要项”，除非使用直接路由或 DHCP。
- `Destination=` 路由的目的地前缀，可加上斜线与前缀长度。
- `Metric=` 路由的优先权。
- `Type=` 路由的类型。
- `Table=` 路由所使用的路由表 ID。
- `GatewayOnLink=` 若设为 `yes`，则内核不再检查网关是否可由本机直接连接（例如：是否连接至本地网络）。可用于解决部分路由问题。

若 `Destination` 未在 `[Route]` 区段中指定，则该区段将视为默认路由。

提示：将 `Address=` 与 `Gateway=` 直接写入 `[Network]` 区段，可作为简写方式，适用于 `[Address]` 仅包含 `Address`、`[Route]` 仅包含 `Gateway` 的情况。

\[RoutingPolicyRule\]

标准路由算法通常仅根据目标地址选择包应发送的网关。但在复杂网络中，这样的判断不一定足够。

`[RoutingPolicyRule]` 区段允许你根据指定条件应用额外的路由规则，只应用至符合条件的包。

以下是部分可用的匹配条件：

- `TypeOfService=`：匹配包的服务类型，值为 0 到 255 间的整数。
- `From=`：源地址前缀。
- `To=`：目的地地址前缀。
- `FirewallMark=`：iptables 设置的防火墙标记值。
- `IncomingInterface=`：包进入的接口。
- `OutgoingInterface=`：包离开的接口。

其他可用选项：

- `Type=`：Routing Policy Database（路由策略数据库）的规则类型。
- `Table=`：当包匹配此规则时要使用的路由表。
- `Priority=`：此规则的优先级。规则会依优先级由小至大排序并应用。
- `InvertRule=`：若设为 `true`，则此规则会反转，即应用于\\不符合\\任何匹配条件的包。
- `SuppressPrefixLength=`：若包的前缀长度小于等于指定值，则此规则不应用。

\[DHCPv4\]

参数	说明	可接受的值	默认值
`UseDNS=`	控制是否使用 DHCP 服务器广播的 DNS 服务器信息。	布尔值	`true`
`Anonymize=`	若设为 true，则根据 RFC:7844（DHCP 客户端匿名配置档）限制发送可识别用户的选项。	布尔值	`false`
`UseDomains=`	控制是否使用 DHCP 服务器提供的域名作为 DNS 搜索域。若设为 `route`，仅用于 DNS 路由查询，不用作搜索。可解决搭配 systemd-resolved 使用时的本地域名解析问题。	布尔值、`route`	`false`
`IPv6OnlyMode=`	若设为 true，DHCP 客户端会通知服务器其支持仅 IPv6 运行（参见 RFC:8925）。若服务器响应该选项，则客户端会中止 DHCPv4 请求，不会获取 IPv4 地址，并改为设置为 IPv6-only 模式。	布尔值	`true`（若启用 IPv6 时）

\[DHCPServer\]

这是一个 DHCP 服务器的示例配置，与 hostapd 搭配使用可建立无线热点。`IPMasquerade` 会新增 NAT 所需的防火墙规则，并隐含 `IPv4Forwarding=yes`，以启用包转发功能。

```
/etc/systemd/network/wlan0.network
[Match]
Name=wlan0

[Network]
Address=10.1.1.1/24
DHCPServer=true
IPMasquerade=ipv4

[DHCPServer]
PoolOffset=100
PoolSize=20
EmitDNS=yes
DNS=9.9.9.9
```

netdev 文件

**这一章节正在考虑移除。**

原因: 与 systemd.netdev(5) 手册页内容重复。 (在 Talk:Systemd-networkd 讨论)

这些文件用于创建虚拟网络设备。它们包含两个区段：`[Match]` 与 `[NetDev]`。以下列出每个区段常用的设置键值。更多信息与示例请参阅 systemd.netdev(5)。

这类文件将创建虚拟网络设备。包含两个小节：`[Match]` 和 `[NetDev]`。下列为每小节的通用配置。详情及范例请参阅systemd.netdev(5)。

\[Match\] 小节

- `Host=` 主机名
- `Virtualization=` 检查是否运行于虚拟机中

\[NetDev\] 小节

最通用的配置为：

- `Name=` 接口名称。**必须提供**
- `Kind=` 例如：*bridge*, *bond*, *vlan*, *veth*, *sit*, 等等。**必须提供**

link 文件

这些文件是自定义 udev 规则的替代方案，会在设备出现时由 udev 自动应用。它们包含两个区段：`[Match]` 与 `[Link]`。以下列出各区段常用的设置键值。更多信息与示例请参阅 systemd.link(5)。

提示：使用 `# udevadm test-builtin net_setup_link /sys/path/to/network/device` 来诊断 *.link* 文件的问题。

\[Match\] 小节

- `MACAddress=` MAC 地址
- `Host=` 主机名
- `Virtualization=`
- `Type=` 设备类型，例如 vlan

\[Link\] 小节

- `MACAddressPolicy=` 持久 (persistent) 或随机 (random) 地址，或
- `MACAddress=` 指定的地址

注意：系统内建的 `/usr/lib/systemd/network/99-default.link` 文件对大部分基本情况已足够使用。

容器下的应用

此服务由 systemd包 提供。你会想要在主机及容器上 enable 并 start `systemd-networkd.service`。

为了方便调试，强烈建议安装 bridge-utils包、net-tools包 与 iproute2包 软件包。

如果你使用 systemd-nspawn，可能需要修改 `systemd-nspawn@.service`，并在 `ExecStart` 行加入启动选项。请参考 systemd-nspawn(1) 了解所有可用选项。

注意：若想利用 DHCP 自动配置 DNS，需要启用 `systemd-resolved`，并将 `/run/systemd/resolve/resolv.conf` 符号链接到 `/etc/resolv.conf`。详细信息请参考 systemd-resolved.service(8)。

在开始配置容器网络前，建议：

- 停用所有 netctl（主机与容器）、dhcpcd（主机与容器）、systemd-networkd（仅容器）、及 `systemd-nspawn@.service`（仅主机）服务，避免冲突并方便调试。
- 若容器要访问互联网，确认已启用数据包转发。确保你的 *.network* 文件中未意外关闭转发，因为如果没有设置 `IPForward=1`，`systemd-networkd` 会在该接口关闭转发，即使全局已启用。
- 确认没有任何 iptables 规则阻挡流量。
- 启动 daemon 后，使用 systemd 的 `networkctl` 指令查看网络接口状态。

以下设定中，

- 我们将限制 `ip a` 指令只显示相关接口
- 假设「主机」是你启动的主要操作系统，「容器」是你的来宾虚拟机
- 所有接口名称及 IP 地址皆为示范用途

基本 DHCP 网络

此设定会为主机与容器启用 DHCP IP。两系统会共用相同 IP，因共用同一接口。

```
/etc/systemd/network/MyDhcp.network
[Match]
Name=en*

[Network]
DHCP=ipv4
```

接着在容器启用并启动 `systemd-networkd.service`。

当然你可以用实际网卡名称替换 `en*`，可用 `ip link` 指令查询。

- 主机与容器上执行：

```
$ ip a
2: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group
default qlen 1000
    link/ether 14:da:e9:b5:7a:88 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.72/24 brd 192.168.1.255 scope global enp7s0
       valid_lft forever preferred_lft forever
    inet6 fe80::16da:e9ff:feb5:7a88/64 scope link
       valid_lft forever preferred_lft forever
```

默认会使用 DHCP 服务器返回的 hostname 作为临时主机名。

若要更改，请在 `[DHCPv4]` 区段加入 `UseHostname=false`：

```
/etc/systemd/network/MyDhcp.network
[DHCPv4]
UseHostname=false
```

若不想自行设定 `/etc/resolv.conf` 的 DNS，想用 DHCP 配置，需 enable `systemd-resolved.service` 并建立符号链接：

```
# ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf
```

详情参见 systemd-resolved.service(8)。

注意：若透过 `/usr/bin/arch-chroot` 进入系统分区，需在 chroot 外的挂载分区上建立符号链接，因为 arch-chroot 会将文件链接到实际运行环境。

DHCP 用于两个独立 IP

桥接端口

先建立虚拟桥接接口，让 systemd 建立名为 *br0* 的以太网桥接设备。

```
/etc/systemd/network/MyBridge.netdev
[NetDev]
Name=br0
Kind=bridge
```

重新 Restart `systemd-networkd.service`，让 systemd 建立桥接。

主机与容器上：

```
$ ip a
3: br0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default
    link/ether ae:bd:35:ea:0c:c9 brd ff:ff:ff:ff:ff:ff
```

此时 *br0* 接口已显示但状态为 DOWN。

绑定以太网卡到桥接端口

接着将符合名称 *en** 的接口加入桥接 *br0*。

```
/etc/systemd/network/bind.network
[Match]
Name=en*

[Network]
Bridge=br0
```

以太网卡本身不得配置 DHCP 或 IP，桥接需要无 IP 的接口，请修改原有 `/etc/systemd/network/*MyEth*.network` 移除 IP 配置。

桥接网络

桥接建立且已绑定以太网卡后，需为桥接接口指定 IP 配置。以下范例为 DHCP。

```
/etc/systemd/network/mybridge.network
[Match]
Name=br0

[Network]
```

```
DHCP=ipv4
```

添加选项以引导容器

若要让主机和容器取得不同 IP，需要将容器网络与主机「断开」。加入启动参数 `--network-bridge=br0`：

```
# systemd-nspawn --network-bridge=br0 -bD /path_to/my_container
```

成果

- 宿主机上

```
$ ip a
3: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 14:da:e9:b5:7a:88 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.87/24 brd 192.168.1.255 scope global br0
       valid_lft forever preferred_lft forever
    inet6 fe80::16da:e9ff:feb5:7a88/64 scope link
       valid_lft forever preferred_lft forever
6: vb-MyContainer: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast master
br0 state UP group default qlen 1000
    link/ether d2:7c:97:97:37:25 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::d07c:97ff:fe97:3725/64 scope link
       valid_lft forever preferred_lft forever
```

- 容器中

```
$ ip a
2: host0: <BROADCAST,MULTICAST,ALLMULTI,AUTOMEDIA,NOTRAILERS,UP,LOWER_UP> mtu 1500
qdisc pfifo_fast state UP group default qlen 1000
    link/ether 5e:96:85:83:a8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.73/24 brd 192.168.1.255 scope global host0
       valid_lft forever preferred_lft forever
    inet6 fe80::5c96:85ff:fe83:a85d/64 scope link
       valid_lft forever preferred_lft forever
```

注意

- 主机的 `br0` 与容器的 `host0` 现有各自的 IP
- 新增两个接口：主机的 `vb-*MyContainer*` 及容器的 `host0`。这是因为使用了 `--network-bridge=br0`，该选项暗示了 `--network-veth`，在主机与容器间建立虚拟以太网桥接连接。
- 容器的 DHCP IP 是由系统 `/usr/lib/systemd/network/80-container-host0.network` 提供。
- 主机上执行：

```
$ brctl show
bridge name   bridge id            STP enabled   interfaces
br0           8000.14dae9b57a88    no            enp7s0
                                                 vb-MyContainer
```

此输出证明桥接有两个绑定接口。

- 宿主机上的路由表：

```
$ ip route
default via 192.168.1.254 dev br0
192.168.1.0/24 dev br0  proto kernel  scope link  src 192.168.1.87
```

- 容器中的路由表：

```
$ ip route
default via 192.168.1.254 dev host0
192.168.1.0/24 dev host0  proto kernel  scope link  src 192.168.1.73
```

以上显示 `br0` 与 `host0` 已启用并拥有 IP 与默认网关 192.168.1.254。网关地址由 systemd-networkd 自动获取。

```
$ cat /run/systemd/resolve/resolv.conf
nameserver 192.168.1.254
```

静态 IP 网络

为每个设备设置静态 IP 有利于部署 FTP、HTTP、SSH 等服务。只要系统中的 `/usr/lib/systemd/network/99-default.link` 包含 `MACAddressPolicy=persistent`（默认启用），每台设备重启后都会保留相同的 MAC 地址，方便将服务路由到目标设备。

配置步骤如下：

主机端

配置与 #DHCP 用于两个独立 IP 类似。先创建虚拟桥接接口并绑定实体网卡，通过以下两个文件实现：

/etc/systemd/network/*MyBridge*.netdev /etc/systemd/network/*MyEth*.network

然后为虚拟桥接接口设置 IP 和 DNS，示例如下：

```
/etc/systemd/network/MyBridge.network
[Match]
Name=br0

[Network]
DNS=192.168.1.254
Address=192.168.1.87/24
Gateway=192.168.1.254
```

容器端

先移除系统默认提供的 DHCP 配置文件 `/usr/lib/systemd/network/80-container-host0.network`，以永久方式（例如 systemd包 更新后仍然生效）在容器中执行：

1. ln -sf /dev/null /etc/systemd/network/80-container-host0.network

若只想在主机端使用静态 IP，容器仍通过 DHCP 获取 IP，则可以保留此文件。

然后为默认接口 `host0` 设置静态 IP，并在容器中 enable and start `systemd-networkd.service`，示例如下：

```
/etc/systemd/network/MyVeth.network
[Match]
Name=host0

[Network]
```

```
DNS=192.168.1.254
Address=192.168.1.94/24
Gateway=192.168.1.254
```

提示与技巧

交互界面及桌面集成

systemd-networkd 并不提供图形交互式管理界面。但仍有一些工具可以用于显示或修改当前网络状态、接收通知或处理无线网络配置：

- *networkctl* 提供了一个命令行接口，用于查询或修改网络接口状态。值得注意的是，若只想修改接口的某些行为，需要先编辑一项或多项位于 `/etc/systemd/network/` 中的配置文件。
- 当 *networkd* 配置了 wpa_supplicant 时，*wpa_cli* 和 *wpa_gui* 可用于动态关联与配置 WLAN 接口。
- networkd-dispatcherAUR 服务允许在网络接口状态变化时执行脚本，功能类似于 *NetworkManager-dispatcher*。
- networkd-notify-gitAUR 会在接口状态发生变化时生成简单通知消息。
- 作为 DNS 解析器，*systemd-resolved* 可以通过 `resolvectl status` 命令可视化当前 DNS 服务器信息。

基于 SSID（位置）配置静态 IP 或 DHCP

常见场景中，家庭无线使用 DHCP，而公司无线则使用静态 IP。这种混合设置可通过如下方式配置：

注意：文件名中的数字决定了处理顺序。用户可以在 \[Match\] 区段按 SSID、BSSID 或两者同时匹配。

```
/etc/systemd/network/24-wireless-office.network
# 专门用于办公 Wi-Fi 的配置
[Match]
Name=wlp2s0
SSID=office_ap_name
#BSSID=aa:bb:cc:dd:ee:ff

[Network]
Address=10.1.10.9/24
Gateway=10.1.10.1
DNS=10.1.10.1
#DNS=8.8.8.8
/etc/systemd/network/25-wireless-dhcp.network
# 对其他任何 Wi-Fi 网络使用 DHCP
[Match]
Name=wlp2s0

[Link]
RequiredForOnline=routable

[Network]
DHCP=yes
```

有线与无线接口绑定

参见 Wireless bonding。

绑定（bonding）允许通过多个接口共享连接，例如当有线接口断开时，无线仍保持连接，网络不中断。

创建一个 bond 接口，此处使用的模式为 *active-backup*，即如果主接口断开，则切换到备份接口：

```
/etc/systemd/network/30-bond0.netdev
[NetDev]
Name=bond0
Kind=bond

[Bond]
Mode=active-backup
PrimaryReselectPolicy=always
MIIMonitorSec=1s
```

将有线接口设为主接口：

```
/etc/systemd/network/30-ethernet-bond0.network
[Match]
Name=enp0s25

[Network]
Bond=bond0
PrimarySlave=true
```

将无线接口设为备份接口：

```
/etc/systemd/network/30-wifi-bond0.network
[Match]
Name=wlan0

[Network]
Bond=bond0
```

注意：在 `[Match]` 区段使用 MAC 地址时，建议使用 `PermanentMACAddress` 而非 `MACAddress`，详见 upstream 讨论。

像配置普通接口一样配置 bond 接口：

```
/etc/systemd/network/30-bond0.network
[Match]
Name=bond0

[Link]
RequiredForOnline=routable

[Network]
BindCarrier=enp0s25 wlan0
DHCP=yes
```

现在，如果有线网络断开，系统将自动切换到无线连接，保持联网状态：

```
$ networkctl
IDX LINK    TYPE     OPERATIONAL      SETUP
  1 lo      loopback carrier          unmanaged
  2 enp0s25 ether    no-carrier       configured
  3 bond0   bond     degraded-carrier configured
  5 wlan0   wlan     enslaved         configured
```

```
4 links listed.
```

加快 TCP 慢启动过程

在带宽较高（如 >10 Mbit/s）且延迟中等的连接（如家庭宽带）上，TCP 默认的慢启动算法偏保守，会导致下载初期速度较慢，需要数秒才能达到满速。在使用 pacman 更新包时尤其明显，每个包下载都从较慢速率起步，常在加速完成前就结束。

通过调整以下参数，可以让 TCP 连接以更大窗口启动，从而避免慢启动阶段造成的性能损失。虽然在低速连接或设置过高的情况下会因丢包而降低性能，但在带宽充足的环境下可显著提升体验。

使用前后请务必进行基准测试，以确认网络实际获得提升；若下载已经没有慢启动问题，则无需调整。测试时，应对高低速服务器都做测试，避免提升高速连接却牺牲低速访问。

编辑对应连接的 \.network\ 文件以调整：

```
/etc/systemd/network/eth0.network
[Match]
Name=eth0

#[Network]
#Gateway=...  <-- 如果已存在，则删除并在下面 Route 中指定

[Route]
# 此选项适用于 DHCP 提供的网关，如手动指定网关，也应写在这里。
Gateway=_dhcp4

# 以下为默认值 10，单位为 MSS (1460 字节) 的倍数
InitialCongestionWindow=10
InitialAdvertisedReceiveWindow=10
```

默认值 10 适合 <10 Mbit/s；若你是 100 Mbit/s 连接，建议使用 30。手册 [systemd.network(5) § \[ROUTE\] SECTION OPTIONS] (https://man.archlinux.org/man/systemd.network.5#%5BROUTE%5D_SECTION_OPTIONS) 表示 100 属于过高。

若启用 sysctl 设置 net.ipv4.tcp_slow_start_after_idle，当连接空闲一段时间后，TCP 将回到上述初始值；若禁用，则保留协商后的更大窗口。无论设置如何，每个 \\新 TCP 连接\\ 都从上述 Initial* 值开始。

sysctl 参数 net.ipv4.tcp_congestion_control 决定拥塞期间窗口的动态调整，与 Initial\ *设置无直接关系。Initial*\ 仅控制每条 TCP 新连接的初始窗口，后续仍由拥塞算法调整。提高初始值可以减少协商延迟，但若设置不当，会适得其反。

防止多个默认路由冲突

systemd-networkd 未按接口类型设置默认路由度量，使用多个网络设备时，需手动设置度量值。例如，下列 \ip route\ 输出显示多个默认路由：

```
ip route
default via 10.30.1.1 dev eno2 proto dhcp src 10.30.1.15 metric 1024
default via 192.168.1.254 dev eno1 proto dhcp src 172.18.105.104 metric 1024
```

由于度量值一致，系统会产生竞争条件。根据接口启用顺序选择默认路由，例如 eno2 先启用则被优先选中，可能导致 eno1 可用时却被忽略。

为避免此问题，应为各接口设置不同的 `RouteMetric=` 值。参考 #有线与无线接口绑定区段的示例。

如某设备不应成为默认路由源，可在其 .network 中使用 `UseRoutes=false` 选项，忽略 DHCP 提供的路由条目。该配置对于仅连接至另一台设备时特别有用。

在已有接口上配置第二个静态 IP 及其独立 MAC

若希望系统在路由器中识别为两个不同设备，可为某接口创建拥有独立 IP 和 MAC 的虚拟接口。

在物理接口上添加一个 macvlan 虚拟接口并指定唯一 MAC：

```
/etc/systemd/network/25-eth210.netdev
[NetDev]
Name=eth210
Kind=macvlan
MACAddress=00:11:22:33:44:55

[MACVLAN]
Mode=bridge
```

然后为该虚拟接口创建配置文件，使用相同子网和网关，但 IP 避开 DHCP 分配范围：

```
/etc/systemd/network/25-eth210.network
[Match]
Name=eth210

[Network]
Address=192.168.132.210/24
Gateway=192.168.132.1

[Route]
Destination=192.168.132.0/24
Metric=2
```

该虚拟 macvlan 接口的路由度量定为 2，因此除非特别指定，否则系统会优先通过主接口（度量值为默认 1）访问网络。

最后，请在主接口的 \.network\ 文件的 \\[Network\]\ 段中添加 `MACVLAN=eth210`！

为了让路由器“认识”这个新 MAC 地址并接受它，可作为 root 运行 `arping -I eth210 192.168.132.1`。配置路由器后，你可以测试虚拟接口的互联网连通性，例如运行：`curl --interface 192.168.132.210 ifconfig.me`应会返回你的公网 IP。

故障排查

启动时"挂载"服务失败

如果你运行的是 Samba 或 NFS 等服务，而它们在网络尚未启动时就尝试启动而失败，可以考虑 enable `systemd-networkd-wait-online.service`。不过这通常不是必要的，因为大多数网络服务即使网络尚未完全配置，也能正常启动。

systemd-resolve 不搜索本地域

systemd-resolved 在仅提供主机名时，可能不会自动搜索本地域，即使相应的 *.network* 文件中设定了 `UseDomains=yes` 或 `Domains=[domain-list]`，并且在 `resolv.conf` 中正确生成了 `search [domain-list]`。

你可以通过运行 `networkctl status` 或 `resolvectl status` 来确认搜索域是否被正确读取。

可能的解决方案：

禁用 LLMNR，让 systemd-resolved 立即继续尝试 DNS 后缀。

简化 `/etc/nsswitch.conf` 中 `hosts` 条目（例如移除 `[!UNAVAIL=return]`）。

使用完整限定域名（FQDN）。

使用 `/etc/hosts` 解析主机名。

放弃使用 systemd 的 `resolve`，改用 glibc 的 DNS 解析方式。

第二台电脑无法使用桥接网络

如果第一台电脑有两个 LAN 接口，第二台电脑只有一个 LAN 并连接到第一台电脑，那么为了让第二台电脑能够通过桥接接口访问所有网络，需要执行以下命令：

```
# sysctl net.bridge.bridge-nf-filter-pppoe-tagged=0
# sysctl net.bridge.bridge-nf-filter-vlan-tagged=0
# sysctl net.bridge.bridge-nf-call-ip6tables=0
# sysctl net.bridge.bridge-nf-call-iptables=0
# sysctl net.bridge.bridge-nf-call-arptables=0
```

参阅

- systemd.networkd man page
- Tom Gundersen, main systemd-networkd developer, G+ home page
- Tom Gundersen posts on Core OS blog
- How to set up systemd-networkd with wpa_supplicant (WonderWoofy's walkthrough on Arch forums)

网络管理界面

服务跑起来后，改配置、查数据、排故障仍要反复敲命令。本章把这些操作搬进浏览器：一节管整机与网络服务的图形化配置，另三节是数据库面板，分别面向通用、PostgreSQL 与 MySQL。

先读《Webmin》与《PhpMyAdmin》：前者最常用，后者配置项最细；查多种数据库可先看《Adminer》；《PhpPgAdmin》只在确实运行 PostgreSQL 时才看。最易踩的坑是把装完软件包当成能用，配置和访问入口没接好，页面只会报错；数据库这类高权限入口不限制访问来源，等于把库摊在网上。

19.1　Webmin

难度 入门 | **篇幅** 2.1 KB | **所属** 网络管理界面

本节将讲到

- 安装
- 配置
- 用法

来自项目主页：

: Webmin 是用于 Unix 的系统管理的基于 Web 的界面。使用任何现代的 Web 浏览器，您都可以设置用户帐户，Apache，DNS，文件共享等等。Webmin 消除了手动编辑 Unix 配置文件（如 `/etc/passwd`）的需要，并允许您从控制台或远程管理系统。请参阅标准模块页面以获取 Webmin 内置的所有功能的列表，或查看屏幕截图。

安装

从 AUR 安装 webminAUR 软件包。

配置

要允许从远程计算机访问 Webmin，请将防火墙配置为允许访问 TCP 端口 10000。您可能希望将防火墙配置为仅限制对某些 IP 地址的访问。

更改端口

要更改端口，请在 `/etc/webmin/miniserv.conf` 文件中编辑 `port` 变量。

```
/etc/webmin/miniserv.conf
[...]
port=10000
[...]
```

（仅）绑定到 localhost

要（仅）将 Webmin 绑定到 `localhost`，请将以下行添加到 `/etc/webmin/miniserv.conf` 文件中。

```
/etc/webmin/miniserv.conf
bind=127.0.0.1
```

用法

启动 `webmin.service` 或启用它（如果您希望在引导时加载 webmin）。

在 Web 浏览器中，输入端口号为 10000 的服务器的 https 地址以访问 Webmin：

```
https://host:10000
```

您将需要输入运行 Webmin 的服务器的 root 密码才能使用 Webmin 界面并管理服务器。

19.2 Adminer

难度 入门 | **篇幅** 4.0 KB | **所属** 网络管理界面

本节将讲到

- 安装
- 配置
- 参见

Adminer 是一个用 PHP 编写的基于网络的数据库管理工具。它可以管理 MySQL、PostgreSQL、Sqlite3、MS SQL、Oracle 数据库和 Elasticsearch。

它是 PhpMyAdmin 的一个更简单的替代方案。你可以在官方网站页或 Wikipedia 找到更多关于这个项目的信息。

安装

安装 adminerAUR 软件包或下载 Adminer，然后将其手动放置在 document-root 中。

使用 adminerAUR 软件包时，Adminer 将安装为 `/usr/share/webapps/adminer/index.php`。

确保未注释 `/etc/php/php.ini` 中的正确扩展，例如，`extension=pdo_mysql` 会提供 MySQL 数据库管理。

配置

Apache

注意：确保Apache 的 PHP 扩展已正确配置

在 `/etc/httpd/conf/httpd.conf` 中添加以下一行：

```
Include conf/extra/httpd-adminer.conf
```

然后重启你的 Apache HTTP 服务器守护进程。

现在可以通过浏览 http://localhost/adminer 访问 Adminer。

Nginx

注意：确保已正确配置PHP FastCGI 接口。

以 `root` 权限使用`/usr/share/webapps/adminer` 创建 server entry：

```
/etc/nginx/sites-available/adminer.conf
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;

    server_name adminer.domain;
    root /usr/share/webapps/adminer;

    # Only allow certain IPs
    #allow 192.168.1.0/24;
    #deny all;

    index index.php;

    location / {
        try_files $uri $uri/ =404;
    }

    error_page 404 /index.php;

    # PHP configuration
    location ~ \.php$ {
      ...
    }
}
```

将 `adminer.conf` 符号链接到 `sites-enabled` 并重启 nginx。

Hiawatha

确保正确配置PHP FastCGI 接口。

然后在 `/etc/hiawatha/hiawatha.conf` 中添加以下 `VirtualHost` 块。

```
/etc/hiawatha/hiawatha.conf
VirtualHost {

    # If you set WebsiteRoot to /usr/share/webapps/adminer you do not need
followsymlinks
    # I symlinked the adminer folder to '/srv/http/adminer' so that I can easily
remember where it's located but
    # still set 'WebsiteRoot' to the real source directory. You could point WebsiteRoot
to the
    # symlinked directory, but you will have to set 'FollowSymlinks = yes' for that to
function properly

    Hostname = db.domainname.dom
    #FollowSymlinks = yes
    #WebsiteRoot = /srv/http/adminer
    WebsiteRoot = /usr/share/webapps/adminer
    AccessLogfile = /var/log/hiawatha/adminer/access.log
```

```
    ErrorLogfile = /var/log/hiawatha/adminer/error.log
    StartFile = index.php
    UseFastCGI = PHP7

}
```

然后重启 `hiawatha.service`。

参见

- Official Adminer 网页
- Author's weblog

19.3 PhpPgAdmin

难度 入门 | **篇幅** 4.7 KB | **所属** 网络管理界面

本节将讲到

- 安装
- 配置
- 访问 phpPgAdmin 安装
- 问题解决

翻译状态：

- 本文（或部分内容）译自 PhpPgAdmin，最近一次同步于 2024-7-30，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

phpPgAdmin 是一个基于网络的工具，可帮助使用 PHP 前端管理 PostgreSQL 数据库。

安装

PhpPgAdmin 需要一个带 PHP 的网络服务器，如 Apache。要设置它，请参阅 Apache HTTP 服务器和Apache HTTP 服务器#PHP。

安装 phppgadminAUR 软件包。

配置

PHP

您需要编辑 `/etc/php/php.ini`，取消注释以下一行，以启用 PHP 中的 `pgsql` 扩展：

```
extension=pgsql
```

您需要确保 PHP 可以访问 `/etc/webapps`。如有必要，请将其添加到 `/etc/php/php.ini` 中的 `open_basedir`：

```
open_basedir
= /srv/http/:/home/:/tmp/:/usr/share/pear/:/usr/share/webapps/:/etc/webapps
```

网络服务器

Apache

创建 Apache 配置文件：

```
/etc/httpd/conf/extra/phppgadmin.conf
Alias /phppgadmin "/usr/share/webapps/phppgadmin"
<Directory "/usr/share/webapps/phppgadmin">
    DirectoryIndex index.php
    AllowOverride All
    Options FollowSymlinks
    Require all granted

    # phppgadmin raises deprecated warnings that lead
    # to parsing errors in JS
    #php_flag display_startup_errors off
    #php_flag display_errors off
    #php_flag html_errors off
</Directory>
```

并将其包含在 `/etc/httpd/conf/httpd.conf` 中：

```
# phpPgAdmin configuration
Include conf/extra/phppgadmin.conf
```

您还需要连接 php7：

```
Include conf/extra/php7_module.conf
LoadModule php7_module modules/libphp7.so
```

默认情况下，每个人都能看到 phpPgAdmin 页面，若要更改，请编辑 `/etc/httpd/conf/extra/phppgadmin.conf` 以满足您的需求。例如，如果只希望在同一台机器上访问，请将 `Require all granted` 替换为 `Require local`。

Lighttpd

lighttpd 的 php 设置与 apache 完全相同。 在 lighttpd 配置中为 phppgadmin 设置一个别名。

```
 alias.url = ( "/phppgadmin" => "/usr/share/webapps/phppgadmin/")
```

然后在配置（server.modules 部分）中启用 mod_alias、mod_fastcgi 和 mod_cgi。

确保 lighttpd 已设置为服务 php 文件，Lighttpd#FastCGI。

重启 lighttpd 并浏览 http://localhost/phppgadmin/index.php 。

nginx

请确保按照 nginx#nginx 配置中的说明，为 PHP 设置 nginx#FastCGI 和单独的配置文件。

使用此方法，您将以 `phppgadmin.<domain>` 的身份访问 PhpPgAdmin。

您可以使用服务器块设置子域（或域），如

```
server {
    server_name     phppgadmin.<domain.tld>;
    root    /usr/share/webapps/phppgadmin;
    index   index.php;
    include php.conf;
}
```

phpPgAdmin 配置

phpPgAdmin 的配置文件位于 `/etc/webapps/phppgadmin/config.inc.php`。

如果 PostgreSQL 服务器位于 `localhost`，则可能需要编辑以下一行：

```
$conf['servers']['host'] = ;
```

到：

```
$conf['servers']['host'] = 'localhost';
```

访问 phpPgAdmin 安装

phpPgAdmin 安装完成。在开始使用之前，您需要通过重启 `httpd.service` 以重启 apache 服务器。

您可以通过 http://localhost/phppgadmin/ 访问 phpPgAdmin 安装。

问题解决

由于安全原因，登录被禁止

如果 extra_login_security 设置为 "true"，那么通过 phpPgAdmin 进行的无密码登录或特定用户名（*psql*、*postgres*、*root*、*administrator*）登录将被拒绝。只有在阅读了常见问题并了解如何修改 PostgreSQL 的 `pg_hba.conf` 以启用带密码的本地连接之后，才能将此设置为 `false`。

编辑 `/etc/webapps/phppgadmin/config.inc.php` 并更改以下行：

```
$conf['extra_login_security'] = true;
```

到：

```
$conf['extra_login_security'] = false;
```

然后重启 `postgresql.service`。

19.4 PhpMyAdmin

难度 进阶 | **篇幅** 10.2 KB | **所属** 网络管理界面

本节将讲到

- 安装
- 运行
- 配置
- 参见

phpMyAdmin是一个基于网页的，帮助管理MySQL数据库的Apache/PHP前端。

安装

安装软件包 phpmyadmin包。

运行

PHP

确保PHP的MySQL扩展已经被启用。

你也可以启用 `extension=bz2` 和 `extension=zip` 扩展以支持压缩。

注意：如果设置了`open_basedir`，`/usr/share/webapps` 和 `/etc/weppapps/` 必须加入到`/etc/php/php.ini` 中的`open_basedir`。参考 PHP#配置\[损坏的链接：无效的章节\]。

Apache

按照Apache HTTP Server#PHP设置PHP。 创建Apache配置文件：

```
/etc/httpd/conf/extra/phpmyadmin.conf
Alias /phpmyadmin "/usr/share/webapps/phpMyAdmin"
<Directory "/usr/share/webapps/phpMyAdmin">
    DirectoryIndex index.php
    AllowOverride All
    Options FollowSymlinks
    Require all granted
</Directory>
```

在`/etc/httpd/conf/httpd.conf`加入配置文件：

```
# phpMyAdmin configuration
Include conf/extra/phpmyadmin.conf
```

注意：默认情况下，每个可以访问Web服务器的人都可以通过这个URL访问phpMyAdmin登录页面。要改变此设置，编辑`/etc/httpd/conf/extra/phpmyadmin.conf`。例如，你只想从本地访问phpMyAdmin，将`Require all granted`改为`Require local`。注意，这将禁止远程访问phpMyAdmin，如果你想安全地远程访问phpMyAdmin，你可以设置一个加密的SOCKS通道。

修改了 Apache 配置文件之后，restart `httpd.service`。

Lighttpd

配置Lighttpd，确保能正常使用PHP，并且启用`mod_alias`。

在配置文件中为phpmyadmin加入下列alias：

```
alias.url = ( "/phpmyadmin" => "/usr/share/webapps/phpMyAdmin/")
```

Nginx

确保正确配置FastCGI，并使用 nginx#Server blocks 来使得管理更简单。

子域名

根据个人喜好，通过子域名访问 phpMyAdmin, 如 `https://*pma.domain.tld*`：

```
/etc/nginx/sites-available/pma.domain.tld
```

```
server {
    server_name pma.domain.tld;
    ; listen 80; # also listen on http
    ; listen [::]:80;
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    index index.php;
    access_log /var/log/nginx/pma.access.log;
    error_log /var/log/nginx/pma.error.log;

    # Allows limiting access to certain client addresses.
    ; allow 192.168.1.0/24;
    ; allow my-ip;
    ; deny all;

    root /usr/share/webapps/phpMyAdmin;
    location / {
        try_files $uri $uri/ =404;
    }

    error_page 404 /index.php;

    location ~ \.php$ {
        try_files $uri $document_root$fastcgi_script_name =404;

        fastcgi_split_path_info ^(.+\.php)(/.*)$;
        fastcgi_pass unix:/run/php-fpm/php-fpm.sock;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;

        fastcgi_param HTTP_PROXY "";
        fastcgi_param HTTPS on;
        fastcgi_request_buffering off;
   }
}
```

子目录

或者使用子目录，如 `https://*domain.tld*/phpMyAdmin`：

```
/etc/nginx/sites-available/domain.tld
server {
    server_name domain.tld;
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    index index.php;
    access_log /var/log/nginx/domain.tld.access.log;
    error_log /var/log/nginx/domain.tld.error.log;

    root /srv/http/domain.tld;
    location / {
        try_files $uri $uri/ =404;
    }

    location /phpMyAdmin {
        root /usr/share/webapps/phpMyAdmin;
    }

    # Deny static files
    location ~ ^/phpMyAdmin/(README|LICENSE|ChangeLog|DCO)$ {
       deny all;
    }

    # Deny .md files
    location ~ ^/phpMyAdmin/(.+\.md)$ {
      deny all;
   }
```

```
    # Deny setup directories
    location ~ ^/phpMyAdmin/(doc|sql|setup)/ {
        deny all;
    }

    #FastCGI config for phpMyAdmin
    location ~ /phpMyAdmin/(.+\.php)$ {
        try_files $uri $document_root$fastcgi_script_name =404;

        fastcgi_split_path_info ^(.+\.php)(/.*)$;
        fastcgi_pass unix:/run/php-fpm/php-fpm.sock;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;

        fastcgi_param HTTP_PROXY "";
        fastcgi_param HTTPS on;
        fastcgi_request_buffering off;
    }
}
```

配置

主配置文件位于 `/usr/share/webapps/phpMyAdmin/config.inc.php`。

定义远程 MySQL 服务器

如果MySQL服务器不在本机上，添加如下行到配置文件:

```
$cfg['Servers'][$i]['host'] =  'example.com';
```

使用安装脚本

要使用phpMyAdmin安装脚本（例如 http://localhost/phpmyadmin/setup ）， 确保`http`用户可以写入`/usr/share/webapps/phpMyAdmin`。

```
# mkdir /usr/share/webapps/phpMyAdmin/config
# chown http:http /usr/share/webapps/phpMyAdmin/config
# chmod 750 /usr/share/webapps/phpMyAdmin/config
```

添加blowfish_secret passphrase

需要设置一个唯一的32位的字符串从而充分地使用blowfish算法，从而避免……（配置文件现在需要一个短语密码）的错误。

(blowfish_secret)*:*

```
/usr/share/webapps/phpMyAdmin/config.inc.php
$cfg['blowfish_secret'] = '...';
```

启用配置存储

例如table linking,change tracking,PDF creation,bookmarking queries的附加功能默认是被禁用的，并且会提示The phpMyAdmin configuration storage is not completely configured, some extended features have been deactivated.

注意：下面的例子将`controluser`假设为**pma**，`controlpass` 假设为**pmapass**。

在`/usr/share/webapps/phpMyAdmin/config.inc.php`取消注释(移除开头的//)，并按需更改为所需的凭据：

```
/usr/share/webapps/phpMyAdmin/config.inc.php
/* User used to manipulate with storage */
// $cfg['Servers'][$i]['controlhost'] = 'my-host';
// $cfg['Servers'][$i]['controlport'] = '3306';
$cfg['Servers'][$i]['controluser'] = 'pma';
$cfg['Servers'][$i]['controlpass'] = 'pmapass';

/* Storage database and tables */
$cfg['Servers'][$i]['pmadb'] = 'phpmyadmin';
$cfg['Servers'][$i]['bookmarktable'] = 'pma__bookmark';
$cfg['Servers'][$i]['relation'] = 'pma__relation';
$cfg['Servers'][$i]['table_info'] = 'pma__table_info';
$cfg['Servers'][$i]['table_coords'] = 'pma__table_coords';
$cfg['Servers'][$i]['pdf_pages'] = 'pma__pdf_pages';
$cfg['Servers'][$i]['column_info'] = 'pma__column_info';
$cfg['Servers'][$i]['history'] = 'pma__history';
$cfg['Servers'][$i]['table_uiprefs'] = 'pma__table_uiprefs';
$cfg['Servers'][$i]['tracking'] = 'pma__tracking';
$cfg['Servers'][$i]['userconfig'] = 'pma__userconfig';
$cfg['Servers'][$i]['recent'] = 'pma__recent';
$cfg['Servers'][$i]['favorite'] = 'pma__favorite';
$cfg['Servers'][$i]['users'] = 'pma__users';
$cfg['Servers'][$i]['usergroups'] = 'pma__usergroups';
$cfg['Servers'][$i]['navigationhiding'] = 'pma__navigationhiding';
$cfg['Servers'][$i]['savedsearches'] = 'pma__savedsearches';
$cfg['Servers'][$i]['central_columns'] = 'pma__central_columns';
$cfg['Servers'][$i]['designer_settings'] = 'pma__designer_settings';
$cfg['Servers'][$i]['export_templates'] = 'pma__export_templates';
```

创建数据库

有两种方法来创建需要的表：

- 使用phpMyAdmin导入 `/usr/share/webapps/phpMyAdmin/sql/create_tables.sql`。
- 在命令行中执行`mysql -u root -p < /usr/share/webapps/phpMyAdmin/sql/create_tables.sql`。

创建数据库用户

要赋予`controluser`所需的权限，执行如下的查询：

注意：确保将所有的`pma`和`pmapass`替换为在`config.inc.php`设置的值。如果正在为远程数据库配置，还需将`localhost`改为适当的主机。

```
GRANT USAGE ON mysql.* TO 'pma'@'localhost' IDENTIFIED BY 'pmapass';
GRANT SELECT (
    Host, User, Select_priv, Insert_priv, Update_priv, Delete_priv,
    Create_priv, Drop_priv, Reload_priv, Shutdown_priv, Process_priv,
    File_priv, Grant_priv, References_priv, Index_priv, Alter_priv,
    Show_db_priv, Super_priv, Create_tmp_table_priv, Lock_tables_priv,
    Execute_priv, Repl_slave_priv, Repl_client_priv
    ) ON mysql.user TO 'pma'@'localhost';
GRANT SELECT ON mysql.db TO 'pma'@'localhost';
GRANT SELECT ON mysql.host TO 'pma'@'localhost';
GRANT SELECT (Host, Db, User, Table_name, Table_priv, Column_priv)
    ON mysql.tables_priv TO 'pma'@'localhost';
```

为了使用书签和关系特性，设置如下的权限：

```
GRANT SELECT, INSERT, UPDATE, DELETE ON phpmyadmin.* TO 'pma'@'localhost';
```

重新登录以启用新特性。

启用templates catching

在/usr/share/webapps/phpMyAdmin/config.inc.php添加如下行：

```
$cfg['TempDir'] = '/tmp/phpmyadmin'
```

移除配置目录

当配置完成之后，可以移除临时配置目录，这也会抑制来自网页交互的警告：

```
# rm -r /usr/share/webapps/phpMyAdmin/config
```

安装主题

主题的位置在 /usr/share/webapps/phpMyAdmin/themes。你可以在 https://www.phpmyadmin.net/themes/ 找到新主题。

直接下载并将主题提取到相应位置，在 phpmyadmin 刷新之后它将正常工作。然而，你下载的主题必须与 phpmyadmin 的版本相对应，适用于老版本的主题无法使用。

参见

- Wikipedia
- Official website

网络配置

有线与无线是联网第一道坎：内核要认出网卡，地址要自动下发，无线要接上加密接入点。以太网讲驱动与链路故障，Dhcpcd 管租约与钩子，Wpa supplicant 与 Iwd 是两套并列的无线方案。

建议先读以太网与 Dhcpcd，二者构成有线环境的最小闭环；无线在 Wpa supplicant 与 Iwd 中择一精读，另一篇略读。两个坑要留意：一是把两套无线方案当成配套同时启用，它们同属一层实现，会互相抢设备；二是链路不通就重装软件包，其实每节末尾的故障排除已按驱动、租约、认证分层写好。

20.1　网络配置/以太网

难度 进阶 | **篇幅** 13.7 KB | **所属** 网络配置

本节将讲到

- 设备驱动
- 提示与技巧
- 故障排除

本页描述了以太网的详细信息，一般的网络配置则为网络配置所涵盖。

设备驱动

检测驱动状态

udev 会探测你的网卡（网络接口控制器）并在启动时自动载入必要的内核模块。通过 `lspci -v` 输出中的“Ethernet controller”(或类似条目)可以确认网络设备所使用的内核驱动和内核模块。例如：

```
$ lspci -v
02:00.0 Ethernet controller: Attansic Technology Corp. L1 Gigabit Ethernet Adapter (rev
b0)
       ...
       Kernel driver in use: atl1
       Kernel modules: atl1
```

`dmesg | grep *module_name*` 命令可以检查是否已经加载了驱动。例如：

```
# dmesg | grep atl1
...
atl1 0000:02:00.0: eth0 link is up 100 Mbps full duplex
```

如果驱动加载成功，就跳过下一节，否则，需要知道特定型号的网络设备需要哪一个模块。

加载设备模块

在互联网上搜索芯片组需要的模块/驱动。常见的驱动模块有用于 Realtek 芯片组网卡的 `8139too`，或者用于 Sis 芯片组网卡的 `sis900`。确定模块后，请尝试手动加载

它。如果碰到了未找到模块的错误，可能是驱动没有包括在 Arch 的内核中。可以在 AUR 中搜索模块名称进行安装。

如果 udev 在引导时不能自动侦测和加载适当的模块，请查阅 Kernel module#自动加载模块。

提示与技巧

用于笔记本的 ifplugd

提示：dhcpcd 也提供了同样的功能。

ifplugd包 是一个守护进程，在网络适配器插入的时候自动配置网络，当网络断开的时候自动取消配置。因为这个程序只会在网络实际接入的时候才会配置网络接口，所以非常适用于笔记本电脑这样的使用移动式的网络，还可以用于不重启电脑、不使用命令行就自动重启网络。

在默认情况下，它会工作于 `eth0` 设备。要更改这个设置（以及更改其他设置，比如等待时间），可以编辑 `/etc/ifplugd/ifplugd.conf`。

注意：netctl 软件包包含了 `netctl-ifplugd@.service`，否则你可以使用 ifplugd包 软件包中的 `ifplugd@.service`。例如启用 `ifplugd@eth0.service`。

故障排除

更换了连接有线调制解调器的计算机

许多有线网络运营商（例如加拿大最大的有线电视公司 Vidéotron，还有中国大陆的有线电视宽带公司）都使用记录网卡 MAC 地址的方法将有线调制解调器配置为只能一台计算机使用。一旦调制解调器获得第一台连接它的设备的 MAC 地址，就不会响应任何其它 MAC 地址。这样如果你换了台计算机（或者路由器），由于新计算机（或者路由器）的 MAC 地址和旧的不同，就没法连接调制解调器了。这时候必须复位调制解调器以使它重新进行识别。 你得关闭调制解调器电源，然后重新打开。一旦调制解调器重启并再次登录完毕（指示灯熄灭），重启新连接的设备以使它发起一个 DHCP 请求，或者手动发起 DHCP 请求。

如果这个方法不能奏效，你需要克隆原来机器上的 MAC 地址。参见 MAC 地址伪装。

显式拥塞通知

显式拥塞通知(Explicit Congestion Notification, ECN) 可能在老旧/劣质的路由器上造成网络拥堵问题。从 systemd 239 开始，它同时为入站流量和出站流量启用。

设置仅当传入连接请求时才启用 ECN（内核默认设置，相当安全）：

```
# sysctl net.ipv4.tcp_ecn=2
```
设置完全禁用 ECN（例如，测试 ECN 是否引起问题）：

```
# sysctl net.ipv4.tcp_ecn=0
```
参见内核文档。

Realtek 没有连接/网络唤醒故障

使用基于 Realtek 8168 8169 8101 8111(C) 芯片网卡（独立网卡和板载）的用户也许会发现这个故障，启动时网卡不可用，网卡上的连接指示灯不亮。这通常会发生在安装了 Windows 的双启动系统上。在 Windows 下使用 Realtek 官方驱动（2007 年 5 月后的版本）会引发故障。新驱动通过在 Windows 关机时禁止网卡来关闭网络唤醒功能，直到下一次 Windows 启动前网卡都会一直不可用。如果是这个原因影响的你，你会观察到连接指示灯在 Windows 启动前一直熄灭，在 Windows 关机过程中也会熄灭。正常情况应该是只要计算机一直开着，即使在 POST 加电过程中，连接指示灯也应该一直亮着。这个故障也会影响其它没有安装新驱动的操作系统（例如各种 Live CD）。这里给出几种解决方案：

在 Linux 中启用网卡

参考网络配置#启用和禁用网络接口来启用网卡。

还原/变更 Windows 驱动

你可以将你的 Windows 网卡驱动还原回微软自带的驱动（如果有的话），或者安装 2007 年 5 月份以前的 Realtek 官方驱动（也许在网卡附带的 CD 上）。

启动 Windows 驱动里的网络唤醒功能

也许最好最快的修复方法就是改变 Windows 驱动里的这个设置。这个方法可以解决很多其它操作系统而不仅仅是 Arch 的麻烦。在 Windows 的设备管理器里，找到你的 Realtek 网卡，双击它。在“高级”标签页中，开启“wake-on-lan after shutdown”选项。

例如在 Windows XP 里

```
右键点击我的电脑
-->管理
  -->设备管理器
    -->网络适配器
      -->双击 Realtek ...
        --> 高级标签页
          --> Wake-On-Lan After Shutdown
            --> 启用。
```
注意：新的 Realtek Windows 驱动程序中（已测试了 2009/01/22 GIGABYTE 上的 *Realtek 8111/8169 LAN Driver v5.708.1030.2008*）可能与这里的选项稍微有点不同，像 *Shutdown Wake-On-LAN > Enable*。似乎把它切换成 `Disable` 没有效果（你仍然可以在Windows关闭时看到连接指示灯熄灭）。一个比较不好的解决方法是引导 Windows，然后立即重启系统（执行非正常重启/关机），不给予 Windows

驱动程序关闭 LAN 的机会。连接指示灯将会保持亮着，网卡也会在 POST 之后保持可用——直到你再次进入 Windows 并正常关机。

在 BIOS/CMOS 中启用 LAN Boot ROM

尽管 Windows 驱动程序在系统关闭时禁用了它，但在 BIOS/CMOS 中设置 *Integrated Peripherals --> Onboard LAN Boot ROM --> Enabled*，系统启动时会重新激活 Realtek LAN 芯片。

注意：这个方法多次在 GIGABYTE GA-G31M-ES2L 主板，2009/02/05 发布的 BIOS 版本 F8 上测试成功。你的情况可能不同。

禁用 USB 自动挂起功能

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：不是所有人都使用了 TLP.（在Talk:网络配置/以太网讨论）

当使用省电功能时，特别是带 TLP 的 USB自动挂起，设备可能无法正确加载，导致 *NO-CARRIER*状态(用RT8156B测试)，也不能建立网络连接。

解决办法是将设备 ID 添加到 TLP 配置的 USB 黑名单（denylist）中，并重新连接设备。

Atheros 芯片组找不到网卡

有些用户的 Atheros 芯片无法正常工作 (至少在 2014 年 2 月的安装镜像中)。可以通过安装 backports-patchedAUR\[损坏的链接：package not found\] 解决。

Broadcom BCM57780

这个 Broadcom 芯片只有你指定模块的加载顺序后才能正常工作。这些模块是 `broadcom` 和 `tg3`，前者需要首先加载。

如果你的电脑有这个芯片，这些步骤应该有用：

- 从 `lspci` 的输出中找到你的网卡：

```
$ lspci | grep Ethernet
02:00.0 Ethernet controller: Broadcom Corporation NetLink BCM57780 Gigabit Ethernet PCIe (rev 01)
```

- 如果你的有线网络不能工作，尝试断开网线，然后以 root 权限实施以下步骤：

```
# modprobe -r tg3
# modprobe broadcom
# modprobe tg3
```

- 插回网线，检查模块是否成功：

```
# dmesg | grep tg3
```

- 如果现在你的故障解决了，你可以把 `broadcom` 和 `tg3` （以此顺序）加入到 `/etc/mkinitcpio.conf` 的 `MODULES` 一行，使得更改持久化：

```
/etc/mkinitcpio.conf
MODULES=(.. broadcom tg3 ..)
```

- 重新生成 initramfs
- 还有一种方法是创建 `/etc/modprobe.d/broadcom.conf`：

```
softdep tg3 pre: broadcom
```

注意：这些方法可能也适用于其它芯片，例如 BCM57760。

Realtek RTL8111/8168B

```
# lspci | grep Ethernet
03:00.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL8111/8168B PCI Express Gigabit Ethernet controller (rev 02)
```

这个网络适配器应被内核的`r8169`模块识别。 但是，对于某些芯片版本，网络连接可能一直是断开状态。 可以在 official repositories中找到r8168包 作为`r8169`的替代品，在这种情况下，其应该可以实现稳定可靠的连接。 将`r8169`列入黑名单 Blacklist，如果 udev没有自动加载r8168包 ，请参见内核模块 Kernel module#自动加载模块

**本文或本章节的事实准确性存在争议。**

原因： "某些修订版本：", 不是驱动程序导致的问题，而是DNS服务器没有正确配置 （在 Talk:网络配置/以太网 中讨论）

对于此网络适配器的某些修订版，驱动程序中的另一个故障是IPv6支持不佳。 如果遇到诸如网页挂起和速度慢之类的问题，禁用IPv6功能 IPv6#Disable functionality可能会有所帮助。

配有 Realtek 8111/8168/8411 的 Gigabyte 主板

使用诸如 Gigabyte GA-990FXA-UD3 这样的主板，在关闭 IOMMU 的配置下启动（可能是默认设置）将会使网络连接出问题，通常是无法连接或连接上了但没有流量。这适用于板载网卡和小节标题中的任何其他 pci-NIC，因为 IOMMU 设置会影响主板上的整个网络接口。启用 IOMMU 并使用安装介质引导会在一秒钟内引发 AMD I-10/xhci 页面错误，但随后会正常启动，板载网卡完全正常运行（即使使用 r8169 模块）。在配置安装引导过程时，添加 `iommu=soft` 作为内核参数，以消除引导时的错误信息，并恢复 USB3.0 功能。

配有 Realtek 8111/8168/8411 的 微星主板

使用诸如 "MicroStar B450M MORTAR TITANIUM" 这样的主板, 拔插网线或者重启路由器的 DHCP 服务可能会导致 `r8169` 进入降档状态, 从千兆带宽（1000Mbit/s）降低到百兆（100 Mbit/s）。内核 log 会打印（可以通过 dmesg 命令查看）：

```
# dmesg | grep r8169
Generic FE-GE Realtek PHY r8169-2200:00: Downshift occurred from negotiated speed 1Gbps
to actual speed 100Mbps, check cabling!
r8169 0000:22:00.0 enp34s0: Link is Up - 100Mbps/Full (downshifted) - flow control
rx/tx
```

这种情况下，可以通过重启网络接口进行恢复。例如执行下面的命令：

```
# ip link set dev enp34s0 down
# ip link set dev enp34s0 up
```

20.2 Dhcpcd

难度 进阶 | **篇幅** 15.0 KB | **所属** 网络配置

本节将讲到

- 安装
- 运行
- 配置
- 钩子
- 小技巧

dhcpcd 是 DHCP 和 DHCPv6 客户端，是目前功能最丰富的开源 DHCP 客户端。项目主页包含了完整的功能列表。

注意：Roy Marple 的 *dhcpcd*（DHCP **client** daemon）和 Internet Systems Consortium 的 dhcpd（DHCP **(server)** daemon）是不同的软件。

安装

安装软件包 dhcpcd包。

dhcpcd-uiAUR 是 dhcpcd 和 GTK 前端，提供了对 wpa_supplicant 的可选支持。功能包括配置对话框、输入无线网络的密码等。

运行

要为*所有的*网络接口启动守护进程，启动或启用 `dhcpcd.service`。

要单独为指定的网络接口启动守护进程，启动或启用模板单元 `dhcpcd@*interface*.service`，其中的 *interface* 名可从这里查询。

建议使用模板单元方式，详情请参阅 #dhcpcd 和 systemd 网络接口。无论哪种方式都会分配一个动态 IP 地址。要分配静态 IP 地址，请参阅 #静态配置。

配置

主配置文件是 `/etc/dhcpcd.conf`，详情请参考 dhcpcd.conf(5)，下面会介绍一些常用选项。

DHCP 静态路由

如果要在客户端设置静态路由，请使用 `/etc/dhcpcd.exit-hook`。下面的例子中添加了一个 VPN 子网 `10.11.12.0/24` 到网关 `192.168.192.5` 的静态路由：

```
/etc/dhcpcd.exit-hook
ip route add 10.11.12.0/24 via 192.168.192.5
```

可以在文件中配置多个路由。

DHCP 客户端标识

服务器可以通过下列方式不同的 DHCP 客户端：

1. 主机名（或客户端发送的主机名），
2. 所用网卡的 MAC 地址,
3. Identity Association ID（身份关联 ID IAID），区分不同使用场景或接口的标识，
4. DHCP 唯一标识（DUID）。

详情请参考 RFC 3315。

DHCP 服务器通过配置决定申请 DHCP IP 租约时，哪些是必须的，哪些是可选的。

注意：*dhcpcd* 默认配置的是最常用的方式。服务器会自动确认上述标识，只有在出现问题时才需要额外配置。

如果无法通过 *dhcpcd* 默认配置获取 IP 地址，可以在 `dhcpcd.conf` 中尝试如下配置：

- `hostname` 发送 `/etc/hostname` 中配置的主机名
- `clientid` 发送 MAC 地址作为标识
- `iaid <interface>` 生成并发送 IAID,可以在接口块（`interface <interface>`）中使用，参考：)，下面的选项更常用：
- `duid` 将联合使用 DUID 和 IAID 作为标识。

DUID 的数值配置在 `/var/lib/dhcpcd/duid` 中，为了保证 DHCP 租约的有效性，需要保证 DUID 在整个网络中是唯一的，而 IAID 需要能区分每一个接口（RFC 4361）。

如果运行的是动态 DNS，请确保三个都是唯一的。如果网络中出现重复的 DUID，例如克隆的虚拟机中，有不同的主机名和 MAC 地址，但是没有修改 DUID 时，最新获取IP地址的客户端会清除之前 DUID 获取的地址。

静态配置

需要的配置在网络配置中均有说明，通常包含网络接口名称、IP 地址、路由地址和域名服务器。

在 `/etc/dhcpcd.conf` 中为 *dhcpcd* 配置静态地址：

```
/etc/dhcpcd.conf
interface eth0
static ip_address=192.168.0.10/24
static routers=192.168.0.1
static domain_name_servers=192.168.0.1 8.8.8.8
```

还可以进行更复杂的配置，比如 `arping` 选项，详情请参考 dhcpcd.conf(5)。

备用配置

可以为 *dhcpcd* 添加备用设置，当 DHCP 续租失败时使用。这对于无显示设备机器特别有用，动态地址无法获取时，使用静态地址作为备用，确保机器有可用的网络连接。

下面示例为 `static_eth0` 配置了 `192.168.1.23` 静态地址，`192.168.1.1` 网关，并将此配置设置为 `eth0` 的备用配置。

```
/etc/dhcpcd.conf
# define static profile
profile static_eth0
static ip_address=192.168.1.23/24
static routers=192.168.1.1
static domain_name_servers=192.168.1.1

# fallback to static profile on eth0
interface eth0
fallback static_eth0
```

钩子

dhcpcd 会按字母顺序执行 `/usr/lib/dhcpcd/dhcpcd-hooks/` 中配置的钩子，详情请参考 dhcpcd.conf(5) 和 dhcpcd-run-hooks(8)。

注意：

- 可以在 `dhcpcd.conf` 中使用 `nohook` 禁用钩子。
- 可以用 `env` 选项为所有钩子设置环境变量。例如要强制设置主机名，请使用 `env force_hostname=YES`。

10-wpa_supplicant

通过创建一个符号链接启用这个钩子，不仅可以在当版本中使用，后期程序更新时也可使用：

```
# ln -s /usr/share/dhcpcd/hooks/10-wpa_supplicant /usr/lib/dhcpcd/dhcpcd-hooks/
```

如果启用了 `10-wpa_supplicant` 钩子，将会在无线接口上自动加载 wpa_supplicant，它仅在以下情况开启：

- 不存在已在接口上监听的 *wpa_supplicant* 进程。
- 至少存在一个 *wpa_supplicant* 配置文件。默认情况下，*dhcpcd* 将会按顺序检查以下配置文件：

```
/etc/wpa_supplicant/wpa_supplicant-interface.conf
/etc/wpa_supplicant/wpa_supplicant.conf
/etc/wpa_supplicant-interface.conf
/etc/wpa_supplicant.conf
```

但也可通过设置 `env wpa_supplicant_conf=*configuration_file_path*` 在 `/etc/dhcpcd.conf` 中添加自定义配置文件路径。

注意：钩子会在找到第一个配置文件时停止执行，因此，如果您有多个 *wpa_supplient* 配置文件，则应该考虑到这一点，否则“dhcpcd”可能会使用错误的配置文件。

如果您通过 *wpa_supplicant* 管理您的无线连接，钩子可能会创建一些不必要的连接事件。例如，如果您关闭了 *wpa_supplicant*，钩子可能会使无线接口再次打开。此外，如果您正在使用 netctl-auto，*wpa_supplicant* 将会根据 `/run/network/wpa_supplicant_*interface*.conf` 配置文件自动启动。因此，从钩子重新启动接口是不必要的，并且可能会导致 `/etc/wpa_supplicant/wpa_supplicant.conf` 文件在启动时解析错误，因为该文件在默认打包的版本中只包含虚拟的预设值。通过移出添加钩子的符号链接以禁用钩子，或者向 `dhcpcd.conf` 中添加 `nohook wpa_supplicant` 参数以解决这个问题。

小技巧

禁用 ARP 探测加速 DHCP 响应

dhcpcd 在内部实现了 DHCP 标准提议（RFC 2131），以通过 ARP 探测验证指定 IP 地址是否已被其他设备使用，在家庭网络环境下这通常是不必要的，因此，可以通过禁用 ARP 探测在每次连接时节约 5 秒左右的时间：

```
/etc/dhcpcd.conf
noarp
```

相当于将 `--noarp` 参数传递给 `dhcpcd`，并禁用 ARP 探测，从而加快使用 DHCP 连接到网络的速度。

移除旧的 DHCP 租约

在文件 `/var/lib/dhcpcd/*interface*.lease` 中，`*interface*` 是获得租约的接口名称，包含 DHCP 服务器发送的实际可用的 DHCP 租约响应。对一个无线接口来说，文件名则为 `/var/lib/dhcpcd/*interface*-*ssid*.lease`，其中 `*ssid*` 是无线网络名称。这些文件用于确定服务器分配的最近一个租约，文件的 `mtime` 属性用于确定租约的公告时间。如果最近的租约信息可用，则该租约信息将用

于请求先前在网络上使用的相同 IP 地址，如果您不想要这样做，只需删除这些文件即可。

如果 DHCP 服务器仍然分配相同的 IP 地址，可能是因为服务器被配置为保持固定分配并且识别出了客户端发送的客户端 id（client id）或者是 DUID（详见 #DHCP 客户端标识），您可以通过停止 *dhcpcd* 并移除或重命名 `/var/lib/dhcpcd/duid` 文件来进行测试，*dhcpcd* 会在下次运行时重新生成这个文件。

请注意，DUID 旨在作为重启后和不同接口之间的持久化的机器标识符。如果要将系统迁移到新计算机上，则不应该留存这个文件。

多重引导获取不同 IP 地址

如果您使用双引导 Arch 和 macOS 或者是 Windows，并且每个系统想要获得不同的 IP 地址，您可以通过在每个操作系统指定不同的 DUID 来控制获得的 IP 地址。

在 Windows 下 DUID 应该保存在以下注册键下：

```
\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters\Dhcpv6DUID
```

macOS 中可以在 `Network\adapter\dhcp preferences panel` 中直接访问修改。

如果您使用 dnsmasq 作为 DHCP 服务器，可以在其配置文件中通过恰当指定 `dhcp-host=` 规则来使用不同的 DUID。

/etc/resolv.conf

如果系统上 resolvconf（英语：resolvconf） 可用，DNS 信息将会发送给它处理，否则，*dhcpcd* 将会自行将 DNS 信息写入 */etc/resolv.conf*。可以通过禁用 `/usr/lib/dhcpcd/dhcpcd-hooks/20-resolv.conf` 钩子防止 *dhcpcd* 重写 `/etc/resolv.conf` 文件，只需在 `/etc/dhcpcd.conf` 最后一段中添加以下内容即可：

```
nohook resolv.conf
```

注意，一般来说，禁用此钩子也会使 dhcpcd 禁用 resolvconf。

或者，您可以创建一个名为 `/etc/resolv.conf.head` 的文件，其中包含您的 DNS 服务器信息。dhcpcd 将把这个文件的内容放在 `/etc/resolv.conf` 的开头。

或者，您可以将 dhcpcd 配置为每次使用相同的 DNS 服务器。想要这么做，请在 `/etc/dhcpcd.conf` 的末尾添加以下行，其中 `*dns-server-ip-addressses*` 是一个以空格分隔的 DNS IP 地址列表。

```
static domain_name_servers=dns-server-ip-addresses
```

例如，要设置 Google DNS 服务器：

```
static domain_name_servers=8.8.8.8 8.8.4.4
```

提示：当使用 openresolv（英语：openresolv） 时，可以在 `/etc/resolvconf.conf` 中设置 DNS 服务器地址，这样，它们就不会被任何支持 *resolvcconf* 的软件覆盖。

问题解决

客户端 ID

如果您处在根据 MAC 地址过滤客户端 ID 的 DHCPv4 的网络中，则可能需要更改以下行：

```
/etc/dhcpcd.conf
# 根据 RFC4361，DHCPv4 使用和 DHCPv6 相同的 DUID+IAID 作为客户端 ID。
duid
```

改为：

```
/etc/dhcpcd.conf
# 使用接口的硬件地址作为客户端 ID（DHCPv4）。
clientid
```

否则，您可能无法获得租约，因为 DHCP 服务器可能无法正确读取 DHCPv6-风格的客户端 ID。有关更多信息，请参阅 RFC 4361。

通过先释放 IP 地址检查 DHCP 问题

此问题可能在 DHCP 获得错误的 IP 分配时发生，例如当两个路由器通过 VPN 相连时。路由器通过 VPN 连接分配到了地址。要修复该问题，请以 root 身份释放该 IP 地址：

```
# dhcpcd -k
```

然后请求一个新地址：

```
# dhcpcd
```

您可能需要多次运行这两个命令。

路由器不兼容导致的问题

对于某些（不兼容的）路由器，除非您注释掉 `/etc/dhcpcd.conf` 中的以下行，否则无法正确连接。

```
require dhcp_server_identifier
```

除非您的网络上有多个 DHCP 服务器（非典型），否则这不会导致问题，详情请见这个页面。

dhcpcd 和 systemd 网络接口

可以在不指定接口的情况下启用 `dhcpcd.service`。但系统启动时，该单元可能会在 *systemd-udevd* 尝试应用一个可预测的网络接口名称时与其产生竞争状态：

```
error changing net interface name wlan0 to wlp4s0: Device or resource busy"
```

要避免该问题，请根据 dhcpcd.conf(5)，使用 `denyinterfaces` 或 `allowinterfaces` 以阻止 dhcpcd 绑定至内核名称，例如：

```
denyinterfaces wlan* eth*
```

也可如 #运行中所述，为每个接口启用 *dhcpcd*。但是，模板单元也有缺点，它不支持有线连接的热插拔，且如果没有连接网线就会失败。为解决这个问题，请参见 #超时延迟。

超时延迟

如果 dhcpcd 在单个接口上运行，且在 30 秒后未能获得租约（例如，当服务器未准备好或网线未插入时），它将退出并返回错误。

要让 *dhcpcd* 无限期等待一次，请编辑单元并将`timeout` 设置为 `0`：

```
/etc/systemd/system/dhcpcd@.service.d/timeout.conf
[Service]
ExecStart=
ExecStart=/usr/bin/dhcpcd -w -q -t 0 %I
```

令单元在退出后重启以使其无限期等待：

```
/etc/systemd/system/dhcpcd@.service.d/dhcpcdrestart.conf
[Service]
Restart=always
```

已知问题

dhcpcd@.service 降低启动速度

在默认配置中，*dhcpcd* 的标识 `-w` 会使 `dhcpcd@.service` 在进入后台守护进程前等待 IP 地址的获取。如果启用了该单元，可能会导致启动时暂停并等待 IP 地址分配以继续启动流程。要修复该问题，请为该单元创建如下的附加配置片段：

```
/etc/systemd/system/dhcpcd@.service.d/no-wait.conf
[Service]
ExecStart=
ExecStart=/usr/bin/dhcpcd -b -q %I
```

另请参阅 FS#49685。

参阅

- dhcpcd(8)
- dhcpcd.conf(5)

20.3 Wpa supplicant

难度 高级 | **篇幅** 25.1 KB | **所属** 网络配置

本节将讲到

- 安装
- 概览

- 使用 wpa_cli 进行连接
- 使用 wpa_passphrase 进行连接
- 高级用法

wpa_supplicant 是跨平台的接入客户端)，支持 WPA，WPA2 和 WPA3（IEEE 802.11i），适用于桌面设备、笔记本及嵌入式系统。它是用于客户端的 IEEE 802.1X/WPA 组件。它实现了与 WPA 验证者交互，控制漫游和无线驱动的 IEEE 802.11 验证和关联。

安装

安装 wpa_supplicant包 软件包。此软件包提供了主程序 *wpa_supplicant*，密码工具 *wpa_passphrase* 和文字界面前端 *wpa_cli*。

还可以额外安装官方的 wpa_supplicant_guiAUR，该软件包提供了 *wpa_supplicant* 的图形界面前端 *wpa_gui*。wpa-cuteAUR 是 *wpa_gui* 的一个分支，提供了额外的修正和改进。

概览

连接到加密无线网络的第一步是让 *wpa_supplicant* 获取 WPA 认证者的认证。为此，*wpa_supplicant* 必须进行配置以使其能够向认证者提交认证信息。

完成认证后需要分配一个 IP 地址，具体步骤请参考网络配置#IP 地址。

使用 wpa_cli 进行连接

这种方法使用了 *wpa_cli* 命令行工具，可以用其扫描可用网络，并配置 *wpa_supplicant*。详细信息可以参考 wpa_cli(8) 。

使用 *wpa_cli* 前，需要先为 *wpa_supplicant* 指定一个控制接口，且它需要获得更新配置的权限。先创建一个最小配置：

```
/etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=/run/wpa_supplicant
update_config=1
```

警告：将 `update_config` 设为 `1` 会允许 *wpa_supplicant* 覆写配置文件。*wpa_supplicant* 会在覆写时根据您的默认 umask 重置文件权限，如果您的系统有多用户，可能会意外使任何人能够读取该文件，由此而泄漏密码。

接下来启动 *wpa_supplicant*：

```
# wpa_supplicant -B -i interface -c /etc/wpa_supplicant/wpa_supplicant.conf
```

提示：可以参考网络配置#列出网络接口查看你的无线网卡名称。

完成后，执行：

```
# wpa_cli
```

这一步完成后会显示一个交互提示符（>），同时带有 tab 补全及命令描述功能，还可以使用 `help` 命令查看帮助。

提示：控制接口的默认位置为 `/var/run/wpa_supplicant/`，可以使用 `-p` 选项手动指定位置，以与 *wpa_supplicant* 的配置相符。另外，可以使用 `-i` 选项指定需配置的接口，否则将使用由 *wpa_supplicant* 管理的第一个网络接口。

提示：

- 控制接口的默认位置为 `/var/run/wpa_supplicant/`，可以使用 `-p` 选项手动指定位置，以与*wpa_supplicant* 的配置相符。
- 可以使用 `-i` 选项指定需配置的接口，否则将使用由 *wpa_supplicant* 管理的第一个无线网络接口。

使用 `scan` 和 `scan_results` 命令查看可用网络：

```
> scan
OK
<3>CTRL-EVENT-SCAN-RESULTS
> scan_results
bssid / frequency / signal level / flags / ssid
00:00:00:00:00:00 2462 -49 [WPA2-PSK-CCMP][ESS] MYSSID
11:11:11:11:11:11 2437 -64 [WPA2-PSK-CCMP][ESS] ANOTHERSSID
```

要将网络与 `*MYSSID*` 进行关联，先添加网络，配置凭证并启用：

```
> add_network
0
> set_network 0 ssid "MYSSID"
> set_network 0 psk "passphrase"
> enable_network 0
<2>CTRL-EVENT-CONNECTED - Connection to 00:00:00:00:00:00 completed (reauth) [id=0
id_str=]
```

如果对应的 SSID 无需密码验证，则需要将命令 `set_network 0 psk "*passphrase*"` 替换为 `set_network 0 key_mgmt NONE`，以将网络指定为无密码。

注意：

- 可以添加多个网络配置，每个网络都按照数字顺序进行排列，所以第一个网络的索引为 0。
- If no connection can be established, some information will be printed, and periodic attempts will be made. As part of the periodic attempts, a user prompt will also be given periodically. Issuing the `disable_network 0` command will stop the periodic attempts and return to a steady user prompt.
- 如果使用 `set_network` 提供了凭证，那么就会由 *引号* 括起的 "passphrase" 字符串生成 PSK。另外，你也可以 *不使用引号* 通过 wpa_passphrase 直接向 `psk` 传入 PSK。

最后，将网络保存到配置文件中，并退出 *wpa_cli*：

```
> save_config
OK
> quit
```

完成后，你需要获取一个 IP 地址，具体步骤请参考网络配置#网络管理。

使用 wpa_passphrase 进行连接

通过使用命令行工具 *wpa_passphrase* 生成 *wpa_supplicant* 所需的最小配置，可以快速连接到已知 SSID 的无线网络。例如：

```
$ wpa_passphrase MYSSID passphrase
network={
    ssid="MYSSID"
    #psk="passphrase"
    psk=59e0d07fa4c7741797a4e394f38a5c321e3bed51d54ad5fcbd3f84bc7415d73d
}
```

上例表明，*wpa_supplicant* 可以与 *wpa_passphrase* 协同工作，只需简单地这样做即可：

```
# wpa_supplicant -B -i interface -c <(wpa_passphrase MYSSID passphrase)
```

提示：

- 上条命令需要使用 root shell。
- 如果输入内容包含空格，请使用引号，例如：`"secret passphrase"`.
- 要找出无线网卡的名字，请参考网络配置#列出网络接口。
- 某些少见的复杂密码需要从文件导入，例如：`wpa_passphrase *MYSSID* < passphrase.txt`，或者从命令行输入，例如：`wpa_passphrase *MYSSID* <<< "*passphrase*"`。
- Alternatively, when using special characters in the passphrase, rather than escaping them, simply invoke `wpa_passphrase` without specifying the passphrase. It will then prompt for it to be entered in the standard input where users can paste it even if it contains special characters.

完成后需要获取一个 IP 地址，具体步骤请参考网络配置#IP 地址。

高级用法

对于各种纷繁复杂的网络，更常见的场景是使用 EAP 管理配置文件。各种配置及其范例可参阅手册页 wpa_supplicant.conf(5)；所有可支持的配置参数可参考范例文件 `/usr/share/doc/wpa_supplicant/wpa_supplicant.conf`。。

配置

如上文中#使用 wpa_passphrase 进行连接一节所述，一个基本的配置文件可以这样生成：

```
# wpa_passphrase MYSSID passphrase > /etc/wpa_supplicant/example.conf
```

这样仅仅是创建了一个 `network` 网络配置节段。 A configuration file with also the ability of #使用 wpa_cli 进行连接 and some other common options may look like：

```
/etc/wpa_supplicant/example.conf
# Giving configuration update rights to wpa_cli
ctrl_interface=/run/wpa_supplicant
ctrl_interface_group=wheel
update_config=1

# AP scanning
ap_scan=1
```

```
# ISO/IEC alpha2 country code in which the device is operating
country=US

# network section generated by wpa_passphrase
network={
    ssid="MYSSID"
    psk=59e0d07fa4c7741797a4e394f38a5c321e3bed51d54ad5fcbd3f84bc7415d73d
}
```

如果可以不顾及安全问题，`network` 中的通行字可以换成由引号包围的纯文本：

```
psk="passphrase"
```

如果网络没有设置通行字，如公共无线网络：

```
network={
   ssid="MYSSID"
   key_mgmt=NONE
}
```

To connect to a WPA-Enterprise network, see #802.1x/radius.

Further `network` blocks may be added manually, or using *wpa_cli* as illustrated in #使用 wpa_cli 进行连接. In order to use *wpa_cli*, a control interface must be set with the `ctrl_interface` option. Setting `ctrl_interface_group=wheel` allows users belonging to such group to execute *wpa_cli*. This setting can be used to enable users without root access (or equivalent via sudo etc) to connect to wireless networks. Also add `update_config=1` so that changes made with *wpa_cli* to `example.conf` can be saved. Note that any user that is a member of the `ctrl_interface_group` group will be able to make changes to the file if this is turned on.

`fast_reauth=1` and `ap_scan=1` are the *wpa_supplicant* options active globally at the time of writing. Whether you need them, or other global options too for that matter, depends on the type of network to connect to. If you need other global options, simply copy them over to the file from `/usr/share/doc/wpa_supplicant/wpa_supplicant.conf`.

Alternatively, `wpa_cli set` can be used to see options' status or set new ones. Multiple network blocks may be appended to this configuration: the supplicant will handle association to and roaming between all of them. The strongest signal defined with a network block usually is connected to by default, one may define `priority=` to influence behaviour. For example to auto-connect to any unsecured network as a fallback with the lowest priority:

```
network={
   key_mgmt=NONE
   priority=-999
}
```

Once you have finished the configuration file, you can optionally use it as a system-wide or per-interface default configuration by naming it according to the paths listed in #At boot (systemd)). This also applies if you use additional network manager tools, which may rely on the paths (for example Dhcpcd#10-wpa_supplicant).

提示：To configure a network block to a hidden wireless *SSID*, which by definition will not turn up in a regular scan, the option `scan_ssid=1` has to be defined in the network block.

连接

手动连接

首先启动 *wpa_supplicant* 命令，其最常用的参数为：

- `-B` - 在后台运行。
- `-c *filename*` - 配置文件的路径。
- `-i *interface*` - 要监听的设备。
- `-D *driver*` - （可选）指定使用的驱动。运行 `wpa_supplicant -h` 以查看所支持的驱动列表。
- `nl80211` 是目前的标准，但是不是所有无线网卡都支持。
- `wext` 目前已经弃用，但是仍被广泛支持。

见 wpa_supplicant(8) 获取全部参数的列表。例如：

```
# wpa_supplicant -B -i interface -c /etc/wpa_supplicant/example.conf
```

接着是用在#概览中提到的方法手动获取 IP，例如：

```
# dhcpcd interface
```

提示：

- *dhcpcd* has a hook that can launch *wpa_supplicant* implicitly, see dhcpcd#10-wpa_supplicant.
- While testing arguments/configuration it may be helpful to launch *wpa_supplicant* in the foreground (i.e. *without* the `-B` option) for better debugging messages.

引导时连接（systemd）

wpa_supplicant 软件包中提供了多个 systemd 服务文件：

- `wpa_supplicant.service` - 使用 D-Bus ，建议 NetworkManager 用户使用。
- `wpa_supplicant@*interface*.service` - 接受网口名作为参数，启动服务于该网口的 *wpa_supplicant* 守护进程。这个服务将读取 `/etc/wpa_supplicant/wpa_supplicant-*interface*.conf` 这个配置文件，在使用 systemd-networkd 时比较有用。
- `wpa_supplicant-nl80211@*interface*.service` - 同样接受网口名作为参数，但明确限定使用 `nl80211` 驱动（详阅下文）。它的配置文件是 `/etc/wpa_supplicant/wpa_supplicant-nl80211-*interface*.conf`。

- `wpa_supplicant-wired@*interface*.service` - 同样接受网口名作为参数，使用 `wired`（有线网络）驱动。它的配置文件是 `/etc/wpa_supplicant/wpa_supplicant-wired-*interface*.conf`。

在引导时激活无线网络，就是激活服务于某个无线网络接口的上述服务单元之一，例如启用 `wpa_supplicant@*interface*`。

现在就可以像概览一节所述，可以选定某个网络接口并启用其服务单元的一个实例，从而获取一个 IP 地址，例如启用 `dhcpcd@*interface*`。

提示：*dhcpcd* 可以后台加载 *wpa_supplicant* ，参阅 dhcpcd#10-wpa_supplicant。

802.1x/radius

To connect a wired adapter using 802.1x/radius you will need to specify some configurations and enable the necessary service for the adapter. This is useful for headless servers using systemd-networkd.

You may need to specify the `*wired*` driver with the `*-D wired*` command line option (see #Manual) if the default driver does not support your adapter.

Replace `*adapter*` with the wired adapter you wish to connect, and adapt the settings to match your 802.1x/radius requirements.

```
/etc/wpa_supplicant/wpa_supplicant-wired-adapter.conf
ctrl_interface=/run/wpa_supplicant
ap_scan=0
network={
  key_mgmt=IEEE8021X
  eap=PEAP
  identity="user_name"
  password="user_password"
  phase2="autheap=MSCHAPV2"
}
```

提示：The same configuration, but for a wireless adapter, would require changing `IEEE8021X` to `WPA-EAP` and removing the `ap_scan=0` line

Since this file is storing a plaintext password, chown it to `root:root` and chmod it to `600`.

To use the hash instead of the plaintext password, you can use the `hash` keyword:

```
password=hash:hash_of_plaintext_password
```

To hash your password:

```
$ iconv -t utf16le | openssl dgst -md4 -provider legacy
```

After invoking the command above, provide your plain password and then press `Ctrl+d`.

注意：Hashing the password does not improve the security nor avoid the risk of exposing secrets.

Before running the `wpa_supplicant-wired@*adapter*.service` service, make sure to set the device down:

```
# ip link set adapter down
```

提示：This setup can be used during system installation as well, though you may want to run using `dhcpcd@*adapter*.service` to solicit an address.

wpa_cli 操作脚本

wpa_cli can run in daemon mode and execute a specified script based on events from *wpa_supplicant*. Two events are supported: `CONNECTED` and `DISCONNECTED`. Some environment variables are available to the script, see wpa_cli(8) for details.

The following example will use notify-send to notify the user about the events:

```
#!/bin/sh

case "$2" in
    CONNECTED)
        notify-send "WPA supplicant: connection established";
        ;;
    DISCONNECTED)
        notify-send "WPA supplicant: connection lost";
        ;;
esac
```

Remember to make the script executable, then use the `-a` flag to pass the script path to *wpa_cli*:

```
$ wpa_cli -a /path/to/script
```

漫游

When connected to a wireless network with multiple access points, *wpa_supplicant* is typically responsible for roaming between access points. Choosing a new access point requires *wpa_supplicant* to perform a scan of available networks, which causes a brief interruption in connectivity to the current access point while the wireless radio scans other frequencies. After a scan, if *wpa_supplicant* detects a closer access point (BSSID) in the current network (SSID), in terms of signal strength (RSSI), it will re-associate to the closer access point.

The default configuration of *wpa_supplicant* has relatively timid roaming: it will rescan only when the association to the current access point is lost. This means that, if a client moves far away from its current access point, but not far enough to completely lose signal, the client will keep using the weak signal instead of roaming to a closer access point.

To make *wpa_supplicant* more aggressive about roaming, set the `bgscan` parameter in the configuration file, such as:

```
bgscan="simple:30:-70:3600"
```

The above example will cause *wpa_supplicant* to scan every 30 seconds when the signal is weak (below -70), and every 3600 seconds otherwise. `bgscan` can be specified either in specific `network` blocks or globally for all networks.

排错

警告：Make sure that you do not have remnant configuration files based on the full documentation example `/usr/share/doc/wpa_supplicant/wpa_supplicant.conf`. It is filled with uncommented network examples that may lead to random errors in practice (FS#40661).

Debugging connection failures

In order to determine why you are unable to connect to an access point you can run *wpa_supplicant* with the `-d` flag for debug messages, wait a couple seconds then look for lines that list SSIDs and the reason they were not connected to. For example:

```
# wpa_supplicant -i wlan0 -c /etc/wpa_supplicant/example.conf -d
wlan0: Selecting BSS from priority group 0
wlan0: 0: d2:93:5b:b7:5d:d2 ssid= wpa_ie_len=26 rsn_ie_len=24 caps=0x511 level=-54
freq=5180
wlan0:    skip - SSID not known
wlan0: 1: f2:93:5b:b7:5d:d2 ssid= wpa_ie_len=26 rsn_ie_len=24 caps=0x511 level=-54
freq=5180
wlan0:    skip - SSID not known
wlan0: 2: b2:93:5b:b7:5d:d2 ssid= wpa_ie_len=26 rsn_ie_len=24 caps=0x511 level=-54
freq=5180
wlan0:    skip - SSID not known
wlan0: 3: b0:93:5b:b7:5d:d2 ssid='Access Point 1' wpa_ie_len=0 rsn_ie_len=20 caps=0x511
level=-55 freq=5180  wps
wlan0:    skip - SSID mismatch
wlan0: 4: c4:13:e2:33:42:20 ssid='\x00\x00\x00\x00' wpa_ie_len=22 rsn_ie_len=0
caps=0x111 level=-69 freq=5260
wlan0:    skip - SSID mismatch
wlan0: 5: c4:13:e2:33:42:24 ssid='Home' wpa_ie_len=0 rsn_ie_len=26 caps=0x1111
level=-69 freq=5260
wlan0:    skip RSN IE - no mgmt frame protection enabled but AP requires it
wlan0:    reject due to mismatch with WPA/WPA2
...
```

In this case we are trying to connect to an access point with the SSID *home*. The reason the connection fails is `skip RSN IE - no mgmt frame protection enabled but AP requires it`, so we need to add `ieee80211w=2` to our configuration file.

某些硬件不支持 nl80211 驱动

On some (especially old) hardware, *wpa_supplicant* may fail with the following error:

```
Successfully initialized wpa_supplicant
nl80211: Driver does not support authentication/association or connect commands
wlan0: Failed to initialize driver interface
```

This indicates that the standard `nl80211` driver does not support the given hardware. The deprecated `wext` driver might still support the device:

```
# wpa_supplicant -B -i wlan0 -D wext -c /etc/wpa_supplicant/example.conf
```

If the command works to connect, and the user wishes to use systemd to manage the wireless connection, it is necessary to edit the `wpa_supplicant@.service` unit provided by the package and modify the `ExecStart` line accordingly:

```
/etc/systemd/system/wpa_supplicant@.service.d/wext.conf
[Service]
ExecStart=
ExecStart=/usr/bin/wpa_supplicant -c/etc/wpa_supplicant/wpa_supplicant-%I.conf -i%I -
Dnl80211,wext
```

注意：Multiple comma separated driver wrappers in option `-Dnl80211,wext` makes *wpa_supplicant* use the first driver wrapper that is able to initialize the interface (see wpa_supplicant(8) § EXAMPLES). This is useful when using mutiple or removable (e.g. USB) wireless devices which use different drivers.

挂载了网络共享（CIFS）时的关机问题

When you use wireless to connect to network shares you might have the problem that the shutdown takes a very long time. That is because systemd runs against a 3 minute timeout. The reason is that WPA supplicant is shut down too early, i.e. before systemd tries to unmount the share(s). A bug report suggests a work-around by editing the `wpa_supplicant@.service` as follows:

```
/etc/systemd/system/wpa_supplicant.service.d/override.conf
[Unit]
After=dbus.service
```

口令相关的问题

wpa_supplicant包 may not work properly if directly passed via stdin particularly long or complex passphrases which include special characters. This may lead to errors such as `failed 4-way WPA handshake, PSK may be wrong` when launching wpa_supplicant包.

In order to solve this try using here strings `wpa_passphrase <MYSSID> <<< "<passphrase>"` or passing a file to the `-c` flag instead:

```
# wpa_supplicant -i <interface> -c /etc/wpa_supplicant/example.conf
```

In some instances it was found that storing the passphrase cleartext in the `psk` key of the `wpa_supplicant.conf` `network` block gave positive results (see). However, this approach is rather insecure. Using `wpa_cli` to create this file instead of manually writing it gives the best results most of the time and therefore is the recommended way to proceed.

Problems with Eduroam

If the institution the user studies or works at did not upgrade their network tunnel's encryption to at least TLS 1.2 yet and still uses TLS 1.0 or 1.1 for network traffic encryption in their Eduroam Wi-Fi infrastructure, OpenSSL 3.x throws an "unsupported protocol" error and the client machine's Wi-Fi backend (either wpa_supplicant or iwd) refuses to establish a connection any further. Fortunately, an easy workaround exists for OpenSSL's TLS 1.0 and 1.1 deprecation without making the client computer's whole Wi-Fi connection stack globally vulnerable to attacks, although it only works with NetworkManager.

Consult NetworkManager#WPA Enterprise connections fail to authenticate with OpenSSL "unsupported protocol" error for the solution.

Connman users can visit the ConnMan#Connecting to eduroam (802.1X)) article for their own version of the fix.

Connections to pure WPA3-SAE access points

Make sure to define the following within the network block of the configuration to enable connections to pure WPA3 access points:

```
ssid="network SSID"
key_mgmt=SAE
sae_password="the.literal.wifi.password"
ieee80211w=2
```

Additionally, Intel Wi-Fi 6 cards may need `sae_pwe=1` in the main (non network) section of the config file.

Connections to mixed WPA2-PSK/WPA3-SAE access points

Mixed WPA2-PSK/WPA3-SAE access points will require an alternative setting for key_mgmt as shown below:

```
ssid="network SSID"
key_mgmt=WPA-PSK-SHA256
psk=xxx
ieee80211w=2
```

Hardware 802.11w support

You can check for hardware support of MFP/PMF (*M*anagement *F*rame *P*rotection / *P*rotected *M*anagement *F*rames) on the interface client by running:

```
$ iw phy phy0 info | grep 00-0f-ac:6
```

Most Wi-Fi devices support this standard introduced in 2009, except some limited (aka non x86_64 related) or old hardware.

参阅

- wpa_supplicant 主页
- wpa_supplicant README - 项目完整文档，包含了 manpage 未列出的 *wpa_cli* 命令
- wpa_cli 用例
- wpa_supplicant(8)
- wpa_supplicant.conf(5)
- wpa_cli(8)
- Kernel.org wpa_supplicant 文档

20.4 lwd

难度 高级 | **篇幅** 29.6 KB | **所属** 网络配置

本节将讲到

- 安装
- 使用方法
- 网络配置
- 可选配置
- 故障排除

iwd (iNet wireless daemon，iNet 无线守护程序) 是由英特尔（Intel）为 Linux 编写的一个无线网络守护程序。该项目的核心目标是不依赖任何外部库，而是最大程度地利用 Linux 内核提供的功能来优化资源利用。

iwd 可以独立工作，也可以和 ConnMan、systemd-networkd 和 NetworkManager 这样更完善的网络管理器结合使用。

注意：当通过 NetworkManager 使用 iwd 时，除非 NetworkManager 的文章中另有说明，否则请不要按照本页上的说明操作。

安装

安装 iwd包 软件包。

也可选择安装第三方包，其提供图形化与终端用户界面：

- **impala** — 一个用于 iwd 的终端用户界面（TUI）。

: https://github.com/pythops/impala || impala包

- **iwdgui** — iwd 的图形界面前端。

: https://gitlab.com/hfernh/iwdgui || iwdguiAUR

- **iwgtk** — iwd 的图形界面前端及指示器（托盘）图标。

: https://github.com/J-Lentz/iwgtk || iwgtkCNRepo

- **iwmenu** — iwd 的菜单驱动界面。

: https://github.com/e-tho/iwmenu || iwmenu-gitAUR

使用方法

iwd包 软件包提供了客户端程序 `iwctl`、守护程序 `iwd` 和 WiFi 监控工具 `iwmon`。

启动/启用 `iwd.service` 以便可以使用 `iwctl` 或 `iwgtk` 对其进行控制。

注意：仅允许root、`network` 或 `wheel` 用户组 的成员与 *iwd* 交互。为了使用 *iwctl* 或 *iwgtk*，您需要 将您的用户添加到其中一个组.

iwctl

要进入交互式提示符（interactive prompt），执行：

```
$ iwctl
```

然后交互式提示符就会以 `[iwd]#` 前缀显示出来了。

提示：

- 在 `iwctl` 提示符中，可以通过 `Tab` 键自动补全命令和设备名称。
- 要退出交互式提示，按下 `Ctrl+d` 发送 EOF 信号。
- 可以在不进入交互式提示符的情况下，将所有命令当作命令行参数使用。例如：`iwctl device wlan0 show`。

要列出所有可用的命令：

```
[iwd]# help
```

连接网络

首先，使用 rfkill 命令确认 Wi-Fi 设备未被屏蔽。

如果不知道你的网络设备名称，请列出所有 WiFi 设备：

```
[iwd]# device list
```

如果设备或其相应的适配器已关闭，请将其打开。

```
[iwd]# device name set-property Powered on
[iwd]# adapter adapter set-property Powered on
```

然后，要开始扫描网络（注意：这个命令不会输出任何内容），执行：

```
[iwd]# station name scan
```

再然后，就可以列出所有可用的网络：

```
[iwd]# station name get-networks
```

最后，要连接到一个网络：

```
[iwd]# station name connect SSID
```

注意：* 对于通过 DHCP 的 IP 和 DNS 自动配置，您需要 手动启用 内置 DHCP 客户端，或配置一个 独立 DHCP 客户端。

提示：用户界面支持自动补全，输入 `station` 后按压 `Tab Tab`，将显示可用的设备，键入设备的首字母和 `Tab` 将自动补全。同样，输入 `connect` 和 `Tab Tab` 以显示可用网络列表。然后，键入所选网络的首字母，后跟 `Tab`，以自动补全。

當網路密碼尚未儲存在 iwd 自動檢查的任何設定檔中時，系統會提示使用者輸入密碼，而連線操作也可以透過命令列參數的方式來指定：

```
$ iwctl --passphrase passphrase station name connect SSID
```

注意：

- `iwd` 会自动将网络密码存储在 `/var/lib/iwd` 目录中，以后就可以使用其自动连接记住的网络。参见 #网络配置 一节。
- 要连接 SSID 里带空格的网络，连接时请用双引号将网络名称括起来。(或用转义符代替空格（"\\ "）)
- iwd 仅支持 8 到 63 位 ASCII 编码字符组成的 PSK 密码。如果没有满足要求，会出现下列错误信息：`PMK generation failed. Ensure Crypto Engine is properly configured`。

使用 WPS/WSC 连接网络

如果网络配置为可以通过按下一个按钮就能进行连接 (Wikipedia:Wi-Fi Protected Setup，WPS，WiFi 保护配置)，先检查设备是否能兼容以下配置步骤：

```
[iwd]# wsc list
```

然后，假设设备出现在了上面的列表中：

```
[iwd]# wsc device push-button
```

接着按下路由器上的按钮。如果提前按下按钮，但只要少于两分钟前，以上步骤也是可以生效的。

如果网络要求验证 PIN 码才能连接，请检查 `help` 命令的输出，了解如何为 `wsc` 命令提供正确的选项。

断开网络连接

要断开网络连接：

```
[iwd]# station device disconnect
```

显示设备和连接信息

要显示 WiFi 设备详细情况，比如 MAC 地址：

```
[iwd]# device device show
```

要显示包括 WiFi 设备的连接网络在内的连接状态：

```
[iwd]# station device show
```

管理已知网络

要列出以前连接过的网络：

```
[iwd]# known-networks list
```

要忘记已知的网络：

```
[iwd]# known-networks SSID forget
```

iwgtk

另外，iwgtkCNRepo 提供了一个 GUI 前端，也通过它可以控制 iwd。

不加任何参数运行 `iwgtk` 将启动一个 iwd 的管理窗口，它能被用于切换适配器的开关、更改操作模式、查看或链接可用的无线网络、管理已保存的网络。

托盘图标

要启动 iwgtk 的托盘图标请运行：

```
$ iwgtk -i
```

如果托盘中的图标没有出现，则你的系统托盘很可能不支持 StatusNotifierItem API，这种情况下你需要使用一个兼容层（例如snixembed-gitAUR）。 下面的系统托盘支持 StatusNotifierItem API，因此无需兼容层：

- KDE Plasma
- swaybar
- xfce4-panel

下面的托盘只支持 XEmbed 因此需要snixembed-gitAUR：

- AwesomeWM
- i3bar

自启动

每次登录桌面时自启动 iwgtk 的托盘图标是常见的用法。如果你的桌面环境支持 XDG Autostart 标准，则 iwgtk 托盘图标的自启动无需额外配置，因为 iwgtkCNRepo 将 `iwgtk-indicator.desktop` 文件放置在了 `/etc/xdg/autostart/`。 作为上面的方法的替代， iwgtkCNRepo提供了一个用来自启动托盘进程的 systemd 单元文件. 如果你的桌面环境支持 systemd 的 `graphical-session.target` 单元, 则 iwgtk 能在 Systemd#使用单元了 `iwgtk.service` 用户单元后被 systemd 自动启动。

网络配置

iwd 默认在 `/var/lib/iwd` 目录下储存网络配置。配置文件名形如 `*网络*.*类别*`，其中网络是该网络的 SSID，类别是网络类别，是 *.open*, *.psk*, *.8021x* 之一。文件储存加密的 `PreSharedKey` 和可选的 cleartext `Passphrase`，可以不调用 `iwctl` 由用户创建。该文件亦可用作隶属该网络 SSID 的其他配置。参见 iwd.network(5) 获得更详尽的设置。

注意：在字符串值中，包括身份和密码，特定字符被反斜杠转义。开头的空格，\\n，\\r 以及反斜杠必须转义表达。参见 iwd.network(5)。

WPA-PSK

一个连接到 WPA-PSK 或 WPA2-PSK 安全网络的最小示例文件，其 SSID "spaceship"，Passphrase "test1234"：

```
/var/lib/iwd/spaceship.psk
[Security]
PreSharedKey=aafb192ce2da24d8c7805c956136f45dd612103f086034c402ed266355297295
```

注意：网络的 SSID 仅当其仅包含字母，数字或 - _ 两字符之一时作为文件名。如果其包含其他字符，文件名将替代为=字符后SSID的十六进制编码（十六进制数使用小写字母）。

下列两种方法可以根据 passphrase 计算 pre-shared key：

- 在配置文件的 cleartext 输入 passphrase：

```
/var/lib/iwd/spaceship.psk
[Security]
Passphrase=test1234
```

pre-shared key 会在第一次连接时自动追加到文件末尾：

```
/var/lib/iwd/spaceship.psk
[Security]
Passphrase=test1234
PreSharedKey=aafb192ce2da24d8c7805c956136f45dd612103f086034c402ed266355297295
```

- 亦可使用 *wpa_passphrase*（自wpa_supplicant包 或 wpa-pskAUR），用 SSID 与 passphrase 计算 pre-shared key。
- 参见 wpa_supplicant#Connecting with wpa_passphrase 获得更多细节。

WPA Enterprise

EAP-PWD

为了连接到受 EAP-PWD 保护的企业（无线）访问接入点 (Access Point, AP)，需要在 `/var/lib/iwd` 文件夹中创建一个 `*essid*.8021x` 格式的文件，并包含以下内容：

```
/var/lib/iwd/essid.8021x
[Security]
EAP-Method=PWD
EAP-Identity=your_enterprise_email
EAP-Password=your_password

[Settings]
AutoConnect=true
```

如果不想自动连接到 AP，可以将相应选项设定为 False 并手动通过 `iwctl` 连接网络。密码同理，如果不想让密码以明文保存，则将相应选项从文件中删去，直接连接企业 AP。

注意：无法更改不同 SSID 的优先级，您可能需要设置 `AutoConnect=false` 作为变通方法。

EAP-PEAP

和 EAP-PWD 一样，同样需要在同一文件夹中创建一个 `*essid*.8021x` 格式的文件。在继续编写配置文件之前，不妨了解一下自己所属组织所使用的 CA 证书。下面是一个使用 MSCHAPv2 密码认证的示例配置文件：

```
/var/lib/iwd/essid.8021x
[Security]
EAP-Method=PEAP
EAP-Identity=anonymous@realm.edu
EAP-PEAP-CACert=/path/to/root.crt
EAP-PEAP-ServerDomainMask=radius.realm.edu
EAP-PEAP-Phase2-Method=MSCHAPV2
EAP-PEAP-Phase2-Identity=johndoe@realm.edu
EAP-PEAP-Phase2-Password=hunter2

[Settings]
AutoConnect=true
```

MsCHAPv2 密码亦可被存为一个加密的哈希值。正确的 md4 值可这样计算：

```
$ iconv -t utf16le | openssl md4 -provider legacy
```

在输完密码后，按下 `Ctrl+d`以输入 EOF，不要按 `Enter`。结果哈希值应储存在 `EAP-PEAP-Phase2-Password-Hash` 值。

注意：如果打算使用 *eduroam*，另请参见 #Eduroam。

TTLS-PAP

和 EAP-PWD 一样，同样需要在同一文件夹中创建一个 `*essid*.8021x` 格式的文件。在继续编写配置文件之前，不妨了解一下自己所属组织所使用的 CA 证书。下面是一个使用 PAP 密码认证的示例配置文件：

```
/var/lib/iwd/essid.8021x
[Security]
EAP-Method=TTLS
EAP-Identity=anonymous@uni-test.de
EAP-TTLS-CACert=cert.pem
EAP-TTLS-ServerDomainMask=*.uni-test.de
EAP-TTLS-Phase2-Method=Tunneled-PAP
EAP-TTLS-Phase2-Identity=user
EAP-TTLS-Phase2-Password=password

[Settings]
AutoConnect=true
```

EAP-TLS

EAP-TLS 使用 x509 *客户端证书*来确认您的身份。和 ssh 密钥一样，这些证书使用公钥加密，因此永不需要向 wifi 验证服务器发送密钥，您也无需复制和重复使用密码。通常情况下，每台设备都会使用不同的证书。理论上，至少可以在不强迫你更改密码或中断其他设备的情况下撤销证书。

与其他企业级方法一样，您需要知道贵组织使用的 CA 证书（`cacert.pem`），该证书可以向设备证明它连接到的目标是正确的。您还需要拥有在每次连接时上传的、代表您的客户端证书（`client-cert.pem`），以及证明您拥有客户端证书的配套的私钥（`client-key.pem`）。

当您收集好凭证，将这个放到您的 `/var/lib/iwd/*essid*.8021x` 文件：

```
/var/lib/iwd/essid.8021x
[Security]

EAP-Method=TLS
EAP-TLS-CACert=/path/to/cacert.pem
EAP-Identity=your_enterprise_email
EAP-TLS-ClientCert=/path/to/client-cert.pem
EAP-TLS-ClientKey=/path/to/client-key.pem
#EAP-TLS-ClientKeyPassphrase=key-passphrase  #如果 client-key.pem 是加密的，請提供其密碼。

[Settings]
AutoConnect=true
```

Eduroam

通过 iwd 连接 eduroam 的方法詳见以下文件，并填写必要的信息：

```
/var/lib/iwd/eduroam.8021x
[Security]
EAP-Method=PEAP
EAP-Identity=anonymous@university.domain
EAP-PEAP-Phase2-Method=MSCHAPV2
EAP-PEAP-Phase2-Identity=username@university.domain
EAP-PEAP-Phase2-Password=password

[Settings]
```

`AutoConnect=true`

**本文或本章节的事实准确性存在争议。**

原因： CAT 中对 iwd 的支持不可用（在 Talk:Iwd 中讨论）

Eduroam 提供了一个配置助手工具 (Configuration Assistant Tool，CAT), 遗憾的是它并不支持 iwd。但是，CAT 的安装程序只是一个 Python 脚本，可以通过点击下载按钮，然后选择你的大学来下载它。可以很轻易地提取出必要的配置选项，包括证书和服务域掩码。此外，一些机构正在升级到 EAP-TLS，并将 `client-cert.pem` 的生成外包给 SecureW2。这种情况下，您也需要使用他们的工具来生成客户端证书。

下列的表格包含了 iwd 配置选项到 Eduroam CAT 安装脚本变量的映射关系：

Iwd 配置选项	**CAT 脚本变量**
essid	某个 `Config.ssids` 文件
`EAP-Method`	`Config.eap_outer`
`EAP-Identity`	`Config.anonymous_identity`
`EAP-*method*-CACert`	`Config.CA`的内容，或者包含`Config.CA`的*.pem*文件的绝对路径
`EAP-*method*-ServerDomainMask`	`Config.servers` 之一
`EAP-*method*-Phase2-Method`	除非 `Config.eap_inner` 等于 `PAP`，否则请使用 `Tunneled-PAP`
`EAP-*method*-Phase2-Identity`	`*username*@Config.user_realm`

其中 `*method*` 是 `EAP-Method` 的内容，应为 `TLS`、`TTLS` 或 `PEAP`。 提取所有必要信息，并转换、使之与 iwd 配置等价后，就可以按照前面的方法将其放入名为 `*essid*.8021x` 的配置文件中。

注意：

- Eduroam 提供方可能不要求使用 `EAP-Identity`，在这种情况下可以在选项字段里写上 `anonymous`。
- 若您的 `EAP-*method*-ServerDomainMask` 以 `DNS:`开始，只使用 `DNS:` 以后的部分。

其他情况

更多测试范例可在上游仓库的测试案例中找到。

内嵌证书

除了使用 PEM 文件的绝对路径（用于证书和密钥）之外，也可以将 PEM 文件本身直接包含在网络配置文件中。

内嵌 PEM 可以在设置文件中的任意位置出现，格式如下：

```
[@pem@my_ca_cert]
----- BEGIN CERTIFICATE -----
PEM data
----- END CERTIFICATE -----
```

其中，*my_ca_cert* 是用来在配置文件中标识该证书的任意名称。

然后，在设置文件中任何需要证书路径的地方，都可以通过在值前加上 `embed:` 来使用内嵌证书，例如：

```
EAP-TTLS-CACert=embed:my_ca_cert
```

这不仅限于 CA 证书。客户端证书、客户端密钥（无论是否加密）以及证书链都可以包含在内。

可选配置

文件 `/etc/iwd/main.conf` 用于存储主要配置。参见 iwd.config(5)。

禁用特定网络的自动连接

创建或编辑 `/var/lib/iwd/*network*.*type*` 文件。在其中添加如下部分：

```
/var/lib/iwd/spaceship.psk (for example)
[Settings]
AutoConnect=false
```

禁用定期扫描可用网络

默认情况下，当 `iwd` 处于未连接状态时，它会定期扫描可用网络。要禁用定期扫描（以便总是手动扫描网络），创建或编辑 `/etc/iwd/main.conf` 文件并添加以下部分：

```
/etc/iwd/main.conf
[Scan]
DisablePeriodicScan=true
```

启用内置网络配置

自 0.19 版本起，iwd 可使用内置的 DHCP 客户端或静态配置来分配（多个）IP 地址并设置路由。它是独立 DHCP 客户端不错的替代品。

要激活 iwd 的网络配置功能，创建或编辑 `/etc/iwd/main.conf` 并添加以下部分：

```
/etc/iwd/main.conf
[General]
EnableNetworkConfiguration=true
```

还可以用 `RoutePriorityOffset` 设置路由指标（route metric）：

```
/etc/iwd/main.conf
[Network]
RoutePriorityOffset=300
```

IPv6 支持

自从 1.10 版本，iwd 支持 IPv6，但在 2.0 版本以下默认禁用。2.0 版本后默认启用。

欲禁用之，在配置文件下添加以下内容：

```
/etc/iwd/main.conf
[Network]
EnableIPv6=false
```

要在高于 1.10 低于 2.0 版本启用它：

```
/etc/iwd/main.conf
[Network]
EnableIPv6=true
```

无论您想用 DHCPv6 还是静态 IPv6 配置，您都需要启用该设置。它也可以设置在每个网络的基础上。

在网络配置中设定静态 IP 地址

将下列部分添加到 `/var/lib/iwd/*network*.*type*` 文件中。例如可以这样写：

```
/var/lib/iwd/spaceship.psk
[IPv4]
Address=192.168.1.10
Netmask=255.255.255.0
Gateway=192.168.1.1
Broadcast=192.168.1.255
DNS=192.168.1.1
```

选择 DNS 管理器

目前，iwd 支持两种 DNS 管理器——systemd-resolved 和 resolvconf。

要使用 `systemd-resolved`，将下列部分添加到 `/etc/iwd/main.conf` 中：

```
/etc/iwd/main.conf
[Network]
NameResolvingService=systemd
```

而对于 `resolvconf`，添加的为:

```
/etc/iwd/main.conf
[Network]
NameResolvingService=resolvconf
```

注意：若未指定，默认使用 systemd-resolved，并推荐其用于systemd.

允许任何用户读取状态信息

如果想允许任何用户读取状态信息，但不允许修改设置，请按如下所示创建一个 D-Bus 配置：

```
/etc/dbus-1/system.d/iwd-allow-read.conf
<!-- Allow any user to read iwd status information. Overrides some part
      of /usr/share/dbus-1/system.d/iwd-dbus.conf. -->

<!DOCTYPE busconfig PUBLIC "-//freedesktop//DTD D-BUS Bus Configuration 1.0//EN"
 "http://www.freedesktop.org/standards/dbus/1.0/busconfig.dtd">
<busconfig>

  <policy context="default">
    <deny send_destination="net.connman.iwd"/>
    <allow send_destination="net.connman.iwd"
send_interface="org.freedesktop.DBus.Properties" send_member="GetAll" />
    <allow send_destination="net.connman.iwd"
send_interface="org.freedesktop.DBus.Properties" send_member="Get" />
    <allow send_destination="net.connman.iwd"
send_interface="org.freedesktop.DBus.ObjectManager" send_member="GetManagedObjects" />
    <allow send_destination="net.connman.iwd" send_interface="net.connman.iwd.Device"
send_member="RegisterSignalLevelAgent" />
    <allow send_destination="net.connman.iwd" send_interface="net.connman.iwd.Device"
send_member="UnregisterSignalLevelAgent" />
  </policy>
```

```
</busconfig>
```

加密网络配置文件

默认情况下，iwd 会以未加密方式在系统中存储网络凭证。自 iwd 1.25 版起，iwd 提供了试验性支持，可为使用 systemd 的系统创建加密配置文件。

首先，创建一个加密凭证。下面的示例使用systemd-creds创建了一个名为*iwd-secret*的加密凭据，该凭据与系统的Trusted Platform Module绑定，将用于创建加密配置文件：

```
# systemd-ask-password -n | systemd-creds --tpm2-device=auto --name=iwd-secret encrypt
- /etc/credstore.encrypted/iwd-secret.cred
```

接下来，为 iwd 服务创建一个 drop-in file，添加 `LoadCredentialEncrypted` 选项。

```
/etc/systemd/system/iwd.service.d/use-creds.conf
[Service]

LoadCredentialEncrypted=iwd-secret:/etc/credstore.encrypted/iwd-secret.cred
```

最终，在 iwd 配置文件中添加 `SystemdEncrypt` 选项（值为该凭证），重载 systemd 管理器，并重启 iwd 服务。

```
/etc/iwd/main.conf
[General]

...

SystemdEncrypt=iwd-secret
```

注意：

- 当前系统中的任何配置文件都将自动加密。此时无需其他操作，今后的任何配置文件都将自动加密。
- 在上例中，加密凭据隐式绑定到 TPM PCR 7。因此，如果安全启动状态或固件证书发生变化，该启动会话将无法连接网络。

故障排除

详细 TLS 调试

如果在配置 MSCHAPv2 或 TTLS 时遇到困难，这会很有用。可以通过一个附加配置片段来设定以下环境变量：

```
/etc/systemd/system/iwd.service.d/tls-debug.conf
[Service]
Environment=IWD_TLS_DEBUG=TRUE
```

之后通过以 root 身份运行 `journalctl -u iwd.service` 命令检查 iwd 日志。

引导后重启 iwd.service

一些电脑上 `iwd.service` 需引导后重启才能正常工作。参见 FS#63912 和 thread 251432。这很可能是因为 *iwd* 在无线网卡上电之前启动了。

作为解决方法，使用命令 `systemctl list-units --type=device | grep *wlan0*` 找到需要等待的单元，并相应地扩展该单元：

```
/etc/systemd/system/iwd.service.d/override.conf
[Unit]
After=sys-XXXX-net-wlan0.device
Wants=sys-XXXX-net-wlan0.device
```

然后重新加载 *systemd* 管理器配置。

如果仍然无效，可以尝试添加以下配置：

```
[Service]
ExecStartPre=ip link set wlan0 up
```

或者

```
[Service]
ExecStartPre=/usr/bin/sleep 2
```

Wireless device is not renamed by udev

1.0 起，iwd 禁用了可预测的无线设备重命名。它设置了阻止 udev 重命名 `wlp#s#` 接口的以下 systemd 网络连接配置文件：

```
/usr/lib/systemd/network/80-iwd.link
[Match]
Type=wlan

[Link]
NamePolicy=keep kernel
```

结果是网络链接名 `wlan#` 在引导后保留。 这使得 *iwd* 和 udev 间存在接口重命名的竞争，如 iwd udev interface renaming 所述。

如果造成问题，尝试以此解决：

```
# ln -s /dev/null /etc/systemd/network/80-iwd.link
```

在 AP 模式下无 DHCP

当在 AP 模式下通过 DHCP连接到 *iwd* 时，客户端可能收不到 IP 地址。这有必要用 *iwd* 在管理好的接口上启用网络配置。

```
/etc/iwd/main.conf
[General]
EnableNetworkConfiguration=True
```

提及文件若不存在则创建。

因 iwd 崩溃无法连接 WiFi

一些用户经历 WiFi 断开的情况，通过不断重连，最终使之稳定并成功连接。

用户通过 journal 报告 `iwd.service` 崩溃 () 。

核心问题 是多个网络连接服务冲突。要解决，请检查您是否同时启用多个服务。

客户端密钥加载错误

要加载密钥文件，*iwd* 需要 `pkcs8_key_parser` 内核参数。引导时，它由 systemd-modules-load.service(8) 使用 `/usr/lib/modules-load.d/pkcs8.conf` 加载，如果才安装 iwd，就不会出现这种情况。

当尝试连接到 WPA 企业网络时，如在 journal 有例如 `Error loading client private key */path/to/key*` 的消息，请手动加载内核：

```
# modprobe pkcs8_key_parser
```

iwd 始终漫游

如果连接太差，iwd 会漫游到其他已知 AP。

出现此问题，系统日志会报：`wlan0: deauthenticating from xx:xx:xx:xx:xx:xx by local choice (Reason: 3=DEAUTH_LEAVING)`

您可通过以下方法查看信号强度：

```
iwctl station wlan0 show | grep RSSI
```

您可调整阈值以允许更差的连接。RoamThreshold 默认值为 -70，RoamThreshold5G 默认值为 -76。

```
/etc/iwd/main.conf
[General]

RoamThreshold=-75

RoamThreshold5G=-80
```

DHCP 请求中未发送主机名

请在网络的配置文件中设置 `SendHostname`，而不是在 `/etc/iwd/main.conf` 中设置。

```
/var/lib/iwd/SomeNetwork.psk
...
[IPv4]
SendHostname=true
```

/etc/resolv.conf：只读文件系统

当使用 **resolvconf** 作为 DNS 解析方法时，可能会出现无法写入 `/etc/resolv.conf` 的问题，报错为只读文件系统：

```
$ journalctl -u iwd.service
Jun 14 14:08:12 host iwd: event: state, old: disconnected, new: autoconnect_quick
Jun 14 14:08:12 host iwd: udev interface=wlan0 ifindex=6
Jun 14 14:08:13 host iwd: event: connect-info, ssid: <redacted>, bss: <redacted>,
signal: -63, load: 0/255
Jun 14 14:08:13 host iwd: event: state, old: autoconnect_quick, new: connecting (auto)
Jun 14 14:08:13 host iwd: event: state, old: connecting (auto), new: connecting
(netconfig)
Jun 14 14:08:14 host iwd: cp: cannot create regular file '/etc/resolv.conf.bak': Read-
only file system
Jun 14 14:08:14 host iwd: /usr/lib/resolvconf/libc: line 257: /etc/resolv.conf: Read-
only file system
```

```
Jun 14 14:08:16 host iwd: cp: cannot create regular file '/etc/resolv.conf.bak': Read-only file system
Jun 14 14:08:16 host iwd: /usr/lib/resolvconf/libc: line 257: /etc/resolv.conf: Read-only file system
Jun 14 14:08:16 host iwd: event: state, old: connecting (netconfig), new: connected
```

解决此问题的方法是，通过添加 drop-in 扩展 `iwd.service` systemd 单元的配置：

```
/etc/systemd/system/iwd.service.d/50-resolvconf.conf
[Service]
RuntimeDirectory=resolvconf
ReadWritePaths=/etc/resolv.conf
```

这将允许 `iwd.service` 系统单元更新 `/etc/resolv.conf`。重启 `iwd.service` 以使更改生效。

注意：关于此问题的讨论请参见上游邮件列表：Re: iwd doesn't work with openresolv

另请参阅

- 开始使用 iwd（英文）
- 网络配置设置（英文）
- WPA Enterprise 的更多示例（英文）
- Arch Linux 论坛上的 IWD 帖子（英文）
- 2017 Update on new WiFi daemon for Linux by Marcel Holtmann - YouTube
- The New Wi-Fi Experience for Linux - Marcel Holtmann, Intel - YouTube
- How to set up a simple access point with iwd

邮件服务器

本章把一台 Arch 机变成能真正收发信、与外部域名互通邮件的服务器：难点不在安装，而在软件选型、端口、MX 记录、身份验证与测试相互牵制。《邮件服务器》把这些串起来，必读且宜最先读；之后再读所选软件的那一节，《Sendmail》讲安装、用户与配置，《Postfix》另含启动与 TLS，Tips and tricks 与 Extras 可按需翻查。《UW IMAP》篇幅短，只在要让客户端拉取信箱时读，用不上可略过。

最易踩的坑是把装上软件、填好域名当成能收信：端口没放行、缺身份验证、MX 记录与真实地址对不上，任一都会让测试失败；TLS 与 SSL 也常留到收尾，而它本该和端口、证书一起定。

21.1　UW IMAP

难度 入门 | **篇幅** 2.6 KB | **所属** 邮件服务器

本节将讲到

- 安装
- 配置
- SSL

翻译状态：

- 本文（或部分内容）译自 UW IMAP，最近一次同步于 2024-8-5，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

摘自 Wikipedia：

: **UW IMAP** 是 University of Washington 开发的 IMAP 协议的参考服务器实现。

虽然多年来它一直没有得到积极开发，但作为基本的 IMAPS 服务器，它仍然可运行良好。(有关其他 IMAP 服务器，请参阅邮件服务器#软件）。

安装

安装 c-clientAUR。它不使用配置文件。

注意：*c-client* 是一个包含 UW IMAP 及其客户端的拆分软件包，你可以根据自己的需要同时或只安装其中一个软件包。

配置

虽然它最初是与 inetd 一起使用的，但在现代 Arch 系统上，更好的解决方案是使用 systemd socket 文件：

```
/etc/systemd/system/imaps.socket
```

```
[Unit]
Description=IMAP Server Activation Socket
Documentation=https://www.washington.edu/imap/

[Socket]
ListenStream=0.0.0.0:993
Accept=true

[Install]
WantedBy=sockets.target
```

此外，还需要创建相应的 .service 文件：

```
/etc/systemd/system/imaps@.service
[Unit]
Description=IMAP Server

[Service]
ExecStart=-/usr/bin/imapd
StandardInput=socket
```

UW-IMAPD 使用 PAM，因此还需要创建一个 PAM 授权文件。本例将使用标准系统密码进行身份验证：

```
/etc/pam.d/imap
auth          required      pam_unix.so
account               required      pam_unix.so
session               required      pam_unix.so
```

启用并启动 `imaps.socket` 并进行测试。

SSL

如果尚未创建通用 SSL 证书和密钥，则会在 `/etc/ssl/certs/imapd.pem` 中创建。可以替换特定服务器的签名证书。

警告：自签名证书（如自动生成的证书）容易受到 MITM 攻击，请确保在生产服务器上使用签名证书。

21.2 Sendmail

难度 进阶 | **篇幅** 8.2 KB | **所属** 邮件服务器

本节将讲到

- 安装
- 添加用户
- 配置
- 小窍门

Sendmail 是来自 UNIX 世界的经典 SMTP 服务器。本篇是基于邮件服务器构建的。

本文的目的是为本地用户账户设置 Sendmail，**不使用 mysql 或者其它数据库**，同时允许建立所谓 *mail-only 账户*。

安装

**本文或本章节的事实准确性存在争议。**

原因： sendmail 可以与其他邮件投递代理（MDA）配合使用，而不只是 procmail。 英文Wiki中存在：Talk:Sendmail#MDA_compatibility（在 Talk:Sendmail 中讨论）

安装软件包 sendmailAUR, procmail包 和 m4包

添加用户

为每个需要接收邮件的用户创建 Linux 用户，每个用户会拥有类似 username@your-domain.com 的电子邮件地址。但是如果你想要添加 *mail-only 账户*，即仅能处理电子邮件，但不能使用 shell 或 X 进行登录的账户，您可以按如下步骤添加该类账户：

```
# useradd -m -s /usr/bin/nologin username
```

注：在此之后，您可以随时通过修改 `/etc/passwd` 等方式更改账户类型与属性。

配置

创建 SSL 证书

警告：如果你部署了 TLS，请务必遵循 weakdh.org 的指南，并禁用 SSLv3，以防止安全漏洞。更多信息请参阅服务端SSL（英文Wiki）

创建证书的过程，请阅读 OpenSSL 以了解更多信息，Let's Encrypt 提供了免费认证。

警告：使用经过口令加密（具有 passphrase）的服务器的密钥文件可能导致 Sendmail 启用 TLS 失败。如果出现问题，请参考 这里 移除已有密钥文件的口令。

注意：建议将服务器密钥与证书统一放在 `/etc/mail/certs/` 目录下，并移除服务器密钥文件的组可读与其它可读权限。之后 `sendmailconfig` 工具可能会自动更改该文件夹下文件的属主、属组与权限。

sendmail.cf

创建文件 `/etc/mail/sendmail.mc`，并以此为基础使用 m4) 工具生成 `sendmail.cf` 文件。

您可以由 `/usr/share/sendmail-cf/README` 文件了解配置 sendmail 的全部选项。

警告：无论是从头创建自己的 `sendmail.mc` 文件，还是在已有的 `sendmail.mc` 文件基础上进行修改，请时刻牢记，在**非 TLS** 情况下使用明文验证是十分危险的行为。除非您明确了解自己行为的意义，请使用以下示例中的方法强制进行 TLS 验证以确保安全性。

下面是在 TLS 之上进行身份验证的配置文件示例。例子包含了解释工作原理的注释，这些注释以 `dnl` 这个单词起始。

```
/etc/mail/sendmail.mc
include(`/usr/share/sendmail-cf/m4/cf.m4')
define(`confDOMAIN_NAME', `your-domain.com')dnl
FEATURE(use_cw_file)
dnl  The following allows relaying if the user authenticates,
```

```
dnl  and disallows plaintext authentication (PLAIN/LOGIN) on
dnl  non-TLS links:
define(`confAUTH_OPTIONS', `A p y')dnl
dnl
dnl  Accept PLAIN and LOGIN authentications:
TRUST_AUTH_MECH(`LOGIN PLAIN')dnl
define(`confAUTH_MECHANISMS', `LOGIN PLAIN')dnl
dnl
dnl Make sure this paths correctly point to your SSL cert files:
define(`confCACERT_PATH',`/etc/ssl/certs')
define(`confCACERT',`/etc/ssl/cacert.pem')
define(`confSERVER_CERT',`/etc/ssl/certs/server.crt')
define(`confSERVER_KEY',`/etc/ssl/private/server.key')
dnl
FEATURE(`virtusertable', `hash /etc/mail/virtusertable.db')dnl
OSTYPE(linux)dnl
MAILER(local)dnl
MAILER(smtp)dnl
```

然后使用使用

```
# m4 /etc/mail/sendmail.mc > /etc/mail/sendmail.cf
```

命令生成 `sendmail.cf` 文件。

注意：如果您对 `sendmail.cf` 语法感兴趣，请参阅 这篇文章 了解详情。

sendmail.conf

`sendmail.conf` 文件，如果存在，则也是配置 Sendmail 并最终生成 `sendmail.cf` 的一个配置文件。请阅读文件内含的注释了解详细信息。

local-host-names

请将您的域名写入 `local-host-names` 文件:

```
/etc/mail/local-host-names
localhost
your-domain.com
mail.your-domain.com
localhost.localdomain
```

请确保您的域名可以被 `/etc/hosts` 文件解析。

access.db

创建文件 `/etc/mail/access` 然后写入规则以配置邮件转发、允许收信与拒信。假设你在 `10.5.0.0/24` 有一个 VPN，而且你希望转发来自该 IP 段的所有邮件：

```
/etc/mail/access
10.5.0 RELAY
127.0.0 RELAY
```

然后使用

```
# makemap hash /etc/mail/access.db < /etc/mail/access
```

命令处理生成 sendmail 可以使用的配置数据库。

aliases.db

编辑文件 `/etc/mail/aliases` ，反注释这一行： `#root: human being here` 并将其修改如下：

```
root:         your-username
```

你可以在此添加用户的别名。例如：

```
coolguy:      your-username
somedude:     your-username
```

然后使用

```
# newaliases
```

命令进行处理。

virtusertable.db

创建 `virtusertable` 文件并在其中写入含有域名的别名：（这项功能在您的服务器绑定多域名时十分有用）

```
/etc/mail/virtusertable
your-username@your-domain.com          your-username
joe@my-other.tk                        joenobody
```

然后使用

```
# makemap hash /etc/mail/virtusertable.db < /etc/mail/virtusertable
```

命令进行处理。

开机自动启动

启用并启动下列服务。请阅读守护进程了解详情。

- `saslauthd.service`
- `sendmail.service`
- `sm-client.service`

SASL 验证

将用户添加至 SASL 数据库并设置用于 SMTP 身份验证的密码。

```
# saslpasswd2 -c your-username
```

注：您可以设置使用异于 SASL 的身份验证途径。这需要进一步修改配置文件，这里不赘述。

小窍门

将某个域名的全部邮件转发至特定邮箱

若需将 **my-other.tk** 域名下所有用户的电子邮件转发到 **your-username@your-domain.com**，请在 `/etc/mail/virtusertable` 文件中添加以下一行：

```
@my-other.tk          your-username@your-domain.com
```

不要忘记再次使用如下命令更新配置数据库：

```
# makemap hash /etc/mail/virtusertable.db < /etc/mail/virtusertable
```

21.3　邮件服务器

难度 进阶 | **篇幅** 13.0 KB | **所属** 邮件服务器

本节将讲到

- 软件
- 端口
- MX record
- 身份验证
- 测试

翻译状态：

- 本文（或部分内容）译自 Mail Server，最近一次同步于 2024-10-20，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

邮件服务器由多个组件组成。邮件传送代理（MTA）通过 SMTP 接收和发送电子邮件。收到和接收的邮件会被传递给邮件投递代理（MDA），后者会将邮件存储在邮箱中（通常为 mbox 或 Maildir 格式）。如果想让用户使用电子邮件客户端（MUA）远程访问邮件，则需要运行 POP3 和/或 IMAP 服务器。

软件

下表列出了所有邮件服务器及其支持的功能。

注意：在大多数情况下，本表中的任何一个程序都无法提供所需的全套支持；postfix 等 MTA 需要与 dovecot 等 MDA 搭配使用。

名称	邮件传送代理	邮件投递代理	发送邮件	邮件列表	JMAP	提示		
发送	接收	POP3	IMAP					
dma	是	是	–	–	是	–	–	不支持电子邮件域名（email domains）；MTA 接收受限（请参阅使用 Google SMTP）.
Exim	是	是	–	–	是	–	–	–
OpenSMTPD	是	是	–	–	是	–	–	–
Postfix	是	是	–	–	是	部分，通过别名功能	–	–
Courier	是	是	是	是	否	是	–	包含一个网络客户端
Cyrus IMAP	–	–	是	是	–	–	是	可通过 cyrus-imapdAUR 安装
Dovecot	–	–	是	是	–	–		–

名称	邮件传送代理	邮件投递代理	发送邮件	邮件列表	JMAP	提示	
							否, 已请求	
UW IMAP	是，使用 c-clientAUR	–	是	是	是，c-clientAUR 具有相同功能	–	–	项目已被放弃
msmtp	是	否	–	–	是，msmtp 具有相同功能	–	–	–
Sendmail	是	否	–	–	是	–	–	Sendmail 实现已过时
sSMTP	是	否	–	–	是	–	–	–
fdm	–	–	是	是	–	–	–	还可以从标准输入（stdin）中获取邮件
Procmail	–	–	部分，参见模块	否	–	–	–	仅通过标准输入（stdin）读取邮件，上游未维护
Maildrop	–	–	否	否	–	–	–	仅支持通过标准输入（stdin）接收电子邮件
Stalwart	是	是	是	是	–	–	是	–

注意：

- Sendmail 可执行程序是一种命令行工具，可通过 SMTP 发送电子邮件。
- "发送 "和 "接收 "栏指的是通过 SMTP 发送邮件和通过 SMTP 接收邮件的功能。
- "邮件列表"软件允许创建邮件列表，参见邮件列表。

端口

用途	端口	协议	加密
接受来自其他 MTA 的邮件。	25	SMTP	STARTTLS
接受 MUA 的 submissions。	587	SMTP	STARTTLS
465	SMTPS	implicit TLS	
允许 MUA 访问邮件。	110	POP3	STARTTLS
995	POP3S	Implicit TLS	
143	IMAP	STARTTLS	
993	IMAPS	implicit TLS	

注意：

- Implicit TLS 比 STARTTLS 更安全，因为后者容易受到 中间人攻击。更多信息，请参阅 和 RFC:8314。

- 上表包含了要使用的通用端口，但邮件服务器可能会使用其他端口，请与邮件提供商确认端口是否正确。唯一不变的端口是 25 端口，中继端口必须打开，否则邮件将无法通过特定服务器进行中继（不过有些软件支持设置自定义中继端口）。

MX record

Hosting a mail server requires a domain name with an MX record pointing to the domain name of your mail transfer agent. The domain name used as the value of the MX record must map to at least one address record (A, AAAA) and must not have a CNAME record to conform with RFC 2181, otherwise you may not get mail from some mail servers. Configuring DNS records is usually done from the configuration interface of your domain name registrar.

身份验证

**本文或本章节可能需要合并到邮件服务器#软件。**

附注： 此部分可以与上面的邮件服务器软件列表合并。（在 Talk:邮件服务器 中讨论）

有多种电子邮件验证技术。

Sender Policy Framework

摘自 Wikipedia：

: **Sender Policy Framework** (**SPF**) 是一种电子邮件验证协议，旨在通过提供一种机制，使接收邮件的邮件交换者能够验证来自某个域的传入邮件是否来自该域管理员授权的 IP 地址，从而检测和阻止电子邮件欺骗行为。

要允许其他邮件交换器验证明显从您的域名发送的邮件，您需要设置 DNS TXT 记录，如 Wikipedia 文章所述（也有一个在线向导）。要使用 SPF 验证收到的邮件，您需要将邮件传输代理配置为使用 SPF 实现。有多种 SPF 实现可供选择： libspf2包、perl-mail-spf包 和 perl-mail-spf-query包。

<table class="wikitable sortable"><caption>SPF 验证支持
</caption><tbody><tr><th><a href="/wzh/index.php?
title=Courier&action=edit&redlink=1" class="new" title="Courier（页面不存在）">Courier</th><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是, 内置
</td></tr><tr><th>Postfix</th><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是</td></tr><tr><th>Sendmail</th><td data-sort-value="5" class="archwiki-table-

cell-background-green" style="vertical-align: middle; text-align: center;">通过 Milter 与 spfmilter-acme^{<small>AUR</small>}</td></tr><tr><th>Exim</th><td data-sort-value="5" class="archwiki-table-cell-background-green" style="vertical-align: middle; text-align: center;">是, 需要 libspf2^{<small>包</small>}</td></tr><tr><th>OpenSMTPD</th><td data-sort-value="1" class="archwiki-table-cell-background-red" style="vertical-align: middle; text-align: center;">否</td></tr></tbody></table>

通过以下网站，您可以验证 SPF 记录：

- SPF Record Checker
- SPF Email test

提示：SPF 甚至对不用于发送电子邮件的域名也有帮助。发布类似 `v=spf1 -all` 这样的策略可使任何执行 SPF 的邮件服务器拒绝来自您域名的电子邮件，从而防止滥用。

Sender Rewriting Scheme

The Sender Rewriting Scheme (SRS) is a secure scheme to allow forwardable bounces for server-side forwarded emails without breaking the Sender Policy Framework.

关于 Postfix，请参阅Postfix#Sender Rewriting Scheme。

DKIM

DomainKeys Identified Mail（DKIM）是一种域名级电子邮件验证方法，旨在检测电子邮件欺骗行为。

可用的 DKIM 实现有 OpenDKIM 和 perl-mail-dkim包。

ARC

ARC 是一个实验性标准，即 Authenticated Received Chain (ARC) 电子邮件验证系统。它允许中间邮件服务器（如邮件列表或转发服务）签署电子邮件的原始验证结果。谷歌、微软、ProtonMail、Fastmail 等公司都采用了 ARC 系统。

可用的 ARC 实现有 rspamd包 （作为 rspamd 模块）和 OpenARC （独立）。

测试

有多个选项可帮助您测试 DNS 记录、传送能力和加密支持。

专用工具

- swaks包

专用网站

有几个方便的网站可以帮助您进行测试。

- https://mxtoolbox.com/
- https://ismyemailworking.com/
- https://www.mail-tester.com/ （仅 3 次免费使用）
- https://www.checktls.com/
- https://pingability.com/zoneinfo.jsp

提示与技巧

移除电子邮件中的 IP 地址

大多数邮件服务器都可以通过配置从外发邮件中删除用户的 IP 地址和用户代理。

扫描电子邮件以发现病毒

有关电子邮件防病毒扫描，请参阅ClamAV。

垃圾过滤

有关垃圾邮件的过滤，请参阅SpamAssassin。

Webmail

有关网络邮件的设置，请参阅 Roundcube 和 Squirrelmail。

参见

- 邮件服务器比较 - 进一步比较邮件服务器，包括 Arch Linux 不支持的服务器
- Sieve) – 一种邮件过滤编程语言
- Webmail

21.4 Postfix

难度 高级 | **篇幅** 21.8 KB | **所属** 邮件服务器

本节将讲到

- 安装
- 配置
- 启动 Postfix
- TLS
- Tips and tricks

翻译状态：

- 本文（或部分内容）译自 Postfix，最近一次同步于 2018-12-06，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

Postfix) 是邮件传送代理软件。按照其官方网站的说法:

: attempts to be fast, easy to administer, and secure, while at the same time being sendmail compatible enough to not upset existing users. Thus, the outside has a sendmail-ish flavor, but the inside is completely different.

: 快速、管理简单、安全， 同时足够兼容 Sendmail，从而不会影响现有用户。 因此，从外面看是 sendmail-ish 风格，但内部是完全不同的。

本文基于邮件服务器。 本文的目标是设置 Postfix 并解释基本配置文件的功能。 这里有两种交付方式的设置说明：本地系统用户方式 和 虚拟用户方式。

安装

安装软件包 postfix包。

配置

请参照软件开发者提供的： Postfix Basic Configuration 基础配置项. 默认的配置文件位于`/etc/postfix` 。 其中两个非常重要的文件是:

- `master.cf`, 定义了启用哪些Postfix服务以及客户端如何连接它们, 请参照 master(5)
- `main.cf`, 主配置文件，请参照 postconf(5)（英文）

配置文件更改过后需要重新加载主服务 `postfix.service`。

别名 Aliases

请参照在线 man 文件： aliases(5)。

别名配置文件： `/etc/postfix/aliases`。你可以在这个文件里指定别名 (有时候也被称为 forwarders) 。

您需要将发往“root”的所有邮件映射到另一个帐户，因为以root身份阅读邮件不是一个好主意。

将下面这行取消注释，并且把 `you` 替换成你要使用的真实账户。

```
root: you
```

一旦你完成了对 `/etc/postfix/aliases` 的编辑， 你就需要运行下面的 postalias 命令:

```
postalias /etc/postfix/aliases
```

对于以后的更改，您可以使用:

```
newaliases
```

提示：或者，你也可以为 root 用户创建这个文件 `~/.forward`, 例如 `/root/.forward`。 指定将root的邮件转发到哪个用户, 例如 *user@localhost*。

```
/root/.forward
user@localhost
```

系统本地用户邮件(Local mail)

要仅向本地系统用户（也就是`/etc/passwd`中存在的用户）发送邮件，请更新配置文件：`/etc/postfix/main.cf`中的以下配置行（取消注释，更改或添加）：

```
myhostname = localhost
mydomain = localdomain
mydestination = $myhostname, localhost.$mydomain, $mydomain
inet_interfaces = $myhostname, localhost
mynetworks_style = host
default_transport = error: outside mail is not deliverable
```

所有其他设置维持不变。 完成上面这个配置后，你可能还想配置一些#别名 Aliases参数， 然后#启动 Postfix。

虚拟用户邮件(Virtual mail)

虚拟用户邮件的邮件账户不存储在本地系统的(`/etc/passwd`文件中。可以配合数据库完成对用户账户的存储。

请参见 Virtual user mail system with Postfix, Dovecot and Roundcube (简体中文) 那是一个如何设置的详细介绍。

检查配置 Check configuration

运行`postfix check` 命令来完成配置检查。它会输出所有你在配置文件中可能写错的东西。

运行postconf命令可以查看所有的配置。运行postconf -n命令可以查看与默认配置的区别。

启动 Postfix

注意：即使你没有设置任何#别名 Aliases，也需要至少运行一次newaliases命令才能让 Postfix 正常运行。

启动 postfix.service 服务。

TLS

有关更多信息，请参见 Postfix TLS Support.

Secure SMTP (sending)

默认情况下，Postfix/sendmail 不会加密发送到其他 SMTP 服务器的电子邮件。要在可用时使用 TLS，请在main.cf 中添加以下行：

```
/etc/postfix/main.cf
smtp_tls_security_level = may
```

要强制使用TLS (这种情况下如果远程服务器不支持的话会导致失败), 只需要把 may 变更为 encrypt 就行了。 值得注意的是，如果此邮件服务是一个公开的服务(相对于企业内部服务，不对公网提供服务的那种)时，这样的做法会违反 RFC:2487 ，所以请慎重考虑。

Secure SMTP (receiving)

警告：如果你想部署 TLS,，请务必遵循weakdh.org's guide以防止FREAK/Logjam攻击。从2015中期以来，默认设置对于POODLE攻击已经是安全的。有关更多信息请参见Server-side TLS。

By default, Postfix will not accept secure mail.默认情况下，Postfix不会接受安全邮件。

要在SMTP上启用STARTTLS（端口587，这是保护SMTP的正确方法），请在main.cf文件中添加以下行：

```
/etc/postfix/main.cf
smtpd_tls_security_level = may
smtpd_tls_cert_file = /path/to/cert.pem
smtpd_tls_key_file = /path/to/key.pem
```

在 master.cf 文件中，找到并取消注释以下行，以在该端口上启用服务并设置正确的配置：

```
/etc/postfix/master.cf
submission inet n       -       n       -       -       smtpd
  -o syslog_name=postfix/submission
  -o smtpd_tls_security_level=encrypt
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_tls_auth_only=yes
  -o smtpd_reject_unlisted_recipient=no
#  -o smtpd_client_restrictions=$mua_client_restrictions
```

```
#  -o smtpd_helo_restrictions=$mua_helo_restrictions
#  -o smtpd_sender_restrictions=$mua_sender_restrictions
  -o smtpd_recipient_restrictions=
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
```

The `smtpd_*_restrictions` options remain commented because `$mua_*_restrictions` are not defined in main.cf by default. If you do decide to set any of `$mua_*_restrictions`, uncomment those lines too.

If you need support for the deprecated SMTPS port 465, also follow the next section.

SMTPS (port 465)

The deprecated method of securing SMTP is using the **wrapper mode** which uses the system service **smtps** as a non-standard service and runs on port 465.

To enable it, uncomment the following lines in `master.cf`:

```
/etc/postfix/master.cf
smtps     inet  n       -       n       -       -       smtpd
  -o syslog_name=postfix/smtps
  -o smtpd_tls_wrappermode=yes
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_reject_unlisted_recipient=no
#  -o smtpd_client_restrictions=$mua_client_restrictions
#  -o smtpd_helo_restrictions=$mua_helo_restrictions
#  -o smtpd_sender_restrictions=$mua_sender_restrictions
  -o smtpd_recipient_restrictions=
  -o smtpd_relay_restrictions=permit_sasl_authenticated,reject
  -o milter_macro_daemon_name=ORIGINATING
```

The rationale surrounding the `$smtpd_*_restrictions` lines is the same as above.

After this, verify that these lines are in `/etc/services`:

```
smtps 465/tcp # Secure SMTP
smtps 465/udp # Secure SMTP
```

If they are not there, go ahead and add them (replace the other listing for port 465). Otherwise Postfix will not start and you will get the following error:

```
postfix/master: fatal: 0.0.0.0:smtps: Servname not supported for ai_socktype
```

Tips and tricks

Blacklist incoming emails

Manually blacklisting incoming emails by sender address can easily be done with Postfix.

Create and open `/etc/postfix/blacklist_incoming` file and append sender email address:

```
user@example.com REJECT
```

Then use the `postmap` command to create a database:

```
# postmap hash:blacklist_incoming
```

Add the following code before the first permit rule in `main.cf`:

```
smtpd_recipient_restrictions = check_sender_access hash:/etc/postfix/blacklist_incoming
```

Finally restart `postfix.service`.

Hide the sender's IP and user agent in the Received header

This is a privacy concern mostly, if you use Thunderbird and send an email. The received header will contain your LAN and WAN IP and info about the email client you used. (Original source: AskUbuntu) What we want to do is remove the Received header from outgoing emails. This can be done by the following steps:

Add the following line to `main.cf`:

```
smtp_header_checks = regexp:/etc/postfix/smtp_header_checks
```

Create `/etc/postfix/smtp_header_checks` with this content:

```
/^Received: .*/     IGNORE
/^User-Agent: .*/   IGNORE
```

Finally, restart `postfix.service`.

Postfix in a chroot jail

Postfix is not put in a chroot jail by default. The Postfix documentation provides details about how to accomplish such a jail. The steps are outlined below and are based on the chroot-setup script provided in the Postfix source code.

First, go into the `master.cf` file in the directory `/etc/postfix` and change all the chroot entries to 'yes' (y) except for the services `qmgr`, `proxymap`, `proxywrite`, `local`, and `virtual`

Second, create two functions that will help us later with copying files over into the chroot jail (see last step)

```
CP="cp -p"
cond_copy() {
  # find files as per pattern in $1
  # if any, copy to directory $2
  dir=`dirname "$1"`
  pat=`basename "$1"`
  lr=`find "$dir" -maxdepth 1 -name "$pat"`
  if test ! -d "$2" ; then exit 1 ; fi
  if test "x$lr" != "x" ; then $CP $1 "$2" ; fi
}
```

Next, make the new directories for the jail:

```
set -e
umask 022
POSTFIX_DIR=${POSTFIX_DIR-/var/spool/postfix}
cd ${POSTFIX_DIR}
mkdir -p etc lib usr/lib/zoneinfo
test -d /lib64 && mkdir -p lib64
```

Find the localtime file

```
lt=/etc/localtime
if test ! -f $lt ; then lt=/usr/lib/zoneinfo/localtime ; fi
if test ! -f $lt ; then lt=/usr/share/zoneinfo/localtime ; fi
if test ! -f $lt ; then echo "cannot find localtime" ; exit 1 ; fi
rm -f etc/localtime
```

Copy localtime and some other system files into the chroot's etc

```
$CP -f $lt /etc/services /etc/resolv.conf /etc/nsswitch.conf etc
$CP -f /etc/host.conf /etc/hosts /etc/passwd etc
ln -s -f /etc/localtime usr/lib/zoneinfo
```

Copy required libraries into the chroot using the previously created function `cond_copy`

```
cond_copy '/usr/lib/libnss_*.so*' lib
cond_copy '/usr/lib/libresolv.so*' lib
cond_copy '/usr/lib/libdb.so*' lib
```

And do not forget to reload Postfix.

DANE (DNSSEC)

Resource Record

警告：This is not a trivial section. Be aware that you make sure you know what you are doing. You better read Common Mistakes before.

DANE supports several types of records, however not all of them are suitable in Postfix.

Certificate usage 0 is unsupported, 1 is mapped to 3 and 2 is optional, thus it is recommendet to publish a "3" record. More on Resource Records.

Configuration

Opportunistic DANE is configured this way:

```
/etc/postfix/main.cf
smtpd_use_tls = yes
smtp_dns_support_level = dnssec
smtp_tls_security_level = dane
/etc/postfix/master.cf
dane       unix  -       -       n       -       -       smtp
  -o smtp_dns_support_level=dnssec
  -o smtp_tls_security_level=dane
```

To use per-domain policies, e.g. opportunistic DANE for example.org and mandatory DANE for example.com, use something like this:

```
/etc/postfix/main.cf
indexed = ${default_database_type}:${config_directory}/

# Per-destination TLS policy
#
smtp_tls_policy_maps = ${indexed}tls_policy

# default_transport = smtp, but some destinations are special:
#
transport_maps = ${indexed}transport
transport
example.com dane
example.org dane
tls_policy
example.com dane-only
```

注意：For global mandatory DANE, change `smtp_tls_security_level` to `dane-only`. Be aware that this makes Postfix tempfail (respond with a `4.X.X` error code) on all deliveries that do not use DANE at all!

Full documentation is found here.

Extras

- **PostfixAdmin** — A web-based administrative interface for Postfix.

: http://postfixadmin.sourceforge.net/ || postfixadmin包

Postgrey

**本文或本章节的语言、语法或风格需要改进。参考：帮助:风格**

原因：See Help:Style（在Talk:Postfix讨论）

Postgrey can be used to enable greylisting for a Postfix mail server.

Installation

Install the postgrey包 package. To get it running quickly edit the Postfix configuration file and add these lines:

```
/etc/postfix/main.cf
smtpd_recipient_restrictions =
  check_policy_service inet:127.0.0.1:10030
```

Then start/enable the `postgrey` service. Afterwards, reload the `postfix` service. Now greylisting should be enabled.

Configuration

Configuration is done via editing the `postgrey.service` file. First copy it over to edit it.

```
# cp /usr/lib/systemd/system/postgrey.service /etc/systemd/system/
```

Whitelisting

To add automatic whitelisting (successful deliveries are whitelisted and do not have to wait any more), you could add the `--auto-whitelist-clients=N` option and replace `N` by a suitably small number (or leave it at its default of 5).

...actually, the preferred method should be the override:

```
cat /etc/systemd/system/postgrey.service.d/override.conf
[Service]
ExecStart=
ExecStart=/usr/bin/postgrey --inet=127.0.0.1:10030 \
       --pidfile=/run/postgrey/postgrey.pid \
       --group=postgrey --user=postgrey \
       --daemonize \
       --greylist-text="Greylisted for %%s seconds" \
       --auto-whitelist-clients
```

To add your own list of whitelisted clients in addition to the default ones, create the file `/etc/postfix/whitelist_clients.local` and enter one host or domain per line, then restart `postgrey.service` so the changes take effect.

Troubleshooting

If you specify `--unix=/path/to/socket` and the socket file is not created ensure you have removed the default `--inet=127.0.0.1:10030` from the service file.

For a full documentation of possible options see `perldoc postgrey`.

SpamAssassin

This section describes how to integrate SpamAssassin.

SpamAssassin stand-alone generic setup

注意：If you want to combine SpamAssassin and Dovecot Mail Filtering, ignore the next two lines and continue further down instead.

Edit `/etc/postfix/master.cf` and add the content filter under smtp.

```
smtp      inet  n       -       n       -       -       smtpd
  -o content_filter=spamassassin
```

Also add the following service entry for SpamAssassin

```
spamassassin unix -     n       n       -       -       pipe
  flags=R user=spamd argv=/usr/bin/vendor_perl/spamc -e /usr/bin/sendmail -oi -f
${sender} ${recipient}
```

Now you can start and enable `spamassassin.service`.

SpamAssassin combined with Dovecot LDA / Sieve (Mailfiltering)

Set up LDA and the Sieve-Plugin as described in Dovecot#Sieve. But ignore the last line `mailbox_command...`.

Instead add a pipe in `/etc/postfix/master.cf`:

```
 dovecot   unix  -       n       n       -       -       pipe
       flags=DRhu user=vmail:vmail argv=/usr/bin/vendor_perl/spamc -u spamd -
e /usr/lib/dovecot/dovecot-lda -f ${sender} -d ${recipient}
```

And activate it in `/etc/postfix/main.cf`:

```
 virtual_transport = dovecot
```

SpamAssassin combined with Dovecot LMTP / Sieve

Set up the LMTP and Sieve as described in Dovecot#Sieve.

Edit `/etc/dovecot/conf.d/90-plugins.conf` and add:

```
 sieve_before = /etc/dovecot/sieve.before.d/
 sieve_extensions = +vnd.dovecot.filter
 sieve_plugins = sieve_extprograms
 sieve_filter_bin_dir = /etc/dovecot/sieve-filter
 sieve_filter_exec_timeout = 120s #this is often needed for the long running
spamassassin scans, default is otherwise 10s
```

Create the directory and put spamassassin in as a binary that can be ran by dovecot:

```
 # mkdir /etc/dovecot/sieve-filter
 # ln -s /usr/bin/vendor_perl/spamc /etc/dovecot/sieve-filter/spamc
```

Create a new file, `/etc/dovecot/sieve.before.d/spamassassin.sieve` which contains:

```
 require [ "vnd.dovecot.filter" ];
 filter "spamc" [ "-d", "127.0.0.1", "--no-safe-fallback" ];
```

Compile the sieve rules `spamassassin.svbin`:

```
 # cd /etc/dovecot/sieve.before.d
 # sievec spamassassin.sieve
```

Finally, restart `dovecot.service`.

Rule-based mail processing

With policy services one can easily finetune Postfix' behaviour of mail delivery. postfwd包 and policydAUR provide services to do so. This allows you to e.g.

implement time-aware grey- and blacklisting of senders and receivers as well as SPF policy checking.

Policy services are standalone services and connected to Postfix like this:

```
/etc/postfix/main.cf
smtpd_recipient_restrictions =
  ...
  check_policy_service unix:/run/policyd.sock
  check_policy_service inet:127.0.0.1:10040
```

Placing policy services at the end of the queue reduces load, as only legitimate mails are processed. Be sure to place it before the first permit statement to catch all incoming messages.

Sender Policy Framework

To use the Sender Policy Framework with Postfix, install python-postfix-policyd-spfAUR.

Edit `/etc/python-policyd-spf/policyd-spf.conf` to your needs. An extensively commented version can be found at `/etc/python-policyd-spf/policyd-spf.conf.commented`. Pay some extra attention to the HELO check policy, as standard settings strictly reject HELO failures.

In the main.cf add a timeout for the policyd:

```
/etc/postfix/main.cf
policy-spf_time_limit = 3600s
```

Then add a transport

```
/etc/postfix/master.cf
policy-spf  unix  -       n       n       -       0       spawn
     user=nobody argv=/usr/bin/policyd-spf
```

Lastly you need to add the policyd to the `smtpd_recipient_restrictions`. To minimize load put it to the end of the restrictions but above any `reject_rbl_client` DNSBL line:

```
/etc/postfix/main.cf
smtpd_recipient_restrictions=
     ...
     permit_sasl_authenticated
     permit_mynetworks
     reject_unauth_destination
     check_policy_service unix:private/policy-spf
```

You can test your Setup with the following:

```
/etc/python-policyd-spf/policyd-spf.conf
defaultSeedOnly = 0
```

Sender Rewriting Scheme

To use the Sender Rewriting Scheme with Postfix, install postsrsdAUR and adjust the settings:

```
/etc/postsrsd/postsrsd
SRS_DOMAIN=yourdomain.tld
SRS_EXCLUDE_DOMAINS=yourotherdomain.tld,yet.anotherdomain.tld
SRS_SEPARATOR==
SRS_SECRET=/etc/postsrsd/postsrsd.secret
SRS_FORWARD_PORT=10001
SRS_REVERSE_PORT=10002
```

```
RUN_AS=postsrsd
CHROOT=/usr/lib/postsrsd
```

Enable and start the daemon, making sure it runs after reboot as well. Then configure Postfix accordingly by tweaking the following lines:

```
/etc/postfix/main.cf
sender_canonical_maps = tcp:localhost:10001
sender_canonical_classes = envelope_sender
recipient_canonical_maps = tcp:localhost:10002
recipient_canonical_classes= envelope_recipient,header_recipient
```

Restart Postfix and start forwarding mail.

Troubleshooting

Warning: "database /etc/postfix/.db is older than source file .."*

If you get one or both warnings with `journalctl`

```
warning: database /etc/postfix/virtual.db is older than source
file /etc/postfix/virtual
warning: database /etc/postfix/transport.db is older than source
file /etc/postfix/transport
```

then you can fix it by using these commands depending on the messages you get

```
postmap /etc/postfix/transport
postmap /etc/postfix/virtual
```

and restart `postfix.service`

See also

- Official documentation
- Postfix Ubuntu documentation
- Out of Office for Squirrelmail

FTP

把远端目录当本地文件夹用，和在不受信任的网络上安全搬运文件，是两件事。本章分开处理：CurlFtpFS 把 FTP 服务器挂成本地目录，SCP 与 SFTP 负责加密传输，SFTP chroot 负责收紧权限。建议按顺序读，前两节必读：第一节含 root 与普通用户两种挂载、连接加密服务器与乱码处理，第二节是日常最常用的命令。SFTP chroot 偏进阶，只在给外部账号开放受限传输权限时才细读，否则可跳。

最易踩的坑是协议关系：SFTP 与 SCP 都建构在 SSH 之上，chroot 改的是 sshd 配置而非 FTP 服务；两种挂载身份的目录路径不同，照抄对方命令常直接失败；乱码要先查挂载选项与服务器编码，而不是反复重挂。

22.1　CurlFtpFS

难度 入门 | **篇幅** 3.9 KB | **所属** FTP

本节将讲到

- 安装
- Mount FTP folder as root
- Mount FTP folder as normal user
- Connect to encrypted server
- 乱码

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： Most of this article has not been translated（在 Talk:CurlFtpFS# 中讨论）

注意：As of February 2015, curlftpfs is reported to be extremely slow, see for example a Ubuntu bug report and a stackoverflow.com question.

CurlFtpFS is a filesystem for accessing FTP hosts based on FUSE and libcurl.

安装

安装软件包 curlftpfs包。

有必要的话，加载fuse内核模块

```
# modprobe fuse
```

Mount FTP folder as root

Create the mount point and then mount the FTP folder.

```
# mkdir /mnt/ftp
# curlftpfs ftp.yourserver.com /mnt/ftp/ -o user=username:password
```

If you want to give other (regular) users access right, use the `allow_other` option:

```
# curlftpfs ftp.yourserver.com /mnt/ftp/ -o user=username:password,allow_other
```

Do not add space after the comma or the `allow_other` argument will not be recognized.

To use FTP in active mode add the option 'ftp_port=-':

```
# curlftpfs ftp.yourserver.com /mnt/ftp/ -o
user=username:password,allow_other,ftp_port=-
```

You can add this line to /etc/fstab to mount automatically.

```
curlftpfs#USER:PASSWORD@ftp.domain.org /mnt/mydomainorg fuse
auto,user,uid=1000,allow_other,_netdev 0 0
```

提示：You can use codepage="*string*" when having problems with non-US English characters on servers that do not support UTF8, e.g. codepage="iso8859-1"

To prevent the password to be shown in the process list, create a `.netrc` file in the home directory of the user running curlftpfs and `chmod 600` with the following content:

```
machine ftp.yourserver.com
login username
password mypassword
```

Mount FTP folder as normal user

You can also mount as normal user (always use the `.netrc` file for the credentials and ssl encryption!):

```
$ mkdir ~/my-server
$ curlftpfs -o ssl,utf8 ftp://my-server.tld/ ~/my-server
```

if the answer is

```
Error connecting to ftp: QUOT command failed with 500
```

then the server does not support the `utf8` option. Leave it out and all will be fine.

提示：If need be try setting the encoding with for example -o codepage="iso8859-1"

To unmount:

```
$ fusermount -u ~/my-server
```

Connect to encrypted server

In its default settings, CurlFtpFS will authenticate in cleartext when connecting to a non encrypted connection port. If the remote server is configured to refuse non encrypted authentification method / force encrypted authentification, CurlFtpFS will return a

```
# Error connecting to ftp: Access denied: 530
```

To authenticate to the ftp server using explicit encrypted authentification, you must specify the ssl or tsl option.

```
# curlftpfs ftp.yourserver.com /mnt/ftp/ -o ssl,user=username:password
```

If your server uses a self-generated certificate not thrusted by your computer, you can specify to ignore it

```
# curlftpfs ftp.yourserver.com /mnt/ftp/ -o
ssl,no_verify_peer,no_verify_hostname,user=username:password
```

An implicit tsl mode is also available. For more details, check the manual page.

乱码

如果中文出现乱码，你可能需要指定编码：

```
sudo curlftpfs -o codepage=gbk -o rw,allow_other
ftp://username:password@ftp.yourserver.com /mnt/ftp
```

22.2 SCP 与 SFTP

难度 进阶 | **篇幅** 7.3 KB | **所属** FTP

本节将讲到

- Secure file transfer protocol (SFTP)
- Secure file transfer protocol (SFTP) with a chroot jail
- Secure copy protocol (SCP)

**本文或本章节可能需要合并到SFTP chroot。**

附注： Instructions seem to be the same as in SFTP chroot and has more content.（在 Talk:SCP 与 SFTP#Incorrect 'Considered for redirection' banner? 中讨论）

翻译状态：

- 本文（或部分内容）译自 SCP and SFTP，最近一次同步于 2024-09-05，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

The Secure copy (SCP) is a protocol to transfer files via a Secure Shell connection. The SSH file transfer protocol (SFTP) is a related protocol, also relying on a secure shell back-end. Both protocols allow secure file transfers, encrypting passwords and transferred data. The SFTP protocol, however, features additional capabilities like, for example, resuming broken transfers or remote file manipulation like deletion.

Secure file transfer protocol (SFTP)

Install and configure OpenSSH. Once running, SFTP is available by default.

Access files with the *sftp* program or SSHFS. Many standard FTP programs should work as well.

Secure file transfer protocol (SFTP) with a chroot jail

Sysadmins can jail a subset of users to a chroot jail using openssh包 thus restricting their access to a particular directory tree. This can be useful to simply share some files without granting full system access or shell access. Users with this type of setup may use SFTP clients such as filezilla包 to put/get files in the chroot jail.

Setup the filesystem

Create a jail directory:

```
# mkdir -p /var/lib/jail
```

Optionally, bind mount the filesystem to be shared to this directory. In this example, `/mnt/data/share` is to be used. It is owned by root and has octal permissions of 755.

```
# mount -o bind /mnt/data/share /var/lib/jail
```

提示：Consider adding an entry to `/etc/fstab` to make the bind mount survive a reboot.

Create an unprivileged user

Create the share user and setup a good password:

```
# useradd -g sshusers -d /var/lib/jail foo
# passwd foo
```

Setup OpenSSH

Add the following to the end of `/etc/ssh/sshd_config` to enable the share and to enforce the restrictions:

```
/etc/ssh/sshd_config
...
 Match group sshusers
  ChrootDirectory %h
  X11Forwarding no
  AllowTcpForwarding no
  PasswordAuthentication yes
  ForceCommand internal-sftp
```

Restart `sshd.service` to re-read the configuration file. See SFTP chroot to configure the keys correctly when using chroot or it will get permission denied.

Test that in fact, the restrictions are enforced by attempting an ssh connection via the shell. The ssh server should return a polite notice of the setup:

```
$ ssh foo@someserver.com
foo@someserver.com's password:
This service allows sftp connections only.
Connection to someserver.com closed.
```

Secure copy protocol (SCP)

Install, configure and start OpenSSH. It contains the *scp* utility to transfer files.

More features are available by installing additional packages, for example rsshAUR or scponly包 described below.

注意：Since OpenSSH 9.0 the *scp* utility uses the SFTP protocol by default. The `-O` option must be used to use the legacy SCP protocol.

警告：The scp protocol is outdated, inflexible and not readily fixed. Its authors recommend the use of more modern protocols like sftp and rsync for file transfer instead.

General Usage

Linux to Linux

Copy file from a remote host to local host SCP example:

```
$ scp username@from_host:file.txt /local/directory/
```
Copy file from local host to a remote host SCP example:

```
$ scp file.txt username@to_host:/remote/directory/
```
Copy directory from a remote host to local host SCP example:

```
$ scp -r username@from_host:/remote/directory/  /local/directory/
```
Copy directory from local host to a remote host SCP example:

```
$ scp -r /local/directory/ username@to_host:/remote/directory/
```
Copy file from remote host to remote host SCP example:

```
$ scp username@from_host:/remote/directory/file.txt username@to_host:/remote/directory/
```

Linux to Windows

Use a Windows program such as WinSCP

Scponly

Scponly is a limited shell for allowing users scp/sftp access and only scp/sftp access. Additionally, one can setup *scponly* to chroot the user into a particular directory increasing the level of security.

install scponly包.

For existing users, simply set the user's shell to scponly:

```
# usermod -s /usr/bin/scponly username
```

Adding a chroot jail

The package comes with a script to create a chroot. To use it, run:

```
# /usr/share/doc/scponly/setup_chroot.sh
```

- Provide answers
- Check that `/path/to/chroot` has `root:root` owner and `r-x` for others
- Change the shell for selected user to `/usr/bin/scponlyc`
- sftp-server may require some libnss modules such as libnss_files. Copy them to chroot's `/lib` path.

Uploads to Chroot jail root dir

For security reasons the directory set as the chroot directory must be owned by root with only root having write access to it otherwise sftp/ssh connections will be denied. This of course means regular users cannot upload files to the root directory. In order to get around this while not compromising security you can create a folder inside the chroot directory which the regular user or group has write access to, e.g:

```
# cd /var/lib/jail
# mkdir uploads
# chown :sshusers uploads
# chmod 730 uploads
```
注意：This will only allow users of group "sshusers" to upload to (but not list the contents of) the "uploads" directory. Use `chmod 770` to allow sshusers to view contents.

Some applications utilizing SFTP do not allow input of sub-directories when performing operations (e.g. uploading files), and will attempt to upload files to the

chroot base directory (which will be denied). In order to force these applications to use a specific sub-directory you can append the following to the "ForceCommand" option:

```
/etc/ssh/sshd_config
...
 Match group sshusers
  ...
  ForceCommand internal-sftp -d /uploads
```

Users on connect will then have their start directory change to the specified sub-directory (remember to restart the sshd server).

22.3 SFTP chroot

难度 进阶 | **篇幅** 8.4 KB | **所属** FTP

本节将讲到

- 安装
- 配置
- 技巧提示
- SFTP 的替代品
- 参见

OpenSSH 4.9+ 包含一个用于 SFTP 的内置 chroot，但需要对普通安装进行一些调整。

安装

安装并配置 OpenSSH。运行后，确保 `sftp-server` 已正确设置：

```
/etc/ssh/sshd_config
Subsystem sftp /usr/lib/ssh/sftp-server
```

使用 *sftp* 或 SSHFS 访问文件。许多标准的FTP 客户端也可能有用。

配置

设置文件系统

注意：

- 读者可以自行选择文件访问方案。例如，选择性地为一块传入（可写）空间和/或一块只读空间创建一个子文件夹。这不需要在 `/srv/ssh/jail`之下直接完成——它可以在 live 绑定挂载的磁盘分区完成。
- 也可以 chroot 到 `/home` 目录从而跳过绑定，但是目标用户家目录应归属根：

```
# chown root:root /home/<username>
# chmod 0755 /home/<username>
```

绑定挂载要共享的 live 文件系统到此目录。此例使用 `/mnt/data/share`，由用户 `root` 拥有并具有 `755` 的八进制 [权限](../../07_开发与工具/命令行/文件权限与属性.md)：

```
# chown root:root /mnt/data/share
# chmod 755 /mnt/data/share
# mkdir -p /srv/ssh/jail
# mount -o bind /mnt/data/share /srv/ssh/jail
```

将以下添加到 fstab 以使绑定挂载在重新启动后不失效：

```
/mnt/data/share /srv/ssh/jail  none   bind   0   0
```

创建一个无特权用户

注意：您无需创建一个组，可以使用 `Match User` 以代替 `Match Group`。

创建 `sftponly` 用户组:

```
# groupadd sftponly
```

创建一个使用 *sftponly* 作为主要组的用户并且 shell 登录访问被拒:

```
# useradd -g sftponly -s /usr/bin/nologin -d /srv/ssh/jail username
```

设置一个（复杂的）密码以防止 `account is locked` 错误（即使使用密钥验证也可能出现）：

```
# passwd username
```

配置 OpenSSH

注意：您可能想使用 `Match User` 而不是 `Match Group`，如上一步所述。

```
/etc/ssh/sshd_config
Subsystem sftp /usr/lib/ssh/sftp-server

Match Group sftponly
  ChrootDirectory %h
  ForceCommand internal-sftp
  AllowTcpForwarding no
  X11Forwarding no
  PasswordAuthentication no
```

重启 `sshd.service` 以应用更改。

为 authorized_keys 修复路径

提示：在 `(pre)auth`错误的情况下，在客户端和服务器上使用 OpenSSH 的 调试模式。

应用*AuthorizedKeysFile*的标准路径时，已 chroot 的用户 SSH 密钥验证会失败。欲修复此问题，附加一个 *AuthorizedKeysFile* 上的属于根的目录到 `/etc/openssh/sshd_config`，以 `/etc/ssh/authorized_keys` 为例:

```
/etc/ssh/sshd_config
AuthorizedKeysFile /etc/ssh/authorized_keys/%u .ssh/authorized_keys
PermitRootLogin no
PasswordAuthentication no
PermitEmptyPasswords no
Subsystem sftp /usr/lib/ssh/sftp-server
```

创建 *authorized_keys* 文件夹，在客户端生成一个 SSH 密钥，复制服务器密钥 `/etc/ssh/authorized_keys`（或任何其他首选方法）的内容并设置正确的权限:

```
# mkdir /etc/ssh/authorized_keys
# chown root:root /etc/ssh/authorized_keys
```

```
# chmod 755 /etc/ssh/authorized_keys
# echo 'ssh-rsa <key> <username@host>' >> /etc/ssh/authorized_keys/username
# chmod 644 /etc/ssh/authorized_keys/username
```

重启 `sshd.service`.

技巧提示

写权限

绑定需要路径完全归属 `root`，但文件和/或子文件夹不需要。 以下示例中，用户 ***www-demo*** 以 `/srv/ssh/www/demo` 作为 jail 目录:

```
# mkdir /srv/ssh/www/demo/public_html
# chown www-demo:sftponly /srv/ssh/www/demo/public_html
# chmod 755 /srv/ssh/www/demo/public_html
```

用户现在应可在此目录中创建文件/子目录。欲详细信息，请参阅文件权限和属性。

仅允许上传

欲仅允许通过 sftp 上传并禁止下载文件，修改 `ForceCommand internal-sftp` 行：

```
/etc/ssh/sshd_config
Match Group sftponly

  ForceCommand internal-sftp -u 0666 -p realpath,open,write,close,lstat
```

日志

用户无法访问 `/dev/log`。当用户连入并尝试下载该文件，即可通过于进程上运行 `strace` 以见此。

创建子目录

在 `ChrootDirectory` 中创建子目录 `dev`，例如：

```
# mkdir /usr/local/chroot/user/dev
# chmod 755 /usr/local/chroot/user/dev
```

现在您应于 `/usr/local/chroot/user/dev/log` 创建被 openssh 接受的套接字。您可以直接将该套接字与 `/dev/log` 或 `/run/systemd/journal/dev-log`（若您使用 journald）绑定或用 `syslog-ng`/`rsyslog` 创建。

绑定 journald

```
# touch /usr/local/chroot/user/dev/log
# mount --bind /run/systemd/journal/dev-log /usr/local/chroot/user/dev/log
```

Syslog-ng 配置

添加新源到 `/etc/syslog-ng/syslog-ng.conf` 日志的并添加配置，例如修改段：

```
source src {
  unix-dgram("/dev/log");
  internal();
  file("/proc/kmsg");
```

```
};
```

成为：

```
source src {
  unix-dgram("/dev/log");
  internal();
  file("/proc/kmsg");
  unix-dgram("/usr/local/chroot/theuser/dev/log");
};
```

并附加：

```
#sftp configuration
destination sftp { file("/var/log/sftp.log"); };
filter f_sftp { program("internal-sftp"); };
log { source(src); filter(f_sftp); destination(sftp); };
```

（可选）如果您想将 SSH 消息类似地记录到其自身的文件中：

```
#sshd configuration
destination ssh { file("/var/log/ssh.log"); };
filter f_ssh { program("sshd"); };
log { source(src); filter(f_ssh); destination(ssh); };
```

（来自 Syslog-ng#Move log to another file）

OpenSSH 配置

编辑 `/etc/ssh/sshd_config` 以替换所有 `internal-sftp` 的实例为 `internal-sftp -f AUTH -l VERBOSE`。

重启服务

重启 `syslog-ng` 与 `sshd` 服务。

`/usr/local/chroot/theuser/dev/log` 现应存在。

SFTP 的替代品

安全复制协议 (SCP)

openssh包 提供 *scp* 命令来传输文件。SCP 可能比使用 SFTP 更快 .

安装 rsshAUR 或 scponly包 作为替代 shell 解决方案。

只使用 SCP (Scponly)

安装 scponly包.

对于现有用户，只需将用户的 shell 设置为 scponly：

```
# usermod -s /usr/bin/scponly username
```

参见 Scponly 维基以获得更多细节。

添加 chroot jail

该软件包附带一个用于创建 chroot 的脚本。要使用它，请运行：

```
# /usr/share/doc/scponly/setup_chroot.sh
```

- 提供答案。
- 核实 `/path/to/chroot` 属于 `root:root` 和有对于其他用户的 `r-x`。
- 将所选用户的 shell 更改为 `/usr/bin/scponlyc`。
- sftp 服务器 可能请求一些 libnss 模块例如 libnss_files。将它们复制到 chroot 的 `/lib` 路径。

参见

- https://www.minstrel.org.uk/papers/sftp/
- sshd_config(5)

NTP

时钟跑偏的麻烦常以别的面目出现：证书校验失败、日志时间戳错乱、跨机任务莫名不同步，追到底才发现是时间在漂。本章要解决的，就是在三套并存的时间同步方案里选一套装好、配好，并让它长期稳定地跑下去。取舍也清楚：Systemd-timesyncd 篇幅最短、配置最少，单机与多数服务器装完即用，属必读且应先读；Chrony 在链路时断时续时收敛更快，还能把同步状态当作网络可用的信号，值得紧接着读；Network Time Protocol daemon 体量与排错篇幅最大，需要精细控制或深入诊断时再翻，普通场景可跳，但排错一节值得留着。

最容易踩的坑是以为三者能同时启用：它们争的是同一份职责，装了两套就必须明确关掉多余的那一套，否则配置看似生效，实际并未接管。

23.1　Systemd-timesyncd

难度 进阶 | **篇幅** 7.6 KB | **所属** NTP

本节将讲到

- 配置
- 使用
- 参阅

摘自 systemd 邮件列表：

: *systemd-timesyncd* 是一个用于跨网络同步系统时钟的守护服务。它实现了一个 SNTP 客户端。与 chrony 或 NTP 参考服务器等 NTP 实现相比，这个服务仅做了客户端实现，摒弃了完整 NTP 实现的复杂性。它只专注于从远程服务器查询然后同步到本地时钟。除非你打算为网络客户端提供 NTP 服务或者连接本地硬件时钟，否则这个简单的 NTP 客户端应该更适合大多数人。守护进程运行只需要最小特权，并且会跟网络服务 networkd 挂钩，仅在网络连接可用时才工作。每次收到一个新的 NTP 同步请求或每 60 秒时，后台服务就把当前时间保存到磁盘，并尽可能在系统启动时修正系统时间，这样处理的目的是为了适应像 Raspberry Pi 和嵌入式设备这种缺少 RTC 的系统，并确保这些系统时单点处理（即使它并不是总是正确的）。如果要使用这个守护进程，需要在安装系统时创建一个新的系统用户和组“systemd-timesync”。

配置

systemd-timesyncd 启动时会读取 `/etc/systemd/timesyncd.conf` 配置文件，内容如下:

```
/etc/systemd/timesyncd.conf
[Time]
#NTP=
```

```
#FallbackNTP=0.arch.pool.ntp.org 1.arch.pool.ntp.org 2.arch.pool.ntp.org
3.arch.pool.ntp.org
#...
```

要增加或者更改时间同步服务器, 取消上文中对应的注释，然后将服务器的域名或 IP 以空格间隔开列出。另外，你也可以将配置文件放入 `/etc/systemd/timesyncd.conf.d/*.conf`，详细信息请参考 timesyncd.conf(5)。

例如，可以使用 NTP 项目或者 Arch 默认（NTP 也有这个站点）提供的任何时间服务器：

```
/etc/systemd/timesyncd.conf 或者 /etc/systemd/timesyncd.conf.d/local.conf
[Time]
NTP=0.arch.pool.ntp.org 1.arch.pool.ntp.org 2.arch.pool.ntp.org 3.arch.pool.ntp.org
FallbackNTP=0.pool.ntp.org 1.pool.ntp.org 0.fr.pool.ntp.org
```

要验证配置:

```
$ timedatectl show-timesync --all
LinkNTPServers=
SystemNTPServers=
FallbackNTPServers=0.arch.pool.ntp.org 1.arch.pool.ntp.org 2.arch.pool.ntp.org
3.arch.pool.ntp.org
ServerName=0.arch.pool.ntp.org
ServerAddress=103.47.76.177
RootDistanceMaxUSec=5s
PollIntervalMinUSec=32s
PollIntervalMaxUSec=34min 8s
PollIntervalUSec=1min 4s
NTPMessage={ Leap=0, Version=4, Mode=4, Stratum=2, Precision=-21, RootDelay=177.398ms,
RootDispersion=142.196ms, Reference=C342F10A, OriginateTimestamp=Mon 2018-07-16
13:53:43 +08, ReceiveTimestamp=Mon 2018-07-16 13:53:43 +08, TransmitTimestamp=Mon
2018-07-16 13:53:43 +08, DestinationTimestamp=Mon 2018-07-16 13:53:43 +08, Ignored=no
PacketCount=1, Jitter=0 }
Frequency=22520548
```

另外，也可以使用 systemd-networkd *.network* 文件的 `NTP=` 选项或 DHCP 服务器（需在 `[DHCPv4]` 或 `[DHCPv6]` 部分启用 `UseNTP` 选项）指定 NTP 服务器。

NTP 服务器的选择会遵循以下规则:

- systemd-networkd.service(8) 中针对每个接口的配置，或者 DHCP 服务优先。
- `/etc/systemd/timesyncd.conf` 中定义的 NTP 服务器会在运行时被添加到针对每个接口的服务器列表中，守护进程会轮流连接这些服务器直到某个有应答。
- 如果上两步没有取到 NTP 服务器， 那么将使用 `FallbackNTP=` 中定义的服务器。

注意：

- 这个服务每次同步或每 60 秒会写入 `/var/lib/systemd/timesync/clock` 本地文件, 这个目录是硬编码的，不可以更换。
- 这一写入机制在只读的 root 分区或者尝试减少 SD 卡写入量时可能会造成问题。

- 写入次数可以通过类似 `PollIntervalMinSec=1d` 和 `SaveIntervalSec=infinity` 这样的配置项降低。

使用

启用并启动

要启用和启动它，执行：

```
# timedatectl set-ntp true
```

其他方法（如在使用 chroot 时）有启动/启用 `systemd-timesyncd.service`。

检查服务

同步过程可能会很慢，请耐心等一会。要检查服务状态，使用:

```
$ timedatectl status
               Local time: Thu 2015-07-09 18:21:33 CEST
           Universal time: Thu 2015-07-09 16:21:33 UTC
                 RTC time: Thu 2015-07-09 16:21:33
                Time zone: Europe/Amsterdam (CEST, +0200)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

查看详细信息

要查看更多的服务信息，请使用:

```
$ timedatectl timesync-status
       Server: 103.47.76.177 (0.arch.pool.ntp.org)
Poll interval: 2min 8s (min: 32s; max 34min 8s)
         Leap: normal
      Version: 4
      Stratum: 2
    Reference: C342F10A
    Precision: 1us (-21)
Root distance: 231.856ms (max: 5s)
       Offset: -19.428ms
        Delay: 36.717ms
       Jitter: 7.343ms
 Packet count: 2
    Frequency: +267.747ppm
```

查看非默认配置

要查看非默认配置选项和文件（即被修改的配置项），使用：

```
$ systemd-analyze cat-config systemd/timesyncd.conf --tldr
# /etc/systemd/timesyncd.conf
[Time]

# /etc/systemd/timesyncd.conf.d/local.conf
[Time]
NTP=0.nl.pool.ntp.org 1.nl.pool.ntp.org 2.nl.pool.ntp.org 3.nl.pool.ntp.org
RootDistanceMaxSec=0.1
PollIntervalMinSec=1d
PollIntervalMaxSec=4w
SaveIntervalSec=infinity
```

查看日志

要查看过去 24 小时的事件记录，使用：

```
# journalctl -u systemd-timesyncd --no-hostname --since "1 day ago"
Jan 19 15:14:20 systemd: Stopping Network Time Synchronization...
Jan 19 15:14:20 systemd: systemd-timesyncd.service: Deactivated successfully.
Jan 19 15:14:20 systemd: Stopped Network Time Synchronization.
Jan 19 15:14:20 systemd: Starting Network Time Synchronization...
Jan 19 15:14:20 systemd: Started Network Time Synchronization.
Jan 19 15:14:20 systemd-timesyncd: Contacted time server 178.215.228.24:123
(0.nl.pool.ntp.org).
Jan 19 15:14:20 systemd-timesyncd: Initial clock synchronization to Fri 2024-01-19
15:14:20.393865 CET.
```

参阅

- 论坛：systemd-timesyncd 没有保持时间同步
- 论坛：使用 systemd-timesync 替代 NTP
- timesyncd 源码
- systemd-timesyncd(8)
- timesyncd.conf(5)

23.2 Chrony

难度 进阶 | **篇幅** 11.7 KB | **所属** NTP

本节将讲到

- 安装
- 配置
- 用法
- 网络状态通知
- 参考

chrony 是一个带有客户端和服务端功能的网络时间协议（NTP）实现，专为不常在线的系统环境设计，还可以跟踪硬件时钟（RTC）漂移。更多信息请参考项目的功能清单页面。

安装

安装 chrony包。

配置

软件包自带了一个被注释掉的 `/etc/chrony.conf` 配置文件。最简可用配置（使用了 IP 而不是域名）如下：

```
/etc/chrony.conf
server 1.2.3.4 offline
server 5.6.7.8 offline
server 9.10.11.12 offline
driftfile /var/lib/chrony/drift
rtconutc

rtcsync
```

`rtcsync` 选项会配置 RTC，但不跟踪 RTC 偏移。可选替代为 `rtcfile`，具体信息请参考下方的#实时时钟一节。

另请参考 项目 FAQ 以及 chronyc(1)，chrony.conf(5) 和 chronyd(8) 手册。

NTP 服务器

在 `/etc/chrony.conf` 中定义的第一项就是设备要连接到的服务器地址。 NTP 服务器处于一分层结构中，其中的每一层被称作 *strata*：被认定为独立时钟源的设备作为 *stratum 0* 时间源；直接连接到 *stratum 0* 的设备为 *stratum 1* 时间源；连接到 *stratum 1* 的为 *stratum 2* 时间源，以此类推。

但请注意，服务器的层级并不能作为其准确度或可靠性的依据。通常来说，stratum 2 服务器被用于时间同步；如果你还不知道要连到哪个服务器，那建议使用 pool.ntp.org 的服务器（备用链接），然后根据你的物理位置选择服务器池。

下一行告知 chrony 从 NTP 池中选择 4 个源（chrony 使用其特定方式处理服务器池，以防止混淆其服务端测偏移跟踪），然后在启动时使用 burst 特性：

```
pool 2.arch.pool.ntp.org iburst maxsources 4
```

离线设备

如果你的设备在启动时不会连到网络，那建议使用其 *offline* 选项。它会让 Chrony 在没有被告知的情况下不要尝试连接到服务器。

```
pool 2.arch.pool.ntp.org iburst maxsources 4 offline
```

使用 NTS 服务器

从 4.0 版本开始，chrony 支持使用 Network Time Security (NTS) 网络时间安全机制，这是 NTP 的安全加密变种。如需使用该功能，请添加一个启用 NTS 的服务器，然后在结尾添加 `nts`，例如：

```
server time.cloudflare.com iburst nts
```

可以在这里找到一些支持 NTS 的服务器。

另外，可以通过 `/etc/chrony.conf` 中的 `authselectmode` 选项配置 NTS 策略。

实时时钟

在系统启动时，会从硬件实时时钟（RTC）读取初始时间，并用其设定系统时间，然后在 chrony 守护进程启动后的几分钟内进行一次同步。如果硬件时间脱离同步，那初始系统时间将可能会和真实时间相差数分钟。Chrony.conf 有三种不同机制来处理 RTC：

- 第一种是 `rtcsync`，它会简单地定期将当前时间写入到 RTC。这也是 ntpd 选用的方式，但会关闭 RTC 偏移跟踪：这对于间断运行的桌面环境很不利，该环境下会依赖 RTC 进行大量计时工作。
- 第二种是 `rtcautotrim`，它会在 RTC 时间偏移超过一定量后进行复写。该方式可与 `rtcfile` 搭配，它会启用 RTC 误差跟踪。
- 最后一种是不操控 RTC，但会将其误差和偏移记录到 `rtcfile`。RTC 时间会一直有误差，但系统时间会保持正常，因为 chrony 记录了其具体的偏移量。chronyc 下的 `rtctrim` *命令*还是可以按需同步 RTC：

```
# chronyc
chronyc> trimrtc
200 OK
chronyc> quit
```

注意：`rtcsync` 和 `rtcfile` 不能同时使用。另外 `rtcfile` 会阻止如 `hwclock` 和 `timedatectl` 之类的工具访问 RTC，详细信息请参考 chrony.conf(5) § System clock。

另外，`rtconutc` 描述了 RTC 是否以 UTC 时间运行。

案例：间断运行的桌面环境

**本文或本章节的事实准确性存在争议。**

原因：该示例需要与系统时间#自动同步一致或进行修改。（在 Talk:Chrony#For intermittently running desktops 2 中讨论）

间断运行的桌面环境将需要使用 `rtcfile` 来跟踪 RTC 误差。装有 Arch Linux 连续运行 5 年的设备上 RTC 可累计有 300 秒的误差，如果使用以上的配置，在重启后 chrony 会需要很长一段时间来调整时间偏移。如果改用如下配置：

注意：默认不存在 `/etc/sysconfig` 文件夹，需要手动创建。

```
/etc/sysconfig/chronyd
OPTIONS='-r -s'
/etc/chrony.conf
dumpdir /var/lib/chrony
rtcfile /var/lib/chrony/rtc
```

有意思的是，RTC 还是会有误差，但在每次重启之后，chrony 都会调整 RTC 的累计误差，就算系统刚刚启动，其时间还能和 NTP 接近同步。

RTC 仍留存误差的原因是我们忘了添加 `rtcautotrim` 一行到配置中，它会让 chrony 调整 RTC 时间。如果加上了这一行，那 RTC 和系统时间都将变正确。

其它有意思的选项

有用的选项：

- `makestep`：允许 chrony 单次大量调整时间，而不是频繁多次调整。可能会对运行中程序造成影响，但有助于修正大量偏移。`makestep 0.1 3` 可能会更

适用于经常离线的系统：只有开头三次调整会是阶跃性的，因此影响仅限于系统启动时段。

精确性：

- `server` 和 `pool`：`xleave` 和 `presend` 可能会在没有兼容性成本的情况下提升精确度。
- `hwtimestamp`：有些网卡会给它的包添加时间戳以测量网络栈的延迟。使用 `hwtimestamp *` 可用启用该功能，对于不支持的网卡将不起任何效果。
- `tempcomp`：跟踪软件时钟偏移（通常由主板晶振温度变化导致）和温度传感器间的关系，适用于要求极致精确度的环境。

用法

启动 chronyd

启用/启动 `chronyd.service` 和 `chrony-wait.service`。第一个服务会启动守护进程，第二个会让 systemd 向依赖时间同步状态的其它服务发送信号。

注意：

- `systemd-timesyncd.service` 会与 `chronyd` 产生冲突，所以如果你要启用 `chronyd`，需要先将其禁用。

告知 chronyd 网络连接已可用

如果你连接到了网络，执行：

```
# chronyc
chronyc> online
200 OK
chronyc> exit
```

你也可以使用 `activity` 选项来查看状态：

```
# chronyc activity
200 OK
3 sources online
0 sources offline
0 sources doing burst (return to online)
0 sources doing burst (return to offline)
0 sources with unknown address
```

Chrony 应该已经连接到配置的时间服务器了，并根据需要更新了你的时钟。使用如下命令告知 chrony 你已经断开网络：

```
# chronyc offline
200 OK

# chronyc activity
200 OK
0 sources online
3 sources offline
0 sources doing burst (return to online)
0 sources doing burst (return to offline)
```

```
0 sources with unknown address
```

在线/离线状态可由 networkmanager包 和 connman包 的调度服务自动处理，详情请参考下文。

检查已配置的 NTP 服务器

使用以下命令查看 chrony 正在使用的 NTP 服务器和其精度：

```
$ chronyc -N 'sources -a -v'

  .-- Source mode  '^' = server, '=' = peer, '#' = local clock.
 / .- Source state '*' = current best, '+' = combined, '-' = not combined,
| /             'x' = may be in error, '~' = too variable, '?' = unusable.
||                                                 .- xxxx [ yyyy ] +/- zzzz
||      Reachability register (octal) -.           |  xxxx = adjusted offset,
||      Log2(Polling interval) --.      |          |  yyyy = measured offset,
||                                \     |          |  zzzz = estimated error.
||                                 |    |           \<br>MS Name/IP address
Stratum Poll Reach LastRx Last
sample<br>===============================================================================
==<br>^+ ptbnts1.ptb.de                  1   6   377    50    -38us[  -13us] +/-
8723us<br>^* ptbnts2.ptb.de                  1   6   377    49  +2061ns[  +27us] +/-
7538us<br>^+ nts.ntp.se                      2   6   377    51   +594us[ +619us] +/-
15ms<br>^+ nts.sth1.ntp.se                 2   6   377    51   +655us[ +680us] +/-
15ms<br>^+ nts.sth2.ntp.se                 2   6   377    53   +991us[+1016us] +/-
15ms<br>^+ time.cloudflare.com             3   6   377    49  -1250us[-1250us] +/-
10ms<br>```

## 网络状态通知

如果你在 `chrony.conf` 中为你的池配置了 offline 选项，需要将网络状态变更告知 *chrony*。

你可以使用 *chronyc* 来通知 *chrony*，也可以使用你的网络配置管理器对应的调度服务。

### NetworkManager

*chronyd* 可以通过 [NetworkManager 调度脚本](../../06_软件与应用/红帽/NetworkManager.md#Network_services_with_NetworkManager_dispatcher)随网络连接状态进入在线/离线状态。创建一个 dispatcher 来使用上游分发的 NetworkManager 调度脚本：

cp /usr/share/doc/chrony/examples/chrony.nm-dispatcher.onoffline /etc/NetworkManager/dispatcher.d/20-chrony-onoffline.sh chmod 700 /etc/NetworkManager/dispatcher.d/20-chrony-onoffline.sh

你也可以选择安装 [networkmanager-dispatcher-chrony](https://aur.archlinux.org/packages/networkmanager-dispatcher-chrony/)AUR。

### netctl

从 AUR 安装 [netctl-dispatcher-chrony](https://aur.archlinux.org/packages/netctl-dispatcher-chrony/)AUR。这会为 netctl 添加一个钩子，随任意网络连接自动运行。

### dhcpcd

创建如下钩子：

/etc/dhcpcd.exit-hook

if $if_up; then chronyc online elif $if_down; then chronyc offline fi

详情可参考 [dhcpcd-run-hooks(8)](https://man.archlinux.org/man/dhcpcd-run-hooks.8)。
```

```
## 参考

- [System time](../../04_硬件与外设/主板和%20BIOS/系统时间.md) (for more information on computer timekeeping)
- [https://chrony-project.org/](https://chrony-project.org/)
- [https://www.ntp.org/](https://www.ntp.org/)
- [https://support.ntp.org/bin/view/Main/WebHome](https://support.ntp.org/bin/view/Main/WebHome)
- [https://www.ntppool.org/en/](https://www.ntppool.org/en/)
- [https://www.eecis.udel.edu/~mills/ntp/html/index.html](https://www.eecis.udel.edu/~mills/ntp/html/index.html)
- [https://www.akadia.com/services/ntp_synchronize.html](https://www.akadia.com/services/ntp_synchronize.html)
```

23.3 Network Time Protocol daemon

难度 进阶 | **篇幅** 14.1 KB | **所属** NTP

本节将讲到

- 安装
- 配置
- 使用
- 技巧
- 排错

Network Time Protocol （网络时间协议）是 GNU/Linux 系统通过互联网时间服务器同步系统软件时钟的最常见方法。设计时考虑到了各种网络延迟，通过公共网络同步时，误差可以降低到10毫秒以内；通过本地网络同步时，误差可以降低到 1 毫秒。

NTP 项目提供了一个名为简单 NTP 的参考实现。本文介绍如何设置和运行服务器和客户端 NTP 进程。

安装

安装 ntp包 软件包。如果不做任何配置， *ntpd* 默认工作于客户端模式。如果使用 Arch Linux 默认的配置，请跳转到 #使用。作为服务器的配置，请参阅 #NTP 服务器模式。

配置

主要的后台进程是 *ntpd*, 可以通过 `/etc/ntp.conf` 配置。详细信息可以参考手册 ntp.conf(5) 和相关的 `man {ntpd|ntp_auth|ntp_mon|ntp_acc|ntp_clock|ntp_misc}`.

连接到 NTP 服务器

NTP 服务器通过一个层级系统进行分类，不同的层级称为 *strata*；独立的时间源为 *stratum 0*；直接连接到 *stratum 0* 的设备为 *stratum 1*；直接连接到 *stratum 1* 的源为 *stratum 2*，以此类推。

服务器的 stratum 并不能完全等同于它的精度和可靠度。通常的时间同步都使用 stratum 2 服务器。通过pool.ntp.org 服务器或这个链接可以选择比较近的服务器池。

下面几行仅仅是例子：

```
/etc/ntp.conf
server 0.fr.pool.ntp.org iburst
server 1.fr.pool.ntp.org iburst
server 2.fr.pool.ntp.org iburst
server 3.fr.pool.ntp.org iburst
```

推荐使用`iburst`选项，如果第一次尝试无法建立连接，程序会发送一系列的包。`burst` 选项则总是发送一系列的包，即使第一次也是这样。如果没有明确的允许的话不要使用 *burst* 选项，有可能被封禁。

NTP 服务器模式

如果建立一个 NTP 服务器，你需要添加 *local clock* 作为一个服务器，这样，即便它失去网络连接，它也可以继续为网络提供服务;添加 *local clock* 作为一个 stratum 10 服务器 (使用 *fudge* 命令)这样它就只会在失去连接时使用本地时钟：

```
server 127.127.1.0
fudge  127.127.1.0 stratum 10
```

下一步，定义规则允许客户端连接你的服务(*localhost* 也被认为是一个客户端)。使用 *restrict* 命令；你应该在文件中已经有一行：

```
restrict default nomodify nopeer noquery
```

这限制了每个人做任何修改并阻止每个人请求你的时间服务器状态：`nomodify` 防止重新配置你的ntpd（使用*ntpq* 或 *ntpdc* ），`noquery` 防止从你的nptd（或是*ntpq* 和 *ntpdc*）获取状态数据。

你也能添加其它选项：

```
restrict default kod nomodify notrap nopeer noquery
```

注意：这会允许其他人查询你的时间服务器。你需要添加 `noserve` 来停止提供时间。

"restrict"选项的完整文档可以从 ntp.conf(5) 中查找到。详见 https://support.ntp.org/bin/view/Support/AccessRestrictions 。

你需要在这一行之后告诉 *ntpd* 什么可以访问你的服务器；如果你不是在配置一台 NTP 服务器的话，下面一行就足够了。

```
restrict 127.0.0.1
```

如果你想要强制DNS解析到IPv6域名，在IP地址或域名前写上 `-6`（`-4` 则强制使用IPv4域名），例如:

```
restrict -6 default kod nomodify notrap nopeer noquery
restrict -6 ::1    # ::1 is the IPv6 equivalent for 127.0.0.1
```

最后，指定drift文件（它能时刻监控你的时钟的时间漂移）和log文件的位置：

```
driftfile /var/lib/ntp/ntp.drift
logfile /var/log/ntp.log
```

一份基础的配置文件是这样的：

```
/etc/ntp.conf
server 0.pool.ntp.org iburst
server 1.pool.ntp.org iburst
server 2.pool.ntp.org iburst
server 3.pool.ntp.org iburst

restrict default kod nomodify notrap nopeer noquery
restrict -6 default kod nomodify notrap nopeer noquery

restrict 127.0.0.1
restrict -6 ::1

driftfile /var/lib/ntp/ntp.drift
logfile /var/log/ntp.log
```

注意：定义日志文件不是必须的，但是它对于反馈 *ntpd* 操作是有好处的。

使用

软件包默认包含客户端模式的配置，并且使用单独的用户和群组，启动时就会移除 root 权限。如果在终端中启动，请使用 `-u` 选项:

```
# ntpd -u ntp:ntp
```

systemd 服务默认使用 `-u` 选项和 `-g` 选项禁用一个阈值(*panic-gate*). 这样即使 ntp-server 的时间和系统时间的差异超过阈值，依然会同步时间。

警告：使用 panic-gate 的原因是某些后台任务或服务会引起时间跳跃. 如果系统的时间从来没有同步过，请考虑先禁用其他服务再进行同步。

两个服务都依赖系统网络状况，会在检测到网络连接时开始同步。

启动时启用 ntpd

启用 `ntpd.service` 服务.

注意：systemd 命令 *timedatectl* 仅可以控制 systemd-timesyncd, 用 root 执行 `timedatectl set-ntp 1` 会停止运行中的 `ntpd.service`.

用 *ntpq* 可以查看同步的状态：

```
$ ntpq -p
```

delay, offset 和 jitter 不应该为零，*ntpd* 同步的服务器前有星号，*ntpd* 可能等待很多分钟后才会进行同步，请等 17 分钟 (1024 秒).

每次启动同步一次

另一种方式是启用 `ntpdate.service` 服务，每次启动都同步一次(`-q`) 并且是 non-forking (`-n`), 进程不会在后台运行。如果是服务器或者很多天才会重启一次，不建议使用此方式。

如果需要把同步到的时间写入硬件时钟，请按照这里的说明修改服务并启动:

```
/etc/systemd/system/ntpdate.service.d/hwclock.conf
[Service]
ExecStart=/usr/bin/hwclock -w
```

技巧

有网络连接的时候启动ntpd

ntpd 可以由你的网络管理器启动, 所以ntp这个守护进程只有在计算机有网络连接的时候才会启动.

Netctl

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 英文页面风格不好。添加 `-u` 参数以及"使用"中提及的可选参数，或尽可能使用 systemctl 。（在 Talk:Network Time Protocol daemon# 中讨论）

给你的 netctl 配置文件添加如下这几行:

```
ExecUpPost='/usr/bin/ntpd || true'
ExecDownPre='killall ntpd || true'
```

NetworkManager

通过网络管理器的 dispatcher 脚本，可以同网络连接一起启动/终止*ntpd* 守护进程。安装networkmanager-dispatcher-ntpdAUR 预配置包 #启动时启用 ntpd 来让网络连接同步ntp启动/终止.

KDE

在KDE中，通过右击时间图标选择 *Adjust date/time*，可以使用 NTP (别忘了安装 NTP)。 注意, 在配置KDE中配置NTP之前，先要把 ntp 守护进程设置为 disabled 状态.

在GPS中使用NTP

大多数联网的交通工具要通过(共享内存) 方式让 *ntpd* 从GPS中接收时间。 但是, 从 *ntpd* 4.2.8版本开始, 一个*更好*的办法出现了----直接与 *gpsd*守护进程交互。这个要先安装 gpsd包 .

在 `/etc/ntp.conf` 配置文件中添加下面这几行:

```
/etc/ntp.conf
#=============================================================
#  GPSD native ntpd driver
#=============================================================
# This driver exists from at least ntp version 4.2.8
# Details at
#   https://www.eecis.udel.edu/~mills/ntp/html/drivers/driver46.html
server 127.127.46.0
```

```
fudge 127.127.46.0 time1 0.0 time2 0.0 refid GPS
```

只要 *gpsd* 一直起着，ntp就会一直正常运行。 ntp会通过一个本地套接字同 *gpsd* 连接, 搜索 *gpsd* 返回的 "gpsd_json" 对象.

要检验安装的话, 首先要检查 *gpsd* 是不是正常运行:

```
 $ cgps -s
```

然后等个几分钟运行 `ntpq -p`. 这个会显示 *ntpd* 是否已经跟 *gpsd* 建立连接了:

```
$ ntpq -p
remote           refid            st t when poll reach   delay   offset  jitter
 ==============================================================================
*GPSD_JSON(0)    .GPS.            0 l   55   64  377    0.000    2.556  14.109
```

提示：如果 *reach* 这列是0的话, 这就说明 *ntpd* 还没有同 *gpsd* 建立连接。 等几分钟再试一次。 有时 *ntpd* 要花点时间来跟*gpsd* 建立连接。

在 chroot 底下运行

注意：在试图把 *ntpd* 放到chroot目录下面之前，*ntpd* 应该在正常路径下启动 (默认在 Arch Linux 安装路径下)。 因为在相对以root身份运行的进程，chroots的用户相对没有啥权限的。

创建一个新目录（如果还没创建的话） `/etc/systemd/system/ntpd.service.d/`，并添加在里面添加文件 `customexec.conf` ,内容如下:

```
[Service]
ExecStart=
ExecStart=/usr/bin/ntpd -g -i /var/lib/ntp -u ntp:ntp -p /run/ntpd.pid
```

然后, 编辑 `/etc/ntp.conf` 来改变 driftfile 路径来让跟它跟 chroot 路径保持一致, 而不是还用原先的普通路径。修改

```
driftfile       /var/lib/ntp/ntp.drift
```

为

```
driftfile       /ntp.drift
```

通过root创建永久目录和文件，作为一个合适的 chroot 环境来支持 getaddrinfo() :

```
# mkdir /var/lib/ntp/etc /var/lib/ntp/lib /var/lib/ntp/proc
# touch /var/lib/ntp/etc/resolv.conf /var/lib/ntp/etc/services
```

然后通过修改 *fstab* 绑定前面提到的文件:

```
/etc/fstab
...
#ntpd chroot mounts
/etc/resolv.conf  /var/lib/ntp/etc/resolv.conf none bind 0 0
/etc/services   /var/lib/ntp/etc/services none bind 0 0
/lib            /var/lib/ntp/lib none bind 0 0
/proc           /var/lib/ntp/proc none bind 0 0
# mount -a
```

最后, 再一次重启 `ntpd` 监控进程。 一旦重启完成你就可以通过检查 `/proc/{PID}/root` 的软连接来判断监控进程是不是在 chrooted 下面工作:

```
# ps -C ntpd | awk '{print $1}' | sed 1d | while read -r PID; do ls -l /proc/$PID/root;
done
```

正常应该链接到 `/var/lib/ntp` 而不是 `/`.

不用等一段时间就要确定 driftfile 配置是否工作正常是比较困难的, 因为 *ntpd* 不会经常读写。 如果配置错误的话，log里面会打印一个error; 如果配置正确的话, 时间戳会更新的。 可以通过等一天来判断，如果没有log中没有错误打印，并且log的时间戳还更新了，那么说明配置成功了。

排错

无法分配请求地址

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 英文章不正。对需要或必须使用 IPv6 的用户而言，此方案不可行。（在 Talk:Network Time Protocol daemon# 中讨论）

如果收到下列*无法分配请求地址*的报错信息：

```
$ journalctl -u ntpd
ntpd: bind(21) AF_INET6 fe80::6ef0:49ff:fe51:4946%2#123 flags 0x11 failed: Cannot assign requested address
ntpd: unable to create socket on eth0 (5) for fe80::6ef0:49ff:fe51:4946%2#123
ntpd: failed to init interface for address fe80::6ef0:49ff:fe51:4946%2
```

可以禁用 IPv6 解决。做法是：编辑 `ntpd.service` 添加 `-4` 参数：

```
[Service]
ExecStart=
ExecStart=/usr/bin/ntpd -g -u ntp:ntp -4
```

参见

- https://www.ntp.org/
- https://support.ntp.org/
- https://www.pool.ntp.org/
- http://www.eecis.udel.edu/~mills/ntp/html/index.html
- https://www.akadia.com/services/ntp_synchronize.html

WebDAV

WebDAV 把远端目录当成文件系统来读写，但服务端、认证、客户端三处任一环节没通都用不成。建议先读《WebDAV》，它篇幅最长，服务器、客户端、认证与排错都在这里，是必读的主干；再读《Davfs2》，讲把 WebDAV 资源挂载到本地，并附提示与常见问题，日常最实用，也建议读；《AgenDAV》只讲安装，最短，可以先跳，需要时再翻。

最容易踩的坑是把挂载成功当成配置完成：davfs2 只是客户端，认证或权限不通时，挂载会失败或只落到只读，此时该查的是服务端那一侧，而不是继续折腾挂载参数。

24.1　AgenDAV

难度 入门 | **篇幅** 1.8 KB | **所属** WebDAV

本节将讲到

- 安装

AgenDAV 是一个用 PHP 编写的开源多语言 CalDAV Web 应用程序，具有丰富的 AJAX 界面和共享日历支持。

安装

安装 agendavAUR 包。

数据库

您必须向 AgenDAV 提供一个 SQL 数据库。 这是一个 PostgreSQL 示例。

根据 PostgreSQL 文章安装 PostgreSQL。创建一个 `agendav` 用户和数据库：

```
# createuser agendav
# createdb -O agendav agendav
```

配置

设置数据库后，必须手动填充它：

```
# psql -U agendav agendav < /usr/share/webapps/agendav/sql/pgsql.schema.sql
# bash /usr/share/webapps/agendav/bin/agendavcli dbupdate
```

确保在 `php.ini` 中启用 `extension=pgsql`（或使用的任何数据库）和 `extension=iconv` 扩展。

根据您的喜好编辑配置文件 `/etc/webapps/agendav/{config,caldav,database}.php`。

通过 apache：`/etc/webapps/agendav/apache.example.conf`，nginx/php-fpm：`/etc/webapps/agendav/nginx.example.conf` 或其他一些 Web 服务器来服务该应用程序。

24.2 Davfs2

难度 进阶 | **篇幅** 6.1 KB | **所属** WebDAV

本节将讲到

- 安装 davfs2
- 挂载 WebDAV 资源
- 提示与技巧
- 常见问题
- 参见

翻译状态：

- 本文（或部分内容）译自 Davfs2，最近一次同步于 2026-03-09，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

davfs2 是一个用于挂载 WebDAV 资源的 Linux 文件系统。WebDAV 是 HTTP/1.1 的一个扩展，用于远程协作共享网络文件资源。

安装 davfs2

安装软件包 davfs2包。

挂载 WebDAV 资源

配置和挂载参数

配置文件分为系统级的 `/etc/davfs2/davfs2.conf` 和用户级的 `~/.davfs2/davfs2.conf`。普通用户调用时，会同时读取两个配置文件，但用户配置的优先级高于系统配置。可以对常规、WebDAV 相关、缓存相关、调试等内容进行配置，具体语法请参见 davfs2.conf(5)。

挂载参数可以用于定义配置文件路径、文件系统的所有者和用户组，以及其他访问权限相关的选项。要查询所有可用的参数，执行以下命令：

```
$ mount.davfs -h
```

也可参见 mount.davfs(8) 了解详细。

使用命令行

要挂载 WebDAV 资源，请使用 `mount`，而非直接使用 `mount.davfs`。

```
# mount -t davfs http(s)://address:<port>/path /mount/point
```

注意：只有在系统级配置文件中定义的 `dav_group` 用户组内的普通成员才能挂载 davfs2 文件系统。确保系统级配置文件中的 `dav_group` 已经被启用并被指定了可用的用户组。

使用 systemd

要使用Systemd 挂载：

```
/etc/systemd/system/mnt-webdav-service.mount
[Unit]
Description=Mount WebDAV Service
After=network-online.target
Wants=network-online.target

[Mount]
What=http(s)://address:<port>/path
Where=/mnt/webdav/service
Options=uid=1000,file_mode=0664,dir_mode=2775,grpid
Type=davfs
TimeoutSec=15

[Install]
WantedBy=multi-user.target
```

也可以创建一个 systemd 自动挂载单元来设置超时时间。

```
/etc/systemd/system/mnt-webdav-service.automount
[Unit]
Description=Mount WebDAV Service

[Automount]
Where=/mnt/webdav/service
TimeoutIdleSec=300

[Install]
WantedBy=remote-fs.target
```

要了解更多关于 systemd 挂载单元的内容，参见 Fstab#通过_systemd_自动挂载。

使用 fstab

要定义如何将 WebDAV 资源挂载在文件系统上，请添加一个以下格式的 fstab 条目：

```
/etc/fstab
https://webdav.example/path /mnt/webdav davfs rw,user,uid=username,noauto 0 0
```

其中，*username* 是所挂载的文件系统的所有者，可以是数字 ID 或是用户名。仅有 *root* 用户能以与挂载用户不同的 UID 进行挂载。挂载参数 `_netdev` 可以用于自动挂载网络驱动器。

提示与技巧

保存认证信息

创建一个密钥文件以保存 WebDAV 服务的认证信息。对于普通用户，使用 `~/.davfs2/secrets`；对于 *root* 用户，使用 `/etc/davfs2/secrets`：

```
/etc/davfs2/secrets
https://webdav.example/path davusername davpassword
```

请确保密钥文件的权限正确。对于 *root* 用户：

```
# chmod 600 /etc/davfs2/secrets
# chown root:root /etc/davfs2/secrets
```

对于普通用户：

```
$ chmod 600 ~/.davfs2/secrets
```

常见问题

无法创建/复制文件，同时可能卡住

如果无法创建/复制文件，同时可能卡住，请编辑#配置和挂载参数，将 `use_locks 0` 加入到配置中。默认配置 `1` 会在打开和写入文件时在服务器上加锁。

密码和密钥文件

如果密码中含有如 `\` 或 `"` 这样的特殊字符，请为其添加转义符 `\`。

网络已连通但域名解析失败

有时 `network-online.target` 已经成功但 WebDAV 服务器的域名仍未被解析。可以手动等待域名解析成功。

首先，配置好#使用 fstab，然后在 15 秒重复使用 ping 测试是否能访问服务器。访问成功时，再进行挂载：

```
/etc/systemd/system/mnt-webdav-service.service
[Unit]
Description=Mount WebDAV Service
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=bash -c 'for i in {1..15}; do if ping -c 1 mywebdav.server.url; then
mount /path/to/mountpoint; break; else sleep 1; fi; done'
ExecStop=umount /path/to/mountpoint
RemainAfterExit=true

[Install]
WantedBy=default.target
```

建议将其配置为用户单元。

挂载 WebDAV 资源时报错 'different mount options in /etc/fstab'

可能是以下原因导致：

1. WebDAV 资源及其挂载点已经在 /etc/fstab 中定义
2. 挂载点已经被作为一个参数传递给 'mount' 命令（即使挂载点与 /etc/fstab 中相同）

因此，请不要在挂载时传递挂载点，而是自动使用 /etc/fstab 中的值。例如：

```
$ mount -t davfs https://mywebdav mymountpoint # 错误
与 /etc/fstab 中的挂载参数不同
$ mount -t davfs https://mywebdav # 正确
```

参见

- https://ajclarkson.co.uk/blog/auto-mount-webdav-raspberry-pi/\[失效链接 2025-03-15 ⓘ\]

24.3 WebDAV

难度 进阶 | **篇幅** 10.8 KB | **所属** WebDAV

本节将讲到

- 服务器
- Client
- Authentication
- Troubleshooting

WebDAV （*Web Distributed Authoring and Versioning*）是 HTTP/1.1 的扩展，因此可以作为一个协议考虑。它包含了一系列概念和由此产生的扩展方法，用以允许通过 HTTP/1.1 读写。WebDAV 不使用 NFS 或 SMB，而是通过HTTP传输文件。

本指南目的是通过 web 服务器配置简单的WebDAV。

服务器

Apache

下载 Apache HTTP 服务器。

取消 DAV 和 auth_digest 这两个模块的注释：

```
LoadModule dav_module modules/mod_dav.so
LoadModule dav_fs_module modules/mod_dav_fs.so
LoadModule dav_lock_module modules/mod_dav_lock.so
LoadModule auth_digest_module modules/mod_auth_digest.so
```

取消注释 `conf/extra/httpd-dav.conf` 文件中的行：

```
# Distributed authoring and versioning (WebDAV)
Include conf/extra/httpd-dav.conf
```

检查文件 `/etc/httpd/conf/extra/httpd-dav.conf` 中的行：

```
DAVLockDB /etc/httpd/var/DavLock
```

确保你把上面的行放在了其它指令的外面，比如在 `DocumentRoot` 定义中：

如果你希望进行干净的部署，可以考虑使用 `/srv/dav` 结构来替代默认的 `/etc/httpd/uploads`。（与原文有出入，请对照英文页面阅读）

接下来，检查 `/etc/httpd/conf/extra/httpd-dav.conf` 中别名相关的设置（同样要放在其它指令的外面）：

```
DavLockDB "/etc/httpd/var/DavLock"

Alias /uploads "/etc/httpd/uploads"
```

```
<Directory "/etc/httpd/uploads">
    Dav On

    AuthType Digest
    AuthName DAV-upload
    # You can use the htdigest program to create the password database:
    #   htdigest -c "/etc/httpd/user.passwd" DAV-upload admin
    AuthUserFile "/etc/httpd/user.passwd"
    AuthDigestProvider file

    # Allow universal read-access, but writes are restricted
    # to the admin user.
    <RequireAny>
        # require that these methods are used (PROPFIND allows directory listing) ...
        Require method GET POST OPTIONS PROPFIND
        # or that the user is admin (f.e. PUT is required to write a file, MKCOL for
folders)
        Require user admin

        # -- Notes ---
        # more info on methods in the webdav rfc:
http://www.webdav.org/specs/rfc4918.html
        # POST treated as PUT: https://datatracker.ietf.org/doc/html/rfc5995

    </RequireAny>
</Directory>
```

创建目录：

```
# mkdir -p /etc/httpd/var
```

检查 DavLockDB 目录的权限并确保Web服务器用户 `http` 有权写入此目录：

```
# chown -R http:http /etc/httpd/var
# mkdir -p /etc/httpd/uploads
# chown -R http:http /etc/httpd/uploads
```

nginx

安装 nginx-mainline包 （nginx的Mainline版本）和 nginx-mainline-mod-dav-extAUR。

在 `/etc/nginx/nginx.conf` 文件的头部以及其它块的外面添加下面的行：

```
load_module /usr/lib/nginx/modules/ngx_http_dav_ext_module.so;
```

在 `server` 块中为WebDAV添加新的 `location`， 例如：

```
location /dav {
    root   /srv/http;

    dav_methods PUT DELETE MKCOL COPY MOVE;
    dav_ext_methods PROPFIND OPTIONS;

    # Adjust as desired:
    dav_access user:rw group:rw all:r;
    client_max_body_size 0;
    create_full_put_path on;
    client_body_temp_path /srv/client-temp;
    autoindex on;

    allow 192.168.178.0/24;
    deny all;
}
```

上面的例子需要 `/srv/http/dav` 目录和 `/srv/client-temp` 目录存在。

你也许希望通过使用绑定挂载来让其它目录也能通过WebDAV访问。

rclone

安装 rclone包 。它支持使用 webdav 导出远程或本地目录。

在不使用任何认证的情况下提供 `/srv/http` 目录下的内容：

```
$ rclone serve webdav /srv/http
```

Caddy

安装 caddy-webdav-gitAUR 或者先安装 xcaddy-binAUR 再使用 WebDAV 模块构建 Caddy ：

```
$ xcaddy build --with github.com/mholt/caddy-webdav
```

若要使用80端口以 `dav` 路径提供 `/srv/webdav` 目录下的内容，将下面的内容添加到 Caddyfile ：

```
:80 {
    rewrite /dav /dav/
    webdav /dav/* {
       root /srv/webdav
       prefix /dav
    }
    file_server
}
```

运行 Caddy ：

```
$ caddy run
```

Client

Cadaver

Install the cadaver包 package.

After installation, test the WebDAV server:

```
$ cadaver http://localhost/dav
dav:/dav/> mkcol test
Creating `test': succeeded.
dav:/dav/> ls
Listing collection `/dav/': succeeded.
Coll: test
```

Dolphin

To create a permanent WebDAV folder in Dolphin select *Network* in the *remotes* section of the places sidebar, then press the *Add Network Folder* button. The network folder wizard will appear. Select *WebFolder (webdav)*, and fill in the subsequent form.

Alternately just click the path bar and then enter the url with *webdav://* protocol specifier.

Nautilus

Install the gvfs包 and gvfs-dnssd包 packages.

In Nautilus choose "connect to server" and enter the address with `dav://` or `davs://` protocol specified:

```
dav://127.0.0.1/dav
```

注意：If you get a "HTTP Error: Moved permanently" with `dav://`, try to use `davs://` as the protocol instead.

rclone

rclone包 is a command line tool that lets you sync to/from, or mount (with many caching options), remote file systems including WebDAV.

Thunar

Install the gvfs包 and gvfs-dnssd包 packages.

In Thunar press `Ctrl+l` and enter the address with *dav* or *davs* protocol specified:

```
davs://webdav.yandex.ru
```

Authentication

There are numerous different protocols you can use:

- plain
- digest
- others

Apache

Using htdigest(1) (remove the `-c` option if the file exists):

```
# htdigest -c /etc/httpd/conf/passwd WebDAV username
```

注意：Make sure digest authentication is enabled in `httpd.conf` by the presence of this entry: `LoadModule auth_digest_module modules/mod_auth_digest.so`

Using plain htpasswd(1) (remove the `-c` option if the file exists):

```
# htpasswd -c /etc/httpd/conf/passwd username
```

Next, `httpd.conf` must be edited to enable authentication. One method would be to require the user `foo` for everything:

```
<Directory "/home/httpd/html/dav">
  DAV On
  AllowOverride None
  Options Indexes FollowSymLinks
  AuthType Digest # substitute "Basic" for "Digest" if you used htpasswd above
  AuthName "WebDAV"
  AuthUserFile /etc/httpd/conf/passwd
  Require user foo
</Directory>
```

注意：`AuthName` must match the realm name passed when using the `htdigest` command for digest authentication. For basic/plain authentication, this line may be removed. Also, make sure that the `AuthUserFile` path matches that used with the `htdigest` or `htpasswd` commands above.

If you want to permit everybody to read, you could use this in your httpd.conf

```
<Directory "/home/httpd/html/dav">
  DAV On
```

```
  AllowOverride None
  Options Indexes FollowSymLinks
  AuthType Digest # substitute "Basic" for "Digest" if you used htpasswd above
  AuthName "WebDAV"
  AuthUserFile /etc/httpd/conf/passwd
  Require all granted
  <LimitExcept GET HEAD OPTIONS PROPFIND>
    Require user foo
  </LimitExcept>
</Directory>
```

Do not forget to restart `httpd.service` after making changes.

注意：If you get an 405 error with Apache, add `DirectoryIndex disabled` to your `Directory` section.

Troubleshooting

Some file explorers cannot edit directories in nginx WebDAV

nginx WebDAV requires a directory path ends with a slash (`/`), but some file explorers does not append a `/` at the end of the path.

This can be worked-around, by either removing the corresponding checking code and recompile it, or by appending the following code in a nginx `server` block to add `/` at the end of a request, if needed:

```
# The configuration was based on: https://nworm.icu/post/nginx-webdav-dolphin-deken/
# if the request method is MKCOL or is to a directory, add / at the end of the request
if it was missing
if ($request_method = MKCOL) {
    rewrite ^(.*[^/])$ $1/ break;
}
if (-d $request_filename) {
    rewrite ^(.*[^/])$ $1/ break;
}

# if the request method is copy or move a directory, add / at the end of the request if
it was missing
set $is_copy_or_move 0;
set $is_dir 0;
if (-d $request_filename) {
    set $is_dir 1;
}
if ($request_method = COPY) {
    set $is_copy_or_move 1;
}
if ($request_method = MOVE) {
    set $is_copy_or_move 1;
}
set $is_rewrite "${is_dir}${is_copy_or_move}";
if ($is_rewrite = 11) {
    rewrite ^(.*[^/])$ $1/ break;
}
```

第 25 章　共 3 节　11 KB

下载工具

这一章解决的是命令行取文件该用哪个程序、怎么把它调到顺手。CURL、Wget、Bookget 都能下载，用法与配置却各不相同。

三节按分量递增排：CURL 最厚，从安装、用法铺到提示与技巧与配置文件，是必读主干，建议先读；Wget 结构相近而多一节配置，安装与用法扫过即可，重点放在配置；Bookget 只有三小节、篇幅最短，要用时再翻，不必通读。

最容易踩的坑是把三者当成可互换的同义词，拿别处的写法来试——配置与用法并不通用；另一误解是跳过安装直接抄用法，跑不通便分不清是环境还是写法问题。

25.1　Bookget

难度 入门 | **篇幅** 1.8 KB | **所属** 下载工具

本节将讲到

- 安装
- 用法
- 参见

bookget 是一款数字古籍图书下载工具，已支持约 50+ 个数字图书馆。

注意：本文及此项目代码仅供学习研究使用。

安装

AUR

有以下三个 AUR 包可选：

- bookgetAUR：从 Github 发布获取源代码。
- bookget-gitAUR：通过 Git 获取源代码并构建。
- bookget-binAUR：从 Github 获取已经编译好的二进制可执行文件，仅支持 x86_64 架构（因为官方仅构建了 amd64 版本）。

手动安装

使用官方提供的安装方法，执行：

```
$ wget -O bookget https://github.com/deweizhu/bookget/releases/download/版本号/bookget-linux
$ chmod +x bookget
# cp bookget /usr/local/bin/
```

例如：对于版本 25.0601，可以执行：

```
$ wget -O bookget
https://github.com/deweizhu/bookget/releases/download/v25.0601/bookget-linux
```

```
$ chmod +x bookget
# cp bookget /usr/local/bin/
```

或者，使用源代码编译安装（需要 golang 开发环境）：

```
git clone https://github.com/deweizhu/bookget.git
cd bookget

# 本地开发时可直接运行
make linux-amd64    # 编译Linux版本
make windows-amd64  # 编译Windows版本
make release        # 编译所有平台
```

用法

参见 Wiki。

要通过链接下载一本古籍：

```
$ bookget
Enter an URL:
-> 古籍的 URL
```

或执行：

```
$ bookget "古籍 URL"
```

参见

- Github 仓库
- 书格 *bookget 专区*

25.2 CURL

难度 入门 | **篇幅** 4.3 KB | **所属** 下载工具

本节将讲到

- 安装
- 用法
- 提示与技巧
- 配置文件
- 参见

cURL 是一个命令行工具和库，用于使用 URL 传输数据。该命令支持多种不同的协议，包括 HTTP、HTTPS、FTP、SCP 和 SFTP。它还可以在没有用户交互的情况下工作，比如在脚本中。

注意：尽管表面上等同于 wget，但事实并非如此。请参阅 Can_I_do_recursive_fetches_with、运行与给定 wget 命令对应的 curl 命令 和 What_is_curl_not。

安装

提示：一般情况下，cURL 已作为 pacman包 等软件包的依赖而被安装。

安装 curl包 软件包。

用法

下载

cURL 的常见用例是将资源下载到指定文件：

```
$ curl --output 文件名 URL
```

如果 URL 包含文件名，则可以直接将资源保存为该文件名：

```
$ curl --remote-name URL
```

同样，你也可以使用 `-J/--remote-header-name` 来接受来自 HTTP 服务器（来自 `Content-Disposition` 头信息）的文件名提示。当与 `-O/--remote-name` 结合使用时，如果 HTTP 服务器没有返回文件名提示，curl 将使用 URL 指定的文件名。

您也可以省略输出选项，将资源打印到 stdout：

```
$ curl URL
```

HTTP POST

您可以使用 cURL 发送 HTTP POST 请求：

```
$ curl --data 'request body' URL
```

如果命令行无法容纳 request body，cURL 可以从文件中读取 request body：

```
$ curl --data @文件名 URL
```

有时，您可能需要为 `Content-Type` 头指定一个自定义值（cURL 的默认值为 `application/x-www-form-urlencoded`）。您可以使用 `-H` 来实现这一功能。例如，如果您想发送一个带有 JSON body 的 POST request，可以使用：

```
$ curl --data 'json body' -H 'Content-Type: application/json' URL
```

请注意，curl 也有一个选项，可以以 json 格式写入 post 数据，并自动更改 header：`--json`:

```
$ curl --json '{"key":"value"}' URL
```

提示与技巧

跟踪重定向

跟踪重定向（如 HTTP 到 HTTPS 重定向）：

```
$ curl --location URL
```

显示下载错误

默认情况下，curl 会忽略错误（例如，在下载文件时，如果出现错误，curl 不会通知你，创建的文件将是空的），因此使用 `--fail` 可以让它显示错误信息：

```
$ curl --fail URL
```

压缩

如果你想压缩传输数据，（例如，在带宽比CPU更有限的情况下，curl 会下载压缩数据，然后在下载后解压缩）：

```
$ curl --compressed URL
```

进度条

当下载文件时，curl 可以选择普通的进度条（例如 `[##### ] 80%`）。

```
$ curl --progress-bar URL
```

通配

也可以在 curl 中使用通配：

```
$ curl "example.com/images/[1-9].png"
$ curl "example.com/{first_page,second_page,third_page}"
```

配置文件

curl 也会在主目录和 `$XDG_CONFIG_HOME` 中搜索名为 `.curlrc` 的配置文件。默认情况下，你只需将想要使用的命令行参数放在 curl 中，例如：

```
$HOME/.curlrc
# this is a comment, the next line would be the option for progressbar:
-#
# to make curl always compress:
--compressed
# or just
compressed
```

参见

- Wikipedia:cURL
- Everything curl - cURL 使用指南详解
- curl(1)

25.3 Wget

难度 入门 | **篇幅** 4.8 KB | **所属** 下载工具

本节将讲到

- 安装
- 配置
- 用法

GNU Wget 是一款自由软件，用于使用HTTP、HTTPS、FTP和 FTPS（*FTPS 的支持自 1.18 版起*）来检索文件。它是一个非交互式命令行工具，因此从脚本中调用它很容易。

安装

wget 1.x

安装 wget包 软件包。git 版本存在于 AUR ，其名称为 wget-gitAUR。

wget2

wget2 是命令行语法与之几乎完全相同的另一个软件，它"是 GNU Wget 的后继者"，"从零开始设计和编写"。这个重写版本"可以多线程运行，并可用许多功能来实现快速操作。由于采用了 HTTP2、HTTP 压缩、并行连接和使用 If-Modified-Since HTTP 请求头技术，在许多情况下，Wget2 的下载速度比 Wget1.x 快得多"。

要使用 wget2，请从 AUR 安装 wget2AUR。需要注意的是，这需要对 pandoc 进行 makedepend，依赖度较高；您也可以安装 AUR中的 wget2-no-docsAUR 来避免依赖。

配置

可在 `/etc/wgetrc` 中配置。默认配置文件不仅文档详尽，而且很少需要修改。更多复杂选项，请参见 wget(1) § OPTIONS。

FTP 自动化

通常，SSH 用于在网络之间安全地传输文件。不过，相比于通过 SSH 运行的 scp 和 rsync，FTP 占用的资源更少。FTP 并不安全，但当在受防火墙保护的环境中，在 CPU 紧张的系统上传输大量数据时，使用 FTP 可以被证明是有好处的。

```
wget ftp://root:somepassword@10.13.X.Y//ifs/home/test/big/"*.tar"
```

```
3,562,035,200 74.4M/s   in 47s
```

在本例中，Wget 以每秒 74.4MB 的速度传输了一个 3.3GiB 的文件。

简而言之，该程序

- 可编写脚本
- 比 ssh 更快
- 可替代字符串变量的语言易于使用
- globbing)capable

代理

Wget 使用标准代理环境变量。请参阅代理设置。

使用代理验证功能

```
$ wget --proxy-user "DOMAIN\USER" --proxy-password "PASSWORD" URL
```

不包括使用 HTML 验证表单的代理。

pacman 集成

要让 pacman 自动使用 Wget 和带有身份验证的代理，请将 Wget 命令写入 `/etc/pacman.conf` 中的 `[options]` 部分：

```
XferCommand = /usr/bin/wget --proxy-user "domain\user" --proxy-password="password" --passive-ftp --quiet --show-progress --continue --output-document=%o %u
```

警告：请注意，以纯文本存储密码并不安全。可使用 `chmod 600 /etc/pacman.conf` 确保只有 root 才能读取该文件。

用法

本节将解释 Wget 的一些使用场景。

基础用法

Wget 最基本、最常见的用途之一就是从互联网上下载文件。

```
$ wget <url>
```

当你已经知道要下载文件的 URL 时，这比通常在浏览器上下载文件并手动将其移动到正确的目录要快得多。不言而喻的是，你可找到一些方法来利用其最简单的用法进行自动下载，如果你想要。

完整网站存档

Wget 可以将绝对链接改为相对链接，从而在存档完整网站的同时保留正确的链接目的地。

```
$ wget --recursive --no-parent --convert-links 'target-url-here'
```

如果是动态网站，还可以使用一些其他选项将其转换为静态 HTML。

```
$ wget --recursive --no-parent --page-requisites --adjust-extension --convert-links --backup-converted 'target-url-here'
```

wget 还提供了绕过下载阻止机制的选项。

```
$ wget --recursive --no-parent --convert-links --random-wait --execute robots=off --user-agent "Mozilla/5.0" 'target-url-here'
```

And if third-party content is to be included in the download, `-H/--span-hosts` switch can be used alongside `-r/--recursive` to recurse to linked hosts.

搜索

要找的可能是文件、文件内容，也可能是网上的资料。本章由窄到宽排开：Locate 最轻，装完即用，专查文件名，必读且应先读；Baloo 面向桌面，范围扩到内容乃至可移动与远程设备，另有命令行用法与禁用索引器的办法，用 KDE 的必读，纯服务器可跳；Surfraw 是终端的网络搜索入口，需求明确再翻，日常可跳。 最容易的误解是把三者混为一谈：Locate 查的是索引而非实时文件系统，结果缺项先怀疑索引过期；Baloo 启用后即持续索引，不用时应当显式禁用索引器；Surfraw 只是前端，装完还需配置才能用。

26.1　Surfraw

难度 入门 | **篇幅** 2.6 KB | **所属** 搜索

本节将讲到

- 安装
- 配置
- 用法

Surfraw 为各种流行的 WWW 搜索引擎提供了一个快速的 UNIX 命令行界面。Surfraw 最初是由 Julian Assange 创建的。

安装

安装 surfraw包 软件包。

配置

**这篇文章或章节的翻译质量不佳。**

原因：译者未充分理解部分内容，如果您的翻译更佳，可以参与贡献。（在 Talk:Surfraw#配置 中讨论）

Surfraw 使用默认浏览器打开成功的查询。如果没有安装任何标准浏览器，Surfraw 将调用 $BROWSER。如果该变量为空，您将收到一条错误信息，因为 Surfraw 无法打开查询。您可以通过 `~/.config/surfraw/conf` 配置浏览器和其他选项：

```
SURFRAW_graphical_browser=/usr/bin/chromium
#SURFRAW_text_browser=/usr/bin/elinks
SURFRAW_graphical=yes
```

默认配置文件安装在 `/etc/xdg/surfraw/conf` 中，其中包含所有可配置选项。

用法

Surfraw 由一组称为 elvi 的 shell 脚本组成，每个脚本都能搜索特定网站。

要查看 elvi 列表，请键入：

```
$ surfraw -elvi
```

您可以使用 surfraw 的全称，也可以使用 surfraw 的简称：

```
$ sr duckduckgo topic_name
```

你也可以将 `/usr/lib/surfraw` 添加到 `$PATH` 中，以直接调用 elvi。

有 100 多种 elvi 可供在网上搜索，如亚马逊网站：

```
$ surfraw amazon -search=books -country=en -q Stanislaw Lem
```

要搜索 AUR:

```
$ sr aur package_name
```

要搜索 ArchWiki:

```
$ sr archwiki article_name
```

有关网站搜索脚本的完整列表，请参见 Elvi 列表

26.2 Baloo

难度 入门 | **篇幅** 4.1 KB | **所属** 搜索

本节将讲到

- 安装
- 用法与配置
- 命令行用法
- 索引可移动设备或远程设备
- 禁用索引器

Baloo 是 KDE Plasma 的文件索引与搜索框架。

安装

安装 baloo包 软件包。

用法与配置

为了在 Plasma 桌面上使用 Baloo 进行搜索，启动 krunner （默认快捷键 `ALT+F2`）并键入查询。若要在 Dophin（文件管理器）内搜索，按`CTRL+F`。

在默认情况下，桌面搜索的 KCM 仅显示两个选项：一个将文件夹放入黑名单的面板以及一种一次点击来禁用它的方法。

或者您可以编辑 `~/.config/baloofilerc` 文件。另外您也可以使用 `balooctl` 进程。运行 `balooctl stop` 和 `balooctl disable`。

将文件夹添加到黑名单或完全禁用了Baloo之后，`baloo_file_cleaner` 进程将会自动删除所有不需要的索引文件。这些文件存储于 `~/.local/share/baloo/` 目录下。

命令行用法

```
$ baloosearch query
```
Support for range queries:

```
$ baloosearch "width>=6000 width<7000"
```
Groups and OR/AND operators:

```
$ baloosearch "tag:coolpicture OR (width>=6000 width<7000)"
```
For a list of all supported filter properties available, see (info).

Note that text search index breaks down all metadata (including filenames) into "words", and does all its searches over it from the beginning of the word only. What this means is that while `baloosearch "tutorial"` would match `my_great_tutorial.txt`, `baloosearch "utorial"` would not match it. To see what "words" baloo stored in the index for a particular file do:

```
$ balooshow -x filename
```

索引可移动设备或远程设备

默认情况下，所有可移动设备都在黑名单内，只需在 KCM 面板中将其移除即可。

禁用索引器

To disable the Baloo file indexer:

```
$ balooctl suspend
$ balooctl disable
```
The indexer will be disabled on next login.

Alternatively, disable *Enable File Search* in *System settings* under *Search* > *File search*.

To permanently delete the index database, run:

```
$ balooctl purge
```
This will also resolve the following error message in file dialogs and other applications (KDE bug 437176):

```
kf.kio.core: "Could not enter folder tags:/."
```

疑难解答

Inotify folder watch limit error

If you get the following error:

```
KDE Baloo Filewatch service reached the inotify folder watch limit. File changes may be ignored.
```
Then you will need to increase the inotify folder watch limit:

```
# echo 524288 > /proc/sys/fs/inotify/max_user_watches
```

To make changes permanent, create a sysctl configuration file:

```
/etc/sysctl.d/40-max-user-watches.conf
fs.inotify.max_user_watches=524288
```

Plasma Vault Files are indexed and available even when vault is closed

This is a major security bug not yet fixed. Any file inside vault is by default indexed and available through file manager search, Krunner and Kickoff.

One workaround is to **stop** folder(s) from being indexed by Baloo. The relevant options are available in *System Settings* > *Search* > *File Search* > *Folder specific configuration* > *Add folder configuration* > *Stop indexing a folder*. After adding the desired folder, the existing Baloo data needs to be removed and freshly indexed again:

```
$ balooctl disable
$ balooctl purge
$ balooctl enable
$ balooctl check
```

26.3 Locate

难度 入门 | **篇幅** 4.3 KB | **所属** 搜索

本节将讲到

- 安装
- 用法
- 问题解决
- 参见

翻译状态：

- 本文（或部分内容）译自 locate，最近一次同步于 2025-03-02，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的**原始**修订历史。
- 本文可能与英文原文存在出入。

locate（即*定位*）是一种常用的 Unix 工具，用于通过文件名快速查找文件。与 find 工具相比，*locate* 通过搜索预先构建的数据库文件，而不是直接搜索文件系统，提高了查找速度。这种方法的缺点是，`locate` 无法检测到数据库文件生成后的更改。通过按计划更新数据库，可以将这一问题降至最低。

随着时间的推移，各种替代实现经历了迭代更替：从仅显示用户可访问文件的 slocate（secure locate，安全定位），到每次更新时合并数据库的 mlocate（merging locate，合并定位）——这种合并机制通过跳过已检查文件实现了性能提升，再到如今基于倒排列表构建的 plocate（posting locate，列表定位），它通过预先处理数据库生成速度更快、体积更小的索引。

安装

安装 plocate包 软件包。它提供了 `plocate`、`updatedb`、`plocate-build` 等命令以及指向 `plocate` 的符号链接 `locate`、`mlocate`。

虽然 GNU findutils 也包含 *locate* 实现，但 Arch 的 findutils包 软件包并不包含。

LS-Shandong 维护了一个 plocate 分叉，这个分叉包含了 Steinar 正考虑合并入官方 plocate 的 i18n 支持及软件的简体中文翻译和手册页翻译。安装 plocate-enhanced-gitAUR 软件包以获取它。

用法

在使用 plocate(1) 之前，需要先创建数据库，这可以通过 updatedb(8) 命令完成，顾名思义，该命令用于更新数据库。

plocate包 包含一个 `plocate-updatedb.timer` 单元，每天都会调用数据库更新，安装时已启用。如果想在重启前使用，请手动启动启动。您也可以随时以根用户身份手动运行 *updatedb*。

为节省时间，（默认情况下）可以通过编辑 `/etc/updatedb.conf` 配置 *updatedb* 忽略某些文件系统和路径。updatedb.conf(5) 介绍了该文件的语义。值得注意的是，默认配置（`PRUNEPATHS`）中忽略的路径包括 `/media` 和 `/mnt`，因此 *locate* 可能无法发现外部设备上的文件。

问题解决

Btrfs

默认配置会阻止结果中包含 Btrfs 文件系统。要允许包含 btrfs 挂载点，请添加

```
/etc/updatedb.conf
PRUNE_BIND_MOUNTS = "no"
```

当然，这也意味着其他绑定挂载点也会被包含在内。如果需要排除这些挂载点，可以使用同一配置文件中的 `PRUNEPATHS` 设置。

参见

- How locate works and rewrite it in one minute
- plocate 官方网站

DHCP

主机开机就要拿到地址、网关和域名服务器，逐台手写既麻烦又容易冲突，本章就是把这件事自动化。Dhcpd 篇幅短，安装、配置、用法一条线讲完，必读，读完即可拉起一份最小可用配置；Dnsmasq 篇幅长，多出提示和技巧一节，不必顺读，配置够用就先动手，技巧按场景挑看，赶时间可跳过，遇问题再翻。

最容易踩的坑是让两套服务在同一子网同时发地址，客户端认到哪台全凭运气，表现为时好时坏的断网；也别把 Dnsmasq 当成只会发地址的纯 DHCP 服务，它还兼管名字解析，只盯着地址池，常常地址发对了名字仍然不通。

27.1　Dhcpd

难度 入门 | **篇幅** 3.9 KB | **所属** DHCP

本节将讲到

- 安装
- 配置
- 用法
- 参阅

dhcpd 是老版本的 ISC DHCP 服务器。

注意：

- *dhcpd* 不再受维护。Internet Systems Consortium（ISC，互联网系统联盟）推荐 Kea 作为其官方替代品。
- *dhcpd* (DHCP **(服务器)** 守护进程) 不是 dhcpcd (DHCP **客户端** 守护进程).

安装

安装 dhcp包 包。

配置

为你想让 *dhcpd* 监听的接口分配静态 IPv4 地址（此处为 `eth0`）。指定的子网不应与其他接口的子网重叠：

```
# ip link set up dev eth0
# ip addr add 139.96.30.100/24 dev eth0 # arbitrary address
```

提示：

- 要在启动时分配静态 IP 地址，参见网络配置#静态 IP 地址。
- RFC 1918 子网是为私有网络保留的，并不会与互联网上的主机冲突。

默认配置文件 `dhcpd.conf` 文件包含许多注释的例子，复制一份该文件：

```
# mv /etc/dhcpd.conf /etc/dhcpd.conf.example
```

欲仅监听子网 `139.96.30.0/24`，可创建最简配置文件如下：

```
/etc/dhcpd.conf
option domain-name-servers 8.8.8.8, 8.8.4.4;
option subnet-mask 255.255.255.0;
option routers 139.96.30.100;
subnet 139.96.30.0 netmask 255.255.255.0 {
  range 139.96.30.150 139.96.30.250;
}
```

配置文件使用的选项如下：

`domain-name-servers` 包含提供给客户的 DNS 服务器地址。此处以谷歌公共 DNS 服务器为例。如果你已经将你自己的 DNS 服务器配置在一台本地机器上，则应在子网中的指定其地址（如 `139.96.30.0/24`）。

`subnet-mask` 和 `routers` 定义子网掩码和子网上可用路由器的列表；`routers` 也定义了提供给客户端的默认网关。对于小型网络，您往往能把 `255.255.255.0` 用作掩码，并为运行 *dhcpd* 的机器指定 IP 地址（此处为 `139.96.30.100`）。

`subnet` 为各子网单独设置选项，作用于 *dhcpd* 所监听的网络接口。此处，我们定义一段可用 IP 地址用作一个子网 `139.96.30.0/24`（在单个接口 `eth0`）。

完整选项列表，参见 dhcpd.conf(5)。

注意：尽管 dhcpd(8) 给出了相反的说法，但是接口必须含有在配置文件中声明的子网，才会被 *dhcpd* 监听。

PXE

PXE（预启动执行环境）的配置通过以下两个选项完成：

```
/etc/dhcpd.conf
next-server 192.168.0.2;
filename "/pxelinux.0";
```

`next-server` 是 TFTP 服务器的 IP 地址，而 `filename` 是用于启动的映像。

该段配置既可置于整个 `subnet` 中，亦可仅置于某个 `host` 定义中。

用法

dhcpd 包括两个单元文件：`dhcpd4.service` 与 `dhcpd6.service`，可用于控制守护进程。它们会分别为 IPv4 和 IPv6 在所有 网络接口 上启动守护进程。

参阅

- ISC DHCP 文档

27.2 Dnsmasq

难度 进阶 | **篇幅** 15.2 KB | **所属** DHCP

本节将讲到

- 安装
- 配置
- 提示和技巧
- 参阅

dnsmasq 提供 DNS 服务器、支持 DHCPv6 和 PXE 的 DHCP 服务器、TFTP 服务器。它设计为轻量且占用空间小，适用于资源受限的路由器和防火墙。还可以将 dnsmasq 配置为 DNS 缓存查询，以提高对以前访问过站点的 DNS 查找速度。

安装

安装 dnsmasq包 软件包，然后启动/启用 `dnsmasq.service`。

还需要重新启动网络，以便 DHCP 客户端可以创建新的 `/etc/resolv.conf`。

配置

要配置 dnsmasq，需要编辑 `/etc/dnsmasq.conf`。该文件包含选项的注释。有关全部可用选项，请参阅 dnsmasq(8)。

注意：dnsmasq 的默认配置启用 DNS 服务器。如果不使用的话，需要显式设置 `port=0` 来禁用它。

如果 dnsmasq 不用作本地 DNS 解析程序，还需要 编辑 `dnsmasq.service`，使其不唤起 `nss-lookup.target`：

```
/etc/systemd/system/dnsmasq.service.d/no-nss-lookup-target.conf
[Unit]
Wants=
```

提示：要检查配置文件语法，请执行：

```
$ dnsmasq --test
```

DNS 服务器

要在单台计算机上将 dnsmasq 设置为 DNS 缓存守护程序，请指定 `listen-address` 指令，添加本地主机 IP 地址：

```
listen-address=::1,127.0.0.1
```

使用此计算机在其 LAN IP 地址上侦听网络上的其他计算机，建议使用静态 LAN IP。例如：

```
listen-address=::1,127.0.0.1,192.168.1.1
```

或者，可以指定网络接口：

```
interface=enp5s0
```

使用 `cache-size=*size*` 设置缓存域名的数量（默认值为 `150`）：

```
cache-size=10000
```

要验证 DNSSEC，请加载 dnsmasq包 软件包提供的 DNSSEC 信任锚，并设置选项 `dnssec`：

```
conf-file=/usr/share/dnsmasq/trust-anchors.conf
dnssec
```

要了解更多选项，请参阅 dnsmasq(8)。

DNS 地址文件和转发

配置 dnsmasq 后，需要将本地主机地址添加为 `/etc/resolv.conf` 中的唯一名称服务器。这会导致所有查询都发送到 dnsmasq。

由于 dnsmasq 是存根解析器而不是递归解析器，因此必须设置转发到外部 DNS 服务器。可以通过 openresolv 自动完成或在 dnsmasq 的配置中手动指定 DNS 服务器完成。

openresolv

如果网络管理器支持 *resolvconf* 而不是直接更改 `/etc/resolv.conf`，可以使用 openresolv 生成 dnsmasq 的配置文件。

编辑 `/etc/resolvconf.conf` 并将 loopback 地址添加为名称服务器，然后配置 openresolv 输出 dnsmasq 配置：

```
/etc/resolvconf.conf
# 使用本地名称服务器
name_servers="::1 127.0.0.1"
resolv_conf_options="trust-ad"

# 输出 dnsmasq 扩展配置和解析文件
dnsmasq_conf=/etc/dnsmasq-conf.conf
dnsmasq_resolv=/etc/dnsmasq-resolv.conf
```

运行 `resolvconf -u` 创建配置文件。如果文件不存在，则 `dnsmasq.service` 无法启动。

编辑 dnsmasq 的配置文件使用 openresolv 生成的配置：

```
# 读取 openresolv 生成的配置文件
conf-file=/etc/dnsmasq-conf.conf
resolv-file=/etc/dnsmasq-resolv.conf
```

手动转发

首先，必须将本地主机地址设置为 `/etc/resolv.conf` 中的唯一名称服务器：

```
/etc/resolv.conf
nameserver ::1
nameserver 127.0.0.1
options trust-ad
```

确保 `/etc/resolv.conf` 不被修改，详述见 Domain name resolution#Overwriting of /etc/resolv.conf。

或者，可以配置 NetworkManager 自动生成特定连接的 `/etc/resolv.conf` 文件，使用以下命令：

```
$ nmcli connection modify 'connection-name' ipv4.dns 127.0.0.1
$ nmcli connection modify 'connection-name' ipv4.dns-options trust-ad
$ nmcli connection modify 'connection-name' ipv4.ignore-auto-dns yes
$ nmcli connection modify 'connection-name' ipv6.dns ::1
$ nmcli connection modify 'connection-name' ipv6.dns-options trust-ad
$ nmcli connection modify 'connection-name' ipv6.ignore-auto-dns yes
```

然后重启 `NetworkManager.service`。

然后，必须在 dnsmasq 的配置文件中设置 `server=*server_address*` 指定上游 DNS 服务器地址。还要添加 `no-resolv`，以便 dnsmasq 不会非必要地读取只包含本地主机地址的 `/etc/resolv.conf`。

```
/etc/dnsmasq.conf
[...]
no-resolv

# 示例：Google 的名称服务器
server=8.8.8.8
server=8.8.4.4
```

现在，DNS 查询将使用 dnsmasq 解析，仅在缓存查询无结果时才会从外部服务器查询。

添加自定域

可以通过以下方式为路由器分配域名：

```
address=/router/192.168.1.1
```

或者，继续为（本地）网络中的主机添加自定域：

```
local=/lan/
domain=lan
```

在此示例中，可以 ping （在 `/etc/hosts` 文件中定义的）主机/设备为 `*hostname*.lan`。

取消注释 `expand-hosts` 将自定域添加到主机条目：

```
expand-hosts
```

如果没有此设置，则必须将域添加到 `/etc/hosts` 的条目中。

测试

要执行查找速度测试，请选择自 dnsmasq 启动以来未访问过的网站（*drill* 是 ldns包软件包的一部分）：

```
$ drill archlinux.org | grep "Query time"
```

再次运行该命令将使用缓存的 DNS IP，如果正确设置了 dnsmasq，则查找时间会缩短：

```
$ drill archlinux.org | grep "Query time"
;; Query time: 18 msec
```

```
$ drill archlinux.org | grep "Query time"
;; Query time: 2 msec
```

若要测试 DNSSEC 验证是否正常工作，请参阅 DNSSEC#Testing。

DHCP 服务器

默认情况下，dnsmasq 关闭了 DHCP 功能，如要使用则必须将其打开。以下是重要的设置：

```
# 仅侦听路由器的 LAN NIC。这样会将 tcp/udp 端口 53 开放给本地主机，并将 udp 端口 67 开放给全世界：
interface=enp0s0

# dnsmasq 将向全世界开放 tcp/udp 端口 53 和 udp 端口 67，以帮助动态接口（分配动态 IP）。
# dnsmasq 将丢弃全部请求，但某些人可能希望关闭它并由内核处理。
# 如果您有其他 dnsmasq 实例在运行（例如由于 libvirtd），您可能也需要此选项。
bind-interfaces

# 设置域名（可选）
domain=example.org

# 设置默认网关
dhcp-option=3,0.0.0.0

# 设置要公布的 DNS 服务器
dhcp-option=6,0.0.0.0

# 如果 dnsmasq 服务器同时也为网络执行路由，则可以使用选项 121 推出静态路由。
# x.x.x.x 是目标 LAN，yy 是 CIDR 表示法（通常为 /24），z.z.z.z 是执行路由的主机。
dhcp-option=121,x.x.x.x/yy,z.z.z.z

# 提供给 LAN PC 的 IP 动态范围和租赁时间。
# 建议首先将租赁时间设置为 5m，以便测试一切正常之后再设置持久记录。
# 这里的地址范围必须位于分配给虚拟接口的地址范围内。
dhcp-range=192.168.111.50,192.168.111.100,12h

# 提供 IPv6 DHCP 租约，使用网络接口作为前缀构建范围
dhcp-range=::f,::ff,constructor:enp0s0

# 如果要让 dnsmasq 将固定 IP 分配给某些客户端，请绑定 LAN 计算机的 NIC MAC 地址：
dhcp-host=aa:bb:cc:dd:ee:ff,192.168.111.50
dhcp-host=aa:bb:cc:ff:dd:ee,192.168.111.51
```

更多选项请参阅 dnsmasq(8)。

Proxy DHCP

如果网络上已经运行了 DHCP 服务器并且您希望与其互操作，可以将 dnsmasq 设置为“代理 DHCP”，因此仅向客户端提供 #PXE 服务器特定信息。此模式仅适用于 IPv4。使用以下语法，提供现有 DHCP 服务器地址：

```
dhcp-range=192.168.0.1,proxy
```

测试

从连接到运行 dnsmasq 的计算机上，将其配置为使用 DHCP 自动分配 IP 地址，然后尝试正常登录网络。

如果您检查服务器上的 `/var/lib/misc/dnsmasq.leases` 文件，您应该能够看到租约。

TFTP 服务器

dnsmasq 内置了 TFTP 服务器。

要使用它，请为 TFTP 创建一个根目录（例如 `/srv/tftp`）以放置可传输文件。

```
enable-tftp
tftp-root=/srv/tftp
```

为了增加安全性，建议使用 dnsmasq 的 TFTP 安全模式。在安全模式下，只有 `dnsmasq` 用户拥有的文件才会通过 TFTP 提供。您需要 chown TFTP 根目录及其中的所有文件为 `dnsmasq` 用户才能使用此功能。

```
tftp-secure
```

有关更多选项，请参阅 dnsmasq(8)。

PXE 服务器

PXE 需要 DHCP 和 TFTP 服务器；两者都可以由 dnsmasq 提供。要设置 PXE 服务器，请按照以下步骤操作：

1. 在 dnsmasq 配置文件中设置 #TFTP 服务器和 #DHCP 服务器（完整 DHCP 或代理模式），
2. 复制并配置 PXE 兼容的引导加载程序（例如 PXELINUX）到 TFTP 根目录，
3. 在 dnsmasq 配置文件中启用 PXE：

要简单发送一个文件：

```
dhcp-boot=lpxelinux.0
```

要根据客户端架构发送文件：

```
pxe-service=x86PC,"PXELINUX (BIOS)",bios/lpxelinux
pxe-service=X86-64_EFI,"PXELINUX (EFI)",efi64/syslinux.efi
```

注意：

- 文件路径相对于 TFTP 根路径
- 如果文件有 *.0* 后缀，您必须在 `pxe-service` 选项中排除后缀

如果 `pxe-service` 无法识别架构（特别是对于基于 UEFI 的客户端），可以使用 `dhcp-match` 和 `dhcp-boot` 的组合。有关更多 `client-arch` 编号，请参阅 RFC 4578 2.1 以用于 dhcp 引导协议。

```
dhcp-match=set:efi-x86_64,option:client-arch,7
dhcp-match=set:efi-x86_64,option:client-arch,9
dhcp-match=set:efi-x86,option:client-arch,6
dhcp-match=set:bios,option:client-arch,0
dhcp-boot=tag:efi-x86_64,efi64/syslinux.efi
dhcp-boot=tag:efi-x86,efi32/syslinux.efi
dhcp-boot=tag:bios,bios/lpxelinux.0
```

有关更多选项，请参阅 dnsmasq(8)。

其余部分取决于引导加载程序。

提示和技巧

防止 OpenDNS 重定向 Google 查询

要防止 OpenDNS 将所有 Google 查询重定向到其自己的搜索服务器，请添加到 `/etc/dnsmasq.conf`：

```
server=/www.google.com/<ISP DNS IP>
```

覆盖地址

在某些情况下，例如操作强制门户时，将特定域名解析为硬编码的地址集可能很有用。这可以通过 `address` 配置完成：

```
address=/example.com/1.2.3.4
```

此外，可以通过使用特殊通配符为所有未从 `/etc/hosts` 或 DHCP 回答的域名返回特定地址：

```
address=/#/1.2.3.4
```

多个实例

如果我们希望每个接口运行两个或更多 dnsmasq 服务器。

静态

要静态执行此操作，每个接口使用一个服务器，请使用 `interface` 和 `bind-interfaces` 选项。这将强制启动第二个 dnsmasq。

动态

在这种情况下，我们可以排除每个接口并绑定任何其他接口：

```
except-interface=lo
bind-dynamic
```

注意：这是 libvirt 中的默认设置。

域名阻止列表

要阻止域名，即使用 NXDOMAIN 回答查询，请使用 `address` 选项而不指定 IP 地址：

```
address=/blocked.example/
address=/anotherblocked.example/
```

注意：与 `/etc/hosts` 文件不同，dnsmasq 将阻止这些域名及其所有子域名，例如 *subdomain.blocked.example*。

还支持通配符。在模式的开头添加 `*`：

```
# 阻止 blocked.example 和 anotherblocked.example 及其所有子域名
```

```
address=/*blocked.example/

# 阻止像 mail.google.com 这样的子域名，但不阻止 google.com
address=/*.google.com/
```

可以使用 # 作为服务器地址来解除阻止某些特定子域名：

```
# 阻止 google.com 及其所有子域名，除了 mail.google.com。
address=/google.com/
server=/mail.google.com/#
```

注意：

- 选项 address=/example.com/ 和 server=/example.com/ 是等效的。两者都将使用 NXDOMAIN 回答查询。
- 选项 address=/example.com/# 和 server=/example.com/# 不等效。
- address=/example.com/# 将使用 NULL 地址（IPv6 为 0.0.0.0 或 ::）回答域名的查询。
- server=/example.com/# 将域名的查询发送到标准配置的服务器。
- 模式 /example.com/ 和 /.example.com/ 是等效的。两者都将匹配 *example.com* 及其所有子域名。

为了方便使用，将阻止列表放在单独的文件中，例如 /etc/dnsmasq.d/blocklist.conf，并从 /etc/dnsmasq.conf 加载它，使用 conf-file=/etc/dnsmasq.d/blocklist.conf 或 conf-dir=/etc/dnsmasq.d/,*.conf。

提示：

- 可以在 OpenWrt 的 adblock 包的 README 中找到阻止列表的潜在来源列表。
- 可以使用 addn-hosts=hosts.txt 选项使用 hosts 文件阻止列表，或者可以使用此 awk 命令将其转换为 dnsmasq 阻止列表：awk '/^[^#]/ { print "address=/"$2"/"$1"" }' hosts.txt。

查看缓存统计信息

可以使用 chaos 请求查询缓存统计信息，使用 ldns包 软件包中的 drill 实用程序：

```
$ drill misses.bind TXT CH
$ drill hits.bind TXT CH
```

输出将分别包含缓存未命中和命中的数量：

```
;; ANSWER SECTION:
misses.bind.    0       CH      TXT     "411"
```

其他选项包括 cachesize.bind、insertions.bind、evictions.bind、auth.bind 和 servers.bind。

参阅

- 使用 dnsmasq 的缓存名称服务器，以及其他一些提示和技巧。

XMPP

本章解决在 Arch 上把 XMPP 跑通：服务端 Jabberd2，客户端 Pidgin。Jabberd2 一节很短，只有安装、配置与参见，够搭出最小服务端骨架；Pidgin 一节是主体，安装后是拼写检查、修复声音、浏览器错误、QIP Encoding bug 与 ICQ，多是使用时才暴露的毛病。

必读两段：Jabberd2 的配置、Pidgin 的安装，其余按症状查；已自建服务端或只用现成账号的，可跳过 Jabberd2 整节。最易踩的坑是把 Pidgin 一节条目当成必做步骤、逐条改配置——它们各对应一种症状，没出现就别动。

28.1　Jabberd2

难度 入门 | **篇幅** 1.7 KB | **所属** XMPP

本节将讲到

- 安装
- 配置
- 参见

jabberd2 是一个 XMPP 服务器，用 C 语言编写，并根据 GNU 通用公共许可作为自由软件许可。它的灵感来自 jabberd14。

安装

安装 jabberd2AUR 包。

配置

编辑 `/etc/jabberd/c2s.xml` 并将标记 `<id register-enable='mu'>` 的内容更改为您的域。

这行将添加您的用户 ID。（如果您将 `example.com` 放在此处，则您的用户 ID 将类似于 `user@example.com`）如果将通过开放的 Internet（而不是 VPN 或 LAN）访问 jabber 服务，则该名称**应由** DNS 解析为您的服务器。

`register-enable='mu'` 部分允许使用标准的 jabber 客户端注册帐户。

还要在 `sm.xml` 上设置服务器：

```
/etc/jabberd/sm.xml
<id>mymachine.com</id>
```

守护进程

配置 `jabberd.service` 以在开机时启动。

阅读守护程序以获取更多信息。

参见

- Jabberd2 主页

28.2 Pidgin

难度 进阶 | **篇幅** 18.1 KB | **所属** XMPP

本节将讲到

- 安装
- 拼写检查
- 修复声音
- 浏览器错误
- QIP Encoding bug

Pidgin （以前的Gaim）是Linux平台上的一个即时信息客户端，它可以连接到许多流行的即时信息网络，比如Live Messenger, Yahoo, IRC, AIM等等。Pidgin的一个关键特性是它可以同时连接许多即时信息网络。

安装

官方软件仓库提供了 pidgin包 的安装包。 Notable variants are:

- **Pidgin Light** — 不包含 GStreamer, Tcl/Tk, XScreenSaver, video/voice 支持的 Light Pidgin 版本。

: https://pidgin.im/ || pidgin-lightAUR

你可以从purple-plugin-pack包处安装其它的插件。

拼写检查

作为依赖Aspell会被默认安装，但为了避免你输入的所有文本都显示为“拼写错误”，你需要安装一个aspell字典（比如：aspell-en包）。执行 `pacman -Ss aspell` 列出可用的语言。 如果拼写检查没有工作，尝试单独运行 aspell 去检查安装是否正确，并且不要分开有用的错误信息。执行：

```
$ echo center | aspell -a
```

注意： **switch spell**插件包含在purple-plugin-pack中，它允许你在不同的语言间切换。

修复声音

为了修复声音需要安装gstreamer0.10-goodAUR包。或者，你可以在配置的声音选项卡中把方法（method）切换为‘命令’（'command'），然后使用下面的命令之一：

如果你使用的是ALSA:

```
aplay %s
```

如果使用的是OSS:

```
ossplay %s
```

浏览器错误

如果在Pidgin中点击网络链接会产生一个关于“试图使用'sensible-browser'打开链接”的错误，试试编辑`~/.purple/prefs.xml`。找到与'sensible-browser'相关的行改为以下的样子：

```
<pref name='command' type='path' value='firefox'/>
```

上面的例子假设你使用Firefox.

QIP Encoding bug

There is another bug in character encoding when communicating between Pidgin and QIP, which especially affects Czech language, but there are also other languages affected. There are two possible solutions. The better one is to upgrade from QIP to QIP Infimum, second solution is to install and enable plugin from pidgin-qip-decoderAUR\[损坏的链接：package not found\] package currently available from AUR.

ICQ

You can change encoding for ICQ account if encoding in Buddy Information is not correct:

```
Account > your ICQ account > Edit account > Advanced tab
```

Select `Encoding: CP1251` (for Cyrillic).

IRC

有一辅助功能用于链接 Libera.chat。它应该也可以用于链接其他 IRC 网络，如果你变更其端口和域名。

选择*账户* > *管理账户* > *添加*. 设置以下选项:

```
协议：IRC
用户名：your username
```

选择*好友* > *加入聊天* (或者按下 `Ctrl+m`), 在 'libera.chat' 文本框中选择 *username*@irc.libera.chat, 然后点击 '确定'，或者输入：

```
/join #你的聊天室名称
```

如果要注册你的账户，输入：

```
/msg nickserv register password email-addres
```

按照Email中指示的做，你也可以输入以下内容来获得帮助：

```
/msg nickserv help
/msg nickserv help command
```

最后一部是把你的聊天室加入 '好友'：选择*好友* > *加入聊天*，填入正确的聊天室 (#archlinux)。

Xfire

Simply install pidgin-gfire包\[损坏的链接：package not found\] and then add a new account, selecting xfire as protocol.

Web QQ

直接安装 pidgin-lwqq包\[损坏的链接：package not found\] 并创建账户，选择协议为 **webQQ** 。

已知问题：目前无法发送图片

Facebook XMPP

Since Facebook Chat supports XMPP, you can use Pidgin without extra plugins. See this article for more information: Facebook Chat Now Available Everywhere

注意：In order to utilise Facebook chat through XMPP and pidgin, you will require a Facebook "username". This is located in *Facebook* > *Account settings* > *username* (below real name)

1\. Go to "Accounts" and select "Manage Accounts."

2\. On the Basic tab, enter the following info:

: Protocol: XMPP : Username: *Your FacebookID* (without e-mail domain, e.g. @yahoo.com, etc) : Domain: chat.facebook.com (make sure you have not typed any extra space) : Resource: Pidgin (leave this empty if you get "username@chat.facebook.com/Pidgin Not Authorized" error message) : Password: *Your Password* : Local alias: *Your Name*

3\. Click the Advanced tab, then enter the following info:

: Connect port: 5222 : Connect server: chat.facebook.com (make sure you have not typed any extra space) : (Uncheck the box labeled "Require SSL/TLS")

注意：

- Newer versions of Pidgin do not have a "Require SSL/TLS" box. Instead, select "Use encryption if available" from the Connection Security dropdown in Advanced
- Changing your Facebook privacy settings to "Turn off all apps" (under Apps and Websites), will disable your ability to send messages via jabber (see Why cannot Pidgin seem to send Facebook messages).
- You may notice that all Facebook contacts are in a separate group every time you login with your XMPP account even though you moved them to other groups or created meta-contacts. If you want to be able to group contacts and create meta-contacts you can use the plugin pidgin-xmpp-ignore-groupsAUR (after installing the plugin activate the option *Ignore server-sent groups* on the Advanced tab in your XMPP account settings). It essentially ignores the group

> data sent by the server roaster and preserves your local changes. It is easier to enable this plugin on your account *before* logging in the first time, so your contacts are put into the default group instead of a group called "Facebook Friends". Enabling it afterward will not move the contacts out of this group.

Privacy

Pidgin has some privacy rules set by default. Namely, the whole world cannot send you messages; only your contacts or people selected from a list. Adjust this, and other settings through:

```
Tools > Privacy
```

Pidgin-OTR

This is a plugin that brings Off-The-Record (OTR) messaging to Pidgin. OTR is a cryptographic protocol that will encrypt your instant messages.

First you need to install pidgin-otr包 from the official repositories. Once this has been done, OTR has been added to Pidgin.

1. To enable OTR, start Pidgin and go to *Tools* > *Plugins* or press `Ctrl+u`. Scroll down to the entry entitled "Off-The-Record Messaging". If the checkbox beside it is not checked, check it.
2. Next, click on the plugin entry and select "Configure plugin" at the bottom. Select which account you wish to generate a key for, then click "Generate". You will have now generated a private key. If you are not sure what the other options do, leave them, the default options will work fine.
3. The next step is to contact a buddy who also has OTR installed. In the chat window, a new icon should appear to the top right of your text input box. Click on it, and select "Start private conversation". This will start an 'Unverified' session. Unverified sessions are encrypted, but not verified - that is, you have started a private conversation with someone using your buddy's account who has OTR, but who might not be your buddy. The steps for verification of a buddy are beyond the scope of this section; however, they might be added in the future.

Pidgin-Encryption

pidgin-encryptionAUR transparently encrypts your instant messages with RSA encryption. Easy-to-use, but very secure.

You can enable it the same way as Pidgin-OTR.

Now you can open conversation window and new icon should appear beside menu. Press it to enable or disable encryption. Also if you want to make encryption enabled by default right-click on a buddy's name (in your buddy list), and select Turn Auto-Encrypt On. Now, whenever a new conversation window for that buddy is opened, encryption will start out as enabled.

Pidgin-GPG

Pidgin-GPG transparently encrypt conversations using GPG, and taking advantage of all the features of a pre-existing WoT.

The plugin is available on AUR as pidgin-gpgAUR\[损坏的链接：package not found\]. It can be enabled the same way as the previously mentioned ones.

Sametime protocol

Sametime support is available by installing two packages from AUR:

- meanwhileAUR
- libpurple-meanwhileAUR

Previously it was required to rebuild Pidgin to remove the `--disable-meanwhile` flag from compilation, this is no longer needed. Once these two packages are installed the 'Sametime' protocol will be available when creating an account.

SIP/Simple protocol for Live Communications Server 2003/2005/2007

The pidgin-sipeAUR plugin is available in official repositories.

Other packages

Arch has other Pidgin-related packages. Here are the most popular (for a thorough list, search the AUR):

- pidgin-libnotify包 - Libnotify support, for theme-consistent notifications
- guifications包\[损坏的链接：package not found\] - Toaster-style popup notifications
- microblog-purpleAUR\[损坏的链接：package not found\] - Libpurple plug-in supporting microblog services like Twitter
- pidgin-latexAUR - A small latex plugin for pidgin. Put math between $$ and have it rendered (recepient also needs to have this installed)

Skype plugin

Install skype4pidgin-svnAUR\[损坏的链接：package not found\] from the AUR.

Auto logout on suspend

If you suspend your computer pidgin seems to stay connected for about 15 minutes. To prevent message loss, it is needed to set your status offline before suspending or hibernating. The status message will not be changed.

Therefore create a new systemd unit `pidgin-suspend` in `/etc/systemd/system` Take the following snippet and replace *myuser* with your user.

```
[Unit]
Description=Suspend Pidgin
```

```
Before=sleep.target
StopWhenUnneeded=yes

[Service]
Type=oneshot
User=myuser
RemainAfterExit=yes
Environment=DISPLAY=:0
ExecStart=-/usr/bin/purple-remote setstatus?status=offline
ExecStop=-/usr/bin/purple-remote setstatus?status=available

[Install]
WantedBy=sleep.target
```

Troubleshooting

- If Facebook XMPP verification does not work for you, there a package in the aur pidgin-facebookchat which does not require a unique user name (you may login with an email address)
- The facebookchat plugin will prompt for varification (enter these two words...), if that fails, hit cancel and log onto Facebook with pidgin open, this will configure the plugin's security setting)

Installing Pidgin after a Carrier installation

If you previously installed carrierAUR\[损坏的链接：package not found\] (aka FunPidgin), follow these steps *before* installing Pidgin:

- Quit Carrier
- Delete your `~/.purple` directory.

警告：This will remove all your user settings for any programs that use libpurple, i.e. Pidgin, Carrier, etc.

```
rm -r ~/.purple
```

- Uninstall **carrier** and **libpurple**.
- Install **pidgin** and **libpurple**.

History import Kopete to Pidgin

- Install xalan-c包 and create `~/bin/history_import_kopete2pidgin.sh` with this code:

```
#!/bin/sh

KOPETE_DIR=~/.kde4/share/apps/kopete/logs
PIDGIN_DIR=~/.purple/logs
CURRENT_DIR=~/bin

cd

if [ ! -d $KOPETE_DIR ];then
    echo "Kopete log directory not found"
    exit 1;
fi

if [ ! -d $PIDGIN_DIR ];then
    echo "Pidgin log directory not found"
    exit 2;
fi
```

```
for KOPETE_PROTODIR in $(ls $KOPETE_DIR); do
    PIDGIN_PROTODIR=$(echo $KOPETE_PROTODIR | sed 's/Protocol//' | tr [:upper:] 
[:lower:])
    for accnum in $(ls $KOPETE_DIR/$KOPETE_PROTODIR); do
        echo "Account number: $accnum"
        for num in $(ls $KOPETE_DIR/$KOPETE_PROTODIR/$accnum); do
            FILENAME=$(Xalan $KOPETE_DIR/$KOPETE_PROTODIR/$accnum/$num 
$CURRENT_DIR/history_import_kopete2pidgin_filename.xslt)
            if [ $? = 0 ]; then
                echo "$KOPETE_DIR/$KOPETE_PROTODIR/$accnum/$num"
                echo " -> $PIDGIN_DIR/$PIDGIN_PROTODIR/$FILENAME"
                mkdir -p $(dirname $PIDGIN_DIR/$PIDGIN_PROTODIR/$FILENAME)
                Xalan -o $PIDGIN_DIR/$PIDGIN_PROTODIR/$FILENAME $KOPETE_DIR/
$KOPETE_PROTODIR/$accnum/$num $CURRENT_DIR/history_import_kopete2pidgin.xslt
            fi
        done
    done
done
```

- Make `~/bin/history_import_kopete2pidgin.sh` executable:

```
chmod +x ~/bin/history_import_kopete2pidgin.sh
```

- Create `~/bin/history_import_kopete2pidgin.xslt` with this code:

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="text" indent="no" />

    <xsl:template match="kopete-history">
        <xsl:apply-templates select="msg"/>
    </xsl:template>

    <xsl:template match="msg">
        <xsl:text>(</xsl:text>
        <xsl:value-of select="translate(substring-after(@time,' '),':',',')"/>
        <xsl:text>) </xsl:text>
        <xsl:value-of select="@nick"/>
        <xsl:if test="not(@nick) or @nick = ">
            <xsl:value-of select="@from"/>
        </xsl:if>
        <xsl:text>: </xsl:text>
        <xsl:value-of select="."/>
              <xsl:text>
</xsl:text>
    </xsl:template>
</xsl:stylesheet>
</nowiki>
```

- Create `~/bin/history_import_kopete2pidgin_filename.xslt` with this code:

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="text" indent="no" />

    <xsl:template match="kopete-history">
        <xsl:value-of select="head/contact[@type = 'myself']/@contactId"/>
        <xsl:text>/</xsl:text>
        <xsl:value-of select="head/contact[not(@type)]/@contactId"/>
        <xsl:text>/</xsl:text>
        <xsl:value-of select="head/date/@year"/>
        <xsl:text>-</xsl:text>
        <xsl:if test="head/date/@month &lt; 10">0</xsl:if>
        <xsl:value-of select="head/date/@month"/>
        <xsl:text>-</xsl:text>
        <xsl:if test="string-length(substring-before(msg/@time,' ')) &lt; 2">0</xsl:if>
        <xsl:value-of select="translate(msg/@time,' :','.')"/>
        <xsl:text>+0200EET.txt</xsl:text>
    </xsl:template>
</xsl:stylesheet>
```

- Execute the command in the shell:

```
~/bin/history_import_kopete2pidgin.sh
```

See also

- Pidgin homepage
- History Import Kopete To Pidgin

互联网中继聊天

装好系统之后遇到疑难，或想确认某个包的状态，社区里最直接的求助去处是 IRC。本章解决“去哪里问”和“用什么待在那里”两件事：先说清官方频道与其它语言频道各自的分工，再把 Irssi 从安装到日常使用走一遍。

建议先看《Arch 的 IRC 频道》，这一节必读，它决定了你该进哪个频道；《Irssi》的安装与使用读完就足以应付日常，设置与技巧偏个人口味，需要时再回头翻。最容易踩的坑是只盯着官方频道，忘了还有按语言划分的频道，也可能在 Irssi 里一上来就照抄一堆配置，基本操作反而没跑通。

29.1　Arch 的 IRC 频道

难度 进阶 | **篇幅** 6.6 KB | **所属** 互联网中继聊天

本节将讲到

- 官方频道
- 其它语言的 IRC 频道

官方频道

这篇条目介绍 Arch Linux 的IRC 主要支持频道 **#archlinux** , 主要的交流频道 **#archlinux-offtopic**。（都在 Libera Chat 上）

#archlinux频道的主要话题是有关 Arch Linux 的支持和总体讨论。频道规则请查看 Code of conduct 和 General guidelines#IRC，常用的缩略语请参看Arch terminology 和 IRC Jargon。

你需要一个IRC客户端才能加入讨论频道。可用的IRC客户端列表请查看List of applications/Internet#IRC clients 和 Wikipedia:Comparison of Internet Relay Chat clients。安装环境中可以使用 irssi 。

用户名注册

#archlinux和**archlinux-offtopic**当前的频道模式为`+r`和`+q $~a`。这意味着你必须通过`NickServ`的身份验证才能加入个别的频道，发送消息。做这个是为了过滤垃圾信息的骚扰。

需要使用NickServ注册，请参照libera FAQ，当连接上了*irc.libera.chat*使用`NickServ help`：

```
/query nickserv help register
/query nickserv help identify
```

如果你并没有进行用户注册和验证，你会被转到**#archlinux-unregisterd**频道。用`msg chanserv access #archlinux list`指令能得到可以帮助你的人的列表，或者加入#liberachat频道，在那里提问。

注意：

- 如果`/query`在你的客户端不起作用，你可以试着用`/quote nickserv <command>`或`/msg nickserv <command>`。
- 有些客户端会在你通过NickServ验证之前就尝试自动加入频道， 你需要开启SASL来解决这个问题。查看你的IRC客户端文档或者在SASL page上寻找教程来实现它。

频道管理员

注意：Arch的管理员在**#archlinux**和**#archlinux-offtopic**都具有管理员身份。查看下面的列或者在liberachat上用`/msg phrik listops`指令。

需要管理员的协助? 别犹豫，直接 `/query` 或者 `/msg` 吧。截止到 2021-09-24 , 操作员有这些:

- alad
- BrainDamage
- gehidore / man
- grawity
- jelle
- MrElendig / Mion
- Namarrgon

其它频道

The size of our community led to the creation of multiple IRC channels. To get a list of all channels on **irc.libera.chat** that contain `archlinux` in their name, use the command `/query alis list *archlinux*`.

Channel	Description
#archlinux64	x86_64 specific discussion channel, mostly in English
#archlinux-aur	AUR general discussion
#archlinux-aurweb	aurweb development discussion
#archlinux-bugs	Bug-centric discussion
#archlinux-classroom	A project that develops and hosts classes for the Arch Linux community.
#archlinux-devops	Arch Linux internal infrastructure and devops discussions.
#archlinux-multilib	Arch Linux Multilib Project discussion and packaging
#archlinux-newbie	A space to learn, try new things, and ask for help without fear of ridicule.
#archlinux-pacman	Pacman development and discussion
#archlinux-projects	Projects development and discussion (mkinitcpio, abs, dbscripts, devtools, ...)
#archlinux-reproducible	Discussion channel for achieving reproducible builds.
#archlinux-security	Discussion of security issues within Arch packages.
#archlinux-testing	Discussion channel regarding the testing repositories.
#archlinux-wiki	Discussion about ArchWiki, its articles and the Arch Linux Forums.

Channel	Description
#archlinux-women	Discussing gender and equality, mostly in English.
#archlinux-proaudio	Discussion of Arch Linux 专业音频. Users also in the unofficial #archaudio

其它语言的 IRC 频道

International discussions are available at the following channels, also located at the irc.libera.chat IRC network, unless stated otherwise.

Channel	Description
#archlinux-za	Discussion (Afrikaans, English)
#archlinux-br	Discussion (Brazilian)
#archlinux-cr	Discussion (Costa Rica)
#archlinux.cz	Discussion (Czech)
#archlinux.dk	Discussion (Danish)
#archlinux.fi	Discussion (Finnish)
#archlinux-fr	Discussion (French)
#archlinux-gaelic	Discussion (Gaelic)
#archlinux.de	Discussion (German)
#archlinux-greece	Discussion (Greek)
#archlinux-il	Discussion (Hebrew)
#archlinux.hu	Discussion (Hungarian)
#archlinux-it	Discussion (Italian); also on **irc.azzurra.org#archlinux**
#archlinux-nordics	Discussion (the nordics: Danish, Finnish, Norwegian and Swedish)
#archlinux-kr	Discussion (Korean)
#archlinux-ir	Discussion (Persian)
#archlinux.org.pl	Discussion (Polish)
#archlinux-pt	Discussion (Portuguese)
#archlinux.ro	Discussion (Romanian)
#archlinux-ru	Discussion (Russian); also on **irc.mibbit.net#archlinux-ru**
#archlinux-rs	Discussion (Serbian)
#archlinux-es	Discussion (Spanish)
#archlinux.se	Discussion (Swedish)
#archlinux-tr	Discussion (Turkish)
#archlinux-ve	Discussion (Venezuela)
#archlinuxvn	Discussion (Vietnamese, Tiếng Việt)

29.2 Irssi

难度 进阶 | **篇幅** 13.8 KB | **所属** 互联网中继聊天

本节将讲到

- 安装
- 使用
- 设置
- 技巧
- 更多内容

Irssi 是一个基于 ncurses、模块化的 IRC (互联网中继聊天 Internet Relay Chat) 客户端。

得益于健壮的社区以及官方插件的支持，使得 Irssi 客户端能够支持 SILC)和 ICB 协议。

安装

安装软件包 irssi包 。

在 AUR 的 irssi-script 下和 Irssi's Script Repository 有一些脚本。

使用

更详细的介绍请阅读官方文档。

注意：这部分假定您了解 IRC 的基本知识并过去用过其他客户端。

推荐使用终端复用器，例如 tmux 或者 GNU Screen 等。它能让用户轻松地断开和重新连接一个会话，并且能够运行如 nicklist.pl 这类依赖次要窗口的脚本。要运行 irssi ，执行：

```
$ irssi
```

参见 irssi(1)。

命令

Irssi 的命令以反斜杠为开始，并且大小写不敏感。您可以输入 `/help` 来获得内建的命令列表。这份列表也能在线查阅。

命令	说明
/help	列出所有命令或者解释给定的命令。
/network	管理您的 IRC 网络。
/server	管理您的 IRC 服务器。
/connect	连接一个服务器或者网络。
/disconnect	断开目前的服务器连接。
`ALT+(1-0,q-p,etc)`	切换活动窗口。`Ctrl+n` 循环到下一个窗口，`Ctrl+p` 循环到前一个。
/window	管理 irssi 窗口。
/layout	保存或者关闭窗口配置。
/statusbar	管理状态栏。
/set	查看或者变更设置。
/alias	管理您的别名。

设置

个人设置文件应当位于 `~/.irssi/config`。在 `/etc/irssi.conf` 有一份模板。使用 `--config` 参数也能使用代替的配置文件启动 irssi。

- 您可以使用 `/save` 来把当前的配置保存到配置文件中。
- 您可以使用 `/layout save` 来保存当前打开的窗口的位置。

使用SASL进行身份验证

本文或本章节的语言、语法或风格需要改进。参考：帮助:风格

原因：ls `-ssl_capath` actually needed?（在Talk:Irssi讨论）

Irssi 是支持 Simple Authentication and Security Layer (SASL)进行登录的。

你可以进行如下设置，实现以SASL的身份认证机制添加IRC聊天服务器（如LiberaChat）

```
/SERVER ADD -ssl -ssl_verify -ssl_capath /etc/ssl/certs -network liberachat -port 6697 irc.libera.chat
/NETWORK ADD -sasl_mechanism plain -sasl_username username -sasl_password password liberachat
```

注意：

- IRC聊天服务器的名字是区分大小写的，请确保服务器的名字输入正确
- 如上的第一条命令，是用于添加允许 #TLS Connection的聊天服务器
- 如果你之前已经添加了聊天服务器，只需输入第二条命令即可
- 如果你的IRC账户的密码中含有 `$`, 你必须额外再加一个 `$` 作为前缀，使得Irssi将`$`作为正常字符进行读取。如将密码设置为`1234$$56`

如果上述操作没问题的话，Irssi在重启之后，你就能看到在所加入的IRC聊天服务器的频道内输出SASL认证成功的信息。

启动时自动连接到 #archlinux

启动 irssi 并输入以下命令：

```
/server add -auto -network liberachat irc.libera.chat
```

`liberachat` 可以用任何喜欢的词代替，例如常用缩写 `fn`。

保证 SASL 设置正确。也可以配合 `-autosendcmd` 命令使用 NickServ 代替 SASL，但是这可能会与您设置的自动加入频道的设置发生冲突。如果愿意的话，可以使用 SSL 证书代替给 NickServ 发送密码认证。

```
/channel add -auto #archlinux liberachat
/channel add -auto #archlinux-offtopic liberachat
```

TLS 连接

LiberaChat 是使用 6697 端口进行 SSL/TLS 连接的。 若想通过TLS连接LiberaChat IRC网络，可以进行以下设置：

```
/server add -auto -tls -tls_verify -network liberachat -port 6697 irc.libera.chat
```

保存更改：

```
/save
```

如果配置成功，那么你将会看到“Z”模组的设置。你应该会看到以下输出："Mode change (+Zi) for user 你的用户名

Client certificates

LiberaChat 和 OFTC 都支持使用SSL证书进行身份认证并为其提供了相应的明文密码的替代方案。 欲知详情，请看LiberaChat的『使用CertFP』的指导。

要创建有效期为730天的无密码证书（当被要求输入详细信息，如状态或甚至通用名称（CN）时，您可以填写任何您想要的内容）：

```
$ openssl req -newkey rsa:4096 -days 730 -x509 -keyout irssi.key -out irssi.crt -nodes
$ cat irssi.crt irssi.key > ~/.irssi/irssi.pem
$ chmod 600 ~/.irssi/irssi.pem
$ rm irssi.crt irssi.key
```

接下来，找出相应的指纹：

```
$ openssl x509 -sha512 -fingerprint -noout -in ~/.irssi/irssi.pem | sed -e 's/
^.*=//;s/://g;y/ABCDEF/abcdef/'
```

这将把SHA512指纹写入至标准输出(stdout)。 如果需要SHA1指纹, 将参数 `-sha512` 改为 `-sha1`。sed命令可用于通过删除不需要的文本和字符来正确格式化指纹。

赋值其指纹字符串，稍后将会使用它。

在irssi中，断开与网络的连接并为其添加客户端证书和密钥。如果您的证书是在没有密码的情况下生成的，请省略-ssl_pass选项：

```
/disconnect liberachat
/server add -ssl_cert ~/.irssi/irssi.pem  -ssl_pass irssi.pem_password -network
liberachat irc.libera.chat 6697
```

现在，您可以进行连接(不是重新连接哦！)以及注册指纹了

```
/connect liberachat
/msg NickServ identify YOUR_PASSWORD
/msg NickServ cert add YOUR_FINGERPRINT
```

此时，您可以从配置文件中删除密码了(如果您将其保存在其中)，并使用以下命令来保存配置

```
/save
```

启用自动日志记录功能

```
/SET autolog ON
/save
```

隐藏频道内其他用户的状态

为了忽略所有频道用户的加入、离开和退出显示，请在Irssi中输入以下内容：

```
/ignore * joins parts quits
```

查阅 smartfilter 来限制活跃用户的上线提醒

鼠标滚动

为了能在Irssi下启动鼠标滚动功能, 输入以下命令:

```
/run scriptassist
/script install mouse.pl
```
欲将其在Irssi运行开始时启用鼠标滚动功能，请输入以下命令：

```
/script autorun mouse.pl
```
如果以上命令不起作用, 你可以手动将mouse.pl脚本放入`~/.irssi/scripts` 或者`~/.irssi/scripts/autorun`, 操作如下:

```
/script load mouse.pl
```

技巧

HTTP 代理

要在Irssi下使用HTTP代理，需要以下命令：

```
/SET use_proxy ON
/SET proxy_address <Proxy host address>
/SET proxy_port <Proxy port>
/SET -clear proxy_string
/SET proxy_string_after conn %s %d
/EVAL SET proxy_string CONNECT %s:%d HTTP/1.0\n\n
```
然后irssi应该相应地修改其配置文件；如果不需要代理，只需将use_proxy设置为OFF。

如果代理需要密码，请尝试：

```
/SET proxy_password your_pass
```
否则：

```
/SET -clear proxy_password
```
注意：如果设置了HTTP代理，那么SSL认证将失效!

在Irssi中使用tmux来显示频道内的用户昵称列表

Irssi 插件 nicklist 提供了一个可显示昵称列表的小窗口，有两种方法能够实现哦：

- **屏幕**，它只是将列表添加到irssi的右侧，但带来的缺点是每次irssi打印一行时都会重新绘制整个窗口。
- **FIFO**，顾名思义，它将列表写入FIFO，然后可以用例如cat~/.irssi/nicklistfifo连续读取。利用FIFO来实现昵称列表的表现效果会更好：

```
/NICKLIST FIFO
```
This fifo can be used in a tmux window split vertically with *irssi* in its left pane and the *cat* from above in a small one in its right. Since the pane is dependent on its creating tmux session's geometry, a subsequent session with a different one needs to recreate it (which also implies a switch in *irssi* windows to refill the fifo).

E.g., the following script first checks for a running *irssi*, presumed to have been run by a previous execution of itself. Unless found it creates a new tmux session, a window named after and running *irssi* and then the pane with *cat*. If however *irssi* was found it merely attaches to the session and recreates the *cat* pane.

```
#!/bin/sh

T3=$(pgrep -u "$USER" -x irssi)

irssi_nickpane() {
    tmux setw main-pane-width $(( $(tput cols) - 21));
    tmux splitw -v "cat ~/.irssi/nicklistfifo";
    tmux selectl main-vertical;
    tmux selectw -t irssi;
    tmux selectp -t 0;
}

irssi_repair() {
    tmux selectw -t irssi
    [ "$(tmux lsp | wc -l)" -gt 1 ] && tmux killp -a -t 0
    irssi_nickpane
}

if [ -z "$T3" ]; then
    tmux new-session -d -s main;
    tmux new-window -t main -n irssi irssi;
    irssi_nickpane ;
fi
    tmux attach-session -d -t main;
    irssi_repair ;
exit 0
```

提示：Instead of doing all this work, this plugin does all the work needed for a nice nicklist inside tmux.

Virtual hostname (vhost)

A vhost can be used to change your hostname when connected to an IRC-server, commonly viewed when joining/parting or doing a whois. This is most commonly done on a server that has a static IP address. Without a vhost it would commonly look like so when doing a 'whois':

```
nick@123.456.78.90.isp.com
```

The result of a successful vhost could be like so if you have the domain example.com available:

```
nick@example.com
```

Keep in mind that not every IRC-server supports the use of vhost. This might be individually set between the servers and not the network, so if you are experiencing issues with one server try another on the same network.

Required preconfigurations

Irssi supports using a vhost as long as the required configurations has been set. This includes especially that your host supports Recursive DNS Lookup (rDNS) using Pointer record (PTR). Additionally you should add an appropriate line to your `/etc/hosts` file.

To see if this is working, test with the 'host' DNS lookup utility included in bind包 like so (where *ip* is a normal IPv4 address):

```
host ip
```

If this returns something in the lines of this then you know that your rDNS is working.

```
ip.in-addr.arpa domain name pointer example.com
```

Enabling the vhost

There are a couple of ways to connect to a server with a given hostname. One is using the 'server' command with a -host argument like so:

```
/server -host example.com irc.libera.chat
```

Another way would be to set your hostname (vhost) with the 'set' command which will save your hostname to `~/.irssi/config`:

```
/set hostname example.com
/save
/server irc.libera.chat
```

更多内容

- Official website
- Official Irssi script repository
- Setting up Irssi
- Guide to efficiently using Irssi and screen by Matt Sparks
- IRC notifications with dzen2 by Jason Ryan
- Irssi's /channel, /network, /server and /connect – What it means by Aaron Toponce
- awesome Wiki Irssi tips (Wayback Machine)
- irssi systemd unit GitHub gist

志愿计算

志愿计算要把机器闲置的算力交给外部项目，并让它长期稳定地跑下去。本章两节都从安装与配置讲起，建议先读 Folding@home 一节：篇幅短、依赖少，适合先跑通，但其中监测工作单位计算情况的部分务必细看；BOINC 一节更长，时间有限时可先读安装、使用与日志文件，再读选择项目时需要考虑的因素，疑难解答与参见、相关资源按需查用。最容易踩的坑是把装完当成结束：工作单位的进度和日志文件，才是判断贡献是否真的在产生的依据；另一个误解是项目越多越好，每个工作单位反被拖慢，不如先跑稳一个再扩展。

30.1　Folding@home

难度 入门 | **篇幅** 4.6 KB | **所属** 志愿计算

本节将讲到

- 安装
- 配置
- 监测工作单位的计算情况
- 问题解决
- 参见

Help Stanford University scientists studying Alzheimer's, Huntington's, Parkinson's, and many cancers by simply running a piece of software on your computer. The problems we are trying to solve require so many calculations, we ask people to donate their unused computer power to crunch some of the numbers. In just 5 minutes... Add your computer to over 333,684 others around the world to form the world's largest distributed supercomputer.

安装

安装 foldingathomeAUR 包。为了使用您的 GPU 进行计算（强烈推荐），您需要 ocl-icd包 和适合您的 GPU 的 OpenCL 包。 Nvidia 用户也可以使用 CUDA。

配置

运行 `FAHClient --configure`，以 root 身份生成 `/etc/foldingathome/config.xml` 配置文件。(Arch Linux 团队编号是 45032) 此外，您可以手动编辑 `/etc/foldingathome/config.xml`。有了配置文件后，您就可以启动守护进程，检查它的状态，并使守护进程在启动时自动启动。

```
$ cd /etc/foldingathome
# FAHClient --configure
```

然后启用 `foldingathome.service` systemd 单元。Nvidia 用户也应该启用 `foldingathome-nvidia.service` systemd 单元。

图形化运行

您可以通过打开网页浏览器并访问 http://localhost:7396/ 来管理守护进程。另外，您也可以安装 fahcontrolAUR 并使用 FAHControl 程序。

守护进程也可以被远程控制。这样做的说明
在 `/etc/foldingathome/config.xml` 中列出。如果有必要，请记得打开防火墙端口。

终端中运行

foldingathome 的行为可以通过编辑 `/opt/fah/config.xml` 来定制。可以指定的一些选项如下：

- 密码，用于识别你的身份。虽然不需要，但它提供了一定程度的安全性。详情请参见

```
<passkey v='passkey'/>
```

- CPU 或 GPU 的并行

```
<slot id='0' type='CPU'/>
```

在限制权限的情况下运行 f@h

更新版的 foldingathomeAUR 包（>7.6.9）已经以有限用户的身份运行。它还安装了一个您可以使用的 systemd 用户脚本，没有 root 权限的用户都可以启用它（您仍然需要 video 组权限才能使用 GPU）。

监测工作单位的计算情况

有几种方法可以在命令行和 GUI 中监控 FAH 客户端的进度。

Folding@home 会将其日志文件写入数据目录。通过检查最后几行，你可以检查它的进度，例如 `tail -10 /var/log/foldingathome/log.txt`。

folding at home 发行的 FAHCONTROLAUR 软件为您提供了高效的控制本地主机和远程主机的方法。只需用相应的“添加”按钮添加另一个客户端，然后输入名称、IP地址、端口和密码（如果你设置了的话），点击保存即可。这时软件应该会尝试建立与远程主机的连接，并在单独的客户端选项卡中显示进度。

要检查 NVidia GPU 的使用率，你可以使用 nvtop-gitAUR。它还可以报告核心温度和电源使用情况。

问题解决

如果您的 GPU 被支持但仍未使用，请编辑 `/etc/foldingathome/config.xml` 文件，尝试自动配置 GPU：

```
<gpu v='true'/>
```

参见

- Folding@home 主页
- Folding@home FAQ
- Folding@home 配置指南
- Arch Folding@home 团队页面
- 更多有关 Arch 团队的统计 extremeoverclocking.com

30.2 BOINC

难度 进阶 | **篇幅** 9.9 KB | **所属** 志愿计算

本节将讲到

- 安装
- 使用
- 日志文件
- 选择项目时需要考虑的因素
- 疑难解答

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 请完成更新后的翻译。（在 Talk:BOINC# 中讨论）

BOINC 官方网站：“利用您的计算机（Windows、Mac 以及 Linux）的空闲的时间，帮助寻找治疗疾病的方法、研究全球变暖、发现脉冲星…… 进行各种各样的科学研究。这一切简单且安全！”

维基百科：“伯克利开放式网络计算平台（Berkeley Open Infrastructure for Network Computing，BOINC）是一个用于志愿计算和网格计算的非商业的中间件系统。开发 BOINC 的最初目的是支援 SETI@home 项目，而后来它发展成为一个支持多种项目的强大的分布式计算平台，这些项目的研究领域涵盖了数学、医学、分子生物学、气候学以及天体物理学等等。BOINC 力图让研究人员享用到全世界数以亿计的个人电脑的强大计算能力。”

安装

官方仓库提供了 boinc包 和 boinc-nox包 软件包，使用安装其一即可。后者不依赖 Xorg，适用于没有物理控制台的服务器。

两个软件包都安装了一个 systemd 单元文件，叫做 `boinc-client.service`。

您还需要将自己添加到`boinc`用户组，以便管理员连接。

要生成下一节中引用的必要文件，请确保启动`boinc-client.service`。

使用

注意：以下内容仅仅适用于：1、Arch 官方软件包提供的 BOINC；2、从 systemd 服务正确启动。如要通过 boinc 命令启动管理，请自行查阅相关文档。

密码文件配置

BOINC 服务会在首次启动后自动创建密码文件 `/var/lib/boinc/gui_rpc_auth.cfg`，任何管理器都需要使用该密码来链接 BOINC 服务。如果从菜单图标启动 BOINC Manager，它会自动读取该文件。但该文件默认只有用户 boinc 可读，为了连接 BOINC 服务，你需要先将当前用户加入 boinc 组，并修改密码文件的访问权限：

```
$ gpasswd -a <用户名> boinc
$ sudo chmod g+r /var/lib/boinc/gui_rpc_auth.cfg
```

重新登录即可生效。

如果你经常使用命令行操作，或需要从终端启动 BOINC Manager，则可将密码文件连接至用户目录，这样 BOINC Manager 或 boinccmd 就能自动连接 BOINC 服务：

```
$ cd ~/
$ ln -s /var/lib/boinc/gui_rpc_auth.cfg gui_rpc_auth.cfg
```

若要手动设置该密码（甚至清空），只需修改 `/var/lib/boinc/gui_rpc_auth.cfg` 文件，并重启 BOINC 服务即可。

图形界面

运行 `boincmgr` 命令启动图形界面的 BOINC Manager：

```
$ boincmgr
```

首次启动时，BOINC Manager 会指引你加入项目。注意，有些项目可以通过 BOINC Manager 直接创建项目，而有些则需要先在其网页上注册。如果存储空间、计算能力和时间等条件允许，可以通过菜单项*工具 / 增加项目*加入多个项目。

如果 BOINC Manager 没有像上面那样指引你加入项目，那么可能是连接 BOINC 服务失败了。点击菜单项*高级 / 选择计算机*，输入计算机名（本机使用 `localhost` 即

可），然后输入 `/var/lib/boinc/gui_rpc_auth.cfg` 中的密码即可。（如果不想每次都这样做，请确保按照上面的步骤正确配置 gui_rpc_auth.cfg 。）

使用 GPU 的项目

要使用 GPU 计算，需要安装闭源的 NVIDIA 和 AMD 驱动。对于 ATI/AMD 显卡，还需要安装 AUR 软件包 libopenclAUR\[损坏的链接：package not found\]。NVIDIA 用户只需安装 \[extra\] 中的 opencl-nvidia包 即可。

此外还需要将 boinc 用户加入 video 组：

```
# gpasswd -a boinc video
```

boinc 用户还需要有访问你的 X session 的权限：

```
xhost local:boinc
```

若要避免每次都输入，可将上述命令加入启动脚本。

注意：对于使用双显卡笔记本电脑的用户，如果使用某些软件（如 bumblebee/bbswitch）禁用了高性能显卡的驱动，在使用 GPU 计算时，可能遇到一些麻烦（找不到显卡、计算错误等等）。如果要使用 GPU 计算，原则上你必须在 BOINC 运行时确保显卡开启，以及加载正确的驱动模块。当前似乎还没有什么好的解决方案。——译者注

命令行界面

对于命令行操作，可参照 `boinccmd` 和 `boinc` 命令的帮助及 man 手册。`boinccmd` 是 BOINC 自带的命令行界面工具。

日志文件

BOINC 的日志文件存放在 `/var/lib/boinc/` 中：

```
/var/lib/boinc/stderrdae.txt
/var/lib/boinc/stdoutdae.txt
```

选择项目时需要考虑的因素

不同项目的硬件要求（CPU、磁盘空间等）有所差异，每个任务的计算时间也不尽相同。如果不能在上报期限前完成任务，服务器会判定任务过期，并把他交给其他计算机。请根据硬件条件选择合适的项目，避免这样的问题发生。

此外，项目的研究方向和公开度也是需要考虑的因素。

64 位系统

某些项目只提供32位计算程序和图形显示程序，因此你需要从 \[multilib\] 安装一些基本的库。

运行计算程序需要通常需要（以 Climateprediction 项目为例）： : lib32-glibc包, lib32-glib2包 **显示图形需要（如 WCG 的几个子项目、Climateprediction）：** : lib32-pango包, lib32-libxdamage包, lib32-libxi包, lib32-mesa-libglAUR\[损坏的链接：replaced by lib32-mesa包\], lib32-libjpeg6AUR\[损坏的链接：package not found\] (AUR), lib32-libxmu包

疑难解答

GPU missing

如果出现这样的错误：

```
GPU Missing
```

且 GPU 计算任务无法启动，则应尝试重启 `boinc-client.service` 服务。

这样的错误通常是因为 BOINC 服务先于 X 启动。

笔记本过热，续航时间缩短

如果在笔记本上运行 BOINC，且 CPU 频率管理策略为 ondemand（默认即是），CPU 频率会始终保持在最大值，导致 CPU 过热、续航时间缩短。解决方法是要求内核调整 CPU 频率时忽略 BOINC 任务的影响：

```
# echo 1 >/sys/devices/system/cpu/cpufreq/ondemand/ignore_nice_load
```

创建以下 tmpfiles.d 配置，可实现开机自动设置：

```
/etc/tmpfiles.d/ondemand-ignore-nice.conf
w /sys/devices/system/cpu/cpufreq/ondemand/ignore_nice_load - - - - 1
```

无法下载 World Community Grid 任务

由于 openssl 的问题，一些用户无法下载 World Community Grid 项目的任务。要解决这一问题，需要对该软件包进行一些修改，然后重新编译。

首先，获取 openssl 的 PKGBUILD，以及打包其他所需文件：

```
$ git clone https://github.com/archlinux/svntogit-packages.git --single-branch --branch packages/openssl
```

如果你安装了 yaourt，也可以使用如下命令：

```
$ yaourt -G openssl
```

执行完第一个命令后，你会看到一个 ./packages/trunk/ 目录，其中有包括 PKGBUILD 在内的若干文件。该 PKGBUILD 需要修改，正确的可以从这里下载。

然后，创建软件软件包：

```
$ makepkg
```

完成后，会出现一个扩展名为 .pkg.tar.xz 的软件包，安装并重启 BOINC 服务即可：

```
# pacman -U openssl-*.pkg.tar.xz
# systemctl restart boinc
```

大功告成。如果还有问题，请参见原讨论贴，发帖请求帮助。

cc_config.xml 相关配置

cc_config.xml 为BOINC管理器的一个高级配置脚本。通过增加不同的参数，可以达到许多意想不到的效果。

将编辑好的文件复制到 /var/lib/boinc 后，还需更改文件为640权限。

相关资源

- 中国分布式计算总站
- BOINC 官方网站
- BOINC 项目列表
- 维基百科的介绍

文件分享

文件分享要么托付给别人，要么自己搭，本章两节分别处理：《Dropbox》把文件同步到外部服务，《Nextcloud》在本机架起一套自主可控的服务，取舍在省事与可控之间。

《Dropbox》读安装、启动与「Securing your Dropbox」即可，多实例与笔记本两节属特定场景，等有第二份账户或要带笔记本出门时再翻。《Nextcloud》篇幅更长，务必从「安装概览」读起，再沿安装、配置、数据库、应用服务器、Web 服务器走一遍。最容易踩的坑是把 Nextcloud 当成装完一个包就了事——数据库、应用服务器与 Web 服务器要各自配好；Dropbox 一侧则常被忽略：不装客户端也能访问文件。

31.1　Dropbox

难度 进阶 | **篇幅** 18.9 KB | **所属** 文件分享

本节将讲到

- 安装
- 启动
- 不安装 Dropbox 而访问文件
- Securing your Dropbox
- Multiple Dropbox instances

翻译状态：

- 本文（或部分内容）译自 Dropbox，最近一次同步于 2014-12-16，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

**这篇文章或章节的翻译质量不佳。**

原因：Last updated in 2014（在 Talk:Dropbox# 中讨论）

Dropbox is a file sharing system that recently introduced a GNU/Linux client. Use it to transparently sync files across computers and architectures. Simply drop files into your `~/Dropbox` folder, and they will automatically sync to your centralized repository.

安装

AUR

您可以从 AUR 安装 dropboxAUR 。

注意：如果 PKGBUILD 已经过时，你可以在下载后更新它，或者在 AUR 页面标记其为过期。

1. 安装完毕，您就可以从程序菜单中的图标或者是在命令行中输入 `dropboxd` 启动 Dropbox。 您将会在系统托盘中看到程序的图标。
2. Dropbox 也许会有弹窗提示（此程序非官网下载等等），你知道这个程序是从 AUR 下载安装的就好啦，然后你可以放心大胆的点击“Don't ask again”来忽视这个提示。
3. 然后会看到一个要求您输入账户信息的窗口，您可以输入已有账户或者是新建一个账户。
4. 再然后您会看到 Dropbox 的欢迎页面，此处将会有一个关于如何使用 Dropbox 的简短的介绍。
5. 点击 "Finish and go to My Dropbox"。

手动安装

**本文或本章节的事实准确性存在争议。**

原因： 应该尽量使用 pacman 来安装和管理软件包，或是从 PKGBUILD 建立软件包。（在 Talk:Dropbox 中讨论）

以下的内容将介绍从命令行下手动安装 Dropbox。以 64 位系统为例，运行：

```
$ cd ~
$ wget -O - "https://www.dropbox.com/download?plat=lnx.x86_64" | tar xzf -
```

然后，启动 Dropbox：

```
$ ~/.dropbox-dist/dropboxd
```

可选软件包

- 纯命令行界面，从 AUR 安装 dropbox-cliAUR。
- GNOME 集成，从 AUR 安装 nautilus-dropboxAUR，此插件将会自动启动 Dropbox。
- Nemo 集成，从 AUR 安装 nemo-dropbox-gitAUR。
- Thunar 集成, 从 AUR 安装 thunar-dropboxAUR。
- 对于 KDE 用户，也可以在 AUR 中找到 KDE 下的应用：kfileboxAUR。

启动

自动启动 Dropbox

对于 KDE 用户来说，在注销用户或者是重启机器后，之前正在运行的程序将会自动重新启动。Xfce 稍有不同，将 `dropbox.desktop` 放置在 {ic|~/.config/autostart}} 下，Dropbox 在下次启动时就会自动启动了。

如果这些都不能正常的工作，那您也可以将 `~/.dropbox-dist/dropboxd &` 添加到 `~/.xinitrc` 来让窗口管理器控制它的启动（也有可能在

`~/.config/openbox/autostart`,看你的设置了）。当然，您也可以将 Dropbox 作为服务启动.

作为 systemd 的服务启动

当前的 Dropbox 版本已经包含了 systemd 服务文件。默认的 systemd 服务文件并不能在系统托盘中显示软件的图标，但是依旧在后台默默的为您同步您的文件。如果您需要托盘图标支持，那么建立 `/etc/systemd/system/dropbox@.service` ，修改环境变量 `DISPLAY` 后替换默认的 systemd 服务文件：

```
/etc/systemd/system/dropbox@.service
.include /usr/lib/systemd/system/dropbox@.service
[Service]
Environment=DISPLAY=:0
```

最后，启动此服务，在登录时 Dropbox 将能够自动运行：

```
# systemctl enable dropbox@<user>
```

Note that you have to manually start Dropbox the first time after installation, so that it runs through the login and setup screen. Further, you need to uncheck the option **Start Dropbox on system startup** in order to prevent Dropbox from being started twice. The daemon can then be used subsequently.

Run as a daemon with systemd user

If you have followed the systemd/User wiki page, you probably want to start dropbox only when you log in or launch your WM/DE. The solution in that case is to create a service in your home directory instead of using the sysadmin account:

```
$HOME/.config/systemd/user/dropbox@.service
[Unit]
Description=Dropbox as a systemd service
After=xorg.target

[Service]
ExecStart=/home/<user>/.dropbox-dist/dropboxd
ExecReload=/bin/kill -HUP $MAINPID
Environment=DISPLAY=%i

[Install]
WantedBy=default.target
```

Then you can start or enable it with:

```
$ systemctl --user {start|enable} dropbox@:0.service
```

That way you can easily start it in your main display (likely :0) or in another one, without having to hard code it.

注意：After a lot of trial and error I found that using `/usr/bin/dropboxd` did not start the service and it did not show any error either (even when running it directly from the terminal worked fine). I believe it has to do that starting it that way systemd does not know which user is actually running the daemon.

不安装 Dropbox 而访问文件

如果您仅需要基本的访问文件的功能，那么您可以访问 https://www.dropbox.com/ 来上传或者是下载您的文件。此方法不需要安装 Dropbox 。

另外，您也可以尝试从 AUR 安装 droxiAUR ，此终端工具提供了类似于 GNU `ftp` 客户端的功能。

Securing your Dropbox

If you want to store sensitive data in your Dropbox, you should encrypt it before. Syncing to Dropbox is encrypted, but all files are (for the time being) stored on the server unencrypted just as you put them in your Dropbox.

- Dropbox works with TrueCrypt, and after you initially uploaded the TrueCrypt volume to Dropbox, performance is quite okay, because Dropbox has a working binary diff.
- Another possibility is to use EncFS, which has the advantage that all files are encrypted separately, i.e. you do not have to determine in advance the size of the content you want to encrypt and your encrypted directory grows and shrinks while you add/delete/modify files in it. You can also mount an encrypted volume at startup using the `-S` option of `encfs` to avoid having to input the passphrase, but note that your encrypted files are not secure from someone who has direct access to your computer.

Setup EncFS with Dropbox

Follow the Wiki instructions to install EncFS.

Assuming you have set your Dropbox directory as `~/Dropbox`:

Create a folder. Files you want synced to Dropbox will go in here.

```
$ mkdir ~/Private
```

Run the following and enter a password when asked:

```
$ encfs ~/Dropbox/Encrypted ~/Private
```

Your secure folder is ready for use; creating any file inside `~/Private` will automatically encrypt it into `~/Dropbox/Encrypted`, which will then be synced to your cloud storage.

To mount your EncFS folder on every boot, follow the instructions in the EncFS wiki page.

提示：Consider using the `ENCFS6_CONFIG` variable and moving the `.encfs6.xml` file to another location (like a USB stick), to help ensure that your encrypted data and the means to realistically decrypt it do not exist together online.

Multiple Dropbox instances

If you need to separate or distinguish your data, personal and work usage for example, you can subscribe to Dropbox with different email addresses and have their directories synced by different Dropbox instances running on a single machine.

The basic principle and general how-to are described in the Dropbox Wiki.

To summarize, you can setup new or additional instances with:

```
mkdir /path/to/.dropbox-alt-1
HOME=/path/to/.dropbox-alt-1 /usr/bin/dropbox start -i
```

Once that is done, stop any Dropbox instance still running and start them like this:

```
HOME=/path/to/.dropbox-alt-1 /path/to/.dropbox-alt-1/.dropbox-dist/dropboxd
HOME=/path/to/.dropbox-alt-2 /path/to/.dropbox-alt-2/.dropbox-dist/dropboxd
```

Pay attention to use different `.../.dropbox-dist/dropboxd` binaries. Even when setting a custom HOME value, the `/opt/dropbox/dropbox` or `/opt/dropbox/dropboxd` wrappers allow only one instance and when started they will kill the one already running.

Dropbox on laptops

Dropbox itself is pretty good at dealing with connectivity problems. If you have a laptop and roam between different network environments, Dropbox will have problems reconnecting if you do not restart it. **Try one of the methods described below first,** if for some reason the problem remains, you may try one of these hackish solutions: , .

注意：When using any of these methods, you need to prevent Dropbox from doing a standard autostart by unchecking *Dropbox - Preferences - General - Start Dropbox on system startup*. This prevents Dropbox from creating the `~/.config/autostart/dropbox.desktop` file and thus from starting twice.

Using netctl

For netctl, use `ExecUpPost` and `ExecDownPre` respectively in every network profile you use, or for example in `/etc/netctl/interfaces/wlan0` to start Dropbox automatically whenever profile on `wlan0` is active. Add '|| true' to your command to make sure netctl will bring up your profile, although Dropbox fails to start.

```
ExecUpPost="any other code; su -c 'DISPLAY=:0 /usr/bin/dropboxd &' your_user || true"
ExecDownPre="any other code; killall dropbox"
```

Obviously, `*your_user*` has to be edited and `*any other code*;` can be omitted if you do not have any. The above will make sure that Dropbox is running only if there is a network profile active.

Using NetworkManager

If you have connectivity problem with NetworkManager, try using a dispatcher script: networkmanager-dispatcher-dropboxAUR\[损坏的链接：package not found\] or networkmanager-dispatcher-dropbox-systemdAUR\[损坏的链接：package not found\].

Using wicd

Create /etc/wicd/scripts/postconnect/dropbox:

```
#!/usr/bin/env bash
su -c 'DISPLAY=:0 /usr/bin/dbus-launch dropboxd &' your_username
```

or, if you use dropbox with systemd:

```
#!/usr/bin/env bash
systemctl restart dropbox@<user>
```

Create /etc/wicd/scripts/postdisconnect/dropbox:

```
#!/usr/bin/env bash
killall dropbox
```

or, if you use dropbox with systemd:

```
#!/usr/bin/env bash
systemctl stop dropbox@<user>
```

注意：If you use PCManFM as your file manager, Dropbox will use 'xdg-open' calls pcmanfm to open the Dropbox folder.However, without a dbus session, you can not use Trash in PCManFM. You should refer to Dbus and General troubleshooting#Session permissionsto edit your ~/.xinitrc based on /etc/skel/.xinitrc to start a D-Bus session before your launch any other program in ~/.xinitrc. Do use 'dbus-launch dropboxd' instead of just 'dropboxd' in wicd postconnect script. otherwise pcmanfm launched by clicking dropbox icon can not use the Trash.

Troubleshooting

Dropbox keeps saying Downloading files

But in fact now files are synced with your box. This problem is likely to appear when your Dropbox folder is located on a NTFS partition whose mount path contains spaces, or permissions are not set for that partition. See more in the forums. To resolve the problem pay attention to your entry in `/etc/fstab`. Avoid spaces in the mount path and set write permissions with the "default_permissions" option:

```
UUID=01CD2ABB65E17DE0 /run/media/username/Windows ntfs-3g
uid=username,gid=users,default_permissions 0 0
```

Change the Dropbox location from the installation wizard

Some users experience the problem during setting-up Dropbox that they cannot select a Dropbox folder other than `/home/username/Dropbox`. In this case when the window for changing the path is shown , hit `Ctrl+l`, enter the location (e.g. /mnt/data/Dropbox) and click on the '"Choose" or "Open" button.

Context menu entries in file manager do not work

Several file managers such as Thunar, GNOME Files or its fork Nemo come with extensions that provide context menu entries for files and folders inside your Dropbox. Most of them will result in a browser action such as opening the file or folder in dropbox.com or sharing the link. If you experience these entries not working, then it is likely you have not set the `$BROWSER` variable which Dropbox requires. See Environment variables for details.

Connecting...

It may happen that Dropbox cannot connect successfully because it was loaded before an internet connection was established. This can happen on wireless connections, or fast loading machines on wired networks. The best solution to this problem, for wired and wireless connections, is #Dropbox on laptops which will ensure that Dropbox is started only after the connection is established.

An alternative solution, for those not using netctl or NetworkManager, is to delay the startup of dropbox:

- `cp ~/.config/autostart/dropbox.desktop ~/.config/autostart/dropbox-delayed.desktop`
- Prevent dropbox from doing a standard autostart by unchecking Dropbox - Preferences - General - Start Dropbox on system startup. This removes `~/.config/autostart/dropbox.desktop`.
- Edit `~/.config/autostart/dropbox-delayed.desktop` and replace `Exec=dropboxd` with `Exec=bash -c "sleep *timeout* && dropboxd"`. Tweak the *timeout* parameter, the value of `3` is a good start.

Dropbox does not start - "This is usually because of a permission error"

Check permissions

Make sure that you own Dropbox's directories before running the application. This includes

- `~/.dropbox` - Dropbox's configuration directory
- `~/Dropbox` - Dropbox's download directory (default)

You can ensure this by changing their owner with `chown -R`.

This error could also be caused by `/var` being full.

Re-linking your account

Dropbox's FAQ suggests that this error may be caused by misconfiguration and is fixed by (re)moving the current configuration folder

```
# mv ~/.dropbox ~/.dropbox.old
```

and restarting Dropbox.

Errors caused by running out of space

A common error that might happen is that there is no more available space on your `/tmp` and `/var` partitions. If this happens, Dropbox will crash on startup with the following error in its log:

```
Exception: Not a valid FileCache file
```

A detailed story of such an occurrence can be found in the forums. Make sure there is enough space available before launching Dropbox.

Another case is when the root partition is full:

```
OperationalError: database or disk is full
```

Check to see the available space on partitions with `df`.

Locale caused errors

Try starting `dropboxd` with this code:

```
LANG=$LOCALE
dropboxd
```

(You can also use a different value for LANG; it must be in the format "en_US.UTF-8") This helps when running from a Bash script or Bash shell where `/etc/rc.d/functions` has been loaded

Filesystem monitoring problem

If you have a lot of files to sync in your Dropbox folder, you might get the following error:

```
Unable to monitor filesystem
Please run: echo 100000 | sudo tee /proc/sys/fs/inotify/max_user_watches and restart
Dropbox to correct the problem.
```

This can be fixed easily by adding

```
fs.inotify.max_user_watches = 100000
```

to `/etc/sysctl.d/99-sysctl.conf` and then reload the kernel parameters

```
# sysctl --system
```

代理设置

最简单的方法是在 Dropbox 的首选项页面设置代理。另外也可以设置为自动选择代理，它将会读取 http_proxy 的变量而使用合适的代理。

```
env http_proxy=http://your.proxy.here:port /usr/bin/dropboxd
```

或者

```
export http_proxy=http://your.proxy.here:port
/usr/bin/dropboxd
```

注意：Dropbox 仅支持此形式的 `http://your.proxy.here:port` 代理表示方法，此表示方法不被支持 `your.proxy.here:port`（大部分程序都不支持此方法）。

不要让它自动升级

```
rm -rf ~/.dropbox-dist
install -dm0 ~/.dropbox-dist
```

31.2 Nextcloud

难度 高级 | **篇幅** 47.1 KB | **所属** 文件分享

本节将讲到

- 安装概览
- 安装
- 配置
- 数据库
- 应用服务器

翻译状态：

- 本文（或部分内容）译自 Nextcloud，最近一次同步于 2024-11-17，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

根据 Wikipedia:Nextcloud:

Nextcloud 是一套客户机——服务器软件（依赖被称为 apps 的组件），能够实现诸如共享，协作以及沟通的需要,比如：

- 文件共享
- 个人信息管理（contacts,calendar,tasks)
- 通讯（mail,chat,video conferencing)
- 合作编辑(text,Office integration)

Nextcloud 是开源的，并且它基于开放标准。数据主权是 Nextcloud 的一大优势，也就是说，你可以部署自己的 Nextcloud 实例来摆脱诸如 Dropbox、Office365 和 Google Drive 等专有（甚至不可信）的服务的束缚。

Nextcloud可以按照你的需求部署在小至单板计算机（比如树莓派），大到有数百万用户的超大型数据中心中。Nextcloud 具有一套精心设计的授权方案以及可选的联邦方案（用于连接多个独立的实例），所以 Nextcloud 同样非常适合在企业环境下部署。

Nextcloud 是 ownCloud的分支，有关其历史，请参看其Wikipedia 页面。

安装概览

完全安装的 Nextcloud 应当（至少）包含以下组件：

一个**web 服务器**；与之配套的**应用服务器**，用来运行 **Nextcloud**（即 PHP 代码）；一个供 Nextcloud 使用的**数据库**。

这篇文章将会讲解使用 MariaDB/MySQL 或 PostgreSQL 作为数据库和以下 web 服务器与应用服务器的组合：

- Nginx->uWSGI(plus uwsgi-plugin-php)
- Nginx->FPM
- Apache HTTP server(using mod_proxy_usgi)->uWSGI(plus uwsgi-plugin-php)
- Apache HTTP server(using mod_proxy_fcgi)->FPM

Nextcloud 包符合 Web 应用包指导规范。它要求 Web 应用程序应当由专门的用户运行——在本例中为 `nextcloud`。这就是为什么要使用应用服务器。出于相同的理由，使用 php-apache直接在 Apache 中执行 Nextcloud 的 PHP 代码也是不可能的。

安装

注意：nextcloud包 提供了基于 php包 或基于 php-legacy包 的安装（依赖元软件包 php-interpreter）。本文强烈建议使用 php-legacy包 包安装以保证安全（也能让你高枕无忧）。详情请看迁移到 php-legacy。本文假定您使用了 php-legacy包 进行安装。

安装 nextcloud包。当被问及时，选择 php-legacy包 作为你的php版本，这会拉取相当多的依赖。大多数必要的 PHP 扩展都将以这种方式进行处理。此外，你必须安装 php-legacy-gd包（最好是作为依赖包安装，利用 pacman 的`--asdeps`选项实现）。

同时建议你安装下面的软件包（同样使用`--asdeps`选项）：

- php-legacy-sodium包 用于argon2散列算法
- php-legacy-imagick包和librsvg包 用于生成预览

其他可选依赖将在后面介绍，具体内容取决于你的安装配置（比如：你选择了什么样的数据库）。

请注意，php-legacy包 自带一部分模块（即*bcmath*、*exif*、*gmp*、*intl*和*sysvsem*），这些模块不必显式安装。

配置

PHP

本指南不会修改 PHP 的主配置文件`/etc/php-legacy/php.ini`,Nextcloud的 PHP 配置会放在单独的地方，不干扰其它使用 PHP 的应用程序。这些文件被放在：

- 一份`php.ini`的副本，存放在`/etc/webapps/nextcloud`（用于`occ`命令行工具以及后台作业）。这是一份初始`php.ini`的完整复制，由php-legacy包提供，Nextcloud 将对其进行一些修改，以供使用。
- 应用程序服务器的相关配置。将在应用程序服务器的部分介绍这些内容。

将`/etc/php-legacy/php.ini`复制到`/etc/webapps/nextcloud`(当然，最好是从*php-legacy*的tarball中解压一份php.ini，*php-legacy*包存放在`/var/cache/pacman/pkg`）。然后，虽然不是绝对必须的，但请纪律性修改文件的所有权：

```
# cp /etc/php-legacy/php.ini /etc/webapps/nextcloud
# chown nextcloud:nextcloud /etc/webapps/nextcloud/php.ini
```

Nextcloud文档中列出的大部分所需的PHP模块已经在刚刚复制的PHP配置文件中启用。但还需手动启用以下扩展：

```
/etc/webapps/nextcloud/php.ini
extension=exif
extension=gd
extension=iconv
extension=intl
extension=sysvsem
; bcmath and gmp for passwordless login
extension=bcmath
extension=gmp
; sodium for the argon2 hashing algorithm
extension=sodium
; in case you installed php-legacy-imagick (as recommended)
```

```
extension=imagick
```

根据你准备使用的数据库，启用相应的`pdo_xxxx`模块。请参阅：数据库。

将`date.timezone`设置为你的首选时区，例如：

```
/etc/webapps/nextcloud/php.ini
date.timezone = Asia/Shanghai
```

将PHP的内存限制放宽到至少512MiB：

```
/etc/webapps/nextcloud/php.ini
memory_limit = 512M
```

为了提高安全性，你也可以配置`open_basedir`，但这不是必要的。这限制了 Nextcloud 的 PHP 代码可以读取和写入文件的位置。经过验证的设置是：

```
/etc/webapps/nextcloud/php.ini
open_basedir=/var/lib/nextcloud:/tmp:/usr/share/webapps/nextcloud:/etc/webapps/nextcloud:/dev/urandom:/usr/lib/php-legacy/modules:/var/log/nextcloud:/proc/meminfo:/proc/cpuinfo
```

根据你安装的其他扩展，你可能需要扩充此列表，例如，如果你选择了Redis，则需要扩展 `/run/redis`。

配置*opcache*是不必要的，因为这份`php.ini`只用于`occ`命令行工具和后台作业，这二者并不经常运行PHP进程。

Nextcloud

将以下条目加入nextcloud的配置文件中：

```
/etc/webapps/nextcloud/config/config.php
'trusted_domains' =>
  array (
    0 => 'localhost',
    1 => 'cloud.mysite.com',
  ),
'overwrite.cli.url' => 'https://cloud.mysite.com/',
'htaccess.RewriteBase' => '/',
```

将示例的主机名*`cloud.mysite.com`*修改成你的。如果你的Nextcloud需要通过子文件夹访问（比如`https://www.mysite.com/nextcloud`）`overwrite.cli.url` 和 `htaccess.RewriteBase` 必须做出相应更改。

系统和环境

确保Nextcloud使用刚刚编辑的`php.ini`作为`occ`工具的配置文件，设置`NEXTCLOUD_PHP_CONFIG`环境变量：

```
$ export NEXTCLOUD_PHP_CONFIG=/etc/webapps/nextcloud/php.ini
```

同时将其加入`.bashrc`（或`.bash_profile`）使其永久生效。出于隐私和安全方面的考量，请为会话数据创建专用目录：

```
# install --owner=nextcloud --group=nextcloud --mode=700 -d /var/lib/nextcloud/sessions
```

网页应用的配置文件在 `/etc/webapps/nextcloud/config/config.php`。

注意：Nextcloud 应当把用户数据存放在 `/var/lib/nextcloud/data/`，因为该目录只能被 root 和应用本身访问。要安装使用这个网页应用的软件，请使用 `/var/lib/nextcloud/apps/`。

数据目录

默认情况下，Nextcloud 将用户数据存放在 `/var/lib/nextcloud/data/`，这个位置可以调节：

```
/etc/webapps/nextcloud/config/config.php
$CONFIG = [
/* [..] */
'datadirectory' => '/var/lib/nextcloud/data',
/* [..] */
]
```

注意： `nextcloud` 用户需要有对 `datadirectory` 的写入权限。

可写应用目录

`nextcloud` 不可写入默认应用目录 `/usr/share/webapps/nextcloud/apps/`，因为它是软件包的一部分。

要从应用商店安装应用，使用一个独立的、可写的目录是可以的。它默认指向 `/var/lib/nextcloud/apps/`，并可以通过一个在网页应用根目录下的符号链接 (`/usr/share/webapps/nextcloud/wapps`) 来访问。

这个目录是可调整的:

```
/etc/webapps/nextcloud/config/config.php
$CONFIG = [
/* [..] */
'apps_paths' => [
        [
                'path'=> '/usr/share/webapps/nextcloud/apps',
                'url' => '/apps',
                'writable' => false,
        ],
        [
                'path'=> '/var/lib/nextcloud/apps',
                'url' => '/wapps',
                'writable' => true,
        ],
],
/* [..] */
]
```

注意：

- 声明为 `writable` 的 `apps_paths` 条目需要可由 `nextcloud` 用户写入。此外，需要在 `/usr/share/webapps/nextcloud/` 中创建指向该目录的符号链接。
- 上面的语法使用 PHP 的短数组语法。这可以用大多数指南使用的语法编写：

```
/etc/webapps/nextcloud/config/config.php
$CONFIG = (
/* [..] */
'apps_paths' => array (
0 => array (
'path' => '/usr/share/webapps/nextcloud/apps',
'url' => '/apps',
'writable' => false,
),
```

```
1 => array (
'path' => '/var/lib/nextcloud/apps',
'url' => '/wapps',
'writable' => true,
),
),
/* [..] */
)
```

日志目录

默认情况下，日志生成在 `/var/log/nextcloud/nextcloud.log`，这个位置是可以调整的：

```
/etc/webapps/nextcloud/config/config.php
$CONFIG = [
/* [..] */
'logfile' => '/var/log/nextcloud/nextcloud.log',
]
/* [..] */
```

数据库

MariaDB/MySQL是Nextcloud的推荐选择。

Nextcloud数据库的相关资料大都与MariaDB / MySQL有关。Nextcloud开发人员承认，他们不太了解其它数据库的专业知识。

PostgreSQL据说可以提供比MariaDB/MySQL更好的性能，并且方言更少。SQLite主要支持测试/开发安装，不建议用于生产环境。受支持的数据库列表中还包括了Oracle database，但本指南不对其作介绍。

MariaDB / MySQL

自从2013年以来，MariaDB一直作为Arch Linux中的MySQL默认实现。

如果你想在安装Nextcloud的主机上同时运行数据库，请配置并启动MariaDB（如果你尚未这样做）。请查看此文档获得更多信息。不要忘记使用`mariadb-install-db`命令初始化MariaDB。为了提高安全性，建议将MariaDB配置为仅侦听本地Unix套接字：

```
/etc/my.cnf.d/server.cnf
[mysqld]
skip_networking
```

Nextcloud的官方文档推荐将事务隔离级别设置为READ-COMMITTED。当你预计有大量并发事务从而造成负载过高时，这一点尤其重要。

```
/etc/my.cnf.d/server.cnf
[mysqld]
transaction_isolation=READ-COMMITTED
```

设置`binlog_format=ROW`的建议已经过时，MariaDB现行版本的默认设置“`MIXED`”的表现已经足够好。

以数据库用户root身份启动命令行工具mysql。（默认密码为空，应当尽快修改）

```
$ mysql -u root -p
```

为Nextcloud创建用户和与之配套的数据库：

```
CREATE USER 'nextcloud'@'localhost' IDENTIFIED BY 'db-password';
CREATE DATABASE IF NOT EXISTS nextcloud CHARACTER SET utf8mb4 COLLATE
utf8mb4_general_ci;
GRANT ALL PRIVILEGES on nextcloud.* to 'nextcloud'@'localhost';
FLUSH privileges;
```

（用自行设置的Nextcloud数据库用户的密码替换dp-password字段。完成后使用\\q命令退出。）

注意：MariaDB对于UTF8编码的解释存在缺陷，这导致了MariaDB无法存储码位在0x10000及以上的字符（比如emoji）。MariaDB在5.5版本引入了一种新的编码来“修复”这个问题，它叫做utf8mb4.所以千万不要使用MariaDB的utf8编码，请使用utf8mb4。如果需要迁移，请参阅此文档。

在PHP中启用相应扩展：

```
/etc/webapps/nextcloud/php.ini
extension=pdo_mysql
```

与Nextcloud管理手册中提供的信息不同，对MariaDB进行额外设置是不必要的。

使用以下命令完成对数据库的安装：

```
$ occ maintenance:install \
    --database=mysql \
    --database-name=nextcloud \
    --database-host=localhost:/run/mysqld/mysqld.sock \
    --database-user=nextcloud \
    --database-pass=db-password \
    --admin-pass=admin-password \
    --admin-email=admin-email \
    --data-dir=/var/lib/nextcloud/data
```

提示：原文大概是为了美观对上面的这条命令进行了排版，在输入的时候不能直接复制此命令，否则会报错，去掉反斜杠，按照正常的命令输入即可。比如`occ maintenance:install --database=pgsql --database-name=nextcloud --database-host=/run/postgresql...`。详情请看下面给出的官方文档。

注意将db-password，admin-password，admin-email替换成自己设定的相对应的值。这个命令可以使Nextcloud与数据库运行在同一台主机上。关于更多选项，请查看官方文档（输入`occ help maintenance:install`）。查看使用“occ”命令行工具来了解此工具的详细信息。

PostgreSQL

如果你想在安装Nextcloud的主机上同时运行数据库，请配置并启动PostgreSQL（如果你尚未这样做）。请查看此文档获得更多信息。为了提高安全性，建议将PostgreSQL配置为仅侦听本地Unix套接字：

```
/var/lib/postgres/data/postgresql.conf
listen_addresses = ''
```

特别的，请不要忘记使用initdb命令初始化数据库。接下来使用PostgreSQL的命令行工具psql创建一个名为nextcloud的用户，然后为其创建一个同样名为nextcloud的数据库：

```
[postgres]$ psql
CREATE USER nextcloud WITH PASSWORD 'db-password';
CREATE DATABASE nextcloud TEMPLATE template0 ENCODING 'UNICODE';
ALTER DATABASE nextcloud OWNER TO nextcloud;
GRANT ALL PRIVILEGES ON DATABASE nextcloud TO nextcloud;
\q
```

（用自行设置的Nextcloud数据库用户的密码替换dp-password字段。）

下载PHP依赖包php-legacy-pgsql包（使用pacman --asdpes选项），然后启用相应的PHP扩展：

```
/etc/webapps/nextcloud/php.ini
extension=pdo_pgsql
```

使用以下命令完成对数据库的安装：

```
$ occ maintenance:install \
    --database=pgsql \
    --database-name=nextcloud \
    --database-host=/run/postgresql \
    --database-user=nextcloud \
    --database-pass=db-password \
    --admin-pass=admin-password \
    --admin-email=admin-email \
    --data-dir=/var/lib/nextcloud/data
```

注意将db-password, admin-password, admin-email替换成自己设定的相对应的值。这个命令可以使Nextcloud与数据库运行在同一台主机上。关于更多选项，请查看官方文档（输入occ help maintenance:install）。查看“occ”工具使用方法来了解此工具的详细信息。

应用服务器

uwsgi包和FPM是两种常用的应用服务器，可用于处理PHP代码。其中FPM专门用于PHP，FPM与web服务器之间使用的通信协议是fastcgi。FPM的文档仍有改进的空间。而uWSGI可以通过安装插件来支持包含PHP在内的一部分语言，uWSGI与web服务器之间使用的通信协议是uwsgi（小写）。uWSGI有大量的文档可供查看，虽然大量的文档可能会导致阅读困难以及混乱。

uWSGI

uWSGI有自己的文章。在那里可以找到很多有用的信息。最好作为依赖安装uwsgi包和它的插件uwsgi-plugin-php-legacy包,比如使用--asdpes选项。若要使用uWSGI运行Nextcloud代码，你必须为uWSGI创建一个专门的配置文件（nextcloud.ini）并且定义一个systemd服务。

警告：必须说明的一点：uWSGI最近维护得很少，其PHP插件更是维护甚少。这已经引发了一些问题，现在只能通过Arch Linux软件包的维护者修补uWSGI代码解决问题，即不能在上游解决。

nextcloud.ini

Nextcloud包已经包含了一个示例文件，该文件已经位于正确的位置`/etc/uwsgi/nextcloud.ini`。通常情况下你都必须根据你的需求调整此文件。你应该找到一份具有大量注释的修改版本（与nextcloud自带的相比）。它提供了一个简洁的Nextcloud配置供个人使用（中等负载）。

通常情况下，应当将启用的扩展，扩展的配置以及`open_basedir`与`/etc/webapps/nextcloud/php.ini`同步（opcache除外）。

提示：对`/etc/uwsgi/nextcloud.ini`的更改应该会变得更广泛。在软件包更新期间，将创建一个名为`nextcloud.ini.pacnew`的文件，以防止nextcloud包提供的原始文件发生更改。为了更好的检查新文件的更改情况并将其应用到`/etc/uwsgi/nextcloud.ini`中去，可以采取以下方法：

获取软件包提供`nextcloud.ini`的文件（例如直接从软件包中解压），存储一份它的副本，并将其命名为`nextcloud.ini.package`。 如果因nextcloud包更新而产生了`nextcloud.ini.pacnew`文件，你可以通过下面的命令比对新旧文件的差异：

```
diff nextcloud.ini.package nextcloud.ini.pacnew
```

有选择性的应用更改到你自己的nextcloud.ini,这具体取决于它们是否适用于你的版本

用`nextcloud.ini.pacnew`替换`nextcloud.ini.package`

uWSGI服务

uwsgi包软件包提供了一个模板单元文件（`uwsgi@.service`）。实例ID（此处为*nextcloud*）用于选择正确的配置文件。启用并启动`uwsgi@nextcloud.service`。

如果你有多个（比如2个）像这样运行，这时可以考虑使用emperor模式，这样更节约资源。

FPM

如果选择FPM作为你的应用服务器，下载php-legacy-fpm包（最好作为依赖包安装 `--asdeps`）

FPM的配置包含一份与它提供的全部应用相关的`php.ini`副本，一份专为每个应用（此处为Nextcloud）生成的pool file文件。最后，还需要调整systemd服务文件。

php-fpm.ini

如前文所述，本指南将不会修改PHP主配置文件`/etc/php-legacy/php.ini`，而是创建并修改它的副本：

```
# cp /etc/php-legacy/php.ini /etc/php-legacy/php-fpm.ini
```

确保该文件由root所有且仅能被root修改。（`-rw-r--r-- 1 root root ... php-fpm.ini`）。启用 op-cache（取消该行的注释）。

```
/etc/php-legacy/php-fpm.ini
zend_extension=opcache
```

然后将下面的内容放到`[opcache]`行下面

```
/etc/php-legacy/php-fpm.ini
opcache.enable = 1
opcache.interned_strings_buffer = 16
opcache.max_accelerated_files = 10000
opcache.memory_consumption = 128
opcache.save_comments = 1
opcache.revalidate_freq = 1
```

警告：不要尝试通过php_value\[...\]和php_flag\[...\]来把上面这些设置放到pool file中。否则FPM进程会在第一个请求处崩溃。

nextcloud.conf

下一步是创建pool file。它负责为Nextcloud应用程序生成专用的FPM进程。创建文件`/etc/php-legacy/php-fpm.d/nextcloud.conf`，你也许可以借鉴这个预先配置好的版本。

确保pool file由root所有且仅能被root修改。（`-rw-r--r-- 1 root root ... nextcloud.conf`）。取决于是否开启访问记录（预配置版本中已开启），应当为日志文件创建相应的目录（预配置版本中是`/var/log/php-fpm-legacy/access`）。按你的想法修改配置（特别是`pm...`，`php_value[...]`以及`php_flag[...]`）。`php_value[...]`和`php_flag[...]`应当与文件`/etc/webapps/nextcloud/php.ini`中的相一致（而不是`/etc/php-legacy/php-fpm.ini`）。

也可以通过修改`php-fpm.ini`文件来达到同样的目的，但是对`php-fpm.ini`的修改将对所有由FPM提供服务的应用生效。

提示：php-legacy-fpm包自带一个名为`www.conf`的pool file的文件，但是在本指南中不发挥任何作用。一个防止其生效的好方法是将其重命名为`www.conf.package`并创建一个仅包含注释行（以分号开头的行）的文件`www.conf`。使用这种方法可以将`www.conf`变成空操作文件。该文件同样不会因php-legacy-fpm包的更新而被覆盖。在更新中，新文件被命名为`www.conf.pacnew`，你通过可以对比`www.conf.package`与`www.conf.pacnew`来检查新文件中是否有重大更改。如果发现了需要重新生成`nextcloud.conf`的更改，请在对`nextcloud.conf`操作完成后，将`www.conf.pacnew`重命名为`www.conf.package`。

systemd服务

FPM作为systemd的一个服务运行。你应该修改服务的配置来使其能够运行Nextcloud。最好的方法是通过drop-in文件（：

```
/etc/systemd/system/php-fpm-legacy.service.d/override.conf
```

```
[Service]
ExecStart=
ExecStart=/usr/bin/php-fpm-legacy --nodaemonize --fpm-config /etc/php-legacy/php-fpm.conf --php-ini /etc/php-legacy/php-fpm.ini
ReadWritePaths=/var/lib/nextcloud
ReadWritePaths=/etc/webapps/nextcloud/config
```

- 它将ExecStart行替换为上一节中提到的php-fpm.ini的启动命令。
- 它将/var/lib/nextcloud和/etc/webapps/nextcloud/config目录（及其下所有内容全部置为可写。原始服务定义的ProtectSystem=full会默认将/usr,/boot和/etc作为只读目录对FPM进程开放。

不要忘记启用和启动php-fpm-legacy服务。

保持/etc目录整洁

Nextcloud会默认创建uWSGI的配置文件`/etc/uwsgi/nextcloud.ini`.该文件对你没有任何作用（虽然也没有任何危害）如果你无论如何都不想它出现在你的目录中。将以下代码添加到`/etc/pacman.conf`中：

```
/etc/pacman.conf
# uWSGI configuration that comes with Nextcloud is not needed
NoExtract = etc/uwsgi/nextcloud.ini
```

Web服务器

有相当数量的web服务器可供选择。但你无论作何选择都要记住，Nextcloud应用必须用其自己的系统用户nextcloud运行。所以才需要将请求转发到上文提到的应用服务器中。

nginx

有关nginx的配置，显然已经超过了本文的覆盖范围。可以查看相关文章了解更多信息。Nextcloud的官方文档中也提供了详细配置文件可供参考。你可以自行决定如何将这些代码引入到你的nginx配置文件中。一种常见的方法是使用`/etc/nginx/sites-available`和`/etc/nginx/sites-enabled`目录来单独配置不同的服务器（也被称作虚拟主机）。请参看：Nginx#管理服务器入口。

如果使用了nextcloud文档中提供的nginx配置，应将根目录更改为：

```
cloud.mysite.com.conf
root /usr/share/webapps/nextcloud;
```

`upstream php-handler { ... }`的部分是不必要的。只需要在`location`中指定`fastcgi_pass unix:/run/php-fpm-legacy/nextcloud.sock;`当使用*uWSGI*替代*FPM*时，应将`location`替换成：

```
cloud.mysite.com.conf
location ~ \.php(?:$|/) {
    include uwsgi_params;
    uwsgi_modifier1 14;
    # Avoid duplicate headers confusing OC checks
    uwsgi_hide_header X-Frame-Options;
```

```
    uwsgi_hide_header X-XSS-Protection;
    uwsgi_hide_header X-Content-Type-Options;
    uwsgi_hide_header X-Robots-Tag;
    uwsgi_hide_header X-Download-Options;
    uwsgi_hide_header X-Permitted-Cross-Domain-Policies;
    uwsgi_pass unix:/run/uwsgi/nextcloud.sock;
}
```

你可能需要解决以下问题（部分）：

- 你的服务器名称，即你的Nextcloud能够访问的服务器部分的URL。
- 用于签名的名称和用于SSL/TLS的密钥。
- 访问记录存放的位置。
- Certbot（或其他ACME客户端）用于存放域验证质询的位置。在这里，`alias`可能比`try_files`更合适。
- 用于访问Nextcloud的路径（访问URL中服务器名称和端口的权限）。
- 你正在使用的应用服务器（uWSGI或FPM），即nginx将以何种方式、向何处传递触发的PHP代码（见上文）。
- 配置OCSP装订。

Nginx无需安装任何其他模块，因为其本就支持这两种协议：FastCGI和uwsgi。

Apache HTTP 服务器

apache HTTP服务器中有许多有用的信息。Nextcloud的文档中同样有一些配置样例，你也可以从`/usr/share/doc/nextcloud/apache.example.conf`中找到它们。对mod_php的隐性依赖将不再可用，需要使用mod_proxy_fcgi或mod_proxy_uwsgi。

你可以在本站中找到使apache与FPM协同工作的方法。uWSGI的文档中有使apache与uWSGI和mod_proxy_uwsgi协同工作以处理PHP代码的方法。注意apache包含了`mod_proxy_fcgi`以及`mod_proxy_uswgi`两个插件。它们应按需开启。

下面是运行Nextcloud所需的模块：

```
/etc/httpd/conf/httpd.conf
# these are already loaded in a standard Apache installation
LoadModule headers_module modules/mod_headers.so
LoadModule env_module modules/mod_env.so
LoadModule dir_module modules/mod_dir.so
LoadModule mime_module modules/mod_mime.so
LoadModule setenvif_module modules/mod_setenvif.so

# these need to be uncommented explicitly
LoadModule rewrite_module modules/mod_rewrite.so
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
LoadModule proxy_module modules/mod_proxy.so

# either this one in case you use FPM
LoadModule proxy_fcgi_module modules/mod_proxy_fcgi.so
```

```
# or this one in case you opt for uWSGI
LoadModule proxy_uwsgi_module modules/mod_proxy_uwsgi.so
```

取消注释下面的代码来引入TLS配置参数：

```
/etc/httpd/conf/httpd.conf
Include conf/extra/httpd-ssl.conf
```

有关如何优化TLS配置的详细信息，参看Mozilla SSL配置工具。

请参阅以下两个示例配置文件，具体取决于你希望如何访问Nextcloud：

- 通过主机名访问（例：`https://cloud.mysite.com/`），把这段代码放入`/etc/httpd/conf/extra/httpd-vhosts.conf`中。
- 通过子文件夹访问（例：`https://www.mysite.com/nextcloud/`），把这段代码放入`/etc/httpd/conf/httpd.conf`中。

当然，你应该按照自己的实际情况来修改示例的配置文件。当你使用*uWSGI*时，用`SetHandler "proxy:unix:/run/uwsgi/nextcloud.sock|uwsgi://nextcloud/"`替换`SetHandler`行。

Nextcloud包自带一个`.htaccess`文件，它已经处理了很多重写和标题内容。运行`occ maintenance:update:htaccess`以适配此文件。`/etc/webapps/nextcloud/config/config.php`中的`htaccess.RewriteBase`参数对此至关重要。

后台作业

Nextcloud要求按计划运行某些任务。请查看相关文档来获取更多信息。实现这个目的最简单（也是最可靠）的方法是使用systemd的“服务”和已经内置在nextcloud包中的计时器单元。为使作业使用正确的`php.ini`文件（而不是全局`php.ini`），服务单元需要进行一些调整。创建一个附加配置片段（drop-in file）并添加:

```
/etc/systemd/system/nextcloud-cron.service.d/override.conf
[Service]
ExecStart=
ExecStart=/usr/bin/php-legacy -c /etc/webapps/nextcloud/php.ini -
f /usr/share/webapps/nextcloud/cron.php
```

之后启用并启动`nextcloud-cron.timer`（不是`.service`）

根据文档的建议，添加如下参数到nextcloud的配置文件中。

```
/etc/webapps/nextcloud/config/config.php
....
'maintenance_window_start' => 0,
....
```

该值是UTC定义的整点时刻，每天从这一时刻开始的4小时，所有高耗时的作业将会被尽数处理，也就是说，请将它安排在主要工作时间以外。

警告：不要下载 nextcloud-systemd-timers包。该软件包已经过时且不再维护。

使用内存缓存（In-memory caching）

Nextcloud 的文档推荐使用存内对象缓存，这样可以显著提高性能。

注意：关注这篇通知：Nextcloud服务以服务器推送取代了客户端轮询，这大大减少了同步延迟，这依赖于redis。

APCu

安装php-legacy-apcu包 （作为依赖`--asdeps`）。在相关配置文件中启用此扩展：

- `occ`命令和后台任务需要用到的 `/etc/webapps/nextcloud/php.ini`。
- 下面的文件由你的应用服务器决定
- *uWSGI*：`/etc/uwsgi/nextcloud.ini`。
- *FPM*：`/etc/php-legacy/php-fpm.d/nextcloud.conf`。

将以下代码添加到`/etc/webapps/nextcloud/php.ini`中：

```
/etc/webapps/nextcloud/php.ini
extension=apcu
apc.ttl=7200
apc.enable_cli = 1
```

（最好是写在`Module Settings`部分的下面什么地方）。

对于另外的那个文件，激活APCu的设置默认已经在其中，只需要取消注释即可。 另外两个与 APCu 相关的配置参数也已存在。不需要再修改（touch）`/etc/php-legacy/php.ini`或者`/etc/php-legacy/conf.d/apcu.ini`这两个文件。

重启应用服务器（不是Web服务器，官方文档中强调了）。将下面的代码添加到Nextcloud配置文件中：

```
/etc/webapps/nextcloud/config/config.php
'memcache.local' => '\OC\Memcache\APCu',
```

Redis

下载php-legacy-igbinary包和php-legacy-redis包（作为依赖`--asdeps`）如果你在本地运行这个组件（与Nextcloud在同一台主机上）。 当然，你也可以选择将Redis服务器安装在其它机器上。详细信息参看： Nextcloud官方文档

注意：“Redis”与“APCu”本地缓存并不冲突。事实上，Nextcloud的文档推荐同时配置这二者。

在有关文件中启用所需的`igbinary`和`redis`扩展：

- `occ`命令和后台任务需要用到的 `/etc/webapps/nextcloud/php.ini`。
- 下面的文件由你的应用服务器决定
- *uWSGI*：`/etc/uwsgi/nextcloud.ini`。
- *FPM*：`/etc/php-legacy/php-fpm.d/nextcloud.conf`。

找到文件中有关启用扩展的部分，然后添加上启动`igbinary`和`redis`的两行。

注意：应当在`extension=redis`之前加载`extension=igbinary`。否则`occ`会报这个错：*/usr/lib/php-legacy/modules/redis.so: undefined symbol: igbinary_serialize*。

如果你在上述的配置文件中指定了`open_basedir` 选项，并且通过本地Unix套接字在本地使用Redis服务，那么则必须扩展允许PHP读取和写入文件的目录列表。在上面提到的文件找到有关的行，然后在其中添加由Redis创建的包含本地Unix套接字的目录，例如：`/run/redis`。

注意：在#Application server 一节中提到的示例配置文件nextcloud.ini和nextcloud.conf中已经启用了`open_basedir`。所以说，如果你使用了这些配置文件，那么必须要调整它们。

将下面的内容添加到Nextcloud配置文件中：

```
/etc/webapps/nextcloud/config/config.php
'memcache.local' => '\OC\Memcache\APCu',
'memcache.distributed' => '\OC\Memcache\Redis',
'memcache.locking' => '\OC\Memcache\Redis',
'redis' => [
     'host'     => '/run/redis/redis.sock',
     'port'     => 0,
     'dbindex'  => 0,
     'password' => '',
     'timeout'  => 1.5,
],
```

相同的，应根据需要调整`/run/redis/redis.sock`。`dbindex`, `password`和`timeout`是可选的。

如果*Redis*运行在不同的机器上：

```
/etc/webapps/nextcloud/config/config.php
'memcache.local' => '\OC\Memcache\APCu',
'memcache.distributed' => '\OC\Memcache\Redis',
'memcache.locking' => '\OC\Memcache\Redis',
'redis' => [
     'host' => 'redis-host.mysite.com',
     'port' => 6379,
],
```

你需要将`*redis-host.mysite.com*`替换成你自己的设置。

提高安全性

请参看Nextcloud官方文档和本百科中的安全条目。Nextcloud也提供了一个安全性检测工具。

同步

提示：不推荐使用自己的个人密码进行认证，而是使用所谓应用程序令牌作为认证工具，通过应用程序令牌，其它软件可以自动认证你的服务器。如此一来，如果你怀疑其中一个软件的凭据已经泄露，你只需要撤消这个受影响的应用程序令牌，而不是更改密码并将新密码重新配置到所有使用此密码的软件上。你可以使用 Nextcloud 的 Web 界面上的设置>设备及会话安全来生成新的令牌。

桌面

官方提供的客户端可以下载 nextcloud-client包。AUR中提供了它的替代版本：nextcloud-client-gitAUR。注意：Nextcloud 不支持 owncloud-client包。

桌面客户端基本上会将计算机的一个或多个目录与 Nextcloud 的文件服务中的目录同步。它可以很好地集成在桌面文件管理器中（KDE plasma 中是 Dolphin、Gnome 中是 Nautilus），在其上显示表示同步与共享状态的叠加层。文件的右键菜单都会多出一个 *Nextcloud* 选项，可以通过此选项来使用 Nextcloud 的相关功能。Nextcloud 的官方文档中专门介绍了其桌面客户端。

如果 Nextcloud 客户端未能成功与文件管理器协同，请查看 nextcloud-client包（`pacman -Qi nextcloud-client`）. 例如 Nautilus（Gnome）需要 python-nautilus包。使用（`pacman -S --asdeps`)作为依赖安装。

注意：Nextcloud 的桌面客户端提供了一个可以指示同步状态的系统托盘图标。但 GNOME 没有提供开箱即用的系统托盘图标 API。如果需要使用此功能，可以通过安装并启用 gnome-shell-extension-appindicator包。详情请看GNOME#Extensions。

Thunderbird

102 版本以后的 Thunderbird 完全支持 CalDAV 和 CardDAV——具有自动检测功能（即使不输入很长的URL也能访问日历和地址簿）。更多信息，请参阅 Nextcloud 的官方文档。

以 davfs2 挂载

如果你想使用 WebDAV 挂载 Nextcloud，请下载 davfs2包（在 davfs2 中有描述）。

使用下面的命令挂载 Nextcloud：

```
# mount -t davfs
https://your_domain/nextcloud/remote.php/dav/files/username/ /path/to/mount
```

也可以在 `/etc/fstab` 创建条目：

```
/etc/fstab
https://your_domain/nextcloud/remote.php/dav/files/username/ /path/to/mount davfs
rw,user,noauto 0 0
```

提示：如果想要实现自动挂载目录，你可以将你的用户名（及密码，可选）存储在 davfs2#Storing credentials 描述的文件中。

注意：如果不能创建、复制文件及目录，参见 davfs2#Creating/copying files not possible and/or freezes。

挂载到 GNOME Files (Nautilus)

你可以直接使用 Nautilus ('+ Other Locations')通过 WebDAV 来访问文件。使用 Nextcloud 服务器的 Web GUI 上显示的链接（例如：

`https://cloud.mysite.com/remote.php/webdav/`）然后把 `davs` 替换成 `https`。当 Nautilus 尝试连接时会询问用户名和密码。

注意：确保你已经安装了 gvfs-dnssd包，如果没有安装，也许会出现“Location is not mountable”错误。

Android

从 Google Play 或 F-Droid 上下载Nextcloud的官方应用。

启用日历及联系人同步（Android 4+）：

1. 下载DAVx5（Play Store，F-Droid）。
2. 创建一个新的 DAVdroid 账户，在*账户*选项中设置你服务器的URL（例如：`https://cloud.mysite.com`）及登录账号和密码。

注意：如前面的 Nextcloud#Web 服务器章节所示，如果你在 web 服务器上配置了适当的重定向，则不需要`/remote.php/{carddav,webdav}`这一部分。DAVdroid 会为自己找到正确的 URL。

iOS

App Store 有Nextcloud的官方应用。

提示和技巧

使用“occ”命令行工具

`occ`是一个非常有用的管理工具。详情请参看Nextcloud官方文档。你可以使用`occ`命令做许多操作，比如管理用户和配置应用。

`/usr/bin/occ`是一个方便的基于原始occ工具`/usr/share/webapps/nextcloud/occ`的包装，它自动使用默认用户（*nextcloud*）运行，使用默认的PHP可执行程序与配置文件。可以使用`NEXTCLOUD_USER`, `NEXTCLOUD_PHP`和`NEXTCLOUD_PHP_CONFIG`环境变量来指定使用别的用户、PHP可执行文件和PHP配置文件）。特别是对于使用了本内容配置和应用服务器配置自己的服务器的用户，使用`NEXTCLOUD_PHP_CONFIG`来指定自己的配置文件是必要的。即使用Next cloud指定的PHP配置文件。在本例中，将`export NEXTCLOUD_PHP_CONFIG=/etc/webapps/nextcloud/php.ini`行添加到你的`.bashrc`中。

当使用php包而非推荐的php-legacy包时你也需要设置`NEXTCLOUD_PHP`环境变量，即`export NEXTCLOUD_PHP=/usr/bin/php`也要被添加到.bashrc中。

警告：当使用php-legacy-apcu包进行缓存时，则需要在 `/etc/webapps/nextcloud/php.ini`中设置`apc.enable_cli=1`。否则`occ`命令将提示APCu的配置有误。

Pacman hook

nextcloud包自带一个pacman hook，它负责在软件包更新后自动升级Nextcloud数据库。查看 `/usr/share/doc/nextcloud/nextcloud.hook`这个文件。

只可惜这个钩子在运行`occ upgrade`命令时无条件的使用全局的`php.ini`文件。也就是说，它并不使用上文提及的账户下的环境变量`NEXTCLOUD_PHP_CONFIG`。

一个可行的解决方案是在适当的地方复制hook文件：

```
# mkdir -vp /etc/pacman.d/hooks
# cp -a /usr/share/doc/nextcloud/nextcloud.hook /etc/pacman.d/hooks/10-nextcloud.hook
```

并且修改以`Exec`开始的行:

```
/etc/pacman.d/hooks/10-nextcloud.hook
Exec = /usr/bin/runuser -u nextcloud -- /usr/bin/php-legacy --php-
ini /etc/webapps/nextcloud/php.ini /usr/share/webapps/nextcloud/occ upgrade
```

在子目录下运行Nextcloud

如果按照Web服务器一节中介绍的方法安装Nextcloud，最后需要一整个主机名才能访问Nextcloud服务。例如，`cloud.mysite.com`。如果你想通过子目录访问Nextcloud的话。例如，`www.mysite.com/nextcloud`。请尝试：

- Nginx：请查看Nextcloud的官方文档，其中对这部分内容有完整介绍。
- Apache：编辑`/etc/httpd/conf/extra/nextcloud.conf`文件，并注释掉`<VirtualHost *:80> ... </VirtualHost>`这部分内容。

注意：别忘了编辑`.well-known`URLs，它用于服务发现。更多内容请参看官方文档：Service discovery

Docker

要使用Docker运行Nextcloud，请查看Docker Hub上的Nextcloud仓库。

Office集成

对于Office集成，目前有三种解决方案：

- Collabora Online
- ONLYOFFICE
- MS Office Online Server

这三者的共同点是需要一个单独的服务器，并且你的Web服务器需要进行一些调整以将某些请求转发到Office服务。实际的集成由上面三者的Nextcloud app完成。

需要注意的是，上面的三个产品都是面向商业客户的，也就是说你必须为其付费。只有Collabora有面向开发者免费的方案。ONLYOFFICE有对于家庭部署的定价方案。

有关安装、配置和如何集成的更多信息，参看：

- Collabora online, app

- ONLYOFFICE, app
- MS Office Online Server, app

禁用应用推荐

默认情况下，Nextcloud会向新用户推荐应用，这可能会导致大量通知。要禁用此功能，请使用`occ app:disable recommendations`。

使用Calcardbackup备份日历和地址簿

可以配置calcardbackupAUR，它可以定期备份日历和地址簿。根据喜好编辑`/etc/calcardbackup/calcardbackup.conf`文件，并且启动和启用`calcardbackup.timer`。

故障排除

有关故障排除的详细信息，请参看本页面的英文版本。

参见

- Nextcloud Documentation Overview
- Nextcloud Admin Manual

网络框架

本章要解决的是在 Arch 上把 Web 框架推到能响应请求：安装来源怎么选、装好怎么配置、如何接上 Web 服务器与数据库。建议以《Ruby on Rails》为主干，篇幅最长、必读，按 RubyGems 安装（推荐）→ 配置 → Web 服务器 → 数据库推进，Mongrel 与 AUR 两段按需查阅；《Django》只讲安装、用法与参见，读完可跳。

最容易踩的坑是安装路径：Rails 给出 RubyGems 与 AUR 两条路且推荐前者，若装了 AUR 版再照 RubyGems 的命令走，版本与路径容易错位；配置与 Web 服务器两步也省不得。

32.1　Django

难度 入门 | **篇幅** 2.1 KB | **所属** 网络框架

本节将讲到

- 安装
- 用法
- 参见

Django 是遵循模型-视图-模板（MVT）架构模式的高级 Python Web 框架。

安装

安装提供最新 Python 3 支持的 python-django包 软件包。可以使用 django-docsAUR 软件包安装文档。

数据库驱动程序

Django 有不同的数据库后端可用：

- 对于 PostgreSQL 后端，请安装 python-psycopg2包 软件包。
- 如果您打算将 MySQL 数据库用作后端，请安装 python-mysqlclient包 软件包。

用法

如果要启动 Django 项目，请使用 `django-admin` 命令

```
$ django-admin startproject mysite
```

这将在当前目录中创建一个 `mysite` 目录。 它还将创建一个 `manage.py` 脚本，该脚本可让您与项目进行交互。

您可以在官方 Django 教程和 Django 文档中找到更多信息。

参见

- awesome-django - Django 应用，项目和资源的精选列表。
- Django vs Flask - Django 和 Flask 框架的比较。

32.2 Ruby on Rails

难度 进阶 | **篇幅** 14.6 KB | **所属** 网络框架

本节将讲到

- 选项A：通过RubyGems来安装 (推荐)
- 选项B：通过 AUR 安装
- 配置
- Web 服务器
- Mongrel

**这篇文章或章节的翻译质量不佳。**

原因：Only halfway translated, still references `rc.conf`（在 Talk:Ruby on Rails# 中讨论）

Ruby on Rails，通常简写为 Rails 或者 RoR，是一个 Ruby 语言的开源的 web 应用框架。它通常被 web 开发人员使用敏捷开发方法做快速开发使用。

本文档描述如何在Arch Linux下建立Ruby on Rails开发环境。

Ruby on Rails 需要首先安装 Ruby，所以阅读首先阅读那篇文章查看安装说明。

选项A：通过RubyGems来安装 (推荐)

注意：如果这个命令不是用root运行的，那么gem会把它安装到用户的家目录中。

```
# gem install rails
```

创建文档需要一会时间. 如果不想要它,加上 --no-ri --no-rdoc 参数到安装命令中.

```
# gem install rails --no-ri --no-rdoc
```

注意：如果安装完成后提示警告在PATH中没有~/.gem/ruby/2.0.0/bin,gem executables will not run，那么要使用命令export PATH="$PATH:$(ruby -rubygems -e 'puts Gem.user_dir')/bin"将rails加入环境变量，最后可以通过命令rails -v查看安装的rails版本确认rails可用。

更新gems

gem 是一个ruby模块包管理器, 类似pacman. 去更新gem, 简单的使用gem命令:

```
# gem update
```

选项B：通过 AUR 安装

警告：This is not recommended, as this might not include the latest Rails version, and additional dependencies may be introduced that may require you to run `gem install` anyway.

There is a ruby-railsAUR package available in the AUR. Note that this is not in an official repository, so you will need to build it manually.

配置

Rails 自带了一个基本的HTTP服务器 WeBrick. 你可以创建一个测试应用来测试它. 首先, 创建一个应用使用rails 命令:

```
# rails new testapp_name
```

然后开始Web服务器. 它默认监听3000端口 :

```
# cd testapp_name
# rails server
```

完成后打开浏览器输入你服务器地址和3000端口. 例如, 如果服务器在本机 :

```
在浏览器地址栏输入http://127.0.0.1:3000/
```

你将看到Rails默认欢迎界面 "Welcome aboard".

Web 服务器

Ruby On Rails 默认HTTP 服务器(WeBrick) 可以方便去测试，但是不推荐在产品中使用。以下是一些合适的选择：

Mongrel

Mongrel is a drop-in replacement for WeBrick, that can be run in precisely the same way, but offers better performance.

通过gem来安装mongrel:

```
# gem install mongrel
```

然后开始mongrel:

```
# mongrel_rails start
```

Alternatively, you can just run "ruby script/server" again, as it replaces WeBrick by default.

Generally, Mongrel is used in a production environment by running multiple instances of mongrel_rails, which are load-balanced behind an Nginx or Apache reverse proxy. However, you might find Phusion Passenger (see below) a much simpler solution for running a production environment.

Apache/Nginx (using Phusion Passenger)

Passenger also known as `mod_rails` is a module available for Nginx and Apache, that greatly simplifies setting up a Rails server environment.

Start by installing the Passenger gem:

```
# gem install passenger
```

If you are aiming to use Apache with Passenger, run:

```
# passenger-install-apache2-module
```

For Nginx:

```
# passenger-install-nginx-module
```

The installer will provide you with any additional information regarding the installation (such as installing additional libraries).

数据库

多数Web应用都需要数据库去保存数据。ActiveRecord (the ORM used by Rails to provide database abstraction) supports several database vendors, the most popular of which are MySQL, SQLite, and PostgreSQL.

SQLite

SQLite 是Ruby on Rails默认的数据库. 要去激活 SQLite, 安装ruby-sqlite3

```
# pacman -S ruby-sqlite3
```

PostgreSQL

（待补充） 安装 postgresql包.

MySQL

注意：在尝试安装 Ruby MySQL 扩展之前，你必须首先安装带有合适头文件(位于`/usr/include`) 的 MySQL(只需要安装 mysqlAUR 就可以了)。

关于如何安装 MySQL 服务器请参考 MySQL。

首先安装通过gem安装mysql模块:

```
# gem install mysql
```

你可以创建使用mysql的Rails应用使用 `-d` 参数:

```
$ rails new testapp_name -d mysql
```

然后你需要编辑 config/database.yml. Rails 为production/tests/development使用3种数据库。下面示例使用development数据库. 填写数据库，用户名，密码字段：

```
 development:
   adapter: mysql
   database: my_application_database
   username: development
   password: my_secret_password
```

Note that you do not have to actually create the database using MySQL, as this can be done via Rails with:

```
# rake db:migrate
```

如果没有错误显示, 那么表示建立正确，并且Rails可以和MySQL数据库通信了。

选项C：完美的 Rails 安装

Phusion Passenger running multiple Ruby versions.

- Arch Linux: A simple, lightweight distribution. ;)

- Nginx: A fast and lightweight **web server** with a strong focus on high concurrency, performance and low memory usage.
- Passenger (a.k.a. mod_rails or mod_rack): Supports both Apache and Nginx web servers. It makes deployment of Ruby web applications, such as those built on Ruby on Rails web framework, a breeze.
- Ruby Enterprise Edition (REE): Passenger allows Ruby on Rails applications to use about 33% less memory, when used in combination with REE.
- Ruby Version Manager (RVM): A command-line tool which allows you to easily install, manage, and work with multiple Ruby environments from interpreters to sets of gems. RVM lets you deploy each project with its own completely self-contained and dedicated environment —from the specific version of ruby, all the way down to the precise set of required gems to run your application—.
- SQLite: The default lightweight **database** for Ruby on Rails.

步骤 0: SQLite

Easy as:

```
$ sudo pacman -S sqlite
```

注意：Of course SQLite is not critical in this setup, you can use MySQL and PostgreSQL as well.

步骤 1: RVM

Make a multi-user RVM installation as specified here.

In the 'adding users to the rvm group' step, do

```
$ sudo usermod -a -G rvm http
$ sudo usermod -a -G rvm nobody
```

. http and nobody are the users related to Nginx and Passenger, respectively.

注意：Maybe adding the 'nodody' user to the 'rvm' group is not necessary.

步骤 2: Rubies

Once you have a working RVM installation in your hands, it is time to install the Ruby Enterprise Edition interpreter

```
$ rvm install ree
```

. Also take the chance to include other interpreters you want to use, like the last Ruby version

```
$ rvm install 1.9.3
```

Advice

I have found useful to delete the 'global' gemsets of the environments that have web applications. Their gems were somehow interfering with Passenger. Do not do

```
$ rvm ree do gemset delete global
$ rvm 1.9.3 do gemset delete global
```

now, but consider this later if you encounter complications.

步驟 3: Nginx with Passenger support

Do not install Nginx via pacman. This web server does not support modules as Apache, so it must be compiled from source with the functionality of *mod_rails* (Passenger). Fortunately this is straightforward thanks to the passenger gem. Get it:

```
$ rvm use ree
$ gem install passenger
```

. The gem will be put into the 'default' gemset. Now execute the following script:

```
$ rvmsudo passenger-install-nginx-module
```

. It will download the sources of Nginx, compile and install it for you. It will guide you through all the process. (The default location for Nginx is /opt/nginx.)

After completion, the script will require you to add two lines into the 'http block' at /opt/nginx/conf/nginx.conf that look like:

```
http {
  ...
  passenger_root /usr/local/rvm/gems/ree-1.8.7-2011.03/gems/passenger-3.0.9;
  passenger_ruby /usr/local/rvm/wrappers/ree-1.8.7-2011.03/ruby;
  ...
}
```

For everything that is not Ruby, use Nginx as usual to serve static pages, PHP and Python. Check the wiki page for more information.

To enable the Nnginx service by default at start-up just add `nginx` to the `DAEMONS` array in `/etc/rc.conf`:

```
DAEMONS=(ntpd syslog-ng ... nginx)
```

注意：It is possible that your Nginx installation has not come with an init script; check your /etc/rc.d/ directory for a file called *nginx*, if that is your case manually create it.

步驟 4: Gemsets and Apps

For each Rails application you should have a gemset. Suppose that you want to try RefineryCMS against BrowserCMS, two open-source Content Management Systems based on Rails. Then you should do:

```
$ rvm use ree@refinery --create
$ gem install rails -v 3.0.11
$ gem install passenger
$ gem install refinerycms refinerycms-i18n sqlite3
```

Deploy a RefineryCMS instance called *refineria*:

```
$ cd /srv/http/
$ rvmsudo refinerycms refineria
```

Again:

```
$ rvm use 1.9.3@browser --create
$ gem install passenger
$ gem install browsercms sqlite3
```

Deploy a BrowserCMS instance called *navegador*:

```
$ cd /srv/http/
$ rvmsudo browsercms demo navegador
$ cd /srv/http/navegador
$ rvmsudo rake db:install
```

Passenger for Nginx and Passenger Standalone

Observe that the passenger gem was installed three times and with different intentions; in the environments

- *ree* => for Nginx,
- *ree@refinery* => Standalone, and
- *1.9.3@browser* => Standalone.

The strategy is to combine Passenger for Nginx with Passenger Standalone. One must first identify the Ruby environment (interpreter plus gemset) that one uses the most; in this setup the REE interpreter and the default gemset were selected. One then proceeds with setting up Passenger for Nginx to use that environment. All applications that are to use a different Ruby version and/or gemset can be served separately through Passenger Standalone and hook into the main web server via a reverse proxy configuration.

步骤 5: .rvmrc 文件和所有者

This step is crucial for the correct behaviour of the setup. RVM seeks for .rvmrc files when changing folders; if it finds one, it reads it. In these files normally one stores a line like

```
rvm <ruby_version>@<gemset_name>
```

so the specified environment is set at the entrance of applications' root folder.

Create /srv/http/refineria/.rvmrc and put

```
sudo sh -c 'echo "rvm ree@refinery" > /srv/http/navegador/.rvmrc'
```

while in /srv/http/navegador/.rvmrc,

```
sudo sh -c 'echo "rvm 1.9.3@browser" > /srv/http/navegador/.rvmrc'
```

You have to enter to both application root folders now, because every first time that RVM finds a .rvmrc it asks you if you trust the given file, consequently you must validate the two files you have just created.

These files aid the programs involved to find the correct gems.

Apart, if applications' files and folders are not owned by the right user you will face database write-access problems. The use of rvmsudo produces *root*\-owned archives when generated by Rails; in the other hand, *nobody* is the user for Passenger —if you have not changed it—: who will use and should posses them. Fix this doing

```
$ sudo chown -R nobody.nobody /srv/http/refineria /srv/http/navegador
```

步骤 6: 反向代理

You have to start the Passenger Standalone web servers for your applications. So, do

```
$ cd /srv/http/refineria
$ rvmsudo passenger start --socket refineria.socket -d
```

and

```
$ cd /srv/http/navegador
$ rvmsudo passenger start --socket navegador.socket -d
```
. The first time that you run a Passenger Standalone it will perform a minor installation.

Note that you are using *unix domain* sockets instead of the commonly-used *TCP* sockets; it turns out that unix domain are significantly faster than TCP sockets.

Launch Passenger Standalone daemons at system start-up

Coming soon!

步骤 7: 部署

Once again edit /opt/nginx/conf/nginx.conf to include some vital instructions:

```
## RefineryCMS ##

upstream refineria_upstream {
    server unix:/srv/http/refineria/refineria.socket;
}

server {
    server_name refinery.domain.com;
    root /srv/http/refineria/public;
    location / {
        proxy_pass http://refineria_upstream;
        proxy_set_header Host $host;
    }
}

## BrowserCMS ##

upstream navegador_upstream {
    server unix:/srv/http/navegador/navegador.socket;
}

server {
    server_name browser.domain.com;
    root /srv/http/navegador/public;
    location / {
        proxy_pass http://navegador_upstream;
        proxy_set_header Host $host;
    }
}
```
At this point you are in conditions to run Nginx with:

```
$ sudo rc.d start nginx
```
and to access both CMSs through *refinery.domain.com* and *browser.domain.com*.

参考文献

- https://rvm.io/integration/passenger
- https://blog.phusion.nl/2010/09/21/phusion-passenger-running-multiple-ruby-versions

参见

- Ruby
- Nginx
- LAMP

- MySQL

参考文献

- Ruby on Rails

邮件

本章处理两件彼此独立的事：自建一个邮件列表，以及为邮件和数据做签名、加密与密钥维护。

建议先读《GnuPG》：篇幅最大，从安装、配置、用法一路到密钥维护、签名与 gpg-agent，属必读；其中签名与 gpg-agent 两节别跳，前者决定收信方能否验证你的信，后者没配好就得反复重输口令。《Mailman》只在确实要自己维护列表时才细读，否则可跳。最容易踩的坑是把《Mailman》当成邮件系统本身：它管的是列表的配置与运行，装完不等于能收发信。

33.1　Mailman

难度 进阶 | **篇幅** 10.7 KB | **所属** 邮件

本节将讲到

- 安装
- 配置
- 运行
- 安装
- 提示与技巧

Mailman 是一个邮件列表管理系统。它与邮件服务器结合使用。

安装

安装mailman3包软件包。

Mailman 可与Postorius（用于配置）和Hyperkitty（用于存档）结合使用。

注意：*Mailman*以`mailman`专用用户的身份运行，因此任何对`mailman`可执行文件的调用都必须以`mailman`用户的身份进行。

配置

All configuration for Mailman takes place in `/etc/mailman.cfg`. The schema explaining all possible configuration options and setting the defaults is stored in `/usr/lib/python3.8/site-packages/mailman/config/schema.cfg`.

The configuration is also accessible via the command line. Run the following command as the `mailman` user (e.g. using sudo or su):

```
[mailman]$ mailman conf
```

数据库

Mailman by default uses an SQLite database in `/var/lib/mailman/data/`, but can be configured to use MariaDB or PostgreSQL.

SQLite

The default location for the SQLite database is already reflected in the `schema.cfg` and therefore does not have to be set in Mailman's configuration.

```
/etc/mailman.cfg
[database]
url: sqlite:////var/lib/mailman/data/mailman.db
```

MariaDB

Install the python-pymysql包 package and configure a database on MariaDB.

```
/etc/mailman.cfg
[database]
class: mailman.database.mysql.MySQLDatabase
url: mysql+pymysql://myuser:mypassword@mymysqlhost/mailman?charset=utf8&use_unicode=1
```

PostgreSQL

Install the python-psycopg2包 package and create a database using PostgreSQL.

```
/etc/mailman.cfg
[database]
class: mailman.database.postgresql.PostgreSQLDatabase
url: postgres://myuser:mypassword@mypghost/mailman
```

REST API

Mailman exposes its REST API based on the settings in the `[webservice]` section of the configuration. Make sure to replace the default values for `admin_user` and `admin_pass` (do **not** use the example values below).

```
/etc/mailman.cfg
[webservice]
admin_user: my_new_admin_user_name
admin_pass: my_new_admin_user_password
```

ARC

By default DMARC and DKIM are disabled. The configuration takes place in the `[ARC]` section of the configuration file. Make sure to set necessary values and read the documentation about the defaults.

```
/etc/mailman.cfg
[ARC]
enabled: yes
authserv_id: mailserver.tld
trusted_authserv_ids: subdomain.mailserver.tld, other.mailserver.tld
privkey: /path/to/privatekey.pem
selector: test
domain: mailserver.tld
```

MTA

To connect a mail-transfer-agent (MTA), it is necessary to configure the `[mta]` section in the configuration file. Upstream documentation covers examples for postfix, exim and sendmail, but other MTAs are technically possible.

Postfix

To connect to a local postfix instance the following configuration section can be used:

```
/etc/mailman.cfg
[mta]
incoming: mailman.mta.postfix.LMTP
```

```
outgoing: mailman.mta.deliver.deliver
lmtp_host: mail.example.com
lmtp_port: 8024
smtp_host: mail.example.com
smtp_port: 25
```

The postfix configuration has to be extended to ensure compatibility (see upstream notes).

```
/etc/postfix/main.cf
[..]
recipient_delimiter = +
unknown_local_recipient_reject_code = 550
owner_request_special = no
[..]
```

Additionally, postfix needs to be made aware of mailman's transport maps. Depending on the postfix configuration these may look similar to the following.

注意： The following examples assume that existing values for `transport_maps`, `local_recipient_maps`, `relay_domains`, `virtual_mailbox_domains` and `virtual_alias_maps` are merged with the additional values.

```
/etc/postfix/main.cf
[..]
transport_maps = hash:/var/lib/mailman/data/postfix_lmtp
local_recipient_maps = hash:/var/lib/mailman/data/postfix_lmtp
relay_domains = hash:/var/lib/mailman/data/postfix_domains
[..]
```

If `postmap` is not directly accessible by mailman for creating the default hash-based alias maps, it is possible to generate regular expression based alias maps instead. To overwrite the default Python-class based configuration, mailman allows the use of a configuration file. Create the following file:

```
/etc/postfix-mailman.cfg
[postfix]
postmap_command: /usr/bin/postmap
transport_file_type: regex
```

Add the file to the `[mta]` section in mailman's configuration file.

```
/etc/mailman.cfg
[mta]
[..]
configuration: /etc/postfix-mailman.cfg
```

Afterwards the `regexp` based exports can then be used in the postfix configuration.

```
/etc/postfix/main.cf
[..]
transport_maps = regexp:/var/lib/mailman/data/postfix_lmtp
local_recipient_maps = regexp:/var/lib/mailman/data/postfix_lmtp
relay_domains = regexp:/var/lib/mailman/data/postfix_domains
[..]
```

注意： Make sure to create the new alias maps after changing their type (e.g. from `hash` to `regexp`): `[mailman]$ mailman aliases`

To connect to a local postfix instance with a virtual mail setup, first set an alias domain. Afterwards alter the respective configuration.

```
/etc/postfix/main.cf
[..]
local_recipient_maps = regexp:/var/lib/mailman/data/postfix_lmtp
transport_maps = regexp:/var/lib/mailman/data/postfix_lmtp
virtual_mailbox_domains = regexp:/var/lib/mailman/data/postfix_domains
virtual_alias_maps = regexp:/var/lib/mailman/data/postfix_vmap
```

```
[..]
```

运行

启用并启动 `mailman3.service`。

有几个systemd定时器服务，负责列表管理的各个方面：

- `mailman3-digests.timer`：用于向订阅者发送每日摘要。
- `mailman3-gatenews.timer`：每小时轮询NNTP服务器以获取新闻
- `mailman3-notify.timer`：用于每天向管理员发送待处理请求的通知。

安装

与邮件服务器集成

与 Hyperkitty 集成

Mailman does not automatically archive mails on its own. The Hyperkitty web application is used for this purpose. Based on a plugin, mailman is able to send mails to a Hyperkitty instance for archival.

Install the mailman3-hyperkitty包 package and configure `/etc/mailman-hyperkitty.cfg`.

```
/etc/mailman-hyperkitty.cfg
[general]
base_url: https://example.tld/hyperkitty/
api_key: super_secret_password
```

Afterwards, make mailman aware of the plugin:

```
/etc/mailman.cfg
[archiver.hyperkitty]
class: mailman_hyperkitty.Archiver
enable: yes
configuration: /etc/mailman-hyperkitty.cfg
```

提示与技巧

从 mailman < 3.0 迁移

Mailman 可以导入基于 mailman < 3.0 的列表数据库（`config.pck`）。以 `mailman` 用户身份（例如使用 sudo 或 su ）运行以下命令：

```
[mailman]$ mailman import21 LISTSPEC PICKLE_FILE
```

这里，`*LISTSPEC*` 表示要导入的列表的全称（例如 `list@example.com`），`*PICKLE_FILE*` 表示列表的 `config.pck` 文件路径。

注意：要导入的 mailman2 列表**必须**已经存在于目标 mailman 实例中。

REST API

Mailman 公开了REST API，可使用基于 python-mailmanclient包 的自定义工具进行连接。

参见

- Mailman 文档 - 上游文档
- Mailmanclient 文档 - 上游文档
- 连接到 Mailman - 关于mailman3-hyperkitty包的上游文档
- Mailman Suite 文档 - 整个 Mailman 套件（Mailman、Hyperkitty 和 Postorius）的（高级）上游文档

33.2 GnuPG

难度 高级 | **篇幅** 55.0 KB | **所属** 邮件

本节将讲到

- 安装
- 配置
- 用法
- 密钥维护
- 签名

根据官方网站：

: GnuPG 是完整实现了 RFC4880（即PGP）所定义的 OpenPGP 标准的自由软件。GnuPG 可以加密和签名你的数据和通讯信息，包含一个通用的密钥管理系统以及用于各种公钥目录的访问模块。GnuPG，简称 GPG，是一个易于与其它程序整合的命令行工具，拥有很多前端程序和函数库。GnuPG 还支持 S/MIME 和 Secure Shell (ssh)。

安装

提示：一般情况下，GnuPG 已作为 pacman包 等软件包的依赖而被安装。

安装软件包 gnupg包。

软件包 pinentry包 也会被同时安装，它是一些简单的 PIN 或 passphrase 输入对话框的合集，GnuPG 需要用这些对话框来输入密码。至于用哪个 pinentry 对话框，则是由 shell 脚本 `/usr/bin/pinentry` 来确定，先后顺序参考 #pinentry。

如果要使用图形界面或集成了 GnuPG 的程序，请查看加密、签名与信息隐藏软件。

配置

主目录

GnuPG 套件将密钥环和私钥存储在 GnuPG 主目录，并从中读取配置。默认路径为 `~/.gnupg`。有两种方法可以改变主目录的路径：

- 设置 `$GNUPGHOME` 环境变量。
- 使用 `--homedir` 参数，如 `$ gpg --homedir */path/to/dir*` 。

默认情况下，主目录的权限设置为 `700`，其包含的文件的权限设置为 `600`。只有目录的所有者有权读取、写入和访问文件。这是出于安全目的，不应更改。如果此目录或其中的任何文件不遵循此安全措施，您将收到有关不安全文件和主目录权限的警告。

配置文件

GnuPG 的所有行为都可以通过命令行参数进行配置。对于您希望成为默认参数的参数，可以将它们添加到相应的配置文件中：

- *gpg* 检查 `*gnupg_home*/gpg.conf`（用户）
  和 `/etc/gnupg/gpg.conf`（全局）。由于 *gpg* 是 GnuPG 的主要入口点，因此大部分感兴趣的配置都在这里。请参阅 GPG 选项获取可能的选项。
- *dirmngr*会检查`*gnupg_home*/dirmngr.conf`
  和`/etc/gnupg/dirmngr.conf`两个配置文件。*dirmngr*是由 `gpg` 内部调用的程序，用于访问 PGP 密钥服务器。请参阅Dirmngr 选项以了解可能的选项。

这两个配置文件涵盖了常见用例，但GnuPG套件中还有更多带有自己选项的辅助程序。请参阅GnuPG 手册获取详细列表。

创建所需的文件，并按照#主目录中讨论的方法设置其权限为`600`。

在这些文件中添加任何你想要的长选项。不要写两个破折号，只需写选项的名称和所需的参数。例如，要始终使GnuPG在特定路径上使用密钥环，就像使用`gpg --no-default-keyring --keyring *keyring-path* ...`调用它一样：

```
gnupg_home/gpg.conf (或 /etc/gnupg/gpg.conf)
no-default-keyring
keyring keyring-path
```

其他示例可以在#参阅中找到。

另外，pacman 在包签名验证方面使用了一组不同的配置文件。有关详细信息，请参阅Pacman/Package signing。

新用户的默认选项

要给新建用户设定一些默认选项，把配置文件放到 `/etc/skel/.gnupg/`。系统创建新用户时，就会把文件复制到 GnuPG 目录。还有一个 *addgnupghome* 命令可以为已有用户创建新 GnuPG 主目录：

```
# addgnupghome user1 user2
```

此命令会检查 `/home/user1/.gnupg/` 和 `/home/user2/.gnupg/`，并从 skeleton 目录复制文件过去。具有已存在的 GnuPG 主目录的用户只需跳过即可。

用法

注意：

- 如果需要一个 *user-id*，可以使用 key ID、指纹、用户名或电邮地址的部分等替代，GnuPG 对此的处理很灵活。
- 如果需要一个 *key-id*，可以给命令加上 `--keyid-format=long` 选项来查询。例如，如果想要查看主密匙，可以使用`gpg --list-secret-keys --keyid-format=long user-id`命令，*key-id* 是和 *sec* 同一行的十六进制散列值。

创建密钥对

用下面命令创建一个密钥对：

```
$ gpg --full-gen-key
```

使用 `--expert` 选项可以选择其它的加密算法，尤其是较新的ECC（椭圆曲线加密）。

命令执行后会需要用户回答一些问题，大部分用户应该需要的是：

- 默认的“RSA 和 RSA”用于加密和解密。
- 默认的密钥长度，即 3072。增大长度到 4096“成本极高，但获益很少”。这个帖子说明了为何 GPG 不默认使用 RSA-4096。
- 过期日期。大部分用户可以选择一年。这样即使无法访问密钥环，用户也知道密钥已经过期。如果有需要，可以不重新签发密钥就延长过期时间。
- 用户名和电子邮件。可以给同样的密钥不同的身份，比如给同一个密钥关联多个电子邮件。
- **不填写**可选注释。注释字段并没有被很好地定义，作用有限。
- 一个安全的密钥口令。可参考如何选择安全的密码。

注意：任何导入密钥的人都可以看到这里的用户名和电子邮件地址。

提示：较简单的 `--gen-key` 选项对密钥类型、密钥长度、过期时间均使用默认值，仅询问姓名和电邮地址。

查看密钥

查看公钥：

```
$ gpg --list-keys
```

查看私钥:

```
$ gpg --list-secret-keys
```

导出公钥

GPG 的主要用途是通过公钥加密信息以确保其私密性。你可以分发自己的公钥，而其他人通过该公钥加密发给你的信息。而你的私钥必须**始终**保密，否则将会威胁信息的私密性。相关内容，请参见公开密钥加密。

所以其他人需要有你的公钥才能给你发加密信息。

以下命令可生成公钥的 ASCII 版本（`--armor` 参数）（例如用于以电子邮件发布）：

```
$ gpg --export --armor --output public-key.asc user-id
```

此外，还可以通过密钥服务器分发公钥。

提示：

- 使用 `--no-emit-version` 可以避免打印版本号，通过配置文件也可以进行此设置。
- 可以省略 `user-id` 以导出密钥环内所有的公钥。这可以用来分享多个身份，或是将其导入到另一个程序，比如 Thunderbird。

导入公共密钥

要给其他人发送加密信息，或者验证他们的签名，就需要他们的公钥。通过文件 `*public.key*` 导入公钥到密钥环：

```
$ gpg --import public.key.asc
```

此外，还可以通过#密钥服务器导入公钥。

如想导入某个 key ID 以安装某个 Arch Linux 软件包，可参见 pacman 的相关说明和 makepkg 的相关说明。

使用公钥服务器

发布公钥

你可以将你的公钥注册到一个公共的密钥服务器，这样其他人不用联系你就能获取到你的公钥：

```
$ gpg --send-keys key-id
```

警告：一旦一个公钥被发送到密钥服务器，它就无法从服务器上删除。这个网页解释了原因。

注意：与公钥相关联的电邮地址一旦公开，可能会被垃圾邮件发送者盯上。请做好相应的防护措施。

搜索和接收公钥

要查询公钥的详细信息而不是导入，执行：

```
$ gpg --search-keys user-id
```

要导入一个公钥：

```
$ gpg --receive-keys key-id
```

要使用密钥服务器中的最新版本刷新/更新钥匙串：

```
$ gpg --refresh-keys
```

警告：

- 您应该通过将其指纹与所有者在独立来源（例如直接联系该人）上发布的指纹进行比较，以验证检索到的公钥的真实性。请参阅 Wikipedia:Public key fingerprint 获取更多信息。
- 接收密钥时，建议使用长密钥 ID 或完整指纹。使用短密钥 ID 可能会导致冲突。所有具有短密钥 ID 的密钥都将被导入，参见 在野外发现的伪密钥作为示例。

提示：将 `auto-key-retrieve` 添加到 GPG 配置文件中，将在需要时自动从密钥服务器获取密钥。这不会对安全性造成妥协，但可以被视为**侵犯隐私**；请参阅gpg(1)中的"web bug"。

公钥服务器

常见的公钥服务器：

- Ubuntu Keyserver：联盟式（federated）、没有验证、公钥不可删除。
- Mailvelope Keyserver：中心式、验证电邮 ID、公钥可删除。
- keys.openpgp.org：中心式、验证电邮 ID、公钥可删除、没有第三方签名（即不支持信任网络）。

维基百科（英文）#Keyserver_examples)上有更多的服务器。

备选公钥服务器可以在#配置文件中的 `keyserver` 选项中注明，例如：

```
~/.gnupg/dirmngr.conf
keyserver hkp://keyserver.ubuntu.com
```

当常规服务器无法正常工作时，临时使用另一台服务器很方便。例如，可以通过以下方法实现：

```
$ gpg --keyserver hkps://keys.openpgp.org/ --search-keys user-id
```

提示：

- 若遇到错误信息 `gpg: keyserver receive failed: General error`，且你使用了默认的 HKPS 公钥服务器池，请在 `dirmngr.conf` 中设置 HKPS 池证书： `hkp-cacert /usr/share/gnupg/sks-keyservers.netCA.pem` 然后杀掉旧的 `dirmngr` 进程。
- 若遇到错误信息 `gpg: keyserver receive failed: Connection refused`，可尝试更换 DNS 服务器。
- 你可以通过 `--use-tor` 选项或 Torsocks 来使用 Tor 连接密钥服务器。详见这个网页。

- 你可以通过设置环境变量 `http_proxy` 并在 `dirmngr.conf` 中写入 `honor-http-proxy` 以使用代理访问密钥服务器。或者，你也可以通过在 `dirmngr.conf` 中写入 `http-proxy *host[:port]*` 以直接覆盖环境变量`http_proxy`来使用代理。重启 `dirmngr.service` 用户服务以使改动生效。
- 如果连接到密钥服务器失败并显示 `gpg: keyserver receive failed: Server indicated a failure`，您可能需要将 gpg 配置为使用备用端口。例如，要在 Ubuntu 的密钥服务器上使用端口 80，请使用 `keyserver hkp://keyserver.ubuntu.com:80`。

网络公钥目录

网络公钥服务（Web Key Service，WKS）协议是公钥分发的新标准。电子邮件域将提供其自己的公钥服务器，称为网络公钥目录（Web Key Directory，WKD）。在高于 2.1.16 版本的 GPG 中，在依电子邮件地址（如`user@example.com`）加密时，如果该公钥不在本地密钥环中，GPG 将以 HTTPS 向电子邮件的域（`example.com`）查询 OpenPGP 公钥。选项 `auto-key-locate` 将在本地密钥环内无该电邮地址的公钥时，按 WKD 协议查找公钥。

```
$ gpg --recipient user@example.org --auto-key-locate --encrypt doc
```

GnuPG Wiki 举出了一些支持 WKD 的电子邮件服务商。若电邮地址域由你控制，可遵循这份指南在你的域上启用 WKD。可用这个地址检查你的公钥是否能在 WKD 中找到。

加密与解密

非对称加解密

在加密（参数`--encrypt`或`-e`）一个文件或一条信息给另外一个人（参数`--recipient`或`-r`）之前，你需要先#导入他的公钥。如果你还没有#创建自己的密钥对，请先创建。

要加密一个名为 doc 的文件：

```
$ gpg --recipient user-id --encrypt doc
```

要解密（参数 `--decrypt` 或 `-d`）一个用你的公钥加密的、名为 *doc*.gpg 的文件：

```
$ gpg --output doc --decrypt doc.gpg
```

gpg 会提示你输入密钥口令，并将 *doc*.gpg 中的数据解密到 *doc*。如果你忽略了参数 `-o`（`--output`），*gpg* 将会直接输出解密的信息。

提示：

- 使用参数 `--armor` 以 ASCII 编码的形式加密文件（适用于复制与粘贴文本文件格式的消息）。
- 使用 `-R *<user-id>*` 或 `--hidden-recipient *user-id*` 代替 `-r` 可以不将收件人的指纹 ID 放入加密的消息中。这有助于隐藏收件人的信息，是针对流量分析的一个有限对策。
- 使用 `--no-emit-version` 以避免打印版本号。也可将相应配置添加到你的配置文件中。

- 你可以使用 GPG 将自己作为收件人来加密敏感文件，但是每次只能压缩一个文件——尽管你可以将多个文件压缩后再进行加密。如果需要加密一个目录或一整个文件系统，请参见 Data-at-rest encryption#Available methods。

对称加解密

对称加密不需要生成密钥对，可用来简单地给文件加上密码。使用 `-c`/`--symmetric` 参数来进行对称加密：

```
$ gpg -c doc
```

下面的例子：

- 用口令给 `*doc*` 进行了对称加密
- 用 AES-256 加密算法对口令进行加密
- 用 SHA-512 摘要算法对口令进行打乱
- 打乱 65536 次

```
$ gpg -c --s2k-cipher-algo AES256 --s2k-digest-algo SHA512 --s2k-count 65536 doc
```

下面的命令可解密以口令对称加密的 `*doc*.gpg` 文件，并将解密的文档输出到同一目录下的 `*doc*` 文件中：

```
$ gpg --output doc --decrypt doc.gpg
```

目录操作

可用 gpgtar(1) 对目录进行加密和解密。

加密：

```
$ gpgtar -c -o dir.gpg dir
```

解密：

```
$ gpgtar -d dir.gpg
```

密钥维护

备份你的私钥

用如下命令备份你的私钥。

```
$ gpg --export-secret-keys --armor --output private-key.asc user-id
```

请注意，上述命令将要求您输入密钥的密码。这是因为，否则任何获得上述导出文件访问权限的人都可以像您一样对文档进行加密和签名，而无需知道您的密码。

警告：

- 口令通常是密钥安全方面最薄弱的环节。最好把导出的文件放在另一个系统或者设备里，比如物理保险柜或者加密驱动器中。这是当你遇到设备被盗、磁盘故障等情况时恢复对密钥控制权的唯一安全措施。

- 这种备份方式有一些安全局限性，这篇文章 https://web.archive.org/web/20210803213236/https://habd.as/post/moving-gpg-keys-privately/ 中有关于用 *gpg* 备份和导入密钥的更加安全的办法。

用如下命令导入你的私钥备份

```
$ gpg --import private-key.asc
```

提示： 你可以用 Paperkey 来把私钥导出为明文文本或条形码，并打印出来存档。

备份你的吊销证书

生成新密钥对的时候会同时生成吊销证书，默认存放在 `~/.gnupg/openpgp-revocs.d/` 下，证书的文件名是对应的密钥的指纹。 你也可以用以下命令手动生成吊销证书：

```
$ gpg --gen-revoke --armor --output revcert.asc user-id
```

如果密钥丢失或泄露，此证书可用于 #吊销密钥。如果你无法访问密钥，则无法使用上述命令生成新的吊销证书，那么备份将非常有用。吊销证书很短，你可以把他打印出来然后在需要使用的时候手动输入到电脑里。

警告：任何能接触到吊销证书的人都可以吊销你的密钥对，而且无法撤消。所以请像保护私钥一样保护你的吊销证书。

编辑你的密钥

运行 `gpg --edit-key *user-id*` 命令将会出现一个菜单，该菜单使你能够执行大部分密钥管理相关的任务。

在编辑密钥子菜单中输入 `help` 命令可以显示完整的命令列表。以下是一些有用的命令：

```
> passwd        # 修改密码短语
> clean         # 压缩任何不再可用的用户ID（例如已撤销或已过期）
> revkey        # 撤销密钥
> addkey        # 向该密钥添加子密钥
> expire        # 更改密钥过期时间
> adduid        # 添加附加的名称、注释和电子邮件地址
> addphoto      # 向密钥添加照片（必须是JPG格式，推荐大小为240x288，当提示时输入完整路径）
```

提示：如果你有多个电子邮件账户，你可以使用 `adduid` 命令将每个账户都添加为一个身份。然后你可以将你最喜欢的账户设置为 `primary`。

导出子密钥

如果您计划在多个设备上使用相同的密钥，您可能希望去除主密钥，只保留在较不安全的系统上的最低限度的加密子密钥。

首先，找出您想要导出的子密钥。

```
$ gpg --list-secret-keys --with-subkey-fingerprint
```

仅选择要导出的子密钥。

```
$ gpg -a --export-secret-subkeys [子密钥id]! > /tmp/subkey.gpg
```

警告：如果您忘记添加“!”，所有子密钥都将被导出。

到此为止，您可以停止了，但最好同时更改密码。将密钥导入临时文件夹。

```
$ gpg --homedir /tmp/gpg --import /tmp/subkey.gpg
$ gpg --homedir /tmp/gpg --edit-key 用户id
> passwd
> save
$ gpg --homedir /tmp/gpg -a --export-secret-subkeys [子密钥id]!
> /tmp/subkey.altpass.gpg
```

注意：您会收到一个警告，说明主密钥不可用且密码未更改，但可以安全地忽略，因为子密钥的密码已经更改。

此时，您现在可以在其他设备上使用 `/tmp/subkey.altpass.gpg`。

延长过期日期

警告：永远不要删除已过期或已吊销的子密钥，除非你有充分的理由。这样做将导致你无法解密使用旧子密钥加密的文件。请**仅**从其他用户那里删除已过期或已吊销的密钥以清理你的密钥环。

在你的子密钥上设置一个过期日期是一个好的做法，这样如果你失去了对密钥的访问（例如，你忘记了密码短语），密钥将不会继续被他人无限期地使用。当密钥过期时，延长过期日期相对简单：

```
$ gpg --edit-key 用户ID
> expire
```

你将被要求输入一个新的过期日期，以及用于签署新过期日期的你的密钥密码短语。

对于任何其他已过期的子密钥，重复此步骤：

```
> key 1
> expire
```

最后，保存更改并退出：

```
> save
```

将其更新到密钥服务器。

```
$ gpg --keyserver keyserver.ubuntu.com --send-keys key-id
```

或者，如果你在多台电脑上使用此密钥，你可以导出带有新签署过期日期的公钥，并在这些计算机上导入它：

```
$ gpg --export --output pubkey.gpg user-id
$ gpg --import pubkey.gpg
```

无需重新导出你的私钥或更新你的备份：主密钥本身永不过期，只需要公钥和子密钥上的过期日期的签名。

轮换子密钥

警告：永远不要删除过期或被撤销的子密钥，除非有充分的理由。这样做将导致您无法解密使用旧子密钥加密的文件。请**仅**从其他用户那里删除过期或被撤销的密钥以清理您的密钥环。

或者，如果您希望在子密钥过期后完全停止使用它们，您可以创建新的子密钥。请提前几周进行此操作，以便其他人可以更新他们的密钥环。

提示：您不需要仅因为密钥过期而创建新密钥。您可以延长过期日期，请参阅部分#延长过期日期。

创建新的子密钥（对签名和加密密钥都重复此操作）

```
$ gpg --edit-key user-id
> addkey
```

回答它询问的以下问题（参见#创建密钥对以获取建议的设置）。

保存更改

```
> save
```

将其更新到密钥服务器。

```
$ gpg --keyserver pgp.mit.edu --send-keys user-id
```

您还需要导出一份最新的私钥备份。有关如何执行此操作的详细信息，请参见#备份私钥。

提示：撤销过期的子密钥是不必要的，而且可能会被认为是不好的做法。如果您不断撤销密钥，可能会导致其他人对您缺乏信心。

吊销密钥

如果密钥被泄露、被取代、不再使用或者您忘记了密码，应该执行密钥撤销操作。这可以通过将密钥与密钥的撤销证书合并来完成。

如果您无法再访问您的密钥对，请首先导入您自己的公钥。 然后，要撤销密钥，请导入保存在撤销证书中的文件：

```
 $ gpg --import revcert.asc
```

现在，撤销操作需要公开。使用密钥服务器将撤销的密钥发送到公共 PGP 服务器（如果您以前使用过），否则，将撤销的密钥导出到文件并分发给您的通信伙伴。

签名

签名用于认证和时间戳文档。如果文档被修改，验证签名将失败。与使用公钥加密文档不同，签名是使用用户的私钥创建的。文档的接收者然后使用发送者的公钥验证签名。

创建签名

签署文件

要签署文件，请使用`-s`/`--sign`标志：

```
$ gpg --output doc.sig --sign doc
```

doc.sig包含原始文件*doc*的压缩内容和以二进制格式表示的签名，但文件并未加密。但是，您可以将签名与加密结合使用。

以可读形式签名文件或消息

要签署文件而无需将其压缩为二进制格式，请使用：

```
$ gpg --output doc.sig --clearsign doc
```

在这里，原始文件*doc*的内容和签名以可读形式存储在*doc*.sig中。

创建独立的签名文件

要创建一个单独的签名文件，以便与文档或文件本身分开分发，请使用--detach-sig标志：

```
$ gpg --output doc.sig --detach-sig doc
```

在这里，签名存储在*doc*.sig中，但*doc*的内容不会存储在其中。这种方法常用于分发软件项目，以允许用户验证程序未被第三方修改。

验证签名

要验证签名，请使用--verify标志：

```
$ gpg --verify doc.sig
```

其中*doc*.sig是包含您要验证的签名的已签名文件。

如果您要验证一个已分离签名，验证时必须同时存在已签名的数据文件和签名文件。例如，要验证 Arch Linux 的最新 iso 文件，您可以执行以下操作：

```
$ gpg --verify archlinux-version.iso.sig
```

其中archlinux-*version*.iso必须位于相同的目录中。

您还可以使用第二个参数指定已签名的数据文件：

```
$ gpg --verify archlinux-version.iso.sig /path/to/archlinux-version.iso
```

如果一个文件除了被签名外还被加密，只需解密该文件，其签名也将被验证。

gpg-agent

gpg-agent 主要用作守护进程，用于请求和缓存密钥链的密码。这在外部程序（如邮件客户端）使用 GnuPG 时十分有用。 gnupg包 带有默认自动启动的 systemd/用户套接字。这些套接字分别是 gpg-agent.socket、gpg-agent-extra.socket、gpg-agent-browser.socket、gpg-agent-ssh.socket 和 dirmngr.socket。

- *gpg* 使用 gpg-agent.socket 连接到 *gpg-agent* 守护进程。

- `gpg-agent-extra.socket` 的作用是在本地建立一个转发自远程系统的 Unix 域套接字。这样就可以在远程系统上使用 *gpg*，而无需向远程系统公开私钥。有关详细信息，请参阅 gpg-agent(1)。
- `gpg-agent-browser.socket` 允许 Web 浏览器访问 *gpg-agent* 守护进程。
- SSH 使用 `gpg-agent-ssh.socket` 缓存 *ssh-add* 程序添加的 SSH keys。有关必要的配置，请参阅 #SSH agent。
- `dirmngr.socket` 启动一个 GnuPG 守护进程来处理与 keyserver 的连接。

注意：如果您没有使用默认的 GnuPG #目录位置, 您需要编辑所有套接字文件让其使用 `gpgconf --list-dirs` 的值。 套接字名称使用非默认 GnuPG 主目录的哈希 ，您可以硬编码它不用担心它的改变。

配置

gpg-agent 用 `~/.gnupg/gpg-agent.conf` 文件配置。配置选项列在 gpg-agent(1) 中。例如，您可以更改默认密钥的缓存 ttl：

```
~/.gnupg/gpg-agent.conf
default-cache-ttl 3600
```

提示：要缓存整个会话的密码(passphrase)，请运行以下命令：

```
$ /usr/lib/gnupg/gpg-preset-passphrase --preset XXXXX
```

其中 XXXXX 是 keygrip。您可以在运行 `gpg --with-keygrip -K` 时获取它的值。密码(passphrase)将一直保存到 `gpg-agent` 重新启动为止。如果设置了 `default-cache-ttl` 值，会优先采用它。

在 Linux 中，为了允许预设的密码短语，需要通过使用 `--allow-preset-passphrase` 启动 gpg-agent，或在 `~/.gnupg/gpg-agent.conf` 中设置`allow-preset-passphrase`。

重新加载 gpg-agent

在修改完配置之后，用 *gpg-connect-agent* 重新加载 gpg-agent：

```
$ gpg-connect-agent reloadagent /bye
```

该命令应该输出 `OK`。

但是在某些情况下，只是重新启动可能不够，比如当 `keep-screen` 被添加到 gpg-agent 配置中时。在这种情况下，您首先需要终止正在进行的 gpg-agent 进程，然后按上述方法重新启动它。

pinentry

`gpg-agent` 可以在 `pinentry-program` 中设定，以便使用特定的 pinentry包 用户界面来提示用户输入(passphrase)。例如：

```
~/.gnupg/gpg-agent.conf
pinentry-program /usr/bin/pinentry-curses
```

还有其他 pinentry 程序可选，参考 `pacman -Ql pinentry | grep /usr/bin/` 的输出结果。

提示：

- 为了使用 `/usr/bin/pinentry-kwallet` 您需要安装软件包 kwalletcliAUR。
- 所有的默认 pinentry 程序（除了 `/usr/bin/pinentry-emacs`）都支持 DBus Secret Service API，它允许通过一个兼容的管理器(如 GNOME Keyring 或 KeePassXC)记住密码。

记得在修改完配置后要#重新加载 gpg-agent。

缓存密码

`max-cache-ttl` 和 `default-cache-ttl` 定义 gpg-agent 的密码缓存时间（秒）。要在会话中只输入一次密码，设置一个非常高的值即可，例如：

```
gpg-agent.conf
max-cache-ttl 60480000
default-cache-ttl 60480000
```

对于 SSH 仿真模式下的密码缓存，需要设置 `default-cache-ttl-ssh` 和 `max-cache-ttl-ssh`，例如：

```
gpg-agent.conf
default-cache-ttl-ssh 60480000
max-cache-ttl-ssh 60480000
```

Unattended passphrase

从 GnuPG 2.1.0 开始，需要使用 gpg-agent 和 pinentry，这可能会破坏使用 `--passphrase-fd 0` 命令行选项从 STDIN 传入的密码短语的向后兼容性。为了拥有与旧版本相同类型的功能，必须做两件事：

首先，编辑 gpg-agent 配置允许 *loopback* pinentry 模式：

```
~/.gnupg/gpg-agent.conf
allow-loopback-pinentry
```

如果 gpg-agent 正在运行，重新加载它使配置生效。

其次，要么应用程序需要更新，以包括一个命令行参数来使用回环模式，例如：

```
$ gpg --pinentry-mode loopback ...
```

如果不可能这样做，则可以将选项添加到配置中：

```
~/.gnupg/gpg.conf
pinentry-mode loopback
```

注意：上游作者指出，在 `gpg.conf` 中设置 `pinentry-mode loopback` 可能会破坏其他用法，如果可能，最好使用命令行选项。

SSH 代理

gpg-agent 具有 OpenSSH 代理仿真功能。如果您正在使用 GnuPG 套件，可以考虑使用 *gpg-agent* 来缓存 SSH 密钥。此外，一些用户可能更喜欢 GnuPG 代理提供的 PIN 输入对话框作为其密码(passphrase)管理的一部分。

设置 SSH_AUTH_SOCK

设置以下环境变量与 *gpg-agent* 通信，替代默认的 *ssh-agent*。

```
SSH_AGENT_PID=""
SSH_AUTH_SOCK="${XDG_RUNTIME_DIR}/gnupg/S.gpg-agent.ssh"
```

注意：

- 如果使用脚本管理环境变量,需要 unset `SSH_AGENT_PID` 而不是将 `unset SSH_AGENT_PID` 设置成 `""`。
- 请记住，在手动设置 `SSH_AUTH_SOCK` 的情况下，如果使用了自定义的 `GNUPGHOME`，那么套接字位置可能不同。您可以使用以下 bash 示例，或者将 `SSH_AUTH_SOCK` 更改为 `gpgconf --list-dirs agent-ssh-socket` 的值。
- 如果安装了 GNOME Keyring，有必要禁用其 ssh 组件。否则，它将覆盖 `SSH_AUTH_SOCK`。

或者通过 Bash 配置，这也适用于非标准的套接字路径：

```
~/.bashrc
unset SSH_AGENT_PID
if [ "${gnupg_SSH_AUTH_SOCK_by:-0}" -ne $$ ]; then
  export SSH_AUTH_SOCK="$(gpgconf --list-dirs agent-ssh-socket)"
fi
```

注意：`gnupg_SSH_AUTH_SOCK_by` 用来检测 *gpg-agent* 是否以 `gpg-agent --daemon /bin/sh` 的方式启动, 在这种情况下，shell 从父进程继承 `SSH_AUTH_SOCK` 变量, *gpg-agent* .

配置 pinentry 使用正确的 TTY

如果用户切换了 X 会话，还需要设置 GPG_TTY 并刷新 TTY，如 gpg-agent(1) 中所述。例如：

```
~/.bashrc
export GPG_TTY=$(tty)
gpg-connect-agent updatestartuptty /bye >/dev/null
```

在同时使用多个终端时，如果想要 *gpg-agent* 通过 *pinentry-curses* 在运行 ssh 的终端上请求密码(passphrase)，添加以下内容到 SSH 配置文件。这将使得每次运行 *ssh* 命令时都会刷新 TTY：

```
~/.ssh/config
Match host * exec "gpg-connect-agent UPDATESTARTUPTTY /bye"
```

请注意，必须设置 `GPG_TTY` 环境变量才能正常工作。

添加 SSH 密钥

gpg-agent 运行后，和 ssh-agent 一样您可以通过 *ssh-add* 添加准许的密钥。 准许的密钥列表储存在 `~/.gnupg/sshcontrol`。

一旦您的密钥被批准，每次需要密码(passphrase)时，都会有 *pinentry* 对话框。要缓存密码，参见 #缓存密码。

使用 PGP 密钥进行 SSH 身份验证

您还可以使用 PGP 密钥作为 SSH 密钥。这需要一个具有验证功能的密钥（请参阅 #Custom ability）。使用 PGP 密钥进行 SSH 身份验证有各种好处，包括:

- 您不再需要维护 SSH 密钥，减少了密钥维护量。
- 可以在智能卡上储存验证密钥。GnuPG 将在卡可用时自动检测密钥，并将其添加到代理中（使用 `ssh-add -l` 或 `ssh-add -L` 检查）。密钥的备注应该会是 `openpgp:*key-id*` 或 `cardno:*card-id*`。

运行 `gpg --export-ssh-key *gpg-key*`获取 GPG/SSH 公钥。如果您的密钥具有身份验证功能，但此命令仍然失败，并显示 “Unusable public key”, 给 *gpg-key* 添加 ！后缀 ()。

除非您的 GPG 密钥在密钥卡上，否则您需要将密钥添加到 `$GNUPGHOME/sshcontrol` 中，才能被识别为 SSH 密钥。如果您的钥匙在钥匙卡上，它的 keygrip 会隐式地添加到 `sshcontrol` 中。如果没有，通过以下方式获取密钥的 keygrip：

```
$ gpg --list-keys --with-keygrip
sub   rsa4096 2018-07-25 [A]
      Keygrip = 1531C8084D16DC4C36911F1585AF0ACE7AAFD7E7
```

然后像这样编辑 `sshcontrol`。添加 keygrip 只需要一次；除非要添加其他密钥，否则不需要再次编辑该文件。

```
$GNUPGHOME/sshcontrol
1531C8084D16DC4C36911F1585AF0ACE7AAFD7E7
```

转发 gpg-agent 和 ssh-agent 到远程主机

提示：设置`ForwardAgent yes`，如此处所示，怎么样?

通过将 gpg 套接字转发到远程机器，可以将 gpg-agent 转发到远程计算机，如 GnuPG wiki 所述。

首先，将以下行添加到远程计算机上的 `/etc/ssh/sshd_config`，以便在连接时自动删除旧的套接字。如果不这样做，则需要手动删除远程计算机上的套接字，然后才能连接启用了转发的代理，以便代理转发正常工作：

```
/etc/ssh/sshd_config
...
StreamLocalBindUnlink yes
...
```

注意：必须在远程计算机上运行 `systemctl reload sshd`，使 sshd 加载新配置。

在客户端上，用 `RemoteForward` SSH 指令来将发往远程端口的通信转发到本地端口。如 ssh_config(5) § RemoteForward 中所述，该指令参数的第一项是远程主机上监听套接字的路径，第二项是本地主机上目标套接字的路径。配置应该如下所示：

```
$HOME/.ssh/config
Host remote_name
    ...
    RemoteForward remote_agent_socket local_agent_extra_socket
    RemoteForward remote_agent_ssh_socket local_agent_ssh_socket
```

第一行配置 gpg-agent 转发 :

- *remote_agent_socket* 是在远程机器上 `gpgconf --list-dir agent-socket` 的输出。
- *local_agent_extra_socket* 是在本地机器上 `gpgconf --list-dir agent-extra-socket` 的输出。

第二行是可选的，配置 ssh-agent 转发:

- *local_agent_ssh_socket* 是在远程机器上 `gpgconf --list-dir agent-ssh-socket` 的输出。
- *remote_agent_ssh_socket* 是在本地机器上 `gpgconf --list-dir agent-ssh-socket` 的输出。

注意：如果要使用 ssh-agent 转发，需要在远程机器上将 `SSH_AUTH_SOCK` 设为 `gpgconf --list-dir agent-ssh-socket` 的输出，如#SSH 代理所述）。

因此，在默认路径下，它应该是：

```
    RemoteForward /run/user/1000/gnupg/S.gpg-agent /run/user/1000/gnupg/S.gpg-agent.extra
    RemoteForward /run/user/1000/gnupg/S.gpg-agent.ssh /run/user/1000/gnupg/S.gpg-agent.ssh
```

有了这个配置，运行 `ssh myremote` 会自动将 gpg-agent 转发到远程，并允许使用 gpg 密钥解密/签名（如果包含第二行 `RemoteForward`，则会允许 gpg 和 ssh-agent 一起工作）。

Smartcards

**本文或本节需要翻译#翻译)。要贡献翻译，请访问简体中文翻译团队。**

附注： 以下内容缺少中文翻译（在 Talk:GnuPG# 中讨论）

GnuPG uses *scdaemon* as an interface to your smartcard reader, please refer to the man page scdaemon(1) for details.

GnuPG only setups

注意： To allow scdaemon direct access to USB smartcard readers the optional dependency libusb-compat包 must be installed

If you do not plan to use other cards but those based on GnuPG, you should check the `reader-port` parameter in `~/.gnupg/scdaemon.conf`. The value '0' refers to the first available serial port reader and a value of '32768' (default) refers to the first USB reader.

GnuPG with pcscd (PCSC Lite)

pcscd(8) is a daemon which handles access to smartcard (SCard API). If GnuPG's scdaemon fails to connect the smartcard directly (e.g. by using its integrated CCID support), it will fallback and try to find a smartcard using the PCSC Lite driver.

To use pscsd install pcsclite包 and ccid包. Then start and/or enable `pcscd.service`. Alternatively start and/or enable `pcscd.socket` to activate the daemon when needed.

Always use pcscd

If you are using any smartcard with an opensc driver (e.g.: ID cards from some countries) you should pay some attention to GnuPG configuration. Out of the box you might receive a message like this when using `gpg --card-status`

```
gpg: selecting openpgp failed: ec=6.108
```

By default, scdaemon will try to connect directly to the device. This connection will fail if the reader is being used by another process. For example: the pcscd daemon used by OpenSC. To cope with this situation we should use the same underlying driver as opensc so they can work well together. In order to point scdaemon to use pcscd you should remove `reader-port` from `~/.gnupg/scdaemon.conf`, specify the location to `libpcsclite.so` library and disable ccid so we make sure that we use pcscd:

```
~/.gnupg/scdaemon.conf
pcsc-driver /usr/lib/libpcsclite.so
card-timeout 5
disable-ccid
```

Please check scdaemon(1) if you do not use OpenSC.

Shared access with pcscd

GnuPG `scdaemon` is the only popular `pcscd` client that uses `PCSC_SHARE_EXCLUSIVE` flag when connecting to `pcscd`. Other clients like OpenSC PKCS#11 that are used by browsers and programs listed in Electronic identification are using `PCSC_SHARE_SHARED` that allows simultaneous access to single smartcard. `pcscd` will not give exclusive access to smartcard while there are other clients connected. This means that to use GnuPG smartcard features you must before have to close all your open browser windows or do some other inconvenient operations. Starting from version 2.2.28 LTS and 2.3.0 you can enable shared

access by modifying your `scdaemon.conf` file and adding `pcsc-shared` line end of it.

Multi applet smart cards

When using YubiKeys or other multi applet USB dongles with OpenSC PKCS#11 may run into problems where OpenSC switches your Yubikey from OpenPGP to PIV applet, breaking the `scdaemon`.

You can hack around the problem by forcing OpenSC to also use the OpenPGP applet. Open `/etc/opensc.conf` file, search for Yubikey and change the `driver = "PIV-II";` line to `driver = "openpgp";`. If there is no such entry, use `pcsc_scan`. Search for the Answer to Reset `ATR: 12 34 56 78 90 AB CD ...`. Then create a new entry.

```
/etc/opensc.conf
...
card_atr 12:23:34:45:67:89:ab:cd:... {
    name = "YubiKey Neo";
    driver = "openpgp"
}
...
```

After that you can test with `pkcs11-tool -O --login` that the OpenPGP applet is selected by default. Other PKCS#11 clients like browsers may need to be restarted for that change to be applied.

Using a smart card on a remote client via SSH

If you log into a machine via SSH and try to use an attached device via pcscd, you will notice errors such as:

```
gpg: selecting card failed: No such device
gpg: OpenPGP card not available: No such device
```

This is due to Polkit restricting access to local clients. To fix this, you can add a rule to allow certain users in all cases. The below rule allows all users in the `wheel` group to access devices via `pcscd`:

```
/etc/polkit-1/rules.d/99-pcscd.rules
polkit.addRule(function(action, subject) {
    if (action.id == "org.debian.pcsc-lite.access_card" &&
        subject.isInGroup("wheel")) {
        return polkit.Result.YES;
    }
});
polkit.addRule(function(action, subject) {
    if (action.id == "org.debian.pcsc-lite.access_pcsc" &&
        subject.isInGroup("wheel")) {
        return polkit.Result.YES;
    }
});
```

After creating the file, make sure to restart `polkit.service`.

Tips and tricks

Different algorithm

You may want to use stronger algorithms:

```
~/.gnupg/gpg.conf
```

```
...

personal-digest-preferences SHA512
cert-digest-algo SHA512
default-preference-list SHA512 SHA384 SHA256 SHA224 AES256 AES192 AES CAST5 ZLIB BZIP2
ZIP Uncompressed
personal-cipher-preferences TWOFISH CAMELLIA256 AES 3DES
```

In the latest version of GnuPG, the default algorithms used are SHA256 and AES, both of which are secure enough for most people. However, if you are using a version of GnuPG older than 2.1, or if you want an even higher level of security, then you should follow the above step.

Encrypt a password

It can be useful to encrypt some password, so it will not be written in clear on a configuration file. A good example is your email password.

First create a file with your password. You **need** to leave **one** empty line after the password, otherwise gpg will return an error message when evaluating the file.

Then run:

```
$ gpg -e -a -r user-id your_password_file
```

`-e` is for encrypt, `-a` for armor (ASCII output), `-r` for recipient user ID.

You will be left with a new `*your_password_file*.asc` file.

提示：pass automates this process.

Change trust model

By default GnuPG uses the Web of Trust as the trust model. You can change this to Trust on first use by adding `--trust-model=tofu` when adding a key or adding this option to your GnuPG configuration file. More details are in this email to the GnuPG list.

Hide all recipient id's

By default the recipient's key ID is in the encrypted message. This can be removed at encryption time for a recipient by using `hidden-recipient *user-id*`. To remove it for all recipients add `throw-keyids` to your configuration file. This helps to hide the receivers of the message and is a limited countermeasure against traffic analysis (i.e. using a little social engineering, anyone who is able to decrypt the message can check whether one of the other recipients is the one they suspect). On the receiving side, it may slow down the decryption process because all available secret keys must be tried (e.g. with `--try-secret-key *user-id*`).

Using caff for keysigning parties

To allow users to validate keys on the keyservers and in their keyrings (i.e. make sure they are from whom they claim to be), PGP/GPG uses the Web of Trust. Keysigning parties allow users to get together at a physical location to validate keys. The Zimmermann-Sassaman key-signing protocol is a way of making these very effective. Here you will find a how-to article.

For an easier process of signing keys and sending signatures to the owners after a keysigning party, you can use the tool *caff*. It can be installed from the AUR with the package caff-gitAUR.

To send the signatures to their owners you need a working MTA. If you do not have already one, install msmtp.

Always show long ID's and fingerprints

To always show long key ID's add `keyid-format 0xlong` to your configuration file. To always show full fingerprints of keys, add `with-fingerprint` to your configuration file.

Custom capabilities

For further customization also possible to set custom capabilities to your keys. The following capabilities are available:

- Certify (only for master keys) - allows the key to create subkeys, mandatory for master keys.
- Sign - allows the key to create cryptographic signatures that others can verify with the public key.
- Encrypt - allows anyone to encrypt data with the public key, that only the private key can decrypt.
- Authenticate - allows the key to authenticate with various non-GnuPG programs. The key can be used as e.g. an SSH key.

It is possible to specify the capabilities of the master key, by running:

```
$ gpg --full-generate-key --expert
```

And select an option that allows you to set your own capabilities.

Comparably, to specify custom capabilities for subkeys, add the `--expert` flag to `gpg --edit-key`, see #Edit your key for more information.

Troubleshooting

su

When using `pinentry`, you must have the proper permissions of the terminal device (e.g. `/dev/tty1`) in use. However, with *su* (or *sudo*), the ownership stays with the original user, not the new one. This means that pinentry will fail with a `Permission denied` error, even as root. If this happens when attempting to use ssh, an error like `sign_and_send_pubkey: signing failed: agent refused operation` will be returned. The fix is to change the permissions of the device at some point before the use of pinentry (i.e. using gpg with an agent). If doing gpg as root, simply change the ownership to root right before using gpg:

```
# chown root /dev/ttyN  # where N is the current tty
```

and then change it back after using gpg the first time. The equivalent is true with `/dev/pts/`.

注意：The owner of tty *must* match with the user for which pinentry is running. Being part of the group `tty` **is not** enough.

提示：If you run gpg with `script` it will use a new tty with the correct ownership:

```
# script -q -c "gpg --gen-key" /dev/null
```

Agent complains end of file

If the pinentry program is `/usr/bin/pinentry-gnome3`, it needs a DBus session bus to run properly. See General troubleshooting#Session permissions for details.

Alternatively, you can use a variety of different options described in #pinentry.

KGpg configuration permissions

There have been issues with kgpg包 being able to access the `~/.gnupg/` options. One issue might be a result of a deprecated *options* file, see the bug report.

GNOME on Wayland overrides SSH agent socket

For Wayland sessions, `gnome-session` sets `SSH_AUTH_SOCK` to the standard gnome-keyring socket, `$XDG_RUNTIME_DIR/keyring/ssh`. This overrides any value set elsewhere.

See GNOME/Keyring#Disabling on how to disable this behavior.

mutt

Mutt might not use *gpg-agent* correctly, you need to set an environment variable `GPG_AGENT_INFO` (the content does not matter) when running mutt. Be also sure to enable password caching correctly, see #Cache passwords.

See this forum thread.

"Lost" keys, upgrading to gnupg version 2.1

When `gpg --list-keys` fails to show keys that used to be there, and applications complain about missing or invalid keys, some keys may not have been migrated to the new format.

Please read GnuPG invalid packet workaround. Basically, it says that there is a bug with keys in the old `pubring.gpg` and `secring.gpg` files, which have now been superseded by the new `pubring.kbx` file and the `private-keys-v1.d/` subdirectory and files. Your missing keys can be recovered with the following commands:

```
$ cd
$ cp -r .gnupg gnupgOLD
$ gpg --export-ownertrust > otrust.txt
$ gpg --import .gnupg/pubring.gpg
$ gpg --import-ownertrust otrust.txt
$ gpg --list-keys
```

gpg hanged for all keyservers (when trying to receive keys)

If gpg hanged with a certain keyserver when trying to receive keys, you might need to kill dirmngr in order to get access to other keyservers which are actually working, otherwise it might keeping hanging for all of them.

Smartcard not detected

Your user might not have the permission to access the smartcard which results in a `card error` to be thrown, even though the card is correctly set up and inserted.

One possible solution is to add a new group `scard` including the users who need access to the smartcard.

Then use udev rules, similar to the following:

```
/etc/udev/rules.d/71-gnupg-ccid.rules
ACTION=="add", SUBSYSTEM=="usb", ENV{ID_VENDOR_ID}=="1050", ENV{ID_MODEL_ID}=="0116|0111", MODE="660", GROUP="scard"
```

One needs to adapt VENDOR and MODEL according to the `lsusb` output, the above example is for a YubikeyNEO.

server 'gpg-agent' is older than us (x < y)

This warning appears if `gnupg` is upgraded and the old gpg-agent is still running. Restart the *user*'s `gpg-agent.socket` (i.e., use the `--user` flag when restarting).

IPC connect call failed

Make sure `gpg-agent` and `dirmngr` are not running with `killall gpg-agent dirmngr` and the `$GNUPGHOME/crls.d/` folder has permission set to `700`.

By default, the gnupg包 package uses the directory `/run/user/$UID/gnupg/` for sockets. GnuPG documentation states this is the preferred directory (not all file systems are supported for sockets). Validate that your `agent-socket` configuration specifies a path that has an appropriate file system. You can find the your path settings for `agent-socket` by running `gpgconf --list-dirs agent-socket`.

Test that `gpg-agent` starts successfully with `gpg-agent --daemon`.

Mitigating Poisoned PGP Certificates

In June 2019, an unknown attacker spammed several high-profile PGP certificates with tens of thousands (or hundreds of thousands) of signatures (CVE-2019-13050) and uploaded these signatures to the SKS keyservers. The existence of these poisoned certificates in a keyring causes gpg to hang with the following message:

```
gpg: removing stale lockfile (created by 7055)
```

Possible mitigation involves removing the poisoned certificate as per this blog post.

Invalid IPC response and Inappropriate ioctl for device

The default pinentry program is `/usr/bin/pinentry-gtk-2`. If gtk2包 is unavailable, pinentry falls back to `/usr/bin/pinentry-curses` and causes signing to fail:

```
gpg: signing failed: Inappropriate ioctl for device
gpg: [stdin]: clear-sign failed: Inappropriate ioctl for device
```

You need to set the `GPG_TTY` environment variable for the pinentry programs `/usr/bin/pinentry-tty` and `/usr/bin/pinentry-curses`.

```
$ export GPG_TTY=$(tty)
```

Keyblock resource does not exist

If you get an error like this when trying to import keys

```
gpg: keyblock resource 'gnupg_home/pubring.kbx': No such file or directory
```

it is because GnuPG will not create its home directory if it does not yet exist. Simply create it manually

```
$ mkdir -m 700 gnupg_home
```

参见

- GNU Privacy Guard Homepage
- 扫盲文件完整性校验——关于散列值和数字签名 @ 编程随想的博客
- Alan Eliasen's GPG Tutorial
- RFC 4880 — "OpenPGP Message Format"
- gpg.conf recommendations and best practices
- Fedora:Creating GPG Keys
- Debian:Subkeys
- Protecting code integrity with PGP
- A more comprehensive gpg Tutorial
- /r/GPGpractice - a subreddit to practice using GnuPG.

FTP 服务器

把一台 Arch 机器变成可供远端读写的 FTP 服务器，是本章要解决的问题。内容只围绕 Very Secure FTP Daemon 展开，从安装、配置走到小技巧、问题解决与更多资源。只有一节，建议顺序读完：安装与配置必读，是后续一切操作的前提；小技巧可按需取用；问题解决与更多资源可先跳过，等真正部署卡住再回头查。最容易踩的误解是"装上、起了服务就完事"——默认配置未必合你的用途，本地用户与匿名访问的取舍、目录与权限的边界都要在配置里逐项落实，跳过这步直接试连，多半只换来一句含糊的登录失败。

34.1　Very Secure FTP Daemon

难度 进阶 | **篇幅** 16.5 KB | **所属** FTP 服务器

本节将讲到

- 安装
- 配置
- 小技巧
- 问题解决
- 更多资源

vsftpd ("Very Secure FTP Daemon") 是一个为 UNIX 类系统开发的轻量，稳定和安全的 FTP 服务器端。

安装

安装 vsftpd包 然后启动/启用 `vsftpd.service` 守护程序。

使用 xinetd 监视和控制 vsftpd 连接，请参见 #使用 xinetd。

配置

vsftpd 的大多数配置都可以通过编辑 `/etc/vsftpd.conf` 文件实现。 该文件本身自带大量注释说明，所以这一章节只就一些重要的配置予以说明。有关所有可用选项和文档，请参阅 vsftpd.conf(5) 手册页。默认情况下，由 `/srv/ftp` 提供文件。

允许的连接 `/etc/hosts.allow`：

```
# 允许所有连接
vsftpd: ALL
# IP 地址范围
vsftpd: 10.0.0.0/255.255.255.0
```

允许上传

需要将 `/etc/vsftpd.conf` 中的 `write_enable` 值设为 YES，以便允许修改文件系统（如上传）：

```
write_enable=YES
```

本地用户登录

需要修改 `/etc/vsftpd.conf` 中的如下值，以便允许 `/etc/passwd` 中的用户登录:

```
local_enable=YES
```

匿名用户登录

以下行控制匿名用户是否可以登录。默认情况下，匿名登录只能从 `/srv/ftp` 下载：

```
/etc/vsftpd.conf
...
# 允许匿名 FTP？ （当心 - 默认情况下允许除非您将其注释）。
anonymous_enable=YES
...
# 取消注释以允许匿名 FTP 用户上传文件。
# 只有上述全局写入被激活，才会有效果。
# 另外，你显然需要创建 FTP 用户有写权限的目录。
# anon_upload_enable=YES
#
# 如果您希望匿名 FTP 用户能够创建新目录，请取消注释。
# anon_mkdir_write_enable=YES
...
```

您也可以添加以下选项（详见 vsftpd.conf(5)）：

```
/etc/vsftpd.conf
# 匿名登录不需要密码
no_anon_password=YES

# 匿名客户端的最大传输速率（以 Bytes/秒 为单位）
anon_max_rate=30000

# 用于匿名登录的目录
anon_root=/example/directory/
```

Chroot 限制

为了阻止用户离开家目录，可以设置 chroot 环境。要启用此功能，请在 `/etc/vsftpd.conf` 中添加以下行：

```
chroot_list_enable=YES
chroot_list_file=/etc/vsftpd.chroot_list
```

`chroot_list_file` 定义了被 chroot 限制的用户列表。

对于更严格的环境，请指定以下行：

```
chroot_local_user=YES
```

这将默认为所有用户启用 chroot 环境。 在这种情况下，`chroot_list_file` 定义了不受 chroot 限制的用户列表。

限制用户登录

可以通过在 `/etc/vsftpd.conf` 中添加以下两行来阻止特定用户登录FTP服务器：

```
userlist_enable=YES
userlist_file=/etc/vsftpd.user_list
```

`userlist_file` 文件列出**不允许**登录的用户。

如果您只想允许特定的用户登录，请添加以下行：

```
userlist_deny=NO
```

此时 `userlist_file` 文件指定**允许**登录的用户。

限制连接数

可以通过在 `/etc/vsftpd.conf` 中添加如下信息来限制本地用户的数据传输速率：

```
local_max_rate=1000000 # 最大数据传输速率（单位：Bytes/秒）
max_clients=50 # 可以同时连接的最大客户端数
max_per_ip=2 # 每个 IP 允许的最大连接数
```

使用 xinetd

Xinetd 提供增强的监控和控制连接功能。对于基本的可以工作的 vsftpd-server 是不必要的

安装 vsftpd 将会添加必要的服务文件 `/etc/xinetd.d/vsftpd`，默认情况下，服务被禁用。

启用ftp服务：

```
service ftp
{
        socket_type             = stream
        wait                    = no
        user                    = root
        server                  = /usr/sbin/vsftpd
        log_on_success  += HOST DURATION
        log_on_failure  += HOST
        disable                 = no
}
```

如果您将 vsftpd 守护程序设置为以独立模式运行，而不是启动 vsftpd 守护程序和启用 `xinetd.service`，请在 `/etc/vsftpd.conf` 中进行以下更改：

```
listen=NO
```

否则连接将失败：

```
500 OOPS: could not bind listening IPv4 socket
```

使用 SSL/TLS 来保护 FTP

本文或本章节的语言、语法或风格需要改进。参考：帮助:风格

原因：Do not duplicate OpenSSL#Certificates.（在Talk:Very Secure FTP Daemon讨论）

首先，您需要一个 *X.509 SSL/TLS* 证书才能使用TLS。如果您没有，可以轻松生成如下的自签名证书：

```
# cd /etc/ssl/certs
# openssl req -x509 -nodes -days 7300 -newkey rsa:2048 -keyout vsftpd.pem -out
vsftpd.pem
# chmod 600 vsftpd.pem
```

你会被问到关于你公司的问题，等等。由于你的证书不是一个可信任的，所填的内容并不重要，它只会被用于加密。要使用可信证书，您可以从 Let's Encrypt 这样的证书颁发机构获得证书。

然后，编辑配置文件：

```
/etc/vsftpd.conf
ssl_enable=YES

# 如果你接受匿名连接你可能要启用这个设置
# allow_anon_ssl=NO

# 默认情况下所有非匿名登录被迫使用 SSL 发送和接收密码和数据，设置为 NO，以允许不安全的连接
force_local_logins_ssl=NO
force_local_data_ssl=NO

# 选择你想要的
force_local_logins_ssl=YES

# TLS v1 协议连接是首选，默认情况下启用此模式，而禁用 SSL v2 和 v3 时，以下设置是默认设置，不需要更改，除非您特别需要 SSL
# ssl_tlsv1=YES
# ssl_sslv2=NO
# ssl_sslv3=NO
# 提供您的证书和私钥注释的路径，它们可以包含在同一个文件中或不同的文件中
rsa_cert_file=/etc/ssl/certs/vsftpd.pem
rsa_private_key_file=/etc/ssl/certs/vsftpd.pem

# 此设置默认设置为 YES，并要求所有数据连接都显示会话重用，这证明他们知道控制通道的秘密。这种方式更安全，但不受许多 FTP 客户端的支持，为了更好的兼容性而设置为 NO
require_ssl_reuse=NO
```

在被动模式下解析主机名

要覆盖 vsftpd 在被动模式下通过服务器的主机名发布的 IP 地址，并在启动时解析 DNS，在 `/etc/vsftpd.conf` 中增加以下两行：

```
pasv_addr_resolve=YES
pasv_address=yourdomain.org
```

注意：

- 对于动态DNS，定期更新 *pasv_address* 并重新启动服务器是不必要的，因为它有时可以被读取。
- 您可能无法通过 LAN 以被动模式连接，在这种情况下，请尝试使用主动模式而不是LAN客户端。

端口配置

可能需要调整默认FTP侦听端口和被动模式数据端口：

- 对于暴露于 Web 的 FTP 服务器，为了减少服务器受到攻击的可能性，可以将侦听端口改为除标准端口 21 以外的端口。
- 要限制被动模式将打开的端口，可以提供一个范围。

这些端口配置更改可以使用以下几行完成：

```
/etc/vsftpd.conf
listen_port=2211
pasv_min_port=5000
pasv_max_port=5003
```

配置 iptables

通常，运行FTP守护进程的服务器受 iptables 防火墙的保护。要允许访问FTP服务器，需要打开相应的端口，如：

```
# iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 21 -j ACCEPT
```

本文不会提供有关如何设置iptables的任何说明，但这里是一个例子：Simple stateful firewall。

有一些内核模块需要在这里引用的 iptables 正确的处理 FTP 连接。 其中特别是 *nf_conntrack_ftp*。这是需要的，因为 FTP 使用给定的 *listen_port*（默认为21） 仅用于命令，所有的数据传输都是通过不同的端口完成的。这些端口由 FTP 守护程序为每个会话随机选择（也取决于是使用主动还是被动模式）。要告诉 iptables 应该接受端口上的数据包，需要 *nf_conntrack_ftp*。要在启动时自动加载，请在 `/etc/modules-load.d` 中创建新文件，例如：

```
# echo nf_conntrack_ftp > /etc/modules-load.d/nf_conntrack_ftp.conf
```

如果内核 >= 4.7，您需要通过 *sysctl* 设置 *net.netfilter.nf_conntrack_helper = 1*，如：

```
# echo net.netfilter.nf_conntrack_helper=1 > /etc/sysctl.d/70-conntrack.conf
```

或使用

```
# iptables -A PREROUTING -t raw -p tcp --dport 21 -j CT --helper ftp
```

小技巧

虚拟用户的 PAM 认证

由于 PAM 不再提供 `pam_userdb.so` 另一个简单的方法是使用 libpam_pwdfileAUR。但是，本节仅限于解释如何通过 `pam_pwdfile.so` 配置 chroot 环境和身份验证。

在此示例中，我们创建目录 `vsftpd`：

```
# mkdir /etc/vsftpd
```

创建和存储用户名和密码的一个选择是使用 Apache 的生成器 htpasswd：

```
# htpasswd -c /etc/vsftpd/.passwd
```

上述命令的一个问题是 vsftpd 可能无法读取生成的 MD5 散列密码。如果使用 -d 开关运行相同的命令，则使用 crypt() 加密，vsftpd 的密码变得可读，但其缺点是安全性较低，密码限制为8个字符。 Openssl 可用于使用算法1生成基于 MD5 的 BSD 密码：

```
# openssl passwd -1
```

无论那个解决方案 `/etc/vsftpd/.passwd` 应该如下所示：

```
username1:hashed_password1
username2:hashed_password2
...
```

接下来，您需要使用 `pam_pwdfile.so` 和生成的 `/etc/vsftpd/.passwd` 文件创建 PAM 服务。在本例中，我们为 *vsftpd* 创建了一个 PAM 策略，其中包含以下内容：

```
/etc/pam.d/vsftpd
auth required pam_pwdfile.so pwdfile /etc/vsftpd/.passwd
account required pam_permit.so
```

现在是为虚拟用户创建一个 home 的时候了。在示例中，决定用 `/srv/ftp` 为虚拟用户托管数据，这也反映了 Arch 的默认目录结构。首先创建 virtual 用户并使 `/srv/ftp` 成为它的 home：

```
# useradd -d /srv/ftp virtual
```

让 virtual 成为所有者

```
# chown virtual:virtual /srv/ftp
```

一个基本的没有私人文件夹配置的 /etc/vsftpd，默认的虚拟用户的 home 文件夹:

```
# 指向正确的 PAM 服务文件
pam_service_name=vsftpd
write_enable=YES
hide_ids=YES
listen=YES
connect_from_port_20=YES
anonymous_enable=NO
local_enable=YES
dirmessage_enable=YES
xferlog_enable=YES
chroot_local_user=YES
guest_enable=YES
guest_username=virtual
virtual_use_local_privs=YES
```

您自己的设置可能不需要一些参数。如果您希望 chroot 环境可写，您将需要将以下内容添加到配置文件中：

```
allow_writeable_chroot = YES
```

否则 vsftpd 的默认安全设置会抱怨，如果它检测到 chroot 是可写的。

Start `vsftpd.service`。

您现在应该可以使用存储在 `/etc/vsftpd/.passwd` 中的任何用户和密码从FTP客户端登录。

为虚拟用户创建私有目录

首先创建用户文件夹

```
# mkdir /srv/ftp/user1
# mkdir /srv/ftp/user2
# chown virtual:virtual /srv/ftp/user?/
```

随后, 在 `/etc/vsftpd.conf` 增加如下行:

```
local_root=/srv/ftp/$USER
user_sub_token=$USER
```

问题解决

vsftpd: Error 500 with kernel 4.18+

seccomp 在 vsftpd 中默认激活，这导致某些内核版本出现兼容性问题。这已经修复了，你可以在 Red Hat Bugzilla＃845980 中看到，但在较新的内核中仍然可能导致问题。

如果在列出目录时遇到故障，将其添加到 `/etc/vsftpd.conf`：

```
seccomp_sandbox=NO
```

vsftpd: refusing to run with writable root inside chroot()

从 vsftpd 2.3.5 开始，用户被锁定的 chroot 目录不可写。这是为了防止安全漏洞。

允许上传的安全方法是保持 chroot 启用，并配置您的 FTP 目录。

```
local_root=/srv/ftp/user
# mkdir -p /srv/ftp/user/upload
#
# chmod 550 /srv/ftp/user
# chmod 750 /srv/ftp/user/upload
```

如果你必须这么做：

您可以将其放入您的 `/etc/vsftpd.conf` 以解决此安全性增强（自vsftpd 3.0.0；来自 Fixing 500 OOPS: vsftpd: refusing to run with writable root inside chroot ())：

```
allow_writeable_chroot=YES
```

FileZilla Client: GnuTLS error -8 -15 -110 when connecting via SSL

vsftpd 尝试在 SSL 会话中显示纯文本错误消息。为了进行调试，暂时禁用加密，您将看到正确的错误消息。

Often these errors can be solved by adding: `seccomp_sandbox=NO`

vsftpd.service fails to run on boot

如果您启用了 `vsftpd.service`，并且无法在启动时运行，请确保它已在服务文件中设置为在 `network.target` 之后加载：

```
/usr/lib/systemd/system/vsftpd.service
[Unit]
Description=vsftpd daemon
After=network.target
```

Passive mode replies with the local IP address to a remote connection

如果 vsftpd 将本地地址返回给远程连接，如：

```
227 Entering Passive Mode (192,168,0,19,192,27).
```

It may be that the FTP server is behind a NAT router and while some devices monitor FTP connections and dynamically replace the IP address specification for packets containing the PASV response, some do not. 可能是 FTP 服务器位于 NAT 路由器后面，而有些设备会监视 FTP 连接并动态替换包含 PASV 响应的数据包的 IP 地址规范，但有些设备却不会。

使用以下命令在 vsftpd 配置中指示外部IP地址：

```
 pasv_address=externalIPaddress
```

或者可选地：

```
pasv_addr_resolve=YES
pasv_address=my.domain.name
```

如果在更改后无法进行内部连接，则可能需要运行 2 个 vsftpd，一个用于内部连接，一个用于外部连接。

提示：要找出 NAT 路由器是否拦截 PASV 响应并用外部 IP 替换内部 IP，可以在 TLS 模式下检查客户端的服务器响应。加密的数据包不能被路由器识别并且未被修改。

ipv6 only fails with: 500 OOPS: run two copies of vsftpd for IPv4 and IPv6

你很有可能已经注释了这一行

```
# When "listen" directive is enabled, vsftpd runs in standalone mode and
# listens on IPv4 sockets. This directive cannot be used in conjunction
# with the listen_ipv6 directive.
#listen=YES
#
# This directive enables listening on IPv6 sockets. To listen on IPv4 and IPv6
# sockets, you must run two copies of vsftpd with two configuration files.
# Make sure, that one of the listen options is commented !!
listen_ipv6=YES
```

而不是设置

```
# When "listen" directive is enabled, vsftpd runs in standalone mode and
# listens on IPv4 sockets. This directive cannot be used in conjunction
# with the listen_ipv6 directive.
listen=NO
```

vsftpd 连接在使用 nis 的机器上失败: yp_bind_client_create_v2: RPC: Unable to send

as mentioned on the vsftpd faq page, "...built-in sandboxing uses network isolation on Linux. This may be interfering with any module that needs to use the network to perform operations or lookups" 正如在 vsftpd fap 页面上提到的那样，“...内置沙盒使用 linux 上的网络隔离可能会干扰任何需要使用网络执行操作或查找的模块”

将这个未公开的行添加到你的 `/etc/vsftpd.conf`

```
isolate_network=NO
```

更多资源

- vsftpd official homepage
- vsftpd.conf man page
- vsftpd FAQ

Usenet 客户端

本章解决的是把 Hellanzb 真正用起来的问题：它是一款 Usenet 客户端，先弄清主要功能能覆盖到哪一步，再完成安装，最后把配置调整到与自己环境相符。

本章只有一节，功能、安装、配置顺次讲完，没有可跳过的部分；若已用过同类客户端，功能部分可以略读，安装与配置仍要逐项过一遍。最容易踩的坑是把安装成功当成任务结束，装完并不等于能用，配置没对齐就始终跑不起来。先看清功能的边界再动手改配置，比事后逐条排错省力得多。

35.1　Hellanzb

难度 入门 | **篇幅** 1.9 KB | **所属** Usenet 客户端

本节将讲到

- 主要功能
- 安装
- 配置

**本文已被弃用。**

本文所述教程或工具由于已经过时而被弃用，请使用其他替代方案： 参见应用程序列表/互联网#新闻组新闻阅读器。 (在Talk:Hellanzb讨论)

Hellanzb 是用 Python 编写的基于控制台的 Usenet 二进制下载器。它可以在守护程序模式下运行，并且可以使用多个 GUI 或 PHP 前端。

注意：尽管开发人员和测试团队认为最新版本（0.13）稳定，但 hellanzb 不再处于活动开发中。

主要功能

- 自动下载放置在用户指定的监视目录中的 nzb 文件。
- Par1/2 验证和修复损坏的文件。
- 取消完全下载。
- 下载完成后，删除不需要的文件。

安装

Hellanzb 在 AUR 软件包 hellanzb-gitAUR\[损坏的链接：package not found\] 中可用。

配置

Hellanzb 可以在两种模式下运行：

- 从 CLI
- 作为守护程序

hellanzb.conf

用户 `.config` 目录中必须存在一个名为 `hellanzb.conf` 的配置文件，hellanzb 才能正常运行。提供了一个示例配置文件。

```
mkdir -p ~/.config
cp /etc/hellanzb.conf.sample ~/.config/hellanzb.conf
```

/etc/conf.d/hellanzb

在守护程序模式下运行时，需要修改 `/etc/conf.d/hellanzb` 文件以配置将要运行 hellanzb 的用户

```
HELLANZB_USER="username"
HELLANZB_CONF="/home/${HELLANZB_USER}/.config/hellanzb.conf"
```

分布式计算

这一章要解决的是大型软件包编译太慢的问题：distcc 把编译任务分发到网络中的其他主机，让多台机器共同承担一次构建。全章只有一节，从名词定义、开始使用，到配置、编译、监视进度，再到交叉编译，前后咬合，没有可跳过的部分；建议照页面顺序读完，其中配置最需细看，后面的行为都由它决定。

最容易踩的坑是把 distcc 当成万能加速器。参与的主机需要一致的编译器与架构，否则产出会出错；它只分发编译，链接仍在本地完成，监视进度时负载不均属正常。交叉编译也不能照搬本地编译的配置。

36.1　Distcc

难度 进阶 | **篇幅** 14.3 KB | **所属** 分布式计算

本节将讲到

- 名词定义
- 开始使用
- 配置
- 编译
- 监视进度

翻译状态：

- 本文（或部分内容）译自 Distcc，最近一次同步于 2021-04-15，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

distcc 是一个将 C、C++、Objective C 或 Objective C++ 等程序的编译任务分发到网络中多个主机的程序。distcc 力求实现和本地编译相同的结果，安装、使用都很方便，而且通常比本地编译快很多。distcc 也可以与 Arch 原生的编译工具，比如 makepkg，很好搭配使用。

名词定义

客户机：启动编译的计算机。 **志愿机**：接受客户机发送的编译请求的计算机。一个 distcc 编译集群可以包含一台或多台志愿机。

开始使用

将整个编译集群中的计算机都安装软件包 distcc包。如果是其他的发行版，甚至包括使用 Cygwin 的 Windows 操作系统,请阅读 distcc 文档或 distcc(1) 和 distccd(1) 的手

册页。请保持 distcc 使用的端口畅通（默认是TCP 3632 端口），参见 Category:Firewalls。

配置

工作模式

Distcc 可以在普通模式（默认模式）或泵模式下使用。简单来讲，这两个模式的核心区别在于如何处理预处理过的源代码。普通模式会传递预处理器展开过的源代码和编译选项，由客户机负责预处理，而泵模式会把预处理和编译工作全部分发到 distcc 编译集群中，通常会更快、效率更高。更多细节请参见手册页 `man distcc`。

志愿机配置

志愿机的配置文件储存在 `/etc/conf.d/distccd` 处。最简单的例子，是添加 --allow-private 选项，这样就能够覆盖整个 ipv4 局域网的范围。保存日志到文件也是排错时经常需要的：

```
DISTCC_ARGS="--allow-private --log-file /tmp/distccd.log"
```

如果设备有很多网卡，请考虑添加 --listen <监听地址>。其它设置请参考 distccd(1)。

在所有志愿机上启动服务 `distccd.service`。要使服务开机自启，请启用此服务。

客户机配置

makepkg 编译选项

编辑`/etc/makepkg.conf`的这些部分：

1. 确保 BUILDENV 数组中的 distcc 没有被禁用(即前面没有感叹号)
2. 取消注释并编辑*DISTCC_HOSTS*行 ，加入可以使用的志愿机的 IP 地址或主机名。可以加上一个正斜杠（"/"）以及最大使用线程数。不同 IP 地址用空白隔开，这个清单应该按照处理器性能从高到低排列。
3. 修改 MAKEFLAGS 中的 -jN 为所有使用线程数的和的两倍。在下面的示例为 2x(9+5+5+3)=44。

注意：虽然 IP 地址或主机名都可以描述志愿机，但如果使用 devtools 的编译脚本，现在仍然不支持主机名解析，此时请使用 IP 地址代替。另外，每次编译都进行的主机名解析可能会塞满一些DNS设备的日志（比如pi-hole），此时指定 IP 地址就不会有这些麻烦。

注意： `CFLAGS` 和 `CXXFLAGS` 中不能使用 `-march=native` 选项，否则 distccd 不会将编译任务发给其它机器。

另外，没有一个万能配置能够覆盖全部（尤其是线程数与 MAKEFLAGS 的并行线程数）。请一个一个测试您的参数，并与其他参数的结果做比较，以获取最优的配置。下面是一些例子。

普通模式例子

```
BUILDENV=(distcc fakeroot color !ccache check !sign)
MAKEFLAGS="-j44"
DISTCC_HOSTS="localhost/9 192.168.10.2/5 192.168.10.3/5 192.168.10.4/3"
```

泵模式例子

```
BUILDENV=(distcc fakeroot color !ccache check !sign)
MAKEFLAGS="-j70"
DISTCC_HOSTS="localhost/9 192.168.10.2,cpp,lzo 192.168.10.3,cpp,lzo
192.168.10.4,cpp,lzo"
```

请注意以下事项：

- 并行数高的话，泵模式的表现通常更好。
- 在泵模式下，IP 地址或主机名会跟上指示泵模式所需要的",cpp,lzo"的后缀。此处的本机localhost没有加上这个后缀，这意味着，distcc会按照普通模式严格向localhost安排9个任务，并激进地以泵模式向志愿机分发其他任务，这是一个大集群中的通常实践：客户机通常不希望编译任务阻塞他们，以此限制客户机的任务数。如果您一视同仁，也可以加上这个后缀。
- 正如提到的那样，没有一个万能配置能够覆盖全部情况。如何选择一个最优的设置通常需要一些经验。

非 makepkg 编译选项

普通模式例子

下面的最小 distcc 客户机配置包括了志愿者设置，并将他们附加到PATH环境变量上。

```
$ export PATH="/usr/lib/distcc/bin:$PATH"
$ export DISTCC_HOSTS="localhost/9 192.168.10.2/5 192.168.10.3/5 192.168.10.4/3"
```

泵模式例子

```
$ export PATH="/usr/lib/distcc/bin:$PATH"
$ export DISTCC_HOSTS="localhost/9 192.168.10.2,cpp,lzo 192.168.10.3,cpp,lzo
192.168.10.4,cpp,lzo"
```

编译

makepkg 编译

普通模式

只要按照上面的过程配置好 `/etc/makepkg.conf` 就可以了。正常运行 makepkg 即可。

泵模式

用户必须在编译前启动分发泵。因为分发泵pump程序会在启动时检查DISTCC_HOSTS是否正确配置，因此我们需要先定义一个 DISTCC_HOSTS 搪塞过去。makepkg 会使用 `/etc/makepkg.conf` 里定义的变量，因此不会影响。

```
$ export DISTCC_HOSTS="localhost,cpp,lzo"
$ eval `pump --startup`
```

然后正常运行 makepkg 即可。

结束时，执行以下指令把分发泵关掉：

```
$ pump --shutdown
```

非 makepkg 编译

普通模式

按照#非 makepkg 编译选项章节配置好需要的变量后，直接调用编译器即可：

```
$ make -j44
```

有些程序可能需要定义CC 和/或 CXX 变量才能正常工作：

```
 $ make -j44 CC=distcc CXX=distcc
```

泵模式

按照上面一节方法操作分发泵。其他内容和普通模式相同。

CMake 编译

使用以下 CMake 选项来使用 distcc 编译一个使用 CMake 管理的项目：

```
$ cmake -DCMAKE_C_COMPILER_LAUNCHER=distcc -DCMAKE_CXX_COMPILER_LAUNCHER=distcc ...
```

监视进度

软件包 distcc包 提供了一个命令行界面的监视器 `distccmon-text` 来检查编译进度。

如果在调用这个命令行界面监视器加上一个空格和一个等待秒数，就可以按照秒数不断更新显示。

```
$ distccmon-text 3
29291 Preprocess  probe_64.c                                192.168.10.2
30954 Compile     apic_noop.c                               192.168.10.2
30932 Preprocess  kfifo.c                                   192.168.10.2
30919 Compile     blk-core.c                                192.168.10.2
30969 Compile     i915_gem_debug.c                          192.168.10.2
30444 Compile     block_dev.c                               192.168.10.3
30904 Compile     compat.c                                  192.168.10.3
30891 Compile     hugetlb.c                                 192.168.10.3
30458 Compile     catalog.c                                 192.168.10.4
30496 Compile     ulpqueue.c                                192.168.10.4
30506 Compile     alloc.c                                   192.168.10.4
```

使用 distcc 交叉编译

交叉编译也可使用 distcc 助力。

- 客户机必须运行在目标架构上。
- 其他架构的志愿机可以协助编译，但必须安装对应的工具链，并配置 distcc。

ARM 架构 Arch Linux 客户机与 x86_64 架构的志愿机

下面一节介绍了如何使用 x86_64 架构的志愿机来帮助 Arch Linux ARM 架构设备编译。只需要一台 x86_64 架构的志愿机就能够加快 2 到 4 倍的编译速度，具体可以参见这些测试。

志愿机

Arch ARM 开发者*强烈*推荐使用并在 x86_64 架构的志愿机上安装官方的工具链。与其手动配置，AUR提供了所有的工具链、配置文件，和系统服务：

- distccd-alarm-armv7hAUR
- distccd-alarm-armv8AUR

这些工具链的配置过程和#志愿机配置一节完全相似，除了配置文件和系统服务文件对应包名有所更改。比如 **armv7h** 的配置文件是 `/etc/conf.d/distccd-**armv7h**`，systemd 服务文件是 `distccd-**armv7h**.service`。

注意这些工具链的端口都不同，使得他们可以共存而不冲突。请一定保持他们使用的端口畅通。（参见Category:Firewalls 和手册页 distcc(1)）

目标架构	distcc 端口
armv7h	3635
armv8h/aarch64	3636

客户机

配置 Arch ARM 客户机的最简单办法是使用distccd-arch-armAUR包。这个包提供了针对Arch ARM架构的所有四个配置文件和系统服务配置文件。比如说如果运行的是 **armv7h**固件，只需要配置 `/etc/conf.d/distccd-**armv7h**` 文件并按照上述内容编辑默认值。再启动 `distccd-**armv7h**.service` 服务以编译。

更详细的介绍，请参见使用示例。

如果希望不使用AUR包而手动配置客户机，可以按照通用的方法配置客户机，但需要更改如下两个内容来更改默认端口。例如（端口假设是按照上面的表格）：

1. `/etc/conf.d/distcc`: armv7h 示例： `DISTCC_ARGS="--allow-private --log-level info --log-file /tmp/distccd-armv7h.log --port 3635"`

2. `/etc/makepkg.conf`: armv7h 示例：
   `DISTCC_HOSTS="192.168.10.2/5:3635 192.168.10.3/5:3635"`

x86_64 架构的 Arch Linux 客户机与 ARM 架构志愿机

下面一节介绍了如何使用 ARM 架构的志愿机来帮助x86_64 架构客户机编译。只需要一台 x86_64 架构的志愿机就能够显著加快编译速度，两台设备就能翻倍。具体可以参见这些测试。

客户机

可以按照通用的方法配置客户机，其对应的标准端口是 3632。

志愿机

distccd-x86_64AUR 提供了装在 Arch ARM 设备上的交叉编译工具。

附加工具链

- EmbToolkit： 创建交叉编译工具链的工具；支持 ARM 和 MIPS 架构；支持创建LLVM工具链
- crosstool-ng：和EmbToolkit相似，支持更多架构（更多信息参见其网站）
- Linaro：提供了ARM开发的工具链

`EmbToolkit` 有一个很好看的配置工具链的图形配置菜单（`make xconfig`）。

故障排除

编译 Arch Linux 内核包时的奇怪现象

如果使用官方的PKGBUILD来编译内核，distcc 将无法工作，因为内核硬编码使用了一些GCC插件，因为一些技术原因distccd不能够支持他们。

可以通过编辑内核、删掉这些对 GCC 插件的要求来迂回。可以在编译前在 PKGBUILD 中用一行 sed 命令搞定：

```
sed -i '/HAVE_GCC_PLUGINS/d' arch/x86/Kconfig
```

如果不这样就无法编译。参见 FS#64275。

另一种办法是编译时传递 CC=distcc 和 CXX=distcc 变量：

```
make all CC=distcc CXX=distcc
```

编译 chromium 包时的奇怪现象

编译 chromium包 会使用 clang，目前受到issue#386的影响。规避这个bug可以在PKGBUILD的`_flags`数组中添加这些：

```
'is_cfi=false'
'use_gold=false'
'clang_use_default_sample_profile=false'
```

```
'chrome_pgo_phase=0'
```

Journalctl

使用 `journalctl` 查看什么地方出错了：

```
# journalctl $(which distccd) -e --since "5 min ago"
```

调整日志记录等级

通常来说，distcc会把log写到 `/var/log/messages.log` 里。一个技巧（这个是distccd的手册页推荐的）是写到其他文件里面，比如写到内存（通过 /tmp）里去。另一个技巧是提高日志记录等级，只记录错误信息。这个等级可以是任何标准系统日志等级，即critical、error、warning、notice、info，或者debug。

在客户机上按照这些参数启动distcc，或修改志愿机的`/etc/conf.d/distccd`文件的DISTCC_ARGS参数：

```
DISTCC_ARGS="--allow 192.168.10.0/24 --log-level error --log-file /tmp/distccd.log"
```

修改 $HOME/.distcc 文件夹位置以减少硬盘读写

distcc 默认会创建 `$HOME/.distcc` 文件夹来暂时保存节点编译相关的信息。下面的命令可以减少不必要的硬盘读写，尤其对 SSD 友好：

```
$ export DISTCC_DIR=/tmp/distcc
```

distccd-alarm 相关

没有那个文件或目录

与下例类似的错误表明用户错误地启动了 distcc包 而不是 distccd-alarm 包（即 distccd-alarm-armv7hAUR，或 distccd-alarm-armv8AUR）提供的distccd。

请一定针对目标架构启动正确的服务。

```
distcc (dcc_execvp) ERROR: failed to exec armv7l-unknown-linux-gnueabihf-g++: No such file or directory
```

另请参见

- https://github.com/distcc/distcc
- icecreamAUR - 一个更好配置的 distcc 分支。

多播DNS

多播 DNS 让同一局域网内的主机不必架设域名服务器，也能互相发现、按名字访问。本章处理的就是这类地址解析问题：怎样装好 Avahi，让本机正常收发查询，再把本机提供的服务广播出去，让别的机器看得见。

全章只有《Avahi》一节，顺序读完即可，没有可跳的部分；末尾的「参见」只作延伸线索，第一遍可以略过。最容易踩的坑是以为装上 Avahi 就万事大吉——它负责的是发现，不等于服务本身就能用；排错时也要先把链路和名字解析两边都确认通了，再去查服务声明，而不是一上来反复重装。

37.1　Avahi

难度 进阶 | **篇幅** 13.5 KB | **所属** 多播DNS

本节将讲到

- 安装
- 使用 Avahi
- 添加服务
- 故障排除
- 参见

摘自 维基百科的 Avahi 条目)：

: Avahi 是一种自由的零配置网络（zeroconf）实现，其中包括用于通过组播 DNS/DNS-SD 发现服务的系统。它允许程序在无需特别配置的情况下，发布和发现局域网上运行的服务与主机。例如，连接到网络后，你便可以立即发现可用的打印机、可访问的文件以及可以与之通信的其他用户。Avahi 采用 GNU 宽通用公共许可证（LGPL）授权。

安装

安装 avahi包 软件包，并启用 `avahi-daemon.service`；如果希望使用套接字激活，也可以启用 `avahi-daemon.socket`。

注意：systemd-resolved 内置了 mDNS 服务。在使用 Avahi 之前，请确保已禁用 systemd-resolved 的组播 DNS 解析器/响应器，或者直接禁用 `systemd-resolved.service`。

使用 Avahi

主机名解析

Avahi 使用“*hostname*.local”命名方式提供本地主机名解析。要启用此功能，请安装 nss-mdns包 软件包，并启动/启用 `avahi-daemon.service`。

接下来编辑 `/etc/nsswitch.conf`，修改 `hosts` 行，在 `resolve` 和 `dns` 之前加入 `mdns_minimal [NOTFOUND=return]`：

```
hosts: mymachines mdns_minimal [NOTFOUND=return] resolve [!UNAVAIL=return] files myhostname dns
```

注意：

- 如果解析 `.local` 主机名时速度较慢（或者你不希望使用 IPv6），可以尝试分别用 `mdns4_minimal` 和 `mdns4` 替换 `mdns_minimal` 和 `mdns`。
- 上述配置使 `nss-mdns` 成为 `.local` 域名的权威解析模块，但以下情况除外：单播 DNS 服务器会响应针对顶级域名 `local` 的 `SOA` 查询，或者查询的名称包含两个以上的标签。参见 `nss-mdns` 的启用说明。
- 即使关闭了 mDNS 支持，systemd-resolved 仍会响应这些查询。参见 #systemd-resolved 导致 nss-mdns 无法工作。
- 有关 NSS 配置的更多信息，请参阅 nss-mymachines(8) 和 Avahi 文档。

为自定义顶级域名配置 mDNS

`mdns_minimal` 模块仅处理针对 `.local` 顶级域名（TLD）的查询。注意 `[NOTFOUND=return]`：它表示如果 `mdns_minimal` 无法找到 `*.local` 主机，就不会继续通过 `dns`、`myhostname` 等模块进行查找。

如果希望 Avahi 支持其他顶级域名，需要：

- 将 `mdns_minimal [NOTFOUND=return]` 替换为完整的 `mdns` 模块。此外还有仅支持 IPv4 或 IPv6 的模块 `mdns[46](_minimal)`。
- 在 `/etc/avahi/avahi-daemon.conf` 中将 `domain-name` 配置为所需的域名。
- 在 `/etc/mdns.allow` 中将 Avahi 使用的自定义顶级域名加入允许列表。

工具

Avahi 提供了多个实用工具，可帮助发现网络中运行的服务。例如，执行以下命令来发现局域网内的服务：

```
$ avahi-browse --all --ignore-local --resolve --terminate
```

如果该命令没有输出，很可能是防火墙阻止了 mDNS 流量。

如果只是想通过 mDNS 查询将 `.local` 主机名解析为 IP 地址（类似于 *dig* 或 *nslookup*），可以使用：

```
$ avahi-resolve-host-name some-host-name.local
```

注意，`getent hosts` 命令既可以执行 DNS 查询，也可以执行 mDNS 查询。

Avahi 零配置网络浏览器 `avahi-discover` 可以显示网络上的各类服务。使用该工具需要安装 Avahi 的可选依赖 gtk3包、python-dbus包 和 python-gobject包。此外，还可以分别使用 `bssh` 和 `bvnc` 浏览 SSH 和 VNC 服务器。

防火墙

如果正在使用防火墙，请确保开放 UDP `5353` 端口。

链路本地（Bonjour/Zeroconf）聊天

Avahi 可用于在 Linux 上提供 Bonjour 协议支持。有关支持 Bonjour 协议的客户端列表，请参阅 即时通信客户端比较或应用程序列表/互联网#即时通讯客户端。

获取 IPv4LL IP 地址

**本文或本章节可能需要合并到dhcpcd。**

附注： 应将此内容合并到主页面（在 Talk:Avahi 中讨论）

如果 dhcpcd 客户端无法通过 DHCP 获取地址，可以尝试获取 IPv4LL 地址。默认情况下，此功能处于禁用状态。要启用它，请将 `noipv4ll` 一行注释掉：

```
/etc/dhcpcd.conf
...
#noipv4ll
...
```

或者运行 `avahi-autoipd`：

```
# avahi-autoipd -D
```

添加服务

Avahi 会发布 `/etc/avahi/services` 目录中 `*.service` 文件所定义的服务。该目录中的文件必须可供 `avahi` 用户或用户组读取。

如果想发布的服务没有对应的 `*.service` 文件，可以很方便地自行创建。 例如，假设你想发布一项运行在本机 TCP `17` 端口、符合 RFC 865 规范的每日名言（QOTD）服务。

首先需要确定 `<type>`。avahi.service(5) 指出，其值应为“该服务的 DNS-SD 服务类型，例如 '_http._tcp'”。由于 DNS-SD 注册表已于 2010 年合并到 IANA 注册表，因此可以在 IANA 注册表或 `/etc/services` 文件中查找服务名称。该服务在注册表中的名称为 `qotd`。由于 QOTD 使用 TCP，因此其服务类型为 `_qotd._tcp`，端口为 `17`（依据 IANA 注册表和 RFC 865）。

于是可以创建以下服务文件：

```
qotd.service
<?xml version="1.0" standalone='no'?><!--*-nxml-*-->
```

```
<!DOCTYPE service-group SYSTEM "avahi-service.dtd">

<service-group>

  <name replace-wildcards="yes">%h</name>

  <service>
    <type>_qotd._tcp</type>
    <port>17</port>
  </service>

</service-group>
```

对于更复杂的场景，例如发布运行在其他服务器上的服务、使用 DNS 子类型等，请参阅 avahi.service(5)。

需要注意，Avahi 不支持在 <type> 字段中使用任意字符串，只能使用 Avahi 服务数据库中已知的值。如果希望注册自定义服务类型，可能需要编辑数据库定义、构建更新后的版本，并将其分发到各台主机。

SSH

Avahi 附带了一个用于发布 SSH 服务器的示例服务文件。要启用它，请执行：

```
# cp /usr/share/doc/avahi/ssh.service /etc/avahi/services/
```

文件共享

NFS

如果已经配置了 NFS 共享，可以通过 Avahi，让支持 Zeroconf 的浏览器（例如 KDE 上的 Konqueror、macOS 上的 Finder）或 GNOME/Files 等文件管理器自动挂载这些共享。

在 `/etc/avahi/services` 中创建一个包含以下内容的 `.service` 文件：

```
/etc/avahi/services/nfs_Zephyrus_Music.service
<?xml version="1.0" standalone='no'?>
<!DOCTYPE service-group SYSTEM "avahi-service.dtd">
<service-group>
  <name replace-wildcards="yes">NFS Music Share on %h</name>
  <service>
    <type>_nfs._tcp</type>
    <port>2049</port>
    <txt-record>path=/data/shared/Music</txt-record>
  </service>
</service-group>
```

如果在 `/etc/exports` 中设置了 *insecure* 选项，上述端口就是正确的；否则需要更改端口（注意，macOS 客户端需要 *insecure* 选项）。路径应为导出的共享目录或其子目录。由于某些原因，Leopard 移除了自动挂载功能，不过仍有一个可用的脚本。该脚本基于这篇帖子。

Samba

如果服务器和客户端上都运行了 Avahi 守护进程，客户端的文件管理器应当能够自动发现服务器。

Vsftpd

还可以自动发现普通的 FTP 服务器，例如 vsftpd。安装 vsftpd包 软件包，并根据个人需要调整 vsftpd 的配置（参见 vsftpd.conf(5)）。

在 `/etc/avahi/services` 中创建一个包含以下内容的 `.service` 文件：

```
/etc/avahi/services/ftp.service
<?xml version="1.0" standalone='no'?>
<!DOCTYPE service-group SYSTEM "avahi-service.dtd">
<service-group>
  <name>FTP file sharing</name>
  <service>
    <type>_ftp._tcp</type>
    <port>21</port>
  </service>
</service-group>
```

此时 Avahi 应当开始发布该 FTP 服务器。你应该能够从局域网中另一台计算机的文件管理器中找到它。客户端可能还需要启用#主机名解析功能。

故障排除

主机名末尾不断追加递增的数字

如果没有禁用 systemd-resolved 的组播 DNS 解析器，就可能出现这种情况。此外，这也是一个由主机名竞态条件引起的已知问题。一种可能的解决方法是禁用 IPv6，以尝试避免竞态条件。如果存在多个网络接口，可以使用 allow-interfaces 将 Avahi 限制在单个接口上。另一种可能的解决方法是禁用缓存，从而阻止 Avahi 检查主机名冲突，但这也会使 Avahi 无法执行查询。

systemd-resolved 导致 nss-mdns 无法工作

只有当 `/etc/resolv.conf` 中列出的 DNS 服务器对“local”域名的 SOA 查询返回 `NXDOMAIN` 时，*nss-mdns* 才能正常工作。

首先检查当前配置的 DNS 服务器是否会对“local”域名的 SOA 查询返回 `NXDOMAIN`。例如：

```
$ host -t SOA local
```

如果 DNS 服务器返回 `NXDOMAIN`，就不需要执行下面的步骤。 即使使用 systemd-resolved，Avahi 也应能正常发现网络中的资源。

在旧版本的 systemd-resolved 中，resolved.conf(5) 中的全局选项 `MulticastDNS=no` 会使其针对“local”域名返回与 Avahi 不兼容的响应代码，导致 Avahi 无法正确发现网络资源（如打印机）。详见 systemd 问题 21659。

不过，如果上述 DNS 查询没有对“local”域名返回 `NXDOMAIN`，可以使用完整的 `mdns` NSS 模块替代 `mdns_minimal`，并创建 `/etc/mdns.allow`，仅允许查询“local”域名。例如：

```
/etc/nsswitch.conf
hosts: mymachines mdns [NOTFOUND=return] resolve [!UNAVAIL=return] files myhostname dns
/etc/mdns.allow
.local.
.local
```

注意：上述方法会破坏 mtr包 和 traceroute包 等网络工具的反向 DNS 查询。这些程序会尝试通过 Avahi 进行反向解析，在超时后也不会回退到其他 DNS 服务。该问题似乎类似于 Debian 问题 433943；遗憾的是，其中建议使用 `mdns_minimal` 而不是 `mdns`。

Avahi 套接字出现 ECONNREFUSED（连接被拒绝）错误

如果 Avahi 实例能够正常启动和运行，但 NSS 似乎没有将请求转发给 mDNS，可能是套接字 `/run/avahi-daemon/socket` 处于卡住的状态。例如，可以通过 strace 验证。在这种情况下，可能需要同时重启 `avahi-daemon.service` 和 `avahi-daemon.socket`，才能恢复正常工作。

参见

- Avahi — 项目官方网站
- 维基百科：Avahi（软件）
- iTunes（包含 Bonjour） — 在 Windows 上启用 Zeroconf
- http://www.zeroconf.org/

目录服务

目录服务要解决的是把账号、组织这类条目集中保管，让多台机器与多个应用查同一份结果，而不是各自维护一份名单。本章只有 OpenLDAP 一节：安装与配置必读，前者是前提，后者才决定服务能否真正立起来；排错不必顺读，先扫一眼它覆盖哪些症状，碰到连不上、查不到再回来查；下一步与参阅是延伸入口，第一遍可跳过。

最容易踩的坑是把装好软件包当成配好了，跳过配置直接看下一步，之后的现象便无从判断；另一个误解是把目录服务当文件存储或通用数据库，它擅长的是条目不多、读远多于写的场景，方向选错只会越调越费劲。

38.1　OpenLDAP

难度 进阶 | **篇幅** 15.9 KB | **所属** 目录服务

本节将讲到

- 安装
- 配置
- 下一步
- 排错
- 参阅

翻译状态：

- 本文（或部分内容）译自 openLDAP，最近一次同步于 2024-05-06，若英文版本有所更改，则您可以帮助同步与翻译更改的内容。
- 您可以在 ArchWiki 的对应页面?action=history)找到本文翻译的原始修订历史。
- 本文可能与英文原文存在出入。

OpenLDAP 是 LDAP 协议的一个开源实现。LDAP 服务器本质上是一个为只读访问而优化的非关系型数据库。它主要用做地址簿查询（如 email 客户端）或对各种服务访问做后台认证以及用户数据权限管控。（例如，访问 Samba 时，它可以起到域控制器的作用；或者 Linux 系统认证时代替 `/etc/passwd` 的作用。）

注意：在跟 OpenLDAP 相关的命令中，以 `ldap` 开头的命令（如：`ldapsearch`）是客户端工具，以 `slap` 开头的命令（如：`slapcat`）是服务端工具。

本页面内容仅基于一个基本的 OpenLDAP 安装做简要配置说明。

提示：目录服务是一个庞大的主题，其配置可以非常复杂。如果你是一个完全的新手，这里有一份详尽的介绍文档。该文档通俗易懂，即使你对 LDAP 一窍不通也完全可以引领你入门。

安装

OpenLDAP 软件包同时包含了服务器和客户端。请安装软件包 openldap包。

配置

服务端

注意：

- 如果你在使用过时的 `slapd.conf` 配置文件，可以使用以下命令将其转换为新的 `cn=config` 数据库：

```
$ slaptest -f /etc/openldap/slapd.conf -F /etc/openldap/slapd.d/
```

- 如果你现在的设备上存在 OpenLDAP 数据库，且想要将其移除，可以清空 `/var/lib/openldap/openldap-data/` 下的所有内容。因此，请备份好 `DB_CONFIG`。

作为服务端，Slapd 将其所有配置保存在其数据库内。因此，我们需要先将配置编写为 LDIF 文件，然后将其导入。

首先创建 `/var/lib/openldap/openldap-data/` 目录，LDAP 数据库将存放在该位置下（OpenLDAP 将其称为“database 1”）：

```
# install -m 0700 -o ldap -g ldap -d /var/lib/openldap/openldap-data/
```

然后为 LDAP 配置数据库（“database 0”）创建新路径：

```
# install -m 0670 -o root -g ldap -d /etc/openldap/slapd.d
```

创建 `/etc/openldap/config.ldif`，其中包括如下最小可用配置：

```
/etc/openldap/config.ldif
# The root config entry
dn: cn=config
objectClass: olcGlobal
cn: config
olcArgsFile: /run/openldap/slapd.args
olcPidFile: /run/openldap/slapd.pid

# Schemas
dn: cn=schema,cn=config
objectClass: olcSchemaConfig
cn: schema

# TODO: Include further schemas as necessary
include: file:///etc/openldap/schema/core.ldif

# The config database
dn: olcDatabase=config,cn=config
objectClass: olcDatabaseConfig
olcDatabase: config
olcRootDN: cn=Manager,$BASEDN

# The database for our entries
dn: olcDatabase=mdb,cn=config
objectClass: olcDatabaseConfig
objectClass: olcMdbConfig
olcDatabase: mdb
olcSuffix: $BASEDN
olcRootDN: cn=Manager,$BASEDN
olcRootPW: $PASSWD
```

```
olcDbDirectory: /var/lib/openldap/openldap-data
# TODO: Create further indexes
olcDbIndex: objectClass eq
```

配置中需要根据需要修改部分内容：

- 所有 `$BASEDN` 都需要被替换为有效 DN。假设你手上的域名为 `example.com`，那你可以使用 `dc=example,dc=com`。
- `$PASSWD` 需替换为加盐并哈希化的密码，可通过运行 `slappasswd` 来生成。

另外，你也可以添加更多 schema，并创建额外索引来提高数据库性能。具体实现取决于用例，但也有几点建议可供参考。对于 LDAP 认证，你需要添加下面三个 schema 以使用 `posixAccount` 对象类来储存用户信息。

注意：额外索引需放置在开头的块中，空行分隔将使配置文件无效。

```
# TODO: Create further indexes
olcDbIndex: objectClass eq
olcDbIndex: uid pres,eq
olcDbIndex: mail pres,sub,eq
olcDbIndex: cn,sn pres,sub,eq
olcDbIndex: dc eq

# Additional schemas
# RFC1274: Cosine and Internet X.500 schema
include: file:///etc/openldap/schema/cosine.ldif
# RFC2307: An Approach for Using LDAP as a Network Information Service
# Check RFC2307bis for nested groups and an auxiliary posixGroup objectClass (way
easier)
include: file:///etc/openldap/schema/nis.ldif
# RFC2798: Internet Organizational Person
include: file:///etc/openldap/schema/inetorgperson.ldif
```

Allow logins to the `ldap` user account with `chsh`, typically selecting the shell `/bin/bash`. 然后以 `ldap` 用户身份导入设置：

```
[ldap]$ slapadd -n 0 -F /etc/openldap/slapd.d/ -l /etc/openldap/config.ldif
```

你也可以直接以 `root` 用户运行该命令。但如果你这么做，请确保 `ldap` 可以查看 `/etc/openldap/slapd.d/` 的内容：

```
# slapadd -n 0 -F /etc/openldap/slapd.d/ -l /etc/openldap/config.ldif
# chown -R ldap:ldap /etc/openldap/*
```

如果一切顺利，你将能看到 `/etc/openldap/slapd.d` 下的目录，名称类似于 `cn=config`。

默认情况下，OpenLDAP 会不加密监听所有网络接口。如果要使其仅监听本地 IP 接口，可以编辑 `slapd.service` 读取的环境变量文件：

```
/etc/conf.d/slapd
SLAPD_URLS="ldap://127.0.0.1/ ldap://[::1]"
SLAPD_OPTIONS=
```

最后，通过启动 `slapd.service` 来启动 *slapd* 守护进程。

注意：

- 如果你想让目录接受来自网络的请求，建议使用 TLS。详细内容请参考#基于 TLS 的 OpenLDAP。
- 如果你计划使用 LDAP 服务器进行认证，建议查看 LDAP 认证#服务端配置 中的访问控制配置部分。
- 不应使用 Berkeley DB（BDB）。对于一般的 *slapd* 数据库，建议使用 slapd(8) 的 mdb 后端作为主后端。它使用了 OpenLDAP 自己的轻量内存映射数据库（LMDB）实现来存储数据，被设计用于取代 Berkeley DB 后端。官方仓库中的 OpenLDAP 包默认使用 mdb。

客户端

客户的配置文件位于 `/etc/openldap/ldap.conf`。

这个配置很简单，只需要将 `BASE` 设置为服务器的前缀，将 `URI` 设置为服务器的地址:

```
/etc/openldap/ldap.conf
BASE            dc=example,dc=com
URI             ldap://localhost
```

要使用 SSL 的话:

- `URI` 的协议 (ldap 或 ldaps) 要和 slapd 配置一致
- 要使用自签名的证书，在 `ldap.conf` 中添加 `TLS_REQCERT allow` 一行
- 要使用由认证机构签名的证书，在 `ldap.conf` 中添加 `TLS_CACERTDIR /usr/share/ca-certificates/trust-source` 一行.

创建初始项

注意：如果你计划使用 LDAP 服务器进行认证，你应该按照 LDAP 认证 一文导入 `base.ldif`，而不是遵循下面的步骤。

配置好客户端后，创建根项和管理员（Manager）角色项：

```
$ ldapadd -x -D 'cn=Manager,dc=example,dc=com' -W
dn: dc=example,dc=com
objectClass: dcObject
objectClass: organization
dc: example
o: Example
description: Example directory

dn: cn=Manager,dc=example,dc=com
objectClass: organizationalRole
cn: Manager
description: Directory Manager
^D
```

第一行后的内容是在 stdin 输入的，或者用 `-f` 选项从文件或重定向读入.

安装后测试

运行下面命令：

```
$ ldapsearch -x '(objectclass=*)' -b 'dc=example,dc=com'
```

或认证为 rootdn (将 `-x` 替换为 `-D *user* -W`), 以上面的配置为例：

```
$ ldapsearch -D "cn=Manager,dc=example,dc=com" -W '(objectclass=*)' -b
'dc=example,dc=com'
```

应该能看到数据库中的信息.

基于 TLS 的 OpenLDAP

注意：上游文档比本节内容更加完整实用。

如果通过网络访问 OpenLDAP 服务器，尤其是当你的服务器上保存有敏感数据时，明文传输这些数据存在被他人嗅探的风险。下面章节将指导你如何设置 LDAP 服务器与客户端之间的 SSL 连接以加密传输数据。

要使用 TLS，你必须获得一个证书。测试时可以使用*自签名*证书。关于证书的详细信息请参阅 OpenSSL。

警告：OpenLDAP 不能使用关联了口令的证书。

创建一个自签名证书

输入下列命令创建一个*自签署*证书：

```
$ openssl req -new -x509 -nodes -out slapdcert.pem -keyout slapdkey.pem -days 365
```

You will be prompted for information about your LDAP server. Much of the information can be left blank. The most important information is the common name. This must be set to the DNS name of your LDAP server. If your LDAP server's IP address resolves to example.org but its server certificate shows a CN of bad.example.org, LDAP clients will reject the certificate and will be unable to negotiate TLS connections (apparently the results are wholly unpredictable).

Now that the certificate files have been created copy them to `/etc/openldap/ssl/` (create this directory if it does not exist) and secure them. `slapdcert.pem` must be world readable because it contains the public key. `slapdkey.pem` on the other hand should only be readable for the ldap user for security reasons:

```
# mv slapdcert.pem slapdkey.pem /etc/openldap/ssl/
# chmod -R 755 /etc/openldap/ssl/
# chmod 400 /etc/openldap/ssl/slapdkey.pem
# chmod 444 /etc/openldap/ssl/slapdcert.pem
# chown ldap /etc/openldap/ssl/slapdkey.pem
```

为 slapd 配置 SSL

Edit the configuration to tell LDAP where the certificate files reside by executing the following command:

注意：The latest version of OpenLDAP (2.4.45) uses OpenSSL and not GnuTLS. This means that current versions of OpenLDAP **do** in fact know how to handle the DEFAULT TLSCipherSuite. To prove this one could run `ldd /usr/bin/slapd`.

```
ldapmodify -D 'cn=Manager,dc=example,dc=com' -W
dn: cn=config
add: olcTLSCertificateFile
olcTLSCertificateFile: /etc/openldap/ssl/slapdcert.pem
```

```
-
add: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/openldap/ssl/slapdkey.pem
```

If you are using a signed SSL Certificate from a certification authority such as Let's Encrypt, you will also need to specify the path to the root certificates database and your intermediary certificate. You will also need to change ownership of the *.pem* files and intermediary directories to make them readable to the user `ldap`:

```
ldapmodify -D 'cn=Manager,dc=example,dc=com' -W
dn: cn=config
add: olcTLSCACertificateFile
olcTLSCACertificateFile: /etc/letsencrypt/live/ldap.my-domain.com/chain.pem
-
add: olcTLSCertificateFile
olcTLSCertificateFile: /etc/letsencrypt/live/ldap.my-domain.com/cert.pem
-
add: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/letsencrypt/live/ldap.my-domain.com/privkey.pem
-
add: olcTLSCACertificatePath
olcTLSCACertificatePath: /usr/share/ca-certificates/trust-source
```

SSLv2/v3

Disable SSLv2/v3 and use strong ciphers.

```
ldapmodify -D 'cn=Manager,dc=example,dc=com' -W
dn: cn=config
add: olcTLSProtocolMin
olcTLSProtocolMin: 3.3
-
add: olcTLSCipherSuite
olcTLSCipherSuite: DEFAULT:!kRSA:!kDHE
-
```

TLSProtocolMin specifies the minimum version in wire format, so "3.3" actually means TLSv1.2.

The TLSCipherSuite specifies a list of OpenSSL ciphers from which slapd will choose when negotiating TLS connections, in decreasing order of preference. In addition to those specific ciphers, you can use any of the wildcards supported by OpenSSL. **Note:** DEFAULT is a wildcard. See ciphers(1ssl) for description of ciphers, wildcards and options supported.

注意：To see which ciphers are supported by your local OpenSSL installation, type the following: `openssl ciphers -v ALL:COMPLEMENTOFALL`. Always test which ciphers will actually be enabled by TLSCipherSuite by providing it to OpenSSL command, like this: `openssl ciphers -v 'DEFAULT'`.

启动基于 SSL 的 slapd

注意：This is not needed for StartTLS which listens on the same port as unencrypted LDAP.

You will have to edit the environment file read by `slapd.service` to change the protocol *slapd* listens on:

```
/etc/conf.d/slapd
SLAPD_URLS="ldaps:///"
SLAPD_OPTIONS=
```

Localhost connections do not need to use SSL. So, if you want to access the server locally you should change the `SLAPD_URLS` line to:

```
SLAPD_URLS="ldap://127.0.0.1 ldaps:///"
```
Then restart `slapd.service`. If it was enabled before, reenable it now.

注意：If you created a self-signed certificate above, be sure to add `TLS_REQCERT allow` to `/etc/openldap/ldap.conf` on the client, or it will not be able connect to the server.

下一步

You now have a basic LDAP installation. The next step is to design your directory. The design is heavily dependent on what you are using it for. If you are new to LDAP, consider starting with a directory design recommended by the specific client services that will use the directory (PAM, Postfix, etc).

A directory for system authentication is the LDAP 认证 article.

一个较好的web客户端实现是phpLDAPadmin.

备份 LDAP

It is imperative that we have a backup of our LDAP database and configuration in case we ever need to restore for any number of reasons.

导出配置

```
[ldap]$ slapcat -vF /etc/openldap/slapd.d -n 0 -l "$(hostname)-ldap-mdb-config-$(date '+%F').ldif"
```

导出数据库

```
[ldap]$ slapcat -v -n 1 -l "$(hostname)-ldap-database-$(date '+%F').ldif"
```

恢复 LDAP

导入配置

```
[ldap]$ slapadd -v -n 0 -F /etc/openldap/slapd.d -l <filename from config export>
```

导入数据库

```
[ldap]$ slapadd -v -n 1 -F /etc/openldap/slapd.d -l <filename from database export>
```

排错

检查 slapd 配置

可以使用该命令检查设置：

```
$ slaptest -F /etc/openldap/slapd.d/ -v
```

检查客户端认证

If you cannot connect to your server for non-secure authentication:

```
$ ldapsearch -x -H ldap://ldaservername:389 -D cn=Manager,dc=example,dc=exampledomain
```
and for TLS secured authentication with:

```
$ ldapsearch -x -H ldaps://ldaservername:636 -D cn=Manager,dc=example,dc=exampledomain
```

LDAP 服务突然停止

如果你发现 *slapd* 似乎启动后突然停止了，可以尝试：

```
# chown -R ldap:ldap /var/lib/openldap
```

来允许 *slapd* 以“ldap”用户身份对数据目录进行写入。

LDAP 服务未启动

尝试从命令行启动服务器并启用调试输出：

```
# slapd -u ldap -g ldap -h ldaps://ldaservername:636 -d Config,Stats
```

参阅

- OpenLDAP 官方管理员文档
- phpLDAPadmin - 类似 phpMyAdmin 的网页管理工具。
- LDAP authentication
- 来自 Arch 用户软件仓库 (AUR) 的 apachedirectorystudioAUR 是一个基于 Eclipse 的 LDAP 查看器，适用于 OpenLDAP。

网络应用程序

这一章解决的是"想在 Arch 上做某件要用网络的事，该装什么"：它把网络连接、网络浏览器、网络服务器、文件共享、通讯、新闻与 RSS 和博客分成几类，各列出候选程序。它不是配置教程，也不把某个工具讲透。

全章只有一节，必读，但一百多 KB 的条目不必从头读到尾：先看网络连接与网络浏览器，把上网这条链路走通，再按需翻网络服务器、文件共享与通讯，新闻、RSS 与博客留到要用时再查。最容易踩的坑是把它当说明书：名单只是可选方案，装哪个要回到自己的需求；网络连接没通之前，后面所有条目都无从谈起。

39.1　应用程序列表/互联网

难度 高级 | **篇幅** 147.5 KB | **所属** 网络应用程序

本节将讲到

- 网络连接
- 网络浏览器
- 网络服务器
- 文件共享
- 通讯

网络连接

网络管理器

见网络配置#网络管理器。

VPN 客户端

- **chrootVPN** — A script for creating a chrooted Linux client CheckPoint VPN middleware+agent setup for connecting via their Web Mobile Access Portal.

: https://github.com/ruyrybeyro/chrootvpn || 未被打包？在 AUR 里搜索

- **GlobalProtect-openconnect** — A GlobalProtect VPN client (GUI) for Linux, based on OpenConnect and built with Qt5, supports SAML auth mode.

: https://github.com/yuezk/GlobalProtect-openconnect/ || globalprotect-openconnect-gitAUR

- **Libreswan** — A free software implementation of the most widely supported and standardized VPN protocol based on ("IPsec") and the Internet Key Exchange ("IKE").

: https://libreswan.org/ || libreswanAUR

- **Mullvad** — A GUI client for the Mullvad VPN service

: https://mullvad.net/ || mullvad-vpnAUR

- **Nebula** — A mesh VPN network

: https://nebula.defined.net/docs/ || nebula包

- **NetworkManager** — Supports a variety of protocols (e.g. MS, Cisco, Fortinet) via a plugin system.

: https://wiki.gnome.org/Projects/NetworkManager/VPN || networkmanager包

- **OpenConnect** — Supports Cisco and Juniper VPNs.

: https://www.infradead.org/openconnect/ || openconnect包

- **Openswan** — IPsec-based VPN Solution.

: https://www.openswan.org/ || openswanAUR

- **OpenVPN** — To connect to OpenVPN VPNs.

: https://openvpn.net/ || openvpn包

- **PPTP Client** — To connect to PPTP VPNs, like Microsoft VPNs (MPPE). (insecure)

: https://pptpclient.sourceforge.net/ || pptpclient包

- **RiseupVPN** — A GUI client for the Riseup VPN service from riseup.net.

: https://riseup.net/en/vpn/ || riseup-vpnAUR or riseup-vpn-configuratorAUR

- **strongSwan** — IPsec-based VPN Solution.

: https://www.strongswan.org/ || strongswan包

- **tinc** — tinc is a free VPN daemon.

: https://www.tinc-vpn.org/ || tinc包

- **vopono** — OpenVPN and Wireguard wrapper to launch applications with VPN tunnels in network namespaces.

: https://github.com/jamesmcm/vopono || voponoAUR

- **vpnc** — To connect to Cisco 3000 VPN Concentrators.

: https://www.unix-ag.uni-kl.de/~massar/vpnc/ || vpnc包

- **WireGuard** — Next generation secure network tunnel.

: https://www.wireguard.com/ || wireguard-tools包

代理服务器

- **Brook** — Proxy focusing on strong encryption and being undetectable.

: https://txthinking.github.io/brook/ || brook包

- **Clash** — A rule-based tunnel in Go.

: https://github.com/Dreamacro/clash\[失效链接 2024-01-13 ⓘ\] || clash包

- **Dante** — SOCKS server and SOCKS client, implementing RFC:1928 and related standards.

: https://www.inet.no/dante/ || dante包

- **Geph** — A modular Internet censorship circumvention system designed specifically to deal with national filtering.

: https://geph.io/en/ || geph4-clientAUR

- **hiddify** — Multiplatform chain proxy client based on sing-boxAUR.

: https://github.com/hiddify/hiddify-next || hiddifyAUR

- **NaïveProxy** — A Proxy using Chrome's network stack to camouflage traffic with strong censorship resistence and low detectablility.

: https://github.com/klzgrad/naiveproxy || naiveproxyAUR

- **Privoxy** — Non-caching web proxy with advanced filtering capabilities for enhancing privacy, modifying web page data and HTTP headers, controlling access, and removing ads and other obnoxious Internet junk.

: https://www.privoxy.org/ || privoxy包

- **V2Ray** — V2Ray is the core of Project V, which is a set of tools to help you build your own privacy network over the internet.

: https://www.v2fly.org/en_US/ || v2ray包

- **V2RayN** — A GUI client for Windows, Linux and macOS, support xrayAUR and sing-boxAUR cores and others.

: https://github.com/2dust/v2rayN || v2rayNAUR

- **Nekoray** — Multiplatform chain proxy client based on sing-boxAUR.

: https://github.com/MatsuriDayo/nekoray || nekorayAUR

- **Shadowsocks** — Secure socks5 proxy, designed to protect your Internet traffic.

: https://shadowsocks.org/ || Python: shadowsocks包, Rust: shadowsocks-rust包

- **Squid** — Caching proxy for the Web supporting HTTP, HTTPS, FTP, and more.

: https://www.squid-cache.org/ || squid包

- **Stunnel** — A server and client to add and remove TLS encryption to TCP data flow.

: https://www.stunnel.org/ || stunnel包

- **Tinyproxy** — Lightweight HTTP/HTTPS proxy daemon.

: https://tinyproxy.github.io/ || tinyproxy包

- **Trojan** — An unidentifiable mechanism that helps you bypass GFW.

: https://trojan-gfw.github.io/trojan/ || trojan包

- **Varnish** — High-performance HTTP accelerator.

: https://varnish-cache.org/ || varnish包

- **Wireproxy** — WireGuard client that exposes itself as a SOCKS5 proxy.

: https://github.com/pufferffish/wireproxy || wireproxy包

- **XX-Net** — Easy to use web proxy tool.

: https://github.com/XX-net/XX-Net/ || xx-netAUR

- **Ziproxy** — Forwarding (non-caching) compressing HTTP proxy server.

: http://ziproxy.sourceforge.net/ || ziproxy包

匿名网络

- **Arti** — Rust implementation of the Tor anonymizing overlay network.

: https://gitlab.torproject.org/tpo/core/arti || arti包

- **Freenet** — An encrypted network without censorship.

: https://freenetproject.org/ || freenetAUR

- **GNUnet** — Framework for secure peer-to-peer networking.

: https://gnunet.org/ || CLI: gnunetAUR, GUI: gnunet-gtkAUR

- **I2P** — Distributed anonymous network.

: https://geti2p.net/ || i2pd包, i2pAUR

- **Lantern** — Peer-to-peer internet censorship circumvention software.

: https://getlantern.org/ || lantern-binAUR

- **Lokinet** — Anonymous, decentralized and IP based overlay network for the internet.

: https://lokinet.org/ || lokinetAUR

- **Tor** — Anonymizing overlay network.

: https://www.torproject.org/ || tor包

网络隧道

- **6tunnel** — Tunnels IPv6 connections for IPv4-only applications.

: https://github.com/wojtekka/6tunnel/ || 6tunnel包

- **iodine** — Tunnel IPv4 data through a DNS server.

: https://code.kryo.se/iodine/ || iodine包

- **isatapd** — Creates and maintains an ISATAP tunnel (RFC:5214).

: http://www.saschahlusiak.de/linux/isatap.htm || isatapd包

- **Ping Tunnel** — A tool for reliably tunneling TCP connections over ICMP echo request and reply packets.

: https://www.cs.uit.no/~daniels/PingTunnel/ || ptunnel包

- **Tuntox** — Tunnel TCP connections over the Tox protocol.

: https://github.com/gjedeer/tuntox/ || tuntox包

Deep packet inspection circumvention

Tools to avoid censorship, bandwidth throttle without anonymization. See Wikipedia:Deep packet inspection, Wikipedia:Internet censorship circumvention for an introduction to the topic.

- **Bypass DPI** — A simple software using SOCKS5, written in C and targeted at Russian censorship.

: https://github.com/hufrea/byedpi || byedpiAUR

- **Cloak** — A pluggable transport that works alongside traditional proxy tools like OpenVPN, written in Go.

: https://github.com/cbeuw/Cloak || cloak-obfuscationAUR

- **DPI Tunnel** — An HTTP/transparent proxy, written in C++ and targeted at Russian censorship.

: https://github.com/txtsd/DPITunnel || dpitunnelAUR

- **Green Tunnel** — An anti-censorship utility using SOCKS5, written in nodejs and targeted at Iranian censorship.

: https://github.com/SadeghHayeri/GreenTunnel || green-tunnelAUR

- **SpoofDPI** — A simple and fast tool using SOCKS5, written in Go.

: https://github.com/xvzc/SpoofDPI || spoofdpiAUR

- **zapret** — A Netfilter queue mode or SOCKS5/transparent proxy, written in C and targeted at Russian censorship.

: https://github.com/bol-van/zapret || zapret-gitAUR

网络浏览器

参见 Wikipedia:Comparison of web browsers。

控制台

- **browsh** — A fully-modern text-based browser. Runs as a frontend to headless Firefox.

: https://www.brow.sh/ || browshAUR

- **Carbonyl** — A Chromium based browser built to run in a terminal.

: https://fathy.fr/carbonyl || carbonyl-binAUR

- **Chawan** — A web browser for your terminal.

: https://sr.ht/~bptato/chawan/ || chawan-gitAUR

- **ELinks** — Advanced and well-established feature-rich text mode web browser with mouse wheel scroll support, frames and tables, extensible with Lua & Guile (links fork).

: http://elinks.or.cz/ || elinks包

- **Links)** — Graphics and text mode web browser. Includes a console version similar to Lynx.

: http://links.twibright.com/ || links包

- **Lynx)** — Text browser for the World Wide Web.

: https://lynx.invisible-island.net/ || lynx包

- **w3m** — Pager/text-based web browser. It has vim-like keybindings, and is able to display images.

: https://salsa.debian.org/debian/w3m || w3m包

图形界面

基于 Gecko

参见 Wikipedia:Gecko (software))。

- **Firefox** — Mozilla 基于 Gecko 的可扩展浏览器，支持快速渲染。

: https://mozilla.com/firefox || firefox包

- **SeaMonkey** — Mozilla Internet Suite 的延续。

: https://www.seamonkey-project.org/ || seamonkeyAUR

Firefox 分支

- **FireDragon** — A fork of Floorp that includes KDE integration patches and custom branding for Garuda.

: https://gitlab.com/garuda-linux/firedragon/settings || firedragonAUR

- **Floorp** — Firefox-based browser with excellent privacy protection, developed by a community of students in Japan.

: https://floorp.ablaze.one || floorpAUR

- **GNU IceCat** — A customized build of Firefox ESR distributed by the GNU Project, stripped of non-free components and with additional privacy extensions. Release cycle may be delayed compared to Mozilla Firefox.

: https://www.gnu.org/software/gnuzilla/ || icecatAUR

- **Mercury** — Firefox fork with compiler optimizations and patches from Librewolf, Waterfox, and GNU IceCat.

: https://thorium.rocks/mercury || mercury-browser-binAUR

- **Midori)** — Light, fast and secure browser.

: https://github.com/goastian/midori-desktop || midoriAUR

- **Mullvad Browser** — Privacy-focused web browser developed by Mullvad VPN and the Tor Project based on Firefox ESR.

: https://mullvad.net/en/browser || mullvad-browser-binAUR

- **LibreWolf** — A fork of Firefox, focused on privacy, security and freedom.

: https://librewolf.net/ || librewolfAUR

- **Tor Browser Launcher** — Securely and easily download, verify, install, and launch Tor Browser, a fork of Firefox ESR.

: https://github.com/micahflee/torbrowser-launcher || torbrowser-launcher包

- **Waterfox** — Fork of Mozilla Firefox featuring some privacy, usability, and speed enhancements.

: https://www.waterfox.net/ || waterfoxAUR

- **Zen Browser** — An experimental, privacy-focused and performance-optimized fork of Firefox with additional features, such as tab groups and a sidebar.

: https://www.zen-browser.app/ || zen-browser-binAUR

基于 Blink

参见 Wikipedia:Blink (web engine))。

- **Chromium** — Web browser developed by Google, the open source project behind Google Chrome.

: https://www.chromium.org/ || chromium包

注重隐私的 chromium 分支

- **Brave)** — Web browser with builtin ad- and tracker blocking.

: https://www.brave.com/ || brave-binAUR

- **Ungoogled Chromium** — Modifications to Google Chromium for removing Google integration and enhancing privacy, control, and transparency

: https://github.com/ungoogled-software/ungoogled-chromium || ungoogled-chromiumAUR或ungoogled-chromiumCNRepo

- **Thorium** — Thorium develops a periodically synchronized fork of the Chromium browser, expanded with additional patches to optimize performance, improve usability and enhance security. According to the developers' tests Thorium is 8-40% ahead of the regular Chromium, mainly due to the inclusion of additional optimizations during compilation.

: https://thorium.rocks/ || thorium-browser-binAUR

- **Cromite** — Cromite is a Chromium fork based on Bromite (Currently unmaintained) with built-in support for ad blocking and an eye for privacy. See List of features.

: https://www.cromite.org/ || cromite-binAUR

专有的 chromium 分支

- **Google Chrome** — Proprietary web browser developed by Google.

: https://www.google.com/chrome/ || google-chromeAUR

- **Microsoft Edge** — Proprietary web browser developed by Microsoft.

: https://www.microsoft.com/edge/ || microsoft-edge-stable-binAUR

- **Opera** — Proprietary browser developed by Opera Software.

: https://opera.com || operaAUR

- **Slimjet** — Fast, smart and powerful proprietary browser based on Chromium.

: https://www.slimjet.com/ || slimjetAUR

- **Vivaldi** — An advanced proprietary browser made with the power user in mind.

: https://vivaldi.com/ || vivaldi包

- **Yandex Browser** — Proprietary browser that combines a minimal design with sophisticated technology to make the web faster, safer, and easier.

: https://browser.yandex.com/ || yandex-browserAUR

基于 Qt WebEngine 的浏览器

- **Dooble** — Colorful Web browser.

: https://textbrowser.github.io/dooble/ || doobleAUR

- **Eric** — QtWebEngine-based HTML browser, part of the eric6 development toolset, can be launched with the `eric6_browser` command.

: https://eric-ide.python-projects.org/ || ericAUR

- **Falkon** — Web browser based on QtWebEngine, written in Qt framework. Part of kde-network包组.

: https://falkon.org/ || falkon包

- **Fiery** — A convergent web browser. Part of maui包组.

: https://mauikit.org/apps/ || fiery包

- **Konqueror** — Web browser based on Qt toolkit and Qt WebEngine. Part of kde-network包组.

: https://apps.kde.org/konqueror/ || konqueror包

- **Liri Browser** — A minimalistic material design web browser written for Liri.

: https://github.com/lirios/browser || liri-browser-gitAUR

- **Otter Browser** — Browser aiming to recreate classic Opera (12.x) UI using Qt5.

: https://otter-browser.org/ || otter-browserAUR

- **qutebrowser** — A keyboard-driven, vim\-like browser based on PyQt5 and QtWebEngine.

: https://qutebrowser.org/ || qutebrowser包

基于 Electron 的浏览器

- **Beaker** — Peer-to-peer web browser with tools to create and host websites.

: https://github.com/beakerbrowser/beaker || beaker-browserAUR

- **Catalyst** — A minimal FOSS web browser with no data collection.

: https://getcatalyst.eu.org || catalyst-browser-binAUR

- **Franz** — Messaging browser for WhatsApp, Facebook Messenger, Slack, Telegram and many other web services.

: https://meetfranz.com/ || franzAUR

- **Hamsket** — A GPL-licensed alternative to Franz, forked from the last open sources of Rambox.

: https://github.com/TheGoddessInari/hamsket || hamsketAUR

- **Min** — A fast, minimal browser that protects your privacy. It includes an interface designed to minimize distractions.

: https://minbrowser.org/ || minAUR

- **Vieb** — Minimalist Electron-based browser with Vim-inspired keybindings and a built-in ad-blocker.

: https://vieb.dev || vieb-binAUR

基于 WebKitGTK

参见 Wikipedia:WebKit。

注意：webkitgtk, webkitgtk2, qt5-webkit and qtwebkit-based browsers were removed from the list, because these are today considered insecure and outdated. More info here and here.

其中大多数支持来自 wyebadblock 的广告拦截。

- **Badwolf** — A minimalist privacy-focused browser.

: https://hacktivis.me/projects/badwolf || badwolfAUR

- **Eolie** — Simple web browser for GNOME.

: https://wiki.gnome.org/Apps/Eolie || eolie包

- **GNOME Web** — Browser which uses the WebKitGTK rendering engine. Part of gnome包组.

: https://apps.gnome.org/Epiphany/ || epiphany包

- **Luakit** — Fast, small, webkit based browser framework extensible by Lua.

: https://luakit.github.io/ || luakit包

- **Nyxt** — Keyboard-oriented, infinitely extensible browser designed for power users. It has familiar key-bindings (Emacs, VI, CUA) and features fuzzy searching between tabs, multiple selections, history as a tree and more.

: https://nyxt.atlas.engineer/ || nyxt包

- **surf** — Lightweight WebKit-based browser, which follows the suckless philosophy (basically, the browser itself is a single C source file).

: https://surf.suckless.org/ || surfAUR

- **Surfer** — Simple keyboard based web browser, written in C. It supports custom JS-scripts.

: https://github.com/nihilowy/surfer || surfer-gitAUR

- **Tangram** — Integration of web applications into the desktop, specifically GNOME.

: https://apps.gnome.org/Tangram/ || tangram包

- **Vimb** — A Vim-like web browser written in C that is inspired by Pentadactyl and Vimprobable. It includes a manpage and a howto for common configurations. It supports custom JS-scripts, dark mode and handles geolocation requests.

: https://fanglingsu.github.io/vimb/ || vimb包

- **wyeb** — A vim-like web browser inspired by dwb and luakit with Adblock.

: https://github.com/jun7/wyeb || wyeb-gitAUR

Goanna-based

See also Wikipedia:Goanna (software)).

- **Pale Moon)** — A Firefox fork focussing on speed, with a pre-Firefox 29 interface. Uses Goanna) layout engine, a fork of Gecko. Firefox add-ons may not be compatible. Without support for newer Firefox features such as WebExtensions, cache2, e10s, and OTMC. Many of the old 93,598 versions of 19,450 Firefox add-ons created by 14,274 developers over the past 15 years using XUL/XPCOM technology in the Classic Add-ons Archive still work.

: https://www.palemoon.org/ || palemoonAUR

- **Basilisk)** — A XUL-based web browser, similar in design to Firefox, aimed at providing a fully functional browsing experience with a classic interface.

: https://basilisk-browser.org/ || basiliskAUR

Servo-based

- **Servo** — Web browser rendering engine written in Rust, with WebGL and WebGPU support, and adaptable to desktop, mobile, and embedded applications.

: https://servo.org/ || servo-gitAUR

- **Verso** — Experimental web browser based on Servo experimenting with multi-view and multi-window and building UI elements entirely from Servo.

: https://github.com/versotile-org/verso/ || verso-gitAUR

其它

- **Dillo** — Small, fast graphical web browser built on FLTK. Uses its own layout engine.

: https://dillo-browser.github.io/ || dillo包

- **Ladybird** — Web browser built from scratch using the SerenityOS LibWeb engine.

: https://ladybird.org/ || ladybirdAUR

- **Links)** — Graphics and text mode web browser. Includes a graphical X-window/framebuffer version with CSS, image rendering, pull-down menus. It can be launched with the `xlinks -g` command.

: http://links.twibright.com/ || links包

- **NetSurf** — Featherweight browser written in C, notable for its slowly developing JavaScript support and fast rendering through its own layout engine.

: https://www.netsurf-browser.org/ || netsurf包

Gemini 浏览器

参见 Wikipedia:Gemini (protocol)#Software#Software)。

- **Amfora** — Terminal browser for the Gemini protocol.

: https://github.com/makeworld-the-better-one/amfora || amfora包

- **Bombabillo** — Non-web client for the terminal, supporting Gopher, Gemini and much more.

: https://bombadillo.colorfield.space/ || bombadillo-gitAUR

- **Castor** — Graphical client for the Gemini, Gopher, and Finger protocols, written in Rust with GTK.

: https://git.sr.ht/~julienxx/castor || castorAUR

- **Geopard** — A graphical gemini client written in rust, using the gtk4 toolkit.

: https://github.com/ranfdev/Geopard || geopardAUR

- **Kristall** — Qt-based Gemini browser.

: https://github.com/MasterQ32/kristall || kristallAUR

- **Lagrange** — Desktop GUI client for browsing Gemini space, offering modern conveniences familiar from web browsers.

: https://gmi.skyjake.fi/lagrange || lagrangeAUR

- **Telescope** — w3m-like browser for Gemini.

: https://www.telescope-browser.org/ || telescopeAUR

网络服务器

网络服务器通过 HTTP 向网络浏览器等客户端提供 HTML 网页和其他文件。 主流的网络服务器可以提供动态内容（网络应用）的程序接口。

参见 分类:网络服务器与 Wikipedia:Comparison of web server software。

- **Apache HTTP Server** — A high performance Unix-based HTTP server.

: https://httpd.apache.org/ || apache包

- **Caddy** — HTTP/3 web server with automatic HTTPS.

: https://caddyserver.com/ || caddy包

- **Hiawatha** — Secure and advanced web server.

: https://hiawatha.leisink.net/ || hiawathaAUR

- **Lighttpd** — A secure, fast, compliant and very flexible web-server.

: https://www.lighttpd.net/ || lighttpd包

- **nginx** — Lightweight HTTP server and IMAP/POP3 proxy server.

: https://nginx.org/ || nginx包

- **sthttpd** — Supported fork of the thttpd web server.

: https://github.com/blueness/sthttpd || sthttpdAUR

- **Traefik** — A modern reverse proxy and load balancer that makes deploying microservices easy.

: https://traefik.io/traefik/ || traefik包

- **yaws** — Web server/framework written in Erlang.

: https://erlyaws.github.io/ || yawsAUR

静态网络服务器

- **Apache Traffic Server** — Fast, scalable and extensible HTTP/1.1 and HTTP/2 compliant caching proxy server.

: https://trafficserver.apache.org/ || trafficserverAUR

- **darkhttpd** — A small and secure static web server, written in C, does not support HTTPS or Auth.

: https://unix4lyfe.org/darkhttpd/ || darkhttpd包

- **miniserve** — Rust alternative to darkhttpd with UTF-8, optional HTTP authentication, file uploading, and more.

: https://github.com/svenstaro/miniserve || miniserve包

- **quark** — An extremely small and simple http get-only web server. It only serves static pages on a single host.

: https://tools.suckless.org/quark/ || quark-gitAUR

- **serve** — Static file serving and directory listing.

: https://github.com/zeit/serve || nodejs-serveAUR

- **Webfs** — Simple and instant web server for mostly static content.

: https://linux.bytesex.org/misc/webfs.html || webfsAUR

也可以从命令行使用 Python 标准库模块 http.server。

专用网络服务器

- **chezdav** — WebDAV server that allows to share a particular directory.

: https://wiki.gnome.org/phodav || phodav包

- **Mongoose** — Embedded web server library, supports WebSocket and MQTT.

: https://github.com/cesanta/mongoose || mongooseAUR

- **OnionShare** — Lets you securely and anonymously send and receive files. It works by starting a web server, making it accessible as a Tor onion service,

and generating an unguessable web address so others can download files from you, or upload files to you.

: https://onionshare.org/ || onionshare包

- **Transfer More** — A minimalist open-source upload HTTP server to store and share files temporarily, written in Crystal, and based on Kemal.

: https://up.sceptique.eu/ || transfer-moreAUR

- **VServer** — GTK application, which opens an http server in the selected folder and shares your files.

: https://github.com/bcedu/ValaSimpleHTTPServer || vserver-gitAUR

- **webhook** — Small server for creating HTTP endpoints (hooks)

: https://github.com/adnanh/webhook || webhook包

- **Woof** — An ad-hoc single file webserver; Web Offer One File.

: http://www.home.unix-ag.org/simon/woof.html || woofAUR

WSGI 服务器

- **Gunicorn** — A Python WSGI HTTP Server for UNIX.

: https://gunicorn.org/ || gunicorn包

- **uWSGI** — A fast, self-healing and developer/sysadmin-friendly application container server written in C.

: https://uwsgi-docs.readthedocs.io/ || uwsgi包

- **Waitress** — A WSGI server for Python 3.

: https://github.com/Pylons/waitress || python-waitress包

Apache 也可以支持 WSGI 用 mod_wsgi。

性能测试

- **http_load** — A webserver performance testing tool, runs in a single process.

: https://www.acme.com/software/http_load/\[失效链接 2025-03-15 ⓘ\] || http_loadAUR

- **httperf** — Can generate various HTTP workloads, written in C.

: https://github.com/httperf/httperf || httperf-gitAUR

- **httping** — A "ping"-like tool for HTTP requests

: https://www.vanheusden.com/httping/ || httping包

- **siege** — An HTTP regression testing and benchmarking utility.

: https://www.joedog.org/siege-home/ || siege包

- **vegeta** — HTTP load testing tool, written in Go.

: https://github.com/tsenart/vegeta || vegeta包

- **Web Bench** — Benchmarking tool, uses fork() for simulating multiple clients.

: http://home.tiscali.cz/~cz210552/webbench.html || webbenchAUR

文件共享

下载管理器

参见 Wikipedia:Comparison of download managers。

控制台

- **aria2** — Lightweight download utility that supports HTTP/S, FTP, SFTP, BitTorrent and Metalink. It can run as a daemon controlled via a built-in JSON-RPC or XML-RPC interface.

: https://aria2.github.io/ || aria2包

- **Axel** — Featherweight command line download accelerator sitting at under 250kB on disk. Supports HTTP/S and FTP.

: https://github.com/eribertomota/axel || axel包

- **cURL** — A URL retrieval utility and library. Supports HTTP, FTP and SFTP.

: https://curl.haxx.se/ || curl包

- **Gtuber** — Fetch media info from websites (bilibili, lbry, niconico, reddit, twitch, youtube). Includes a command-line downloader (`gtuber-dl`) and a GStreamer plugin, which is used automatically when you try playing video from website that Gtuber can handle in your GStreamer based application.

: https://github.com/Rafostar/gtuber || gtuber包

- **HTTPie** — Human-friendly command-line HTTP client for the API era.

: https://github.com/httpie/httpie || httpie包

- **HTTrack** — An easy-to-use offline browser utility. It allows you to download a World Wide Web site from the Internet to a local directory, building recursively all directories, getting HTML, images, and other files from the server to your computer.

: https://www.httrack.com/ || httrack包

- **LFTP** — Sophisticated file transfer program. Supports HTTP, FTP, SFTP, FISH, and BitTorrent.

: https://lftp.yar.ru/ || lftp包

- **Plowshare** — A set of command-line tools designed for managing file-sharing websites (aka Hosters).

: https://github.com/mcrapet/plowshare || plowshareAUR

- **RTMPDump** — Download FLV videos through RTMP (Adobe's proprietary protocol for Flash video players)

: http://rtmpdump.mplayerhq.hu/ || rtmpdump包

- **snarf** — Command-line URL retrieval tool. Supports HTTP and FTP.

: https://www.xach.com/snarf/ || snarfAUR

- **Streamlink** — Launch streams from various streaming services in a custom video player or save them to a file.

: https://streamlink.github.io/ || streamlink包

- **Streamripper** — Records and splits streaming mp3 into tracks.

: https://streamripper.sourceforge.net/ || streamripperAUR

- **yewtube** — Terminal-based YouTube player and downloader.

: https://github.com/mps-youtube/yewtube || yewtubeAUR

- **You-Get** — Download media contents (videos, audios, images) from the Web.

: https://you-get.org/ || you-getAUR

- **youtube-dl** — Download videos from YouTube and many other web sites.

: https://rg3.github.io/youtube-dl/ || youtube-dlAUR

- **youtube-viewer** — Command line utility for viewing YouTube videos.

: https://github.com/trizen/youtube-viewer || youtube-viewerAUR

- **ytfzf** — A POSIX script to find and watch youtube videos from the terminal.

: https://github.com/pystardust/ytfzf || ytfzf包

- **yt-dlp** — A youtube-dl fork with additional features and fixes.

: https://github.com/yt-dlp/yt-dlp || yt-dlp包

- **Wget** — A network utility to retrieve files from the Web. Supports HTTP and FTP.

: https://www.gnu.org/software/wget/ || wget包

图形界面

- **ClipGrab** — Downloader and converter for YouTube, Vimeo and many other online video sites.

: https://clipgrab.org/ || clipgrab包

- **FatRat** — Qt based download manager with support for HTTP, FTP, SFTP, BitTorrent and Metalink.

: https://github.com/LubosD/fatrat || fatrat-gitAUR

- **Forklift** — Simple GUI for youtube-dl using PyGObject.

: https://github.com/Johnn3y/Forklift || forklift-gitAUR

- **gtk-youtube-viewer** — GTK utility for viewing YouTube videos. See optional dependencies for the GUI.

: https://github.com/trizen/youtube-viewer || youtube-viewerAUR

- **Gwget** — Download manager for GNOME. Supports HTTP and FTP.

: https://gitlab.gnome.org/Archive/gwget || gwgetAUR

- **Gydl** — GUI wrapper around the already existing youtube-dl program to download content from sites like YouTube.

: https://github.com/JannikHv/gydl || gydl-gitAUR

- **Gyre** — GTK3 downloader for videos from Coub.

: https://github.com/HelpSeeker/Gyre || gyreAUR

- **JDownloader** — Java-based downloader for one-click hosting sites.

: https://jdownloader.org/ || jdownloader2AUR

- **KGet** — Download manager for KDE. Supports HTTP, FTP, BitTorrent and Metalink. Part of kde-network包组.

: https://apps.kde.org/kget/ || kget包

- **MegaBasterd** — Yet another unofficial MEGA downloader/uploader/streaming suite.

: https://github.com/tonikelope/megabasterd || megabasterd-binAUR

- **Motrix** — Full-featured download manager that supports downloading HTTP, FTP, BitTorrent, Magnet, etc. Based on the Electron platform.

: https://motrix.app/ || motrixAUR

- **Persepolis** — Graphical front-end for aria2 download manager with lots of features. Supports HTTP and FTP.

: https://persepolisdm.github.io/ || persepolis包

- **pyLoad** — Downloader written in Python and designed to be extremely lightweight, easily extensible and fully manageable via web.

: https://pyload.net/ || pyload-ngAUR

- **Steadyflow** — Simple download manager for GNOME. Supports HTTP and FTP.

: https://launchpad.net/steadyflow || steadyflow包

- **Streamtuner2** — Internet radio station and video browser. It simply lists stations in categories from different directories and launches your preferred media apps for playback.

: https://sourceforge.net/projects/streamtuner2/ || streamtuner2AUR

- **uGet** — GTK download manager featuring download classification and HTML import. Supports HTTP, FTP, BitTorrent, Metalink, YouTube and Mega.

: https://ugetdm.com/ || uget包

- **Video Downloader** — GTK application to download videos from websites like YouTube and many others (based on youtube-dl).

: https://github.com/Unrud/video-downloader || video-downloaderAUR

- **Xtreme Download Manager** — Powerful tool to increase download speed up-to 500%. Supports HTTP and FTP. Video grabber works in a general way and is not limited to certain websites.

: https://subhra74.github.io/xdm/ || xdmanAUR

- **youtubedl-gui** — Simple-to-use graphical interface for youtube-dl.

: https://github.com/JaGoLi/ytdl-gui || youtubedl-guiAUR

局域网文件传输

参见 #局域网 messengers。

- **LAN Share** — Cross-platform local area network file transfer application, built using Qt GUI framework. It can be used to transfer a whole folder, one or more files, large or small immediately without any additional configuration.

: https://github.com/abdularis/LAN-Share || lanshareAUR

- **LocalSend** — Cross-platform app to securely share files and messages with nearby devices over the local network without needing an internet connection.

: https://localsend.org/ || localsendAUR 或 localsendCNRepo

- **Magic Wormhole** — Command-line tool `wormhole` to securely transfer data between computers.

: https://github.com/magic-wormhole/magic-wormhole || magic-wormhole包

- **NitroShare** — Cross-platform network file transfer application, built using Qt GUI framework. It is designed to make transferring files from one device to another on the local network extremely simple.

: https://nitroshare.net/ || nitroshare包

- **Teleport** — Native GTK3 application to effortlessly share files on the local network.

: https://gitlab.gnome.org/jsparber/teleport || teleport-share-gitAUR

- **Warp** — Securely send files to each other via the internet or local network by exchanging a word-based code. It uses the Magic Wormhole protocol.

: https://apps.gnome.org/Warp/ || warp包

- **Warpinator** — GTK application to share files across the LAN.

: https://github.com/linuxmint/warpinator || warpinator包

云存储服务器

- **Cozy** — A personal cloud you can hack, host and delete.

: https://cozy.io/ || cozy-stack包

- **Nextcloud** — A cloud server to store your files centrally on a hardware controlled by you.

: https://nextcloud.com || nextcloud包

- **Pydio** — Mature open source web application for file sharing and synchronization.

: https://pydio.com/ || pydioAUR

- **Seafile** — An online file storage and collaboration tool with advanced support for file syncing, privacy protection and teamwork.

: https://www.seafile.com/ || seafile-serverAUR

云同步客户端

提示：

- 一些 同步和备份程序 直接支持某些云储存服务。
- 一些 FUSE文件系统 提供了将云储存挂载为文件系统的方法。例如对于Google Drive，在使用Gnome虚拟文件系统(gvfs)的应用(如Nautilus)可以安装 gvfs-google包 来进行访问，或者在使用KIO框架的应用(如Dolphin)中安装 kio-gdrive包 进行访问。
- 如果你想实现零知识(客户端透明加密)(zero-knowledge(client-side transparent encryption))储存，参见静态数据加密#云存储优化。

多协议客户端

- **CloudCross** — Synchronize local files and folders with many cloud providers. Mail.ru Cloud, Yandex Disk, Google Drive, OneDrive and Dropbox support is available.

: https://github.com/MasterSoft24/CloudCross || cloudcrossAUR

- **Rclone** — Multi-provider sync, copy, and mount client.

: https://rclone.org/ || rclone包

- **Rclone Browser** — GUI client for Rclone.

: https://github.com/kapitainsky/RcloneBrowser || rclone-browserAUR

Google Drive 客户端

- **DriveSync** — Command line utility that synchronizes your Google Drive files with a local folder on your machine.

: https://github.com/MStadlmeier/drivesync || drivesyncAUR

- **gdrive** — Command line utility for interacting with Google Drive.

: https://github.com/prasmussen/gdrive || gdriveAUR

- **Grive** — Google Drive client with support for new Drive REST API and partial sync.

: https://github.com/vitalif/grive2 || griveAUR

- **Insync** — Unofficial proprietary Google Drive desktop client.

: https://www.insynchq.com/ || insyncAUR

- **Google Drive OCamlFUSE** — A FUSE filesystem for Google Drive, written in OCaml.

: https://github.com/astrada/google-drive-ocamlfuse || google-drive-ocamlfuseAUR

其他同步客户端

- **aws-cli** — CLI for Amazon Web Services, including efficient file transfers to and from Amazon S3.

: https://aws.amazon.com/cli/ || aws-cli包

- **Backblaze B2** — Backblaze B2 open-source command-line client.

: https://www.backblaze.com/b2/cloud-storage.html || backblaze-b2AUR

- **Baidu Netdisk** — Proprietary client for cloud storage service launched by Baidu (formerly Baidu Cloud).

: https://pan.baidu.com || baidunetdisk-binAUR

- **Cozy Drive** — Desktop client for Cozy.

: https://cozy-labs.github.io/cozy-desktop/ || cozy-desktop包

- **Dropbox** — Proprietary desktop client for Dropbox.

: https://www.dropbox.com/ || dropboxAUR

- **Mega) Sync Client** — Proprietary (though source-available) desktop client to sync files with Mega.

: https://mega.nz/ || CLI: megacmdAUR, GUI: megasyncAUR

- **Megatools** — Unofficial CLI for Mega.

: https://megatools.megous.com/ || megatools包

- **Nextcloud Client** — Desktop client for Nextcloud.

: https://nextcloud.com/ || nextcloud-client包

- **Nutstore** — Proprietary desktop client for Nutstore.

: https://www.jianguoyun.com/ || nutstoreAUR

- **OneDrive** — Unofficial CLI for OneDrive.

: https://github.com/skilion/onedrive || onedriveAUR

- **OneDrive** — Fork of the unofficial CLI for OneDrive.

: https://github.com/abraunegg/onedrive || onedrive-abrauneggAUR

- **ownCloud Desktop Client** — Desktop syncing client for ownCloud.

: https://owncloud.com/client/ || owncloud-client包

- **pCloud Drive** — Proprietary desktop syncing client for pCloud. Based on the Electron platform.

: https://www.pcloud.com/download-free-online-cloud-file-storage.html || pcloud-driveAUR

- **PydioSync** — Desktop client for Pydio.

: https://pydio.com/ || pydio-syncAUR

- **S3cmd** — Unofficial CLI for Amazon S3.

: https://s3tools.org/s3cmd || s3cmd包

- **Seafile Client** — GUI client for Seafile.

: https://www.seafile.com/ || seafile-clientAUR

- **Versiobit** — Desktop file sync client with versioning and end-to-end encryption.

: https://versiobit.com || versiobitAUR

- **SpiderOak One** — Proprietary client for SpiderOak One.

: https://spideroak.com/ || spideroak-oneAUR

- **Synology Drive** — Proprietary GUI client to sync and share files between a centralized Synology NAS and multiple client computers.

: https://www.synology.com/ || synology-driveAUR

- **Tresorit** — Proprietary desktop syncing client for Tresorit.

: https://tresorit.com/download || tresoritAUR

- **Yandex Disk** — Proprietary CLI for Yandex Disk.

: https://disk.yandex.ru/ || yandex-diskAUR

FTP

FTP 客户端

参见 Wikipedia:Comparison of FTP client software。

- **FileZilla** — Fast and reliable FTP, FTPS and SFTP client.

: https://filezilla-project.org/ || filezilla包

- **gFTP** — Multithreaded FTP client for Linux.

: https://www.gftp.org/ || gftp包

- **ftp** — Simple ftp client provided by GNU Inetutils

: https://www.gnu.org/software/inetutils/manual/inetutils.html#ftp-invocation || inetutils包

- **ncftp** — A set of free application programs implementing FTP.

: https://www.ncftp.com/ || ncftp包

- **tnftp** — FTP client with several advanced features for NetBSD.

: https://freshmeat.sourceforge.net/projects/tnftp || tnftp包

一些文件管理器比如 Dolphin, GNOME Files 和 Thunar 也提供 FTP 功能。

FTP 服务器

参见 zhwp:FTP服务器列表。

- **bftpd** — Small, easy-to-configure FTP server

: https://bftpd.sourceforge.net/ || bftpd包

- **ftpd** — Simple ftp server provided by GNU Inetutils

: https://www.gnu.org/software/inetutils/manual/inetutils.html#ftpd-invocation || inetutils包

- **proFTPd** — A secure and configurable FTP server

: http://www.proftpd.org/ || proftpdAUR

- **Pure-FTPd** — Free (BSD-licensed), secure, production-quality and standard-compliant FTP server.

: https://www.pureftpd.org/project/pure-ftpd/ || pure-ftpdAUR

- **vsftpd** — Lightweight, stable and secure FTP server for UNIX-like systems.

: https://security.appspot.com/vsftpd.html || vsftpd包

BitTorrent 客户端

一些下载管理器也可以连接到 BitTorrent 网络：Aria2, LFTP, FatRat, KGet, MLDonkey, uGet。

另见 Wikipedia:Comparison of BitTorrent clients。

控制台

- **Ctorrent** — CTorrent is a BitTorrent client implemented in C++ to be lightweight and quick.

: http://www.rahul.net/dholmes/ctorrent/ || enhanced-ctorrentAUR

- **Deluge** — BitTorrent client with multiple user interfaces in a client/server model. This package includes a console client.

: https://deluge-torrent.org/ || deluge包

- **peerflix** — Streaming torrent client for node.js.

: https://github.com/mafintosh/peerflix || peerflixAUR

- **rTorrent** — Simple and lightweight ncurses BitTorrent client.

: https://rakshasa.github.io/rtorrent/ || rtorrent包

- **Transmission CLI** — Simple and easy-to-use BitTorrent client with a daemon version and multiple front-ends. This package includes backend, daemon, command-line interface, and a Web UI interface.

: https://transmissionbt.com/ || transmission-cli包

图形界面

- **Deluge (GTK interface)** — User-friendly BitTorrent client written in Python using GTK.

: https://deluge-torrent.org/ || deluge-gtk包

- **Fragments** — Easy to use BitTorrent client for the GNOME desktop environment.

: https://apps.gnome.org/Fragments/ || fragments包

- **FrostWire** — Easy to use cloud downloader, BitTorrent client and media player.

: https://www.frostwire.com/ || frostwireAUR

- **Ktorrent** — Feature-rich BitTorrent client for KDE. Part of kde-network包组.

: https://apps.kde.org/ktorrent/ || ktorrent包

- **Powder Player** — Hybrid between a streaming BitTorrent client and a player. Based on the Electron platform.

: https://powder.media/ || powder-player-binAUR

- **qBittorrent** — Open source (GPLv2) BitTorrent client with an integrated torrent search engine that strongly resembles µTorrent.

: https://www.qbittorrent.org/ || qbittorrent包

- **Tixati** — Proprietary peer-to-peer file sharing program that uses the popular BitTorrent protocol.

: https://tixati.com/ || tixatiAUR

- **Torrential** — Simple torrent client for elementary OS.

: https://github.com/davidmhewitt/torrential || torrentialAUR

- **Transmission** — Simple and easy-to-use BitTorrent client with a daemon version and multiple front-ends.

: https://transmissionbt.com/ || GTK: transmission-gtk包, Qt: transmission-qt包

- **Transmission Remote** — GTK client for remote management of the Transmission BitTorrent client, using its HTTP RPC protocol.

: https://github.com/transmission-remote-gtk/transmission-remote-gtk || transmission-remote-gtk包

- **Tremotesf** — Qt client for remote management of the Transmission BitTorrent client, using its HTTP RPC protocol.

: https://github.com/equeim/tremotesf2 || tremotesfAUR

- **Tribler** — 4th generation file sharing system BitTorrent client.

: https://www.tribler.org || triblerAUR

- **Vuze** — Feature-rich BitTorrent client written in Java (formerly Azureus).

: https://www.vuze.com/ || vuzeAUR

- **WebTorrent Desktop** — Streaming BitTorrent application. Based on the Electron platform.

: https://webtorrent.io/desktop/ || webtorrent-desktopAUR

其它 P2P 网络

参见 Wikipedia:Comparison of file-sharing applications。

- **aMule** — Well-known eDonkey/Kad client with a daemon version and GTK, web, and CLI front-ends.

: https://www.amule.org/ || amule包

- **EiskaltDC++** — Direct Connect and ADC client.

: https://github.com/eiskaltdcpp/eiskaltdcpp || GTK: eiskaltdcpp-gtkAUR, Qt: eiskaltdcpp-qtAUR

- **gtk-gnutella** — GTK server/client for the Gnutella peer-to-peer network.

: https://gtk-gnutella.sourceforge.net/ || gtk-gnutellaAUR

- **KaMule** — KDE graphical front-end for aMule.

: https://github.com/nihui/kamule/ || kamuleAUR

- **LBRY** — Browser and wallet for LBRY, the decentralized, user-controlled content marketplace. Based on the Electron platform.

: https://lbry.io/ || lbry-desktopAUR

- **lbt** — Small set of command-line tools for LBRY.

: https://gitlab.com/gardenappl/lbt || lbtAUR

- **MLDonkey** — Multi-protocol P2P client that supports HTTP, FTP, BitTorrent, Direct Connect, eDonkey and FastTrack.

: https://mldonkey.sourceforge.net/ || mldonkeyAUR

- **ncdc** — Modern and lightweight Direct Connect and ADC client with a friendly ncurses interface.

: https://dev.yorhel.nl/ncdc || ncdcAUR

- **Nicotine+** — A graphical client for the Soulseek P2P network.

: https://github.com/Nicotine-Plus/nicotine-plus || nicotine+包

- **Send Anywhere** — Proprietary file sharing service where users can directly share digital content in real time. Based on the Electron platform.

: https://send-anywhere.com/ || sendanywhereAUR

- **IPFS** — IPFS is a P2P Network capable of sharing and receiving files.

: https://ipfs.io/ || kubo包

网络剪切板

参见 Wikipedia:Pastebin。

网络剪切板通常用于在协作和故障排除时引用文本或图像。Pastebin 客户端提供了从命令行发布消息的便捷方式。

注意：通常来说，建议使用不需要JavaScript即可查看、不显示广告、不会操控上传内容、不需要登录的网络剪切板。 pastebin.com 已被部分人屏蔽，除去一直以来的使用问题（必须启用JavaScript、显示广告、自行插入CRLF、随机显示验证码）。**非常不建议**使用该服务。

不用专用客户端

一些服务可以与更通用的命令行工具一起使用，例如 cURL。为了扩展，例如行号，可以使用更多的命令行工具。例如 `cat -n`。

- 0x0.st 是一个文件托管服务。用法示例如下：

: ``` $ command | curl -F 'file=@-' 0x0.st

或上传文件：

$ curl -F 'file=@-' 0x0.st < file

: > **注意：**Read the instructions on the website for further options such as file removal tokens, expiration, and private URLs.

- termbin.com works with nc.
- paste.c-net.org accepts HTTP requests and works with nc.

专用客户端

- **Fb-client** — Client for the paste.xinu.at pastebin.

: https://paste.xinu.at || [fb-client](https://archlinux.org/packages/?name=fb-client)包

- **Gist** — Command-line interface for the gist.github.com pastebin service.

: https://github.com/defunkt/gist || [gist](https://archlinux.org/packages/?name=gist)包

- **imgur** — A CLI client which can upload image to imgur.com image sharing service.

: https://github.com/tremby/imgur.sh || [imgur.sh](https://aur.archlinux.org/packages/imgur.sh/)AUR

- **Pastebinit** — Really small Python script that acts as a Pastebin client (see `pastebinit -l` for the list of servers).

: https://launchpad.net/pastebinit || [pastebinit](https://archlinux.org/packages/?name=pastebinit)包

- **ruby-haste** — Client for hastebin.com.

: https://github.com/seejohnrun/haste-client || [ruby-haste](https://aur.archlinux.org/packages/ruby-haste/)AUR

- **Wgetpaste** — Bash script that automates pasting to a number of pastebin services.

: http://wgetpaste.zlin.dk/ || [wgetpaste](https://archlinux.org/packages/?name=wgetpaste)包

通讯

Email 客户端

参见 [Wikipedia:Comparison of email clients](https://en.wikipedia.org/wiki/Comparison_of_email_clients)。

控制台

- **[aerc](../邮件客户端/Aerc.md)** — Work in progress asynchronous email client.

: https://sr.ht/~rjarry/aerc || [aerc](https://archlinux.org/packages/?name=aerc)包

- **alot** — An experimental terminal MUA based on [notmuch mail](https://notmuchmail.org/). It is written in python using the [urwid](https://urwid.org/) toolkit.

: https://github.com/pazz/alot || [alot](https://archlinux.org/packages/?name=alot)包

- **Alpine** — Fast, easy-to-use and Apache-licensed email client based on [Pine](https://en.wikipedia.org/wiki/Pine_(email_client)).

: https://alpineapp.email/ || [alpine-git](https://aur.archlinux.org/packages/alpine-git/)AUR

- **S-nail** — a mail processing system with a command syntax reminiscent of *ed* with lines replaced by messages. Provides the functionality of [mailx](https://en.wikipedia.org/wiki/mailx).

: https://www.sdaoden.eu/code.html#s-mailx || [s-nail](https://archlinux.org/packages/?name=s-nail)包

- **mu/mu4e** — Email indexer (mu) and client for emacs (mu4e). Xapian based for fast searches.

: https://www.djcbsoftware.nl/code/mu/mu4e.html || [mu](https://aur.archlinux.org/packages/mu/)AUR

- **[Mutt](../邮件客户端/Mutt.md)** — Small but very powerful text-based mail client.

: http://www.mutt.org/ || [mutt](https://archlinux.org/packages/?name=mutt)包

- **[NeoMutt](../邮件客户端/Mutt.md)** — Command line mail reader (or MUA). It is a fork of Mutt with added features.

: https://neomutt.org/ || [neomutt](https://archlinux.org/packages/?name=neomutt)包

- **nmh** — A modular mail handling system.

: https://www.nongnu.org/nmh/ || [nmh](https://aur.archlinux.org/packages/nmh/)AUR

- **[notmuch](../邮件客户端/Notmuch.md)** — A fast mail indexer built on top of *xapian*.

: https://notmuchmail.org/ || [notmuch](https://archlinux.org/packages/?name=notmuch)包

- **sendemail** — A lightweight command line SMTP email client written in Perl.

: http://caspian.dotconf.net/menu/Software/SendEmail/ || [sendemail](https://aur.archlinux.org/packages/sendemail/)AUR

- **[Sup](../邮件客户端/Sup.md)** — CLI mail client with very fast searching, tagging, threading and GMail like operation.

: https://sup-heliotrope.github.io/ || [sup](https://aur.archlinux.org/packages/sup/)AUR

- **swaks** — Swiss Army Knife SMTP; Command line SMTP testing, including TLS and AUTH, can be used to send emails.

: https://jetmore.org/john/code/swaks/ || [swaks](https://archlinux.org/packages/?name=swaks)包

- **Wanderlust** — Email client and news reader for Emacs.

: https://github.com/wanderlust/wanderlust/ || [wanderlust](https://archlinux.org/packages/?name=wanderlust)包

图形界面

- **Balsa** — Simple and light email client for GNOME.

: https://pawsa.fedorapeople.org/balsa/ || [balsa](https://archlinux.org/packages/?name=balsa)包

- **Betterbird** — Fork of thunderbird.

: https://www.betterbird.eu/ || [betterbird-bin](https://aur.archlinux.org/packages/betterbird-bin/)AUR

- **[Claws Mail](https://en.wikipedia.org/wiki/Claws_Mail)** — Lightweight GTK-based email client and news reader.

: https://www.claws-mail.org/ || [claws-mail](https://archlinux.org/packages/?name=claws-mail)包

- **ElectronMail** — Unofficial desktop application for several end-to-end encrypted email providers (like ProtonMail, Tutanota). Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/vladimiry/ElectronMail || [electronmail-bin](https://aur.archlinux.org/packages/electronmail-bin/)AUR

- **Evolution** — Mature and feature-rich e-mail client that is part of the GNOME project. Part of [gnome-extra](https://archlinux.org/groups/x86_64/gnome-extra/)包组.

: https://gitlab.gnome.org/GNOME/evolution/-/wikis/home || [evolution](https://archlinux.org/packages/?name=evolution)包

- **Geary** — Simple desktop mail client built in [Vala](https://en.wikipedia.org/wiki/Vala_(programming_language)). Part of [gnome-extra](https://archlinux.org/groups/x86_64/gnome-extra/)包组.

: https://wiki.gnome.org/Apps/Geary || [geary](https://archlinux.org/packages/?name=geary)包

- **[Kmail](https://en.wikipedia.org/wiki/Kmail)** — Mature and feature-rich email client. Part of [kde-pim](https://archlinux.org/groups/x86_64/kde-pim/)包组.

: https://kontact.kde.org/components/kmail/ || [kmail](https://archlinux.org/packages/?name=kmail)包

- **Kube** — Modern communication and collaboration client built with QtQuick.

: https://kube-project.com/ || [kube](https://aur.archlinux.org/packages/kube/)AUR

- **Mailspring** — Fork of [Nylas Mail](https://en.wikipedia.org/wiki/Nylas_Mail) by one of the original authors. The paid "Pro" version requires a Mailspring ID and has extra features like snooze, send later. Based on the [Electron](https://electronjs.org/) platform.

: https://getmailspring.com/ || [mailspring](https://aur.archlinux.org/packages/mailspring/)AUR

- **[SeaMonkey Mail & Newsgroups](https://en.wikipedia.org/wiki/SeaMonkey#Mail)** — Email client included in the SeaMonkey suite.

: https://www.seamonkey-project.org/ || [seamonkey](https://aur.archlinux.org/packages/seamonkey/)AUR

- **[Thunderbird](../../06_软件与应用/Mozilla/Thunderbird.md)** — Feature-rich email client from Mozilla written in GTK.

: https://www.thunderbird.net/ || [thunderbird](https://archlinux.org/packages/?name=thunderbird)包

- **Tutanota** — Email client for Tutanota mail service. Based on the [Electron](https://electronjs.org/) platform.

: https://tutanota.com/ || [tutanota-desktop](https://aur.archlinux.org/packages/tutanota-desktop/)AUR

- **Viagee** — It allows desktop mail actions, such as 'Send File as Email' or web 'mailto' links, to be handled by the Gmail web client.

: https://davesteele.github.io/viagee/ || [viagee](https://aur.archlinux.org/packages/viagee/)AUR

基于网络

- **[Nextcloud](../文件分享/Nextcloud.md) Mail** — An email webapp for NextCloud.

: https://github.com/nextcloud/mail || [nextcloud-app-mail](https://archlinux.org/packages/?name=nextcloud-app-mail)包

- **Roundcube** — Browser-based multilingual IMAP client webapp with a native application-like user interface.

: https://roundcube.net/ || [roundcubemail](https://archlinux.org/packages/?name=roundcubemail)包

- **SquirrelMail** — Webmail for Nuts!

: https://squirrelmail.org/ || [squirrelmail](https://aur.archlinux.org/packages/squirrelmail/)AUR

邮件通知程序

- **Ayatana Webmail** — Webmail notifications and actions for any desktop.

: https://tari.in/www/software/ayatana-webmail/ || [ayatana-webmail](https://aur.archlinux.org/packages/ayatana-webmail/)AUR

- **Bubblemail** — New and Unread mail notification service for local mailboxes, pop, imap, and gnome online accounts. A fork of Mailnag.

: http://bubblemail.free.fr/ || [bubblemail](https://aur.archlinux.org/packages/bubblemail/)AUR

- **Gnubiff** — Mail notification program that checks for mail and displays headers when new mail has arrived.

: https://gnubiff.sourceforge.net/ || [gnubiff](https://archlinux.org/packages/?name=gnubiff)包

- **Mailnag** — Extensible mail notification daemon.

: https://github.com/pulb/mailnag || [mailnag](https://archlinux.org/packages/?name=mailnag)包

邮件服务器

参见 [Mail server](../邮件服务器/邮件服务器.md)。

- **DavMail** — POP/IMAP/SMTP/Caldav/Carddav/LDAP exchange gateway allowing users to use any mail/calendar client with an Exchange server.

: https://davmail.sourceforge.net/ || [davmail](https://aur.archlinux.org/packages/davmail/)AUR

- **Modoboa** — A modular mail hosting and management platform, written in Python.

: https://modoboa.org/ || [modoboa](https://aur.archlinux.org/packages/modoboa/)AUR

邮件检索代理

参见 [Wikipedia:Mail retrieval agent](https://en.wikipedia.org/wiki/Mail_retrieval_agent)。

- **fdm** — Program to fetch and deliver mail.

: https://github.com/nicm/fdm || [fdm](https://archlinux.org/packages/?name=fdm)包

- **[Fetchmail](https://en.wikipedia.org/wiki/Fetchmail)** — A remote-mail retrieval utility.

: https://www.fetchmail.info/ || [fetchmail](https://archlinux.org/packages/?name=fetchmail)包

- **getmail** — A POP3/IMAP4 mail retriever with reliable Maildir and command delivery.

: http://pyropus.ca/software/getmail/ || [getmail](https://aur.archlinux.org/packages/getmail/)AUR

- **hydroxide** — A third-party, open-source ProtonMail CardDAV, IMAP and SMTP bridge

: https://github.com/emersion/hydroxide || [hydroxide](https://aur.archlinux.org/packages/hydroxide/)AUR

- **imapsync** — IMAP synchronisation, sync, copy or migration tool

: https://imapsync.lamiral.info/ || [imapsync](https://archlinux.org/packages/?name=imapsync)包

- **isync** — IMAP and MailDir mailbox synchronizer

: https://isync.sourceforge.net/ || [isync](https://archlinux.org/packages/?name=isync)包

- **mpop** — A small, fast POP3 client suitable as a fetchmail replacement

: https://marlam.de/mpop/ || [mpop](https://archlinux.org/packages/?name=mpop)包

- **vomit** — Rust utility to sync between Maildir mailbox and IMAP mailbox

: https://git.sr.ht/~bitfehler/vomit-sync/tree/master/item/cli/README.md || [vsync](https://aur.archlinux.org/packages/vsync/)AUR (sync-only part of [vomit](https://aur.archlinux.org/packages/vomit/)AUR mail tookit)

- **OfflineIMAP** — Synchronizes emails between two repositories.

: https://www.offlineimap.org/ || [offlineimap](https://archlinux.org/packages/?name=offlineimap)包

即时通讯客户端

参见 [Wikipedia:Comparison of instant messaging clients](https://en.wikipedia.org/wiki/Comparison_of_instant_messaging_clients) 和 [Wikipedia:Comparison of VoIP software](https://en.wikipedia.org/wiki/Comparison_of_VoIP_software)。

本节列出了所有支持[即时通讯](https://en.wikipedia.org/wiki/Instant_messaging)的客户端软件。

多协议客户端

> **注意：**所有通过直接连接支持多个网络的 messengers 都属于本节。

这些客户端支持的网络数量非常多，但它们（像任何多协议客户端一样）通常对网络特定功能的支持非常有限或没有。

控制台

- **BitlBee** — IRC gateway to popular chat networks.

: https://bitlbee.org/ || [bitlbee](https://aur.archlinux.org/packages/bitlbee/)AUR

- **Finch** — Ncurses-based chat client that uses libpurple and supports all its protocols (Bonjour, Gadu-Gadu, Groupwise, IRC, SIMPLE, XMPP, Zephyr).

: https://developer.pidgin.im/wiki/Using%20Finch || [finch](https://archlinux.org/packages/?name=finch)包

- **WeeChat** — Modular, lightweight ncurses-based IRC client. A variety of other protocols are supported through plugins.

: https://weechat.org/ || [weechat](https://archlinux.org/packages/?name=weechat)包

图形界面

- **[Jitsi](https://en.wikipedia.org/wiki/Jitsi)** — Audio/video VoIP phone and instant messenger written in Java that supports protocols such as SIP, XMPP, IRC and many other useful features.

: https://jitsi.org/ || [jitsi](https://aur.archlinux.org/packages/jitsi/)AUR

- **Lith** — WeeChat Relay client, allowing to connect to a running WeeChat instance from anywhere.

: https://lith.app/ || [lith-git](https://aur.archlinux.org/packages/lith-git/)AUR

- **[Pidgin](../XMPP/Pidgin.md)** — Multi-protocol instant messaging client with audio support that uses libpurple and supports all its protocols (Bonjour, Gadu-Gadu, Groupwise, IRC, SIMPLE, XMPP, Zephyr).

: https://pidgin.im/ || [pidgin](https://archlinux.org/packages/?name=pidgin)包

- **[Smuxi](https://en.wikipedia.org/wiki/Smuxi)** — Cross-platform IRC client that also supports XMPP.

: https://smuxi.im/ || [smuxi](https://aur.archlinux.org/packages/smuxi/)AUR

- **[Thunderbird](../../06_软件与应用/Mozilla/Thunderbird.md)** — Feature-rich email client supports instant messaging and chat using IRC and XMPP.

: https://www.thunderbird.net/ || [thunderbird](https://archlinux.org/packages/?name=thunderbird)包

IRC 客户端

参见 [Wikipedia:Comparison of Internet Relay Chat clients](https://en.wikipedia.org/wiki/Comparison_of_Internet_Relay_Chat_clients)。

控制台

- **[BitchX](https://en.wikipedia.org/wiki/BitchX)** — Console-based IRC client developed from the popular [ircII](https://en.wikipedia.org/wiki/ircII).

: https://bitchx.sourceforge.net/ || [bitchx-git](https://aur.archlinux.org/packages/bitchx-git/)AUR

- **catgirl** — TLS-only terminal IRC client.

: https://git.causal.agency/catgirl || [catgirl](https://aur.archlinux.org/packages/catgirl/)AUR

- **ERC** — Powerful, modular and extensible IRC client for [Emacs](../../07_开发与工具/Emacs/Emacs.md).

: https://savannah.gnu.org/projects/erc/ || included with [emacs](https://archlinux.org/packages/?name=emacs)包

- **ii** — Featherweight IRC client, literally `tail -f` the conversation and `echo` back your replies to a file.

: https://tools.suckless.org/ii/ || [ii](https://aur.archlinux.org/packages/ii/)AUR

- **ircii** — Oldest maintained IRC client which lays claim to being small and fast owing to its reduced feature set.

: http://www.eterna23.net/ircii/ || [ircii](https://aur.archlinux.org/packages/ircii/)AUR

- **[Irssi](../互联网中继聊天/Irssi.md)** — Highly-configurable ncurses-based IRC client.

: https://irssi.org/ || [irssi](https://archlinux.org/packages/?name=irssi)包

- **pork** — Programmable, ncurses-based IRC client that mostly looks and feels like ircII.

: http://dev.ojnk.net/ || [pork](https://archlinux.org/packages/?name=pork)包

- **ScrollZ** — Advanced IRC client based on [ircII](https://en.wikipedia.org/wiki/ircII).

: https://www.scrollz.info/ || [scrollz](https://aur.archlinux.org/packages/scrollz/)AUR

- **senpai** — An IRC client that works best with bouncers (e.g. [soju-git](https://aur.archlinux.org/packages/soju-git/)AUR): no logs are kept, history is

fetched from the server via [CHATHISTORY](https://ircv3.net/specs/extensions/chathistory), networks are fetched from the server via [bouncer-networks](https://git.sr.ht/~emersion/soju/tree/master/item/doc/ext/bouncer-networks.md).

: https://sr.ht/~delthas/senpai/ || [senpai](https://archlinux.org/packages/?name=senpai)包

- **sic** — Extremely simple IRC client, similar to [ii](https://en.wikipedia.org/wiki/Ii_(IRC_client)).

: https://tools.suckless.org/sic/ || [sic](https://aur.archlinux.org/packages/sic/)AUR

- **tiny** — an IRC client written in Rust with a clutter-free interface

: https://github.com/osa1/tiny || [tiny](https://archlinux.org/packages/?name=tiny)包

图形界面

- **[ChatZilla](https://en.wikipedia.org/wiki/ChatZilla)** — Clean, easy to use and highly extensible Internet Relay Chat (IRC) client, built on the Mozilla platform using [XULRunner](https://en.wikipedia.org/wiki/XULRunner). Included in the [SeaMonkey](https://en.wikipedia.org/wiki/SeaMonkey) suite.

: http://chatzilla.hacksrus.com/ || [seamonkey](https://aur.archlinux.org/packages/seamonkey/)AUR

- **Halloy** — An open-source IRC client written in Rust, with the iced GUI library.

: https://halloy.squidowl.org/ || [halloy](https://aur.archlinux.org/packages/halloy/)AUR

- **HexChat** — Fork of XChat for Linux and Windows.

: https://hexchat.github.io/ || [hexchat](https://archlinux.org/packages/?name=hexchat)包

- **[Konversation](https://en.wikipedia.org/wiki/Konversation)** — Qt-based IRC client for the KDE desktop. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://konversation.kde.org/ || [konversation](https://archlinux.org/packages/?name=konversation)包

- **[KVIrc](https://en.wikipedia.org/wiki/KVIrc)** — Qt-based IRC client featuring extensive themes support.

: https://kvirc.net/ || [kvirc](https://archlinux.org/packages/?name=kvirc)包

- **Loqui** — GTK IRC client.

: https://loqui.sunnyone.org/ || [loqui](https://aur.archlinux.org/packages/loqui/)AUR

- **LostIRC** — Simple GTK IRC client with tab-autocompletion, multiple server support, logging and others.

: https://lostirc.sourceforge.net || [lostirc](https://aur.archlinux.org/packages/lostirc/)AUR

- **Polari** — Simple IRC client by the GNOME project. Part of [gnome-extra](https://archlinux.org/groups/x86_64/gnome-extra/)包组.

: https://apps.gnome.org/Polari/ || [polari](https://archlinux.org/packages/?name=polari)包

- **Quassel** — Modern, cross-platform, distributed IRC client.

: https://quassel-irc.org/ || KDE: [quassel-monolithic](https://archlinux.org/packages/?name=quassel-monolithic)包, Qt: [quassel-monolithic-qt](https://archlinux.org/packages/?name=quassel-monolithic-qt)包

- **Srain** — Modern, beautiful IRC client written in GTK 3.

: https://srain.silverrainz.me || [srain](https://aur.archlinux.org/packages/srain/)AUR

- **Thelounge** — Modern self-hosted web IRC client

: https://thelounge.chat/ || [thelounge](https://aur.archlinux.org/packages/thelounge/)AUR

XMPP 客户端

参见 [Wikipedia:XMPP](https://en.wikipedia.org/wiki/XMPP) 和 [Wikipedia:Comparison of XMPP clients](https://en.wikipedia.org/wiki/Comparison_of_XMPP_clients)。

控制台

- **Freetalk** — Console-based XMPP client.

: https://www.gnu.org/software/freetalk/ || [freetalk](https://aur.archlinux.org/packages/freetalk/)AUR

- **jabber.el** — Minimal XMPP client for [Emacs](../../07_开发与工具/Emacs/Emacs.md).

: https://emacs-jabber.sourceforge.net/ || [emacs-jabber](https://aur.archlinux.org/packages/emacs-jabber/)AUR

- **MCabber** — Small XMPP console client, includes features: SSL, PGP, MUC, OTR and UTF8.

: https://mcabber.com/ || [mcabber](https://archlinux.org/packages/?name=mcabber)包

- **Poezio** — XMPP client with IRC feeling

: https://poez.io/ || [poezio](https://aur.archlinux.org/packages/poezio/)AUR

- **Profanity** — A console based XMPP client inspired by Irssi.

: https://profanity-im.github.io/ || [profanity](https://archlinux.org/packages/?name=profanity)包

图形界面

- **Converse.js** — Web-based XMPP chat client written in JavaScript.

: https://conversejs.org/ || [conversejs-git](https://aur.archlinux.org/packages/conversejs-git/)AUR

- **Dino** — A modern, easy to use XMPP client, with PGP and OMEMO support.

: https://dino.im/ || [dino](https://archlinux.org/packages/?name=dino)包

- **Gajim** — XMPP client with audio support written in Python using GTK.

: https://gajim.org/ || [gajim](https://archlinux.org/packages/?name=gajim)包

- **Kaidan** — A simple, user-friendly Jabber/XMPP client providing a modern user interface using Kirigami and QtQuick.

: https://www.kaidan.im/ || [kaidan](https://archlinux.org/packages/?name=kaidan)包

- **Libervia (Salut à Toi)** — Web frontend for Salut à Toi, multi-purpose XMPP client

: https://libervia.org/ || [libervia-web-hg](https://aur.archlinux.org/packages/libervia-web-hg/)AUR

- **Nextcloud JavaScript XMPP Client** — Chat app for Nextcloud with XMPP, end-to-end encryption, video calls, file transfer & group chat.

: https://github.com/nextcloud/jsxc.nextcloud || [nextcloud-app-jsxc](https://aur.archlinux.org/packages/nextcloud-app-jsxc/)AUR

- **[Psi](https://en.wikipedia.org/wiki/Psi_(instant_messaging_client))** — Qt-based XMPP client.

: https://psi-im.org/ || [psi](https://archlinux.org/packages/?name=psi)包 or [psi-nowebengine](https://archlinux.org/packages/?name=psi-nowebengine)包

- **[Spark](https://en.wikipedia.org/wiki/Spark_(XMPP_client))** — Cross-platform real-time XMPP collaboration client optimized for business and organizations.

: https://www.igniterealtime.org/projects/spark/ || [spark](https://aur.archlinux.org/packages/spark/)AUR

- **Swift** — XMPP client written in C++ with Qt and Swiften.

: https://swift.im/ || [swift-im](https://aur.archlinux.org/packages/swift-im/)AUR

- **[Tkabber](https://en.wikipedia.org/wiki/Tkabber)** — Easy to hack feature-rich XMPP client by the author of the ejabberd XMPP server.

: https://tkabber.jabber.ru/ || [tkabber](https://aur.archlinux.org/packages/tkabber/)AUR

- **Vacuum IM** — Full-featured crossplatform XMPP client.

: https://github.com/Vacuum-IM/vacuum-im || [vacuum-im](https://aur.archlinux.org/packages/vacuum-im/)AUR

SIP 客户端

参见 [Wikipedia:List of SIP software#Clients](https://en.wikipedia.org/wiki/List_of_SIP_software#Clients)。

- **[Blink](https://en.wikipedia.org/wiki/Blink_(SIP_client))** — State of the art, easy to use SIP client.

: https://icanblink.com/ || [blink](https://aur.archlinux.org/packages/blink/)AUR

- **baresip** — portable and modular SIP User-Agent with audio and video support.

: https://github.com/baresip/baresip || [baresip](https://archlinux.org/packages/?name=baresip)包

- **Calls** — Simple, elegant phone dialer and call handler for GNOME. It can be used with a cellular modem for plain old telephone calls as well as VoIP calls using the SIP protocol.

: https://gitlab.gnome.org/GNOME/calls || [gnome-calls](https://archlinux.org/packages/?name=gnome-calls)包

- **Jami** — SIP-compatible softphone and instant messenger for the decentralized Jami network. Formerly known as Ring and SFLphone.

: https://jami.net/ || [jami-qt](https://archlinux.org/packages/?name=jami-qt)包

- **[Linphone](https://en.wikipedia.org/wiki/Linphone)** — VoIP phone application (SIP client) for communicating freely with people over the internet, with voice, video, and text instant messaging.

: https://www.linphone.org/ || [linphone-desktop](https://aur.archlinux.org/packages/linphone-desktop/)AUR

- **[Twinkle](https://en.wikipedia.org/wiki/Twinkle_(software))** — Qt softphone for VoIP and IM communication using SIP.

: http://twinkle.dolezel.info/ || [twinkle](https://aur.archlinux.org/packages/twinkle/)AUR

- **Zoiper** — Proprietary SIP and IAX2 VoIP softphone

: https://zoiper.com || [zoiper-bin](https://aur.archlinux.org/packages/zoiper-bin/)AUR

Matrix 客户端

参见 [Matrix](../网络电话/Matrix.md) 和 [Matrix Clients](https://matrix.org/clients/)。

- **Chatty** — Simple to use SMS and Matrix messaging application.

: https://gitlab.gnome.org/World/Chatty || [chatty](https://archlinux.org/packages/?name=chatty)包

- **Cinny** — Cinny is a matrix client focusing primarily on simple, elegant and secure interface. The desktop app is made with [Tauri](https://github.com/tauri-apps/tauri).

: https://github.com/cinnyapp/cinny-desktop || [cinny-desktop](https://aur.archlinux.org/packages/cinny-desktop/)AUR

- **Element** — Glossy Matrix client with an emphasis on performance and usability. Web application and desktop application based on the [Electron](https://electronjs.org/) platform.

: https://element.io/ || [element-web](https://archlinux.org/packages/?name=element-web)包, [element-desktop](https://archlinux.org/packages/?name=element-desktop)包

- **FluffyChat** — Multi-platform Matrix client with a simple and clean UI written in Dart/Flutter.

: https://fluffychat.im/ || [fluffychat](https://aur.archlinux.org/packages/fluffychat/)AUR

- **Fractal** — Matrix client for GNOME written in Rust.

: https://wiki.gnome.org/Apps/Fractal || [fractal](https://archlinux.org/packages/?name=fractal)包

- **Gomuks** — Terminal Matrix client written in Go using [mautrix](https://github.com/tulir/mautrix-go) and [mauview](https://github.com/tulir/mauview).

: https://maunium.net/go/gomuks || [gomuks](https://archlinux.org/packages/?name=gomuks)包

- **iamb** — A terminal-based Matrix client with Vim keybindings written in Rust.

: https://github.com/ulyssa/iamb || [iamb](https://aur.archlinux.org/packages/iamb/)AUR

- **Moment** — A fancy, customizable, keyboard-operable Matrix chat client for encrypted and decentralized communication. Written in Qt/QML + Python with nio, fork of the now-abandoned Mirage.

: https://mx-moment.xyz/ || [moment](https://aur.archlinux.org/packages/moment/)AUR

- **Neochat** — KDE client for the Matrix protocol. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://apps.kde.org/neochat/ || [neochat](https://archlinux.org/packages/?name=neochat)包

- **nheko** — Desktop client for the Matrix protocol.

: https://github.com/Nheko-Reborn/nheko || [nheko](https://archlinux.org/packages/?name=nheko)包

- **Quaternion** — Qt5-based IM client for the Matrix protocol.

: https://github.com/QMatrixClient/Quaternion || [quaternion](https://aur.archlinux.org/packages/quaternion/)AUR

- **QuickMedia** — A rofi inspired native client for web services. Supports Matrix and several other sites.

: https://git.dec05eba.com/QuickMedia/about/ || [quickmedia](https://aur.archlinux.org/packages/quickmedia/)AUR

- **SchildiChat** — Matrix client based on Element with a more traditional instant messaging experience. Based on the [Electron](https://electronjs.org/) platform.

: https://schildi.chat/ || [schildichat-desktop](https://aur.archlinux.org/packages/schildichat-desktop/)AUR

- **Spectral** — Qt5-based Glossy cross-platform client for Matrix.

: https://gitlab.com/spectral-im/spectral || [spectral-matrix](https://aur.archlinux.org/packages/spectral-matrix/)AUR

- **Syphon** — Privacy-centric cross-platform Matrix client with E2EE support, currently in alpha.

: https://github.com/syphon-org/syphon || [syphon-bin](https://aur.archlinux.org/packages/syphon-bin/)AUR

Tox 客户端

参见 Tox 和 [comparison clients](https://wiki.tox.chat/clients)。

- **ratox** — FIFO based tox client.

: https://git.z3bra.org/ratox/file/README.html || [ratox-git](https://aur.archlinux.org/packages/ratox-git/)AUR

- **Toxic** — ncurses-based Tox client

: https://github.com/Jfreegman/toxic || [toxic](https://archlinux.org/packages/?name=toxic)包

- **Toxygen** — Tox client written in pure Python3.

: https://github.com/toxygen-project/toxygen || [toxygen-git](https://aur.archlinux.org/packages/toxygen-git/)AUR

- **Venom** — a modern Tox client for the GNU/Linux desktop

: https://github.com/naxuroqa/Venom || [venom](https://aur.archlinux.org/packages/venom/)AUR

- **µTox** — Lightweight Tox client.

: https://github.com/uTox/uTox || [utox](https://archlinux.org/packages/?name=utox)包

局域网 messengers

参见 [Avahi#Link-Local (Bonjour/Zeroconf) chat](../多播DNS/Avahi.md#Link-Local_(Bonjour/Zeroconf)_chat) 和 [Wikipedia:Comparison of LAN messengers](https://en.wikipedia.org/wiki/Comparison_of_LAN_messengers)。

- **BeeBEEP** — Secure LAN Messenger.

: https://www.beebeep.net/ || [beebeep](https://archlinux.org/packages/?name=beebeep)包

- **iptux** — LAN communication software, compatible with IP Messenger.

: https://github.com/iptux-src/iptux || [iptux](https://archlinux.org/packages/?name=iptux)包

P2P 消息客户端

参见 Ring 和 Tox。

- **[Briar](https://en.wikipedia.org/wiki/Briar_(software))** — Briar is a messaging application designed for activists, journalists, and anyone else who needs a safe, easy and robust way to communicate.

: https://briarproject.org/ || [briar-desktop](https://aur.archlinux.org/packages/briar-desktop/)AUR, [briar-headless](https://aur.archlinux.org/packages/briar-headless/)AUR

- **Patchwork** — Decentralized messaging and sharing application built on top of Secure Scuttlebutt (SSB). Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/ssbc/patchwork || [ssb-patchwork](https://aur.archlinux.org/packages/ssb-patchwork/)AUR

- **Manyverse** — Modern decentralized messaging and sharing application built on top of Secure Scuttlebutt (SSB).

: https://gitlab.com/staltz/manyverse || [manyverse-bin](https://aur.archlinux.org/packages/manyverse-bin/)AUR

- **RetroShare** — Serverless encrypted instant messenger with filesharing, chatgroups, mail.

: https://retroshare.cc/ || [retroshare](https://aur.archlinux.org/packages/retroshare/)AUR

- **[Ricochet](https://en.wikipedia.org/wiki/Ricochet_(software))** — Anonymous peer-to-peer instant messaging system built on [Tor](../互联网应用/Tor.md) hidden services.

: https://www.ricochetrefresh.net/ || [ricochet-refresh](https://aur.archlinux.org/packages/ricochet-refresh/)AUR

其它即时通讯客户端

- **Caprine** — Unofficial Facebook Messenger app. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/sindresorhus/caprine || [caprine](https://aur.archlinux.org/packages/caprine/)AUR

- **Chatterino** — Chat client for Twitch chat.

: https://chatterino.com/ || [chatterino2-git](https://aur.archlinux.org/packages/chatterino2-git/)AUR

- **Delta Chat** — A privacy oriented chat application built on e-mail. Based on the [Electron](https://electronjs.org/) platform.

: https://delta.chat/ || [deltachat-desktop](https://archlinux.org/packages/?name=deltachat-desktop)包

- **[Discord](../即时通信/Discord.md)** — Proprietary all-in-one voice and text chat application for gamers that's free and works on both your desktop and phone. Based on the [Electron](https://electronjs.org/) platform.

: https://discordapp.com/ || [discord](https://archlinux.org/packages/?name=discord)包

- **Flare** — Unofficial Signal client based on GTK.

: https://gitlab.com/schmiddi-on-mobile/flare || [flare](https://aur.archlinux.org/packages/flare/)AUR

- **[Gitter](https://en.wikipedia.org/wiki/Gitter)** — Communication product for communities and teams on GitLab and GitHub.

: https://gitter.im/ || [gitter-bin](https://aur.archlinux.org/packages/gitter-bin/)AUR

- **[Jitsi](https://en.wikipedia.org/wiki/Jitsi) Meet** — Desktop application for Jitsi Meet. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/jitsi/jitsi-meet-electron || [jitsi-meet-desktop](https://aur.archlinux.org/packages/jitsi-meet-desktop/)AUR

- **Kotatogram Desktop** — Experimental fork of Telegram Desktop.

: https://kotatogram.github.io/ || [kotatogram-desktop](https://aur.archlinux.org/packages/kotatogram-desktop/)AUR

- **Matterhorn** — Console client for the Mattermost chat system.

: https://github.com/matterhorn-chat/matterhorn || [matterhorn](https://aur.archlinux.org/packages/matterhorn/)AUR

- **Mattermost Desktop** — Desktop application for Mattermost. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/mattermost/desktop || [mattermost-desktop](https://archlinux.org/packages/?name=mattermost-desktop)包

- **[Microsoft Teams](https://en.wikipedia.org/wiki/Microsoft_Teams)** — Official proprietary client for Microsoft Teams. Based on the [Electron](https://electronjs.org/) platform.

: https://teams.microsoft.com/downloads || [teams](https://aur.archlinux.org/packages/teams/)AUR

- **Mumble** — Voice chat application similar to TeamSpeak.

: https://www.mumble.info/ || [mumble](https://archlinux.org/packages/?name=mumble)包

- **Paper Plane** — Chat over Telegram on a modern and elegant client.

: https://github.com/paper-plane-developers/paper-plane || [paper-plane-git](https://aur.archlinux.org/packages/paper-plane-git/)AUR

- **[QQ](../即时通信/腾讯%20QQ.md)** — Proprietary instant messaging software developed by Tencent (imitating ICQ).

: https://im.qq.com/ || [linuxqq](https://aur.archlinux.org/packages/linuxqq/)AUR

- **Rocket.Chat Desktop** — Desktop application for Rocket.Chat. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/RocketChat/Rocket.Chat.Electron || [rocketchat-desktop](https://aur.archlinux.org/packages/rocketchat-desktop/)AUR

- **Ruqola** — Rocket.Chat client for the KDE desktop.

: https://apps.kde.org/ruqola/ || [ruqola](https://aur.archlinux.org/packages/ruqola/)AUR

- **Session Desktop** — Onion routing based messenger. Based on the [Electron](https://electronjs.org/) platform.

: https://getsession.org/ || [session-desktop](https://aur.archlinux.org/packages/session-desktop/)AUR

- **[Signal](https://en.wikipedia.org/wiki/Signal_(software)) Desktop** — Desktop application for Signal private messenger. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/signalapp/Signal-Desktop || [signal-desktop](https://archlinux.org/packages/?name=signal-desktop)包

- **[Skype](https://en.wikipedia.org/wiki/Skype)** — Popular but proprietary application for voice and video communication. Based on the [Electron](https://electronjs.org/) platform.

: https://www.skype.com/ || [skypeforlinux-bin](https://aur.archlinux.org/packages/skypeforlinux-bin/)AUR

- **[Slack](https://en.wikipedia.org/wiki/Slack_(software))** — Proprietary Slack client for desktop. Based on the [Electron](https://electronjs.org/) platform.

: https://slack.com/ || [slack-desktop](https://aur.archlinux.org/packages/slack-desktop/)AUR

- **teams-for-linux** — Unofficial Microsoft Teams for Linux client. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/IsmaelMartinez/teams-for-linux || [teams-for-linux](https://aur.archlinux.org/packages/teams-for-linux/)AUR

- **TeamSpeak** — Proprietary VoIP application with gamers as its target audience.

: https://www.teamspeak.com/ || [teamspeak3](https://archlinux.org/packages/?name=teamspeak3)包

- **TeamTalk** — Proprietary VoIP application with video chat, file and desktop sharing. Desktop sharing does not appear to be working in Linux though. AUR package is server only, but client is built in the make process.

: https://bearware.dk || [teamtalk](https://aur.archlinux.org/packages/teamtalk/)AUR

- **[Telegram](../即时通信/Telegram.md) Desktop** — Official Telegram desktop client.

: https://desktop.telegram.org/ || [telegram-desktop](https://archlinux.org/packages/?name=telegram-desktop)包

- **[Viber](https://en.wikipedia.org/wiki/Viber)** — Proprietary cross-platform IM and VoIP software.

: https://www.viber.com/products/linux/ || [viber](https://aur.archlinux.org/packages/viber/)AUR

- **[Wire](https://en.wikipedia.org/wiki/Wire_(software))** — Modern, private messenger. Based on the [Electron](https://electronjs.org/) platform.

: https://wire.com/ || [wire-desktop](https://archlinux.org/packages/?name=wire-desktop)包

- **Zoom** — Proprietary video conferencing, online meetings and group messaging application.

: https://zoom.us/ || [zoom](https://aur.archlinux.org/packages/zoom/)AUR

- **[Zulip](https://en.wikipedia.org/wiki/Zulip)** — Desktop client for Zulip group chat. Based on the [Electron](https://electronjs.org/) platform.

: https://zulipchat.com/apps/linux || [zulip-desktop](https://aur.archlinux.org/packages/zulip-desktop/)AUR

即时通讯服务器

参见 [Wikipedia:Comparison of instant messaging protocols](https://en.wikipedia.org/wiki/Comparison_of_instant_messaging_protocols)。

IRC 服务器

- **InspIRCd** — A stable, modern and lightweight IRC daemon.

: https://www.inspircd.org/ || [inspircd](https://aur.archlinux.org/packages/inspircd/)AUR

- **IRCD-Hybrid** — A lightweight, high-performance internet relay chat daemon.

: https://www.ircd-hybrid.org/ || [ircd-hybrid](https://aur.archlinux.org/packages/ircd-hybrid/)AUR

- **miniircd** — A small and configuration free IRC server, suitable for private use.

: https://github.com/jrosdahl/miniircd || [miniircd-git](https://aur.archlinux.org/packages/miniircd-git/)AUR

- **ngIRCd** — A free, portable and lightweight Internet Relay Chat server for small or private networks.

: https://ngircd.barton.de/ || [ngircd](https://aur.archlinux.org/packages/ngircd/)AUR

- **Ergo** — A modern and simple to set up IRC server written in Go. Combines the features of an IRCd, a services framework, and a bouncer.

: https://ergo.chat/ || [ergochat](https://aur.archlinux.org/packages/ergochat/)AUR

- **UnrealIRCd** — Open Source IRC Server.

: https://www.unrealircd.org/ || [unrealircd](https://archlinux.org/packages/?name=unrealircd)包

- **Solanum** — Solanum is the upcoming IRCd for unified networks that is being worked on by a collaboration of Libera Chat and OFTC staff.

: https://solanum.chat/ || [solanum-ircd-git](https://aur.archlinux.org/packages/solanum-ircd-git/)AUR

XMPP 服务器

参见 [Wikipedia:Comparison of XMPP server software](https://en.wikipedia.org/wiki/Comparison_of_XMPP_server_software)。

- **Prosody** — An XMPP server written in the [Lua](https://www.lua.org/) programming language. Prosody is designed to be lightweight and highly extensible. It is licensed under a permissive [MIT license](https://prosody.im/source/mit).

: https://prosody.im/ || [prosody](https://archlinux.org/packages/?name=prosody)包

- **Ejabberd** — Robust, scalable and extensible XMPP Server written in Erlang

: https://www.ejabberd.im/ || [ejabberd](https://archlinux.org/packages/?name=ejabberd)包

- **Jabberd2** — An XMPP server written in the C language and licensed under the GNU General Public License. It was inspired by jabberd14.

: https://jabberd2.org/ || [jabberd2](https://aur.archlinux.org/packages/jabberd2/)AUR

- **Openfire** — An XMPP IM multiplatform server written in Java

: https://www.igniterealtime.org/projects/openfire/ || [openfire](https://archlinux.org/packages/?name=openfire)包

SIP 服务器

参见 [Wikipedia:List of SIP software#Servers](https://en.wikipedia.org/wiki/List_of_SIP_software#Servers)。

- **Asterisk** — A complete PBX solution.

: https://www.asterisk.org/ || [asterisk](https://aur.archlinux.org/packages/asterisk/)AUR

- **Kamailio** — SIP server for large VoIP and real-time communication platforms.

: https://www.kamailio.org/ || [kamailio](https://aur.archlinux.org/packages/kamailio/)AUR

- **openSIPS** — SIP proxy/server for voice, video, IM, presence and any other SIP extensions.

: https://opensips.org/ || [opensips](https://archlinux.org/packages/?name=opensips)包

- **Repro** — An open-source, free SIP server.

: https://www.resiprocate.org/About_Repro || [repro](https://aur.archlinux.org/packages/repro/)AUR

- **[Yate](https://en.wikipedia.org/wiki/Yate_(telephony_engine))** — Advanced, mature, flexible telephony server that is used for VoIP and fixed networks, and for traditional mobile operators and MVNOs.

: https://yate.ro/ || [yate](https://archlinux.org/packages/?name=yate)包

其它即时通讯服务器

- **Mattermost** — Open source private cloud server, Slack-alternative.

: https://github.com/mattermost/mattermost-server || [mattermost](https://archlinux.org/packages/?name=mattermost)包

- **Murmur** — The voice chat application server for Mumble.

: https://www.mumble.info/ || [mumble-server](https://archlinux.org/packages/?name=mumble-server)包

- **Nextcloud Talk** — Video- and audio-conferencing app for Nextcloud.

: https://github.com/nextcloud/spreed || [nextcloud-app-spreed](https://archlinux.org/packages/?name=nextcloud-app-spreed)包

- **Rocket.Chat** — Web chat server, developed in JavaScript, using the Meteor fullstack framework.

: https://github.com/RocketChat/Rocket.Chat || [rocketchat-server](https://aur.archlinux.org/packages/rocketchat-server/)AUR

- **[Synapse](../网络电话/Matrix.md)** — Reference homeserver for the Matrix protocol.

: https://github.com/element-hq/synapse || [matrix-synapse](https://archlinux.org/packages/?name=matrix-synapse)包

- **TeamSpeak Server** — Proprietary VoIP conference server.

: https://teamspeak.com/ || [teamspeak3-server](https://archlinux.org/packages/?name=teamspeak3-server)包

- **uMurmur** — Minimalistic Mumble server.

: https://umurmur.net/ || [umurmur](https://archlinux.org/packages/?name=umurmur)包

协作软件

参见 [Wikipedia:Collaborative software](https://en.wikipedia.org/wiki/Collaborative_software)。

- **SOGo** — Groupware server built around OpenGroupware.org (OGo) and the SOPE application server.

: https://sogo.nu/ || [sogo](https://aur.archlinux.org/packages/sogo/)AUR

短链接服务器

- **microbin** — A tiny, self-contained, configurable paste bin and URL shortener written in Rust.

: https://github.com/szabodanika/microbin || [microbin](https://aur.archlinux.org/packages/microbin/)AUR

- **shlink** — Self-proclaimed definitive self-hosted URL shortener.

: https://shlink.io/ || [shlink](https://aur.archlinux.org/packages/shlink/)AUR

- **YOURLS** — A self-hosted link shortening service written in PHP.

: https://yourls.org/ || [yourls](https://aur.archlinux.org/packages/yourls/)AUR

新闻，RSS，与博客

新闻聚合器

Web 源聚合器。一些 [email 客户端](#Email_客户端)也可以作为新闻聚合器：[Claws Mail](https://en.wikipedia.org/wiki/Claws_Mail) RSSyl 插件，Evolution，[SeaMonkey Mail & Newsgroups](https://en.wikipedia.org/wiki/SeaMonkey#Mail)，[Thunderbird](../../06_软件与应用/Mozilla/Thunderbird.md)。

参见 [Wikipedia:Comparison of feed aggregators](https://en.wikipedia.org/wiki/Comparison_of_feed_aggregators)。

控制台

- **[Canto](https://en.wikipedia.org/wiki/Canto_(news_aggregator))** — Ncurses RSS aggregator.

: https://github.com/themoken/canto-curses || [canto-curses](https://archlinux.org/packages/?name=canto-curses)包

- **Ditch The Bell** — A highly configurable Linux-based desktop notifier for RSS/Atom feeds.

: https://github.com/eschermoore/ditchthebell || [dtbell-git](https://aur.archlinux.org/packages/dtbell-git/)AUR

- **feed2imap-go** — [feed2imap](https://github.com/feed2imap/feed2imap) reimplemented in Go that aggregating RSS/Atom/jsonfeed into folders of your IMAP mailbox.

: https://github.com/Necoro/feed2imap-go || [feed2imap-go](https://aur.archlinux.org/packages/feed2imap-go/)AUR

- **[Gnus](https://en.wikipedia.org/wiki/Gnus)** — Email, NNTP and RSS client for Emacs.

: https://www.gnus.org/ || [emacs](https://archlinux.org/packages/?name=emacs)包

- **feed2imap-go** — [feed2imap](https://github.com/feed2imap/feed2imap) reimplemented in Go that aggregating RSS/Atom/jsonfeed into folders of your IMAP mailbox.

: https://github.com/Necoro/feed2imap-go || [feed2imap-go](https://aur.archlinux.org/packages/feed2imap-go/)AUR

- **Newsboat** — Ncurses RSS aggregator with layout and keybinding similar to the [Mutt](../邮件客户端/Mutt.md) email client.

: https://newsboat.org/ || [newsboat](https://archlinux.org/packages/?name=newsboat)包

- **Newsraft** — Feed reader with ncurses user interface. It is greatly inspired by Newsboat and tries to be its lightweight counterpart.

: https://codeberg.org/grisha/newsraft/ || [newsraft](https://aur.archlinux.org/packages/newsraft/)AUR

- **Rawdog** — "RSS Aggregator Without Delusions Of Grandeur" that parses RSS/CDF/Atom feeds into a static HTML page of articles in chronological order.

: https://offog.org/code/rawdog/ || [rawdog](https://aur.archlinux.org/packages/rawdog/)AUR

- **rss2email** — Aggregating your RSS/Atom feed into your IMAP/Maildir mailbox as a cronjob.

: https://github.com/rss2email/rss2email || [rss2email](https://archlinux.org/packages/?name=rss2email)包

- **sfeed** — Crontab oriented shell-scriptable feed aggregator setup with an RSS/Atom parser utility plus a simple ncurses reader.

: https://codemadness.org/sfeed-simple-feed-parser.html || [sfeed](https://aur.archlinux.org/packages/sfeed/)AUR

- **Snownews** — Text mode RSS news reader.

: https://sourceforge.net/projects/snownews/ || [snownews](https://aur.archlinux.org/packages/snownews/)AUR

图形界面

- **[Akregator](https://en.wikipedia.org/wiki/Kontact#News_feed_aggregator)** — News aggregator for KDE, part of [kde-pim](https://archlinux.org/groups/x86_64/kde-pim/)包组.

: https://apps.kde.org/akregator/ || [akregator](https://archlinux.org/packages/?name=akregator)包

- **Alligator** — Kirigami-based RSS/Atom feed reader for mobile devices. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://apps.kde.org/alligator/ || [alligator](https://archlinux.org/packages/?name=alligator)包

- **Feeds** — An RSS/Atom feed reader for GNOME.

: https://gfeeds.gabmus.org/ || [gfeeds](https://archlinux.org/packages/?name=gfeeds)包

- **Fluent Reader** — Modern desktop RSS reader built with React and Fluent UI. Based on the [Electron](https://electronjs.org/) platform.

: https://hyliu.me/fluent-reader/ || [fluent-reader](https://aur.archlinux.org/packages/fluent-reader/)AUR

- **HackUp** — Read Hacker News from the desktop.

: https://github.com/mdh34/hackup || [hackup-git](https://aur.archlinux.org/packages/hackup-git/)AUR

- **[Liferea](https://en.wikipedia.org/wiki/Liferea)** — GTK news aggregator for online news feeds and weblogs.

: https://lzone.de/liferea/ || [liferea](https://archlinux.org/packages/?name=liferea)包

- **Newsflash** — Modern feed reader designed for the GNOME desktop. The spiritual successor to FeedReader.

: https://apps.gnome.org/NewsFlash/ || [newsflash](https://archlinux.org/packages/?name=newsflash)包

- **[Nextcloud](../文件分享/Nextcloud.md) News** — RSS/Atom feed reader for Nextcloud.

: https://github.com/nextcloud/news || [nextcloud-app-news](https://archlinux.org/packages/?name=nextcloud-app-news)包

- **Raven** — Simple desktop RSS reader made using VueJS. Based on the [Electron](https://electronjs.org/) platform.

: https://ravenreader.app/ || [raven-reader](https://aur.archlinux.org/packages/raven-reader/)AUR

- **RSS Guard** — Very tiny RSS and ATOM news reader developed using Qt framework.

: https://github.com/martinrotter/rssguard || [rssguard](https://archlinux.org/packages/?name=rssguard)包 or [rssguard-lite](https://archlinux.org/packages/?name=rssguard-lite)包

- **selfoss** — The new multipurpose RSS reader, live stream, mashup, aggregation web application.

: https://selfoss.aditu.de/ || [selfoss](https://aur.archlinux.org/packages/selfoss/)AUR

- **Tickr** — GTK-based RSS Reader that displays feeds as a smooth scrolling line on your desktop, as known from TV stations.

: https://www.open-tickr.net/ || [tickr](https://aur.archlinux.org/packages/tickr/)AUR

- **Tiny Tiny RSS** — Web-based news feed (RSS/Atom) aggregator.

: https://tt-rss.org/ || [tt-rss](https://archlinux.org/packages/?name=tt-rss)包

播客客户端

一些媒体播放器也可以作为播客客户端：[Amarok](../../05_桌面与图形/KDE/Amarok.md), Cantata, [Clementine](https://en.wikipedia.org/wiki/Clementine_(software)), Goggles Music Manager, [Rhythmbox](https://en.wikipedia.org/wiki/Rhythmbox), VLC media player。[git-annex](https://en.wikipedia.org/wiki/git-annex) 参见 [function as podcatcher](https://git-annex.branchable.com/tips/downloading_podcasts/)。

参见 [Wikipedia:List of podcatchers](https://en.wikipedia.org/wiki/List_of_podcatchers)。

控制台

- **castero** — A TUI podcast client for the terminal.

: https://github.com/xgi/castero || [castero-git](https://aur.archlinux.org/packages/castero-git/)AUR

- **castget** — Simple, command-line RSS enclosure downloader, primarily intended for automatic, unattended downloading of podcasts.

: https://castget.johndal.com/ || [castget](https://archlinux.org/packages/?name=castget)包

- **gpo** — Text mode interface of gPodder.

: https://gpodder.github.io/ || [gpodder](https://archlinux.org/packages/?name=gpodder)包

- **Greg** — A command-line podcast aggregator.

: https://github.com/manolomartinez/greg || [greg-git](https://aur.archlinux.org/packages/greg-git/)AUR

- **pcd** — A minimal podcast client written in go

: https://github.com/kvannotten/pcd || [pcd](https://aur.archlinux.org/packages/pcd/)AUR

图形界面

- **CPod** — Simple, beautiful podcast app. Based on the [Electron](https://electronjs.org/) platform.

: https://github.com/z-------------/CPod || [cpod](https://aur.archlinux.org/packages/cpod/)AUR

- **gPodder** — Podcast client and media aggregator (GTK interface).

: https://gpodder.github.io/ || [gpodder](https://archlinux.org/packages/?name=gpodder)包

- **Kasts** — Convergent podcast application that looks good on desktop and mobile. Part of [kde-multimedia](https://archlinux.org/groups/x86_64/kde-multimedia/)包组.

: https://apps.kde.org/kasts/ || [kasts](https://archlinux.org/packages/?name=kasts)包

- **Pocket Casts** — Electron wrapper around the Pocket Casts web app with support for MPRIS (media controls).

: https://pocketcasts.com/ || [pocket-casts-linux](https://aur.archlinux.org/packages/pocket-casts-linux/)AUR

- **Podcasts** — Podcast client for the GNOME desktop written in Rust.

: https://apps.gnome.org/Podcasts/ || [gnome-podcasts](https://archlinux.org/packages/?name=gnome-podcasts)包

- **Vocal** — Simple podcast client for the Modern Desktop (GTK).

: https://vocalproject.net/ || [vocal](https://aur.archlinux.org/packages/vocal/)AUR

新闻组新闻阅读器

一些 [email 客户端](#Email_客户端)也可以作为新闻组新闻阅读器：[Claws Mail](https://en.wikipedia.org/wiki/Claws_Mail), Evolution, [NeoMutt](../邮件客户端/Mutt.md), [SeaMonkey Mail & Newsgroups](https://en.wikipedia.org/wiki/SeaMonkey#Mail), [Sylpheed](https://en.wikipedia.org/wiki/Sylpheed), [Thunderbird](../../06_软件与应用/Mozilla/Thunderbird.md)。

参见：[Wikipedia:List of Usenet newsreaders](https://en.wikipedia.org/wiki/List_of_Usenet_newsreaders), [Wikipedia:Comparison of Usenet newsreaders](https://en.wikipedia.org/wiki/Comparison_of_Usenet_newsreaders)。

控制台

- **nn** — Alternative more user-friendly (curses-based) Usenet newsreader for UNIX.

: http://www.nndev.org/ || [nn](https://aur.archlinux.org/packages/nn/)AUR

- **[slrn](https://en.wikipedia.org/wiki/slrn)** — Text-based news client.

: https://www.slrn.org/ || [slrn](https://aur.archlinux.org/packages/slrn/)AUR

- **[tin](https://en.wikipedia.org/wiki/Tin_(newsreader))** — A cross-platform threaded NNTP and spool based UseNet newsreader.

: http://tin.org/ || [tin](https://aur.archlinux.org/packages/tin/)AUR

- **trn** — A text-based Threaded Usenet newsreader.

: https://trn.sourceforge.net/ || [trn](https://aur.archlinux.org/packages/trn/)AUR

图形界面

- **NZBGet** — Usenet binary downloader for .nzb files with web and CLI interface.

: https://nzbget.net/ || [nzbget](https://archlinux.org/packages/?name=nzbget)包

- {{App|[Pan](https://en.wikipedia.org/wiki/Pan_(newsreader))|GTK Usenet newsreader that's good at both text and binaries.|https://gitlab.gnome.org/GNOME/pan%7C[\[https://aur.archlinux.org/packages/pan/ pan](https://archlinux.org/packages/?name=%3Cspan+class%3D%22nobreak%22%3E%3Cspan+class%3D%22plainlinks+archwiki-template-pkg%22%3E%5Bhttps%3A%2F%2Faur.archlinux.org%2Fpackages%2Fpan%2F+pan%5D%3C%2Fspan%3E%3Csup%3E%3Csmall%3EAUR%3C%2Fsmall%3E%3C%2Fsup%3E%3C%2Fspan%3E)AUR\]包
- **SABnzbd** — An open-source binary newsreader webapp written in Python.

: https://sabnzbd.org/ || [sabnzbd](https://aur.archlinux.org/packages/sabnzbd/)AUR

- **XRN** — Usenet newsreader for X Window System.

: http://www.mit.edu/people/jik/software/xrn.html || [xrn](https://aur.archlinux.org/packages/xrn/)AUR

微博客户端

参见 [Wikipedia:List of Twitter services and applications](https://en.wikipedia.org/wiki/List_of_Twitter_services_and_applications)。

- **Choqok** — Microblogging client for KDE that supports Mastodon, Pump.io and GNU social.

: https://choqok.kde.org/ || [choqok-git](https://aur.archlinux.org/packages/choqok-git/)AUR

- **Dianara** — Pump.io client written in Qt.

: https://jancoding.wordpress.com/dianara/ || [dianara](https://aur.archlinux.org/packages/dianara/)AUR

- **Lemmy-UI** — Official web app for lemmy.

: https://github.com/LemmyNet/lemmy-ui || [lemmy-ui](https://aur.archlinux.org/packages/lemmy-ui/)AUR

- **Lemoa** — Native Gtk client for Lemmy.

: https://github.com/lemmygtk/lemoa || [lemoa-git](https://aur.archlinux.org/packages/lemoa-git/)AUR

- **Liftoff** — A mobile and desktop lemmy client written in flutter.

: https://github.com/liftoff-app/liftoff || [liftoff-bin](https://aur.archlinux.org/packages/liftoff-bin/)AUR

- **Mikutter** — Simple, powerful Mastodon client using GTK and Ruby.

: https://mikutter.hachune.net/ || [mikutter](https://aur.archlinux.org/packages/mikutter/)AUR

- **Pumpa** — Pump.io client written in C++ and Qt.

: https://pumpa.branchable.com/ || [pumpa-git](https://aur.archlinux.org/packages/pumpa-git/)AUR

- **Tokodon** — Mastodon client for KDE. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://apps.kde.org/tokodon/ || [tokodon](https://archlinux.org/packages/?name=tokodon)包

- **Toot** — CLI and TUI tool for interacting with Mastodon instances.

: https://github.com/ihabunek/toot || [toot](https://archlinux.org/packages/?name=toot)包

- **Tuba** — GTK4 client for Mastodon.

: https://apps.gnome.org/Tuba/ || [tuba](https://archlinux.org/packages/?name=tuba)包

- **Whalebird** — Mastodon client application. Based on the [Electron](https://electronjs.org/) platform.

: https://whalebird.social/ || [whalebird](https://aur.archlinux.org/packages/whalebird/)AUR

博客引擎

参见 [Wikipedia:Blog software](https://en.wikipedia.org/wiki/Blog_software) 和 [Wikipedia:List of content management systems](https://en.wikipedia.org/wiki/List_of_content_management_systems)。

> **注意：**内容管理器、社交网络和博客发布器在许多功能上重叠。

- **Diaspora** — A distributed privacy aware social network.

: https://diasporafoundation.org || [diaspora-mysql](https://aur.archlinux.org/packages/diaspora-mysql/)AUR or [diaspora-postgresql](https://aur.archlinux.org/packages/diaspora-postgresql/)AUR

- **[Drupal](../../06_软件与应用/内容管理系统/Drupal.md)** — A PHP-based content management platform.

: https://www.drupal.org/ || [drupal](https://archlinux.org/packages/?name=drupal)包

- **Joomla** — A php Content Management System (CMS) which enables you to build websites and powerful online applications.

: https://www.joomla.org/ || [joomla](https://aur.archlinux.org/packages/joomla/)AUR

- **[Wordpress](../网络服务器/Wordpress.md)** — Blog tool and publishing platform.

: https://wordpress.org/ || [wordpress](https://archlinux.org/packages/?name=wordpress)包

静态网站生成器

- **Hexo** — Fast, simple and powerful blog framework.

: https://hexo.io/ || [hexo-cli](https://aur.archlinux.org/packages/hexo-cli/)AUR

- **Hugo** — Hugo is a static HTML and CSS website generator written in Go. It is optimized for speed, ease of use, and configurability.

: https://gohugo.io/ || [hugo](https://archlinux.org/packages/?name=hugo)包

- **Jekyll** — Static blog engine, written in Ruby, which supports Markdown, textile and other formats.

: https://jekyllrb.com/ || [jekyll](https://aur.archlinux.org/packages/jekyll/)AUR

- **Nikola** — Static site generator written in Python, with incremental rebuilds and multiple markup formats.

: https://getnikola.com/ || [nikola](https://archlinux.org/packages/?name=nikola)包

- **Pelican** — Static site generator, powered by Python.

: https://docs.getpelican.com/ || [pelican](https://archlinux.org/packages/?name=pelican)包

- **Zola** — An opinionated static site generator, written in Rust.

: https://www.getzola.org/ || [zola](https://archlinux.org/packages/?name=zola)包

Gallery software

- **fgallery** — A static photo gallery generator with no frills that has a stylish, minimalist look.

: https://www.thregr.org/wavexx/software/fgallery/ || [fgallery](https://aur.archlinux.org/packages/fgallery/)AUR

- **jAlbum** — A freeware cross-platform software for managing and creating digital photo albums or galleries.

: https://jalbum.net/en/ || [jalbum](https://aur.archlinux.org/packages/jalbum/)AUR

- **jolly** — A tool for statically generating galleries from images.

: https://gitlab.com/prior99/jolly || [jolly](https://aur.archlinux.org/packages/jolly/)AUR

- **llgal** — An easy and fast on-line gallery generator based on iGal.

: http://bgoglin.free.fr/llgal/ || [llgal](https://aur.archlinux.org/packages/llgal/)AUR

- **thumbsup** — A static web galleries for all your photos and videos.

: https://thumbsup.github.io/ || [nodejs-thumbsup](https://aur.archlinux.org/packages/nodejs-thumbsup/)AUR

- **Photoview** — A photo gallery for self-hosted personal servers.

: https://photoview.github.io/ || [photoview](https://aur.archlinux.org/packages/photoview/)AUR

- **Piwigo** — A web application to manage your collection of photos, and other medias.

: https://piwigo.org/ || [piwigo](https://aur.archlinux.org/packages/piwigo/)AUR

- **revela** — A static web image gallery generator.

: https://sr.ht/~yaroslav/revela/ || [revela](https://aur.archlinux.org/packages/revela/)AUR

- **Sigal** — A simple static gallery generator.

: http://sigal.saimon.org/en/latest/ || [sigal](https://archlinux.org/packages/?name=sigal)包

- **ZenphotoCMS** — A CMS for selfhosted, gallery focused websites.

: https://www.zenphoto.org/ || [zenphoto](https://aur.archlinux.org/packages/zenphoto/)AUR

远程桌面

参见 [Wikipedia:Remote desktop software](https://en.wikipedia.org/wiki/Remote_desktop_software) 和 [Wikipedia:Comparison of remote desktop software](https://en.wikipedia.org/wiki/Comparison_of_remote_desktop_software)。

参见 [Chrome Remote Desktop](https://remotedesktop.google.com) 获得基于 web 浏览器的解决方案。

远程桌面客户端

- **[AnyDesk](https://en.wikipedia.org/wiki/AnyDesk)** — Proprietary remote desktop software.

: https://anydesk.com/ || [anydesk-bin](https://aur.archlinux.org/packages/anydesk-bin/)AUR

- **GNOME Connections** — Remote desktop client for GNOME. Supports RDP and VNC. Part of [gnome](https://archlinux.org/groups/x86_64/gnome/)包组.

: https://apps.gnome.org/Connections/ || [gnome-connections](https://archlinux.org/packages/?name=gnome-connections)包

- **GVncViewer** — Simple VNC Client on Gtk-VNC. Run with `gvncviewer`.

: https://wiki.gnome.org/Projects/gtk(2d)vnc || [gtk-vnc](https://archlinux.org/packages/?name=gtk-vnc)包

- **[KRDC](https://en.wikipedia.org/wiki/KRDC)** — Remote Desktop Client for KDE. Supports RDP and VNC. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://apps.kde.org/krdc/ || [krdc](https://archlinux.org/packages/?name=krdc)包

- **[Remmina](../远程服务器/Remmina.md)** — Remote desktop client written in GTK. Supports RDP, VNC, NX, XDMCP and SSH.

: https://remmina.org/ || [remmina](https://archlinux.org/packages/?name=remmina)包

- **Remote Viewer** — Simple remote display client. Supports SPICE and VNC.

: https://virt-manager.org/ || [virt-viewer](https://archlinux.org/packages/?name=virt-viewer)包

- **RustDesk** — A remote desktop software, open source, written in Rust.

: https://rustdesk.com || [rustdesk](https://aur.archlinux.org/packages/rustdesk/)AUR

- **Sunlogin Remote Control** — Proprietary software that supports remote control of mobile devices, Windows, Mac, Linux and other systems. It uses its own proprietary protocol.

: https://sunlogin.oray.com/en/about/about || [sunloginclient](https://aur.archlinux.org/packages/sunloginclient/)AUR

- **ToDesk** — Proprietary remote desktop client that suits for remote teamwork. It uses its own proprietary protocol.

: https://www.todesk.com/ || [todesk-bin](https://aur.archlinux.org/packages/todesk-bin/)AUR

- **[TeamViewer](https://en.wikipedia.org/wiki/TeamViewer)** — Proprietary remote desktop client. It uses its own proprietary protocol.

: https://www.teamviewer.com/ || [teamviewer](https://aur.archlinux.org/packages/teamviewer/)AUR

- **[vncviewer (TigerVNC)](../远程服务器/TigerVNC.md)** — VNC viewer for X.

: https://tigervnc.org/ || [tigervnc](https://archlinux.org/packages/?name=tigervnc)包

- **[Vinagre](https://en.wikipedia.org/wiki/Vinagre)** — Remote desktop viewer for GNOME. Supports RDP, VNC, SPICE and SSH.

: https://wiki.gnome.org/Apps/Vinagre || [vinagre](https://archlinux.org/packages/?name=vinagre)包

- **xfreerdp** — FreeRDP X11 client. Run with `xfreerdp`.

: https://www.freerdp.com/ || [freerdp](https://archlinux.org/packages/?name=freerdp)包

- **[X2Go](../远程服务器/X2Go.md) Client** — A graphical client (Qt5) for the X2Go system that uses the [NX technology](https://en.wikipedia.org/wiki/NX_technology) protocol.

: https://wiki.x2go.org/doku.php || [x2goclient](https://aur.archlinux.org/packages/x2goclient/)AUR

远程桌面服务器

- **freerdp-shadow-cli** — A utility for sharing an X display via RDP.

: https://www.freerdp.com/ || [freerdp](https://archlinux.org/packages/?name=freerdp)包

- **kmsvnc** — A VNC server for DRM/KMS capable GNU/Linux devices.

: https://github.com/isjerryxiao/kmsvnc || [kmsvnc](https://aur.archlinux.org/packages/kmsvnc/)AUR

- **KRdp** — A limited RDP server for KDE.

: https://invent.kde.org/plasma/krdp || [krdp](https://archlinux.org/packages/?name=krdp)包

- **Krfb** — VNC server for KDE. Part of [kde-network](https://archlinux.org/groups/x86_64/kde-network/)包组.

: https://apps.kde.org/krfb/ || [krfb](https://archlinux.org/packages/?name=krfb)包

- **MeshCentral** — Device management server for files, terminal access and remote desktop into Linux (X11), macOS and Windows.

: https://meshcentral.com/ || [meshcentral](https://aur.archlinux.org/packages/meshcentral/)AUR

- **NoMachine** — Proprietary remote desktop server and client based on [NX technology](https://en.wikipedia.org/wiki/NX_technology).

: https://nomachine.com/ || [nomachine](https://aur.archlinux.org/packages/nomachine/)AUR

- **wayvnc** — VNC server for wlroots based wayland compositors (such as [sway](https://archlinux.org/packages/?name=sway)包).

: https://github.com/any1/wayvnc || [wayvnc](https://archlinux.org/packages/?name=wayvnc)包

- **[x0vncserver (TigerVNC)](../远程服务器/TigerVNC.md)** — VNC Server for X displays.

: https://tigervnc.org/ || [tigervnc](https://archlinux.org/packages/?name=tigervnc)包

- **[x11vnc](../远程服务器/X11vnc.md)** — VNC server for real X displays.

: http://www.karlrunge.com/x11vnc/ || [x11vnc](https://archlinux.org/packages/?name=x11vnc)包

- **[X2Go](../远程服务器/X2Go.md) Server** — An open source remote desktop software that uses the [NX technology](https://en.wikipedia.org/wiki/NX_technology) protocol.

: https://wiki.x2go.org/doku.php || [x2goserver](https://aur.archlinux.org/packages/x2goserver/)AUR

- **Xpra** — A multi-platform screen and application forwarding system.

: https://xpra.org/ || [xpra](https://archlinux.org/packages/?name=xpra)包

- **[Xrdp](../远程服务器/Xrdp.md)** — A daemon that supports RDP. It uses Xvnc, X11rdp or xorgxrdp as a backend.

: https://www.xrdp.org/ || [xrdp](https://aur.archlinux.org/packages/xrdp/)AUR

网络电话

本章解决的是把 Matrix 自己跑起来的问题：装包、写配置、接数据库、拉起服务，再管理用户、启用蜘蛛爬虫，一条链路走完，才算有一套能用的通信服务。

本章只有一节，没有整节可跳，重点在顺序：安装、配置、数据库、服务这四段必须依次读、依次做，前一环没落地就别往下推；用户管理与蜘蛛爬虫属按需部分，眼下用不到可先跳过，需要时再回补。最容易踩的坑是顺序和生效——数据库还没就绪就启动服务，或者改完配置没重启服务就以为已经生效；蜘蛛爬虫也不是纯装饰，它会按配置主动对外发起请求，启用前先弄清它要做什么。

40.1　Matrix

难度 进阶 | **篇幅** 6.9 KB | **所属** 网络电话

本节将讲到

- 安装
- 配置
- 数据库
- 服务
- 用户管理

Matrix 是一个新的开放式联邦化的即时通信和 VoIP 生态系统。其包含服务端、客户端和桥接工具，可以连接到已有的通信解决方案如IRC。

Arch Linux 的官方 Matrix 空间位于#public-space:archlinux.org。要想加入空间，请联系已加入空间的成员或 *@heftig:archlinux.org* 寻求邀请。想加入 Arch Linux CN 的官方 Matrix 房间可以参考 Arch Linux 中文社区#Matrix 房间。一些国际社区有其自己的 Matrix 房间，请参阅国际社区以了解详细信息。

对于客户端，请参阅应用程序列表/互联网#Matrix 客户端。

您可以使用已有的 Matrix 服务器如 https://matrix.org 或创建您自己的服务器。下文是创建一台广泛使用的服务器的教程。

安装

Matrix 的服务端实现 **Synapse** 由 matrix-synapse包 提供，其会创建一个 *synapse* 用户。

配置

安装完成后需要生成一个配置文件，*synapse* 用户应当能够读取该文件：

```
# generate_config --server-name my.domain.name \
                  --config-dir /etc/synapse \
                  --data-dir /var/lib/synapse \
                  --report-stats yes \
                  --generate-secrets \
                  --output-file /etc/synapse/homeserver.yaml
# generate_log_config -o /etc/synapse/my.domain.name.log.config \
                      -f /var/log/synapse/homeserver.log
# generate_signing_key --output_file /etc/synapse/my.domain.name.signing.key
# chown -R synapse:synapse /etc/synapse /var/log/synapse
```

请注意这会为指定的服务器名称生成相应的 SSL 和自签名证书，若您更改了服务器名称，则应重新生成。

注意：若您的 Synapse 服务器可通过互联网访问，那么非常建议您为其设置反向代理。

数据库

Synapse 仅用 SQLite 测试服务器，操作时使用 Postgres。

要使用 Postgres，请安装 python-psycopg2包。

注意：您可在官网的使用 Postgres 文档中读到更多有关安装和配置的信息，注意如果您错过了这个步骤，您*需要*稍后执行迁移。在这个领域 SQlite 并不是很精通。

服务

matrix-synapse包 包含 systemd 服务 `synapse.service`，其会按照配置文件 `/etc/synapse/homeserver.yaml` 以用户 *synapse* 启动 Synapse 服务端。

用户管理

您的新 synapse 服务器需要至少一个用户，可以执行以下命令创建您的默认非 root 用户：

```
[synapse]$ register_new_matrix_user -c /etc/synapse/homeserver.yaml
http://127.0.0.1:8008
```

也可以使用一个 matrix 客户端，例如 element-desktop包，或 libpurpleAUR 的 purple-matrix-gitAUR 插件。

蜘蛛爬虫

如需为链接预览等功能启用蜘蛛爬虫，请安装 python-lxml包 和 python-netaddr包。之后在 `homeserver.yaml` 中设定 `url_preview_enabled: True`。欲使 synapse 服务器拒绝指向内部服务器的 GET 请求，请设定 `url_preview_ip_range_blacklist:`。

警告：黑名单默认空白：不配置的话 synapse 服务器可以爬取您所有的内部主机。

可以取消一些黑名单示例的注释。把您本地的 IP 地址范围添加到列表中可以阻止 synapse 服务器尝试爬取它们。在编辑 `homeserver.yaml` 后，必须重启服务。

您可能感兴趣的频道

KDE 社区有一系列的 matrix 房间用于特定的应用、语言、事件等。详情请查看 https://community.kde.org/Matrix。

GNOME 社区也有一个有一系列房间的 matrix 实例用于社区即时交流。详情请查看 https://wiki.gnome.org/GettingInTouch/Matrix。

故障排除

只读文件系统

默认情况下，synapse 只能写入由服务文件设置的工作目录 (`/var/lib/synapse`)。当 synapse 写入另一个路径时，可能会发生写入错误，例如您设置了媒体存储于 `/var/lib/matrix-synapse/media`。

您可以创建一个 `synapse.service` 的替换单元文件并添加 `ReadWritePaths=*`欲写入的路径`*` 到 `[Service]` 节来允许 synapse 访问其他路径。

高内存占用

synapse 的内存占用可以通过安装 jemalloc包 被大幅减少。要启用 ***jemalloc*** 内存管理器，环境变量 `LD_PRELOAD` 需要按下方示例设置。可以修改 `/etc/default/synapse` 文件，systemd 会自动加载该配置中的环境变量。

```
/etc/default/synapse
LD_PRELOAD=/usr/lib/libjemalloc.so
```

启用 ***jemalloc*** 后，通过调整缓存设置可以进一步降低内存占用。

```
/etc/synapse/homeserver.yaml
caches:
  cache_autotuning:
    max_cache_memory_usage: 1024M
    target_cache_memory_usage: 758M
    min_cache_ttl: 5m
```

只有 ***jemalloc*** 启用时，`cache_autotuning` 中的设置才会工作。

网络监控

网络是"偶尔慢"还是"一直不稳"，临时测一次得不出结论，得把链路质量按时间轴记成曲线才看得清，这正是 Smokeping 要干的事。

本章只有一节，没有整节可跳的部分：安装、配置、安装 Web 界面三步构成最短可用路径，顺序不宜调换——只装主程序不会自动得到图表，Web 界面是分开的一步。高级配置可以先略读，等基础曲线跑通、要调整监控范围时再回看；Troubleshooting 与注意两处建议动手前先扫一遍。最容易踩的坑是把"能跑起来"当成"数据可信"：配置写错时工具往往不报错，只安静地给出一张没有意义的图。

41.1　Smokeping

难度 进阶 | **篇幅** 11.1 KB | **所属** 网络监控

本节将讲到

- 安装
- 配置
- 安装Web界面
- 高级配置
- Troubleshooting

Smokeping允许你监测多台服务器。 Smokeping使用RRDtool来存储数据，另外，其可基于RRDtool输出生成相应的统计图表。 Smokeping由两个部分组成。一个运行在后台、定期收集数据的服务。一个以图表形式展示数据的Web界面。

这个wiki页面包括安装smokeping后台服务和Web界面的基本步骤。

安装

这一节包括使用smokeping包包安装Smokeping。 FastCGI的安装步骤可以参见Apache and FastCGI。

smokeping软件包包括两个部分：

- smokeping的后台服务和在`/etc/smokeping/`的配置文件。这个后台服务执行监测任务。
- 在`/srv/http/smokeping`的“htdocs”。这些文件被Web界面所使用。

除了安装smokeping包包之外，您还需要：

- 一个smokeping监测的工具。fping包 是默认ping探针的最简洁方法。
- apache包 和mod_fcgidAUR 被用于Web界面。

- 一个图像缓存目录，其可被FastCGI脚本写入，例如：`/srv/smokeping/imgcache`
- 一个数据存储目录，其可被smokeping后台服务写入，同时可被FastCGI脚本读取，例如：`/srv/smokeping/data`
- 确保主配置文件可被smokeping后台服务读取。

可选的安装

如果你想使用其他探针，例如DNS、http探针，你需要安装如下所示的其他软件包。这些探针的配置并不包括在这个wiki页面中。

探针	所需软件包
Curl	curl包
DNS	bind包 (被用于Dig)
EchoPing	echoping包
SSH	openssh包
TelnetIOSPing	perl-net-telnet包
AnotherDNS	perl-net-dns包
LDAP	perl-ldap包
LDAP (tls)	perl-io-socket-ssl包
Authen	perl-authen-radius包

配置

Smokeping需要你编辑许多文件。 未编辑的文件的扩展名为`.dist`。将在`/etc/smokeping`目录中以`.dist`结尾的文件去除`.dist`后缀。 *find* 可以完成这件任务，同时打印出所有被重命名与需要编辑的文件。

```
# cd /etc/smokeping
# find . -name '*.dist' -print -execdir sh -c 'mv {} $(basename {} .dist)' \;
# mv /srv/http/smokeping/smokeping.fcgi.dist /srv/http/smokeping/smokeping.fcgi
```

编辑配置文件

下一步，编辑 `/etc/smokeping/config` 文件。这个文件是smokeping的主配置文件。 下面将用一个完整的配置文件例子简要介绍各节的功用。

`/etc/smokeping/config` 中 *General* 节是最容易被编辑的。 根据你个人信息个性化上述的配置文件。相应的条目均有注释。

注意：如果你没有安装`sendmail`的软件（例如：postfix 或 sendmail），使用一些别的东西代替，例如`/bin/false`。 你编辑的文件必须存在，否则smokeping将会报错。

Alerts 节。这个最小化的配置文件例子中并不需要 *Alerts* 节，所以你可以将其注释或删除。

Database 节不需要做任何改动。

Presentation 节中，模板文件的路径需要更新。

Probes 节规定哪些探针被激活。默认情况下，仅 *FPing* 探针被激活。这一节不需要做任何更改。

Slaves 节。这个最小化的配置文件例子中并不需要 *Slaves* 节，所以你可以将其注释或删除。注意：如果你在*Slaves*节中使用`smokeping_secrets`设置，你必须确保那个文件不能被其他用户访问，否则smokeping将会报错。`chmod 600 /etc/smokeping/smokeping_secrets`

Targets 节指定哪些主机将被探测（在这个例子中为ping的目标）。像如下的例子，根据你想要搜集统计数据的主机，个性化 *Targets* 节。

你可以在这个网址 https://oss.oetiker.ch/smokeping/doc/smokeping_examples.en.html 了解更多关于 Smokeping 配置文件的知识。

```
/etc/smokeping/config
*** General ***

owner    = Your Name Here                           # 你的名字
contact  = your.email@host.bla                      # 你的电子邮件地址
mailhost = your.smtp.server.bla                     # 你的邮件服务器
sendmail = /bin/false                               # sendmail程序路径
imgcache = /srv/smokeping/imgcache                  # filesystem directory where we
store files
imgurl   = imgcache                                 # URL directory to find them
datadir  = /srv/smokeping/data                      # daemon 与 webapp 共享的数据目录
piddir   = /var/run                                 # filesystem directory to store
PID file
cgiurl   = http://localhost/smokeping/smokeping.fcgi  #  外部URL
smokemail = /etc/smokeping/smokemail
tmail    = /etc/smokeping/tmail
syslogfacility = local0
# each probe is now run in its own process
# disable this to revert to the old behaviour
# concurrentprobes = no

*** Database ***

step     = 300
pings    = 20

# consfn mrhb steps total

AVERAGE  0.5   1  1008
AVERAGE  0.5  12  4320
    MIN  0.5  12  4320
    MAX  0.5  12  4320
AVERAGE  0.5 144   720
    MAX  0.5 144   720
    MIN  0.5 144   720

*** Presentation ***
```

```
template = /etc/smokeping/basepage.html

+ charts

menu = Charts
title = The most interesting destinations
++ stddev
sorter = StdDev(entries=>4)
title = Top Standard Deviation
menu = Std Deviation
format = Standard Deviation %f

++ max
sorter = Max(entries=>5)
title = Top Max Roundtrip Time
menu = by Max
format = Max Roundtrip Time %f seconds

++ loss
sorter = Loss(entries=>5)
title = Top Packet Loss
menu = Loss
format = Packets Lost %f

++ median
sorter = Median(entries=>5)
title = Top Median Roundtrip Time
menu = by Median
format = Median RTT %f seconds

+ overview

width = 600
height = 50
range = 10h

+ detail

width = 600
height = 200
unison_tolerance = 2

"Last 3 Hours"    3h
"Last 30 Hours"   30h
"Last 10 Days"    10d
"Last 400 Days"   400d

*** Probes ***

+ FPing

binary = /usr/sbin/fping

*** Targets ***

probe = FPing

menu = Top
title = Network Latency Grapher
remark = Welcome to the SmokePing website of Arch User. \
         Here you will learn all about the latency of our network.

+ targets
menu = Targets

++ ArchLinux
```

```
menu = Arch Linux
title = Arch Linux Website
host = 66.211.214.131

++ GoogleDNS

menu = Google DNS
title = Google DNS server
host = 8.8.8.8

++ MultiHost

menu = Multihost example
title = Arch Wiki and Google DNS
host = /targets/ArchLinux /targets/GoogleDNS
```

有关smokeping配置文件语法的注意事项

每一个 **+** 号定义了一个有着层次等级的节。在节名称中不允许含有空格。英文句号 **.** 和正斜杠 **/** 同样应当尽量避免出现在节名称中。这是因为存储在数据目录下的RRD文件有着和节名称一样的文件名。

在*Targets* 节，你可以用真实的主机名或其他节名称的路径来定义`host`，从而生成含有多个主机结果的图表，具体情况可以参考上面例子中的`MultiHost`。

安装系统中的其他部分

创建配置文件提到的额外目录：

```
# mkdir -p /srv/smokeping/data
# mkdir -p /srv/smokeping/imgcache
# chown -R smokeping:smokeping /srv/smokeping
# chown -R http:http /srv/smokeping/imgcache
# chmod a+rx /srv/smokeping
# chmod -R a+rx /srv/smokeping/data
```

由于smokeping配置文件同时被smokeping服务和FastCGI脚本读取，所以它应当具有可读取权限：

```
# chmod a+rx /etc/smokeping
# chmod a+r /etc/smokeping/config
```

启动服务

启动 Start `smokeping.service`服务，并且确认它正在运行。

```
# systemctl start smokeping.service
# systemctl enable smokeping.service
```

安装Web界面

编辑`/etc/httpd/conf/httpd.conf`，使其包含以下内容：

```
/etc/httpd/conf/httpd.conf
 LoadModule fcgid_module modules/mod_fcgid.so
 <IfModule fcgid_module>
   AddHandler fcgid-script .fcgi
 </IfModule>

 Alias /smokeping/imgcache /srv/smokeping/imgcache
 Alias /smokeping /srv/http/smokeping
```

```
<Directory "/srv/smokeping/imgcache">
  AllowOverride all
  Require all granted
</Directory>

<Directory "/srv/http/smokeping">
 Options FollowSymLinks ExecCGI
 AllowOverride all
 Require all granted
</Directory>
```

通过 `httpd.service`，启动 Start Apache 。

检查 `[http://localhost/smokeping/smokeping.fcgi](http://localhost/smokeping/smokeping.fcgi)` 是否载入。第一批数据将在几分钟后出现。

如果由于字体原因导致图像不可读，你应当安装ttf-dejavu包。

高级配置

Smokeping是一个可以被多种方式配置的有力工具。你可以启动许多不同类型的探针。你可以设置发送统计信息的从属smokeping服务器（slave smokeping servers），并显示来自其他服务器的探测结果。 你也可以使用perl创建你自己的探针。这篇教程目前并没有讲解这些选项。如果你对这些内容感兴趣，可以查阅Smokeping官方网站上的资料。

Troubleshooting

The smokeping package is currently broken in several ways. To get the service to run as a daemon, you will need to modify the provided systemd unit file to have this parameter:

```
   type=forking
```

Smokemail is also not currently included, and is a required file for smokeping to run. You will need to download the file and add it manually from smokeping's github:

```
   https://github.com/oetiker/SmokePing/blob/master/etc/smokemail.dist
```

注意

Smoketrace (Tr.cgi)

根据更新记录，v2.5.5版本后SmokeTraceroute功能正式上线。

网页

本章解决的是让机器自己去取网上的订阅源：把想跟的站点交给它，更新自己送上门，不必每天手动翻一遍。范围很窄，只做获取网络订阅源这一件事。

本章只有《网络订阅源》一节，属必读，没有可跳过的内容；篇幅不长，建议一次读完，读完就拿手头想跟的源试一遍。最容易踩的坑是把它当一次性抓取——地址当下能打开，不等于长期可用：源会搬迁、停更或改变结构，取用太频繁也可能被对端拒绝。先确认源还在正常更新，再谈定期获取，比事后逐条排查失效的源省事得多。

42.1　网络订阅源

难度 入门 | **篇幅** 3.3 KB | **所属** 网页

本节将讲到

- 获取网络订阅源

一些网站提供了网络订阅源（Web feed） 或 **新闻订阅源**，一般以 RSS， \[Atom)\], 或 \[JSON\] 格式提供。 **新闻聚合器**可以读取这些订阅源的内容，检查这些内容是否有更新，从而让用户订阅博客或播客内容。

获取网络订阅源

即使一个网站并没有声明其拥有网络订阅源，也有可能提供这项服务。您可以尝试在网址后添加`/feed`或`/rss`来查看网站是否支持。如果这样还是不行的话，可以使用`Ctrl+u`查看网页源代码，然后使用`Ctrl+f`搜索`<link rel="alternate" type="application/atom+xml"`或`<link rel="alternate" type="application/rss+xml"`。火狐浏览器插件Awesome RSS会在访问支持网络订阅源的网站时于地址栏添加一个可以点击的图标。

如果网站不支持网页订阅源，可以试试这个应用RSS-Bridge.

以下章节描述了在特定网站上如何获得订阅链接。

Arch Linux

- 新闻（英文）
- 订阅链接汇总（英文）

GitHub

对提交commit订阅使用如下链接语法：

```
https://github.com/user/repo/commits/branch/path/to/subdir.atom
```

Reddit

Reddit provides feeds for subreddits, posts and users: simply add `.rss` to the URL. If you have a Reddit account, you can find your personal feeds in the preferences.

Reddit提供RSS订阅链接。对于子版块、帖子和用户，仅需要在网址后添加`.rss`即可。如果你已经拥有Reddit账户，你可以在偏好中找到自己的订阅链接。

YouTube

订阅一个频道：

```
https://www.youtube.com/feeds/videos.xml?channel_id=UCxxxxxxxxxxxxxxxxxxxxxx
```

如果这个频道使用以`/user/*username*`结尾的传统网址，你也可以使用：

```
https://www.youtube.com/feeds/videos.xml?user=username
```

你可以从Google Takeout上下载你的订阅链接，然后使用这个项目将其转换为OMPL格式。